

Vitis Unified Software Platform Documentation

Application Acceleration Development

UG1393 (v2023.2) December 13, 2023

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

Section I: Getting Started with Vitis.....	13
Chapter 1: Navigating Content by Design Process.....	14
Chapter 2: Vitis Software Platform Release Notes.....	15
What's New.....	15
Supported Platforms.....	15
Embedded GNU Toolchain Details.....	16
Changed Behavior.....	16
Known Issues.....	18
Chapter 3: Installation.....	19
Installation Requirements.....	19
Vitis Software Platform Installation.....	21
Section II: Introduction to Vitis Flows.....	25
Chapter 4: Introduction to the Vitis Unified IDE.....	27
Features.....	30
Components and System Projects.....	31
Migrating from Existing Tools.....	33
Chapter 5: Introduction to Vitis Tools for Embedded System	
Designers.....	37
Terminology for Embedded System Design.....	38
Fixed Platforms versus Extensible Platforms.....	39
PS Software Development.....	43
Vitis PL Kernel Development Flow.....	43
Versal AI Engine Graph Development.....	45
Building and Packaging the System.....	46
Next Steps.....	49

Chapter 6: Introduction to Data Center Acceleration for Software Programmers.....	50
Terminology.....	51
Working with Alveo Accelerator Cards.....	52
A Sample Application.....	54
Vitis Application Development Flow.....	61
Best Practices for Kernel Development.....	66
Best Practices for Host Programming.....	69
Next Steps.....	71
Chapter 7: Introduction to Data Center Acceleration for RTL Designers.....	72
Terminology.....	73
Vitis Development Flow for RTL Designers.....	74
Using Alveo Accelerator Cards.....	76
Differences between Vivado and Vitis Development Flows.....	78
Creating and Packaging RTL Kernels.....	79
Building Kernels and Managing Implementation	81
Writing Host Applications with XRT API.....	81
Next Steps.....	82
Chapter 8: Tutorials and Examples.....	84
Section III: Developing Applications.....	85
Chapter 9: Programming Model.....	86
Design Topology.....	86
PL Kernel Properties.....	87
Chapter 10: Writing the Software Application.....	92
Specifying the Device ID and Loading the XCLBIN.....	93
Setting Up XRT-Managed Kernels and Kernel Arguments.....	93
Transferring Data between Software and PL Kernels.....	95
Working with XRT-Managed Kernels.....	96
Working with User-Managed Kernels.....	97
Working with AI Engine Graphs.....	101
Summary.....	102

Chapter 11: Developing PL Kernels using C++.....	103
Chapter 12: Packaging RTL Kernels.....	104
Requirements of an RTL Kernel.....	104
Creating User-Managed RTL Kernels.....	109
RTL Kernel Development Flow.....	109
Design Recommendations for RTL Kernels.....	115
Chapter 13: Programming Versal AI Engines.....	118
Section IV: Building and Running the System.....	119
Chapter 14: Setting Up the Vitis Environment.....	123
Chapter 15: Working with Build Targets.....	124
Software Emulation.....	124
Hardware Emulation.....	125
System Hardware Target.....	126
Chapter 16: Building the Software Application.....	128
Compiling and Linking for x86.....	128
Compiling the Embedded Application for the Cortex-A72 Processor.....	131
Chapter 17: Building the Device Binary.....	133
Compiling C/C++ PL Kernels.....	134
Compiling AI Engine Graph Applications.....	136
Linking the System.....	140
Chapter 18: Managing Vivado Synthesis, Implementation, and Timing Closure.....	165
Working with Vivado in the Vitis Integrated Flow.....	166
Vitis Export to Vivado Flow.....	176
Chapter 19: Packaging the System.....	183
Packaging for Embedded Platforms.....	184
Packaging for Data Center Platforms.....	186
Packaging for DFX Platforms.....	187

Chapter 20: Building a Bare-Metal System.....	191
Chapter 21: Running the System on Hardware.....	193
Section V: Simulating the Application with the Emulation Flow.....	195
Chapter 22: Running Emulation Targets.....	197
Data Center vs. Embedded Platforms.....	199
QEMU.....	199
Running Emulation on Data Center Accelerator Cards.....	200
Running Emulation on an Embedded Processor Platform.....	201
Chapter 23: Speed and Accuracy of Hardware Emulation.....	207
Chapter 24: Working with Simulators in Hardware Emulation..	209
Simulator Support.....	209
Using the Simulator Waveform Viewer	211
AXI Transactions Display in XSIM Waveform.....	212
Generating Test Vectors for Vitis HLS during Hardware Emulation.....	212
Chapter 25: Working with Functional Model of the HLS Kernel..	214
Chapter 26: Working with SystemC Models.....	217
Coding a SystemC Model.....	217
Creating the XO.....	220
Linking with the v++ Command.....	220
Xilinx TLM – SystemC Library for ESL Modelling.....	220
Chapter 27: Debug Techniques in Hardware Emulation.....	223
Chapter 28: Working with I/O Traffic Generators.....	226
Adding Traffic Generators to Your Design.....	227
Using Traffic Generators for AI Engine Designs.....	228
AXI4-Stream I/O Model for Streaming Traffic.....	238
AXI4 Memory Map External Traffic through Python/C++.....	278
Section VI: Profiling and Debugging the Application.....	282
Chapter 29: Profiling the Application.....	283

Enabling Profiling in Your Application.....	284
Guidance.....	295
System Estimate Report.....	298
HLS Synthesis Report.....	303
Profile Summary Report.....	305
Timeline Trace.....	309
Detailed Kernel Trace.....	313
Waveform View and Live Waveform Viewer.....	318
Chapter 30: Debugging System Projects.....	324
Debugging in Software Emulation.....	325
Debugging in Hardware Emulation.....	333
Debugging During Hardware Execution.....	339
Section VII: Vitis Commands and Utilities.....	367
Chapter 31: Launching the Vitis Unified IDE.....	369
Launch Options.....	370
Chapter 32: v++ Command.....	372
v++ General Options.....	372
v++ Compilation Options.....	390
v++ Linking and Packaging Options.....	480
--advanced Options.....	480
--clock Options.....	486
--connectivity Options.....	489
--debug Options.....	494
--drc Options.....	495
--hls Options.....	496
--linkhook Options.....	499
--package Options.....	500
--profile Options.....	506
--vivado Options.....	510
Vitis Compiler Configuration File.....	513
Using the Message Rule File.....	515
Chapter 33: emconfigutil Utility.....	517
Chapter 34: kernelinfo Utility.....	519

Kernel Definition.....	520
Arguments.....	520
Ports.....	521
Chapter 35: launch_emulator Utility.....	522
Versal PS and PMC Arguments for QEMU.....	528
Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU.....	531
Zynq 7000 PS Arguments for QEMU.....	535
Chapter 36: manage_ipcache Utility.....	537
Chapter 37: package_xo Command.....	538
RTL Kernel XML File.....	540
Chapter 38: platforminfo Utility.....	544
Basic Platform Information.....	545
Hardware Platform Information.....	546
Interface Information.....	546
Clock Information.....	546
Valid SLRs.....	547
Resource Availability.....	547
Memory Information.....	548
Feature ROM Information.....	549
Software Platform Information.....	549
Platforminfo for xilinx_zcu104_base_202010_1.....	550
Chapter 39: vitis-run Command.....	552
Chapter 40: xbutil Utility.....	556
Chapter 41: xbmgmt Utility.....	557
Chapter 42: xclbinutil Utility.....	558
xclbin Information.....	559
Hardware Platform Information.....	560
Clocks.....	560
Memory Configuration.....	560
Kernel Information.....	561
Tool Generation Information.....	562

Chapter 43: xrt.ini File.....	564
Section VIII: Using the Vitis Unified IDE.....	574
Chapter 44: Launching Vitis Unified IDE.....	575
Chapter 45: Working with the Vitis Unified IDE.....	577
Features of the Vitis Unified IDE.....	577
Using Vitis Component Explorer.....	578
Search View.....	580
Source Control.....	581
Launch Configurations.....	584
Debug View.....	585
Examples View.....	587
Managing Platform Repositories.....	589
Code View and Smart Editor Features.....	589
Preferences.....	590
Jupyter Notebook Support.....	593
Parallel Compiling.....	594
Running Scripts in Batch Mode.....	595
Scrollbar control tips.....	596
Chapter 46: Creating an AI Engine Component.....	597
Chapter 47: Creating an HLS Component.....	598
Chapter 48: Creating an Application Component.....	599
Building the Application Component	600
Chapter 49: Creating a System Project for Heterogeneous	
 Computing.....	602
System Project Structure.....	604
Creating a System Project.....	606
Building the System Project	618
Launching Run or Debug of the System Project.....	620
Chapter 50: Creating a Bare-metal System.....	623

Chapter 51: Working with User-Managed Flows.....	627
Chapter 52: Debugging the System Project and AI Engine	
Components.....	632
Launching Debug in the Vitis unified IDE.....	633
Using the Debug Environment.....	641
Chapter 53: Programming Device and Flash Memory.....	656
Chapter 54: Vitis Interactive Python Shell.....	657
Auto-Completion of Python Statements	657
Shell Basic Commands.....	657
Commands to Edit a File.....	658
Commands to Run Python Scripts.....	658
Setting and Listing Environment Variables.....	659
Command to Display History.....	659
Command to Search the History.....	660
Command to Display Execution Time.....	660
Command to Clear the Session.....	660
Library Function References.....	660
Section IX: Working with the Analysis View (Vitis Analyzer).....	662
Chapter 55: Configuring the Analysis View.....	664
Chapter 56: Open Summary Reports.....	666
Chapter 57: Analysis View Window Manager.....	670
Chapter 58: Viewing AI Engine Graphs and Arrays.....	673
Chapter 59: Importing JSON Output Files.....	675
Section X: Using Vitis Embedded Platforms.....	676
Chapter 60: Vitis Embedded Platforms.....	677
Introduction.....	677
Platform Types.....	677
Platform Naming Convention.....	683

Embedded Platform Components and Architecture.....	684
Chapter 61: Using Vitis Embedded Platforms.....	686
Packaging Images.....	686
Writing Images to the SD Card.....	688
Configuring the PL Kernel in DFX Platforms and Non-DFX Platforms.....	689
Running an Acceleration Application on the Board.....	690
Software Package Management in PetaLinux rootfs.....	690
Chapter 62: Creating Embedded Platforms in Vitis.....	693
Platform Creation Basics.....	693
Platform Creation Requirements.....	696
Creating an Embedded Platform.....	697
Validating an Embedded Platform.....	723
Chapter 63: Creating a Platform Component.....	729
Creating a hardware platform.....	731
Preparing the software component.....	732
Packaging a platform component.....	733
Add a Domain to an Existing Platform.....	736
Configuring the BSP (Board support package)	738
Platform Component Command Line Flow.....	738
Section XI: Additional Information.....	742
Chapter 64: Accelerating Data Center Applications with Vitis	
Software Platform.....	743
Introduction.....	743
Methodology for Architecting a Device Accelerated Application.....	746
Methodology for Developing C/C++ Kernels.....	759
Best Practices for Data Center Acceleration with Vitis.....	772
Chapter 65: Migrating to a New Target Platform.....	774
Design Migration.....	774
Migrating Releases.....	779
Modifying Kernel Placement.....	780
Address Timing.....	786
Chapter 66: OpenCL Programming.....	789

OpenCL Host Application.....	789
Compiling and Linking the Host Application.....	820
Chapter 67: Output Structure of the Vitis Tools.....	821
Output Directories of the v++ Command.....	821
Output Directories of v++ -c --mode aie.....	827
Output Directories of v++ -c --mode hls.....	831
Output Directories of the Vitis Unified IDE.....	835
Chapter 68: Python XSDB Commands.....	846
Breakpoints.....	846
Connections.....	848
Device.....	849
Download.....	851
IPI.....	852
JTAG.....	853
Memory.....	856
Miscellaneous.....	859
Registers.....	861
Reset.....	862
Running.....	863
STAPL.....	868
Streams.....	869
SVF.....	870
TFile.....	871
Usage Examples.....	875
Chapter 69: Streaming Data Transfers.....	877
Streaming Kernel Coding Guidelines.....	877
Free-Running Kernels.....	878
Chapter 70: Using Vitis System Compilation Mode.....	880
Introduction to Vitis System Compilation Mode.....	881
Quick Start Example.....	885
Creating a VSC Makefile.....	890
Building Hardware.....	893
Application Software Interface.....	912
Debugging and Validation.....	934
Supported Platforms and Startup Examples.....	942



Section XII: Additional Resources and Legal Notices.....945

Finding Additional Documentation..... 945

Support Resources..... 946

Revision History..... 946

Please Read: Important Legal Notices..... 950

Getting Started with Vitis

This section contains the following chapters:

- [Navigating Content by Design Process](#)
- [Vitis Software Platform Release Notes](#)
- [Installation](#)

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs.

- **System and Solution Planning:** Identifying the components, performance, I/O, and data transfer requirements at a system level. Includes application mapping for the solution to PS, PL, and AI Engine.
- **Embedded Software Development:** Creating the software platform from the hardware platform and developing the application code using the embedded CPU. Also covers XRT and Graph APIs.
- **Host Software Development:** Developing the application code, accelerator development, including library, XRT, and Graph API use.
- **AI Engine Development:** Creating the AI Engine graph and kernels, library use, simulation debugging and profiling, and algorithm development. Also includes the integration of the PL and AI Engine kernels.
- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration.
- **Software Development for Acceleration:** Create an algorithm accelerator kernel with HLS and/or AI Engine. Includes platform design, organization, and management.

Vitis Software Platform Release Notes

This section contains information regarding the features and updates of the AMD Vitis™ software platform in this release. It also contains information regarding the features and updates of the Vitis software platform for AMD Versal™ AI Engine development.

What's New

For information about what's new in this version of the AMD Vitis™ unified software development platform, see the [Vitis What's New Page](#).

Supported Platforms

Data Center Accelerator Cards

Access the latest Vitis target platforms for AMD Alveo™ Data Center accelerator cards at www.xilinx.com/products/boards-and-kits/alveo.html.

Refer to the *Alveo Data Center Accelerator Card Platforms User Guide* ([UG1120](#)) for specifications of each accelerator card and available target platforms. The *Getting Started* section for each accelerator card has information for deploying your applications on that card.

Refer to Installing Xilinx Runtime and Platforms in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)) for more information on setting up XRT and platforms.

Embedded Platforms

Pre-built embedded base platforms are installed with the Vitis installer. The following platforms are included:

- `xilinx_vck190_base_202320_1`
- `xilinx_vck190_base_dfx_202320_1`

- `xilinx_vmk180_base_202320_1`
- `xilinx_zcu102_base_202320_1`
- `xilinx_zcu102_base_dfx_202320_1`
- `xilinx_zcu104_base_202320_1`
- `xilinx_vek280_es1_base_202320_1`

Common software images can be downloaded from the [Embedded Platforms download page](#). They provide pre-built Linux kernel, `rootfs`, `sysroot`, and boot components and can also work with the pre-built embedded base platforms mentioned above. They can work with the pre-built embedded base platforms mentioned above.

Versal Platform for AI Engine Development

The VCK190 platform is available for use with the Vitis application acceleration development flow, as described in the *AI Engine Tools and Flows User Guide* ([UG1076](#)). The platform enables development of designs that include:

- AI Engine graphs and kernels
- Programmable logic kernels
- Host application targeting the Linux or a bare metal OS running on the Arm® processor in the Versal device.

Embedded GNU Toolchain Details

The following GNU toolchain components are installed with the Vitis unified software platform:

- **binutils:** 2.39
- **gcc:** 12.2
- **gdb:** 12.1
- **glibc:** 2.36
- **newlib:** 4.2

Changed Behavior

The following table specifies differences between this release and prior releases that impact behavior or flow when migrating.

Table 1: Changed Behavior Summary

Area	Behavior
Vitis IDE	The Vitis Unified IDE is now the default GUI when launching with the <code>vitis</code> command. To launch the classic Vitis IDE, run <code>vitis --classic</code>. To refer to information about the classic Vitis IDE, visit the 2023.1 or earlier version of the documentation.
v++	The <code>--freqhz</code> command has been introduced to replace most other clock management commands. Use <code>--freqhz</code> to specify clocks for AI Engine components, HLS components, and <code>v++</code> + <code>--link</code> commands for System projects, as described in Managing Clock Frequencies.
Vitis HLS	<code>xparameters.h</code> no longer contains <code>DEVICE_ID</code> definitions. Instead the <code>BASEADDR</code> definition is used. AMD drivers and libraries handle this change. However, if you have legacy code, you need to be aware of this change. You need to change the way your software application addresses the hardware module through <code>X<DUT>_Initialize</code> and <code>X<DUT>_LookupConfig</code> driver functions.
Vitis HLS	The initialization behavior of <code>ap_int/ap_uint/ap_fixed/ap_ufixed</code> is changed to be more compatible with the C builtin type (<code>int</code>, <code>float</code>...). <pre>ap_int<N> A[N]; // uninitialized, both 23.2 and previous releases</pre> <pre>ap_int<N> B[N] = {0}; // With 23.2, the whole array is initialized \ to zero. With previous releases, only the first element is initialized to \ zero, other elements are uninitialized.</pre> <pre>ap_int<N> B[N] = { }; // With 23.2, the whole array is initialized \ to zero. With previous releases, it is uninitialized.</pre>
Vitis HLS	The original <code>qdma_axis<w, 0, 0, 0></code> has been removed from <code>clib</code> and is replaced with: <pre>template <std::size_t WData, std::size_t WUser, std::size_t WId, std::size_t WDest> using qdma_axis = hls::axis<ap_uint<WData>, WUser, WId, WDest, AXIS_ENABLE_ALL ^ AXIS_ENABLE_STRB, false>;</pre>
Vitis HLS	In the 2023.2 release the timing model used by the scheduler was changed so that the HLS compiler timing predictions more closely match the Vivado Design Suite. This causes the delay model to be more pessimistic. Major changes include the delay of an AXIS stream interface, and the use of complex enabling conditions for pipeline control. As a result of these changes it is expected that both latency and II can change for designs that were passing before this release. If a design was meeting timing in HLS and Vivado before, you can restore the original II and latency by adding such constraints via pragmas or directives. You can also reduce clock uncertainty using the <code>add_clock</code> or <code>set_clock</code> command. With a better HLS timing model, uncertainty can be reduced. But II constraints (and latency constraints if needed) are a more deterministic mechanism to achieve the same goal.
Vitis HLS	Assert is now supported.

Table 1: Changed Behavior Summary (cont'd)

Area	Behavior
Vitis Embedded Software Development	The SDT (System Device Tree) is a new concept in the Vitis Unified flow. Previously, in the Vitis Classic flow, the HW metadata was extracted directly from the XSA using the HSI API in an "AD Hoc" manner as required by Vitis tools, such as extracting processors for platform creation or extracting IP for BSP creation. In the Vitis Unified flow, the SDT is generated when you generate the platform. This is used to provide the hardware metadata to Vitis via the Lopper framework.
	Lopper is a Python based framework that is used to extract system metadata from the System Device Tree. The Lopper Framework API is not exposed to you via Vitis. Instead, the Vitis Python API such as Platform component creation would use the underlying Lopper Framework API. The Lopper framework is also used to generate the <code>xparameters.h</code> , and the driver initialization files too.
	MLD/MDD MSS filesets are removed and replaced by YAML and CMake filesets.
	While <code>xparameters.h</code> is still generated by Lopper Framework, it does not contain <code>DEVICE_ID</code> definitions. Instead, the <code>BASEADDR</code> definition is used. AMD drivers and libraries handle this change. However, users with legacy code need to be aware of this change.

Known Issues

Known issues for the Vitis software platform are available on the [Vitis 2023 Known Issues](#) web page.

Installation

Installation Requirements

The AMD Vitis™ unified software platform consists of an integrated design environment (IDE) for interactive project development, and command-line tools for scripted or manual application development.

Note: The Vitis application acceleration development flow is not supported for the Windows OS. However, the Vitis IDE runs on Windows OS to support both the embedded software flow, and high-level synthesis (HLS) development flow.

Table 2: Application Acceleration Development Flow Minimum System Requirements

Component	Requirement	
	Development (Build Machine OS)	Deployment (Host OS) Enabled via XRT
Operating system	Linux, 64-bit: - Red Hat Enterprise Linux Workstation/Server 7/CentOS: 7.9 - Red Hat Enterprise Linux Workstation/Server: 8.5, 8.6, 8.7, 8.8 - Red Hat Enterprise Linux Workstation/Server: 9.0, 9.1, 9.2 - Ubuntu 20.04.4 LTS, 20.04.5 LTS, 20.04.6 LTS - Ubuntu 22.04 LTS, 22.04.1 LTS, 22.04.2 LTS - Amazon Linux 2 AL2 LTS Windows, 64-bit for Embedded Software and HLS Development: - Windows 10: 21H2, 22H2 - Windows 11: 21H2, 22H2 - Windows Server 2022	For on-premise acceleration (Alveo Data Center accelerator cards): - Red Hat Enterprise Linux Workstation/Server 7/CentOS: 7.9 - Red Hat Enterprise Linux Workstation/Server: 8.5, 8.6, 8.7, 8.8 - Red Hat Enterprise Linux Workstation/Server: 9.0, 9.1, 9.2 - Ubuntu 20.04.4 LTS, 20.04.5 LTS, 20.04.6 LTS - Ubuntu 22.04 LTS, 22.04.1 LTS, 22.04.2 LTS - Amazon Linux 2 AL2 LTS - Linux in VM on Windpows Server 2019 For edge acceleration (embedded platforms): - PetaLinux 2023.2
System memory	For Alveo cards: 64 GB (80 GB is recommended) For embedded: 32 GB	
Internet connection	Required for downloading drivers and utilities.	

Table 2: Application Acceleration Development Flow Minimum System Requirements
(cont'd)

Component	Requirement	
	Development (Build Machine OS)	Deployment (Host OS) Enabled via XRT
Hard disk space	150 GB	

Table 3: AI Engine Development Flow Minimum System Requirements

Component	Requirement
Operating system	Linux, 64-bit: - Red Hat Enterprise Workstation/Server 7/CentOS: 7.9 - Red Hat Enterprise Workstation/Server: 8.5, 8.6, 8.7, 8.8 - Red Hat Enterprise Workstation/Server: 9.0, 9.1, 9.2 - Ubuntu 18.04.4 LTS, 18.04.5 LTS, 18.04.6 LTS - Ubuntu 20.04.2 LTS, 20.04.3 LTS, 20.04.4 LTS, 20.04.5 LTS, 20.04.6 LTS - Ubuntu 22.04 LTS, 22.04.1 LTS, 22.04.2 LTS IMPORTANT! The 2023.2 release is the last release that AI Engine development supports the following operating systems: - Ubuntu 18.04.4 LTS, 18.04.5 LTS, 18.04.6 LTS - Ubuntu 20.04.2 LTS, 20.04.3 LTS
System memory	64 GB (80 GB is recommended)
Internet connection	Required for downloading drivers and utilities.
Hard disk space	100 GB

OpenCL Installable Client Driver Loader

The Vitis environment supports the OpenCL™ Installable Client Driver (ICD) extension (`cl_khr_icd`). This extension allows multiple implementations of OpenCL to co-exist on the same system. The ICD Loader acts as a supervisor for all installed platforms, and provides a standard handler for all API calls.

Refer to [OpenCL Host Application](#) for information about the OpenCL host application and how to install the ICD Loader.

Vitis Software Platform Installation

Installing the Vitis Software Platform

Ensure your system meets all requirements described in [Installation Requirements](#).



TIP: To reduce installation time, disable anti-virus software and close all open programs that are not needed.

1. Go to the [AMD Adaptive Computing Downloads Website](#).
2. Download the installer for your operating system.
3. Run the installer, xsetup, or xsetup.exe, which opens the Welcome page. Extract the installer package.
4. Click **Next** to open the Select Install Type page of the Installer.
5. If installing with the web installer, enter your AMD user account credentials, and select **Download and Install Now** (only needed by web installer).
6. Click **Next** to open the Accept License Agreements page of the Installer.
7. Accept the terms and conditions by clicking each **I Agree** check box.
8. Click **Next** to open the Select Product to Install page of the Installer.
9. Select **Vitis** and click **Next** to open the Vitis Unified Software Platform page of the Installer.
10. Customize your installation by selecting design tools and devices (optional).

The default Design Tools selections are for standard Vitis Unified Software Platform installations, and include Vitis, Vivado, and Vitis HLS. You do not need to separately install Vivado tools. You can also install Model Composer and System Generator if needed.

You can enable **Vitis IP Cache** to install cache files for example designs found in the release. This is not required, but when selected, the files are installed at `<installdir>/Vitis/<release>/data/cache/xilinx`.

The default Devices selections are for devices used on standard acceleration platforms supported by the Vitis tools. You can disable some devices that might not be of interest in your installation.



IMPORTANT! Do not deselect the following option. It is required for installation.

- **Devices → Install devices for Alveo and Edge acceleration platforms**

11. Click **Next** to open the Accept License Agreements page of the Installer and accept as appropriate.
12. Click **Next** to open the Select Destination Directory page of the Installer.

13. Specify the installation directory, review the location summary, review the disk space required to insure there is enough space, and click **Next** to open the Installation Summary page of the Installer.
14. Click **Install** to begin the installation of the software.

After a successful installation of the full Vitis unified software, a confirmation message is displayed, with a prompt to run the `installLibs.sh` script.

1. Locate the script at: `<install_dir>/Vitis/<release>/scripts/installLibs.sh`, where `<install_dir>` is the location of your installation, and `<release>` is the installation version.
2. Run the script using `sudo` privileges as follows:

```
sudo installLibs.sh
```

The command installs a number of necessary packages for the Vitis tools based on the OS of your system.

★ **IMPORTANT!** Pay attention to any messages returned by the script. You might need to install any missing packages manually. For example, if your installation of Linux does not include the `zip` command-line utility, you need to manually install it. The utility is required by some of the Vitis tools and the `installLibs.sh` script does not install it for you.

Installing Xilinx Runtime and Platforms

Xilinx Runtime (XRT) is implemented as a combination of user-space and kernel driver components. XRT supports AMD Alveo™ PCIe®-based cards, in addition to AMD Versal™ and AMD Zynq™ UltraScale+™ MPSoC-based embedded system platforms, and provides a software interface to AMD programmable logic devices.

You must install XRT for use in the Vitis application acceleration development flow. You do not need to reinstall it for every additional platform you choose to download.

Note: Installing XRT is *not* required when targeting Arm®-based embedded platforms: Vitis compiler has its own copy of `xclbinutil` for hardware generation, and for software compilation, you can use the XRT from the `sysroot`. Look for **Common images for Embedded Vitis platforms** on the downloads page.

★ **IMPORTANT!** XRT installation uses standard Linux RPM and Linux DEB distribution files, and root access is required for all software and firmware installations.

`<rpm-dir>` or `<deb-dir>` is the directory where you downloaded the packages to install.

To download and install the XRT package for your operating system, do the following.

1. Go to <https://www.xilinx.com/xrt>.

2. From the [Getting Started](#) page, you can choose to download the XRT package for a specific Alveo Data Center accelerator card, or for Embedded Platforms. After choosing the platform, you will be redirected to a website with instructions for downloading XRT and the required files for the selected platform.
3. Follow the directions to install XRT and your selected platform.



TIP: The instructions for installing the Alveo Data Center accelerator cards are provided on the platform download page. Instructions for installing the Embedded Platforms can be found in the following section.

Installing Embedded Platforms

To support the Vitis application acceleration development flow, embedded platforms require a Linux kernel, a `rootfs` with integrated Xilinx Runtime, and a `sysroot` to cross-compile the host application.

To install a platform, download the zip file and extract it into `/opt/xilinx/platforms`, or extract it into a separate location and add that location to the `PLATFORM_REPO_PATHS` environment variable.

You can build your own platform software, or use a pre-built software common image. To download a pre-built common image, look for the **Common images for Embedded Vitis platforms** block on the downloads page, and download and extract the common image for your platform architecture.

Running `sdk.sh` extracts and installs the `sysroot`. The option `-d` gives you the option to choose where to install the `sysroot`. This package also provides a pre-compiled kernel image and `rootfs`.

You can add the `sysroot` to a Makefile for your command line project, or the Vitis IDE will prompt you to add it to your application project. For example, in your Makefile point `<SYSROOT>` to `/<install_path>/cortexa72-cortexa53-xilinx-linux`, which is generated when running `sdk.sh`.

Setting Up the Environment to Run the Vitis Software Platform

To configure the environment to run the Vitis unified software platform, run the following scripts in a command shell to set up the tools to run in that shell:

```
#set up XILINX_VITIS and XILINX_VIVADO variables
source <Vitis_install_path>/Vitis/2023.2/settings64.sh
#set up XILINX_XRT
source /opt/xilinx/xrt/setup.sh
```



TIP: `.csh` scripts are also provided.

This sets up the tools for the Vitis application acceleration development flow, the Vitis embedded software development flow, and the AI Engine tools for development on Versal adaptive SoC AI Engine devices.

To use any platforms you have downloaded as described in the [Installing Xilinx Runtime and Platforms](#), set the following environment variable to point to the location of the platforms:

```
export PLATFORM_REPO_PATHS=<path to platforms>
```

This identifies the location of platform files for the tools, and makes them accessible to your design projects.

Introduction to Vitis Flows

The AMD Vitis™ unified software platform is a development environment for heterogeneous applications supporting AMD devices such as AMD Alveo™ Data Center Accelerator cards, AMD Versal™ adaptive SoC devices, AMD Kria™ SOM, and AMD Zynq™ MPSoC. In the Vitis environment, heterogeneous systems include software applications running on x86 host processors or Arm® embedded processors, compute kernels running in programmable-logic (PL) regions or Versal AI Engine arrays, and extensible platform designs that provide the foundation for building and running the heterogeneous systems. The Vitis unified software platform consists of the following elements:

- The software development tool stack, such as compilers and cross-compilers to build your software application.
- Debuggers to help you locate and fix any problems in your system design.
- Program analyzers to let you profile and analyze the performance of your application.
- Xilinx Runtime (XRT) provides an API and drivers for your software program to connect with the target platform, and handles transactions and data transfers between the software application and the hardware design.
- Vitis accelerated libraries provide performance-optimized hardware functions with minimal code changes, and without the need to re-implement your algorithms to harness the benefits of AMD adaptive computing. Vitis accelerated libraries are available for common functions of math, statistics, linear algebra and DSP, in addition to domain specific applications, like vision and image processing, quantitative finance, database, data analytics, and data compression. For more information on Vitis accelerated libraries, refer to https://xilinx.github.io/Vitis_Libraries/.

The Vitis unified software platform combines all aspects of AMD hardware and software development into one unified environment using standard C/C++ for both software and hardware components. The Vitis tools provide compilation, linking, profiling and debug capabilities for heterogeneous systems in a number of different design flows including [Data Center application acceleration](#), [RTL kernel design](#), [Embedded System design](#), and traditional embedded hardware and software design. Each of these flows is described in more detail in the following chapters:

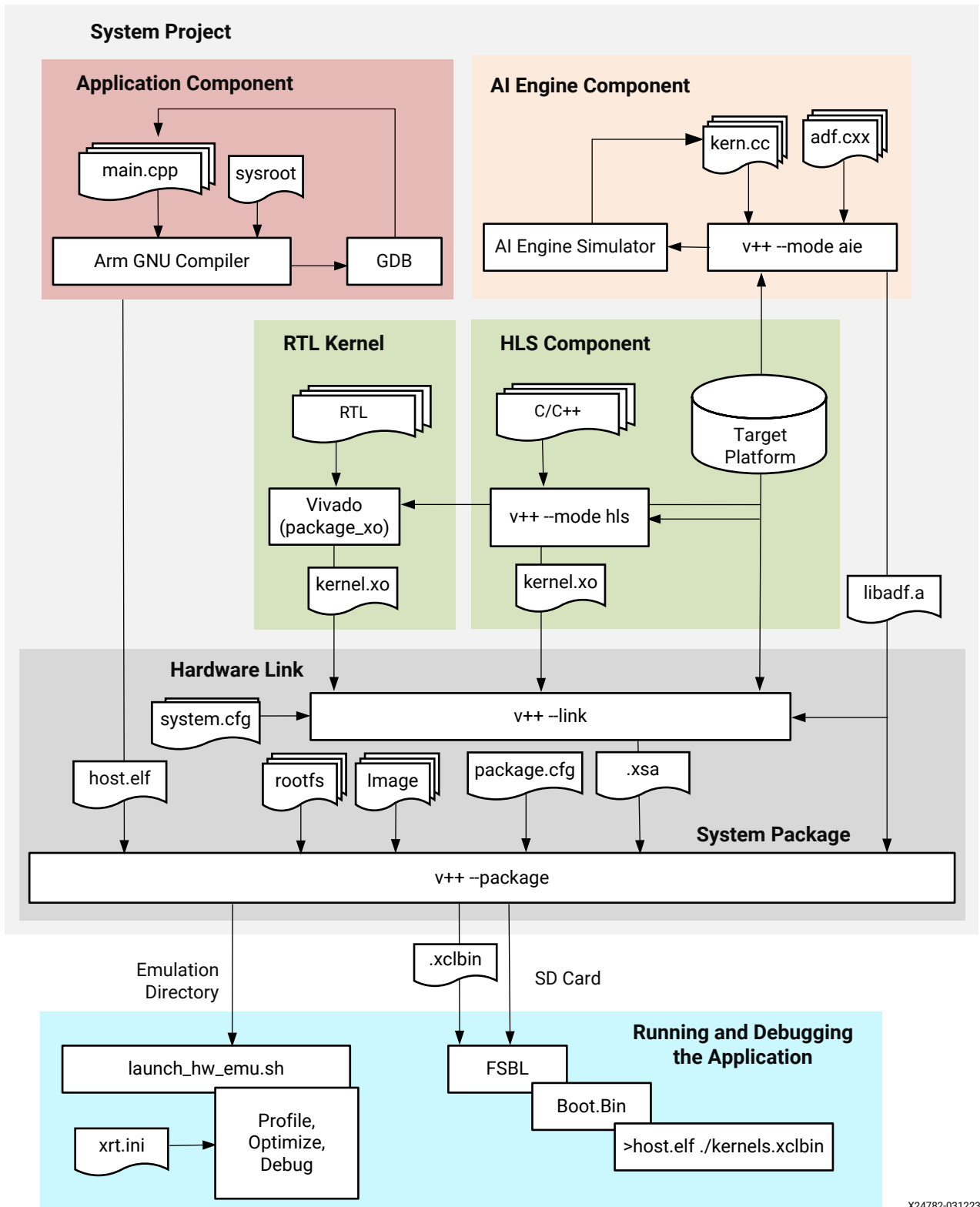
- [Introduction to the Vitis Unified IDE](#)
- [Introduction to Vitis Tools for Embedded System Designers](#)
- [Introduction to Data Center Acceleration for Software Programmers](#)

- [Introduction to Data Center Acceleration for RTL Designers](#)
- [Tutorials and Examples](#)

Introduction to the Vitis Unified IDE

The AMD Vitis™ Unified IDE is a design environment for developing applications for AMD Adaptive SoC and FPGA devices. The Vitis unified IDE embraces a bottom-up design flow that lets you develop components of a system: AI Engine graphs, C/C++ sourced HLS components, RTL design kernels, software applications, and then integrate the components into a top-level system project as shown in the following figure. The single unified development environment provides all the features you need to compile, run, debug, and analyze the different elements of an heterogeneous embedded system design, or an FPGA-accelerated Data Center application.

Figure 1: Vitis Tools Flow



X24782-031223

The Vitis Unified IDE lets you create AI Engine components using the very-long instruction word (VLIW) processor arrays of AMD Versal™ devices; synthesize C/C++ code into RTL designs using HLS components, run C-simulation and C/RTL Co-simulation; review and analyze build and run summaries in the newly integrated Vitis analyzer tool.

The unified IDE works with the common command-line features of the `v++` and `vitis-run` commands. Whether working from the command-line or from the Vitis Unified IDE, the environment provides a single tool set for end-to-end application development without the need to jump between multiple tools for design, debug, integration and analysis.

You can perform the following tasks with the Vitis Unified IDE:

- Develop embedded applications that run on processors for Adaptive SoC, including AMD Versal™ and AMD Zynq™ UltraScale+™ MPSoC devices
- Develop AI Engine applications and kernels for Versal Adaptive SoC
- Design programmable logic with C/C++ by creating HLS components
- Develop System projects for AMD Alveo™ Data Center accelerator cards and Adaptive SoC devices

The Vitis Unified IDE provides the following benefits:

- **Architected for ease-of-use:**
 - The Flow Navigator helps you manage the work flow for different designs
 - The design flow supports example template for new users to view all available examples, increasing productivity
 - Non-blocking commands can now run multiple build and analysis jobs at the same time
 - Software Emulation runs the host application in x86 mode for faster design iterations as it does not need to launch QEMU with the Linux operating system
 - The AI Engine pipeline view is enhanced from single core to multi-core; you can select the pipeline view for any active cores
 - The AI Engine microcode view is enhanced with user selectable filters
- **Easier switching between GUI and command-line (CLI) mode:**
 - Combining strengths of both GUI and CLI
 - Configuration files are rendered in real time
 - CLI can be used to build projects, and the unified IDE for debugging and analysis
 - GUI operations are saved in python log for batch rebuilding
- **Modern look and framework:**
 - Light and dark themes

- Quick actions with fully customizable shortcut keys
- User-friendly command palette
- Up-to-date C++ syntax highlighting and IntelliSense

Features

The next generation Vitis Unified IDE supports the following features:

- Support for data center and embedded application acceleration development
- Support for AI Engine applications on embedded platforms
- Support for standalone AI Engine component workflow
- Creation of HLS components for synthesizing RTL from C/C++ code
- C-Simulation and C/RTL Co-Simulation
- Integrated Vitis analyzer for helping you review, analyze, and iterate in a single tool
- Ability to create applications from templates or examples
- Support for run and debug applications in emulation or on hardware
- Support for project flow and custom Makefile flow
- Runs in Jupyter Notebook environment
- Runs in batch or interactive mode
- Support for source control with git
- Command line interface for running project creation and building actions and query project status

The editor supports the following features:

- Syntax highlighting for supported languages including C, C++, Python, `CMakeList.txt`
- Smart editor for `v++` config file and `xrt.ini`
- Quick viewing of function definitions

Components and System Projects

The new Vitis Unified IDE manages designs at the component level and the System project level. The top-level System project can contain different components, such as the Application component that holds the source code for PS or X86 applications, the HLS component (PL kernel), the AI Engine component (graph), and the Platform component, all of which can be separately developed, built, and analyzed in the new Vitis Unified IDE.

Components are the basic building blocks in the Vitis Unified IDE. There are multiple types of components:

- The AI Engine Component as described in Using the Vitis Unified IDE in *Vitis High-Level Synthesis User Guide* ([UG1399](#))
- The HLS Component as described in Building and Running an HLS Component in *Vitis High-Level Synthesis User Guide* ([UG1399](#))
- The Application Component described in [Creating an Application Component](#)
- The Platform Component as described in [Creating a Platform Component](#)

Each component contains source code and configuration files. The component can be built within their own scope and can be run or debug on their own, or together with other components in a System project. The System assembles multiple components for a system design. It contains the component link configuration. When building a System project, the new Vitis Unified IDE will run, link and package using the build output of the included components. If the required component output files are not available, the Vitis Unified IDE will prompt you to build each component.

The metadata for components are stored in `vitis-comp.json`; the metadata for applications are stored in `vitis-sys.json`, in addition to links to the configuration files for building, linking and packaging the system and its components.

System Project Configuration Files

There are two types of configuration files for a Vitis system project:

CMake Files

The parameters defined in this file are translated to configuration settings to the compilers by CMake, a commonly used build utility. The Vitis Unified IDE provides a context editor for `CMakeList.txt` file. For more information see [CMake.org](#).

In the `CMakeList.txt` file, Vitis Unified IDE creates templates for you to edit. The options managed by you are wrapped by comments such as:

```
####      START OF THE USER SETTINGS      ####
####      END OF USER SETTINGS SECTION    ####
```

The template provides explanations and examples to each user setting. For example:

```
# Add any compiler definitions, they will be added as extra definitions
# Example adding VERBOSE=1 will pass -DVERBOSE=1 to the compiler.
set(USER_COMPILE_DEFINITIONS
" "
)
```

You should read the explanations and add the contents of the settings to the quote. For example;

```
# Add any compiler definitions, they will be added as extra definitions
# Example adding VERBOSE=1 will pass -DVERBOSE=1 to the compiler.
set(USER_COMPILE_DEFINITIONS
"VERBOSE=1"
)
```

Note: If a parameter has multiple values, each value with quote should take a new line. For more information about `CMakeList.txt` syntax, see [CMake help document](#).

Configuration Files

The new Vitis Unified IDE provides a Config File editor with GUI rendering for option selection with example syntax. The default view is GUI rendering, but you can view the text form of the config file by clicking the `</>` button to switch to the code view. The code view can be used for manual entry of configuration commands, which can be necessary in some circumstances. There are four configuration files used in the system:

- **AI Engine Configuration (`aiecompiler.cfg`):** This is the configuration file for the AI Engine component for compilation and simulation of the component. The `aiecompiler.cfg` file is used by the `v++ -c --mode aie` command, in addition to the `x86simulator` and `aiesimulator` commands.
- **HLS Configuration (`hls_config.cfg`):** This is the configuration file for the HLS component to drive synthesis of the C/C++ code into an RTL module. It can be used to define properties of the design for synthesis, simulation, co-simulation, and implementation. The `hls_config.cfg` file is used by the `v++ -c --mode hls` command in addition to the `vitis-run` command.
- **Hardware Link Configuration (`hw_link/binary_containers_1-link.cfg`):** This is a configuration file for the System project and defines build instructions for linking the system of components with the platform. The `hw_link/binary_containers_1-link.cfg` configuration file is used by the `v++ --link` command.

- **Package Configuration** (`package/package.cfg`): This is another configuration file for the System project and defines packaging instructions for packaging boot files for the system, and creating an SD card. The `package/package.cfg` configuration file is used by the `v++ --link` command.

Migrating from Existing Tools

If you have experience working with the classic AMD Vitis™ IDE, you can migrate to the new Vitis unified IDE by understanding a few concepts and operation changes. For each of the various components supported by the Vitis unified IDE the links to appropriate migration content are as follows:

- Embedded Applications: Refer to "Migrating from Vitis Classic IDE" in *Vitis Unified Software Platform Documentation: Embedded Software Development* ([UG1400](#))
- AI Engine Applications: Refer to "Migrating from Vitis Classic IDE" in *AI Engine Tools and Flows User Guide* ([UG1076](#))
- HLS Components: Refer to "Migrating from Vitis HLS" in *Vitis High-Level Synthesis User Guide* ([UG1399](#))
- System Projects: Documented [here](#)

Terminology Changes

The top-level System project has several components such as the Application component that stores the host or ELF application, HLS component that defines the PL kernels, and the AI Engine component that defines the graph. The HLS components and AI Engine components can be grouped into binary containers and linked with a hardware link, similar to the existing Vitis IDE flow.

Following table provides a brief comparison of the concepts and terminology.

Table 4: Concepts and Terminology Comparison

New Vitis IDE	Classic Vitis IDE
Workspace	Workspace
System project	System project
Application component	Host application
HLS component	PL kernel application
AI Engine Component	AI Engine design

Control File Differences

The new Vitis Unified IDE uses the unified command-line in the `v++` compiler. The `v++ --mode aie` and `--mode hls` are both driven by new configurations commands that are based on the existing tools. For AI Engine component, the configuration command language is essentially the existing commands for the `aiecompiler`, `aiesimulator`, and `x86simulator` tools.

For the HLS component, the configuration file command syntax is based on the existing AMD Vitis™ HLS Tcl command language. However, there are sufficient differences in the configuration file command language and syntax that it can seem unfamiliar to start. The new Vitis Unified IDE provides a simple user interface to add to and manage configuration files, so that you can use the IDE to edit configuration files and switch to text editor mode to help you become familiar with the new command syntax.

The Vitis Unified IDE uses CMake to generate `Makfile` for the project flow. You can update the `CMakeList.txt` to add your compile options. If you prefer using custom `Makfile`, you can use the custom flow.

Migrating Projects from Classic IDE

Classic Vitis IDE workspaces cannot be opened directly in Vitis Unified IDE. To facilitate the migration of projects from the classic IDE to the unified IDE, the classic IDE provides a feature to enable moving a project or workspace into the Vitis unified IDE. To use this command first launch the Vitis classic IDE on the workspace:

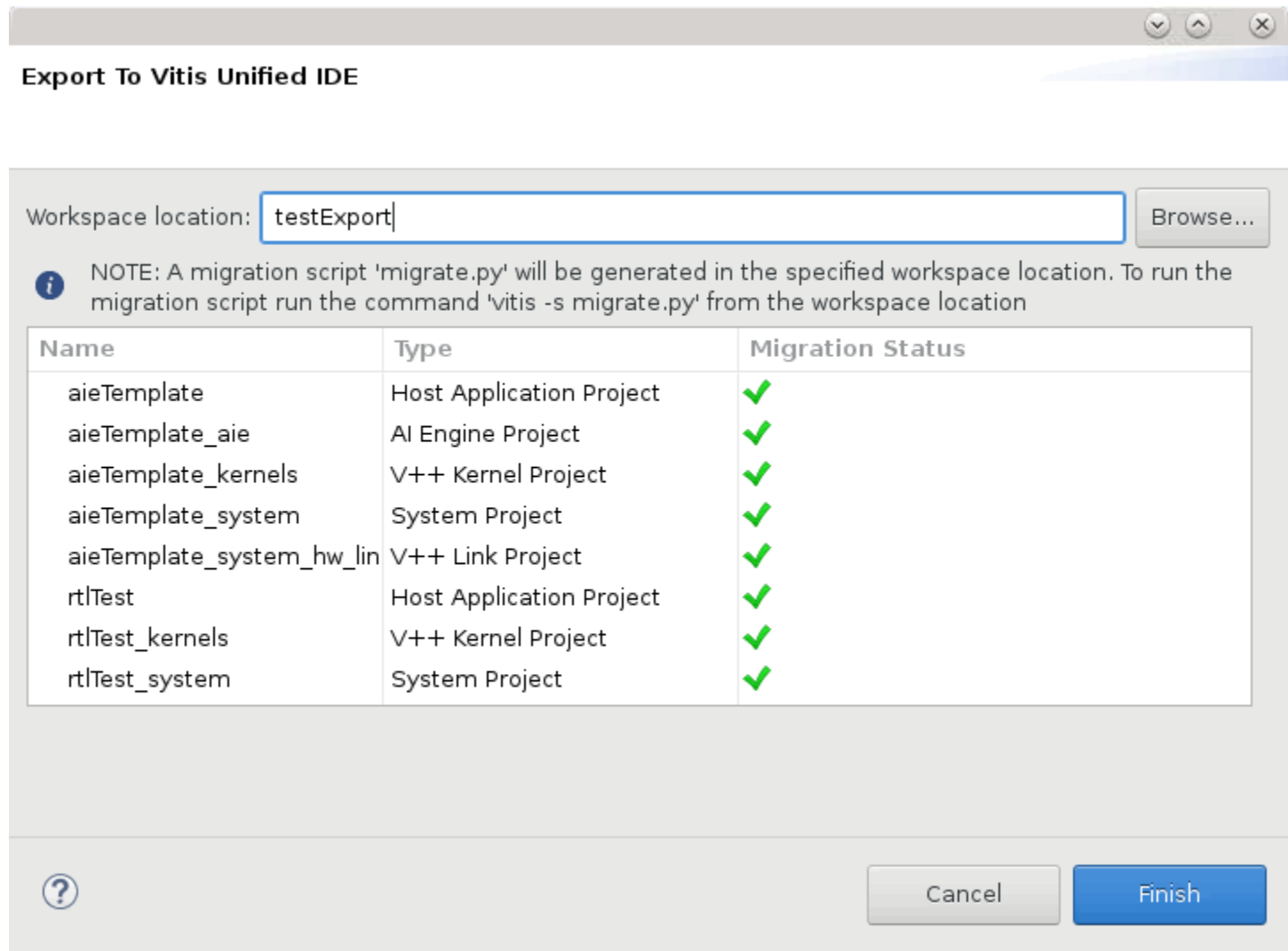
```
vitis --classic -workspace <workspace>
```

After the Vitis IDE opens the workspace, use the **Vitis → Export Workspace to Unified IDE** command from the main menu. The tool opens a dialog box listing the projects contained in the workspace, and prompts you for a new workspace to export the projects to. Specify the new workspace location and click **Finish**. The tool will generate a Python migration script (`migrate.py`) and write it to the specified workspace folder.



TIP: The Workspace location specified for the export script must be a new empty workspace. If you want to make changes to the classic IDE projects and re-export them, you must start with a clean export workspace, without hidden files.

Figure 2: Export to Vitis Unified IDE



After generating the migration script a pop-up window will show the location of the script. You will need to run the script in the Vitis unified IDE by using the `vitis -s <script>` form of the command as described in [Launch Options](#).

```
vitis -s migrate.py
```

Table 5: Supported Project Types

Project Type	Limitation	Workaround
Embedded Platforms	Local changes to BSP sources will not be migrated to the new workspace. A new BSP will be created, and the settings are applied on the new BSP.	Copy the sources to the new BSP manually.

Table 5: Supported Project Types (cont'd)

Project Type	Limitation	Workaround
	Any embedded software repositories added to Vitis IDE will not be migrated. A warning will be shown in the wizard if the project has any local SW repos.	All software repositories need to be migrated to lopper first. Refer to UG1400 for more information. The path to the migrated repository can be manually added to the migration script prior to running the script.
	IP drivers includes in XSAs created with 2023.1 or older releases needs will not work.	Need to regenerate the XSA with 2023.2 release.
Embedded Software Applications	Applications referring to platforms that are outside the current workspace cannot be migrated.	Migrate the platform first and update the application to use the new platform before migrating the application.
HW Link	Hardware linker options defined using the Extra V++ command line options will not be migrated through the script	You will need to manually define these options in the <code>hw_link.cfg</code> for the System project
Accelerated Host applications	Only compile definitions (-D), include path (-I), library paths (-L) and libraries (-l) are migrated for accelerated host applications.	Any other compiler or linker settings from the C/C++ build settings window will need to be set manually in the Application component.

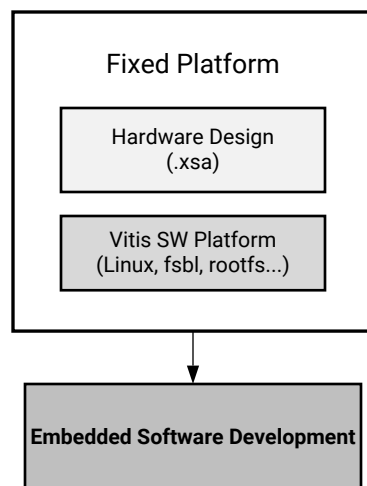
Introduction to Vitis Tools for Embedded System Designers

This chapter is intended for Embedded System Designers who are developing systems using heterogeneous compute elements supported by AMD Vitis™ tools. The chapter introduces key concepts for understanding and using Vitis tools for embedded system design. The elements of the embedded system can include AMD Vivado™ exported hardware designs, Vitis extensible platforms, Arm processor applications, PL kernels, and AI Engine graph applications. Refer to [Terminology for Embedded System Design](#) for definitions of these terms.

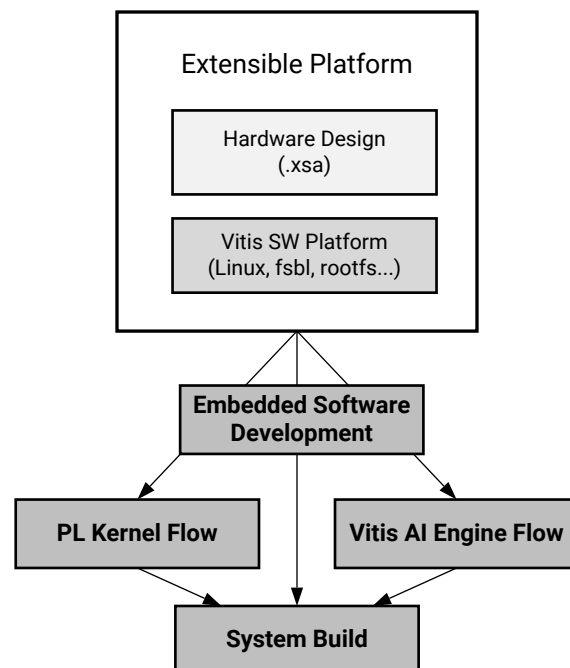
As shown in the figure below, Vitis tools support two different embedded design flows.

Figure 3: Vitis Embedded System Design Flows

Traditional Vitis Embedded SW Flow



Vitis Heterogeneous System Design Flow



X27308-101922

The traditional embedded software development flow relies on a fixed hardware design, processor domains and OS, boot files, software drivers, and Arm processor based software applications. This traditional design flow is described briefly in [Fixed Platforms versus Extensible Platforms](#), and is documented with more detail in the *Vitis Unified Software Platform Documentation: Embedded Software Development (UG1400)*. Refer to that document for more information on the traditional embedded software flow.

The Vitis heterogeneous system design flow enables designing and building embedded system designs using AMD Versal™ adaptive SoC devices, AMD Kria™ SOM, and AMD Zynq™ MPSoC devices. The Embedded Software flow is part of the Vitis Heterogeneous System Design flow. The heterogeneous system design flow is the primary focus of this *Introduction*, highlighting the elements from four different disciplines included in the flow:

1. Hardware design and custom platform development
2. Embedded processor software design
3. Programmable-logic (PL) kernel design
4. Versal AI Engine graph design

The tools and techniques for creating and integrating these different components are the focus of the following sections, beginning with some terminology for a common understanding.

Terminology for Embedded System Design

The following introduces some of the tools and terms you will encounter in this document.

- **Vitis core development kit:** Provides a framework for designing, building, and debugging heterogeneous applications using standard programming languages for both software and hardware components.
- **Vivado Design Suite:** An RTL language design, synthesis, and implementation tool that enables hardware designers to create and export hardware designs (.xsa).
- **Xilinx Support Archive (.xsa):** Is a hardware container exported from the Vivado Design Suite for multiple uses, including in a fixed or extensible platform.
- **Fixed Platform (.xpfm):** Includes a completed hardware design (.xsa) and supporting software files defining the operating system, libraries, and boot files. In this context, "fixed" simply means that the hardware design is complete.
- **Extensible Platform (.xpfm):** The target platform of the Vitis heterogeneous system design flow. In this context, the "extensible" design can be further customized by adding programmable content such as PL kernels and AI Engine graph applications to the platform to build the embedded system. Extensible Platform can also be used to develop software like the fixed platform.

- **PL kernel (.xo):** A hardware function that can be added to the PL region of an extensible platform to define custom hardware. PL kernels can be defined using C++ code in Vitis HLS, or using RTL code and the IP packager feature of the Vivado Design Suite.
- **Vitis HLS:** A high-level synthesis tool that translates C/C++ functions into RTL for implementation in the programmable logic (PL) region of a device. Vitis HLS generates a compiled object (.xo) file that can be imported into the Vitis environment.
- **Vitis Compiler:** The `v++` command used to compile PL kernels (.xo) from C++ code, and to link multiple PL kernels with hardware platforms and AI Engine graph applications to build the device binary.
- **PS Application:** A user-defined software application to be run on an Arm processor in an AMD MPSoC or adaptive SoC device, that can control and interact with PL kernels and AI Engine graph.
- **Xilinx runtime library (XRT):** Provides an API and drivers to let your software application control, transfer data to, and read the status of the PL kernels and AI Engine graph application in the hardware design.
- **AI Engine kernel and graph applications:** Compiled by the Vitis `aiecompiler` and linked into the embedded system with `v++`. Kernels are functions that run on Versal AI Engines and form the fundamental building blocks of a data flow graph application. The AI Engine graph application is an adaptive dataflow graph with deterministic behavior.
- `aiecompiler/aiesimulator`: Vitis tools for the compilation and simulation of AI Engine graph applications.
- **Device Binary (.xc1bin) file:** Contains the programmable device image (PDI) for Versal adaptive SoC or the bitstream for Zynq MPSoC, and metadata needed to control the hardware design.

Fixed Platforms versus Extensible Platforms

A platform provides design context to integrate AI Engine design and PL kernels. Platform hardware is represented by an XSA container built using the Vivado Design Suite, and a platform software that includes a board support package (BSP), in addition to operating systems, boot files, drivers, libraries, and file systems to link application software and build boot images. The Vitis SDK and PetaLinux tools can be used to build BSPs, in addition to system and application software customized to your implemented hardware that includes AI Engine and PL kernels. There are two kinds of platforms as described below.

Traditional Fixed-Platform Design

A fixed platform design is a hardware design completed in the Vivado Design Suite. You can use the Vivado IP integrator feature to stitch together IP in a block design, or you can describe the hardware system in RTL. You can also use Vitis HLS to create hardware functions in C++ and generate Vivado IP to add to your hardware design.

You will use the `write_hw_platform -fixed` command to create a Xilinx Shell Archive (`.xsa`) of the completed hardware design; and use either the Vitis IDE, or the Xilinx Software Command-line Tool (XSCT) to define a platform project, import the fixed `.xsa`, and configure and define processor domains, or BSPs for the fixed-platform. The fixed platform (`.xpfm`) can be used in a traditional embedded software design flow to create embedded applications for Versal adaptive SoC devices, or Zynq MPSoC devices.

The development of software applications for embedded processors requires the use of hardware drivers delivered as part of the exported hardware container (`.xsa`). You must manage and use the drivers to access your hardware design from your software application. This traditional embedded software design flow is fully documented in the *Vitis Unified Software Platform Documentation: Embedded Software Development* ([UG1400](#)).

Extensible Platform-Based Design

The extensible platform also starts with the Vivado Design Suite to define an extensible hardware design (`.xsa`) that allows the addition of programmable components such as AI Engine and PL kernels and subsystem features to quickly customize the resulting embedded system design. The extensible `.xsa` is a container that provides the foundation of the hardware design for the `v++` linker to extend by updating a dynamic region of the design. The extensible platform allows the embedded system designer to easily extend the functionality of the extensible hardware foundation.

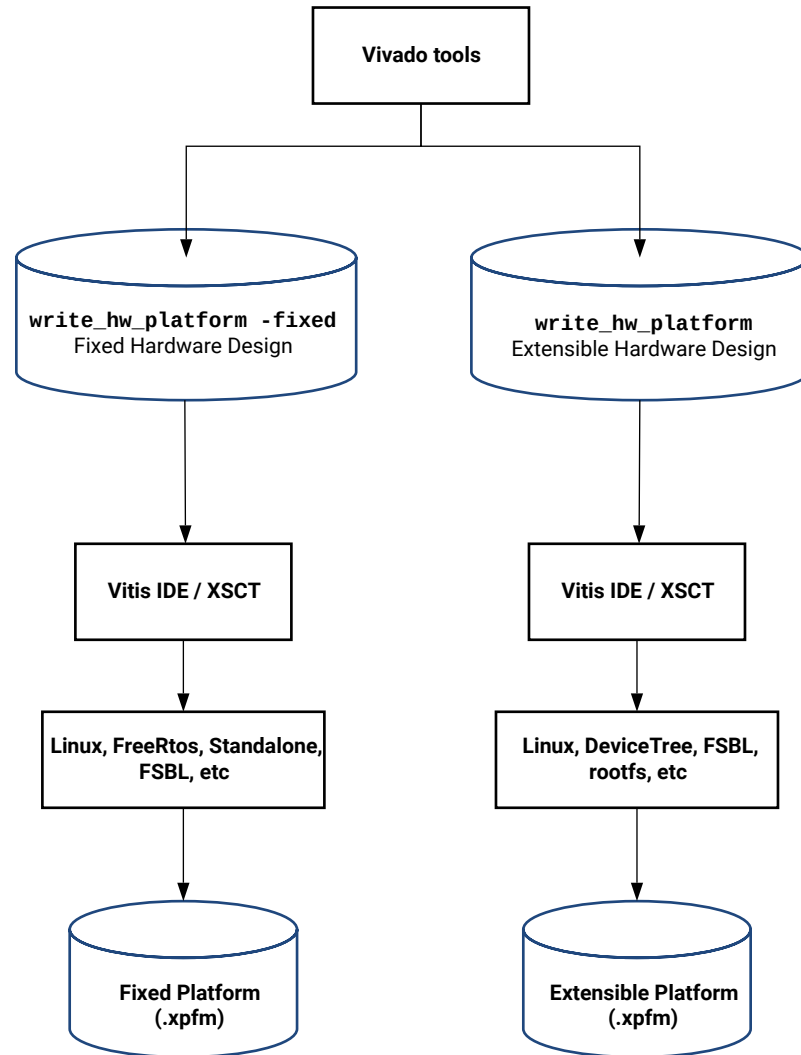
For embedded system designs, a parallel development process can be used where different elements of the heterogeneous system can be developed concurrently. The application team starts work with an AMD base platform, such as the VCK190 or ZCU104, to develop the programmable components of the system. Using an available platform means that the application can be developed and tested independently using a known-good system. At the same time the platform team works to build and validate a custom hardware platform. You are strongly encouraged to use an existing platform design as a reference for your own custom platform designs. Current AMD embedded platforms can be found in the Vitis installation, and their source code can be found in the [Vitis Embedded Platform Source](#) repository on GitHub.

The hardware part of the extensible platform is a Vivado project containing any required IP such as an interrupt controller and clock wizard, in addition to any additional features you want to implement into the hardware design. Extensible platform designs have specific requirements as described in [Creating Embedded Platforms in Vitis](#), such as AXI-4 interfaces for linking PL kernels into the system, memory controllers, one or more clock and reset signals, and a single interrupt. More information on required interfaces can be found at [Adding Hardware Interfaces](#).



TIP: As described in Versal Adaptive SoC Design Guide ([UG1273](#)), you must use the IP integrator to configure and connect required IP such as the CIPS IP, the NoC/DDRMC IP, and hardware debug IP to take advantage of block design automation when building and updating your hardware design. However, you can use the IP integrator feature to configure and connect these critical Versal adaptive SoC IP and then instantiate the resulting hierarchical module into a higher-level RTL design.

Figure 4: Create Fixed or Extensible Platform



X27309-101922

The Tcl command to export a fixed XSA is `write_hw_platform -fixed <.xsa>` and the command to export an extensible XSA is `write_hw_platform -force <.xsa>`. Synthesis and implementation must run before exporting the fixed XS from Vivado.

The hardware design file (`.xsa`) is exported from the Vivado project using the `write_hw_platform` command and imported into a Platform project in the Vitis IDE, or XSCT. In the Platform project the software components (processor domains, DeviceTree, OS) are packaged with the hardware (`.xsa`) to create a custom extensible embedded processor platform (`.xpfm`):

- Operating System includes standalone (or baremetal), FreeRTOS, or Linux. For systems running XRT, Linux is required.
- Processor specifies the Arm core to use for the specified OS domain. The choice of OS determines which processors are available to configure.



TIP: The initial Platform project setup lets you configure one processor domain, but the IDE lets you configure additional domains once the project has been established. Advanced settings of the platform, such as for DFX platforms, must be accessed through XSCT.

When the platform has been built, an `export` folder is added to the hierarchy of the project, and the platform `.xpfm` file and `./hw` and `./sw` folders are added to the platform. The new platform is added to the `$PLATFORM_REPOSTORY_PATHS` environment variable, and the platform can be used for application development with the Vitis tools.

The Vitis platform based design flow extends standard Vivado Embedded Design Methodology to build embedded system hardware, employing Vitis via the `v++` command to link PL kernels and AI Engine graph applications with the extensible hardware platform. This flow promotes concurrent development of the different elements of the system and facilitates the integration process of heterogeneous systems.



IMPORTANT! The platform-based embedded system design flow facilitates hardware platform reuse and re-targeting of accelerator and AI Engine subsystems from pre-validated to custom platforms. This flow can be used broadly, and is required for AI Engine graph applications on Versal AI Core devices and for hardware support of software applications that employ the Xilinx Runtime (XRT).

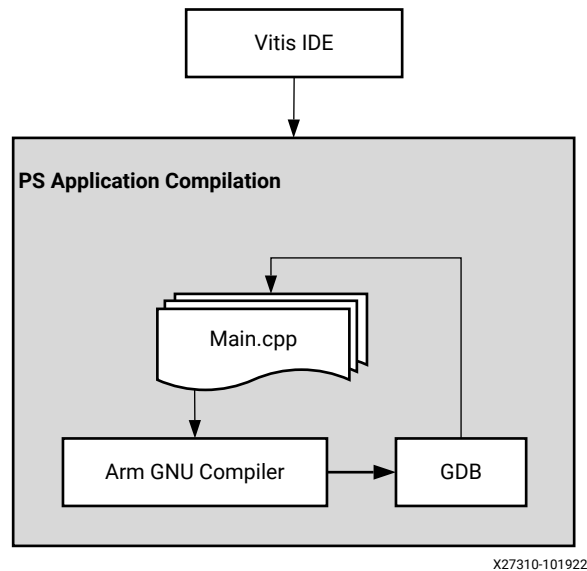
The extensible platform hardware is customized by linking PL kernels (`.xo`) and AI Engine graph applications (`libadf.a`) to the hardware design using the `v++ --link` command. In designing an extensible platform, you have considerable control over the structure and content of the hardware. As described in [Generate XSA file](#), there are minimum requirements associated with CPU control, memory, and clocking; otherwise, you have control to employ the best tools for specific tasks in your platform design.

A minimal hardware platform is ideal for AI Engine and PL kernel subsystem development, and by aligning platform PFM interfaces, you can readily re-target such subsystems to more complete system platforms with minimal or no changes to your source code.

PS Software Development

The PS software development flow is classic software development, with compilation on an Arm-based `g++` cross-compiler, and debugging using GDB. This step can be done in an Application project in the Vitis IDE; or it can be done from the command line or from a `Makefile` using `g++` and `gdb` commands.

Figure 5: Embedded Software Development



As described in [Compiling and Linking the Host Application](#), you will compile the software program to run on a Cortex®-A72 or Cortex-A53 core processor using the GNU Arm cross-compiler to create an ELF file.

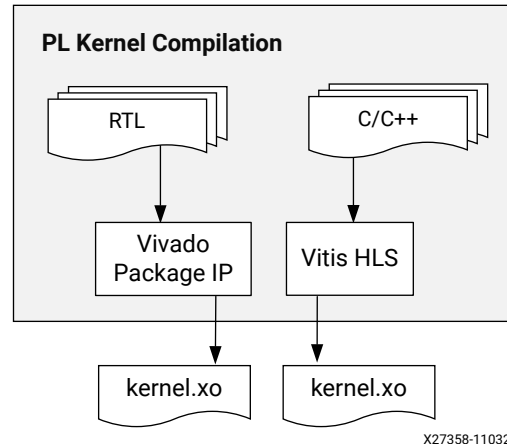
In platform-based design, the software program interacts with kernels in the PL and AI Engine regions of the device through the XRT API. For more information on writing the software program using the API, refer to [Section III: Developing Applications](#). For more information on compiling the software program, refer to [Building the Software Application](#).

Vitis PL Kernel Development Flow

In the platform-based design flow, the platform provides an expandable foundation for hardware development. The hardware design can be expanded through the use of PL kernels linked into the platform using the Vitis tools. PL kernels are hardware functions that can be generated from C++ in the Vitis HLS tool, or described directly in RTL in the Vivado Design Suite.

This section provides a brief overview of the Vitis development flow for PL kernels. A representation of this flow is shown below.

Figure 6: Vitis PL Kernel Development Flow



PL kernels can be written in C++ and synthesized into RTL IP using the Vitis HLS tool. As described in *Vitis High-Level Synthesis User Guide* ([UG1399](#)), there are fundamental concepts that need to be understood to design and write good synthesizable software in such a way that it can be successfully converted to hardware using high-level synthesis (HLS) tools.

PL kernels written in C++ can also be compiled directly into Xilinx object files (.xo) using the `v++ --compile` command as described in [Compiling C/C++ PL Kernels](#). However, the recommended method for C/C++ kernels is to use a bottom-up methodology in Vitis HLS to let you perform analysis and optimization directly on the kernel design to obtain the best results. In fact, the `v++ --compile` command uses Vitis HLS as a key part of the compilation process.

PL kernels written in C++ can also be written in RTL, or may already exist in catalogs of custom IP for use with the Vivado tools. As described in [Packaging RTL Kernels](#), these kernels have specific interface requirements to allow them to be linked into the system design. Embedded system designers can also decide to integrate the RTL as part of their custom platform, or link the RTL kernel into the system using the `v++` command as explained below.

Whether coming from Vitis HLS, RTL kernel, or `v++ --compile` command, the results of kernel development or compilation is a Xilinx object (.xo) file. The PL kernels have specific requirements regarding the different types of supported interfaces, and clocks and reset signals as explained in [PL Kernel Properties](#). The clocks and resets let XRT manage the execution of the kernels at run time, and the interfaces let the PL kernels be linked with an extensible hardware platform (.xpfm) and other .xo files, and AI Engine graph applications (.libadf.a) as described in [Building and Packaging the System](#).

Versal AI Engine Graph Development

Versal adaptive SoCs combine Scalar Engines, Adaptable Engines, and Intelligent Engines with leading-edge memory and interfacing technologies to deliver powerful heterogeneous computing for any application. Most importantly, Versal adaptive SoC hardware and software are targeted for programming and optimization by data scientists, software and hardware developers.

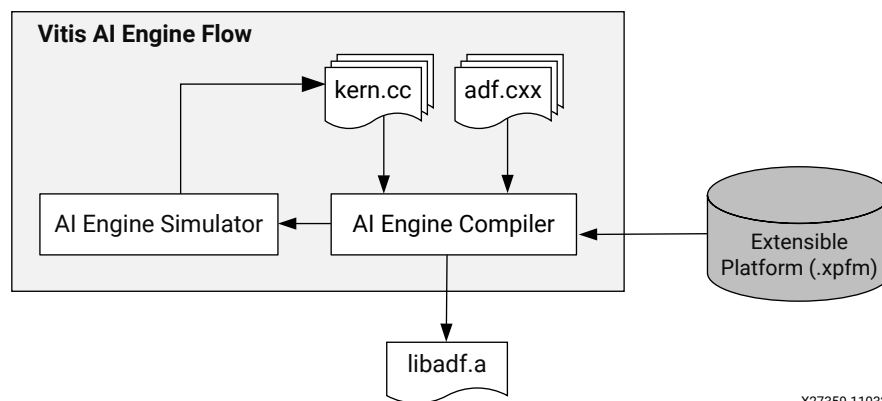
Some Versal adaptive SoC devices incorporate an array of very-long instruction word (VLIW) processors with single instruction multiple data (SIMD) vector units called, AI Engines, that are highly optimized for compute-intensive applications such as 5G wireless and artificial intelligence (AI) applications. Embedded system designs using Versal devices can include AI Engine adaptive data flow (ADF) graph applications developed and tested using Vitis tools.

As described in *AI Engine Kernel and Graph Programming Guide* (UG1079), the ADF graph application consists of nodes and edges where nodes represent compute kernel functions, and edges represent data connections. The ADF graph is a static dataflow graph with the kernels operating in parallel. An AI Engine kernel is a C/C++ program written using specialized intrinsic calls that target the VLIW SIMD vector processor. Kernels operate on data streams, and are the fundamental building blocks of an ADF graph specification. These kernels consume input blocks of data and produce output blocks of data.

The following figure shows the development flow for AI Engine graph applications in which individual kernel code is compiled and combined into the graph application. The AI Engine graph and kernel code is compiled using the AI Engine compiler (`aiecompiler`) or `v++ --mode aie` command to produce an executable file that is run on AI Engine processors.

★ **IMPORTANT!** The inclusion of AI Engine graph applications require the use of extensible platforms in your embedded system design to control the execution of the application.

Figure 7: Vitis AI Engine Development Flow



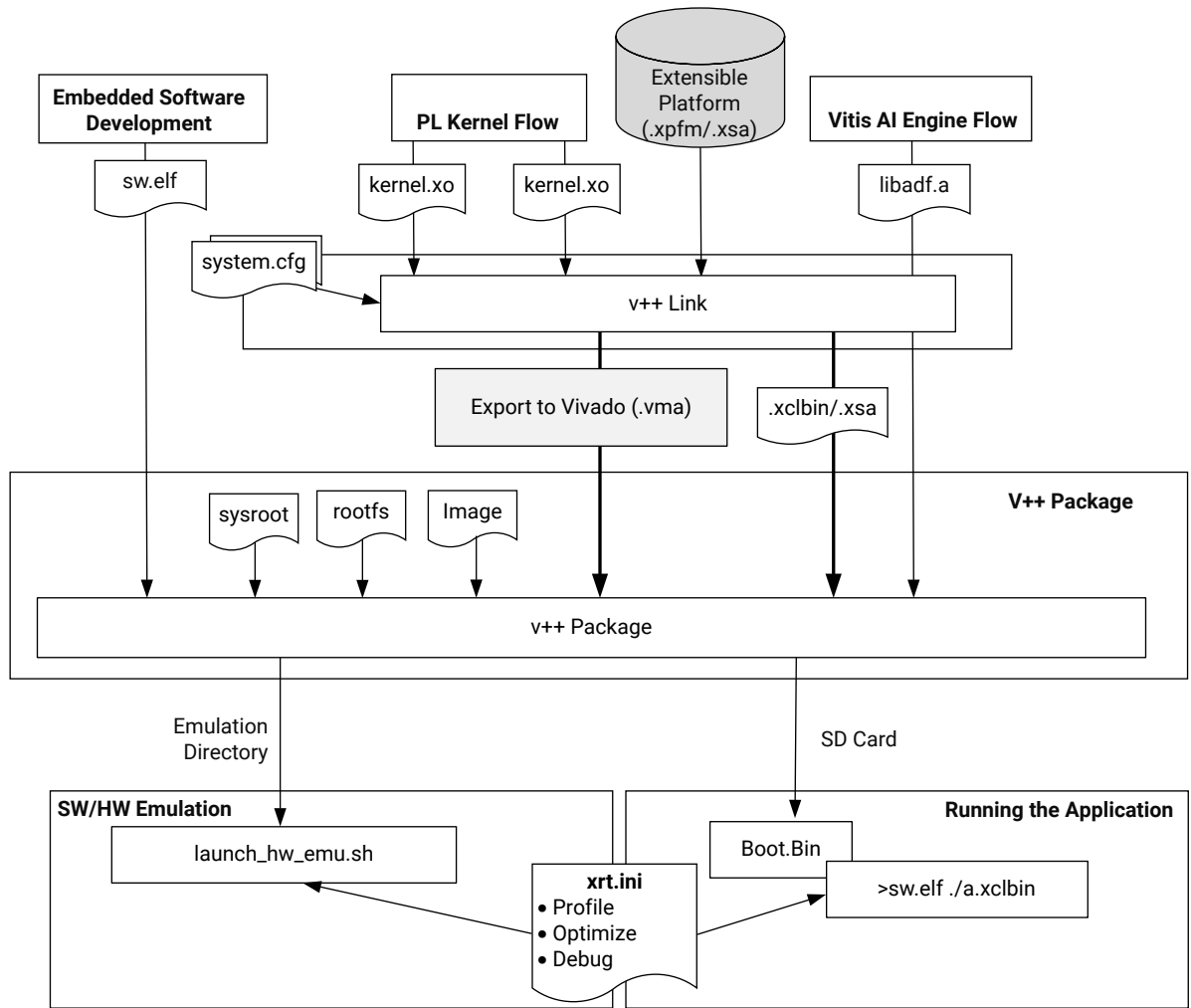
An AI Engine component targeting the `aiengine` domain of a Versal device can be built from the command line as described in [Section IV: Building and Running the System](#), or in the Vitis unified IDE as described in [Section VIII: Using the Vitis Unified IDE](#). The process for building and analyzing an AI Engine graph application is described in brief in this document, and detailed in *AI Engine Tools and Flows User Guide* ([UG1076](#)).

By default, the AI Engine compiler writes all outputs to a directory called `./Work` and creates a file called `libadf.a`, where `Work` is a sub-directory of the current directory where the tool was launched, and `libadf.a` is written to the same directory as the AI Engine compiler was launched from and is used for linking with PL kernels and the extensible platform using the `v++` command as explained in the next section.

Building and Packaging the System

With the elements of the heterogeneous embedded system design available, you are ready to build the system using the Vitis tools. The `v++ --link` command is a key part of the system build process, as is the `v++ --package` command. Both of these commands are shown in the figure below, which highlights the different elements of the system, and how they fit together. At the top of the figure are the extensible platform (`.xpfm`) or hardware `.xsa` file, the embedded software program (`.elf`), the PL kernels (`.xo`), and the AI Engine graph (`libadf.a`) that are elements of the heterogeneous system. Below those are the linking and packaging commands that build the system.

Figure 8: Building and Packaging the Embedded System Design



X27360-110422

System Linking

After the various elements of the embedded system design become available, at least in some initial state, the system can be built using the `v++ --link` command. The Vitis compiler links the kernel `.xo` files of the PL kernels and the `libadf.a` of the AI Engine with the extensible platform (`.xpfm`) or hardware design (`.xsa`) in one of two ways:

- The Vitis integrated flow in which the `v++ --link` command links the elements of the system, synthesizes the design, builds the placed and routed hardware, and produces a device executable (`.xclbin`) file for AMD Kria™ SOM, and Zynq MPSoC devices, or a fixed platform (`.xsa`) for Versal adaptive SoC devices. This integrated flow is required for designs based on standard platforms, such as Alveo Data Center accelerator cards or embedded system platforms such as the `vck190_base`.

- The Vitis Export to Vivado flow that uses the `v++ --link --export_archive` command to create a linked system design and export it to a Vitis MetaData Archive (`.vma`) file that can be imported into the Vivado Design Suite for further design. The Vitis Export to Vivado flow is only supported for system designs on custom Versal platforms specifically designed to take advantage of this flow.

Both of the flows described above use the Vivado Design Suite for synthesis and implementation of the linked system design. The Vitis integrated flow launches synthesis and place and route automatically as part of the linking process. The Vitis Export to Vivado flow exports the linked system design as a `.vma` file, letting you import the file into your Vivado project and continue to design and achieve timing closure directly within the Vivado tool.

The Vitis integrated flow is automated as described in [Linking the System](#), though does offer some opportunity for manual intervention. The linking, synthesis, and implementation processes are broken down into a series of major steps, that can be interrupted to enable customization as explained in [Working with Vivado in the Vitis Integrated Flow](#).

The Export to Vivado flow supports a more hands-on approach for experienced RTL designers as described in [Vitis Export to Vivado Flow](#). This is the recommended approach for developers working with custom Versal platforms; using the Vitis tools to develop and link the AI Engine graph application and connect PL kernels, while using the Vivado tools for greater control over the placement, routing, and timing resolution of the system.

Both of these flows lead into the packaging step described next.

Packaging the System

Meanwhile, in the Vitis tools the `v++ --package` command packages the final product at the end of system design, and configures the boot system for the device. As stated in [Packaging the System](#), the `v++ --package` command will also generate the device binary (`.xclbin`) for Versal platform designs.

The package command controls several aspects of the completed system. For example, in the case of AI Engine designs, the `--package.defer_aie_run` command indicates that the graph application in the `libadf.a` should not be started at boot time, and should instead wait until called expressly from a software application. The `--package.boot_mode` indicates that the system is booted from an SD card, or QSPI/OSPI, and the output produced by the package process is generated accordingly. Refer to [--package Options](#) for a complete list of options.

Finally, the `--package` command lets you define the required files for all platforms to boot and run the embedded system design for software or hardware emulation, or to create an SD card to run your system on hardware. For an embedded system design this process is described in [Packaging for Embedded Platforms](#).

Booting and Running the System

When running the application, you can run software emulation, hardware emulation, or run on the actual physical platform. Running the application on embedded processor platforms is different from running on data center accelerator cards. For more information, refer to [Running the System on Hardware](#) or [Section V: Simulating the Application with the Emulation Flow](#).

- When the build target is software or hardware emulation, the QEMU environment models the hardware device. The Vitis compiler generates simulation models of the kernels in the device binary and running the application runs in the QEMU model of the system. As described in [Working with Build Targets](#), emulation targets let you build, run, and iterate the design over relatively quick cycles; debugging the application and evaluating performance.
- When the build target is the hardware system, the target platform is the physical device. The Vitis compiler generates the `.xclbin` using the Vivado Design Suite to run synthesis and implementation, and resolve timing. Running the application runs your system on the hardware.

Next Steps

This introduction provided numerous references to more detailed information related to topics discussed here. You are strongly encouraged to review that material to develop a deeper understanding of the tools and flows described here.

For an example, on the different elements of the flow, and putting them together to build a complete system, you can refer to the [Versal Custom Thin Platform Extensible System](#) tutorial, based on a thin custom platform (minimal clocks and AXI exposed to PL) including both HLS and RTL kernels and AI Engine graph using a full Makefile build-flow.

You can also look through the [Vitis-Tutorials/AI Engine Development](#) repository for additional examples of the extensible platform development flow.

Finally, there is a chapter on [Vitis-Tutorials/Vitis Platform Creation](#) for some examples of custom embedded processor platform development.

Introduction to Data Center Acceleration for Software Programmers

This chapter is intended for C/C++ software developers who want to accelerate their data center applications using AMD FPGA-based AMD Alveo™ Accelerator Cards. The goal of this guide is to introduce key concepts and provide a pathway for software developers to begin accelerating applications using the AMD Vitis™ compiler and integrated development environment (IDE).

FPGAs offer many advantages over traditional CPU/GPU acceleration, including a custom architecture capable of implementing any function that can run on a processor, resulting in better performance at lower power dissipation. When compared with processor architectures, the structures that comprise the programmable logic (PL) fabric in an AMD device enable a high degree of parallelism in application execution.

The following are key concepts for creating accelerated applications on FPGAs resulting in greater acceleration performance vs CPU:

- Applications written for CPU and FPGA are quite different, and rewriting the functions to be accelerated on an FPGA is required. Functions are executed sequentially on the CPU and must infer parallelism on FPGA for greater performance.
- For application acceleration, the software program is split into a host application that runs on the CPU and compute functions, or kernels that run on the Alveo data center accelerator card. The XRT runtime library provides an API enabling the host application to interact with the kernels on the accelerator cards.
- Data transfers between the host and global memory introduce latency, which can be costly to the overall application. To achieve acceleration in a real system, the performance achieved by the hardware acceleration kernels must outweigh the added latency of the data transfers.
- The software developer should profile the original application and identify functions with the potential to be accelerated. Once the target functions are identified, a performance budget for each kernel should be determined to meet the overall application performance goal.
- The memory hierarchy plays a key role in overall application performance. The memory accessed by kernels should be grouped as memory reads and writes in separate functions using a load-compute-store architecture. The kernels should access contiguous memory if possible and the number of accesses should be optimized by removing redundant accesses or by creating a local cache.

You are encouraged to review this material and the extended material referred to in the following topics. After reviewing this document that lists key concepts and examples along with extended reference material, you should have a practical understanding for developing or modifying existing functions to be targeted for acceleration with proper architecture that meets your performance needs.

Terminology

The following introduces some of the tools and terms you will encounter in this document.

- **Vitis core development kit:** Provides a framework for developing and delivering FPGA accelerated applications using standard programming languages for both software and hardware components. The software component, or host program, is developed using C/C++ to run on a CPU with XRT API calls to manage runtime interactions with the accelerator. The hardware component, or kernel, can be developed using C/C++ or using Verilog or VHDL.
- **Alveo data center accelerator cards:** Are PCI Express® Gen3 x16 compliant cards designed to accelerate compute intensive applications such as machine learning, data analytics, and video processing.
- **Platform:** A predefined configuration of the Alveo accelerator card with features implemented for specific applications. A platform has multiple partitions. The base logic partition (BLP) is a static region that contains fixed logic for essential functions (such as PCIe and DMA). A user logic partition (ULP), which is a dynamic region where the C++ or RTL kernel logic, is programmed for execution.
- **XRT:** The Xilinx Runtime library that provides an API and drivers for your host program to connect with the target FPGA platform and handles transactions between your host program and accelerated kernels.
- **Host and Global Memory:** The distinction between memory on the host machine used by the CPU, and memory on the Alveo data center accelerator card used by the accelerated functions.
- **Vitis HLS:** A high-level synthesis tool that translates C/C++ functions into device logic using programmable logic (PL) elements and RAM/DSP blocks. Vitis HLS synthesizes the C/C++ code into an RTL design and packages it as a compiled object (.xo) file that can be imported into the Vitis environment. There can be multiple functions targeted on the FPGA, each being a separate kernel. Vitis HLS will synthesize these kernels one by one and generate separate .xo files.
- **Register transfer level (RTL):** An abstraction level used for modeling digital circuits. Often the term RTL is used interchangeably for Verilog or VHDL which are both hardware description languages.

- **PL Kernel (.xo) file:** Is the term to designate the custom logic implementation of your accelerator function. Each kernel is packaged as an `.xo` file and contains the IP for the function and associated metadata used by the Vitis tool.
- **RTL Kernel:** Is an RTL design that uses standard AXI4 interfaces to enable the Vitis compiler to link it into the target platform to quickly build the system design. The RTL kernel is packaged as an `.xo` file.
- **Vivado Design Suite:** An RTL language synthesis and implementation tool that takes the RTL design generated by Vitis HLS or an RTL designer and generates the bitstream that can be loaded and executed on the FPGA.
- **Bitstream:** Is the configuration data that is used to program the FPGA so that its functionality can be changed. The kernel design will result in a bitstream used to program the dynamic region of the FPGA on the Alveo accelerator card.
- **Device Binary (.xclbin) file:** Contains the bitstream and other metadata needed to be used to program the FPGA. This is used by XRT APIs to actually program the FPGA. In the Vitis flow, the device binary files have the extension `.xclbin`.
- **Vitis analyzer:** A utility that allows you to view and analyze the reports generated while building and running the application through Vitis, Vitis HLS, and AMD Vivado™.

Working with Alveo Accelerator Cards

What is an FPGA

An FPGA (field-programmable gate array) is an integrated circuit that uses an array of interconnected programmable logic elements to implement any type of digital function as a physical circuit. Because the elements and the routing resources that connect them are configured after power-up, the FPGA can be repeatedly programmed to implement any set of functions required. By creating multiple copies of these functions, FPGAs are particularly well suited at implementing functions in parallel, making them extraordinarily good at serving as hardware accelerators for applications that contain high levels of parallelism. FPGAs come in different sizes with different quantities of programmable logic resources. Larger devices contain more resources, allowing designers to implement more parallel circuits, leading to higher levels of acceleration. The variety of devices provides designers with multiple cost/performance trade-offs.

Unlike GPUs, which contain processing cores that must fetch and execute instructions, FPGAs have a flexible architecture that maps code to physical logic circuitry. Like GPUs, however, it will be necessary for you to understand some of the basics of how this is done to architect your code for best results.

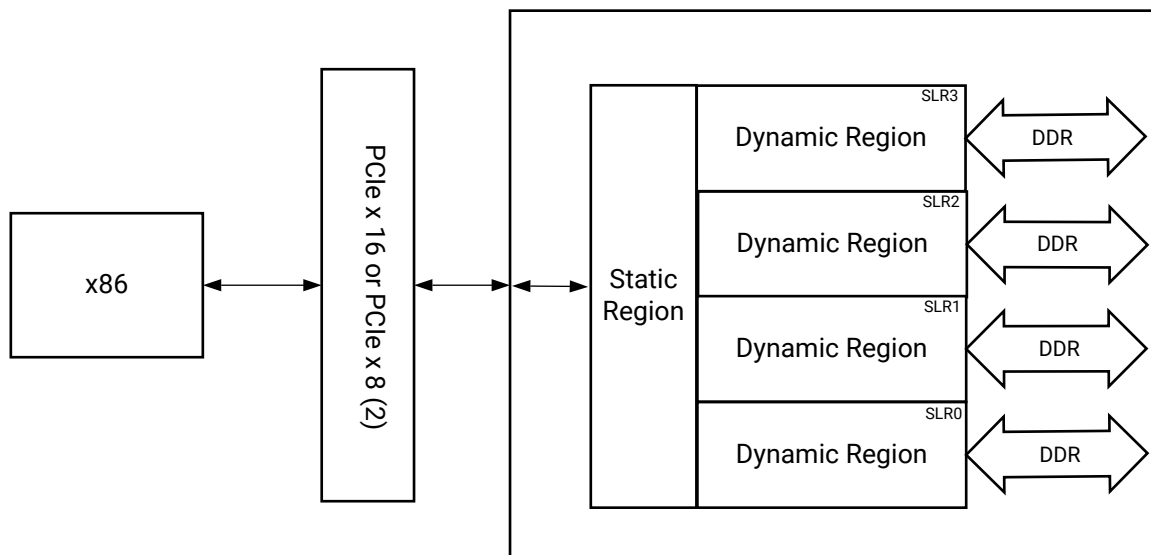
Alveo Block Diagram and Data Movement between Host and FPGA

Using FPGAs at its core, AMD has developed the Alveo family of PCIe Data Center accelerator cards. Each Alveo card combines three essential things: a powerful FPGA for acceleration, high-bandwidth device memory banks, and connectivity to a host server via a high-bandwidth PCIe Gen3x16 link. A number of different cards are available to provide designers with a choice of features and quantity of programmable resources. Below is the block diagram for the Alveo U250.

Although FPGAs are essentially blank devices that get configured at power-up, all Alveo cards are shipped with target platforms that provide the firmware to configure the accelerator card for specific uses. The platform must be installed with Xilinx Runtime (XRT); flashed into the device during installation, or when changing the configuration of the accelerator card.

On the AMD device, the platform consists of two physical FPGA partitions: *Shell* and *User*. Shell partition is a static region and provides basic infrastructure for the platform like PCIe connectivity, board management, sensors, clocking, and reset. User partition is a dynamic region that contains user compiled binary called `.xclbin` which is loaded by XRT during execution. RTL kernels are the custom logic created by the developer and programmed into the dynamic region. In this document, kernels refer to the functions that the designer is implementing into the dynamic region of the Alveo accelerator card.

Figure 9: Alveo Block Diagram



X26735-060122

The PCIe interface is used for communication between the host and accelerator card, and to transfer data from the host into the Alveo card's device memory. This device memory serves as a Global memory, accessible by both host and hardware accelerators. The device memory included on the Alveo platform are PLRAM (small size but fast access with the lowest latency), HBM (moderate size and access speed with some latency), and DDR (large size but slow access with high latency). Depending upon the Alveo card, you may have DDR or HBM, or even both.

The block diagram shown above is of U250 and has 4 banks of DDR, each with 16GB of memory. The FPGA on the Alveo card is further subdivided into multiple super logic regions (SLRs), which aid in the architecture of very high-performance designs. But this is a slightly more advanced topic that will remain largely unnoticed as you take your first steps into Alveo development.

To further improve performance, and minimize access to DDR memory, FPGAs have large quantities of small, internal RAM blocks. These are completely configurable by the compiler to ensure that buffering can be created between tasks to enable pipeline-style computation. This effectively eliminates the need for caches and is one of the key strengths of FPGAs.

There are many more details you could learn about the FPGA architecture and Alveo cards, but this is sufficient for introductory purposes. From the perspective of designing an FPGA-based acceleration architecture, the important points to remember are:

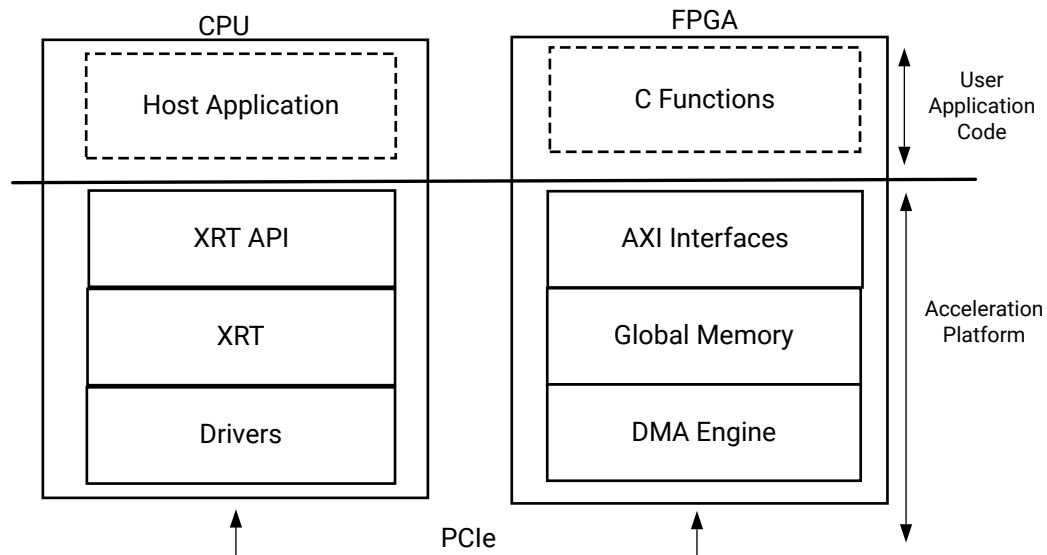
- Moving data across PCIe is expensive, even at Gen3x16, latency is high. For larger data transfers, bandwidth can easily become a system bottleneck.
- Bandwidth and latency between the DDR4 and the FPGA are significantly better than over PCIe, but touching external memory is still expensive in terms of overall system performance.

A Sample Application

This section provides a snapshot of the evolution of a program written for CPU into an application written for FPGA-based acceleration. This section is primarily intended to showcase key ideas for building your application without going into details. You may come across several new terms here, but you can refer to [Terminology](#) for some definitions.

The figure below illustrates the execution flow of the Vitis application acceleration environment. The application program is split between an application running on CPU (called the host program) and hardware-accelerated kernels running on FPGA with a communication channel between them. The host program, written in C/C++ and using the XRT API, is compiled into an executable that runs on an x86 based host processor while hardware-accelerated kernels are compiled into an executable device binary (.xclbin) that runs within the programmable logic (PL) region of an AMD device on the Alveo accelerator card.

Figure 10: CPU/FPGA Interaction



X27432-112922

The API calls managed by XRT are used to process transactions between the host program and the hardware accelerators. Communication between the host and the kernel, including control and data transfers, occurs across the PCIe bus. The execution model of a Vitis application can be broken down into the following steps:

1. The host program writes the data needed by a kernel into the global memory of the attached device through the PCIe interface on an Alveo Data Center accelerator card.
2. The host program sets up the kernel with its input parameters.
3. The host program triggers the execution of the kernel function on the FPGA.
4. The kernel performs the required computation while reading data from global memory, as necessary.
5. The kernel writes data back to global memory and notifies the host that it has completed its task.
6. The host program reads data back from global memory into the host memory and continues processing as needed.

The following is a simple program written in C++ for execution on the CPU. This program includes the `compute()` function to be accelerated as a kernel on an Alveo accelerator card.

```
#include <vector>
#include <iostream>
#include <ap_int.h>
#include "hls_vector.h"

#define totalNumWords 512
unsigned char data_t;

int main(int, char**) {
```

```

// initialize input vector arrays on CPU
for (int i = 0; i < totalNumWords; i++) {
    in[i] = i;
}
compute(data_t in[totalNumWords], data_t Out[totalNumWords]);
check_results();
}

void compute (data_t in[totalNumWords ], data_t Out[totalNumWords ]) {
    data_t tmp1[totalNumWords], tmp2[totalNumWords];
    A: for (int i = 0; i < totalNumWords ; ++i) {
        tmp1[i] = in[i] * 3;
        tmp2[i] = in[i] * 3;
    }
    B: for (int i = 0; i < totalNumWords ; ++i) {
        tmp1[i] = tmp1[i] + 25;
    }
    C: for (int i = 0; i < totalNumWords ; ++i) {
        tmp2[i] = tmp2[i] * 2;
    }
    D: for (int i = 0; i < totalNumWords ; ++i) {
        out[i] = tmp1[i] + tmp2[i] * 2;
    }
}

```

The program looks very similar to any other C++ program where the main function calls a compute function, setting up the data to be sent to compute function, and checking the results with golden results after compute function completes. The execution of this program is sequential on the CPU. This program can also run sequentially on an FPGA, producing correct results without any performance gain compared to the CPU. For the application to execute with higher performance on an FPGA, the program needs to be re-architected to enable parallelism at various levels. Examples of parallelism can include:

- The compute function can start before all the data is transferred from the host to the compute function
- Multiple compute functions can run in an overlapping fashion, for example a "for" loop can start the next iteration before the previous iteration has completed
- The operations within a "for" loop can run concurrently on multiple words and doesn't need to be executed on a per-word basis

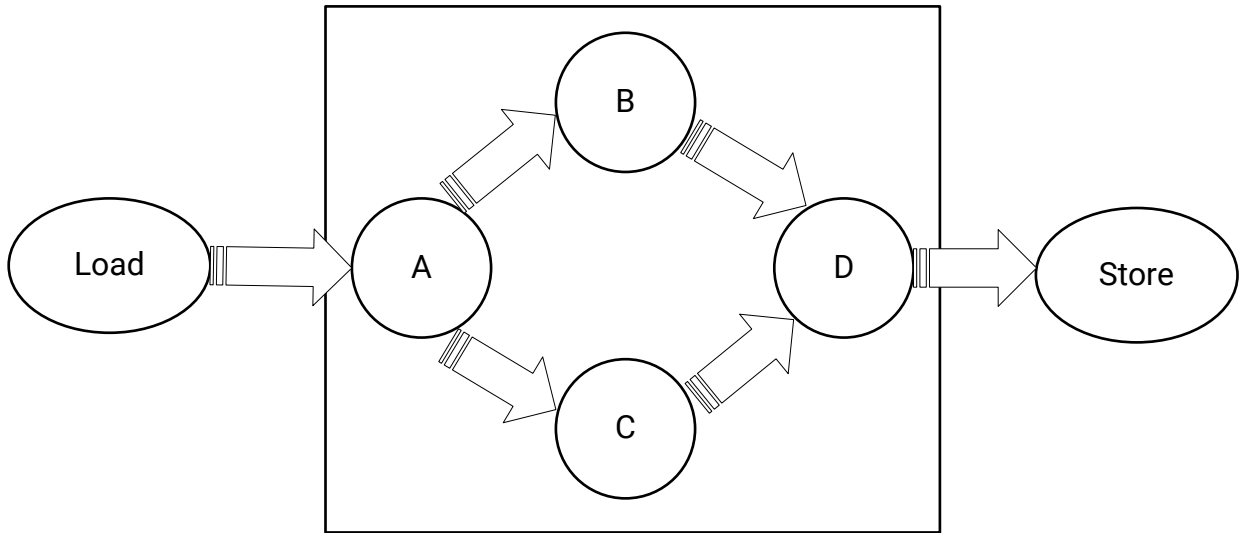
You will need to re-architect the compute function that resides on the FPGA as an accelerated kernel, and the host application that runs on the CPU and communicates with the accelerated kernels.

Re-Architecting Kernel Code

From the prior example it is the `compute()` function that needs to be re-architected for FPGA-based acceleration.

In the `compute()` function Loop A multiplies the input with 3 and creates two separate paths, B and C. Loop B and C performs operations and feed the data to D. This is a simple representation of a realistic case where you have several tasks to be performed one after another and these tasks are connected to each other as a network like the one shown below.

Figure 11: Kernel Architecture



X27496-120522

Here are the key takeaways for re-architecting the kernel code are:

- Task-level parallelism is implemented at the function level. To implement task-level parallelism loops are pushed into separate functions. The original `compute()` function is split into multiple sub-functions. As a rule of thumb, sequential functions can be made to execute concurrently, but sequential loops will execute sequentially.
- These tasks (or sub-functions) are communicating with each other using `hls::stream` which acts as a FIFO channel. The `hls::stream` class is a C++ template class for modeling streams behavior between functions.
- Instruction-level parallelism is implemented by reading 16 32-bit words from memory (or 512-bits of data). Computations can be performed on all these words in parallel. The `hls::vector` class is a C++ template class for executing vector operations on multiple samples concurrently.
- The `compute()` function needs to be re-architected into load-compute-store sub-functions, as shown in the example below. The load and store functions encapsulate the data accesses and isolate the computations performed by the various compute functions.

- Additionally, there are compiler directives starting with `#pragma` that can transform the sequential code into parallel execution.

```
#include "diamond.h"
#define NUM_WORDS 16
extern "C" {

void diamond(vecOf16Words* vecIn, vecOf16Words* vecOut, int size)
{
    hls::stream<vecOf16Words> c0, c1, c2, c3, c4, c5;
    assert(size % 16 == 0);

    #pragma HLS dataflow
    load(vecIn, c0, size);
    compute_A(c0, c1, c2, size);
    compute_B(c1, c3, size);
    compute_C(c2, c4, size);
    compute_D(c3, c4, c5, size);
    store(c5, vecOut, size);
}
}

void load(vecOf16Words *in, hls::stream<vecOf16Words >& out, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS performance target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in[i]);
    }
}

void compute_A(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out1, hls::stream<vecOf16Words >& out2, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS performance target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        vecOf16Words t = in.read();
        out1.write(t * 3);
        out2.write(t * 3);
    }
}

void compute_B(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{
    Loop0:
    for (int i = 0; i < size; i++)
    {
        #pragma HLS performance target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in.read() + 25);
    }
}

void compute_C(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{

```

```

Loop0:
    for (data_t i = 0; i < size; i++)
    {
        #pragma HLS performance target_ti=32
        #pragma HLS LOOP_TRIPCOUNT max=32
        out.write(in.read() * 2);
    }
}

void compute_D(hls::stream<vecOf16Words >& in1, hls::stream<vecOf16Words >&
in2, hls::stream<vecOf16Words >& out, int size)
{
    Loop0:
        for (data_t i = 0; i < size; i++)
        {
            #pragma HLS performance target_ti=32
            #pragma HLS LOOP_TRIPCOUNT max=32
            out.write(in1.read() + in2.read());
        }
}

void store(hls::stream<vecOf16Words >& in, vecOf16Words *out, int size)
{
    Loop0:
        for (int i = 0; i < size; i++)
        {
            #pragma HLS performance target_ti=32
            #pragma HLS LOOP_TRIPCOUNT max=32
            out[i] = in.read();
        }
}

```

Re-Architecting the Host Application

The main function in the original program is responsible for setting up the data, calling the compute function, checking the results, etc. In the case of an accelerated application, the host code is responsible for initializing the data to be sent/received over the PCIe® bus to the device memory. It also sets the kernel function arguments similar to how the main function calls compute functions. The API calls, managed by XRT, are used to process transactions between the host program and the hardware accelerators.

In general, the structure of the host application can be divided into the following steps:

1. Loading the `.xclbin` generated into the program.
2. Allocate buffers in the global memory
3. Create the input test data and map the buffers to the host memory
4. Setting up the kernel and kernel arguments.
5. Transferring buffers between the host and kernels
6. Execute the kernel.
7. Receive the output results back to the host into output buffers

The host application re-written for the `compute()` function described above, making use of the XRT native API to run on the Alveo accelerator card is shown below:

```
// XRT includes
#include "experimental/xrt_bo.h"
#include "experimental/xrt_device.h"
#include "experimental/xrt_kernel.h"

#include "types.h"

int main(int argc, char** argv) {
    unsigned int device_index = 0;
    auto uuid = device.load_xclbin("diamond.hw.xclbin");

    size_t vector_size_bytes = sizeof(int) * totalNumWords;
    auto krnl = xrt::kernel(device, uuid, "diamond");

    std::cout << "Allocate Buffer in Global Memory\n";
    auto bufIn = xrt::bo(device, vector_size_bytes, krnl.group_id(0));
    auto bufOut = xrt::bo(device, vector_size_bytes, krnl.group_id(1));

    // Map the contents of the buffer object into host memory
    auto bufIn_map = bufIn.map<int*>();
    auto bufOut_map = bufOut.map<int*>();
    std::fill(bufIn_map, bufIn_map + totalNumWords, 0);
    std::fill(bufOut_map, bufOut_map + totalNumWords, 0);

    // Create the input data
    for (int i = 0; i < totalNumWords; i++)
        bufIn_map[i] = (uint32_t)i;

    // Create the output golden data
    int bufReference[totalNumWords];
    for (int i = 0; i < totalNumWords; ++i) {
        bufReference[i] = ((i*3)+25)+((i*3)*2);
    }

    // Synchronize buffer content with device side
    bufIn.sync(XCL_BO_SYNC_BO_TO_DEVICE);

    std::cout << "Execution of the kernel\n";
    auto run = krnl(bufIn, bufOut, totalNumWords/16);
    run.wait();

    // Get the output;
    std::cout << "Get the output data from the device" << std::endl;
    bufOut.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

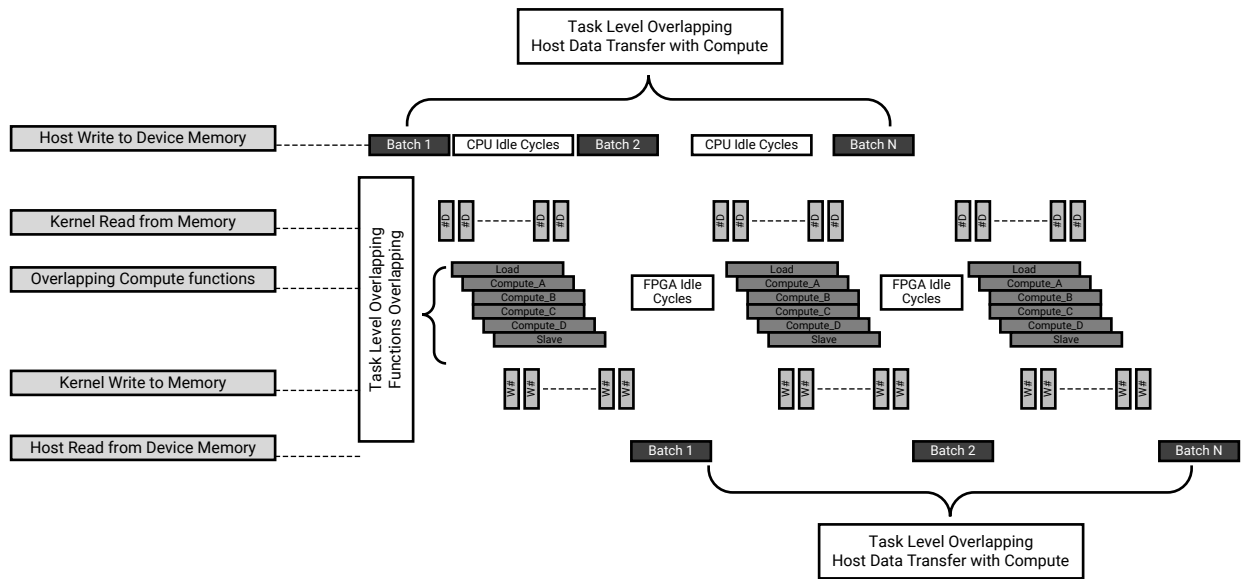
    for (int i = 0; i < totalNumWords; i++)
    {
        std::cout << "Referece  " << bufReference[i] << std::endl;
        std::cout << "Out      " << bufOut_map[i] << std::endl;
    }

    // Validate our results
    if (std::memcmp(bufOut_map, bufReference, totalNumWords))
        throw std::runtime_error("Value read back does not match
reference");
    std::cout << "TEST PASSED\n";
}
```


Application Execution Timeline

When run on the Alveo accelerator card, the application timeline looks like the following.

Figure 12: Application Timeline



X27500-120522

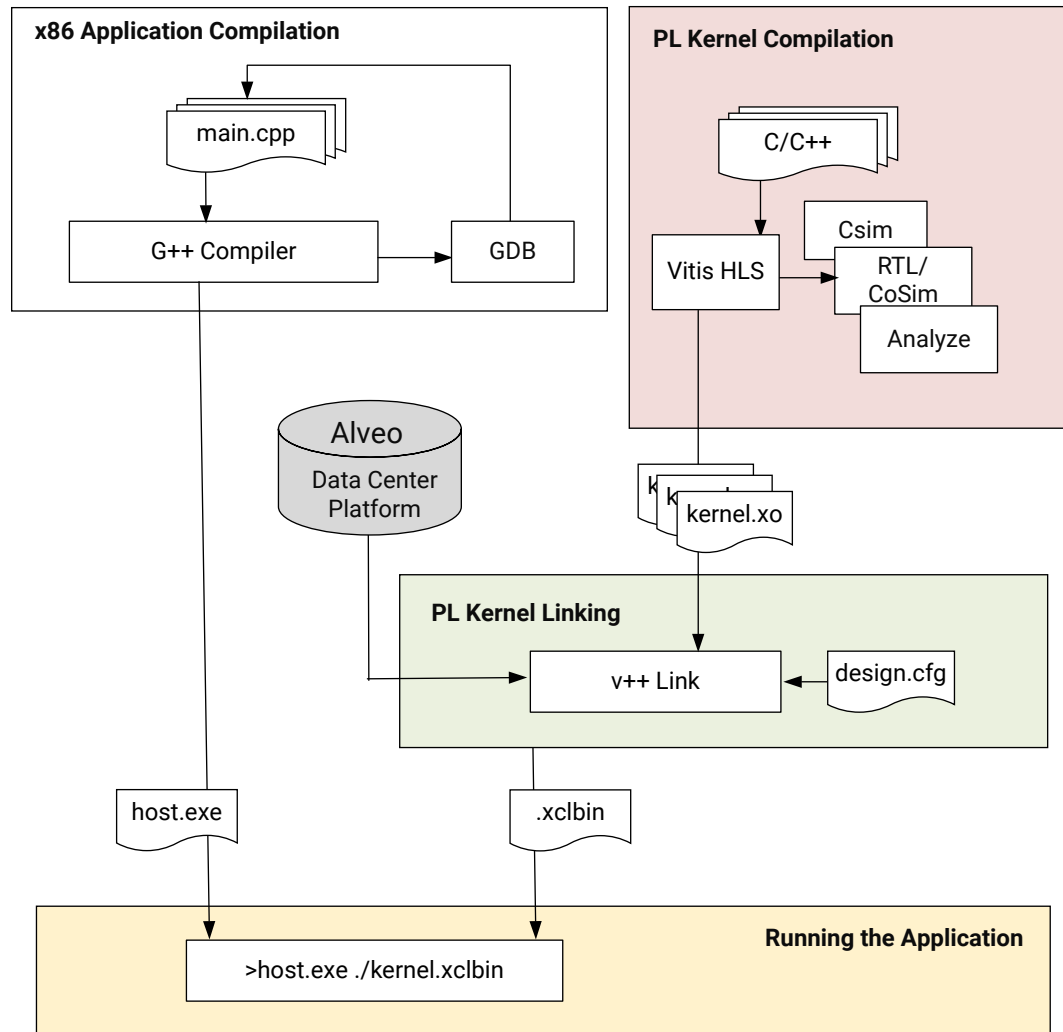
The execution of the application on an FPGA is quite different than on a CPU due to several types of parallelism that can be observed from figure above. The kernel code was written to leverage task-level parallelism by creating sub-functions for each loop. The result is that `compute_A`, `compute_B`, `compute_C`, and `compute_D` are running in an overlapping fashion. In fact, `compute_A`, `compute_B`, `compute_C`, and `compute_D` are sub-function calls within the compute function. A similar execution overlap can be accomplished for multiple kernels.

While the hardware device and its kernels are designed to offer potential parallelism, the software application must be engineered to take advantage of this potential parallelism. Task-level parallelism is further enabled by overlapping host-to-device data transfers and overlapping the compute function execution.

Vitis Application Development Flow

In this section, you will learn about the Vitis application development flow first and then review a methodology for re-architecting a CPU application for FPGA-based acceleration; re-coding each kernel to meet the overall performance objective. An image of the Vitis application development flow is shown below.

Figure 13: Vitis Development Flow



X24704-052421

The development flow includes the following steps:

- **Application Compilation using G++:**

The host program written in C/C++, and using XRT native API, is compiled using a g++ compiler to create a host executable file to run on the x86 processor. The host program interacts with kernels in the PL region on the FPGA device.

- **Kernel Compilation using Vitis HLS:**

Vitis HLS is a compiler that takes C/C++ source code as an input and synthesizes it into an RTL design that is optimized for AMD FPGA products. Each C++ kernel must be synthesized using Vitis HLS to produce a Xilinx object (`.xo`) file. One or more `.xo` files can be paired for linking using Vitis linker to produce the `.xclbin` file.

The steps for kernel development in Vitis HLS are as follows:

1. Write the C/C++ code for the function
2. Verify the Code using C-simulation
3. Build the kernel using C-synthesis
4. Verify the kernel generated with C++ outputs
5. Review the HLS synthesis reports and co-simulation reports to analyze performance
6. Repeat previous steps until performance goals are met.

- **PL Kernel Linking using Vitis Tools:**

Xilinx object (.xo) files are linked with the target hardware platform by the Vitis linker to create a device binary file (.xclbin) that is loaded for execution on the Alveo accelerator card.



TIP: This step will call Vivado place and route to generate the .xclbin file.

To help define the architecture of the device binary, a configuration file can be created specifying option like how many instances of a kernel (or Compute Unit) should be built in the device binary, how are the kernels connected to the global memory, or to other kernels, etc. This configuration file is passed to the Vitis linker to generate the .xclbin.

There are three different build targets of the Vitis Compiler that defines the nature and contents of the generated .xclbin file. Two emulation targets used for validation and debugging purposes: software emulation for C-based simulation, and hardware emulation for RTL co-simulation; and one hardware target for building the final project output to run on the Alveo card. The same host program can be used to run any of the .xclbin targets.



TIP: Compiling for an emulation target is significantly faster than compiling for actual hardware. The emulation run is performed in a simulation environment, which offers enhanced debug visibility and does not require a physical accelerator card.

- **Running the Application:** Finally, when you run the application the host program loads the .xclbin file generated by Vitis Compiler. The host application always runs on the CPU and can be run in emulation mode on x86, or run on the actual physical accelerator platform.

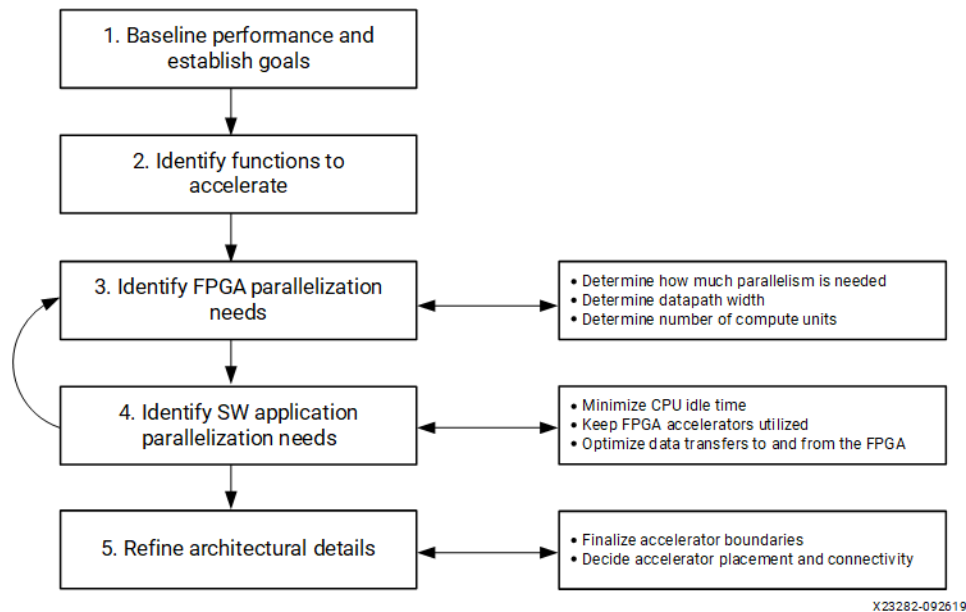
Developing Vitis Accelerated Applications

The methodology is comprised of two major phases:

1. Architecting the application and identifying kernels with performance goals defined. The developer makes key decisions about the application architecture by determining which software functions should be mapped to device kernels, how much parallelism is needed, and how it should be delivered.

2. Developing the C/C++ kernels to meet the goals established. The developer implements the kernels. This task primarily involves structuring source code and applying the desired compiler pragma to create the desired kernel architecture and meet the performance target. Review the [Design Principles for Software Programmers](#) intended for software developers who want to understand the process of synthesizing accelerated hardware from a software algorithm written in C/C++

Figure 14: Methodology for Architecting the Application



The preceding image highlights the key tasks related to architecting the application:

- Profile the C++ application using Valgrind, callgrind, and gprof to create the baseline for analysis. The functions that consume the most execution time are good candidates to be offloaded and accelerated onto FPGAs.
- The maximum achievable throughput is limited by the PCIe bus. PCIe performance is influenced by many different aspects, such as motherboard, drivers, target platform, and transfer sizes. Run DMA tests upfront to measure the effective throughput of PCIe transfers and thereby determine the upper bound of the acceleration potential, such as the `xbutil dma` test.
- Identify the performance bottlenecks by reviewing the algorithm and analyzing any parallel paths. Accelerating one path may not give the expected acceleration for the overall application. When looking for acceleration candidates, consider the performance of the entire application instead of only individual functions.
- Identify the overall acceleration potential, set the application performance goal.
- After the functions to be accelerated are identified and the overall acceleration goals are established, the next step is to determine what level of parallelization is needed to meet the goals.

- Enable parallelism between host and device data transfer and compute on FPGA so that there is minimal idle time. Keep the device kernels active performing new computations as early and often as possible. Optimize data transfers to and from the device.

For a more complete examination of this topic, refer to [Accelerating Data Center Applications with Vitis Software Platform](#), or refer to the Design Principles for Software Programmers section of the Vitis HLS User Guide ([UG1399](#)).

Methodology for Developing C/C++ Kernels

The software program can be automatically converted (or synthesized) into hardware, but achieving acceptable quality of results (QoR) will require additional work such as rewriting the software to help the Vitis HLS tool achieve the desired performance goals. To help, you need to understand the best practices for writing good software for execution on the FPGA device. The next few sections will discuss how you can first identify some macro-level architectural optimizations to structure your program and then focus on some fine-grained micro-level architectural optimizations to boost your performance goals. The following key kernel requirements for optimal application performance should have already been identified during the architecture definition phase:

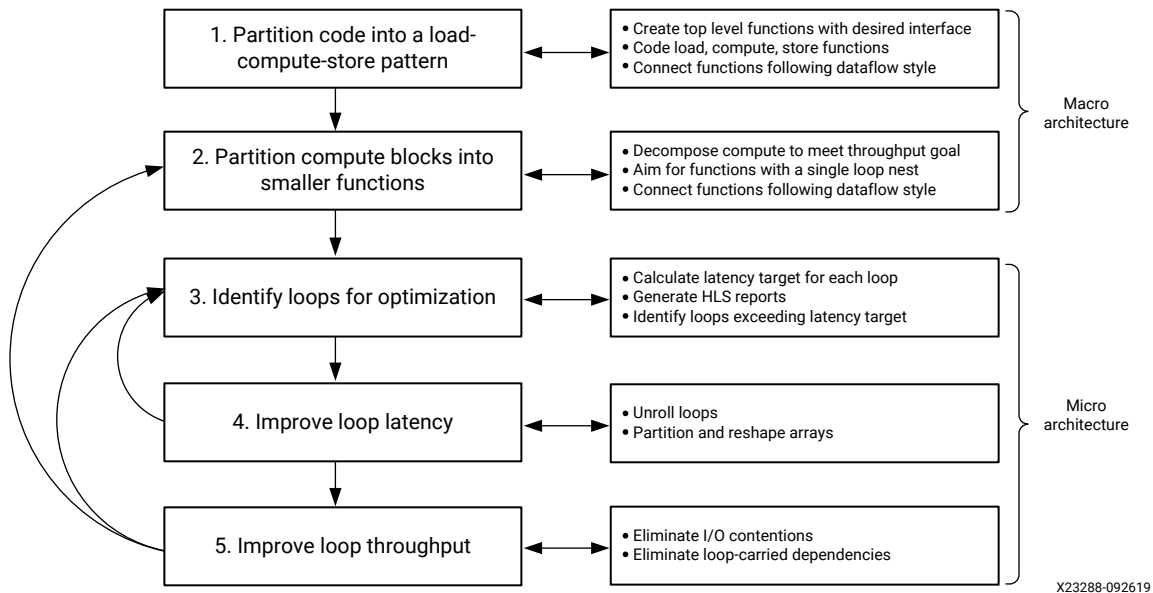
- Throughput goal
- Latency goal
- Datapath width
- The number of accelerated kernels.
- Interface bandwidth

These requirements drive the kernel development and optimization process. Achieving the kernel throughput goal is the primary objective, as overall application performance is predicated on each kernel meeting the specified throughput.

The kernel development methodology, therefore, follows a throughput-driven approach and works from the outside-in. This approach has two phases, as also described in the following figure:

1. Defining and implementing the macro-architecture of the kernel
2. Coding and optimizing the micro-architecture of the kernel

Figure 15: Kernel Development Methodology



Refer to [Methodology for Developing C/C++ Kernels](#) for a detailed view on requirements, considerations, and how to re-architect the code for achieving higher performance.

Best Practices for Kernel Development

You have reviewed some basic understanding of the Alveo accelerator card and its key components, how the data moves between the CPU and Alveo card. You have also been exposed to the recommended guidelines for creating Vitis applications. This section will cover more in-depth topics that are key concepts of coding using Vitis HLS.

Mapping Function Arguments to HW Interfaces

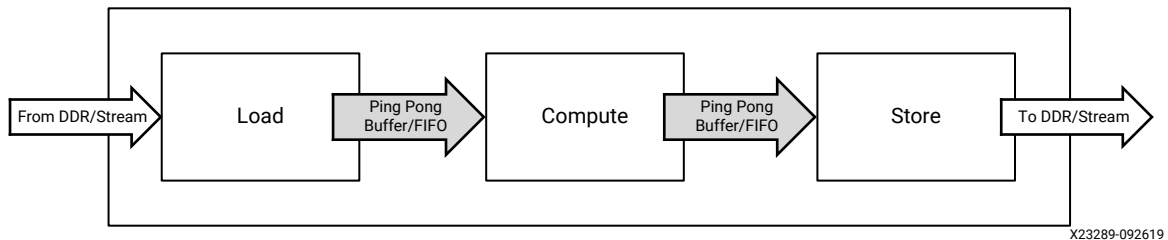
The Vitis HLS tool automatically assigns interface ports for the arguments of your C/C++ kernel function. These function arguments are of either scalar or pointer/array types. The parameters from the host are written directly to the registers of the accelerators. The buffers are kept external in the global memory and the accelerator reads and writes from this global memory.

The scalar type function arguments are used for parameters and the pointer or array type arguments are used for accessing global memory. The Vitis HLS implements these interface ports as AXI Protocol. Refer to [Introduction to AXI](#) for more information on this interface protocol.

Load - Compute - Store

The algorithm should be structured as load-compute-store with communications channels in between as shown below.

Figure 16: Load-Compute-Store Pattern



- The `load` function is responsible for moving data into the kernel from the device memory. This function does not perform any data processing but focuses on efficient data transfers, including buffering and caching if necessary.
- The `compute` function, as its name suggests, is where all the processing is done. At this stage of the development flow, the internal structure of the compute function is not important.
- The `store` function mirrors the load function. It is responsible for moving data out of the kernel, taking the results of the compute function, and transferring them to global memory outside the kernel.

The developer needs to code memory accesses in a way to minimize the overhead of global memory accesses, which means maximizing the use of consecutive accesses so that bursting can be inferred. The burst access hides the memory access latency and improves the memory bandwidth.

Additionally, the maximum data width from the global memory to and from the kernel is 512 bits. To maximize the data transfer rate, it is recommended that you use this full data width. By default in the Vitis kernel flow the Vitis HLS tool automatically re-sizes the kernel interface ports up to 512-bits to improve burst access.

Creating a load-compute-store structure that meets the performance goals starts by engineering the flow of data within the kernel. Some factors to consider are:

- How does the data flow from outside the kernel into the kernel?
- How fast does the kernel need to process this data?
- How is the processed data written to the output of the kernel?

Load-Compute and Compute-Store communicate over the streaming channel. Streaming is a type of data transfer in which data samples are sent in sequential order starting from the first sample. Streaming requires no address management and can be implemented with FIFOs. As soon as sufficient data is available for the compute function, the computation can start. Similarly, as soon as the data is available for the Store function, the data can be sent to the DDR over the AXI4 Master interface.

Example - Dataflow using FIFOs

Task Level Parallelism

The developer needs to assess the algorithm and determine how task-level parallelism can be accomplished. This type of parallelism can be enabled in two dimensions.

1. The tasks can execute in an overlapping fashion with each other. In other words, Compute functions can start based on the data availability and don't require the previous function to finish first. With the data flow enabled, the tool will infer this type of parallelism.
2. The task can restart itself within a given time, called the "Transaction Interval." In other words, the next invocation of the same compute function can be restarted before its previous invocation is completely done. The Vitis tool provides the compiler directive for the performance target for any loop. When this directive is added, the compiler will automatically do the necessary transformations or combinations of transformations like partitioning the arrays, unrolling the nested loops, or pipeline the loops to meet the "Target Interval" goal.

For more information on function and loop pipelining, loop unrolling, and array partitioning, see *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

Verifying Functional Correctness of the Kernel

When using the Vitis HLS design flow, it is time-consuming to synthesize functionally incorrect C code and then analyze the implementation details to determine why the function does not perform as expected. Therefore, the first step in high-level synthesis should be to validate that the C function is correct, before generating RTL code, by performing C-simulation using a well-written test bench. Writing a good test bench can greatly increase your productivity, as C functions execute in orders of magnitude faster than RTL simulations. Using C to develop and validate the algorithm before synthesis is much faster than developing and debugging RTL code. The same C-based test bench can be used to run C/RTL co-simulation to automatically verify the RTL design generated.

For further review of this subject, see *Vitis High-Level Synthesis User Guide* ([UG1399](#)), which includes the following material:

- Writing a Test Bench
- Verifying Code with C Simulation
- C/RTL Co-Simulation

- The Vitis HLS Analysis and Optimization Tutorial will work through the Vitis HLS tool GUI to build, analyze, and optimize a hardware kernel.
- Review the checklist in [Best Practices for Designing with M_AXI](#) for best practices to use when designing interfaces for your application.

Best Practices for Host Programming

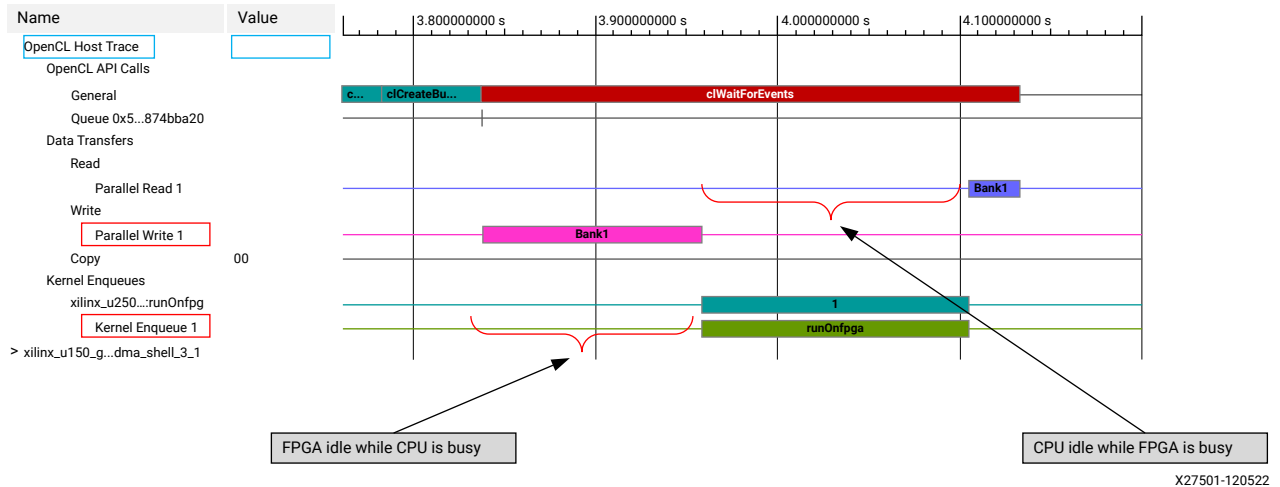
In the Vitis environment, the host application can be written in C++ using the Xilinx Runtime (XRT) native C++ API. Just as re-architecting the kernel code is required to enable parallelism on the hardware and optimize the memory accesses, host programming is equally important to ensure high performance over the CPU. You can have an optimized kernel to meet the required performance but the application performance won't be optimal if the utilization of CPU and FPGA is not high. Here are some of the considerations while creating a host program:

- Reducing the overhead of kernel enqueueing: There is an overhead of dispatching the commands and arguments by the host to the kernel before enqueueing the kernel. You can reduce the impact of this overhead by minimizing the number of times the kernel needs to be enqueued by the host.
- Maximize the data transfer bandwidth between the host and device: The data transfer between host and device memory should be large enough to maximize the PCIe bandwidth. At the same time, the buffer shouldn't be too large to initiate the kernel execution.
- Data availability for the kernel compute: The host should send the data to the FPGA memory as soon as possible so that the compute can be initiated and the kernel is not starved due to data availability.
- Overlapping data transfers with kernel computation: Applications, such as database analytics or video, have a much larger data set than can be stored in the available global device memory on the acceleration device. They require the data to be processed in blocks. Techniques that overlap the data transfers with the computation are critical to achieving high performance for these applications.

You may need to try out different buffer sizes for data movement between host and device memory to optimize the application performance. Using the Vitis tools, you can explore applications using an emulation target for estimated performance results and finally run on the hardware for the accurate performance results. After running the application, you can use Vitis analyzer to visualize the data movement between host and FPGA memory, in addition to kernel and device memory.

The following snapshot is based on the Timeline Trace (host application timeline) as seen in the Analysis view of the Vitis analyzer. This view displays the data movement on the horizontal axis as "Time" to identify any potential performance improvement opportunities.

Figure 17: Timeline Trace (host application timeline)

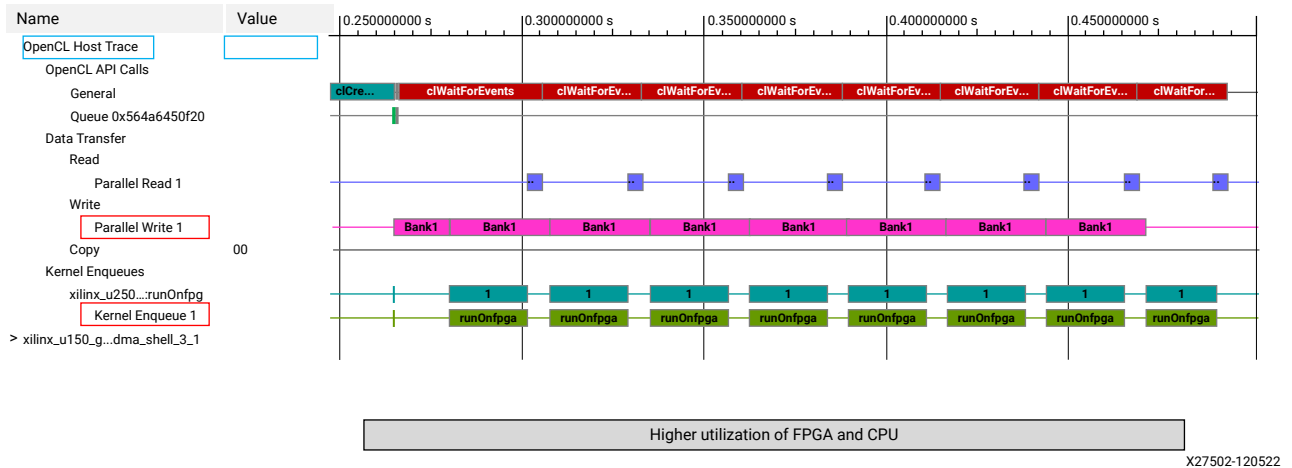


The row with "Data Transfer:Read" displays when the host is reading the data from the device memory and "Data Transfer:Write" displays when the host is writing the data to the device memory. The row with "Kernel Enqueue" displays when the kernel is executing.

Like Task level parallelism achieved within the kernel, similar parallelism can also be achieved between the host and CPU. By enabling this, both CPU and FPGA can be active at the same time and result in high performance. As you program the host code, you need to think of ways to keep the FPGA and CPU both busy at the same time. This is usually the property of the algorithm and the designer has to carefully write the host program to get the max utilization of FPGA and CPU.

Looking at the above Application Timeline, it's easy to identify the gaps when the CPU is idle. Similarly, the FPGA is idle when the host is transferring the data. So the host program needs to feed the data to the device memory as soon as possible so that kernel on FPGA is not starved for data. Here, a large buffer is sent one time from the host to the CPU for computing. The FPGA is idle until the data is completely transferred to the device memory. When the FPGA is executing the compute function, the CPU is idle at that time. For this application, it's not required to send the large buffer and the application can produce better results by overlapping host transfer and kernel compute.

Figure 18: Improved Timeline



With the change on the host side by splitting the data into multiple chunks, the kernel compute can be initiated earlier and data transfer between the host and FPGA can be hidden. The host data transfer and the accelerated function on FPGA can now run in parallel and improve overall application performance. This technique has been demonstrated and explained in detail in [Bloom Filter Tutorial](#).

Vitis analyzer is not limited to showing the timeline trace but also provides application guidance on profiling, presents the synthesis reports with timing and resource information, in addition to the critical path in your design. You can refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for more information.

For more information on host programming refer to [XRT Native API](#), and the [Example - Overlap of Host program and Kernel execution](#).

Next Steps

The following tutorials walk through the methodologies on architecting the application, developing the accelerator to meet performance goals, and writing the host code. The tutorials also demonstrate the debugging and visualization capabilities of Vitis.

[Bloom Filter Tutorial](#) - Demonstrates how to make key decisions about the architecture of the application, develop the accelerator to meet your desired performance goals, and optimize the host code.

[Convolution Example](#) - Walks you through the process of analyzing and optimizing a 2D convolution used for real-time processing of a video stream.

Introduction to Data Center Acceleration for RTL Designers

This chapter is intended for RTL designers who want to accelerate data center applications using AMD FPGA-based Alveo accelerator cards. The AMD Vitis™ application acceleration development flow provides a model to combine RTL designs and a host application into a unified system running on AMD Alveo™ accelerator cards. The goal of this guide is to introduce key concepts for understanding and using the Vitis tools for RTL designers.

The following are key concepts for using RTL designs to create accelerated applications on FPGAs:

- The accelerated data center application is split into host code that runs on the CPU and the RTL design, or RTL kernels that run on programmable logic (PL) region of an Alveo accelerator card.
- Vitis enables existing RTL designs to be used, with limited changes to satisfy interface requirements, by packaging the IP as an RTL kernel using the AMD Vivado™ IP packager.
- The host application running on the x86 CPU uses Xilinx Runtime (XRT) APIs to interact with the device and the accelerators. The XRT APIs let the application read or write any address-mapped register in the accelerators and transfer data buffers to and from global memory in the Alveo card.
- Data transfers between the host and global memory of the accelerator card introduce latency which can be costly to the overall application. To achieve acceleration in a real system, the performance of the RTL kernels must outweigh the added latency of data transfers.
- RTL kernels from Vivado IP have few signal requirements for integration by the Vitis tools; but they should include AXI4-Lite interfaces for accessing address-mapped registers, AXI4 memory-mapped interfaces for connecting to global memory, and clocks and resets for operation.

The next sections of this guide provide an overview discussion of the details of working with Alveo accelerator cards, packaging RTL designs as kernels for use by the Vitis compiler, and using the XRT native API to create host programs for the integrated application. The sections provide references to additional information which you will need to review for a deeper understanding of the Vitis tools and development environment.

Terminology

The following introduces some of the tools and terms you will encounter in this document.

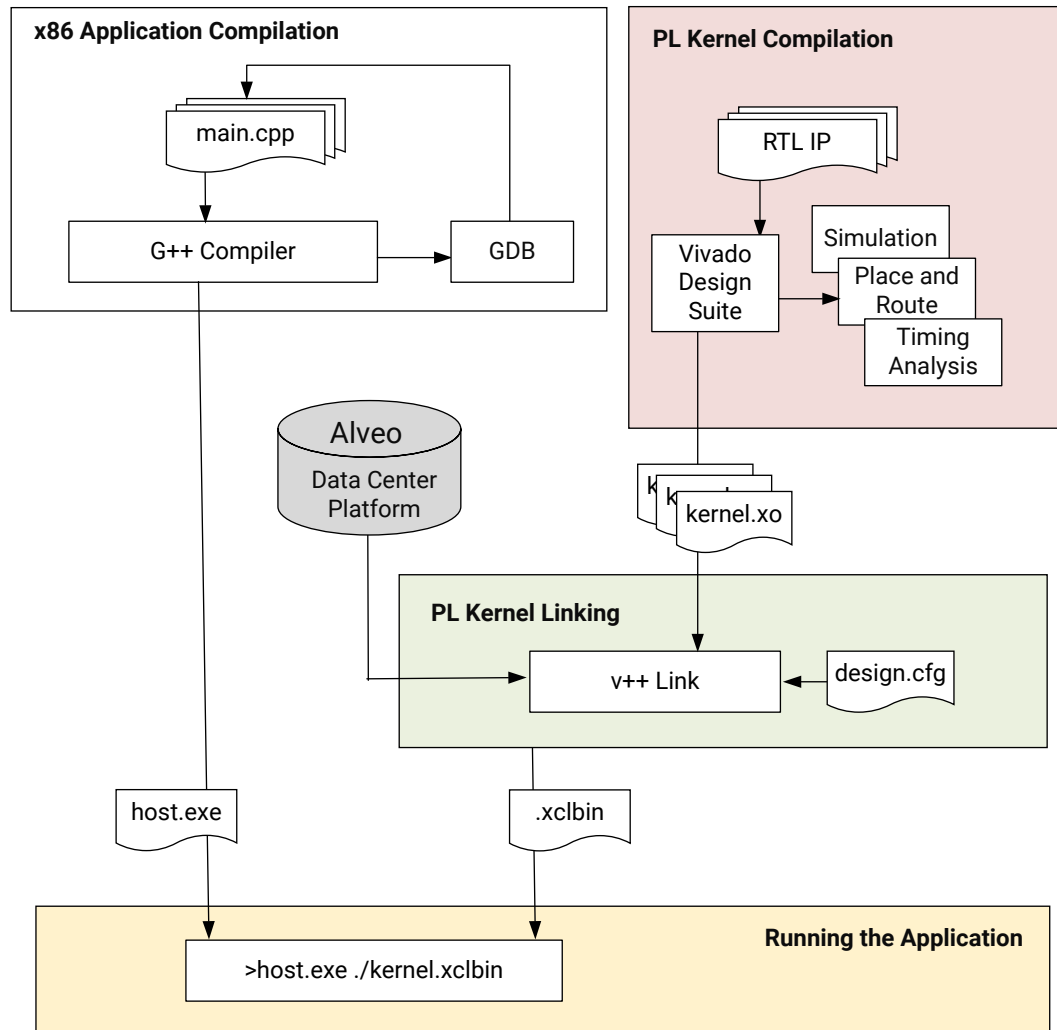
- **Vitis core development kit:** Provides a framework for developing and delivering FPGA accelerated applications using standard programming languages for both software and hardware components. The software component, or host program, is developed using C/C++ to run on a CPU with XRT API calls to manage runtime interactions with the accelerator. The hardware component, or kernel, can be developed using C/C++ or using Verilog or VHDL.
- **Alveo data center accelerator cards:** Are PCI Express® Gen3 x16 compliant cards designed to accelerate compute intensive applications such as machine learning, data analytics, and video processing.
- **Platform:** A predefined configuration of the Alveo accelerator card with features implemented for specific applications. A platform has multiple partitions. The base logic partition (BLP) is a static region that contains fixed logic for essential functions (such as PCIe and DMA). A user logic partition (ULP), which is a dynamic region where the C++ or RTL kernel logic, is programmed for execution.
- **XRT:** The Xilinx Runtime library that provides an API and drivers for your host program to connect with the target FPGA platform and handles transactions between your host program and accelerated kernels.
- **Host and Global Memory:** The distinction between memory on the host machine used by the CPU, and memory on the Alveo data center accelerator card used by the accelerated functions.
- **Vitis HLS:** A high-level synthesis tool that translates C/C++ functions into device logic using programmable logic (PL) elements and RAM/DSP blocks. Vitis HLS synthesizes the C/C++ code into an RTL design and packages it as a compiled object (.xo) file that can be imported into the Vitis environment. There can be multiple functions targeted on the FPGA, each being a separate kernel. Vitis HLS will synthesize these kernels one by one and generate separate .xo files.
- **Register transfer level (RTL):** An abstraction level used for modeling digital circuits. Often the term RTL is used interchangeably for Verilog or VHDL which are both hardware description languages.
- **PL Kernel (.xo) file:** Is the term to designate the custom logic implementation of your accelerator function. Each kernel is packaged as an .xo file and contains the IP for the function and associated metadata used by the Vitis tool.
- **RTL Kernel:** Is an RTL design that uses standard AXI4 interfaces to enable the Vitis compiler to link it into the target platform to quickly build the system design. The RTL kernel is packaged as an .xo file.

- **Vivado Design Suite:** An RTL language synthesis and implementation tool that takes the RTL design generated by Vitis HLS or an RTL designer and generates the bitstream that can be loaded and executed on the FPGA.
- **Bitstream:** Is the configuration data that is used to program the FPGA so that its functionality can be changed. The kernel design will result in a bitstream used to program the dynamic region of the FPGA on the Alveo accelerator card.
- **Device Binary (.xclbin) file:** Contains the bitstream and other metadata needed to be used to program the FPGA. This is used by XRT APIs to actually program the FPGA. In the Vitis flow, the device binary files have the extension `.xclbin`.
- **Vitis analyzer:** A utility that allows you to view and analyze the reports generated while building and running the application through Vitis, Vitis HLS, and AMD Vivado™.

Vitis Development Flow for RTL Designers

This section provides a brief overview of the Vitis application development flow for RTL designers. An image of this flow is shown below.

Figure 19: Vitis Development Flow with RTL Kernels



X24704-042122

The development flow includes the following steps:

- **Application Compilation using G++:**

The host program is written in C/C++ using XRT native API, and compiled using a `g++` compiler to create a host executable file to run on the x86 processor. The host program interacts with RTL kernels in the PL region on the FPGA to complete the accelerated application.

- **PL Kernel Creation using Vivado Design Suite:**

The RTL design is developed and optimized for AMD FPGA devices using the Vivado tools. The steps for kernel development in the Vivado tools include:

1. Edit RTL code to design the function

2. Verify the RTL using behavioral simulation in Vivado simulator
 3. Verify the RTL synthesis, place and route, and timing
 4. Analyze performance by reviewing timing reports
 5. Repeat previous steps until performance goals are met
 6. Package the RTL IP as a kernel (.xo) for use in the Vitis flow
- **PL Kernel Linking using Vitis Tools:**

Xilinx object (.xo) files are linked with the target hardware platform by the Vitis linker to create a device binary file (.xclbin) that is loaded for execution on the Alveo accelerator card.



TIP: This step will call Vivado place and route to generate the bitstream as part of the .xclbin file.

To help define the architecture of the device binary a configuration file (design.cfg) can be created to specify how the kernels are connected to the global memory or to each other, or define how many instances of a kernel (or Compute Unit) should be built in the device binary to allow multiple functions to run in parallel. This configuration file is passed to the Vitis linker to generate the .xclbin.

- **Running the Application:** Finally, when you run the application the host program loads the .xclbin file generated by Vitis Compiler. The host application always runs on the CPU, and the RTL kernel can be run in emulation mode on the x86, or load the .xclbin on the FPGA on the Alveo accelerator card.

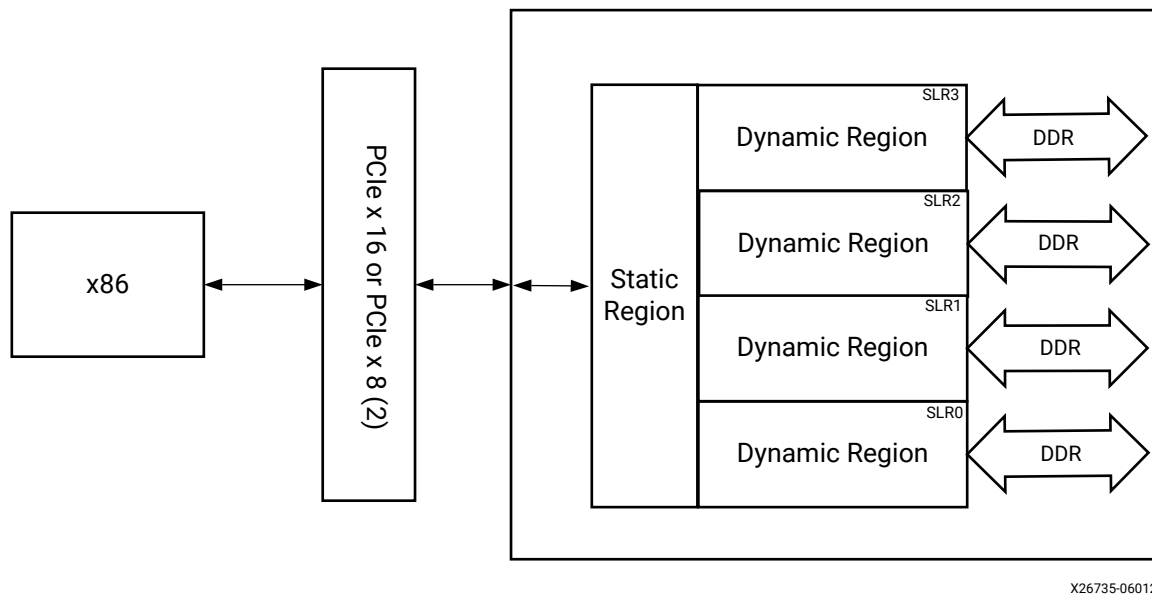
Using Alveo Accelerator Cards

AMD has developed the Alveo family of PCIe Data Center accelerator cards using FPGAs at its core. Each Alveo card combines three essential things: a powerful FPGA for acceleration, high-bandwidth device memory banks, and connectivity to a host server via a high-bandwidth PCIe Gen3x16 link. A number of different cards are available to provide designers with a choice of features and quantity of programmable resources. Below is the block diagram for the Alveo U250.

Although FPGAs are essentially blank devices that get configured at power-up, all Alveo cards are shipped with target platforms that provide the firmware to configure the accelerator card for specific uses. The platform must be installed with Xilinx Runtime (XRT); flashed into the device during installation, or when changing the configuration of the accelerator card.

On the AMD device, the platform consists of two physical FPGA partitions: *Shell* and *User*. The Shell partition is a static region and provides basic infrastructure for the platform like PCIe connectivity, board management, sensors, clocking, and reset. The User partition is a dynamic region that contains user compiled binary called `.xclbin` which is loaded by XRT during execution. RTL kernels are the custom logic created by the developer and programmed into the dynamic region. In this document, kernels refer to the functions that the designer is implementing into the dynamic region of the Alveo accelerator card.

Figure 20: Alveo Block Diagram



The PCIe interface is used for communication between the host and accelerator card, and to transfer data from the host into the Alveo card's device memory. This device memory serves as a global memory, accessible by both host and hardware accelerators. The device memory included on the Alveo platform are PLRAM (small size but fast access with the lowest latency), HBM (moderate size and access speed with some latency), and DDR (large size but slow access with high latency). Depending upon the Alveo card, you may have DDR or HBM, or even both.

The block diagram shown above is of U250 and has 4 banks of DDR, each with 16 GB of memory. The FPGA on the Alveo card is further subdivided into multiple super logic regions (SLRs), which aid in the architecture of very high-performance designs. As you develop RTL kernels for implementation into the dynamic region of the platform you will need to manage the design constraints of SLRs and global memory.

To further improve performance, and minimize access to DDR memory, FPGAs have large quantities of small, internal RAM blocks. These are completely configurable by the compiler to ensure that buffering can be created between tasks to enable pipeline-style computation. This effectively eliminates the need for caches and is one of the key strengths of FPGAs.

There are many more details you could learn about the FPGA architecture and Alveo cards, but this is sufficient for introductory purposes. From the perspective of designing an FPGA-based acceleration architecture, the important points to remember are:

- Moving data across PCIe is expensive - even at Gen3x16, latency is high. For larger data transfers, bandwidth can easily become a system bottleneck.
- Bandwidth and latency between the DDR4 and the FPGA are significantly better than over PCIe, but touching external memory is still expensive in terms of overall system performance.

Differences between Vivado and Vitis Development Flows

As an RTL designer, you might be familiar with or have worked with the Vivado Design Suite. This tool is also at the heart of the Vitis design environment, and your experience working with the Vivado tool will benefit you in this design flow.

RTL Development

The RTL kernel development flow requires you to standardize any RTL IP you have for use in the Vitis design flow. This means creating RTL kernel (.xo) files from existing IP, making any necessary modifications to support the AXI4 interfaces required for the Vitis tools as described in [Requirements of an RTL Kernel](#). The RTL kernel development flow lets you modify existing custom IP that you might have already packaged for use in the Vivado tool, or start your RTL design from scratch and package it for use in the Vitis flow.

Creating an RTL kernel follows the tradition RTL IP development process: you will write the RTL code, simulate it with the Vivado logic simulator, or any supported third-party simulator, and place and route the design to insure it passes timing and routes to completion. You might also define XDC constraints for use with your design as needed to complete implementation. When you are ready, or when your design is complete, you can package the RTL design as an IP and generate the RTL kernel (.xo) files for use by the Vitis compiler in building the system design. In terms of the RTL design and IP packaging process everything is the same, with the additional output of the .xo file.

System Design

In the Vivado development flow you would manually add and stitch the required IP together using the IP integrator of the tool, or define your top-down system using RTL. In the Vivado flow you will need to specify the overall system design, complete with PCIe bus, global memory, and peripheral features, outside of the FPGA design. You would need to create the custom host code incorporating drivers to access features of the system card or the programmable logic.

In the Vitis application acceleration flow the compiler links the RTL kernel, or multiple kernels, with the target platform of the Alveo accelerator card, automatically building the system design using the IP integrator feature. The Vitis compiler automatically instantiates a Memory Subsystem (MSS) IP into the system design to manage AXI traffic between kernels, the host processor, and memory resources. Configuration of the MSS is derived from the connectivity section of a configuration file used during linking as described in [Linking the System](#). XRT provides the underlying runtime and drivers, and provides an API for developing the host application to access the accelerator card.

The Vivado development flow requires synthesis, place and route, and timing closure for your design. The Vitis flow creates the Vivado project during the linking process, and automates the synthesis and implementation of the design. While this is automated within the Vitis tool flow, you can completely control the process, using Tcl scripts or working interactively in the Vivado tool to address design problems and generate the desired results.

While the Vivado and Vitis tools both provide the system design capability, the Vitis tools standardize much of the required ecosystem. The Vitis flow automates several steps like integrating with PCIe, and adding global memories. This lets you focus on developing the RTL function and reduces the overall development time. With the Vitis flow, it is also easier to migrate to another accelerator card seamlessly, and most of the time without any change in the RTL component or the host code.

Creating and Packaging RTL Kernels

Creating an RTL kernel begins with creating an IP within the Vivado Design Suite. You might have an existing RTL IP in your repository, or might want to create a new RTL design to package as an IP. Either approach is a good place to start in creating a new RTL kernel.

Execution Protocol

The RTL kernel employs a user-managed execution scheme where the host application generally uses register reads and writes to manage the execution and completion of the RTL kernel function. This user-managed execution protocol lets you use the control scheme of existing IP in the Vitis environment with little or no redesign, as explained in [Creating User-Managed RTL Kernels](#). Typically this includes the use of an `s_axilite` interface and using XRT native API object classes and methods for reading and writing to register addresses on the kernel.

Port Interface Protocols

RTL kernels, like all kernels in the Vitis development flow, support four types of interfaces:

- AXI4-Lite (`S_AXILITE`) for control registers, buffer pointers, scalar values, and kernel interactions with the host. The data is accessed by register reads and writes

- AXI4 Memory Mapped (`M_AXI`) for access from the kernel to global memory or host memory. Data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM
- AXI4-Stream ports to stream data between kernels, or other streaming sources such as a video processor or camera
- Custom (non-AXI) interfaces are also supported using the `--connectivity.connect` command to make connection to these ports during the `v++` linking process. This process can be used to connect your kernel to GT ports on the platform for instance

In addition, the kernels must have at least one clock, but can support multiple clocks, reset signals, and interrupts, as discussed in [Kernel Interface Requirements](#). If your original IP does not use AXI4 interfaces, or provide the needed clock signal, you will need to modify and repackage the current IP to provide these signals.

A platform can have scalable clocks and fixed clocks. The Vitis flow can generate any number of derived fixed clocks that are not provided by the platform and are commonly used for RTL kernel flow. When fixed clocks are used, Vitis flow will insert an MMCM into the system design to generate the required frequencies. Refer to [Managing Clock Frequencies](#) for more information.

The data bus width of the AXI master and stream ports are configurable. Normally this depends on data transfer bandwidth and FPGA resource considerations.

The design of the RTL kernel is highly flexible. You can decide the interaction method between the kernel and the host application, the internal clock generation scheme, clock gating strategy, handling of interrupts.



TIP: You might need to use the AXI Clock Converter as part of the AXI Interconnect Core IP to manage cross-domain clocking from outside the kernel to inside. It might be hard to achieve timing closure if you treat the external bus clock and internal bus clock as synchronous.

Packaging the IP

To generate the RTL kernel you must run the Package IP process in the Vivado tool, as described in [Packaging the RTL Code as a Vitis XO](#). This is the standard IP packaging flow as described in *Vivado Design Suite User Guide: Creating and Packaging Custom IP* ([UG1118](#)), with the additional step of specifying the kernel for use in the Vitis design flow.

Building Kernels and Managing Implementation

With the RTL kernel generated from a packaged IP into a compiled `.xo` file you are ready to link the kernel with the target platform and with other kernels and build the system design. Designs with user-managed RTL kernels support hardware emulation builds and hardware builds as described at [Working with Build Targets](#), but do not support software emulation builds, as the RTL kernel does not support inclusion of a C-model.

During the build process the Vitis compiler launches the Vivado Design Suite to automatically place and route the design, and generate the bitstream and the `.xclbin` file. While the build process is automated by the Vitis compiler, it also offers the opportunity for specifying constraints on complex designs or interactively working in the Vivado tools to help you resolve timing and generate the `.xclbin` file, as described in [Managing Vivado Synthesis, Implementation, and Timing Closure](#).

The Vivado tools also provide a rich Tcl programming language for scripting and automating elements of the design flow. You can provide Tcl scripts to be run at different stages of the build process for the Vitis compiler or the Vivado tool. These scripts can be enabled for specific steps in the build process using [--linkhook Options](#), or using Vivado properties as explained in [--vivado Options](#).

Writing Host Applications with XRT API

The general structure of a host application in the Vitis development flow can be divided into the following steps, with example code provided:

1. Specify the accelerator device ID and load the `.xclbin`:

```
auto device = xrt::device(device_index);
auto uuid = device.load_xclbin(binaryFile);
```

2. Setup the kernel IP, and define the argument buffers:

```
auto ip1 = xrt::ip(device, uuid, "Vadd_A_B:{Vadd_A_B_1}");
// Allocate Buffer in Global Memory
auto ip1_boA = xrt::bo(device, vector_size_bytes, 1);
auto ip1_boB = xrt::bo(device, vector_size_bytes, 1);
// Map the contents of the buffer object into host memory
auto bo0_map = ip1_boA.map<int*>();
auto bo1_map = ip1_boB.map<int*>();
```

3. Transfer data from the host to the device memory, if the kernel is written to access device memory:



TIP: The kernel can also be written to access host memory directly when the platform supports that ability, as described in [Host Memory Access](#).

```
// Get the buffer physical address
buf_addr[0] = ip1_boA.address();
buf_addr[1] = ip1_boB.address();
// Synchronize buffer content with device side
ip1_boA.sync(XCL_BO_SYNC_BO_TO_DEVICE);
ip1_boB.sync(XCL_BO_SYNC_BO_TO_DEVICE);

// Setting Register A (Input Address)
ip1.write_register(A_OFFSET, buf_addr[0]);
ip1.write_register(A_OFFSET + 4, buf_addr[0] >> 32);

//Setting Register B (Input Address)
ip1.write_register(B_OFFSET, buf_addr[1]);
ip1.write_register(B_OFFSET + 4, buf_addr[1] >> 32);
```

4. Run the kernel and returning results:

```
// Start Kernel by writing to the Control Register
uint32_t axi_ctrl = IP_START;
ip1.write_register(USER_OFFSET, axi_ctrl);

// Wait until the IP is done by reading from the Control Register
while (axi_ctrl != IP_IDLE) {
    axi_ctrl = ip1.read_register(USER_OFFSET);
}

// Sync the output buffer back to the Host memory
ip1_boB.sync(XCL_BO_SYNC_BO_FROM_DEVICE);
```

To support user-managed RTL kernels, you will write your host application using the [XRT Native API](#), defining the RTL kernel as an `xrt::ip` object as shown above, and accessing the kernel through register read and write commands. Refer to [Writing the Software Application](#) for more information.

With the host application written, you can compile it using the standard g++ compiler as described in [Compiling and Linking for x86](#).

Next Steps

The Vitis development flow provides a rich environment for RTL designers. The environment provides the tools necessary to develop use existing IP or develop new IP for use in Data Center applications. The Alveo accelerator cards provide advanced systems for application acceleration, and the Vitis compiler provides the tools and drivers to let you quickly connect your RTL designs with the target platform and host applications, greatly increasing your productivity for RTL design based application acceleration.

For an example on creating and building a user-managed RTL kernel and host application refer to [RTL Kernel Workflow tutorial](#) for step by step instructions on the process. The [Vitis Accel Examples](#) repository provides additional examples of RTL kernels and host application.

Tutorials and Examples

To help you quickly get started with the AMD Vitis™ core development kit, you can find tutorials, example applications, and hardware kernels in the following repositories on <https://github.com/xilinx/Vitis-Tutorials>.

- **Vitis Tutorials:** Provides a number of tutorials that can be worked through to teach specific concepts regarding the tool flow and application development.

The [Getting Started](#) pathway tutorials are an excellent place to start as a new user.

- **Vitis Examples:** Hosts many examples to demonstrate good design practices, coding guidelines, design pattern for common applications, and most importantly, optimization techniques to maximize application performance. The on-boarding examples are divided into several main categories. Each category has various key concepts illustrated by individual examples in both C and C++, when applicable. All examples include a Makefile to enable building for software emulation, hardware emulation, running on hardware, and a `README.md` file with a detailed explanation of the example.

Developing Applications

This section contains the following chapters:

- [Programming Model](#) describes the elements of the Vitis development environment
- [Writing the Software Application](#) specifies how to create software applications for x86 or Arm processors using the XRT API to access PL kernels
- [Developing PL Kernels using C++](#) provides an introduction to creating PL kernels from C++ source code using the HLS compiler
- [Packaging RTL Kernels](#) describes creating PL kernels from Vivado packaged IP
- [Programming Versal AI Engines](#) provides an introduction to systems using AI Engine graph applications

Programming Model

The Vitis development flow supports building heterogeneous computing systems as described in [Introduction to Vitis Tools for Embedded System Designers](#), and [Introduction to Data Center Acceleration for Software Programmers](#). In this environment a software application running on an x86 or Arm® processor uses the Xilinx Runtime (XRT) API to offload compute intensive tasks to execute a hardware kernel running on the programmable logic (PL) of an AMD device or on Versal AI Engines.

The elements of these heterogeneous computing systems include the following:

- Software applications using XRT running on x86 processors or embedded processors on AMD Adaptive SoC devices, as described in [Writing the Software Application](#)
- C/C++ sourced programmable logic functions as described in [Developing PL Kernels using C++](#)
- ADF graph applications running on Versal Adaptive SoC as described in [Programming Versal AI Engines](#)
- [AMD Alveo™ Data Center acceleration cards](#), or custom embedded platforms as described in [Section X: Using Vitis Embedded Platforms](#)
- A system project to link and package the various elements as described in [Section IV: Building and Running the System](#), or [Creating a System Project for Heterogeneous Computing](#)

Design Topology

In the Vitis core development kit, Alveo Data Center acceleration cards and custom embedded platforms provide a foundation for designs. Targeted devices can include AMD Versal™ adaptive SoCs, Zynq UltraScale+ MPSoCs, Kria SOMs, or AMD UltraScale+™ FPGAs. These devices contain a programmable logic (PL) region that loads and executes a device binary (.xclbin) file that contains and connects PL kernels as compiled object (.xo) files and can also contain AI Engine graphs.

Extensible Alveo acceleration cards and custom embedded platforms contain one or more interfaces to global memory (DDR or HBM), and optional streaming interfaces connected to other resources such as AI Engines and external I/O. PL kernels can access data through global memory interfaces (`m_axi`) or streaming interfaces (`axis`). The memory interfaces of PL kernels must be connected to memory interfaces of the extensible platform. The streaming interfaces of PL kernels can be connected to any streaming interfaces of the platform, of other PL kernels, or of the AI Engine array. Both memory-based and streaming connections are defined through Vitis linking options, as described in [Linking the System](#).

Multiple kernels (`.xo`) can be implemented in the PL region of the AMD device binary (`.xclbin`), allowing for significant application acceleration. A single kernel can also be instantiated multiple times. The number of instances, or compute units of a kernel is programmable up to 31, and determined by linking options specified when building the device binary.

For Versal devices the `.xclbin` file can also contain the compiled AI Engine graph application (`libadf.a`). The graph application consists of nodes and edges where nodes represent compute kernel functions, and edges represent data connections. Kernel functions are the fundamental building blocks of an ADF graph application. Kernels operate on data streams, consuming input blocks of data and producing output blocks of data. The `libadf.a` and PL kernels (`.xo`) are linked with the target platform (`.xpfm`) to define the hardware design. The AI Engine can be driven by PL kernels through `axis` interfaces. The AI Engine can also be controlled through the Arm processor (PS) via runtime parameters (RTP) in the graph and GMIO on Versal adaptive SoC devices. Refer to *AI Engine Tools and Flows User Guide* ([UG1076](#)) for more information.

PL Kernel Properties

In the Vitis development flow, PL kernels as compiled object files (`.xo`) are the processing elements executing in the programmable logic region of the AMD device. The Vitis core development kit supports PL kernels written in C/C++ compiled by the `v++` HLS compiler, and RTL IP packaged in the Vivado Design Suite.

Regardless of source language, all PL kernels have the same properties and must adhere to same set of requirements.

- **Control Scheme:** Kernels can be defined as software controllable, or data-driven. This means that the PL kernel is controlled through a software application running XRT or using available drivers, or is not managed by software and is instead driven by the arrival of data.
- **HW Interfaces:** Kernels must use AXI4 interfaces in the design regardless of whether software controlled or data driven.
- **Clocks and Resets:** Kernels must have clocks to sync kernels and platforms into a system, and optional resets for additional control.

SW-Controllable Kernels

Software controllable kernels expose a programmable register interface, allowing a host software application to interact with kernels through register reads and write. These are the most common and widely applicable types of PL kernels. There are two types of SW controllable kernels: user-managed and XRT-managed.

Note: XRT-managed kernels are a specialized form of user-managed kernels.

The primary difference between user-managed and XRT-managed kernels is related to the kernel execution mode. Because XRT relies on the `ap_ctrl_chain` and `ap_ctrl_hs` execution protocols generated by the HLS compiler, XRT-managed kernels are better for C++ developers as described in [Developing PL Kernels using C++](#). Alternatively, user-managed kernels can support many different user-defined execution protocols as found in existing Vivado RTL IP, and so are a better fit for RTL designers described in [Packaging RTL Kernels](#).

The Vitis development flow supports software applications written for use with PL kernels using the XRT native C/C++ API, which supports both user-managed kernels and XRT-managed kernels, in addition to some advanced designs as discussed in [Execution Modes](#). The next sections briefly describe the programming API and the different hardware interfaces required for XRT-managed or user-managed kernels.



TIP: The general documentation for XRT can be found at [Xilinx Runtime Architecture](#). The XRT API described below can be found at [XRT Native C++ API](#).

Table 6: Software Control Using the XRT API

XRT-Managed Kernels	User-Managed Kernels
- The object class for an XRT-managed kernel is <code>xrt::kernel</code> - The software application communicates with the XRT-managed kernel using higher-level commands such as <code>set_arg</code>, <code>run</code>, and <code>wait</code> - The user does not need to know the low-level details of the programmable registers and kernel execution protocols - Control and status registers provide XRT with a known interface to interact with the kernel, which makes these high-level commands possible - If needed, it is also possible to control an XRT-managed kernel as a user-managed kernel (using atomic register reads and write) - OpenCL API can also be used with XRT-managed kernels (<code>cl::kernel</code>)	- The object class for a user-managed kernel is <code>xrt::ip</code> - The software application communicates with the user-managed kernel using atomic register reads and writes through the AXI4-Lite interface - The application developer is responsible for knowing the address offset and purpose of each register in the kernel, and using them properly - There are no checks, high-level controls, or profiling capabilities. The user is responsible for running the simulations for performance analysis/debugging.

Design Languages

SW controllable kernels can be developed using either RTL or C/C++:

- **RTL:** User-managed kernels are the most natural and recommended type of kernel for RTL developers. They offer greater flexibility, offer a wider range of control possibilities, and have fewer requirements than XRT-managed kernels. For more information, see [Packaging RTL Kernels](#).
- **C++:** XRT-managed kernels are the default and recommended type of kernel for C/C++ developers as described in [Developing PL Kernels using C++](#). The Vitis compiler (`v++`) automatically generates interfaces compatible with the high-level XRT API, leaving fewer details for the developer to worry about.

HW Interfaces

The kernel interfaces are used to exchange data with the host application, other kernels, or device I/Os. Both user-managed and XRT-managed have exactly the same interface requirements as listed here:

- **Programmable interface:** AXI4-Lite slave interface. Kernels can only have a single AXI4-Lite interface.
- **Data interfaces:** Any number and combination of AXI4 memory mapped and AXI4-Stream interfaces.
- **Clock and resets:** As described in [Clock and Reset Requirements](#).



TIP: XRT-managed kernels have specific requirements for control registers in the AXI4-Lite interface (including start and stop bits) as described in [Control Requirements for XRT-Managed Kernels](#). User-managed kernels can implement whatever control scheme the user specifies.

The following table elaborates the type of interface required based on the characteristics of the data movement in your application.

Table 7: Kernel Interface Types

Register (AXI4-Lite)	Memory Mapped (M_AXI)	Streaming (AXI4-Stream)
• Register interfaces must be implemented using a single AXI4-Lite interface. • Designed for transferring scalars between the host application and the kernel. • Register reads and writes are initiated by the host application. • The kernel acts as a slave.	• Memory mapped interfaces must be implemented using one or more AXI4 Masters interfaces. • Designed for bi-directional data transfers with global memory (DDR, PLRAM, HBM). • Introduces additional latency for memory transfers. • The kernel acts as a master accessing data stored into global memory. • The host application allocates the buffer for the size of the dataset. • The base address of the buffer is provided by the host application to the kernel via the AXI4-Lite interface.	• Streaming interfaces must be implemented using one or more AXI4-Stream interfaces. • Designed for uni-directional data transfers between kernels. • The access pattern is sequential. • Does not use global memory. • Data set is unbounded. • A sideband signal can be used to indicate the last value in the stream.

Execution Modes

User-managed PL kernels have no predefined execution mode. It is up to the kernel designer to implement the control protocol and the execution mechanism. It is the application developer's responsibility to manage the operation of the kernel by executing appropriate sequences of register reads and writes from the software application, in accordance with the user-defined control protocol of the kernel.

XRT-managed PL kernels, as described in [Supported Kernel Execution Models](#) in the XRT documentation, provide defined kernel execution modes supporting overlapping execution of the kernel, or sequential execution.

- A kernel is started by the software application using an XRT API call. When the kernel is ready for new data it notifies the host application through bits in the control register.
- The default control protocol, `ap_ctrl_chain`, supports pipelined execution enabling multiple executions of the same PL kernel to be overlapped to improve the overall application throughput.
- If required, pipelined execution can be disabled by using the `ap_ctrl_hs` control protocol which forces kernels to run sequentially, waiting until the prior run has completed before starting the next run.
- Finally, a kernel can be auto-restarting, allowing it to run for a specified number of iterations, or until reset by the host application as described in Auto-Restarting Kernels in *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

Data Driven Kernels

These kernels are present in the device but are not directly managed by the software application, instead triggered by the arrival of data at the interface. The kernels must have at least one AXI4-Stream interface. The kernel synchronizes with the rest of the system through these streaming interfaces. These kernels can have scalar inputs and outputs connected through the AXI4-Lite interface. You can read/write to the kernels by native XRT APIs (`xrt::ip::read_register`, `xrt::ip::write_register`).

Data driven kernels do not require a software API, as the host application is not required to interact directly with the kernel. The kernel can be developed as either an RTL IP or synthesized from C/C++ source code.

HW Interfaces

The kernel interfaces are used to exchange the data with the host application, other kernels, or device I/Os. Data driven kernels have the interface requirements listed here:

- **Programmable interface:** The AXI4-Lite interface is optional and used to pass scalar values to the kernel.

- **Data interfaces:** The kernel requires at least one AXI4-Stream interfaces.
- **Clock and resets:** As described in [Clock and Reset Requirements](#).

Clock and Reset Requirements

These clock and reset requirements apply to both software controllable and non-software controllable kernels.

Table 8: Requirements

C/C++/OpenCL C Kernel	RTL Kernel
• C kernel does not require any input from user on clock ports and reset ports. The HLS tool always generates RTL with clock port <code>ap_clk</code> and reset port <code>ap_rst_n</code>. • HLS kernels can only have one clock/reset.	• RTL kernels require at least one clock port, but a kernel can have multiple clocks. The number of clocks that an RTL can have is primarily determined by the number of clocks that the platform supports. Most data center platforms only support two clocks, but most embedded platforms can have multiple clocks. • An active-Low reset port can optionally be associated with a clock through the <code>ASSOCIATED_RESET</code> parameter on the clock.

Writing the Software Application

In the AMD Vitis™ environment the software application component can be written in native C++ using the Xilinx Runtime (XRT) native API. The XRT native API is described here in brief, with additional details available under [XRT Native API](#) on the XRT documentation site. The Application component is generally referred to as the host application for Data Center acceleration as it runs on an x86 server, and simply as the application for embedded processor-based systems in the same device as the PL kernel and the AI Engine graph.



TIP: For examples of software application programming using the XRT native API refer to [host_xrt](#) in the [Vitis_Accel_Examples](#).

In general, the structure of the host application can be divided into the following steps:

1. Specifying the platform device ID and loading the `.xclbin`
2. Setting up the PL kernel and kernel arguments
3. Transferring data between the software application and PL kernels
4. Loading and launching the AI Engine graph application
5. Running the system and returning results

For AMD Versal™ adaptive SoC devices, the PS application manages the whole heterogeneous system, including the PL hardware and the AI Engine graph application. Refer to Programming the PS Host Application in the *AI Engine Tools and Flows User Guide* ([UG1076](#)) for more information.

To use the native XRT APIs, the host application must link with the `xrt_coreutil` library. For example:

```
g++ -g -std=c++17 -I$XILINX_XRT/include -L$XILINX_XRT/lib -lxrt_coreutil -pthread
```

Compiling host code with XRT native C++ API requires C++ standard with `-std=c++17`. On GCC version older than 4.9.0, use `-std=c++1y` instead because `-std=c++17` is introduced to GCC from 4.9.0.



IMPORTANT! For multithreading the host application, exercise caution when calling a `fork()` system call. The `fork()` does not duplicate all the runtime threads. Hence, the child process cannot run as a complete application in the Vitis core development kit. It is advisable to use the `posix_spawn()` system call to launch another process from the Vitis software platform application.

Specifying the Device ID and Loading the XCLBIN

To use the Xilinx Runtime (XRT) environment properly, the software application needs to identify the accelerator card, or target platform, and the device ID that the kernel will run on. Then it needs to load the device binary (.xclbin) into the device to program the PL kernels.

The XRT API includes a Device class (`xrt::device`) that can be used to specify the device ID on the target platform, and an XCLBIN class (`xrt::xclbin`) that defines the hardware and PL kernels for the runtime. You must use the following `#include` statements in your source code to load these classes:

```
#include <xrt/xrt_device.h>
#include <experimental/xrt_xclbin.h>
```

The following code snippet creates a device object by specifying the device ID from the target platform, and then loads the .xclbin into the device, returning the UUID for the program.

```
//Setup the Environment
unsigned int device_index = 0;
std::string binaryFile = parser.value("kernel.xclbin");
std::cout << "Open the device" << device_index << std::endl;
auto device = xrt::device(device_index);
std::cout << "Load the xclbin " << binaryFile << std::endl;
auto uuid = device.load_xclbin(binaryFile);
```



TIP: The device ID can be obtained using the `xbutil` command for a specific accelerator card or hardware platform.

Setting Up XRT-Managed Kernels and Kernel Arguments

After identifying devices and loading the program, the software application should identify the kernels that execute on the device, and set up the kernel arguments. All kernels the software application interacts with are defined within the loaded .xclbin file, and so should be identified from there.

For XRT-managed kernels, the XRT API provides a Kernel class (`xrt::kernel`), that is used to access the kernels contained within the .xclbin file. The kernel object identifies an XRT-managed kernel in the .xclbin loaded into the AMD device that can be run by the host application.



TIP: As discussed in [Working with User-Managed Kernels](#), you should use the IP class (`xrt::ip`) to identify the user-managed kernels in the `.xclbin` file.

The use of the kernel and buffer objects require the addition of the following `include` statements in your source code:

```
#include <xrt/xrt_kernel.h>
#include <xrt/xrt_bo.h>
```

The following code example identifies a kernel (`"vadd"`) defined in the program (`uuid`) loaded onto the device:

```
auto krnl = xrt::kernel(device, uuid, "vadd");
```



TIP: You can use the `xclbinutil` command to examine the contents of an existing `.xclbin` file and determine the kernels contained within.

After identifying the kernel, or kernels to be run, you need to define buffer objects to associate with the kernel arguments and enable data transfer from the host application to the kernel instance or compute unit (CU):

```
std::cout << "Allocate Buffer in Global Memory\n";
auto bo0 = xrt::bo(device, vector_size_bytes, krnl.group_id(0));
auto bo1 = xrt::bo(device, vector_size_bytes, krnl.group_id(1));
auto bo_out = xrt::bo(device, vector_size_bytes, krnl.group_id(2));
```

The kernel object (`xrt::kernel`) includes a method to return the memory associated with each kernel argument, `kernel.group_id()`.

Note: A buffer is not required for scalar arguments.

Creating Multiple Compute Units

When building the `.xclbin` file you can specify the number of kernel instances, or compute units (CU) to implement into the hardware by using the `--connectivity.nk` option as described in [Creating Multiple Instances of a Kernel](#). After the `.xclbin` has been built, you can access specific CUs from the software application.

A single kernel object (`xrt::kernel`) can be used to execute multiple CUs as long as the CUs have identical interface connectivity, meaning the CUs have the same memory connections (`krnl.group_id`). If all CUs do not have the same kernel connectivity, then you can create a separate kernel object for each unique configuration of the kernel, as shown in the example below.

```
krnl1 = xrt::kernel(device, xclbin_uuid, "vadd:{vadd_1,vadd_2}");
krnl2 = xrt::kernel(device, xclbin_uuid, "vadd:{vadd_3}");
```

In the example above, `krnl1` can be used to launch the CUs `vadd_1` and `vadd_2` which have matching connectivity, and `krnl2` can be used to launch `vadd_3`, which has different connectivity.



IMPORTANT! If you create a single kernel object for multiple CUs without matching connectivity, then XRT assigns one or more CUs with matching connectivity to the kernel object, and ignores the other CUs in the hardware when executing the kernel.

Transferring Data between Software and PL Kernels

Transferring data to and from the memory in the accelerator card or device uses the buffer objects (`xrt::bo`) created when [Setting Up XRT-Managed Kernels and Kernel Arguments](#).

The class constructor typically allocates a regular 4K aligned buffer object. The following code creates regular buffer objects that have a software application backing pointer allocated by user space in heap memory, and a device-side buffer allocated in the memory bank associated with the kernel argument (`krnl.group_id`). Optional flags in the `xrt::bo` constructor let you create non-standard types of buffers for use in special circumstances as described in [Creating Special Buffers](#).

```
std::cout << "Allocate Buffer in Global Memory\n";
auto bo0 = xrt::bo(device, vector_size_bytes, krnl.group_id(0));
auto bo1 = xrt::bo(device, vector_size_bytes, krnl.group_id(1));
auto bo_out = xrt::bo(device, vector_size_bytes, krnl.group_id(2));
```



IMPORTANT! A single buffer cannot be bigger than 4 GB, yet to maximize throughput from the host to global memory, AMD also recommends keeping the buffer size at least 2 MB if possible.

With the buffer established, and filled with data, there are a number of methods to enable transfers between the software application and the kernel, as described below:

- **Using `xrt::bo::sync()`:** Use `xrt::bo::sync` to sync data from the host to the device with `XCL_BO_SYNC_TO_DEVICE` flag, or from the device to the host with `XCL_BO_SYNC_FROM_DEVICE` flag using `xrt::bo::write`, or `xrt::bo::read` to write the buffer from the host application, or read the buffer from the device.

```
bo0.write(buff_data);
bo0.sync(XCL_BO_SYNC_BO_TO_DEVICE);
bo1.write(buff_data);
bo1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
...
bo_out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);
bo_out.read(buff_data);
```

Note: If the buffer is created using a user-pointer as described in [Creating Buffers from User Pointers](#), the `xrt::bo::sync` call is sufficient, and the `xrt::bo::write` or `xrt::bo::read` commands are not required.

- **Using `xrt::bo::map()`:** This method maps the host-side buffer backing pointer to a user pointer.

```
// Map the contents of the buffer object into host memory
auto bo0_map = bo0.map<int*>();
auto bo1_map = bo1.map<int*>();
auto bo_out_map = bo_out.map<int*>();
```

The software code can subsequently exercise the user pointer for data reads and writes. However, after writing to the mapped pointer (or before reading from the mapped pointer) the `xrt::bo::sync()` command should be used with the required direction flag for the DMA operation.

```
for (int i = 0; i < DATA_SIZE; ++i) {
    bo0_map[i] = i;
    bo1_map[i] = i;
}

// Synchronize buffer content with device side
bo0.sync(XCL_BO_SYNC_BO_TO_DEVICE);
bo1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
```

There are additional buffer types and transfer scenarios supported by the XRT native API, as described in [Miscellaneous Other Buffers](#).

Working with XRT-Managed Kernels

The execution of a PL kernel is associated with a class called `xrt::run` that implements methods to start and wait for kernel execution. Most interaction with kernel objects are accomplished through `xrt::run` objects, created from a kernel to represent an execution of the kernel.

The run object can be explicitly constructed from a kernel object, or implicitly constructed by starting a kernel execution as shown below.

```
std::cout << "Execution of the kernel\n";
auto run = krnl(bo0, bo1, bo_out, DATA_SIZE);
run.wait();
```

The above code example demonstrates launching the kernel execution using the `xrt::kernel()` operator with the list of arguments for the kernel that returns an `xrt::run` object. This is an asynchronous operator that returns after starting the run. The `xrt::run::wait()` member function is used to block the current thread until the run is complete.



TIP: Upon finishing the kernel execution, the `xrt::run` object can be used to relaunch the same kernel function if desired.

An alternative approach to run the kernel is shown in the code below:

```
auto run = xrt::run(krnl);
run.set_arg(0,bo0); // Arguments are specified starting from 0
run.set_arg(0,bo1);
run.set_arg(0,bo_out);
run.start();
run.wait();
```

In this example, the run object is explicitly constructed from the kernel object, the kernel arguments are specified with `run.set_args()`, and the run execution is launched by the `run.start()` command. Finally, the current thread is blocked as it waits for the kernel to finish.

After the kernel has completed its execution, you can sync the kernel results back to the host application using code similar to the following example:

```
// Get the output;
bo_out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

// Validate our results
if (std::memcmp(bo_out_map, bufReference, DATA_SIZE))
    throw std::runtime_error("Value read back does not match reference");
```

Working with User-Managed Kernels



TIP: For an example of a software application working with user-managed RTL kernel, refer to [Vitis Tutorials: Hardware Acceleration](#) or the [rtl_user_managed](#) example in the [Vitis_Accel_Examples](#) on GitHub.

User-managed kernels require the use of the XRT native API for the software application, and are specified as an IP object of the `xrt::ip` class. The following is a high-level overview of how to structure your host application to access user-managed kernels from an `.xclbin` file.



IMPORTANT! XRT has a 16-bit (64K) address width limitation for the `s_axilite` interface.

1. Add the following header files to include the XRT native API:

```
#include "experimental/xrt_ip.h"
#include "xrt/xrt_bo.h"
```

- `experimental/xrt_ip.h`: Defines the IP as an object of `xrt::ip`.
- `xrt/xrt_bo.h`: Lets you create buffer objects in the XRT native API.

2. Set up the application environment as described in [Specifying the Device ID and Loading the XCLBIN](#).

3. The IP object (`xrt::ip`) is constructed from the `xrt::device` object, the `uuid` of the loaded `.xclbin`, and the name of the user-managed kernel:

```
//User Managed Kernel = IP
auto ip = xrt::ip(device, uuid, "Vadd_A_B");
```

4. Optionally, for Versal AI Core devices with AI Engine graph applications, you can also specify the graph application to load at run time. This process requires a few sub-tasks as shown:
- Add required header to `#include` statement:

```
#include <experimental/xrt_aie.h>
```

- Identify the AI Engine graph from the `xrt::device` object, the `uuid` of the loaded `.xclbin`, and the name of the graph application:

```
auto my_graph = xrt::graph(device, uuid, "mygraph_top");
```

- Reset and run the graph application from the software program as needed:

```
my_graph.reset();
std::cout << STR_PASSED << "my_graph.reset()" << std::endl;
my_graph.run(0);
std::cout << STR_PASSED << "my_graph.run()" << std::endl;
```



TIP: For more information on building and running AI Engine applications, refer to *AI Engine Tools and Flows User Guide* ([UG1076](#)).

5. Create buffers for the IP arguments:

```
auto <buf_name> = xrt::bo(<device>, <DATA_SIZE>, <flag>, <bank_id>);
```

Where the buffer object constructor uses the following fields:

- `<device>`: `xrt::device` object of the accelerator card.
- `<DATA_SIZE>`: Size of the buffer as defined by the width and quantity of data.
- `<flag>`: Flag for creating the buffer objects.
- `<bank_id>`: Defines the memory bank on the device where the buffer should be allocated for IP access. The memory bank specified must match with the corresponding IP port's connection inside the `.xclbin` file. Otherwise you will get `bad_alloc` when running the application. You can specify the assignment of the kernel argument using the `--connectivity.sp` command as explained in [Mapping Kernel Ports to Memory](#).

For example:

```
auto buf_in_a = xrt::bo(device, DATA_SIZE, xrt::bo::flags::normal, 0);
auto buf_in_b = xrt::bo(device, DATA_SIZE, xrt::bo::flags::normal, 0);
```



TIP: Verify the IP connectivity to determine the specific memory bank, or you can get this information from the Vitis generated `.xclbin.info` file.

For example, the following information for a user-managed kernel from the `.xclbin` could guide the construction of buffer objects in your host code:

```
Instance:      Vadd_A_B_1
Base Address: 0x1c00000

Argument:      scalar00
Register Offset: 0x10
Port:          s_axi_control
Memory:        <not applicable>

Argument:      A
Register Offset: 0x18
Port:          m00_axi
Memory:        bank0 (MEM_DDR4)

Argument:      B
Register Offset: 0x24
Port:          m01_axi
Memory:        bank0 (MEM_DDR4)
```

6. Get the buffer addresses and transfer data between host and device:

```
auto a_data = buf_in_a.map<int*>();
auto b_data = buf_in_b.map<int*>();

// Get the buffer physical address
long long a_addr=buf_in_a.address();
long long b_addr=buf_in_b.address();

// Sync Buffers
buf_in_a.sync(XCL_BO_SYNC_BO_TO_DEVICE);
buf_in_b.sync(XCL_BO_SYNC_BO_TO_DEVICE);
```

`xrt::bo::map()` allows mapping the host-side buffer backing pointer to a user pointer. However, before reading from the mapped pointer or after writing to the mapped pointer, you should use `xrt::bo::sync()` with direction flag for the DMA operation.

7. After preparing the buffer (buffer create, sync operation as shown above), you are free to pass all the necessary information to the IP with the direct register write operation.



IMPORTANT! The `xrt::ip` differs from the standard `xrt::kernel`, and indicates that XRT does not manage the IP but does provide access to read or write the registers.

For example, the code below shows the information passing the buffer base address through the `xrt::ip::write_register()` command.

```
ip.write_register(REG_OFFSET_A, a_addr);
ip.write_register(REG_OFFSET_A+4, a_addr>>32);

ip.write_register(REG_OFFSET_B, b_addr);
ip.write_register(REG_OFFSET_B+4, b_addr>>32);
```

8. Start the IP execution. Because the IP is user-managed, you can employ any number of register write/read to control the start/check status/restart the IP to trigger the execution of the IP. The following example uses an `s_axilite` interface to access control signals in the control register:

```
uint32_t axi_ctrl = 0;
std::cout << "INFO:IP Start" << std::endl;
axi_ctrl = IP_START;
ip.write_register(CSR_OFFSET, axi_ctrl);

// Wait until the IP is DONE
axi_ctrl = 0;
while((axi_ctrl & IP_IDLE) != IP_IDLE) {
    axi_ctrl = ip.read_register(CSR_OFFSET);
}
```

9. After IP execution is finished, you can transfer the data back to host by the `xrt::bo::sync` command with the appropriate flag to dictate the buffer transfer direction.

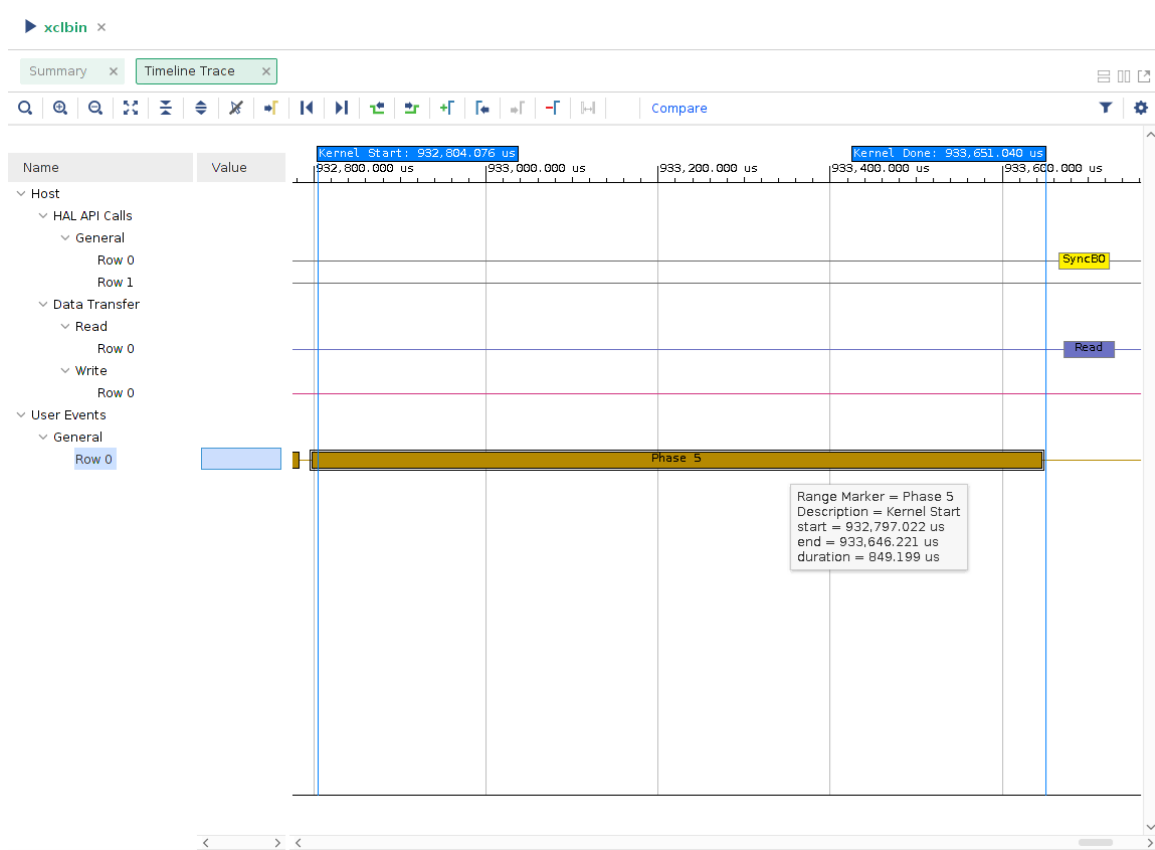
```
buf_in_b.sync(XCL_BO_SYNC_BO_FROM_DEVICE);
```

10. Optionally profile the application.

Because XRT is not in charge of starting or stopping the kernel, you cannot directly profile the operation of `user_managed` kernels as you would XRT managed kernels. However, you can use the `user_range` and `user_event` objects as discussed in [Custom Profiling of the Host Application](#) to profile elements of the host application. For example the following code captures the time it takes to write the registers from the host application:

```
// Write Registers
range.start("Phase 4a", "Write A Register");
ip.write_register(REG_OFFSET_A, a_addr);
ip.write_register(REG_OFFSET_A+4, a_addr>>32);
range.end();
range.start("Phase 4b", "Write B Register");
ip.write_register(REG_OFFSET_B, b_addr);
ip.write_register(REG_OFFSET_B+4, b_addr>>32);
range.end();
```

You can observe some aspects of the application and kernel operation in the Vitis analyzer as shown in the following figure.



Working with AI Engine Graphs



TIP: For an example of an embedded system design that includes an AI Engine graph application refer to [Vitis-Tutorials/Developer_Contributed/01-Versal_Custom_Thin_Platform_Extensible_System/](https://vitis.tutorials.developer.com/01-Versal_Custom_Thin_Platform_Extensible_System/).

For Versal devices, the AI Engine graph application is an adaptive data flow (ADF) graph application. For more information on programming the AI Engine kernels and graph refer to *AI Engine Kernel and Graph Programming Guide* (UG1079).

The following is an example of how you would load and run the graph application from the software program.

1. Add required header to `#include` statement:

```
#include <experimental/xrt_aie.h>
```

2. Set up the application environment as described in [Specifying the Device ID and Loading the XCLBIN](#).

3. Identify the AI Engine graph from the `xrt::device` object, the `uuid` of the loaded `.xclbin`, and the name of the graph application:

```
auto my_graph = xrt::graph(device, uuid, "mygraph_top");
```

4. Reset and run the graph application from the software program as needed:

```
my_graph.reset();  
std::cout << STR_PASSED << "my_graph.reset()" << std::endl;  
my_graph.run(0);  
std::cout << STR_PASSED << "my_graph.run()" << std::endl;
```

The graph application (`libadf.a`) is packaged as part of the device binary (`.xclbin`) by the `v++ --package` command, and also copied to the SD card. For more information refer to [Compiling AI Engine Graph Applications](#).

Summary

As discussed in earlier topics, the recommended coding style for the host program in the Vitis core development kit includes the following points:

1. In the Vitis core development kit, one or more kernels are separately compiled/linked to build the `.xclbin` file. The `device.load_xclbin(binaryFile)` command is used to load the kernel binary.
2. Create `xrt::kernel` objects from the loaded device binary, and associate buffer objects (`xrt::bo`) with the memory banks assigned to kernel arguments.
3. Transfer data back and forth from the host application to the kernel using `xrt::bo::sync` commands and buffer reads and write commands.
4. Execute the kernel using an `xrt::run` object to start the kernel and wait for kernel execution.
5. Additionally, you can add error checking after XRT API calls for debugging purpose, if required.

Developing PL Kernels using C++

The PL kernel code is generally a compute-intensive part of the system intended to run on the programmable logic (PL) region of an AMD device. The Vitis development environment supports PL kernels written in C or C++, and also written in RTL.

For C/C++ based kernels the HLS compiler (`v++`) is available as a standalone command, or as part of the Vitis unified IDE. The IDE provides a language sensitive editor, simulation and code analysis tools, high-level synthesis of RTL from the C or C++ code, and C-RTL co-simulation for an in-depth examination of the resulting hardware design. The HLS compiler is the recommended tool for developing PL kernels for use in the AMD Vivado™ traditional design flow, or in the Vitis heterogeneous design flow.

Generally, off-the-shelf software cannot be efficiently converted into hardware running on programmable logic. Even if the software program can be automatically converted (or synthesized) into hardware, achieving acceptable quality of results (QoR) will require additional work such as structuring the algorithm to help the HLS compiler achieve the desired performance goals. To help, you need to understand the best practices for writing good software for execution on programmable logic as discussed in [Design Principles](#) in the *Vitis HLS User Guide* (UG1399).

The code for C++ kernels written and optimized in an HLS component can be compiled in the Vitis tools either from the `v++` command line as explained in [Compiling C/C++ PL Kernels](#), or from a System project in the Vitis unified IDE as explained in [Section VIII: Using the Vitis Unified IDE](#).



IMPORTANT! The kernel function declaration must be wrapped with the `extern "C"` linkage in the header file, or the whole function in the kernel source code must be wrapped.

```
extern "C" {  
    void kernel_function(int *in, int *out, int size);  
}
```

Packaging RTL Kernels

In the AMD Vitis™ development flow, RTL IP from the Vivado Design Suite can be packaged as compiled AMD object (.xo) files that can be linked into a device binary (.xclbin), as long as they adhere to Vivado IP Packaging guidelines, and requirements of the Vitis compiler for linking the system.

As explained in [PL Kernel Properties](#), RTL kernels can be user-managed kernels that do not meet XRT requirements for execution control, but rather implement any number of possible control schemes specified by existing RTL designs. Alternatively, RTL kernels can adhere to the requirements of the `ap_ctrl_chain` or `ap_ctrl_hs` control protocols needed for XRT-managed kernels.

RTL kernels support the hardware emulation build, and the hardware build described in [Working with Build Targets](#), but an RTL kernel in its native form does not support software emulation. To support software emulation, you must add a C-model to the packaged RTL kernel, as described in [Adding C-Models to RTL Kernels](#).

The following sections describe the kernel interface requirements for the Vitis compiler to link kernels into a system. These requirements are common to software controllable and non-software controlled kernels. The control requirements for XRT-managed kernels are also described, in addition to any additional requirements. Finally, the development flow is described to help you package RTL IP in the Vivado Design Suite as RTL kernels for use in the Vitis environment.

Requirements of an RTL Kernel

To be integrated into the Vitis tool flow, an RTL module must minimally meet the requirements enumerated in [Kernel Interface Requirements](#). The need to meet the kernel interface requirements applies to both XRT-managed and user-managed kernels.

In addition, XRT-managed kernels must satisfy the requirements described in [Control Requirements for XRT-Managed Kernels](#) to be executed and profiled by XRT.

User-managed kernels must have the signal interfaces needed by the Vitis compiler to allow it to link the kernels to other kernels and to the target platform, but do not need to adhere to the strict execution protocol of XRT. In this way, existing RTL IP can be more rapidly and simply integrated into the Vitis environment.

It might be necessary to revise your RTL module to meet the kernel requirements outlined in the following sections.

Kernel Interface Requirements

To enable the Vitis compiler to connect kernels into the target platform, an RTL kernel must adhere to the requirements described in [PL Kernel Properties](#). The various interface requirements are summarized in the following table.



IMPORTANT! In some cases, the port names must be defined exactly as shown.

Table 9: RTL Kernel Interface and Port Requirements

Port or Interface	Description	Comment
Clock	One or more clock inputs.	- At least one clock is required for the kernel.¹ - Can be named anything, but must be packaged with a bus interface. IMPORTANT! All ports in the RTL IP must be associated with an interface when packaging the RTL for use in the Vitis environment. If this is not the case, an error similar to the following occurs: <pre>ERROR: UNDEF When packaging for Vitis, pins that are not part of an interface are not supported</pre>
Reset	Primary active-Low reset input port	- Optional port. - Can be named anything, but must be associated with a Clock signal through the ASSOCIATED_RESET property on the Clock. - This signal should be internally pipelined to improve timing. - The signal is driven by a synchronous reset in the associated Clock domain.
interrupt	Active-High interrupt.	- Optional port. - When used, the name must be exactly as shown.

Table 9: RTL Kernel Interface and Port Requirements (cont'd)

Port or Interface	Description	Comment
s_axi_control	One (and only one) AXI4-Lite slave control interface	- Required port. The <code>s_axilite</code> interface is generally required with exception for some cases using AXI4-Stream interfaces. It is not required for non-software controlled kernels. - When used, the name must be exactly as shown, and is case-sensitive. TIP: The address range of the <code>s_axilite</code> interface can be edited in the <code>kernel.xml</code> file and repackaged using the <code>package_xo</code> command if needed. However, XRT imposes a 64K (16 bit) address range limitation. The tool will return an error if the <code>s_axilite</code> interface is greater than 16 bits wide.
AXI4_Memory Mapped Interface (m_axi)	AXI4 memory mapped interfaces for global memory access	- Optional port. - All AXI4 memory mapped interfaces must have 64-bit addresses (32 bits on AMD Zynq™ 7000 devices). - The RTL kernel developer is responsible for partitioning global memory spaces. Each partition in the global memory becomes a kernel argument. The memory offset for each partition must be provided by the SW applications to the kernel through a register in the AXI4-Lite interface. - AXI4 memory mapped must not use Wrap or Fixed burst types and must not use narrow (sub-size) bursts. This means that <code>AxSIZE</code> should match the width of the AXI data bus. - Any user logic or RTL code that does not conform to the requirements above, must be wrapped or bridged to satisfy these requirements.
AXI4_STREAM (axis)	AXI4-Stream interfaces for one-way data transfers between kernels or between the host application and kernels.	- Optional port. - Cannot be used with bi-directional ports. - Use the STREAM interface template in the Vivado Design Suite. - Refer to AXI4-Stream Interfaces in the <i>Vitis HLS User Guide</i> (UG1399) for additional information on interface requirements.

Notes:

- The clock requirements listed here are for newer platform shells which include fixed clocks as discussed in [Managing Clock Frequencies](#). RTL kernels for use on legacy platforms support two clocks named `ap_clk` and `ap_clk_2` specifically, and two optional resets named `ap_rst_n` and `ap_rst_n_2`.

Control Requirements for XRT-Managed Kernels



IMPORTANT! User-managed kernels do not require the control registers and signals described below, but they can implement a control structure using registers in an `s_axilite` interface as discussed in [Creating User-Managed RTL Kernels](#). If your RTL module implements a different control structure, you can define it as a `user_managed` kernel or it must be adapted to conform to the XRT-managed requirements described here.

The following table outlines the required register map for an XRT-managed kernel to be used within the Vitis tools and XRT. The control register is required by kernels that specify `ap_ctrl_hs` and `ap_ctrl_chain` control protocols as described in [Execution Modes](#). Kernels that implement `ap_ctrl_none` and `user_managed` control protocols do not require the control registers described below.



TIP: The interrupt related registers are only required for designs that implement interrupts.

All user-defined registers must begin at location `0x10`; locations below this are reserved. These include registers for kernel arguments such as scalar values and address offsets passed to memory mapped interfaces.

Table 10: Register Address Map

Offset	Name	Description
0x0	Control	Controls and provides kernel status.
0x4	Global Interrupt Enable	Used to enable interrupt to the host.
0x8	IP Interrupt Enable	Used to control which IP generated signals are used to generate an interrupt.
0xC	IP Interrupt Status	Provides interrupt status.
0x10	Kernel arguments	This would include scalars and global memory arguments for example.

The following table shows the control signals that are accessed through the control register (offset `0x0`). The control register and its signals are determined by the kernel execution mode, `ap_ctrl_hs` and `ap_ctrl_chain`.

The available signals are used by the different control protocols as explained in [Supported Kernel Execution Models](#) in the XRT documentation. For example, for the sequential execution mode `ap_ctrl_hs` the host typically writes `0x00000001` to the offset 0 control register which sets Bit 0, clears Bits 1 and 2, and polls on reading `ap_done` signal until it is a 1.

Table 11: Control Register Signals

Bit	Name	Description
0	<code>ap_start</code>	Asserted when the kernel can start processing data. Cleared on handshake with <code>ap_done</code> being asserted.
1	<code>ap_done</code>	Asserted when the kernel has completed operation. Cleared on read.
2	<code>ap_idle</code>	Asserted when the kernel is idle.
3	<code>ap_ready</code>	Asserted by the kernel when it is ready to accept the new data
4	<code>ap_continue</code>	Asserted by the XRT to allow kernel keep running
7	<code>auto_restart</code>	Used to enable automatic kernel restart as described in the chapter Auto-Restarting Kernels in <i>Vitis High-Level Synthesis User Guide</i> (UG1399).
31:5	Reserved	Reserved

The following interrupt related registers are only required if the kernel has an interrupt.

Table 12: Global Interrupt Enable (0x4)

Bit	Name	Description
0	Global Interrupt Enable	When asserted, along with the IP Interrupt Enable bit, the interrupt is enabled.
31:1	Reserved	Reserved

Table 13: IP Interrupt Enable (0x8)

Bit	Name	Description
0	Interrupt Enable	When asserted, along with the Global Interrupt Enable bit, the interrupt is enabled.
31:1	Reserved	Reserved

Table 14: IP Interrupt Status (0xC)

Bit	Name	Description
0	Interrupt Status	Toggle on write.
31:1	Reserved	Reserved

Interrupt

XRT-managed RTL kernels can optionally have an `interrupt` port containing a single interrupt. The port name must be called `interrupt` and be active-High. It is enabled when both the global interrupt enable (GIE) and interrupt enable register (IER) bits are asserted in the Control Register block.

The Vitis compiler (v++) will link the interrupt signal of a PL kernel into the available signals on the platform, provided the platform has interrupts available for connection as described in [Adding Hardware Interfaces](#). If no interrupt is enabled on the platform, then you must manually connect the interrupt of the kernel.

By default, the IER uses the internal `ap_done` signal to trigger an interrupt. Further, the interrupt is cleared only when writing a 1 to bit-0 of the IP Interrupt Status Register.

This logic should be reflected in the Verilog code for the RTL kernel, and also in the associated `component.xml` and `kernel.xml` files. The `kernel.xml` file is stored inside the `kernel.xo` file and is generated automatically when using the `package_xo` command or RTL Kernel Wizard.



IMPORTANT! The XRT native API does not support triggering or catching interrupts in the host application for user-managed RTL kernels.

Creating User-Managed RTL Kernels

If your RTL IP does not satisfy the AXI interface requirements for the Vitis compiler as outlined in [Kernel Interface Requirements](#), you must modify the IP to implement the required interfaces. However, if your RTL IP does not satisfy the XRT control protocols of `ap_ctrl_hs` or `ap_ctrl_chain`, you can define it as a user-managed kernel rather than having to rewrite your IP.

A user-managed kernel does not need to satisfy the control requirements of XRT, and can implement any of a variety of execution mechanisms. User-managed kernels are meant to let you take advantage of the system building capabilities of the Vitis compiler, while letting your kernel implement your own control scheme. There is no prescribed method of starting or stopping, or otherwise controlling your kernel. This is largely up to you, and the specific requirements of your application or system. Some of the available control schemes include:

- Accessing registers through an `s_axilite` control interface, similar to the method used by XRT though open to your own implementation
- Accessing the hardware through software drivers, such as UIO drivers, implemented in your host application
- Triggering the start or stop response of your kernel from a signal provided by a separate component, or from another kernel
- Providing a data-driven approach, as described in the topic Auto-Restarting Mode in the Vitis HLS User Guide ([UG1399](#)).



IMPORTANT! One limitation of implementing a control register in an `s_axilite` interface for a user-managed kernel is that the control register cannot be named `CTRL`. That name is specifically reserved for XRT-managed kernels, and returns a Critical Warning when found on a user-managed kernel, or `ap_ctrl_none` kernel.

RTL Kernel Development Flow

This section explains the process of creating RTL kernels using the Package IP feature inside the Vivado Design Suite. The Package IP command provides a Package for Vitis option which greatly simplifies packaging an existing RTL IP as a compiled object (.xo) file for use in the Vitis environment.

The packaged XO file is a container encapsulating the Vivado IP object (including source files) and associated kernel XML file. Using the Vitis compiler, the XO file can be combined with other kernels, and linked with the target platform and built for hardware or hardware emulation flows.

The Package for Vitis feature provides DRCs to check the completeness of the packaged IP prior to generating the XO file, and also automates the `package_xo` command to simplify the production of the packaged RTL kernel.

Packaging the RTL Code as a Vitis XO



IMPORTANT! The RTL IP should first be thoroughly verified with traditional RTL verification methods before being packaged as a kernel.

As discussed in [Kernel Interface Requirements](#), the RTL kernel must be packaged with the following required interfaces:

- The AXI4-Lite interface name must be packaged as `S_AXI_CONTROL`, but the underlying AXI ports can be named differently.
- Any memory-mapped AXI4 interfaces must be packaged as AXI4 master endpoints with 64-bit address support.



RECOMMENDED: AMD strongly recommends that AXI4 interfaces be packaged with AXI meta data `HAS_BURST=0` and `SUPPORTS_NARROW_BURST=0`. These properties can be set in an IP-level `bd.tcl` file. This indicates wrap and fixed burst type is not used, and narrow (sub-size burst) is not used.

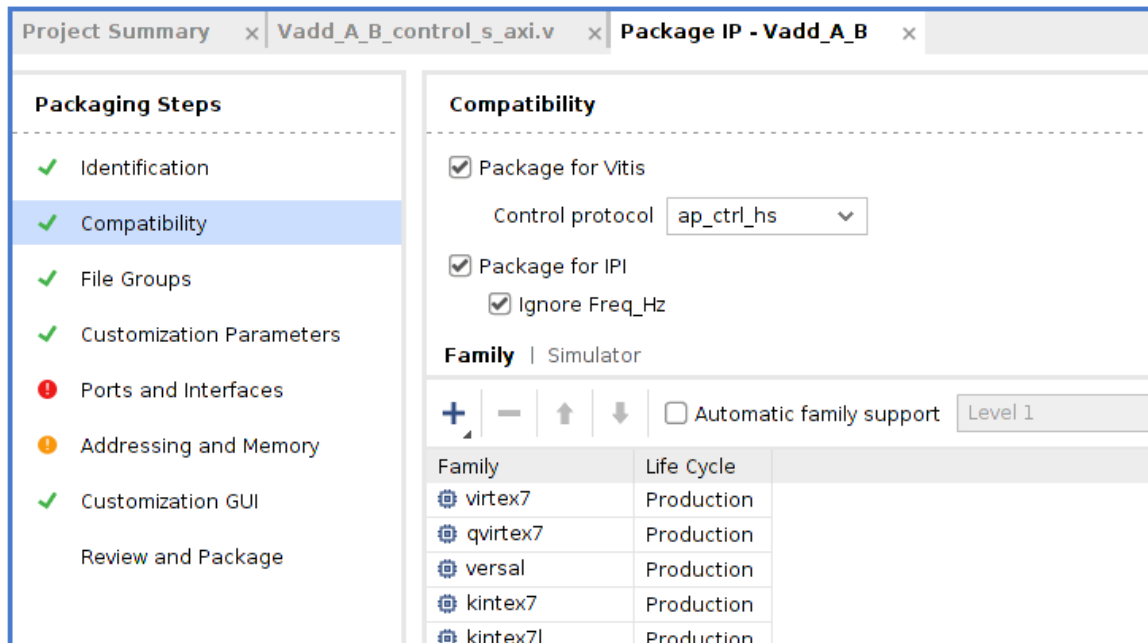
- You can also implement the AXI4-Stream interface.
- At least one clock is required for the kernel, though it can support multiple clocks.
 - Each clock must have an associated Bus Interface identifying it as a clock.
 - Each clock can have an optional active-Low reset, specified by the `ASSOCIATED_RESET` property on the clock.
 - A clock must be associated with each AXI4-Lite, AXI4, and AXI4-Stream interface on the kernel.

To package the IP, use the following steps:

1. Create and package a new IP.
 - a. From a Vivado project, with your RTL source files added, select **Tools → Create and Package New IP**.
 - b. Select **Package your current project**, and click **Next**.
 - c. Specify the location for your packaged IP. You can select the default location, or choose a different location.
 - d. Review the Summary page and click **Finish** to open the Package IP window.

The Package IP window opens to display the Identification page. For details on working with the IP packager in the Vivado tool, refer to the *Vivado Design Suite User Guide: Creating and Packaging Custom IP* ([UG1118](#)).

2. Select **Compatibility** under the Packaging Steps. This displays the Compatibility view as shown in the following figure.



- a. Select the **Package for Vitis** check box to enable the process of packaging the RTL IP as an XO for use in the Vitis environment.
- b. Select the **Control Protocol** for the RTL Kernel. This determines the control mechanism used to operate the kernel. The choices are:
 - `user_managed`: Defines a SW-controllable kernel, that is user-managed rather than XRT-managed. This is the preferred option. Refer to [Creating User-Managed RTL Kernels](#) for additional information.
 - `ap_ctrl_hs`: This is the default, and specifies the simple sequential execution model for XRT -managed kernels as described in [SW-Controllable Kernels](#).
 - `ap_ctrl_chain`: Specifies a pipelined execution model for XRT-managed kernels.
 - `ap_ctrl_none`: Indicates no control protocol as described in [Data Driven Kernels](#).
- c. Check to ensure that both **Package for IPI** and **Ignore Freq_Hz** are enabled as well.

Enabling these check boxes enables design rule checks (DRC) that the `ipx::check_integrity` command runs prior to packaging the IP and generating the XO. The DRCs include checks for required signals as described in [Requirements of an RTL Kernel](#), and checks for control protocols and registers for XRT-managed kernels. As shown in the preceding figure, any issues are reported to the Package IP tool as they are encountered.

3. Associate the clock to the AXI interfaces.

Select the **Ports and Interfaces** step of the Package IP window, you can associate the primary kernel clock with the AXI4 interfaces, and reset signal if needed.

- a. Right-click an AXI4 interface, and select **Associate Clocks**.

This opens the Associate Clocks dialog box which lists any identified clock signals.

- b. Select the appropriate clock and click **OK** to associate it with the interface.
 - c. Ensure to repeat this step to a clock signal with each of the AXI interfaces.
4. Click the **Addressing and Memory** step to add control registers and offsets.

XRT-managed kernels using the `ap_ctrl_hs` or `ap_ctrl_chain` control protocol require control registers as discussed in [Control Requirements for XRT-Managed Kernels](#). The following table shows a list of the required registers.



TIP: While `ap_ctrl_none` and `user_managed` control protocols do not require control registers, they can still use them if an `s_axilite` interface is included as part of the RTL design. In this case, the specific registers can differ from the following table, but the process of assigning `names`, `offsets`, and `widths` is the same.

Table 15: Address Map

Register Name	Description	Address Offset	Size
CTRL	Control Signals as described in Control Requirements for XRT-Managed Kernels .	0x000	32
GIER	Global Interrupt Enable Register. Used to enable interrupt to the host.	0x004	32
IP_IER	IP Interrupt Enable Register. Used to control which IP generated signal are used to generate an interrupt.	0x008	32
IP_ISR	IP Interrupt Status Register. Provides interrupt status.	0x00C	32
<kernel_args>	This includes a separate entry for each kernel argument as needed on the software function interface. All user-defined registers must begin at location 0x10; locations below this are reserved.	0x010	32/64 Scalar arguments are 32-bits wide. <code>m_axi</code> and <code>axis</code> interfaces are 64 bits wide.

- a. To create the address map described in the table, right-click in the **Address Blocks** and select the **Add Register** command.

This opens the Add Register view in which you can enter one of the register names from the table above.



IMPORTANT! The **Range** value under **Address Blocks** specifies the address range for the `s_axilite` interface. You can modify this value to change the range for the kernel.

- b. Repeat as needed to add all required registers.

This creates a Registers table in the Addressing and Memory section. You can edit the table to add the Description, Address Offset, and Size to each register. The Registers table should look similar to the following example.

Addressing and Memory

Name	Display Name	Description
 s_axi_control		

Address Blocks

Name	Display Name	Description	Base Address	Range	Range Dependency
 reg0			0	4096	$\text{pow}(2, (\text{C_S_AXI_CONTROL_ADDR_WIDTH} - 1) + 1)$

Registers

Name	Display Name	Description	Address Offset	Size
 CTRL		Control signals	0x000	32
 GIER		Global Interrupt Enable Register	0x004	32
 IP_IER		IP Interrupt Enable Register	0x008	32
 IP_ISR		IP Interrupt Status Register	0x00C	32
 scalar00			0x010	32
  A			0x018	64

Register Parameters

Name	Description	Display Name	Value	Value Bit String Length	Value Format	Value Source	Value Validation List	Maximum	Minimum	Parameter Types
 ASSOCIATED_BUSIF			m00_axi	0	string	default				
  B							0x024			64

Register Parameters

Name	Description	Display Name	Value	Value Bit String Length	Value Format	Value Source	Value Validation List	Maximum	Minimum	Parameter Types
 ASSOCIATED			m01_axi	0	string	default				

Memory Maps (for slaves)

Address Spaces (for masters)



TIP: The Tcl commands for each step of this process are written to the Tcl Console. You can use this fact to execute the process, and then use the Tcl transcript to create scripts to automate the process for future iterations.

- c. Finally, select the register for each of the pointer arguments from your table, right-click and select the **Add Register Parameter** command. Enter the name `ASSOCIATED_BUSIF` into the dialog box that opens, and click **OK**.

This lets you define an association between the register and the AXI4 Interface. In the value field of the added parameter, enter the name of the `m_axi` interface assigned to the specific argument you are defining. In the example above, the argument `A` uses the `m00_axi` interface, and the argument `B` uses the `m01_axi` interface.

5. At this point you should be ready to package your IP.
 - a. Select the **Review and Package** section of the Package IP view, review the Summary and After Packaging sections, and make whatever changes are needed.



IMPORTANT! You must enable the generation of an IP archive file. If the After Packaging section indicates An archive will not be generated, you must select the **Edit packaging settings** link and enable the **Create archive of IP** setting.

- b. When you are ready, click **Package IP**.

The Vivado tool packages your kernel IP, automatically runs the `package_xo` command as needed to produce the XO file, and opens a dialog box to inform you of success.

The generated XO file for the RTL kernel can be used by the Vitis compiler during the linking process to connect to other HLS or RTL kernels, and for linking with the target platform to complete the system. Refer to [Section IV: Building and Running the System](#) for more information.

- c. If your RTL kernel has some custom features that are not standard for the `package_xo` command that is run automatically, you can run the command manually to regenerate the XO file and kernel with custom settings. Refer to [package_xo Command](#) for details of the command. Some specific reasons why you might need to manually run the `package_xo` command include:

- Specify a different IP directory or XO path
- Output a copy of the `kernel.xml` file to modify it and repackage
- Include a C-model using the `package_xo -kernel_files` option to enable software emulation for your RTL kernel as described in [Adding C-Models to RTL Kernels](#)

6. Optional: Test the Packaged IP.

To test if the RTL kernel is packaged correctly for the IP integrator, try to instantiate the packaged kernel IP into a block design in the IP integrator. For information on the tool, refer to *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator* (UG994).

The kernel IP should show the various interfaces described above. Examine the IP in the canvas view. The properties of the AXI interface can be viewed by selecting the interface on the canvas. Then in the Block Interface Properties view, select the **Properties** tab and expand the CONFIG table entry. If an interface is to be read-only or write-only, the unused AXI channels can be removed and the `READ_WRITE_MODE` is set to read-only or write-only.

7. Optional: Configure Design Constraints.

If the RTL kernel has design constraints (`.xdc`) which refer to elements of the static region of the platform, such as clocks, then the constraint file needs to be marked as **late processing order** to ensure RTL kernel constraints are correctly applied.

There are two methods to mark constraints for late processing:

- a. If the constraints are given in a `.tcl` file, add `<: setFileProcessingOrder "late" :>` to the `.tcl` preamble section of the file as follows:

```
<: set ComponentName [getComponentNameString] :>
<: setOutputDirectory "/" :>
<: setFileName $ComponentName :>
<: setFileExtension ".xdc" :>
<: setFileProcessingOrder "late" :>
```

- b. If constraints are defined in an `.xdc` file, then add the following four lines starting at `<spirit:define>` in the `component.xml`. The four lines in the `component.xml` need to be next to the area where the `.xdc` file is called. In the following example, `my_ip_constraint.xdc` file is being called with the subsequent late processing order defined.

```
<spirit:file>
  <spirit:name>ttcl/my_ip_constraint.xdc</spirit:name>
  <spirit:userFileType>ttcl</spirit:userFileType>
  <spirit:userFileType>USED_IN_implementation</
spirit:userFileType>
  <spirit:userFileType>USED_IN_synthesis</spirit:userFileType>
  <spirit:define>
    <spirit:name>processing_order</spirit:name>
    <spirit:value>late</spirit:value>
  </spirit:define>
</spirit:file>
```

Adding C-Models to RTL Kernels



TIP: C-models are not supported for user-managed kernels. They are only supported for `ap_ctrl_hs` and `ap_ctrl_chain` kernels.

RTL kernels support the hardware emulation build, and the hardware build described in [Working with Build Targets](#), but an RTL kernel in its native form does not support software emulation. To support software emulation, you must add a C-model to the packaged RTL kernel using the `package_xo -kernel_files` option.

The C-model can be a simple C/C++ application defining the RTL function that is packaged with the RTL code for the kernel. The C-model is included in the packaged kernel (`.xo`) in a folder called `cpu_sources`, which the Vitis compiler uses during software emulation. For the hardware emulation and hardware builds, the Vitis compiler always use the RTL implementation of the kernel when creating the device binary (`.xclbin`).

Design Recommendations for RTL Kernels

RTL kernels should be designed with recommendations from the *UltraFast Design Methodology Guide for FPGAs and SoCs* ([UG949](#)). In addition to adhering to the interface and packaging requirements, the kernels should be designed with the following performance goals in mind:

- [Memory Performance Optimizations for AXI4 Interface](#)
- [Quality of Results Considerations](#)
- [Debug and Verification Considerations](#)

Memory Performance Optimizations for AXI4 Interface

The AXI4 interfaces typically connects to DDR memory controllers in the platform.



RECOMMENDED: *For optimal frequency and resource usage, it is recommended that one interface is used per memory controller.*

For best performance from the memory controller, the following is the recommended AXI interface behavior:

- Use an AXI data width that matches the native memory controller AXI data width, typically 512-bits.
- Do not use `WRAP`, `FIXED`, or sub-sized bursts.
- Use burst transfer as large as possible (up to 4k byte AXI4 protocol limit).
- Avoid use of deasserted write strobes. Deasserted write strobes can cause error-correction code (ECC) logic in the DDR memory controller to perform read-modify-write operations.
- Use pipelined AXI transactions.
- Avoid using threads if an AXI interface is only connected to one DDR controller.
- Avoid generating write address commands if the kernel does not have the ability to deliver the full write transaction (non-blocking write requests).
- Avoid generating read address commands if the kernel does not have the capacity to accept all the read data without back pressure (non-blocking read requests).
- If a read-only or write-only interfaces are desired, the ports of the unused channels can be commented out in the top level RTL file before the project is packaged into a kernel.
- Using multiple threads can cause larger resource requirements in the infrastructure IP between the kernel and the memory controllers.

Quality of Results Considerations

The following recommendations help improve results for timing and area:

- Pipeline all reset inputs and internally distribute resets avoiding high fanout nets.
- Reset only essential control logic flip-flops.
- Consider registering input and output signals to the extent possible.
- Understand the size of the kernel relative to the capacity of the target platforms to ensure fit, especially if multiple kernels will be instantiated.
- Recognize platforms that use stacked silicon interconnect (SSI) technology. These devices have multiple die and any logic that must cross between them should be flip-flop to flip-flop timing paths.

Debug and Verification Considerations

- RTL kernels should be verified in their own test bench using advanced verification techniques including verification components, randomization, and protocol checkers. The AXI Verification IP (VIP) is available in the Vivado IP catalog and can help with the verification of AXI interfaces. The RTL kernel example designs contain an AXI VIP-based test bench with sample stimulus files.
- You can add ILA inside of RTL kernels as described in [Adding Debug IP to RTL Kernels](#).
- Hardware emulation should be used to test the host code software integration or to view the interaction between multiple kernels.

Programming Versal AI Engines

As described in *AI Engine Kernel and Graph Programming Guide* ([UG1079](#)), an AI Engine kernel is a C/C++ program that is written using the AI Engine API and specialized intrinsic functions that target the VLIW scalar and vector processors of AMD Versal™ AI Core devices.

The AI Engine compiler produces ELF files that are run on the AI Engine processors. Multiple AI Engine kernels are combined in an adaptive data flow (ADF) graph that consists of nodes and edges where nodes represent compute kernel functions, and edges represent data connections. The ADF graph is a static data flow graph with kernels operating in parallel on data streams. The ADF graph interacts with the PL kernels in the device binary (`.xclbin`), global memory, and the host application as previously described. The AI Engine compiler produces a `libadf.a` file containing the graph application, which can be linked with PL kernels and extensible platform by the Vitis compiler to produce the device binary.

As described in *AI Engine Tools and Flows User Guide* ([UG1076](#)), the AI Engine compiler (`v++` or `aiecompiler`) is available as a standalone command, or as part of the Vitis unified IDE. The IDE provides a language sensitive editor, simulation, profiling, and debug of the component. The AI Engine component can be designed as a standalone component and integrated into a system project from the command line as described in [Section IV: Building and Running the System](#), or in the Vitis unified IDE as described in [Section VIII: Using the Vitis Unified IDE](#).

Building and Running the System

After the software program and the kernel code is written, you can build the application, which includes compiling the software program, and linking the platform with the PL kernel file to create the device binary (`.xclbin`). The build process follows a standard compilation and linking process for both the software program and the kernel code, followed by packaging the outputs for use. However, the first step in building the application is to identify the build target, indicating if you are building for test or simulation of the application, or building for the target hardware. After building, both the host program and the FPGA binary, you will be ready to run the application.

This section contains the following chapters:

- [Setting Up the Vitis Environment](#)
- [Working with Build Targets](#)
- [Building the Software Application](#)
- [Building the Device Binary](#)
- [Packaging the System](#)
- [Managing Vivado Synthesis, Implementation, and Timing Closure](#)
- [Running the System on Hardware](#)

While developing an AI Engine design graph, many design iterations are typically performed using the AI Engine compiler or AI Engine simulator tools. This method provides quick design iterations when focused on developing the AI Engine application. When ready, the AI Engine design can be integrated into a larger system design using the flow described in this chapter.

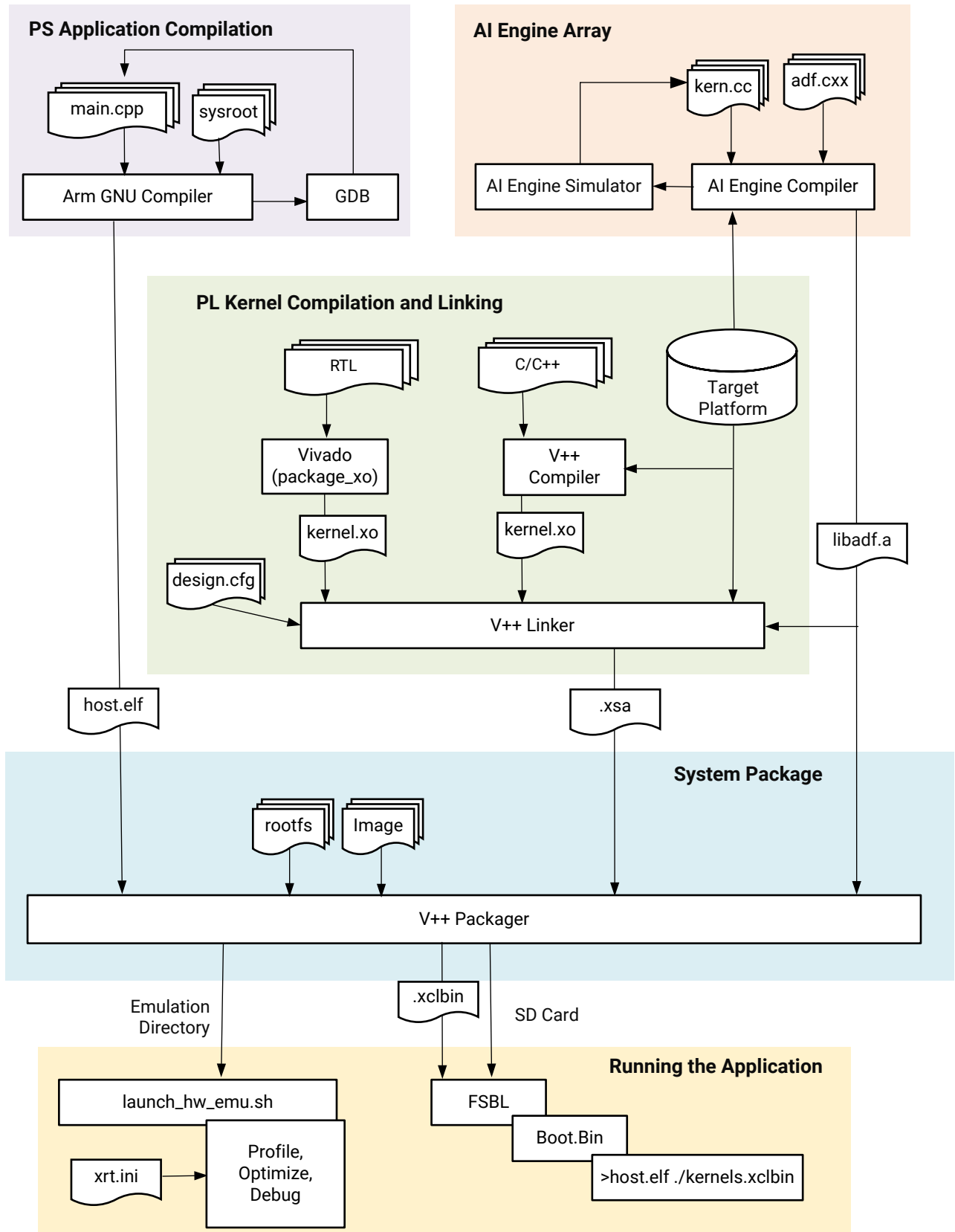
The AMD Vitis™ tools flow simplifies hardware design and integration with a software-like compilation and linking flow, integrating the four domains of the AMD Versal™ device: the AI Engine array, the programmable logic (PL) region, the network-on-chip (NoC) and the processing system (CIPS). The Vitis compiler flow lets you integrate your compiled AI Engine design graphs (`libadf.a`) with additional kernels implemented in the PL region of the device, including HLS and RTL kernels, and link them for use on a target platform. The Vitis compiler provides abstract

directives for accessing system memory, CPU control, and streaming I/O, so it is often possible to develop AI Engine graphs and kernels on a standard development platform and quickly re-target the AI Engine code to a custom platform developed for your specific application. You can control AI Engine graphs and PL kernels from code running on an embedded ARM processor in the Versal device or from an external CPU.

The following figure shows the high-level steps required to use the Vitis tools flow to integrate your application. The command-line process to run this flow is described here.

Note: You can also use this flow from within the Vitis IDE as explained in [Launching Vitis Unified IDE](#).

Figure 21: Vitis Tools Flow



X24782-040422



IMPORTANT! Using Vitis tools and AI Engine tools require the setup described in [Setting Up the Vitis Environment](#).

The following steps can be adapted to any AI Engine design in a Versal device.

1. As described in [Compiling AI Engine Graph Applications](#), the first step is to create and compile the AI Engine graph into a `libadf.a` file using the AI Engine compiler. You can iterate between the AI Engine compiler, and the AI Engine simulator to develop the graph, until you are ready to proceed.
2. [Compiling C/C++ PL Kernels](#): PL kernels are compiled for implementation in the PL region of the target platform using the `v++ --mode hls` command. In addition to Vitis compilation you can use Vivado to package RTL modules as kernels in the compiled `.xo` format as described in [Packaging RTL Kernels](#).
3. [Linking the System](#): Link the compiled AI Engine graph with the C/C++ kernels and RTL kernels onto a target platform. The process creates an XSA file to encapsulate the implemented hardware system to create boot and loadable images.
4. [Compiling the Embedded Application for the Cortex-A72 Processor](#): Optionally compile a host application to run on the Cortex®-A72 core processor using the GNU Arm cross-compiler to create an ELF file. The host program interacts with the AI Engine kernels and kernels in the PL region. This compilation step is optional because there are several ways to deploy and interact with the AI Engine kernels, and the host program running in the PS is one way.
5. [Packaging the System](#): Use the `v++ --package` process to gather the required files to configure and boot the system, to load and run the application, including the AI Engine graph and PL kernels. The packager can also be invoked to build the necessary package to run emulation and debug, or run your application on hardware.

Setting Up the Vitis Environment

The AMD Vitis™ unified software platform includes three elements that must be installed and configured to work together properly: the Vitis core development kit, the Xilinx Runtime (XRT), and an accelerator card such as the AMD Alveo™ Data Center accelerator card. The requirements of installation and configuration are described in [Installation](#).

If you have the elements of the Vitis software platform installed, you need to setup the environment to run in a specific command shell by running the following scripts (.csh scripts are also provided):

```
#setup XILINX_VITIS and XILINX_VIVADO variables
source <Vitis_install_path>/settings64.sh
#setup XILINX_XRT
source /opt/xilinx/xrt/setup.sh
```

You can also specify the location of the available platforms for use with your Vitis IDE by setting the following environment variable:

```
export PLATFORM_REPO_PATHS=<path to platforms>
```



TIP: The `PLATFORM_REPO_PATHS` environment variable points to directories containing platform files (.xpfm).

Working with Build Targets

The build target of the AMD Vitis™ tool defines the nature and contents of the FPGA binary (.xclbin) created during compilation and linking. There are three different build targets: two emulation targets used for validation and debugging purposes: software emulation and hardware emulation, and the default system hardware target used to generate the FPGA binary (.xclbin) loaded into the AMD device.

Compiling for an emulation target is significantly faster than compiling for the real hardware. The emulation run is performed in a simulation environment, which offers enhanced debug visibility and does not require an actual accelerator card.

Table 16: Comparison of Emulation Flows with Hardware Execution

Software Emulation	Hardware Emulation	Hardware Execution
Host application runs with a C/C++ model of the kernels.	Host application runs with a simulated RTL model of the kernels. SystemC models and external TGs are also supported.	Host application runs with actual hardware implementation of the kernels.
Used to confirm functional correctness of the system.	Test the host / kernel integration, get performance estimates.	Confirm that the system runs correctly and with desired performance.
Fastest build time supports quick design iterations.	Best debug capabilities, moderate compilation time with increased visibility of the kernels.	Final FPGA implementation, long build time with accurate (actual) performance results.

Software Emulation

The main goal of software emulation (sw_emu) is to ensure functional correctness of the host program and kernels. Software emulation provides a purely functional execution, without any modeling of timing delays, or latency; it does not give any indication of the accelerator performance.

The kernel code is always compiled and running natively on an x86 processor, as described in [Compiling PL Kernels for Software Emulation](#). The application code is either:

- Compiled and running natively on an x86 processor for Alveo Data Center acceleration cards, or for embedded platforms as described in [Embedded Processor Emulation Using PS on x86](#)
- Cross-compiled to the Arm® processor and running in an emulator as described in [Section V: Simulating the Application with the Emulation Flow](#)

Thus, software emulation is typically used for algorithm refinement, debugging functional issues, and letting developers iterate quickly through the code to make improvements. The software programming model of fast compilation and run iterations is preserved.

The `v++` compiler does the minimum transformation of the kernel code to create the FPGA binary to run the host program and kernel code together. Software emulation takes the C-based kernel code and compiles it with GCC. It runs each kernel as a separate C-thread. If there are multiple compute units of a single kernel, each CU is run as a separate thread. Therefore, it mimics the parallel execution model of the hardware. However, within each kernel the execution is modeled sequentially although there might be parallelism within a kernel when running on hardware. The software emulation driver implements the XRT API and acts as a bridge between the user application running XRT and the device process modeling the hardware components.



TIP: For RTL kernels, software emulation can be supported if a C model is associated with the kernel. The [RTL Kernel Development Flow](#) provides an option to associate C model files with the RTL kernel for support of software emulation flows.

The following describes the software emulation limitations:

- There is a global memory limit of 16 GB which should not be exceeded for simulation purposes.
- Software emulation does not support AXI4-Stream Interfaces without Side-Channels in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

As discussed in [v++ Command](#), the software emulation target is specified in the `v++` command with the `-t` option:

```
v++ -t sw_emu ...
```

You can use the GDB debugger for both the host application and the kernel code, set breakpoints or use `printf()` to print information and checkpoints. For details on how to debug the host application or the kernel during software emulation, refer to [Debugging in Software Emulation](#).

Hardware Emulation

Hardware emulation runs an RTL simulation of the programmable logic design, where the PL kernels are integrated with a cycle-approximate model of the hardware platform.

Hardware emulation is especially useful for the following tasks:

- Checking the functional correctness of the RTL code synthesized from the C, C++ kernel code
- Testing the interactions between different kernels or multiple CUs
- Using hardware waveforms to gain detailed visibility into internal activity of the kernels

- Getting initial performance estimates for the application
- SystemC models of HLS kernels can be used to speed up simulation when needed
- C/C++ or Python traffic generators can be used to inject traffic in the simulation
- Cycle-approx SystemC model of AI Engine kernels are available

Each kernel is compiled to a hardware model (RTL). During hardware emulation, kernels are run in the AMD Vivado™ logic simulator, with a waveform viewer to examine the kernel design. Some third-party simulators are also supported as described in [Simulator Support](#). In addition, hardware emulation provides performance and resource estimates for the hardware implementation.

SystemC models are provided for the key IP used in the hardware platform, like AMD Versal™ NoC/DDR memory, CIPS, PS block, AI Engine, AMD UltraScale+™ MIG DDR memory, and AXI4 SmartConnect. These IP models are used during hardware emulation to improve simulation performance and results.

In hardware emulation, compile and execution times are longer than software emulation, but it provides a detailed, cycle-accurate, view of kernel activity. AMD recommends using small data sets for validation during hardware emulation to keep runtimes manageable.

★ **IMPORTANT!** *The DDR memory model and the memory interface generator (MIG) model used in hardware emulation are high-level simulation models. These models provide good simulation performance, but only approximate latency values and are not cycle-accurate like the kernels. Therefore, performance numbers shown in the profile summary report are approximate, and should be used for guidance and for comparing relative performance between different kernel implementations.*

As discussed in [v++ Command](#), the hardware emulation target is specified in the `v++` command with the `-t` option:

```
v++ -t hw_emu ...
```

System Hardware Target

When the build target is the hardware, `v++` builds the FPGA binary for the AMD device by running Vivado synthesis and implementation on the design. It is normal for this build target to take a longer period of time than generating either the software or hardware emulation targets in the Vitis IDE. However, the final FPGA binary can be loaded into the hardware of the accelerator card, or embedded processor platform, and the application can be run in its actual operating environment.

As discussed in [v++ Command](#), the system hardware target is specified in the `v++` command with the `-t` option:

```
v++ -t hw ...
```

Building the Software Application

The software application, written in C/C++ using the XRT native API, is built using the GNU C++ compiler (`g++`) which is based on GNU compiler collection (GCC). Each source file is compiled to an object file (`.o`) and linked with the Xilinx Runtime (XRT) shared library to create the executable which runs on an x86 or embedded Arm processor.

To use the native XRT APIs, the host application must link with the `xrt_coreutil` library. For example:

```
g++ -g -std=c++17 -I$XILINX_XRT/include -L$XILINX_XRT/lib -lxrt_coreutil -pthread
```

Compiling host code with XRT native C++ API requires C++ standard with `-std=c++17`. On GCC version older than 4.9.0, use `-std=c++1y` instead because `-std=c++17` is introduced to GCC from 4.9.0.



TIP: `g++` supports many standard GCC options which are not documented here. For information refer to the [GCC Option Summary](#).

Compiling and Linking for x86



TIP: Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the tools.

Compiling and linking for x86 follows the standard `g++` flow. The only requirement is to include the XRT header files and link the XRT shared libraries. Each source file of the host application is compiled into an object file (`.o`) using the `g++` compiler. The generated object files (`.o`) are linked with the Xilinx Runtime (XRT) shared library to create the executable host program using the `g++ -l` option.

The host application can be written in native C++ using the Xilinx Runtime (XRT) native C++ API. The required include files and libraries depend on the API your host application uses, and any specific requirements of your host code.

To use the native XRT API, the host application must link with the `xrt_coreutil` library. The command line uses a few different settings as shown in the following example, which combines compilation and linking:

```
$CXX -std=c++17 -O0 -g -Wall -c -I./src -o host.o sw/host.cpp
```

When compiling the source code using XRT native API, the following `g++` options are required:

- `-std=c++17`: Define the C++ language standard. Compiling host code with XRT native C++ API requires C++ standard with `-std=c++17` or newer. However, on GCC versions older than 4.9.0, use `-std=c++1y` instead.
- `-I${XILINX_XRT}/include/`: XRT include directory
- `-I${XILINX_VITIS}/aietools/include`: AI Engine Include directory
- `-I./src/aie`: Include AI Engine graph and kernel source files

When linking the executable, the following `g++` options are required:

- `-L${XILINX_XRT}/lib/`: Look in XRT library.
- `-lxrt_coreutil`: Search the named library during linking.
- `-pthread`: Search the named library during linking.

Command to Link the Host Application Against Required XRT APIs and Generate the Executable:

```
g++ *.o -lxrt_coreutil -L${XILINX_XRT}/lib -o ${EXECUTABLE}
```

ADF API:

To use the ADF API, here is the command to compile the host application:

```
g++ -Wall -c -std=c++17 -Wno-int-to-pointer-cast -I${XILINX_XRT}/include \
-I./src/aie -I./ -I${XILINX_VITIS}/aietools/include \
-o aie_control_xrt.o ./Work/ps/c_rts/aie_control_xrt.cpp$
```

The following is the command to link the host application against required XRT APIs and generate the executable:

```
-ladf_api_xrt -L${XILINX_VITIS}/aietools/lib/lx64.o
```

Compiling an Embedded Application for PS on x86

In order to compile an embedded application to run on an x86 processor, as described in [Embedded Processor Emulation Using PS on x86](#) you must use the x86 version of gcc or g++ compiler version 8.3 or later.



TIP: x86 compilation is supported for the software emulation flow only, and requires the x86 installation of XRT as described in [Installing Xilinx Runtime and Platforms](#). This sets the `LD_LIBRARY_PATH` variable to point to XRT libraries.

Compile and Link the Host Application Against Required XRT APIs and Generate the Executable

```
g++ -Wall -c -std=c++17 -Wno-int-to-pointer-cast -I${XILINX_XRT}/include \
-I./src/aie -I./ -I${XILINX_VITIS}/aietools/include \
-o aie_control_xrt.o ./Work/ps/c_rts/aie_control_xrt.cpp $(HOST_SRCS) -o
main.o
```

```
g++ *.o -lxrt_coreutil -ladf_api_xrt -L${XILINX_VITIS}/aietools/lib/lnx64.o \
-L${XILINX_XRT}/lib -o $(EXECUTABLE)
```

Note: For `ps_on_x86` compilation, you must set the GCC path from the Vitis Install:

```
setenv PATH ${XILINX_VITIS}/aietools/tps/lnx64/gcc/bin:$PATH
```

If you want to switch between the embedded (ARM-GCC) flow and x86 compilation flow on the same setup (terminal), you need to explicitly specify ARM-GCC and `SYSROOT` paths for compiling and linking the application for ARM-GCC based flow. The scenario is applicable if you are trying to run the emulation using cross compilation of the host application and simultaneously want to run software emulation using native x86 compilation of the host application on the same shell (terminal).

Note: After running native x86 compilation, if you are trying to compile the host application using ARM-GCC compiler in the same terminal, you will source the environment setup script `xilinx-versal-common-v2023.1/environment-setup-cortexa72-cortexa53-xilinx-linux`. Ensure the `LD_LIBRARY_PATH` variable is not set because a warning will be issued if this variable is set. You must then unset the `LD_LIBRARY_PATH` and re-run the environment setup script.

For information on how to run PS on x86 for an AI Engine kernel, see the [AIE Adder](#) example on GitHub.

Limitations

Arm-only data types or libraries are not supported in the host code while running the x86 compilation.

Below is the host code example using `_fp16` (floating point 16) data types which is only supported in the ARM-GCC based compiler. The x86 compiler issues error while compiling the same host code. In such situation, AMD recommends using the QEMU model of the PS and compiling the PS application with ARM-GCC based compiler.

```
#define LENGTH (1024)
#define HALF __fp16
int main(int argc, char* argv[])
{
    unsigned fileBufSize;
```

```
std::string binaryFile = argv[1];
size_t vector_size_bytes = sizeof(HALF) * LENGTH;
//Source Memories
std::vector<HALF> source_a(LENGTH);
std::vector<HALF> source_b(LENGTH);
std::vector<HALF> result_sim (LENGTH);
std::vector<HALF> result_krnl(LENGTH);
/* Create the test data and golden data locally */
for(int i=0; i < LENGTH; i++){
    source_a[i] = i;
    source_b[i] = i*2;
    result_sim[i] = source_a[i] + source_b[i];
}
```



TIP: As described in [Compiling the Embedded Application for the Cortex-A72 Processor](#), you can also target the QEMU model of the PS by compiling the embedded application in software emulation flow using the ARM-GCC compiler.

Compiling the Embedded Application for the Cortex-A72 Processor

After linking the AI Engine graph and PL kernels, the focus moves to the embedded application running in the PS that interacts with the AI Engine graph and kernels. The PS application is written in C/C++, using API calls to control the initialization, running, and closing of the AI Engine graph as described in [Run-Time Graph Control API](#) in *AI Engine Kernel and Graph Programming Guide* ([UG1079](#)).

For details about compiling and linking host code using the XRT API, see [Compiling and Linking Host Code for Linux](#) in the *AI Engine Tools and Flows User Guide* ([UG1076](#)).

```
$CXX -std=c++17 -O0 -g -Wall -c \
-I./src -I${XILINX_VITIS}/aietools/include -o sw/host.o sw/host.cpp

$CXX -std=c++17 -O0 -g -Wall -c \
-I./src -I${XILINX_VITIS}/aietools/include -o
```

Many of the options in the preceding command are standard and can be found in a description of the `g++` command. The more important options are listed as follows:

- `-std=c++17`
- `-I./src`
- `-I${XILINX_VITIS}/aietools/include`
- `-o sw/host.o sw/host.cpp`

`aie_control_xrt.cpp` is copied from the directory `Work/ps/c_rts`.

```
$CXX -lgcc -lc -lpthread -lrt -ldl \
-lcrypt -lstdc++ -lxrt_coreutil \
-L<platform_path>/sysroots/aarch64-xilinx-linux/usr/lib \
-o sw/host.exe sw/host.o sw/aie_control_xrt.o
```

Note in the preceding linker script that it links the `adf_api_xrt` libraries, which is necessary for the ADF API to work with the XRT API.

`xrt_coreutil` are required libraries for XRT and for the XRT API.

While many of the options can be found in a description of the `g++` command, some of the more important options are listed in the following table.

If any ADF API is used in the host code, compiling host code requires additional ADF API related include files and libraries files, and also `aie_control_xrt.cpp` from `aiecompiler` result, such as:

```
$CXX -std=c++17 -O0 -g -Wall -c \
-I./src -I${XILINX_VITIS}/aietools/include -o sw/host.o sw/host.cpp

$CXX -std=c++17 -O0 -g -Wall -c \
-I./src -I${XILINX_VITIS}/aietools/include -o sw/aie_control_xrt.o Work/ps/
c_rts/aie_control_xrt.cpp
```

`aie_control_xrt.cpp` is copied from the directory `Work/ps/c_rts`.

```
$CXX -ladf_api_xrt -lgcc -lc -lpthread -lrt -ldl \
-lcrypt -lstdc++ -lxrt_coreutil \
-L<platform_path>/sysroots/aarch64-xilinx-linux/usr/lib \
-L${XILINX_VITIS}/aietools/lib/aarch64.o -o sw/host.exe sw/host.o sw/
aie_control_xrt.o
```

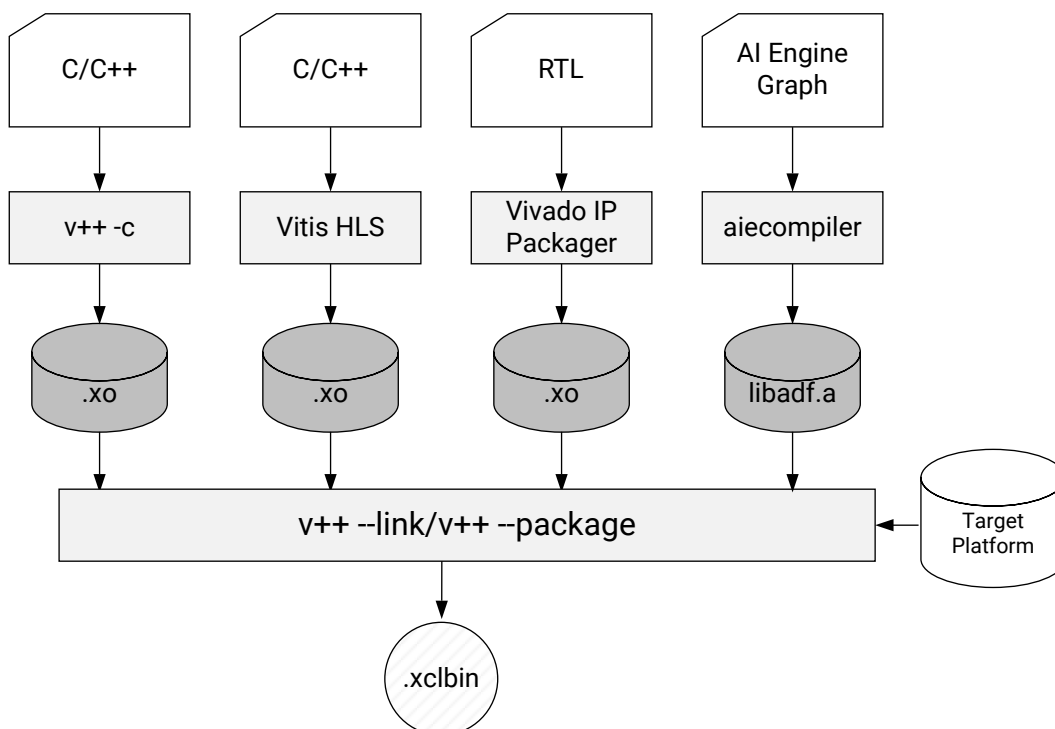
Table 17: Command Options

Option	Description
<code>-ladf_api_xrt</code>	Required for the ADF API. This is used to control the AI Engine through XRT. If not controlling with XRT, use <code>-ladf_api</code> with the path <code>-L\${XILINX_VITIS}/aietools/lib/aarch64none.so</code> . For more information see Host Programming for Bare-Metal in <i>AI Engine Tools and Flows User Guide</i> (UG1076).
<code>-lxrt_coreutil</code>	Required for the XRT API.
<code>-L<platform_path>/sysroots/aarch64-xilinx-linux/usr/lib</code>	
<code>-L\${XILINX_VITIS}/aietools/lib/aarch64.o</code>	
<code>-o sw/host.exe</code>	

Building the Device Binary

After the C/C++ kernels are compiled into an object (XO) file, and any RTL kernels are packaged as XO files, and the AI Engine graph application is compiled into a `libadf.a` file, the Vitis `v++ --link` command links them with the target platform to build the platform file (XSA), or device binary (`.xclbin`) file as shown in the following figure.

Figure 22: Device Build Process



X21155-060321

The process, as outlined above, has two steps:

1. Compile the design components from the source code.
 - For C, C++ kernels, the `v++ -c --mode hls` command compiles the source code into object (XO) files. Multiple kernels are compiled into separate XO files.

- For RTL kernels, the Vivado IP packager command produces the XO file to be used for linking. Refer to [Packaging RTL Kernels](#) for more information.
 - Any AI Engine graph application is compiled with the `v++ -c --mode aie` command to create the `libadf.a` file.
2. After compilation, the `v++ -l` command links one or multiple kernel objects (XO), and the `libadf.a` file if present, together with the hardware platform, to produce the Xilinx binary `.xclbin` file or updated `.xsa` file.

The following sections describe building the different components of the system design to produce the device binary.



TIP: The `v++` command can be used from the command line, in scripts, or a build system like `make`, and can also be used through the Vitis unified IDE as discussed in [Section VIII: Using the Vitis Unified IDE](#).

Compiling C/C++ PL Kernels



IMPORTANT! Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the tools.

The techniques of programming, simulating, and synthesizing the PL kernel objects are described in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)). The kernel object file (`.xo`) can be compiled by the HLS compiler (`v++ -c --mode hls`) as described below. The compiled object file can be linked with the AI Engine graph application, and the target platform, and used with the `v++ --link` command as described in [Linking the System](#).



TIP: The PL kernel must be compiled (`-c`) and linked (`-l`) in two separate steps.

To compile the PL kernel use the following command line as an example:

```
v++ -c --mode hls --config ./src/hls_config.cfg --work_dir vadd
```

The various arguments used are described below:

- `-c`: Specifies the compilation mode of the `v++` command. This can be specified using `-c` or `--compile`
- `--mode hls`: Launches the HLS compiler form of the Vitis compiler
- `--config <config_filename>`: Specify a Configuration file for use with the HLS compiler. The configuration file has HLS compiler options as described in [v++ Mode HLS](#). Refer to *Creating an HLS Component* in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)) for additional details. The Configuration file will specify:
 - The target platform or part for the build. The platform specified in the `v++ --link` command must match the target platform or part used for the HLS compile command

- The source C++ file or files for the PL kernel
- The Configuration file will also specify an output name if one is needed. The default output name is the same as the top-level function with the `.xo` extension
- `--work_dir`: Specifies the location of the HLS component created by the `v++` compiler



TIP: The HLS compiler mode does not require a target. The compiled object file (`.xo`) is suitable for both the hardware emulation build and the hardware build. However, the `v++ -c --mode hls` command does not produce a PL kernel for use in software emulation. To generate the software emulation target you must use the method explained in [Compiling PL Kernels for Software Emulation](#).

Refer to [Output Directories of the v++ Command](#) to get an understanding of the location of various output files generated by the HLS compiler.

After the compilation step is complete, any reports generated during this process are collected into the `<kernel_name>.compile_summary`. This collection of reports can be viewed by opening the `compile_summary` in the Analysis view of Vitis unified IDE, and includes a Summary report, Kernel Estimate for timing and resource estimates, Kernel Guidance offering any suggestions for compilation, and the HLS Synthesis log. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for additional information.

Compiling PL Kernels for Software Emulation

Compiling a PL kernel for use in the software emulation flow requires the `v++ -c -k` form of the command. Use the following command line as an example to build the software emulation target:

```
v++ -t sw_emu --platform xilinx_u200_gen3x16_xdma_2_202110_1 -c -k vadd \
-I'./src' -o'vadd.sw_emu.xo' ./src/vadd.cpp
```

The following list details the options specified in the command shown above, as described in [v++ General Options](#):

- `-t <arg>`: Specifies the build target as software emulation (`sw_emu`)
- `-c`: Compile the kernel. Required. The kernel must be compiled (`-c`) and linked (`-l`) in two separate steps
- `--platform <arg>`: Specifies the target platform to compile the PL kernel for. The platform must match the platform specified in the `v++ --link` command
- `-k <arg>`: Name of the PL kernel associated with the source files
- `-o <arg>`: Specify the output file `.xo` for the compiler
- `<input_file>`: Can be specified as a positional argument on the command line, or using the `--input` option

Compiling AI Engine Graph Applications



IMPORTANT! Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the tools.

The techniques of programming, simulating, and compiling an AI Engine component are described in *AI Engine Tools and Flows User Guide* ([UG1076](#)). Versal AI Engine graph applications are written in C++ and compiled using the `v++ -c --mode aie` command, as shown below. The command uses a configuration file command language as described in [v++ Mode AI Engine](#).

The compiled graph application can be linked with PL kernel object files (`.xo`), and the target platform as described in [Linking the System](#).

Compiling for x86simulator

To build the AI Engine component for simulation with the x86simulator, the command-line is as follows:

```
v++ -c --mode aie --target x86sim --platform xilinx_vck190_base_202310_1 \
--config ./aiecompiler.cfg --work_dir ./aie-sys-design-aie/Work ./src/
graph.cpp
```

The various arguments used are described below.

- `-c`: Specifies the compilation mode of the `v++` command. This can be specified using `-c` or `--compile`
- `--mode aie`: Launches the AI Engine compiler form of the Vitis compiler
- `-target=x86sim`: Specifies the build target as either for simulation purposes (`x86`) or for running on the physical device (`hw`). The default is `hw`.
- `--platform=xilinx_vck190_base_202320_1`: Specifies the platform to use when compiling the graph application.



TIP: AI Engine components require platforms using Versal AI Engine devices.

- `--config <config_filename>`: Specify a Configuration file for use with the AI Engine compiler. An example config file is shown below.
- `--work_dir`: Specifies the location of the output files created by the `v++` compiler
- `<source_file>`: Specify source files for the kernel and graph. Multiple source files can be specified.

The contents of a configuration file can vary, but the example uses the following commands in the `aiecompiler.cfg` file:

```
include=./aie_sys_design_aie
include=./aie_sys_design_common/src
include=./aie_sys_design_aie/src

[aie]
Xchess=main:darts.xargs=-nb
log-level=1
stacksize=1024
heapsize=1024
```

After compilation the AI Engine component can then be run through the `x86simulator` using the following command:

```
x86simulator --pkg-dir=./Work --i=../..
```

Where `--pkg-dir` specifies the `Work` directory where compilation occurs, and the `--i` specifies the input directory path for the simulator

Compiling for AI Engine Simulator / Hardware

To build the AI Engine component for simulation with the `aiesimulator`, the command-line is as follows:

```
v++ -c --mode aie --target hw --platform xilinx_vck190_base_202310_1
--config ./aiecompiler.cfg --work_dir ./aie_sys_design_aie/Work ./src/
graph.cpp
```



TIP: Notice only the target has changed between the `x86simulator` and `aiesimulator` builds.

The AI Engine component can then be run through the `aiesimulator` using the following command:

```
aiesimulator --pkg-dir=./Work --i=../..
```

The default output is an archive of the AI Engine graph application called `libadf.a`. Refer to the chapter on Compiling an AI Engine Graph Application in *AI Engine Tools and Flows User Guide* ([UG1076](#)) for additional information. The archive file can be used by the `v++ --link` command to build a fixed platform of the integrated system design as described in the next section.

Refer to [Output Directories of the v++ Command](#) to get an understanding of the location of various output files generated by the AI Engine compiler.

Deploying the AI Engine Graph

The Vitis design execution model has multiple considerations that impact how the AI Engine graph is loaded onto the board, run, reset, and reloaded. Depending on the needs of the application you have a choice of loading the AI Engine graph at board boot up time, or using the PS host application. In addition, you can also control running the graph as soon as the graph is loaded or defer it to a later time. You also have the option of running the graph infinitely or for a fixed number of iterations or cycles.

AI Engine Graph Load and Run

The AI Engine graph can be loaded and run immediately at boot, or it can be loaded by the host PS application. Additionally, you also have the option of deferring the running of the graph after the graph has been loaded using the `graph.run()` host API XRT call.

By default, the AMD platform management controller (PMC) loads the graph from the PDI and runs the graph immediately after loading. However the `v++ --package.defer_aie_run` command will let you defer the graph run until after the graph has been loaded using the `graph.run()` API call in the PS application. The following table lists the deployment options.

Table 18: Deploying the AI Engine Graph

Run Forever	Host Control
By default the graph is enabled in the PDI (device binary) and the graph runs until the system exits	Specify <code>v++ --package.defer_aie_run</code> command when packaging the system to prevent the AI Engine graph application from starting at boot-up. Enable the graph as needed from the PS application using <code>graph.run()</code>

AI Engine Run Iterations

By default, the AI Engine graph application runs indefinitely. However, it can be made to run for a limited number of iterations. Use the `graph.run(run_iterations)` or `graph.end(cycles)` to limit the number of graph runs to a specific number of iterations or for a specific number of clock cycles. See [Run-Time Graph Control API](#) in *AI Engine Kernel and Graph Programming Guide* (UG1079).

Iterative AI Engine Application Compilation

AI Engine application development can start early in the system development stage. Gradually, the AI Engine development team and the hardware development team converge on an interface between the Programmable Logic and the AI Engine array. At some point, this interface is fixed and should not be changed, but the AI Engine logic can continue to evolve as long as this interface remains unchanged. Essentially the logic inside the AI Engine application can change as long as the interface to the PL kernels and the platform does not.

The hardware and the AI Engine development teams can have independent development processes. After an AI Engine application compilation, the compiler generates many files in the `Workdirectory` that identify the decisions made during the entire process. One of the files called `Work/temp/graph_aie_routed.aiecst` contains, among other information, all the interface specification between the AI Engine array, and the PL and PS in JSON format. The `NodeConstraints` area contains the description of all the interfaces you defined in your graph, with their column location and channel selection.

1. You can extract this data and store it in another file in JSON format:

```
{
  "NodeConstraints": {
    "DataIn1": {
      "shim": {
        "column": 24,
        "channel": 0
      }
    },
    "clip_in": {
      "shim": {
        "column": 24,
        "channel": 0
      }
    },
    "clip_out": {
      "shim": {
        "column": 25,
        "channel": 0
      }
    },
    "DataOut1": {
      "shim": {
        "column": 25,
        "channel": 0
      }
    }
  }
}
```

2. The AI Engine application can be modified and recompiled by providing the previously extracted interface constraints file to the compiler:

```
v++ -c --mode aie $(AIE_FLAGS) --workdir=./Work2 --
constraints=interface.aiecst graph.cpp
```

A new `libadf.a` is created, unless you specify another name, and can be directly packaged with the XSA that has been generated previously by the link stage and the host executable.

You can manually change the constraints file without having to wait for a new link of the system and provide your `libadf.a` file afterwards to the hardware team.

A tutorial including all the steps to perform this task is available at: https://github.com/Xilinx/Vitis-Tutorials/tree/2023.1/AI_Engine_Development/Feature_Tutorials/15-post-link-recompile.

Developing the AI Engine Software using a Fixed Hardware Platform

After building the hardware design, the AI Engine / Programmable Logic (PL) interface is fixed, but you can recompile AI Engine graphs and kernels against this fixed hardware as often as desired. In fact, the kernels and graphs can be quite different across such AI Engine software-only compiles as long as the AI Engine / Programmable Logic interface remains fixed. The AI Engine compiler will issue an error when you attempt a software-only AI Engine compile that cannot conform the fixed AI Engine / PL interface. After the system link stage, Vitis generates a new platform with an `.xsa` extension. This file contains a lot of information beside the bitstream, in particular the AI Engine-PL interface constraints. The AI Engine developer can develop software based on this hardware platform. This file can be used by the `v++` command to integrate automatically all the constraints:

```
v++ -c --mode aie -include ... -platform=LinkedPlatform.xsa newgraph.cpp
```

The host code can also be adapted to the new graph, compiled and packaged with the new `libadf.a` to generate an image.



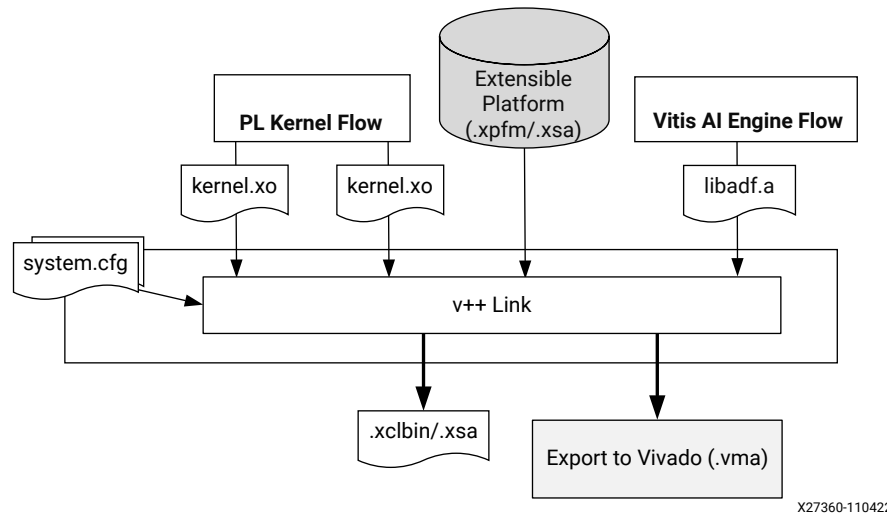
TIP: The Vitis `v++` linker and platform interfaces abstract CPU control, DDR memory, and streaming I/O, so it can be advantageous to build an application-specific hardware test harness targeting a standard development board so that you can develop, compile, and run AI Engine code at speed in hardware. Re-targeting the AI Engine application code to a custom application-specific platform consists simply of reapplying the `v++` linker directives for the AI Engine when targeting the custom platform.

Linking the System



TIP: Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the tools.

Figure 23: Linking the System Design



X27360-110422

The preceding figure shows two paths through the linking process, starting with the `v++ --link` command. The first path is the Vitis Integrated Flow in which the `v++` command links the elements of the system, automatically launches the Vivado tools for implementation of the design, and outputs an `.xclbin` or `.xsa` file. The second path is the Vitis Export to Vivado flow in which the `v++` command links the elements of the system and outputs a `.vma` file for you to use in the Vivado tools for synthesis, implementation, and timing closure. These two paths are explained below.

Note: The Export to Vivado flow is currently only supported for custom Versal platforms.

Vitis Integrated Flow

As shown in the preceding figure, the PL kernel compilation process results in an object (`.xo`) file whether the kernel is written in C/C++ or RTL. During the linking stage, one or more PL kernels are linked with the extensible platform to create the FPGA binary container file (`.xclbin`). For Versal adaptive SoC devices, the linking process also includes an AI Engine graph application (`libadf.a`) and creates a fixed hardware platform (`.xsa`) for use by the Vitis packaging process as described in [Packaging the System](#).

Note: The Integrated Flow automatically runs Vivado synthesis and implementation under the hood (hence the name "integrated"). This is a major difference.

The following is an example command line to link the `vadd` kernel (`.xo`) with a `libadf.a` graph archive and a Versal adaptive SoC platform, specifying the `.xsa` file as the output:

```
v++ -t hw_emu --platform xilinx_vck190_base_202310_1 --link vadd.xo
libadf.a \
--config ./system.cfg -o binary_container.xsa
```

This command contains the following arguments:

- **-t <arg>**: Specifies the build target. When linking, you must use the same **-t** and **--platform** arguments specified when compiling the PL kernels and AI Engine graph application.
- **--platform <arg>**: Specifies the platform to link with the system design. In the example command above the `custom_vck190` platform is a custom platform designed to work with the `--export_archive` command.
- **--link**: Link the kernels, graph, and platform into a system design.
- **<input>.xo**: Specifies the input PL kernel object files (`.xo`) to link with the AI Engine graph and the target platform. This is a positional parameter.
- **libadf.a**: Specifies the input compiled AI Engine graph application to link with the PL kernels and the target platform. This is a positional parameter.
- **--config ./system.cfg**: Specify a configuration file that is used to provide `v++` command options for a variety of uses. Refer to [Vitis Compiler Configuration File](#) for more information on the `--config` option.
- **-o binary_container.xsa**: Specifies the output file name. The output file in the link stage will be an `.xsa` file due to the use of a Versal platform. The default output name is `a.xsa`.



TIP: For Alveo accelerator cards, or AMD Zynq™ MPSoC based platforms, the output of the link command will be an `.xclbin` file rather than the `.xsa`.

Vitis Export to Vivado Flow

Alternatively, for custom Versal platforms designed specifically to support this feature, `v++ --link --export_archive` creates a Vitis Metadata Archive (`.vma`) file for export to the Vivado Design Suite. This flow lets you open the linked system design in the Vivado tools for more directed synthesis, place and route, and timing closure.

Note: In this Export flow, the linker stops pre-synthesis.

An example command follows:

```
v++ --platform custom_vck190 --link vadd.xo libadf.a --config ./system.cfg \
--export_archive -o hw-vadd.vma
```



IMPORTANT! The `--export_archive` command cannot be used with the `--target` (or `-t`) command. An error will be returned.

This command is similar to the prior command, with the following differences:

- **--export_archive**: Specifies the creation of the `.vma` file to export to the Vivado Design Suite. This option stops `v++` from automatically running Vivado synthesis and place and route, and instead lets you manually launch and direct the implementation and timing closure of the design as described in [Vitis Export to Vivado Flow](#).

- `--platform custom_vck190`: The `--export_archive` command can only be used with a custom Versal platform designed with block design containers (BDC) required for this design flow. Refer to the chapter on Introduction to Block Design Containers (*Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)) for more information.
- `-o hw-vadd.vma`: Specifies the output file name for the `.vma` file produced by the `--export_archive` command.

After Linking

After the linking step is complete, any reports generated during this process are collected into the `<kernel_name>.link_summary`. This collection of reports can be viewed by opening the `link_summary` in the Analysis view of Vitis analyzer, and includes a Summary report, System Estimate providing timing and resources estimates, System Guidance offering any suggestions for improving linking and the performance of the system. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for additional information.



TIP: Refer to [Output Directories of the v++ Command](#) to get an understanding of the location of various output files.

The linking process defines important architectural details of the system design. In particular, this is where the design is enabled for profiling or debug, where you specify the number of compute unit (CUs) to instantiate into hardware, where CUs are assigned to SLRs, and where you define connections from PL kernel ports to global memory or to AI Engine applications. The following sections discuss some of these build options.

Enabling Profile and Debug when Linking

To capture and visualize profiling and trace information, or to enable your design for debugging, you will need to add specific commands during the `v++` linking phase, and sometimes during `v++` compilation. The tool must instrument the profile IP using [--profile Options](#) in the `v++` linking phase and enable the profiling during the runtime. To enable debugging the application you can specify on of the [--debug Options](#).

During `v++` linking, the application developer needs to add profile IP to the design to capture the profiling data and preferably choose the memory resources for storing and offloading data during the runtime.

- You can add PL monitors to capture tracing information on their design by using `--profile` command. This adds the logic to capture profile data for data traffic between the kernel and host, kernel stalls, the execution times of kernels, and compute units (CUs). The instrumentation can be added using options, `--profile.data`, `--profile.stall`, and `--profile.exec`, as described in the [--profile Options](#).

- You also can specify the choice of memory resources or FIFO in the PL to store captured data. On large designs that span multiple SLRs, the tracing infrastructure can cause timing issues as there is one offload point and all trace data must cross SLRs to reach it. For these use cases, multiple memory resources can be used for offloading the trace data.

To enable the capture of profile data or trace information during the application runtime, you can choose from a variety of options to add to the [xrt.ini File](#) which configures the runtime. See [Profiling the Application](#) for more information.

Creating Multiple Instances of a Kernel

By default, the linker builds a single hardware instance from a kernel. If the host program will execute the same kernel multiple times, due to data processing requirements for instance, then it must execute the kernel on the hardware accelerator in a sequential manner. This can impact overall application performance. However, you can customize the kernel linking stage to instantiate multiple hardware compute units (CUs) from a single kernel. This can improve performance as the host program can now make multiple overlapping kernel calls, executing kernels concurrently by running separate compute units.

Multiple CUs of a kernel can be created by using the `connectivity.nk` option in the `v++` config file during linking. Edit a config file to include the needed options, and specify it in the `v++` command line with the `--config` option, as described in [v++ Command](#).

For example, for the `vadd` kernel, two hardware instances can be implemented in the config file as follows:

```
[connectivity]
#nk=<kernel name>:<number>:<cu_name>,<cu_name>...
nk=vadd:2
```

Where:

- `<kernel_name>`: Specifies the name of the kernel to instantiate multiple times.
- `<number>`: The number of kernel instances, or CUs, to implement in hardware.
- `<cu_name>,<cu_name>...`: Specifies the instance names for the specified number of instances. This is optional, and the CU name will default to `kernel_1` when it is not specified. The delimiter between kernel instances is a comma. In the example above, the `kernel_name` and the `number` of CUs are specified, but not the `cu_name`. In this case `vadd_1` and `vadd_2` will be added to the design.

Then the config file is specified on the `v++` command line:

```
v++ --config vadd_config.cfg ...
```



TIP: You can check the results by using the `xclbinutil` command to examine the contents of the `xclbin` file. Refer to [xclbinutil Utility](#).

The following example results in three CUs of the `vadd` kernel, named `vadd_X`, `vadd_Y`, and `vadd_Z` in the `xclbin` binary file:

```
[connectivity]
nk=vadd:3:vadd_X,vadd_Y,vadd_Z
```

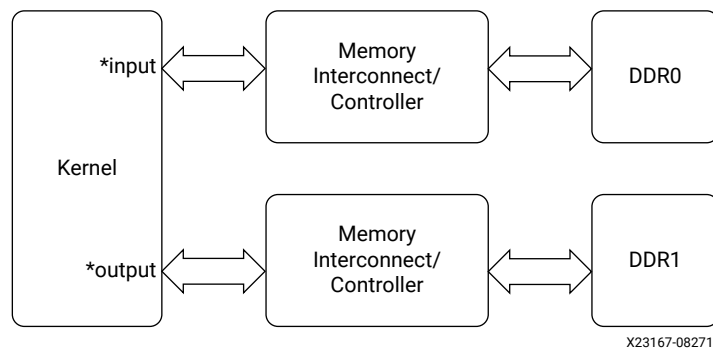
Mapping Kernel Ports to Memory

The link phase is when the memory ports of the kernels are connected to memory resources which include DDR, HBM, and PLRAM. By default, when the `xclbin` file is produced during the `v++` linking process, all kernel memory interfaces are connected to the same global memory bank (or `gmem`). As a result, only one kernel interface can transfer data to/from the memory bank at one time, limiting the performance of the application due to memory access.

While the Vitis compiler can automatically connect CU to global memory resources, you can also manually specify which global memory bank each kernel argument (or interface) is connected to. Proper configuration of kernel to memory connectivity is important to maximize bandwidth, optimize data transfers, and improve overall performance. Even if there is only one compute unit in the device, mapping its input and output arguments to different global memory banks can improve performance by enabling simultaneous accesses to input and output data.

The following block diagram shows the [Global Memory Two Banks](#) example in [Vitis Examples](#) on GitHub. This example connects the input pointer interface of the kernel to DDR bank 0, and the output pointer interface to DDR bank 1.

Figure 24: Global Memory Two Banks Example



IMPORTANT! Up to 15 separate kernel interfaces can be connected to a single global memory bank. Therefore, if there are more than 15 memory interfaces in the design you must explicitly perform the memory mapping as described here, using the `--connectivity.sp` option to distribute connections across different memory banks.

Start by assigning the kernel arguments to separate bundles to increase the available interface ports, then assign the arguments to separate memory banks. The following example uses the interfaces described at [HW Interfaces](#).

1. In the C/C++ kernel code assign arguments to separate bundles using the `INTERFACE` pragma prior to compiling them:

```
void cnn( int *pixel, // Input pixel
         int *weights, // Input Weight Matrix
         int *out, // Output pixel
         ... // Other input or Output ports

#pragma HLS INTERFACE m_axi port=pixel offset=slave bundle=gmem
#pragma HLS INTERFACE m_axi port=weights offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=out offset=slave bundle=gmem
```

The memory interface inputs `pixel` and `weights` are assigned different bundle names in the example above, while `out` is bundled with `pixel`. This creates two separate interface ports: `gmem` and `gmem1`.



IMPORTANT! You must specify `bundle=` names using all lowercase characters to be able to assign it to a specific memory bank using the `--connectivity.sp` option.

2. Use the `--connectivity.sp` option, or include it in a config file, as described in [--connectivity Options](#).

For example, for the `cnn` kernel shown above, the `connectivity.sp` option in the config file would be as follows:

```
[connectivity]
#sp=<compute_unit_name>.<argument>:<bank name>
sp=cnn_1.pixel:DDR[0]
sp=cnn_1.weights:DDR[1]
sp=cnn_1.out:DDR[0]
```

Where:

- `<compute_unit_name>` is an instance name of the CU as determined by the `connectivity.nk` option, described in [Creating Multiple Instances of a Kernel](#), or is simply `<kernel_name>_1` if multiple CUs are not specified.
- `<argument>` is the name of the kernel argument. Alternatively, you can specify the name of the kernel interface as defined by the HLS `INTERFACE` pragma for C/C++ kernels, including `m_axi_` and the bundle name. In the `cnn` kernel above, the ports would be `m_axi_gmem` and `m_axi_gmem1`.



TIP: For RTL kernels, the interface is specified by the interface name defined in the `kernel.xml` file.

- `<bank_name>` is denoted as `DDR[0]`, `DDR[1]`, `DDR[2]`, and `DDR[3]` for a platform with four DDR banks. You can also specify the memory as a contiguous range of banks, such as `DDR[0:2]`, in which case XRT will assign the memory bank at run time.

Some platforms also provide support for PLRAM, HBM, HP or MIG memory, in which case you would use `PLRAM[0]`, `HBM[0]`, `HP[0]` or `MIG[0]`. You can use the `platforminfo` utility to get information on the global memory banks available in a specified platform. Refer to [platforminfo Utility](#) for more information.

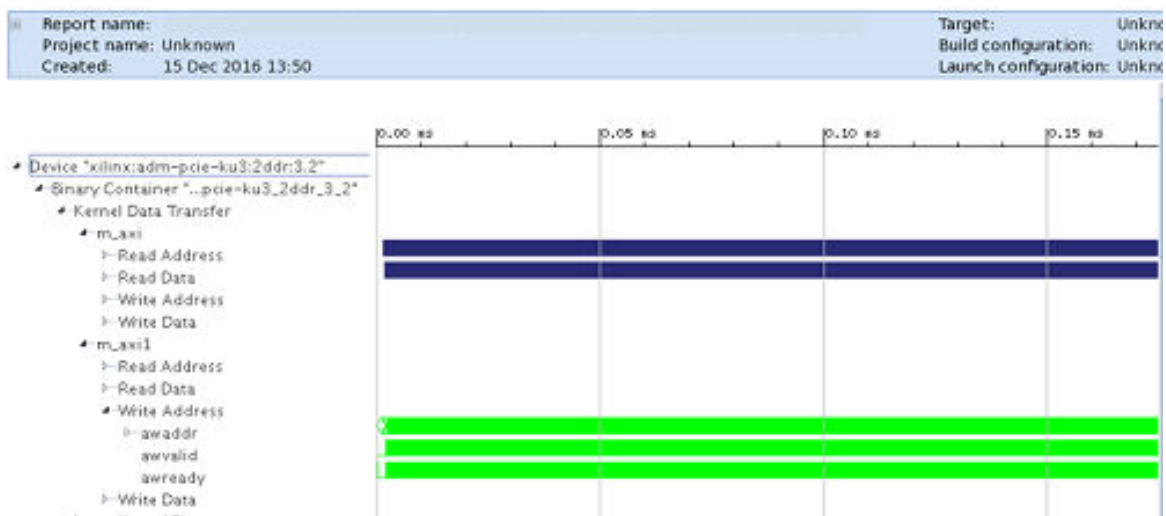
In platforms that include both DDR and HBM memory banks, kernels must use separate AXI interfaces to access the different memories. DDR and PLRAM access can be shared from a single port.



TIP: Assigning kernel interfaces to specific memory banks may also require you to specify the SLR to place the kernel into. For more information see [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#).

You can use the Device Hardware Transaction view in Vitis Analyzer to observe the actual DDR Bank communication, and to analyze DDR usage.

Figure 25: Device Hardware Transaction View Transactions on DDR Bank



Additional Memory Mapping Techniques for Alveo Accelerator Cards

Alveo accelerator cards support additional features such as host-memory access, HBM, or PLRAM utilization, depending on the specific card you are working with. The following sections describe some of these additional techniques.

Assigning AXI Interfaces to PLRAM

For platforms that support PLRAM use the `--connectivity.sp` option as previously described in [Mapping Kernel Ports to Memory](#), but use the name `PLRAM[id]` to specify the bank. Valid PLRAM banks supported by a platform can be found in the Memory Information section reported by the `platforminfo` command.

For example, in the Alveo U250 platform the following PLRAM information is reported:

```
Bus SP Tag: PLRAM
Segment Index: 0
Consumption: explicit
SLR:          SLR0
Max Masters: 60
```

```
Segment Index: 1
Consumption: explicit
SLR: SLR1
Max Masters: 60
Segment Index: 2
Consumption: explicit
SLR: SLR2
Max Masters: 60
Segment Index: 3
Consumption: explicit
SLR: SLR3
Max Masters: 60
```

To access the PLRAM you would use the following command for example:

```
--connectivity.sp cnn_1.weights:PLRAM3
```

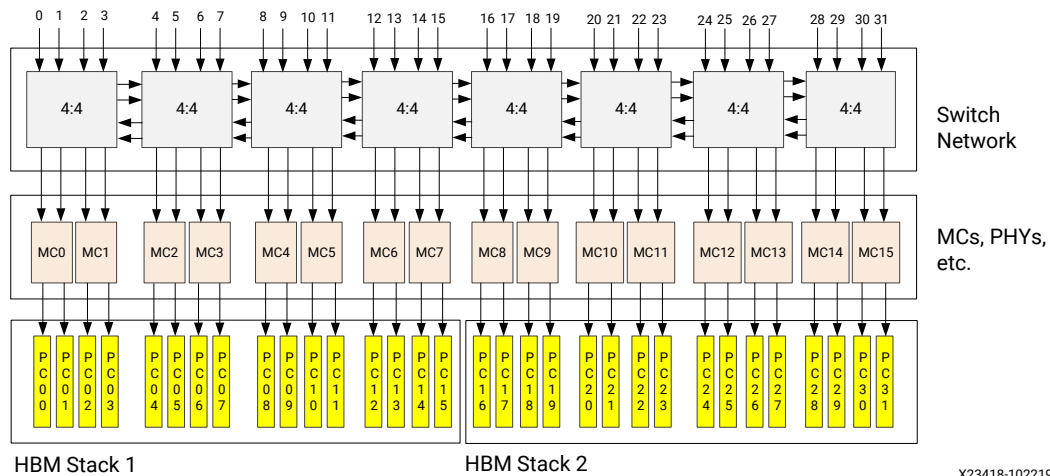


TIP: To reduce latency and improve performance the kernel accessing this PLRAM should be placed into SLR3 in addition to [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#).

HBM Configuration and Use

Some algorithms are memory bound, limited by the 77 Gb/s bandwidth available on DDR-based Alveo cards. For those applications there are High Bandwidth Memory (HBM) based Alveo cards, providing up to 460 Gb/s memory bandwidth. For the Alveo implementation, two 16-layer HBM (HBM2 specification) stacks are incorporated into the FPGA package and connected into the FPGA fabric with an interposer. A high-level diagram of the two HBM stacks is as follows.

Figure 26: High-Level Diagram of Two HBM Stacks



This implementation provides:

- 16 GB HBM using 512 MB pseudo channels (PCs) for Alveo U55C accelerator cards, as described in [Alveo U55C Data Center Accelerator Cards Data Sheet \(DS978\)](#)
- 8 GB total HBM using 256 MB PCs for Alveo U280 accelerator cards, and U50 cards as described in [Alveo U50 Data Center Accelerator Cards Data Sheet \(DS965\)](#)

- An independent AXI channel for communication between the Vitis kernels and the HBM PCs through a segmented crossbar switch network
- A two-channel memory controller for addressing two PCs
- 14.375 Gb/s max theoretical bandwidth per PC
- 460 Gb/s (32×14.375 Gb/s) max theoretical bandwidth for the HBM subsystem

Although each PC has a theoretical maximum performance of 14.375 Gb/s, this is less than the theoretical maximum of 19.25 Gb/s for a DDR channel. To get better than DDR performance, designs must efficiently integrate multiple AXI masters into the HBM subsystem. The programmable logic has 32 HBM AXI interfaces that can access any memory location in any of the PCs on either of the HBM stacks through a built-in switch network providing full access to the memory space.

Connection to the HBM is managed by the HBM Memory Subsystem (HMSS) IP, which enables all HBM PCs, and automatically connects the XDMA to the HBM for host access to global memory. When used with the Vitis compiler, the HMSS is automatically customized to activate only the necessary memory controllers and ports as specified by the `--connectivity.sp` option to connect both the user kernels and the XDMA to those memory controllers for optimal bandwidth and latency.

Note: Because of the complexity and flexibility of the built-in switch network, there are many implementations that result in congestion at a particular memory location or in the switch network. Interleaved read and write transactions cause a drop in efficiency with respect to read-only or write-only due to memory controller timing parameters (bus turnaround). Write transactions that span both HBM stacks will also experience degraded performance, and should be avoided. It is important to plan memory accesses so that kernels access limited memory where possible, and configure kernel connectivity to isolate the memory accesses for different kernels into different HBM PCs.

The `--connectivity.sp` option to connect kernels to HBM PCs is:

```
sp=<compute_unit_name>.<argument>:<HBM_PC>
```

In the following config file example, the kernel input ports `in1` and `in2` are connected to HBM PCs 0 and 1 respectively, and the output buffer `out` is connected to HBM PCs 3–4

```
[connectivity]
sp=krnl.in1:HBM[0]
sp=krnl.in2:HBM[1]
sp=krnl.out:HBM[3:4]
```

Each HBM PC is 256 MB, giving a total of 1 GB of memory access for this kernel. Refer to the [Using HBM Tutorial](#) for additional information and examples.



TIP: The config file specifies the mapping of a kernel argument to one or more HBM PCs. When mapping to multiple PCs each AXI interface should only access a contiguous subset of the available 32 HBM PCs. For example, `HBM[3:7]`.

When implementing the connection between the kernel argument and the specified PC, the HMSS automatically selects the appropriate channel in the switch network to connect the AXI Slave interface port to access memory, maximize bandwidth, and minimize latency given the pseudo channel number or range. However, the `--connectivity.sp` syntax for HBM also lets you specify which channel index for the switch network the HMSS should use for connecting the kernel interface. The `--connectivity.sp` syntax for specifying the switch network index is:

```
sp=<compute_unit_name>.<argument>:<bank_name>.<index>
```

When specifying the switch index, only one index can be specified per `sp` option. You cannot reuse an index which has already been used by another `sp` option or line in the config file.

In this case, the last line of the previous example could be rewritten as:

```
sp=krnl.out:HBM[3:4].3 to use switch channel 3 (S_AXI03) or  
sp=krnl.out:HBM[3:4].4 to use switch channel 4 (S_AXI04), as shown in the figure above.
```

Either `sp` option would route the kernel's data transactions through one of the left-most network switch blocks to reduce implementation complexity. Using any index in the range 0 to 7 would use one of the two left-most switch blocks in the network. Using any other index would force the use of additional switch blocks, which adds routing complexity and could negatively impact performance depending on the application.

The HBM ports are located in the bottom SLR of the device. The HMSS automatically handles the placement and timing complexities of AXI interfaces crossing super logic regions (SLR) in SSI technology devices. By default, without specifying the `--connectivity.sp` or `--connectivity.slr` options on `v++`, all kernel AXI interfaces access HBM[0] and all kernels are assigned to SLR0.

However, you can specify the SLR assignments of kernels using the `--connectivity.slr` option. For devices or platforms that use multiple SLRs, you are strongly recommended to define CU assignments to specific SLRs. Refer to [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#) for more information.

Random Access and the RAMA IP

HBM performs well in applications where sequential data access is required. However, for applications requiring random data access, performance can vary significantly depending on the application requirements (for example, the ratio of read and write operations, minimum transaction size, and size of the memory space being addressed). In these cases, the addition of the Random Access Memory Attachment (RAMA) IP to the target platform can significantly improve random memory access efficiency in cases where the required memory exceeds the 256 MB limit of a single HBM PC.

The RAMA IP will improve random access performance when two or more HBM PC are used. Refer to *RAMA LogiCORE IP Product Guide* ([PG310](#)) for more information.



TIP: To effectively use the RAMA IP in your application the kernel should access memory from multiple HBM PCs and should use a static single ID on the AXI transaction ID ports (AxID), or slowly changing (pseudo-static) AXI transaction IDs. If these conditions are not met, the thread creation used in the RAMA IP to improve performance has little effect, and consumes programmable logic resources for no purpose.

To use the RAMA IP add the keyword `RAMA` to the `sp` option in the config file, with the following format.

Note: The `--connectivity.sp` option requires the use of the `<index>` as described in the prior section.

```
sp=<compute_unit_name>.<argument>:<bank_name>.<index>.RAMA
```

For example:

```
sp=krnl.out:HBM[3:4].3.RAMA
```

Directly Accessing Host Memory on Alveo Accelerator Cards

The PCIe® Slave-Bridge IP is provided on some data center platforms to let kernels access directly to host memory. Only certain Alveo cards support the host memory connection, as reported in the Memory Information section returned by the `platforminfo` command.

For example, the following information is returned for the Alveo U250 card:

```
Name: HOST[0]
Index: 8
Type: MEM_DRAM
Base Address: 0x2000000000
Address Size: 0x400000000
Bank Used: No
```

Configuring the device binary to connect directly to host memory uses the `--connectivity.sp` command below.

```
[connectivity]
## Syntax
##sp=<cu_name>.<interface_name>:HOST[0]
sp=cnn_1.weights:HOST[0]
```



IMPORTANT! Using host memory also requires changes to the accelerator card setup and your host application as described at [Host-Memory Access](#) in the XRT documentation.

Specifying Streaming Connections

Support for hardware accelerator pipelines that communicate through streams is one of the major advantages of FPGAs, FPGA-based SoCs, and Versal adaptive SoC devices; it can be used in DSP and image processing applications, in addition to communication systems. Kernel ports involved in streaming are defined within the kernel, and are not addressed by the host program. There is no need to send data back to global memory before it is forwarded to another kernel for processing. The connections between the kernels are directly defined during the `v++` linking process as described below.

A streaming data output port of one kernel can be connected to the streaming data input port of another kernel, or between a PL kernel and the PLIO of an ADF graph application, during linking using the `--connectivity.sc` option. This option can be specified at the command line, or from a `config` file that is specified using the `--config` option, as described in [v++ Command](#).



IMPORTANT! An error occurs if the `--connectivity.sc` kernel drives itself.

To connect the streaming output port of a producer kernel to the streaming input port of a consumer kernel, set up the connection in the `v++` config file using the `connectivity.stream_connect` option as follows:

```
[connectivity]
#stream_connect=<cu_name>.<output_port>:<cu_name>.<input_port>:
[<fifo_depth>]
stream_connect=vadd_1.stream_out:vadd_2.stream_in
stream_connect=vadd_2.stream_in:ai_engine_0.DataIn0
```

Where:

- `<cu_name>` is an instance name of the CU as determined by the `connectivity.nk` option, described in [Creating Multiple Instances of a Kernel](#). The `cu_name` can be specified in the config file as described in [Creating Multiple Instances of a Kernel](#), or is defined automatically by the tool when not otherwise specified.



TIP: When specifying connections to Versal AI Engine ports, the `<cu_name>` is `ai_engine_0` as shown above.

- `<output_port>` or `<input_port>` is the streaming port defined in the producer or consumer kernel.



IMPORTANT! If the port-width of the output and input ports do not match, the Vitis compiler automatically inserts a data-width converter between the two ports as part of the build process. The inclusion of the data-width converter is either truncate a larger bit-width output to a smaller bit-width input, or expand a smaller bit-width to a larger bit-width.

- `[:<fifo_depth>]` inserts a FIFO of the specified depth between the two streaming ports to prevent stalls. The value is specified as an integer.

Assigning Compute Units to SLRs on Alveo Accelerator Cards

Alveo Data Center accelerator cards use stacked silicon devices consisting of multiple Super Logic Regions (SLR) to provide device resources, including global memory. Kernel compute unit (CU) instance and DDR memory resource floorplanning are keys to meeting quality of results of your design in terms of frequency and resources. Floorplanning involves explicitly allocating CUs (a kernel instance) to SLRs. For best performance, you should assign kernels or CUs to specific SLRs to improve placement and timing results. SLR assignment is especially important when assigning kernel ports to specific memory banks as described in [Mapping Kernel Ports to Memory](#).

Specific availability of SLR in an Alveo accelerator card can be determined with the `platforminfo` command. For instance, the U250 card reports the following information with regard to SLRs:

```
Valid SLRs
:
SLR0, SLR1, SLR2, SLR3
```

You can use the actual kernel resource usage values to help distribute CUs across SLRs to reduce congestion in any one SLR. The system estimate report lists the number of resources (LUTs, Flip-Flops, BRAMs, etc.) used by the kernels early in the design cycle. Use this information along with the available SLR resources to help assign CUs to SLRs such that no one SLR is over-utilized.



IMPORTANT! If your kernel is too large to fit into a single SLR, the Vitis compiler automatically places the logic across multiple SLRs. In this case you should not assign the SLR or this could result in an error during implementation.

A CU can be assigned to an SLR using the `connectivity.slr` option in a config file. The syntax of the `connectivity.slr` option in the config file is as follows:

```
[connectivity]
#slr=<compute_unit_name>:<slr_ID>
slr=vadd_1:SLR2
slr=vadd_2:SLR3
```

Where:

- `<compute_unit_name>` is an instance name of the CU as determined by the `connectivity.nk` option, described in [Creating Multiple Instances of a Kernel](#), or is simply `<kernel_name>_1` if multiple CUs are not specified.
- `<slr_ID>` is the SLR number to which the CU is assigned, in the form SLR0, SLR1,...

AMD recommends assigning a kernel to a DDR memory resource in the same SLR as the kernel is placed. This reduces competition for limited SLR-crossing connection resources, and the use of super long line (SLL) routing resources which incur a greater delay than a standard routing. It might be necessary to connect a kernel to a DDR resource in a different SLR. However, if both the `connectivity.sp` and the `connectivity.slr` directives are explicitly defined, the tool automatically adds additional crossing logic to minimize the effect of the SLL delay, and facilitates better timing closure.



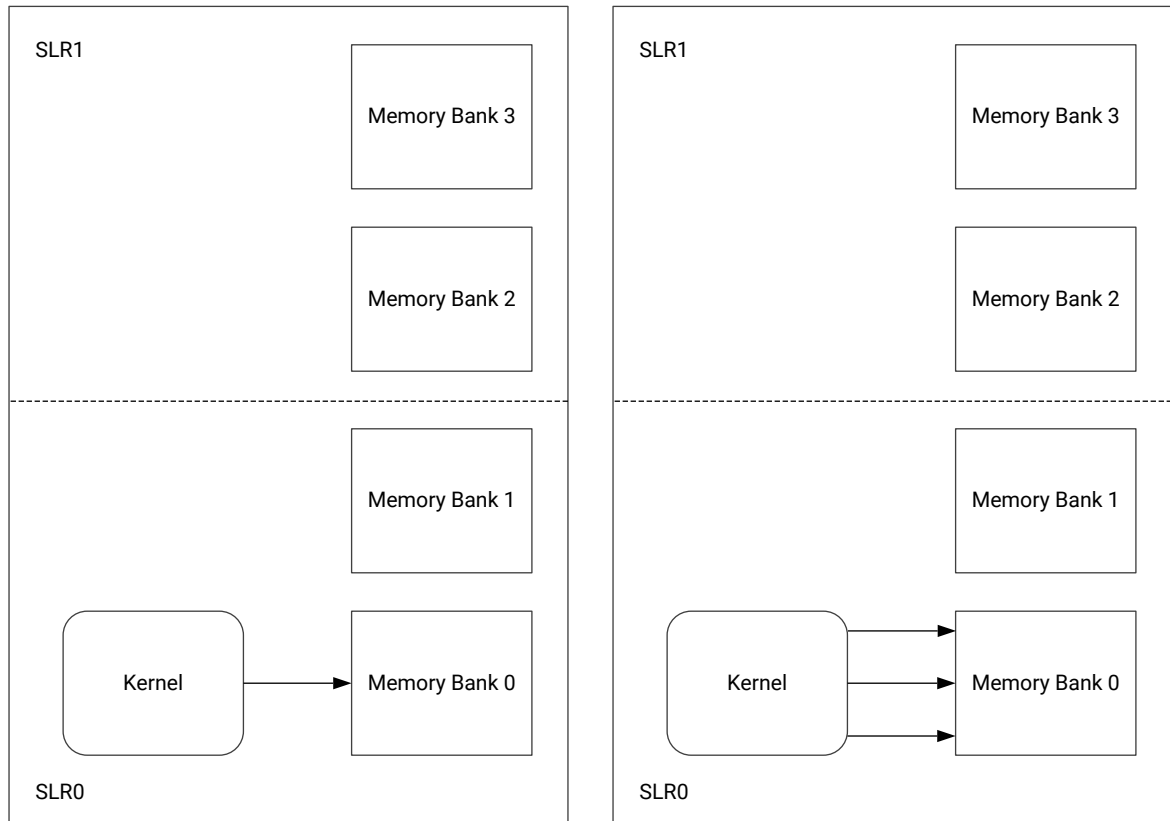
IMPORTANT! When building the hardware and specifying trace memory for the profile command, as described in [--profile Options](#), you should also be aware of CU placement and assign memory resources according to SLR use. This can improve timing by limiting and managing SLR crossings. The command syntax is as follows:

```
--profile.trace_memory <memory>:<SLR>
```

SLR Guidelines for Kernels that Access Multiple Memory Banks

The DDR memory resources are distributed across the super logic regions (SLRs) of the platform. Because the number of connections available for crossing between SLRs is limited, the general guidance is to place a kernel in the same SLR as the DDR memory resource with which it has the most connections. This reduces competition for SLR-crossing connections and avoids consuming extra logic resources associated with SLR crossing.

Figure 27: Kernel and Memory in Same SLR



X22194-010919

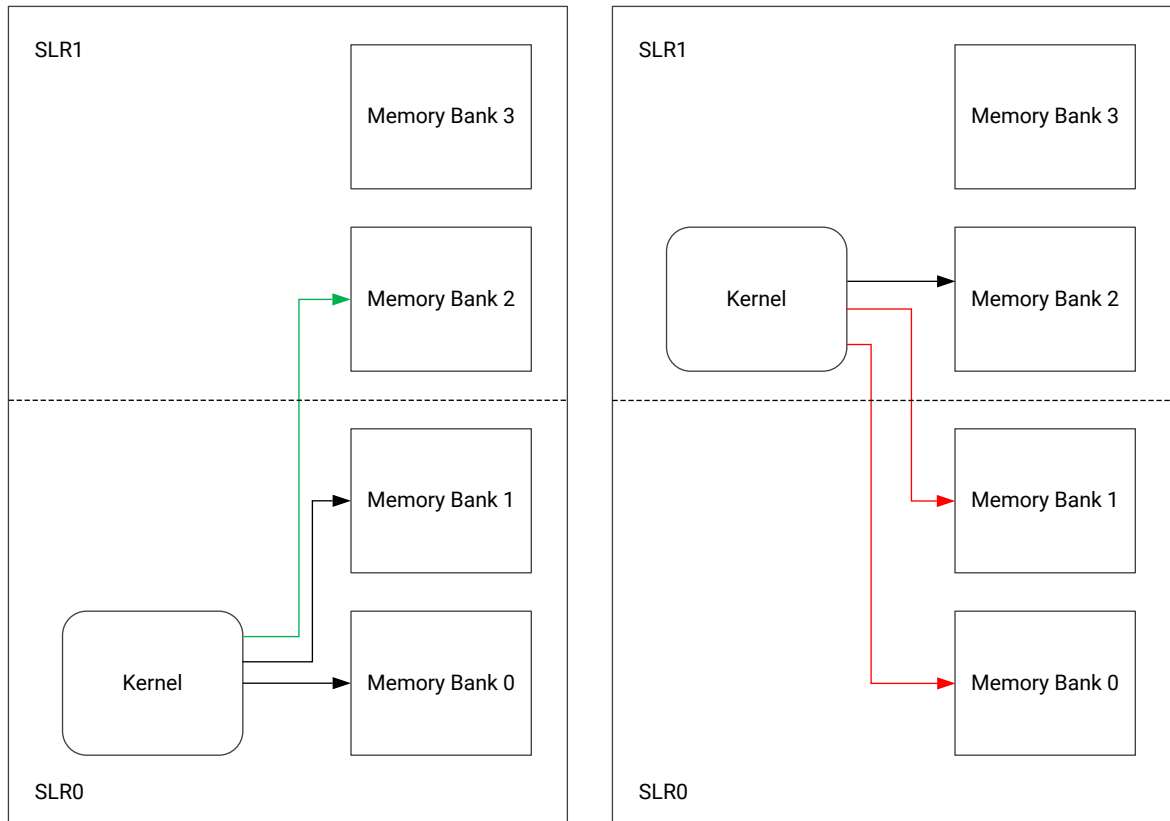
Note: The image on the left shows a single AXI interface mapped to a single memory bank. The image on the right shows multiple AXI interfaces mapped to the same memory bank.

As shown in the previous figure, when a kernel has a single AXI interface that maps only a single memory bank, the `platforminfo` utility described in [platforminfo Utility](#) lists the SLR that is associated with the memory bank of the kernel; therefore, the SLR where the kernel would be best placed. In this scenario, the design tools might automatically place the kernel in that SLR without need for extra input; however, you might need to provide an explicit SLR assignment for some of the kernels under the following conditions:

- If the design contains a large number of kernels accessing the same memory bank.
- A kernel requires some specialized logic resources that are not available in the SLR of the memory bank.

When a kernel has multiple AXI interfaces and all of the interfaces of the kernel access the same memory bank, it can be treated in a very similar way to the kernel with a single AXI interface, and the kernel should reside in the same SLR as the memory bank that its AXI interfaces are mapping.

Figure 28: Memory Bank in Adjoining SLR



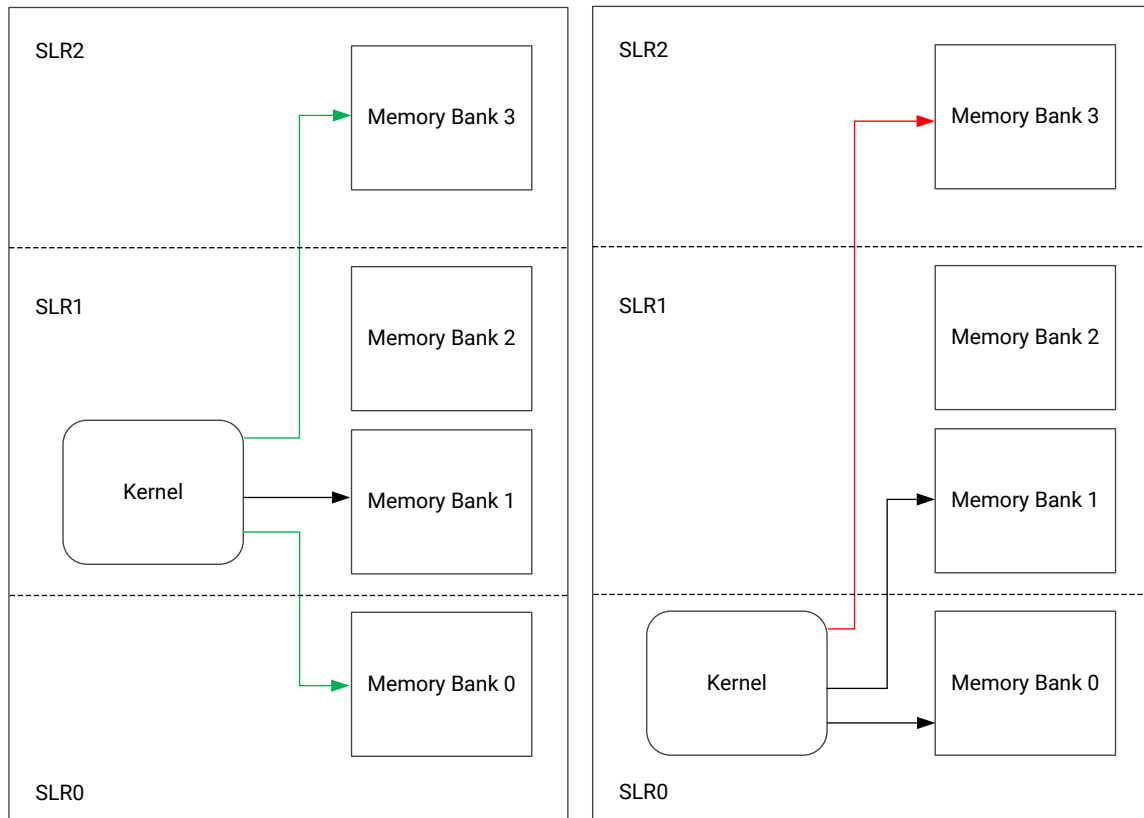
X22195-010919

Note: The image on the left shows one SLR crossing is required when the kernel is placed in SLR0. The image on the right shows two SLR crossings are required for kernel to access memory banks.

When a kernel has multiple AXI interfaces to multiple memory banks in different SLRs, the recommendation is to place the kernel in the SLR that has the majority of the memory banks accessed by the kernel (shown in the figure above). This minimizes the number of SLR crossings required by this kernel which leaves more SLR crossing resources available for other kernels in your design to reach your memory banks.

When the kernel is mapping memory banks from different SLRs, explicitly specify the SLR assignment as described in [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#).

Figure 29: Memory Banks Two SLRs Away



X22196-010919

Note: The image on the left shows two SLR crossings are required to access all of the mapped memory banks. The image on the right shows three SLR crossings are required to access all of the mapped memory banks.

As shown in the previous figure, when a platform contains more than two SLRs, it is possible that the kernel might map a memory bank that is not in the immediately adjacent SLR to its most commonly mapped memory bank. When this scenario arises, memory accesses to the distant memory bank must cross more than one SLR boundary and incur additional SLR-crossing resource costs. To avoid such costs it might be better to place the kernel in an intermediate SLR where it only requires less expensive crossings into the adjacent SLRs.

Managing Clock Frequencies

The Vitis tools let you create designs using the default clocks defined in a platform, or software scalable clocks, or fixed-frequency clocks generated from MMCMs in the hardware design. This solution is engineered to meet broad design requirements for multi-clocking of and between PL kernels, AI Engine graphs, and platform IP like GTs.

As described in [Identifying Platform Clocks](#), platforms include scalable clocks that can be used at the default frequency, or that XRT can scale to a different specified frequency, or that can also be scaled to meet timing if needed. Platforms also include fixed clocks, that can be used at the default frequency, or that can have additional frequencies generated using MMCMs to divide or multiply the default frequency.

Clock frequencies can be specified using the `--freqhz` option for AI Engines, for PL kernels, or for linking the System project. When you specify the `--freqhz` option, the `v++` linker examines the frequency relative to the predefined platform clocks, and chooses a clock to use. If the frequency matches one of the predefined clocks, the tool uses that clock. If the frequency is close to a platform clock frequency, the platform clock will be selected, but if there is no clock close to the desired frequency, a clock will be generated by inserting a clock wizard into the design.

During the system linking process you can connect multiple kernels to the platform using different clock frequencies specifying multiple `--freqhz` options. Each kernel, or unique instance (CU) of the kernel can connect to a specified clock frequency as shown for the `--freqhz` option syntax in [v++ General Options](#).

```
v++ -l -t hw -platform <pfm_name> --freqhz=200000000:mm2s \
--freqhz=200000000:s2mm -config system.cfg
```



TIP: You can view the `automation_summary_pre_synthesis.txt` produced during the `v++ --link` command to verify the clock assignments for AI Engine kernels, and PL kernels in the system, as shown in the following figure.

5. Clock Summary =====

AIE

Instance	Kernel	PLIO	Annotated Arg	Clock Port	Compile (MHz)	Link (MHz)
ai_engine_0	ai_engine	M00_AXIS	DataOut1	aclk0	200.00	208.33
ai_engine_0	ai_engine	S00_AXIS	DataIn1	aclk0	200.00	208.33
ai_engine_0	ai_engine	M01_AXIS	clip_in	aclk1	200.00	104.17
ai_engine_0	ai_engine	S01_AXIS	clip_out	aclk1	200.00	104.17

PL

Instance	Kernel	Clock Port	Compile (MHz)	Link (MHz)
mm2s	mm2s	ap_clk	149.99	208.33
s2mm	s2mm	ap_clk	149.99	208.33
polar_clip	polar_clip	ap_clk	200.00	104.17

In some cases, the clock frequency can only be achieved in some approximation to the specified `--freqhz`. In these cases you can specify the `--clock.default_tolerance` to indicate an acceptable range for the frequency. If the specified clock frequency can not be met within the acceptable tolerance, a warning is issued and the closest default clock is used.

You can refer to the `automation_summary_pre_synthesis.txt` report to identify the clock frequency used for compilation and linking. This report is generated at pre-synthesis step during linking, so you do not have to wait for the process to finish. In compilation, the specified clock frequency is used, but for linking the clock frequency is determined by the specified value, the clock tolerance, and the available clocks.

Platform clocks also have clock indexes which identify a specific clock ID, rather than simply the clock frequency. A clock ID defines a network path, including startpoints and endpoints in the platform design. To specify clocks in your designs you should generally use the `--freqhz` option, instead of clock ID. Specify the clock ID (`--clock.ID`) only when you want a common clock network for all IPs inserted by `v++` along the path to this kernel.

Using `--freqhz` for Clock Management

In the common command-line flow the Vitis compiler supports a single approach to clock management using the `v++ --freqhz` option. It is supported for AI Engine graph compilation, HLS component synthesis, and System project linking.

The process for managing clock frequencies in AI Engine components, HLS components, or System projects includes the following:

- **Default values:**
 - `--part`: The device doesn't contain a default clock, so the default depends upon whether you are targeting HLS, AI Engine, or system linking as described below.
 - `--platform`: A platform specifies a default clock, which is passed to the `v++` option. You can override the default platform clock by using the `--freqhz` option.
- **`v++ -c --mode aie`:**
 - `--part`: The default clock is $\frac{1}{4}$ of the AI Engine PLL frequency.
 - `--platform`: The default clock is derived from the platform, and can be overridden using the `--freqhz` option.

```
v++ -c --mode aie --platform <pfm_name> --freqhz=200000000.. aie/
graph.cpp
```

- **`v++ -c --mode hls`:**
 - `--part`: The default clock is derived from `--hls.clock=10ns`.
 - `--platform`: The default clock is derived from the platform, and can be overridden using the `--freqhz` option. Specifying a different `--hls.clock` frequency will result in an error during the `v++ --mode hls` option.

```
v++ -c --mode hls -platform <pfm_name> --freqhz=150000000 -config hls.cfg
```

- **`v++ -c -k`:**

- `--part`: This is unsupported for `v++ -c -k`. The default clock is defined by the platform.
- `--platform`: The default clock is derived from the platform, and can be overridden using the `--freqhz` option.

```
v++ -c -k vadd -platform <pfm_name> --freqhz=150000000
```

- **v++ --link:**

- `--part`: For Versal devices, the `--part` option specifies a default value of 300 MHz. For other devices, `--part` is unsupported. A default clock is defined by the platform.
- `--platform`: The default clock is derived from the platform, and can be overridden using the `--freqhz` option.

```
v++ -l -t hw -platform <pfm_name> --freqhz=200000000:mm2s \
--freqhz=200000000:s2mm -config system.cfg
```

Identifying Platform Clocks

The handling of clocks in platforms has evolved to support multiple platform clocks and clock frequencies. The kernels can have any number of independent and edge-aligned clocks, and platforms can have multiple kernels running at different clock frequencies under user-control. Platforms have a variety of clocking: processor, PL, and AI Engine clocking. The following table explains the clocking for each.

Table 19: Platform Clocks

Clock	Description
AI Engine	Can be configured in the platform in the AI Engine IP.
Processor	Can be configured in the platform in the CIPS IP.
Programmable Logic (PL)	Can have multiple clocks and can be configured in the platform.
NoC	Device dependent and can be configured in the platform in the CIPS and NoC IP.

Notes:

1. These clocks are derived from the platform and are affected by the device, speed grade, and operating voltage.

A platform can have scalable clocks and fixed clocks.

- **Scalable Clocks:**

An AMD Alveo™ platform provides a frequency scalable kernel clock with ID (or Index) 0 that drives all XRT managed kernels. XRT can set the clock frequency of this clock according to the metadata contained in the `xclbin` file, when loading the file. An Alveo platform also provides a second scalable clock with ID 1 that is also controllable based on `xclbin` metadata. You do not need to provide an option to connect to scalable clocks, but you can specify the clock frequency during `v++` linking using the `--freqhz` option or `--kernel-frequency`, as described in [v++ General Options](#). The `v++` linker automatically connects PL kernel clocks `ap_clk` to clock ID 0, and `ap_clk2` to clock ID 1.



TIP: In practice, `ap_clk2` is primarily found on RTL kernels, because the HLS compiler does not generate kernels with multiple clocks.

- **Fixed Clocks:**

Fixed clocks can be found on both AMD Alveo™ platforms and embedded platforms using Versal devices or Zynq MPSoC. Fixed clocks are also typically used in RTL kernels.

Scalable clocks are scaled by XRT to meet the clock specified in the `v++` command and the timing requirements of the design. Fixed clocks use MMCMs added to the design to generate frequencies other than the fixed frequencies defined on the platform. For example, if you specify clock frequencies: 60, 200, and 450, the Vitis compiler adds all the necessary logic to generate the required clocks from the available fixed clocks.

Use the `--freqhz` option to specify the clock frequency for the design. The [--clock Options](#) can be also used to specify PL kernel connections to specific platform clocks, or to specify clock frequencies that are generated from fixed clocks on the platform.

Specifying the `--clock` options directs `v++` to use the fixed clocks of a platform, rather than the scalable clocks. For HLS kernels, the clock ID 0 is always used. For RTL kernels with one clock, clock ID 0 is used by default, but you can select a different clock.



TIP: You cannot mix fixed and scalable clocks on a single kernel, but these can be mixed across different kernels within a single `.xclbin` file.

If Vivado place and route tools are unable to meet the frequency specification, the tools can scale the clock frequency to an achievable frequency if the scalable clock has been used (`--kernel-frequency`). However, if a fixed clock is used (`--clock`), then you need to re-run the implementation to update the frequency target.

You can determine the clocks available in the target platform by using the `platforminfo` command. For example, the following command returns verbose information related to the specified platform and writes it to an output file:

```
platforminfo -v -p xilinx_u250_gen3x16_xdma_4_1_202210_1 -o pfmClocks.txt
```

The *Clock Information* is reported in the output file as shown below, indicating the scalable and fixed clocks in the platform:

```
=====
Clock Information
=====
Default Clock Index: 0
Default Clock Frequency: 300.000000
Default Clock Pretty Name: PL 0
Clock Index: 0
  Frequency: 300.000000
  Name: ss_ucs_aclk_kernel_00
  Pretty Name: PL 0
  Inst Ref: ss_ucs
  Comp Ref: shell_ucs_subsystem
  Period: 3.333333
  Normalized Period: .003333
  Status: scalable
Clock Index: 1
  Frequency: 500.000000
  Name: ss_ucs_aclk_kernel_01
  Pretty Name: PL 1
  Inst Ref: ss_ucs
  Comp Ref: shell_ucs_subsystem
  Period: 2.000000
  Normalized Period: .002000
  Status: scalable
Clock Index: 2
  Frequency: 50.000000
  Name: ii_level1_wire_ulp_m_aclk_ctrl_00
  Pretty Name: PL 2
  Inst Ref: ii_level1_wire
  Comp Ref: ii_level1_wire
  Period: 20.000000
  Normalized Period: .020000
  Status: fixed
Clock Index: 3
...
```

There are a couple of high-level considerations in practice. Scalable clocks help you to achieve timing without having to regenerate the `.xclbin` file, by allowing the tool to scale the clock as needed to achieve a specific frequency or to meet timing. However, scaling does not increase a scaled clock frequency above the platform frequency. Designs requiring different kernels to run at different frequencies to close timing or to meet performance targets can make use of fixed clocks. For most serious designs, the expectation is that you will at some point know what the achievable frequency targets are for the design, and will need to specify the fixed clocks when they exceed the scalable clock frequencies.

For Versal devices, the `--part` option can be used instead of `--platform` with the `v++ --link` and `v++ --package` commands. With `--part` the tool generates a base design for use on the device, and can generally be used while waiting for the development of a full platform specification. However, the `v++` linker generated base platform design employs PLRAM only, so is unsuitable for running Linux applications on the target.

Syncing Clocks with the AI Engine

PLIO represents an ADF graph interface to a PL component. This component could be a PL kernel, a platform IP representing a signal source or sink, or it could be a data mover to interface the ADF graph to memory. You should provide clock frequency values for these interfaces to ensure simulation results match the results from running the design in hardware. In addition, when you link the ADF graph into the platform using the `v++ -link` command, you can direct the tools to generate precisely the clock frequencies required by your application. PL kernels can be independently clocked, and the `v++` linker will automatically insert clock domain crossing circuitry into the design as needed.

The recommended best practice is to use the `--freqhz` option to specify all clocks as follows:

1. When all PLIOs will be clocked at the same frequency, use `v++ -c --mode aie --freqhz` to specify. When different PLIOs will run at different clock frequencies, specify frequencies in the ADF graph PLIO constructors. If the PLIO clock frequencies are not known at AI Engine compilation time, they can be specified at `v++ link` time.

```
v++ -c --mode aie --freqhz <frequency>
```

2. Provide the same frequency to the Vitis compilation of the attached PL kernel:

```
v++ -c --mode hls -freqhz <frequency>
```

3. Provide the same frequency to the Vitis linker (`v++ -l`) when linking the AI Engine graph to the PL kernels and the platform in the System project. At `v++ link` time, individual kernel clocks can also be specified with the `--clock.freqhz` directive.

```
v++ -l --platform <pfm_name> --freqhz <frequency>
```

Controlling Report Generation

The `v++ -R` option (or `--report_level`) controls the level of information to report during compilation or linking for hardware emulation and system targets. Builds that generate fewer reports will typically run more quickly.

The command line option is as follows:

```
$ v++ -R <report_level>
```

Where `<report_level>` is one of the following options:

- `-R0`: Minimal reports and no intermediate design checkpoints (DCP).
- `-R1`: Includes `R0` reports plus:
 - Identifies design characteristics to review for each kernel (`report_failfast`).
 - Identifies design characteristics to review for the full post-optimization design.

- Saves post-optimization design checkpoint (DCP) file for later examination or use in the Vivado Design Suite.



TIP: *report_failfast* is a utility that highlights potential device usage challenges, clock constraint problems, and potential unreachable target frequency (MHz).

- -R2: Includes R1 reports plus:
 - Includes all standard reports from the Vivado tools, including saved DCPs after each implementation step.
 - Design characteristics to review for each SLR after placement.
- -Restimate: Forces Vitis HLS to generate a System Estimate report, as described in [System Estimate Report](#).



TIP: This option is useful for the software emulation build (`-t sw_emu`).

Managing Vivado Synthesis, Implementation, and Timing Closure

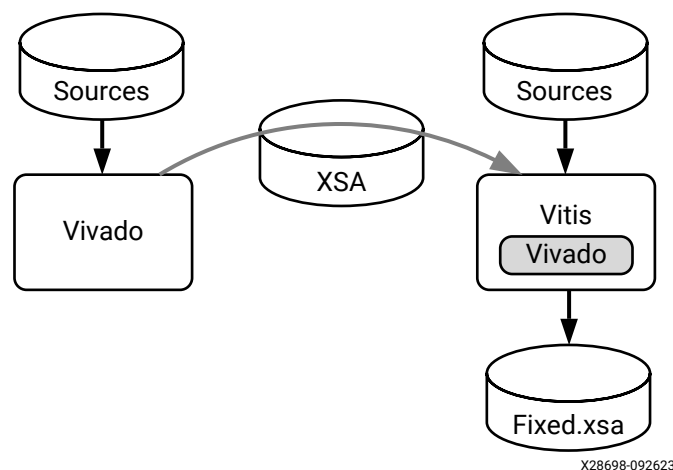


TIP: This topic requires an understanding of the Vivado Design Suite tools and design methodology as described in *UltraFast Design Methodology Guide for FPGAs and SoCs* (UG949), or the *Versal Adaptive SoC Design Guide* (UG1273).

As explained before, the Vitis linking process supports two different flows for implementing the linked hardware design: the Vitis Integrated Flow, and the Vitis Export to Vivado flow. Both of the flows use the Vivado Design Suite for synthesis and implementation of the linked system design. The way the Vivado tools are used is different between the two flows.

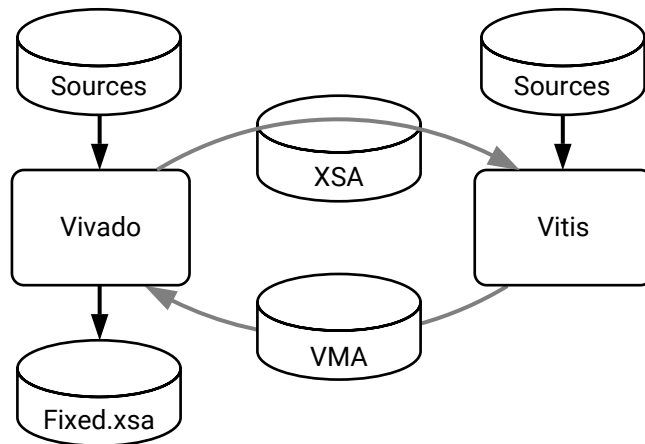
The Vitis Integrated Flow is shown in the following figure. An extensible hardware platform (.xsa) is built using the Vivado Design Suite and passed to the Vitis tool, where the platform is used for developing the AI Engine graph application and additional PL kernels for the system design. In the Vitis Integrated Flow, the system design is automatically synthesized and implemented in the Vivado Design Suite in the Vitis Integrated Flow during the v++ linking phase.

Figure 30: Vitis Integrated Flow



The Vitis Export to Vivado flow performs standard linking design modifications, but stops before running Vivado synthesis and instead encapsulates all relevant design data into a Vitis Metadata Archive (.vma) file to be imported back into the Vivado tools, as shown in the following figure. In this flow the .xsa file is passed to the Vitis design team for developing the AI Engine graph application and PL kernels for the system design, and the .vma file is returned to the Vivado tools, leaving simulation, synthesis and implementation to be manually completed by the user.

Figure 31: Vitis Export to Vivado Flow



X28697-092623

Working with Vivado in the Vitis Integrated Flow

The Vitis Integrated Flow (also known as the Traditional Platform Development Flow) automatically launches the Vivado Design Suite to synthesize the linked system design, place and route the elements of the design, resolve timing, and generate the bitstream or PDI for the design. In most cases, the Vitis tools completely abstract away the underlying process of synthesis and implementation of the hardware design. This removes the application developer from the typical hardware development process, and the need to manage constraints such as logic placement and routing delays. The Vitis tool automates much of the FPGA implementation process.

While automated, this flow does offer some opportunity for manual intervention. The process is broken down into a series of major steps, that can be interrupted to enable customization when necessary. In some cases you might want to exercise some control over the synthesis and implementation processes deployed by the Vitis compiler, especially when large designs are being implemented. Towards this end, the Vitis tool offers some control through specific options that can be specified in a `v++` configuration file, or from the command line. The following sections describe some of the methods you can use to control the Vivado synthesis and implementation results.

- Using the `--vivado` options to manage the Vivado tool.
- Using multiple implementation strategies to achieve timing closure on challenging designs.
- Using the `-to_step` and `-from_step` options to run the compilation or linking process to a specific step, perform some manual intervention on the design, and resume from that step.
- Interactively editing the Vivado project, and using the results for generating the FPGA binary.

Using the `--vivado` and `--advanced` Options

Using the `--vivado` option, as described in [--vivado Options](#), and the `--advanced` option as described in [--advanced Options](#), you can perform a number of interventions on the standard Vivado synthesis or implementation.

1. You can specify the strategy to be used by the Vivado tool during synthesis, implementation, or when generating reports during the build process. The strategy specified can be one of the standard tool strategies, or can be a custom-defined strategy that you have previously created in the Vivado tool. Use the `--vivado.prop` command as shown below.

The original Tcl command to set the property on a run object looks like the following:

```
set_property strategy Flow_AreaOptimized_medium [get_runs synth_1]
```

The `v++` command is rewritten as shown below:

- Synthesis Strategy:

```
--vivado.prop run.synth_1.strategy=Flow_AreaOptimized_medium
```

- Implementation Strategy:

```
--vivado.prop run.impl_1.strategy=Performance_ExtraTimingOpt
```

- Report Strategy: Can be specified for synthesis or implementation runs.

```
--vivado.prop run.synth_1.report_strategy=MyCustom_Reports
```

```
--vivado.prop run.impl_1.report_strategy={Timing Closure Reports}
```

The command line is broken down as follows:

- `--vivado.prop` is the `v++` command-line option to assign properties to objects as described in [--vivado Options](#).
- `run.<run_name>.strategy=<strategy_name>` to assign a `strategy` property (or `report_strategy`) to the specified synthesis or implementation run. Default run names are `synth_1` for synthesis or `impl_1` for implementation.
- Strategy names with spaces in them, such as `{Timing Closure Reports}` require braces or double-quotes to group the words as shown above.



TIP: You can also specify multiple implementation strategies to run as described in [Running Multiple Implementation Strategies for Timing Closure](#).

2. Pass Tcl scripts with custom design constraints or scripted operations.

You can create Tcl scripts to assign XDC design constraints to objects in the design, and pass these Tcl scripts to the Vivado tools using the PRE and POST Tcl script properties of the synthesis and implementation steps. For more information on Tcl scripting, refer to the *Vivado Design Suite User Guide: Using Tcl Scripting* (UG894). While there is only one synthesis step, there are a number of implementation steps as described in the *Vivado Design Suite User Guide: Implementation* (UG904). You can assign Tcl scripts for the Vivado tool to run before the step (PRE), or after the step (POST). The specific steps you can assign Tcl scripts to include the following: `SYNTH_DESIGN`, `INIT_DESIGN`, `OPT_DESIGN`, `PLACE_DESIGN`, `ROUTE_DESIGN`, `WRITE_BITSTREAM`.



TIP: There are also some optional steps that can be enabled using the `--vivado.prop run.impl_1.steps.phys_opt_design.is_enabled=1` option. When enabled, these steps can also have Tcl PRE and POST scripts.

An example of the Tcl PRE and POST script assignments follow:

```
--vivado.prop run.impl_1.STEPS.PLACE_DESIGN.TCL.PRE=/.../xxx.tcl
```

In the preceding example a script has been assigned to run before the `PLACE_DESIGN` step. The command line is broken down as follows:

- `--vivado` is the `v++` command-line option to specify directives for the Vivado tools.
- `prop` keyword to indicate you are passing a property setting.
- `run.` keyword to indicate that you are passing a run property.
- `impl_1.` indicates the name of the run.
- `STEPS.PLACE_DESIGN.TCL.PRE` indicates the run property you are specifying.
- `/.../xx.tcl` indicates the property value.



TIP: Both the `--advanced` and `--vivado` options can be specified on the `v++` command line, or in a configuration file specified by the `--config` option. The example above shows the command line use, and the following example shows the config file usage. Refer to [Vitis Compiler Configuration File](#) for more information.

3. Setting properties on run, file, and fileset design objects.

This is very similar to passing Tcl scripts as described above, but in this case you are passing values to different properties on multiple design objects. For example, to use a specific implementation strategy such as `Performance_Explore` and disable global buffer insertion during placement, you can define the properties as shown below:

```
[vivado]
prop=run.impl_1.STEPS.OPT_DESIGN.ARGS.DIRECTIVE=Explore
prop=run.impl_1.STEPS.PLACE_DESIGN.ARGS.DIRECTIVE=Explore
prop=run.impl_1.{STEPS.PLACE_DESIGN.ARGS.MORE_OPTIONS}={-no_bufg_opt}
prop=run.impl_1.STEPS.PHYS_OPT_DESIGN.IS_ENABLED=true
prop=run.impl_1.STEPS.PHYS_OPT_DESIGN.ARGS.DIRECTIVE=Explore
prop=run.impl_1.STEPS.ROUTE_DESIGN.ARGS.DIRECTIVE=Explore
```

In the example above, the `Explore` value is assigned to the `STEPS.XXX.DIRECTIVE` property of various steps of the implementation run. Note the syntax for defining these properties is:

```
<object>.<instance>.property=<value>
```

Where:

- `<object>` can be a design run, a file, or a fileset object.
- `<instance>` indicates a specific instance of the object.
- `<property>` specifies the property to assign.
- `<value>` defines the value of the property.

In this example the object is a run, the instance is the default implementation run, `impl_1`, and the property is an argument of the different step names. In this case the `DIRECTIVE`, `IS_ENABLED`, and `{MORE_OPTIONS}`. Refer to [--vivado Options](#) for more information on the command syntax.

4. Enabling optional steps in the Vivado implementation process.

The build process runs Vivado synthesis and implementation to generate the device binary. Some of the implementation steps are enable and run as part of the default build process, and some of the implementation steps can be optionally enabled at your discretion.

Optional steps can be listed using the `--list_steps` command, and include:

```
vp1.impl.power_opt_design, vp1.impl.post_place_power_opt_design,
vp1.impl.phys_opt_design, and vp1.impl.post_route_phys_opt_design.
```

An optional step can be enabled using the `--vivado.prop` option. For example, to enable `PHYS_OPT_DESIGN` step, use the following config file content:

```
[vivado]
prop=run.impl_1.steps.phys_opt_design.is_enabled=1
```

When an optional step is enabled as shown above, the step can be specified as part of the `-from_step/-to_step` command as described below in *Running --to_step or --from_step*, or enable a Tcl script to run before or after the step as described in *--linkhook Options*.

5. Passing parameters to the tool to control processing.

The `--vivado` option also allows you to pass parameters to the Vivado tools. The parameters are used to configure the tool features or behavior prior to launching the tool. The syntax for specifying a parameter uses the following form:

```
--vivado.param <object><parameter>=<value>
```

The keyword `param` indicates that you are passing a parameter for the Vivado tools, rather than a property for a design object. You must also define the `<object>` it applies to, the `<parameter>` that you are specifying, and the `<value>` to assign it.

The following example project indicates the current Vivado project, `writeIntermediateCheckpoints`, is the parameter being passed and the value is 1, which enables this boolean parameter.

```
--vivado.param project.writeIntermediateCheckpoints=1
```

6. Managing the reports generated during synthesis and implementation.



IMPORTANT! You must also specify `--save-temps` on the `v++` command line when customizing the reports generated by the Vivado tool to preserve the temporary files created during synthesis and implementation, including any generated reports.

You might also want to generate or save more than the standard reports provided by the Vivado tools when run as part of the Vitis tools build process. You can customize the reports generated using the `--advanced.misc` option as follows:

```
[advanced]
misc=report-type report_utilization name
synth_report_utilization_summary steps {synth_design} runs {__KERNEL__}
options {}
misc=report-type report_timing_summary name
impl_report_timing_summary_init_design_summary steps {init_design} runs
{impl_1} options {-max_paths 10}
misc=report-type report_utilization name
impl_report_utilization_init_design_summary steps {init_design} runs
{impl_1} options {}
misc=report-type report_control_sets name
impl_report_control_sets_place_design_summary steps {place_design} runs
{impl_1} options {-verbose}
misc=report-type report_utilization name
impl_report_utilization_place_design_summary steps {place_design} runs
{impl_1} options {}
misc=report-type report_io name impl_report_io_place_design_summary
```

```
steps {place_design} runs {impl_1} options {}
misc=report=type report_bus_skew name
impl_report_bus_skew_route_design_summary steps {route_design} runs
{impl_1} options {-warn_on_violation}
misc=report=type report_clock_utilization name
impl_report_clock_utilization_route_design_summary steps {route_design}
runs {impl_1} options {}
```

The syntax of the command line is explained using the following example:

```
misc=report=type report_bus_skew name
impl_report_bus_skew_route_design_summary steps {route_design} runs
{impl_1} options {-warn_on_violation}
```

- **misc=report=:** Specifies the `--advanced.misc` option as described in [--advanced Options](#), and defines the report configuration for the Vivado tool. The rest of the command line is specified in name/value pairs, reflecting the options of the `create_report_config` Tcl command as described in *Vivado Design Suite Tcl Command Reference Guide* (UG835).
- **type report_bus_skew:** Relates to the `-report_type` argument, and specifies the type of the report as the `report_bus_skew`. Most of the `report_*` Tcl commands can be specified as the report type.
- **name impl_report_bus_skew_route_design_summary:** Relates to the `-report_name` argument, and specifies the name of the report. Note this is not the file name of the report, and generally this option can be skipped as the report names will be auto-generated by the tool.
- **steps {route_design}:** Relates to the `-steps` option, and specifies the synthesis and implementation steps that the report applies to. The report can be specified for use with multiple steps to have the report regenerated at each step, in which case the name of the report will be automatically defined.
- **runs {impl_1}:** Relates to the `-runs` option, and specifies the name of the design runs to apply the report to.
- **options {-warn_on_violation}:** Specifies various options of the `report_*` Tcl command to be used when generating the report. In this example, the `-warn_on_violation` option is a feature of the `report_bus_skew` command.



IMPORTANT! *There is no error checking to ensure the specified options are correct and applicable to the report type specified. If you indicate options that are incorrect the report will return an error when it is run.*

Associating an ELF file with MicroBlaze

You can use the following steps to associate an ELF file with a MicroBlaze™ processor in your design. Associating the ELF file configures a memory target, such as a set of BRAMs. The information that is needed for ELF file association includes:

- The location of the ELF file to be loaded

- The address space accessible via a master interface to a memory location, which the ELF file will be stored
- The mapped peripheral within that address space representing the memory where ELF file will be stored and from where it will be accessed at run time



IMPORTANT! This flow requires you to have access to the level of design hierarchy containing the MicroBlaze processor, and an existing ELF file.

This process uses `SCOPED_TO_REF` and `SCOPED_TO_CELLS` properties on the MicroBlaze processor itself, and not to the BRAMs that are the actual target of the ELF file data.

You can associate the ELF file to the MicroBlaze processor during the `v++ --link` process using the `--advanced.param <param_name>=<param_value>` command as described in [--advanced Options](#). An example for a config file is shown below.

```
[advanced]
param=hw_emu.post_sim_settings=<file_path>/link.tcl
```

The `link.tcl` should add the ELF file to the Vivado Design Suite project, exclude it for use in simulation, and associate it with the MicroBlaze processor, as shown in the example below.

```
add_files <file_path>/executable.elf
set_property used_in_simulation 0 [get_files <file_path>/executable.elf]
set_property SCOPED_TO_REF base_microblaze_design [get_files -all \
-of-objects [get_fileset sources_1] {<file_path>/executable.elf}]
set_property SCOPED_TO_CELLS { microblaze_0 } \
[get_files -all -of-objects [get_fileset sources_1] {<file_path>/
executable.elf}]
```

This information will be used to generate a BMM file which will be used by programs such as `data2mem` to generate a `.mem` file that will populate the BRAMs that are generated from the `block_memory_generator`.

Running Multiple Implementation Strategies for Timing Closure

For challenging designs, it can take multiple iterations of Vivado implementation using multiple different strategies to achieve timing closure. This topic shows you how to launch multiple implementation strategies at the same time in the hardware build (`-t hw`), and how to identify and use successful runs to generate the device binary and complete the build.

As explained in [--vivado Options](#) the `--vivado.impl.strategies` command enables you to specify multiple strategies to run in a single build pass. The command line would look as follows:

```
v++ --link -s -g -t hw --platform xilinx_zcu102_base_202010_1 -I . \
--vivado.impl.strategies "Performance_Explore,Area_Explore" -o
kernel.xclbin hello.xo
```


In the example above, the `Performance_Explore` and `Area_Explore` strategies are run simultaneously in the Vivado build to see which returns the best results. You can specify the `ALL` to have all available strategies run within the tool.



IMPORTANT! Running ALL implementation strategies might launch 30 or more runs in the Vivado tool, including any user-defined strategies stored in your home directory (`~/Xilinx/Vivado/202X.X/strategies`). This can be a tremendous drain on resources, and is not advised. You can prevent this by defining specific strategies to run, and using a command queue to distribute the process load in some managed way, such as through the `--vivado.impl.jobs` or the `--vivado.impl.lsf` commands.

You can also determine this option in a configuration file in the following form:

```
#Vivado Implementation Strategies
[vivado]
impl.strategies=Performance_Explore,Area_Explore
```

The Vitis compiler automatically picks the first completed run results that meets timing to proceed with the build process and generate the device binary. However, you can also direct the tool to wait for all runs to complete and pick the best results from the completed runs before proceeding. This would require the use of the `--advanced.compiler` directive as shown:

```
[advanced]
param=compiler.multiStrategiesWaitOnAllRuns=1
```

`compiler.multiStrategiesWaitOnAllRuns=0` represents the default behavior. If you want `v++` to wait for all runs to complete, which will get their report files, change that parameter value to 1.

The Vitis analyzer displays the implementation results for the all strategies that are allowed to run to completion. This includes an overview of the implementation results, in addition to a Timing Summary report. You can use this feature to review the different strategies and results.

You can also manually review the results of all implementation strategies after they have completed. Then, use the results of any of the implementation runs by using the `--reuse_impl` option as described in [Using --to_step and Launching Vivado Interactively](#).

Using --to_step and Launching Vivado Interactively

The Vitis compiler lets you stop the build process after completing a specified step (`--to_step`), manually intervene in the design or files in some way, and then continue the build by specifying a step the build should resume from (`--from_step`). The `--from_step` directs the Vitis compiler to resume compilation from the step where `--to_step` left off, or some earlier step in the process. The `--to_step` and `--from_step` are described in [v++ General Options](#).



IMPORTANT! The `--to_step` and `--from_step` options are sequential build options that require you to use the same project directory when launching `v++ --link --from_step` as you specified when using `v++ --link --to_step`.

The Vitis compiler also provides a `--list_steps` option to list the available steps for the compilation or linking processes of a specific build target. For example, the list of steps for the link process of the hardware build can be found by:

```
v++ --list_steps --target hw --link
```

This command returns a number of steps, both default steps and optional steps that the Vitis compiler goes through during the linking process of the hardware build. Some of the default steps include: `system_link`, `vpl`, `vpl.create_project`, `vpl.create_bd`, `vpl.generate_target`, `vpl.synth`, `vpl.impl.opt_design`, `vpl.impl.place_design`, `vpl.impl.route_design`, and `vpl.impl.write_bitstream`.

Optional steps include: `vpl.impl.power_opt_design`, `vpl.impl.post_place_power_opt_design`, `vpl.impl.phys_opt_design`, and `vpl.impl.post_route_phys_opt_design`.



TIP: An optional step must be enabled before specifying it with `--from_step` or `--to_step` as previously described in [Using the --vivado and --advanced Options](#).

Launching the Vivado IDE for Interactive Design

Note: The [Vitis Export to Vivado Flow](#) is the preferred method.

With the `--to_step` command, you can launch the build process to Vivado synthesis and then start the Vivado IDE on the project to manually place and route the design. To perform this you would use the following command syntax:

```
v++ --target hw --link --to_step vpl.synth --save-temps --platform  
<PLATFORM_NAME> <XO_FILES>
```



TIP: As shown in the example above, you must also specify `--save-temps` when using `--to_step` to preserve any temporary files created by the build process.

This command specifies the link process of the hardware build, runs the build through the synthesis step, and saves the temporary files produced by the build process.

You can launch the Vivado tool directly on the project built by the Vitis compiler using the `--interactive` command. This opens the Vivado project found at `<temp_dir>/link/vivado/vpl/prj` in your build directory, letting you interactively edit the design:

```
v++ --target hw --link --interactive impl --save-temps --platform  
<PLATFORM_NAME> <XO_FILES>
```

When invoking the Vivado IDE in this mode, you can open the synthesis or implementation runs to manage and modify the project. You can change the run details as needed to close timing and try different approaches to implementation. You can save the results to a design checkpoint (DCP), or generate the project bitstream (.bit) to use in the Vitis environment to generate the device binary.

After saving the DCP from within the Vivado IDE, close the tool and return to the Vitis environment. Use the `--reuse_impl` option to apply a previously implemented DCP file in the `v++` command line to generate the `xclbin`.



IMPORTANT! The `--reuse_impl` option is an incremental build option that requires you to apply the same project directory when resuming the Vitis compiler with `--reuse_impl` that you specified when using `--to_step` to start the build.

The following command completes the linking process by using the specified DCP file from the Vivado tool to create the `project.xclbin` from the input files.

```
v++ --link --platform <PLATFORM_NAME> -o'project.xclbin' project.xo --reuse_impl ./_x/link/vivado/routed.dcp
```

You can also use a bitstream file generated by the Vivado tool to create the `project.xclbin`:

```
v++ --link --platform <PLATFORM_NAME> -o'project.xclbin' project.xo --reuse_bit ./_x/link/vivado/project.bit
```

Note: The `project.bit` used for `--reuse_bit` is a partial bit and not a full bit.

Additional Vivado Options

Some additional switches that can be used in the `v++` command line or config file include the following:

- `--export_script/--custom_script` edit and use Tcl scripts to modify the compilation or linking process.
- `--remote_ip_cache` specify a remote IP cache directory for Vivado synthesis.
- `--no_ip_cache` turn off the IP cache for Vivado synthesis. This causes all IP to be re-synthesized as part of the build process, scrubbing out cached data.

Vitis Export to Vivado Flow

Often the hardware design comprising the platform hardware is co-developed with Vitis AI Engine graphs and PL kernels, possibly by different teams. The Vitis Export flow enables bi-directional hardware hand-offs between the Vivado Design Suite and the Vitis tools to enable these design teams to work with loose coupling, sharing system design updates at convenient check points rather than with every iteration.

The Vitis Export flow enabled by `v++ --link --export_archive` performs standard linking design modifications, but stops before running Vivado synthesis and instead encapsulates all relevant design data (BD, IP repository) and XCLBIN metadata for the Xilinx Runtime (XRT) into a Vitis Metadata Archive (`.vma`) file, forgoing the normal automated Vivado simulation, synthesis and implementation runs.

At the design level, the `v++` command operates entirely within a *dynamic region* block design container (BDC) in the extensible hardware platform. Based on source input files, the `v++` linker instantiates user-defined PL kernels, configures platform IP such as AI Engine, NoC, and soft interconnects, adds required design IP for AXI buses, clock domain crossing, data width conversion, and FIFO buffering, adds networks for hardware debugging, trace, and clocking, and creates connections between IP within the dynamic region.

The `.vma` file is imported into the original extensible platform project in the Vivado Design Suite using the `vitis::import_archive` Tcl procedure. The `.vma` IP repository is added to the Vivado project's IP repository and a new active variant for the dynamic region BDC is created. Development can then continue in the Vivado project, including additional design modifications, simulation, synthesis, and implementation.

The Vitis Export flow is mainly used to make modifications to the platform in Vivado after importing the `.vma` file. After implementation and timing closure, the `write_hw_platform -fixed` command has been enhanced to encapsulate the XRT metadata from the `.vma` file into the output `.xsa`. You can also export the XSA for the hardware emulation target from the Vivado tool and run emulation in the Vitis tool.

The flow described here can be repeated through any number of design iterations. The platform `.xsa` file can be exported from Vivado and passed back to the Vitis tools. The updated system design `.vma` file produced by the `v++ --link --export_archive` command can be re-imported into the Vivado tools. The flow supports testing the design on hardware and hardware emulation.

Vitis Export Flow Guidelines and Limitations

The `v++` compiler operates on a Vivado project that has been encapsulated in an extensible XSA built in Vivado. Conversely, the block design of the VMA is imported into a project as a design source that the user can continue to modify in Vivado.

In general, any modification to the Vivado project after `vitis::import_archive` that does not invalidate the contract between the imported design and the `.xclbin` metadata contained within the VMA is supported. The following table enumerates supported and prohibited operations.

Supported operations include:

- Adding, removing, and reconfiguring IPs and RTL modules outside of and unconnected to the Vitis region hierarchy within the dynamic region block design.
- Add, removing, or changing connections unconnected to the Vitis region hierarchy within the dynamic region block design.
- Changing clock frequencies on clock wizard instances outside of the Vitis region hierarchy within the dynamic region block design.
- Changing QoS settings on `axi_noc` instances in the dynamic region block design.
- Adding `.xdc` constraints associated with any part of the design, including within the Vitis region hierarchy within the dynamic region block design.

Prohibited operations include:

- Adding or deleting any IP instances or connections within the Vitis region hierarchy within the dynamic region block design.
- Adding or deleting connections between the dynamic region and the Vitis region hierarchy.
- Changes to the address map that modifies any address APERTURES or IP addressing in the Vitis region hierarchy within the dynamic region block design.
- Project changes that modify the netlist path to the Vitis region hierarchy within the dynamic region block design.

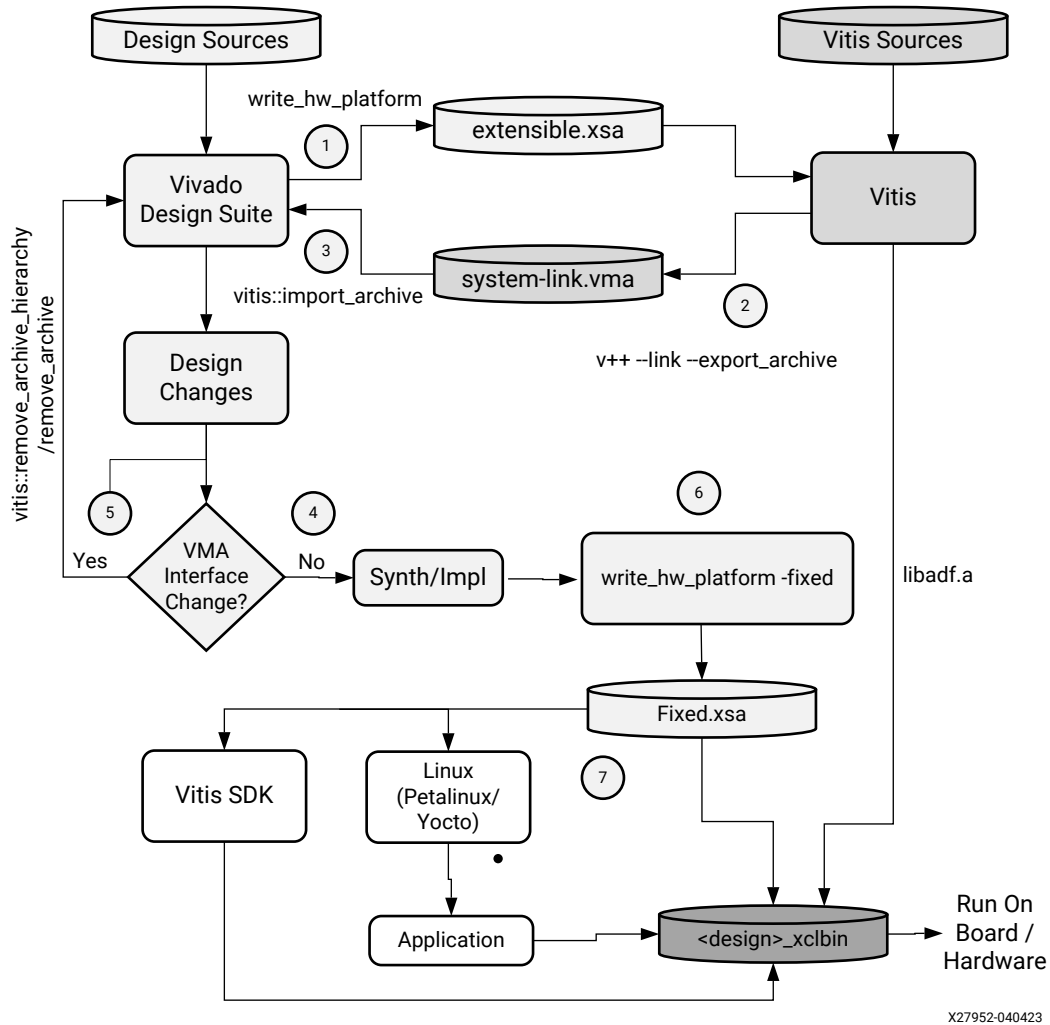
Current limitations of the Vitis Export flow include the following:

- Supported for Versal platforms only
- The Vivado dynamic region block design must be a block design container, and as a result, some base platforms, for example `vck190_base`, currently do not support the flow

Flow Details and Implementation

This section explains the Vitis Export flow as shown in the following diagram. The complete flow (from hardware design creation to export `.xclbin`) is divided into seven steps.

Figure 32: Vitis Export to Vivado Flow

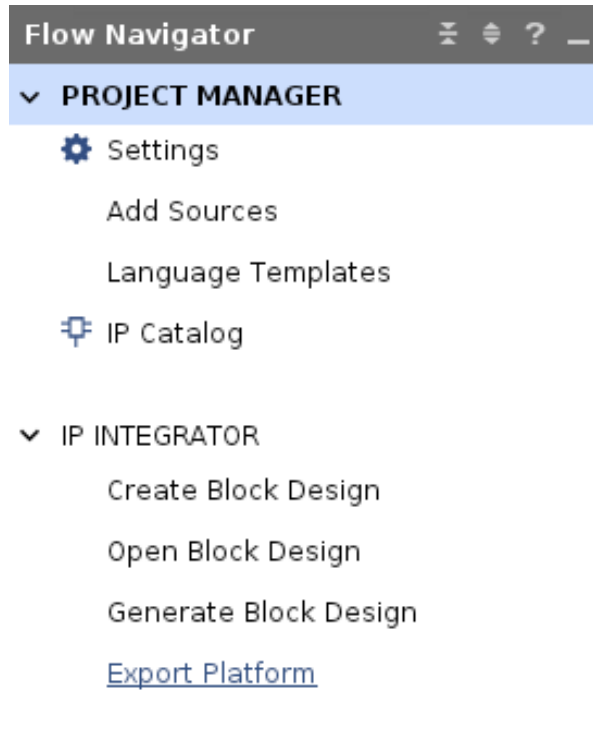


You can execute the flow in the following steps:

1. Start by creating the Versal custom platform design. The Vitis export flow requires block-design container (BDC) designs as described in [Generate XSA file](#), and does not support flat designs. You can use HLS-based components, or RTL-based packaged IP, or standard IP from the IP catalog in your Vivado design.

After creating the platform, run synthesis to clean any issues related to hardware realization. Only use the flow for the Versal device platforms. Export the platform into `extensible.xsa` using following steps:

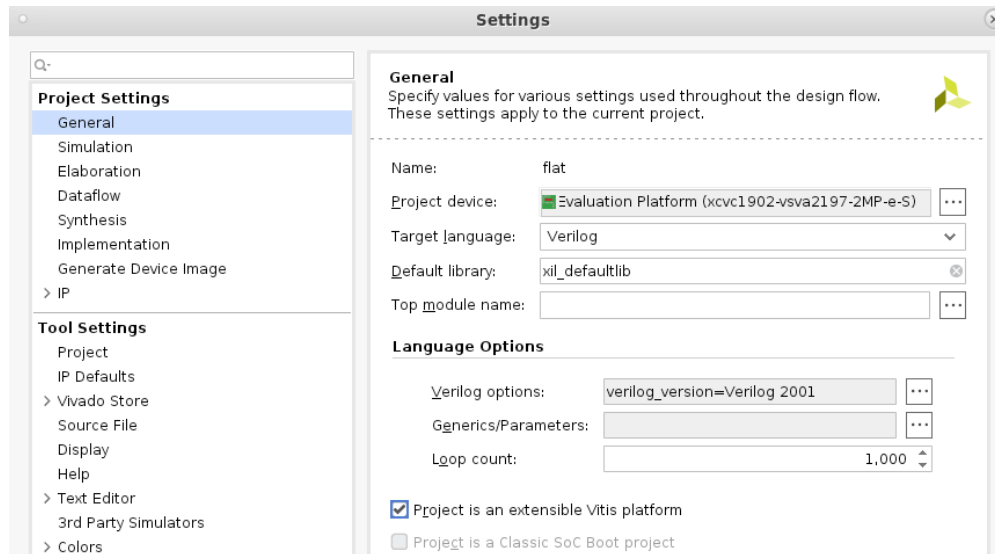
- a. Click on **Export Platform** under IP Integrator in the Flow Navigator.



In Export Platform, select the **Presynthesis** option to generate the `extensible.xsa`.



TIP: If the **Export Platform** option is disabled, go to Project Settings and select **Project is an extensible Vitis Platform**.



- b. From the Tcl console, enter the command `write_hw_platform -f <filename>.xsa`
2. In the Vitis tool, the exported `.xsa` becomes the platform of choice, and the system design is completed.

- Compile the AI Engine graph (`libadf.a`)
- Compile PL kernels (`.xo`)
- Update `system.cfg` file for connectivity
- Run `v++` linker with `--export_archive` option

In the Vitis Export to Vivado flow the system linking process occurs as usual, but the automatic launch of Vivado synthesis and place and route is skipped. Instead the `v++ --link --export_archive` command is used to generate the Vitis metadata archive (`.vma`) to export to the Vivado Design Suite.

```
v++ --link --export_archive --platform ../<>.xsa --config ../system.cfg \
<>.xo ./libadf.a -o <vma_file>.vma
```



IMPORTANT! The `--export_archive` command cannot be used with the `--target` (or `-t`) command. An error is returned.

3. Open the Vivado project after generating the `.vma` file from the Vitis tools. Import the `.vma` file in the Vivado project, or new project, with the following Tcl procedure:

```
vitis::import_archive ./vma_path/<vma_file>.vma
```

- a. A new variant for the dynamic region block design container will be created by cloning the VMA's block design and made active.
 - b. Within the dynamic region block design, Vitis content is encapsulated within a level of hierarchy that the user should consider essentially read-only whenever they intend to use XRT after implementing the design in Vivado. See the [Vitis Export Flow Guidelines and Limitations](#) section for more details on how to preserve XRT metadata consistency while updating the design in Vivado.
 - c. The `.vma` region can be opened in Vivado. Alternatively, to rerun Vitis flows, you can re-export an extensible XSA after removing Vitis content via `vitis::remove_archive_hierarchy`.
4. After importing the `.vma` to the Vivado tools, design modifications can be done in Vivado only. See the [Vitis Export Flow Guidelines and Limitations](#) section for more information.
 - a. If there are changes to the `.vma` file, such as changes to the AI Engine design, PL kernels, or to the PLIO boundary, go to Step 5 to regenerate the `.xsa` file and reiterate through the Vitis Export to Vivado flow.
 - b. If the design changes are related to Vivado only, simulate the design, synthesize and implement the design to meet timing, and go to Step 6 for generating the `fixed.xsa`.
 5. If the design requires changes related to the AI Engine, PL kernels, or updates in the PLIO boundary, these changes require updates to the linked system design, and regenerating the `.vma` in the Vitis tool. Thus, you must first remove the previously imported `.vma` from the Vivado project using one of two Tcl procedures:

- a. Use the `vitis::remove_archive_hierarchy` procedure to remove the imported `.vma` file while preserving any work done to the Vivado project after importing the `.vma`.
- b. Use the `vitis::remove_archive` procedure to restore the Vivado project to its state prior to importing the `.vma` file, removing both the `.vma` and any changes to the project.

After removing `.vma` from the Vivado design, you can make any changes to the project. Vitis depends only upon the dynamic region block design and to potential connectivity points declared through PFM APIs. Update `system.cfg` to update the boundary connections. If there is a need to export the `extensible.xsa` for the second iteration or later from Vivado, use the `vitis::remove_archive` command and repeat Step 1 to export the `extensible.vma`; repeat Step 2 and 3 to export the VMA from Vitis and import VMA to Vivado respectively.

6. Once you have implemented your design, you can generate a `fixed.xsa` from the Vivado project by using the following command:

```
write_hw_platform -fixed ./<fixed_xsa>.xsa
```

This XSA can be used to perform application development for PetaLinux / Yocto or XRT-based apps development, PS-based apps development through Vitis embedded software flow, or baremetal flow as it has been done traditionally.

The fixed XSA generated from the preceding command can be used to test the design on the hardware target only. If you want to run hardware emulation, generate the XSA using the following command instead:

```
write_hw_platform -fixed -include_sim_content ./<fixed_xsa>.xsa
```

7. After modifying the design, you can test the design on hardware or run hardware emulation. Follow the steps below to generate the fixed XSA for testing the design on hardware and hardware emulation.

To test the design on hardware, first run synthesis and implementation on the design. Use the command `write_hw_platform -fixed ./<path to fixed.xsa>` to generate fixed XSA.

To run hardware emulation, follow the steps below to generate the fixed XSA:

- a. Re-generate the output products
- b. Execute the command `launch_simulation -scripts only`
- c. Run `compile.sh`
- d. Run `elaborate.sh`
- e. Execute the command `write_hw_platform -fixed -include_sim_content <path to fixed xsa>`

When you are ready to run the design on the hardware target (`-t=hw`) or hardware emulation target (`-t=hw_emu`), run `v++ --package` to generate the `.xclbin`. To generate the `xclbin` for hardware and hardware emulation, use the respective `fixed.xsa`. Set the `t= hw` or `hw_emu`, then provide the required software binaries and files to generate the `.xclbin`.

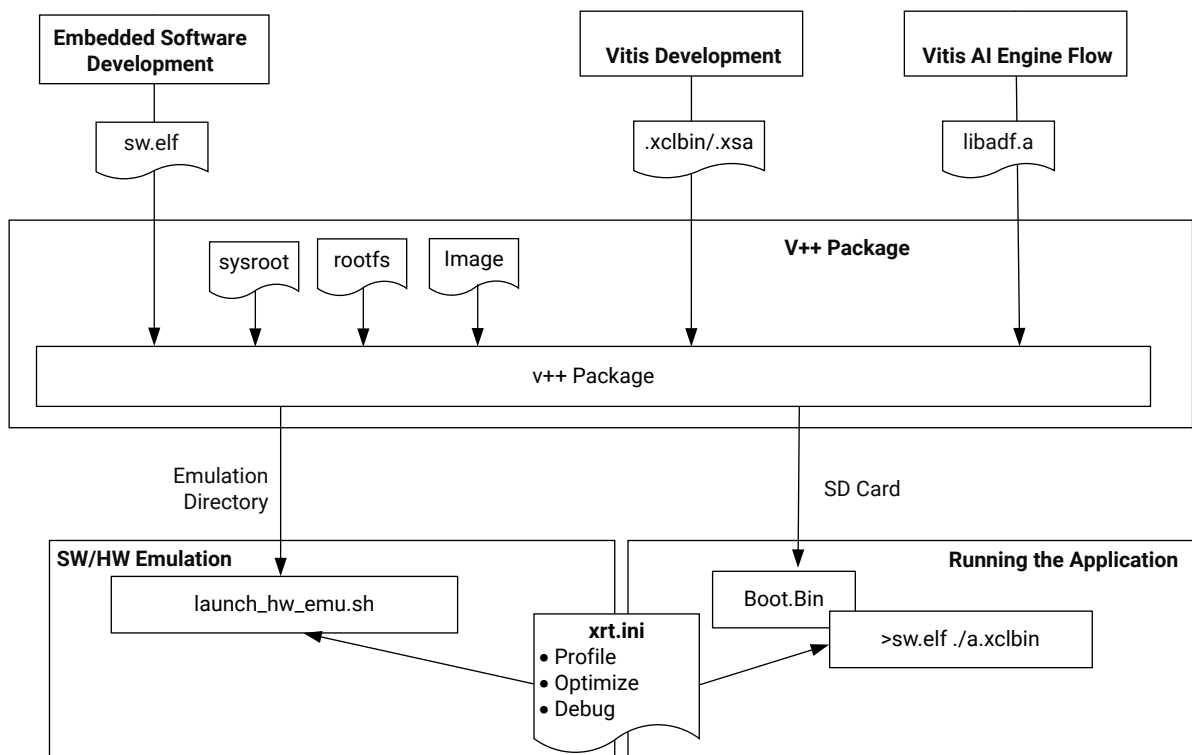
Guidelines for Revision Control

It is recommended to revision control the project sources while developing the design using the Vitis and Vivado tools. In this flow, a number of design iterations are required for the hardware design development. You can adopt revision control mechanism according to your standards. The following files are recommended for revision control:

- C++ source files
- AI Engine graphs (`.libadf`)
- Config files
- Tcl scripts
- *.vma
- *.xsa

Packaging the System

Figure 33: Linking the System Design



X27360-110422

The Vitis `v++ --package` command generates SD card and other Flash images required for booting the system, in addition to the `.xclbin` device binary from the `.xsa` generated for Versal devices. The `v++ --package` step, or `-p` option, packages the final system at the end of the `v++` compile, link, and package process. As described in [Packaging for Embedded Platforms](#), this is a required step for all Versal platforms, including AI Engine platforms, and embedded processor platforms.

A fixed `.xsa` can be used to create custom boot, and software platform images as described in the *Versal adaptive SoC System Software Developers Guide* ([UG1304](#)). But the [--package Options](#) let you package your design and define various files required for booting and configuring the AMD device for use during emulation or in production systems. It collects the various elements to create an SD card, or other means to program the device, to define the operating system, and to load the application and kernel code.

In the Vitis unified IDE, the package process is automated and the tool creates the required files based on the build target, platform, and OS. However, in the command line flow, you must specify the Vitis packaging command (`v++ --package`) with the correct options for the job.

After packaging the design the AMD Vitis™ compiler generates a `v++ .package_summary` that includes the packaging command and log file. The summary file can be viewed in Analysis view of the Vitis analyzer alongside the compile, link, and run summaries as explained in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Packaging for Embedded Platforms

For AMD Zynq™ UltraScale+™ MPSoC and AMD Zynq™ 7000 embedded platforms, the `--package` command line is shown below:

```
v++ --package -t [sw_emu | hw_emu | hw] --platform <platform> input.xclbin  
[ -o output.xclbin ]
```

Note: If the output option (`-o`) is not specified, the tool creates an output file with the default name of `a.xclbin`.

For Versal devices the `v++ --link` command creates an `.xsa` file instead of the `.xclbin` file. In this case the `.xsa` must be provided to the package process to generate the `.xclbin` file. The `--package` command line for Versal devices is as follows:

```
v++ --package -t [sw_emu | hw_emu | hw] --platform <platform> input.xsa [ -  
o output.xclbin ]
```

In the case of Versal platforms, the package process takes the `.xsa` file generated during the `v++ --link` command, and also takes the `libadf.a` file produced by the `aiecompiler` command and integrates it into the output device binary, as described in [Packaging Versal Designs](#).

The `--package` command has a range of options for use with the different platforms and build targets supported by the Vitis tools. In the Vitis IDE, the package process is automated and the tool creates the required files as needed. However, in the command line flow, you must specify the `v++ --package` command or add the `[package]` tag in the `config` file with the right options for the job. The following is an example command for hardware emulation:

```
v++ --package --config package.cfg ./aie_graph/libadf.a \  
./project.xsa -o aie_graph.xclbin
```

Where, the `--config package.cfg` option specifies a configuration file for the Vitis compiler with the various options specified for the package process. The following is an example configuration file:

```
platform=xilinx_vck190_base_202310_1
target=hw_emu
save-temps=1

[package]
boot_mode=sd
out_dir=./emulation
enable_aie_debug=1
rootfs=<path_to_platform>/sw/versal/xilinx-versal-common-v2023.1/rootfs.ext4
image_format=ext4
kernel_image=<path_to_platform>/sw/versal/xilinx-versal-common-v2023.1/Image
sd_file=host.exe
```

For software and hardware emulation, the command takes the `.xclbin` or `.xsa` file as input, produces a script to launch emulation (`launch_sw_emu.sh` or `launch_hw_emu.sh`), and writes needed support files to a specified output folder, `--package.out_dir`.

Additional files required for running the application, such as data files needed as input or to validate the application, or the `xrt.ini` file for profiling and debug, must be included in the output files, and can be transferred individually using the `--package.sd_file` option, or transferred as a directory using the `sd_dir` option as explained in [--package Options](#).

For hardware builds, the `--package` command creates an `sd_card` folder, or the `QSPI.img` depending on the boot mode specified with the `--package.boot_mode` option.



TIP: For bare metal ELF files running on PS cores, you should also add the following option to the command line:

```
--package.ps_elf <elf>,<core>
```

The package command creates an output folder called `sd_card`, that contains all of the files needed to run hardware emulation for the application, modeling the boot process of an `sd_card`. For hardware builds, it contains the files required for creating an SD card to boot the device. The following is an example of the packaging output for hardware emulation:

```
|-- BOOT_bh.bin      //Boot header
|-- BOOT.BIN        //Boot File
|-- boot_image.bif
|-- launch_hw_emu.sh //Hardware emulation launch script
|-- libadf          //AIE emulation data folder
|  |-- cfg
|  |  |-- aie.control.config.json
|  |  |-- aie.partial.aiecompile_summary
|  |  |-- aie.shim.solution.aiesol
|  |  |-- aie.sim.config.txt
|  |-- aie.xpe
|-- plm.bin          //PLM boot file
|-- pmc_args.txt     //PMC command argument specification file
|-- pmc_cdo.bin      //PMC boot file
|-- qemu_args.txt    //QEMU command argument specification file
```

```
| -- sd_card  
| | -- BOOT.BIN  
| | -- boot.scr  
| | -- aie_graph.xclbin  
| | -- host.exe  
| | -- Image  
| -- sd_card.img  
`-- sim                                //Vivado simulation folder
```

After creating the `sd_card` folder for the hardware build, copy the contents to an SD card to create the boot image for your physical device.

Note: On Windows OS you must use a third-party tool, such as Etcher, to write on the SD card for use in booting the AMD device.

Packaging Versal Designs

The Versal AI Engine compiler generates output in the form of a library file, `libadf.a`, which contains ELF and CDO files, as well as tool-specific data and metadata, for hardware and hardware emulation flows. To create a loadable image binary, this data must be combined with PL-based configuration data, boot loaders, and other binaries. The Vitis packager performs this function, combining information from `libadf.a` and the Vitis linker generated XSA file.

For Versal adaptive SoCs, the programmable device image (PDI) file is used to boot and program the hardware device. For hardware emulation the `--package` command adds the PDI, `EMULATION_DATA` sections and the XSA file, and outputs an XCLBIN file. For hardware builds, the package process creates an XCLBIN file containing ELF files and graph configuration data objects (CDOs) for the AI Engine application. The XCLBIN file includes the following information:

- **PDI:** Programming information for the AI Engine array
- **Debug data:** Debug information when included in the build
- **Memory topology:** Defines the memory resources and structure for the target platform
- **IP Layout:** Defines layout information for the implemented hardware design
- **Metadata:** Various elements of platform meta data to let the tool load and run the XCLBIN file on the target platform

Packaging for Data Center Platforms



TIP: For Data Center accelerator cards the `--package` command is only required for cards incorporating Versal devices.

For Versal Data Center accelerator cards, the `--package` command line is as follows:

```
v++ --package -t [sw_emu | hw_emu | hw] --platform <platform> libadf.a
input.xsa [ -o output.xclbin ]
```

In the Versal AI Engine flow, the `--package` command combines the `libadf.a` output from the `aiecompiler` with the input `.xsa` file and generates the `.xclbin` file:

If the output option (`-o`) is not specified, the tool creates an output file with the default name of `a.xclbin`.

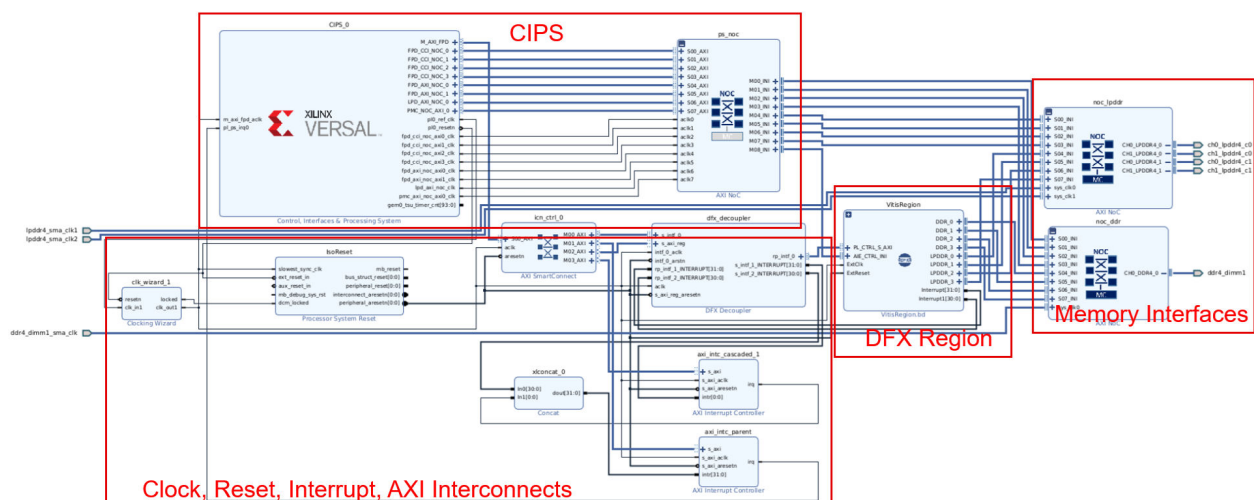
Packaging for DFX Platforms

Vitis Dynamic Function eXchange (DFX) platforms contain a static region and DFX regions.

- **Static Region:** Hardened block that is loaded in system booting and is not reconfigurable at runtime.
- **DFX region:** A reconfigurable partition (RP) that is implemented by the Vivado IP integrator block design container (BDC). This partition can be reprogrammed repeatedly during runtime to dynamically trade out one function, or reconfigurable module (RM), for another.

provides base DFX platforms, such as `xilinx_zcu102_base_dfx_202320_1` and `xilinx_vck190_base_dfx_202310_1`. These are single RP platforms, meaning that it contains a static region and only one DFX region in which RMs can be dynamically exchanged at runtime. In the case of `xilinx_vck190_base_dfx_202310_1`, the RM contains an AI Engine array and PL kernels. When loading the RM the entire AI Engine array and all PL kernels are loaded at the same time. The following picture shows the block design (BD) of the platform.

Figure 34: Block Design of the Base DFX Platform

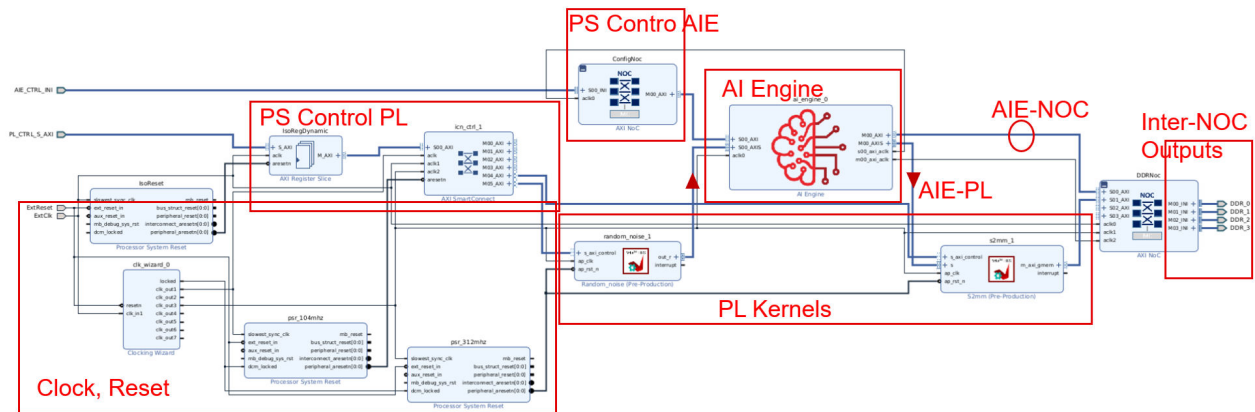


The static partition contains the following:

- CIPS and PS NoC
- Clock, Reset, Interrupt, and AXI interconnections
- NoC Interface and DDR memory controller

The reconfigurable module (RM) to load into the DX region of the `xilinx_vck190_base_dfx_202310_1` platform contains an AI Engine array and PL kernels that are linked together by the Vitis linker (`v++ -l`). Refer to *Versal Adaptive SoC Hardware, IP, and Platform Development Methodology Guide* ([UG1387](#)). The following picture shows an example of the RM.

Figure 35: Block Design of the RM



The RM, with interfaces to the static region, includes:

- Clock and Reset
- Inter-NoC Interface (INI) outputs to general memory
- INI input from PS to control AI Engine
- AXI Interface from PS to control PL kernels

The contents of the RM can include:

- AI Engine
- PL kernels
- AXI-NoC IP
- Clock and reset modules
- AXI Interconnect module
- ILA
- FIFO, Data Width Converter (DWC) and CDC modules

The `v++ --compile` and `v++ --link` commands are the same for both a DFX platform and a non-DFX platform. However, the `v++ --package` command is different for the DFX platform than the non-DFX platform, and is different between the hardware and hardware emulation targets as described below.

Hardware Packaging and Deployment

For a `hw` target the `v++ --package` command for the DFX platform requires two steps:

1. The first step generates an `xclbin` file that contains the RM PDI that will be loaded at runtime.

```
v++ -p -t hw -f xilinx_vck190_base_dfx_202310_1
--package.defer_aie_run \
-o rm.xclbin \
${XSA} \
libadf.a
```

2. The second step packages the RM `xclbin` file (`rm.xclbin`) into the SD card that can be read by the host program.

```
v++ -p -t hw -f xilinx_vck190_base_dfx_202310_1
--package.rootfs ${ROOTFS}
--package.kernel_image ${IMAGE} \
--package.boot_mode=sd \
--package.image_format=ext4 \
--package.sd_dir data \
--package.sd_file ${HOST_EXE} \
--package.sd_file rm.xclbin
```

When running the hardware, after Linux has started type the following command to load the RM:

```
cd /run/media/mmcblk0p1
./host.exe rm.xclbin
```

Note: If the XRT detects that the same XCLBIN has been downloaded already, it will not download the XCLBIN again by default. Instead, it loads metadata from the `xclbin` file only.

To make sure XCLBIN is downloaded and the device is clean every time, set `xrt.ini` to enforce XCLBIN download to the device as follows:

1. Add following configuration to the `xrt.ini` file:

```
[Runtime]
force_program_xclbin=true
```

2. Add the `xrt.ini` file to the SD card during the package process: `--package.sd_file xrt.ini`

This ensures the XRT downloads the XCLBIN to device every time.

Hardware Emulation Packaging and Deployment

For `hw_emu` target the `v++ --package` command can be:

```
v++ -p -t hw_emu -f xilinx_vck190_base_dfx_202310_1 \  
--package.defer_aie_run \  
--package.rootfs ${ROOTFS} \  
--package.kernel_image ${IMAGE} \  
--package.boot_mode=sd \  
--package.image_format=ext4 \  
--package.sd_dir data \  
--package.sd_file ${HOST_EXE} \  
--package.sd_file emconfig.json \  
-o rm.xclbin \  
${XSA} \  
libadf.a
```

Launch the emulation with the following command:

```
./launch_hw_emu.sh
```



IMPORTANT! In hardware emulation you can load the XCLBIN only once. The RM cannot be reloaded.

In QEMU, following commands can be run:

```
cd /run/media/mmcblk0p1  
export XCL_EMULATION_MODE=hw_emu  
./host.exe rm.xclbin
```

To run multiple RMs on a DFX platform during hardware emulation you must start and stop the emulator, and load each RM `.xclbin` for a separate test run.

Building a Bare-Metal System

Building a bare-metal system requires a few additional steps from the Linux-based system flow previously described. The specific steps required are described here.

1. Build the bare-metal platform.

Building bare-metal applications requires a bare-metal domain in the platform. The base platform `xilinx_vck190_base_202320_1` does not have a bare-metal domain, which means you must create a custom platform from the base platform. You must create a custom platform because the PS application needs bare-metal drivers for the PL kernels in the design.

Starting with the `xsa` file generated by the `v++` linking process as described in [Linking the System](#) use the following shell script:

```
generate-platform.sh -name vck190_baremetal -hw <filename>.xsa \  
                    -domain psv_cortexa72_0:standalone
```

where:

- `-name vck190_baremetal`: Specifies a name for the new platform. In this example the platform will be written to: `./vck190_baremetal/export/vck190_baremetal`
- `-hw <filename>.xsa`: Specifies the name of the input `xsa` file generated during the `v++ --link` command.

Note: The `.xsa` file must be for the hardware target as generated by the `v++ --link` command.

- `-domain psv_cortexa72_0:standalone`: Specifies the processor domain and operating system to create in the new platform.

You can add the new platform to your platform repository by adding the file location to your `$PLATFORM_REPO_PATHS` environment variable. This makes it accessible to the Vitis unified IDE for instance, or allows you to specify the platform in command-lines by simply referring to the name rather than the whole path.



IMPORTANT! *The generated platform will be used only for building the bare-metal PS application and is not used any other places across the flow.*

2. Compile and link the PS application as described in [Compiling and Linking Host Code for Bare-Metal in AI Engine Tools and Flows User Guide \(UG1076\)](#). You will specify the custom platform drivers to include in the build.
3. Package the System

Finally, you must run the `v++ --package` command to generate the final bootable image (PDI) for running the design on the bare-metal platform. This command produces the SD card content for booting the device and running the application, as described in [Packaging the System](#)

```
v++ -p -t hw \
  -f xilinx_vck190_base_202310_1 \
  libadf.a project.xsa \
  --package.out_dir ./sd_card \
  --package.domain aiengine \
  --package.defer_aie_run \
  --package.boot_mode sd \
  --package.ps_elf main.elf,a72-0 \
  -o aie_graph.xclbin
```



TIP: For bare-metal ELF files running on PS cores, you should also add the `package.ps_elf` option to the `--package` command.

The use of `--package.defer_aie_run` is related to the way the AI Engine graph is run. If the application is loaded and launched at boot time, this option is not required. If your host application launches and controls the graph, then you need to use these options when compiling and packaging your system.

The `./sd_card` folder, specified by the `--out_dir` option, contains the following files produced for the hardware build:

```
|-- BOOT.BIN      //BOOT.BIN file containing PDI and the application ELF
|-- boot_image.bif //bootgen input file used to create BOOT.BIN
`-- sd_card      //SD card folder
    |-- aie_graph.xclbin //xclbin output file (not used)
    `-- BOOT.BIN      //BOOT.BIN file containing PDI and the
        application ELF
```

Copy the contents of the `sd_card` folder to an SD card to create a boot device for your system.

Now you have built the bare-metal system, you can run it or debug it.

Running the System on Hardware

Running the system depends on the build target. The process of running on the physical hardware is different from running either software or hardware emulation. Refer to [Running Emulation Targets](#) for more information on running emulation.

Running the System project hardware build lets you see your application running on an embedded processor platform targeting an AMD Versal™ adaptive SoC or AMD Zynq™ UltraScale+™ MPSoC devices, or an accelerator card such as the AMD Alveo™ Data Center accelerator card. The performance data and results captured here are the actual performance of your accelerated application. Yet the profiling data from this run might still reveal opportunities to optimize your design.



TIP: To use the accelerator card, you must have it installed as described in *Getting Started with Alveo Data Center Accelerator Cards* ([UG1301](#)).

1. Edit the `xrt.ini` file as described in [xrt.ini File](#).

This is optional, but recommended when running on hardware for evaluation purposes. You can configure XRT with the `xrt.ini` file to capture debugging and profile data as the application is running. To capture event trace data when running the hardware, refer to [Enabling Profiling in Your Application](#). To debug the running hardware, refer to [Debugging During Hardware Execution](#).



TIP: Ensure to use the `++ -g` option when compiling your kernel code for debugging.

2. Unset the `XCL_EMULATION_MODE` environment variable.



IMPORTANT! The hardware build will not run if the `XCL_EMULATION_MODE` environment variable is set to an emulation target.

3. For embedded platforms, boot the SD card.



TIP: This step is only required for platforms using AMD embedded devices such as Versal adaptive SoC or Zynq UltraScale+ MPSoC.

For an embedded processor platform, copy the contents of the `./sd_card` folder produced by the `++ --package` command to an SD card as the boot device for your system. Boot your system from the SD card. You will need to login to PetaLinux to run the application from the SD card, using the following steps:

- a. Login to the user name *petalinux* and set a password for this user if it is the first time logging in. This password will also be the `sudo` password.
 - b. Switch user to `sudo` to run the host application from the SD card: `sudo -i`
 - c. Change directory to the SD card: `cd /run/media/mmcblk0p1`
 - d. Run the application as described in the next step.
4. Run your application.

The specific command line to run the application will depend on your host code. A common implementation used in AMD tutorials and examples is as follows:

```
./host.exe kernel.xclbin
```



TIP: This command line assumes that the host program is written to take the name of the *xclbin* file as an argument, as most AMD Vitis™ examples and tutorials do. However, your application can have the name of the *xclbin* file hard-coded into the host program, or can require a different approach to running the application.

When running the design you can specify a number of trace options as described in [Enabling Profiling in Your Application](#) to capture design data during runtime. Any reports generated during the run are collected into the `xrt.run_summary` file. This collection of reports can be viewed by opening the `run_summary` in Vitis analyzer, and includes a Summary report, System and Platform Diagrams to illustrate the hardware design, Run Guidance offering any suggestions for improving the performance of the system, and a Profile Summary and Timeline Trace when enabled in the `xrt.ini` file during runtime. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for additional information.

Simulating the Application with the Emulation Flow

Development of an application and hardware kernels targeting an FPGA requires a phased development approach. Because FPGA, AMD Versal™ adaptive SoC, and AMD Zynq™ UltraScale+™ MPSoC are programmable devices, building the device binary for hardware takes some time. To enable quicker iterations without having to go through the full hardware compilation flow, the AMD Vitis™ tool provides software emulation targets to perform C-based simulation of the design, and hardware emulation targets to perform C-RTL co-simulation of the software application and PL kernels. Compiling for emulation targets is significantly faster than compiling for the actual hardware. Additionally, emulation targets provide full visibility into the application or accelerator, thus making it easier to perform debugging. Once your design passes in emulation, then in the late stages of development you can compile and run the application on the hardware platform.

The Vitis tool provides two emulation targets:

- **Software emulation (sw_emu):** The software emulation build compiles and links quickly, and the host program runs either natively on an x86 processor or in the QEMU emulation environment. The PL kernels are natively compiled and running on the host machine. This build target lets you quickly iterate on both the host code and kernel logic.
- **Hardware emulation (hw_emu):** The host program runs in `sw_emu`, natively on x86 or in the QEMU, but the kernel code is compiled into an RTL behavioral model which is run in the AMD Vivado™ simulator or other supported third-party simulators. This build and run loop takes longer but provides a cycle-accurate view of kernel logic.

Compiling and linking for either of the emulation targets is seamlessly integrated into the Vitis command line and IDE flows. You can compile your host and kernel source code for either emulation target, without making any change to the source code. For your host code, you do not need to compile differently for emulation as the same host executable or PS application ELF binary can be used in emulation. Emulation targets support most of the features including XRT APIs, buffer transfer, platform memory SP tags, kernel-to-kernel connections, etc. The following sections detail the features and requirements of both the software and hardware emulation flows.

While running emulation you can specify a number of trace options as described in [Enabling Profiling in Your Application](#) to capture design data during runtime. Any reports generated during the run are collected into the `xrt.run_summary` file. This collection of reports can be viewed by opening the `run_summary` in Vitis analyzer, and includes a Summary report, System and Platform Diagrams to illustrate the hardware design, Run Guidance offering any suggestions for improving the performance of the system, and a Profile Summary and Timeline Trace when enabled in the `xrt.ini` file during runtime. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for additional information.

SW Emulation is an abstract model and does not use any of the PetaLinux drivers like such as Zynq OpenCL (ZOCL), interrupt controller, or Device Tree Binary (DTB). Hence, the overhead of creating `sd_card.img` and booting PetaLinux on full QEMU machines can be avoided for SW Emulation. This enables faster SW_EMU as QEMU is slow and requires PetaLinux. Thus, for this approach, you are not required to provide fields such as `sysroot`, `rootfs` and `sd_Card Image`.

Note: If you are sourcing the environment setup script "`xilinx-versal-common-v2022.2/environment-setup-cortexa72-cortexa53-xilinx-linux`", they may find a warning or error to unset the `LD_LIBRARY_PATH` (if already set) when executing the embedded XRT. The environment setup script sets up the arm gcc tool chain path along with the required additional environment variables.

Installing the x86 XRT automatically sets the `LD_LIBRARY_PATH` variable to point to XRT libraries. For running both the embedded XRT and x86 XRT on the same setup (terminal), you must specify `arm-gcc` and `SYSROOT` paths for embedded systems.

This section contains the following chapters:

- [Running Emulation Targets](#)
- [Data Center vs. Embedded Platforms](#)
- [QEMU](#)
- [Running Emulation on Data Center Accelerator Cards](#)
- [Running Emulation on an Embedded Processor Platform](#)
- [Speed and Accuracy of Hardware Emulation](#)
- [Working with Simulators in Hardware Emulation](#)
- [Working with Functional Model of the HLS Kernel](#)
- [Working with SystemC Models](#)
- [Debug Techniques in Hardware Emulation](#)
- [Working with I/O Traffic Generators](#)

Running Emulation Targets

In the typical development flow, the steps will be as follows.

1. Software emulation of the system for verifying the functional correctness of the complete system

Note: This can also include running C simulation on HLS components, or running the `x86simulator` for verifying AI Engine components

2. AI Engine simulator for verifying that the AI Engine kernel and graph meets performance needs of the application

3. Hardware emulation of the system for verifying timing of the design

Note: This can also include running C/RTL co-simulation on HLS components, or running the `aiesimulator` for verifying AI Engine components

4. Testing and debugging on hardware

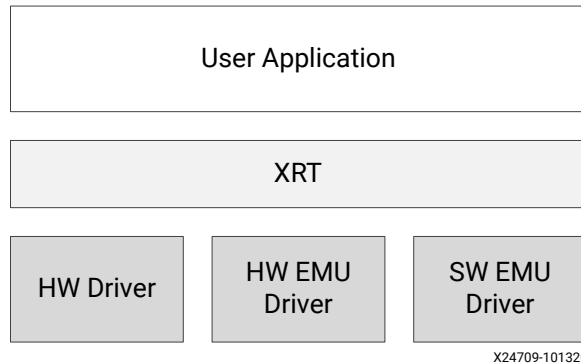
While running on hardware runs the system design on the physical device of the platform or accelerator card. However, running emulation targets occurs on a virtual system on the x86 processor, or in the QEMU environment on embedded processors. Software and hardware emulation targets have their own target specific drivers which are loaded at runtime by XRT. Thus, the same CPU binary can be run as-is without recompiling, by changing the target mode during runtime.

Based on the value of the `XCL_EMULATION_MODE` environment variable, XRT loads the target specific driver and makes the application interface with an emulation model of the hardware. The allowed values of `XCL_EMULATION_MODE` are `sw_emu` and `hw_emu`. If `XCL_EMULATION_MODE` is not set, then XRT will load the hardware driver.



IMPORTANT! It is required to set `XCL_EMULATION_MODE` when running emulation.

Figure 36: XRT Drivers



You can also use the `xrt.ini` file to configure various options applicable to emulation. There is an `[Emulation]` specific section in `xrt.ini`, as described in [xrt.ini File](#).

Emulation for a system with the AI Engine can be useful in:

- Checking initial system behavior with a limited known data set
- Functional integration and debugging of PS, PL, and ADF graph using GDB
- Testing the system with external traffic generator using Python, or C++
- Running system with C-based models for RTL kernels
- Applying AI Engine simulation options through the `x86.options` file in `Work/options` or the `aiesim_options.txt` found in the `aiesimulator_output` directory



IMPORTANT! *Software emulation requires all PL kernels to be HLS based, or contain C/C++ models. RTL kernels do not generally support software emulation.*

When writing host code to execute in software emulation, sync all buffer objects (`xrtBO`) with `xrtBOSync` API call. To run data-driven PL kernels more than once, you can run it inside a `while (1)` loop in the software application.

When launching software or hardware emulation, you can specify options for the AI Engine component according to the simulator being used. These options can be specified from the `launch_sw_emu.sh` script using the `-x86-sim-options`, or from the `launch_hw_emu.sh` script using the `-aie-sim-options`. For more information refer to *Reusing AI Engine Simulator Options in AI Engine Tools and Flows User Guide* ([UG1076](#)).

When the emulation environment is fully booted and the Linux prompt is up, make sure to set the following environment variables in the QEMU environment to ensure that the host application works. These must also be set when running on hardware.

```
export XILINX_XRT=/usr
export LD_LIBRARY_PATH=/mnt/sd*1:
export XCL_EMULATION_MODE=hw_emu
```

Data Center vs. Embedded Platforms

Emulation is supported for both data center and embedded platforms. For data center platforms, the host application is compiled for x86 server, while the device is modeled as separate x86 process emulating the hardware. The user host code and the device model process communicate using RPC calls. For embedded platforms, where the CPU code is running on the embedded Arm processor, emulation flows use QEMU (Quick Emulator) to mimic the Arm-based PS-subsystem. In QEMU, you can boot embedded Linux and run Arm binaries on the emulation targets.

For running software emulation (`sw_emu`) and hardware emulation (`hw_emu`) of a data center application, you must compile an emulation model of the accelerator card using the `emconfigutil` command and set the `XCL_EMULATION_MODE` environment variable prior to launching your application. The steps are detailed in [Running Emulation on Data Center Accelerator Cards](#).

For running `sw_emu` or `hw_emu` of an embedded application, you will compile the application for an Arm processor using Arm-GCC and launch the QEMU emulation environment on an x86 processor to model the execution environment of the Arm processor. This requires the use of the `launch_emulator.py` command, or `launch_emulator.sh` shell scripts generated during the build process. The details of this flow are explained in [Running Emulation on an Embedded Processor Platform](#).



TIP: You can also compile and run the simulation for an embedded processor application directly on the x86 processor as described in [Embedded Processor Emulation Using PS on x86](#). This compiles using x86 GCC and does not require QEMU.

QEMU

QEMU stands for Quick Emulator. It is a generic and open source machine emulator. AMD provides a customized QEMU model that mimics the Arm based processing system present on AMD Versal™ adaptive SoC, AMD Zynq™ UltraScale+™ MPSoC, and Zynq 7000 SoC. The QEMU model provides the ability to execute CPU instructions at almost real time without the need for real hardware. For more information, refer to the [QEMU User Documentation](#).

For hardware emulation, the AMD Vitis™ emulation targets use QEMU and co-simulate it with an RTL and SystemC-based model for rest of the design to provide a complete execution model of the entire platform. You can boot an embedded Linux kernel on it and run the XRT-based accelerator application. Because QEMU can execute the Arm instructions, you can take the Arm binaries and run them in emulation flows as-is without the need to recompile. QEMU also allows you to debug your application using GDB and TCF-based target connections from Xilinx System Debugger (XSDB).

The Vitis emulation flow also uses QEMU to emulate the MicroBlaze™ processor to model the platform management modules (PLM and PMU) of the devices. On Versal devices, the PLM firmware is used to load the PDI to program sections of the PS and AI Engine model.

To ensure that the QEMU configuration matches the platform, there are additional files that must be provided as part of `sw` directory of Vitis platforms. Two common files, `qemu_args.txt` and `pmc_args.txt`, contain the command line arguments to be used when launching QEMU. When you create a custom platform, these two files are automatically added to your platform with default contents. You can review the files and edit as needed to model your custom platform. Refer to an AMD embedded platform for an example.

Because QEMU is a generic model, it uses a Linux device tree style DTB formatted file to enable and configure various hardware modules. A default QEMU hardware DTB file is shipped with the Vitis tools in the `<vitis_installation>/data/emulation/dtbs` folder. However, if your platform requires a different QEMU DTB, you can package it as part of your platform.



TIP: The QEMU DTB represents the hardware configuration for QEMU, and is different from the DTB used by the Linux kernel.

Running Emulation on Data Center Accelerator Cards



TIP: Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the builds.

1. Set the desired runtime settings in the `xrt.ini` file. This step is optional.

As described in [xrt.ini File](#), the file specifies various parameters to control debugging, profiling, and message logging in XRT when running the host application and kernel execution. This enables the runtime to capture debugging and profile data as the application is running. The `Emulation` group in the `xrt.ini` provides features that affect your emulation run.



TIP: Be sure to use the `v++ -g` option when compiling your kernel code for emulation mode.

2. Create an `emconfig.json` file from the target platform as described in [emconfigutil Utility](#). This is required for running hardware or software emulation.

The emulation configuration file, `emconfig.json`, is generated from the specified platform using the `emconfigutil` command, and provides information used by the XRT library during emulation. The following example creates the `emconfig.json` file for the specified target platform:

```
emconfigutil --platform xilinx_u200_xdma_201830_2
```

In emulation mode, the runtime looks for the `emconfig.json` file at a location specified by the `$EMCONFIG_PATH` variable, or in the same directory as the host executable.



TIP: It is mandatory to have an up-to-date JSON file for running emulation on your target platform.

3. Set the `XCL_EMULATION_MODE` environment variable to `sw_emu` (software emulation) or `hw_emu` (hardware emulation) as appropriate. This changes the application execution to emulation mode.

Use the following syntax to set the environment variable for C shell (csh):

```
setenv XCL_EMULATION_MODE sw_emu
```

Bash shell:

```
export XCL_EMULATION_MODE=sw_emu
```



IMPORTANT! The emulation targets will not run if the `XCL_EMULATION_MODE` environment variable is not properly set.

4. Run the application.

With the runtime initialization file (`xrt.ini`), emulation configuration file (`emconfig.json`), and the `XCL_EMULATION_MODE` environment set, run the host executable with the desired command line argument.

For example:

```
./host.exe kernel.xclbin
```



TIP: This command line assumes that the host program is written to take the name of the `xclbin` file as an argument, as most AMD Vitis™ examples and tutorials do. However, your application might have the name of the `xclbin` file hard-coded into the host program, or might require a different approach to running the application.

Running Emulation on an Embedded Processor Platform



RECOMMENDED: The file size limit on your machine should either be set to unlimited or a higher value (over 16 GB) because embedded HW Emulation can create files with larger file size for memory.



TIP: Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the builds.

1. Set the desired runtime settings in the `xrt.ini` file.

As described in [xrt.ini File](#), the file specifies various parameters to control debugging, profiling, and message logging in XRT when running the host application and kernel execution. As described in [Enabling Profiling in Your Application](#) this enables the runtime to capture debugging and profile data as your application is running.

The `xrt.ini` file, as well as any additional files required for running the application, must be included in the output files as explained in [Packaging for Embedded Platforms](#).



TIP: Be sure to use the `++ -g` option when compiling your kernel code for emulation mode.

2. Launch the QEMU emulation environment by running the `launch_sw_emu.sh` script or `launch_hw_emu.sh` script.

```
launch_sw_emu.sh -forward-port 1440 22
```

The script is created in the emulation directory during the packaging process, and uses the `launch_emulator.py` command to setup and launch QEMU. When launching the emulation script you can also specify options for the `launch_emulator.py` command. Such as the `-forward-port` option to forward the QEMU port to an open port on the local system. This is needed when trying to copy files from QEMU as discussed in Step 5 below. Refer to [launch_emulator Utility](#) for details of the command.

Another example would be to specify `launch_hw_emu.sh -enable-debug` to configure additional XTERMs to be opened for QEMU and PL processes to observe live transcripts of command execution to aid in debugging the application. This is not enabled by default, but can be useful when needed for debug.

3. Mount and configure the QEMU shell with the required settings.

The AMD embedded base platforms have `rootfs` on a separate EXT4 partition on the SD card. After booting Linux, this partition needs to be mounted. If you are running emulation manually, you need to run the following commands from the QEMU shell:

```
mount /dev/mmcblk0p1 /mnt
cd /mnt
export LD_LIBRARY_PATH=/mnt:/tmp:$LD_LIBRARY_PATH
export XCL_EMULATION_MODE=hw_emu
export XILINX_XRT=/usr
export XILINX_VITIS=/mnt
```



TIP: You can set the `XCL_EMULATION_MODE` environment variable to `sw_emu` for software emulation, or `hw_emu` for hardware emulation. This configures the host application to run in emulation mode.

4. Run the application from within the QEMU shell.

With the runtime initialization (`xrt.ini`), the `XCL_EMULATION_MODE` environment set, run the host executable with the command line as required by the host application. For example:

```
./host.elf kernel.xclbin
```



TIP: This command line assumes that the host program is written to take the name of the `xclbin` file as an argument, as most AMD Vitis™ examples and tutorials do. However, your application can have the name of the `xclbin` file hard-coded into the host program, or can require a different approach to running the application.

5. After the application run has completed, it is possible to have files that were produced by the runtime, such as `opencl_summary.csv`, `opencl_trace.csv`, and `xrt.run_summary`. These files can be found in the `/mnt` folder inside the QEMU environment. However, to view these files you must copy them from the QEMU Linux system back to your local system.

Note: The files can be copied either from the Guest or Host machine using the `scp` command. Key terminology to notice here are *Host* and *Guest*. The *Host* machine is the machine hosting QEMU, and the *Guest* machine is the one with PetaLinux running on QEMU. `Host-ip-address` is the IP address of the machine from which QEMU is launched.

a. Copying files from the Host machine:

- i. First, copy the required files from the root area to the `petalinux` (default) user home area using the following command from the Guest machine: `cp -rf /mnt/<files> /home/petalinux/`
- ii. Switch from the default login user `petalinux` using the `exit` command
- iii. Change the permission of the copied files using `sudo chmod 755 <files>`

The files can be copied using the `scp` command from the Host machine as follows: `scp -P <port-num> <guest-machine-user-name>@<host-ip-address>:<source-file> <dest-path>`. For example: `scp -P 1440 root@172.55.12.26:/mnt/xrt.run_summary.`

<port-num>: To copy Guest machine files to the Host, the Port number through which Host and Guest are connected is required. 1440 is the QEMU port to connect to the Guest port. The `-forward-port 1440 22` is passed to the `launch_emulator` to map the Guest port to the Host port. Here, 22 is the Guest TCP port and 1440 is the Host port. Both ports are mapped. To access the Guest TCP services (on port 22), use the Host machine's mapped port (i.e. 1440). Accessing the Host's port 1440 means accessing Guest port 22.

<guest-machine-user-name> The guest machine user name can be found by using the `whoami` command from the PetaLinux terminal.

<host-ip-address>: The host IP address can be found with a linux command such as `nslookup <machine name>` or `hostname -i`

<source-file>: The path and name of the file you want to copy from the QEMU environment.

<dest-path>: The destination path to copy the file to on the local system.

b. Copying files from the Guest machine:

You can copy from the guest machine using the `scp` command: `scp <guest-file> <userid-of-hostmachine>@<host-ip-address>:<host's-dir-path>` For example: `scp /mnt/xrt.run_summary userabcd@172.55.12.26:~`

- When your application has completed emulation and you have copied the required files, press the **Ctrl + a + x** keys to terminate the QEMU shell and return to the Linux shell.

Note: If you have trouble terminating the QEMU environment, you can kill the processes it launches to run the environment. The tool reports the process IDs (pids) at the start of the transcript, or you can specify the `-pid-file` option to capture the pids when launching emulation.

Embedded Processor Emulation Using PS on x86

Running emulation under the QEMU environment for embedded processors is compute-intensive and requires additional setup and configuration. You must cross-compile the embedded application using `arm-gcc`, create an SD card image, and boot Linux under the QEMU environment before running the application. The PS on x86 feature lets you simulate your embedded system design on an x86 processor with much less effort. This feature requires you to compile the PS application using an x86 version of `gcc` or `g++` compiler, and to build your `.xclbin` file to run on an emulation platform for software emulation created by `emconfigutil`.



IMPORTANT! PS on x86 compilation is only valid for software emulation and requires GCC 8.3 or later. This feature also requires the x86 installation of XRT as described in [Installing Xilinx Runtime and Platforms](#). In addition, for AI Engine designs, you must enable the `--package.defer_aie_run` option as shown in the config file examples below.

The process for building and running software emulation using PS on x86 is as follows:

- Compile the PS application using the standard `gcc` compiler rather than the Arm cross-compiler. This is described in [Compiling and Linking for x86](#).
- Compile and link the device binary (`.xclbin`) using standard `v++` commands as described in [Building the Device Binary](#).
- For the PS on x86 flow the `v++ --package` command tells the tool what processor to use for emulation (`--package.emu_ps`).

For AMD Versal™ platforms it is also needed to convert the `.xsa` produced by the `v++ --link` command into an `.xclbin` file. This step is not required for AMD Zynq™ UltraScale+™ MPSoC based platforms.

- Use the [emconfigutil Utility](#) to create an `emconfig.json` file for software emulation.

```
emconfigutil --platform xilinx_zcu102_base_202220_1
```

- Set the `XCL_EMULATION_MODE` environment variable to `sw_emu` for software emulation mode.



IMPORTANT! The emulation targets will not run if the `XCL_EMULATION_MODE` environment variable is not properly set.

6. Run the application. For example:

```
./host.exe kernel.xclbin
```

The following are example `emconfigutil` and `v++ --package` commands. Use the `emconfigutil` to create the virtual platform needed for emulation, and the `v++ --package` command to create the system for emulation.

```
emconfigutil --platform $PLATFORM_REPO_PATHS/xilinx_vck190_base_202310_1/
xilinx_vck190_base_202310_1.xpfm --nd 1
```

followed by,

```
v++ -p -t sw_emu \
--package.defer_aie_run \
--platform $PLATFORM_REPO_PATHS/xilinx_vck190_base_202310_1/
xilinx_vck190_base_202310_1.xpfm \
--package.sd_dir $PLATFORM_REPO_PATHS/sw/versal/xrt \
--package.out_dir ./package.sw_emu \
--package.ps_on_x86 \
--package.sd_file ./emconfig.json \
../tutorial.xsa ../libadf.a
```

To run the software emulation use the following commands:

```
setenv XCL_EMULATION_MODE sw_emu
./host_ps_on_x86 a.xclbin
```

The following table describes the differences in building the PS application and device binary (`.xclbin`) for running software emulation under QEMU and for running PS on x86.

Table 20: Running SW Emulation for PS on X86

Process	Running SW_EMU under QEMU	SW_EMU with PS on X86
Host compilation	Requires cross-compilation using <code>arm-gcc</code> and requires <code>SYSROOT</code> definition, and embedded include and library files: <pre>aarch64-linux-gnu-g++ -o hello_world \ host.cpp -lxrt_coreutil -pthread \ --sysroot=<path-to-sysroot> \ -I<path-to-sysroot>/usr/include/xrt \ -L<path-to-sysroot>/usr/lib</pre>	Requires x86 <code>gcc</code> and requires x86 installation of XRT: <pre>g++ -o hello_world host.cpp \ -lxrt_coreutil -pthread \ -I\$(XILINX_XRT)/include \ -L\$(XILINX_XRT)/lib</pre>

Table 20: Running SW Emulation for PS on X86 (cont'd)

Process	Running SW_EMU under QEMU	SW_EMU with PS on X86
Package	Requires SD card image creation to run under QEMU, specifies the emulation processor to be QEMU: <pre>v++ -p -t sw_emu \$(LINK_XCLBIN) \$(LIBADF) \ --platform \$(PLATFORM_PATH) \ --package.out_dir ./package.sw_emu \ --package.emu_ps qemu \ --package.rootfs \$(EDGE_COMMON_SW)/ rootfs.ext4 \ --package.kernel_image \$(EDGE_COMMON_SW)/ Image \ --package.sd_file xrt.ini \ --package.sd_file ./run_app.sh \ -o vadd.xclbin --package.sd_file ./ hello-world</pre>	For PS on x86 simulation the <code>--package</code> command is needed to identify the PS for emulation, and to generate the <code>.xclbin</code> file: <pre>v++ -p \$(LINK_OUTPUT) \$ (VPP_FLAGS) \ --package.emu_ps x86 \ --package.defer_aie_run \ --package.out_dir \$ (PACKAGE_OUT) \ -o \$(BUILD_DIR)/ kernel.xclbin</pre>
Running Simulation	Requires use of <code>launch_emulation</code> shell script to setup QEMU and run the simulation: <pre>./launch_sw_emu.sh -run-app \ \$(RUN_APP_SCRIPT) tee run_app.log;</pre>	Set emulation mode environment variable and launch application directly: <pre>XCL_EMULATION_MODE=sw_emu ./\$(EXECUTABLE) \$ (CMD_ARGS)</pre>

You can compile the same PS code using `arm-gcc` in `x86 gcc`. However, any ARM-only data types or libraries that are linked in your PS code will not work when compiled for x86. Refer to [ps_on_x86](#) on GitHub for an example of running software emulation using this feature.



TIP: In order to compile and run AI Engine applications for PS on x86, refer to *AI Engine Tools and Flows User Guide* ([UG1076](#)) for additional information.

Speed and Accuracy of Hardware Emulation

Hardware emulation uses a mix of SystemC and RTL co-simulation to provide a balance between accuracy and speed of simulation. The SystemC models are a mix of purely functional models and performance approximate models. Hardware emulation does not mimic hardware accuracy 100%, therefore you should expect some differences in behavior between running emulation and executing your application on hardware. This can lead to significant differences in application performance, and sometimes differences in functionality can also be observed.

Functional differences with hardware typically point to a race condition or some unpredictable behavior in your design. So, an issue seen in hardware might not always be reproducible in hardware emulation, though most behavior related to interactions between the host and the accelerator, or the accelerator and the memory are reproducible in hardware emulation. This makes hardware emulation an excellent tool to debug issues with your accelerator prior to running on hardware.

The following table lists models that are used to mimic the hardware platform and their accuracy levels.

Table 21: Hardware Platform

Hardware Functionality	Description
Host to Card PCIe® Connection and DMA (XDMA, SlaveBridge)	For data center platforms, the connection to the x86 host server over PCIe is done as a purely functional model and does not have any performance modeling. Thus, any issues related to PCIe bandwidth cannot be reflected in hardware emulation runs.
AMD UltraScale™ DDR Memory, SmartConnect	The SystemC models for the DDR memory controller, AXI SmartConnect, and other data path IPs are usually throughput approximate. They typically do not model the exact latency of the hardware IP. The model can be used to gauge a relative performance trend as you modify your application or the accelerator kernel.
AI Engine	The AI Engine SystemC model is cycle approximate, though it is not intended to be 100% cycle accurate. A common model is used between AI Engine Simulator and hardware emulation, thus enabling a reasonable comparison between the two stages.
AMD Versal™ NoC and DDR Models	The Versal NoC and DDR SystemC models are cycle approximate.

Table 21: Hardware Platform (cont'd)

Hardware Functionality	Description
Arm Processing Subsystem (PS, CIPS)	The Arm PS is modeled using QEMU, which is a purely functional execution model. For more information, see QEMU .
User Kernel (accelerator)	Hardware emulation uses RTL for the user accelerator. As follows, the accelerator behavior by itself is 100% accurate. However, the accelerator is surrounded by other approximate models.
Other I/O Models	For hardware emulation, there is generic Python or C-based traffic generator which can be interfaced with the emulation process. You can generate abstract traffic at AXI protocol level which mimics the I/O in your design. Because these models are abstract, any issues observed on the specific hardware board will not be shown in hardware emulation.

Because hardware emulation uses RTL co-simulation as its execution model, the speed of execution is orders of magnitude slower as compared to real hardware. AMD recommends using small data buffers. For example, if you have a configurable vector addition and in hardware you are performing a 1024 element `vadd`, in emulation you might restrict it to 16 elements. This will enable you to test your application with the accelerator, while still completing execution in reasonable time.

Working with Simulators in Hardware Emulation

Simulator Support

The AMD Vitis™ tool uses the AMD Vivado™ logic simulator (`xsim`) as the default simulator for all platforms, including AMD Alveo™ Data Center accelerator cards, and AMD Versal™ and AMD Zynq™ UltraScale+™ MPSoC embedded platforms. However, for Versal embedded platforms, like `xilinx_vck190_base` or custom platforms similar to it, the Vitis tool also supports the use of third-party simulators for hardware emulation: Mentor Graphics Questa Advanced Simulator, Xcelium, and VCS. The specific versions of the supported simulators are the same as the versions supported by Vivado Design Suite.



TIP: For data center platforms, hardware emulation supports the U250_XDMA platform with Questa Advanced Simulator. This support does not include features like peer-to-peer (P2P), SlaveBridge, or other features unless explicitly mentioned.

Enabling a third-party simulator requires some additional configuration options to be implemented during generation of the device binary (`.xclbin`) and supporting Tcl scripts. The specific requirements for each simulator is discussed below. In addition, you should run the Vivado setup for third-party simulators before using those simulators in Vitis. Specifically, you must pre-compile the simulation models using the `compile_sim_lib` Tcl command. For more details, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)) for third-party simulator setup.

- **Questa:** Add the following advanced parameters and Vivado properties to a configuration file for use during linking:

```
## Final set of additional options required for running simulation using
Questa Simulator
[advanced]
param=hw_emu.simulator=QUESTA
[vivado]
prop=project.__CURRENT__.simulator.questa_install_dir=<Questa_install_dir>
prop=project.__CURRENT__.compplib.questa_pre-
compiled_library_dir=<Questa_compiled_lib_path>
prop=fileset.sim_1.questa.compile.sccom.cores={16}
```

After generating the configuration file you can use it in the `v++` command line as follows:

```
v++ -link --config questa_sim.cfg
```

- **Xcelium:** Add the following advanced parameters and Vivado properties to a configuration file for use during linking:

```
## Final set of additional options required for running simulation using
Xcelium Simulator
[advanced]
param=hw_emu.simulator=XCELIUM
[vivado]
prop=project.__CURRENT__.simulator.xcelium_install_dir=<Xcelium_install_dir>
prop=project.__CURRENT__.compplib.xcelium_compiled_library_dir=<Xcelium_pre-compiled_lib_path>
prop=fileset.sim_1.xcelium.elaborate.xmelab.more_options={-timescale 1ns/1ps -STATUS}
```

After generating the configuration file you can use it in the `v++` command line as follows:

```
v++ -link --config xcelium.cfg
```

- **VCS:** Add the following advanced parameters and Vivado properties to a configuration file for use during linking:

```
## Final set of additional options required for running simulation using
VCS Simulator
[advanced]
param=hw_emu.simulator=VCS
[vivado]
prop=project.__CURRENT__.simulator.vcs_install_dir=<VCS_install_dir>
prop=project.__CURRENT__.compplib.vcs_compiled_library_dir=<Vcs_pre-compiled_lib_path>
prop=project.__CURRENT__.simulator.vcs_gcc_install_dir=<VCS_gnu_pkg_install_dir>
param=project.alignLibraryPathEnvForVCS=true
prop=fileset.sim_1.vcs.compile.vlogan.more_options={-v2005}
```

After generating the configuration file you can use it in the `v++` command line as follows:

```
v++ -link --config vcs_sim.cfg
```

- **Riviera:** Add the following advanced parameters and Vivado properties to a configuration file for use during linking:

```
## Final set of additional options required for running simulation using
VCS Simulator
[advanced]
param=hw_emu.simulator=RIVIERA
[vivado]
prop=project.__CURRENT__.simulator.riviera_install_dir=<Riviera_install_dir>
```

```
prop=project.__CURRENT__.complib.riviera_compiled_library_dir=<Riviera_pre-compiled_lib_path>
prop=project.__CURRENT__.simulator.riviera_gcc_install_dir=<Riviera_gcc_path>
prop=fileset.sim_1.riviera.simulate.asim.more_options={+access +r}
```

After generating the configuration file you can use it in the `v++` command line as follows:

```
v++ -link --config riviera.cfg
```

You can use the `-user-pre-sim-script` and `-user-post-sim-script` options from the `launch_emulator.py` command to specify Tcl scripts to run before the start of simulation, or after simulation completes. As an example, in these scripts, you can use the `$cwd` command to get the run directory of the simulator and copy any files needed prior to simulation, or copy any output files generated at the end of simulation.

To enable hardware emulation, you must set up the environment for simulation in the Vivado Design Suite. A key step for setup is pre-compiling the RTL and SystemC models for use with the simulator. To do this, you must run the `compile_sim_lib` command in the Vivado tool. For more information on pre-compilation of simulation models, refer to the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

When creating your Versal platform ready for simulation, the Vivado tool generates a simulation wrapper which must be instantiated in your simulation test bench. So, if the top most design module is `<top>`, then when calling `launch_simulation` in the Vivado tool, it will generate a `<top>_sim_wrapper` module, and also generates `xlnoc.bd`. These files are generated as simulation-only sources and will be overwritten whenever `launch_simulation` is called in the Vivado tool. Platform developers need to instantiate this module in the test bench and not their own `<top>` module.

Using the Simulator Waveform Viewer

Hardware emulation uses RTL and SystemC models for execution. A regular application and HLS-based kernel developer does not need to be aware of the hardware level details. The Vitis analyzer provides sufficient details of the hardware execution model. However, for advanced users who are familiar with HW signal and protocols, they can launch hardware emulation with the simulator waveform running, as described in [Waveform View and Live Waveform Viewer](#).

By default, when running `v++ -link -t hw_emu`, the tool compiles the simulation models in optimized mode. However, when you also specify the `-g` switch, you enable hardware emulation models to be compiled in debug mode. During the application runtime, use the `-g` switch with the `launch_hw_emu.sh` command to run the simulator interactively in GUI mode with waveforms displayed. By default, the hardware emulation flow adds common signals of interest to the waveform window. However, you can pause the simulator to add signals of interest and resume simulation.

AXI Transactions Display in XSIM Waveform

Many models in hardware emulation use SystemC transaction-level modeling (TLM). In these cases, interactions between the models cannot be viewed as RTL waveforms. However, Vivado simulator (xsim) provides a transaction level viewer. For standard platforms, these interface objects can be added to the waveform view, similar to how RTL signals are added. As an example, to add an AXI interface to the waveform, use the following Tcl command in xsim:

```
add_wave <HDL_objects>
```

Using the `add_wave` command, you can specify full or relative paths to HDL objects. For additional details on how to interpret the TLM waveform see [Interpreting TLM Waveform Data for Third-Party Simulators](#), or refer to *Vivado Design Suite User Guide: Logic Simulation (UG900)*.

Generating Test Vectors for Vitis HLS during Hardware Emulation

Now it is possible to instruct the Vitis tool to generate test vectors for simulation during hardware emulation, without re-running `v++` compilation and linking. The test vectors will enable Vitis HLS to run C/RTL Co-simulation without a dedicated C++ test bench for:

- deadlock analysis
- FIFO depth optimization
- other performance optimizations

Use the following steps:

1. Create an `hlsPre.tcl` file and insert this command:

```
config_export -cosim_trace_generation
```

2. Run `v++ --compile` and keep the HLS project directories, under the `<compile_dir>`
3. Run `v++ --link --target hw_emu`
4. Run the application for hardware emulation
5. Locate the `hls_cosim` inside the `HW_EMU` run directory
 - a. This directory contains one directory for each kernel, with one directory for each kernel CU (kernel instance) below it:

```
<build_dir>/run/<run_number>/hw_em/device0/binary_0/behav_waveform/  
xsim/hls_cosim/<kernel_name>
```


6. Copy the appropriate kernel directory to the HLS project directory, i.e.:

```
cp -r <build_dir>/run/<run_number>/../xsim/hls_cosim/<kernel_name>  
<compile_dir>/<kernel_name>/<kernel_name>
```

7. Open the Vitis HLS tool and run C/RTL Co-simulation in batch mode or GUI mode:

```
cosim_design -hwemu_trace_dir <kernel_name>/<instance_name> ...
```

The traces generated from HW_EMU are valid only as long as:

- The functionality of the kernel does not change
- The top interface of the kernel does not change
- The number of top interface reads and writes (`s_axilite` registers, `m_axi` interfaces, `axis` interfaces) does not change.

Working with Functional Model of the HLS Kernel

Using a functional model of the AMD Vitis™ HLS kernel during hardware emulation is an advanced use case that enables compilation of kernels in functional mode that generates the XO with a SystemC wrapper around the C code. An IP, whether generated by the HLS compiler or not, can have multiple types of simulation models, such as TLM and RTL, as indicated by the `allowed_sim_models` property. However, the IP needs to indicate which of these models is the current model as defined by the `selected_sim_model` property. The method described here lets you specify which type of sim model you want to be selected when the HLS compiler generates the output IP or XO.

HW Emulation is mainly targeted for hardware kernel debug with detailed, cycle-accurate view of kernel activity. The functional (TLM) model speeds up emulation by compiling the kernel of interest in functional mode rather than as RTL code, and can be used as an early model when the RTL is not yet available. This provides faster compile time for the kernel as it does not need full C to RTL synthesis, and faster execution time as C-code is simulated instead of RTL simulation. You can also mix and match of C and RTL kernels in hardware emulation for faster debug of RTL blocks.

The functional model feature supports modeling AXI4-Stream interfaces (`axis`) and AXI4 memory-mapped interfaces (`m_axi`), in addition to register reads and writes of the AXI4-Lite (`s_axilite`) interfaces. However, with this approach, the kernel will be purely functional without latency information, unlike cycle-accurate models.

The user HLS function is wrapped into a SystemC module with TLM interfaces and IP is created out of the generated code which will allow generating HW_EMU compatible XO that can be used in IP integrator for stitching v++ link designs in HW Emulation flows. This also allows the Wrapper IP to talk to other RTL and SystemC models. So, the HLS C/C++ kernels compiled in functional mode will have TLM transactions during simulation and users can see traffic between the memory models (for example DDR memory) and the TLM kernels.



TIP: The functional model uses C-code performing C-simulation for the kernel. The kernel will be purely functional without any latency information unlike cycle-accurate models. Although, you can see boundary transactions via TLM interfaces during HW Emulation.

XO Generation with Functional Model



IMPORTANT! The `v++ -c --mode hls` command does not support the functional simulation model as described below. To generate an IP or kernel to use the TLM model for functional simulation you must use the `v++ -c -t=hw_emu` command.

During the `v++ -c -t=hw_emu` compile step, while creating the hardware emulation (`hw_emu`) XO files, you can provide an option enabling a functional simulation model for the PL kernel that will generate the XO with a SystemC wrapper on the C code. You need to provide the `--advanced.param compiler.emulationMode=func` option during compilation, as described in [--advanced Options](#).

The default setting for this is `compiler.emulationMode=rtl`. When building the XO you can either provide the default value using `--advanced.param compiler.emulationMode=rtl` so you can simply toggle between RTL and TLM models for a specific XO; or you can remove the `--advanced.param` command to restore the default value and add it back when building for functional simulation. In either case, if you want to change the model from RTL to TLM or back, you must recompile the XO using the `v++ -c -t=hw_emu` command.

The generated functional simulation XO is linked using the `v++ --link` command like the regular XO. For an example refer to [mm_stream_func_mode](#) on GitHub.

Limitations of the Functional Model

1. The functional mode is not supported on Windows OS.
2. Limitations in HLS are applied "as is." For example, HLS does not support double pointers so the functional model does not identify it.
3. HLS designs which operate on multiple data iteration from host with single kernel `ap_start` (for example `ap_ctrl_chain`) might not operate if the restart is triggered from the kernel code. Mailboxing works fine.
4. Application Binary Interface (ABI) changes for FPGA are not available in Functional Mode x86 ABI. For most optimizations where is ABI is used, they need to be disabled in functional compiler.
5. Limiting DDR Analysis by Casting/Inter procedural uses:
 - a. Typecasting DDR memory pointers from scalars will not work.

```
kernel void vadd(size_t a_s, size_t b_s, size_t c){
    int* a = (size_t)a;
    int* b = (size_t)b;
    int* c = (size_t)c;
    for(int i=0; i < 64; i++){
        c[i] = a[i] + b[i];
    }
}
```

- b. Caching DDR memory pointers across procedural context will not work.

```
class Cache{
int* local;
Cache(int *a) : local(a){}
int read(){
void write(int x){}
};
kernel void vadd(int *a,int *b, int *c){
    Cache ca(a);
    for(int i=0; i < 64; i++){
        c[i] = ca.read() + b[i];
    }
}
```

6. HLS features implemented in binary and consuming DDR memory access are not supported and require functional rewrite.
7. Burst transactions are not automatically detected in the functional model.

Coding guidelines for working with functional models: For kernel compute units that run multiple times and expect static value reset to zero in each iteration you must initialize all static variables at the entry of the kernel function. The following example shows code that returns an error and also demonstrates the recommended approach:

```
// User code that errors out
static int i = 0;
void hls_kernel_logic(...) {
    ...
}
// Recommended
static int i = 0;
void hls_kernel_logic(...) {
    i = 0;
    ...
}
```

Working with SystemC Models

SystemC models in the AMD Vitis™ application acceleration development flow allow you to quickly model an RTL algorithm for rapid analysis in software and hardware emulation. Using this approach you can model portions of your system while the RTL kernel is still in development, but you want to move forward with some system analysis.

The SystemC model feature supports all the XRT-managed kernel execution models using `ap_ctrl_hs` and `ap_ctrl_chain`. It also supports modeling both AXI4 memory mapped interfaces (`m_axi`) and AXI4-Stream interfaces (`axis`), in addition to register reads and write of the `s_axilite` interface.

You can model your kernel code in SystemC TLM models, provide interfaces to other kernels and the host application, and use it during emulation. You can create a Xilinx object file (XO) to link the SystemC model to other kernels in your `xclbin`. The sections that follow discuss the creation of SystemC models, the use of the `create_sc_xo` command to create the XO, and generating the `xclbin` using the `v++` command.



TIP: Keep in mind that the SystemC model is not cycle accurate, and therefore impacts the timing results of your emulation. It does not reflect the true bandwidth, latency, or throughput of the RTL code.

Coding a SystemC Model

The process for defining a SystemC model uses the following steps:

1. Include header files `"xtlm_ap_ctrl.h"` and `"xtlm.h"`.
2. Derive your kernel from a predefined class based on the supported kernel types: `ap_ctrl_chain`, `ap_ctrl_hs`, etc.
3. Declare and define the AXI interfaces used on your kernel.
4. Add required kernel arguments with the correct address offset and size.
5. Write the kernel body in `main()` thread.

This process is demonstrated in the code example below.

When creating the SystemC model, you derive the kernel from a class-based on a supported control protocol: `xtlm_ap_ctrl_chain`, `xtlm_ap_ctrl_hs`, and `xtlm_ap_ctrl_none`. Use the following structure to create your SystemC model.



TIP: The following example is based on the simple vector addition (VADD) example design found in the [Vitis_Accel_Examples](#) on GitHub.

```
#include "xtlm.h"
#include "xtlm_ap_ctrl.h"

class vadd : public xsc::xtlm_ap_ctrl_hs
{
public:
    SC_HAS_PROCESS(vadd);
    vadd(sc_module_name name, xsc::common_cpp::properties& _properties):
        xsc::xtlm_ap_ctrl_hs(name)
    {
        DEFINE_XTLM_AXIMM_MASTER_IF(in1, 32);
        DEFINE_XTLM_AXIMM_MASTER_IF(in2, 32);
        DEFINE_XTLM_AXIMM_MASTER_IF(out_r, 32);

        ADD_MEMORY_IF_ARG(in1, 0x10, 0x8);
        ADD_MEMORY_IF_ARG(in2, 0x18, 0x8);
        ADD_MEMORY_IF_ARG(out_r, 0x20, 0x8);
        ADD_SCALAR_ARG(size, 0x28, 0x4);

        SC_THREAD(main_thread);
    }

    //! Declare aximm interfaces..
    DECLARE_XTLM_AXIMM_MASTER_IF(in1);
    DECLARE_XTLM_AXIMM_MASTER_IF(in2);
    DECLARE_XTLM_AXIMM_MASTER_IF(out_r);

    //! Declare scalar args...
    unsigned int size;

    void main_thread()
    {
        wait(ev_ap_start); //! Wait on ap_start event...

        //! Copy kernel args configured by host...
        uint64_t in1_base_addr = kernel_args[0];
        uint64_t in2_base_addr = kernel_args[1];
        uint64_t out_r_base_addr = kernel_args[2];
        size = kernel_args[3];

        unsigned data1, data2, data_r;
        for(int i = 0; i < size; i++) {
            in1->read(in1_base_addr + (i*4), (unsigned
char*)&data1); //! Read from in1 interface
            in2->read(in2_base_addr + (i*4), (unsigned
char*)&data2); //! Read from in2 interface

            //! Add data1 & data2 and write back result
            data_r = data1 + data2; //! Add
            out_r->write(out_r_base_addr + (i*4), (unsigned
char*)&data_r); //! Write the result
        }
    }
}
```

```

        ap_done(); //! completed Kernel computation...
    }

};

```

The include files are available in the Vitis installation hierarchy under the `$XILINX_Vivado/data/systemc/simlibs/` folder.

The kernel name is specified when defining the class for the SystemC model, as shown above, inheriting from the `xtlm_ap_ctrl_hs` class.

You must declare and define the AXI interfaces associated with the kernel arguments as shown by the following constructs:

```

DECLARE_XTLM_AXIMM_MASTER_IF(in1);
DEFINE_XTLM_AXIMM_MASTER_IF(in1, 32);

```

The declaration associates the interface type with the argument. The definition defines the data width of the interface. You must also declare the register offsets and size for the kernel arguments as shown in the following:

```

ADD_MEMORY_IF_ARG(in1, 0x10, 0x8);

```

When specifying the kernel arguments, offsets, and size, these values should match the values reflected in the AXI4-Lite interface of the XRT-managed kernel as described in [SW-Controllable Kernels](#) and [Control Requirements for XRT-Managed Kernels](#).

The addresses below `0x10` are reserved for use by XRT for managing kernel execution. Kernel arguments can be specified from `0x10` onwards. Most importantly the arguments, offsets, and size specified in the SystemC model should match the values used in the Vitis HLS or RTL kernel.

The kernel is executed in the SystemC `main_thread`. This thread waits until the `ap_start` bit is set by the host application, or XRT, at which time the kernel can process argument values as shown:

1. The kernel waits for a signal to begin from XRT (`ev_ap_start`).
2. Kernel arguments are mapped to variables in the SystemC model.
3. The inputs are read.
4. The vectors are added and the result is captured.
5. The output is written back to the host.
6. The finished signal is sent to XRT (`ap_done`).

Creating the XO

To generate XO file from the SystemC model, use the `create_sc_xo` command. This takes the SystemC kernel source file as input and creates IP that generates the XO, which can be used for linking with the target platform and other kernels with the Vitis compiler. For example:

```
create_sc_xo vadd.cpp
```

Generating an XO file from the source file involves a number of intermediate steps like generating a Package IP script, and running the `package_xo` command. These intermediate commands can be used for debugging if necessary.

The output of the above `create_sc_xo` command is `vadd.xo`.

Linking with the v++ Command

Link the SystemC model XO file by adding the kernel to the `v++ --link` command line:

```
v++ --link --platform <platform> --target hw_emu \  
--config ./vadd.cfg --input_files ../vadd.xo --output ../vadd.link.xclbin \  
--optimize 0 --save-temps --temp_dir ./hw_emu
```

The SystemC model can be used for both software emulation and hardware emulation, but is not supported for hardware build targets.

When a SystemC model is included in the `xclbin`, the design is no longer clock cycle accurate due to the limitations of the TLM.

Xilinx TLM – SystemC Library for ESL Modelling

Xilinx TLM (XTLM) is an AMD extension of Accellera SystemC TLM 2.0 library for modeling AXI protocols. It provides simulation infrastructure between SystemC to SystemC using custom TLM sockets and also provides the co-simulation infrastructure between RTL and SystemC through Transactors. Though TLM 2.0 consists of sockets, payload, and phases, these default elements are not sufficient to support the full functionality of AXI. Therefore, XTLM has defined its own set of sockets, payload, and phases.

A user of XTLM should be familiar with the basics of SystemC and the TLM 2.0 specifications. The following table provides a brief introduction to classes available inside the XTLM library.

Table 22: XTLM Features and Usage

S. No.	XTLM Feature/Classes	Usage
1	aximm_payload	Transaction object class to describe data over the AXI4 memory map bus in a single transaction. Based on TLM 2.0 generic payload but not derived from.
2	axis_payload	Transaction object class to describe data over the AXI4-Stream bus in a single transaction. Based on TLM 2.0 generic payload but not derived from it.
3	xtlm_aximm_initiator_socket	Basic socket to be used for AXI4 memory map interface in master/initiator. One socket shall be instantiated for each READ and WRITE socket.
4	xtlm_aximm_simple_initiator_socket_tagged	Basic socket to be used for AXI4 memory map interface in master/initiator to bind to multiple interfaces. One socket shall be instantiated for each READ and WRITE socket. Each interface has to be added with new ID.
5	xtlm_aximm_target_socket	Basic socket to be used for AXI4 memory map interface in slave/target. One socket shall be instantiated for each READ and WRITE socket.
6	xtlm_aximm_passthrough_target_socket_tagged	Basic socket to be used for AXI4 memory map interface in slave/target to bind to multiple interfaces. One socket shall be instantiated for each READ and WRITE socket. Each interface has to be added with new ID.
7	xtlm_axis_initiator_socket	Basic socket to be used for AXI4-Stream interface in master/initiator.
8	xtlm_axis_simple_initiator_socket_tagged	Basic socket to be used for AXI4-Stream interface in master/initiator to bind to multiple interfaces. Each interface has to be added with new ID.
9	xtlm_axis_target_socket	Basic socket to be used for AXI4-Stream interface in slave/target.
10	xtlm_axis_passthrough_target_socket_tagged	Basic socket to be used for AXI4-Stream interface in slave/target to bind to multiple interfaces. Each interface has to be added with new ID.
11	xtlm_aximm_initiator_stub	One Initiator socket to stub XTLM slave interface. This socket is useful where there is no XTLM master. This avoids port binding errors in SystemC.
12	xtlm_aximm_target_stub	One Target socket to stub XTLM master interface. This socket is useful where there is no XTLM slave. This avoids port binding errors in SystemC
13	xtlm_axis_initiator_stub	One Initiator socket to stub XTLM AXI4-Stream slave interface. This socket is useful where there is no XTLM AXI4-Stream master. This avoids port binding errors in SystemC.
14	xtlm_aximm_target_stub	One Target socket to stub XTLM master interface. This socket is useful where there is no XTLM slave. This avoids port binding errors in SystemC.

Table 22: XTLM Features and Usage (cont'd)

S. No.	XTLM Feature/Classes	Usage
15	xtlm_aximm_initiator_rd_socket_util	Utility for AXI4 Memory Map Initiator Read Socket. Works only with xtlm_aximm_initiator_socket type.
16	xtlm_aximm_initiator_wr_socket_util	Utility for AXI4 Memory Map Initiator Write Socket. Works only with xtlm_aximm_initiator_socket type.
17	xtlm_aximm_target_rd_socket_util	Utility for AXI4 Memory Map Target Read Socket. Works only with xtlm_aximm_target_socket type.
18	xtlm_aximm_target_wr_socket_util	Utility for AXI4 Memory Map Target Write Socket. Works only with xtlm_aximm_target_socket type.

Debug Techniques in Hardware Emulation

The following table shows some specific techniques for debugging different situations in hardware emulation.

Table 23: Techniques for Debugging in Hardware Emulation

Debug Focus	Description	Steps
x86 host (XRT)	Enable detailed XRT logs by updating <code>xrt.ini</code> .	Add the following to <code>xrt.ini</code> file. Run the test case with the updated <code>xrt.ini</code> . Review the generated <code>xrt_hal.log</code> . <pre>[runtime]hal_log=xrt_hal.log</pre>
AXI traffic on SystemC models like AI Engine, NOC, CIPS	The host transactions are routed through <code>sim_qdma</code> SystemC module. These transactions can be dumped into log file. If PL is also SystemC, then even PL transactions can be viewed there. If PL is RTL, then boundary of PL can be viewed in waveform.	In <code>xrt.ini</code> add, <pre>[Emulation]xtlm_aximm_log=true xtlm_axis_log=true</pre> View file <code>xsc_report.log</code> .
Viewing DDR memory content	The DDR model saves its contents in a binary file. In the folder where simulation is run (<code>package.hw_emu/sim/behav_waveform/xsim dir</code>), look for files named like below. Each such binary file corresponds to a region of DDR memory at a particular offset. <code>qemu-memory-_ddr@0x00000000 or qemu-memory- _mem_0xc080000000@0xc08000000 ULL</code> These files represent DDR/LPDDR memory contents in binary form. There is a backdoor connection between QEMU to NOC_DDR model. Thus, users will not see any transactions in the waveform nor will they see any logs. The shared memory is directly updated. To view memory contents, users can directly dump the memory contents.	The contents of the memory can be seen by using <code>hexdump</code> command. an example command is shown below. <pre>hexdump qemu-memory- _mem_0x60000000000@0x60000000 00ULL -s 0x80000000 -n 4096 - v -e '1/4 "%02X" "\n" > dump_600.log</pre>

Table 23: Techniques for Debugging in Hardware Emulation (cont'd)

Debug Focus	Description	Steps
PS (QEMU)	During firmware and software execution, the Arm APU can run into issues. You see details of various transactions and state on PS using these mechanisms.	1. Setenv variable <code>ENABLE_RP_LOGS=true</code> in the shell where QEMU is being run. Look for a log file named <code>sim/behav_waveform/xsim/qemu_rp.log</code>. This contains transactions from PS to rest of the peripherals (other than DDR). 2. Review <code>package.hw_emu/qemu_output.log</code> to see the output. This contains PS output to STDOUT (UART). If there is any error during PLM or other SW execution, those will be captured here. Look at the details of the error (CDO address, U-Boot stage etc) to narrow down which CDO is causing the error. 3. Run <code>launch_hw_emu.sh -enable_debug</code> mode which will direct QEMU logs in its own xterm window.
AI Engine	PDI is downloaded from PS to AI Engine through fast mode, thus transactions cannot be seen in the waveform. Users can enable logging all transactions at AI Engine AXI interfaces. There is a separate file per AXI interface	1. Enable setenv <code>ENABLE_AIE_DBG_TRACE</code>. View transaction log in the simulation folder at <code>aie_log/S00_AXI.txt</code> file. 2. Enable AI Engine VCD Dump and view contents in Vitis Analyzer by adding switch <code>-aie-sim-options</code> to <code>launch_hw_emu.sh</code> cmd line. See UG1076 for details of AI Engine-sim-options file. These are common between <code>aiesim</code> and <code>hw_emu</code>. 3. Create a file (<code>aie_sim_config.txt</code>) to enable AI Engine simulation options. a. In this file, add <code>"AIE_DEBUG_AXIMM=True"</code> b. Pass this file on launch emulator cmd line: <pre>./package.hw_emu/ launch_hw_emu.sh -aie-sim- options <Absolute path to the options .txt file></pre>

Table 23: Techniques for Debugging in Hardware Emulation (cont'd)

Debug Focus	Description	Steps
Dumping Waveform in DC flow (Batch Mode)	Make sure the design is linked with the <code>-g</code> option and you have run the design at least once in Xsim GUI mode to save the required <code>.wcfg</code> file and save the list of relevant signals. Dump the waveform and open it in xsim along with saved <code>.wcfg</code> file	Update following options in <code>xrt.ini</code> option <pre>[Emulation] user_pre_sim_script=<use pre sim script absolute path></pre> Pre-simulation script is needed to enable signal dumping: <pre>log_wave -r *</pre>

Working with I/O Traffic Generators

Some user applications such as video streaming and Ethernet-based applications make use of I/O ports on the platform to stream data into and out of the platform. For these applications, performing software and hardware emulation of the design, or running AI Engine simulation, requires a mechanism to mimic the hardware behavior of the I/O port, and to simulate data traffic running through the ports. I/O traffic generators let you model traffic through the I/O ports during software and hardware emulation in the AMD Vitis™ application acceleration development flow, during the AI Engine simulation flows (x86sim, AI Enginesim), or during logic simulation in the AMD Vivado™ Design Suite.



IMPORTANT! *Software emulation supports only AXI4-Stream I/O emulation and is not supported on AMD Zynq™ 7000 devices. Hardware emulation supports both AXI4-Stream and AXI4 memory map interface I/O emulation.*

As described in [AXI4-Stream I/O Model for Streaming Traffic](#), traffic generators can be written in Python, MATLAB, C/C++, or RTL (Verilog/SV) modules. They are launched in an external process which communicates with Vitis Emulation process or AI Engine simulation process using Inter Process Communication (IPC). The IPC connections are established using IPC AXI4-Stream master/slave modules as described in [Running Traffic Generators in Python/C++/MATLAB](#).

Traffic generators are designed to pass data into your system, or receive data from your system. The APIs are provided to handle data transfer for standard datatypes, or for more complex datatypes. The API you use to create the traffic generator is determined by the datatype your system requires. For instance, standard datatypes are covered by simple API such as `send_data`, or `receive_data` as described in [Python API for AI Engine Graphs](#) or [Python API for PL Kernels](#). More complex datatypes require API like those described in [General Purpose Python API](#) or [Advanced Python Traffic Generator API](#).

The following are additional details on ways to integrate traffic generators in Python/C/C++/Verilog with subsequent PL kernels or the AI Engine kernels based on your application.

Adding Traffic Generators to Your Design

AXI Traffic Generator kernels provide a method to inject traffic onto the I/O of your system design, AI Engine graph, or PL kernels during simulation. AMD provides a library that enables interfacing AXI4-Stream to mimic a streaming data flow for software and hardware emulation, and AXI4 memory mapped interface to mimic memory mapped data transfers for hardware emulation.

The AXI Traffic generators are provided as XO files which can be linked into your System project using the Vitis compiler (v++). These XO files are called `sim_ipc_axis_master_XY.xo` and `sim_ipc_axis_slave_ZW.xo` where XY and ZW correspond to the number of bits in the PLIO interface. For example `sim_ipc_axis_master_128.xo` provides an AXI4-Stream master data bus that is 128 bits wide. A wider interface allows the PL to achieve the same throughput at a lower clock frequency and allows the AI Engine array to maximize its memory bandwidth. However, the PLIO interface tiles are each 64 bits wide and they are a limited resource. Using one 64-bit PLIO interface at twice the clock speed provides an equivalent bandwidth to a 128-bit PLIO while using only one PLIO tile. This requires the PL to run at twice the clock speed and the optimal choice will vary from application to application.

Two steps are required to use the traffic generators in your system design:

1. Specify the connections between the traffic generator (`sim_ipc`) modules and their corresponding AXI4-Stream ports on the AI Engine array. This is typically done in the `system.cfg` file using the `--connectivity.nk` and `--connectivity.sc` commands, as described in [Linking the System](#). The following is an example:

```
[connectivity]
nk=sim_ipc_axis_master:1:inst_sim_ipc_axis_master
nk=sim_ipc_axis_slave:1:inst_sim_ipc_axis_slave
stream_connect=sim_ipc_axis_master.M00_AXIS:ai_engine_0.DataIn
stream_connect=ai_engine_0.DataOut:sim_ipc_axis_slave.S00_AXIS
```

The syntax for connecting the `sim_ipc_axis` XO files is as follows.

```
nk=sim_ipc_axis_master:<Number Of
Masters>:<inst_name_1>.<inst_name_2>.<...>
nk=sim_ipc_axis_slave:<Number Of
Slaves>:<inst_name_1>.<inst_name_2>.<...>
```

Where:

- The `sim_ipc_axis_master/slave` specifies the XO kernel in your design
- The `<Number Of Masters>` or `<Number Of Slaves>` field lets you specify up to 8 different traffic generator kernels in your design
- The `<inst_name>` should be meaningful in your application

2. Next, add the XO files to the Vitis link command as shown below.



IMPORTANT! The traffic generator XO can only be used in hardware emulation with `hw_emu` target.

```
v++ -l --platform <platform.xpfm> sim_ipc_axis_master_128.xo  
sim_ipc_axis_slave_128.xo libadf.a -target hw_emu --config system.cfg
```

For additional information on how to use XO files with the Vitis compiler see [Building the Device Binary](#).

Using Traffic Generators for AI Engine Designs

Overview

This section describes how to provide input and capture the output from the AI Engine array in all simulation and emulation modes using AXI traffic generators. In the AI Engine simulator the input data stimulus is provided using the PLIO object which specifies a text file containing the data:

```
input_plio plin = input_plio::create("DataIn", adf::plio_32_bits, "data/  
input.txt");
```

While this is really fast to get your first simulation in place, the main limitation of this approach is that if you want to change the input file name for another simulation, you have to recompile the entire application. That's why there is a possibility to avoid file name specification and rely on independent External Traffic Generator to generate data traffic on the PLIO:

```
input_plio plin = input_plio::create("DataIn", adf::plio_32_bits);
```

For hardware and software emulation an equivalent feature exists that emulates the behavior of this PLIO and AXI4-Stream interface. Both Python and C++ APIs are provided to create these External Traffic Generators that will be connected seamlessly on any of these simulation or emulation modes.

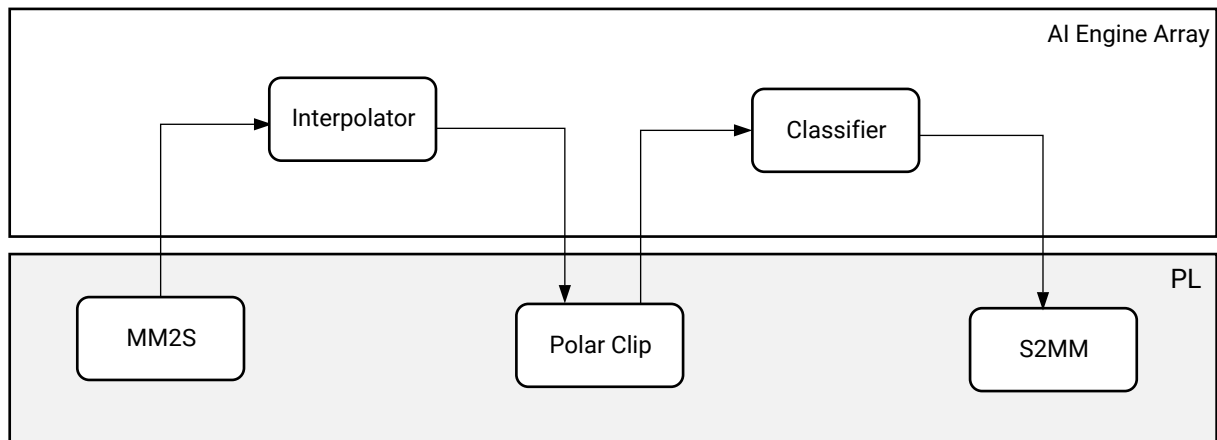
The primary external data interfaces for the AI Engine array are AXI4-Stream interfaces. These are known as PLIOs and allow the AI Engine to receive data, operate on the data, and send data back on a separate AXI4-Stream interface. The input interface to the AI Engine is an AXI4-Stream consumer and the output is an AXI4-Stream producer. To interact with these top level interfaces during hardware and software emulation complementary AXI4-Stream modules are provided. These complementary modules are referred to as the AXI traffic generators.



TIP: The width of a PLIO interface is an important system level design decision. The wider the interface the more data can be sent per PL clock cycle.

When you develop an AI Engine application you can test it standalone either in simulation (`x86simulator`, `aiesimulator`), or test it as part of a system project in emulation (`sw_emu`, `hw_emu`). In either case you need to send the input data from a predefined reference file, and capture the output data in a separate file. Furthermore, if your AI Engine graph is intertwined with kernels which are located in the Programmable Logic (HLS C++ or RTL) then you also have to deal with these data flow interruptions. For example, a full system design might look like the following figure:

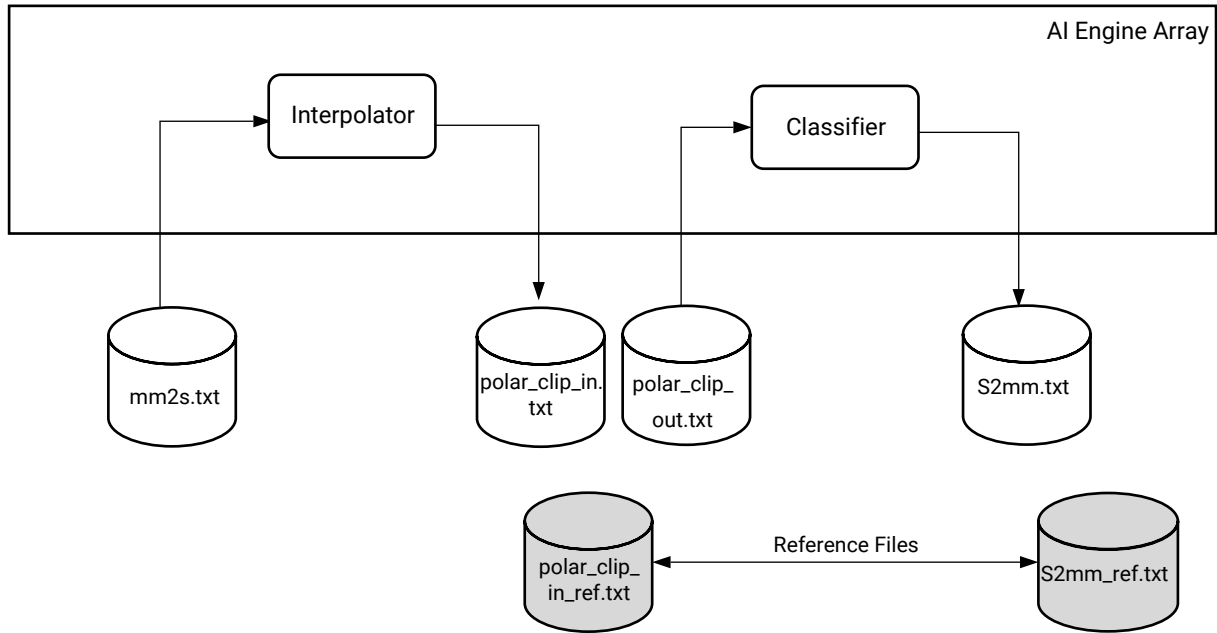
Figure 37: AI Engine + Programmable Logic Application



X26421-041122

In a first step you replace all the connections which are not in the AI Engine array with text files to provide input data or capture output data:

Figure 38: Initial Simulation Framework



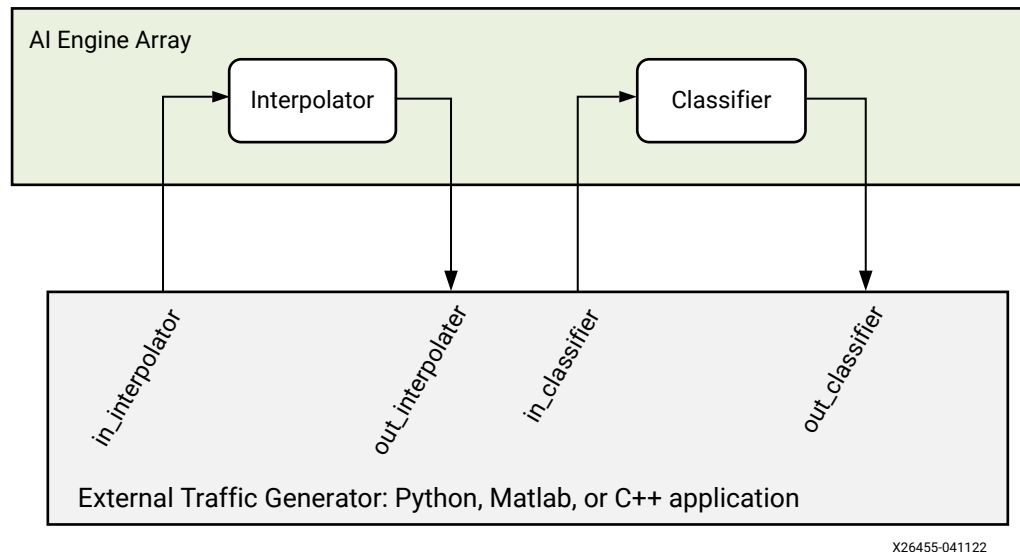
X26422-041122

For more flexibility in data generation and verification you can exchange the text files with external traffic generators which enable dynamic simulated communication between the PL and the AI Engine array through AXI4-Stream TLM connected to Unix sockets. The power of these external traffic generator is that they can be used in all simulation/emulation framework without modification:

- x86 Simulation
- AI Engine simulation
- SW Emulation
- HW Emulation

The overall simulation framework would look like the following figure:

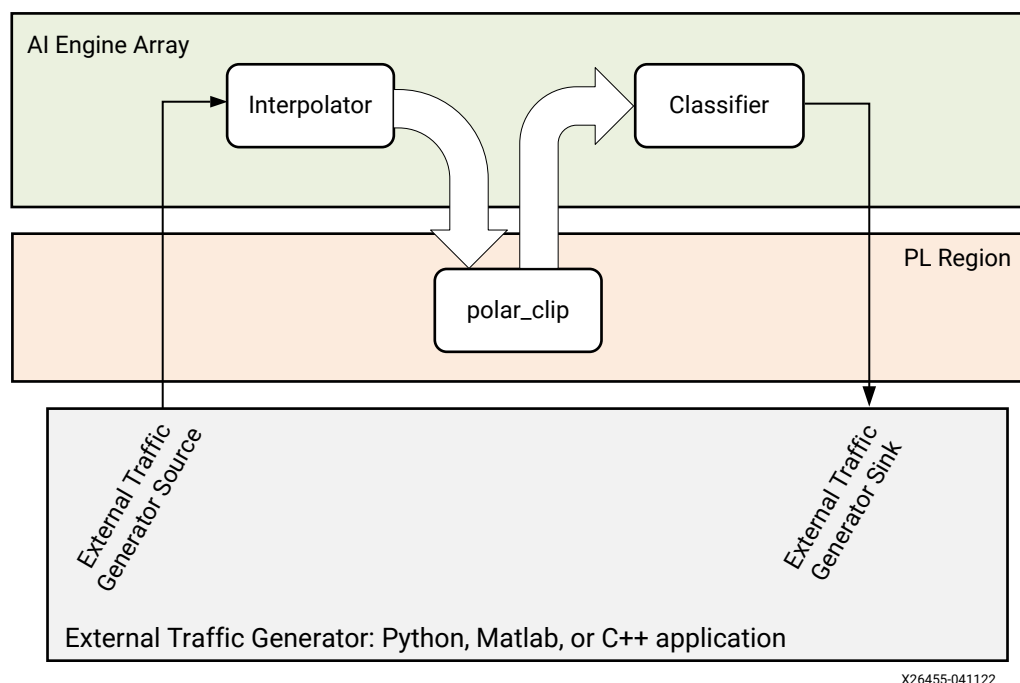
Figure 39: External Traffic Generator-Based AI Engine Simulation Flow



Each AI Engine block can be validated using an external test bench written in Python, MATLAB, or C++. An example implementation can be found in the [External-Traffic-Generator-AIE](#) tutorial on GitHub.

For System projects, incorporating AI Engine graph applications and PL kernels, the datamovers are replaced with external Traffic Generators source and sink, and the PL processing kernel is a streaming kernel connected to the AI Engine kernels, as shown in the following figure.

Figure 40: Full System Emulation



The traffic generators are used to feed and flush the data into the full system with PL logic as well. You do not need to model the PS code writing data to DDR memory and model the data moving to the AI Engine kernels. This approach replaces the datamovers with external traffic generators dynamically producing data. The following sections describe the step-by-step changes that are needed to interface external traffic generators with an AI Engine system design for the emulation flow.

AI Engine Graph Modifications

Nothing has to be changed within the graph concerning the kernel connections. The definition of the traffic generators as the source of data from the PLIO port is the only change required as shown below. The example below is based on the code in *Design Flow Using RTL Programmable Logic in AI Engine Kernel and Graph Programming Guide* ([UG1079](#)).

```
plin = input_plio::create("DataIn1",adf::plio_32_bits);
clip_in = output_plio::create("clip_in",adf::plio_32_bits);
clip_out = input_plio::create("clip_out",adf::plio_32_bits);
plout = output_plio::create("DataOut1",adf::plio_32_bits);
```

The first parameter of the input/output plio declaration is important as this is the name that will be used on the traffic generator side to connect to the right socket.

x86 simulation and AI Engine simulation can be launched working with the traffic generators. Launching simulation requires running the `aiesimulator` or the `x86simulator` in parallel with the external traffic generator.

PL Kernels Change

In considering the AI Engine application and its environment for system emulation modes (`sw` and `hw`), you need to model the data transfers to/from the programmable logic side. At the beginning of the development the kernels are not ready to be used in a `sw_emu` or `hw_emu` framework, so the Xilinx object files (`.xo`) cannot be created to be used in the Vitis link stage. You must introduce hooks in the programmable logic so that you can connect external traffic generators to them. provides a complete set of pre-compiled `.xo` files that can be used for that purpose:

- `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_slave_32.xo`, `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_master_32.xo`
- `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_slave_64.xo`, `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_master_64.xo`
- `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_slave_128.xo`, `$(XILINX_VITIS)/data/emulation/XO/sim_ipc_axis_master_128.xo`

The right set of `.xo` files should be copied to the right location on your project to use them in your configuration file during the Vitis link stage.

Preparing Connectivity to Link the Traffic Generators

During the Vitis link stage (v++ -1) the previously defined .xo files will be used to connect the related compute units to the AI Engine graph. The `hw_link.cfg` configuration file is created in such a way that the compute unit names matches the names you defined in the graph for the `input_plio` and the `output_plio`. For example, the code below matches the PLIO assignments in the example above:

```
[connectivity]

nk=sim_ipc_axis_master_32:1:in_interpolator
nk=sim_ipc_axis_slave_32:1:out_classifier
nk=polar_clip:1:polar_clip

sc=in_interpolator.M00_AXIS:ai_engine_0.in_interpolator
sc=ai_engine_0.out_interpolator:polar_clip.in_sample
sc=polar_clip.out_sample:ai_engine_0.in_classifier
sc=ai_engine_0.out_classifier:out_classifier.S00_AXIS
```

The format of the `--connectivity.nk` command is the kernel name such as `sim_ipc_axis_master_32`, the number of kernel instances (or Compute Units) to create (1), and the names of each kernel instance (`in_interpolator`). Refer to [--connectivity Options](#) for more information the command.

The `--connectivity.sc` command defines the streaming connections between PL kernels, or between PL kernels and the AI Engine graph. In the example above the output port of the traffic generator `in_interpolator.M00_AXIS` is connected to the input port `ai_engine_0.in_interpolator`.

With this naming approach, the same external traffic generator can be used for multiple simulation or emulation runs. In the case of software emulation (`sw_emu`) and hardware emulation (`hw_emu`), you can write the external traffic generator in C++, Python, MATLAB, or HDL if familiar with RTL coding.

Host Code

The host code creation is very simple. As there are no programmable logic kernels, you can avoid all the stages where you look for and run the PL kernels as well as the parts where you allocate memory for all the buffer objects. The stages are just:

- Open the device
- Load the `xclbin` file
- Register XRT to connect to the design
- Run the AI Engine graph

After compiling the host code you can package the whole project. Running the emulation consists in running the external traffic generator in parallel with the standard emulation launch.

Using Traffic Generators in AI Engine Graphs

Overview

This section describes how to interface external traffic generators with AI Engine components. Simulation using external traffic generators can be run by launching the simulator/emulator and the traffic generator in parallel. The traffic generators can be written either in Python, MATLAB, or in C++, using multi-threading capabilities of these languages. Then the AI Engine must be written to work with the traffic generator during simulation or emulation. The steps below describe interfacing the traffic generator into your AI Engine graph for standalone AI Engine simulation or the full system emulation flow (hw_emu/sw_emu).

In order to interface external traffic generators with AI Engine components, you need to use certain APIs in your external script or source code written in Python, MATLAB, or C++:

Using Python or Matlab API

You must write a Python script using dedicated APIs to interface the external traffic generator process with the AI Engine simulation process running the AI Engine graph.

```
# Mandatory
import os, sys

import multiprocessing as mp
import threading
import struct

from aie_input_plio import aie_input_plio
from aie_output_plio import aie_output_plio

# Optional for ease of use
import numpy as np
import logging
```

The Python traffic generator uses API described in [Writing Python Traffic Generators](#). MATLAB API is described in [Writing Traffic Generators in MATLAB](#).

Use the following steps to integrate the external traffic generator data into the graph:

1. Modify the graph code to declare the PLIO, but do not specify the PLIO based data text file in the PLIO constructors. This is needed for the external traffic generator to work properly. Take a note of the names (first argument) of the PLIO constructors. These will be used to hook up the external traffic generators and the same name should be used for creating the ports in the external traffic generators:

```
plin = input_plio::create("DataIn1",adf::plio_32_bits);
plout = output_plio::create("DataOut1",adf::plio_32_bits);
```

2. Instantiate the corresponding AXI master and slave connections in the Python script. This will establish connections to the PLIO ports of the graph:

```
pl_in = aie_input_plio("DataIn1", 'int16')
pl_out = aie_output_plio("DataOut1", 'int16')
```



TIP: You need to specify the PLIO datatype during object creation.

3. Send data to the AI Engine.

The data to be sent to the PLIO port in the AI Engine graph must be set up in the Python script. Using the `send_data()` API, you can send the data in the form of a list.

```
pl_in.send_data send_data(<data_list>, tlast)
```

4. Receive Data from the AI Engine.

Use the `receive_data_with_size()` API to receive data from the AI Engine. This API returns the received values in `recv_data` from the following example. This is a blocking API and will wait until expected bytes of data are received. For more information, refer to [Writing Python Traffic Generators](#).

```
recv_data = plout.receive_data_with_size(<expected_bytes>)
```

Using C++ Traffic Generator APIs

When using the C++ language to implement an external traffic generator, various headers are necessary to use some libraries in the external traffic generator source code. The headers useful for handling these libraries are:

```
# For the traffic generator
#include "xtlm_ipc.h"
#include <thread>
```

Also, the C++ traffic generator uses APIs available as part of `xtlm_ipc` sources. For a list of the APIs and their corresponding usage, see [Writing Traffic Generators in C++](#).

The Makefile dependencies are:

```
# Libraries directories
PROTO_PATH=$(XILINX_VIVADO)/data/simmodels/xsim/2023.1/lnx64/6.2.0/ext/
protobuf/
IPC_XTLM= $(XILINX_VIVADO)/data/emulation/ip_utils/xtlm_ipc/
xtlm_ipc_v1_0/cpp/src/
IPC_XTLM_INC= $(XILINX_VIVADO)/data/emulation/ip_utils/xtlm_ipc/
xtlm_ipc_v1_0/cpp/inc/
LOCAL_IPC= $(IPC_XTLM)../

LD_LIBRARY_PATH:=$(XILINX_VIVADO)/data/simmodels/xsim/2023.1/
lnx64/6.2.0/ext/protobuf/:$(XILINX_VIVADO)/lib/lnx64.o/Default:$(
(XILINX_VIVADO)/lib/lnx64.o/:$(LD_LIBRARY_PATH)

# Kernel directories
PLKernels_DIR := ../../pl_kernels
```

```

PLKernels := $(PLKernels_DIR)/polar_clip.cpp
PLheaders := $(PLKernels_DIR)/polar_clip.hpp $(PLKernels_DIR)/s2mm.hpp $(
    $(PLKernels_DIR)/mm2s.hpp

# XTLM source files
IPC_SRC := $(LOCAL_IPC)/src/axis/*.cpp $(LOCAL_IPC)/src/common/*.cpp $(
    $(LOCAL_IPC)/src/common/*.cc

# Compiler/linker flags
INC_FLAGS := -I$(LOCAL_IPC)/inc -I$(LOCAL_IPC)/inc/axis/ -I$(LOCAL_IPC)/inc/
common/ -I$(PROTO_PATH)/include/ -I$(PLKernels_DIR) -I$(XILINX_HLS)/include
LIB_FLAGS := -L$(PROTO_PATH)/ -lprotobuf -L$(XILINX_VIVADO)/lib/lnx64.o/ -
lrdizlib -L$(GCC)/../lib64/ -lstdc++ -lpthread

# Compilation
compile: main.cpp $(PLheaders) $(PLKernels)
    $(GCC) -g main.cpp $(PLKernels) $(IPC_SRC) $(INC_FLAGS) $(LIB_FLAGS) -o
chain

```

Below are the steps to integrate external traffic generator using C++ APIs in the AI Engine component:

1. Declare the external PLIOs in the graph code as below:

```

plin = input_plio::create("DataIn1",adf::plio_32_bits);
plout = output_plio::create("DataOut1",adf::plio_32_bits);

```

2. Instantiate AXI master and sender.

```

xtlm_ipc::axis_master plin("DataIn1");
xtlm_ipc::axis_slave plout("DataOut1");

```

3. Prepare the data. This is the user logic.
4. Send the data.

A simple API is available if you prefer not to have fine granular control and send the data.

```

std::vector<char> data; // The sender API expects data to be in the form
of vector of char

```

The C++ XTLM library provides the utility conversion APIs to easily convert user readable data type to byte array:

```

std::vector<char> type
std::vector<char> byte_array = plin.uInt8ToByteArray(user_list)

```

Here `uint8` is the user data type and contains user list/array for `uint8` type, such as `std::vector<uint8> user_list;`. For more details on conversion API, see [Writing Traffic Generators in C++](#).

```

// Write a user logic to fill in the data
// Send the data using send_data() API call
plin.send_data(byte_array, tlast)

```

For advanced users who need fine granular control over AXI4-Stream, see [Advanced C++ Traffic Generator API](#).

5. Receive the data.

A simple API is available if you do not need fine grain control. It returns the data in the form of byte array (`std::vector<char>`) and waits until expected `data_size` is received.

```
std::vector<char> data; // create empty vector of char
plout.receive_data_with_size(data, data_size);
```

The C++ library provides another set of utility conversion API to convert the received byte array into user readable data format. For example:

```
std::vector<int8> recv_data;
recv_data = plout .byteArrayToInt8(data)
```

For advanced users who need fine grain control over AXI4-Stream, see [Advanced C++ Traffic Generator API](#).

For more details on the API and their usage in the external traffic generator CPP code, see [Writing Traffic Generators in C++](#).

The interest of the C++ traffic generator is that you can use and test your HLS kernels as soon as they are created, without having to synthesize them in a `.xo` file. This allows you to add more and more realism and flexibility to your simulations without having to recreate a `.xclbin` file.

Using SystemVerilog Traffic Generator APIs

You can also drive traffic generators from external System Verilog/Verilog traffic generators and test benches to the `aiesimulator` or `x86simulator`.

Prior to integrating a traffic generator module, declare the external PLIOs in the graph.

```
pl_in0 = adf::input_plio::create("in_classifier",adf::plio_32_bits);
out0 = adf::output_plio::create("out_interpolator",adf::plio_32_bits);
```

To establish the connection between the SystemVerilog/Verilog traffic generator and the AI Engine graph's external PLIOs, the `xtlm_ipc` SystemC modules are required. The external AI Engine wrapper is generated based on the external PLIO declarations in the ADF graph. Follow the steps below to generate this wrapper module for the AI Engine.

1. Use the following command to perform the ADF graph compilation to generate a `scsim_config.json` file that resides in the `work/config/scsim_config.json` directory. This config file contains information on the PLIOs declared in the graph.

```
aiecompiler --platform=$(PLATFORM) -v -log-level=3 --pl-freq=500 -
include=./aie --output=graph.json aie/graph.cpp
```

2. This config file is passed as an argument to the python script available inside `{XILINX_VITIS}/data/emulation/scripts/gen_aie_wrapper.py` to generate the Verilog-based AI Engine wrapper module. For more details on how to generate the AI Engine wrapper module, see [External RTL Traffic Generator and AI Engine Simulation](#).

3. After generating the AI Engine wrapper module, you need to instantiate it in an external test bench to make the connection between the System Verilog/Verilog traffic generator and the AI Engine graph PLIOs. For details on integrating the wrapper module into the external RTL test bench, see [Instantiating AI Engine Wrapper in the Test Bench](#).

After the connection is established, you can launch the HDL simulation in parallel with AI Enginesimulation or x86 simulator. For details on how to launch the HDL simulation, see [Running the RTL Traffic Generator with AI Engine Simulation](#).

AXI4-Stream I/O Model for Streaming Traffic

The following section is specific to AXI4-Stream. The streaming I/O model can be used to emulate streaming traffic on the platform, and also support delay modeling. You can add streaming I/O to your application when targeted either for software emulation or hardware emulation, or add them to your custom platform design in the context of hardware emulation as described below:

- Streaming I/O kernels can be added to the device binary (`.xclbin`) file like any other compiled kernel object (XO) file, using the `v++ --link` command. Using these pre-compiled XO files reduces `v++` compile time. The Vitis installation provides kernels for AXI4-Stream interfaces of various data widths. The standard bit widths supported are 8, 16, 32, 64, 128, 256, 512. These can be found in the software installation at `$XILINX_VITIS/data/emulation/XO`.

Add these to your designs using the following example command:

```
v++ -t hw_emu --link $XILINX_VITIS/data/emulation/XO/
sim_ipc_axis_master_32.xo $XILINX_VITIS/data/emulation/XO/
sim_ipc_axis_slave_32.xo ...
```

In the example above, the `sim_ipc_axis_master_32.xo` and `sim_ipc_axis_slave_32.xo` provide 32-bit master and slave kernels that can be linked with the target platform and other kernels in your design to create the `.xclbin` file for the emulation build.

- In case of hardware emulation, IPC modules can also be added to a platform block design using the Vivado IP integrator for AMD Versal™ and AMD Zynq™ UltraScale+™ MPSoC custom platforms. The tool provides `sim_ipc_axis_master_v1_0` and `sim_ipc_axis_slave_v1_0` IP to add to your platform design. These can be found in the software installation at `$XILINX_VIVADO/data/emulation/hw-em/ip-repo`.

The following is an example Tcl script used to add IPC IP to your platform design, which will enable you to inject data traffic into your simulation from an external process written in Python or C++:

```
#Update IP Repository path if required
set_property ip_repo_paths $XILINX_VIVADO/data/emulation/hw_em/ip_repo
[current_project]
## Add AXIS Master
create_bd_cell -type ip -vlnv xilinx.com:ip:sim_ipc_axis_master:1.0
sim_ipc_axis_master_0
#Change Model Property if required
set_property -dict [list CONFIG.C_M00_AXIS_TDATA_WIDTH {64}]
[get_bd_cells sim_ipc_axis_master_0]

##Add AXIS Slave
create_bd_cell -type ip -vlnv xilinx.com:ip:sim_ipc_axis_slave:1.0
sim_ipc_axis_slave_0
#Change Model Property if required
set_property -dict [list CONFIG.C_S00_AXIS_TDATA_WIDTH {64}]
[get_bd_cells sim_ipc_axis_slave_0]
```

Writing Python Traffic Generators

Introduction

You can include an external traffic generator process while simulating your application to dynamically generate data traffic on the I/O traffic generators, or to capture output data from the emulation process. The AMD provided Python library can be used to create the traffic generator code as described in the following sections.

The Python library having the API is divided into API for sending and receiving user data on instantiated traffic generator objects. There are also advanced APIs for packet level granularity if you need more control over transactions. For seamless interaction of traffic generators with simulation process, the API gives you control over sending or receiving the data without worrying too much about managing packet level details like `TLAST`, `TKEEP`, `TSRB`, etc. This is auto managed by the `sim_IPC` infrastructure when using the API. Also, an application can communicate to multiple I/O interface. It is not necessary to have each instance of I/O utilities to be in a separate process/thread. If your application requires it, you might consider the non-blocking API to send the data.

The Python API are categorized as follows:

- [Python API for AI Engine Graphs](#)
- [Python API for PL Kernels](#)
- [General Purpose Python API](#)
- [Advanced Python Traffic Generator API](#)

Python API for AI Engine Graphs

Here is the list of AI Engine specific APIs and their usage in integrating the traffic generators. With the Python API you can create the traffic generator code to generate data to pass into or collect data from your AI Engine graph. Use the following API to instantiate objects to send and receive data. You can provide any datatype vector/list to `send_data` or `receive_data`.

- **Instantiating Classes to Send or Receive Data:**

```
aie_input_plio( name, datatype)
aie_output_plio(name, datatype)
```

Parameters:

- **name:** A string value that should match a PLIO name from the graph.
- **datatype:** A string with the following supported values ["int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", "uint64", "float", "bfloat16"]

Table 24: Supported AI Engine Data Types

#num	AIE kernel Datatype	API Enum String	Usage	User Value list representation (In Python)
1	int8	int8	input_port = aie_input_plio('input_port_name',"int8")	data = [12 23 2 -2 23]
2	int16	int16	input_port = aie_input_plio('input_port_name',"int16")	data = [1212 121 232 -2323]
3	int32	int32	input_port = aie_input_plio('input_port_name',"int32")	data = [121 23234 2323232 23 -878787]
4	int64	int64	input_port = aie_input_plio('input_port_name',"int64")	data = [232 2342232 -23482947 238273]
5	uint8	uint8	input_port = aie_input_plio('input_port_name',"uint8")	data = [23 23 12]
6	uint16	uint16	input_port = aie_input_plio('input_port_name',"uint16")	data = [236 23728 2378 8237]
7	uint32	uint32	input_port = aie_input_plio('input_port_name',"uint32")	data = [267 2376 2362736 232767362]
8	uint64	uint64	input_port = aie_input_plio('input_port_name',"uint64")	data = [347 2348 327 98932 872389]
9	float	float	input_port = aie_input_plio('input_port_name',"float")	data = [3.14 2.3234 3472.23]

Table 24: Supported AI Engine Data Types (cont'd)

#num	AIE kernel Datatype	API Enum String	Usage	User Value list representation (In Python)
10	bfloat16	bfloat16	input_port = aie_input_plio('input_port_name', "bfloat16")	data = [3.14 2.3234 3472.23] NOTE : Float and Bfloat16 representations in floating point are similar. But when transferred to AIE, float will be transmitted in 32 bits. bfloat16 will be transmitted in 16 bits.
11	cfloat	float	input_port = aie_input_plio('input_port_name', "float")	data = [3.14 2.2323 23.3 23.33] NOTE : From left, even positions will be real and odd positions will be imaginary
12	cint16	int16	input_port = aie_input_plio('input_port_name', "int16")	data = [32 232 232 234] NOTE : From left, even positions will be real and odd positions will be imaginary
13	cint32	int32	input_port = aie_input_plio('input_port_name', "int32")	data = [23 -23 23 -232] NOTE : From left, even positions will be real and odd positions will be imaginary

Note: For some types like `cint32` you can use `int32` as it is expected that you will provide pairs of real and imaginary values in the same list. This means the size of the user list for complex types is always even.

For example, before creating external ports, you need to do AI Engine graph modifications as described at [Using Traffic Generators for AI Engine Designs](#). Once graph modifications are done, you can create the sender and receiver objects for the AI Engine in your python script.

```
input = aie_input_plio("in_data", 'int16')
output = aie_output_plio("out_data", 'int16')
```

Here, `in_data` and `out_data` define the PLIO matching in the graph code; `input` and `output` are the objects created.

The second parameter `int16` is the datatype of the interfaced AI Engine kernel with traffic generator.

- **send_data():**

```
send_data(data, tlast)  
creates a non-blocking call to send data
```

Parameters:

- **data:** The list of specified datatype
- **tlast:** Boolean value, can be true (1) or false (0)

Note: The datatype must be specified during object instantiation.

The following is an example of creating the traffic generator object and sending data through it:

```
input = aie_input_plio("in_data", "uint32")  
input.send_data(<data_list>, "false")
```

This API call sends the data to the AI Engine kernel via "in_data" PLIO connected to the traffic generator.

- **receive_data():**

```
receive_data()  
creates a blocking call to receive data
```

RETURNS a list of specified datatype

Note: The datatype must be specified during object instantiation

For example:

```
recv_data = output.receive_data()
```

`recv_data` is a list containing the returned value of `receive_data()`

- **receive_data_with_size():**

```
receive_data_with_size(data_size)  
creates a blocking call to receive a specified amount of data
```

Parameters:

- **data_size:** integer value indicating the amount of data in bytes to receive

RETURNS a list of specified datatype

Note: Data size is specified in bytes only.

For example:

```
recv_data = output.receive_data_with_size(1024)
```

This is a blocking API which blocks until the specified bytes (1024 bytes) of data is received.

- **receive_data_on_tlast():**

```
receive_data_on_tlast()
creates a blocking call returning data after receiving tlast packet
```

RETURNS a list of specified datatype

Note: The datatype must be specified during object instantiation.

For example:

```
recv_data = output.receive_data_on_tlast()
```

recv_data contains the returned data from `receive_data_on_tlast()`. This is a blocking API which will wait until it gets the TLAST signal.

Python API for PL Kernels

Here is the list of PL kernel specific APIs and their usage in integrating the traffic generators. With the Python API you can create the traffic generator code to generate data to pass into or collect data from PL kernels. Use the following API to instantiate objects to send and receive data. You can provide any datatype vector/list to `send_data` or `receive_data`.

- **Instantiating Classes to Send or Receive Data:**

The APIs are found in the following library path:

```
${XILINX_VIVADO}/data/emulation/python/xtlm_ipc_v2/
```

You need to set `PYTHONPATH` to point to this library.

```
export PYTHONPATH=${XILINX_VIVADO}/data/emulation/python/xtlm_ipc_v2/
```

```
hls_input_stream(name, datatype)
hls_output_stream(name, datatype)
```

Parameters:

- **name:** string name to match the HLS kernel
- **datatype:** A string value based on HLS kernel datatype. The supported values are
["int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", "uint64", "float", "bfloat16"]

Table 25: Supported PL Kernel Data Types

	PL Kernel Data Type	EOU API Enum String	Example
1	int8	int8	input_port = hls_input_plio('input_port_name',"int 8")
2	int16	int16	input_port = hls_input_plio('input_port_name',"int 16")
3	int32	int32	input_port = hls_input_plio('input_port_name',"int 32")
4	int64	int64	input_port = hls_input_plio('input_port_name',"int 64")
5	uint8	uint8	input_port = hls_input_plio('input_port_name',"ui nt8")
6	uint16	uint16	input_port = hls_input_plio('input_port_name',"ui nt16")
7	uint32	uint32	input_port = hls_input_plio('input_port_name',"ui nt32")
8	uint64	uint64	input_port = hls_input_plio('input_port_name',"ui nt64")
9	float	float	input_port = hls_input_plio('input_port_name',"flo at")
10	double	double	input_port = hls_input_plio('input_port_name',"do uble")

- **send_data():**

```
send_data(data, tlast)
creates a non-blocking call to send data
```

Parameters:

- data: list of specified datatype for the object
- tlast: boolean value, can be true or false

Note: The datatype must be specified during object instantiation

- **receive_data():**

```
receive_data()
creates a blocking call to receive data
```

RETURNS a list of specified datatype

Note: The datatype must be specified during object instantiation

- **receive_data_with_size():**

```
receive_data_with_size(data_size)
creates a blocking call to receive a specified amount of data
```

RETURNS a list of specified datatype

Parameters:

- **data_size:** integer value indicating the amount of data in bytes to receive

Note: Data size must be specified in bytes.

- **receive_data_on_tlast():**

```
receive_data_on_tlast()
creates a blocking call returning data after receiving tlast packet
```

RETURNS list of specified data type

Note: The data type must be specified during object instantiation.

General Purpose Python API

Here is the list of general purpose APIs that can be used with custom user-defined datatypes. With these API you can send and receive data in the form of `byte_arrays` only. You must convert your custom datatype to `byte_arrays` prior to transport, and convert the received value back to your custom datatype for use by your application. Conversion API are provided as described below.

- **Instantiating Classes to Send or Receive Data:**

The APIs are found in the following library path:

```
${XILINX_VIVADO}/data/emulation/python/xtlm_ipc_v2/
```

You need to set `PYTHONPATH` to point to this library:

```
export PYTHONPATH=${XILINX_VIVADO}/data/emulation/python/xtlm_ipc_v2/
```

```
input_port = axis_master(name)
output_port = axis_slave(name)
```

Parameters:

- **name:** string matching the AXI4-Stream interface
- **send_data():**



TIP: The general purpose API expects data in the form of *byte_array* only. To convert it from a user data type, use the conversion API such as *uInt16ToByteArray* described below.

```
send_data(data, tlast)  
creates a non-blocking call to send data
```

RETURNS nothing

Parameters:

- **data:** list created using `create_byte_array()` or conversion API described below
- **tlast:** boolean value, can be true or false

For example:

```
input.send_data(data_byte_array, tlast)
```

- **receive_data():**



TIP: The general purpose API expects data in the form of *byte_array* only. To convert it into a user readable format with data types after receiving it, use the conversion API such as *byteArrayToInt16* described below.

```
receive_data(data)  
creates a blocking call to receive data
```

RETURNS nothing

Parameters:

- **data:** a byte array that can be converted with conversion API described below

- **receive_data_with_size():**

```
receive_data_with_size(data, data_size)  
creates a blocking call to receive a specified amount of data
```

RETURNS a list of specified datatype

Parameters:

- **data:** a byte array that can be converted with conversion API described below
- **data_size:** integer value indicating the amount of data in bytes to receive

Note: Data size is specified in bytes

For example:

```
output.receive_data_with_size(<recv_vector>, 512)
```

Where `recv_vector` is an empty byte array that gets filled with the data received in the form of a byte array

- **receive_data_on_tlast():**

```
receive_data_on_tlast(data)
creates a blocking call returning data after receiving tlast packet
```

RETURNS a list of specified datatype

Parameters:

- `data`: a byte array that can be converted with the conversion API described below

Table 26: Byte_Array Conversion API

API	Description
<code>byte_array = input_port.uInt8ToByteArray(user_list)</code> <code>byte_array = input_port.uInt16ToByteArray(user_list)</code> <code>byte_array = input_port.uInt32ToByteArray(user_list)</code> <code>byte_array = input_port.uInt64ToByteArray(user_list)</code> <code>byte_array = input_port.int8ToByteArray(user_list)</code> <code>byte_array = input_port.int16ToByteArray(user_list)</code> <code>byte_array = input_port.int32ToByteArray(user_list)</code> <code>byte_array = input_port.int64ToByteArray(user_list)</code> <code>byte_array = input_port.floatToByteArray(user_list)</code> <code>byte_array = input_port.doubleToByteArray(user_list)</code> <code>byte_array = input_port.bfloat16ToByteArray(user_list)</code>	Convert the specified data type to <code>byte_array</code> value to send the data <pre>byte_array = input_port.uInt64ToByteArray(user_list)</pre> Parameters: Returns list of specified data type converted from <code>byte_array</code> <code>user_list</code> → <code>std::vector<T></code> // T is the data type present in function signature. // For example in <code>byteArrayToFloat</code> <code>user_list</code> is a vector of type float <code>byte_array</code> → list created using <code>create_byte_array</code> or conversion APIs
<code>user_list = output_port.byteArrayToInt8(byte_array)</code> <code>user_list = output_port.byteArrayToInt16(byte_array)</code> <code>user_list = output_port.byteArrayToInt32(byte_array)</code> <code>user_list = output_port.byteArrayToInt64(byte_array)</code> <code>user_list = output_port.byteArrayToInt8(byte_array)</code> <code>user_list = output_port.byteArrayToInt16(byte_array)</code> <code>user_list = output_port.byteArrayToInt32(byte_array)</code> <code>user_list = output_port.byteArrayToInt64(byte_array)</code> <code>user_list = output_port.byteArrayToFloat(byte_array)</code> <code>user_list = output_port.byteArrayToDouble(byte_array)</code> <code>user_list = output_port.byteArrayToBfloat16(byte_array)</code>	Convert the <code>byte_array</code> value to the specified data type after receiving <pre>user_list = output_port.byteArrayToInt16(byte_array)</pre> Parameters: Returns list of specified data type converted from <code>byte_array</code> <code>byte_array</code> → list created using <code>create_byte_array</code> or conversion APIs <code>user_list</code> → <code>std::vector<T></code> // T is the data type present in function signature. // For example in <code>byteArrayToFloat</code> <code>user_list</code> is a vector of type float

Advanced Python Traffic Generator API

The following is the list of the advanced Python traffic generator API along with their usage.



TIP: A full system-level example using these API is available for external IO in Python in the [External_IO_PY](#) tutorial on GitHub.

Table 27: List of Advanced Python API used in external traffic generator

API	Behavior
<code>ipc_axis_master_util("<sender_io_name>")</code>	To instantiate the master traffic generator (TG)
<code>ipc_axis_slave_util("<receiver_io_name>")</code>	To instantiate the slave TG
<code>b_transport(packet)</code>	Blocking API to send the AXI4-Stream packets
<code>sample_transaction()</code>	Blocking API to receive the AXI4-Stream packets. Data is received beat by beat (one <code>sample_transaction()</code> call gives one beat).
<code>ipc_axis_master_nb_util("<sender_io_name>")</code>	To instantiate a non-blocking master TG
<code>ipc_axis_slave_nb_util("<receiver_io_name>")</code>	To instantiate a non-blocking slave TG
<code>nb_transport(packet)</code>	Non-blocking API call to send the AXI4-Stream packets
<code>nb_sample_transaction()</code>	Non-blocking API call to receive the AXI4-Stream packets. Data is received beat by beat (one <code>sample_transaction()</code> call gives one beat).
<code>no_of_available_txns()</code>	Used to get the number of transactions received for non-blocking TG
<code>end_of_simulation()</code>	End and exit the simulation using this API Preferable use in external TG with <code>sw_emu/hw_emu</code>
<code>disconnect()</code>	To disconnect the socket connections (DataIn, DataOut etc) Preferable for use in external TG with <code>x86simulator</code> or <code>aiesimulator</code>

`ipc_axis_master_util (<sender_io_name>)`

Used to instantiate the sender port in the Python script.

- **API usage:** Creates a user object using the API for sender:

```
sender_obj = ipc_axis_master_util (<sender_io_name>)
```

- **Parameters:** Name of the external port that sends data:

```
<sender_io_name>
```

- **Example:**

```
#Instantiating AXI Master Utilities
data_in = ipc_axis_master_util("DataIn")
```

DataIn is the external port name and data_in is the user object.

`ipc_axis_slave_util(<receiver_io_name>)`

Used to instantiate the receiver port in the Python script.

- **API usage:** Creates a user object using the API for the receiver:

```
receiver_obj = ipc_axis_slave_util (<receiver_io_name>)
```

- **Parameters:** Name of the external port that receives the data:

```
receiver_io_name
```

- **Example:**

```
#Instantiating AXI Slave Utilities
data_out = ipc_axis_slave_util("DataOut")
```

In the example, `DataOut` is the external port and `data_out` is the user object.

b_transport(packet)

The blocking API to send the AXI4-Stream.

- **API usage:** Calls the API using the object created

```
sender_obj.b_transport(<packet>)
```

- **Parameters:** AXI4-Stream packet:

```
packet
```

This AXI4-Stream has the following attributes that need to be set. The data that needs to be sent:

```
data
```

The TLAST value to mark the end of the packet. This is the user's choice based on the application.

```
tlast
```

If not specified, the default TLAST value is true so that every data beat is a transaction. If `tlast=false` no TLAST is driven. If `tlast=true` (specified by the user based on the application), the last data beat has `tlast=true`. Some other optional AXI4-Stream packet fields are: `tuser` and `tkeep`

- **Example:**

```
#Create packet
data_packet = xtlm_ipc.axi_stream_packet() // API to generate the stream
packet
data = generate_data() # custom user code to generate the data
data_packet.data = data

data_packet.tlast = True #AXI Stream Fields
#Optional AXI Stream Parameters
data_packet.tuser = optional data
data_packet.tkeep = optional data

#Send Transaction
data_in.b_transport(data_packet)
```

sample_transaction()

Blocking API to receive the AXI4-Stream packets.

- **API Usage:** Calls the API using the receiver_obj created:

```
packet = receiver_obj.sample_transaction()
```

- **Example:**

```
#Sample the packets
data_packet = data_out.sample_transaction() # Will wait until data is
received.
```

Here, data_out is the receiver object and data_packet is the received packet.

ipc_axis_master_nb_util(<sender_io_name>)

Instantiates the sender port for non-blocking in the python script.

- **API Usage:** Creates a user object using the API for sender:

```
sender_obj = ipc_axis_master_nb_util (<sender_io_name>)
```

- **Parameters:** Name of the external port that sends the data:

```
<sender_io_name>
```

- **Example:**

```
#Instantiating AXI Master Utilities
data_in = ipc_axis_master_nb_util("DataIn")
```

ipc_axis_slave_nb_util(<receiver_io_name>)

Used to instantiate the receiver port for non-blocking in the Python script

- **API Usage:** Creates a user object using the API for receiver

```
receiver_obj = ipc_axis_slave_util (<receiver_io_name>)
```

- **Parameters:** Name of the external port that sends the data

```
receiver_io_name
```

- **Example:**

```
#Instantiating AXI Slave Utilities
data_out = ipc_axis_slave_nb_util("DataOut")
```

nb_transport(<packet>)

Non-blocking API used to send AXI4-Stream packets. Prior to transporting data you need to format it into data packets as described in *Formatting Data with Traffic Generators in Python* below.

- **API Usage:** Calls the API using sender_obj:

```
sender_obj.nb_transport(<packet>)
```

- **Parameters:** AXI4-Stream packet:

```
packet
```

This AXI4-Stream packet is created by the user which has the following attributes: The data that needs to be sent:

```
data
```

The TLAST value to mark the end of the packet. This is the user's choice based on the application.

```
tlast
```

If not specified, the default TLAST value is true so that every data beat is a transaction. If `tlast=false` no TLAST is driven. If `tlast=true` (specified by the user based on the application), the last data beat has `tlast=true`. Some other optional AXI4-Stream packet fields are `tuser` and `tkeep`.

- **Example:**

```
#Create packet
data_packet = xtlm_ipc.axi_stream_packet()
data = generate_data() # custom user code to generate the desired data
data_packet.data_length = "DATA_LENGTH"
data_packet.data = data
data_packet.tlast = True #to mark end of packet. This is user's choice
based on application
data_in.nb_transport(data_packet)
```

nb_sample_transaction()

A non-blocking API to receive the AXI stream packets.

- **API Usage:**

```
packet = receiver_obj.nb_sample_transaction()
```

- **Example:**

```
#Sample the packets
# Will return data if it is available
# Will return empty list if nothing is available
# Will not wait or stop the execution until data is present to sample
data_packet = data_out.nb_sample_transaction()
```

Here, `data_out` is the receiver object and `data_packet` is the received packet.

no_of_available_txns

For non-blocking, this API is used to get to get number of transactions received.

- **API Usage:** Calls the API using the user object for the receiver.

- **Example:**

```
data_out = ipc_axis_slave_nb_util("ReceiverName")
data = []

# Checks if any data is present to receive
if(slave.no_of_available_txns != 0) :
    data = data_out.nb_sample_transaction() # fill data with respective
    received data
```

end_of_simulation()

Ends and exits the simulation.

- **API Usage:** Call the API using the sender object

```
sender_obj.end_of_simulation()
```

- **Example:**

```
data_in.end_of_simulation()
```

Preferably used with `sw_emu/hw_emu`.

disconnect()

To disconnect the socket connections (DataIn, DataOut etc)

- **API Usage:** Calls the API using sender or receiver object:

```
sender_obj.disconnect()
receiver_obj.disconnect()
```

- **Example:**

```
data_in.disconnect()
data_out.disconnect()
```


Preferably used with `x86simulator` or `aiesimulator`.

Formatting Data with Traffic Generators in Python

Data formatting is not necessary when using the AI Engine API as described in [Python API for AI Engine Graphs](#). However, to send or receive data using the advanced API, which expects data to be in the form of packets, you need to format the data prior to transport.

To emulate AXI4-Stream transactions AXI Traffic Generators require the payload data to be broken into appropriately sized bursts. For example, to send 128 bytes with a PLIO width of 32 bits (4 bytes) requires $128 \text{ bytes} / 4 \text{ bytes} = 32$ AXI4-Stream transactions. Converting between bytes arrays and AXI transactions can be handled in Python.

The Python `struct` library provides a mechanism to convert between Python and C data types. Specifically, the `struct.pack` and `struct.unpack` functions pack and unpack byte arrays according to a format string argument. The following table shows format strings for common C data types and PLIO widths.

For more information see: <https://docs.python.org/3/library/struct.html>

Table 28: Format Strings for C Data Types and PLIO Widths

Data Type	PLIO Width	Python Code Snippet
cfloat	PLIO32	N/A
	PLIO64	<pre>rVec = np.real(data) iVec = np.imag(data) out2column = np.zeros((L,2)).astype(np.single) out2column.tobytes() formatString = "<" + str(len(byte_array)//4) + "f"</pre>
	PLIO128	
cint16	PLIO32	<pre>rVec = np.real(data).astype(np.int16) iVec = np.imag(data).astype(np.int16) formatString = "<" + str(len(byte_array)//2) + "h"</pre>
	PLIO64	
	PLIO128	
int8	PLIO32	<pre>intvec = np.real(data).astype(np.int8) formatString = "<" + str(len(byte_array)//1) + "b"</pre>
	PLIO64	
	PLIO128	
int32	PLIO32	<pre>intvec = np.real(data).astype(np.int32) formatString = "<" + str(len(byte_array)//4) + "i"</pre>
	PLIO64	
	PLIO128	

Writing Traffic Generators in MATLAB

Introduction

You can include external traffic generators created in MATLAB to dynamically generate data traffic on the I/O traffic generators while simulating the design, or to capture output data from the emulation process. The AMD provided MATLAB API can be used to create the traffic generator code as described in the following sections.

The MATLAB API is divided into API for sending and receiving user data on instantiated traffic generator objects. There are also advanced API for packet level granularity if you need more control over transactions. For seamless interaction of traffic generators with simulation process, high-level APIs give you control over sending or receiving the data without worrying much about managing packet level details like `TLAST`, `TKEEP`, `TSRB`. This is auto managed by the `sim_IPC` infrastructure. Also, an application can communicate to multiple I/O interface. It is not necessary to have each instance of I/O utilities to be in a separate process/thread. If your application requires it, you might consider the non-blocking API to send the data.

The MATLAB API are categorized as follows:

- [MATLAB API for AI Engine Graphs](#)
- [MATLAB API for PL Kernels](#)
- [General Purpose MATLAB API](#)

MATLAB API for AI Engine Graphs

Here is the list of AI Engine specific APIs and their usage in integrating the traffic generators. With the MATLAB API you can create the traffic generator code to generate data to pass into or collect data from your AI Engine graph. Use the following API to instantiate objects to send and receive data. You can provide any datatype vector/list to `send_data` or `receive_data`.

- **Instantiating Classes to Send or Receive Data:**

You need to set the MATLAB library path to use the APIs:

```
vivado = getenv("XILINX_VIVADO");  
libPath = fullfile(vivado, "/data/emulation/matlab/xtlm_ipc");  
addpath(libPath)
```

```
aie_input_plio(name, datatype)  
aie_output_plio(name, datatype)
```

Parameters:

- **name:** A string value that should match a PLIO name from the graph

- **datatype:** A string with the supported values ["int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", "uint64", "float", "bfloat16"]

For example, before creating external ports you need to make ADF graph modifications as described *Graph Modification* in [Using Traffic Generators for AI Engine Designs](#). Once graph modifications are done, you can create the sender and receiver objects for the AI Engine in your MATLAB script.

```
input = aie_input_plio("in_data", 'int16')
output = aie_output_plio("out_data", 'int16')
```

Here `in_data` and `out_data` are the PLIOs defined in the graph code, `input` and `output` are the instantiated traffic generator objects.

Table 29: Supported AI Engine Data Types

#num	AIE kernel Datatype	API Enum String	Usage	User Value list representation (In MATLAB)
1	int8	int8	input_port = aie_input_plio('input_port_name',"int8")	data = [12 23 2 -2 23]
2	int16	int16	input_port = aie_input_plio('input_port_name',"int16")	data = [1212 121 232 -2323]
3	int32	int32	input_port = aie_input_plio('input_port_name',"int32")	data = [121 23234 2323232 23 -878787]
4	int64	int64	input_port = aie_input_plio('input_port_name',"int64")	data = [232 2342232 -23482947 238273]
5	uint8	uint8	input_port = aie_input_plio('input_port_name',"uint8")	data = [23 23 12]
6	uint16	uint16	input_port = aie_input_plio('input_port_name',"uint16")	data = [236 23728 2378 8237]
7	uint32	uint32	input_port = aie_input_plio('input_port_name',"uint32")	data = [267 2376 2362736 232767362]
8	uint64	uint64	input_port = aie_input_plio('input_port_name',"uint64")	data = [347 2348 327 98932 872389]
9	float	float	input_port = aie_input_plio('input_port_name',"float")	data = [3.14 2.3234 3472.23]

Table 29: Supported AI Engine Data Types (cont'd)

#num	AIE kernel Datatype	API Enum String	Usage	User Value list representation (In MATLAB)
10	bfloat16	bfloat16	input_port = aie_input_plio('input_port_name', "bfloat16")	data = [3.14 2.3234 3472.23] Float and Bfloat16 representations in floating point are similar. But when transferred to AIE, float will be transmitted in 32 bits. bfloat16 will be transmitted in 16 bits.
11	cfloat	float	input_port = aie_input_plio('input_port_name', "float")	data = [3.14 2.2323 23.3 23.33] NOTE : From left, even positions will be real and odd positions will be imaginary
12	cint16	int16	input_port = aie_input_plio('input_port_name', "int16")	data = [32 232 232 234] NOTE : From left, even positions will be real and odd positions will be imaginary
13	cint32	int32	input_port = aie_input_plio('input_port_name', "int32")	data = [23 -23 23 -232] NOTE : From left, even positions will be real and odd positions will be imaginary

- **send_data():**

```
send_data(data, tlast)
creates a non-blocking call to send data
```

RETURNS nothing

Parameters:

- **data:** list of specified datatype for the object
- **tlast:** boolean value, can be true or false

Note: The datatype must be specified during object instantiation.

The following is an example of creating the traffic generator object and sending data through it:

```
input = aie_input_plio("in_data", "uint32")
input.send_data(<data_set>, "false")
```

This API call sends the data to the AI Engine kernel via "in_data" PLIO connected to the traffic generator.

- **receive_data():**

```
receive_data()  
creates a blocking call to receive data
```

RETURNS a list of specified datatype

Note: The datatype must be specified during object instantiation

For example:

```
recv_data = output.receive_data()
```

recv_data is a list containing the returned value of receive_data()

- **receive_data_with_size():**

```
receive_data_with_size(data_size)  
creates a blocking call to receive a specified amount of data
```

RETURNS a list of specified datatype

Parameters:

- **data_size:** integer value indicating the amount of data in bytes to receive

Note: Data size is specified in bytes

For example:

```
recv_data = output.receive_data_with_size(1024)
```

This is a blocking API which blocks until the specified bytes of data is received.

- **receive_data_on_tlast():**

```
receive_data_on_tlast()  
creates a blocking call returning data after receiving tlast packet
```

RETURNS list of specified data type

Note: The data type must be specified during object instantiation

For example:

```
recv_data = receive_data_on_tlast()
```

recv_data contains the returned data from receive_data_on_tlast(). This is a blocking API which will wait until it gets the TLAST signal.

MATLAB API for PL Kernels

Here is the list of PL kernel specific APIs and their usage in integrating the traffic generators. With the MATLAB API you can create the traffic generator code to generate data to pass into or collect data from PL kernels. Use the following API to instantiate objects to send and receive data. You can provide any datatype vector/list to `send_data` or `receive_data`.

You need to set the MATLAB library path to use the APIs.

```
vivado = getenv("XILINX_VIVADO");
libPath = fullfile(vivado, "/data/emulation/matlab/xtlm_ipc");
addpath(libPath)
```

• Instantiating Classes to Send or Receive Data:

```
hls_input_stream(name, datatype)
hls_output_stream(name, datatype)
```

Parameters:

- **name:** string name to match the PL kernel
- **datatype:** A string value based on HLS kernel datatype. The supported values are
["int8", "uint8", "int16", "uint16", "int32", "uint32", "int64",
"uint64", "float", "bfloat16"]

Table 30: Supported PL Kernel Data Types

	PL Kernel Data Type	EOU API Enum String	Example
1	int8	int8	input_port = hls_input_plio('input_port_name','int 8")
2	int16	int16	input_port = hls_input_plio('input_port_name','int 16")
3	int32	int32	input_port = hls_input_plio('input_port_name','int 32")
4	int64	int64	input_port = hls_input_plio('input_port_name','int 64")
5	uint8	uint8	input_port = hls_input_plio('input_port_name','ui nt8")
6	uint16	uint16	input_port = hls_input_plio('input_port_name','ui nt16")
7	uint32	uint32	input_port = hls_input_plio('input_port_name','ui nt32")
8	uint64	uint64	input_port = hls_input_plio('input_port_name','ui nt64")

Table 30: Supported PL Kernel Data Types (cont'd)

	PL Kernel Data Type	EOU API Enum String	Example
9	float	float	input_port = hls_input_plio('input_port_name',"float")
10	double	double	input_port = hls_input_plio('input_port_name',"double")

- **send_data():**

```
send_data(data, tlast)
creates a non-blocking call to send data
```

RETURNS nothing

Parameters:

- **data:** list of specified datatype for the object
- **tlast:** boolean value, can be true (1) or false (0)

Note: The datatype must be specified during object instantiation

- **receive_data():**

```
receive_data()
creates a blocking call to receive data
```

RETURNS a list of specified datatype

Note: The datatype must be specified during object instantiation

- **receive_data_with_size():**

```
receive_data_with_size(data_size)
creates a blocking call to receive a specified amount of data
```

RETURNS a list of specified datatype

Parameters:

- **data_size:** integer value indicating the amount of data in bytes to receive

Note: Data size is specified in bytes

- **receive_data_on_tlast():**

```
receive_data_on_tlast()
creates a blocking call returning data after receiving tlast packet
```

RETURNS list of specified data type

Note: The data type must be specified during object instantiation

General Purpose MATLAB API

Here is the list of general purpose APIs that can be used with custom user-defined datatypes. With these APIs, you can send and receive data only in the form of `byte_arrays`. You must convert your custom datatype to `byte_arrays` prior to transport, and convert the received value back to your custom datatype for use by your application. The conversion API is described below.

You need to set the MATLAB library path to use the APIs:

```
vivado = getenv("XILINX_VIVADO");  
libPath = fullfile(vivado, "/data/emulation/matlab/xtlm_ipc");  
addpath(libPath)
```

- **Instantiating Classes to Send or Receive Data:**

```
input_port = axis_master( name)  
output_port = axis_slave(name)
```

Parameters:

- `name`: string matching the interface name of the block connected to the traffic generator

- **`send_data()`:**



TIP: The general purpose API expects data in the form of `byte_array` only. To convert it from a user data type, use the conversion API such as `uInt16ToByteArray` described below.

```
send_data(data, tlast)  
creates a non-blocking call to send data
```

RETURNS nothing

Parameters:

- `data`: list created using `create_byte_array()` or conversion API described below
- `tlast`: boolean value, can be true or false

For example:

```
input.send_data(data_byte_array, tlast)
```

- **`receive_data()`:**



TIP: The general purpose API expects data in the form of `byte_array` only. To convert it into a user readable format with data types after receiving it, use the conversion API such as `byteArrayToInt16` described below.

```
receive_data(data)  
creates a blocking call to receive data
```

RETURNS a list of data received in the form of a byte array

Parameters:

- `data`: a byte array that can be converted with conversion API described below
- **receive_data_with_size():**

```
receive_data_with_size(data, data_size)  
creates a blocking call to receive a specified amount of data
```

RETURNS a list of data received in the form of a byte array

Parameters:

- `data`: a byte array that can be converted with conversion API described below
- `data_size`: integer value indicating the amount of data in bytes to receive

Note: Data size is specified in bytes

For example:

```
output.receive_data_with_size(<recv_vector>, 512)
```

Where `recv_vector` is an empty byte array that gets filled with the data received in the form of a byte array

- **receive_data_on_tlast():**

```
receive_data_on_tlast(data)  
creates a blocking call returning <data> after receiving tlast packet
```

RETURNS a list of data received in the form of a byte array

Parameters:

- `data`: a byte array that can be converted with conversion API described below

Table 31: Byte_Array Conversion API

API	Description
<pre>byte_array = input_port.uInt8ToByteArray(user_list) byte_array = input_port.uInt16ToByteArray(user_list) byte_array = input_port.uInt32ToByteArray(user_list) byte_array = input_port.uInt64ToByteArray(user_list) byte_array = input_port.int8ToByteArray(user_list) byte_array = input_port.int16ToByteArray(user_list) byte_array = input_port.int32ToByteArray(user_list) byte_array = input_port.int64ToByteArray(user_list) byte_array = input_port.floatToByteArray(user_list) byte_array = input_port.doubleToByteArray(user_list) byte_array = input_port.bfloat16ToByteArray(user_list)</pre>	Convert the specified data type to byte_array value to send the data <pre>byte_array = input_port.uInt64ToByteArray(user_list)</pre> Parameters: Returns list of specified data type converted from byte_array user_list → std::vector<T> // T is the data type present in function signature. // For example in byteArrayToFloat user_list is a vector of type float byte_array → list created using create_byte_array or conversion APIs
<pre>user_list = output_port.byteArrayTouInt8(byte_array) user_list = output_port.byteArrayTouInt16(byte_array) user_list = output_port.byteArrayTouInt32(byte_array) user_list = output_port.byteArrayTouInt64(byte_array) user_list = output_port.byteArrayToInt8(byte_array) user_list = output_port.byteArrayToInt16(byte_array) user_list = output_port.byteArrayToInt32(byte_array) user_list = output_port.byteArrayToInt64(byte_array) user_list = output_port.byteArrayToFloat(byte_array) user_list = output_port.byteArrayToDouble(byte_array) user_list = output_port.byteArrayToBfloat16(byte_array)</pre>	Convert the byte_array value to the specified data type after receiving <pre>user_list = output_port.byteArrayTouInt16(byte_array)</pre> Parameters: Returns list of specified data type converted from byte_array byte_array → list created using create_byte_array or conversion APIs user_list → std::vector<T> // T is the data type present in function signature. // For example in byteArrayToFloat user_list is a vector of type float

Writing Traffic Generators in C++

Introduction

You can include an external traffic generator process while simulating your application to dynamically generate data traffic on the I/O, or to capture output data from the emulation process. The AMD provided C++ library can be used to create the traffic generator code as described in the following sections.

The C++ library having the API is divided into API for sending/receiving data in the form of byte array. The user data before sending needs to be converted into byte array and the data is received in the form of byte array which can be further converted to user data type using conversion APIs. There are also advanced API for packet level granularity if you need more control over transactions. For seamless interaction of traffic generators with simulation process, the API gives you control over sending or receiving the data without much worrying about managing packet level details like TLAST, this is auto managed by the `sim_IPC` infrastructure.

For C++, the APIs are available at:

```
$XILINX_VIVADO/data/emulation/cpp/inc/xtlm_ipc/axis/
```

You can build the executable with the following include path:

```
-I $XILINX_VIVADO/data/emulation/cpp/inc/xtlm_ipc/axis/
```

and linked against library as:

```
-L $XILINX_VIVADO/data/emulation/cpp/lib/
```

And use `-lxtlm_ipc` with a `gcc` compiler.

The C++ API are categorized as follows:

- [General Purpose C++ API](#)
- [Advanced C++ Traffic Generator API](#)

A full system-level example is available in the [External_IO_CPP](#) tutorial on GitHub.

General Purpose C++ API

Here is the list of general purpose APIs that can be used with custom user-defined datatypes. With these API you can send and receive data in the form of `byte_arrays` only. You must convert your custom datatype to `byte_arrays` prior to transport, and convert the received value back to your custom datatype for use by your application. Conversion API are provided as described below.

- **Instantiating Classes to Send or Receive Data:**

```
xtlm_ipc::axis_master    input_port(std::string name)
xtlm_ipc::axis_slave     output_port(std::string name)
```

Parameters:

name → string matching the AXI4-Stream interface

- **send_data():**



TIP: The general purpose API expects data in the form of `byte_array` only. To convert it from a user data type, use the conversion API such as `uInt16ToByteArray` described below.

```
send_data(data, tlast)
creates a non-blocking call to send data
```

Parameters:

RETURNS nothing

data → `std::vector<char>`

tlast → boolean value, can be true or false

- **receive_data():**



TIP: The general purpose API expects data in the form of `byte_array` only. To convert it into a user readable format with data types after receiving it, use the conversion API such as `byteArrayToInt16` described below.

```
receive_data(data)
creates a blocking call to receive data
```

Parameters:

RETURNS the received data as a `byte_array`
data → `std::vector<char>`

The following example creates an empty vector for data and receives data:

```
std::vector<char> recv_data;
output.receive_data(<recv_data>)
```

- **receive_data_with_size():**

```
receive_data_with_size(data, data_size)
creates a blocking call to receive a specified amount of data
```

Parameters:

RETURNS the received data as a `byte_array`
data → `std::vector<char>`
data_size → integer value indicating the amount of data in bytes to receive

Note: data_size is specified in bytes

The following example creates an empty vector for data and receives data:

```
std::vector<char> recv_data;
output.receive_data_with_size(<recv_data>, 1024)
```

- **receive_data_on_tlast():**

```
receive_data_on_tlast(data)
creates a blocking call returning data after receiving tlast packet
```

Parameters:

RETURNS the received data as a `byte_array`
data → `std::vector<char>`

Note: The datatype must be specified during object instantiation

The following example creates an empty vector for data and receives data on the TLAST signal:

```
std::vector<char> recv_data;
output.receive_data_on_tlast(<recv_data>)
```

Table 32: Byte_Array Conversion API

API	Description
<pre>byte_array = input_port.uInt8ToByteArray(user_list) byte_array = input_port.uInt16ToByteArray(user_list) byte_array = input_port.uInt32ToByteArray(user_list) byte_array = input_port.uInt64ToByteArray(user_list) byte_array = input_port.int8ToByteArray(user_list) byte_array = input_port.int16ToByteArray(user_list) byte_array = input_port.int32ToByteArray(user_list) byte_array = input_port.int64ToByteArray(user_list) byte_array = input_port.floatToByteArray(user_list) byte_array = input_port.doubleToByteArray(user_list) byte_array = input_port.bfloat16ToByteArray(user_list)</pre>	Convert the specified data type to byte_array value to send the data <pre>byte_array = input_port.uInt64ToByteArray(user_list)</pre> Parameters: Returns list of specified data type converted from byte_array user_list → std::vector<T> // T is the data type present in function signature. // For example in byteArrayToFloat user_list is a vector of type float byte_array → list created using create_byte_array or conversion APIs
<pre>user_list = output_port.byteArrayTouInt8(byte_array) user_list = output_port.byteArrayTouInt16(byte_array) user_list = output_port.byteArrayTouInt32(byte_array) user_list = output_port.byteArrayTouInt64(byte_array) user_list = output_port.byteArrayToInt8(byte_array) user_list = output_port.byteArrayToInt16(byte_array) user_list = output_port.byteArrayToInt32(byte_array) user_list = output_port.byteArrayToInt64(byte_array) user_list = output_port.byteArrayToFloat(byte_array) user_list = output_port.byteArrayToDouble(byte_array) user_list = output_port.byteArrayToBfloat16(byte_array)</pre>	Convert the byte_array value to the specified data type after receiving <pre>user_list = output_port.byteArrayTouInt16(byte_array)</pre> Parameters: Returns list of specified data type converted from byte_array byte_array → list created using create_byte_array or conversion APIs user_list → std::vector<T> // T is the data type present in function signature. // For example in byteArrayToFloat user_list is a vector of type float

Advanced C++ Traffic Generator API

The C++ API provides both blocking and non-blocking function support. The following table shows the usage:

Table 33: List of Advanced C++ APIs used in external traffic generator

API	Behavior
<code>xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::BLOCKING> socket_util("<sender_io_name>")</code>	To instantiate the master for blocking API calls
<code>xtlm_ipc::axis_target_socket_util<xtlm_ipc::BLOCKING> socket_util("<receiver_io_name>")</code>	To instantiate the slave for blocking API calls
<code>xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::NON_BLOCKING> socket_util("<sender_io_name>")</code>	To instantiate the master for non-blocking API calls
<code>xtlm_ipc::axis_target_socket_util<xtlm_ipc::NON_BLOCKING> socket_util("<receiver_io_name>")</code>	To instantiate the slave for non-blocking API calls

Table 33: List of Advanced C++ APIs used in external traffic generator (cont'd)

API	Behavior
<code>transport(data, size(), tlast)</code>	API to send data in the form of vector of char, specify the size of the data Optional argument TLAST if <code>tlast = false</code> , No TLAST is driven if <code>tlast = true</code> , last data beat has TLAST=true Default value of TLAST is true.
<code>transport(data, tlast)</code>	API to send data in the form of vector of char Optional argument TLAST if <code>tlast = false</code> , No TLAST is driven if <code>tlast = true</code> , last beat has TLAST=true Default value of TLAST is true.
<code>transport(packet)</code>	API to send the AXI4-Stream packets
<code>sample_transaction(data)</code>	API to receive data in the form of vector of char Receives data beat by beat (one <code>sample_transaction()</code> call gives one beat of data).
<code>sample_transaction(data, tlast)</code>	API to receive data in the form of vector of char Receives data beat by beat (one <code>sample_transaction()</code> call gives one beat of data). This API returns the <code>tlast</code> value.
<code>sample_transaction(packet)</code>	Receives data beat by beat (one <code>sample_transaction()</code> call gives one beat of data).
<code>end_of_simulation()</code>	End and exit the simulation using this API Preferably use in External TG with <code>sw_emu/hw_emu</code>
<code>disconnect()</code>	To disconnect the socket connections (DataIn, DataOut etc) Preferably use in External TG with AI Engine sim/x86sim

xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::BLOCKING>

Instantiates the sender in the C++ code for blocking send calls.

- **API Usage:** Creates the user object using the API for blocking API calls:

```
xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::BLOCKING>
sender_obj("<sender_io_name>")
```

- **Parameters:** Name of the external port that sends the data:

```
sender_io_name
```

- **Example:**

```
xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::BLOCKING>
data_in("DataIn");
```

Here, `data_in` is the sender object and `DataIn` is the external port name.

xtlm_ipc::axis_target_socket_util<xtlm_ipc::BLOCKING>

Instantiates the receiver in the C++ code for blocking API calls.

- **API Usage:** Creates the user object for the receiver for blocking receive calls:

```
xtlm_ipc::axis_target_socket_util<xtlm_ipc::BLOCKING>  
receiver_obj("<receiver_io_name>")
```

- **Parameters:** Name of the external port that receives the data:

```
receiver_io_name
```

- **Example:**

```
xtlm_ipc::axis_target_socket_util<xtlm_ipc::BLOCKING> data_out("DataOut");
```

Here, `data_out` is the user object and `DataOut` is the external port name.

xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::NON_BLOCKING>

Instantiates the sender in the C++ code for non-blocking API calls.

- **API Usage:** Creates the user object for the sender:

```
xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::NON_BLOCKING>  
sender_obj("<sender_io_name>")
```

- **Parameters:** Name of the external port that receives the data:

```
sender_io_name
```

- **Example:**

```
xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::NON_BLOCKING>  
data_in("DataIn");
```

Here, `data_in` is the user object and `DataIn` is the external port name.

xtlm_ipc::axis_target_socket_util<xtlm_ipc::NON_BLOCKING>

Instantiates the receiver in the C++ code for non-blocking API calls.

- **API Usage:** Creates the user object using the API:

```
xtlm_ipc::axis_target_socket_util<xtlm_ipc::NON_BLOCKING>  
receiver_obj("<receiver_io_name>")
```

- **Parameters:** Name of the external port that receives the data:

```
receiver_io_name
```

- **Example:**

```
xtlm_ipc::axis_target_socket_util<xtlm_ipc::NON_BLOCKING>
data_out("DataOut");
```

Here, `data_out` is the user object and `DataOut` is the external port name.

transport(std::vector<char> data, int size(), bool tlast=true)

Sends the data if you prefer not to have fine granular control (recommended).

- **API Usage:** Calls the API using the sender object:

```
data_in.transport(data, size(), tlast)
```

- **Parameters:** The generated data in the form of vector of char:

```
data
```

The data size:

```
size()
```

The TLAST is True/False (user's choice based on the application). If not specified, the default value is true (i.e every data beat is a transaction). If `tlast = false`, no TLAST is driven. If `tlast = true` (specified by the user based on the application), the last data beat has TLAST=true.

```
tlast
```

- **Example:**

```
const unsigned int NUM_TRANSACTIONS = 8;
bool tlast;
std::vector<char> data; // Prepare data in the form of vector of char
for(int i = 0; i < NUM_TRANSACTIONS; i++) {

data = generate_data(); // user custom code to generate the data

// print(data); // user code to print the data
if (i==NUM_TRANSACTIONS-1){
    tlast = true;
}
else{
    tlast = false;
}
data_in.transport(data.data(), data.size(), tlast);
}
```

transport(std::vector<char> data, bool tlast=true)

Sends the data if you prefer not to have fine granular control (recommended).

- **API Usage:** Calls the API using the sender object:

```
data_in.transport(data, tlast)
```

- **Parameters:** The generated data in the form of vector of char:

```
data
```

The TLAST is True/False (user's choice based on the application). If not specified, default value is true (i.e every data beat is a transaction). If tlast = false, no TLAST is driven. If tlast = true (specified by the user based on the application), the last data beat has TLAST=true

```
tlast
```

transport(xtlm_ipc::axis_payload packet)

Sends the stream packets if you need fine granular control.

- **API Usage:** Calls the API using the sender object:

```
sender_obj.transport(packet)
```

- **Parameters:** AXI stream packet:

```
packet
```

This AXI stream has the following attributes to be set. The data that needs to be sent:

```
data
```

The TLAST value to mark the end of the packet. This is the user's choice to be set based on the application.

```
tlast
```

Some other optional AXI stream packet attributes are:

```
tuser
```

```
tkeep
```

- **Example:**

```
const unsigned int NUM_TRANSACTIONS = 8;

xtlm_ipc::axis_payload& packet;

for(int i = 0; i < NUM_TRANSACTIONS; i++) {
    //generate_data() is your custom code to generate traffic

    std::vector<char> data = generate_data();
```

```

//! Set packet attributes...
packet.data = data;
packet.tlast = 1; //Additional AXIS attributes can be set if required
//Blocking transport API to send the transaction
data_in.transport(packet);
}

```

sample_transaction(std::vector<char>& data)

Receives the data if you prefer not to have fine granular control (recommended).

- **API Usage:** Calls the API using the receiver object.
- **Parameters:** The empty vector of char to receive the data:

```
data
```

- **Example:**

```

const unsigned int NUM_TRANSACTIONS = 100;
unsigned int num_received = 0;
std::vector<char> data; // Empty vector for receiving transactions

while(num_received < NUM_TRANSACTIONS) {
    data_out.sample_transaction(data); // sample_transaction fills the data
    vector
    // print(data); user code to print the data
    num_received += 1; // user logic to process data further
}

```

Receives data available on the interface up to TLAST. For example, if the master generates TLAST every 16 beats, then the vector<> will be populated with 16 beats. If the master does not drive TLAST, then every beat is considered a transaction by itself (i.e one beat per call of the API).

sample_transaction(std::vector<char>& data, bool& tlast)

Receives data if you prefer not to have fine granular control (recommended).

Here `bool tlast` is passed as an argument to check if the current data is the last beat of transaction.

You can use this `tlast` for further processing.

- **API Usage:** Calls the API using the receiver object
- **Parameters:** The empty vector of char to receive the data:

```
data
```

This is the received tlast value that the API returns:

```
tlast
```

- **Example:**

```
const unsigned int NUM_TRANSACTIONS = 100;
unsigned int num_received = 0;
std::vector<char> data; // Empty vector of char
bool recv_tlast;

while(num_received < NUM_TRANSACTIONS) {
    data_out.sample_transaction(data, recv_tlast); // data vector will be
    filled

    // If the current sample has tlast as true, recv_tlast will be set to
    true
    if (recv_tlast) {
        // do relevent logic if tlast is true
    }
    num_received += 1; // user logic to process data further
}
```

Receives data available on the interface up to TLAST. For example, if the master generates TLAST every 16 beats, then the vector<> will be populated with 16 beats. If the master does not drive TLAST, then every beat is considered a transaction by itself (i.e one beat per call of the API).

sample_transaction(xtlm_ipc::axis_payload& packet)

Receives AXI stream packets.

- **API Usage:** Calls the API using the receiver object:

```
receiver_obj.sample_transaction(packet)
```

- **Parameters:** Empty stream packet to be filled with samples:

```
packet
```

- **Example:**

```
unsigned int num_received = 0;
xtlm_ipc::axis_payload packet;

data_out.sample_transaction(packet); //API to sample the transaction
which fills the empty packet of type xtlm_ipc::axis_payload

//Process the packet as per requirement.
num_received += 1;
```

end_of_simulation();

Ends and exits the simulation.

- **API Usage:** Calls the API using the sender object:

```
sender_obj.end_of_simulation()
```

- **Example:**

```
//! Instantiate IPC socket with name matching in IPI diagram...
xtlm_ipc::axis_initiator_socket_util<xtlm_ipc::BLOCKING>
data_in("DataIn");

const unsigned int NUM_TRANSACTIONS = 100;
xtlm_ipc::axis_payload packet;

std::cout << "Sending " << NUM_TRANSACTIONS << " Packets..." <<
std::endl;
for (int i = 0; i < NUM_TRANSACTIONS; i++) {
    packet = generate_packet();
    print(packet);
    socket_util.transport(packet);
}
file.close();
usleep(10000);
data_in.end_of_simulation();
```

Preferably used with sw_emu/hw_emu.

barrier_wait()

For non-blocking, the API waits for all transactions to complete.

- **API Usage:** Calls the API using the sender object.

- **Example:**

```
const unsigned int NUM_TRANSACTIONS = 8;
std::vector<char> data;
std::cout << "Sending " << NUM_TRANSACTIONS << " data transactions..."
<<std::endl;

for(int i = 0; i < NUM_TRANSACTIONS/2; i++) {
    data = generate_data();
    print(data);
    data_in.transport(data.data(), data.size());
}

// "Adding Barrier to complete all outstanding transactions..." <<
std::endl;
data_in.barrier_wait();

for(int i = NUM_TRANSACTIONS/2; i < NUM_TRANSACTIONS; i++) {
    data = generate_data(); // user code to generate the data
    print(data); // user code to print the data
    data_in.transport(data.data(), data.size());
}
```

get_num_transactions()

For non-blocking, the API is used to check if any transaction is available to receive.

This method is useful in the non-blocking version because `sample_transaction` may not fill the vector/payload if no transaction is available. It is recommended to check before calling `sample_transaction` with `NON_BLOCKING` receiver.

- **API Usage:** Calls the API using the sender object.
- **Example:**

```
xtlm_ipc::axis_target_socket_util<xtlm_ipc::NON_BLOCKING>
receiverObj("Receiver");
xtlm_ipc::axis_payload receiverPayload;
// std::vector<char> receiveData;
if ( receiverObj.get_num_transactions() != 0 ) { // Checks if transaction
is available
receiverObj.sample_transaction(receiverPayload); // Fills the
receiverPayload

// One can use with vector<char> as parameter also
// such as receiverObj.sample_transaction(receiveData);
}
```

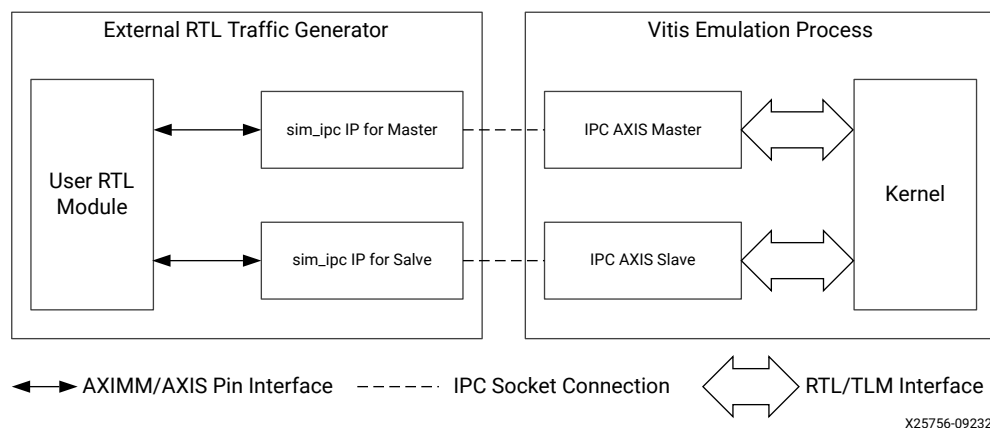
External RTL Traffic Generators using SV/Verilog

Generate traffic using the existing test bench written in the System Verilog/Verilog with slight modification to your test bench hierarchy, as explained below.

External RTL Traffic Generator and Emulation Process

External RTL traffic generators are used to drive traffic to Vitis emulation process or AI Engine simulation process using SystemVerilog or Verilog modules.

Figure 41: Test Bench Hierarchy



As shown in the figure above, the external test bench (on the left) and the Vitis emulation (on the right), both run as separate simulation processes. To establish communication between two processes using IPC, you must instantiate `SIM_IPC` Master/Slave modules.

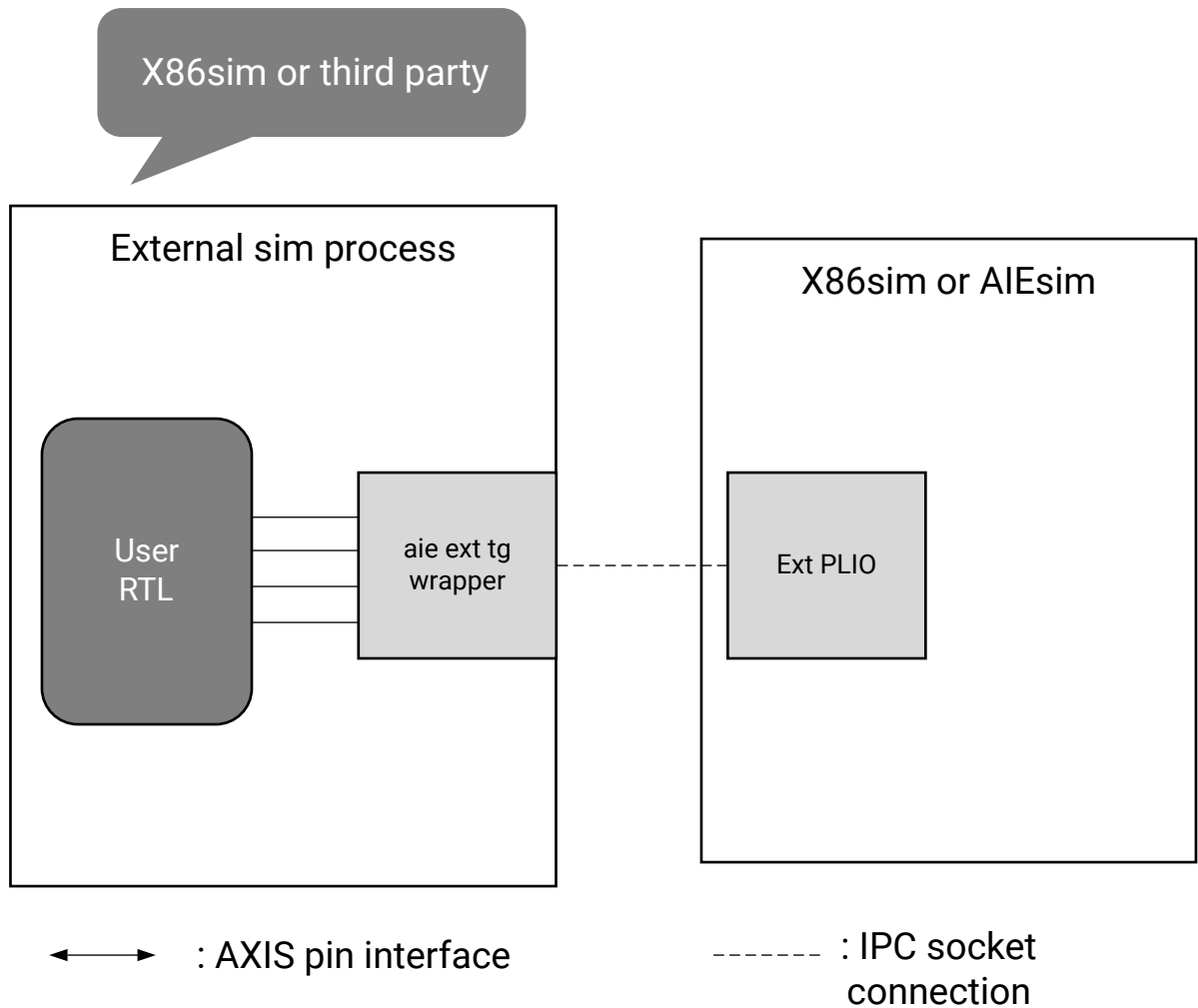
Perform the following modifications:

1. You need to create a project in Vivado simulator. For details on how to create a project, refer *Vivado Design Suite User Guide: Design Flows Overview* ([UG892](#))
2. Once the project is created, you need to instantiate `sim_ipc` IP in the external SV/Verilog testbench.
3. Then run the `export_simulation` command in Vivado to generate the scripts for the simulation
4. Run the simulation in Vivado simulator. For details running simulation refer to *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

External RTL Traffic Generator and AI Engine Simulation

The same technique can be deployed to drive traffic from the external System Verilog/Verilog traffic generators/test benches to the AI Engine simulator or x86-simulator.

Figure 42: XTLM Test Bench Hierarchy



To generate the AI Engine wrapper stub module (`aie_wrapper_ext_tb.v`) use the following steps:

1. The wrapper stubs will be generated based on the external PLIO declarations in the ADF graph. You need to perform the ADF graph compilation to generate `scsim_config.json` file that resides in `./Work/config/scsim_config.json` directory. This config file contains information on the PLIOs declared in the graph. For more details on how to perform ADF graph compilation and external PLIOs declaration, refer to *AI Engine Tools and Flows User Guide* (UG1076).
2. You can use this config file as argument to the `gen_aie_wrapper.py` script to auto-generate Verilog stub modules based on ext PLIO declared in ADF Graph:

```
python3 ${XILINX_VITIS}/data/emulation/scripts/gen_aie_wrapper.py \
-json Work/config/scsim_config.json --mode <wrapper/vivado>
```



TIP: The python script is available in the Vitis installation area as shown in the example above. There are two modes for the script: wrapper and Vivado mode. By default, the script runs in Vivado mode.

The name of the instance stubs must be identical to the name of the corresponding external PLIOs in the graph and will be reflected in the generated `aie_wrapper_ext_tb.v` file.

After running the `gen_aie_wrapper.py` script, the `aie_wrapper_ext_tb.v` is generated with instances of `sim_ipc_axis` modules that can be directly instantiated in your external test bench.

Note: The module used to send data to/from external traffic generator to AI Engine simulator/x86sim are the XTLM IPC SystemC modules which are present inside the wrapper stub module which includes all the XTLM IPC modules. This wrapper needs to be instantiated in the external test bench to establish the connection as shown in the figure above.

Instantiating AI Engine Wrapper in the Test Bench

The aie wrapper module (`aie_wrapper_ext_tb.v`) needs to be instantiated in the external test bench. The `aiesim` expects data to be in beats instead of transaction, so you need to keep `tlast` at high (`1'b1`) all the time.

Note: You can add timescale directive as per your requirement in `aie_wrapper_ext_tb.v`.

Generating sim_ipc_axis IP for Vivado Project

By default, the python script generates `aie_wrapper_ext_tb_ip.tcl` and `aie_wrapper_ext_tb_proj.tcl` along with the wrapper Verilog file as mentioned in the previous section.

There are two ways to proceed based on the existence of a Vivado project:

1. If you have already created Vivado project, use the IP flow described here. From the **Tcl console** source the `aie_wrapper_ext_tb_ip.tcl` script:

```
source <absolute_path>/aie_wrapper_ext_tb_ip.tcl
```

This Tcl script can be used for generating required `sim_ipc_axis` IP. After sourcing the Tcl file you will see hierarchy created under `simulation_sources`. You can add the required files and directories for your project .

2. If a Vivado project is not already created, use the project script `aie_wrapper_ext_tb_proj.tcl` to create one. From a terminal use the following command:

```
vivado -mode batch -source aie_wrapper_ext_tb_proj.tcl
```

Note: To use third party simulators, you need to update the required paths for `SIMULATOR_GCC_PATH`, `SIMULATOR_CLIBS_PATH` and `INSTALL_BIN_PATH`. For more details on how to set the third party simulators, refer to the Logic Simulation chapter of *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

After sourcing `aie_wrapper_ext_tb_proj.tcl`, the tool will generate the `export_sim` directory with sub-directories and scripts required for use with other simulators. This Tcl script sources the `aie_wrapper_ext_tb_ip.tcl` script.



TIP: The scripts mentioned above only contain the `sim_ipc_axis` modules, so you must add any additional required RTL modules and options to the script. You can modify and directly include required RTL the needed script.

Running the RTL Traffic Generator with AI Engine Simulation

You can launch the external process using the following steps:

1. If already inside the Vivado project, once the project hierarchy is updated after adding the required sources, you can run the simulation from Vivado. Refer to *Vivado Design Suite User Guide: Logic Simulation (UG900)* for more information.
2. If outside the Vivado project, once the `export_sim` directory is generated with required simulation scripts, you can traverse inside the appropriate simulator directory and run the `<top_module_name>.sh` script to launch the RTL simulation.
3. Also, simultaneously launch the AI Engine simulator process as already mentioned. For details on how to run the `aiesimulator` command, refer to *AI Engine Tools and Flows User Guide (UG1076)*.

After simulation is launched you can see the traffic propagating to and from the user RTL.



TIP: For more details on how to integrate and launch the external RTL traffic generator with AI simulation process refer to the tutorial.

Running Traffic Generators in Python/C++/MATLAB

After generating an external process binary as shown above using the headers and sources available at `$XILINX_VIVADO/data/emulation/ip_utils/xtlm_ipc/xtlm_ipc_v1_0/<supported_language>`, you can run the emulation using the following steps:

1. For C++, set the `LD_LIBRARY_PATH` as `export LD_LIBRARY_PATH=$XILINX_VIVADO/data/emulation/cpp/lib:$LD_LIBRARY_PATH`
2. For Python, set the `PYTHONPATH` as: `export PYTHONPATH=$XILINX_VIVADO/data/emulation/hw_em/lib/python:$XILINX_VIVADO/data/emulation/python/xtlm_ipc/`
3. For MATLAB, set the following inside the script:

```
vivado = getenv("XILINX_VIVADO");
libPath = fullfile(vivado, "/data/emulation/matlab/xtlm_ipc");
addpath(libPath)
```

- For Advanced C++ Traffic Generator APIs, launch the AMD Vitis™ emulation, the AMD Vivado™ simulation or x86sim/AI Enginesim using the standard process and wait for the simulation to start. For AI Engine specific APIs, use `$XILINX_VIVADO/data/emulation/python/xtlm_ipc_v2/`.
- From another terminal, launch the external process such as Python/C++/C. If you are running multiple I/O or traffic generator-based solutions on the same machine, set `XTLM_IPC_SOCKET_DIR` to be unique to each test case on both the emulation terminal in addition to the external process terminal. For example, `setenv XTLM_IPC_SOCKET_DIR <test_case_dir>` (same environment on both emulation process and external process).

Note: The traffic generator executable and `hw_emu/sw_emu` or `x86sim/aiesim` should be run on the same server/machine.



WARNING! AMD provides an `end_of_simulation()` API to terminate emulation from master utilities of memory mapped AXI4 and AXI4-Stream interfaces. However, you are warned not to use this method unless there is no way to terminate emulation from host. In a normal course of emulation, external process is not expected to terminate emulation. Use this in exceptional scenarios.

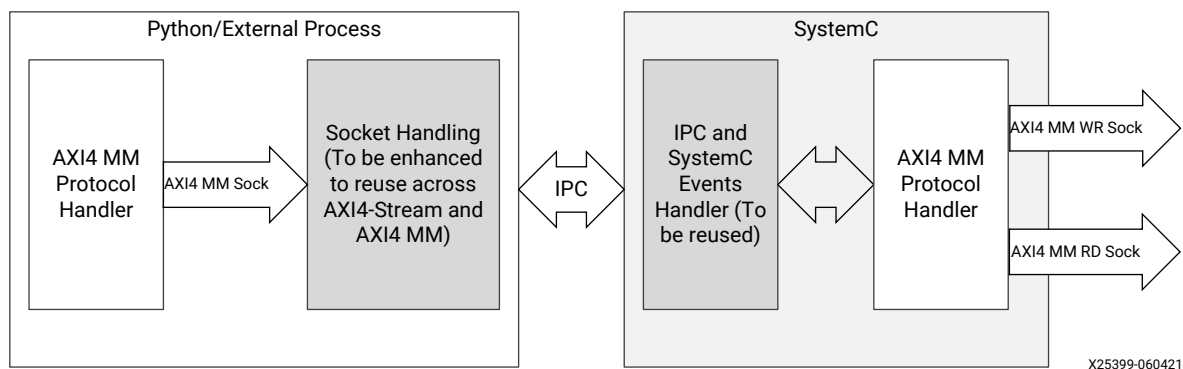
AXI4 Memory Map External Traffic through Python/C++

The AXI4 memory map external traffic which is supported only for hardware emulation has the following specifications:

- Only transaction-level granularity is supported.
- Re-ordering of transactions is not supported.
- Parallel Read, Write transactions are not supported (transactions will be serialized).
- Unaligned transactions are not supported.

The following figure shows the high-level design.

Figure 43: AXI4 Memory Map External Traffic Design



Use Cases

The use cases include the following:

- Emulate AXI4 memory map Master/Slave through an external process such as Python/C++. This can help you with emulating design with quick design time of AXI4 Master/Slave without investing resources in developing AXI4 Master.
- Chip-to-chip connection between two FPGAs can be emulated with AXI4 memory map Interprocess communication.

Writing Traffic Generators in C++

For API/pseudo code, a single instance of AXI4 memory map transaction is used for the complete transaction. This is in line with how payload is used in the AMD SystemC modules. For AXI4 Master, there is a `b_transport(aximm_packet)` API. After the call, `aximm_packet` is updated with a response given by AXI4 Slave. For AXI4 Slave, there are `sample_transaction()` and `send_response(aximm_packet)` APIs.

The following code snippets show the API usage in the context of C++.

- Code snippet for C++ Master:

```
auto payload = generate_random_transaction(); //Custom Random transaction
generator. Users can configure AXI properties on the payload.
/* Or User can set the AXI transaction properties as follows
payload->set_addr(std::rand() * 4);
payload->set_len(1 + (std::rand() % 255));
payload->set_size(1 <= (std::rand() % 3));
*/

master_util.b_transport(*payload.get(), std::rand() % 0x10); //A blocking
call. Response will be updated in the same payload. Each AXI MM
transaction will use same payload for whole transaction
std::cout << "-----Transaction Response-----" << std::endl;
std::cout << *payload << std::endl; //Prints AXI transaction info
```

- Code snippet for C++ Slave:

```
auto& payload = slave_util.sample_transaction(); // Sample the transaction

//If it is read transaction, give read data
if(payload.cmd() == xtlm_ipc::aximm_packet_command_READ)
{
    rd_resp.resize(payload.len()*payload.size());
    std::generate(rd_resp.begin(), rd_resp.end(), []()
    { return std::rand()%0x100;});
}

//Set AXI response (for Read & Write)
payload.set_resp(std::rand()%4);
slave_util.send_response(payload); //Send the response to the master
```

Writing Traffic Generators in Python

The following code snippets show the API usage in the context of Python.

You need to set PYTHONPATH as follows:

- For example, on C Shell:

```
setenv PYTHONPATH $XILINX_VIVADO/data/emulation/hw_em/lib/python:
$XILINX_VIVADO/data/emulation/ip_utils/xtlm-ipc/xtlm-ipc-v1_0/python
```

- Code snippet of Python Master:

```
aximm_payload = xtlm_ipc.aximm_packet()
random_packet(aximm_payload) # Custom function to set AXI Properties
randomly
#Or user can set AXI properties as required
#aximm_payload.addr = int(random.randint(0, 1000000)*4)
#aximm_payload.len = random.randint(1, 64)
#aximm_payload.size = 4

master_util.b_transport(aximm_payload)
#After this call aximm_payload will have updated response as set by the
AXI Slave.
```

- Code snippet of Python Slave:

```
aximm_payload = slave_util.sample_transaction()
aximm_payload.resp = random.randint(0,3)
if not aximm_payload.cmd: #if it is a read transaction set Random data
    tot_bytes = aximm_payload.len * aximm_payload.size
    for i in range(0, int(tot_bytes/SIZE_OF_EACH_DATA_IN_BYTES)):
        aximm_payload.data += bytes(bytearray(struct.pack(">I",
            random.randint(0,60000)))) # Binary data should be aligned with C struct

slave_util.send_resp(aximm_payload)
```

AXI4 Memory Map I/O Limitations in the Platform

The following shows the AXI4 memory map I/O limitations in the platform:

- During platform development, AXI4 memory map I/O can be connected to any memory/slave.
- Master AXI4 memory map I/O cannot connect to kernel as kernel cannot provide an additional slave interface.
- AXI4 memory map Slave I/O can be used without any restrictions.
- AXI4 memory map Master I/O can be used where data needs to be driven from external process to memory/slave.

XO Usage

The use cases of AXI4 memory map I/O XO differs from AXI4-Stream I/O XO. AXI4 memory map XOs have few limitations on usage during the link stage of Vitis listed as below:

- Only AXI4 memory map Master I/O can be used.
- AXI4 memory map Master I/O can connect only with available slaves in the platform.
- AXI4 memory map Master I/O cannot communicate with kernel in the design.

For XO usage during link stage:

- To generate XO, developers can use the script available at `$XILINX_VITIS/data/emulation/XO/scripts/aximm-xo-creation.sh`
- Required configuration of XO can be generated using the above script.

```
$XILINX_VITIS/data/emulation/XO/scripts/aximm-xo-creation.sh --  
address_width <adr_width> --data_width <data_width> --id_width <id_width>  
--output_path <output_path>.xo  
$XILINX_VITIS/data/emulation/XO/scripts/aximm-xo-creation.sh --  
address_width 64 --data_width 64 --id_width 4 --output_path  
sim_ipc_aximm-master.xo
```

- After generating XO, it can be used in the design with configuration as shown below (sample usage, actual connection to be done based on the requirement):

```
[connectivity]  
nk=sim_ipc_aximm-master:1:aximm-master  
sp=aximm-master.M_AXIMM:HBM[0]
```

Profiling and Debugging the Application

Running the system, either in emulation or on the system hardware, presents a series of potential challenges and opportunities. Running the system for the first time, you can profile the application to identify bottlenecks, or performance issues that offer opportunities to optimize the design, as discussed in the sections below. Of course, running the application can also reveal coding errors, or design errors that need to be debugged to get the system running as expected.

This section contains the following chapters:

- [Profiling the Application](#)
- [Debugging System Projects](#)

Profiling the Application

The AMD Vitis™ core development kit generates various system and kernel resource performance reports during compilation. These reports help you establish a baseline of performance for your application, identify bottlenecks, and help to identify target functions that can be accelerated in hardware kernels as discussed in [Methodology for Architecting a Device Accelerated Application](#). The Xilinx Runtime (XRT) collects profiling data during application execution in both emulation and hardware builds. Examples of profiling and event data that can be reported includes:

- Host and device timeline events
- OpenCL™ or XRT native API call sequences
- Kernel execution sequence
- Kernel start and stop signals
- FPGA trace data including AXI transactions
- Power profile data for the accelerator card
- AI Engine profiling and event trace
- User event and range profiling

Profiling reports and data can be used to isolate performance bottlenecks in the application, identify problems in the system, and optimize the design to improve performance. Optimizing an application requires optimizing both the application host code and any hardware accelerated kernels. The host code must be optimized to facilitate data transfers and kernel execution, while the kernel should be optimized for performance and resource usage.

There are four distinct areas to be considered when performing algorithm optimization in the Vitis environment: System resource usage and performance, kernel optimization, host optimization, and data transfer optimization. The following Vitis reports and graphical tools support your efforts to profile and optimize these areas:

- [Guidance](#)
- [System Estimate Report](#)
- [HLS Synthesis Report](#)
- [Profile Summary Report](#)
- [Timeline Trace](#)

- [Waveform View and Live Waveform Viewer](#)

When enabled as described in [Enabling Profiling in Your Application](#), these reports are automatically generated while running the active build, either from the command line as described in [Section IV: Building and Running the System](#), or when [Section VIII: Using the Vitis Unified IDE](#). Separate reports are generated for the different build targets and can be found in the respective report directories. Reports can be viewed in the Analysis view in the IDE as described in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Enabling Profiling in Your Application

To enable profiling and capturing event trace data during the execution of your application, you must instrument your application for this task. You must enable additional logic, and consume additional device resources to track the host and kernel execution steps, and capture event data. This process requires optionally modifying your host application to capture custom data, modifying your kernel XO during compilation and the `xclbin` during linking to capture different types of profile data from the device side activity, and configuring the Xilinx Runtime (XRT) as described in the [xrt.ini File](#) to capture data during the application runtime.



TIP: While capturing profile data is a critical part of the profiling and optimization process for building your accelerated application, it does consume additional resources and impacts performance. You should be sure to clean these elements out of your final production build.

There are many different types of profiling for your applications, depending on which elements your system includes, and what type of data you want to capture. The following table shows some of the levels of profiling that can be enabled, and discusses which are complimentary and which are not.

Table 34: Profiling Host and Kernels

Profile/Trace	Description	Comments
Host Application OpenCL API and some limited device side (kernel) profiling.	Specified by the use of the <code>opencl_trace</code> option in the <code>xrt.ini</code> file.	Generates the <code>opencl_trace.csv</code> file and the <code>xrt.run_summary</code> for viewing in Vitis analyzer.
Host Application XRT Native API	Specified by the use of the <code>native_xrt_trace</code> option in the <code>xrt.ini</code> file.	Generates profile summary and trace events for the XRT API as described in Writing the Software Application .
Host Application User-Event Profiling	Requires additional code in the host application as described in Custom Profiling of the Host Application .	Generates user range data and user events for the host application. TIP: Can be used to capture event data for user-managed kernels as described in Working with User-Managed Kernels .

Table 34: Profiling Host and Kernels (cont'd)

Profile/Trace	Description	Comments
Low Overhead Profiling	Specified by the use of the <code>lop_trace</code> option in the <code>xrt.ini</code> file.	Generates the <code>lop_trace.csv</code> file as described in Enabling Low Overhead Profiling .
Device Side Profiling	Enabled by the use of <code>--profile</code> options during <code>v++</code> compilation and linking, as described in --profile Options , and the use of <code>device_trace</code> in the <code>xrt.ini</code> file.	Enables capturing data traffic between the host and kernel, kernel stalls, the execution times of kernels and compute units (CUs), in addition to monitoring activity in AMD Versal™ AI Engines.
AI Engine Graph and Kernels	Specified by the use of the <code>aie_profile</code> and <code>aie_trace</code> options in the <code>xrt.ini</code> file. These options can be specified together or separately.	Generates the <code>default.aierun_summary</code> report containing the Profile and/or Trace reports. The <code>aierun_summary</code> can be found in the <code>aiesimulator_output</code> folder of the AI Engine graph build directory. Refer to the AI Engine Simulation-Based Profiling chapter in the <i>AI Engine Tools and Flows User Guide</i> (UG1076) for more information.
Power Profile	Specified by the use of the <code>power_profile</code> option in the <code>xrt.ini</code> file.	Generates the <code>power_profile-<device>.csv</code> report. Note: This feature is not supported on embedded platforms or AWS.
Vitis AI Profiling	Specified by the use of the <code>vitis_ai_profile</code> option in the <code>xrt.ini</code> file.	Enables counter profiling of DPUs to generate the <code>opencl_summary.csv</code> file and the <code>xrt.run_summary</code> for viewing in Vitis analyzer.

The device binary (`xclbin`) file is configured for capturing limited device-side profiling data by default. However, using the `--profile` option during the Vitis compiler linking process instruments the device binary by adding Acceleration Monitors, AXI Performance Monitors, and Memory Monitors to the system. This option has multiple instrumentation options: `--profile.data`, `--profile.stall`, and `--profile.exec`, as described in the [--profile Options](#).

As an example, add `--profile.data` to the `v++` linking command line:

```
v++ -g -l --profile.data all:all:all ...
```



TIP: Be sure to also use the `v++ -g` option when compiling your kernel code for debugging with software or hardware emulation.

After your application is enabled for profiling during the `v++` compile and link process, data gathering during application runtime must also be enabled in XRT by editing the `xrt.ini` file as discussed above. For example, the following `xrt.ini` file enables OpenCL profiling, power profiling, and event and stall trace capture when the application is run:

```
[Debug]
opencl_trace=true
power_profile=true
device_trace=fine
stall_trace=all
```

To enable the profiling of Kernel Internals data, you must also add the `debug_mode` tag in the `[Emulation]` section of the `xrt.ini`:

```
[Emulation]
debug_mode=batch
```

If you are collecting a large amount of trace data, you can increase the amount of available memory for capturing data by specifying the `--profile.trace_memory` option during `v++` linking, and add the `trace_buffer_size` keyword in the `xrt.ini`.

- `--profile.trace_memory`: Indicates what type of memory to use for capturing trace data.
- `trace_buffer_size`: Specifies the amount of memory to use for capturing the trace data during the application runtime.



TIP: When `--profile.trace_memory` is not specified but `device_trace` is enabled in the [xrt.ini File](#), the profile data is captured to the default platform memory with 1 MB allocated for the trace buffer size.

Finally, as discussed in [Continuous Trace Capture](#) you can enable continuous trace capture to continuously offload device trace data while the application is running, so in the event of an application or system crash, some trace data is available to help debug the application.

Continuous Trace Capture

The Vitis tool supports recording continuous trace data while the application is running. The application can run for a very long time thus leading to the capture of significant trace data, which can result in issues like incomplete trace data especially when the memory resource used for trace data is not large enough. Using continuous trace, analysis of the trace can be carried out while the application is still running or if the application has crashed before completion.

With the ability to continuously capture trace data, the Timeline Trace reports can be dynamically updated in the Vitis analyzer tool while your application is running. Once these reports are loaded in Vitis Analyzer, there is a hyperlink available indicating that the current report is being modified on the disk. If new data needs to be loaded, **Reload** or **Auto-Reload** options are available on the banner to let you view the updated report as your application runs and trace data is generated.

Continuous trace is not enabled by default. Additionally, the memory resources of an FPGA are not unlimited. So if the application generates large trace data, a circular buffer for storing the data can be used. The circular buffer can be written, offloaded to the host, and reused again. By enabling a circular buffer with continuous trace, the memory resources needed are even smaller thus saving available resources on the device. However, an application run with continuous trace/circular buffer may result in multiple device trace files.



TIP: For Hardware emulation, only host side continuous trace is available, for hardware runs both host side and device side continuous trace are available.

Here are some scenarios where it is recommended to use the memory resource as a circular buffer.

The circular buffer implementation is automatically turned on when continuous trace is enabled in the `xrt.ini`. The flow requires the following settings for enabling continuous trace.

- In the `xrt.ini` file, `continuous_trace` is set to `TRUE`
- v++ linking option `--profile.trace_memory` is set to `DDR` or `HBM`

You can optionally set:

- The size of the trace buffer using `trace_buffer_size` in the `xrt.ini` file. This defaults to 1 MB.
- The interval at which the trace buffer is offloaded from the device using `trace_buffer_offload_interval_ms` in the `xrt.ini` file. The default is 10 ms.
- The interval at which files are dumped by setting `trace_file_dump_interval_s`. The default is 3 seconds.



IMPORTANT! Circular Buffer can be force enabled by setting `trace_buffer_offload_interval_ms` to 0 ms.

As an example, if you enable `continuous_trace` with `trace_buffer_size` as 8k and default `trace_buffer_offload_interval_ms` of 10 ms, the trace data rate is 819200 bytes/s which is less than the default of 100 MB/s. In this scenario, the circular buffer is *NOT* enabled by default and an XRT warning is reported:

```
[XRT] WARNING: Unable to use circular buffer for continuous trace offload.
Please increase trace buffer size and/or reduce continuous
trace interval. Minimum required offload rate (bytes per second) :
104857600 Requested offload rate : 819200
```

Here is an example of `xrt.ini` settings:

```
[Debug]
opencl_trace=true
device_trace=fine
stall_trace=all
continuous_trace=true
```

```
// The following are optional and needed only in rare circumstances

trace_buffer_size=20M
trace_buffer_offload_interval_ms=10
trace_file_dump_interval_s=2
```

The following are the results of these settings:

- `opencl_trace`: Enables the generation of host-related OpenCL API trace, `opencl_trace.csv` files is created.
- `device_trace`: Enables the collection of kernel activity to be added to profile summary and trace, `device_trace_0.csv` files are created with 0 being the device number.
- `stall_trace`: Enables the hardware generation of stalls into compute units.
- `continuous_trace`: Enables the continuous dumping of files for trace and the continuous reading of device data into the host.
- `trace_buffer_size`: Specifies the amount of memory to consume for trace data capture.
- `trace_buffer_offload_interval_ms`: Controls the reading of device data from the device to the host in milliseconds.
- `trace_file_dump_interval_s`: Controls the time between dumping of trace files in seconds.

As a result, there are several CSV files generated in addition to the `xrt.run_summary` as part of the application run using the above `xrt.ini` file. Vitis Analyzer only needs the generated `run_summary` file and will use the relevant CSV files to display the profile summary and timeline trace.

Here are the recommendations on setting up an application for trace data dumping:

1. By default the memory used for trace capture is the first memory resource on the platform, which can be determined using the [platforminfo Utility](#). In most platforms this is either DDR or HBM. The amount of memory reserved for trace data is determined by the `trace_buffer_size` switch in the `xrt.ini` file, which defaults to 1 MB.
Note: You can also specify the use of FIFO and the size to allocate using the `--profile.trace-memory` option.
2. If still unable to dump maximum trace, disable stall trace by setting `stall_trace=off` or `stall_trace=on` with `data_transfer_trace=coarse`.
3. If the application requires larger size of trace buffer, enable circular buffer by setting `continuous_trace=true` with default settings of `trace_buffer_offload_interval_ms=10` and `trace_file_dump_interval_s=5`. Ideally, a continuous trace feature should be used for the following cases:
 - Long-running design with minimal trace generated
 - Debugging application crashes where some `.csv` files might still be available for debugging

4. If the application run is still unable to dump the maximum trace, the `trace_buffer_size` can further be increased.
5. If the application still creates huge trace data that the host cannot keep up, use the smaller size of `trace_file_dump_interval`, which creates multiple files equivalent to the interval provided.
6. Lastly, continuous trace can generate several trace files as part of the application run in addition to `xrt.run_summary` file. The Vitis Analyzer only needs the generated `run_summary` file and can pick the relevant CSV files generated to display profile summary and timeline trace to provide a better experience.

Custom Profiling of the Host Application

All XRT related actions from the host application are automatically tracked for profiling, through either the OpenCL API calls, or the XRT API calls. However, you can also profile the host application beyond the XRT related events, capturing event data based on user-specified actions or events.

This feature provides two types of custom profiling:

- **User range:** Profiles the specified start/end times across a range of code. This captures the span of time within which an action occurs in the host application.
- **User events:** Marks an event in the timeline. The user event is added to the timeline waveform at whatever point in time it occurs.

The `user_range` and `user_event` data can be captured to the Profile Summary and Timeline Trace reports for display in Vitis analyzer. As seen in the figure below, the Profile Summary shows the number of occurrences of a given event and the range. The User Ranges table also reports the Min/Max/Avg/Total duration of the user-defined ranges in the host code. In the Timeline Trace report `user_range` elements in the host code are displayed in a separate row, and `user_event` markers are added at specific points on the timeline.

Figure 44: Profile Summary – User Range

Q

⌕

⚙

ℹ

Settings

Summary

✓ Kernels & Compute Units

Kernel Execution

Top Kernel Execution

Compute Unit Utilization

Compute Unit Stalls

✓ Kernel Data Transfers

Kernel Transfer

Top Kernel Transfer

✓ Host Data Transfers

Host Transfer

Top Memory Writes

Top Memory Reads

API Calls

User Events & Ranges

User Events & Ranges

User Events

Label	Count
Kernel Done	1
Kernel Enqueued	1

User Ranges

Label	Count	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	Description
Buffer Creation	2	0.096	0.044	0.048	0.052	Setting up Buffer
Host2Device Migrate	2	0.558	0.09	0.279	0.468	Migrating Buffers from Host to Device
Device2Host Migrate	2	0.221	0.11	0.111	0.111	Migrating Buffers from Device to Host

Using custom profiling requires a few changes in your host application source code and build process. You must make use of C or C++ API in your code, as described below, and you must include the `xrt_coreutil` library when linking your host application.

- The C/C++ API are described below, but can also be found at the following URL: https://github.com/Xilinx/XRT/blob/master/src/runtime_src/core/include/experimental/xrt_profile.h.
- For both C and C++ you must add the following:

```
#include experimental/xrt_profile.h
```

- When linking host code, add `-lxrt_coreutil` to the compiler command line.



TIP: An example of `user_range` and `user_event` can be seen in the host code at https://github.com/Xilinx/Vitis_Accel_Examples/blob/master/host/debug_profile/src/host.cpp.

Profiling of C++ Code

For C++ code the provided objects are:

- `user_range`:** This object captures the start time and end time of a measured range of activity with the specified ID. The object constructor is:

```
user_range(const char* label, const char* tooltip);
```

- `user_event`:** This object marks an event occurring at single point in time, adding the specified label onto the timeline trace. The object constructor is:

```
user_event()
```

Use the `user_range` to construct an object and start keeping track of time immediately upon construction. Usage details of the `user_range` objects:

- If a `user_range` is instantiated using the default constructor, no time is marked until the user calls `user_range.start()` with the label and tooltip.
- You can instantiate a `user_range` object passing the label and tooltip strings. This starts monitoring the range immediately.
- You must call `user_range.start()` and `user_range.end()` to capture ranges of time you are interested in.
- If `user_range.end()` is not called, then any range being tracked lasts until the `user_range` object is destructed.
- The `user_range` object can be reused any number of times, by calling `user_range.start()/user_range.end()` pairs in the host code.
- Sequential calls to `user_range.start()` ignore all but the first call until `user_range.end()` terminates the range.
- Sequential calls to `user_range.end()` ignore all but the first call until `user_range.start()` starts a new range.

Usage of the `user_event` objects:

- A `user_event` object must be instantiated using the default constructor.
- Calls to `user_event.mark()` creates a user marker on the timeline trace at that particular time.
- `user_event.mark()` takes an optional `const char*` argument which appears as a label on the timeline trace.

The [debug_profile](#) example of the [Vitis_Accel_Examples](#) demonstrates user event profiling in a host application. With your host application properly instrumented, XRT can capture profile data from these user-defined ranges and events, in addition to the standard XRT API-based events. You must enable profiling in the `xrt.ini` file as explained previously.

Profiling of C Code

For C code the provided functions are:

- `xrtURStart()`: This function establishes the start time of a measured range of activity with the specified ID. The function signature is:

```
void xrtURStart(unsigned int id, const char* label, const char* tooltip)
```

- `xrtUReEnd()`: This function marks the end time of a measured range with the specified ID. The function signature is:

```
void xrtUReEnd(unsigned int id)
```

- `xrtUEMark()`: This function marks an event occurring at single point in time, adding the specified label onto the timeline trace. The function signature is:

```
void xrtUEMark(const char* label)
```

Use the `xrtURStart()` and `xrtUREnd()` functions to start keeping track of time immediately, and specify an ID to pair the start/end calls and define the user range. Usage details of the `user_range` functions:

- Start/End ranges of one ID can be nested inside other Start/End ranges of a different ID.
- It is your responsibility to make sure the IDs match for the Start/End range you are profiling.



IMPORTANT! Multiple calls to `xrtURStart` and `xrtUREnd` with the same ID can cause unexpected behavior.

- The user range can have a label that is added to the timeline, and a tooltip that is displayed when you place the cursor over the user range.

A call to `xrtUEMark()` will create a user marker on the timeline trace at the point of the event.

- `xrtUEMark()` lets you specify a label for the event. The label will appear on the timeline with the mark.
- You can use `NULL` for the label to add an unlabeled mark.

The following is example code:

```
int main(int argc, char* argv[]) {
    58
    59     xrtURStart(0, "Software execution", "Whole program execution") ;
    60     ...
    61     //TARGET_DEVICE macro needs to be passed from gcc command line
    62     if(argc != 2) {
    63         std::cout << "Usage: " << argv[0] << " <xclbin>" << std::endl;
    64         return EXIT_FAILURE;
    65     }
    ....
    153     q.enqueueTask(krnl_vector_add);
    154
    155     // The result of the previous kernel execution will need to be
    156     // retrieved in
    157     // order to view the results. This call will transfer the data from
    158     // FPGA to
    159     // source_results vector
    160     q.enqueueMigrateMemObjects({buffer_result}, CL_MIGRATE_MEM_OBJECT_HOST);
    161     q.finish();
    162     xrtUEMark("Starting verification") ;
    163 }
```


Enabling Low Overhead Profiling

The Vitis software platform supports low overhead profiling that provides minimal information with little effect on execution time. Using this option during runtime, the timeline trace is still available but with a reduced amount of information. Low overhead profiling captures minimal information on OpenCL events and dumps a CSV file called `lop_trace.csv` at the end of execution. Low overhead profiling can be run in all three flows (hardware, hardware emulation, and software emulation).

To enable low overhead profiling, there is a new flag in the "Debug" section of the [xrt.ini File](#) called `lop_trace`. By default, `lop_trace` is FALSE and must be enabled by setting the `ini` parameter to TRUE.

```
xrt.ini file
[Debug]
lop_trace=true
```



TIP: The `lop_trace` parameter can be enabled alongside other profiling parameters, but doing so eliminates any benefit of low overhead profiling by capturing all profiling data as well.

When `lop_trace=true` is enabled, the runtime will generate `lop_trace.csv` which can be viewed in the Run Summary within Vitis analyzer.

```
vitis_analyzer xrt.run-summary
```

To obtain the lowest possible overhead, information collected in normal OpenCL profiling is omitted. Specifically, the following information is expected to not be available in the low overhead profiling trace:

- Device events, such as compute unit executions or kernel memory transfers
- Information about memory reads or writes, such as destination address or size
- Information about kernel enqueues, such as kernel name or NDRange sizes
- Dependencies between buffer transfers and kernel enqueue

Enabling No Overhead Profiling

When profiling is enabled in `xrt.ini` with `opencl_trace` there is operational overhead added and events in the timeline can show longer delay in between events. However, XRT provides a no-overhead option to dump the OpenCL events on the timeline. It is a simple method of displaying events on the timeline without any overhead.



TIP: You cannot specify any host side profiling or this overrides the no-overhead approach. However, you can use this approach with `device_trace`, which profiles the device but does not add overhead to the host application.

Because the goal is to provide visibility into events with absolutely zero overhead, there are limitations to the number of events that can be logged. Additionally, there is no command queue information in this view, so this view is not intended as a replacement of the more detailed Timeline Trace.

You can use the "no overhead" view to confirm OpenCL command dependencies and to observe actual event overhead for the command execution from the host application.

Add the following switch in `xrt.ini` to enable OpenCL events. The beginning and end of event capturing can be controlled as shown below:

```
[Debug]
xocl_debug=true
#xocl_event_begin= 0 (default)
#xocl_event_end=1000 (default)
```

By default only 1000 events can be visualized.

After the run, if no `xrt.run_summary` is generated, you can use the following steps to generate a `.wdb` file to view in Vitis Analyzer:

```
vp_analyze xocl -i xocl.log // generates debug_log.csv
vp_analyze trace -i debug_log.csv // generates debug_log.wdb
vitis_analyzer debug_log.wdb // loads the wdb file in Vitis analyzer
```

Figure 45: No Overhead Timeline

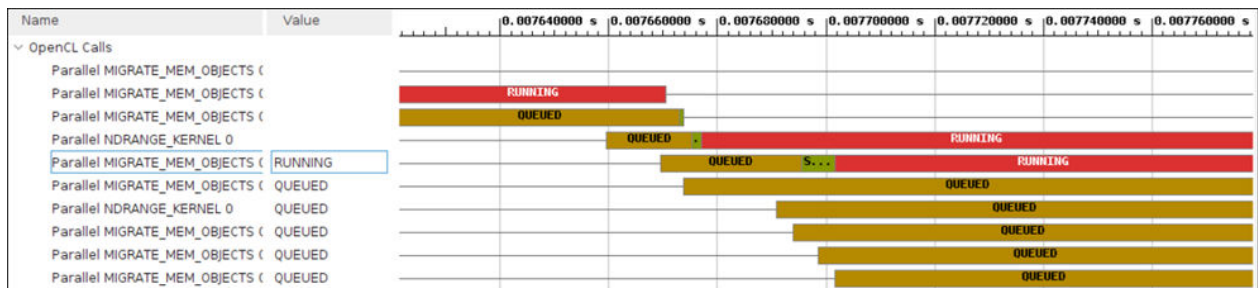


Table 35: Options for OpenCL Profiling

Trace Information Captured	Profile Overhead	Use Case	xrt.ini Switches
Complete	High	For debug purposes like the early stage of application development.	<code>opencl_trace = true</code>
Partial	Low	When profile overhead is High and unexpected delay is experienced.	<code>lop_trace = true</code>
Minimum	No	Only to confirm the cause of delay between events.	<code>xocl_debug = true</code>

Enabling NoC-DDRMC Profiling

The Versal device programmable Network on Chip (NoC) is an AXI-interconnecting network used for sharing data between IP endpoints in the programmable logic (PL), the processing system (PS), and other integrated blocks. This device-wide infrastructure is a high-speed, integrated data path with dedicated switching. The NoC system is a large-scale interconnection of instances of NoC master units (NMUs), NoC slave units (NSUs), and NoC packet switches (NPSs), each controlled and programmed from a NoC programming interface (NPI). There are 16 NMUs/NSUs on the VC1902, each one is capable of 16 Gb/s of throughput in each direction.

Network performance of the NoC interconnecting network can be monitored by the `Vperf` utility. `Vperf` is a Vitis tool that uses the ChipScoPy functionality to profile the NoC and DDRMC in applications built using a `v++` flow. ChipScoPy is an open-source Python project that enables communication with and control of Versal device debug solutions. The ChipScoPy Python package allows users to program designs and begin debugging in a few simple steps. Refer to *Profiling the NoC in AI Engine Tools and Flows User Guide* ([UG1076](#)) for more information on this process.

Guidance

The Vitis core development kit has a comprehensive design guidance tool that provides immediate, actionable guidance to the software developer for issues detected in their designs. These issues might be related to the source code, or due to missed tool optimizations. Also, the rules are generic rules based on an extensive set of reference designs. Therefore, these rules might not be applicable for your specific design. It is up to you to understand the specific guidance rules and take appropriate action based on your specific algorithm and requirements.

Guidance is generated from the Vitis HLS, Vitis profiler, and AMD Vivado™ Design Suite when invoked by the `v++` compiler. The generated design guidance can have several severity levels: errors, warning messages, and informational messages are provided during software emulation, hardware emulation, and system builds. The profile design guidance helps you interpret the profiling results which allows you to focus on improving performance.

Guidance includes message text for reported violations, a brief suggested resolution, and a detailed resolution provided as a web link. You can determine your next course of action based on the suggested resolution. This helps improve productivity by quickly highlighting issues and directing you to additional information in using the Vitis technology.

Design guidance is automatically generated after building or running an application from the command line or Vitis unified IDE.

You can open the Guidance report as discussed in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#). To access the Guidance report, open the Compile Summary, the Link Summary, or the Run Summary, and open the Guidance report.

- Kernel Guidance is generated by the Vitis HLS tool after kernel is built using `v++ compile` command. This can be viewed in the Vitis analyzer by opening the Compile Summary report. Kernel guidance and Compile Summary files are generated for each kernel compiled. Kernel guidance includes recommendations on using Dataflow; and possible reasons why the expected throughout could not be achieved.
- System Guidance is generated after the `.xclbin` or `.xsa` is built using the `v++ link` command. This can be viewed in the Vitis analyzer by opening the Link Summary report. System guidance includes all Kernel Guidance checks, and provides comprehensive review before running your application.
- Run Guidance is generated when your generated `.xclbin` is run, and is a feature of the XRT. This can be viewed by opening the Run Summary in the Vitis analyzer. Run Guidance includes checks like if Kernel Stall is above 50%, recommendations if PLRAM can be used instead of DDR memory, etc.

With the Guidance report open, the Guidance view displays the messages along with resolution columns. The resolutions also have extended weblink help available.

The following image shows an example of the Guidance report displayed in the Vitis analyzer. For example, clicking a link in the Name column opens a description of the rule check. Links in the Details column can open source code, select a design object such as a kernel, or navigate to another report.

Figure 46: Design Guidance Example

Name	Threshold	Actual	Details	Resolution	Impact
DATAFLOW_ACCELERATION	> 1.500	1.000	Compute unit <code>runOnFpga_1</code> had dataflow acceleration of 1.000 .	Improve dataflow acceleration to maximize performance. Click here .	Medium
KERNEL_COUNT (1)	> 1				
KERNEL_COUNT	> 1	1	Kernel <code>runOnFpga</code> was executed 17 time(s) with 1 compute unit(s).	Ensure kernel utilizes multiple compute units. Click here .	Medium
KERNEL_PORT_DATA_WIDTH (1)					
KERNEL_PORT_DATA_WIDTH	= 512	32	Port <code>m_axi_maxiport1</code> has a data width of 32.	Utilize the entire memory data width. Click here .	Medium
Resolution The specific port, that is not scalar based, is not utilizing the optimum data width. Kernel arguments are implemented through memory mapped AXI ports. For detailed explanation and recommendation: KERNEL_PORT_DATA_WIDTH					
Host Data Transfers (1)					
HOST_READ_TRANSFER_UTIL (1)	> 70.0				
HOST_READ_TRANSFER_UTIL	> 70.0	0.569	Host read bandwidth utilization was 0.569%.	Improve efficiency of host read transfers. Click here .	Medium

There is one HTML guidance report for each run of the `v++` command, including compile and link. The report files are located in the `--report_dir` under the specific output name. For example:

- `v++_compile-<output>_guidance.html` for v++ compilation
- `v++_link-<output>_guidance.html` for v++ linking

You can click the web link in the Resolution column to get additional details about the resolution. The [Vitis HLS Messaging](#) guidance web page lists all of the current messages for your review.

Kernel and Compute Unit objects, in addition to profile reported data values, can also be cross-probed to other views like the Profile Report.

Opening the Guidance Report

When kernels are compiled and when the FPGA binary is linked, guidance reports are generated automatically by the v++ command. You can view these reports in the Vitis analyzer by opening the `<output_filename>.compile_summary` or the `<output_filename>.link_summary` for the application project. The `<output_filename>` is the output of the v++ command.

As an example, launch the Vitis analyzer and open the report using this command:

```
vitis_analyzer <output_filename>.link_summary
```

When the Vitis analyzer opens, it displays the link summary report, as well as the compile summaries, and a collection of reports generated during the compile and link processes. Both the compile and link steps generate Guidance reports to view by clicking the **Build** heading on the left-hand side. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for more information.

Interpreting Guidance Data

The Guidance view places each entry in a separate row. Each row might contain the name of the guidance rule, threshold value, actual value, and a brief but specific description of the rule. The last field provides a link to reference material intended to assist in understanding and resolving any of the rule violations.

In the GUI Guidance view, guidance rules are grouped by categories and unique IDs in the Name column and annotated with symbols representing the severity. These are listed individually in the HTML report. In addition, as the HTML report does not show tooltips, a full Name column is included in the HTML report as well.

The following list describes all fields and their purpose as included in the HTML guidance reports.

- **Id:** Each guidance rule is assigned a unique ID. Use this id to uniquely identify a specific message from the guidance report.
- **Full Name:** The Full Name provides a less cryptic name compared to the mnemonic name in the Name column.

- **Categories:** Most messages are grouped within different categories. This allows the GUI to display groups of messages within logical categories under common tree nodes in the Guidance view.
- **Threshold:** The Threshold column displays an expected threshold value, which determines whether or not a rule is met. The threshold values are determined from many applications that follow good design and coding practices.
- **Actual:** The Actual column displays the values actually encountered on the specific design. This value is compared against the expected value to see if the rule is met.
- **Details:** The Details column describes the specifics of the current rule.
- **Resolution:** The Resolution column provides a pointer to common ways the model source code or tool transformations can be modified to meet the current rule. Clicking the link brings up a popup window or the documentation with tips and code snippets that you can apply to the specific issue.

System Estimate Report

The process step with the longest execution time includes building the hardware system and the FPGA binary to run on AMD devices. Build time is also affected by the target device and the number of compute units instantiated onto the FPGA fabric. Therefore, it is useful to estimate the performance of an application without needing to build it for the system hardware.

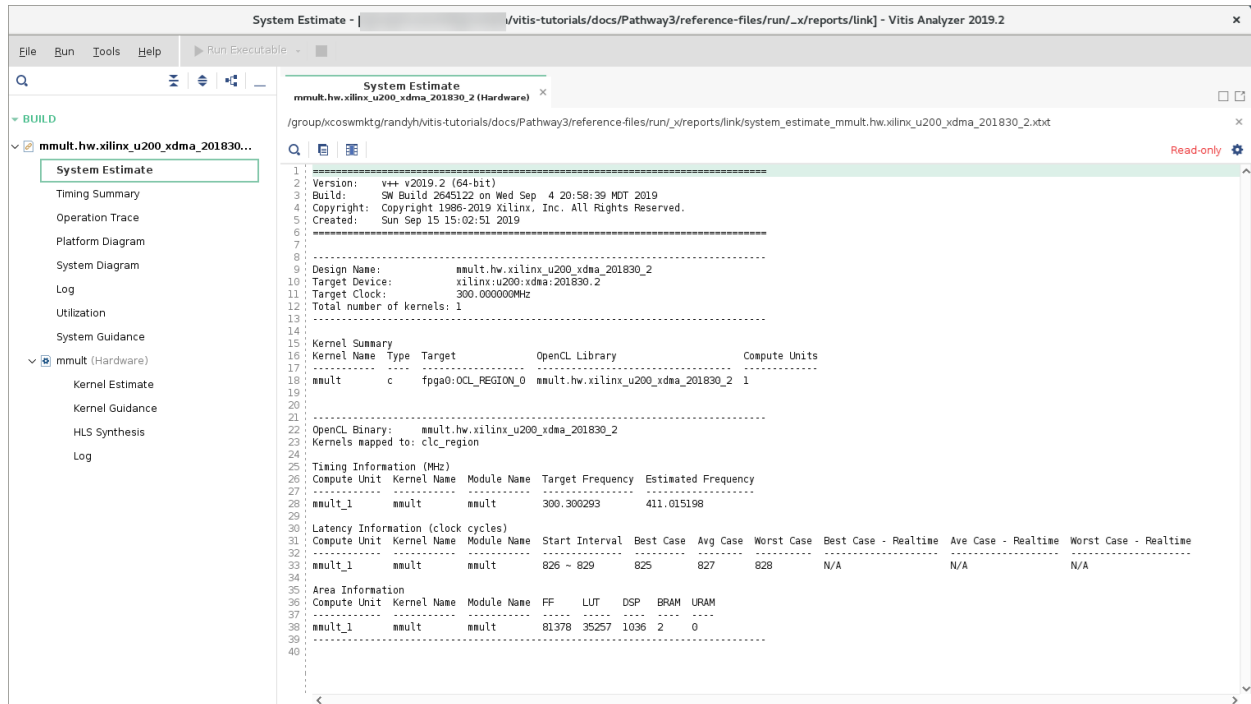
The System Estimate report provides estimates of FPGA resource usage and the estimated frequency at which the hardware accelerated kernels can operate. The report is automatically generated for hardware emulation and system hardware builds. The report contains high-level details of the user kernels, including resource usage and estimated frequency. This report can be used to guide design optimization.

You can also force the generation of the System Estimate report with the following option:

```
v++ .. --report_level estimate
```

An example report is shown in the figure:

Figure 47: System Estimate



Opening the System Estimate Report

The System Estimate report can be opened in the Vitis analyzer tool, intended for viewing reports from the Vitis compiler when the application is built, and the XRT library when the application is run. You can launch the Vitis analyzer and open the report using the following command:

```
vitis_analyzer <output_filename>.link_summary
```

The <output_filename> is the output of the v++ command. This opens the Link Summary for the application project in the Vitis analyzer tool. Then, select the System Estimate report (see [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#)).

Interpreting the System Estimate Report

The System Estimate report generated by the v++ command provides information on every binary container in the application, in addition to every compute unit in the design. The report is structured as follows:

- Target device information
- Summary of every kernel in the application
- Detailed information on every binary container in the solution

The following example report file represents the information generated for the estimate report:

```
-----
Design Name:          mmult.hw_emu.xilinx_u200_xdma_201830_2
Target Device:        xilinx:u200:xdma:201830.2
Target Clock:         300.000000MHz
Total number of kernels: 1
-----

Kernel Summary
Kernel Name  Type  Target                OpenCL Library                Compute Units
-----
mmult        c    fpga0:OCL_REGION_0    mmult.hw_emu.xilinx_u200_xdma_201830_2    1
-----

OpenCL Binary:        mmult.hw_emu.xilinx_u200_xdma_201830_2
Kernels mapped to:    clc_region

Timing Information (MHz)
Compute Unit  Kernel Name  Module Name  Target Frequency  Estimated Frequency
-----
mmult_1      mmult        mmult        300.300293       411.015198

Latency Information (clock cycles)
Compute Unit  Kernel Name  Module Name  Start Interval  Best Case  Avg Case  Worst Case
-----
mmult_1      mmult        mmult        826 ~ 829       825        827       828

Area Information
Compute Unit  Kernel Name  Module Name  FF      LUT      DSP      BRAM      URAM
-----
mmult_1      mmult        mmult        81378   35257    1036     2         0
-----
```

Design and Target Device Summary

All design estimate reports begin with an application summary and information about the target device. The device information is provided in the following section of the report:

```
-----
Design Name:          mmult.hw_emu.xilinx_u200_xdma_201830_2
Target Device:        xilinx:u200:xdma:201830.2
Target Clock:         300.000000MHz
Total number of kernels: 1
-----
```

For the design summary, the information provided includes the following:

- **Target Device:** Name of the AMD device on the target platform that runs the FPGA binary built by the Vitis compiler.
- **Target Clock:** Specifies the target operating frequency for the compute units (CUs) mapped to the FPGA fabric.

Kernel Summary

This section lists all of the kernels defined for the application project. The following example shows the kernel summary:

Kernel Summary				
Kernel Name	Type	Target	OpenCL Library	Compute Units
mmult	c	fpga0:OCL_REGION_0	mmult.hw_emu.xilinx_u200_xdma_201830_2	1

In addition to the kernel name, the summary also provides the execution target and type of the input source. Because there is a difference in compilation and optimization methodology for OpenCL™, C, and C/C++ source files, the type of kernel source file is specified.

The Kernel Summary section is the last summary information in the report. From here, detailed information on each compute unit binary container is presented.

Timing Information

For each binary container, the detail section begins with the execution target of all compute units (CUs). It also provides timing information for every CU. As a general rule, if the estimated frequency for the FPGA binary is higher than the target frequency, the CU will be able to run in the device. If the estimated frequency is below the target frequency, the kernel code for the CU needs to be further optimized to run correctly on the FPGA fabric. This information is shown in the following example:

OpenCL Binary: mmult.hw_emu.xilinx_u200_xdma_201830_2				
Kernels mapped to: clc_region				
Timing Information (MHz)				
Compute Unit	Kernel Name	Module Name	Target Frequency	Estimated Frequency
mmult_1	mmult	mmult	300.300293	411.015198

It is important to understand the difference between the target and estimated frequencies. CUs are not placed in isolation into the FPGA fabric. CUs are placed as part of a valid FPGA design that can include other components defined by the device developer to support a class of applications.

Because the CU custom logic is generated one kernel at a time, an estimated frequency that is higher than the target frequency indicates that the CU can run at the higher estimated frequency. Therefore, CU should meet timing at the target frequency during implementation of the FPGA binary.

Latency Information

The latency information presents the execution profile of each CU in the binary container. When analyzing this data, it is important to recognize that all values are measured from the CU boundary through the custom logic. In-system latencies associated with data transfers to global memory are not reported as part of these values. Also, the latency numbers reported are only for CUs targeted at the FPGA fabric. The following is an example of the latency report:

Latency Information (clock cycles)							
Compute Unit	Kernel Name	Module Name	Start	Interval	Best Case	Avg Case	Worst Case
mmult_1	mmult	mmult	826	~ 829	825	827	828

The latency report is divided into the following fields:

- Start interval
- Best case latency
- Average case latency
- Worst case latency

The start interval defines the amount of time that has to pass between invocations of a CU for a given kernel.

The best, average, and worst case latency numbers refer to how much time it takes the CU to generate the results of one ND Range data tile for the kernel. For cases where the kernel does not have data dependent computation loops, the latency values will be the same. Data dependent execution of loops introduces data specific latency variation that is captured by the latency report.

The interval or latency numbers will be reported as "undef" for kernels with one or more conditions listed below:

- OpenCL kernels that do not have explicit `reqd_work_group_size(x, y, z)`
- Kernels that have loops with variable bounds

Note: The latency information reflects estimates based on the analysis of the loop transformations and exploited parallelism of the model. These advanced transformations such as pipelining and data flow can heavily change the actual throughput numbers. Therefore, latency can only be used as relative guides between different runs.

Area Information

Although the FPGA can be thought of as a blank computational canvas, there are a limited number of fundamental building blocks available in each FPGA. These fundamental blocks (FF, LUT, DSP, block RAM) are used by the Vitis compiler to generate the custom logic for each CU in the design. The quantity of fundamental resources needed to implement the custom logic for a single CU determines how many CUs can be simultaneously loaded into the FPGA fabric. The following example shows the area information reported for a single CU:

Area Information							
Compute Unit	Kernel Name	Module Name	FF	LUT	DSP	BRAM	URAM
mmult_1	mmult	mmult	81378	35257	1036	2	0

HLS Synthesis Report

The HLS compiler generates a number of reports for simulation, synthesis, co-simulation. These reports provide details about the high-level synthesis (HLS) process of a PL kernel. The main report is the Synthesis Summary report that provides estimated FPGA resource usage, operating frequency, latency, and interface signals of the custom-generated hardware logic. These details provide many insights to guide kernel optimization.

When running from the Vitis unified IDE, this report can be found in the HLS component directory named `<hls_component>.hlscompile_summary`. The Summary report can be opened from the Flow Navigator in the HLS component under the C Synthesis/Reports heading, or by opening the Compile Summary, or the Link Summary as described in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Figure 48: Synthesis Summary Report

Summary Synthesis Report - dct

Estimated Quality of Results

Timing Estimate

TARGET	ESTIMATED	UNCERTAINTY
8.00 ns	7.040 ns	0.96 ns

Performance & Resource Estimates

MODULES & LOOPS	LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	BRAM	DSP	FF	LUT	URAM
dct (4)	331	2.648E3	-	181	-	dataflow	62	320	14813	76737	0
entry_proc	0	0.0	-	0	-	no	0	0	2	6	0
read_data (1)	136	1.088E3	-	136	-	no	0	0	177	2536	0
RD_Loop_Row_RD_Loop_Col	134	1.072E3	72	1	64	yes	-	-	-	-	-
dct_2d	13	104.000	-	4	-	yes	0	320	10182	70348	0
write_data (1)	180	1.440E3	-	180	-	no	0	0	730	674	0
WR_Loop_Row (1)	112	896.000	14	-	8	no	-	-	-	-	-
write_data_Pipeline_WR_Loop_Col (1)	10	80.000	-	10	-	no	0	0	135	189	0
WR_Loop_Col	8	64.000	1	1	8	yes	-	-	-	-	-

Generating and Opening the HLS Report

★ IMPORTANT! You must specify the `--save-temps` option during the build process to preserve the intermediate files produced by Vitis HLS, including the reports. The HLS report and HLS guidance are only generated for hardware emulation and system builds for C and OpenCL kernels. They are not generated for software emulation or RTL kernels.

The HLS report can be viewed through the Vitis analyzer by opening the `<output_filename>.compile_summary` or the `<output_filename>.link_summary` for the application project. The `<output_filename>` is the output of the `v++` command.

You can launch the Vitis analyzer and open the report using the following command:

```
vitis_analyzer <output_filename>.compile_summary
```

When the Vitis analyzer opens, it displays the Compile Summary and a collection of reports generated during the compile process. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for more information.

Interpreting the HLS Report

The HLS Synthesis report is a spreadsheet listing the module hierarchy in the left column. This section is describing one section of the HLS report: Performance and Resource Estimates. Each module and loop generated by the HLS run is represented in this hierarchy. The HLS Synthesis report contains the following columns:

- Issue Type

- Slack
- Latency in clock cycles
- Latency in absolute time (ns)
- Iteration Latency
- Interval
- Trip Count
- Pipelined
- Utilization Estimates of BRAM, DSP, FF, and LUT

If this information is part of a hierarchical block, it will sum up the information of the blocks contained in the hierarchy. Therefore, the hierarchy can also be navigated from within the report when it is clear which instance contributes to the overall design.



CAUTION! The absolute counts of cycles and latency numbers are based on estimates identified during HLS synthesis, especially with advanced transformations, such as pipelining and dataflow. Therefore, these numbers might not accurately reflect the final results. If you encounter question marks in the report, this might be due to variable bound loops, and you are encouraged to set trip counts for such loops to have some relative estimates presented in this report.

Profile Summary Report

As described in [Enabling Profiling in Your Application](#), the Xilinx Runtime (XRT) collects profiling data on host applications and kernels when specific options are enabled in the `xrt.ini` file, such as `opencl_trace`, `xrt_native_api`, and `device_trace`. XRT captures profiling data for the host application as it makes calls to the runtime either through OpenCL or XRT API calls. You can also add user calls to your host application to capture additional profiling information, as explained in [Custom Profiling of the Host Application](#). To capture details of the kernel operations, you must implement kernels in the `.xclbin` using the [--profile Options](#) as explained in the next section.

After the application finishes running, the Profile Summary report is saved as `.csv` files in the directory where the compiled host code is executed. The Profile Summary provides annotated details regarding the overall application performance. All data generated during the execution of the application is grouped into categories. The Profile Summary lets you examine the kernel execution and data transfer statistics.



TIP: The Profile Summary report can be generated for all build configurations. However, with the software emulation build, the report will not include any data transfer details under kernel execution efficiency and data transfer efficiency. This information is only generated in hardware emulation or system builds.

An example of the Profile Summary report is shown below.

Figure 49: Profile Summary

The screenshot shows the Vitis Analyzer's Profile Summary report. The sidebar on the left contains a search bar and a list of navigation items: Settings, Summary, Kernels & Compute Units (selected), Kernel Execution, Top Kernel Execution, Compute Unit Utilization, Host Data Transfers, Top Memory Writes, Top Memory Reads, and API Calls. The main content area is titled 'Kernels & Compute Units' and contains three tables.

Kernel Execution Table:

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
dct	1	3.079	3.079	3.079	3.079

Top Kernel Execution Table:

Kernel	Kernel Instance Address	Context ID	Command Queue ID	Device	Start Time (ms)	Duration (ms)
dct	0x9d3990	0	0	xilinx_u200_xdma_201830_2-0	326.922	3.079

Compute Unit Utilization Table:

Compute Unit	Kernel	Device	Calls	Dataflow Execution	Max Parallel Executions	Dataflow Acceleration	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	Clock Freq (MHz)	CU Utilization (%)
dct_1	dct	xilinx_u200_xdma_201830_2-0	1	No	0	1.000000x	1.054	1.054	1.054	1.054	300.000	34.218

Generating and Opening the Profile Summary Report

Capturing the data required for the Profile Summary requires a few steps prior to actually running the application.

1. The FPGA binary (`xclbin`) file is configured for capturing profiling data by default. However, using the `v++ --profile` option during the linking process enables a greater level of detail in the profiling data captured. For more information, see the [--profile Options](#).
2. The runtime requires the presence of an `xrt.ini` file, as described in [xrt.ini File](#), that includes options for capturing trace data:

```
[Debug]
opencl_trace = true
xrt_native_api = true
device_trace = fine
```

3. To enable the profiling of Kernel Internals data, you must also add the `debug_mode` tag in the `[Emulation]` section of the `xrt.ini`:

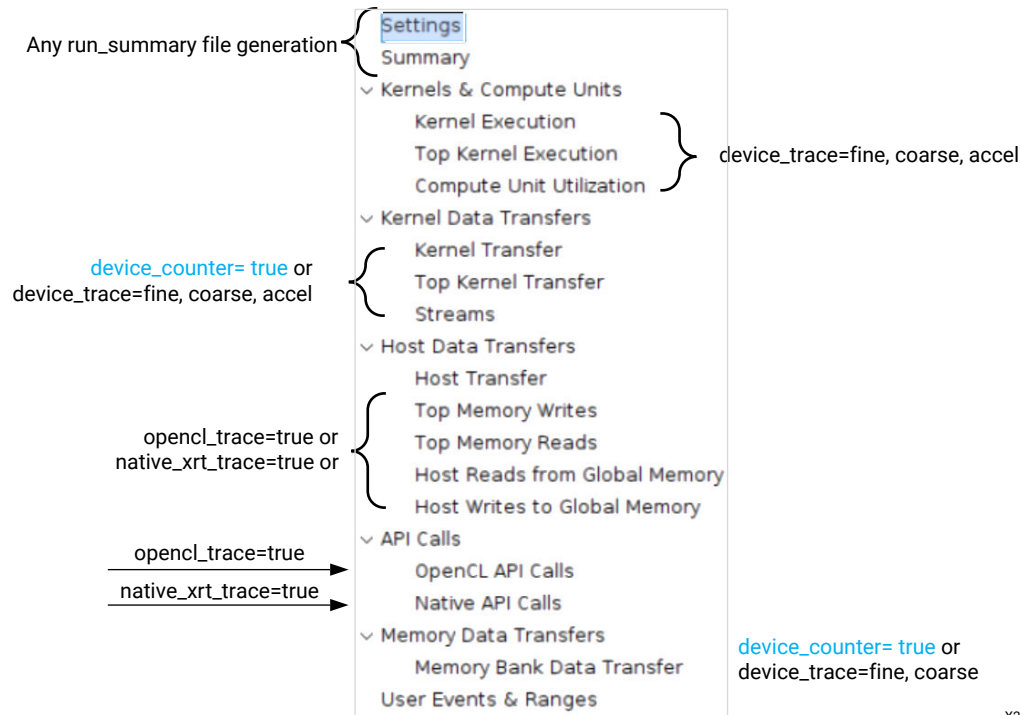
```
[Emulation]
debug_mode = batch
```

With profiling enabled in the device binary and in the `xrt.ini` file, the runtime creates different report files when running the application depending on which options are enabled. The Profile Summary report can be viewed in the [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) by opening the Run Summary using the following command:

```
vitis_analyzer xrt.run_summary
```

As previously stated, the data contained in the Profile Summary report depends on the various options enabled in the `xrt.ini` file. The following figure illustrates some of the available data tables and the options to enable them.

Figure 50: Profile Summary Tables



Related Information

[Running the System on Hardware](#)

[Section IX: Working with the Analysis View \(Vitis Analyzer\)](#)

Interpreting the Profile Summary

The profile summary includes a number of useful statistics for your host application and kernels. The report provides a general idea of the functional bottlenecks in your application.



TIP: When viewing a table in the Analysis view, you can hover your mouse over any field to get a definition of the field contents.

Settings

This displays the report and XRT configuration settings.

Summary

This displays summary statistics including device execution time and device power.

Kernels & Compute Units

Displays the profile summary data for all kernel functions scheduled and executed.

Kernel Data Transfers

Displays the data transfer for kernels to the global memory, and the top data transfer for kernels to the global memory, and the data transfer streams.

Host Data Transfers

Displays profile data for all write transfers between the host and device memory through PCI Express® link, and profile data for all read transfers between the host and device memory through PCI Express® link, and the data transfer for host to the global memory.

API Calls

Displays the profile data for all OpenCL host API function calls executed in the host application. The top displays a bar graph of the API call time as a percent of total time.

Device Power

This displays the profile data for device power.

Kernel Internals

Displays the running time for compute units in microseconds (μ s) and reports stall time as a percent of the running time. This section of the Profile Summary displays the data transfer for specific ports on the compute unit, and displays the functional port data transfers on the compute unit, and displays the running time and stalls on the compute unit.



TIP: The Kernel Internals tab reports time in μ s, while the rest of the Profile Summary reports time in milliseconds (ms).

Shell Data Transfers

This following table displays the DMA data transfers.



TIP: For DMA bypass and Global Memory to Global Memory data transfers, see the DMA Data Transfer table in Kernel Internals.

NoC Counters



TIP: This data is not displayed unless it has been specifically generated during implementation.

NoC Counters display the NoC Counters Read and NoC Counters Write. These sections are only displayed if there is a non-zero NoC counter data.

Each section has a table containing summary data with line graphs for transfer rate and latency. The graphs can have multiple NoC counters, so you can toggle the counters ON/OFF through check boxes in the Chart column of the table.

Depending on the design, it can be possible to correlate NoC counters to CU ports. In this case, the CU port appears in the table, and selecting it cross-probes to the system diagram, profile summary, and any other views that include CU ports as selectable objects.

AI Engine Counters

AI Engine counters display if there is a non-zero AI Engine counter data. If there is an incompatible configuration of the AI Engine counters, this section displays a message stating that the configuration does not support performance profiling. This section of the Profile Summary can contain three sub-sections:

- AI Engine & Memory
- Interface Channels
- Memory Channels

Note: For more information, see *AI Engine Tools and Flows User Guide* ([UG1076](#)).

Timeline Trace

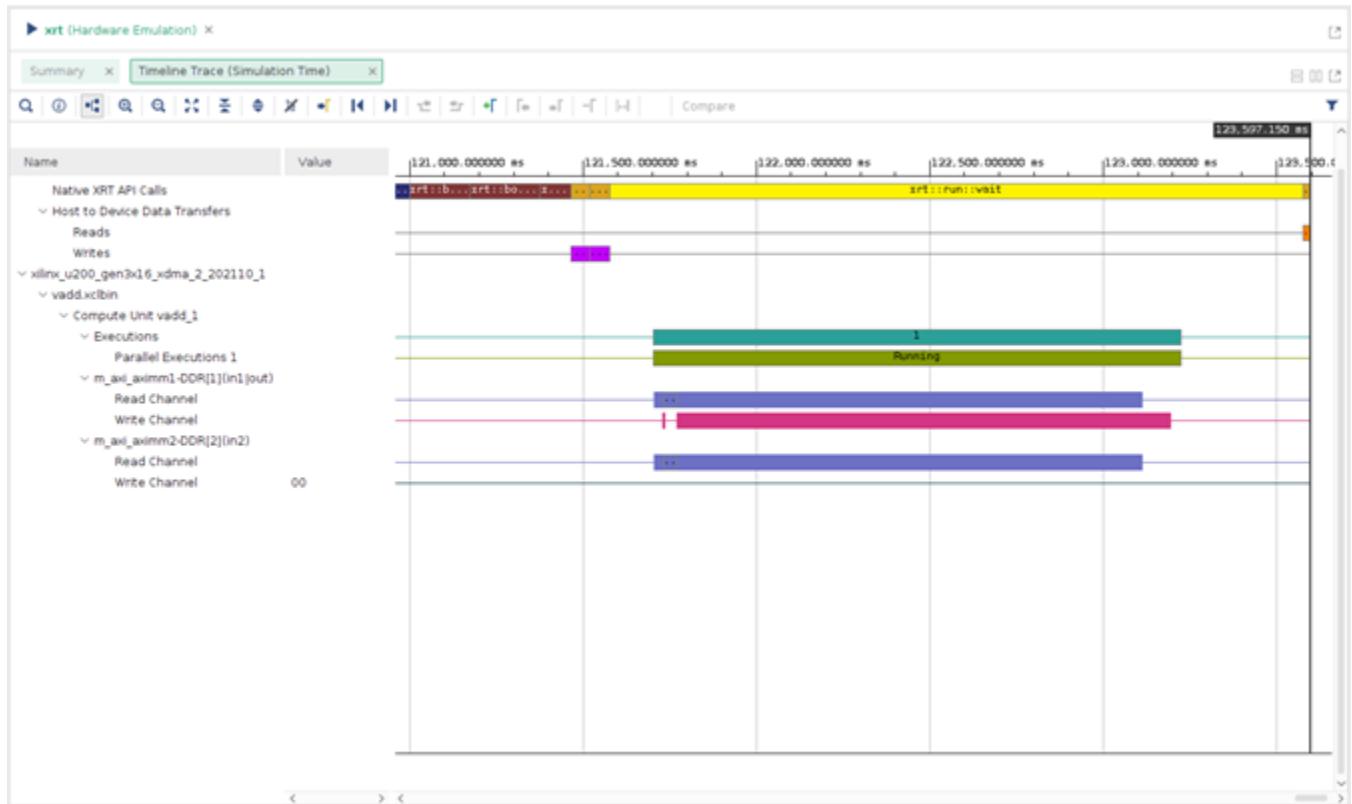
The Timeline Trace collects and displays host and kernel events on a common timeline to help you understand and visualize the overall health and performance of your systems. The graphical representation lets you see issues regarding kernel synchronization and efficient concurrent execution. The displayed events include:

- XRT API calls from the host code
- Device trace data including compute units, AXI transaction start/stop
- Host events and kernel start/stops

While the timeline and device trace data are useful for debugging and profiling the application, collecting the data can affect application performance by adding time to the execution. However, the trace data is collected with dedicated resources in the kernel, and so does not affect kernel functionality. By default, the collected trace data is offloaded at the end of the run, though the tool can be configured to offload data continuously which can help when debugging application crashes.

The following is a snapshot of the Timeline Trace window which displays host and device events on a common timeline. Host activity is displayed at the top of the image and kernel activity is shown on the bottom of the image. Host activities include creating the program, running the kernel and data transfers between global memory and the host. The kernel activities include read/write accesses and transfers between global memory and the kernel(s). This information helps you understand details of application execution and identify potential areas for improvements.

Figure 51: Timeline Trace



Timeline data can be enabled and collected through the command line flow. However, viewing must be done in the Vitis analyzer as described in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Generating and Opening the Timeline Trace

To generate the Timeline Trace report, you must complete the following steps to enable timeline and device trace data collection in the command line flow:

1. Instrument the FPGA binary during linking, by adding Acceleration Monitors and AXI Performance Monitors to kernels using the `v++ --profile` option as described in [--profile Options](#). As an example, add `--profile.data` to the `v++` linking command line:

```
v++ -g -l --profile.data all:all:all ...
```

2. After the kernels are instrumented during the build process, data gathering must also be enabled during the runtime execution of the application by editing the `xrt.ini` file. Refer to [xrt.ini File](#) for more information.

The following `xrt.ini` file enables maximum information gathering when the application is run:

```
[Debug]
openccl_trace=true
device_trace=fine
stall_trace=all
```



TIP: If you are collecting a large amount of trace data, you might need to use the `--profile.trace_memory` with the `v++` command, and the `trace_buffer_size` keyword in the `xrt.ini`.

After running the application, the Timeline Trace data is captured in CSV files called `openccl_trace.csv` and `device_trace_0.csv`.

3. The CSV report can be viewed in the Vitis analyzer tool by opening the Run Summary produced during the application execution. You can launch the Vitis analyzer and open the Run Summary using the following command:

```
vitis_analyzer xrt.run_summary
```



TIP: By default, the Timeline Trace is displayed in a hierarchical view, which presents the information according to design hierarchy but consumes significant real estate in the display. As an alternative, you can "flatten" the timeline display to eliminate unnecessary spacing between lines. Perform this by selecting the **Flatten Signal** command on the toolbar. This feature is useful when there is less display area to work with, or when comparing multiple trace files.

Interpreting the Timeline Trace

The Timeline Trace window displays host and device events on a common timeline. This information helps you understand details of application execution and identify potential areas for improvements. The Timeline Trace report has two main sections: Host and Device. The Host section shows the trace of all the activity originating from the host side. The Device section shows the activity of the CUs on the FPGA.

The report has the following structure:

- Host

- **OpenCL API Calls:** All OpenCL API calls are traced here. The activity time is measured from the host perspective.
 - **General:** All general OpenCL API calls such as `clCreateProgramWithBinary`, `clCreateContext`, and `clCreateCommandQueue`, are traced here.
 - **Queue:** OpenCL API calls that are associated with a specific command queue are traced here. This includes commands such as `clEnqueueMigrateMemObjects`, and `clEnqueueNDRangeKernel`. If the user application creates multiple command queues, then this section shows all the queues and activities.
 - **Data Transfer:** In this section the DMA transfers from the host to the device memory are traced. There are multiple DMA threads implemented in the OpenCL runtime and there is typically an equal number of DMA channels. The DMA transfer is initiated by the user application by calling OpenCL APIs such as `clEnqueueMigrateMemObjects`. These DMA requests are forwarded to the runtime which delegates to one of the threads. The data transfer from the host to the device appear under **Write** as they are written by the host, and the transfers from device to host appear under **Read**.
 - **Kernel Enqueues:** The kernels enqueued by the host program are shown here. The kernels here should not be confused with the kernels/CUs on the device. Here kernel refers to the `NDRangeKernels` and tasks created by the OpenCL commands `clEnqueueNDRangeKernels` and `clEnqueueTask`. These are plotted against the time measured from the host's perspective. Multiple kernels can be scheduled to be executed at the same time, and they are traced from the point they are scheduled to run until the end of the kernel execution. This is the reason for multiple entries. The number of rows depend on the number of overlapping kernel executions.

Note: Overlapping of the kernels should not be mistaken for actual parallel execution on the device as the process might not be ready to execute right away.
- **Device "name"**
 - **Binary Container "name":** Binary container name.
 - **Compute Unit "name":** Name of the compute unit on the FPGA.
 - **User Functions:** In the case of the Vitis HLS tool kernels, functions that are implemented as data flow processes are traced here. The trace for these functions show the number of active instances of these functions that are currently executing in parallel. These names are generated in hardware emulation when waveform is enabled.

Note: Function level activity is only possible in hardware emulation.

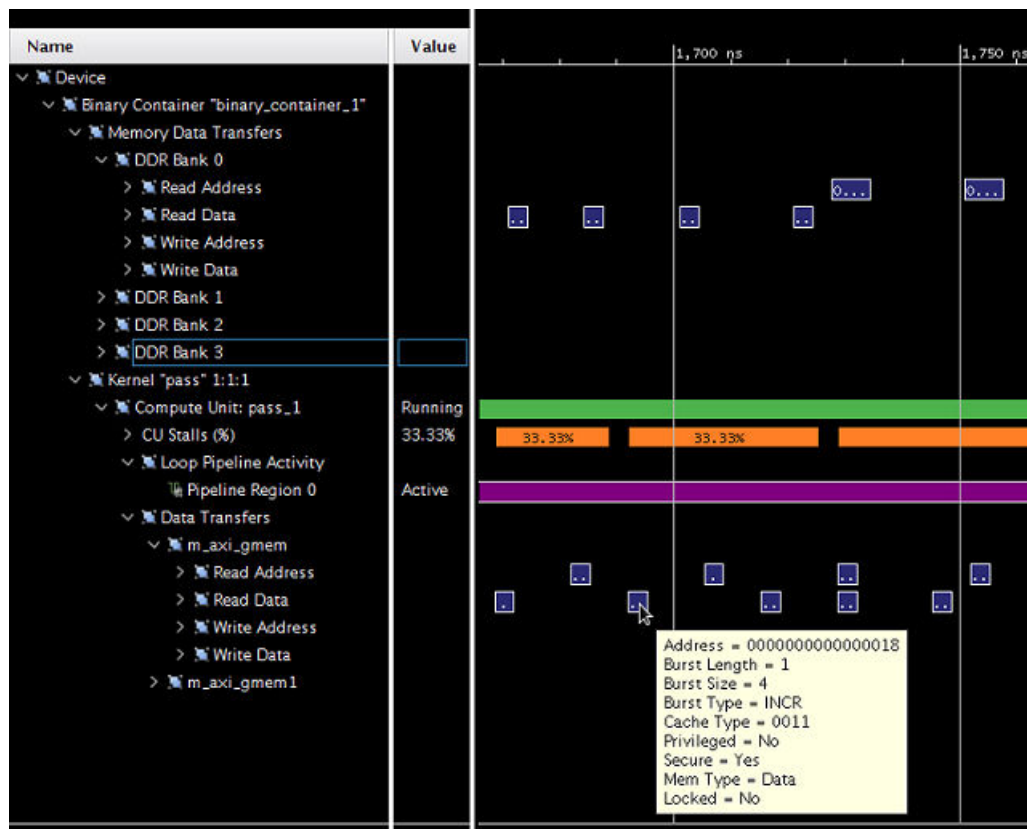
 - **Function: "name a"**
 - **Function: "name b"**
 - **Read:** A CU reads from the DDR over AXI-MM ports. The trace of a data read by a CU is shown here. The activity is shown as transaction and the tool-tip for each transaction shows more details of the AXI transaction. These names are generated when `--profile.data` is for the CU.

- **Write:** A CU writes to the DDR over AXI-MM ports. The trace of data written by a CU is shown here. The activity is shown as transactions and the tool-tip for each transaction shows more details of the AXI transaction. This is generated when `--profile.data` is specified for the CU.

Detailed Kernel Trace

The detailed kernel trace provides easy access to the AXI transactions and their properties. The AXI transactions are presented for the global memory, in addition to the Kernel side (Kernel "pass" 1:1:1) of the AXI interconnect. The following figure illustrates a typical kernel trace of a newly accelerated algorithm.

Figure 52: Accelerated Algorithm Kernel Trace



Most interesting with respect to performance are the fields:

- **Burst Length:** Describes how many packages are sent within one transaction.
- **Burst Size:** Describes the number of bytes being transferred as part of one package.

Given a burst length of 1 and 4 bytes per package, it will require many individual AXI transactions to transfer any reasonable amount of data.

Note: The Vitis core development kit never creates burst sizes less than 4 bytes, even if smaller data is transmitted. In this case, if consecutive items are accessed without AXI bursts enabled, it is possible to observe multiple AXI reads to the same address.

Small burst lengths and burst sizes considerably less than 512 bits are therefore good opportunities to optimize interface performance.

Using Burst Data Transfers

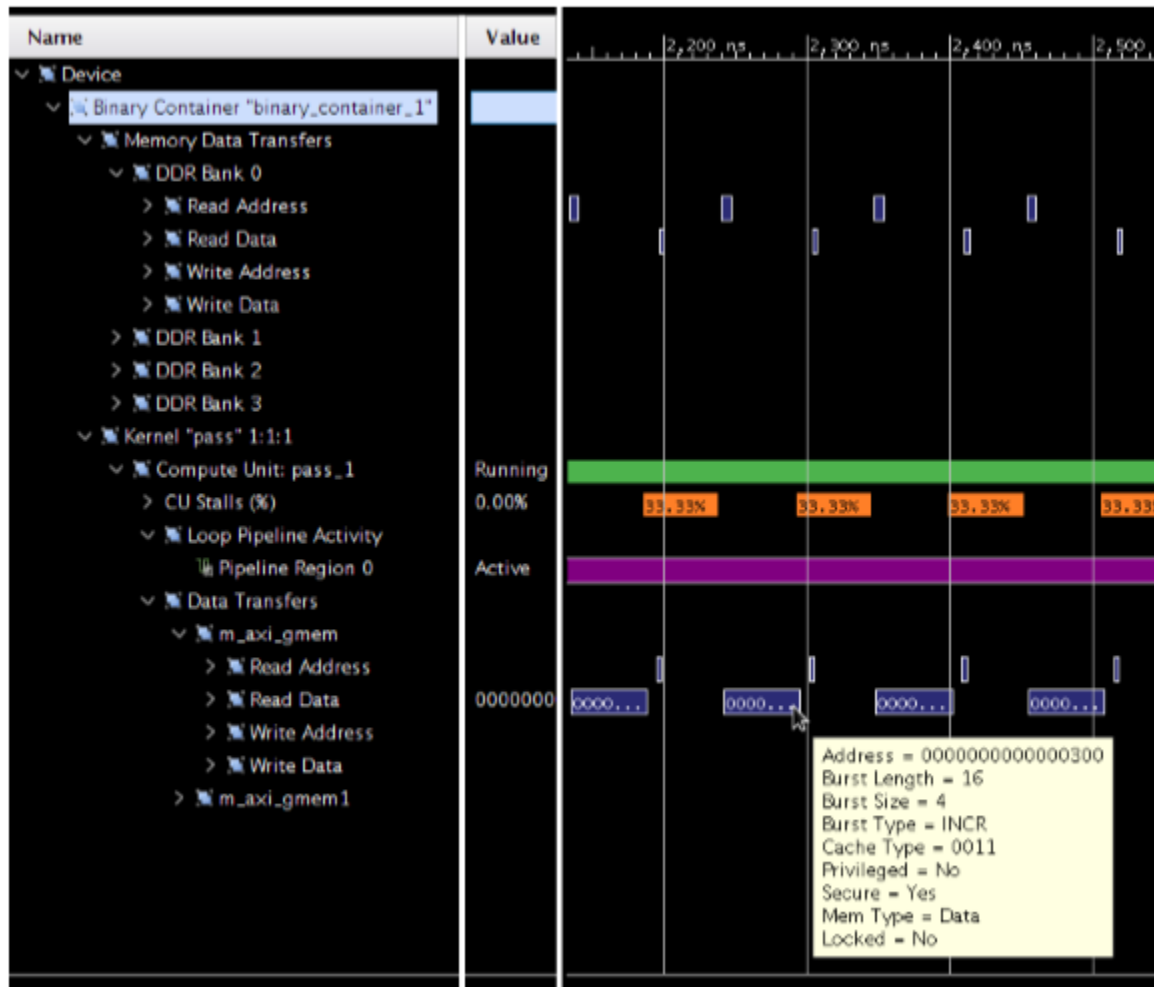
Transferring data in bursts hides the memory access latency and improves bandwidth usage and efficiency of the memory controller. Also, check the HLS report for bursting information.



RECOMMENDED: *Infer burst transfers from successive requests of data from consecutive address locations. Refer to the topic AXI Burst Transfers in the Vitis HLS User Guide ([UG1399](#)) for more details.*

If burst data transfers occur, the detailed kernel trace will reflect the higher burst rate as a larger burst length number:

Figure 53: Burst Data Transfer with Detailed Kernel Trace



In the previous figure, it is also possible to observe that the memory data transfers following the AXI interconnect are actually implemented rather differently (shorter transaction time). Hover over these transactions, you would see that the AXI interconnect has packed the 16 x 4 byte transaction into a single package transaction of 1 x 64 bytes. This effectively uses the AXI4 bandwidth which is even more favorable. The next section focuses on this optimization technique in more detail.

Burst inference is heavily dependent on coding style and access pattern. However, you can ease burst detection and improve performance by isolating data transfer and computation, as shown in the following code snippet:

```
void kernel(T in[1024], T out[1024]) {
    T tmpIn[1024];
    T tmpOu[1024];
    read(in, tmpIn);
    process(tmpIn, tmpOut);
    write(tmpOut, out);
}
```

In short, the function `read` is responsible for reading from the AXI input to an internal variable (`tmpIn`). The computation is implemented by the function `process` working on the internal variables `tmpIn` and `tmpOut`. The function `write` takes the produced output and writes to the AXI output. For more information on burst, see the [AXI Burst Transfers](#) in the *Vitis HLS User Guide* (UG1399).

The isolation of the read and write function from the computation results in:

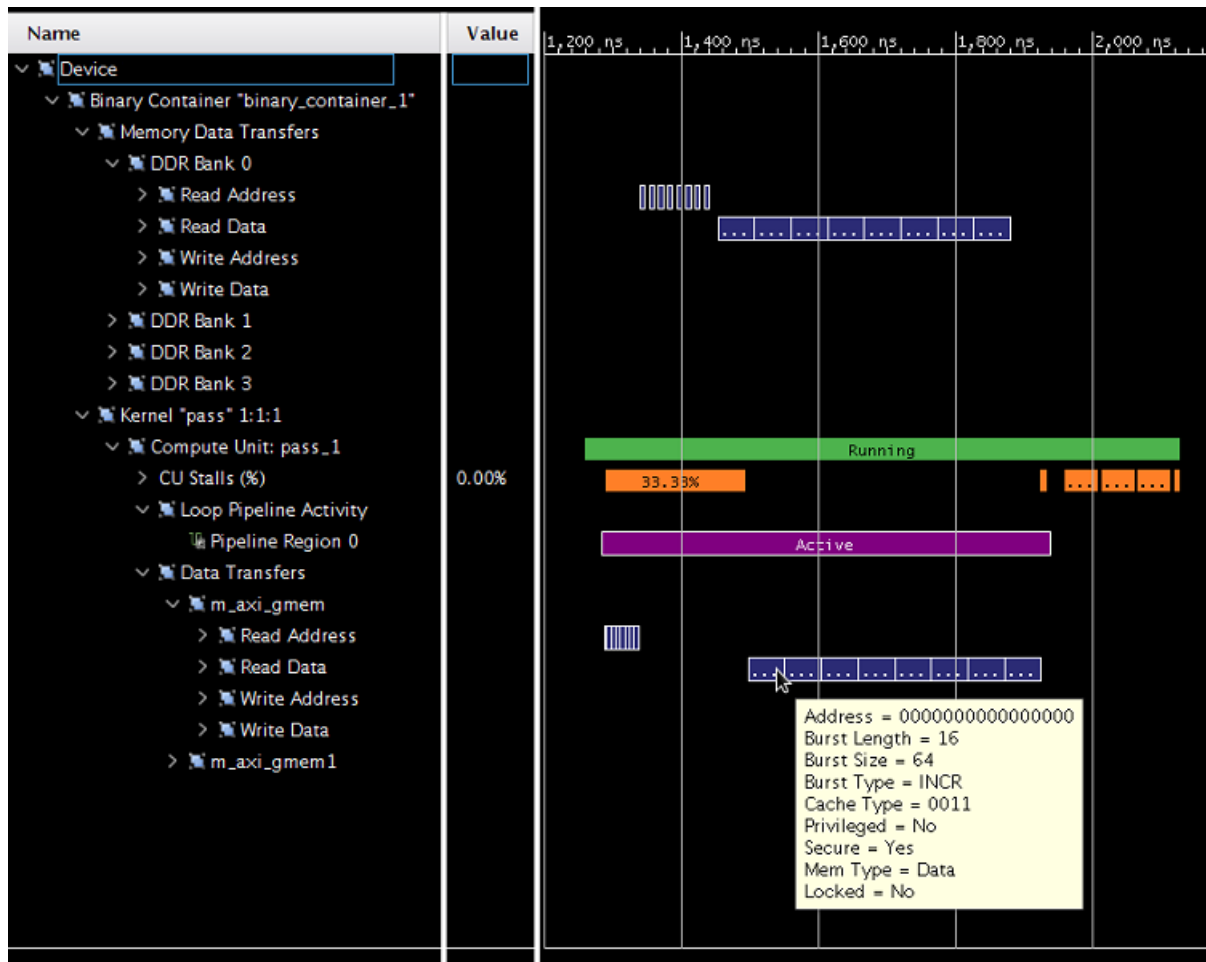
- Simple control structures (loops) in the read/write function which makes burst detection simpler.
- The isolation of the computational function away from the AXI interfaces, simplifies potential kernel optimization.
- The internal variables are mapped to on-chip memory, which allow faster access compared to AXI transactions. Acceleration platforms supported in the Vitis core development kit can have as much as 10 MB on-chip memories that can be used as pipes, local memories, and private memories. Using these resources effectively can greatly improve the efficiency and performance of your applications.

Using Full AXI Data Width

The user data width between the kernel and the memory controller can be configured by the Vitis compiler based on the data types of the kernel arguments. To maximize the data throughput, AMD recommends that you choose data types map to the full data width on the memory controller. The memory controller in all supported acceleration cards supports 512-bit user interface, which can be mapped to C/C++ arbitrary precision data type `ap_int<512>` or OpenCL vector data types such as `int16`.

The default is for Vitis HLS to automatically re-size the kernel interface ports up to 512-bits to improve burst access. As shown on the following figure, you can observe burst AXI transactions (Burst Length 16) and a 512-bit package size (Burst Size 64 bytes).

Figure 54: Burst AXI Transactions



This example shows good interface configuration as it maximizes AXI data width, in addition to actual burst transactions.

Complex structs or classes, used to declare interfaces, can lead to very complex hardware interfaces due to memory layout and data packing differences. This can introduce potential issues that are very difficult to debug in a complex system.



RECOMMENDED: Use simple structs for kernel arguments that can be packed to 32-bit boundary.

Waveform View and Live Waveform Viewer

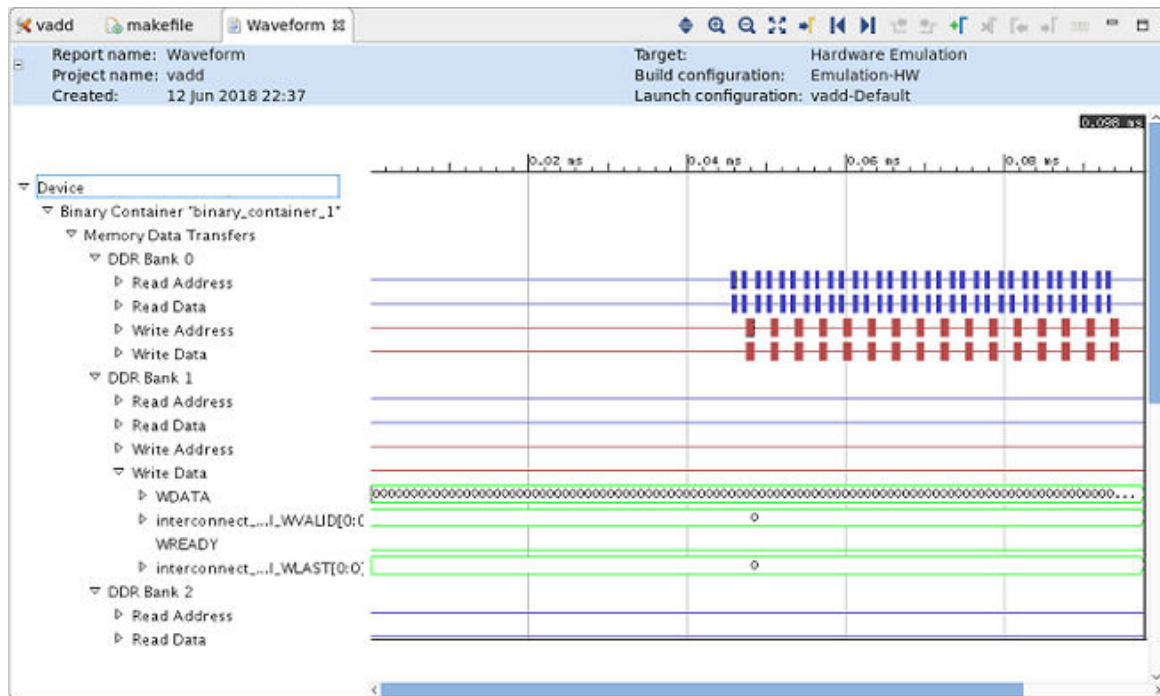
The Vitis core development kit can generate a Waveform view when running hardware emulation. It displays in-depth details at the system-level, CU level, and at the function level. The details include data transfers between the kernel and global memory and data flow through inter-kernel pipes. These details provide many insights into performance bottlenecks from the system-level down to individual function calls to help optimize your application.

The Live Waveform Viewer is similar to the Waveform view, however, it provides even lower-level details with some degree of interactivity. The Live Waveform Viewer can also be opened using the Vivado logic simulator, `xsim`.

Note: The Waveform view allows you to examine the device transactions from within the Vitis analyzer (see [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#)). In contrast, the Live Waveform Viewer opens the Vivado simulation waveform viewer to examine the hardware transactions in addition to any user selected signals.

Waveform data is not collected by default because it requires the runtime to generate simulation waveforms during hardware emulation, which consumes more time and disk space. Refer to [Generating and Opening the Waveform Reports](#) for instructions on enabling these features.

Figure 55: Waveform View



You can also open the waveform database (.wdb) file with the Vivado logic simulator through the Linux command line:

```
xsim -gui <filename.wdb> &
```



TIP: The .wdb file is written to the directory where the compiled host code is executed.

Generating and Opening the Waveform Reports

Follow these instructions to enable waveform data collection from the command line during hardware emulation and open the viewer.

1. Enable debug code generation during compilation and linking using the `-g` option.

```
v++ -c -g -t hw_emu ...
```

2. Create an `xrt.ini` file in the same directory as the host executable with the following contents (see [xrt.ini File](#) for more information).

```
[Emulation]
debug_mode=batch
```

The `debug_mode=batch` enables the capture of waveform data (.wdb) by running simulation in batch mode. You can also enable the Live Waveform Viewer to launch simulation in interactive mode using the following setting in the `xrt.ini`.

```
[Emulation]
debug_mode=gui
```



TIP: If Live Waveform Viewer is enabled, the simulation waveform opens during the hardware emulation run.

3. Run the hardware emulation build of the application as described in [Section V: Simulating the Application with the Emulation Flow](#). The hardware transaction data is collected in the waveform database file, `<hardware_platform>-<device_id>-<xclbin_name>.wdb`. Refer to [Output Directories of the v++ Command](#) or [Output Directories of the Vitis Unified IDE](#) for more information on locating these reports.
4. Open the Waveform view in the Vitis analyzer by opening the Run Summary, and opening the Waveform report.

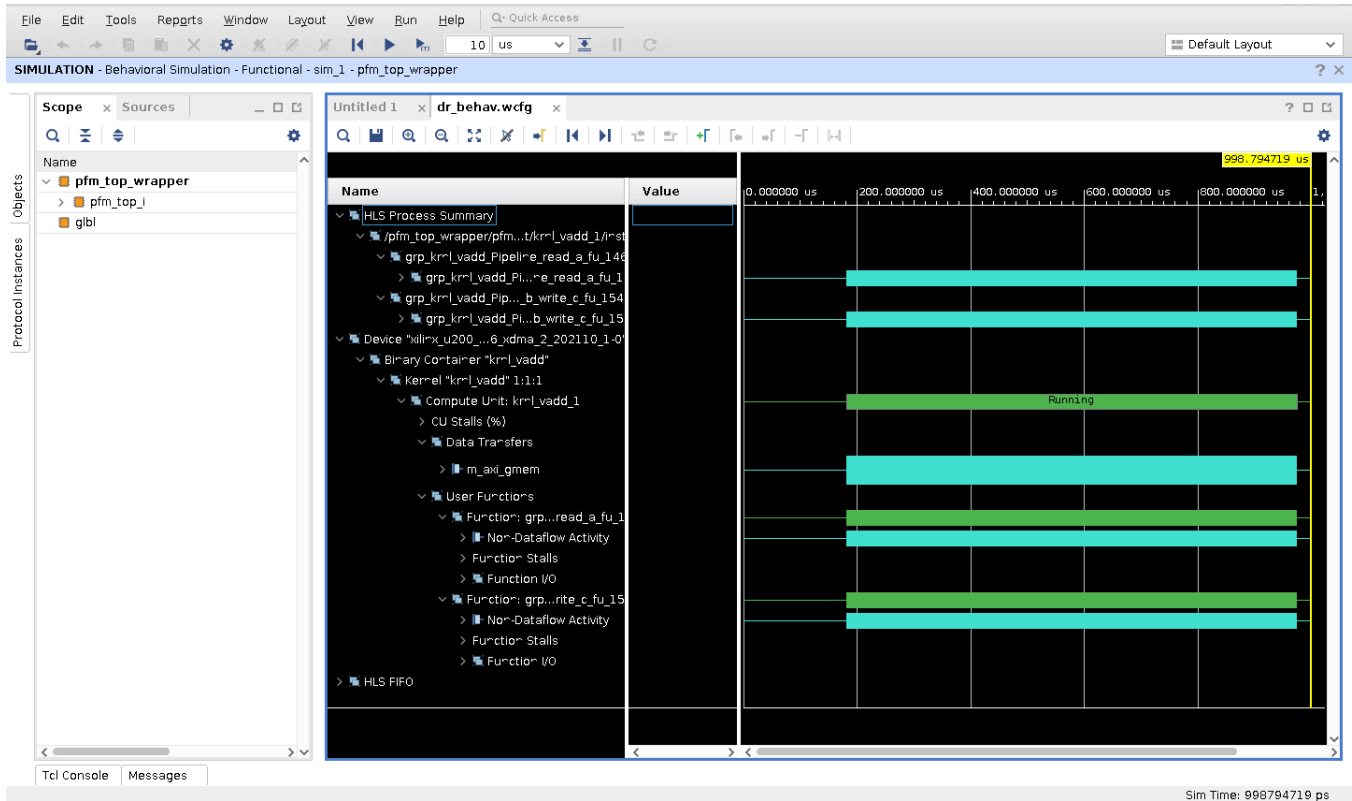
```
vitis_analyzer xrt.run_summary
```

5. Waveforms for TLM transactions can also be dumped for third-party simulators (support limited to Mentor Graphics Questa Advanced Simulator and Cadence Xcelium). Wave data dump is enabled when `v++` link is done with `-g` option (as mentioned in step 1). The format of the wave database dumped is simulator specific (for example, `.wlf` for Questa Advanced Simulator and `.shm` for Xcelium).

Interpreting Data in the Waveform Views

The following image shows the Waveform view:

Figure 56: Waveform View



The Waveform and Live Waveform views are organized hierarchically for easy navigation.

- The Waveform view is based on the actual waveforms generated during hardware emulation (Kernel Trace). This allows the viewer to descend all the way down to the individual signals responsible for the abstracted data. However, because the Waveform view is generated from the post-processed data, no additional signals can be added to the report, and some of the runtime analysis cannot be visualized, such as DATAFLOW transactions.
- The Live Waveform viewer is displaying the Vivado logic simulator (`xsim`) run, so you can add extra signals and internals of the register transfer (RTL) design to the live view. Refer to the *Vivado Design Suite User Guide: Logic Simulation (UG900)* for information on working with the Waveform viewer.

The hierarchy of the Waveform and Live Waveform views include the following:

- **HLS Process Summary:**
 - Hierarchical view of processes and their sub-processes corresponding to the user functions in CU

- Entry for each kernel instance which has processes modeling user function in it (entries can be unfolded to show the processes in it)
- Handshake transactions on all the processes corresponding to user-functions
- Both dataflow and non-dataflow/non-pipeline processes
- The transactions on the processes including stalls, are shown using the corresponding protocol analyzer instance
- Allows you to get an overview about the usage of the individual processes over the execution time (similar to C/C++ profile capabilities)
- **Device "name":** Target device name.
- **Binary Container "name":** Binary container name.
 - **Memory Data Transfers:** For each DDR Bank, this shows the trace of all the read and write request transactions arriving at the bank from the host.
 - **Kernel "name" 1:1:1:** For each kernel and for each compute unit of that kernel, this section breaks down the activities originating from the compute unit.
 - **Compute Unit: "name":** Compute unit name.
 - **CU Stalls (%):** Stall signals are provided by the Vitis HLS tool to inform you when a portion of the circuit is stalling because of external memory accesses, internal streams (that is, dataflow), or external streams (that is, OpenCL pipes). The stall bus shown in detailed kernel trace compiles all of the lowest level stall signals and reports the percentage that are stalling at any point in time. This provides a factor of how much of the kernel is stalling at any point in the simulation.

For example, if there are 100 lowest level stall signals and 10 are active on a given clock cycle, then the CU Stall percentage is 10%. If one goes inactive, then it is 9%.
- **Data Transfers:** This shows the read/write data transfer accesses originating from each Master AXI port of the compute unit to the DDR.
- **User Functions:** This information is available for the HLS kernels and shows the user functions.
 - **Function: "name":** Function name.
 - **Dataflow/Pipeline Activity:** This shows the number of parallel executions of the function if the function is implemented as a dataflow process.
 - **Active Iterations:** This shows the currently active iterations of the dataflow. The number of rows is dynamically incremented to accommodate the visualization of any concurrent execution.
 - **StallNoContinue:** This is a stall signal that tells if there were any output stalls experienced by the dataflow processes (function is done, but it has not received a continue from the adjacent dataflow process).

- **RTL Signals:** These are the underlying RTL control signals that were used to interpret the above transaction view of the dataflow process.
- **Function Stalls:** Shows the different types of stalls experienced by the process.
 - **External Memory:** Stalls experienced while accessing the DDR memory.
 - **Internal-Kernel Pipe:** If the compute units communicated between each other through pipes, then this shows the related stalls.
- **Intra-Kernel Dataflow:** FIFO activity internal to the kernel.
- **Function I/O:** Actual interface signals.
- **HLS FIFO:**
 - This shows waveform for size of HLS FIFOs created inside non-RTL kernels
 - The waveform is in Analog style
 - It shows one entry for each kernel instance which has FIFO in it
 - The analog waveform is produced by tracing on an internal HDL signal of the kernel which gives the current number of elements in the FIFO during simulation.
 - **CU name:** Name of the CU containing FIFO
 - **FIFO instance name:** Name of the FIFO instance
 - **mOutPtr["size":"0"]:** HDL signal which gives the number of elements currently in the FIFO during simulation

Interpreting TLM Waveform Data for Third-Party Simulators

1. Under the respective design hierarchy in the waveform windows, for each TLM–TLM socket connection, the following information is visible as waveforms.
 - a. For memory mapped AXI4 interfaces, the bus transaction is visible as six channels:
 - **<socket_name>:** This channel contains statistics such as whether transaction is read/write and a unique mark to differentiate information in sub channels. This also indicates the number of transactions completed.
 - **<socket_name>_AW:** This channel contains the transaction information of Write Address.
 - **<socket_name>_W:** This channel contains the transaction information of Write Data.
 - **<socket_name>_B:** This channel contains the transaction information of the corresponding Write Response.

- `<socket_name>_AR`: This channel contains the transaction information of Read Address.
- `<socket_name>_R`: This channel contains the transaction information of Read Data.

Detailed attributes for each channel like burst size, burst type, response, etc. are visible as attributes in each channel.

- b. For AXI4-Stream, the bus transaction is visible in one channel only named after the `socket_name`. This contains the information like TID, TDEST, TDATA, etc. as attributes.

Note: TLM waveform viewing is only supported for Questa Advanced Simulator and Xcelium. The information on the usage of waveform, adding socket to waveform view, and detailed view of attributes can be referred to its respective third-party simulator user guide.

Debugging System Projects

The AMD Vitis™ unified software platform provides application-level debug features and techniques that allow the Application component, AI Engine component, PL kernels, and the interactions between them to be debugged. However, debugging projects built from the command line is a challenge because the various elements of the system, the compiled AI Engine graph application (`libadf.a`), the device binary (`.xclbin`), and the top-level application (`host.elf`), must be gathered together and presented as a system.

The Vitis unified IDE provides an excellent framework for debugging these heterogeneous systems. There are many advantages to working in the Debug view in the IDE. In fact, you are strongly recommended to debug your command-line driven projects in the IDE. The process for doing this is broken down into two steps:

1. Import your command-line project into the IDE as described in [Working with User-Managed Flows](#)
2. Debug the system in the IDE as described in [Debugging the System Project and AI Engine Components](#)

The Vitis tools provide application-level debug features which let the host code, the system project, and the interactions between them be efficiently debugged in the Vitis unified IDE. These features and techniques are split between software debugging and hardware debugging flows. The recommended debugging flow consists of three levels of debugging:

- [Debugging in Software Emulation](#) to confirm the algorithm design of the application as represented in both your software application and hardware design. For software debugging, the application and system project can be debugged using the Vitis unified IDE, or using GDB from the command line as a standard debug tool.
- [Debugging in Hardware Emulation](#) to compile the PL kernels into RTL, confirm the behavior of the generated logic, and evaluate the simulated performance of the hardware.
- [Debugging During Hardware Execution](#) to implement the device binary and debug the application running on hardware using Xilinx virtual cable (XVC) running over the PCIe® bus, or debugged using USB-JTAG cables for both Alveo accelerator cards and embedded processor platforms.

This three-tiered approach enables debugging the Application component and System project at different levels of abstraction. Each provides specific insights into the design providing a comprehensive view of the system from software to hardware. All flows are supported through the Vitis unified IDE using basic compile time and runtime setup options.

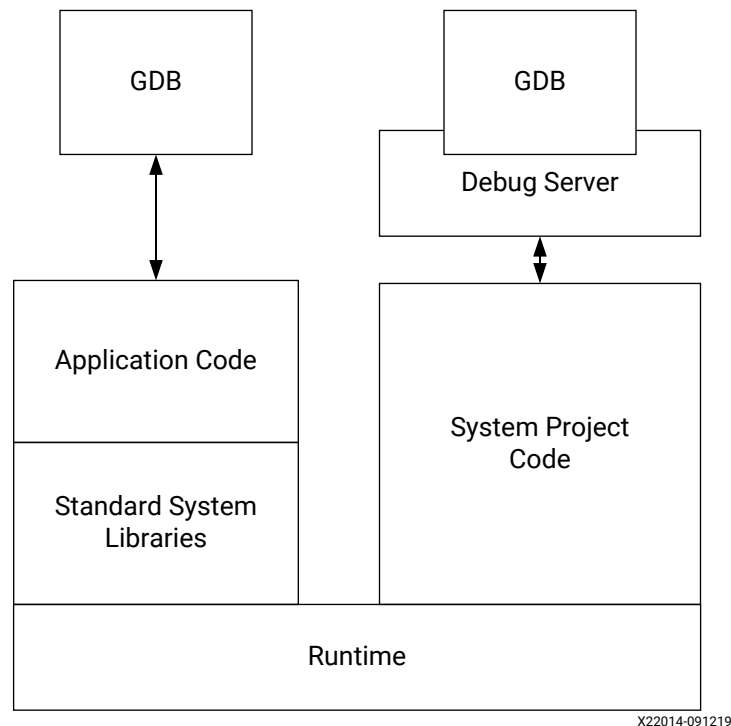
Debugging in Software Emulation

★ **IMPORTANT!** The following steps describe debugging from the command line. However, you can also debug command-line projects in the Vitis unified IDE by opening them in a workspace as described in [Working with User-Managed Flows](#), and debugging them as described in [Debugging the System Project and AI Engine Components](#).

The Vitis tools supports typical software debugging for the host code at all times, and the linked system project when running in software emulation mode (and also at points during hardware emulation). This is a standard software debug flow using breakpoints, stepping through code, analyzing variables, and forcing the code into specific states.

The following figure shows the debug flow during software emulation for the host and kernel code (written in C/C++ or OpenCL™) using the GNU debugging (GDB) tool. Notice the two instances of GDB to separately debug the host and kernel processes, and the use of the debug server (`xrt_server`) to connect to the system project.

Figure 57: Software Emulation



AMD recommends iterating through the design as much as possible in software emulation, which takes little compile time and executes quickly. For more detailed information on software emulation, see [Software Emulation](#).

Using printf() or cout to Debug Kernels

The basic approach to debugging algorithms is to verify key code steps and key data values throughout the execution of the program. For application developers, printing checkpoint statements, and outputting current values in the code is a simple and effective method of identifying issues within the execution of a program. This can be done using the `printf()` function, or `cout` for standard output.

For C/C++ kernel models, `printf()` is only supported during software emulation and should be excluded from the Vitis HLS synthesis step. In this case, any `printf()` statement should be surrounded by the following compiler macros:

```
#ifndef __SYNTHESIS__
    printf("Checkpoint 1 reached");
#endif
```

For C++ kernels, you can also use `cout` in your code to add checkpoints or messages used for debugging the code. For example, you might add the following:

```
std::cout << "TEST " << (match ? "PASSED" : "FAILED") << std::endl;
```

GDB-Based Debugging



IMPORTANT! Both the host and kernel code must be compiled for debugging using the `-g` option.

For the GNU debugging (GDB), you can debug the kernel or host code, adding breakpoints, and inspecting variables. This familiar software debug flow allows quick design, compile, and debug to validate the functionality of your application. The Vitis debugger also provides extensions to GDB to let you examine the content of the Xilinx Runtime (XRT) library from the host program. These extensions can be used to debug protocol synchronization issues between the host and the kernel.

The Vitis core development kit supports GDB host program debugging in all flows, but kernel debugging is limited to the software emulation mode. Debugging features need to be enabled in your host and kernel code by using the `-g` option during compilation and linking.

This section shows how host and kernel debugging can be performed with the help of GDB. Because this flow should be familiar to most software developers, this section focuses on the extensions of host code debugging capabilities for the XRT library and the requirements of kernel debug.

Xilinx Runtime Library GDB Extensions

The Vitis debugger (`xgdb`) enables new GDB commands that give you visibility from the host application into the XRT library.

Note: If you launch GDB outside of the Vitis debugger, the command extensions need to be enabled using the `appdebug.py` script as described in [Launching Host and Kernel Debug](#).

There are two kinds of commands which can be called from the `gdb` command line:

1. `xprint` commands that give visibility into XRT library data structures (`cl_command_queue`, `cl_event`, and `cl_mem`). These commands are explained below.
2. `xstatus` commands that give visibility into IP running on the Vitis target platform when debugging during hardware execution.

You can get more information about the `xprint` and `xstatus` commands by using the `help <command>` from the `gdb` command prompt.

A typical application for these commands is when you see the host application hang. In this case, the host application could be waiting for the command queue to finish, or waiting on an event list. Printing the command queue using the `xprint queue` command can tell you what events are unfinished, allowing you to analyze dependencies between events.

The output of both of these commands is automatically tracked when debugging with the Vitis IDE. In this case, three tabs are provided next to the common tabs for Variables, Breakpoints, and Registers in the upper left corner of the debug perspective. These are labeled Command Queue, Memory Buffers, and Platform Debug, showing the output of `xprint queue`, `xprint mem`, and `xstatus`, respectively.

xprint Commands

The arguments to `xprint queue` and `xprint mem` are optional. The application debug environment keeps track of all the XRT library objects and automatically prints all valid queues and `cl_mem` objects if the argument is not specified. In addition, the commands do a proper validation of supplied command `queue`, `event`, and `cl_mem` arguments.

```
xprint queue [<cl_command_queue>]
xprint event <cl_event>
xprint mem [<cl_mem>]
xprint kernel
xprint all
```

xstatus Commands

This functionality is only available in the system flow (hardware execution) and not in any of the emulation flows.

```
xstatus all
xstatus --<ipname>
```

GDB Kernel-Based Debugging

GDB kernel debugging is supported for the software emulation flow. When the GDB executable is connected to the kernel in the IDE or command line flows, you can set breakpoints and query the content of variables in the kernel, similar to normal host code debugging. This is fully supported in the software emulation flow because the kernel GDB processes attach to the spawned software processes.

Command Line Debug Flow for Software Emulation



IMPORTANT! You should import your command-line project into the Vitis unified IDE as described in [Working with User-Managed Flows](#), and debug it as described in [Debugging the System Project and AI Engine Components](#).

The following describes the steps required to run the debug flow for software emulation from the command line:

1. Set up the command shell or window as described in [Setting Up the Vitis Environment](#) prior to running the tools.
2. Compile and Link the host code and System project for debugging by adding the `-g` option to the `g++` command line as described in [Section IV: Building and Running the System](#).
3. Launch GDB to debug the application. There are currently two flows supporting debug during software emulation:
 - a. Debugging PL or AI Engine kernels using the `kernel-dbg` option as explained in [Software Emulation Debug for Embedded Processors](#), or [Software Emulation Debug for Alveo Accelerators](#). This is a simple flow, but only enables debugging of the kernels in the `.xclbin`, and not the host application.
 - b. Debugging the host application and PL kernels using three command terminals and `xrt_server` as described in [Launching Host and Kernel Debug](#). This is a more complex flow, but supports concurrent debug of both the host and kernels.

Software Emulation Debug for Embedded Processors

You can use the following process to debug the system in software emulation for an embedded processor platform, such as an AMD Versal™ VCK190 or AMD Zynq™ UltraScale+™ MPSoC ZCU102 or ZCU102. This process launches a new terminal that allows for `gdb` commands to be used, in addition to visual of the files for code stepping.

1. After completing the `sw_emu` build process, launch the QEMU emulation environment with debug using one of the following methods:
 - If you are using [PS on x86](#) flow, add `kernel-dbg` to the `xrt.ini` file as follows:

```
[Emulation]
kernel-dbg=true
```

For more information on all options available in the `xrt.ini` file, see [xrt.ini File](#).

- Otherwise, use the following command from your build directory to launch the emulation environment with debug.

```
./emulation/launch_sw_emu.sh -kernel-dbg true
```

Where,

- `./emulation` is the output directory of the `v++ --package` command.
 - `-kernel-dbg true` sets up the emulator to run `gdb` at the execution of the application kernels (PL, AI Engine).
2. As described in [Running Emulation on an Embedded Processor Platform](#), run the following commands in the QEMU shell once you see the `qemu%` prompt:

```
mount /dev/mmcblk0p1 /mnt
cd /mnt
export LD_LIBRARY_PATH=/mnt:/tmp:$LD_LIBRARY_PATH
export XCL_EMULATION_MODE=sw_emu
export XILINX_XRT=/usr
```

3. Run the PS application from the QEMU environment. For example: `./host.exe a.xclbin`

Launching the host application also launches `gdb` in a separate terminal to let you debug the PL and AI Engine kernels in the `.xclbin`. In `gdb`, you can perform all the typical activities to set up your debug environment, such as breakpoint insertion for PL and AI Engine kernels, and step or continue commands to walk through the code.

4. You can set a breakpoint on either function name or line number in `gdb` using this syntax. During execution, when it reaches that breakpoint, `gdb` automatically opens the file with the right line number, as long as it can locate the sources. For example,

```
break <filename>:<function name>
break <filename>:<line_num >
```

Also, you can set a breakpoint for a kernel using a command such as `b Vadd_A_B`. This command pauses `.xclbin` execution in `gdb` at the invocation of the specified kernel, in this example the `Vadd_A_B` kernel. In an `.xclbin` file with multiple kernels, you can add breakpoints for all or specific kernels.

Note: When you set the breakpoint in `gdb`, you will see a note that the function is not defined and prompting you to make the breakpoint pending on future load. Enter `y` to proceed.

5. In the `gdb` terminal, press `r` to run the application and step through the code, set variable values, and continue. To get a textual user interface of `gdb`, select with **Ctrl + X**, **Ctrl + A**. If the text interface is used the source code for the kernel is displayed.
6. Step through the code, or press `c` to let the code run through to completion. The results of your software emulation are displayed in the original command terminal.

Software Emulation Debug for Alveo Accelerators

You can use the following process to debug the system in software emulation for an AMD Alveo™ accelerator card. This process launches a new terminal that allows for `gdb` commands to be used, in addition to displaying the source files for code stepping.

1. Complete the `sw_emu` build process, using the `-g` or `--debug` options when building the host and kernels.
2. Add the `kernel-dbg=true` option to the `xrt.ini` file prior to running the application:

```
[Emulation]
kernel-dbg=true
```

3. As described in [Running Emulation on Data Center Accelerator Cards](#), set the `XCL_EMULATION_MODE` to `sw_emu`:

```
setenv XCL_EMULATION_MODE sw_emu
```

4. Launch the host application with the `.xclbin` file: `./host.exe a.xclbin`

Launching the host application also launches `gdb` in a separate terminal to let you debug the PL kernels in the `.xclbin`. In `gdb`, you can perform all the typical activities to set up your debug environment, such as breakpoint insertion for PL kernels, and step or continue commands to walk through the code.

5. You can set a breakpoint on either function name or line number in `gdb` using this syntax. During execution, when it reaches that breakpoint, `gdb` automatically opens the file with the right line number, as long as it can locate the sources. For example,

```
break <filename>:<function name>
break <filename>:<line_num >
```

Also, you can set a breakpoint for a kernel using a command such as `b Vadd_A_B`. This command pauses `.xclbin` execution in `gdb` at the invocation of the specified kernel, in this example the `Vadd_A_B` kernel. In an `.xclbin` file with multiple kernels, you can add breakpoints for all or specific kernels.

Note: When you set the breakpoint in `gdb`, a message is displayed that the function is not defined and prompt you to make the breakpoint pending on future load. Enter `y` to proceed.

6. In the `gdb` terminal, press `r` to run the application and step through the code, set variable values, and continue.



TIP: To use a Textual User Interface (TUI) in `gdb`, use the following keystrokes:

```
Ctrl+x Ctrl+a
```

7. If the TUI is displayed, the source code for the kernel is displayed, as well as the path to the source code defined in the `.xclbin` by the software emulation build process.
8. Step through the code, or press `c` to let the code run through to completion. The results of your software emulation are displayed in the original command terminal.

9. Press **q** to quit `gdb`.
10. Close the `gdb` command terminal.

Launching Host and Kernel Debug

In software emulation, to better model the hardware accelerator, the execution of the FPGA binary is spawned as a separate process. If you are using GDB to debug the host code, breakpoints set in kernel code are not encountered because the kernel code is not run within the host code process. To support the concurrent debugging of the host and kernel code, the Vitis debugger provides a system to attach to spawned kernels through the use of the debug server (`xrt_server`). To connect the host and kernel code to the debug server, you must open three terminal windows using the following process.



TIP: This flow should also work while using a graphical front-end for GDB, such as the data display debugger (DDD) available from GNU. The following steps are the instructions for launching GDB.

1. Open three terminal windows, and set up each window as described in [Setting Up the Vitis Environment](#). The three windows are for:
 - Running `xrt_server`
 - Running GDB (`xgdb`) on the Host Code
 - Running GDB (`xgdb`) on the Kernel Code
2. In the first terminal, after setting up the terminal environment, start the Vitis debug server using the following command:

```
xrt_server --sdx-url
```

The debug server listens for debug commands from the host and kernel, connecting the two processes to create a single debug environment. The `xrt_server` returns a `listener port <num>` on standard out. To control this process, you must start new GDB instances and connect to the `xrt_server` through the listener port. This is done in the next steps.



TIP: The initial listener port should not be connected to as it is used for connection by TCF agents such as the Vitis IDE. In this command-line flow the first listener port should be ignored.



IMPORTANT! With the `xrt_server` running, all spawned GDB processes wait for control from you. If no GDB ever attaches to the `xrt_server`, or provides commands, the kernel code appears to hang.

3. In a second terminal, after setting up the terminal environment, launch GDB for the host code as described in the following steps:
 - a. Set the `ENABLE_KERNEL_DEBUG` environment variable. For example, in a C-shell use the following:

```
setenv ENABLE_KERNEL_DEBUG true
```

- b. Set the `XCL_EMULATION_MODE` environment variable to `sw_emu` mode as described in [Running the System on Hardware](#). For example, in a C-shell use the following:

```
setenv XCL_EMULATION_MODE sw_emu
```

- c. The runtime debug feature must be enabled using an entry in the `xrt.ini` file, as described in [xrt.ini File](#). Create an `xrt.ini` file in the same directory as your host executable, and include the following lines:

```
[Debug]
app_debug=true
```

This informs the runtime library that the kernel has been compiled for debug, and that XRT library should enable debug features.

- d. Start `gdb` through the AMD wrapper:

```
xgdb --args <host> <xclbin>
```

Where `<host>` is the name of your host executable, and `<xclbin>` is the name of the FPGA binary. For example:

```
xgdb --args host.exe vadd.xclbin
```

Launching GDB from the `xgdb` wrapper performs the following setup steps for the Vitis debugger:

- Loads GDB with the specified host program.
- Sources the Python script from the GDB command prompt to enable the Vitis debugger extensions:

```
gdb> source ${XILINX_XRT}/share/appdebug/appdebug.py
```

When you run the host application in `gdb`, it spawns off the software emulation process. The software emulation process detects that `xrt_server` is running and connects to it. At that point, a listener port for the kernel `gdb` is created and the software emulation process waits until it receives commands. Now, you should connect the kernel `gdb` to the `xrt_server` listener port as described in the next step.

4. In a third terminal, after setting up the terminal environment, launch the `xgdb` command, and run the following commands from the (`gdb`) prompt:

- For software emulation:

```
file <Vitis_path>/data/emulation/unified/cpu_em/generic_pcie/model/genericpciemodel
```

Where `<Vitis_path>` is the installation path of the Vitis core development kit. Using the `$XILINX_VITIS` environment variable does not work inside GDB.

- Connect to the kernel process:

```
target remote :<num>
```


Where `<num>` is the listener port number returned by the `xrt_server`.

With the three terminal windows running the `xrt_server`, GDB for the host, and GDB for the kernels, you can set breakpoints on your host or kernels as needed, run the `continue` command, and debug your application. When the all kernel invocations have finished, the host code continues and the `xrt_server` connection drops.

Debugging in Hardware Emulation

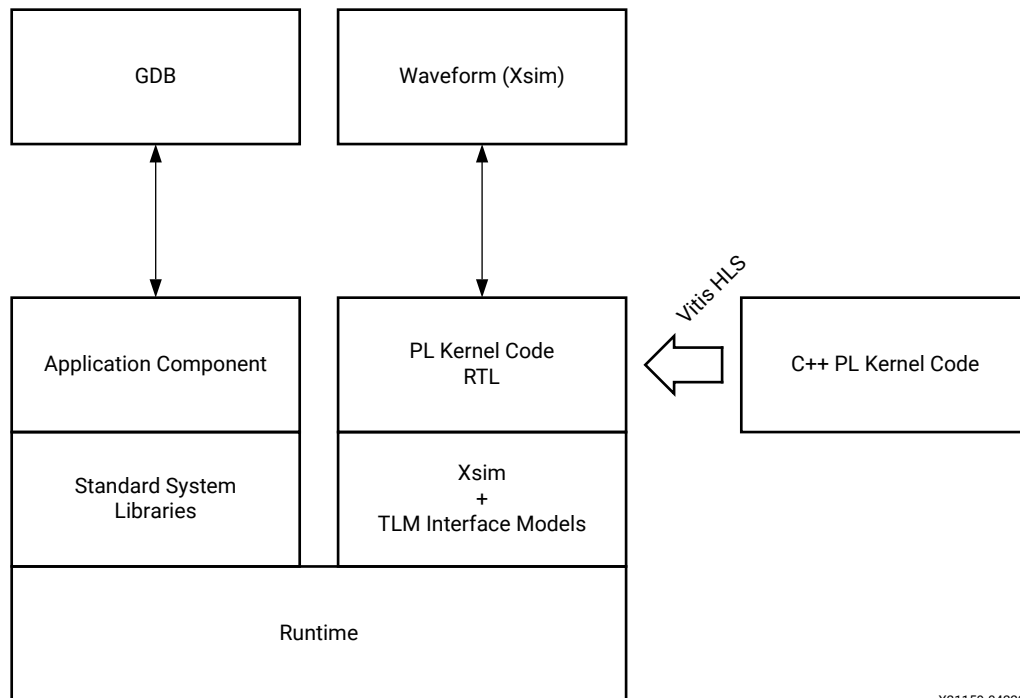


IMPORTANT! *The following steps describe debugging from the command line. However, you are strongly encouraged to debug command-line projects in the Vitis unified IDE by opening them in a workspace as described in [Working with User-Managed Flows](#), and debugging them as described in [Debugging the System Project and AI Engine Components](#).*

During hardware emulation, kernel code is compiled into RTL code so that you can evaluate the RTL logic of kernels prior to implementation into the AMD device. The host code can be executed concurrently with a behavioral simulation of the RTL model of the kernel, directly imported, or created through Vitis HLS from the C/C++/OpenCL kernel code. For more information, see [Hardware Emulation](#).

The following figure shows the hardware emulation flow diagram which can be used in the Vitis debugger to validate the host code, profile host and kernel performance, give estimated FPGA resource usage, and verify the kernel using an accurate model of the hardware (RTL). The RTL kernel code is analyzed in an AMD Vivado™ simulator or third-party RTL simulator. GDB is used for more traditional software-style debugging of the Application component.

Figure 58: Hardware Emulation



XZ1159-042221

Verify the host code and the kernel hardware implementation is correct by running hardware emulation on a data set. The hardware emulation flow invokes the Vivado logic simulator in the Vitis core development kit to test the kernel logic that is to be executed on the FPGA fabric. The interface between the models is represented by a transaction-level model (TLM) to limit impact of interface model on the overall execution time. The execution time for hardware emulation is longer than software emulation.



TIP: AMD recommends that you use small data sets for debug and validation in hardware emulation as it takes considerably longer than running in the hardware device.

During hardware emulation, you can optionally modify the kernel code to improve performance. Iterate your host and kernel code design in hardware emulation until the functionality is correct, and the estimated kernel performance is satisfactory.

Enable Waveform Debugging with the Vitis Compiler Command

The waveform debugging process can be enabled through the `v++` command using the following steps:

1. Compile and Link the host code and System project for debugging by adding the `-g` option to the `g++` command line as described in [Section IV: Building and Running the System](#).

2. Create an `xrt.ini` file in the same directory as the host executable, as described in [xrt.ini File](#), with the following contents:

```
[Emulation]
debug_mode=batch
```



TIP: Use `debug_mode=gui` to run the Vivado simulator in GUI mode and enable displaying the waveform for interactive use, as described in [Running Live Waveform Mode](#). This is especially useful when debugging a `hw_emu` hang issue, because you can interrupt the simulation process in the simulator and observe the waveform up to that time.

3. Launch the hardware emulation using the `launch_hw_emu.sh` script that is generated during the `v++ --package` process.

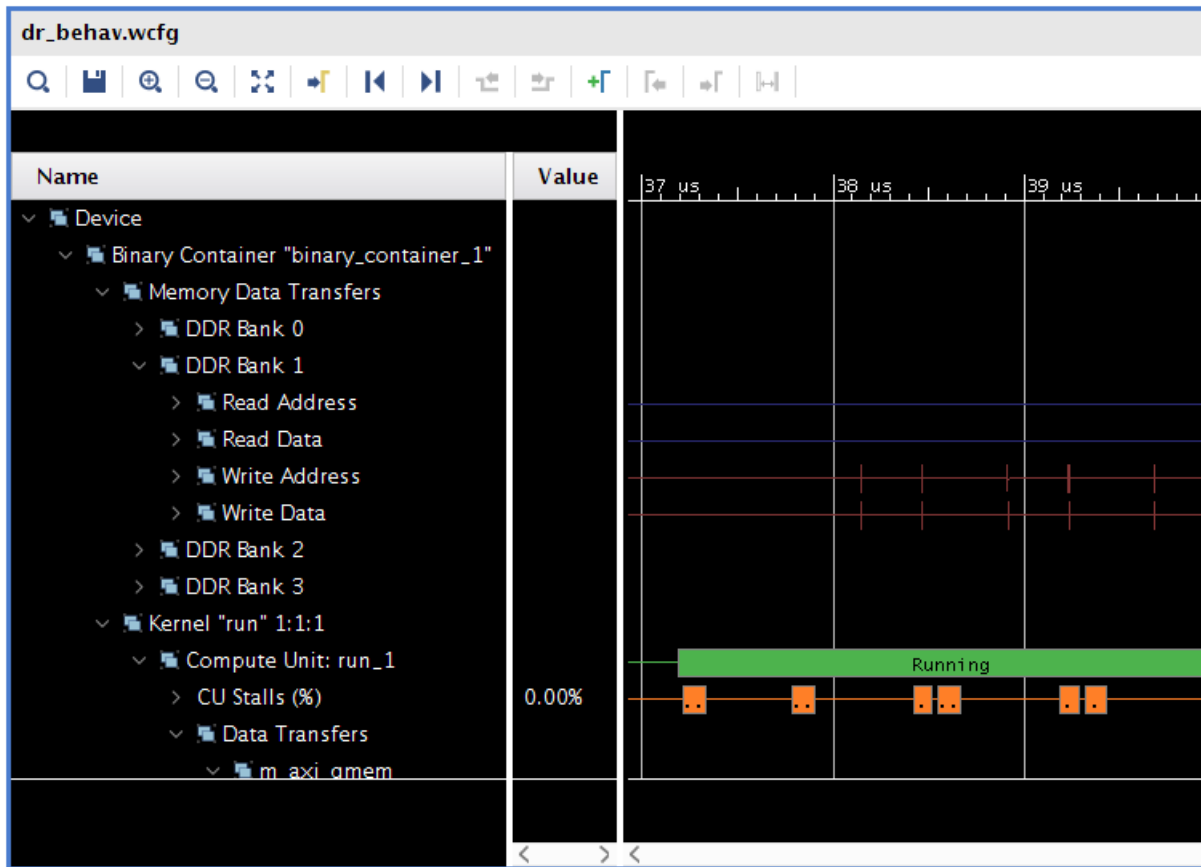
This script will launch the Vivado simulator and enable batch simulation to capture the simulation waveform database. The waveform database is written to a `.wdb` that can be opened in the Analysis view of the Vitis unified IDE as described in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Running Live Waveform Mode

The Vitis unified IDE provides live waveform-based HDL debugging in the hardware emulation mode. The waveform is opened in the Vivado waveform viewer which should be familiar to Vivado logic simulation users. The Vitis IDE lets you display kernel interfaces, internal signals, and includes debug controls such as restart, HDL breakpoints, as well as HDL code lookup and waveform markers. In addition, it provides top-level DDR data transfers (per bank) along with kernel-specific details including compute unit stalls, loop pipeline activity, and data transfers. For details, see [Waveform View and Live Waveform Viewer](#).

If the live waveform viewer is activated, the waveform viewer automatically opens when running the executable. By default, the waveform viewer shows all interface signals and the following debug hierarchy:

Figure 59: Waveform Viewer



- **Memory Data Transfers:** Shows data transfers from all compute units funnel through these interfaces.



TIP: These interfaces could be a different bit width from the compute units. If so, then the burst lengths would be different. For example, a burst of sixteen 32-bit words at a compute unit would be a burst of one 512-bit word at the OCL master.

- **Kernel <kernel name><workgroup size> Compute Unit<CU name>:** Kernel name, workgroup size, and compute unit name.
- **CU Stalls (%):** This shows a summary of stalls for the entire CU. A bus of all lowest-level stall signals is created, and the bus is represented in the waveform as a percentage (%) of those signals that are active at any point in time.
- **Data Transfers:** This shows the data transfers for all AXI masters on the CU.
- **User Functions:** This lists all of the functions within the hierarchy of the CU.
- **Function: <function name>:** This is the function name.
- **Dataflow/Pipeline Activity:** This shows the function-level loop dataflow/pipeline signals for a CU.

- **Function Stalls:** This lists the three stall signals within this function.
- **Function I/O:** This lists the I/O for the function. These I/O are of protocol `-m_axi`, `ap_fifo`, `ap_memory`, or `ap_none`.



TIP: As with any waveform debugger, additional debug data of internal signals can be added by selecting the instance of interest from the scope menu and the signals of interest from the object menu. Similarly, debug controls such as HDL breakpoints, as well as HDL code lookup and waveform markers are supported. Refer to the Vivado Design Suite User Guide: Logic Simulation ([UG900](#)) for more information on working with the waveform viewer.

Debug Techniques for Hardware Emulation

Due to the approximate models used in hardware emulation, the behavior of an emulated system might not match the hardware. The following list provides some common issues to examine if your application does not give expected results during hardware emulation:

1. Review the host application to ensure that the event dependency between different kernel runs is correctly captured. Such issues can lead to unpredictable behavior. It is also possible that the application can pass in hardware, but there could be a logical bug in your application which can be triggered on hardware under slightly different conditions.
2. If you have an RTL kernel, run the application in debug mode and ensure that there are no "X" (undriven values) in simulation in the kernel. This indicates incorrect code which can work in hardware but will fail in simulation with unpredictable behavior. If it is an HLS-generated kernel, confirm that all the variables are initialized to appropriate values.
3. Ensure that the amount of data being processed by kernels in hardware emulation is small so that emulation can finish in a reasonable time. Otherwise, it can appear that the application is running forever or has "hung". In this case, when running the application in hardware emulation look for `INFO: [Vitis-EM 22]` messages in the host application console. Check that the amount of data being read/written to or from global memory is increasing:
 - a. If the RD/WR data is increasing, this indicates that application and hardware execution is progressing. The application is not hung, but is taking a really long time to complete. This could be due to large data size or due to kernels performing memory read/write in an inefficient manner. The application and kernel need to be optimized.
 - b. If the RD/WR data is not increasing in successive messages, this indicates that simulation is running but there is a deadlock in the hardware somewhere – either in the kernel or rest of the platform. Review the AXI transactions at the boundary of kernel, interconnect (for example, `sdx_memss`), and other places to check if there is an incomplete transaction or whether any transaction is being generated by the kernel.
4. Run hardware emulation in waveform mode and also review at the timeline trace. Check whether the kernel is getting "started" and "done" by observing the traffic on its AXI4-Lite interface, or by observing the output interrupt from the kernel.
5. Review the `[Emulation]` section of the [xrt.ini File](#) to enable applicable settings that can help to narrow down the issue in your application or kernel.

Hardware Emulation Debug from the Command Line

With the Hardware Emulation build complete, including the AI Engine graph, the PL region kernels, and the PS application, you can use the following steps to debug the system design.

1. Launch the QEMU emulator environment using the `launch_hw_emu.sh` script, that is generated during the `--package` process.
2. For AI Engine platforms, the files required for emulation of the system are defined by the `--package` command, including the emulation script. To launch the emulation environment use the following command from your build directory:

```
./emulation/launch_hw_emu.sh -pid-file emulation.pid -no-reboot \  
-add-env ENABLE_RDWR_DEBUG=true -add-env RDWR_DEBUG_PORT=10100 -forward-  
port 1440 1534
```

Where:

- `./emulation` is the output directory of the package process as described in [Packaging Versal Designs](#), and is where the `launch_hw_emu.sh` script can be found.
- `-add-env RDWR_DEBUG_PORT=${aie_mem_sock_port}` defines the port for communicating with the AI Engine domain. In the previous example, it is 10100.
- `-forward-port ${linux_tcf_agent_port} 1534` defines the port for the Linux TCF agent. In the previous example, it is 1440, which is the default.



TIP: Any free ports can be used for `aie_mem_sock_port` and `linux_tcf_agent_port` in the above command template. However, these port are mandatory for enabling the AI Engine application and Linux application debug respectively.

This command launches the emulator and then waits until Linux is booted within the QEMU. The QEMU shell shows a transcript of the QEMU launch and Linux boot process. You can tell when the process has completed when the `qemu%` prompt is displayed. At that time you are ready to proceed.

3. In a second terminal window, launch the XRT server application using the following command:

```
xrt_server -I300 -S -s tcp::4352
```

where:

- `-I300` defines an idle timeout, in which the server quits if there is no response.
- `-S` specifies print server properties in JSON format to stdout.
- `-s tcp::${xrt_server_port}` defines the agent listening protocol and port. In the previous example, it is 4352, but can be any free port.

From this point you can do all the debug activities like step in/step over/viewing variables/plant break points in the emulation environment.

Debugging During Hardware Execution

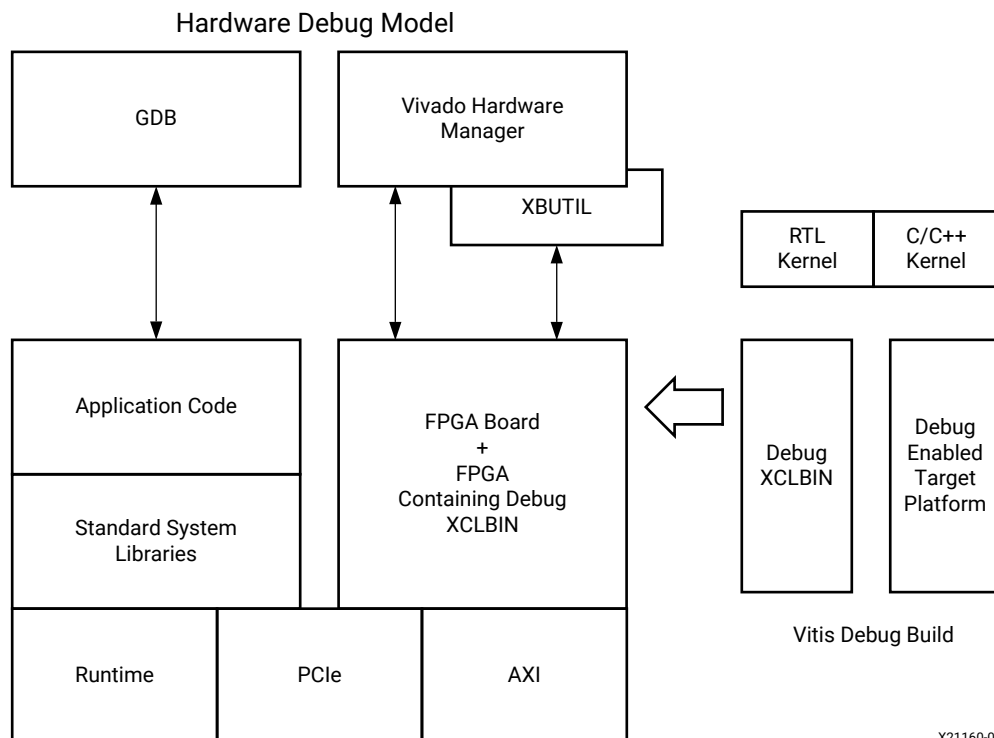
★ **IMPORTANT!** The following steps describe debugging from the command line. However, you are strongly encouraged to debug command-line projects in the Vitis unified IDE by opening them in a workspace as described in [Working with User-Managed Flows](#), and debugging them as described in [Debugging the System Project and AI Engine Components](#).

During hardware execution, the actual hardware platform is used to execute the kernels, and you can evaluate the performance of the host program and accelerated kernels by running the application. However, debugging the hardware build requires additional logic to be incorporated into the application. This will impact both the FPGA resources consumed by the kernel and the performance of the kernel running in hardware. The debug configuration of the hardware build includes special ChipScope debug cores, such as Integrated Logic Analyzer (ILA) and Virtual Input/Output (VIO) cores, and AXI performance monitors for debug purposes.

💡 **TIP:** The additional logic required for debugging the hardware should be removed from the final production build.

The following figure shows the debug process for the hardware build, including debugging the host code using GDB, and using the Vivado hardware manager, with waveform analysis, kernel activity reports, and memory access analysis to identify and localize hardware issues.

Figure 60: Hardware Execution



X21160-092519

With the system hardware build configured for debugging, the host program running on the CPU and the Vitis accelerated kernels running on the AMD device can be confirmed to be executing correctly on the actual hardware of the target platform. Some of the conditions that can be identified and analyzed include the following:

- System hangs caused by protocol violations:
 - These violations can take down the entire system.
 - These violations can cause the kernel to get invalid data or to hang.
 - It is hard to determine where or when these violations originated.
 - To debug this condition, you should use an ILA triggered off of the AXI protocol checker, which needs to be configured on the Vitis target platform.
- Problems with the hardware kernel:
 - Problems sometimes caused by the implementation: timing issues, race conditions, and bad design constraints.
 - Functional bugs that hardware emulation does not reveal.
- Performance issues:
 - For example, the frames per second processing is not what you expect.
 - You can examine data beats and pipelining.
 - Using an ILA with trigger sequencer, you can examine the burst size, pipelining, and data width to locate the bottleneck.

Enabling Kernels for Debugging with Chipscope

System ILA

The key to hardware debugging lies in instrumenting the kernels with the required debug logic. The following topic discusses the `v++` linker options that can be used to list the available kernel ports, enable the System Integrated Logic Analyzer (ILA) core on selected ports, and enable the AXI Protocol Checker debug core for checking for protocol violations.

The ILA core provides transaction-level visibility into an instance of a compute unit (CU) running on hardware. AXI traffic of interest can also be captured and viewed using the ILA core. The ILA provides custom event triggering on one or more signals to allow waveform capture at system speeds. The waveforms can be analyzed in a viewer and used to debug hardware, finding protocol violations, or performance issues. It can also be crucial for debugging difficult situation like application hangs.

Captured data can be accessed through the AMD virtual cable (XVC) using the Vivado tools. See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)) for complete details.

The ILA core can be added to an existing RTL kernel to enable debugging features within that design, or it can be inserted automatically by the `v++` compiler during the linking stage. The `v++` command provides the `--debug` option as described in [--debug Options](#) to attach System ILA cores at the interfaces to the kernels for debugging and performance monitoring purposes.



IMPORTANT! ILA debug cores require system resources, including logic and local memory to capture and store the signal data. Therefore they provide excellent visibility into your kernel, but they can affect both performance and resource utilization.

The `--debug` option to enable ILA IP core insertion has the following syntax:

```
--debug.chipscope <cu_name>[:<interface_name>]>
```



TIP: The `<interface_name>` is optional, and if not specified all ports on the CU will be analyzed. You can use the `--debug.list_ports` option to return the interface names on the kernel to use with `--debug` options.

In case of a flattened design or any design where there would be multiple debug bridges in master mode, the flow will not pick one to stitch the debug cores, a constraint is needed to define the connectivity. For example in a Samsung Smart SSD U.2 flat shell, there is no partitioning between the static and dynamic regions while generating the kernels with the debug (ILA) options enabled. It is required to specify the connectivity of the kernel AXI ports that needs to be under debug to the user debug bridge in the dynamic region.

To specify the connectivity, you must provide the option below in the `v++` command line:

```
--advanced.paramcompiler.userPostDebugProfileOverlayTcl=<path to  
post_dbg_profile_overlay.tcl >
```

Inside the `post_dbg_profile_overlay.tcl`, the file must call the XDC file with the connect debug core command and mention its processing order.

For example, the contents in the `post_dbg_profile_overlay.tcl` file are given below.

```
read_xdc < path to the connect_debug_core.xdc file>  
set_property used_in_implementation TRUE [get_files <path to the  
connect_debug_core.xdc file>]  
set_property PROCESSING_ORDER EARLY [get_files <path to the  
connect_debug_core.xdc file>]]
```

In the `connect_debug_core.xdc` file, you have to specify the `connect_debug_cores` constraint.

For example:

```
connect_debug_cores -master [get_cells -hierarchical -filter {NAME =~  
*debug_bridge_xsdbm/inst/xsdbm}]  
-slaves [get_cells -hierarchical -filter {NAME =~ level0_i/ulp/  
system_ila_0}]
```

AXI Protocol Checker

The AXI Protocol Checker core monitors AXI interfaces. When attached to an interface, it actively checks for protocol violations and provides an indication of which violation occurred. You can assign it for all CUs in the design, or for specific CUs and ports.

The `--debug` option to enable AXI Protocol Checker insertion has the following syntax:

```
--debug.protocol all
```

The protocol checker can be specified with the keyword `all`, or the `<cu_name>:<interface_name>`.

Note: The `--debug.list_ports` option can be specified to return the actual names of ports on the kernel to use with `protocol` or `chipscope`.

An example flow you could use for adding ILA or protocol checkers to your design is outlined below:

1. Compile the kernel source files into an XO file, using the `-g` option to instrument the kernel for debug features:

```
v++ -c -g -k <kernel_name> --platform <platform> -o <kernel_xo_file>.xo  
<kernel_source_files>
```

2. After the kernel has been compiled into an XO file, use `--debug.list_ports` to cause the `v++` compiler to print the list of valid compute units and port combinations for the kernel:

```
v++ -l -g --platform <platform> --connectivity.nk  
<kernel_name>:<compute_units>:<kernel_nameN>  
--debug.list_ports <kernel_xo_file>.xo
```

3. Add the ILA or AXI debug cores on the desired ports by replacing `list_ports` with the appropriate `--debug.chipscope` or `--debug.protocol` command syntax:

```
v++ -l -g --platform <platform> --connectivity.nk  
<kernel_name>:<compute_units>:<kernel_nameN>  
--debug.chipscope <compute_unit_name>:<interface_name>  
<kernel_xo_file>.xo
```



TIP: The `--debug` option can be specified multiple times in a single `v++` command line, or configuration file to specify multiple CUs and interfaces.

When the design is built, you can debug the design using the Vivado hardware manager as described in [Debugging with ChipScope](#).

Adding Debug IP to RTL Kernels



IMPORTANT! This debug technique requires familiarity with the Vivado Design Suite, and RTL design.

You can also enable debugging in RTL kernels by manually adding ChipScope debug cores like the ILA and VIO in your RTL kernel code before packaging it for use in the Vitis development flow. From within the Vivado Design Suite, edit the RTL kernel code to manually instantiate an ILA debug core, or VIO IP from the AMD IP catalog, similar to using any other IP in Vivado IDE. Refer to the HDL Instantiation flow in the *Vivado Design Suite User Guide: Programming and Debugging* (UG908) to learn more about adding debug cores to your design.

The best time to add debug cores to your RTL kernel is when you create it. However, debug cores consume device resources and can affect performance, so it is good practice to make one kernel for debug and a second kernel for production use. The `rtl_vadd_hw_debug` of the [RTL Kernels](#) examples on GitHub shows an ILA debug core instantiated into the RTL kernel source file. The ILA monitors the output of the combinatorial adder as specified in the `src/hdl/krn1_vadd_rtl_int.sv` file.

```
// ILA monitoring combinatorial adder
ila_0 i_ila_0 (
    .clk(ap_clk),           // input wire      clk
    .probe0(areset),        // input wire [0:0] probe0
    .probe1(rd_fifo_tvalid_n), // input wire [0:0] probe1
    .probe2(rd_fifo_tready), // input wire [0:0] probe2
    .probe3(rd_fifo_tdata),  // input wire [63:0] probe3
    .probe4(adderr_tvalid),  // input wire [0:0] probe4
    .probe5(adderr_tready_n), // input wire [0:0] probe5
    .probe6(adderr_tdata)   // input wire [31:0] probe6
);
```

You can also add the ILA debug core using a Tcl script from within an open Vivado project, using the Netlist Insertion flow described in *Vivado Design Suite User Guide: Programming and Debugging* (UG908), as shown in the following Tcl script example:

```
create_ip -name ila -vendor xilinx.com -library ip -version 6.2 -
module_name ila_0
set_property -dict [list CONFIG.C_PROBE6_WIDTH {32} CONFIG.C_PROBE3_WIDTH
{64} \
CONFIG.C_NUM_OF_PROBES {7} CONFIG.C_EN_STRG_QUAL {1}
CONFIG.C_INPUT_PIPE_STAGES {2} \
CONFIG.C_ADV_TRIGGER {true} CONFIG.ALL_PROBE_SAME_MU_CNT {4}
CONFIG.C_PROBE6_MU_CNT {4} \
CONFIG.C_PROBE5_MU_CNT {4} CONFIG.C_PROBE4_MU_CNT {4}
CONFIG.C_PROBE3_MU_CNT {4} \
CONFIG.C_PROBE2_MU_CNT {4} CONFIG.C_PROBE1_MU_CNT {4}
CONFIG.C_PROBE0_MU_CNT {4}] [get_ips ila_0]
```

After the RTL kernel has been instrumented for debug with the appropriate debug cores, you can analyze the hardware in the Vivado hardware manager as described in [Debugging with ChipScope](#).

Enabling ILA Triggers for Hardware Debug

To perform hardware debug of both the host program and the kernel code running on the target platform, the application host code must be modified to let you set up the ILA trigger conditions *after* the kernel has been programmed into the device, but *before* starting the kernel.

Adding ILA Triggers Before Starting Kernels

Pausing the host program can be accomplished through the use of a pause, or wait step in the code, such as the `wait_for_enter` function used in the [RTL Kernel](#) example on GitHub. The function is defined in the `src/host.cpp` code as follows:

```
void wait_for_enter(const std::string &msg) {
    std::cout << msg << std::endl;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

The `wait_for_enter` function is used in the `main` function as follows:

```
....
    std::string binaryFile = xcl::find_binary_file(device_name, "vadd");

    cl::Program::Binaries bins = xcl::import_binary_file(binaryFile);
    devices.resize(1);
    cl::Program program(context, devices, bins);
    cl::Kernel krnl_vadd(program, "krnl_vadd_rtl");

    wait_for_enter("\nPress ENTER to continue after setting up ILA
trigger...");

    //Allocate Buffer in Global Memory
    std::vector<cl::Memory> inBufVec, outBufVec;
    cl::Buffer buffer_r1(context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY,
        vector_size_bytes, source_input1.data());
    ...

    //Copy input data to device global memory
    q.enqueueMigrateMemObjects(inBufVec, 0/* 0 means from host */);

    //Set the Kernel Arguments
    ...

    //Launch the Kernel
    q.enqueueTask(krnl_vadd);
```

The use of the `wait_for_enter` function pauses the host program to give you time to set up the required ILA triggers and prepare to capture data from the kernel. After the Vivado hardware manager is set up and configured, press `Enter` to continue running the application.

- For C++ host code, add a pause after the creation of the `cl::Kernel` object, as shown in the example above.
- For C-language host code, add a pause after the `clCreateKernel()` function call:

```
// Build the program executable
//
err = clBuildProgram(program, 0, NULL, NULL, NULL, NULL);
if (err != CL_SUCCESS)
{
    size_t len;
    char buffer[2048];

    printf("Error: Failed to build program executable!\n");
    clGetProgramBuildInfo(program, device_id, CL_PROGRAM_BUILD_LOG, sizeof(buffer), buffer, &len);
    printf("%s\n", buffer);
    printf("Test failed\n");
    return EXIT_FAILURE;
}

// Create the compute kernel in the program we wish to run
//
kernel = clCreateKernel(program, "vadd", &err);
if (!kernel || err != CL_SUCCESS)
{
    printf("Error: Failed to create compute kernel!\n");
    printf("Test failed\n");
    return EXIT_FAILURE;
}

// PAUSE
wait_for_enter("\nPress ENTER to continue after setting up ILA trigger...");

// Create the input and output arrays in device memory for our calculation
//
d_a = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(int) * LENGTH, NULL, NULL);
d_b = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(int) * LENGTH, NULL, NULL);
d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, sizeof(int) * LENGTH, NULL, NULL);
if (!d_a || !d_b || !d_c)
{
    printf("Error: Failed to allocate device memory!\n");
    printf("Test failed\n");
    return EXIT_FAILURE;
}
```

Pausing the Host Application Using GDB

If you are running GDB to debug the host program at the same time as performing hardware debug on the kernels, you can also pause the host program as needed by inserting a breakpoint at the appropriate line of code. Instead of making changes to the host program to pause the application as needed, you can set a breakpoint prior to the kernel execution in the host code. When the breakpoint is reached, you can set up the debug ILA triggers in Vivado hardware manager, arm the trigger, and then resume the host program in GDB.

Debugging with ChipScope

You can use the ChipScope debugging environment and the Vivado hardware manager to help you debug your host application and kernels quickly and more effectively. These tools enable a wide range of capabilities from logic to system-level debug while your kernel is running in hardware. To achieve this, at least one of the following must be true:

- Your Vitis application project has been designed with debug cores, using the `--debug.xxx` compiler switch, as described in [Enabling Kernels for Debugging with Chipscope](#).
- The RTL kernels used in your project need to be instantiated with debug cores (as described in [Adding Debug IP to RTL Kernels](#)).

Checking the FPGA Board for Hardware Debug Support

Supporting hardware debugging requires the platform to support several IP components, most notably the Debug Bridge. Talk to your platform designer to determine if these components are included in the target platform. If an AMD platform is used, debug availability can be verified using the `platforminfo` utility to query the platform. Debug capabilities are listed under the `chipscope_debug` objects.

For example, to query the a platform for hardware debug support, the following `platforminfo` command can be used:

```
$ platforminfo --json="hardwarePlatform.extensions.chipscope_debug"
xilinx_u250_gen3x16_xdma_4_1_202210_1
{
  "debug_networks": {
    "user": {
      "bar_number": "0",
      "supports_jtag_fallback": "false",
      "name": "User Debug Network",
      "supports_microblaze_debug": "true",
      "pcie_pf": "1",
      "axi_baseaddr": "0x000001C00000",
      "is_user_visible": "true"
    },
    "mgmt": {
      "bar_number": "0",
      "supports_jtag_fallback": "true",
      "name": "Management Debug Network",
      "supports_microblaze_debug": "true",
      "pcie_pf": "0",
      "axi_baseaddr": "0x01F60000",
      "is_user_visible": "false"
    }
  }
}
```

The response shows that the target platform contains `user` and `mgmt` debug networks, supports debugging a MicroBlaze™ processor, and also supports JTAG fallback for the Management Debug Network.

Running XVC and HW Servers

The following steps are required to run the Xilinx virtual cable (XVC) and HW servers, host applications, and also trigger and arm the debug cores in the Vivado hardware manager.

1. Add debug IP to the kernel as discussed in [Enabling Kernels for Debugging with Chipscope](#).
2. Modify the host program to pause at the appropriate point as described in [Enabling ILA Triggers for Hardware Debug](#).
3. Set up the environment for hardware debug, using an automated script described in [Automated Setup for Hardware Debug](#), or manually as described in [Manual Setup for Hardware Debug](#).
4. Run the hardware debug flow using the following process:
 - a. Launch the required XVC and the `hw_server` of the Vivado hardware manager.
 - b. Run the host program and pause at the appropriate point to enable setup of the ILA triggers.
 - c. Open the Vivado hardware manager and connect to the XVC server.
 - d. Set up ILA trigger conditions for the design.
 - e. Continue execution of the host program.
 - f. Inspect kernel activity in the Vivado hardware manager.
 - g. Rerun iteratively from step b (above) as required.

Automated Setup for Hardware Debug

1. Set up your Vitis core development kit as described in [Setting Up the Vitis Environment](#).
2. Use the `debug_hw` script to launch the `xvc_pcie` and `hw_server` apps as follows:

```
debug_hw --xvc_pcie /dev/xfpga/xvc_pub.<driver_id> --hw_server
```

The `debug_hw` script returns the following:

```
launching xvc_pcie...
xvc_pcie -d /dev/xfpga/xvc_pub.<driver_id> -s TCP::10200
launching hw_server...
hw_server -sTCP::3121
```



TIP: The `/dev/xfpga/xvc_pub.<driver_id>` driver character path is defined on your machine, and can be found by examining the `/dev` folder.

3. Modify the host code to include a pause statement *after* the kernel has been created/downloaded and *before* the kernel execution is started, as described in [Enabling ILA Triggers for Hardware Debug](#).
4. Run your modified host program.

5. Launch Vivado Design Suite using the `debug_hw` script:

```
debug_hw --vivado --host <host_name> --ltx_file ./_x/link/vivado/vpl/prj/
prj.runs/impl_1/debug_nets.ltx
```



TIP: The `<host_name>` is the name of your system.

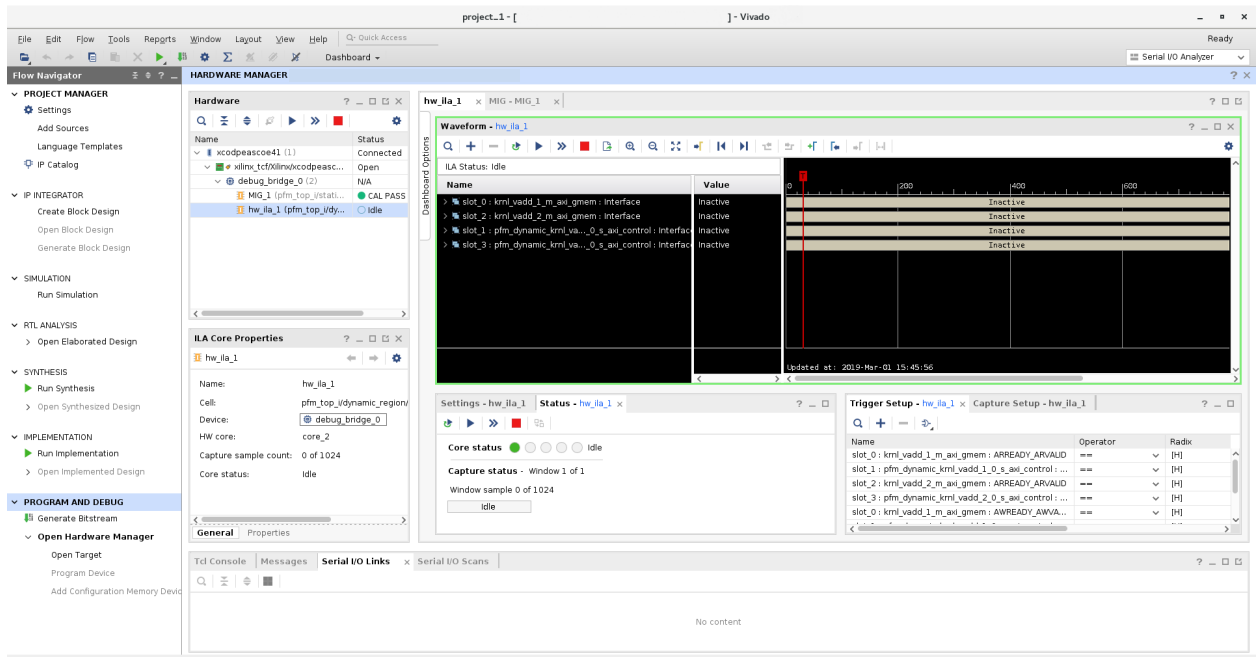
As an example, the command window displays the following results:

```
launching vivado... ['vivado', '-source', 'vitis_hw_debug.tcl', '-
tclargs',
'/tmp/project_1/project_1.xpr', 'workspace/vadd_test/System/
pfm_top-wrapper.ltx',
'host_name', '10200', '3121']

***** Vivado v2019.2 (64-bit)
**** SW Build 2245749 on Date Time
**** IP Build 2245576 on Date Time
** Copyright 1986-2019 Xilinx, Inc. All Rights Reserved.

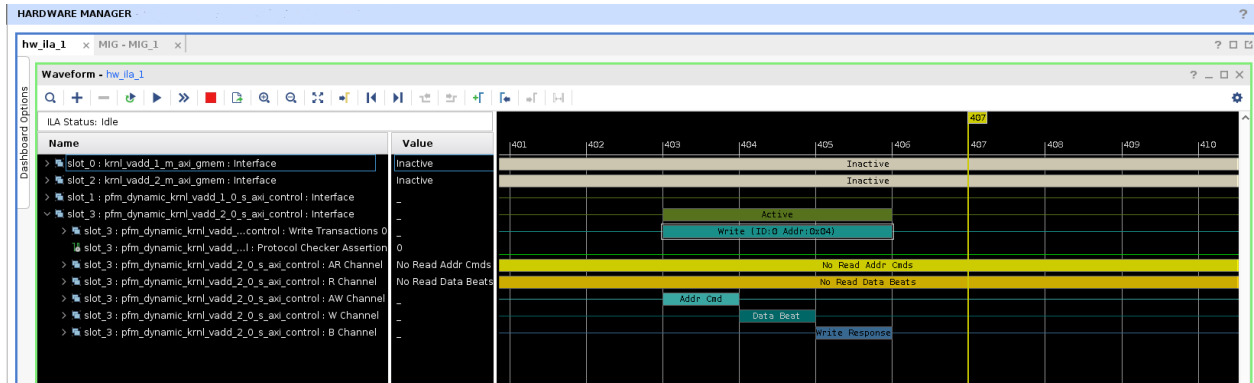
start_gui
```

6. In Vivado Design Suite, run the ILA trigger.



7. Press **Enter** to continue running the host program.

8. In the Vivado hardware manager, see the interface transactions on the kernel compute unit slave control interface in the Waveform view.



Manual Setup for Hardware Debug



TIP: The following steps can be used when setting up Nimbix and other cloud platforms.

There are a few steps required to start the debug servers prior to debugging the design in the Vivado hardware manager.

1. Set up your Vitis core development kit as described in [Setting Up the Vitis Environment](#).
2. Launch the `xvc_pcie` server. The file name passed to `xvc_pcie` must match the character driver file installed with the kernel device driver, where `<driver_id>` can be found by examining the `/dev` folder.

```
>xvc_pcie -d /dev/xfpga/xvc_pub.<device_id>
```



TIP: The `xvc_pcie` server has many useful command line options. You can issue `xvc_pcie -help` to obtain the full list of available options.

3. Start the `hw_server` on port 3121, and connect to the XVC server on port 10201 using the following command:

```
>hw_server -e "set auto-open-servers xilinx-xvc:localhost:10201" -e "set always-open-jtag 1"
```

4. Launch Vivado Design Suite and open the hardware manager:

```
vivado
```

Starting Debug Servers on an Amazon F1 Instance

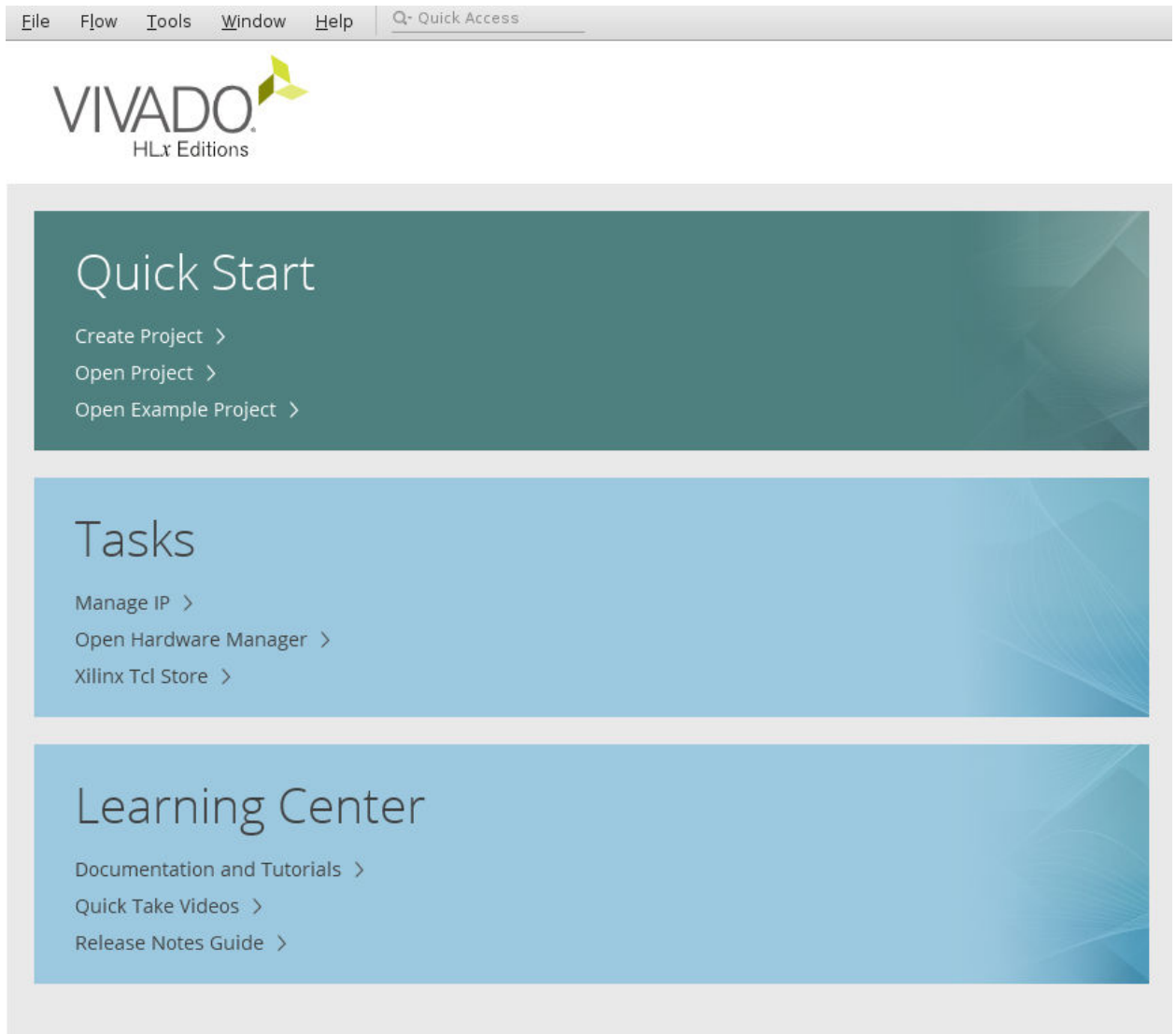
Instructions to start the debug servers on an Amazon F1 instance can be found here: https://github.com/aws/aws-fpga/blob/master/hdk/docs/Virtual_JTAG_XVC.md.

Debugging Designs Using Vivado Hardware Manager

Traditionally, a physical JTAG connection is used to perform hardware debug for AMD devices with the Vivado hardware manager. The Vitis unified software platforms also makes use of the Xilinx virtual cable (XVC) for hardware debugging on remote accelerator cards. To take advantage of this capability, the Vitis debugger uses the XVC server, an implementation of the XVC protocol that allows the Vivado hardware manager to connect to a local or remote target device for debug, using the standard AMD debug cores like the ILA or the VIO IP.

The Vivado hardware manager, from the Vivado Design Suite or Vivado debug feature, can be running on the target instance or it can be running remotely on a different host. The TCP port on which the XVC server is listening must be accessible to the host running Vivado hardware manager. To connect the Vivado hardware manager to XVC server on the target, the following steps should be followed on the machine hosting the Vivado tools:

1. Launch the Vivado debug feature, or the full Vivado Design Suite.
2. Select **Open Hardware Manager** from the Tasks menu, as shown in the following figure.



3. Connect to the Vivado tools `hw_server`, specifying a local or remote connection, and the **Host name** and **Port**, as shown below.

Open New Hardware Target

Hardware Server Settings

Select local or remote hardware server, then configure the host name and port settings. Use Local server if the target is attached to the local machine; otherwise, use Remote server.

Connect to:

Local server (target is on local machine)

Click Next to launch and/or connect to the hw_server (port 3121) application on the local machine.

?

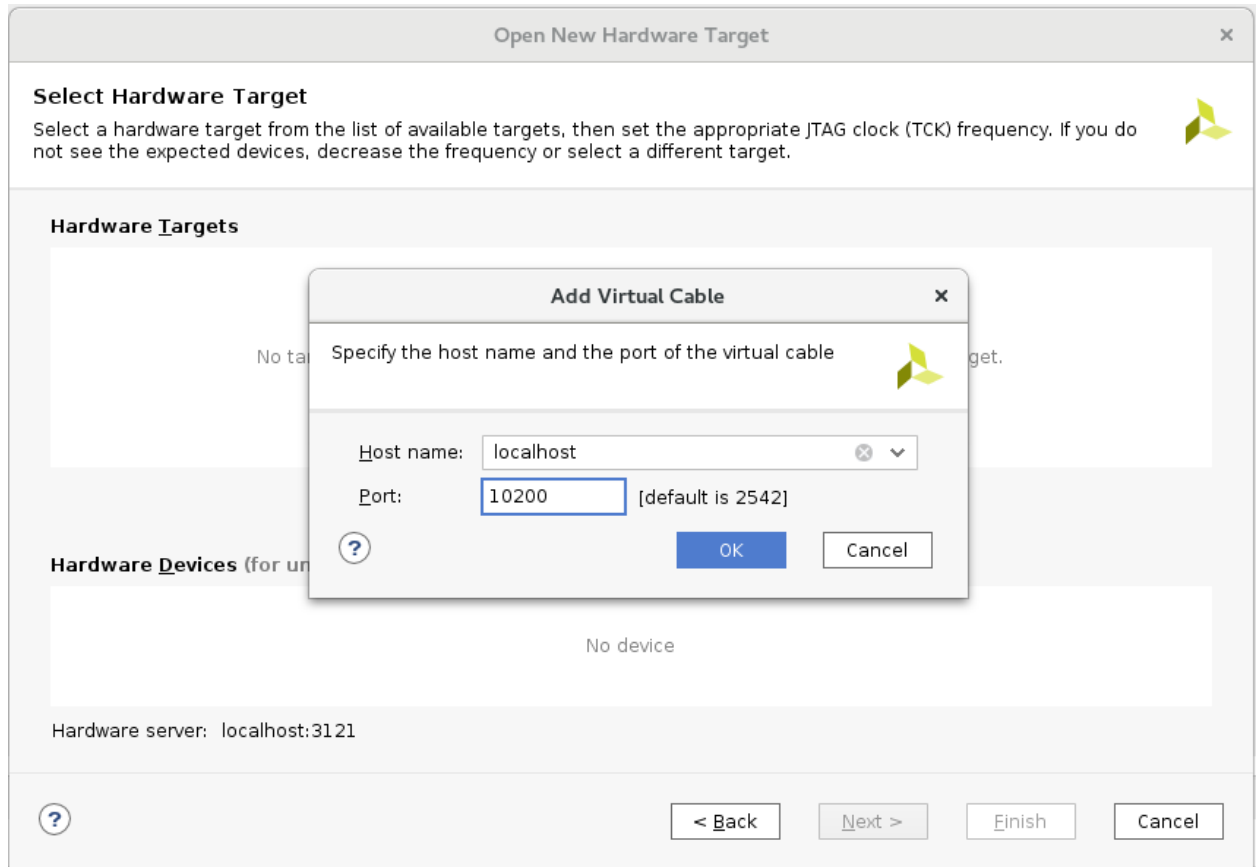
< Back

Next >

Finish

Cancel

4. Connect to the target instance Virtual JTAG XVC server.



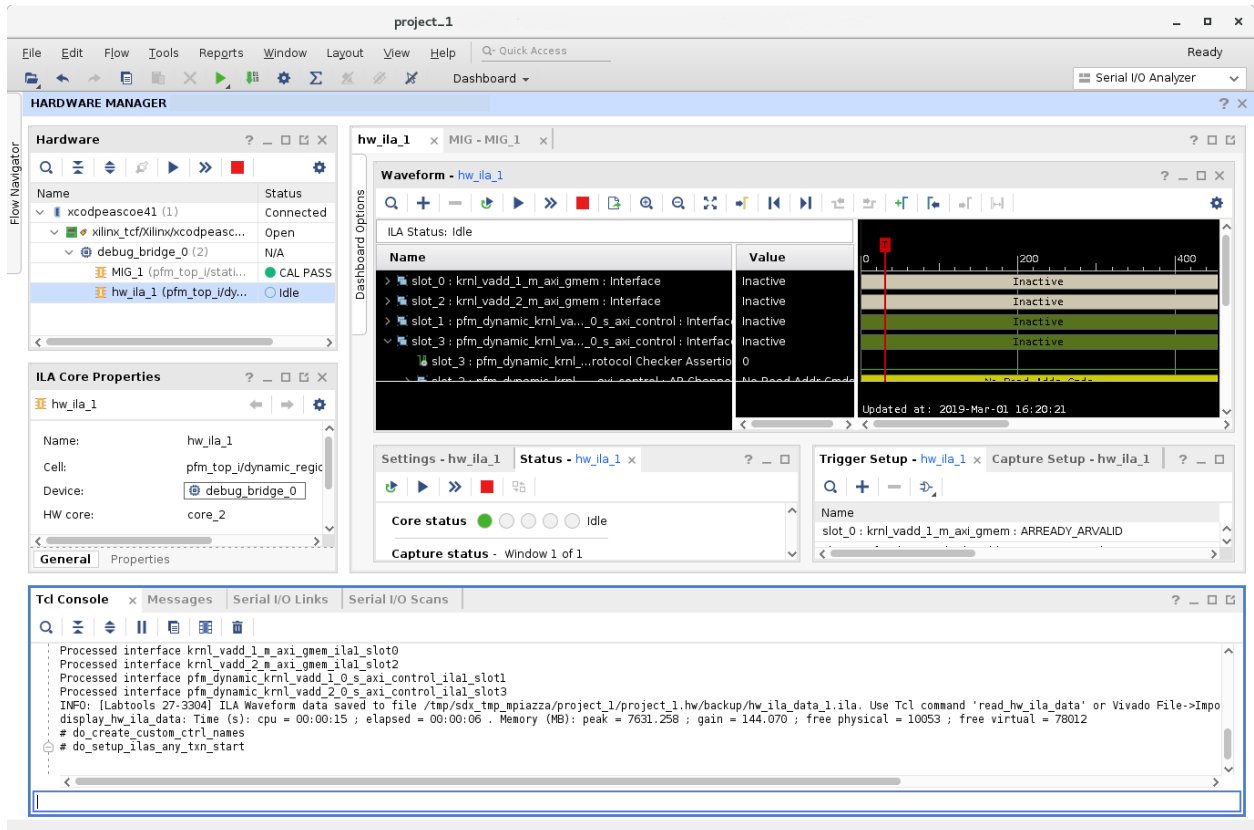
5. Select the `debug_bridge` instance from the Hardware window in the Vivado hardware manager.

Specify the probes file (.ltx) for your design adding it to the **Probes → File** entry in the Hardware Device Properties window. Adding the probes file refreshes the hardware device, and Hardware window should now show the debug cores in your design.



TIP: If the kernel has debug cores as specified in [Enabling Kernels for Debugging with Chipscope](#), the probes file (.ltx) is written out during the implementation of the kernel by the Vivado tool.

6. The Vivado hardware manager can now be used to debug the kernels running on the Vitis software platform. Arm the ILA cores in your kernels and run your host application.



TIP: Refer to the Vivado Design Suite User Guide: Programming and Debugging ([UG908](#)) for more information on working with the Vivado hardware manager to debug the design.

JTAG Fallback for Private Debug Network

Hardware debug for the Alveo Data Center accelerator cards typically uses the XVC-over-PCIe connection due to the inaccessibility of the physical card, and the JTAG connector on the card. While XVC-over-PCIe allows you to remotely debug your application running on the target platform, certain conditions such as AXI interconnect system hangs can prevent you from accessing the hardware debug functionality that depends on these PCIe/AXI features. Being able to debug these kinds of conditions is especially important for platform designers.

The JTAG Fallback feature is designed to provide access to debug networks that were previously only accessible through XVC-over-PCIe. The JTAG Fallback feature can be enabled without having to change the XVC-over-PCIe-based debug network in the platform design.

On the host side, when the Vivado hardware manager user connects through the `hw_server` to a JTAG cable that is connected to the physical JTAG pins of the accelerator card, or device under test (DUT), the `hw_server` disables the XVC-over-PCIe pathway to the hardware. This lets you use the XVC-over-PCIe cable as your primary debug path, but enable debug over the JTAG cable directly when it is required in certain situations. When you disconnect from the JTAG cable, the `hw_server` re-enables the XVC-over-PCIe pathway to the hardware.

JTAG Fallback Steps

Here are the steps required to enable JTAG Fallback:

1. Enable the JTAG Fallback feature of the Debug Bridge (AXI-to-BSCAN mode) master of the debug network to which you want to provide JTAG access. This step enables a BSCAN slave interface on this Debug Bridge instance.
2. Instantiate another Debug Bridge (BSCAN Primitive mode) in the static logic partition of the platform design.
3. Connect the BSCAN master port of the Debug Bridge (BSCAN Primitive mode) from step 2 to the BSCAN slave interface of the Debug Bridge (AXI-to-BSCAN mode) from step 1.

Utilities for Hardware Debugging

In some cases, the normal Vitis IDE and command line debug features are limited in their ability to isolate an issue. This is especially true when the software or hardware appears not to make any progress (hangs). These kinds of system issues are best analyzed with the help of the utilities mentioned in this section.

Using the Linux dmesg Utility

Well-designed kernels and modules report issues through the kernel ring buffer. This is also true for Vitis technology modules that allow you to debug the interaction with the accelerator board on the lowest Linux level.

The `dmesg` utility is a Linux tool that lets you read the kernel ring buffer. The kernel ring buffer holds kernel information messages in a circular buffer. A circular buffer of fixed size is used to limit the resource requirements by overwriting the oldest entry with the next incoming message.

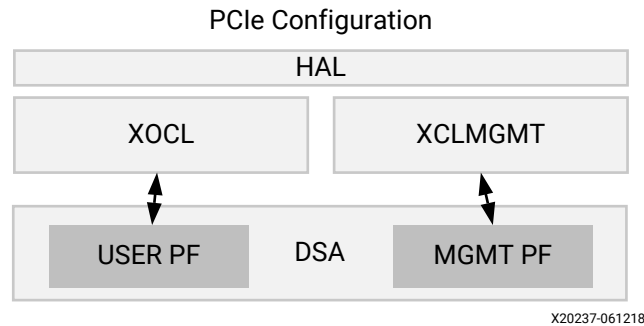


TIP: In most cases, it is sufficient to work with the less verbose `xbutil` feature to localize an issue. Refer to [Using the xbutil Utility](#) for more information on using this tool for debug.

In the Vitis technology, the `xocl` module and `xclmgmt` driver modules write informational messages to the ring buffer. Thus, for an application hang, crash, or any unexpected behavior (like being unable to program the bitstream, etc.), the `dmesg` tool should be used to check the ring buffer.

The following image shows the layers of the software platform associated with the target platform.

Figure 61: Software Platform Layers



To review messages from the Linux tool, you should first clear the ring buffer:

```
sudo dmesg -c
```

This flushes all messages from the ring buffer and makes it easier to spot messages from the `xocl` and `xclmgmt`. After that, start your application and run `dmesg` in another terminal.

```
sudo dmesg
```

The `dmesg` utility prints a record shown in the following example:

Figure 62: dmesg Utility Example

```
[ 9902.316729] xclmgmt: AXI Firewall 2 has tripped. Status: 0x00000
[ 9902.316874] xclmgmt: xclmgmt_killall_processes
[ 9902.317007] xclmgmt: Killing pid: 19891
[ 9902.317501] xocl:xdma_xfer_submit: xfer 0xffff8801c1be1018, 268435456, s 0x1 timed out, ep 0x10000000.
[ 9902.317911] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e0000) = 0x1fc00006 (id).
[ 9902.318410] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e0040) = 0x00000001 (status).
[ 9902.318895] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e0004) = 0x00f83e1f (control)
[ 9902.319370] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e4080) = 0xa7a30000 (first_desc_lo)
[ 9902.319848] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e4084) = 0x00000000 (first_desc_hi)
[ 9902.320336] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e4088) = 0x0000000f (first_desc_adjacent).
[ 9902.320802] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e0048) = 0x00000000 (completed_desc_count).
[ 9902.321279] xocl:engine_reg_dump: 0-H2C0-MM: ioread32(0xffffc900064e0090) = 0x00f83e1e (interrupt_enable_mask)
[ 9902.321759] xocl:engine_status_dump: SG engine 0-H2C0-MM status: 0x00000001: BUSY
[ 9902.322233] xocl:transfer_abort: abort transfer 0xffff8801c1be1018, desc 240, engine desc queued 0.
[ 9902.322752] [drm:xdma_migrate_bo [xocl]] *ERROR* DMA failed to device addr 0x0, tid 19897, channel 0
[ 9902.323232] [drm:xdma_migrate_bo [xocl]] *ERROR* Dumping SG Page Table
```

In the example shown above, the AXI Firewall 2 has tripped, which is better examined using the `xbutil` utility.

Using the xbutil Utility

The Xilinx board utility (`xbutil`) is a powerful standalone command line utility that can be used to debug lower level hardware/software interaction issues. A full description of this utility can be found in [xbutil Utility](#).

With respect to debugging, the following `xbutil` options are of special interest:

- **examine:** Provides an overall status of a card including information on the kernels in card memory.
- **program:** Downloads a binary (`xclbin`) to the programmable region of the AMD device.
- **`xbutil examine -r debug-ip-status -d <BDF>`:** Extracts the status of the Performance Monitors (`aim` and `asm`) and the Lightweight AXI Protocol Checkers (`lapc`).

Techniques for Debugging Application Hangs

This section discusses debugging issues related to the interaction of the host code and the accelerated kernels. Problems with these interactions manifest as issues such as machine hangs or application hangs. Although the GDB debug environment might help with isolating the errors in some cases (`xprint`), such as hangs associated with specific kernels, these issues are best debugged using the `dmesg` and `xbutil` commands as shown here.

If the process of hardware debugging does not resolve the problem, it is necessary to perform hardware debugging using the ChipScope feature.

AXI Firewall Trips

The AXI firewall should prevent host hangs. This is why the AXI Protocol Firewall IP is included in all production Vitis platforms. When the firewall trips, one of the first checks to perform is confirming if the host code and kernels are set up to use the same memory banks. The following steps detail how to perform this check.

1. Use `xbutil` to program the FPGA:

```
xbutil program -p <xclbin>
```



TIP: Refer to [xbutil Utility](#) for more information on `xbutil`.

2. Run the `xbutil examine` option to check memory topology:

```
xbutil examine -r memory -d <bdf>
```

In the following example, there are no kernels associated with memory banks:

```
#####
```

Mem Topology				Device Memory Usage		
Tag	Type	Temp	Size	Mem Usage	BO nums	
[0] bank0	MEM_DDR4	Not Supp	16 GB	0 Byte	0	
[1] bank1	MEM_DDR4	Not Supp	16 GB	0 Byte	0	
[2] bank2	**UNUSED**	Not Supp	16 GB	0 Byte	0	
[3] bank3	**UNUSED**	Not Supp	16 GB	0 Byte	0	
[4] PLRAM[0]	MEM_DRAM	Not Supp	128 KB	0 Byte	0	
[5] PLRAM[1]	**UNUSED**	Not Supp	128 KB	0 Byte	0	
[6] PLRAM[2]	**UNUSED**	Not Supp	128 KB	0 Byte	0	

```

Total DMA Transfer Metrics:
Chan[0].h2c: 416 MB
Chan[0].c2h: 328 MB
Chan[1].h2c: 96 MB
Chan[1].c2h: 184 MB

```

3. If the host code expects any DDR banks/PLRAMs to be used, this report should indicate an issue. In this case, it is necessary to check kernel and host code expectations. If the host code is using the AMD OpenCL extensions, it is necessary to check which DDR banks should be used by the kernel. These should match the `connectivity.sp` options specified as discussed in [Mapping Kernel Ports to Memory](#).

Kernel Hangs Due to AXI Violations

It is possible for the kernels to hang due to bad AXI transactions between the kernels and the memory controller. To debug these issues, it is required to instrument the kernels.

1. The Vitis core development kit provides two options for instrumentation to be applied during `v++` linking (`--link`). Both of these options add hardware to your implementation, and based on resource usage it can be necessary to limit instrumentation.
 - a. Add Lightweight AXI Protocol Checkers (`lapc`). These protocol checkers are added using the `--debug.protocol` option, as explained in [--debug Options](#). The following syntax is used:

```
--debug.protocol <compute_unit_name>:<interface_name>
```

In general, the `<interface_name>` is optional. If not specified, all ports on the CU are expected to be analyzed. The `--debug.protocol` option is used to define the protocol checkers to be inserted. This option can accept a special keyword, `all`, for `<compute_unit_name>` and/or `<interface_name>`.

Note: Multiple `--debug.xxx` options can be specified in a single command line, or configuration file.

- b. Adding Performance Monitors (`am`, `aim`, `asm`) enables the listing of detailed communication statistics (counters). Although this is most useful for performance analysis, it provides insight during debugging on pending port activities. The Performance Monitors are added using the `--profile` option as described in [--profile Options](#). The basic syntax for the `--profile` option is:

```
--profile.data <krnl_name>|all:<cu_name>|all:<intrfc_name>|
all:<counters>|all
```

Three fields are required to determine the specific interface to attach the performance monitor to. However, if resource consumption is not an issue, the keyword `all` lets you apply the monitoring to all existing kernels, compute units, and interfaces with a single option. Otherwise, you can specify the `kernel_name`, `cu_name`, and `interface_name` explicitly to limit instrumentation.

The last option, `<counters>|all`, allows you to restrict the information gathering to `counters` for large designs, while `all` (default) includes the collection of actual trace information.

Note: Multiple `--profile` options can be specified in a single command line, or configuration file.

```
[profile]
dataernel1:cu1:m_axi_gmem0
dataernel1:cu1:m_axi_gmem1
dataernel2:cu2:m_axi_gmem
```

- When the application is rebuilt, rerun the host application using the `xclbin` with the added AIM IP and LAPC IP.
- When the application hangs, you can use `xbutil examine` to check for any errors or anomalies.
- Check the AIM output:
 - Run the following command a couple of times to check if any counters are moving. If they are moving then the kernels are active.

```
xbutil examine -d <bdf> -r debug-ip-status -e aim
```



TIP: Testing AIM output is also supported through GDB debugging using the command extension `xstatus aim`.

- If the counters are stagnant, the outstanding counts greater than zero might mean some AXI transactions are hung.
- Check the LAPC output:
 - Run the following command to check if there are any AXI violations.

```
xbutil examine -d <bdf> -r debug-ip-status -e lapc
```



TIP: Testing LAPC output is also supported through GDB debugging using the command extension `xstatus lapc`.

- If there are any AXI violations, it implies that there are issues in the kernel implementation.

Host Application Hangs When Accessing Memory

Application hangs can also be caused by incomplete DMA transfers initiated from the host code. This does not necessarily mean that the host code is wrong; it might also be that the kernels have issued illegal transactions and locked up the AXI.

1. If the platform has an AXI firewall, such as in the Vitis target platforms, it is likely to trip. The driver issues a `SIGBUS` error, kills the application, and resets the device. You can check this by running the following command:

```
xbutil examine -d <bdf> -r firewall
```

The following figure shows such an error in the firewall status:

```
Firewall Last Error Status:
0:          0x0      (GOOD)
1:          0x0      (GOOD)
2:          0x80000 (RECS_WRITE_TO_BVALID_MAX_WAIT).
                Error occurred on Tue 2017-12-19 11:39:13 PST

Xclbin ID:      0x5a39da87
```



TIP: If the firewall has not tripped, the Linux tool, `dmesg`, can provide additional insight.

2. When you know that the firewall has tripped, it is important to determine the cause of the DMA timeout. The issue could be an illegal DMA transfer, or kernel misbehavior. However, a side effect of the AXI firewall tripping is that the health check functionality in the driver resets the board after killing the application; any information on the device that might help with debugging the root cause is lost. To debug this issue, disable the health check thread in the `xclmgmt` kernel module to capture the error. This uses common Unix kernel tools in the following sequence:
 - a. `sudo modinfo xclmgmt`: This command lists the current configuration of the module and indicates if the `health_check` parameter is ON or OFF. It also returns the path to the `xclmgmt` module.
 - b. `sudo rmmod xclmgmt`: This removes and disables the `xclmgmt` kernel module.
 - c. `sudo insmod <path to module>/xclmgmt.ko health_check=0`: This re-installs the `xclmgmt` kernel module with the health check disabled.



TIP: The path to this module is reported in the output of the call to `modinfo`.

3. With the health check disabled, rerun the application. You can use the kernel instrumentation to isolate this issue as previously described.

Typical Errors Leading to Application Hangs

The user errors that typically create application hangs are listed below:

- Read-before-write in 5.0+ target platforms causes a Memory Interface Generator error correction code (MIG ECC) error. This is typically a user error. For example, this error might occur when a kernel is expected to write 4 KB of data in DDR, but it produces only 1 KB of data, and then try to transfer the full 4 KB of data to the host. It can also happen if you supply a 1 KB buffer to a kernel, but the kernel tries to read 4 KB of data.
- An ECC read-before-write error also occurs if no data has been written to a memory location as the last bitstream download which results in MIG initialization, but a read request is made for that same memory location. ECC errors stall the affected MIG because kernels are usually not able to handle this error. This can manifest in two different ways:
 1. The CU might hang or stall because it cannot handle this error while reading or writing to or from the affected MIG. The `xbutil` query shows that the CU is stuck in a `BUSY` state and is not making progress.
 2. The AXI Firewall might trip if a PCIe® DMA request is made to the affected MIG, because the DMA engine is unable to complete the request. AXI Firewall trips result in the Linux kernel driver killing all processes which have opened the device node with the `SIGBUS` signal. The `xbutil` query shows if an AXI Firewall has indeed tripped and includes a timestamp.

If the above hang does not occur, the host code might not read back the correct data. This incorrect data is typically 0s and is located in the last part of the data. It is important to review the host code carefully. One common example is compression, where the size of the compressed data is not known up front, and an application might try to migrate more data to the host than was produced by the kernel.

Defensive Programming

The Vitis compiler is capable of creating very efficient implementations. In some cases, however, implementation issues can occur. One such case is if a write request is emitted before there is enough data available in the process to complete the write transaction. This can cause deadlock conditions when multiple concurrent kernels are affected by this issue and the write request of a kernel depends on the input read being completed.

To avoid these situations, a conservative mode is available on the adapter. In principle, it delays the write request until it has all of the data necessary to complete the write. This mode is enabled during compilation by applying the following `--advanced.param` option to the `v++` compiler:

```
--advanced.param:compiler.axiDeadLockFree=yes
```

Because enabling this mode can impact performance, you might prefer to use this as a defensive programming technique where this option is inserted during development and testing and then removed during optimization. You might also want to add this option when the accelerator hangs repeatedly.

Hardware Debug for Embedded Processors

For hardware builds the setup involves the following steps:

1. Copy the contents of the `<project>/Hardware/sd_card/sd_card` folder to a physical SD card. This creates a bootable medium for your target platform.
2. Insert the SD card into the card reader of your embedded processor platform.
3. Change the boot-mode settings of the platform to SD boot mode, and power up the board.
4. After the device is booted, enter the `mount` command at the command prompt to get a list of mount points. As shown in the following figure, the `mount` command displays mounting information for the system.



TIP: Be sure to capture the proper path for the `cd` command in the next step, and subsequent commands, based on the results of the `mount` command.

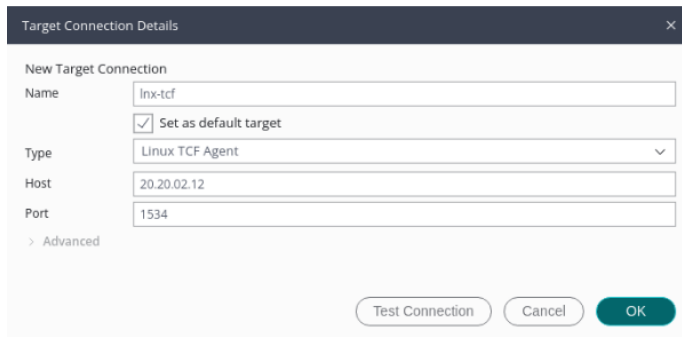
```
systest - Konsole
File Edit View Bookmarks Settings Help
root@versal-rootfs-common-2020_1:~# mount
/dev/mmcblk0p2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,relatime,size=893208k,nr_inodes=223302,mode=755)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
debugfs on /sys/kernel/debug type debugfs (rw,relatime)
configfs on /sys/kernel/config type configfs (rw,relatime)
tmpfs on /run type tmpfs (rw,nosuid,nodev,mode=755)
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /run/media/mmcblk0p1 type vfat (rw,relatime,gid=6,fmask=0007,dm
ask=0007,allow_utime=0020,codepage=437,iocharset=iso8859-1,shortname=mixed,errors
=remount-ro)
devpts on /dev/pts type devpts (rw,relatime,gid=5,mode=620,ptmxmode=000)
root@versal-rootfs-common-2020_1:~#
```

5. Execute the following commands, for example:

```
cd /run/media/mmcblk0p1
source init.sh
cat /etc/xocl.txt
```

The `cat` command will display the platform name `xilinx_vck190_base_202310_1` to let you confirm it is the same as your specified platform and that your setup is correct.

6. Run `ifconfig` to get the IP address of the target card. You will use the IP address to set up a TCF agent connection in the Vitis unified IDE to connect to the assigned IP address of the embedded processor platform.
7. Create a target connection to the remote accelerator card. Use the **Vitis → Target Connections** menu command to open the Target Connections dialog box.
8. Right-click the **Linux TCF Agent** and select the **New Target** command to open the New Target Connection dialog box.
9. Specify the **Target Name**, enable the **Set as default target** check box, and specify the **Host IP** address of the accelerator card that you obtained in an earlier step.



Target Connection Details

New Target Connection

Name:

☒ Set as default target

Type:

Host:

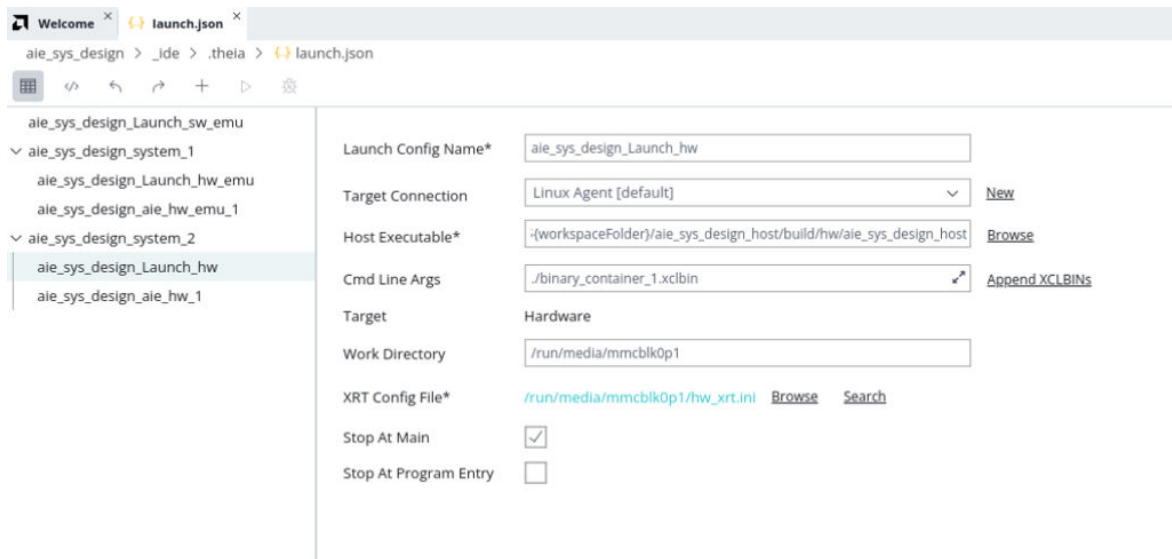
Port:

> Advanced

Test Connection Cancel OK

10. Click **OK** to close and continue.

11. In the Flow Navigator view, click the Open Settings command to open the Launch Configuration editor to create a new launch configuration for the hardware design



Welcome x launch.json x

ale_sys_design > _ide > .theia > launch.json

ale_sys_design_Launch_sw_emu

ale_sys_design_system_1

ale_sys_design_Launch_hw_emu

ale_sys_design_aie_hw_emu_1

ale_sys_design_system_2

ale_sys_design_Launch_hw

ale_sys_design_aie_hw_1

Launch Config Name*

Target Connection [New](#)

Host Executable* [Browse](#)

Cmd Line Args [Append XCLBin](#)

Target Hardware

Work Directory

XRT Config File* [Browse](#) [Search](#)

Stop At Main ☒

Stop At Program Entry ☐

Set the following fields on the **Main** tab of the dialog box:

- **Name:** Specifies a name for your Hardware debug configuration.
- **Target Connection:** Select a Linux TCF agent as previously configured
- **Host Executable:** Specify the location of the software application to drive the hardware
- **Cmd Line Args:** Specify any needed command line arguments for the host application, such as the `.xclbin` file to load
- **Work Directory:** Specifies the location where the system is run and output files will be written
- **XRT Config File:** Specifies the `xrt.ini` file to add to the hardware run as described in [Enabling Profiling in Your Application](#)
- **Stop at Main:** Puts a breakpoint in the host application to stop at the entry to the `main()` function to enable debug operations

- **Stop At Program Entry:** Places a breakpoint at the entry to the hardware program to enable debug operations

12. Select **Debug** from the Flow Navigator to open the Debug view as described in [Debug View](#).

This opens the Debug view in the Vitis unified IDE, and connects to the PS application on your hardware platform. The application automatically breaks at the `main()` function to let you set up and configure the debug environment.

Example of Command Line Debugging

To help you get familiar with debugging using the command line flow, this example walks you through building and debugging the [hello_world example](#) available from the Xilinx GitHub.

1. In a terminal, set up your environment as described in [Setting Up the Vitis Environment](#).
2. If you have not already done it, clone the [Vitis Examples](#) GitHub repository to acquire all of the Vitis examples:

```
git clone https://github.com/Xilinx/Vitis_Accel_Examples.git
```

This creates a `Vitis_Examples` directory which includes the IDCT example.

3. CD to the IDCT example directory:

```
cd Vitis_Accel_Examples/hello_world/
```

The host code is fully contained in `src/host.cpp` and the kernel code is part of `src/vadd.cpp`.

4. Build the kernel software for software emulation as discussed in [Building the Device Binary](#).
 - a. Compile the kernel object file for debugging using the `v++` compiler, where `-g` indicates that the code is compiled for debugging:

```
v++ -t sw_emu --platform <DEVICE> -g -c -k vadd \
-o vadd.xo ./src/vadd.cpp
```

- b. Link the kernel object file, also specifying `-g`:

```
v++ -g -l -t sw_emu --platform <DEVICE> -o vadd.xclbin vadd.xo
```

5. Compile and link the host code for debugging using the GNU compiler chain, `g++` as described in [Building the Software Application](#):

Note: For embedded processor target platforms, use the GNU Arm cross-compiler as described in [Compiling and Linking the Host Application](#).

- a. Compile host code C++ files for debugging using the `-g` option:

```
g++ -c -I${XILINX_XRT}/include -g -o host.o src/host.cpp
```


- b. Link the object files for debugging using `-g`:

```
g++ -g -lOpenCL -lpthread -lrt -lstdc++ -L${XILINX_XRT}/lib/ -o host
host.o
```

6. As described in [emconfigutil Utility](#), prepare the emulation environment using the following command:

```
emconfigutil --platform <device>
```

The actual emulation mode (`sw_emu` or `hw_emu`) then needs to be set through the `XCL_EMULATION_MODE` environment variable. In C-shell this would be as follows:

```
setenv XCL_EMULATION_MODE sw_emu
```

7. As described in [xrt.ini File](#), you must setup the runtime for debug. In the same directory as the compiled host application, create an `xrt.ini` file with the following content:

```
[Debug]
app-debug=true
```

8. Run GDB on the host and kernel code. The following steps guide you through the command line debug process which requires three separate command terminals, setup as described in [Setting Up the Vitis Environment](#).

- a. In the first terminal, start the XRT debug server, which handles the transactions between the host and kernel code:

```
${XILINX_VITIS}/bin/xrt_server --sdx-url
```

- b. In a second terminal, set the emulation mode:

```
setenv XCL_EMULATION_MODE sw_emu
```

Run GDB by executing the following:

```
xgdb --args host vadd.xclbin
```

Enter the following on the `gdb` prompt:

```
run
```

- c. In the third terminal, attach the software emulation model to GDB to allow stepping through the design. Start up another `xgdb`:

```
xgdb
```

- For debugging in software emulation:
 - Type the following on the `gdb` prompt:

```
file <XILINX_VITIS>/data/emulation/unified/cpu_em/generic_pcie/
model/genericpciemodel
```

Note: Because GDB does not expand the environment variable, you must specify the path to the Vitis software platform installation as represented by `<XILINX_VITIS>`

- Connect to the kernel process:

```
target remote :NUM
```

Where `NUM` is the number returned by the `xrt_server` as the GDB listener port.

At this point, debugging the host and kernel code can be done as usual with GDB, with the host code and the kernel code running in two different GDB sessions. This is common when dealing with different processes.



IMPORTANT! *Be aware that the application might hit a breakpoint in one process before the next breakpoint in the other process is hit. In these cases, the debugging session in one terminal appears to hang, while the second terminal is waiting for input.*

Vitis Commands and Utilities

The AMD Vitis™ IDE uses the `v++` command, and the `vitis-run` command to compile and run the various components of the design. This section describes these commands and their options. There are additional commands that can be used when building, running, or debugging components or applications, which are also documented here.

The reference materials contained here include the following:

- [v++ Command](#): Provides a description of the compiler options (`--compile`) including the commands for the AI Engine and HLS compiler modes; the linking options (`--link`) for creating the device binary, the packaging options (`--package`) for building the boot files and generating an SD card for the system, and a discussion of the `--config` command and configuration files.
- [HLS Pragmas](#) provides a description of pragmas for use in HLS components.
- Various AMD utilities are provided for the Vitis tools and Xilinx Runtime (XRT) to provide detailed information about the platform resources, including SLR and memory resource availability, to help you construct the `v++` command line, and manage the build and run process.
 - [emconfigutil Utility](#)
 - [kernelinfo Utility](#)
 - [launch_emulator Utility](#)
 - [manage_ipcache Utility](#)
 - [package_xo Command](#)
 - [platforminfo Utility](#)
 - [vitis-run Command](#)
 - [xbutil Utility](#)
 - [xbmgmt Utility](#)
 - [xclbinutil Utility](#)



TIP: The Xilinx Runtime (XRT) Architecture reference material is available on the [Xilinx Runtime GitHub repository](#).

- The `xrt.ini` file is used to initialize XRT to produce reports, debug, and profiling data as it transacts business between the host and kernels. This file is used when the application is run, for emulation or hardware builds, and must be created manually when the build process is run from the command line.

Launching the Vitis Unified IDE

To launch the Vitis Unified IDE, do the following:

1. Use the following command to load the Vitis software platform environment.

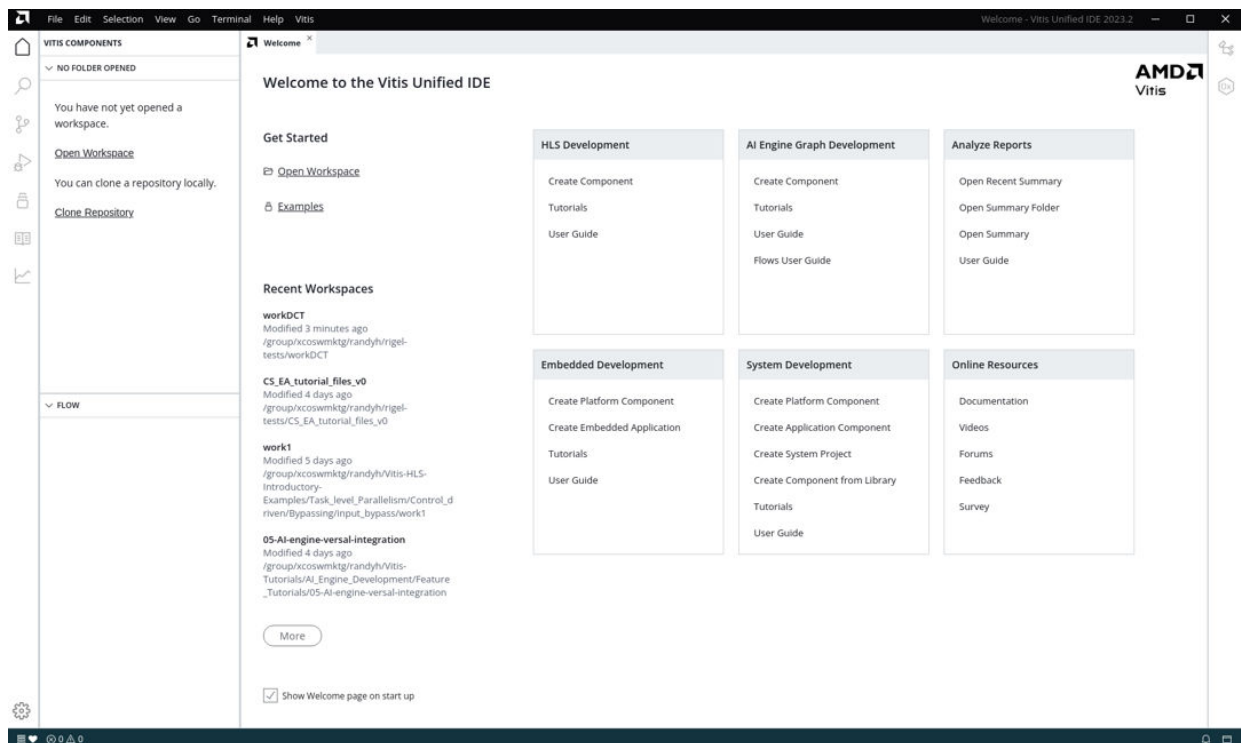
```
source <Vitis_Installation_Directory>/settings64.sh
```

2. Use the following command to launch the next generation Vitis IDE.

```
vitis -w <workspace>
```

where `<workspace>` indicates a folder to hold all of the contents of your design project. The workspace is used to group together the source and data files that make up a design, or multiple designs, and stores the configuration of the tool for that workspace.

Figure 63: Welcome Screen



For other supported launch modes, see [Launch Options](#).

Note: Before launching the Vitis unified IDE, you can set up other environmental settings to ensure that the tool can pick up these settings. For example, you can set up Xilinx Runtime (XRT) for building and running data center acceleration applications:

```
source <XRT_Install_Path>/setup.sh
```

You can set up the platform repository path with the following example:

```
export PLATFORM_REPO_PATHS=<platform_path>
```

Launch Options

The Vitis Unified IDE supports the following modes:

- **Classic Mode:** Use to open the legacy Vitis IDE.

```
vitis --classic
```

- **Workspace Mode:** Open the Vitis unified IDE with the specified workspace.

```
vitis -w <workspace>
```

- **Analysis Mode:** Analysis mode launches the tool directly into the [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) letting you review the summary reports generated during the build, run, and debug processes.

```
vitis -a
```

- **Interactive Mode:** Interactive mode lets you enter commands through the command-line interface, outside of the GUI.

```
vitis -i
```

Note: Type `help()` from the interactive command prompt to explore the available command modules.

- **Batch Mode:** Batch mode executes the specified script and exits.

```
vitis -s <script>.py
```

- **Jupyter Notebook Mode:** Launches Jupyter Notebook server with the Vitis environment and the front end UI in your default web browser.

```
vitis -j
```

You can use `vitis_server` command line interface in this environment as described in [Vitis Interactive Python Shell](#).

You can use `-h` to print the supported options of `vitis -new`.

```
vitis -h

Syntax: vitis [--classic | -a | -w | -i | -s | -h | -v]

Options:
  -classic/--classic
    Launch the classic Vitis IDE.
  -a/--analyze [<summary file | folder | waveform file: *.*[wdb|wcfg]>]
    Open the summary file in the Analysis view.
    Opening a folder opens the summary files found in the folder.
    Open the waveform file in a waveform view tab.
    If no file or folder is specified, opens the Analysis view.
  -w/--workspace <workspace_location>
    Launches Vitis IDE with the given workspace location.
  -i/--interactive
    Launches Vitis python interactive shell.
  -s/--source <python_script>
    Runs the given python script.
  -j/--jupyter
    Launches Vitis Jupyter Web UI.
  -h/--help
    Display help message.
  -v/--version
    Display Vitis version.
```

v++ Command

This section describes the AMD Vitis™ compiler command, `v++`, and the various options it supports for building the device binary. `v++` is a standalone command line utility with three command modes:

- `--compile (-c)`: For launching the HLS or AI Engine compiler modes of the `v++` tool to compile C/C++ code into AI Engine kernels and graph applications into a `libadf.a` file as described in [Compiling AI Engine Graph Applications](#), or PL kernel object files (`.xo`) as described in [Compiling C/C++ PL Kernels](#)
- `--link (-l)`: For linking PL kernels(`.xo`), AI Engine graph applications (`libadf.a`), and a target hardware platform (`.xpfm`) into a device binary (`.xclbin`), a hardware design (`.xsa`), or a Vivado export file (`.vma`) as described in [Linking the System](#)
- `--package (-p)`: For packaging an AI Engine `libadf.a` file into the `xclbin`, and generating an SD card file or QSPI/OSPI file as needed to initialize and boot the accelerated system as described in [Packaging the System](#)

Beyond these three command modes there are many options available to customize the build process as described in the following sections. Some of the options are supported for all three command modes, and some options are specific to compilation, linking, or packaging.

v++ General Options

The `v++` command supports many options for the compilation, linking, and packaging processes as described below.



TIP: All `v++` command-line options can be specified in a configuration file for use with the `--config` option, as discussed in the [Vitis Compiler Configuration File](#). For example, the `--platform` option can be specified in a configuration file using the following syntax:

```
platform=xilinx_u50_gen3x16_xdma_5_202210_1
```

--advanced

- **Applies to:** Compile, Link, Package

Specify parameters and properties for use by the `v++` command. See [--advanced Options](#) for more information.

--board_connection

- **Applies to:** Link

```
--board_connection
```

Specifies a dual in-line memory module (DIMM) board file for each DIMM connector slot. The board is specified using the Vendor:Board:Name:Version (vbnv) attribute of the DIMM card as it appears in the board repository.

For example:

```
<DIMM_connector>:<vbnv_of_DIMM_board>
```

-c | --compile

- **Applies to:** Compile

```
--compile
```

Required for compilation, but mutually exclusive with `--link` and `--package`. Run `v++ -c` to generate XO files from kernel source files.

--clock

- **Applies to:** Link

Provide a method for assigning clocks to kernels during the linking process. See [--clock Options](#) for more information.

--config

- **Applies to:** Compile, Link, Package

```
--config <config_file> ...
```

Specifies a configuration file containing `v++` command options. The configuration file can be used to capture compilation, linking, or packaging strategies, that can be easily reused by referring to the config file on the `v++` command line. In addition, the config file allows the `v++` command line to be shortened to include only the options that are not specified in the config file. Refer to the [Vitis Compiler Configuration File](#) for more information.



TIP: Multiple configuration files can be specified on the `v++` command line. A separate `--config` switch is required for each file used. For example:

```
v++ -l --config system.cfg --config vivado.cfg ...
```

--connectivity

- **Applies to:** Link

Used to specify important architectural details of the device binary during the linking process. See [--connectivity Options](#) for more information.

--custom_script

- **Applies to:** Compile, Link

```
--custom_script <kernel_name>:<file_name>
```

This option lets you specify custom Tcl scripts to be used in the build process during compilation or linking. Use with the `--export_script` option to create, edit, and run the scripts to customize the build process.

When used with the `v++ --compile` command, this option lets you specify a custom HLS script to be used when compiling the specified kernel. The script lets you modify or customize the Vitis HLS tool. Use the `--export_script` option to extract a Tcl script Vitis HLS uses to compile the kernel, modify the script as needed, and resubmit using the `--custom_script` option to better manage the kernel build process.

The argument lets you specify the kernel name, and path to the Tcl script to apply to that kernel. For example:

```
v++ -c -k kernel1 -export_script ...
*** Modify the exported script to customize in some way, then resubmit. ***
v++ -c --custom_script kernel1:./kernel1.tcl ...
```

When used with the `v++ --link` command for the hardware build target (`-t hw`), this option lets you specify the absolute path to an edited `run_script_map.dat` file. This file contains a list of steps in the build process, and Tcl scripts that are run by the Vitis and AMD Vivado™ tools during those steps. You can edit `run_script_map.dat` to specify custom Tcl scripts to run at those steps in the build process. You must use the following steps to customize the Tcl scripts:

1. Run the build process specifying the `--export_script` option as follows:

```
v++ -t hw -l -k kernel1 -export_script ...
```

2. Copy the Tcl scripts referenced in the `run_script_map.dat` file for any of the steps you want to customize. For example, copy the Tcl file specified for the synthesis run, or the implementation run. You must copy the file to a separate location, outside of the project build structure.

3. Edit the Tcl script to add or modify any of the existing commands to create a new custom Tcl script.
4. Edit the `run_script_map.dat` file to point a specific implementation step to the new custom script.
5. Relaunch the build process using the `--custom_script` option, specifying the absolute path to the `run_script_map.dat` file as shown below:

```
v++ -t hw -l -k kernel1 -custom_script /path/to/run_script_map.dat
```



IMPORTANT! When editing a custom synthesis run script, you must either comment out the lines related to the `dont_touch.xdc` file, or edit the lines to point to a new user-specified `dont_touch.xdc` file. The specific lines to comment or edit are shown below:

```
read_xdc dont_touch.xdc
set_property used_in_implementation false [get_files dont_touch.xdc]
```

The synthesis run returns an error related to a missing `dont_touch.xdc` file if this is not done.

--debug

- **Applies to:** Link

Specify the addition of debug IP core insertion into the hardware design. See [--debug Options](#) for more information.

-D | --define

- **Applies to:** Compile, Link

```
--define <arg>
```

Valid macro name and definition pair: `<name>=<definition>`.

Predefine name as a macro with definition. This option is passed to the `v++` preprocessor.

--export_archive

- **Applies to:** Link

```
--export_archive
```

This command produces a Vitis metadata archive file (`.vma`) from the linking process rather than the `.xclbin` device binary, or `.xsa` fixed platform. The flow for this command and the use of the `.vma` file is described in [Vitis Export to Vivado Flow](#).

--export_script

- **Applies to:** Compile, Link, Package

```
--export_script
```

This option runs the build process up to the point of exporting a script file, or list of script files, and then stops execution. The build process must be completed using the `--custom_script` option. This lets you edit the exported script, or list of scripts, and then rerun the build using your custom scripts.

When used with the `v++ --compile` command, this option exports a Tcl script for the specified kernel, `<kernel_name>.tcl`, that can be used to execute Vitis HLS, but stops the build process before actually launching the HLS tool. This lets you interrupt the build process to edit the generated Tcl script, and then restart the build process using the `--custom_script` option, as shown in the following example:

```
v++ -c -k kernel1 -export_script ...
```



TIP: This option is not supported for software emulation (`-t sw_emu`) of OpenCL kernels.

When used with the `v++ --link` command for the hardware build target (`-t hw`), this option exports a `run_script_map.dat` file in the current directory. This file contains a list of steps in the build process, and Tcl scripts that are run by the Vitis and Vivado tools during those steps. You can edit the specified Tcl scripts, customizing the build process in those scripts, and relaunch the build using the `--custom_script` option. Export the `run_script_map.dat` file using the following command:

```
v++ -l -t hw -export_script ...
```

--freqhz

- **Applies to:** Compile and Link

```
--freqhz <value>::<cu>[.<clk_pin>][,<cu_n>[.<clk_pin_n>]]
```

Specify a frequency for a component during compilation, or for the system during linking. The `--freqhz` option overrides any default clock frequency and applies the stated frequency during compilation and linking.

Specify a clock frequency in Hertz and a list of associated compute unit names and optionally their clock pins

- **<value>:** Must be specified in Hertz. For example 150 MHz is specified as 150000000

- **[cu[.clk_pin]]:** Optionally specifies a PL kernel or instance of a PL kernel (Compute Unit) target for the specified clock frequency. This lets you specify a different frequency for different instances of kernels. You can also specify the clock pin of the CU by adding the `.clk_pin` syntax. You can add multiple CU and clock pins in a single `--freqhz` option with a single frequency value specified.

For example:

```
v++ -l -t hw -platform <pfm_name> --freqhz=200000000:mm2s \
--freqhz=300000000:s2mm -config system.cfg
```



IMPORTANT! The `--freqhz` option is provided to consolidate the various methods or commands associated with specifying clocks in the AI Engine, PL kernels, or system design. The `--freqhz` option takes precedence over other clock commands, and `v++` can return an error if multiple commands are used.

--from_step

- **Applies to:** Compile, Link, Package

```
--from_step <arg>
```

Specifies a step name for the Vitis compiler build process, to start the build process from that step. If intermediate results are available, the link process fast forwards and begins execution at the named step if possible. This allows you to run the build through a `--to_step`, and then resume the build process at the `--from_step`, after interacting with your project in some method. You can use the `--list_step` option to determine the list of valid steps.



IMPORTANT! `--to_step/--from_step` are sequential build options that require you to use `--from_step` to resume the build in the same project directory that you used when starting the build with `--to_step`.

For example:

```
v++ --link --from_step vpl.update_bd
```

-g

- **Applies to:** Compile, Link

```
-g
```

Generates code for debugging the kernel during software emulation. Using this option adds features to facilitate debugging the kernel as it is compiled.

For example:

```
v++ -g ...
```

-h | --help

- **Applies to:** Compile, Link, Package

```
-h
```

Prints the help contents for the `v++` command. For example:

```
v++ -h
```

--hls

- **Applies to:** Compile

Specify commands for the Vitis HLS synthesis process during kernel compilation.



IMPORTANT! With the introduction of `v++ --mode hls` the [--hls Options](#) should only be used for the `v++ -c -k` mode for software emulation. The commands for `v++ --mode hls` are defined in [v++ Mode HLS](#).

-I | --include

- **Applies to:** Compile, Link

```
--include <arg>
```

Add the specified directory to the list of directories to be searched for header files. This option is passed to the Vitis compiler pre-processor.

--input_files <input_file>

- **Applies to:** Compile, Link, Package

```
--input_files <input_file1> <input_file2> ...
```

Specifies an OpenCL or C/C++ kernel source file for `v++` compilation, or Xilinx object (XO) files for `v++` linking.

For example:

```
v++ -l --input_files kernel1.xo kernelRTL.xo ...
```



TIP: Input files can also be specified positionally without the `--input_files` option. For example:

```
v++ -l kernel1.xo kernelRTL.xo ...
```

--interactive

- **Applies to:** Link

```
--interactive [ impl ]
```

v++ configures the needed environment and launches the Vivado tool with the implementation project.

Because you are interactively launching the Vivado tool, the linking process is stopped after the `vp1` step, which is the equivalent of using the `--to_step vp1` option in your v++ command.

When you are done interactively working with the Vivado tool, and you save the design checkpoint (DCP), you can resume the Vitis compiler linking process using the `v++ --from_step rtdgen`, or use the `--reuse_impl` or `--reuse_bit` options to read in the implemented DCP file or bitstream.

For example:

```
v++ --interactive impl
## Interactively use the Vivado tool
v++ --from_step rtdgen
```

-k | --kernel

- **Applies to:** Compile

```
--kernel <arg>
```

Compile only the specified kernel from the input file. Only one `-k` option is allowed per v++ command. Valid values include the name of the kernel to be compiled from the input `.cl` or `.c/.cpp` kernel source code.

This is required for C/C++ kernels, but is optional for OpenCL kernels. OpenCL uses the `kernel` keyword to identify a kernel. For C/C++ kernels, you must identify the kernel by `-k` or `--kernel`.

When an OpenCL source file is compiled without the `-k` option, all the kernels in the file are compiled. Use `-k` to target a specific kernel.

For example:

```
v++ -c --kernel vadd
```

--kernel_frequency

IMPORTANT! This command is used to specify kernel frequencies for Alveo platforms with scalable clocks. Platforms using fixed clocks, including both Alveo and embedded platforms, use the [--clock](#) Options for clock management. Refer to [Managing Clock Frequencies](#) for more information.

- **Applies to:** Link

```
--kernel_frequency <freq> | <clockID>:<freq>[<clockID>:<freq>]
```

Specifies a user-defined clock frequency (in MHz) for the kernel, overriding the default clock frequency defined on the hardware platform. The `<freq>` specifies a single frequency for kernels with only a single clock, or can be used to specify the `<clockID>` and the `<freq>` for kernels that support two clocks.

The syntax for overriding the clock on a platform with only one kernel clock, is to simply specify the frequency in MHz:

```
v++ --kernel_frequency 300
```

To override a specific clock on a platform with two clocks, specify the clock ID and frequency:

```
v++ --kernel_frequency 0:300
```

To override both clocks on a multi-clock platform, specify each clock ID and the corresponding frequency. For example:

```
v++ --kernel_frequency 0:300|1:500
```



TIP: During implementation of the design, if Vivado place and route tools are unable to meet the frequency specification, the tools can scale the clock frequency to an achievable frequency.

-l | --link

- **Applies to:** Link

```
--link
```

This is a required option for the linking process, which follows compilation, but is mutually exclusive with `--compile` or `--package`. Run `v++` in link mode to link XO input files and generate an `.xclbin` or and `.xsa` output file.



IMPORTANT! As discussed in [Linking the System](#), the `--link` option generates an `.xclbin` file for most platforms, but generates an `.xsa` file for AMD Versal™ platforms.

--linkhook

- **Applies to:** Link

Lets you customize the build process for the device binary by specifying Tcl scripts to be run at specific steps in the implementation flow. See [--linkhook Options](#) for more information.

--list_steps

- **Applies to:** Compile, Link, Package

```
--list_steps
```

List valid run steps for a given target. This option returns a list of steps that can be used in the `--from_step` or `--to_step` options. The command must be specified with the following options:

- `-t | --target [sw_emu | hw_emu | hw ]`:
- `[ --compile | --link ]`: Specifies the list of steps from either the compile or link process for the specified build target.

For example:

```
v++ -t hw_emu --link --list_steps
```

--log_dir

- **Applies to:** Compile, Link, Package

```
--log_dir <dir_name>
```

Specifies a directory to store log files into. If `--log_dir` is not specified, the tool saves the log files to `./_x/logs`. Refer to [Output Directories of the v++ Command](#) for more information.

For example:

```
v++ --log_dir /tmp/myProj_logs ...
```

--message_rules

- **Applies to:** Compile, Link, Package

```
--message-rules <file_name>
```

Specifies a message rule file with rules for controlling messages. Refer to [Using the Message Rule File](#) for more information.

For example:

```
v++ --message_rules ./minimum_out.mrf ...
```

--mode

- **Applies to:** Compile

Specifies a compilation mode for the `v++` command, with accepted values being `aie` or `hls`. The `v++ --mode aie` creates an AI Engine component as described in *AI Engine Tools and Flows User Guide* (UG1076), and can be used with configuration commands described under [v++ Mode AI Engine](#). The `v++ --mode hls` creates an HLS component as described in *Vitis HLS User Guide* (UG1399), and can be used with configuration file commands described in [v++ Mode HLS](#).

--no_ip_cache

- **Applies to:** Link

```
--no_ip_cache
```

Disables the IP cache for out-of-context (OOC) synthesis for Vivado Synthesis. Disabling the IP cache repository requires the tool to regenerate the IP synthesis results for every build, and can increase the build time. However, it also results in a clean build, eliminating earlier results for IP in the design.

For example:

```
v++ --no_ip_cache ...
```

-O | --optimize

- **Applies to:** Link

```
--optimize <arg>
```

This option specifies the optimization level of the Vivado implementation results. Valid optimization values include the following:

- `0`: Default optimization. Reduces compilation time.
- `1`: Optimizes to reduce power consumption by running Vivado implementation strategy `Power_DefaultOpt`. This takes more time to build the design.
- `2`: Optimizes to increase kernel speed. This option increases build time, but also improves the performance of the generated kernel by adding the `PHYS_OPT_DESIGN` step to implementation.
- `3`: This optimization provides the highest level performance in the generated code, but compilation time can increase considerably. This option specifies retiming during synthesis, and enables both `PHYS_OPT_DESIGN` and `POST_ROUTE_PHYS_OPT_DESIGN` during implementation.
- `s`: Optimizes the design for size. This reduces the logic resources of the device used by the kernel by running the `Area_Explore` implementation strategy.
- `quick`: Reduces Vivado implementation time, but can reduce kernel performance, and increases the resources used by the kernel. This enables the `Flow_RuntimeOptimized` strategy for both synthesis and implementation.

For example:

```
v++ --link --optimize 2
```

-o | --output

- **Applies to:** Compile, Link, Package

```
-o <output_name>
```

Specifies the name of the output file generated by the `v++` command. The compilation (`-c`) process output name must end with the `.xo` file suffix, for Xilinx object file. The linking (`-l`) process output file must end with the `.xclbin` file suffix, or `.xsa` suffix. The `--export_archive` command requires the `.vma` suffix. The `--package` command takes the `.xsa` as input and outputs the `.xclbin`.

For example:

```
v++ -o krnl_vadd.xo
```

If `--o` or `--output` are not specified, the output file names default to the following:

- **Compilation:** `a.o`
- **Linking:** `a.xclbin` (`a.xsa` for Versal platforms)
- **Packaging:** `a.xclbin`

-p | --package

- **Applies to:** Package

Specify options for the Vitis compiler to package your design for either running emulation or running on hardware. See [--package Options](#) for more information.

--part

- **Applies to:** Compile, Link, Package

```
--part <part_name>
```

Specifies an AMD part for use in compiling, linking, and packaging the components of your design. The `--part` command can be used to specify a device rather than a full platform or XSA. Specifying `v++ --link --part <Versal part only>` instructs the tools to generate a simple base hardware platform for the specified AMD Versal™ device.

For example:

```
v++ --part xcvc1902-vsva2197-2MP-e-S --target hw_emu ...
```



IMPORTANT! `--part` must be used for the software (AI Engine and PL kernel) development only for future platforms where no platform changes are required. Create a platform in Vivado and use it if there are hardware changes required for the software development. The default clock provided for the `--part` for Versal design is 300 MHz. The default values are determined by a different process as described in [Managing Clock Frequencies](#).

-f | --platform

- **Applies to:** Compile, Link, Package

```
--platform <platform_name>
```

Specifies the name of a supported acceleration platform as specified by the `$PLATFORM_REPO_PATHS` environment variable, or the full path to the platform `.xpfm` file. For a list of supported platforms for the release, see the [Vitis Software Platform Release Notes](#).

This is a required option for both compilation and linking, to define the target AMD platform of the build process. The `--platform` option accepts either a platform name, or the path to a platform file `xpfm`, using the full or relative path.



IMPORTANT! The specified platform and build targets for compiling, linking, and packaging must match. The `--platform` and `-t` options specified when the XO file is generated by compilation, must be the `--platform` and `-t` used during linking, and packaging. For more information, see [platforminfo Utility](#).

For example:

```
v++ --platform xilinx_u50_gen3x16_xdma_5_202210_1 ...
```

--profile

- **Applies to:** Compile, Link

Specify options to configure the Xilinx Runtime environment to capture application performance information. See [--profile Options](#) for more information.

--remote_ip_cache

- **Applies to:** Link

```
--remote_ip_cache <dir_name>
```

Specifies the location of the remote IP cache directory for Vivado Synthesis to use during out-of-context (OOC) synthesis of IP. OOC synthesis lets the Vivado synthesis tool reuse synthesis results for IP that have not been changed in iterations of a design. This can reduce the time required to build your `.xclbin` files, due to reusing synthesis results.

When the `--remote_ip_cache` option is not specified the IP cache is written to the current working directory from which `v++` was launched. You can use this option to provide a different cache location, used across multiple projects for instance.

For example:

```
v++ --remote_ip_cache /tmp/IP_cache_dir ...
```

--report_dir

- **Applies to:** Compile, Link, Package

```
--report_dir <dir_name>
```

Specifies a directory to store report files into. If `--report_dir` is not specified, the tool saves the report files to `./_x/reports`. Refer to [Output Directories of the v++ Command](#) for more information.

For example:

```
v++ --report_dir /tmp/myProj_reports ...
```

-R | --report_level

- **Applies to:** Compile, Link, Package

```
--report_level <arg>
```

Valid report levels: 0, 1, 2, estimate.

These report levels have mappings kept in the `optMap.xml` file. You can override the installed `optMap.xml` to define custom report levels.

- `-R0` specification turns off all intermediate design checkpoint (DCP) generation during Vivado implementation. Turns on post-route timing report generation.
- The `-R1` specification includes everything from `-R0`, plus `report_failfast pre-opt_design`, `report_failfast post-opt_design`, and enables all intermediate DCP generation.
- The `-R2` specification includes everything from `-R1`, plus `report_failfast post-route_design`.
- The `-Restimate` specification forces Vitis HLS to generate a `design.xml` file if it does not exist and then generates a System Estimate report, as described in [System Estimate Report](#).



TIP: This option is useful for the software emulation build (`-t sw_emu`), when `design.xml` is not generated by default.

For example:

```
v++ -R2 ...
```

--reuse_bit

```
--reuse_bit <arg>
```

- **Applies to:** Link

Specifies the path and file name of generated bitstream file (.bit) to use when generating the device binary (xclbin) file. As described in [Using --to_step and Launching Vivado Interactively](#), you can specify the --to_step option to interrupt the Vitis build process and manually place and route a synthesized design to generate the bitstream.



IMPORTANT! The --reuse_bit option is a sequential build option that requires you to use the same project directory when resuming the Vitis compiler with --reuse_bit that you specified when using --to_step to start the build.

For example:

```
v++ --link --reuse_bit ./project.bit
```

--reuse_impl

```
--reuse_impl <arg>
```

- **Applies to:** Link

Specifies the path and file name of an implemented design checkpoint (DCP) file to use when generating the device binary (xclbin) file. The link process uses the specified implemented DCP to extract the FPGA bitstream and generates the xclbin. You can manually edit the Vivado project created by a previously completed Vitis build, or specify the --to_step option to interrupt the Vitis build process and manually place and route a synthesized design, for instance. This allows you to work interactively with Vivado Design Suite to change the design and use DCP in the build process.



IMPORTANT! The --reuse_impl option is an incremental build option that requires you to use the same project directory when resuming the Vitis compiler with --reuse_impl that you specified when using --to_step to start the build.

For example:

```
v++ --link --reuse_impl ./manual_design.dcp
```

-s | --save-temps

- **Applies to:** Compile, Link, Package

```
--save-temps
```

Directs the `v++` command to save intermediate files/directories created during the compilation and link process. Use the `--temp_dir` option to specify a location to write the intermediate files to.



TIP: This option is useful for debugging when you encounter issues in the build process.

For example:

```
v++ --save-temps ...
```

-t | --target

- **Applies to:** Compile, Link, Package

```
-t [ sw_emu | hw_emu | hw ]
```

Specifies the build target, as described in [Working with Build Targets](#). The build target determines the results of the compilation and linking processes. You can choose to build an emulation model for debug and test, or build the actual system to run in hardware. The build target defaults to `hw` if `-t` is not specified.



IMPORTANT! The specified platform and build targets for compiling and linking must match. The `--platform` and `-t` options specified when the XO file is generated by compilation must be the `--platform` and `-t` used during linking.

The valid values are:

- `sw_emu`: Software emulation
- `hw_emu`: Hardware emulation
- `hw`: Hardware

For example:

```
v++ --link -t hw_emu
```

--temp_dir

- **Applies to:** Compile, Link, Package

```
--temp_dir <dir_name>
```

This allows you to manage the location where the tool writes temporary files created during the build process. The temporary results are written by the `v++` compiler, and then removed, unless the `--save-temps` option is also specified.

If `--temp_dir` is not specified, the tool saves the temporary files to `./_x/temp`. Refer to [Output Directories of the v++ Command](#) for more information.

For example:

```
v++ --temp_dir /tmp/myProj_temp ...
```

--to_step

- **Applies to:** Compile, Link, Package

```
--to_step <arg>
```

Specifies a step name, for either the compile or link process, to run the build process through that step. You can use the `--list_step` option to determine the list of valid compile or link steps.

The build process terminates after completing the named step. At this time, you can interact with the build results. For example, manually accessing the HLS project or the Vivado Design Suite project to perform specific tasks before returning to the build flow, launch the `v++` command with the `--from_step` option.



IMPORTANT! `--to_step/--from_step` are sequential build options that require you to use `--from_step` to resume the build in the same project directory that you used when starting the build with `--to_step`.

You must also specify `--save-temps` when using `--to_step` to preserve the temporary files required by the Vivado tools. For example:

```
v++ --link --save-temps --to_step vpl.update_bd
```

--user_board_repo_paths

- **Applies to:** Link

```
--user_board_repo_paths
```

Specifies an existing user board repository for DIMM board files. This value is pre-pended to the `board_part_repo_paths` property of the Vivado project.

--user_ip_repo_paths

- **Applies to:** Link

```
--user_ip_repo_paths <repo_dir>
```


Specifies the directory location of one or more user IP repository paths to be searched first for IP used in the kernel design. This value is appended to the start of the `ip_repo_paths` used by the Vivado tool to locate IP cores. IP definitions from these specified paths are used ahead of IP repositories from the hardware platform (`.xsa`) or from the AMD IP catalog.



TIP: Multiple `--user_ip_repo_paths` can be specified on the `v++` command line.

The following lists show the priority order in which IP definitions are found during the build process, from high to low.

Note: All of following entries can possibly include multiple directories in them.

- For the system hardware build (`-t hw`):
 1. IP definitions from `--user_ip_repo_paths`.
 2. Kernel IP definitions (`vpl --iprepo` switch value).
 3. IP definitions from the IP repository associated with the platform.
 4. IP cache from the installation area (for example, `<Install_Dir>/Vitis/2019.2/data/cache/`).
 5. AMD IP catalog from the installation area (for example, `<Install_Dir>/Vitis/2019.2/data/ip/`)
- For the hardware emulation build (`-t hw_emu`):
 1. IP definitions and User emulation IP repository from `--user_ip_repo_paths`.
 2. Kernel IP definitions (`vpl --iprepo` switch value).
 3. IP definitions from the IP repository associated with the platform.
 4. IP cache from the installation area (for example, `<Install_Dir>/Vitis/2019.2/data/cache/`).
 5. `$::env(XILINX_VITIS)/data/emulation/hw_em/ip_repo`
 6. `$::env(XILINX_VIVADO)/data/emulation/hw_em/ip_repo`
 7. AMD IP catalog from the installation area (for example, `<Install_Dir>/Vitis/2019.2/data/ip/`)

For example:

```
v++ --user_ip_repo_paths ./myIP_repo ...
```

-v | --version

```
-v
```

Prints the version and build information for the `v++` command. For example:

```
v++ -v
```

--vivado

- **Applies to:** Link

Specify properties and parameters to configure the Vivado synthesis and implementation environment prior to building the device binary. See [--vivado Options](#) for more information.

v++ Compilation Options

The `v++` command provides several features for compilation of PL kernels and AI Engine graphs. These options can be broken down into the following categories:

- `--compile`: These are the classic top-down compilation commands for the Vitis acceleration flow. This flow is still supported, and in some cases necessary. For more information refer to the section on `v++` Command.
- `--mode aie`: This is a new compilation mode in the `v++` compiler which enables the generation of the AI Engine graph application, and also supports the use of the `x86simulator` and `aiesimulator` tools for analysis. These options are described in detail in [v++ Mode AI Engine](#).
- `--mode hls`: This is a new compilation mode in the `v++` compiler that enables the bottom-up generation of an HLS component, either for the AMD Vivado™ IP flow or the Vitis Kernel flow as described in the *Vitis High-Level Synthesis User Guide* (UG1399). The options to configure and analyze an HLS component are described in [v++ Mode HLS](#).

v++ General Compilation Options

The general `v++` compilation options typically appear on the command-line rather than a configuration file. It can include the `-c` (or `--compile`) option, a `--mode` when generating an AI Engine component or HLS component, the `--platform` or `--part`, the build `--target`, and a `--config` file as shown below:

```
v++ -c --mode aie --platform xilinx_vck190_base_202310_1 --target hw \
-config ./aie_config.cfg <input_files...>
```



TIP: Relative paths used on the command line are relative to the current working directory where the command is launched. Relative paths in the config file are relative to the location of the config file.

Command options for `v++` can be generally be used in a config file or on the command-line. However, the following options are only available for use on the command-line:

- **-c [--compile] :**

Run compilation mode.

- **--config <file_name>:** Specifies the config file path and name.

- **--mode [aie | hls] :**

Specifies the compilation mode as either compile an AI Engine component, or an HLS component.



IMPORTANT! The absence of `--mode` puts the `v++ -c` command in the traditional top-down compilation mode for the Vitis kernel flow as described in [Compiling C/C++ PL Kernels](#).

- **-h [--help]:**

Print usage message for options. Can be combined with the `--mode` option to list the available options for the AI Engine or HLS components.

- **-v [--version]:**

Prints version information.

- **--input_files <arg>:**

Specify input file(s). Input file(s) can also be specified positionally without using the `--input_files` option.

- **-o [--output] <arg>:**

Specify the output file name and path. Default: `libadf.a` for the AI Engine compilation mode, `<kernel_name>.xo` for the HLS compilation mode, `a.xclbin` for the `v++ --link` command.

- **-f [--platform]:**

This is a path to a Vitis platform file that defines the hardware and software components available for AI Engine components, HLS components, and Applications. The platform can be a platform specification (XPFM) or a hardware specification (XSA).

- **--log_dir <arg>:**

Specify a directory to copy internally generated log files.

- **--macro_dir <arg>:**

Specify a macro to define a base directory: `<macro_name>=<base_directory>` Macros with a `$` sign can be used to specify a relative path.

- **--report_dir <arg>:**

Specify a directory to copy report files.

- **--work_dir <arg>:**

Optionally specify a working directory for the build and output files. For `--mode aie` the work directory defaults to `./Work`. For `--mode hls` it defaults to the top-level function specified by `hls.syn.top`. This option can only be specified on the command line.

- **--aie_legacy:**

Use legacy options for `aiecompiler`.

The following options can either be used in a config file or on the command-line:



TIP: As these are general options they do not need to be placed under a header (such as `[AIE]` or `[HLS]`) in the config file.

- **-t [--target] <arg>:**

The `--target` argument has different values when used with or without `--mode`.

- Used with `--mode` the target defines the compilation target of the AI Engine or HLS component.
 - `x86`: Specifies compilation for x86 simulation of an AI Engine component, or for C-simulation of the HLS component.
 - `hw`: Specifies compilation for use with AI Engine simulator for AI Engine components, for C/RTL Co-simulation for HLS components, or to run on the physical device for either components.
- Used without `--mode`: the target defines the tradition build targets (`sw_emu`, `hw_emu`, `hw`) of the Application flow as described in [Creating a System Project for Heterogeneous Computing](#).

Specify a target as `sw_emu`, `hw_emu`, or `hw` for the classic `v++` compilation mode; or as `x86` simulation for AI Engine compilation mode or `hw` for AI Engine simulation. The default target is `hw`.

- **--part <arg>:**

Specify part family or part value. Note this option is mutually exclusive with `--platform` or `--hls.board`.

- **-D [--define] <name=definition>:**

Predefine `<name>` as a macro with `<definition>`.

- **-I [--include] <arg>:**

Include the specified directory in the list of directories to be searched for header files.

v++ Mode AI Engine

The AI Engine compilation mode provides for the development, optimization, analysis, and export of AI Engine graph applications (`libadf.a`). The AI Engine compilation mode uses the following command:

```
v++ -c --mode aie --target hw --platform xilinx_vck190_base_202320_1 \
--work_dir ./myWork --config ./config.cfg <Input File>
```

- `v++ -c --mode aie --target hw`: Indicates compilation of an AI Engine component for the HW target indicating the generation of the `libadf.a` file for use in AI Engine simulation and to run on the physical device. You can also specify `x86sim` for the target to build the component for x86 functional simulator. The contents of the output directory depend on the target.
- `--platform xilinx_vck190_base_202320_1`: Specifies the platform or physical device to build the component for.
- `--work_dir ./myWork`: Specifies the work directory to use for building the AI Engine component.



TIP: By default the compiler writes all outputs to a directory called *Work* in a sub-directory of the current directory where the tool was launched, and creates a file called *libadf.a* in the current directory.

- `--config ./config.cfg`: Specifies the AI Engine component config file containing a variety of options available to the AI Engine compiler, x86 Simulator, and AI Engine Simulator which are part of the analysis and profiling tools for the AI Engine component.
- `<Input File>` specifies the data flow graph code that defines the `main()` application for the AI Engine graph. The input flow graph is specified using a data flow graph language. For a description of the data flow graph, refer to [Introduction to Graph Programming](#) in the *AI Engine Kernel and Graph Programming Guide (UG1079)*.

The `v++ -c --mode aie` command options are described in the following sections. The AI Engine options should be added to a configuration file under the `[AIE]` heading.



TIP: Some `[AIE]` options can be specified on the command line instead of in a config file, although using a config file is the recommended approach. To use an `[AIE]` option on the command line add `--aie.` to the start of the command name from the following sections. For example, to specify the constraints from the command line use `--aie.constraints`

AI Engine Options

The following options apply to the AI Engine compilation process.

- **constraints:**

Constraints (location, bounding box, etc.) can be specified using a JSON file. This option lets you specify one or more constraint files.

```
constraints=constraints.json
```

- **heapsize:**

Heap size (in bytes) used by each AI Engine

The stack, heap, and sync buffer (32 bytes, includes the graph run iteration number information) are allocated up to 32768 bytes of data memory. The default heap size is set to 1024 bytes. Before changing the heap size to a different value, ensure that the sum of the stack, heap, and sync buffer sizes does not exceed 32768 bytes.

Used for allocating any remaining file-scoped data that is not explicitly connected in the user graph.

```
heapsize=512
```

- **log-level:**

Log level for verbose logging. Only applicable when used with `verbose`. The specified range can be from 0 to 5, with increasing details provided in the log file as the number increases.

- 0 is the same as the `quiet` option
- 1 is the default logging of the compiler
- 2 is the default logging specified with `verbose`
- 3 - 5 are increased logging details when used with `verbose`

```
log-level=4
```

- **pl-freq:**

Specifies the interface frequency (in MHz) for all PL kernels and PLIOs. The default frequency is a quarter of the AI Engine frequency dependent on the specific device speed grade, and the maximum frequency is half of the AI Engine frequency. The PL frequency specific to each interface is provided in the graph.

```
pl-freq=500
```

- **pl-register-threshold:**

Specifies the frequency (in MHz) threshold for registered AI Engine-PL crossings. The default frequency is one-eighth of the AI Engine frequency dependent on the specific device speed grade.

```
pl-register-threshold=300
```



IMPORTANT! Values above a quarter of the AI Engine array frequency are ignored, and a quarter is used instead.

- **stacksize:**

Stack size (in bytes) used by each AI Engine tile. The default stack size is set to 1024 bytes. Used as a standard compiler calling convention including stack-allocated local variables and register spilling.



TIP: The stack, heap, and sync buffer (32 bytes) are allocated up to 32768 bytes of data memory. Before changing the stack size to a different value, ensure that the sum of the stack, heap, and sync buffer sizes does not exceed 32768 bytes.

```
stacksize=512
```

CDO Options

The CDO options apply to generator code for graph configuration and initialization in configuration data object (CDO) format. This is used during SystemC-RTL simulation and during actual hardware execution.

- **broadcast-enable-core:**

Enables all AI Engine cores associated with a graph using broadcast. This option reserves one broadcast channel in the array for core enabling purpose. This is enabled by default, and you can disable it by setting the argument to false.

```
broadcast-enable-core=false
```

Compiler Debugging Options

The Debugging options specify features of the compiler to enable troubleshooting errors during the build process, such as increased log details and consistency checking in the source code.

- **adf-api-log-level:**

ADF API log-level. Available values are as follows:

- 0: errors
- 1: level-0 + warnings
- 2: level-1 + info messages
- 3: level-2 + debug messages

The default is 2.

```
adf-api-log-level=3
```

- **kernel-linting:**

Perform consistency checking between graphs and kernels. Accepted values are true and false. The default is false.

```
kernel-linting=true
```

- **quiet:**

Suppress output of the compiler. Accepted values are true and false. The default is false.

```
quiet=true
```

- **verbose:**

Verbose output of the compiler. Accepted values are true and false. The default is false.

```
verbose=true
```



TIP: Can be used with `log-level` to increase the verbosity of the logs. For example:

```
log-level=4
```

Design Rule Check Options

The AI Engine mapper and router support configuring a list of Design Rule Checks (DRCs). The DRC commands shown below should be entered into the config file outside of any heading ([AIE] for instance).

- **drc.disable:**

Disable the DRC rule check for the specified ID. A disabled check is not executed.

```
drc.disable=AIE-ROUTER-3
```

- **drc.enable:**

Enable a previously disabled DRC rule check for the specified ID.

```
drc.enable=AIE-ROUTER-3
```

- **drc.severity:**

Change the severity of a DRC rule check. The format of the argument is `<ID>:<severity>[:context]`

Where:

- `<ID>` is the DRC rule ID which can be found on DRC messages reported by the tool.

- <severity> represents the new severity to assign to the specified ID.
- [context]

```
drc.severity=AIE-ROUTER-3:warning
```

- **drc.waive:**

Waive the Design Rule Check for the specified ID. A waived check is still performed, but marked as waived, meaning that you don't care about this specific check for some reason.

```
drc.waive=AIE-ROUTER-3
```

File Options

The file options are used for file management activities through the config file, such as specifying include files and output file names.

- **include:**

This option can be used to include additional directories in the include path for the compiler front-end processing. Specify one or more include directories.

- **output:**

Specifies an output.json file that is produced by the front end for an input data flow graph file. The output file is passed to the back-end for mapping and code generation of the AI Engine device. This is ignored for other types of input.

- **output-archive:**

Specify output archive name which will contain compiled AI Engine artifacts (default: `libadf.a`). The output archive is written to the component directory above the Work directory. The `libadf.a` file is still written by the tool.

```
output-archive=myGraph.a
```

Module Specific Options

Options that apply to kernel implementation on AI Engine tiles.



IMPORTANT! Only AI Engine kernels that were modified are recompiled in subsequent compilations of the AI Engine graph. Any un-modified kernels will not be recompiled; therefore, the application of these options might require touching kernel source to be recompiled.

- **Xchess arg:**

Used to pass kernel specific options to the CHES compiler that is used to compile code for each AI Engine. The option string is specified as `<kernel>:<optionid>=<value>`. The option string is included during compilation of generated source files on the AI Engine where the specified kernel is mapped.

```
Xchess=main:darts.xargs=-nb
```

- **Xelfgen arg:**

Can be used to pass additional command-line options to the ELF generation phase of the compiler, which is currently run as a `make` command to build all AI Engine ELF files.

For example, to limit the number of parallel compilations to four, you write `Xelfgen="-j4"`.

Note: If you see errors with `bad_alloc` in the log, or if the Vitis IDE crashes, this could be due to insufficient memory on your workstation. A possible workaround (other than increasing the available memory on your machine) is to limit the parallelism used by the compiler during code generation by passing this option to the Elf Generator Makefile (`-j1` or `-j2`).

```
Xelfgen=-j2
```

- **Xmapper arg:**

Can be used to pass additional command-line options to the mapper phase of the compiler. These are options to try when the design is either failing to converge in the mapping or routing phase, or when you are trying to achieve better performance via reduction in memory bank conflict. See the [Mapper and Router Options](#) for a list and description of options.

```
Xmapper=DisableFloorplanning
```

- **Xpreproc arg:**

Pass general option to the PREPROCESSOR phase for all source code compilations AIE/PS/PL/x86sim of the AI Engine component (e.g. `-D...`).

```
Xpreproc=-D<var>=<value>
```

- **Xpslinker arg:** Pass general option to the PS LINKER phase of the AI Engine component (e.g. `-L...` `-l...`).

```
Xpslinker=-L<libpath> -l<libname>
```

- **Xrouter arg:**

Pass general option to the ROUTER phase.

```
Xrouter=dmaFIFOsInFreeBankOnly
```

- **Xx86sim arg:**

Pass x86sim-specific option to the compiler.

```
Xx86sim=clangStaticAnalyzer
```

- **fast-floats:**

Enable fast implementation for linear floating point scalar operations like `add`, `sub`, `mul` and `compare`. Accepted values are `true` and `false`. The default is `false`.

```
fast-floats=true
```

- **fast-nonlinearfloats:**

Enable fast implementation for non-linear floating point scalar operations like `sine/cosine`, `sqrt` and `inv`. Accepted values are `true` and `false`. The default is `false`.

```
fast-nonlinearfloats=true
```

- **fastmath:**

Enable fast implementations of `float2fix`, `fplt` and `fpge`. Accepted values are `true` and `false`. The default is `false`.

```
fastmath=true
```

- **float-accuracy:**

Option available only for *AI Engine ML*. It selects the required floating-point compute accuracy emulated with `bfloat16` numerical type:

```
float-accuracy=<arg>
```

Available argument values are:

- `safe`: Accuracy is slightly better than FP32.
- `fast`: Improved performances with similar accuracy to FP32.
- `low`: Best performances with better accuracy than FP16 and `bfloat16`.

Event Tracing Options

Options to define the event-tracing characteristics of the compiled kernels and graph. These options prepare the design for event tracing during runtime. See [Event Trace Build Flow](#) for more information.

- **event-trace:**

Event trace configuration value. Where the specified `<value>` indicates the following:

- **functions:** Function transition view without stalls.
- **functions_partial_stalls:** Function transition view with stream/lock/cascade stalls.
- **functions_all_stalls:** Function transition view with all stalls (stream/lock/cascade/memory).
- **runtime:** Run-time event tracing configuration.

```
event-trace=functions_all_stalls
```

- **event-trace-mem-tile:**

Event trace configuration levels for memory tile. Accepted value is `L1` which specifies tracing DMA port running events.

```
event-trace-mem-tile=L1
```

- **event-trace-port:**

Set the AI Engine event tracing port. Accepted values are `plio` and `gmio`. The default value is `gmio` which is the recommended event-trace-port configuration.

```
event-trace-port=plio
```

- **graph-iterator-event:**

Generates `user_event()` whenever the graph iterator is incremented. This provides the ability to delay the start of the hardware event trace based on the graph iteration.

- **num-trace-streams:**

Specifies the number of trace streams. The default value is 16.

```
num-trace-streams=32
```

- **trace-plio-width:**

PLIO width for trace streams (default: 64)

Optimization Options

- **xlopt:**

Enable kernel optimizations based on the specified value:

- 0: No kernel optimizations.
- 1: Computation of heap size, generation of guidance based on LLVM IR analysis, and insertion of loop pragmas.

- 2: The same optimizations as 1 with the addition of loop peeling for unrolled loops, and automatic inlining.

```
xlopt=2
```

- **Xxloptstr:**

Option string to enable or disable optimizations in XLOpt level 1,2.

- -xinline-threshold=T : set inlining threshold to T (default T = 5000, only for optlevel=2).
- -annotate-pragma : insertion of loop unrolling, pipelining, and flattening pragmas (default = true)

```
Xxloptstr=-annotate-pragma
```

Miscellaneous Options

- **--evaluate-fifo-depth:**

This option, available only for the AI Engine, analyzes re-convergent data paths. Data might be sent on multiple paths and sometimes they can re-converge which can result in a deadlock. Such deadlocks can be resolved by adding FIFOs to the appropriate data paths.

The steps for evaluating and resolving deadlocks as a result of re-convergent data paths is as follows:

1. Compile the design with this option.
2. Run `aiesimulator` on the design.
3. Open Vitis unified IDE with the simulation `run_summary`
4. Note the Estimated FIFO column.
5. Apply the recommended number of FIFOs from the Estimated FIFO column using the `fifo_depth` constraint on specified nets on the graph, and recompile the design.

- **disable-multirate:**

Disable multirate in ADF graphs. Accepted values are true and false. The default is false.

```
disable-multirate=true
```

- **no-init:**

This option disables initialization of window buffers in AI Engine data memory. This option enables faster loading of the binary images into the SystemC-RTL co-simulation framework. Accepted values are true and false. The default is false.

```
no-init=true
```

- **nodot-graph:**

By default, the AI Engine compiler produces .dot and .png files to visualize the user-specified graph and its partitioning onto the AI Engines. This option can be used to eliminate the dot graph output. Accepted values are true and false. The default is false.

```
nodot-graph=true
```

v++ Mode HLS

The HLS compilation mode provides access to numerous features for the development, optimization, analysis, and export of Vitis kernels (.xo) or Vivado IP (.xci) files. The HLS mode can be reached by using the following command:

```
v++ -c --mode hls -h [options] <input_files...>
```

The HLS compilation options should be entered into a configuration file for use with the `v++` command using the `--config` option. The HLS options should be placed under a section head of `[HLS]` in the config file. For example the following config file specifies the part, the source file, the test bench files, and the flow target. `part` is not specified under the `[HLS]` header because this is a general option for the `v++` compiler.

```
part=xcvu11p-flga2577-1-e

[hls]
clock=8
flow_target=vitis
syn.file=../../src/dct.cpp
syn.top=dct
tb.file=../../src/out.golden.dat
tb.file=../../src/in.dat
tb.file=../../src/dct_test.cpp
tb.file=../../src/dct_coeff_table.txt
syn.output.format=xo
clock_uncertainty=15%
```



TIP: To use the HLS config options on the `v++` command line you can simply begin the command name with `--hls`. For instance, to specify the `flow_target` from the command line use `--hls.flow_target`.

The HLS mode command options are described in the following sections.

HLS General Options



IMPORTANT! The following options must appear in the HLS configuration file under the `[hls]` header.

- **clock:**

Specify the clock period in `ns` or `MHz` (`ns` is default). If no period is specified a default period of 10 ns is used.

```
clock=8ns
```



IMPORTANT! If your HLS configuration file uses `platform=` instead of `part=` then you must also specify `freqhz=` instead of `clock=` as shown here to change the default clock frequency of the platform.

- **clock_uncertainty:**

Specify how much of the clock period is used as a margin by HLS. The margin of uncertainty is subtracted from the clock period to create an effective clock period. The clock uncertainty is defined in `ns`, or as a percentage of the clock period. The clock uncertainty defaults to 27% of the clock period. When specifying a value, the default units is `ns` but `%` or `MHz` can also be used.

```
clock_uncertainty=15%
```

- **flow_target:**

Set the flow target to synthesize either a Vitis kernel (`.xo`) or a Vivado IP (`.xci`). The Vitis kernel is used in the Application Acceleration flow, while the Vivado IP can be used in the embedded software design flow.



IMPORTANT! There are differences in the interface definition supported by Vivado IP or Vitis kernels.

C-Synthesis Sources

- **syn.cflags:**

Defines compilation flags to be applied to all `syn.file` defined source files for use during synthesis.

```
syn.cflags=-I../src/
```

- **syn.csimflags:**

Defines compilation flags to be applied to all `syn.file` source files for use during C-simulation or RTL/Co-simulation.

- **syn.file:**

Specify the file path and name of a source file to be used during synthesis of the HLS component. Multiple files require multiple `syn.file` statements.

The file paths can be specified as either absolute or relative, where relative paths are relative to the location of the config file, whether inside the HLS component or outside the component.

```
syn.file=../../src/dct.cpp
```

- **syn.file_cflags:**

Apply a compilation flag for synthesis to the specified source file. Specify the file path and name first, followed by a comma, followed by the cflags:

```
syn.file_cflags=../../src/dct.cpp,-I../../src/
```

- **syn.file_csimflags:**

Apply a compilation flag for simulation to the specified source file. Specify the file path and name first, followed by a comma, followed by the csimflags.

```
syn.file_csimflags=../../src/dct.cpp,-Wno-unknown-pragmas
```

- **syn.blackbox.file:**

Specify the JSON file to be used for an RTL blackbox. The information in this file is used by the HLS compiler during synthesis and when running RTL/Co-simulation, as described in [Adding RTL Blackbox Functions](#).

```
syn.blackbox.file=../../RTL/fft.json
```

- **syn.top:**

Specifies the name of the function to be synthesized as the top-level function for the HLS component. This can be used to identify the top function in source code where multiple functions are defined.

```
syn.top=dct
```



IMPORTANT! Any functions called by the top-level function will also become part of the HLS component.

Test bench Sources

- **tb.cflags arg:**

Defines compilation flags to be applied to all `tb.file` defined source files for use during simulation or co-simulation.

```
tb.cflags=-Wno-unknown-pragmas
```

- **tb.file arg:**

Specify the file path and name of a test bench source file to be used during simulation or co-simulation of the HLS component. Multiple files require multiple `tb.file` statements.

The file paths can be specified as either absolute or relative, where relative paths are relative to the location of the config file, whether inside the HLS component or outside the component.

```
tb.file=../../src/dct_test.cpp
```

- **tb.file_cflags arg:**

Apply a compilation flag for simulation or co-simulation to the specified test bench source file. Specify the file path and name first, followed by a comma, followed by the cflags:

```
syn.file_cflags=../../src/dct.cpp,-Wno-unknown-pragmas
```

Array Partition Configuration

The `syn.array_partition` commands specify the default behavior for array partitioning for the whole design. These settings can be overridden for specific arrays using the `syn.directive.array_partition`.

- **syn.array_partition.complete_threshold <value>:**

Sets the threshold for completely partitioning arrays. Arrays which have fewer elements than the specified value will be completely partitioned into individual elements.

```
syn.array_partition.complete_threshold=12
```

- **syn.array_partition.throughput_driven [auto | off]:**

Enable automatic partial and/or complete array partitioning.

- `auto` : Enable automatic array partitioning with smart trade-offs between area and throughput. This is the default value.
- `off` : Disable automatic array partitioning.

```
syn.array_partition.throughput_driven=off
```

Array Stencil Configuration

The `syn.array_stencil` command specifies the default behavior of array stenciling for the whole design. These settings can be overridden for specific arrays using the `syn.directive.array_partition`.

- **syn.array_stencil.throughput_driven:**

Enable automatic array stenciling with trade-offs between area and throughput.

- `auto` : Enable automatic array stenciling with smart trade-offs between area and throughput.
- `off` : Disable automatic array stenciling. This is the default value.

```
syn.array_partition.throughput_driven=auto
```

C-Simulation Configuration

The `csim` options apply to the C-simulation process used to validate the C/C++ language for the design. Refer to Running C Simulation in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)) for more information.

- **csim.O:**

Enables optimizing compilation which eliminates debug constructs. The default is false and compilation is done in debug mode to enable debugging.

```
csim.O=true
```

- **csim.argv:**

Specifies an argument list for the behavioral test bench. The specified `<arg>` will be passed to the `main()` function in the C test bench.

```
csim.argv=arg1 arg2
```

- **csim.clean:**

Enables clean build. The default is false. Without this option the design will compile incrementally.

```
csim.clean=true
```

- **csim.code_analyzer:**

Enable code analysis and interactive Code Analyzer report.

```
csim.code_analyzer=0
```

- **csim.ldflags:**

Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.

```
csim.ldflags=ldExample
```

- **csim.mflags:**

Provides for options to be passed to the compiler for C simulation. This is typically used to speed up compilation.

```
csim.mflags=mExample
```

- **csim.profile:**

Enable the creation of the [Pre-Synthesis Control Flow](#).

```
csim.profile=true
```

- **csim.setup:**

When this option is specified, the simulation binary will be created in the `csim` directory of the current HLS component, but simulation will not be executed. Simulation can be launched later from the compiled executable. The default is false, and simulation is run after setup is complete.

```
csim.setup=true
```

Co-Simulation Configuration

The `cosim` options apply to the C/RTL Co-Simulation process used to validate the RTL produced by HLS synthesis. This includes using the C/C++ test bench used earlier in C-simulation and using the RTL design in behavioral simulation as described in C/RTL Co-Simulation in Vitis HLS in the Vitis HLS User Guide ([UG1399](#)).

- **cosim.O:**

Enables optimizing compilation which eliminates debug constructs. The default is false and compilation is done in debug mode to enable debugging. Enabling optimized compilation of the C/C++ test bench and RTL wrapper increases compilation time, but results in better run time performance.

```
cosim.O=true
```

- **cosim.argv:**

Specifies an argument list for the behavioral test bench. The specified `<arg>` will be passed to the `main()` function in the C test bench.

```
cosim.argv=arg1 arg2
```

- **cosim.compiled_library_dir:**

Specifies the compiled library directory used during simulation with third-party simulators. The `<arg>` is the path name to the compiled library directory. The library must be compiled ahead of time using the `compile_simlib` command as explained in the *Vivado Design Suite Tcl Command Reference Guide* ([UG835](#)).

```
cosim.compiled_library_dir=../../simLib
```

- **cosim.coverage:**

Enables the coverage feature during simulation with the VCS simulator.

```
cosim.coverage=true
```

- **cosim.disable_binary_tv :**

Disables the binary test vector format in co-simulation.

```
cosim.disable_binary_tv=true
```

- **cosim.disable_deadlock_detection:**

Disables deadlock detection, and opening the [Cosim Deadlock Viewer](#) in co-simulation.

```
cosim.disable_deadlock_detection=true
```

- **cosim.disable_dependency_check:**

Disables dependency checks when running co-simulation.

```
cosim.disable_dependency_check=true
```

- **cosim.enable_dataflow_profiling:**

This option enables the dataflow channel profiling to track channel sizes during co-simulation. You must enable this feature to capture dataflow data as described in the Dataflow viewer section of the Vitis HLS User Guide ([UG1399](#)).

```
cosim.enable_dataflow_profiling=true
```

- **cosim.enable_fifo_sizing:**

Enables automatic FIFO channel size tuning for dataflow profiling during co-simulation.

```
cosim.enable_fifo_sizing=true
```

- **cosim.enable_tasks_with_m_axi:**

Enables stable `m_axi` interfaces for use with `hls::task`.

```
cosim.enable_tasks_with_m_axi=true
```

- **cosim.hwemu_trace_dir:**

Specifies the location of test vectors generated during hardware emulation to be used as a test bench during co-simulation. The test vectors are generated by the `syn.rtl.cosim_trace_generation` command as described in [RTL Configuration](#). This argument lets you specify the kernel and instance name of the Vitis kernel in the hardware emulation simulation results to locate the test vectors for the HLS component.

```
cosim.hwemu_trace_dir=../../dct/dct_2
```

- **cosim.ldflags <arg>:**

Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.

```
cosim.ldflags=ldExample
```

- **cosim.mflags <arg>:**

Provides for options to be passed to the compiler for C simulation. This is typically used to speed up compilation.

```
cosim.mflags=mExample
```

- **cosim.random_stall:**

Enable random stalling of top level interfaces during co-simulation.

```
cosim.random_stall=true
```

- **cosim.rtl:**

Specifies either Verilog or VHDL as the language to use for C/RTL co-simulation. The default is Verilog.

```
cosim.rtl=vhdl
```

- **cosim.setup:**

When this option is specified, the simulation binary will be created in the `cosim` directory of the current HLS component, but simulation will not be executed. Simulation can be launched later from the compiled executable. The default is false, and co-simulation is run after setup is complete.

```
cosim.setup=true
```

- **cosim.stable_axilite_update:**

Enable `s_axilite` to configure registers which are stable compared with the prior transaction.

```
cosim.stable_axilite_update=true
```

- **cosim.tool :**

Specify the HDL simulator to be used to co-simulate the RTL with the C testbench. The Vivado simulator (`xsim`) is the default, unless otherwise specified.

- `auto`
- `vcs`
- `modelsim`
- `riviera`
- `isim`
- `xsim`
- `ncsim`
- `xceillum`

```
cosim.tool=modelsim
```

- **cosim.trace_level:**

Determines the level of waveform trace data to save during C/RTL co-simulation.

- `none` does not save trace data. This is the default.
- `all` results in all port and signal waveforms being saved to the trace file.
- `port` only saves waveform traces for the top-level ports.
- `port_hier` save the trace information for all ports in the design hierarchy.

```
cosim.trace_level=port
```

The trace file is saved in the `sim/Verilog` or `sim/VHDL` folder of the component when the simulation executes, depending on the selection used with the `cosim.rtl` option.

- **cosim.user_stall:**

Specifies the JSON stall file to be used during co-simulation. The stall file can be generated using the `cosim_stall` command.

```
cosim.user_stall=../../stall.json
```

- **cosim.wave_debug:**

Opens the Vivado simulator GUI to view waveforms and simulation results. Enables waveform viewing of all processes in the generated RTL, as in the dataflow and sequential processes. This option is only supported when using Vitis simulator for co-simulation by setting `cosim.tool=xsim`. See [Viewing Simulation Waveforms](#) for more information.

```
cosim.wave_debug=true
```

Compile Options

The `syn.compile` commands specify the default behavior for compilation of the HLS component.

- **syn.compile.design_size_max_warning:** Specifies the design size that triggers a warning related to slow compilation or poor QoR.

```
syn.compile.design_size_maximum_warning=300000
```

- **syn.compile.enable_auto_rewind:**

When true enables an alternative HLS implementation for pipelined loops which uses automatic loop rewind as described in the Rewinding Pipelined Loops for Performance section of the Vitis HLS User Guide ([UG1399](#)) for additional information.

```
syn.compile.enable_auto_rewind=1
```

- **syn.compile.ignore_long_run_time:**

Disable long run time warning.

```
syn.compile.ignore_long_run_time=1
```

- **syn.compile.name_max_length <arg>:**

The `name_max_length` option will specify the maximum length of function names. If the length is over the threshold, the last part of the name will be truncated.

```
syn.compile.name_max_length=13
```

- **syn.compile.no_signed_zeros:**

The `no_signed_zeros` option will ignore the signedness of floating point zero so that compiler can do aggressive optimizations on floating point operations.

```
syn.compile.no_signed_zeros=1
```

- **syn.compile.pipeline_flush_in_task arg:** Specifies that pipelines in `hls::tasks` will be flushing (`flp`) by default to reduce the probability of deadlocks in C/RTL Co-simulation. This option applies to pipelines that achieve an `II=1` with the default option of `ii1`. However, you can also specify it as applying `always` to enable flushing pipelines in either `hls::tasks` or dataflow, or can be completely disabled using `never`. Refer to Flushing Pipelines and Pipeline Types in *Vitis High-Level Synthesis User Guide* ([UG1399](#)) for more information.



IMPORTANT! Flushing pipelines (`flp`) are compatible with the `rewind` option specified in the `PIPELINE` pragma or directive when the `syn.compile.enable_auto_rewind` option is also specified.

- **always:** Always make pipelines flushable in `hls::tasks` or dataflow regardless of `II`.
- **never:** Never make pipeline flushable unless specifically overridden by other directives or pragmas.
- **ii1:** Make pipelines that achieve `II=1` flushable in `hls::tasks`. This is the default setting.

```
syn.compile.pipeline_flush_in_task=always
```

- **syn.compile.pipeline_loops arg:**

Loops with a tripcount equal to or greater than the specified value will be pipelined automatically.

```
syn.compile.pipeline_loops=20
```

- **syn.compile.pipeline_style arg:**

Set default pipeline style, this is a preference not a hard constraint. Refer to Flushing Pipelines and Pipeline Types in *Vitis High-Level Synthesis User Guide* ([UG1399](#)) for more information. The three styles are: stallable (`stp`), flushable (`flp`), and free-running (`frp`). The default is free-running.

```
syn.compile.pipeline_style=flp
```

- **syn.compile.pragma_strict_mode:**

Enable errors instead of warnings for unrecognized and improper pragma syntax.

```
syn.compile.pragma_strict_mode=1
```

- **syn.compile.unsafe_math_optimizations:**

The `unsafe_math_optimizations` option will ignore the signedness of floating point zero and enable associative floating point operations so that compiler can do aggressive optimizations on floating point operations.

```
syn.compile.unsafe_math_optimizations=1
```




IMPORTANT! Using this option might change the result of any floating point calculations and result in a mismatch in C/RTL co-simulation. Ensure your test bench is tolerant of differences and checks for a margin of difference, not exact values.

Dataflow Configuration

Synthesis: Dataflow Options

The `syn.dataflow` commands configure the dataflow analysis for the whole design. These settings specify the default memory channel and FIFO depth used by `syn.directive.dataflow`.

- **syn.dataflow.default_channel:**

By default a RAM memory configured in ping-pong fashion is used to buffer the data between functions or loops when dataflow pipelining is used. When streaming data is used (where the data is always read and written in consecutive order), a FIFO memory will be more efficient and can be selected as the default memory type.

The available channels are `fifo` and `pingpong`. The default is `pingpong`.

```
syn.dataflow.default_channel=fifo
```



TIP: Arrays must be set to streaming using the `set_directive_stream` command to perform FIFO accesses.

- **syn.dataflow.disable_fifo_sizing_opt:**

Disable FIFO sizing optimizations that increase resource usage; this can improve performance and reduce deadlocks.

```
syn.dataflow.disable_fifo_sizing_opt=1
```

- **syn.dataflow.fifo_depth:**

An integer value specifying the default depth of FIFOs. This option has no effect when `pingpong` memories are used. By default the FIFOs depth used in the channel will be set to the size of the largest producer or consumer (whichever is largest). In some cases this approach can be too conservative and result in FIFOs which are larger than needed. This option can be used to specify the depth when you know the FIFOs are larger than required.

```
syn.dataflow.fifo_depth=6
```



IMPORTANT! Be careful when using this option as incorrect use can result in a design which fails to operate correctly

- **syn.dataflow.override_user_fifo_depth:**

Specify a depth for every `hls::stream`, overriding any user settings.

```
syn.dataflow.override_user_fifo_depth=12
```

This is useful for checking if a deadlock is due to insufficient FIFO depths in the design. By setting the override to a very large value (for example, the maximum depth printed by co-simulation at the end of simulation), if there is no deadlock, then you can use the FIFO depth profiling options of co-simulation and the GUI to find the minimum depth that ensures performance and avoids deadlocks.

- **syn.dataflow.scalar_fifo_depth:**

An integer value specifying the minimum depth of the scalar value propagation FIFOs as described in the Specifying Compiler-Created FIFO Depth section of the *Vitis High-Level Synthesis User Guide* ([UG1399](#)). These FIFOs are used to forward the value of scalar arguments of the dataflow regions to processes which have predecessors in the region itself. They do not affect functional correctness, but an insufficient automatically computed size can result in loss of performance and even deadlock.

```
syn.dataflow.scalar_fifo_depth=4
```

When this option is not specified, the minimum depth is the value of the `syn.dataflow.fifo_depth` option, or it is 2. As a rule of thumb, a good value is the average number of times the process forwarding the scalar value can start before the last process that reads it starts.

- **syn.dataflow.start_fifo_depth:**

An integer value specifying the minimum depth of the start propagation FIFOs as described in [Specifying Compiler-Created FIFO Depth](#). These FIFOs are used to forward the `ap_start` handshake signal to processes which have predecessors in the region. They do not affect functional correctness, but an insufficient automatically computed size can result in loss of performance.

```
syn.dataflow.start_fifo_depth=5
```

When this option is not specified, the minimum depth is the value of the `syn.dataflow.fifo_depth` option, or it is 2. As a rule of thumb, a good value is the expected average number of times a process should be allowed to start in advance compared to its successors.

- **syn.dataflow.strict_mode:**

Set severity for dataflow canonical form messages. The available modes are: `error`, `warning`, `off`. The default is `warning`.

```
syn.dataflow.strict_mode=error
```

- **syn.dataflow.strict_stable_sync:**

Force synchronization of stable ports with `ap_done`. Refer to the Stable Arrays section of the Vitis HLS User Guide ([UG1399](#)) for more information.

```
syn.dataflow.strict_stable_sync=1
```

- **syn.dataflow.task_level_fifo_depth:**

Default task-level FIFO depth (used for FIFOs automatically created to transfer scalars between processes). A FIFO is synchronized by `ap_ctrl_chain`. The write is the `ap_done` of the producer, the read is the `ap_ready` of the consumer. Like a PIPO in terms of synchronization, and like a FIFO in terms of access.

```
syn.dataflow.task_level_fifo_depth=7
```

Debug Options

The `syn.debug` commands enable debugging in the HLS component and specify where to write debugging output.

- **syn.debug.enable:**

Enable debug file generation. When not enabled the HLS component can be optimized during compilation, however it will not support debug.

```
syn.debug.enable=1
```



TIP: This option relates to the `v++ -c -g` option, and must be manually enabled in the HLS component when debugging is needed at the Application level.

- **syn.debug.directory:**

Specifies the location of to write the output of HLS debugging. When not specified the location is set to `hls/.debug`.

```
syn.debug.directory=../../debug
```

Interface Configuration

The `syn.interface` commands configure the default interface settings applied to the HLS component. These settings can be overridden for specific top-level interface ports using `syn.directive.interface`.

- **syn.interface.clock_enable:**

Add a clock-enable port (`ap_ce`) to the design. The clock enable prevents all clock operations when it is active low, and disables all sequential operations. The default is false.

```
syn.interface.clock_enable=1
```

- **syn.interface.default_slave_interface:**

Specify the default slave interface. The two modes are `s_axilite` and `none`. The default is `s_axilite`.

```
syn.interface.default_slave_interface=none
```

- **syn.interface.m_axi_addr64:**

Enable 64-bit addressing for all `m_axi` interfaces. This is enabled by default. When disabled, the `m_axi` interfaces uses 32-bit addressing.

```
syn.interface.m_axi_addr64=1
```

- **syn.interface.m_axi_alignment_byte_size:**

Specify default alignment byte size for all `m_axi` interfaces. The default when this option is not specified is 64 bytes for Vitis kernel flow, or 1 byte for Vivado IP flow as described in the Default of Vivado/Vitis Flows section of the Vitis HLS User Guide ([UG1399](#)).

```
syn.interface.m_axi_alignment_byte_size=16
```



TIP: A value of 0 is not valid.

- **syn.interface.m_axi_auto_id_channel:**

Enable automatic assignment of channel IDs for `m_axi` interfaces. This is disabled by default. Refer to the AXI4 Master Interface section of the Vitis HLS User Guide ([UG1399](#)) for additional information.

```
syn.interface.m_axi_auto_id_channel=1
```

- **syn.interface.m_axi_auto_max_ports:**

Enable automatic creation of separate `m_axi` interface adapters for each argument or port on the interface. This is disabled by default, reducing the `m_axi` interfaces to minimum needed. Refer to the section M_AXI Bundles of the Vitis HLS User Guide ([UG1399](#)) for more information.

```
syn.interface.m_axi_auto_max_ports=1
```

- **syn.interface.m_axi_buffer_impl:**

Specify the implementation resource for all buffers internal to the `m_axi` adapters. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `bram`.

```
syn.interface.m_axi_buffer_impl=lutram
```

- **syn.interface.m_axi_cache_impl:**

Specify the implementation resource for cache added to the `m_axi` adapters. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `auto`.

```
syn.interface.m_axi_cache_impl=lutram
```

- **syn.interface.m_axi_conservative_mode:**

Configure all `m_axi` adapters to work in conservative mode, waiting to issue a write request until the associated write data is entirely available (typically, buffered into the adapter or already emitted). It uses a buffer inside the MAXI adapter to store all the data for a burst (both in case of reading and writing). This feature is enabled by default, and might slightly increase write latency but can resolve deadlock due to concurrent requests (read or write) on the memory subsystem. Disable conservative mode by setting it to `false`.

```
syn.interface.m_axi_conservative_mode=0
```

- **syn.interface.m_axi_flush_mode:**

Configure all `m_axi` adapters to be flushable, writing or reading garbage data if a burst is interrupted due to pipeline blocking (missing data inputs when not in conservative mode or missing output space). This is disabled by default.

```
syn.interface.m_axi_flush_mode=1
```

- **syn.interface.m_axi_latency:**

Globally specify the expected latency of the `m_axi` interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected. The default value is 64 for the Vitis Kernel flow, and 0 for the Vivado IP flow, as described in the section Defaults of Vivado and Vitis Flows in the Vitis HLS User Guide ([UG1399](#)).

```
syn.interface.m_axi_latency=5
```

- **syn.interface.m_axi_max_bitwidth:**

Specifies the maximum bitwidth for the `m_axi` interfaces data channel. The default is 1024 bits. The specified value must be a power-of-two, between 8 and 1024. This decreases throughput if the actual accesses are bigger than the required interface, as they will be split into a multi-cycle burst of accesses.

```
syn.interface.m_axi_max_bitwidth=128
```

- **syn.interface.m_axi_max_read_burst_length:**

Specifies a global maximum number of data values read during a burst transfer for all `m_axi` interfaces. The default is 16.

```
syn.interface.m_axi_max_read_burst_length=12
```

- **syn.interface.m_axi_max_widen_bitwidth:**

Enable automatic port width resizing to widen bursts for the `m_axi` interface, up to the chosen bitwidth. The specified value must be a power of 2 between 8 and 1024, and must align with the `-m_axi_alignment_size`. The default value is 512 for the Vitis Kernel flow, and 0 for the Vivado IP flow.

```
syn.interface.m_axi_max_widen_bitwidth=64
```

- **syn.interface.m_axi_max_write_burst_length:**

Specifies a global maximum number of data values written during a burst transfer for all `m_axi` interfaces. The default is 16.

```
syn.interface.m_axi_max_write_burst_length=12
```

- **syn.interface.m_axi_min_bitwidth:**

Specifies the minimum bitwidth for `m_axi` interfaces data channel. The default is 8 bits. The value must be a power of 2, between 8 and 1024. This does not necessarily increase throughput if the actual accesses are smaller than the required interface.

```
syn.interface.m_axi_min_bitwidth=64
```

- **syn.interface.m_axi_num_read_outstanding:**

Specifies how many read requests can be made to the `m_axi` interface without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

```
num_read_outstanding*max_read_burst_length*word_size
```

The default value is 16.

```
syn.interface.m_axi_num_read_outstanding=8
```

- **syn.interface.m_axi_num_write_outstanding:**

Specifies how many write requests can be made to the `m_axi` interface without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

```
num_write_outstanding*max_write_burst_length*word_size
```

The default value is 16.

```
syn.interface.m_axi_num_write_outstanding=8
```

- **syn.interface.m_axi_offset:**

Specify the default offset mechanism for all `m_axi` interfaces. The options are:

- `off`: No offset port is generated.
- `slave`: Generates an offset port and automatically maps it to an `s_axilite` interface. This is the default value.
- `direct`: Generates a scalar input offset port for directly passing the address offset into the IP through the offset port.

```
syn.interface.m_axi_offset=slave
```

- **syn.interface.register_io:**

Globally enables registers for all scalar inputs, outputs, or all scalar ports on the top function (arrays are always registered). The options are `off`, `scalar_in`, `scalar_out`, `scalar_all`. The default is `off`.

```
syn.interface.register_io=scalar_out
```

- **syn.interface.s_axilite_auto_restart_counter:**

Enables the auto-restart behavior for kernels. Use 1 to enable the auto-restart feature, or 0 to disable it which is the default. When enabled, the tool establishes the auto-restart bit in the `ap_ctrl_chain` control protocol for the `s_axilite` interface. For more information refer to Auto-Restarting Mode in the Vitis HLS User Guide ([UG1399](#)).

```
syn.interface.s_axilite_auto_restart_counter=1
```

- **syn.interface.s_axilite_data64:**

Enable 64 bit data width for `s_axilite` interface. Use 1 to enable 64 bit data width and 0 to disable it. The default is 32 bit data width.

```
syn.interface.s_axilite_data64=1
```

- **syn.interface.s_axilite_interrupt_mode:**

Specify the interrupt mode for `s_axilite` interface to be Clear on Read (`cor`) or Toggle on Write (`tow`). Clear on Read interrupt can be completed in a single transaction, while `tow` requires two. `Tow` is the default interrupt mode.

```
syn.interface.s_axilite_interrupt_mode=cor
```

- **syn.interface.s_axilite_mailbox:** Enables the creation of a mailbox for non-stream non-stable `s_axilite` arguments. The mailbox feature is used in the setting and management of auto-restart kernels as described in the Auto-Restarting Mode section in the Vitis HLS User Guide (UG1399). The available options are:
 - `in`: Enable mailbox for only input arguments
 - `out`: Enable mailbox for only output arguments
 - `both`: Enable mailbox for input and output arguments
 - `none`: No mailbox created. This is the default value.

```
syn.interface.s_axilite_mailbox=in
```

- **syn.interface.s_axilite_status_regs:**

Enables exposure of ECC error bits in the `s_axilite` register file via two clear-on-read (COR) counters per BRAM or URAM with ECC enabled. Options are `ecc` to enable or `off` to disable the feature. The default is disabled.

```
syn.interface.s_axilite_status_regs=ecc
```

- **syn.interface.s_axilite_sw_reset:**

Enable SW reset in `s_axilite` adapter.

```
syn.interface.s_axilite_sw_reset=1
```

Package Options

Output Options

The `package.output` options are used to specify the nature of the exported files generated from the synthesized RTL design. These options include the following:

- **package.output.file:** Specifies the output file path and name for the exported file. When not specified the Vitis unified IDE will name the output file after the HLS component. An example command would be:

```
package.output.file=../kernel.xo
```
- **package.output.format:** Specifies the exported format of the output generated after synthesis. The supported values are:
 - `package.output.format=ip_catalog`: A format suitable for adding to the Vivado IP catalog.
 - `package.output.format=xo`: A format accepted by the `v++` compiler for linking in the Vitis application acceleration flow.

- `package.output.format=syn_dcp`: Synthesized checkpoint file for Vivado Design Suite. If this option is used, RTL synthesis is automatically executed. Vivado implementation can be optionally added using `vivado.flow=impl`
- `package.output.format=sysgen`: Generate an IP and .zip archive for use in System Generator.
- `package.output.format=rtl`: Outputs the RTL files that are generated by C synthesis, and does not package the IP or XO used for downstream processes.



TIP: The RTL format is useful for development, analysis, and debug of the HLS component. However, to export files you must specify one of the other output formats.

- **package.output.syn:** Enables the running of the Package step as part of the C synthesis step to generate the required export files during synthesis.

IP Options

The `package.ip` commands specify details of the IP being generated by the HLS component. These are settings to define the IP Vendor, Library, Name, and Version (VLNV) for example to identify it in the IP catalog.

- **package.ip.description:**

Provides a description for the catalog entry for the generated IP, used when packaging the IP.

```
package.ip.description=details of IP
```

- **package.ip.display_name:**

Provides a display name for the catalog entry for the generated IP, used when packaging the IP.

```
package.ip.display_name=randy1
```

- **package.ip.library:**

Provides the library component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.

```
package.ip.library=testLib
```

- **package.ip.name:**

Provides the name component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for the generated IP.

```
package.ip.name=randy1
```

- **package.ip.taxonomy:**

Specifies the taxonomy for the catalog entry for the generated IP, used when packaging the IP. Taxonomy is a category to help identify the purpose or application of the IP.

```
package.ip.taxonomy=video
```

- **package.ip.vendor:**

Provides the vendor component of the <Vendor>:<Library>:<Name>:<Version> (VLNV) identifier for generated IP.

```
package.ip.vendor=randyCom
```

- **package.ip.version:**

Provides the version component of the <Vendor>:<Library>:<Name>:<Version> (VLNV) identifier for generated IP.

```
package.ip.version=2.3
```

- **package.ip.xdc_file:**

Specify an XDC file whose contents will be included in the packaged IP for use during implementation in the Vivado tool.

```
package.ip.xdc_file=../../ip.xdc
```

- **package.ip.xdc_ooc_file:**

Specify an out-of-context (OOC) XDC file whose contents will be included in packaged IP and used during out-of-context Vivado synthesis for the exported IP.

```
package.ip.xdc_ooc_file=../../ooc.xdc
```

Operator Configuration

The `syn.op` command lets you configure the default implementation style, latency, and precision for different operators used for the HLS component. You can add multiple `syn.op` commands to a config file to specify the details of different operators. If an operator is not specified then the tool determines the default values for the component.

You can override the default settings specified by the `syn.op` command by using the [syn.directive.bind_op](#) command for specific variables.

- **syn.op:**

The syntax for the `syn.op` command is as follows:

```
syn.op=op:mul impl:dsp
syn.op=op:add impl:fabric latency:6
syn.op=op:fmacc precision:high
syn.op=op:hdiv latency:5
```

- `syn.op=`: Starts the command
- `op:<operator>`: Specifies the `op` keyword followed by the operator being defined. The supported operators include:
 - `mul`: integer multiplication operation
 - `add`: integer add operation
 - `sub`: integer subtraction operation
 - `fadd`: single precision floating-point add operation
 - `fsub`: single precision floating-point subtraction operation
 - `fdiv`: single precision floating-point divide operation
 - `fexp`: single precision floating-point exponential operation
 - `flog`: single precision floating-point logarithmic operation
 - `fmul`: single precision floating-point multiplication operation
 - `frsqrt`: single precision floating-point reciprocal square root operation
 - `frecip`: single precision floating-point reciprocal operation
 - `fsqrt`: single precision floating-point square root operation
 - `dadd`: double precision floating-point add operation
 - `dsub`: double precision floating-point subtraction operation
 - `ddiv`: double precision floating-point divide operation
 - `dexp`: double precision floating-point exponential operation
 - `dlog`: double precision floating-point logarithmic operation
 - `dmul`: double precision floating-point multiplication operation
 - `drsqrt`: double precision floating-point reciprocal square root operation
 - `drecip`: double precision floating-point reciprocal operation
 - `dsqrt`: double precision floating-point square root operation
 - `hadd`: half precision floating-point add operation

- `hsub`: half precision floating-point subtraction operation
- `hdiv`: half precision floating-point divide operation
- `hmul`: half precision floating-point multiplication operation
- `hsqrt`: half precision floating-point square root operation
- `facc`: single precision floating-point accumulate operation
- `fmacc`: single precision floating-point multiply-accumulate operation
- `fmadd`: single precision floating-point multiply-add operation



TIP: Comparison operators, such as `dcmp`, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the `syn.op` or `syn.directive.bind_op` commands.

- `impl:<value>`: Specifies the implementation (`impl`) keyword followed by the value for the specified operator. When `impl` is not specified, the default is for the tool to determine the best implementation for a given operator. Supported values include:
 - `all`: All implementations. This is the default setting.
 - `dsp`: Use DSP resources
 - `fabric`: Use non-DSP resources
 - `meddsp`: Floating Point IP Medium Usage of DSP resources
 - `fulldsp`: Floating Point IP Full Usage of DSP resources
 - `maxdsp`: Floating Point IP Max Usage of DSP resources
 - `primitivedsp`: Floating Point IP Primitive Usage of DSP resources
 - `auto`: enable inference of combined `facc` | `fmacc` | `fmadd` operators
 - `none`: disable inference of combined `facc` | `fmacc` | `fmadd` operators
- `latency:<value>`: Specifies the `latency` keyword followed by the value. Defines the default latency for the binding of the operator to the implementation resource. The valid value range varies for each implementation (`impl`) of the operation. The default is -1, which lets the tool apply the standard latency for the implementation resource.



TIP: You can specify the latency for a specific operation without specifying the implementation. This leaves the tool free to choose the implementation while managing the latency.

- `precision:<value>`: Used for floating point operators (`facc`, `fmacc`, and `fmadd`), this specifies the `precision` keyword followed by one of the following:

- **low**: Use a low precision (60 bit and 100 bit integer) accumulation implementation when available. This option is only available on certain non-AMD Versal™ devices, and can cause RTL/Co-Sim mismatches due to insufficient precision with respect to C++ simulation. However, it can always be pipelined with an `II=1` without source code changes, though it uses approximately 3X the resources of `standard` precision floating point accumulation.
- **high**: Use high precision (one extra bit) fused multiply-add implementation when available. This option is useful for high-precision applications and is very efficient on Versal devices, although it can cause RTL/Co-Sim mismatches due to the extra precision with respect to C++ simulation. It uses more resources than `standard` precision floating point accumulation.
- **standard**: standard precision floating point accumulation and multiply-add is suitable for most uses of floating-point, and is the default setting. It always uses a true floating-point accumulator that can be pipelined with `II=1` on Versal devices, and `II` that is typically between 3 and 5 (depending on clock frequency and target device) on non-Versal devices.

RTL Configuration

The `syn.rtl` commands configure various attributes of the compiled RTL, the type of reset used, and the encoding of the state machines. It also allows you to use specific identification in the RTL. By default, these options are applied to the top-level design and all RTL blocks within the design.

- **syn.rtl.cosim_trace_generation:**

Generate test vectors during hardware emulation in the Vitis tool flow when the kernel is synthesized as a Vitis kernel, to be used as a test bench for C/RTL Co-simulation in future design iterations.

```
syn.rtl.cosim_trace_generation=1
```

- **syn.rtl.deadlock_detection:**

Enables simulation or synthesis deadlock detection in the top-level RTL of an exported IP/XO file. The options are as follows:

- **none** : Deadlock detection disabled
- **sim** : Enables deadlock detection only for simulation/emulation. This is the default setting.
- **hw** : Deadlock detection enabled in synthesized RTL IP. Adds `ap_local_deadlock` and `ap_local_block` signals to the IP to enable local and global deadlock detection.

```
syn.rtl.deadlock_detection=hw
```

- **syn.rtl.deadlock_diagnosis:**

Enable deadlock detection diagnosis for Vitis kernels (.xo) during hardware emulation of the Application.

```
syn.rtl.deadlock_diagnosis=1
```

- **syn.rtl.header:**

Specify a file whose contents will be inserted at the beginning of all generated RTL files. This allows you to ensure that the generated RTL files contain user specified content.

```
syn.rtl.header=../../myHeader.txt
```

- **syn.rtl.kernel_profile:**

Add top level event and stall ports required by kernel profiling.

```
syn.rtl.kernel_profile=1
```



IMPORTANT! This option relates to the `v++ -c --profile.stall` command, and must be manually added to the HLS component to ensure the stall profiling is available for use in the linked Application.

- **syn.rtl.module_auto_prefix:**

Specifies the top level function name as the prefix value for generated RTL modules. This option is ignored if `syn.rtl.module_prefix` is also specified. This is enabled by default.

```
syn.rtl.module_auto_prefix=1
```

- **syn.rtl.module_prefix:**

Specify a prefix to be used for all generated RTL module names. Use this to override the default module prefix of the top-level function.

```
syn.rtl.module_prefix=newTop
```

- **syn.rtl.mult_keep_attribute:**

Enable keep attribute.

```
syn.rtl.mult_keep_attribute=1
```

- **syn.rtl.register_all_io:**

Use a register by default for all I/O signals.

```
syn.rtl.register_all_io=1
```

- **syn.rtl.register_reset_num:**

Number of registers to add to reset signal.

```
syn.rtl.register_reset_num=2
```

- **syn.rtl.reset:** Variables initialized in the C/C++ code are always initialized to the same value in the RTL and therefore in the bitstream. This initialization is performed only at power-on. It is not repeated when a reset is applied to the design.

The setting applied with the `-reset` option determines how registers and memories are reset.

- `none`: No reset is added to the design.
- `control`: Resets control registers, such as those used in state machines and those used to generate I/O protocol signals. This is the default setting.
- `state`: Resets control registers and registers or memories derived from static or global variables in the C/C++ code. Any static or global variable initialized in the C/C++ code is reset to its initialized value.
- `all`: Resets all registers and memories in the design. Any static or global variable initialized in the C/C++ code is reset to its initialized value.

```
syn.rtl.reset=state
```

- **syn.rtl.reset_async:**

Causes all registers to use an asynchronous reset. If this option is not specified a synchronous reset is used.

```
syn.rtl.reset_async=1
```

- **syn.rtl.reset_level:**

Defines the polarity of the reset signal to be either active-Low or active-High. The default setting is active-High.

```
syn.rtl.reset_level=low
```



TIP: The AXI protocol requires an active-Low reset. If your design uses AXI interfaces the tool will define this reset level with a warning if the `syn.rtl.reset_level` is active-High.

Schedule Setting

The `syn.schedule` command configures the default type of scheduling performed.

- **syn.schedule.enable_dsp_full_reg:**

Specifies that the DSP signals should be fully registered. This is enabled by default.

```
syn.schedule.enable_dsp_full_reg=0
```

Storage Configuration

Sets the global default options for the HLS micro-architecture binding of FIFO storage elements to memory resources.

The default configuration defined by `syn.storage` for FIFO storage can be overridden by [syn.directive.bind_storage](#) for a specific design element, or specifying the `storage_type` option for [syn.directive.interface](#) for objects on the interface.

- **syn.storage:**

The syntax for the `syn.storage` command is as follows:

```
syn.storage=fifo impl=auto auto_srl_max_bits=512 auto_srl_max_depth=3
```

- `syn.storage=fifo`: Starts the command to configure FIFOs.
Note: FIFO is the only type supported at this time.
- `impl=<value>`: Specifies the implementation (`impl`) keyword followed by the value. When `impl` is not specified, the default is `auto`, allowing the tool to determine the best implementation for a given operator. Supported values include: `auto`, `bram`, `lutram`, `uram`, `memory`, `srl`
- `auto_srl_max_bits=<value>`: Only valid when for `impl:auto` (the default). Specifies the maximum allowed SRL total bits (depth * width) for auto implementations. The default is 1024.
- `auto_srl_max_depth=<value>`: Specifies the maximum allowed SRL depth for auto-srl implementation. The default is 2.

Implementation Configuration

The `syn.vivado` commands configure the Vivado synthesis and implementation runs used to derive resource utilization and timing estimates. These settings generally do not affect the RTL created for the HLS component, but can affect the example hardware design used for estimates.

- **vivado.clock:**

Override the HLS clock constraint used in the Vivado out-of-context run for determination of timing analysis. This does not affect the output IP or XO files.

```
vivado.clock=37
```

- **vivado.flow:**

Run the Vivado flow for synthesis only (`syn`), or for implementation (`impl`). Running synthesis will be faster than running implementation, but will lack some of details of the implementation run. The default setting is `impl`.

```
vivado.flow=syn
```



TIP: This is different from the `flow_target` option that defines the target as being either the Vitis kernel flow or the Vivado IP flow.

- **vivado.impl_strategy:**

Specify a Vivado synthesis or implementation strategy. Strategy names can be found in *Defining Implementation Strategies in the Vivado Design Suite User Guide: Implementation (UG904)*. This is used for resource usage and timing analysis and will not affect the output files.

```
vivado.impl_strategy=Performance_Explore
```

- **vivado.max_timing_paths:**

Specify the max number of timing paths to report when the timing is not met in the Vivado synthesis or implementation.

```
vivado.max_timing_paths=12
```

- **vivado.optimization_level:**

Vivado optimization level, sets other `vivado_*` options. This will not apply to IP/XO output. The choices are 0, 1, 2, 3. This only applies for report generation and will not apply to the exported IP. The default setting is 0.

```
vivado.optimization_level=2
```

- **vivado.pblock:**

Specify a Pblock range to use during implementation for reporting purposes.

```
vivado.pblock={SLICE_X8Y105:SLICE_X23Y149}
```

- **vivado.phys_opt:**

Run Vivado physical optimization at the specified implementation stage. The choices are no optimization (`none`), placement optimization (`place`), routing optimization (`route`), or both placement and routing (`all`). This option only applies to the implementation run used to generate resource and timing estimates.

```
vivado.phys_opt=route
```

- **vivado.report_level:**

Specify the report level for Vivado synthesis and implementation run. The valid values and the associated reports are:

- 0: Post-synthesis utilization. Post-implementation utilization and timing.
- 1: Post-synthesis utilization, timing, and analysis. Post-implementation utilization, timing, and analysis.
- 2: Post-synthesis utilization, timing, analysis, and failfast. Post-implementation utilization, timing, and failfast. This is the default setting.

```
vivado.report_level=1
```

- **vivado.rtl:**

Specifies RTL language (`verilog/vhdl`) to select in Vivado flow.

```
vivado.rtl=vhdl
```

- **vivado.synth_design_args:**

Specifies arguments for the Vivado `synth_design` command. Available `synth_design` arguments can be found in the *Vivado Design Suite Tcl Command Reference Guide* ([UG835](#)).

```
vivado.synth_design_args=arg1 arg2
```

- **vivado.synth_strategy:**

Specifies a Vivado synthesis strategy to use when generating the example design to use for resource and timing estimates. Specific synthesis strategies can be found in *Vivado Design Suite User Guide: Synthesis* ([UG901](#)).

```
vivado.synth_strategy=Synthesis_Defaults
```

Unroll Setting

The `syn.unroll` setting provides a global tripcount threshold below which loops are automatically unrolled. Loops with a greater tripcount can be unrolled using `syn.directive.unroll`.

- **syn.unroll.tripcount_threshold:**

All loops which have fewer iterations than the specified value are automatically unrolled. The default value is 0, which means that loops are not automatically unrolled.

```
syn.unroll.tripcount_threshold=6
```

HLS Optimization Directives



IMPORTANT! The following options must appear in the HLS configuration file under the `[hls]` header.

Directives, or the `syn.directive.xxx` commands allow you to customize the synthesis results for the same source code across multiple implementations. Change the directives to change the results. The `syn.directive.xxx` commands are intended for use in the config files associated with the new Vitis IDE as described in Building and Running an HLS Component in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

Note: You can also use pragmas in your source code, rather than directives in your config file. This will have the same result, but also have the added advantage of being stored directly in your source code. Refer to the HLS Pragmas section of the Vitis HLS User Guide ([UG1399](#)) for more information.

Directives applied through a configuration file must include a `<location>` as an argument to the directive. The `<location>` defines what element of the source code the directive applies to, such as function, loop, region, or variable.

The example syntax for `syn.directive.xxx` commands include the location and the arguments for the directive. For example:

```
syn.directive.pipeline=dct2d II=4
```

Where `dct2d` is the location (function name) to apply the PIPELINE directive to, and `II=4` is one of the possible arguments to the directive.

syn.directive.aggregate

Description

This directive collects the data fields of a struct into a single wide scalar. Any arrays declared within the struct and Vitis HLS performs a similar operation as `syn.directive.array_reshape`, and completely partitions and reshapes the array into a wide scalar and packs it with other elements of the struct.



TIP: Arrays of structs are restructured as arrays of aggregated elements. The optimization does not support structs that contain other structs.

The bit alignment of the resulting new wide-word can be inferred from the declaration order of the struct elements. The first element takes the least significant sector of the word and so forth until all fields are mapped.

Syntax

```
syn.directive.aggregate=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable which will be packed.
- `<variable>` is the struct variable to be packed.

Options

- `compact=[bit | byte | none | auto]`: Specifies the alignment of the aggregated struct. Alignment can be on the bit-level (packed), the byte-level (padded), none, or automatically determined by the tool which is the default behavior.

Examples

The following example aggregates the variable `AB` which is a struct pointer containing three 8-bit fields (`typedef struct {unsigned char R, G, B;}pixel`) in function `func`, into a new 24-bit pointer, aligning data at the bit-level.

```
syn.directive.aggregate=func AB compact=bit
```

See Also

- [syn.directive.array_reshape](#)
- [syn.directive.disaggregate](#)

syn.directive.alias

Description

Specify that two or more `M_AXI` pointer arguments point to the same underlying buffer in memory (DDR or HBM) and indicate any aliasing between the pointers by setting the distance or offset between them.



IMPORTANT! *syn.directive.alias* applies to top-level function arguments mapped to `M_AXI` interfaces.

Vitis HLS considers different pointers to be independent channels and generally does not provide any dependency analysis. However, in cases where the host allocates a single buffer for multiple pointers, this relationship can be communicated through `syn.directive.alias` and dependency analysis can be maintained. This enables data dependence analysis in Vitis HLS by defining the distance between pointers in the buffer.

Requirements for `syn.directive.alias`:

- All ports must be assigned to `M_AXI` interfaces and also assigned to different bundles, as shown in the example below
- Each port can only be used in one `syn.directive.alias` command

- All ports assigned to `syn.directive.alias` must have the same depth
- When offset is specified, each port can have one offset
- The offset for the `M_AXI` port must be specified as `slave` or `direct`, `offset=off` is not supported

Syntax

```
syn.directive.alias=[OPTIONS] <location> <ports>
```

- `<location>` is the location string in the format `function[/label]` that the `ALIAS` pragma applies to.
- `<ports>` specifies the ports to alias.

Options

- `distance=<integer>`: Specifies the difference between the pointer values passed to the ports in the list.
- `offset=<string>`: Specifies the offset of the pointer passed to each port in the `ports` list with respect to the origin of the array.

Example

For the following function `top`:

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
    #pragma HLS interface M_AXI port=arr0 bundle=hbm0 depth=0x40000000
    #pragma HLS interface M_AXI port=arr1 bundle=hbm1 depth=0x40000000
    #pragma HLS interface M_AXI port=arr2 bundle=hbm2 depth=0x40000000
    #pragma HLS interface M_AXI port=arr3 bundle=hbm3 depth=0x40000000
}
```

The following command defines aliasing for the specified array pointers, and defines the `distance` between them:

```
syn.directive.alias=top arr0,arr1,arr2,arr3 distance=10000000
```

Alternatively, the following command specifies the `offset` between pointers, to accomplish the same effect:

```
syn.directive.alias=top arr0,arr1,arr2,arr3
offset=00000000,10000000,20000000,30000000
```

See Also

- [syn.directive.interface](#)

syn.directive.allocation

Description

Specifies instance restrictions for resource allocation.

`syn.directive.allocation` can limit the number of RTL instances and hardware resources used to implement specific functions, loops, or operations. For example, if the C/C++ source has four instances of a function `foo_sub`, the `syn.directive.allocation` command can ensure that there is only one instance of `foo_sub` in the final RTL. All four instances are implemented using the same RTL block. This reduces resources used by the function, but negatively impacts performance by sharing those resources.

The operations in the C/C++ code, such as additions, multiplications, array reads, and writes, can also be limited by the `syn.directive.allocation` command.

Syntax

```
syn.directive.allocation=[OPTIONS] <location> <instances>
```

- `<location>` is the location string in the format `function[/label]`.
- `<instances>` is a function or operator. The function can be any function in the original C/C++ code that has not been either inlined by the `syn.directive.allocation` command or inlined automatically by Vitis HLS.

Options

- `limit=<integer>`:

Sets a maximum limit on the number of instances (of the type defined by the `type` option) to be used in the RTL design.

- `type=[function|operation]`: The instance type can be `function` (default) or `operation`. For a complete list of supported operations refer to [Operator Configuration](#).

Examples

Given a design `foo_top` with multiple instances of function `foo`, limits the number of instances of `foo` in the RTL to 2.

```
syn.directive.allocation=limit=2 type=function foo_top foo
```

Limits the number of multipliers used in the implementation of `My_func` to 1. This limit does not apply to any multipliers that might reside in sub-functions of `My_func`. To limit the multipliers used in the implementation of any sub-functions, specify an allocation directive on the sub-functions or inline the sub-function into function `My_func`.

```
syn.directive.allocation=limit=1 type=operation My_func mul
```

See Also

- [syn.directive.inline](#)

syn.directive.array_partition

Description



IMPORTANT! *syn.directive.array_partition and syn.directive.array_reshape are not supported for M_AXI Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in the Vector Data Types section of the Vitis HLS User Guide (UG1399).*

`syn.directive.array_partition` partitions an array into smaller arrays or individual elements.

This partitioning:

- Results in RTL with multiple small memories or multiple registers instead of one large memory
- Effectively increases the number of read and write ports for the storage
- Potentially improves the throughput of the design
- Requires more memory instances or registers

Syntax

```
syn.directive.array_partition=[OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<array>` is the array variable to be partitioned.

Options

- `dim=<integer>`: Specifies which dimension of the array is to be partitioned.
 - The dimension is relevant for multi-dimensional arrays only.
 - If a value of 0 is used, all dimensions are partitioned with the specified options.

- Any other value partitions only that dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- `type=(block|cyclic|complete):`
 - `block` partitioning creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into N equal blocks where N is the integer defined by the `-factor` option.
 - `cyclic` partitioning creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used:
 - Element 0 is assigned to the first new array.
 - Element 1 is assigned to the second new array.
 - Element 2 is assigned to the third new array.
 - Element 3 is assigned to the first new array again.
 - `complete` partitioning decomposes the array into individual elements. For a one-dimensional array, this corresponds to resolving a memory into individual registers. For multi-dimensional arrays, specify the partitioning of each dimension, or use `-dim 0` to partition all dimensions.

The default is `complete`.

- `factor=<integer>`: Used for `block` or `cyclic` partitioning only, this option specifies the number of smaller arrays that are to be created.
- `off=true`: Disables the `ARRAY_PARTITION` feature for the specified variable. Does not work with `dim`, `factor`, or `type`.

Example 1

Partitions array `AB[13]` in function `func` into four arrays. Because four is not an integer factor of 13:

- Three arrays have three elements.
- One array has four elements (`AB[9:12]`).

```
syn.directive.array_partition=func AB type=block factor=4
```

Partitions array `AB[6][4]` in function `func` into two arrays, each of dimension `[6][2]`.

```
syn.directive.array_partition=func AB type=block factor=2 dim=2
```

Partitions all dimensions of `AB[4][10][6]` in function `func` into individual elements.

```
syn.directive.array_partition=func AB type=complete dim=0
```


Example 2

Partitioned arrays can be addressed in your code by the new structure of the partitioned array, as shown in the following code example. When using the following directive:

```
syn.directive.array_partition=top b type=complete dim=1
```

The code can be structured as follows:

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}

syn.directive.interface mode=ap_memory top b[0]
syn.directive.interface mode=ap_memory top b[1]
syn.directive.interface mode=ap_memory top b[2]
syn.directive.interface mode=ap_memory top b[3]
```

See Also

- [syn.directive.array_reshape](#)

syn.directive.array_reshape

Description



IMPORTANT! *syn.directive.array_partition* and *syn.directive.array_reshape* are not supported for *M_AXI* Interfaces on the top-level function. Instead you can use the *hls::vector* data types as described in [Vector Data Types](#).

`syn.directive.array_reshape` combines array partitioning with vertical array mapping to create a single new array with fewer elements but wider words.

The `syn.directive.array_reshape` command has the following features:

- Splits the array into multiple arrays (like `syn.directive.array_partition`).
- Automatically recombine the arrays vertically to create a new array with wider words.

Syntax

```
syn.directive.array_reshape=[OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) that contains the array variable.
- `<array>` is the array variable to be reshaped.

Options

- **dim=<integer>**: Specifies which dimension of the array is to be partitioned.
 - The dimension is relevant for multidimensional arrays only.
 - If a value of 0 is used, all dimensions are partitioned with the specified options.
 - Any other value partitions only that dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- **type=(block|cyclic|complete)**:
 - **block** reshaping creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into N equal blocks where N is the integer defined by the `-factor` option and then combines the N blocks into a single array with `word-width*N`. The default is `complete`.
 - **cyclic** reshaping creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used, element 0 is assigned to the first new array, element 1 to the second new array, element 2 is assigned to the third new array, and then element 3 is assigned to the first new array again. The final array is a vertical concatenation (word concatenation, to create longer words) of the new arrays into a single array.
 - **complete** reshaping decomposes the array into temporary individual elements and then recombines them into an array with a wider word. For a one-dimension array this is equivalent to creating a very-wide register (if the original array was N elements of M bits, the result is a register with $N*M$ bits). This is the default.
- **factor=<integer>**: Used for `block` or `cyclic` partitioning only, this option specifies the number of smaller arrays that are to be created.
- **object**:

Note: Relevant for container arrays only.

Applies reshape on the objects within the container. If the option is specified, all dimensions of the objects will be reshaped, but all dimensions of the container will be kept.

- **off=true**: Disables the `ARRAY_RESHAPE` feature for the specified variable.

Example 1

Reshapes 8-bit array `AB[17]` in function `func` into a new 32-bit array with five elements.

Because four is not an integer factor of 17:

- Index 17 of the array, `AB[17]`, is in the lower eight bits of the reshaped fifth element.
- The upper eight bits of the fifth element are unused.

```
syn.directive.array_reshape=type=block factor=4 func AB
```

Partitions array AB[6][4] in function `func`, into a new array of dimension [6][2], in which dimension 2 is twice the width.

```
syn.directive.array_reshape=type=block factor=2 dim=2 func AB
```

Reshapes 8-bit array AB[4][2][2] in function `func` into a new single element array (a register), 4*2*2*8 (= 128)-bits wide.

```
syn.directive.array_reshape=type=complete dim=0 func AB
```

Example 2

Reshaped arrays can be addressed in your code by the new structure of the array, as shown in the following code example. When using the following directive:

```
syn.directive.array_reshape=top b type=complete dim=0
```

The code can be structured as follows:

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}
```

And the array interface defined as:

```
syn.directive.interface=mode=ap_memory top b[0]
```

See Also

- [syn.directive.array_partition](#)
- [syn.directive.interface](#)

syn.directive.bind_op

Description



TIP: This pragma or directive does not play a part in DSP multi-operation matching (MULADD, AMA, ADDMUL and corresponding Subtraction operations). In fact the use of BIND_OP for assigning an operator to DSP can prevent the HLS compiler from matching multi-operation expressions.

Vitis HLS implements the different operations in the code using specific hardware resources (or implementations). The tool automatically determines the resource to use, or you can define `syn.op` can be defined to generally specify the implementation as explained in [Operator Configuration](#).

The `syn.directive.bind_op` command indicates that for the specified variable an operation (for example `mul`, `add`, or `sub`) should be mapped to a specific implementation (`impl`) in the RTL. For example, to indicate that a specific multiplier operation (`mul`) is implemented in the device fabric rather than a DSP, you can use the `syn.directive.bind_op` command.

You can also specify the latency with `syn.directive.bind_op` using the `latency` option as described below.



IMPORTANT! To use the `latency` option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all basic arithmetic operations (add, subtract, multiply, and divide), and all floating-point operations.

Syntax

```
syn.directive.bind_op=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.
- `<variable>` is the variable to be assigned. The variable in this case is one that is assigned the result of the operation that is the target of this directive.

Options

- `op=<value>`: Defines the operation to bind to a specific implementation resource. Supported functional operations include: `mul`, `add`, `sub`

Supported floating point operations include: `fadd`, `fsub`, `fdiv`, `fexp`, `flog`, `fmul`, `frsqrt`, `frecip`, `fsqrt`, `dadd`, `dsub`, `ddiv`, `dexp`, `dlog`, `dmul`, `drsqrt`, `drecip`, `dsqrt`, `hadd`, `hsub`, `hdiv`, `hmul`, and `hsqrt`



TIP: Floating-point operations include single precision (f), double-precision (d), and half-precision (h).

- `impl=<value>`: Defines the implementation to use for the specified operation. Supported implementations for functional operations include `fabric` and `dsp`. Supported implementations for floating point operations include: `fabric`, `meddsp`, `fulldsp`, `maxdsp`, and `primitivedsp`.

Note: `primitivedsp` is only available on Versal devices.

- `latency=<int>`: Defines the default latency for the implementation of the operation. The valid latency varies according to the specified `op` and `impl`. The default is -1, which lets Vitis HLS choose the latency. The tables below reflect the supported combinations of operation, implementation, and latency.

Table 36: Supported Combinations of Functional Operations, Implementation, and Latency

Operation	Implementation	Min Latency	Max Latency
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0



TIP: Comparison operators, such as `dcmp`, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the `config_op` or `bind_op` commands.

Table 37: Supported Combinations of Floating Point Operations, Implementation, and Latency

Operation	Implementation	Min Latency	Max Latency
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13

Table 37: Supported Combinations of Floating Point Operations, Implementation, and Latency (cont'd)

Operation	Implementation	Min Latency	Max Latency
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqrt	fabric	0	58
drsqr	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

Examples

In the following example, a two-stage pipelined multiplier using fabric logic is specified to implement the multiplication for variable `c` of the function `foo`.

```
int foo (int a, int b) {
    int c, d;
    c = a*b;
    d = a*c;
    return d;
}
```

And the `syn.directive.bind_op` command is as follows:

```
syn.directive.bind_op=op=mul impl=fabric latency=2 foo c
```



TIP: The HLS tool selects the implementation to use for variable `d` because no `syn.directive.bind_op` is specified for that variable.

See Also

- [syn.directive.bind_storage](#)

syn.directive.bind_storage

Description

Vitis HLS assigns arrays in the code using specific memory resources (or types). The tool automatically determines the resource to use. Alternatively, you can define `syn.storage` to generally specify the memory type, as explained in [Storage Configuration](#).

The `syn.directive.bind_storage` command assigns a specific variable in the code (an array, or function argument) to a specific memory type (`type`) in the RTL. For example, you can use the `syn.directive.bind_storage` command to specify which type of memory, and which implementation to use for an array variable. Also, this allows you to control whether the array is implemented as a single or a dual-port RAM.



IMPORTANT! This feature is important for arrays on the top-level function interface, because the memory type associated with the array determines the number and type of ports needed in the RTL, as discussed in the Arrays on the Interface section of the Vitis High-Level Synthesis User Guide ([UG1399](#)). However, for variables assigned to top-level function arguments you must assign the memory type and implementation using the `storage_type` and `storage_impl` options of `syn.directive.interface`.

You can also use the `latency` option of `syn.directive.bind_storage` to specify the latency of the implementation. For block RAMs on the interface, the `latency` option allows you to model off-chip, non-standard SRAMs at the interface, for example supporting an SRAM with a latency of 2 or 3. For internal operations, the `latency` option allows the operation to be implemented using more pipelined stages. These additional pipeline stages can help resolve timing issues during RTL synthesis.



IMPORTANT! To use the `latency` option, the memory must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all block RAMs.

Syntax

```
syn.directive.bind_storage=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.

- `<variable>` is the variable to be assigned.



TIP: If the variable is an argument of a top-level function, then use the `storage_type` and `storage_impl` options of `syn.directive.interface`.

Options

- `type=<value>`: Defines the type of memory to bind to the specified variable. Supported types include: `fifo`, `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

Table 38: Storage Types

Type	Description
FIFO	A FIFO. Vitis HLS determines how to implement this in the RTL, unless the <code>-impl</code> option is specified.
RAM_1P	A single-port RAM. Vitis HLS determines how to implement this in the RTL, unless the <code>-impl</code> option is specified.
RAM_1WNR	A RAM with 1 write port and N read ports, using N banks internally.
RAM_2P	A dual-port RAM that allows read operations on one port and both read and write operations on the other port.
RAM_S2P	A dual-port RAM that allows read operations on one port and write operations on the other port.
RAM_T2P	A true dual-port RAM with support for both read and write on both ports.
ROM_1P	A single-port ROM. Vitis HLS determines how to implement this in the RTL, unless the <code>-impl</code> option is specified.
ROM_2P	A dual-port ROM.
ROM_NP	A multi-port ROM.



TIP: If you specify a single port RAM for a PIPO, then the binder will typically allocate a merged-PIPO, with only one bank, and reader and writer accessing different halves of the bank. The info message reported by the HLS compiler says "Implementing PIPO using a single memory for all blocks." This is often cheaper for small PIPOs, where all banks can share a single RAM block. However, if you specify a dual-port RAM for a PIPO to get higher bandwidth and the scheduler uses both ports either in the producer or in the consumer, then the binder will typically allocate a split-PIPO, where the producer will use 1 port and the consumer 2 ports of different banks. The info message reported by the HLS compiler in this case says "Implementing PIPO using a separate memory for each block."

- `impl=<value>`: Defines the implementation for the specified memory type. Supported implementations include: `bram`, `bram_ecc`, `lutram`, `uram`, `uram_ecc`, `srl`, `memory`, and `auto` as described below.

Table 39: Supported Implementation

Name	Description
MEMORY	Generic memory for FIFO, lets the Vivado tool choose the implementation.
URAM	UltraRAM resource

Table 39: Supported Implementation (cont'd)

Name	Description
URAM_ECC	UltraRAM with ECC
SRL	Shift Register Logic resource
LUTRAM	Distributed RAM resource
BRAM	Block RAM resource
BRAM_ECC	Block RAM with ECC
AUTO	Vitis HLS automatically determine the implementation of the variable.

Table 40: Supported Implementations by FIFO/RAM/ROM

Type	Command/Pragma	Scope	Supported Implementations
FIFO	<code>bind_storage</code> ¹	local	AUTO , BRAM, LUTRAM, URAM, MEMORY, SRL
FIFO	<code>config_storage</code>	global	AUTO , BRAM, LUTRAM, URAM, MEMORY, SRL
RAM* ROM*	<code>bind_storage</code>	local	AUTO BRAM, BRAM_ECC, LUTRAM, URAM, URAM_ECC
RAM* ROM*	<code>config_storage</code> ²	global	N/A
RAM_1P	<code>set_directive_interface_s_axilite - storage_impl</code>	local	AUTO , BRAM, URAM
	<code>config_interface - m_axi_buffer_impl</code>	global	AUTO , BRAM , LUTRAM, URAM

Notes:

- When no implementation is specified the directive uses AUTOSRL behavior as a default. However, this value cannot be specified.
 - `config_storage` only supports FIFO types.
- `latency=<int>`: Defines the default latency for the binding of the storage type to the implementation. The valid latency varies according to the specified `type` and `impl`. The default is -1, which lets Vitis HLS choose the latency.

Table 41: Supported Combinations of Memory Type, Implementation, and Latency

Type	Implementation	Min Latency	Max Latency
FIFO	BRAM	1	4
FIFO	LUTRAM	1	4
FIFO	MEMORY	1	4
FIFO	SRL	1	4
FIFO	URAM	1	4
RAM_1P	AUTO	1	3
RAM_1P	BRAM	1	3
RAM_1P	LUTRAM	1	3

Table 41: Supported Combinations of Memory Type, Implementation, and Latency
(cont'd)

Type	Implementation	Min Latency	Max Latency
RAM_1P	URAM	1	3
RAM_1WNR	AUTO	1	3
RAM_1WNR	BRAM	1	3
RAM_1WNR	LUTRAM	1	3
RAM_1WNR	URAM	1	3
RAM_2P	AUTO	1	3
RAM_2P	BRAM	1	3
RAM_2P	LUTRAM	1	3
RAM_2P	URAM	1	3
RAM_S2P	BRAM	1	3
RAM_S2P	BRAM_ECC	1	3
RAM_S2P	LUTRAM	1	3
RAM_S2P	URAM	1	3
RAM_S2P	URAM_ECC	1	3
RAM_T2P	BRAM	1	3
RAM_T2P	URAM	1	3
ROM_1P	AUTO	1	3
ROM_1P	BRAM	1	3
ROM_1P	LUTRAM	1	3
ROM_2P	AUTO	1	3
ROM_2P	BRAM	1	3
ROM_2P	LUTRAM	1	3
ROM_NP	BRAM	1	3
ROM_NP	LUTRAM	1	3



IMPORTANT! Any combinations of memory type and implementation that are not listed in the prior table are not supported by `syn.directive.bind_storage`.

Examples

In the following example, the `coeffs[128]` variable is an argument of the function `func1`. The directive specifies that `coeffs` uses a single port RAM implemented on a BRAM core from the library.

```
syn.directive.bind_storage=func1 coeffs type=RAM_1P impl=bram
```



TIP: The ports created in the RTL to access the values of `coeffs` are defined in the `RAM_1P` core.

See Also

- [syn.directive.bind_op](#)

syn.directive.dataflow**Description**

All operations are performed sequentially in a C/C++ description. In the absence of any directives that limit resources (such as `set_directive_allocation`), Vitis HLS seeks to minimize latency and improve concurrency. Data dependencies can limit this. For example, functions or loops that access arrays must finish all read/write accesses to the arrays before they complete. This prevents the next function or loop that consumes the data from starting operation.

However, it is possible for the operations in a function or loop to start operation before the previous function or loop completes all its operations. `syn.directive.dataflow` specifies that dataflow optimization be performed on the functions or loops, improving the concurrency of the RTL implementation. When `syn.directive.dataflow` is specified, the HLS tool analyzes the dataflow between sequential functions or loops and creates channels (based on ping-pong RAMs or FIFOs) that allow consumer functions or loops to start operation before the producer functions or loops have completed. This allows functions or loops to operate in parallel, which decreases latency and improves the throughput of the RTL.



TIP: The `syn.dataflow.xxx` command specifies the default memory channel and FIFO depth used by `syn.directive.dataflow` as explained in [Dataflow Configuration](#).

If no initiation interval (number of cycles between the start of one function or loop and the next) is specified, Vitis HLS attempts to minimize the initiation interval and start operation as soon as data is available. For the DATAFLOW optimization to work, the data must flow through the design from one task to the next. The following coding styles prevent the HLS tool from performing the DATAFLOW optimization.

- Single-producer-consumer violations
- Feedback between tasks
- Conditional execution of tasks
- Loops with multiple exit conditions



IMPORTANT! If any of these coding styles are present, the HLS tool issues a message and does not perform DATAFLOW optimization.

Finally, the DATAFLOW optimization has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from the DATAFLOW optimization, you must apply the optimization to the loop and the sub-function, or inline the sub-function.

Syntax

```
syn.directive.dataflow=<location> disable_start_propagation
```

- `<location>` is the location (in the format `function[/label]`) at which dataflow optimization is to be performed.
- `disable_start_propagation` disables the creation of a start FIFO used to propagate a start token to an internal process. Such FIFOs can sometimes be a bottleneck for performance.

Examples

Specifies DATAFLOW optimization within function `foo`.

```
syn.directive.dataflow=foo
```

See Also

- [syn.directive.allocation](#)

syn.directive.dependence

Description

Vitis HLS detects dependencies within loops: dependencies within the same iteration of a loop are loop-independent dependencies, and dependencies between different iterations of a loop are loop-carried dependencies.

These dependencies are impacted when operations can be scheduled, especially during function and loop pipelining.

- **Loop-independent dependence:** The same element is accessed in a single loop iteration.

```
for (i=1;i<N;i++) {  
    A[i]=x;  
    y=A[i];  
}
```

- **Loop-carried dependence:** The same element is accessed from a different loop iteration.

```
for (i=1;i<N;i++) {  
    A[i]=A[i-1]*2;  
}
```

Under certain circumstances, such as variable dependent array indexing or when an external requirement needs to be enforced (for example, two inputs are never the same index), the dependence analysis might be too conservative and fail to filter out false dependencies. The `syn.directive.dependence` command allows you to explicitly define the dependencies and eliminate a false dependence.



IMPORTANT! Specifying a false dependency when the dependency is not false can result in incorrect hardware. Ensure dependencies are correct (true or false) before specifying them.

Syntax

```
syn.directive.dependence=[OPTIONS] <location>
```

- **<location>**: The location in the code, specified as `function[/label]`, where the dependence is defined.

Options

- **class=(array | pointer)**: Specifies a class of variables in which the dependence needs clarification. This is mutually exclusive with the `variable` option.
- **variable=<variable>**: Defines a specific variable to apply the dependence directive. Mutually exclusive with the `class` option.



IMPORTANT! You cannot specify a *dependence* for function arguments that are bundled with other arguments in an *m_axi* interface. This is the default configuration for *m_axi* interfaces on the function. You also cannot specify a dependence for an element of a struct, unless the struct has been disaggregated.

- **dependent=(true | false)**: This argument should be specified to indicate whether a dependence is `true` and needs to be enforced, or is `false` and should be removed. However, when not specified, the tool will return a warning that the value was not specified and will assume a value of `false`.

- **direction=(RAW | WAR | WAW)**:

Note: Relevant only for loop-carried dependencies.

Specifies the direction for a dependence:

- **RAW (Read-After-Write - true dependence)**: The write instruction uses a value used by the read instruction.
- **WAR (Write-After-Read - anti dependence)**: The read instruction gets a value that is overwritten by the write instruction.
- **WAW (Write-After-Write - output dependence)**: Two write instructions write to the same location, in a certain order.
- **distance=<integer>**:

Note: Relevant only for loop-carried dependencies where `-dependent` is set to `true`.

Specifies the inter-iteration distance for array access.

- **type=(intra | inter)**: Specifies whether the dependence is:

- Within the same loop iteration (`intra`), or
- Between different loop iterations (`inter`) (default).

Examples

Removes the dependence between `Var1` in the same iterations of `loop_1` in function `func`.

```
syn.directive.dependence=variable=Var1 type=intra \  
dependent=false func/loop_1
```

The dependence on all arrays in `loop_2` of function `func` informs Vitis HLS that all reads must happen *after* writes in the same loop iteration.

```
syn.directive.dependence=class=array type=intra \  
dependent=true direction=RAW func/loop_2
```

See Also

- [syn.directive.disaggregate](#)
- [syn.directive.pipeline](#)

syn.directive.disaggregate

Description

The `syn.directive.disaggregate` command lets you deconstruct a `struct` variable into its individual elements. The number and type of elements created are determined by the contents of the struct itself.



IMPORTANT! *Structs used as arguments to the top-level function are aggregated by default, but can be disaggregated with this directive or pragma.*

Syntax

```
syn.directive.disaggregate=<location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) where the variable to disaggregate is found.
- `<variable>` specifies the struct variable name.

Options

This command has no options.

Example 1

The following example shows disaggregates the struct variable `a` in function `top`:

```
syn.directive.disaggregate=top a
```

Example 2

Disaggregated structs can be addressed in your code by the using standard C/C++ coding style as shown below. When using the directives:

```
syn.directive.disaggregate=top a
syn.directive.disaggregate=top c
```

The code can be structured as follows.

```
struct SS
{
    int x[N];
    int y[N];
};

int top(SS *a, int b[4][6], SS &c) {

syn.directive.interface=mode=s_axilite top a->x
syn.directive.interface=mode=s_axilite top a->y

syn.directive.interface=mode=ap_memory top c.x
syn.directive.interface=mode=ap_memory top c.y
```



IMPORTANT! Notice the different methods shown above for accessing an element of a pointer (*a*) versus an element of a reference (*c*)

Example 3

The following example shows the `Dot` struct containing the `RGB` struct as an element. If you apply `syn.directive.disaggregate` to variable `Arr`, then only the top-level `Dot` struct is disaggregated.

```
struct Pixel {
char R;
char G;
char B;
};

struct Dot {
Pixel RGB;
unsigned Size;
};

#define N 1086
```

```
void DUT(Dot Arr[N]) {
    ...
}

syn.directive.disaggregate=DUT Arr
```

If you want to disaggregate the whole struct, `Dot` and `RGB`, then you can assign the `set_directive_disaggregate` as shown below.

```
void DUT(Dot Arr[N]) {
    #pragma HLS disaggregate variable=Arr->RGB
    ...
}

syn.directive.disaggregate=DUT Arr->RGB
```

The results in this case will be:

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
    ...
}
```

See Also

- [syn.directive.aggregate](#)

syn.directive.expression_balance

Description

Sometimes C/C++ code is written with a sequence of operations, resulting in a long chain of operations in RTL. With a small clock period, this can increase the latency in the design. By default, the Vitis HLS tool rearranges the operations using associative and commutative properties. The rearrangement creates a balanced tree that can shorten the chain, potentially reducing latency in the design at the cost of extra hardware.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but can be disabled.
- For floating-point operations, expression balancing is off by default but can be enabled.

The `syn.directive.expression_balance` command allows this expression balancing to be turned off, or on, within a specified scope.

Syntax

```
syn.directive.expression_balance=[OPTIONS] <location>
```

- `<location>` is the location (in the format `function[/label]`) where expression balancing should be disabled, or enabled.

Options

- **off**: Turns off expression balancing at the specified location. Specifying the `syn.directive.expression_balance` command enables expression balancing in the specified scope. Adding the `off` option disables it.

Examples

Disables expression balancing within function `My_Func`.

```
syn.directive.expression_balance=off My_Func
```

Explicitly enables expression balancing in function `My_Func2`.

```
set_directive_expression_balance=My_Func2
```

syn.directive.function_instantiate

Description

By default:

- Functions remain as separate hierarchy blocks in the RTL, or are decomposed (inlined) into higher-level functions.
- All instances of a function, at the same level of hierarchy, uses the same RTL implementation (block).

The `syn.directive.function_instantiate` command is used to create a unique RTL implementation for each instance of a function, allowing each instance to be optimized around a specific argument or variable.

By default, the following code results in a single RTL implementation of function `func_sub` for all three instances, or if `func_sub` is a small function it is inlined into function `func`.



TIP: By default, the Vitis HLS tool automatically inlines small functions. This is true even for function instantiations. Using the `set_directive_inline off` option can be used to prevent this automatic inlining.

```
char func_sub(char inval, char incr)
{
    return inval + incr;
}
void func(char inval1, char inval2, char inval3,
          char *outval1, char *outval2, char * outval3)
{
    *outval1 = func_sub(inval1, 1);
    *outval2 = func_sub(inval2, 2);
    *outval3 = func_sub(inval3, 3);
}
```

Using the directive as shown in the example section below results in three versions of function `func_sub`, each independently optimized for variable `incr`.

Syntax

```
syn.directive.function_instantiate=<location> <variable>
```

- `<location>` is the location (in the format `function[/region label]`) where the instances of a function are to be made unique.
- `<variable>` specifies the function argument to be specified as a constant in the various function instantiations.

Options

This command has no options.

Examples

For the example code shown above, the following Tcl (or pragma placed in function `func_sub`) allows each instance of function `func_sub` to be independently optimized with respect to input `incr`.

```
syn.directive.inline off=func_sub  
syn.directive.function_instantiate=func_sub incr
```

See Also

- [syn.directive.allocation](#)
- [syn.directive.inline](#)

syn.directive.inline

Description

`syn.directive.inline` removes a function as a separate entity in the RTL hierarchy. After inlining, the function is dissolved into the calling function, and no longer appears as a separate level of hierarchy.



IMPORTANT! *Inlining a child function also dissolves any pragmas or directives applied to that function. In Vitis HLS, any directives applied in the child context are ignored.*

In some cases, inlining a function allows operations within the function to be shared and optimized more effectively with the calling function. However, an inlined function cannot be shared or reused, so if the parent function calls the inlined function multiple times, this can increase the area and resource utilization.

By default, inlining is only performed on the next level of function hierarchy.

Syntax

```
syn.directive.inline=[OPTIONS] <location>
```

- **<location>** is the location (in the format `function[/label]`) where inlining is to be performed.

Options

- **off**: By default, Vitis HLS performs inlining of smaller functions in the code. Using the `off` option disables inlining for the specified function.
- **recursive**: By default, only one level of function inlining is performed. The functions within the specified function are not inlined. The `recursive` option inlines all functions recursively within the specified function hierarchy.

Examples

The following example inlines function `func_sub1`, but no sub-functions called by `func_sub1`.

```
syn.directive.inline=func_sub1
```

This example inlines function `func_sub1`, recursively down the hierarchy, excluding function `func_sub2`:

```
syn.directive.inline=recursive func_sub1  
syn.directive.inline=off func_sub2
```

See Also

- [syn.directive.allocation](#)

syn.directive.interface

Description

`syn.directive.interface` is only supported for use on the top-level function, and cannot be used for sub-functions of the HLS component. The `INTERFACE` pragma or directive specifies how RTL ports are created from the function arguments during interface synthesis, as described in the Defining Interfaces section of the Vitis HLS User Guide ([UG1399](#)). The Vitis HLS tool automatically determines the I/O protocol used by any sub-functions.

Ports in the RTL implementation are derived from the data type and direction of the arguments of the top-level function and function return, the `flow_target` for the HLS component, the default interface configuration settings as specified by `syn.interface.xxx` commands described in [Interface Configuration](#), and by `syn.directive.interface`. Each function argument can be specified to have its own I/O protocol (such as valid handshake or acknowledge handshake).



TIP: Global variables required on the interface must be explicitly defined as an argument of the top-level function as described in the Global Variables section of the Vitis HLS User Guide ([UG1399](#)). However, if a global variable is accessed, but all read and write operations are local to the design, the resource is created in the design. There is no need for an I/O port in the RTL.

The interface also defines the execution control protocol of the HLS component as described in the Block-Level Control Protocols section of the Vitis HLS User Guide ([UG1399](#)). The control protocol controls when the HLS component (or block) starts execution, and when the block completes operation, is idle and ready for new inputs.

Syntax

```
syn.directive.interface=[OPTIONS] <location> <port>
```

- `<location>` is the location (in the format `function[/label]`) where the function interface or registered output is to be specified.
- `<port>` is the parameter (function argument) for which the interface has to be synthesized. The port name is not required when block control modes are specified: `ap_ctrl_chain`, `ap_ctrl_hs`, or `ap_ctrl_none`.

Options



TIP: Many of the options specified below have default values that are defined with `syn.interface.xxx` commands. You can define local values for the interface defined here to override the default values.

- `mode=<mode>`:

The supported modes, and how the tool implements them in RTL, can be broken down into three categories as follows:

1. Port-Level Protocols:

- `ap_none`: No port protocol. The interface is a simple data port.
- `ap_stable`: No protocol. The interface is a simple data port, but the Vitis HLS tool assumes the data port is always stable after reset which allows optimizations to remove unnecessary registers.
- `ap_vld`: Implements the data port with an associated `valid` signal to indicate when the data is valid for reading or writing.

- `ap_ack`: Implements the data port with an associated `acknowledge` signal to acknowledge that the data was read or written.
- `ap_hs`: Implements the data port with both `valid` and `acknowledge` signals to provide a two-way handshake to indicate when the data is valid for reading and writing and to acknowledge that the data was read or written.
- `ap_ovld`: Implements the output data port with an associated `valid` signal to indicate when the data is valid for reading or writing.



TIP: For `ap_ovld` Vitis HLS implements the input argument or the input half of any read/write arguments with mode `ap_none`.

- `ap_memory`: Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the interface is composed of separate ports.
- `bram`: Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the memory interface is composed of a single port.
- `ap_fifo`: Implements the port with a standard FIFO interface using data input and output ports with associated active-Low FIFO `empty` and `full` ports.

Note: You can only use the `ap_fifo` interface on read arguments or write arguments. `ap_fifo` mode does not support bidirectional read/write arguments.

2. AXI Interface Protocols:

- `s_axilite`: Implements the port as an AXI4-Lite interface. The tool produces an associated set of C driver files when exporting the generated RT for the HLS component.
- `m_axi`: Implements the port as an AXI4 interface. You can use the `syn.interface.m_axi_addr64` command to specify either 32-bit (default) or 64-bit address ports and to control any address offset.
- `axis`: Implements the port as an AXI4-Stream interface.

3. Block-Level Control Protocols:

- `ap_ctrl_chain`: Implements a set of block-level control ports to `start` the design operation, `continue` operation, and indicate when the design is `idle`, `done`, and `ready` for new input data.
- `ap_ctrl_hs`: Implements a set of block-level control ports to `start` the design operation and to indicate when the design is `idle`, `done`, and `ready` for new input data.
- `ap_ctrl_none`: No block-level I/O protocol.

Note: Using the `ap_ctrl_none` mode might prevent the design from being verified using C/RTL co-simulation.

- `bundle=<string>`:

By default, the HLS tool groups (or bundles) function arguments that are compatible into a single interface port in the RTL code. All interface ports with compatible options, such as `mode`, `offset`, and `bundle`, are grouped into a single interface port.



TIP: This default can be changed using the `syn.interface.m_axi_auto_max_ports` command.

This `bundle=<string>` option lets you define bundles to group ports together, overriding the default behavior. The `<string>` specifies the bundle name. The port name used in the generated RTL code is derived automatically from a combination of the `mode` and `bundle`, or is named as specified by `name`.



IMPORTANT! `bundle` names must be specified using lower-case characters.

- **clock=<string>:** By default, the AXI4-Lite interface clock is the same clock as the system clock. This option is used to set specify a separate clock for an AXI4-Lite interface. If the `bundle` option is used to group multiple top-level function arguments into a single AXI4-Lite interface, the clock option need only be specified on one of bundle members.
- **channel=<string>:** To enable multiple channels on an `m_axi` interface specify the channel ID. Multiple `m_axi` interfaces can be combined into a single `m_axi` adapter using separate channel IDs.
- **depth=<int>:** Specifies the maximum number of samples for the test bench to process. This setting indicates the maximum size of the FIFO needed in the verification adapter that the HLS tool creates for RTL co-simulation.



TIP: While `depth` is usually an option, it is needed to specify the size of pointer arguments for RTL co-simulation.

- **interrupt=<int>:** Only used by `ap_vld/ap_hs`. This option enables the I/O to be managed in interrupt, by creating the corresponding bits in the `ISR` and `IER` in the `s_axilite` register file. The integer value `N=16..31` specifies the bit position in both registers (by default assigned contiguously from 16).
- **latency=<value>:** This option can be used on `ap_memory` and `m_axi` interfaces.
 - In an `ap_memory` interface the option specifies the read latency of the RAM resource driving the interface. By default, a read operation of 1 clock cycle is used. This option allows an external RAM with more than 1 clock cycle of read latency to be modeled.
 - In an `m_axi` interface the option specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request the specified number of cycles (latency) before the read or write is expected. If this figure is too low, the design will be ready too soon and might stall waiting for the bus. If this figure is too high, bus access might be idle waiting on the design to start the access.

- **max_read_burst_length=<int>**: For use with the `m_axi` interface, this option specifies the maximum number of data values read during a burst transfer. Refer to the AXI Burst Transfers section of the Vitis HLS User Guide (UG1399) for more information.
- **max_write_burst_length=<int>**: For use with the `m_axi` interface, this option specifies the maximum number of data values written during a burst transfer.
- **max_widen_bitwidth=<int>**: Specifies the maximum bit width available for the interface when automatically widening the interface. This overrides the default value specified by the `syn.interface.m_axi_max_bitwidth` command. Refer to Automatic Port Width Resizing in the Vitis High-Level Synthesis User Guide (UG1399) for more information.
- **name=<string>**: Specifies a name for the port which will be used in the generated RTL. By default the port name is derived automatically from a combination of the `mode` and `bundle` unless specified by `name`.
- **num_read_outstanding=<int>**: For use with the `m_axi` interface, this option specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

```
num_read_outstanding*max_read_burst_length*word_size
```

- **num_write_outstanding=<int>**: For use with the `m_axi` interface, this option specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

```
num_write_outstanding*max_write_burst_length*word_size
```

- **offset=<string>**: Controls the address offset in AXI4-Lite (`s_axilite`) and AXI4 memory mapped (`m_axi`) interfaces for the specified port.
 - In an `s_axilite` interface, `<string>` specifies the address in the register map.
 - In an `m_axi` interface this option overrides the global option specified by the `config_interface -m_axi_offset` option, and `<string>` is specified as:
 - `off`: Do not generate an offset port.
 - `direct`: Generate a scalar input offset port.
 - `slave`: Generate an offset port and automatically map it to an AXI4-Lite slave interface. This is the default offset.
- **register**: Registers the signal and any associated protocol signals and instructs the signals to persist until at least the last cycle of the function execution. The `syn.interface.register_io` command controls default registering of all interfaces on the top function, while this option lets you override the default for the current interface. This option applies to the following interface modes:
 - `s_axilite`
 - `ap_fifo`

- `ap_none`
- `ap_stable`
- `ap_hs`
- `ap_ack`
- `ap_vld`
- `ap_ovld`



TIP: The `register` option cannot be used on the return port of the function (`port=return`). Use the `syn.directive.latency` instead.

- **`register_mode=(both|forward|reverse|off)`:** This option applies to AXI4-Stream interfaces, and specifies if registers are placed on the forward path (`TDATA` and `TVALID`), the reverse path (`TREADY`), on both paths, or if none of the ports signals are to be registered (`off`). The default is `register_mode=both`.



TIP: AXI4-Stream side-channel signals are considered to be data signals and are registered whenever the `TDATA` is registered.

- **`storage_impl=<impl>`:** For use with `mode=s_axilite` only, this options defines a storage implementation to assign to the interface. Supported `<impl>` values include `auto`, `bram`, and `uram`. The default is `auto`.



TIP: `uram` is a synchronous memory with a single clock for two ports available only on certain devices. Therefore `uram` cannot be specified for an `s_axilite` adapter with two clocks, or when the specified part does not support `uram`.

- **`storage_type=<type>`:**

For use with `mode=ap_memory` or `mode=bram` only, this options defines a storage type (for example, `RAM_T2P`) to assign to the variable.

Supported types include: `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.



TIP: For objects that are not defined on the interface of the top-level function this can be defined using `syn.directive.bind_storage`.

Example 1

This example disables function-level handshakes for function `func`.

```
syn.directive.interface=mode=ap_ctrl_none func return
```


Example 2

Argument `InData` in function `func` is specified to have a `ap_vld` interface and the input should be registered.

```
set_directive_interface=func InData mode=ap_vld register
```

See Also

- [syn.directive.bind_storage](#)
- [syn.directive.latency](#)

syn.directive.latency

Description

Specifies a maximum or minimum latency value, or both, on a function, loop, or region.

Vitis HLS always aims for minimum latency. The behavior of the tool when minimum and maximum latency values are specified is as follows:

- Latency is less than the minimum: If Vitis HLS can achieve less than the minimum specified latency, it extends the latency to the specified value, potentially enabling increased sharing.
- Latency is greater than the minimum: The constraint is satisfied. No further optimizations are performed.
- Latency is less than the maximum: The constraint is satisfied. No further optimizations are performed.
- Latency is greater than the maximum: If Vitis HLS cannot schedule within the maximum limit, it increases effort to achieve the specified constraint. If it still fails to meet the maximum latency, it issues a warning. Vitis HLS then produces a design with the smallest achievable latency.



TIP: You can also use `syn.directive.latency` to limit the efforts of the tool to find an optimum solution. Specifying latency constraints for scopes within the code: loops, functions, or regions, reduces the possible solutions within that scope and can improve tool compilation time. Refer to the *Improving Synthesis Runtime and Capacity* section of the Vitis HLS User Guide ([UG1399](#)) for more information.

To limit the total latency of all loop iterations, the latency directive should be applied to a region that encompasses the entire loop, as in this example: `syn.directive.latency`

```
Region_All_Loop_A max=10
```

```
Region_All_Loop_A: {
  Loop_A: for (i=0; i<N; i++)
  {
    ..Loop Body...
  }
}
```

In this case, even if the loop is unrolled, the latency directive sets a maximum limit on all loop operations.

Syntax

```
syn.directive.latency=[OPTIONS] <location>
```

- `<location>` is the location (function, loop or region) (in the format `function[/label]`) to be constrained.

Options

- `max=<integer>`: Limits the maximum latency.
- `min=<integer>`: Limits the minimum latency.

Examples

Function `foo` is specified to have a minimum latency of 4 and a maximum latency of 8.

```
syn.directive.latency=min=4 max=8 foo
```

In function `foo`, loop `loop_row` is specified to have a maximum latency of 12.

```
syn.directive.latency=max=12 foo/loop_row
```

syn.directive.loop_flatten

Description

Flattens nested loops into a single loop hierarchy.

In the RTL implementation, it costs a clock cycle to move between loops in the loop hierarchy. Flattening nested loops allows them to be optimized as a single loop. This saves clock cycles, potentially allowing for greater optimization of the loop body logic.



RECOMMENDED: Apply this directive to the inner-most loop in the loop hierarchy. Only perfect and semi-perfect loops can be flattened in this manner.

- **Perfect loop nests:**
 - Only the innermost loop has loop body content.
 - There is no logic specified between the loop statements.
 - All loop bounds are constant.
- **Semi-perfect loop nests:**
 - Only the innermost loop has loop body content.

- There is no logic specified between the loop statements.
- The outermost loop bound can be a variable.
- **Imperfect loop nests:**

When the inner loop has variable bounds (or the loop body is not exclusively inside the inner loop), try to restructure the code, or unroll the loops in the loop body to create a perfect loop nest.

Syntax

```
syn.directive.loop_flatten=[OPTIONS] <location>
```

- `<location>` is the location (inner-most loop), in the format `function[/label]`.

Options

- `off`: Option to prevent loop flattening from taking place, and can prevent some loops from being flattened while all others in the specified location are flattened.



IMPORTANT! The presence of the `LOOP_FLATTEN` pragma or directive enables the optimization. The addition of `off` disables it.

Examples

Flattens `loop_1` in function `foo` and all (perfect or semi-perfect) loops above it in the loop hierarchy, into a single loop.

```
set_directive_loop_flatten=foo/loop_1
```

Prevents loop flattening in `loop_2` of function `foo`.

```
set_directive_loop_flatten=off foo/loop_2
```

See Also

- [syn.directive.loop_flatten](#)

syn.directive.loop_merge

Description

Merges all loops into a single loop. Merging loops:

- Reduces the number of clock cycles required in the RTL to transition between the loop-body implementations.
- Allows the loops be implemented in parallel (if possible).

The rules for loop merging are:

- If the loop bounds are variables, they must have the same value (number of iterations).
- If loops bounds are constants, the maximum constant value is used as the bound of the merged loop.
- Loops with both variable bound and constant bound cannot be merged.
- The code between loops to be merged cannot have side effects. Multiple execution of this code should generate the same results.
 - `a=b` is allowed
 - `a=a+1` is not allowed.
- Loops cannot be merged when they contain FIFO reads. Merging changes the order of the reads. Reads from a FIFO or FIFO interface must always be in sequence.

Syntax

```
syn.directive.loop_merge=[options] <location>
```

- `<location>` is the location (in the format `function[/label]`) at which the loops reside.

Options

- `force`: Forces loops to be merged even when Vitis HLS issues a warning. You must assure that the merged loop will function correctly.

Examples

Merges all consecutive loops in function `foo` into a single loop.

```
syn.directive.loop_merge=foo
```

All loops inside `loop_2` of function `foo` (but not `loop_2` itself) are merged by using the `force` option.

```
syn.directive.loop_merge=force foo/loop_2
```

See Also

- [syn.directive.loop_flatten](#)
- [syn.directive.unroll](#)

syn.directive.loop_tripcount

Description

The *loop_tripcount* is the total number of iterations performed by a loop. Vitis HLS reports the total latency of each loop (the number of cycles to execute all iterations of the loop). This loop latency is therefore a function of the tripcount (number of loop iterations).



IMPORTANT! *syn.directive.loop_tripcount* is for analysis only, and does not impact the results of synthesis.

The tripcount can be a constant value. It might depend on the value of variables used in the loop expression (for example, $x < y$) or control statements used inside the loop.

Vitis HLS cannot determine the tripcount in some cases. These cases include, for example, those in which the variables used to determine the tripcount are:

- Input arguments, or
- Variables calculated by dynamic operation

In the following example, the maximum iteration of the for-loop is determined by the value of input `num_samples`. The value of `num_samples` is not defined in the C function, but comes into the function from the outside.

```
void foo (num_samples, ...) {
    int i;
    ...
    loop_1: for(i=0;i< num_samples;i++) {
        ...
        result = a + b;
    }
}
```

In cases where the loop latency is unknown or cannot be calculated, *syn.directive.loop_tripcount* allows you to specify minimum, maximum, and average iterations for a loop. This lets the tool analyze how the loop latency contributes to the total design latency in the reports and helps you determine appropriate optimizations for the design.



TIP: If a C assert macro is used to limit the size of a loop variable, Vitis HLS can use it to both define loop limits for reporting and create hardware that is exactly sized to these limits.

Syntax

```
syn.directive.loop_tripcount=[OPTIONS] <location>
```

- `<location>` is the location of the loop (in the format `function[/label]`) at which the tripcount is specified.

Options

- `avg=<integer>`: Specifies the average number of iterations.
- `max=<integer>`: Limits the maximum number of iterations.
- `min=<integer>`: Limits the minimum number of iterations.

Examples

`loop_1` in function `foo` is specified to have a minimum tripcount of 12, and a maximum tripcount of 16:

```
syn.directive.loop_tripcount=min=12 max=16 avg=14 foo/loop_1
```

syn.directive.occurrence

Description

When pipelining functions or loops, the OCCURRENCE pragma or directive specifies that the code in a pipelined function call within the pipelined function or loop is executed at a lower rate than the surrounding function or loop. This allows the pipelined call that is executed at the lower rate to be pipelined at a slower rate, and potentially shared within the top-level pipeline. For example:

- A loop iterates N times.
- Part of the loop is protected by a conditional statement and only executes M times, where N is an integer multiple of M .
- The code protected by the conditional is said to have an occurrence of N/M .

Identifying a region with an OCCURRENCE rate allows the functions and loops in this region to be pipelined with an initiation interval that is slower than the enclosing function or loop.

Syntax

```
syn.directive.occurrence=[OPTIONS] <location>
```

- `<location>` specifies the block of code that contains the pipelined function call(s) with a slower rate of execution.

Options

- `cycle=<int>`: Specifies the occurrence N/M where:
 - N is the number of times the enclosing function or loop is executed.
 - M is the number of times the conditional region is executed.



IMPORTANT! *N must be an integer multiple of M.*

- `off=true`: Disable occurrence for the specified function.

Examples

Region `Cond_Region` in function `foo` has an occurrence of 4. It executes at a rate four times slower than the code that encompasses it.

```
syn.directive.occurrence=cycle=4 foo/Cond_Region
```

See Also

- [syn.directive.pipeline](#)

syn.directive.performance

Description



TIP: *syn.directive.performance applies to loops and loop nests, and requires a known loop tripcount to determine the performance. If your loop has a variable tripcount then you must also specify `syn.directive.tripcount`.*

The `syn.directive.performance` lets you specify a high-level constraint, `target_ti` or `target_tl`, defining the number of clock cycles between successive starts of a loop, and lets the tool infer lower-level UNROLL, PIPELINE, ARRAY_PARTITION, and INLINE directives needed to achieve the desired result. The `syn.directive.performance` does not guarantee the specified value will be achieved, and so it is only a target.

The `target_ti` is the interval between successive starts of the loop, or between the start of the first iteration of the loop, and the next start of the first iteration of the loop. In the following code example, a `target_ti=T` would mean the target interval for the start of loop `L2` between two consecutive iterations of `L1` should be 100 cycles.

```
const int T = 100;
L1: for (int i=0; i<N; i++){
  L2: for (int j=0; j<M; j++){
    #pragma HLS PERFORMANCE target_ti=T
    ...
  }
}
```

The `target_tl` is the interval between start of the loop and end of the loop, or between the start of the first iteration of the loop and the completion of the last iteration of the loop. In the preceding code example a `target_tl=T` means the target completion of loop `L2` for a single iteration of `L1` should be 100 cycles.

Note: `syn.directive.inline` is applied automatically to functions inside any pipelined loop that has `II=1` to improve throughput. If you apply the `PERFORMANCE` pragma or directive that infers a pipeline with `II=1`, it will also trigger the auto-inline optimization. You can disable this for specific functions by using `syn.directive.inline off`.

The transaction interval is the initiation interval (II) of the loop times the number of iterations, or tripcount: $\text{target_ti} = \text{II} * \text{loop tripcount}$. Conversely, $\text{target_ti} = \text{FreqHz} / \text{Operations per second}$.

For example, assuming an image processing function that processes a single frame per invocation with a throughput goal of 60 fps, then the target throughput for the function is 60 invocations per second. If the clock frequency is 180 MHz, then `target_ti` is 180M/60, or 3 million clock cycles per function invocation.

Syntax

Specify the directive for a labeled loop.

```
syn.directive.performance=<location> [Options]
```

Where:

`<location>` specifies the loop in the format `function/loop_label`.

Options

- **target_ti=<value>:** Specifies a target transaction interval defined as the number of clock cycles for the loop to complete an iteration. The transaction interval refers to the number of clock cycles from the first transaction of a loop, or nested loop, to the start of the next transaction of the loop. The `<value>` can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

Note: A warning will be returned if truncation occurs.

- **target_tl=<value>:**

Specifies a target latency defined as the number of clock cycles for the loop to complete all iterations. The transaction latency is defined as the interval between the start of the first iteration of the loop, and the completion of the last iteration of the loop. The `<value>` can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

- **unit=[sec | cycle]:** Specifies the unit associated with the `target_ti` or `target_tl` values. The unit can either be specified as seconds, or clock cycles. When the unit is specified as seconds, a unit can be specified with the value to indicate nanoseconds (ns), picoseconds (ps), microseconds (us).

Example 1

The loop labeled `loop_1` is specified to have target transaction interval of 4 clock cycles:

```
syn.directive.performance=loop_1 target_ti=4
```

See Also

- [syn.directive.inline](#)

syn.directive.pipeline

Description

Reduces the initiation interval (II) for a function or loop by allowing the concurrent execution of operations as described in the Pipelining Paradigm section of Vitis HLS User Guide (UG1399). A pipelined function or loop can process new inputs every N clock cycles, where N is the initiation interval (II). An II of 1 processes a new input every clock cycle.

As a default behavior, with `syn.directive.pipeline` the tool will generate the minimum II for the design according to the specified clock period constraint. The emphasis will be on meeting timing, rather than on achieving II unless the `-II` option is specified.

The default type of pipeline is defined by the `syn.compile.pipeline_style` command as described in [Compile Options](#), but can be overridden in the PIPELINE pragma or directive.

If Vitis HLS cannot create a design with the specified II, it issues a warning and creates a design with the lowest achievable II. You can then analyze the design with the warning messages to determine what steps must be taken to create a design that satisfies the required initiation interval.

Syntax

```
syn.directive.pipeline=[OPTIONS] <location>
```

Where:

- `<location>` is the location (in the format `function[/label]`) to be pipelined.

Options

- `II=<integer>`: Specifies the desired initiation interval for the pipeline. Vitis HLS tries to meet this request. Based on data dependencies, the actual result might have a larger II.
- `off`: Turns off pipeline for a specific loop or function. This can be used when `config_compile -pipeline_loops` is used to globally pipeline loops.
- `rewind`:

Note: Applicable only to a loop.

Optional keyword. Enables rewinding as described in [Rewinding Pipelined Loops for Performance](#). This enables continuous loop pipelining with no pause between one execution of the loop ending and the next execution starting. Rewinding is effective only if there is one single loop (or a perfect loop nest) inside the top-level function. The code segment before the loop:

- Is considered as initialization.
- Is executed only once in the pipeline.
- Cannot contain any conditional operations (`if-else`).
- `style=<stp | frp | flp>`: Specifies the type of pipeline to use for the specified function or loop. For more information on pipeline styles refer to the Flushing Pipeline and Pipeline Types section of the Vitis HLS User Guide ([UG1399](#)). The types of pipelines include:
 - `stp`: Stall pipeline. Runs only when input data is available otherwise it stalls. This is the default setting, and is the type of pipeline used by Vitis HLS for both loop and function pipelining. Use this when a flushable pipeline is not required. For example, when there are no performance or deadlock issue due to stalls.
 - `flp`: This option defines the pipeline as a flushable pipeline. This type of pipeline typically consumes more resources and/or can have a larger II because resources cannot be shared among pipeline iterations.
 - `frp`: Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power as the pipeline registers are clocked even if there is no data.



IMPORTANT! This is a hint not a hard constraint. The tool checks design conditions for enabling pipelining. Some loops might not conform to a particular style and the tool reverts to the default style (`stp`) if necessary.

Examples

Function `func` is pipelined with the specified initiation interval.

```
syn.directive.pipeline=func II=1
```

See Also

- [syn.directive.dependence](#)

syn.directive.protocol

Description

This command specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code. The tool will not insert any clocks between operations in the region, including those which read from or write to function arguments. The order of read and writes will therefore be strictly followed in the synthesized RTL.

A region of code can be created in the C/C++ code by enclosing the region in braces "{}" and naming it. The following defines a region named `io_section`:

```
io_section:{  
...  
lines of code  
...  
}
```

A clock operation can be explicitly specified in C code using an `ap_wait()` statement, and can be specified in C++ code by using the `wait()` statement.



TIP: The `ap_wait` and `wait` statements have no effect on the simulation of the design.

Syntax

```
syn.directive.protocol=[OPTIONS] <location>
```

The `<location>` specifies the location (in the format `function[/label]`) at which the protocol region is defined.

Options

- `mode=[floating | fixed]:`
 - `floating`: Lets code statements outside the protocol region overlap and execute in parallel with statements in the protocol region in the final RTL. The protocol region remains cycle accurate, but outside operations can occur at the same time. This is the default mode.
 - `fixed`: The fixed mode ensures that statements outside the protocol region do not execute in parallel with the protocol region.

Examples

The example code defines a protocol region, `io_section` in function `foo`. The following directive defines that region as a fixed mode protocol region:

```
syn.directive.protocol=mode=fixed foo/io_section
```

syn.directive.reset

Description

The RESET pragma or directive adds or disables reset ports for specific state variables (global or static).

The reset port is used to restore the registers and block RAM, connected to the port, to an initial value any time the reset signal is applied. Globally, the presence and behavior of the RTL reset port is controlled using the `syn.rtl.reset` configuration settings. The reset configuration settings include the ability to define the polarity of the reset, and specify whether the reset is synchronous or asynchronous, but more importantly it controls, through the reset option, which registers are reset when the reset signal is applied. For more information, see Controlling Initialization and Reset Behaviour in *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

More specific control over reset is provided through the RESET pragma. For global or static variables the RESET pragma is used to explicitly enable a reset when none is present, or the variable can be removed from the reset by turning `off` the pragma. This can be particularly useful when static or global arrays are present in the design.

Note: For public variables of a class, the RESET pragma must be used as the reset configuration settings only apply to variables declared at the function or file level. In addition, the RESET pragma must be applied to an object of the class in the top-function or sub-function, and cannot be applied to private variables of the class.

Syntax

Place the pragma in the C source within the boundaries of the variable life cycle.

```
syn.directive.reset=<location> [ variable=<a> | off=true ]
```

Where:

- **location>:** Specifies the location (in the format `function[/label]`) at which the variable is defined.
- **variable=<a>:** Specifies the variable to which the RESET pragma is applied.
- **off or off=true:** Indicates that reset is not generated for the specified variable.

Examples

Adds reset to variable `a` in function `foo` even when the global reset (`syn.rtl.reset`) setting is `none` or `control`.

```
syn.directive.reset=foo a
```

Removes reset from variable `static int a` in function `foo` even when the global reset setting is `state` or `all`.

```
syn.directive.reset=off foo a
```

The following example shows the use of the `RESET` pragma or directive with class variables and methods. A variable declared in a class must be a public static variable in the class and can have the `RESET` pragma specified for the variable in a method of the class, or after an object of the class has been created in the top function or sub-function of the HLS design.

```
class bar {
public:
    static int a; // class level static; must be public
    int a1;      // class-level non-static; must be public
    bar(...) {
        // #pragma reset does not work here for a or a1
    }
    // this is the function that is called in the top
    void run(...) {
        // #pragma reset does not work here for non-static variable a1
        // but does work for static variable a
        #pragma reset variable=a
    }
};

static int c; // file level
int d; // file level
void top(...) {
    #pragma HLS top
    static int b; // function level
    bar t;
    static bar s;

    // #pragma reset works here for b, c and d, as well as t and s members
    // except for t.a1 because it is not static
    #pragma reset variable=b
    #pragma reset variable=c
    #pragma reset variable=d
    #pragma reset variable=t.a
    #pragma reset variable=s.a
    #pragma reset variable=s.a1
    t.run(...);
    s.run(...);
}
```

The `syn.directive.reset` command specified for the function `top` shown above would be as follows:

```
syn.directive.reset=top variable=b
syn.directive.reset=top variable=c
syn.directive.reset=top variable=d
syn.directive.reset=top variable=t.a
syn.directive.reset=top variable=s.a
syn.directive.reset=top variable=s.a1
```

See Also

- [RTL Configuration](#)

syn.directive.stable

Description

`syn.directive.stable` is applied to arguments of a DATAFLOW or PIPELINE region and is used to indicate that an input or output of this region can be ignored when generating the synchronizations at entry and exit of the DATAFLOW region. This means that the reading accesses of that argument do not need to be part of the “first stage” of the task-level (resp. fine-grain) pipeline for inputs, and the write accesses do not need to be part of the last stage of the task-level (resp. fine-grain) pipeline for outputs.

The pragma can be specified at any point in the hierarchy, on a scalar or an array, and automatically applies to all the DATAFLOW or PIPELINE regions below that point. The effect of STABLE for an input is that a DATAFLOW or PIPELINE region can start another iteration even though the value of the previous iteration has not been read yet. For an output, this implies that a write of the next iteration can occur although the previous iteration is not done.

Syntax

```
syn.directive.stable=<location> <variable>
```

- `<location>` is the function name or loop name where the directive is to be constrained.
- `<variable>` is the name of the array to be constrained.

Examples

In the following example, without the STABLE directive, `proc1` and `proc2` would be synchronized to acknowledge the reading of their inputs (including `A`). With the directive, `A` is no longer considered as an input that needs synchronization.

```
void dataflow_region(int A[...], int B[...] ...  
    proc1(...);  
    proc2(A, ...);
```

The directives for this example would be scripted as:

```
syn.directive.stable=dataflow_region variable=A  
syn.directive.dataflow dataflow_region
```

See Also

- [syn.directive.dataflow](#)
- [syn.directive.pipeline](#)

syn.directive.stream

Description

By default, array variables are implemented as RAM:

- Top-level function array parameters are implemented as a RAM interface port.
- General arrays are implemented as RAMs for read-write access.
- Arrays involved in sub-functions, or loop-based DATAFLOW optimizations are implemented as a RAM ping-pong buffer channel.

If the data stored in the array is consumed or produced in a sequential manner, a more efficient communication mechanism is to use streaming data, where FIFOs are used instead of RAMs. When an argument of the top-level function is specified as `INTERFACE mode=ap_fifo`, the array is automatically implemented as streaming. See the Interfaces for Vivado IP Flow section of the Vitis HLS User Guide ([UG1399](#)) for more information.



IMPORTANT! To preserve the accesses, it might be necessary to prevent compiler optimizations (in particular dead code elimination) by using the *volatile* qualifier as described in the Type Qualifiers section of the Vitis HLS User Guide ([UG1399](#)).

Syntax

```
syn.directive.stream=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<variable>` is the array variable to be implemented as a FIFO.

Options

- `depth=<integer>`:

Note: Relevant only for array streaming in dataflow channels.

By default, the depth of the FIFO implemented in the RTL is the same size as the array specified in the C code. This options allows you to modify the size of the FIFO.

When the array is implemented in a DATAFLOW region, it is common to use the `-depth` option to reduce the size of the FIFO. For example, in a DATAFLOW region where all loops and functions are processing data at a rate of $11 = 1$, there is no need for a large FIFO because data is produced and consumed in each clock cycle. In this case, you can use the `-depth` to reduce the FIFO size to 2 to substantially reduce the area of the RTL design.

This same functionality is provided for all arrays in a DATAFLOW region using the `config_dataflow` command with the `-depth` option. The `-depth` option used with `set_directive_stream` overrides the default specified using `config_dataflow`.

- **type=<arg>**: Specify a mechanism to select between FIFO, PIPO, synchronized shared (`shared`), and un-synchronized shared (`unsync`). The supported types include:
 - `fifo`: A FIFO buffer with the specified `depth`.
 - `pipo`: A regular Ping-Pong buffer, with as many “banks” as the specified depth (default is 2).
 - `shared`: A shared channel, synchronized like a regular Ping-Pong buffer, with depth, but without duplicating the array data. Consistency can be ensured by setting the depth small enough, which acts as the distance of synchronization between the producer and consumer.



TIP: The default depth for `shared` is 1.

- `unsync`: Does not have any synchronization except for individual memory reads and writes. Consistency (read-write and write-write order) must be ensured by the design itself.

Examples

Specifies array `A[10]` in function `func` to be streaming and implemented as a FIFO.

```
syn.directive.stream=func A type=fifo
```

Array `B` in named loop `loop_1` of function `func` is set to streaming with a FIFO depth of 12. In this case, place the pragma inside `loop_1`.

```
syn.directive.stream=depth=12 type=fifo func/loop_1 B
```

Array `C` has streaming implemented as a PIPO.

```
syn.directive.stream=type=pipo func C
```

See Also

- [syn.directive.dataflow](#)
- [syn.directive.interface](#)
- [Dataflow Configuration](#)

syn.directive.top

Description

Attaches a name to a function, which can then be used by the `set_top` command to set the named function as the top. This is typically used to synthesize member functions of a class in C++.

Syntax

```
syn.directive.top=[OPTIONS] <location>
```

- `<location>` is the function to be renamed.

Options

- `name=<string>`: Specifies the name of the function to be used by the `syn.top` command as described in [HLS General Options](#).

Examples

Function `foo_long_name` is renamed to `DESIGN_TOP`, which is then specified as the top-level. If the pragma is placed in the code, the `set_top` command must still be issued in the top-level specified in the GUI project settings.

```
syn.directive.top=name=DESIGN_TOP foo_long_name
```

Followed by the `syn.top=DESIGN_TOP` command.

See Also

- `syn.top` in [HLS General Options](#)

syn.directive.unroll

Description

Transforms loops by creating multiples copies of the loop body.

A loop is executed for the number of iterations specified by the loop induction variable. The number of iterations can also be impacted by logic inside the loop body (for example, break or modifications to any loop exit variable). The loop is implemented in the RTL by a block of logic representing the loop body, which is executed for the same number of iterations.

The `syn.directive.unroll` command allows the loop to be fully or partially unrolled. Fully unrolling the loop creates as many copies of the loop body in the RTL as there are loop iterations. Partially unrolling a loop by a factor *N*, creates *N* copies of the loop body and adjusting the loop iteration accordingly.

If the factor *N* used for partial unrolling is not an integer multiple of the original loop iteration count, the original exit condition must be checked after each unrolled fragment of the loop body.

To unroll a loop completely, the loop bounds must be known at compile time. This is not required for partial unrolling.

Syntax

```
syn.directive.unroll=[OPTIONS] <location>
```

- **<location>** is the location of the loop (in the format `function[/label]`) to be unrolled.

Options

- **factor=<integer>**: Specifies a non-zero integer indicating that partial unrolling is requested.

The loop body is repeated this number of times. The iteration information is adjusted accordingly.

- **skip_exit_check**: Effective only if a factor is specified (partial unrolling).
 - Fixed bounds
No exit condition check is performed if the iteration count is a multiple of the factor.
If the iteration count is *not* an integer multiple of the factor, the tool:
 - Prevents unrolling.
 - Issues a warning that the exit check must be performed to proceed.
 - Variable bounds
The exit condition check is removed. You must ensure that:
 - The variable bounds is an integer multiple of the factor.
 - No exit check is in fact required.
- **off=true**: Disable unroll for the specified loop.

Examples

Unrolls loop `L1` in function `foo`. Place the pragma in the body of loop `L1`.

```
syn.directive.unroll=foo/L1
```

Specifies an unroll factor of 4 on loop `L2` of function `foo`. Removes the exit check. Place the pragma in the body of loop `L2`.

```
syn.directive.unroll=skip_exit_check factor=4 foo/L2
```

See Also

- [syn.directive.loop_flatten](#)

- [syn.directive.loop_merge](#)

HLS Pragmas

Optimizations in Vitis HLS

In the AMD Vitis™ software platform, a kernel defined in the C/C++ language, or OpenCL™ C, must be compiled into the register transfer level (RTL) that can be implemented into the programmable logic of an AMD device. The v++ compiler calls the Vitis High-Level Synthesis (HLS) tool to synthesize the RTL code from the kernel source code.

The HLS tool is intended to work with the Vitis IDE project without interaction. However, the HLS tool also provides pragmas that can be used to optimize the design, reduce latency, improve throughput performance, and reduce area and device resource usage of the resulting RTL code. These pragmas can be added directly to the source code for the kernel.

The HLS pragmas include the optimization types specified in the following table.

For detailed pragma information, refer to the *Vitis High-Level Synthesis User Guide* ([UG1399](#)).

Table 42: Vitis HLS Pragmas by Type

Type	Attributes
Kernel Optimization	• pragma HLS aggregate • pragma HLS bind_op • pragma HLS bind_storage • pragma HLS expression_balance • pragma HLS latency • pragma HLS reset • pragma HLS top
Function Inlining	• pragma HLS inline
Interface Synthesis	• pragma HLS interface
Task-level Pipeline	• pragma HLS dataflow • pragma HLS stream
Pipeline	• pragma HLS pipeline • pragma HLS occurence
Loop Unrolling	• pragma HLS unroll • pragma HLS dependence
Loop Optimization	• pragma HLS loop_flatten • pragma HLS loop_merge • pragma HLS loop_tripcount
Array Optimization	• pragma HLS array_partition • pragma HLS array_reshape

Table 42: Vitis HLS Pragmas by Type (cont'd)

Type	Attributes
Structure Packing	pragma HLS aggregate pragma HLS dataflow

v++ Linking and Packaging Options

Linking

The details of the v++ linking process are described in [Building the Device Binary](#). For systems that incorporate the AI Engine graph applications, additional information can be found in *AI Engine Tools and Flows User Guide* ([UG1076](#)) under the heading of Linking the System.

The individual v++ `--link` commands can be found in the v++ Command chapter.

Packaging

The details of the v++ packaging process are provided in [Building and Packaging the System](#). For systems that incorporate the AI Engine graph applications, additional information can be found in *AI Engine Tools and Flows User Guide* ([UG1076](#)) under the heading of Packaging.

The individual v++ `--package` commands can be found in [--package Options](#).

--advanced Options

The `--advanced.param` and `--advanced.prop` options specify parameters and properties for use by the v++ command. When compiling or linking, these options offer fine-grain control over the hardware generated by the Vitis core development kit, and the hardware emulation process.

The arguments for the `--advanced.xxx` options are specified as `<param_name>=<param_value>`. For example:

```
v++ --link --advanced.param compiler.enableXSAIntegrityCheck=true
--advanced.prop kernel.foo.kernel_flags="-std=c++0x"
```



TIP: The order of precedence between the command line and the config file, and multiple entries in the config file are described in [Vitis Compiler Configuration File](#). However, the `--advanced` commands described here have the opposite precedence: config file commands take precedence over command-line arguments, and the last command encountered takes precedence over any earlier occurrences.

--advanced.param

```
--advanced.param <param_name>=<param_value>
```

Specifies advanced parameters as described in the following table.

Param Options*Table 43: Param Options*

Parameter Name	Valid Values	Description
<code>compiler.acceleratorBinaryContent</code>	Type: String Default Value: <code><empty></code>	Design content to insert in the generated <code>xclbin</code> file. Valid options include <code>bitstream</code>, <code>pdi</code>, or <code>dcp</code>. <code>bitstream</code> and <code>pdi</code> are mutually exclusive. <code>pdi</code> applies to Versal platforms, <code>bitstream</code> applies to non-Versal platforms. TIP: You can specify two values to have <code>v++</code> generate two <code>xclbin</code> files: one containing a DCP file, and the other containing either a bitstream or PDI file. For example: <pre>--advanced.param compiler.acceleratorBinaryContent=dcp,bitstream --advanced.param compiler.acceleratorBinaryContent=dcp,pdi</pre> This parameter is used while building the hardware target, this option applies to: <code>v++ --link</code> <code>vpl.impl</code> <code>xclbinutil</code>
<code>compiler.addOutputTypes</code>	Type: String Default Value: <code><empty></code>	Additional output types produced by the Vitis compiler. Valid values include: <code>xclbin</code> and <code>hw_export</code>. Use <code>hw_export</code> to create a fixed XSA from dynamic hardware platforms for use in the Embedded Software Development Flow. Applies to: <code>v++ --link</code> <code>vpl.impl</code> - XSA generation
<code>compiler.axisDeadLockFree</code>	Type: Boolean Default Value: <code>TRUE</code>	Avoid dead locks. This option is enabled by default for Vitis HLS.

Table 43: Param Options (cont'd)

Parameter Name	Valid Values	Description
<code>compiler.deadlockDetection</code>	Type: Boolean Default Value: FALSE	Enables detection of kernel deadlocks during the simulation run as part of hardware emulation. The tool posts an Error message to the console and the log file when the application is deadlocked: <pre>// ERROR!!! DEADLOCK DETECTED at 42979000 ns! SIMULATION WILL BE STOPPED! //</pre> The message is repeated until the deadlock is terminated. You must manually terminate the application to end the deadlock condition. TIP: When deadlocks are encountered during simulation, you can open the kernel code in Vitis HLS for additional deadlock detection and debug capability. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config-export</code>
<code>compiler.emulationMode=<mode></code>	Type: String func rtl	Indicates that the kernel should be compiled as RTL code for use in hardware emulation and hardware design, or as a C functional model with a SystemC wrapper for use in hardware emulation as described in Working with Functional Model of the HLS Kernel. This option applies to <code>v++ --compile</code>. The default is to compile the kernel as RTL code.
<code>compiler.enableIncrHwEmu</code>	Type: Boolean Default Value: FALSE	Use to enable incremental compilation of the hardware emulation <code>xclbin</code> when there are minor changes made to the platform. This enables a quick rebuild of the device binary for hardware emulation when the platform has been updated. Applies to: <code>v++ --link</code> <code>vpl.impl</code>
<code>compiler.errorOnHoldViolation</code>	Type: Boolean Default Value: TRUE	After the last step of Vivado implementation, during timing analysis check, and clock scaling if needed. If hold violations are found, <code>v++</code> quits and returns an error by default, and does not generate an <code>xclbin</code>. This parameter lets you over ride the default behavior. Applies to: <code>v++ --link</code> <code>vpl.impl</code>
<code>compiler.fsanitize</code>	Type: String Default Value: <empty>	Enables additional memory access checks for OpenCL kernels as described in Debugging OpenCL Kernels. Valid values include: address, memory. Applies to Software Emulation and Debug.
<code>compiler.interfaceRdBurstLen</code>	Type: Int Range Default Value: 0	Specifies the expected length of AXI read bursts on the kernel AXI interface. This is used with option <code>compiler.interfaceRdOutstanding</code> to determine the hardware buffer sizes. Values are 1 through 256. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config-interface</code>

Table 43: Param Options (cont'd)

Parameter Name	Valid Values	Description
<code>compiler.interfaceWrBurstLen</code>	Type: Int Range Default Value: 0	Specifies the expected length of AXI write bursts on the kernel AXI interface. This is used with option <code>compiler.interfaceWrOutstanding</code> to determine the hardware buffer sizes. Values are 1 through 256. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config_interface</code>
<code>compiler.interfaceRdOutstanding</code>	Type: Int Range Default Value: 0	Specifies how many outstanding reads to buffer are on the kernel AXI interface. Values are 1 through 256. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config_interface</code>
<code>compiler.interfaceWrOutstanding</code>	Type: Int Range Default Value: 0	Specifies how many outstanding writes to buffer are on the kernel AXI interface. Values are 1 through 256. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config_interface</code>
<code>compiler.maxComputeUnits</code>	Type: Int Default Value: -1	Maximum compute units allowed in the system. The default is 60 compute units, or is specified in the hardware platform (<code>.xsa</code>) with the <code>numComputeUnits</code> property. The specified value overrides the default value or the hardware platform. The default value of -1 preserves the default. Applies to <code>v++ --link</code> .
<code>compiler.skipTimingCheckAndFrequencyScaling</code>	Type: Boolean Default Value: FALSE	This parameter causes the Vivado tool to skip the timing check and optional clock frequency scaling that occurs after the last step of implementation process, which is either <code>route_design</code> or post-route <code>phys_opt_design</code> . Applies to: <code>v++ --link</code> <code>vpl.impl</code>
<code>compiler.userPreCreateProjectTcl</code>	Type: String Default Value: <empty>	Specifies a Tcl script to run before creating the Vivado project in the Vitis build process. Applies to: <code>v++ --link</code> <code>vpl.create_project</code>
<code>compiler.userPreSysLinkOverlayTcl</code>	Type: String Default Value: <empty>	Specifies a Tcl script to run after opening the Vivado IP integrator block design, before running the compiler-generated <code>dr.bd.tcl</code> script in the Vitis build process. Applies to: <code>v++ --link</code> <code>vpl.create_bd</code>

Table 43: Param Options (cont'd)

Parameter Name	Valid Values	Description
<code>compiler.userPostSysLinkOverlayTcl</code>	Type: String Default Value: <code><empty></code>	Specifies a Tcl script to run after running the compiler-generated <code>dr.bd.tcl</code> script. Applies to: <code>v++ --link</code> <code>vp1.update_bd</code>
<code>compiler.userPostDebugProfileOverlayTcl</code>	Type: String Default Value: <code><empty></code>	Specifies a Tcl script to run after debug profile overlay insertion in Vivado IP integrator block design in the <code>vp1.update_bd</code> step. Applies to: <code>v++ --link</code> <code>vp1.updated_bd</code>
<code>compiler.worstNegativeSlack</code>	Type: Float Default Value: 0	During timing analysis check, this specifies the worst acceptable negative slack for the design, specified in nanoseconds (ns). When negative slack exceeds the specified value, the tool might try to scale the clock frequency to achieve timing results. This specifies an acceptable negative slack value instead of zero slack. Applies to: <code>v++ --link</code> <code>vp1.impl</code>
<code>compiler.xclDataflowFifoDepth</code>	Type: Int Default Value: -1	Specifies the depth of FIFOs used in kernel data flow region. Applies to: <code>v++ --compile</code> - Vitis HLS <code>config_dataflow</code>
<code>hw_emu.aie_shim_sol_path</code>	Type: String Default Value: <code><empty></code>	For use by Versal platforms, this option specifies the path to the AI Engine SHIM Solution constraints file which is generated by the <code>aiecompiler</code> . Used during simulation, compilation, and elaboration, the file provides a logical mapping to the physical interface. This is needed for third-party simulators like Mentor Graphics Questa Advanced Simulator or Cadence Xcelium Logic Simulation.
<code>hw_emu.compiledLibs</code>	Type: String Default Value: <code><empty></code>	Uses mentioned <code>clibs</code> for the specified simulator. Applies to Hardware Emulation and Debug.
<code>hw_emu.debugMode</code>	<code>wdb</code> Default Value: <code>wdb</code>	The default value is WDB and runs simulation in waveform mode. This option only works in combination with the <code>-g</code> or <code>--debug</code> options. Applies to Hardware Emulation and Debug.
<code>hw_emu.enableProtocolChecker</code>	Type: Boolean Default Value: FALSE	Enables the lightweight AXI protocol checker (lapc) during HW emulation. This is used to confirm the accuracy of any AXI interfaces in the design. Applies to Hardware Emulation and Debug.
<code>hw_emu.json_device_file_path</code>	Type: String Default Value: <code><empty></code>	For use by Versal platforms, this option specifies the path to the AI Engine JSON Device file located in the Vitis software installation area. Used during simulation, compilation, and elaboration, the file specifies the size of the AI Engine array. This is needed for third-party simulators like Mentor Graphics Questa Advanced Simulator or Cadence Xcelium Logic Simulation.

Table 43: Param Options (cont'd)

Parameter Name	Valid Values	Description
<code>hw_emu.platformPath</code>	Type: String Default Value: <empty>	Specifies the path to the custom platform directory. The <platformPath> directory should meet the following requirements to be used in platform creation: - The directory should contain a subdirectory called <code>ip_repo</code>. - The directory should contain a subdirectory called <code>scripts</code> and this <code>scripts</code> directory should contain a <code>hw_em_util.tcl</code> file. The <code>hw_em_util.tcl</code> file should have the following two procedures defined in it: <code>hw_em_util::add_base_platform</code> <code>hw_em_util::generate_simulation_scripts_and_compile</code> Applies to Hardware Emulation and Debug.
<code>hw_emu.post_sim_settings</code>	Type: String	Specifies the path to a Tcl script that is used to configure the settings of the Vivado simulator prior to running hardware emulation. This script is run after the default configuration of the tool, but prior to launching simulation. You can use the Tcl script to override specific settings, or to custom configure the simulator as needed. Applies to Hardware Emulation and Debug.
<code>hw_emu.reduceHwEmuCompileTime</code>	Type: Boolean Default Value: FALSE	Move the generation of the top-level block design into the Generate Targets step of <code>v++ --link</code> . Applies to Hardware Emulation and Debug.
<code>hw_emu.scDebugLevel</code>	none waveform log waveform_and_log Default Value: waveform_and_log	Sets the TLM transaction debug level of the Vivado logic simulator (<code>xsim</code>). - NONE to disable TLM debug - LOG to dump TLM transaction log info into report file - WAVEFORM for enabling the TLM transaction waveform view - WAVEFORM_AND_LOG for both the Log Messages and Waveform view Applies to Hardware Emulation and Debug.
<code>hw_emu.simulator</code>	XSIM QUESTA Default Value: XSIM	Uses the specified simulator for the hardware emulation run. Applies to Hardware Emulation and Debug.

For example:

```
--advanced.param compiler.addOutputTypes="hw_export"
```



TIP: This option can be specified in a configuration file under the `[advanced]` section head using the following format:

```
[advanced]
param=compiler.addOutputTypes="hw_export"
```

--advanced.prop

```
--advanced.prop <arg>
```

Specifies advanced kernel or solution properties for kernel compilation where `<arg>` is one of the values described in the table below.

Table 44: Prop Options

Property Name	Valid Values	Description
<code>kernel.<kernel_name>.kernel_flags</code>	Type: String Default Value: <code><empty></code>	Sets specific compile flags on the kernel <code><kernel_name></code> .
<code>solution.device_repo_path</code>	Type: String Default Value: <code><empty></code>	Specifies the path to a repository of hardware platforms. The <code>--platform</code> option with full path to the <code>.xpfm</code> platform file should be used instead.
<code>solution.kernel_compiler_margin</code>	Type: Float Default Value: 12.5% of the kernel clock period.	The clock margin (in ns) for the kernel. This value is subtracted from the kernel clock period prior to synthesis to provide some margin for place and route delays.

--advanced.misc

```
--advanced.misc <arg>
```

Specifies advanced tool directives for kernel compilation.

--clock Options



IMPORTANT! The `--clock` options described here can be used with `--frequency` to manage clocks as described in [Managing Clock Frequencies](#).

The `--clock` options are used to assign the clock frequency during the compilation of AI Engine, HLS kernels and linking of the design. It is supported in both `--part` and `--platform` options. If no option is provided, Vitis assigns the default clock per the respective mode selected as shown in the following table.

Table 45: Default clock assigned

V++	-c --mode aie	-c --mode hls	-c -k	-l
<code>--part</code>	¼ of AIE PLL Freq	Default HLS (100 MHz)	N/A	300 MHz for Versal device
<code>--platform</code>	Default Platform Clock Freq	Default Platform Clock Freq	Default Platform Clock Freq	Default Platform Clock Freq

There are different clock directives for different `v++ -c` modes and `v++ -l`. `--frequency` is a new clock directive introduced in 2023.2 that can be used in all `v++ -c` modes and `v++ -l`, helping you to use one clock directive in all different modes. Older directives are still supported.

There are two ways to connect the kernel to the specified frequency using the `--clock` option:

1. `--clock.freqhz` or `--freqhz`: This option is used to connect the specified clock frequency from the platform to the kernel. If a platform has one or more clocks at the specified frequency, one of them will be selected. If a platform doesn't have the specified frequency, the specified frequency is generated using the platform reference clock.
2. `--clock.id`: This option can also be used to connect a specified clock frequency to the kernel; however, in this option the clock is considered as a clock source. This option can be used to connect the kernel or multiple kernels on the same clock source. See the following example for details.

Note: You can determine the available clock IDs for a target platform using the `platforminfo` utility as described in [platforminfo Utility](#).

To determine the fixed clocks available in the platform

```
xilinx_vck190_base_bdc_202320_1.xpfm:
```

```
Platforminfo ../ xilinx_vck190_base_bdc_202320_1.xpfm
```

Refer to “Clock Information” in the report:

```
=====
Clock Information
=====
Default Clock Index: 2
Clock Index: 0
Frequency: 104.166666
Clock Index: 1
Frequency: 156.250000
Clock Index: 2
Frequency: 312.500000
Clock Index: 3
Frequency: 78.125000
Clock Index: 4
Frequency: 208.333333
Clock Index: 5
Frequency: 416.666666
Clock Index: 6
Frequency: 625.000000
```

Suppose you want to connect kernel X to 312.5 MHz and kernel Y to 625.00 MHz, considering that both frequencies are generated in the platform. It is recommended to use the `--freqhz` option to assign the specific clock to kernel X and Y during the compilation and linking phase.

In compile mode (`v++ -c --mode aie` or `v++ -c --mode hls`), the `--clock` option `--freqhz` can be used in CLI as:

```
v++ -c --mode hls --platform <xx.xpfm> --freqhz=312500000 --config ./
X.cfg
```

```
v++ -c --mode hls --platform <xx.xpfm> --freqhz=625000000 --config ./
Y.cfg
```

In the config file, it can be given as per the following.

For the X.cfg file:

```
[clock]
freqhz=312500000
```

For the Y.cfg file:

```
[clock]
freqhz=625000000
```

In link mode (`v++ -l`), the syntax is `<cu>.<clk_pin_name>`.

In CLI, the command can be used as:

```
v++ -l -t hw --platform ./<.xpfm> ./libadf.a ./Y.xo --
freqhz=312500000:X.clk --freqhz=625000000:Y.clk -s --config ./
system.cfg -o fixed.xsa
```

Note: The key portion is `--freqhz=312500000:X.clk --freqhz=625000000:Y.clk`.

In the config file, it can be given as:

```
[clock]
freqhz=312500000: X.clk
freqhz=625000000 : Y.clk
```

Here, `clk` is the pin name of the kernel.

Note: The `clock` directive can be used either in CLI or in the configuration file.

--clock.tolerance

```
--clock.tolerance <arg>
```

Specifies a clock tolerance as a value, or as a percentage of the clock frequency. When specifying `--clock.freqhz`, you can also specify the tolerance with either a value or percentage. This updates the timing constraints to reflect the accepted tolerance. `<arg>` is specified as `<tolerance>:<cu_0>[.<clk_pin_0>][,<cu_n>[.<clk_pin_n>]]`

- **<tolerance>:**
 - Can be specified either as a whole number, indicating the `clock.freqhz ±` the specified tolerance value; or as a percentage of the clock frequency specified as a decimal value.
 - `<cu_0>[.<clk_pin_0>][,<cu_n>[.<clk_pin_n>]]`: Applies the defined clock tolerance to the specified CUs, and optionally to the specified clock pin on the CU.
 - `<cu_0>[.<clk_pin_0>][,<cu_n>[.<clk_pin_n>]]`: Applies the defined clock tolerance to the specified CUs, and optionally to the specified clock pin on the CU.



IMPORTANT! The default clock tolerance is 5% of the specified frequency when this option is not specified.

For example:

```
v++ --link --clock.tolerance 0.10:vadd_1,vadd_3
```



TIP: This option can be specified in a configuration file under the `[clock]` section head using the following format:

```
[clock]
tolerance=0.10:vadd_1,vadd_3
```

--connectivity Options

As discussed in [Linking the System](#), there are a number of `--connectivity.XXX` options that let you define the topology of the FPGA binary, specifying the number of CUs, assigning them to SLRs, connecting kernel ports to global memory, and establishing streaming port connections. These commands are an integral part of the build process, critical to the definition and construction of the application.

--connectivity.nk

```
--connectivity.nk <arg>
```

Where `<arg>` is specified as

```
<kernel_name>:#:<cu_name1>,<cu_name2>,...<cu_name#>.
```

This instantiates the specified number of CU (#) for the specified kernel (`kernel_name`) in the generated FPGA binary (`.xclbin`) file during the linking process. The `cu_name` is optional. If the `cu_name` is not specified, the instances of the kernel are simply numbered: `kernel_name_1`, `kernel_name_2`, and so forth. By default, the Vitis compiler instantiates one compute unit for each kernel.

For example:

```
v++ --link --connectivity.nk vadd:3:vadd_A,vadd_B,vadd_C
```



TIP: This option can be specified in a configuration file under the `[connectivity]` section head using the following format:

```
[connectivity]
nk=vadd:3:vadd_A,vadd_B,vadd_C
```

--connectivity.sc

```
--connectivity.sc <arg>
```

Create a streaming connection between two compute units through their AXI4-Stream interfaces. Use a separate `--connectivity.sc` option for each streaming interface connection. The order of connection must be from a streaming output port of the first kernel to a streaming input port of the second kernel. Valid values include:

```
<cu_name>.<streaming_output_port>:<cu_name>.<streaming_input_port>[:<fifo_depth>]
```

Where:

- `<cu_name>` is the compute unit name specified in the `--connectivity.nk` option. Generally this is `<kernel_name>_1` unless a different name was specified.
- `<streaming_output_port>/<streaming_input_port>` is the function argument for the compute unit port that is declared as an AXI4-Stream.
- `[:<fifo_depth>]` inserts a FIFO of the specified depth between the two streaming ports to prevent stalls. The value is specified as an integer.



IMPORTANT! An error will occur if the `--connectivity.sc` kernel drives itself.

For example, to connect the AXI4-Stream port `s_out` of the compute unit `mem_read_1` to AXI4-Stream port `s_in` of the compute unit `increment_1`, use the following:

```
--connectivity.sc mem_read_1.s_out:increment_1.s_in
```



TIP: This option can be specified in a configuration file under the `[connectivity]` section head using the following format:

```
[connectivity]
sc=mem_read_1.s_out:increment_1.s_in
```

The inclusion of the optional `<fifo_depth>` value lets the `v++` linker add a FIFO between the two kernels to help prevent stalls. This uses BRAM resources from the device when specified, but eliminates the need to update the HLS kernel to contain FIFOs. The tool also instantiates a Clock Converter (CDC) or Datawidth Converter (DWC) IP if the connections have different clocks, or different bus widths.

--connectivity.slr

```
--connectivity.slr <arg>
```

Use this option to assign a CU to a specific SLR on the device. The option must be repeated for each kernel or CU being assigned to an SLR.



IMPORTANT! If you use `--connectivity.slr` to assign the kernel placement, then you must also use `--connectivity.sp` to assign memory access for the kernel.

Valid values include:

```
<cu_name>:<SLR_NUM>
```

Where:

- `<cu_name>` is the name of the compute unit as specified in the `--connectivity.nk` option. Generally this is `<kernel_name>_1` unless a different name was specified.
- `<SLR_NUM>` is the SLR number to assign the CU to. For example, SLR0, SLR1.

For example, to assign CU `vadd_2` to SLR2, and CU `fft_1` to SLR1, use the following:

```
v++ --link --connectivity.slr vadd_2:SLR2 --connectivity.slr fft_1:SLR1
```



TIP: This option can be specified in a configuration file under the `[connectivity]` section head using the following format:

```
[connectivity]
slr=vadd_2:SLR2
slr=fft_1:SLR1
```

--connectivity.sp

```
--connectivity.sp <arg>
```

Use this option to specify the assignment of kernel arguments to system ports within the platform. A primary use case for this option is to connect kernel arguments to specific memory resources. A separate `--connectivity.sp` option is required to map each argument of a kernel to a particular memory resource. Any argument not explicitly mapped to a memory resource through the `--connectivity.sp` option is automatically connected to an available memory resource during the build process.



RECOMMENDED: AMD recommends specifying argument names when using the `--connectivity.sp` option as this provides the greatest connection flexibility. However, you can also specify kernel interface ports with this option.

Valid values include:

```
<cu_name>.<kernel_argument_name>:<sptag[min:max]>
```

Where:

- `<cu_name>` is the name of the compute unit as specified in the `--connectivity.nk` option. Generally this is `<kernel_name>_1` unless a different name was specified.

- `<kernel_argument_name>` is the name of the function argument for the kernel, or the compute unit interface port.
- `<sptag>` represents a system port tag, such as for memory controller interface names from the target platform. Valid `<sptag>` names include DDR, PLRAM, and HBM.
- `[min:max]` enables the use of a range of memory, such as DDR[0:2]. A single index is also supported: DDR[2].



TIP: The supported `<sptag>` and range of memory resources for a target platform can be obtained using the `platforminfo` command. Refer to [platforminfo Utility](#) for more information.

The following example maps the input argument (A) for the specified CU of the VADD kernel to DDR[0:3], input argument (B) to HBM[0:31], and writes the output argument (C) to PLRAM[2]:

```
v++ --link --connectivity.sp vadd_1.A:DDR[0:3] --connectivity.sp
vadd_1.B:HBM[0:31] \
--connectivity.sp vadd_1.C:PLRAM[2]
```



TIP: This option can be specified in a configuration file under the `[connectivity]` section head using the following format:

```
[connectivity]
sp=vadd_1.A:DDR[0:3]
sp=vadd_1.B:HBM[0:31]
sp=vadd_1.C:PLRAM[2]
```

--connectivity.noc.connect

```
--connectivity.noc.connect <arg>
```

Where `<arg>` is in the form of

`<compute_unit_name>.<kernel_interface_name>:<noc interface>`, and specifies a connection between the PL kernel interface and the Versal NoC. Valid values are internal memory controllers, or master interfaces on the Versal NoC cell.

The Vitis compiler estimates kernel bandwidth requirements based on NoC connectivity and M_AXI properties (datawidth * clock freq) across the dynamic region, and automatically sets NoC configuration settings for read and write bandwidth, scaling as needed to avoid exceeding the available bandwidth.

For example:

```
[connectivity]
noc.read_bw=mm2s.M_AXI:2000.16
noc.write_bw=mm2s.M_AXI:2010.16
noc.connect=mm2s.M_AXI:M00_INI
```

--connectivity.noc.read_bw

```
--connectivity.noc.read_bw <arg>
```


Where `<arg>` is in the form

`<compute_unit_name>.<kernel_interface_name>:<Bandwidth>.<Avg_burst_length>`, and specifies both the bandwidth and burst length of the connection. The bandwidth is specified in MB/s.

This option specifies expected read traffic characteristics on M_AXI interfaces to let you override the automatic Versal NoC configuration.

--connectivity.noc.write_bw

```
--connectivity.noc.write_bw <arg>
```

Where `<arg>` is in the form

`<compute_unit_name>.<kernel_interface_name>:<Bandwidth>.<Avg_burst_length>`, and specifies both the bandwidth and burst length of the connection. The bandwidth is specified in MB/s.

This option specifies expected write traffic characteristics on M_AXI interfaces to let you override the automatic Versal NoC configuration.

--connectivity.connect

```
--connectivity.connect <X:Y>
```

This option can be used to make connections through the Vivado IP integrator, but `v++` does not perform any error checking on the specified connections. Use this to specify general connections between kernels and non-AXI elements of the target platform, such as connections to GT ports.

The X and Y connections must be specified as arguments compatible with either the IP integrator `connect_bd_net` or `connect_bd_intf_net` commands. The specific format of `<X:Y>` is:

```
src/hierarchy_name/cell_name/pin_name:dst/hierarchy_name/cell_name/pin_name
```

These cannot include connections between AXI4-Stream interfaces which require the use of `--connectivity.sc`, or M_AXI interfaces which require the use of `--connectivity.sp` as described above.



TIP: This option can be specified in a configuration file under the `[connectivity]` section head using the following format:

```
[connectivity]
connect=<X:Y>
```

--debug Options

This option enables debug IP core insertion in the device binary (.xclbin) for hardware debugging. This option lets you specify the type of debug core to add, and which compute unit and interfaces to monitor with ChipScope™ as described in [Debugging During Hardware Execution](#). The --debug.xxx options lets you attach AXI protocol checkers and System ILA cores at the interfaces to kernels or specific compute units (CUs) for debugging and performance monitoring purposes:

- The System Integrated Logic Analyzer (ILA) provides transaction level visibility into an accelerated kernel or function running on hardware. AXI traffic of interest can also be captured and viewed using the [System ILA](#) core.
- The AXI Protocol Checker debug core is designed to monitor AXI interfaces on the accelerated kernel. When attached to an interface of a CU, the [AXI Protocol Checker](#) actively checks for protocol violations and provides an indication of which violation occurred.

The --debug.xxx commands can be specified in a configuration file under the [debug] section head using the following format as an example:

```
[debug]
protocol=all:all           # Protocol analyzers on all CUs
protocol=cu2:port3        # Protocol analyzer on port3 of cu2
chipscope=cu2             # ILA on cu2
```

The various options of --debug include the following:

--debug.aie.chipscope

```
--debug.aie.chipscope <interface_name> | <adf_graph_arg_name>
```

Enables hardware debug for the Versal AI Engine through ChipScope. The <interface_name> argument applies to non-PL kernel interfaces such as AI Engine PLIO interfaces, or AXIS interfaces. The <adf_graph_arg_name> specifies arguments of the graph.

--debug.chipscope

```
--debug.chipscope <cu_name>[:<interface_name>]
```

Adds the System Integrated Logic Analyzer debug core to the specified CUs in the design.



IMPORTANT! The --debug.chipscope option requires the <cu_name> to be specified and does not accept the keyword all. You can optionally specify an <interface_name>.

For example, the following command adds an ILA core to the vadd_1 CU:

```
v++ --link --debug.chipscope vadd_1
```

--debug.list_ports

Shows a list of valid compute units and port combinations in the current design. This is informational to help you with crafting a command line or config file for the `--debug` command.

This option needs to be specified during linking, but does not run the linking process. The required elements of the command line are shown in the following example, which returns the available ports when linking the specified kernels with the listed platform:

```
v++ --platform <platform> --link --debug.list_ports <kernel.xo>
```

--debug.protocol

```
--debug.protocol all|<cu_name>[:<interface_name>]
```

Adds the AXI Protocol Checker debug core to the design. This can be specified with the keyword `all`, or the `<cu_name>` and optional `<interface_name>` to add the protocol checker to the specified CU and interface.

For example:

```
v++ --link --debug.protocol all
```

--drc Options

This option lets you manage the Design Rule Check (DRC) messages returned by the Vivado Design Suite during implementation of the design. Messages can be disabled or enabled, reduced in severity, or waived to allow the design to proceed.

--drc.disable

```
--drc.disable <arg>
```

Lets you disable the DRC message specified by the DRC ID. Disabled DRCs are not checked.

For example:

```
v++ --link --drc.disable TIMING-18
```

--drc.enable

```
--drc.enable <arg>
```

Lets you enable the DRC message specified by the DRC ID. DRC checks that have been disabled can be re-enabled as needed.

For example:

```
v++ --link --drc.enable TIMING-18
```

--drc.severity

```
--drc.severity <arg>
```

Change the severity of a DRC check. For example, instead of a {CRITICAL WARNING} a violation is only reported as a WARNING. The DRC must be specified by ID, and the new severity to apply.

For example:

```
v++ --link --drc.severity TIMING-18:Warning
```

--drc.waive

```
--drc.waive <arg>
```

Lets you waive the specified DRC ID. This lets the Vivado tool proceed through implementation, ignoring DRCs that might otherwise prevent completion of the design. A waived rule is still checked, but it is marked as waived.

For example:

```
v++ --link --drc.waive TIMING-18
```

--hls Options

The `--hls.XXX` options described below are used to specify options for the Vitis HLS synthesis process invoked during kernel compilation.

--hls.clock

```
--hls.clock <arg>
```

Specifies a frequency in Hertz at which the listed kernel(s) should be compiled by Vitis HLS.

Where `<arg>` is specified as:

```
<frequency_in_Hz>:<cu_name1>,<cu_name2>,...,<cu_nameN>
```

- `<frequency_in_Hz>`: Defines the kernel frequency specified in Hz.
- `<cu_name1>,<cu_name2>,...`: Defines a list of kernels or kernel instances (CUs) to be compiled at the specified target frequency.

For example:

```
v++ -c --hls.clock 300000000:mmult,mmadd --hls.clock 100000000:fifo_1
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
clock=300000000:mmult,mmadd  
clock=100000000:fifo_1
```

--hls.export_mode

```
--hls.export_mode <file_type>:<file_path>
```

Specifies the RTL export mode for Vitis HLS and the path and name of the exported file. As a v++ compiler option, the only supported `<file_type>` is XO.

For example:

```
v++ --hls.export_mode xo:./kernel.xo
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
export_mode=xo:./kernel.xo
```

--hls.export_project

```
--hls.export_project <arg>
```

Specifies a directory where the Vitis HLS project setup script is exported.

For example:

```
v++ --hls.export_project ./hls_export
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
export_project=./hls_export
```

--hls.jobs

```
--hls.jobs <arg>
```

Specifies the number of jobs for launching HLS runs.

This option specifies the number of parallel jobs Vitis HLS uses to synthesize the RTL kernel code. Increasing the number of jobs allows the tool to spawn more parallel processes and complete faster.

For example:

```
v++ --hls.jobs 4
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
jobs=4
```

--hls.lsf

```
--hls.lsf <arg>
```

Specifies a `bsub` command to submit a job to LSF for HLS runs.

Specifies the `bsub` command line as a string to pass to an LSF cluster. This option is required to use the IBM Platform Load Sharing Facility (LSF) for Vitis HLS synthesis.

For example:

```
v++ --compile --hls.lsf '{bsub -R \"select[type=X86_64]\" -N -q medium}'
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
lsf='{bsub...}
```

--hls.post_tcl

```
--hls.post_tcl <arg>
```

Specifies a Tcl file containing Tcl commands for `vitis_hls` to source after `csynth_design`.

For example:

```
v++ --hls.post_tcl ./runPost.tcl
```



TIP: This option can be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
post_tcl=./runPost.tcl
```

--hls.pre_tcl

```
--hls.pre_tcl <arg>
```

Specifies a Tcl file containing Tcl commands for `vitis_hls` to source before running `csynth_design`.

For example:

```
v++ --hls.pre_tcl ./runPre.tcl
```

Where `runPre.tcl` contains the following commands to configure `m_axi` interfaces in Vitis HLS:

```
config_interface -m_axi_auto_max_ports=1  
config_interface -m_axi_max_bitwidth 512
```



TIP: This option can also be specified in a configuration file under the `[hls]` section head using the following format:

```
[hls]  
pre_tcl=./runPre.tcl
```

--linkhook Options

The `--linkhook.XXX` options described below are used to specify Tcl scripts to run at specific steps during the Vitis linking process. Valid steps can be determined using the `--linkhook.list_steps` command as described below.

--linkhook.custom

```
--linkhook.custom <step name, path to script file>
```

Specifies a Tcl script to execute at a predefined point in the build process. The path to specify the script can be an absolute path, or partial path relative to the build directory.

For example, the following command runs the specified Tcl script before the SysLink step in the build:

```
v++ -l --linkhook.custom preSysLink,./runScript.tcl
```

-linkhook.do_first

```
--linkhook.do_first <step name, path to script file>
```

Specifies a Tcl script to execute before the given step name. The path to specify the script can be an absolute path, or partial path relative to the build directory.

For example, the following command runs the specified Tcl script before the `place_design` step in the build:

```
v++ -l --linkhook.do_first vpl.impl.place_design,runScript.tcl
```

-linkhook.do_last

```
--linkhook.do_last <step name, path to script file>
```

Specifies a Tcl script to execute immediately after the given step completes. The path to specify the script can be an absolute path, or partial path relative to the build directory.

For example, the following command runs the specified Tcl script after the `place_design` step in the build:

```
v++ -l --linkhook.do_last vpl.impl.place_design,runScript.tcl
```

-linkhook.list_steps

```
--linkhook.list_steps
```

List default and optional build steps that support script hooks for a specified target. This command requires the `--target` to be specified in addition to the `--link` option.

For example:

```
v++ --target hw -l --linkhook.list_steps
```

The command returns both default steps that are always enabled during the build process, and optional steps that you can enable if needed. Refer to [Managing Vivado Synthesis, Implementation, and Timing Closure](#) for directions on enabling optional steps.

--package Options

Introduction

As described in [Packaging the System](#), the `v++ --package`, or `-p` step, generates and packages the final product at the end of the `v++` compile and link build process. This is a required step for all embedded platforms, including Versal devices, AI Engine, and AMD Zynq™ UltraScale+™ MPSoC devices.

The syntax of the `--package` command for Versal platforms is as follows:

```
v++ --package -t <sw_emu | hw_emu | hw> --platform <platform> input.xsa \  
[ -o output.xclbin --package.<options> ]
```



TIP: Package command options can be specified in a configuration file for use with the `--config` option, as discussed in the [Vitis Compiler Configuration File](#).

For non-Versal platforms the syntax for the `--package` command is:

```
v++ --package -t <sw_emu | hw_emu | hw> --platform <platform> input.xclbin \  
[ -o output.xclbin --package.<options> ]
```

The various options to specify as `--package.<options>` as shown in the syntax above include the following:

--package.aie_debug_port

```
--package.aie_debug_port <arg>
```

Where `<arg>` specifies a TCP port where emulator listens for incoming connections from the debugger to debug Versal AI Engine cores. The default port value is 10100.

For example:

```
v++ -l --package.aie_debug_port 1440
```

--package.bl31_elf

```
--package.bl31_elf <arg>
```

Where `<arg>` specifies the absolute or relative path to Arm trusted FW ELF that executes on A72 #0 core. If this option is not specified, then the Vitis compiler searches for the bl31 in the platform.

For example:

```
v++ -l --package.bl31_elf ./arm_trusted.elf
```

--package.boot_mode

```
--package.boot_mode <arg>
```

Where `<arg>` specifies `<ospi | qspi | sd>` Boot mode used for running the application in emulation or on hardware. For embedded platforms, the default boot mode is `sd`. For Data Center platforms, it is either `qspi` or `ospi` as appropriate.



TIP: The `xilinx_vck190-v202310-1` platform does not support the `qspi` option. Custom platforms that are configured to support it will work. The `ospi` option is for use with Versal Data Center platforms only.

For example:

```
v++ -l --package.boot_mode sd
```

--package.defer_aie_run

```
--package.defer_aie_run
```

Where this option specifies that the Versal AI Engine cores are enabled by an embedded processor (PS) application. When not specified, the tool generates CDO commands to enable the AI Engine cores during PDI load instead. By default this option is disabled, or FALSE.

For example:

```
v++ -l --package.defer_aie_run
```

--package.domain

```
--package.domain <arg>
```

Where `<arg>` specifies a domain name. If this option is not specified, then the Vitis compiler picks up the default domain from the software platform (SPFM) file. For AI Engine designs, this should always be `aiengine`.

For example:

```
v++ -l --package.domain xrt
```

--package.dtb

```
--package.dtb <arg>
```

Where `<arg>` specifies the absolute or relative path to device tree binary (DTB) used for loading Linux on the APU. If this options is not specified, then Vitis compiler searches for the `dtb` in the platform.

For example:

```
v++ -l --package.dtb ./device_tree.image
```

--package.emu_ps

```
--package.emu_ps <x86 | qemu>
```

Specifies that the PS application code has been compiled for the x86 processor instead of an Arm® processor, and will run under the system OS for C-simulation instead of PetaLinux/QEMU. The default setting is x86. To compile the PS application for use on the x86 processor refer to [Embedded Processor Emulation Using PS on x86](#).



IMPORTANT! *This option is only valid for the `sw_emu` target.*

For example:

```
v++ -l --package.emu_ps x86
```

--package.enable_aie_debug

```
--package.enable_aie_debug
```

When enabled, the tool generates CDO commands to stop the AI Engine cores during PDI load. By default this option is disabled, or FALSE.

For example:

```
v++ -l --package.enable_aie_debug
```

--package.image_format

```
--package.image_format <arg>
```

Where `<arg>` specifies `<ext4 | fat32>` output image file format used on the SD card. For embedded platforms with a Linux domain, the default image format is `ext4`. For all others, the image format is `fat32`.

- `ext4`: Linux file system
- `fat32`: Windows file system



IMPORTANT! *EXT4 format is not supported on Windows.*

For example:

```
v++ -l --package.image_format fat32
```

--package.kernel_image

```
--package.kernel_image <arg>
```

Where `<arg>` specifies the absolute or relative path to a Linux kernel image file. Overrides the existing image available in the platform. The platform image file is available for download from [xilinx.com](https://www.xilinx.com). Refer to the [Vitis Software Platform Installation](#) for more information. If this option is not specified, then the Vitis compiler copies the Linux image from the platform to the SD card folder.

For example:

```
v++ -l --package.kernel_image ./kernel_image
```

--package.no_image

```
--package.no_image
```

Bypass SD card image creation. Valid for `--package.boot_mode sd`. By default this option is disabled, or FALSE.

--package.out_dir

```
--package.out_dir <arg>
```

Where `<arg>` specifies the absolute or relative path to the output directory of the `--package` command. The default output directory is the directory from which the Vitis compiler is launched.

For example:

```
v++ -l --package.out_dir ./out_dir
```

--package.ps_debug_port

```
--package.ps_debug_port <arg>
```

Where `<arg>` specifies the TCP port where emulator listens for incoming connections from the debugger to debug PS cores.

For example:

```
v++ -l --package.debug_port 3200
```

--package.ps_elf

```
--package.ps_elf <arg>
```

Where `<arg>` specifies `<path_to_elf_file,core>`.

- `path_to_elf_file`: Specifies the ELF file for the PS core.

- `core`: Indicates the PS core it should run on.

Used when a baremetal ELF file is running on a device processor core. This option specifies an ELF file and processor core pair to be included in the boot image. The available processors for supported devices are listed below:

- Versal processor core values include: `a72-0`, `a72-1`, `a72-2`, and `a72-3`.
- Zynq UltraScale+ MPSoC processor core values include: `a53-0`, `a53-1`, `a53-2`, `a53-3`, `r5-0`, and `r5-1`.
- Zynq 7000 processor core values include: `a9-0` and `a9-1`.



TIP: Specify the option separately for each ELF/Core pair.

For example:

```
v++ -l --package.ps_elf a53_0.elf,a53-0 --package.ps_elf r5_0.elf,r5-0
```

--package.rootfs

```
--package.rootfs <arg>
```

Where `<arg>` specifies the absolute or relative path to a processed Linux root file system file. The platform RootFS file is available for download from Xilinx.com. Refer to the [Vitis Software Platform Installation](#) for more information. If this option is not specified, then the Vitis compiler picks up the default `rootfs` path from the software platform (SPFM) file.

For example:

```
v++ -l --package.rootfs ./rootfs.ext4
```

--package.sd_dir

```
--package.sd_dir <arg>
```

Where `<arg>` specifies a folder to package into the `sd_card` directory/image. The contents of the directory are copied to a sub-folder of the `sd_card` folder.

For example:

```
v++ -l --package.sd_dir ./test_data
```

--package.sd_file

```
--package.sd_file <arg>
```

Where `<arg>` specifies an ELF or other data file to package into the `sd_card` directory/image. This option can be used repeatedly to specify multiple files to add to the `sd_card`. The `.xclbin` and `libadf.a` files are automatically copied to the `out-dir` or `sd_card` folder.

For example:

```
v++ -l --package.sd_file ./arm_trusted.elf
```

--package.uboot

```
--package.uboot <arg>
```

Where `<arg>` specifies a path to U-Boot ELF file which overrides a platform U-Boot. If this option is not specified, then the Vitis compiler searches for the `uboot` in the platform.

For example:

```
v++ -l --package.uboot ./uboot.elf
```

--profile Options

As discussed in [Enabling Profiling in Your Application](#), there are a number of `--profile` options that let you enable profiling of the application and kernel events during runtime execution. This option enables capturing transaction details for data traffic between the kernel and host, kernel stalls, the execution times of kernels and compute units (CUs), in addition to monitoring activity in Versal AI Engines.



IMPORTANT! Using the `--profile` option in `v++` also requires the addition of one of the profile or trace options in the `xrt.ini` file. Refer to [xrt.ini File](#) for more information.

The `--profile` commands can be specified in a configuration file under the `[profile]` section head using the following format, for example:

```
[profile]
data=all:all:all           # Monitor data on all kernels and CUs
data=k1:all:all            # Monitor data on all instances of kernel k1
data=k1:cu2:port3         # Specific CU master
data=k1:cu2:port3:counters # Specific CU master (counters only, no trace)
memory=all                # Monitor transfers for all memories
memory=<sptag>             # Monitor transfers for the specified memory
stall=all:all             # Monitor stalls for all CUs of all kernels
stall=k1:cu2              # Stalls only for cu2
exec=all:all              # Monitor execution times for all CUs
exec=k1:cu2               # Execution times only for cu2
aie=all                   # Monitor all AIE streams
aie=DataIn1               # Monitor the specific input stream in the SDF
graph
aie=M02_AXIS              # Monitor specific stream interface
```

The various options of the command are described below:

--profile.aie:<arg>

Enables profiling of AI Engine streams in adaptive data flow (ADF) applications, where <arg> is:

```
<ADF_graph_argument|pin_name|all>
```

- <ADF_graph_argument>: Specifies an argument name from the ADF graph application.
- <pin_name>: Indicates a port on an AI Engine kernel.
- <all>: Indicates monitoring all stream connections in the ADF application.

For example, to monitor the `DataIn1` input stream use the following command:

```
v++ --link --profile.aie:DataIn1
```

--profile.aie_trace_offload:<arg>

Enables profiling of AI Engine streams in adaptive data flow (ADF) applications, where <arg> is HSDP, or high-speed debug port.

```
v++ --link --profile.aie_trace_offload:HSDP
```

The use of the HSDP is described in *Event Trace Offload using High Speed Debug Port in AI Engine Tools and Flows User Guide* ([UG1076](#)).

--profile.data:<arg>

Enables monitoring of data ports through monitor IP that are added into the design. This option needs to be specified during linking.

Where <arg> is:

```
[<kernel_name>|all]: [<cu_name>|all]: [<interface_name>|all] (: [<counters>|all])
```

- [<kernel_name>|all] defines either a specific kernel to apply the command to. However, you can also specify the keyword `all` to apply the monitoring to all existing kernels, compute units, and interfaces with a single option.
- [<cu_name>|all] when <kernel_name> has been specified, you can also define a specific CU to apply the command to, or indicate that it should be applied to all CUs for the kernel.
- [<interface_name>|all] defines the specific interface on the kernel or CU to monitor for data activity, or monitor all interfaces.
- [<counters>|all] is an optional argument, as it defaults to `all` when not specified. It allows restricting the information gathering to only `counters` for larger designs, while `all` will include the collection of actual trace information.

For example, to assign the data profile to all CUs and interfaces of kernel `k1` use the following command:

```
v++ --link --profile.data k1:all:all
```

--profile.exec:<arg>

This option records the execution times of the kernel and provides minimum port data collection during the system run. This option needs to be specified during linking.



TIP: The execution time of a kernel is collected by default when `--profile.data` or `--profile.stall` is specified. You can specify `--profile.exec` for any CUs not covered by `data` or `stall`.

The syntax for `exec` profiling is:

```
[<kernel_name>|all]: [<cu_name>|all] (: [<counters>|all])
```

For example, to profile to execution of `cu2` for kernel `k1` use the following command:

```
v++ --link --profile.exec:k1:cu2
```

--profile.stall:<arg>



IMPORTANT! This option must be specified during both `v++` compilation and linking.

Adds stall monitoring logic to the device binary (`.xclbin`) which requires the addition of stall ports on the kernel interface. To facilitate this, the `stall` option must be specified during both compilation and linking.

The syntax for `stall` profiling is:

```
[<kernel_name>|all]: [<cu_name>|all] (: [<counters>|all])
```

For example, to monitor stalls of `cu2` for kernel `k1` use the following command:

```
v++ --compile -k k1 --profile.stall ...
v++ --link --profile.stall:k1:cu2 ...
```

--profile.trace_memory:<arg>



TIP: This option applies to hardware build targets (`-t=hw`) only, and should not be used for software or hardware emulation flows.

When building the hardware target (`-t=hw`), use this option to specify the type and amount of memory to use for capturing trace data. You can specify the argument as follows:

```
<FIFO>:<size>|<MEMORY> [<n>] [:<SLR>]
```


This argument specifies memory type to use for capturing trace data. Use the `--profile.trace_memory` command to define the type or memory to use, with the `trace_buffer_size` switch in the `xrt.ini` file to define the amount of memory to use as described in [xrt.ini File](#). The default memory type used is the first memory defined in the platform, and the default buffer size is 1 MB.

When `trace_memory` is not specified but `device_trace` is enabled in the [xrt.ini File](#), the profile data is captured to the default platform memory with 1 MB allocated for the trace buffer.

- **FIFO:<size>**: Specified in KB. The maximum is 128K, although 64K is the maximum recommended.
- **Memory**: Specifies the type and number of memory resource on the platform. Memory resources for the target platform can be identified with the `platforminfo` command. Supported memory types include HBM, DDR, PLRAM, HP, ACP, MIG, and MC_NOC. For example, `DDR[1]`.
- **[:<SLR>]**: Optionally indicates that CUs assigned to the specified `<SLR>` should use the DDR or HBM resources specified in the `<MEMORY>` field. Note that this syntax can only be used with DDR or HBM memory banks.

You can specify the `--profile.trace_memory` command with the memory size and unit such as `FIFO:8k`, or specify the memory bank such as `DDR[0]` or `HBM[3]`. In this case, the profile data for all CUs are captured in the specified memory.

Or you can specify the memory to use to capture profile data, and the SLR assignment for that memory. In this case, the SLR assignment indicates that any CUs assigned to the specified SLR should have profile data captured in the specified memory. This is shown in the following config file example:

```
[profile]
trace_memory=DDR[1]:SLR0
trace_memory=DDR[2]:SLR1
```

In the example above, profile data for CUs assigned to SLR0 are captured in DDR bank 1, and CUs assigned to SLR1 are captured in DDR bank 2. CUs are assigned to SLRs using the `--connectivity.slr` command as described in [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#).



IMPORTANT! You cannot mix DDR and HBM memory banks in a single design, and when you specify the `<SLR>` syntax you must use that syntax for all `trace_memory` commands in the design.

--vivado Options

The `--vivado.XXX` options are used to configure the Vivado tools for synthesis and implementation of your device binary (`.xclbin`). For instance, you can specify the number of jobs to spawn, LSF commands to use for implementation runs, or the specific implementation strategies to use. You can also configure optimization, placement, timing, or specify which reports to output.



IMPORTANT! Familiarity with the Vivado Design Suite is required to make the best use of these options. See the Vivado Design Suite User Guide: Implementation ([UG904](#)) for more information.

--vivado.impl.jobs

```
--vivado.impl.jobs <arg>
```

Specifies the number of parallel jobs the Vivado Design Suite uses to implement the device binary. Increasing the number of jobs allows the Vivado implementation step to spawn more parallel processes and complete faster jobs.

For example:

```
v++ --link --vivado.impl.jobs 4
```

--vivado.impl.lsf

```
--vivado.impl.lsf <arg>
```

Specifies the `bsub` command line as a string to pass to an LSF cluster. This option is required to use the IBM Platform Load Sharing Facility (LSF) for Vivado implementation.

For example:

```
v++ --link --vivado.impl.lsf '{bsub -R \"select[type=X86_64]\" -N -q  
medium}'
```

--vivado.impl.strategies

```
--vivado.impl.strategies <arg>
```

Specifies a comma-separated list of strategy names for Vivado implementation runs. Use `ALL` to run all available implementation strategies. This lets you run a variety of implementation strategies at the same time during the build process and allows you to more quickly resolve the timing and routing issues of the design.



IMPORTANT! Running ALL implementation strategies might launch 30 or more runs in the Vivado tool. This can be a tremendous drain on resources, and is not advised. You can prevent this by defining a set of strategies to run, and using a command queue to distribute the process load in some managed way, such as through the `--vivado.impl.jobs` or the `--vivado.impl.lsf` commands.

--vivado.param

```
--vivado.param <arg>
```

Specifies parameters for the Vivado Design Suite to be used during synthesis and implementation of the FPGA binary (xclbin).



TIP: You can use the `report_param` Tcl command in the Vivado tool to identify the available parameters.

--vivado.prop

```
--vivado.prop <arg>
```

Specifies properties for the Vivado Design Suite to be used during synthesis and implementation of the FPGA binary (xclbin).

Table 46: Prop Options

Property Name	Valid Values	Description
<code>vivado.prop</code> <code><object_type>.<object_name>.<prop_name></code>	Type: Various	This allows you to specify any property used in the Vivado hardware compilation flow. <code><object_type></code> is <code>run fileset file project</code>. The <code><object_name></code> and <code><prop_name></code> values are described in <i>Vivado Design Suite Properties Reference Guide</i> (UG912). Examples: <pre>vivado.prop run.impl_1. {STEPS.PLACE_DESIGN.ARGS.MORE OPTIONS}={-no_bufg_opt}</pre> <pre>vivado.prop fileset. current.top=foo</pre> If <code><object_type></code> is set to <code>file</code>, <code>current</code> is not supported. If <code><object_type></code> is set to <code>run</code>, the special value of <code>__KERNEL__</code> can be used to specify run optimization settings for ALL kernels, instead of the need to specify them one by one.

For example, from the command line:

```
v++ --link --vivado.prop run.impl_1.STEPS.PHYS_OPT_DESIGN.IS_ENABLED=true
--vivado.prop run.impl_1.STEPS.PHYS_OPT_DESIGN.ARGS.DIRECTIVE=Explore
--vivado.prop run.impl_1.STEPS.PLACE_DESIGN.TCL.PRE=/.../xxx.tcl
```

The example above enables the optional `PHYS_OPT_DESIGN` step as part of the Vivado implementation process, sets the `Explore` directive for that step, and specifies a Tcl script to run before the `PLACE_DESIGN` step.



TIP: As described in [Managing Vivado Synthesis, Implementation, and Timing Closure](#), each step in the Vivado synthesis and implementation process can have a Tcl prescript to run before the step, and a Tcl postscript to run after the step. This lets you customize the build process by inserting pre-processing or post-processing Tcl commands around the different steps. These scripts can be specified as shown in the example above.

These options can also be specified in a configuration file under the `[vivado]` section head using the following format:

```
[vivado]
prop=run.impl_1.STEPS.PHYS_OPT_DESIGN.IS_ENABLED=true
prop=run.impl_1.STEPS.PHYS_OPT_DESIGN.ARGS.DIRECTIVE=Explore
prop=run.impl_1.STEPS.PLACE_DESIGN.TCL.PRE=/.../xxx.tcl
```



IMPORTANT! Some Vivado properties have spaces in their name, such as `MORE OPTIONS` and Tcl syntax requires these properties to be enclosed in braces, `{}`. However, the placement of the braces in the `--vivado` options is important. You must surround the complete property name with braces, rather than a portion of it. For instance, the correct placement would be:

```
--vivado.prop run.impl_1.{STEPS.PLACE_DESIGN.ARGS.MORE OPTIONS}={-
no_bufg_opt}
```

While the following would result in an error during the build process:

```
--vivado.prop "run.impl_1.{STEPS.PLACE_DESIGN.ARGS.MORE OPTIONS}={-
no_bufg_opt}"
```

--vivado.synth.jobs

```
--vivado.synth.jobs <arg>
```

Specifies the number of parallel jobs the Vivado Design Suite uses to synthesize the device binary. Increasing the number of jobs allows the Vivado synthesis to spawn more parallel processes and complete faster jobs.

For example:

```
v++ --link --vivado.synth.jobs 4
```

--vivado.synth.lsf

```
--vivado.synth.lsf <arg>
```

Specifies the `bsub` command line as a string to pass to an LSF cluster. This option is required to use the IBM Platform Load Sharing Facility (LSF) for Vivado synthesis.

For example:

```
v++ --link --vivado.synth.lsf '{bsub -R \"select[type=X86_64]\" -N -q  
medium}'
```

Vitis Compiler Configuration File

Configuration files are the recommended way of working with either the Vitis Unified IDE, or the common command-line supported by `v++`. A configuration file provides an organized way of passing options to the tools by grouping similar commands together, creating reusable configuration files to perform specific tasks like connectivity or profiling, and minimizing and simplifying the `v++` command line. Some of the features that can be controlled through config file entries include:

- AI Engine commands to configure component creation using `v++ -c --mode aie`
- HLS commands to configure interface definition, configure synthesis and simulation, and control packaging when using `v++ -c --mode hls`



TIP: Almost all HLS commands must be specified in a configuration file. The exceptions are `--platform` and `--freqhz`, which are permitted on the command line or in a config file.

- Connectivity directives for system linking specifying the number of kernels to instantiate, or assigning AI Engine streaming ports to PL kernel ports when using `v++ --link`
- Package directives to configure the creation of boot files, the generation of the `.xclbin` file from a `.xsa`, and the creation of an SD card when using `v++ --package`
- Directives for the Vivado Design Suite to manage hardware synthesis and implementation.
- Comments can be added to the configuration file by starting the line with a `"#"`:

```
# This is a comment line
```

The Vitis Unified IDE use configuration files to drive component build and simulation processes. The configuration file can be created by the tool at the time of component creation, can be imported from an existing component, or can be custom created and added to the component separately. For the command-line flow the configuration file is specified through the use of the `v++ --config` option as discussed in the [v++ General Options](#). An example of the `--config` option follows:

```
v++ --link --config ../src/system.cfg
```

Config file commands can be non-composing, which means they are only permitted once such as `--platform`. In this case the commands are read from the config file in the order they are encountered. The first command read is used. Config file commands can also be composing, which means that multiple values can be specified to apply to specific ports or CUs in the design. In this case the commands are cumulative, with any later conflicting commands either ignored or resulting in an error.

Commands are read in the order they are encountered. If the same switch is repeated with conflicting information, the first switch read is used or an error is returned. The order of precedence for switches is as follows, where item one takes highest precedence:

1. Command line switches.
2. Config files as specified on the command line from left-to-right.
3. Within a single config file, precedence is as encountered from top-to-bottom.

In general, any `v++` command option can be specified in a configuration file. However, the configuration file supports defining sections containing groups of related commands under command headers to help manage build options and strategies. The following table lists the defined section headers.



TIP: Multiple processes can be defined in a single configuration file. However, it is recommended to keep separate configuration files for separate processes, such as HLS component creation, system linking, and system packaging.

Table 47: Section Tags of the Configuration File

Section Name	Description
Unlabeled	Generally, an unlabelled section can be placed at the top of a configuration file to contain commands that are not specific to one of the following section heads. Examples of these command can include <code>--part</code> or <code>--platform</code> , <code>--freqhz</code> , and <code>--debug</code> .
[advanced]	--advanced Options:
[aie]	Used for AI Engine compilation with the <code>v++ -c --mode aie</code> command
[clock]	--clock Options:
[connectivity]	--connectivity Options:
[debug]	--debug Options

Table 47: Section Tags of the Configuration File (cont'd)

Section Name	Description
[hls]	Used for HLS component compilation using the <code>v++ -c --mode hls</code> command. These commands are described in v++ Mode HLS Note: There are [hls] commands intended for use with the <code>v++ -c -k</code> mode for compiling the PL kernel for software emulation. These commands are described in --hls Options
[linkhook]	--linkhook Options
[package]	--package Options
[profile]	--profile Options
[vivado]	--vivado Options:

Using the Message Rule File

The `v++` command executes various AMD tools during kernel compilation and linking. These tools generate many messages that provide build status to you. These messages might or might not be relevant to you depending on your focus and design phase. The Message Rule file (`.mrf`) can be used to better manage these messages. It provides commands to promote important messages to the terminal or suppress unimportant ones. This helps you better understand the kernel build result and explore methods to optimize the kernel.

The Message Rule file is a text file consisting of comments and supported commands. Only one command is allowed on each line.

Comment

Any line with “#” as the first non-white space character is a comment.

Supported Commands

By default, `v++` recursively scans the entire working directory and promotes all error messages to the `v++` output. The `promote` and `suppress` commands below provide more control on the `v++` output.

- `promote`: This command indicates that matching messages should be promoted to the `v++` output.
- `suppress`: This command indicates that matching messages should be suppressed or filtered from the `v++` output. Errors cannot be suppressed.

Enter only one command per line.

Command Options

The Message Rule file can have multiple `promote` and `suppress` commands. Each command can have one and only one of the options below. The options are case-sensitive.

- `-id [<message_id>]`: All messages matching the specified message ID are promoted or suppressed. The message ID is in format of `nnn-mmm`. As an example, the following is a warning message from HLS. The message ID in this case is 204-68.

```
WARNING: [V++ 204-68] Unable to enforce a carried dependence constraint
(II = 1, distance = 1, offset = 1)
between bus request on port 'gmem'
(/matrix_multiply_cl_kernel/mmult1.cl:57) and bus request on port 'gmem'-
severity [severity_level]
```

For example, to suppress messages with message ID 204-68, specify the following:
`suppress -id 204-68.`

- `-severity [<severity_level>]`: The following are valid values for the severity level. All messages matching the specified severity level will be promoted or suppressed.
 - `info`
 - `warning`
 - `critical_warning`

For example, to promote messages with severity of 'critical-warning', specify the following:
`promote -severity critical_warning.`

Precedence of Message Rules

The `suppress` rules take precedence over `promote` rules. If the same message ID or severity level is passed to both `promote` and `suppress` commands in the Message Rule file, the matching messages are suppressed and not displayed.

Example of Message Rule File

The following is an example of a valid Message Rule file:

```
# promote all warning, critical warning
promote -severity warning
promote -severity critical_warning
# suppress the critical warning message with id 19-2342
suppress -id 19-2342
```


emconfigutil Utility

When running software or hardware emulation in the command line flow, it is necessary to create an emulation configuration file, `emconfig.json`, used by the runtime library during emulation. The emulation configuration file defines the device type and quantity of devices to emulate for the specified platform. A single `emconfig.json` file can be used for both software and hardware emulation.

Note: When running on real hardware, the runtime and drivers query the installed hardware to determine the device type and quantity installed, along with the device characteristics.

To use the `emconfigutil` utility to automate the creation of the emulation file, specify the target platform and additional options in the `emconfigutil` command line:

```
emconfigutil --platform <platform_name> [options]
```

At a minimum, you must specify the target platform through the `-f` or `--platform` option to generate the required `emconfig.json` file. The specified platform must be the same as specified during the host and hardware builds.

The `emconfigutil` options are provided in the following table.

Table 48: emconfigutil Options

Option	Valid Values	Description
<code>-f</code> or <code>--platform</code>	Target device	Required. Defines the target device from the specified platform. For a list of supported devices, refer to Supported Platforms .
<code>--nd</code>	Any positive integer	Optional. Specifies number of devices. The default is 1.
<code>--od</code>	Valid directory	Optional. Specifies the output directory. When running emulation, the <code>emconfig.json</code> file must be in the same directory as the host executable. The default is to write the output in the current directory.
<code>-s</code> or <code>--save-temps</code>	N/A	Optional. Specifies that intermediate files are not deleted and remain after the command is executed. The default is to remove temporary files.
<code>--xp</code>	Valid AMD parameters and properties.	Optional. Specifies additional parameters and properties. For example: <pre>--xp prop:solution.platform_repo_paths=<xsa_path></pre> This example sets the search path for the target platforms.
<code>-h</code> or <code>--help</code>	N/A	Prints command help.

The `emconfigutil` command generates the `emconfig.json` configuration file in the output directory or the current working directory.



TIP: When running emulation, the location of the `emconfig.json` file can be specified by the `$EMCONFIG_PATH` variable, or must be found in the same directory as the host executable.

The following example creates a configuration file targeting two `xilinx_u200_qdma_201910_1` devices.

```
$emconfigutil --xilinx_u200_qdma_201910_1 --nd 2
```

kernelinfo Utility

The `kernelinfo` utility extracts and displays information from Xilinx object (XO) files which can be used during host code development. This information includes hardware function names, arguments, offsets, and port data.

The following command options are available:

Table 49: kernelinfo Commands

Option	Description
<code>-h [--help]</code>	Print help message.
<code>-x [--xo_path] <arg></code>	Absolute path to file including file name and <code>.xo</code> extension.
<code>-l [--log] <arg></code>	By default, information is displayed on the screen. Otherwise, you can use the <code>--log</code> option to output the information as a file.
<code>-j [--json]</code>	Output the file in JSON format.
<code>[input_file]</code>	XO file. Specify the XO file positionally or use the <code>--xo_path</code> option.
<code>[output_file]</code>	Output from AMD OpenCL Compiler. Specify the output file positionally, or use the <code>--log</code> option.

To run the `kernelinfo` utility, enter the following in a Linux terminal:

```
kernelinfo <filename.o>
```

The output is divided into three sections:

- Kernel Definitions
- Arguments
- Ports

The report generated by the following command is reviewed to help better understand the report content. The report is broken down into specific sections for better understandability.

```
kernelinfo krnl_vadd.xo
```

Where `krnl_vadd.xo` is a compiled kernel.

Kernel Definition

Reports high-level kernel definition information. Importantly, for the host code development, the kernel name is given in the `name` field. In this example, the kernel name is `krnl_vadd`.

```
=== Kernel Definition ===
name: krnl_vadd
language: c
vlnv: xilinx.com:hls:krnl_vadd:1.0
preferredWorkGroupSizeMultiple: 1
workGroupSize: 1
debug: true
containsDebugDir: 1
sourceFile: krnl_vadd/cpu_sources/krnl_vadd.cpp
```

Arguments

Reports kernel function arguments.

In the following example, there are four arguments: `a`, `b`, `c`, and `n_elements`.

```
=== Arg ===
name: a
addressQualifier: 1
id: 0
port: M_AXI_GMEM
size: 0x8
offset: 0x10
hostOffset: 0x0
hostSize: 0x8
type: int*

=== Arg ===
name: b
addressQualifier: 1
id: 1
port: M_AXI_GMEM
size: 0x8
offset: 0x1C
hostOffset: 0x0
hostSize: 0x8
type: int*

=== Arg ===
name: c
addressQualifier: 1
id: 2
port: M_AXI_GMEM1
size: 0x8
offset: 0x28
hostOffset: 0x0
hostSize: 0x8
type: int*
```

```
=== Arg ===  
name: n_elements  
addressQualifier: 0  
id: 3  
port: S_AXI_CONTROL  
size: 0x4  
offset: 0x34  
hostOffset: 0x0  
hostSize: 0x4  
type: int const
```

Ports

Reports the memory and control ports used by the kernel.

```
=== Port ===  
name: M_AXI_GMEM  
mode: master  
range: 0xFFFFFFFF  
dataWidth: 32  
portType: addressable  
base: 0x0  
  
=== Port ===  
name: M_AXI_GMEM1  
mode: master  
range: 0xFFFFFFFF  
dataWidth: 32  
portType: addressable  
base: 0x0  
  
=== Port ===  
name: S_AXI_CONTROL  
mode: slave  
range: 0x1000  
dataWidth: 32  
portType: addressable  
base: 0x0
```

launch_emulator Utility

For embedded platforms that have an Arm® subsystem, the AMD Vitis™ tool uses QEMU to emulate the PS subsystem. The QEMU processes must be run along with the RTL simulator process to emulate the entire system in hardware emulation. The `launch_emulator.py` is a utility which launches QEMU and manages the synchronization of the PL simulator processes. It launches QEMU and the simulation process with provided arguments. The Vitis IDE also calls this command when starting and stopping the emulator.



TIP: For help inside QEMU, press **Ctrl + a h** while in the emulator shell. To terminate the QEMU command, press **Ctrl + a x** while in the emulator shell.

For embedded platforms, the `--package Options` command generates scripts, `launch_hw_emu.sh`, or `launch_sw_emu.sh` to call the `launch_emulator.py` command with the required arguments based on the platform and the target application.

You can pass additional arguments to the `launch_emulator` utility from the command line when using the `launch_hw_emu.sh` or `launch_sw_emu.sh` wrapper scripts. Simply append the option to the command line when running the script. This allows you to customize the `launch_emulator` utility as needed to support your specific platform or application.

The following table shows the list of available options.

Table 50: Common Options for launch_emulator

Option	Accepted Value	Description
<code>-add-env ADD_ENV_CMD</code>	N/A	Specify additional Environment Variables for the emulation shell.

Table 50: Common Options for launch_emulator (cont'd)

Option	Accepted Value	Description
-aie-sim-options	AIE_SIM_OPTIONS file	Points to an AI Engine sim options file that has various AI Engine debug flags that are required for debugging the AI Engine SystemC module. Refer to the section on Reusing AI Engine Simulator Options in the <i>AI Engine Tools and Flows User Guide</i> (UG1076) for more information. The options file should be specified with a relative path with respect to <code>package.hw_emu/sim/behav_waveform/xsim/</code>. TIP: This is optional and only applies to AI Engine designs. You can enable profiling options that were used during <code>aiesimulator</code> to be applied to hardware emulation of the system-level design. To do this, add the following command: <pre>-aie-sim-options ../aiesimulator_output/aiesim_options.txt</pre>
-enable-debug	N/A	Debug mode opening two different XTERMs for QEMU and PL. IMPORTANT! This is very useful for the batch mode users to understand the flow and handshake between the QEMU and PL process.
-graphic-qemu	N/A	Start the Quick Emulator(QEMU) in GUI mode
-help	N/A	Prints help message.
-kernel-dbg	true, false	Enable debug in SW_EMU. This is used only for software emulation.
-pl-kernel-debug	true, false	Enable debug for the PL kernel.
-run-app	<application_script_name>	Ensure that the application script is packaged using <code>--package.sd_file</code> option during package step. Only when it is packaged in <code>sd_card</code>, the application script is available to run after QEMU is up running and mounted. TIP: When using the <code>-run-app</code> option, all QEMU messages are initially written to a file called <code>qemu_output.log</code> inside the <code>package.hw_emu</code> or <code>package.sw_emu</code> folder, based on your emulation target, and then re-written to the console after some delay. This delay can cause you to think there is a problem with QEMU, but you can examine the contents of the <code>qemu_output.log</code> if needed.
-timeout	<n>	Terminates emulation after <n> seconds. The default value when <code>-run-app</code> is used is 4000 seconds. This means the application terminates after running for 4000 seconds without user intervention.
-user-post-sim-script	Path to Tcl script required to be done post simulation before quit	Creates Tcl for any post operations into a Tcl file and pass the Tcl script to this switch.

Table 50: Common Options for launch_emulator (cont'd)

Option	Accepted Value	Description
-user-pre-sim-script	Path to Tcl script	For first run, <code>launch_emulator.py</code> in GUI mode and add the signals that you want to observe. Copies the commands from the Tcl console and save into a Tcl script. From the next run, pass the Tcl script in batch mode, <code>launch_emulator.py -user-pre-sim-script <path_to_saved_tcl_script></code> . Only supports the Vivado simulator (xsim).
-verbose	N/A	Enable additional debug messages
-wcfg-file-path	N/A	Specify the wcfg file created by the XSIM to open during GUI simulation. Requires complete absolute path of the file.
-wdb-File	Path to WDB file	Specify the wdb file to load. Requires complete absolute path of the file.
-x86-sim-options	N/A	Points to the x86 simulation options file which has various AI Engine debug flags that are required for debugging the AI Engine Model. Used only for software emulation.
-xtlm-aximm-log	N/A	This switch generates xTLM AXI4 transaction logs for interface connection between two SystemC models (with information like address/data/size, etc.). While running the emulation log is available at (directory structure can vary based on v++ options and simulator used): <code>package.hw_emu/sim/behav_waveform/xsim/xsc_report.log</code>
-xtlm-axis-log	N/A	This switch generates xTLM AXI4-Stream transaction logs for interface connection between two SystemC models. While running emulation log is available at (directory structure can vary based on v++ options and simulator used): <code>package.hw_emu/sim/behav_waveform/xsim/xsc_report.log</code>

Table 51: Advanced Options for launch_emulator

Option	Accepted Value	Description
-disable-host-completion-check	N/A	Skip the check for host/test completion. Generally used in applications where python scripts check for the test completion status PASS/FAIL. By default, you search for "TEST PASSED" string when <code>-run-app</code> switch is used.
-enable-tcp-sockets	N/A	Enables TCP Sockets
-kill	<pid>	Kills a specified emulator process.
-kill-pid-file	N/A	Specifies the file to be used to kill the process. This file stores the group PID of the process. Can be created using <code>-pid-file</code> .
-no-reboot	N/A	Exit QEMU instead of rebooting. Used to exit gracefully from QEMU by executing command <code>reboot -f</code> at the embedded Linux prompt.

Table 51: Advanced Options for launch_emulator (cont'd)

Option	Accepted Value	Description
-no_build	N/A	Enables a check of the build command without running the build process.
-no_run	N/A	Build but don't run the emulation.
-ospi-image	OSPI Image file	Specify an OSPI image file for booting.
-pl-sim-args	Arguments to simulator	These arguments gets appended to simulator command line. Alternative to pm-sim-args-file.
-pmc-args	Arguments to PMC	The PMC/PMU is emulated by <code>qemu-system-microblazeel</code>. Most common command line switches of the PMC are captured in <code>pmc_args.txt</code>. Instead of writing into a file called <code>pmc_args.txt</code>, you can directly provide all the arguments that need to be appended to the PMC command line. This is an alternative to <code>-pmc-args-file</code>. PMC/PMU arguments for specific devices can be found in Versal PS and PMC Arguments for QEMU and Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU. TIP: This option is not supported for AMD Zynq™ 7000 devices.
-pmc-args-file	PMC QEMU arguments file name	Any options to be passed to PMU/PMC can be given in this file. The specific format is determined by the base file on your chosen platform. This is auto passed in the <code>v++</code> package generated script. PMC/PMU arguments for specific devices can be found in Versal PS and PMC Arguments for QEMU and Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU. TIP: This option is not supported for Zynq 7000 devices.
-print-qemu-version	N/A	Prints the version of QEMU being used.
-qemu-args	Arguments to QEMU	The PS is emulated by <code>qemu-system-aarch64</code>. Most common command line switches of the PS are captured in <code>qemu_args.txt</code>. Instead of writing into a file called <code>qemu_args.txt</code>, you can directly provide all the arguments that need to be appended to the QEMU command line. This is an alternative to <code>qemu-args-file</code>. PS arguments for specific devices can be found in Versal PS and PMC Arguments for QEMU and following sections for AMD Zynq™ UltraScale+™ MPSoC.
-qemu-dtb	<path_to_DTB_file>	<code>v++ --package</code> automatically creates a DTB file based on the addressing in the design and passes it to the <code>launch_emulator</code> command. This option lets you specify the DTB file to override the defaults. Note: Ensure the DTB is compatible with the <code>noc_memory_config.txt</code> file used.

Table 51: Advanced Options for launch_emulator (cont'd)

Option	Accepted Value	Description
-qspi-high-image	Specify QSPI high image file	The image file which is passed as a QEMU argument in the form of boot mode. This is auto passed in the v++ package generated script. Required only when DUAL QSPI mode is used.
-qspi-image	Specify qspi.bin	The image file is passed as a QEMU argument in the form of boot mode. This is auto passed in the v++ package generated script. Required only when you opt for QSPI mode.
-qspi-low-image	Specify QSPI low image file	The image file is passed as a QEMU argument in the form of boot mode. This is auto passed in the v++ package generated script. Required only when DUAL QSPI mode is used.
-result-string	N/A	Result string searches for the status of test completion. Default = "TEST PASSED."
-use-qemu-version-v4	N/A	Use QEMU version 4.2

Table 52: Auto-Populated by V++ in Emulation Script

Option	Accepted Value	Description
-aie-sim-config	N/A	Points to the AI Engine sim config file that provides various AI Engine files that are required for the SystemC Model of AI Engine. This is auto passed by the v++ package. Required for AI Engine designs.
-boot-bh	Path to BH file	Specify the boot BH file path
-device-family	7Series UltraScale Versal	Required to specify the device family for the platform. This is auto passed by the v++ package generated scripts launch_hw_emu.sh or launch_sw_emu.sh based on the target chosen. This needs to be passed manually for direct usage of the launch_emulator command.
-enable-prep-target	N/A	Enable pre-process target.
-forward-port	<target> <host>	Forwards TCP port from target to host.
-gdb-port	Port number	QEMU waits for GDB connection on <port>.
-noc-memory-config <path/to/noc_memory_config.txt>	N/A	By default, v++ --package creates the NoC memory configuration based on the design configuration, and you can see this file parallel to simulation binaries. You can override this file by replacing the file specified in the simulation binary folder. Use the -user-pre-sim-script option to copy your noc_memory_config.txt file to the simulation binary area and to get the configuration applied.
-pid-file	File name	Write process ID to <file> for later use with -kill. Used by the Vitis software platform to kill once emulation is successful.

Table 52: Auto-Populated by V++ in Emulation Script (cont'd)

Option	Accepted Value	Description
-pl-sim-dir	Simulation directory	Start the Programmable Logic Simulator by launching the scripts from this directory. This is auto passed in the v++ package generated script. The tool expects a file called <code>simulate.sh</code> in the specified directory and will execute it to launch the PL simulator (for example, XSIM).
-pl-sim-script	Simulation script location	Advanced users can have one direct script to launch simulation (for example, Vivado users). When this option is given, run the script, other options are of no value.
-platform-name	NAME	The platform name
-pmc-args-file	PMC QEMU arguments file name	Any options to be passed to PMU/PMC can be given in this file. The specific format is determined by the base file on your chosen platform. This is auto passed in the v++ package generated script. PMC/PMU arguments for specific devices can be found in Versal PS and PMC Arguments for QEMU and Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU. TIP: This option is not supported for Zynq 7000 devices.
-pmc-dtb	<path_to_DTB_file>	v++ --package automatically creates a device-tree binary (DTB) file based on the addressing in the design and passes it to the <code>launch_emulator</code> command. This option lets you specify the DTB file to override the defaults. Note: Ensure the DTB is compatible with the <code>noc_memory_config.txt</code> file used. PMC/PMU arguments for specific devices can be found in Versal PS and PMC Arguments for QEMU and Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU. TIP: This option is not supported for Zynq 7000 devices.
-protoinst-File	Path to ProtoInst file	Specify the protoinst file to load. Provide the complete absolute path.
-qemu-args-file	PS QEMU Arguments file name	Any options to be passed to QEMU can be given in this file. This is specific format where you fetch the base file from the platform chosen. This is auto passed in the v++ package generated script.
-qemu-dtb	<path_to_DTB_file>	v++ --package automatically creates a DTB file based on the addressing in the design and passes it to the <code>launch_emulator</code> command. This option lets you specify the DTB file to override the defaults. Note: Ensure the DTB is compatible with the <code>noc_memory_config.txt</code> file used.

Table 52: Auto-Populated by V++ in Emulation Script (cont'd)

Option	Accepted Value	Description
<code>-sd-card-image</code>	Specify <code>sd_card.img</code>	The image file is passed as a QEMU argument in the form of boot mode. This is auto passed in the V++ package generated script. Required only when SD mode is used.
<code>-t -target</code>	<code>sw_emu</code> or <code>hw_emu</code>	Specify to run <code>sw_emu</code> or <code>hw_emu</code> . Based on the target chosen in the v++, respective script is generated by the v++ package. For <code>sw_emu</code> target, <code>launch_sw_emu.sh</code> is generated and for <code>hw_emu</code> target, <code>launch_hw_emu.sh</code> is generated.
<code>-xtlm-log-state</code>	WAVEFORM LOG BOTH	Option to specify what the XTLM Log should contain. It can contain the waveform, the text log, or both. This option is used only for hardware emulation.

Versal PS and PMC Arguments for QEMU

In the AMD Versal™ device, the PS(a72) is emulated by `qemu-system-aarch64` and PMC is emulated by `qemu-system-microblazeel`. Most common command line switches of PS are captured in `qemu_args.txt` and PMC command line switches are captured in `pmc_args.txt`.



TIP: You can add comments to the `pmc_args.txt`, `qemu_args.txt`, and `pmu_args.txt` files using the '#' symbol at the start of the line.

Table 53: Versal Options for qemu_args.txt

Arg Name	Value	Description	Source	How to Extract the Info
<code>-boot</code>	<code>mode=<boot-number></code> Ex. for sd1 <code>-boot mode=5</code>	Specify boot mode on your platform: • <code>qspi24 = 1</code> • <code>qspi32 = 2</code> • <code>sd0 = 3</code> • <code>sd1 = 5</code> • <code>emmc0 = 6</code> • <code>ospi = 8</code>	v++ -p	DRC needed between CIPS enabled boot modes and v++ -p selection
<code>-display</code>	<code>none</code>	By default QEMU creates display for user I/O. This option disables the display	Static	Specify none to disable the display
<code>-hw-dtb</code>	<code><ps-dtb-file></code>	Device tree file which describes the PS (a72). The Vitis compiler --package command generates the <code>dtb</code> file and appends it to <code>qemu_args.txt</code> .	v++ -p	

Table 53: Versal Options for qemu_args.txt (cont'd)

Arg Name	Value	Description	Source	How to Extract the Info
-M	arm-generic-fdt	This specifies the QEMU machine to create. arm-generic-fdt machine option tells QEMU to parse dtb for machine generation, passes by -hw-dtb user.dtb.	Device specific	Hard coded for Versal
-serial	-serial null -serial null -serial mon:stdio	By default, QEMU connects invoking terminal to UART0 for user I/O operations. You can override this behavior by specifying this option. Versal platforms have four UARTs specified using positional arguments: the first two are for debug and the last two are UART0 and UART1. To connect UART0 to the terminal, specify -serial null -serial null -serial mon:stdio which ignores debug related UARTS and connects to UART0 to terminal. Similarly to connect only UART1 to terminal specify -serial null -serial null -serial mon:stdio	Based on UARTs enabled on CIPS configuration.	The Versal device has four UARTs. The first two are debug related UARTs. When enabling UART0: CONFIG.PS_UART0_PERIPHERAL_ENABLE = 1 CONFIG.PS_UART1_PERIPHERAL_ENABLE = 0 or 1 Then specify -serial null -serial null -serial mon:stdio When enabling only UART1: CONFIG.PS_UART0_PERIPHERAL_ENABLE = 0 CONFIG.PS_UART1_PERIPHERAL_ENABLE = 1 Then specify: -serial null -serial null -serial mon:stdio
-sync-quantum	Time in milliseconds	Specifies how frequently QEMU will sync with the RTL simulator. Modifying this can have an impact on the speed of simulation.	Static	Hard coded for Versal devices (user need to change)

Versal CIPS has two Ethernet interfaces. Most of AMD Versal CIPS board has eth0 enabled. If no -net or -netdev is specified then QEMU by default enables eth0 and maps to user mode backend.

Table 54: Versal Options for pmc_args.txt

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-M	microblaze-fdt	Specifies the QEMU machine to create. QEMU creates MicroBlaze™ with nodes from dtb.	Static	Hard coded for Versal Devices
-display	none	By default, QEMU creates display for user I/O. This option instructs QEMU that there is no need for display.	Static	Hard coded for Versal Devices
-device	loader,file=<BOOT_bh.bin>,addr=0xf201e000,force-raw	Specifies Boot header file with load address as 0xF201E000 (BOOT_bh.bin is loaded at address 0xF201E000). This is fixed argument in pmc_args.txt which is processed by v++ -p for final argument which has absolute path of BOOT_bh.bin file. BOOT_bh.bin is generated by v++ -p from final PDI. BOOT_bh.bin is directly loaded onto QEMU because there is no BOOT ROM access for QEMU to load boot header from PDI.	v++ -pack	v++ pack extracts BOOT_bh.bin and generates this switch
-device	loader,file=<pmc_cdo.bin>,addr=0xf2000000,force-raw	Specifies pmc_cdo.bin with load address as 0xF2000000. This is fixed argument in pmc_args.txt. This is processed by v++ -p for final argument which has absolute path of pmc_cdo.bin file.	v++ -pack	v++ pack extracts pmc_cdo.bin and generates this switch
-device	loader,file=<plm.bin>,addr=0xF0200000,force-raw	Specifies plm binary firmware with load address as 0xF0200000. This is a fixed argument in pmc_args.txt which is processed by v++ -p for final argument. This has an absolute path of plm.bin file. This plm is executed by PPU1 when it is out of reset.	v++ -pack	v++ pack extracts plm.bin and generates this switch
-device	loader,addr=0xf0000000,data=0xba020004,data-len=4 -device loader,addr=0xf0000004,data=0xb800fffc,data-len=4	Specifies PPU0 process to be in boot loop. As there is no BOOTROM access for QEMU, PPU0 is put in bootloop which generally loads BOOT header from PDI to memory.	Static	Hard coded for Versal Devices
-device	loader,addr=0xF1110624,data=0x0,data-len=4 -device loader,addr=0xF1110620,data=0x1,data-len=4	Makes PPU1 out of reset and puts in executing mode.	Static	Hard coded for Versal Devices

Table 54: Versal Options for pmc_args.txt (cont'd)

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-hw-dtb	<ps-dtb-file>	dtb file which describes about PS(a72). v++ pack generates this dtb file and appends to qemu-args.txt.	v++ pack	v++ pack generates dtb files based on DDR config on device

Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU

The Zynq UltraScale+ MPSoC PS(a53) is emulated by `qemu-system-aarch64` and PMU is emulated by `qemu-system-microblazeel`. Most common command line switches of PS are captured in `qemu_args.txt` and PMC command line switches are captured in `pmu_args.txt`.



TIP: You can add comments to the `pmc_args.txt`, `qemu_args.txt`, and `pmu_args.txt` files using the '#' symbol at the start of the line.

Table 55: Zynq UltraScale+ MPSoC Options for qemu_args.txt

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-M	arm-generic-fdt	This specifies the QEMU machine to create. <code>arm-generic-fdt</code> machine option tells QEMU to parse dtb for machine generation, passes by <code>-hw-dtb user.dtb</code> .	Static	Hard-coded for Zynq UltraScale+ MPSoC devices

Table 55: Zynq UltraScale+ MPSoC Options for qemu_args.txt (cont'd)

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-serial	mon:stdio	-serial is a positional argument. Redirect the serial port to specified char dev (i.e., stdio, tcp port, file, etc.).	Based on UART configuration on Zynq UltraScale+ MPSoC	Zynq UltraScale+ MPSoC has two UARTs. When enabling UART0: <pre>CONFIG.PSU__UART0__PERIPHERAL__ENABLE = 1 CONFIG.PSU__UART1__PERIPHERAL__ENABLE = 0 or 1</pre> Then specify: -serial mon:stdio When enabling only UART1: <pre>CONFIG.PSU__UART0__PERIPHERAL__ENABLE = 0 CONFIG.PSU__UART1__PERIPHERAL__ENABLE = 1</pre> Then specify: -serial null -serial mon:stdio
-global	xlnx,zynqmp-boot.cpu-num=0	Make the specified CPU come out of reset.	Static	Hard coded for Zynq UltraScale+ MPSoC devices

Table 55: Zynq UltraScale+ MPSoC Options for qemu_args.txt (cont'd)

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-net	-net nic -net nic -net nic -net user	-net is positional argument. Initialize network interfaces gem3. Connect the specified network adapter to user mode network. TIP: <i>-net none will disable all Ethernet interfaces.</i>	Static	Based on Ethernet configurations: If gem0(eth0) is enabled: <pre>CONFIG.PSU__ENET0__PERIPHERAL__ENABLE = 1</pre> Then specify -net nic -net user If gem1 is enabled: <pre>CONFIG.PSU__ENET1__PERIPHERAL__ENABLE = 1</pre> Then specify -net nic -net nic -net user If gem2 is enabled: <pre>CONFIG.PSU__ENET2__PERIPHERAL__ENABLE = 1</pre> Then specify -net nic -net nic -net nic -net user If gem 3 is enabled: <pre>CONFIG.PSU__ENET3__PERIPHERAL__ENABLE = 1</pre> Then specify -net nic -net nic -net nic -net user TIP: <i>If no -net (and/or -netdev) is mentioned then by default QEMU will enable the first Ethernet (gem0) and map it to user mode backend.</i>
-m	4G	Enabling 4 GB DDR on Zynq UltraScale+ MPSoC.	Static	Emulating full DDR on Zynq UltraScale+ MPSoC
-device	loader,file=<bl31.elf>,cpu-num=0	Load bl31.elf file on A53 core 0.	Static	v++ --package should replace bl31.elf with absolute path of bl31.elf

Table 55: Zynq UltraScale+ MPSoC Options for qemu_args.txt (cont'd)

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-device	loader,file=<u-boot.elf>	Loading u-boot.elf.	Static	v++ --package should replace bl31.elf with absolute path of u-boot.elf
-hw-dtb	<ps-dtb-file>	dtb file which describes PS which is emulated by QEMU can be specified using -hw-dtb.	Static	Hard coded for Zynq UltraScale+ MPSoC devices: <ps-dtb-file>=/proj/xbuils/HEAD_daily_latest/installs/lin64/Vitis/HEAD//data/emulation/dtbs/zynqmp/zynqmp-arm-cosim.dtb

Table 56: Zynq UltraScale+ MPSoC Options for pmu_args.txt

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-M	microblaze-fdt	This specifies the QEMU machine to create. microblaze-fdt tells QEMU to parse dtb for machine generation, passes by -hw-dtb user.dtb.	Static	Hard coded for Zynq UltraScale+ MPSoC devices
-device	loader,file=<pmufw.elf>	Load pmufw.elf file on PMU RAM.	Static	Hard coded for Zynq UltraScale+ MPSoC devices
-machine-path	<path-to-xsim-dir>	Point -machine-path to folder to create shared RAM and remote-port sockets.	Static	launch_emulator command will set this machine path
-display	none	By default, QEMU creates display for user I/O. This option disables the display.	Static	Hard coded for Zynq UltraScale+ MPSoC devices
-hw-dtb	<pmu-dtb-file>	dtb file which describes PMU which is emulated by QEMU can be specified using -hw-dtb.	Static	<pmu-dtb-file>=/proj/xbuils/HEAD_daily_latest/installs/lin64/Vitis/HEAD//data/emulation/dtbs/zynqmp/zynqmp-pmu.dtb



TIP: Although the file is called *pmu_args.txt* here, the file is specified for *launch_emulator* using the *-pmc-args-file* command.

Zynq 7000 PS Arguments for QEMU

Zynq 7000 PS(a9) is emulated by `qemu-system-aarch64` QEMU binary. Most common command line switches of PS are captured in `qemu_args.txt`.



TIP: You can add comments to the `pmc_args.txt`, `qemu_args.txt`, and `pmu_args.txt` files using the '#' symbol at the start of the line.

Table 57: Zynq 7000 Options for `qemu_args.txt`

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-M	arm-generic-fdt-7series	Indicates the QEMU machine to create. arm-generic-fdt-7series tells QEMU to parse <code>dtb</code> for machine generation, passes by <code>-hw-dtb user.dtb</code> .	Static	Hard coded for Zynq 7000 devices
-serial	-serial /dev/null -serial mon:stdio	Redirect the serial port to specified char dev (i.e., stdio, tcp port, file, etc.)	Based on the UART configuration of the Zynq IP.	Zynq 7000 has two UARTs. When enabling UART0: <pre>CONFIG.PCW_UART0_PERIPHERAL_ENABLE = 1 CONFIG.PCW_UART1_PERIPHERAL_ENABLE = 0 or 1</pre> Then specify: <code>-serial mon:stdio</code> When enabling only UART1: <pre>CONFIG.PCW_UART1_PERIPHERAL_ENABLE = 1</pre> Then specify: <code>-serial null -serial mon:stdio</code>
-device	loader,addr=0xf8000008,data=0xDF0D,data-len=4 - device loader,addr=0xf8000140,data=0x00500801,data-len=4 - device loader,addr=0xf800012c,data=0x1ed044d,data-len=4 - device loader,addr=0xf8000108,data=0x0001e008,data-len=4 - device loader,addr=0xf800025C,data=0x00000005,data-len=4 - device loader,addr=0xf8000240,data=0x00000000,data-len=4	Register writes to SLCR block, to set PLL and CLK_CTRL regs (required for Linux).	Static	Hard coded for Zynq 7000 devices

Table 57: Zynq 7000 Options for qemu_args.txt (cont'd)

Switch Name	Value	Description	Source of the Config	How to Extract the Info
-boot	mode=5	Boot mode 5 is for SD boot.	v++ -p	
-kernel	<u-boot.elf>	Guest software to load during boot up.	Static	<u-boot.elf> is replaced with the absolute path of u-boot.elf from the target platform
-machine	linux=on	Make QEMU itself a loader of the Linux image.	Static	Hard coded for Zynq 7000 devices

manage_ipcache Utility

To provide better performance during synthesis of kernels in your application designs, the AMD Vitis™ compiler uses an IP cache to store and reuse synthesis results. This lets the build process for the `.xclbin` file avoid having to repeat synthesis for kernels and CUs that have not changed. The IP cache stores the synthesis results and applies them for unchanged kernels in the design.

By default, the IP cache is stored inside the Vitis IDE workspace for a project, or at the level of your builds when running `v++` from the command line. You can customize the location for the IP cache using `--remote_ip_cache` to specify a new location, or disable the use of the IP cache using `--no_ip_cache`. See [v++ General Options](#) for information on these options.

The `manage_ipcache` utility is a standalone utility to help you manage the contents of your IP cache repository. It lets you report statistics on the IP cache repository and remove entries based on a variety of criteria.

Table 58: manage_ipcache Options

Option	Description
<code>-c --cache</code>	Required. Specifies the IP Cache directory to work on.
<code>-d --disk_space <size></code>	Delete all but the most recently used entries that fit in the disk space specified in MB.
<code>-h --help</code>	Prints help for the <code>manage_ipcache</code> command.
<code>-k --keep_top <N></code>	Delete all but the top N most recently used entries (N is an integer).
<code>-o --outfile <file></code>	Report stats for the IP cache to the specified file.
<code>-p --purge</code>	Delete ALL cache entries.
<code>-r --report</code>	Report stats for the IP cache to stdout.
<code>-u --unused</code>	Delete cache entries that have never been used (no cache hits).

The following example reports on the entries of the specified IP cache:

```
manage_ipcache --cache ./ip_cache --report
```

The `manage_ipcache` command returns 0 if successful, or returns -1 if an error occurs.

package_xo Command

Syntax

```
package_xo -kernel_name <arg> [-force] [-kernel_xml <arg>]
           [-output_kernel_xml <arg>] [-design_xml <arg>]
           [-ip_directory <arg>] [-parent_ip_directory <arg>]
           [-kernel_files <args>] [-kernel_xml_args <args>]
           [-kernel_xml_pipes <args>] [-kernel_xml_connections <args>]
           [-ctrl_protocol <arg>] -xo_path <arg> [-quiet] [-verbose]
```

Description

The `package_xo` command is a Tcl command within the AMD Vivado™ Design Suite. Kernels written in RTL are compiled in the Vivado tool using the `package_xo` command line utility which generates a Xilinx object (XO) file which can subsequently be used by the `v++` command, during the linking stage.

Table 59: Arguments

Argument	Description
<code>-kernel_name <arg></code>	(Required) Specify the name of the RTL kernel.
<code>-force</code>	(Optional) Overwrite an existing XO file if one exists.
<code>-kernel_xml <arg></code>	(Optional) Specify the path to an existing kernel XML file. The Vivado tool will create a <code>kernel.xml</code> file for the XO file if one is not specified.
<code>-output_kernel_xml</code>	(Optional) Specify the path to write the kernel XML file. The Vivado tool will create a <code>kernel.xml</code> file to include in the XO file, and also write it to the specified output file. TIP: You can use this option to generate a <code>kernel.xml</code> file which you can edit and use as an input in the <code>package_xo</code> command.
<code>-design_xml <arg></code>	(Optional) Specify the path to an existing design XML file
<code>-ip_directory <arg></code>	(Optional) Specify the path to the packaged IP directory.
<code>-parent_ip_directory</code>	(Optional) If the kernel IP directory specified contains multiple IPs, specify a directory path to the parent IP where its <code>component.xml</code> is located directly below.
<code>-kernel_files</code>	(Optional) Kernel file name(s). Can be used to add a C-model to your kernel XO to enable software emulation for your kernel.

Table 59: Arguments (cont'd)

Argument	Description
<code>-kernel_xml_args <args></code>	(Optional) Generate the <code>kernel.xml</code> with the specified function arguments. Each argument value should use the following format: <pre>{name:addressQualifier:id:port:size:offset:type:memSize}</pre> Note: <code>memSize</code> is optional.
<code>-kernel_xml_pipes <args></code>	(Optional) Generate the <code>kernel.xml</code> with the specified pipe(s). Each pipe value use the following format: <pre>{name:width:depth}</pre>
<code>-kernel_xml_connections <args></code>	(Optional) Generate the <code>kernel.xml</code> file with the specified connections. Each connection value should use the following format: <pre>{srcInst:srcPort:dstInst:dstPort}</pre>
<code>-ctrl_protocol</code>	Kernel control protocol as described in PL Kernel Properties. Valid values: <code>ap_ctrl_hs</code>, <code>ap_ctrl_chain</code>, <code>ap_ctrl_none</code>, <code>user_managed</code>. TIP: The default <code>ap_ctrl_hs</code> is written to the <code>kernel.xml</code> file when <code>-ctrl_protocol</code> is not specified.
<code>-xo_path <arg></code>	(Required) Specify the path and file name of the compiled object (XO) file.
<code>-quiet</code>	(Optional) Execute the command quietly, returning no messages from the command. The command also returns <code>TCL_OK</code> regardless of any errors encountered during execution. Note: Any errors encountered on the command-line, while launching the command, will be returned. Only errors occurring inside the command will be trapped.
<code>-verbose</code>	(Optional) Temporarily override any message limits and return all messages from this command. Note: Message limits can be defined with the <code>set_msg_config</code> command.

Examples

The following example creates the specified XO file containing an RTL kernel of the specified name using the `ap_ctrl_chain` control protocol, and creates the `kernel.xml` file because one has not been specified:

```
package_xo -xo_path Vadd_A_B.xo -kernel_name Vadd_A_B -ctrl_protocol
ap_ctrl_chain -ip_directory ./ip
```

The following example creates the XO file using the specified `kernel.xml` file:

```
package_xo -xo_path Vadd_A_B.xo -kernel_name Vadd_A_B -kernel_xml
kernel.xml -ip_directory ./ip
```



TIP: The control protocol will be defined in the specified `kernel.xml` file.

RTL Kernel XML File



TIP: The `package_xo` command will create a `kernel.xml` file from the `component.xml` of a packaged IP, so you do not need to manually provide one, or generate one using the RTL Kernel wizard.

An XML kernel description file, called `kernel.xml`, must be created for each RTL kernel, so that it can be used in the AMD Vitis™ application acceleration development flow. The `kernel.xml` file specifies kernel attributes like the register map and ports needed by the runtime and Vitis tool flows. The following code shows is an example of a `kernel.xml` file.

```
<?xml version="1.0" encoding="UTF-8"?>
<root versionMajor="1" versionMinor="6">
  <kernel name="vitis_kernel_wizard_0" language="ip_c"
    vlnv="mycompany.com:kernel:vitis_kernel_wizard_0:1.0"
    attributes="" preferredWorkGroupSizeMultiple="0" workGroupSize="1"
    interrupt="true">
    <ports>
      <port name="s_axi_control" mode="slave" range="0x1000" dataWidth="32"
        portType="addressable" base="0x0"/>
      <port name="m00_axi" mode="master" range="0xFFFFFFFFFFFFFFFF"
        dataWidth="512" portType="addressable"
        base="0x0"/>
    </ports>
    <args>
      <arg name="axi00_ptr0" addressQualifier="1" id="0" port="m00_axi"
        size="0x8" offset="0x010" type="int*"
        hostOffset="0x0" hostSize="0x8"/>
    </args>
  </kernel>
</root>
```

Note: The `kernel.xml` file can be created automatically using the RTL Kernel Wizard to specify the interface specification of your RTL kernel.

The following table describes the format of the `kernel.xml` in detail:

Table 60: Kernel XML File Content

Tag	Attribute	Description
<root>	versionMajor	For the current release of Vitis software platform, set to 1.
	versionMinor	For the current release of Vitis software platform, set to 6.

Table 60: Kernel XML File Content (cont'd)

Tag	Attribute	Description
<kernel>	name	Kernel name
	language	Always set to ip_c for RTL kernels.
	vlnv	Must match the vendor, library, name, and version attributes in the component.xml of an IP. For example, if component.xml has the following tags: <pre><spirit:vendor>xilinx.com</spirit:vendor> <spirit:library>hls</spirit:library> <spirit:name>test_sincos</spirit:name> <spirit:version>1.0</spirit:version></pre> The vlnv attribute in kernel XML must be set to:xilinx.com:hls:test_sincos:1.0
	attributes	Reserved. Set it to empty string: ""
	preferredWorkGroupSizeMultiple	Reserved. Set it to 0.
	workGroupSize	Reserved. Set it to 1.
	interrupt	Set to "true" (interrupt="true") if the RTL kernel has an interrupt, otherwise omit.
	hwControlProtocol	Specifies the control protocol for the RTL kernel. - ap_ctrl_hs: Default control protocol for RTL kernels. - ap_ctrl_chain: Control protocol for chained kernels that support dataflow. Adds ap_continue to the control registers to enable ap_done/ap_continue completion acknowledgment. - ap_ctrl_none: Control protocol (none) applied for data driven kernels. - user_managed: Specifies a kernel that meets the interface requirements for Vitis compiler to link the kernel with other kernels and a target platform, but does not adhere to the requirements for execution management by XRT. Refer to Creating User-Managed RTL Kernels for more information.

Table 60: Kernel XML File Content (cont'd)

Tag	Attribute	Description
<port>	name	Specifies the port name. IMPORTANT! The AXI4-Lite interface must be named <code>S_AXI_CONTROL</code> .
	mode	At least one AXI4 master port and one AXI4-Lite slave control port are required. AXI4-Stream ports can be specified to stream data between kernels. - For AXI4 master port, set to "master." - For AXI4 slave port, set to "slave." - For AXI4-Stream master port, set to "write_only." - For AXI4-Stream slave port, set it "read_only."
	range	The range of the address space for the port.
	dataWidth	The width of the data that goes through the port, default is 32-bits.
	portType	Indicate whether or not the port is addressable or streaming. - For AXI4 master and slave ports, set it to "addressable." - For AXI4-Stream ports, set it to "stream."
	base	For AXI4 master and slave ports, set to 0x0. This tag is not applicable to AXI4-Stream ports.
<arg>	name	Specifies the kernel software argument name.
	addressQualifier	Valid values: 0: Scalar kernel input argument 1: global memory 2: local memory 3: constant memory 4: pipe
	id	Only applicable for AXI4 master and slave ports. The ID needs to be sequential. It is used to determine the order of kernel arguments. Not applicable for AXI4-Stream ports.
	port	Specifies the <port> name to which the <code>arg</code> is connected.
	size	Size of the argument in bytes. The default is 4 bytes.
	offset	Indicates the register memory address.
	type	The C data type of the argument. For example, <code>uint*</code> , <code>int*</code> , or <code>float*</code> .
	hostOffset	Reserved. Set to 0x0.
	hostSize	Size of the argument. The default is 4 bytes.
	memSize	For AXI4-Stream ports, <code>memSize</code> sets the depth of the created FIFO. TIP: Not applicable to AXI4 ports.

Table 60: Kernel XML File Content (cont'd)

Tag	Attribute	Description
The following tags specify additional tags for AXI4-Stream ports. They do not apply to AXI4 ports.		
<connection>	The connection tag describes the actual connection in hardware, either from the kernel to the FIFO inserted for the PIPE, or from the FIFO to the kernel.	
	srcInst	Specifies the source instance of the connection.
	srcPort	Specifies the port on the source instance for the connection.
	dstInst	Specifies the destination instance of the connection.
	dstPort	Specifies the port on the destination instance of the connection.

platforminfo Utility

The `platforminfo` command line utility reports platform meta-data including information on interface, clock, valid SLRs and allocated resources, and memory in a structured format. This information can be referenced when allocating kernels to SLRs or memory resources for instance.

The following command options are available to use with `platforminfo`:

Table 61: platforminfo Commands

Option	Description
<code>-f [--force]</code>	Overwrite an existing output file.
<code>-h [--help]</code>	Print help message and exit.
<code>-k [--keys]</code>	Get keys for a given platform. Returns a list of JSON paths.
<code>-l [--list]</code>	List platforms. Searches the user repo paths <code>\$PLATFORM_REPO_PATHS</code> and then the install locations to find <code>.xpfm</code> files.
<code>-e [--extended]</code>	List platforms with extended information. Use with '--list'.
<code>-d [--hw] <arg></code>	Specify the platform definition (<code>*.dsa</code>) on which to operate. The value must be a full path, including file name and <code>.dsa</code> extension.
<code>-s [--sw] <arg></code>	Specify the software platform definition (<code>*.spfm</code>) on which to operate. The value must be a full path, including file name and <code>.spfm</code> extension.
<code>-p [--platform] <arg></code>	AMD platform definition (<code>*.xpfm</code>) on which to operate. The value for <code>--platform</code> can be a full path including file name and <code>.xpfm</code> extension, as shown in example 1 below. If supplying a file name and <code>.xpfm</code> extension without a path, this utility will search only the current working directory. You can also specify just the base name for the platform. When the value is a base name, this utility will search the <code>\$PLATFORM_REPO_PATHS</code>, and the install locations, to find a corresponding <code>.xpfm</code> file, as shown in example 2 below. Example 1: <code>--platform /opt/xilinx/platforms/xilinx_u50_gen3x16_xdma_201920_3.xpfm</code> Example 2: <code>--platform xilinx_u200_gen3x16_xdma_2_202110_1</code>
<code>-o [--output] <arg></code>	Specify an output file to write the results to. By default the output is returned to the terminal (stdout).

Table 61: platforminfo Commands (cont'd)

Option	Description
<code>-j [--json] <arg></code>	Specify JSON format for the generated output. When used with no value, the <code>platforminfo</code> utility prints the entire platform in JSON format. This option also accepts an argument that specifies a JSON path, as returned by the <code>-k</code> option. The JSON path, when valid, is used to fetch a JSON subtree, list, or value. Example 1: <code>platforminfo --json="hardwarePlatform" --platform <platform base name></code> Example 2: Specify the index when referring to an item in a list. <code>platforminfo --json="hardwarePlatform.devices[0].name" --platform <platform base name></code> Example 3: When using the short option form (<code>-j</code>), the value must follow immediately. <code>platforminfo -j"hardwarePlatform.systemClocks[]" -p <platform base name></code>
<code>-v [--verbose]</code>	Specify more detailed information output. The default behavior is to produce a human-readable report containing the most important characteristics of the specified platform.

Note: Except when using the `--help` or `--list` options, a platform must be specified. You can specify the platform using the `--platform` option, or using either `--hw`, `--sw`. You can also simply insert the platform name or full path into the command line positionally.

To understand the generated report, condensed output logs, based on the following command are reviewed. The report is broken down into specific sections for better understandability.

```
platforminfo -p $PLATFORM_REPO_PATHS/
xilinx_u200_gen3x16_xdma_2_202110_1.xpfm
```



TIP: See [Platforminfo for xilinx_zcu104_base_202010_1](#) for an example of embedded processor platforms.

Basic Platform Information

Platform information and high-level description are reported.

```
Platform:          xdma
File:              /proj/xbuilds/2020.2_daily_latest/internal_platforms/
xilinx_u200_xdma_201830_3/xilinx_u200_xdma_201830_3.xpfm
Description:
```

Hardware Platform Information

General information on the hardware platform is reported. For the Software Emulation and Hardware Emulation field, a "1" indicates this platform is suitable for these configurations.

```
Vendor:                xilinx
Board:                 U200 (xdma)
Name:                  xdma
Version:               201830.3
Generated Version:     2018.3
Hardware:              1
Software Emulation:    1
Hardware Emulation:    1
Hardware Emulation Platform: 0
FPGA Family:          virtexuplus
FPGA Device:           xcu200
Board Vendor:          xilinx.com
Board Name:            xilinx.com:au200:1.0
Board Part:            xcu200-fsgd2104-2-e
```

Interface Information

The following shows the reported PCIe interface information.

```
Interface Name: PCIe
Interface Type: gen3x16
PCIe Vendor Id:    0x10EE
PCIe Device Id:    0x5000
PCIe Subsystem Id: 0x000E
```

Clock Information

Reports the maximum kernel clock frequencies available. The Clock Index is the reference used in the `--kernel_frequency v++` directive when overriding the default value.

```
Default Clock Index: 0
Clock Index:         1
Frequency:           500.000000
Clock Index:         0
Frequency:           300.000000
```

Valid SLRs

Reports the valid SLRs in the platform.

```
SLR0, SLR1, SLR2
```

Resource Availability

The total available resources and resources available per SLR are reported. This information can be used to assess applicability of the platform for the design and help guide allocation of compute unit to available SLRs.

```
====  
Total  
====  
LUTs: 1047139  
FFs: 2186064  
BRAMs: 1896  
DSPs: 6833  
  
=====  
Per SLR  
=====  
SLR0:  
LUTs: 354690  
FFs: 723308  
BRAMs: 638  
DSPs: 2265  
SLR1:  
LUTs: 159739  
FFs: 331654  
BRAMs: 326  
DSPs: 1317  
SLR2:  
LUTs: 354839  
FFs: 723294  
BRAMs: 638  
DSPs: 2265
```

Memory Information

Reports the available DDR and PLRAM memory connections per SLR as shown in the example output below.

```
Type: ddr4
Bus SP Tag: DDR
  Segment Index: 0
    Consumption: automatic
    SP Tag:      bank0
    SLR:         SLR0
    Max Masters: 15
  Segment Index: 1
    Consumption: default
    SP Tag:      bank1
    SLR:         SLR1
    Max Masters: 15
  Segment Index: 2
    Consumption: automatic
    SP Tag:      bank2
    SLR:         SLR1
    Max Masters: 15
  Segment Index: 3
    Consumption: automatic
    SP Tag:      bank3
    SLR:         SLR2
    Max Masters: 15
Bus SP Tag: PLRAM
  Segment Index: 0
    Consumption: explicit
    SLR:         SLR0
    Max Masters: 15
  Segment Index: 1
    Consumption: explicit
    SLR:         SLR1
    Max Masters: 15
  Segment Index: 2
    Consumption: explicit
    SLR:         SLR2
    Max Masters: 15
```

The `Bus SP Tag` heading can be DDR or PLRAM and gives associated information below.

The `Segment Index` field is used in association with the `SP Tag` to generate the associated memory resource index as shown below.

```
Bus SP Tag[Segment Index]
```

For example, if `Segment Index` is 0, then the associated DDR resource index would be `DDR[0]`.

This memory index is used when specifying memory resources in the `v++` command as shown below:

```
v++ ... --connectivity.sp vadd.m_axi_gmem:DDR[3]
```


There can be more than one `Segment Index` associated with an SLR. For instance, in the output above, SLR1 has both `Segment Index 1` and `2`.

The `Consumption` field indicates how a memory resource is used when building the design:

- **default:** If the `--connectivity.sp` directive is not specified, it uses this memory resource by default during `v++ build`. For example in the report below, DDR with `Segment Index 1` is used by default.
- **automatic:** When the maximum number of memory interfaces are used and under `Consumption: default` is fully applied, then the interfaces under `automatic` is used. The maximum number of interfaces per memory resource are given in the **Max Masters** field.
- **explicit:** For PLRAM, consumption is set to `explicit` which indicates this memory resource is only used when explicitly indicated through the `v++` command line.

Feature ROM Information

The feature ROM information provides build related information on ROM platform and can be requested by [Support](#) when debugging system issues.

```
ROM Major Version:      10
ROM Minor Version:      1
ROM Vivado Build ID:    2388429
ROM DDR Channel Count:  5
ROM DDR Channel Size:   16
ROM Feature Bit Map:    655885
ROM UUID:               00194bb3-707b-49c4-911e-a66899000b6b
ROM CDMA Base Address 0: 620756992
ROM CDMA Base Address 1: 0
ROM CDMA Base Address 2: 0
ROM CDMA Base Address 3: 0
```

Software Platform Information

Although software platform information is reported, it is only useful for users that have an OS running on the device, and not applicable to users that use a host machine.

```
Number of Runtimes:      1
Default System Configuration: config0_0
System Configurations:
  System Config Name:      config0_0
  System Config Description: config0_0 Linux OS on x86_0
  System Config Default Processor Group: x86_0
  System Config Default Boot Image:
  System Config Is QEMU Supported:      0
  System Config Processor Groups:
    Processor Group Name:    x86_0
```

```

Processor Group CPU Type:  x86
Processor Group OS Name:   Linux OS
System Config Boot Images:
Supported Runtimes:
Runtime: OpenCL

```

Platforminfo for xilinx_zcu104_base_202010_1

Use the following command to return the platforminfo for the xilinx_zcu104_base_202010_1 platform:

```
platforminfo -p xilinx_zcu104_base_202010_1
```

The results returned are as follows:

```

=====
Basic Platform Information
=====
Platform:          xilinx_zcu104_base_202010_1
File:              /platforms/xilinx_zcu104_base_202010_1/
xilinx_zcu104_base_202010_1.xpfm
Description:
A basic static platform targeting the ZCU104 evaluation board, which
includes 2GB DDR4, GEM, USB, SDIO interface and UART of the Processing
System. It reserves most of the PL resources for user to add acceleration
kernels

=====
Hardware Platform (Shell) Information
=====
Vendor:            xilinx
Board:             zcu104_base
Name:              zcu104_base
Version:           1.0
Generated Version: 2020.1
Software Emulation: 1
Hardware Emulation: 0
FPGA Family:       zynqplus
FPGA Device:       xczu7ev
Board Vendor:       xilinx.com
Board Name:         xilinx.com:zcu104:1.1
Board Part:         xczu7ev-ffvc1156-2-e
Maximum Number of Compute Units: 60

=====
Clock Information
=====
Default Clock Index: 0
Clock Index:         0
Frequency:           150.000000
Clock Index:         1
Frequency:           300.000000
Clock Index:         2

```

```

Frequency:          75.000000
Clock Index:        3
Frequency:          100.000000
Clock Index:        4
Frequency:          200.000000
Clock Index:        5
Frequency:          400.000000
Clock Index:        6
Frequency:          600.000000

=====
Memory Information
=====
Bus SP Tag: HP0
Bus SP Tag: HP1
Bus SP Tag: HP2
Bus SP Tag: HP3
Bus SP Tag: HPC0
Bus SP Tag: HPC1
=====
Feature ROM Information
=====

=====
Software Platform Information
=====
Number of Runtimes:          1
Default System Configuration: xilinx_zcu104_base_202010_1
System Configurations:
  System Config Name:          xilinx_zcu104_base_202010_1
  System Config Description:    xilinx_zcu104_base_202010_1
  System Config Default Processor Group: xrt
  System Config Default Boot Image:  standard
  System Config Is QEMU Supported:  1
  System Config Processor Groups:
    Processor Group Name:      xrt
    Processor Group CPU Type:   cortex-a53
    Processor Group OS Name:    linux
  System Config Boot Images:
    Boot Image Name:           standard
    Boot Image Type:
    Boot Image BIF:             xilinx_zcu104_base_202010_1/boot/linux.bif
    Boot Image Data:            xilinx_zcu104_base_202010_1/xrt/image
    Boot Image Boot Mode:       sd
    Boot Image RootFileSystem:
    Boot Image Mount Path:      /mnt
    Boot Image Read Me:         xilinx_zcu104_base_202010_1/boot/
generic.readme
  Boot Image QEMU Args:         xilinx_zcu104_base_202010_1/qemu/
pmu_args.txt:xilinx_zcu104_base_202010_1/qemu/qemu_args.txt
  Boot Image QEMU Boot:
  Boot Image QEMU Dev Tree:
Supported Runtimes:
Runtime: OpenCL

```

vitis-run Command

The `vitis-run` command is provided to enable C-simulation, C/RTL Co-simulation, and Vivado implementation of an HLS component created with the `v++ -c --mode hls` command.

The options of the `vitis-run` command include:

- **--mode hls:** Specifies the use of `vitis-run` for HLS components. This is the only mode currently supported.
- **--csim:** Run C-simulation on an HLS component.
- **--cosim:** Run C/RTL Co-simulation on an HLS component.
- **--package:** Run the package process to generate the IP or XO files from the RTL design of the HLS component. This generates the required export files to use the RTL design in the Vivado Design Suite, or Vitis development flow.
- **--impl:** Run Vivado implementation out-of-context (OOC) run on the HLS component. This is used to provide resource usage and timing estimates from the synthesized or implemented design.
- **--tcl:** Run Vitis HLS using the Tcl scripting language. Tcl commands are described in the *Vitis High-Level Synthesis User Guide* ([UG1399](#)) under the heading of Vitis HLS Command Reference.
- **--work_dir:** Specify the working directory. For `-cosim` and `-impl`, the specified working directory must contain a previously compiled HLS component.
- **--config arg:** Specify a config file for use with the tool. Refer to [v++ Mode HLS](#) for specific commands to use in the config file.
- **-h [--help]:**

Display command help for the tool.



TIP: The list of options above is not complete list. You can use the `--help` command to display the complete list of `vitis-run` commands.

You can use the `vitis-run` command to run C-simulation without an existing design, or run C/RTL Co-simulation or Vivado implementation on an existing HLS component. Some examples follow.

Run C-Simulation

You can run C-simulation on an existing work directory containing a previously compiled HLS component, or specify a new work directory to run C-simulation on the source files directly. The config file for an existing HLS component only needs to specify commands for C-Simulation as described in [C-Simulation Configuration](#). The previously built HLS component provides the foundation for simulation, defining the source files, test bench files, and part or platform for the design. Running C-simulation on a new work directory requires a complete config file, specifying the required source files and part, in addition to any C-simulation options.

An example command line:

```
vitis-run --mode hls --csim --config ./hls_csim.cfg --work_dir newTest
```

The example config file:

```
part=xcvu11p-flga2577-1-e

[hls]
clock=8
flow_target=vitis
syn.file=../../src/dct.cpp
syn.top=dct
tb.file=../../src/out.golden.dat
tb.file=../../src/in.dat
tb.file=../../src/dct_test.cpp
tb.file=../../src/dct_coeff_table.txt
syn.output.format=xo
clock_uncertainty=15%
csim.O=true
csim.code_analyzer=0
csim.clean=true
csim.profile=true
```

Run C/RTL Co-Simulation

You can run Co-simulation on an existing work directory containing a previously compiled HLS component, or specify a new work directory to run C-simulation on the source files directly. The config file for an existing HLS component only needs to specify commands for C-Simulation as described in [Co-Simulation Configuration](#). The previously built HLS component provides the foundation for simulation, defining the source files, test bench files, and part or platform for the design. Running C-simulation on a new work directory requires a complete config file, specifying the required source files and part, in addition to any C-simulation options.

An example command line:

```
vitis-run --mode hls --cosim --config ./cosim.cfg --work_dir myHLS
```

The example config file:

```
part=xcvu11p-flga2577-1-e

[HLS]
clock=8
flow_target=vitis
syn.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct.cpp
syn.top=dct
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/out.golden.dat
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/in.dat
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct_test.cpp
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct_coeff_table.txt
syn.output.format=xo
clock_uncertainty=15%
cosim.trace_level=port
#cosim.wave_debug=true
cosim.random_stall=true
cosim.enable_dataflow_profiling=true
```

Run Vivado Implementation

You can run the Vivado Design Suite to synthesize and run place and route on the RTL generated by the HLS synthesis process. The synthesis and implementation processes are managed by commands specified in the configuration file as described in [Implementation Configuration](#).

An example command line:

```
vitis-run --mode hls --impl --config ./impl.cfg --work_dir myHLS
```

The example config file:

```
part=xcvu11p-flga2577-1-e

[HLS]
clock=8
flow_target=vitis
syn.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct.cpp
syn.top=dct
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/out.golden.dat
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/in.dat
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct_test.cpp
tb.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct_coeff_table.txt
syn.output.format=xo
```

```
clock_uncertainty=15%  
cosim.trace_level=port  
#cosim.wave_debug=true  
cosim.random_stall=true  
cosim.enable_dataflow_profiling=true
```

Run Tcl Script

You can also use `vitis-run` to run an existing Tcl script, such as the `script.tcl` from an existing project, to build the project and then write a config file from the Tcl script. The example below shows the `vitis-run` command to perform these actions.

An example command line:

```
vitis-run --mode hls --tcl dct-build.tcl
```

An example Tcl script:

```
open_project dctProj  
set_top dct  
add_files ../03-Vitis_HLS/reference-files/src/dct.cpp  
add_files -tb ../03-Vitis_HLS/reference-files/src/dct_coeff_table.txt  
add_files -tb ../03-Vitis_HLS/reference-files/src/dct_test.cpp  
add_files -tb ../03-Vitis_HLS/reference-files/src/in.dat  
add_files -tb ../03-Vitis_HLS/reference-files/src/out.golden.dat  
open_solution "solution1" -flow_target vivado  
set_part {xcvu11p-flga2577-1-e}  
create_clock -period 10 -name default  
csynth_design  
write_ini ./dct-build.cfg  
exit
```



TIP: The addition of the `write_ini` command at the end of the script creates a config file from the Tcl script, thus providing you with a config file to use with `v++ -c --mode hls`.

xbutil Utility

The Xilinx Board Utility (`xbutil`) is a standalone command line utility that is included with the Xilinx Runtime (XRT) installation package. Details of the `xbutil` command can be found at <https://xilinx.github.io/XRT/master/html/xbutil.html>.

`xbutil` includes multiple commands to validate and identify the installed accelerator card(s) along with additional card details including on card memory, host interface, target platform name, and system information. This information can be used for both card administration and application debugging.

Accelerator cards are partitioned into a user function and a management function to provide different levels of card access. The user function lets you load and run applications, while the management function is for system administrators to manage the card. The `xbutil` utility interacts with the user function. The `xbmgmt` utility, which requires root privilege, is for interacting with the management function. The reason for splitting the function access between the two utilities is to provide some security for the management features of the tool.

You can use the `help` command to list the available `xbutil` commands and options:

```
xbutil --help
```

Set up the `xbutil` command as part of the XRT installation using the following scripts:

- For `csh` shell:

```
$ source /opt/xilinx/xrt/setup.csh
```

- For `bash` shell:

```
$ source /opt/xilinx/xrt/setup.sh
```


xbmgmt Utility

AMD Board Management (`xbmgmt`) utility is a standalone command line tool that is included with the Xilinx Runtime (XRT) installation package. Details of the `xbmgmt` command can be found at <https://xilinx.github.io/XRT/master/html/xbmgmt.html>.

Accelerator cards are partitioned into a user function and a management function to provide different levels of card access. The user function lets you load and run applications, while the management function is for system administrators to manage the card. The `xbutil` utility interacts with the user function.

The `xbmgmt` utility is used for card installation and administration, and requires `sudo` privileges when running it. The `xbmgmt` supported tasks include flashing the card firmware, and scanning the current device configuration.

You can use the `help` command to list the available `xbmgmt` commands and options, and access help for individual commands by using the following:

```
xbmgmt --help <command>
```

For detailed help of each sub-command, use the following:

```
xbmgmt help <subcommand>
```

Set up the `xbmgmt` command as part of the XRT installation using the following scripts:

- For `csh` shell:

```
$ source /opt/xilinx/xrt/setup.csh
```

- For `bash` shell:

```
$ source /opt/xilinx/xrt/setup.sh
```

xclbinutil Utility

The `xclbinutil` utility can create, modify, and report `xclbin` content information.

The available command options are shown in the following table.

Table 62: xclbinutil Commands

Option	Description
<code>-h [--help]</code>	Print help messages.
<code>-i [--input]<arg></code>	Input file name. Reads <code>xclbin</code> into memory.
<code>-o [--output]<arg></code>	Output file name. Writes in memory <code>xclbin</code> image to a file.
<code>--target <arg></code>	Target flow for this image. Valid values: <code>hw</code> , <code>hw_emu</code> , and <code>sw_emu</code> .
<code>---private-key <arg></code>	Private key used in signing the <code>xclbin</code> image.
<code>--certificate <arg></code>	Certificate used in signing and validating the <code>xclbin</code> image.
<code>--digest-algorithm <arg></code>	Digest algorithm. Default: <code>sha512</code>
<code>--validate-signature</code>	Validates the signature for the given <code>xclbin</code> archive.
<code>-v [--verbose]</code>	Display verbose/debug information
<code>-q [--quiet]</code>	Minimize reporting information.
<code>--migrate-forward</code>	Migrate the <code>xclbin</code> archive forward to the new binary format.
<code>--add-section <arg></code>	Section name to add to the <code>xclbin</code> image. Format: <code><section>:<format>:<file></code>
<code>--add-replace-section <arg></code>	Replace an existing section or add the section of the <code>xclbin</code> image if it does not exist. Format: <code><section>:<format>:<file></code>
<code>--add-merge-section <arg></code>	Add the section if it does not exist, or merge content with an existing section. Format: <code><section>:<format>:<file></code>
<code>--remove-section<arg></code>	Section name to remove from the <code>xclbin</code> image.
<code>--dump-section<arg></code>	Section to dump. Format: <code><section>:<format>:<file></code>
<code>--replace-section<arg></code>	Section to replace.
<code>--key-value<arg></code>	Key value pairs. Format: <code>[USER SYS]:<key>:<value></code>
<code>--remove-key<arg></code>	Removes the given user key from the <code>xclbin</code> archive.
<code>--add-signature<arg></code>	Adds a user defined signature to the given <code>xclbin</code> image.
<code>--remove-signature</code>	Removes the signature from the <code>xclbin</code> image.
<code>--get-signature</code>	Returns the user defined signature (if set) of the <code>xclbin</code> image.
<code>--info</code>	Report accelerator binary content. Including: generation and packaging data, kernel signatures, connectivity, clocks, sections, etc
<code>--list-sections</code>	List all possible section names (standalone option).

Table 62: xclbinutil Commands (cont'd)

Option	Description
<code>--version</code>	Version of this executable.
<code>--force</code>	Forces a file overwrite.

The following are various use examples of the tool.

- **Reporting xclbin information:** `xclbinutil --info --input binary_container_1.xclbin`
- **Extracting the bitstream image:** `xclbinutil --dump-section BITSTREAM:RAW:bitstream.bit --input binary_container_1.xclbin`
- **Extracting the build metadata:** `xclbinutil --dump-section BUILD_METADATA:HTML:buildMetadata.json --input binary_container_1.xclbin`
- **Removing a section:** `xclbinutil --remove-section BITSTREAM --input binary_container_1.xclbin --output binary_container_modified.xclbin`

For most users, details about the contents and how the `xclbin` was created is desired. This information can be obtained through the `--info` option and reports information on the `xclbin`, hardware platform, clocks, memory configuration, kernel, and how the `xclbin` was generated.

The output of the `xclbinutil` command using the `--info` option is shown below divided into sections.

```
xclbinutil -i binary_container_1.xclbin --info
```

xclbin Information

```
Generated by:      v++ (2020.1) on Mon Apr 13 20:19:40 MDT 2020
Version:          2.6.436
Kernels:          CopyKernel
Signature:
Content:          Bitstream
UUID (xclbin):    d081de98-3fd3-4e9b-bab3-108b42c73101
UUID (IINTF):     862c7020a250293e32036f19956669e5
Sections:        DEBUG_IP_LAYOUT, BITSTREAM, MEM_TOPOLOGY,
IP_LAYOUT,
CONNECTIVITY, CLOCK_FREQ_TOPOLOGY,
BUILD_METADATA,
EMBEDDED_METADATA, SYSTEM_METADATA,
PARTITION_METADATA
```

Hardware Platform Information

```
Vendor:           xilinx
Board:            u200
Name:             xdma
Version:          201830.1
Generated Version: Vivado 2018.3 (SW Build: 2388429)
Created:          Wed Nov 14 20:06:10 2018
FPGA Device:      xcuv200
Board Vendor:     xilinx.com
Board Name:       xilinx.com:au200:1.0
Board Part:       xilinx.com:au200:part0:1.0
Platform VBNV:    xilinx_u200_xdma_201830_1
Static UUID:      00194bb3-707b-49c4-911e-a66899000b6b
Feature ROM TimeStamp: 1542252769
```

Clocks

Reports the maximum kernel clock frequencies available. Both the clock names and clock indexes are provided. The clock indexes are identical to those reported in [platforminfo Utility](#).

```
Name:      DATA_CLK
Index:      0
Type:       DATA
Frequency:  300 MHz

Name:      KERNEL_CLK
Index:      1
Type:       KERNEL
Frequency:  500 MHz
```

Memory Configuration

```
Name:      bank0
Index:      0
Type:       MEM_DDR4
Base Address: 0x0
Address Size: 0x400000000
Bank Used:  No

Name:      bank1
Index:      1
Type:       MEM_DDR4
Base Address: 0x400000000
Address Size: 0x400000000
Bank Used:  Yes

Name:      bank2
Index:      2
```

```

Type:      MEM_DDR4
Base Address: 0x800000000
Address Size: 0x400000000
Bank Used:  No

Name:      bank3
Index:     3
Type:      MEM_DDR4
Base Address: 0xc00000000
Address Size: 0x400000000
Bank Used:  No

Name:      PLRAM[0]
Index:     4
Type:      MEM_DDR4
Base Address: 0x1000000000
Address Size: 0x20000
Bank Used:  No

Name:      PLRAM[1]
Index:     5
Type:      MEM_DRAM
Base Address: 0x1000020000
Address Size: 0x20000
Bank Used:  No

Name:      PLRAM[2]
Index:     6
Type:      MEM_DRAM
Base Address: 0x1000040000
Address Size: 0x20000
Bank Used:  No

```

Kernel Information

For each kernel in the `xclbin`, the function definition, ports, and instance information is reported.

The following is an example of the reported function definition.

```

Definition
-----
Signature: krnl_vadd (int* a, int* b, int* c,
                    int const n_elements)

```

The following is an example of the reported ports.

```

Ports
-----
Port:      M_AXI_GMEM
Mode:      master
Range (bytes): 0xFFFFFFFF
Data Width: 32 bits
Port Type:  addressable

Port:      M_AXI_GMEM1

```

```

Mode:          master
Range (bytes): 0xFFFFFFFF
Data Width:    32 bits
Port Type:     addressable

Port:          S_AXI_CONTROL
Mode:          slave
Range (bytes): 0x1000
Data Width:    32 bits
Port Type:     addressable

```

The following is an example of the reported instance(s) of the kernel.

```

Instance:      krnl_vadd_1
Base Address:  0x0

Argument:      a
Register Offset: 0x10
Port:          M_AXI_GMEM
Memory:        bank1 (MEM_DDR4)

Argument:      b
Register Offset: 0x1C
Port:          M_AXI_GMEM
Memory:        bank1 (MEM_DDR4)

Argument:      c
Register Offset: 0x28
Port:          M_AXI_GMEM1
Memory:        bank1 (MEM_DDR4)

Argument:      n_elements
Register Offset: 0x34
Port:          S_AXI_CONTROL
Memory:        <not applicable>

```

Tool Generation Information

The utility also reports the `v++` command line used to generate the `xclbin`. The Command Line section gives the actual `v++` command line used, while the Options section displays each option used in the command line, but in a more readable format with one option per line.

```

Generated By
-----
Command:      v++
Version:      2018.3 - Tue Nov 20 19:42:42 MST 2018 (SW BUILD: 2394611)
Command Line: v++ -t hw_emu --platform /opt/xilinx/platforms/
xilinx_u200_xdma_201830_1/xilinx_
u200_xdma_201830_1.xpfm --save-temps -l --connectivity.nk
krnl_vadd:1
-g --messageDb binary_container_1.mdb
--temp_dir binary_container_1
--report_dir binary_container_1/reports --log_dir
binary_container_1/logs
--remote_ip_cache /wrk/tutorials/ip_cache

```

```

                                -obinary_container_1.xclbin binary_container_1/
krnl_vadd.o
Options:
    -t hw_emu
    --platform /opt/xilinx/platforms/xilinx_u200_xdma_201830_1/
xilinx_u200_xdma_201830_1.xpfm
    --save-temps
    -l
    --connectivity.nk krnl_vadd:1
    -g
    --messageDb binary_container_1.mdb
    --temp_dir binary_container_1
    --report_dir binary_container_1/reports
    --log_dir binary_container_1/logs
    --remote_ip_cache /wrk/tutorials/ip_cache
    -obinary_container_1.xclbin binary_container_1/krnl_vadd.o
=====
==
User Added Key Value Pairs
-----
    <empty>
=====
==

```

xrt.ini File

The Xilinx Runtime (XRT) library uses various control parameters to specify debugging, profiling, and message logging when running the host application and kernel execution. These control parameters are specified in a runtime initialization file, `xrt.ini` and used to configure features of XRT at start-up.

If you are a command line user, the `xrt.ini` file needs to be created manually and saved to the same directory as the host executable.

The runtime library checks if `xrt.ini` exists in the same directory as the host executable and automatically reads the file to configure the runtime. You can also specify the location of an `xrt.ini` file at runtime by setting the `XRT_INI_PATH` environment variable to point to the file, for example:

```
export XRT_INI_PATH=/path/to/xrt.ini
```



TIP: The AMD Vitis™ IDE creates an `xrt.ini` file automatically based on your run configuration and saves it in the run configuration folder.

Runtime Initialization File Format

The `xrt.ini` file is a simple text file with groups of keys and their values. Any line beginning with a semicolon (;) or a hash (#) is a comment. The group names, keys, and key values are all case sensitive.

The following is an example `xrt.ini` file that enables the timeline trace feature, and directs the runtime log messages to the Console view.

```
#Start of Debug group
[Debug]
native_xrt_trace = true
device_trace = fine

#Start of Runtime group
[Runtime]
runtime_log = console
```

There are three groups of initialization keys:

- Runtime

- Debug
 - AIE_profile_settings
 - AIE_trace_settings
- Emulation

The following tables list all supported keys for each group, the supported values for each key, and a short description of the purpose of the key.

Runtime Group

The Runtime group of switches lets you configure elements of the runtime operation as described below.

Table 63: Runtime Group Keys and Values

Key	Valid Values	Description
api_checks	[true false]	Enables or disables OpenCL API checks. • true: Enable. This is the default value. • false: Disable.
cpu_affinity	{N,N,...}	Pins all runtime threads to specified CPUs. Example: <pre>cpu_affinity = {4,5,6}</pre>
exclusive_cu_context	[true false]	This allows the host application to direct OpenCL to acquire exclusive CU access, so that low-level AXI read/write (xclRegRead and xclRegWrite) can be used for regular kernels.
force_program_xclbin	[true false]	Forces a reconfigurable module (RM) .xclbin file to be reloaded into the device, even if it is already loaded. This is used to force the reloading of a DFX region in the platform with the .xclbin when requested.
runtime_log	[null console syslog <filename>]	Specifies where the runtime logs are printed • null: Do not print any logs. This is the default value. • console: Print logs to stdout • syslog: Print logs to Linux syslog. • <filename>: Print logs to the specified file. For example, <code>runtime_log=my_run.log</code>.
verbosity	[0 1 2 3 4 5 6 7]	Verbosity of the log messages. The default value is 4.

Debug Group

The Debug group of switches define key options for the enabling profiling of the application during runtime, or tracing data transfers and execution. These switches apply to both AI Engine and PL kernels in the Vitis acceleration flow, and let you configure aspects of the runtime to control the frequency of data capture, the events to capture, and the amount of memory to reserve or use for recording trace and profile data.

Table 64: Debug Options

Key	Valid Values	Description
<code>aie_profile</code>	<code>[true false]</code>	Enables the runtime configuration and polling of AI Engine hardware performance counters. Available on VCK190 hardware and hardware emulation runs. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
<code>aie_trace</code>	<code>[true false]</code>	Enables the runtime configuration and collection of AI Engine event trace. Available on VCK190 hardware runs only. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
<code>aie_status</code>	<code>[true false]</code>	Enables the polling of AI Engine status information. Available on VCK190 hardware and hardware emulation runs.
<code>aie_status_interval_us</code>	integer (default=1000us)	Controls the interval at which AI Engine status information is captured. Specified in microseconds.
<code>app-debug</code>	<code>[true false]</code>	Enables the OpenCL application debug for the host code when debugging with GDB. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
<code>continuous_trace</code>	<code>[true false]</code>	Enables the continuous dumping of files for trace and the continuous reading of device data into the host. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value. Note: This switch only has an effect if <code>device_trace</code> is enabled.
<code>device_counters</code>	<code>[true false]</code>	Enables device counter offload only, without enabling trace functionality.

Table 64: Debug Options (cont'd)

Key	Valid Values	Description
device_trace	[off fine coarse accel]	Enables the collection of data from monitors inserted on the PL to add to summary and trace. • <code>accel</code>: Traces compute unit starts/stops. • <code>coarse</code>: Lumps all reads/writes together under each execution of a compute unit. • <code>fine</code>: Tracks everything as it happens. • <code>off</code>: Turns off reading and reporting of device-level trace during runtime. This is the default value.
host_trace	[true false]	Enables trace of host code based on the first protocol encountered. TIP: If your host application uses both OpenCL and XRT native API you should manually specify both <code>opencl_trace</code> and <code>native_xrt_trace</code> to capture all events.
lop_trace	[true false]	Enables generation of lower overhead OpenCL API host trace. Should not be used with other OpenCL options. • <code>true</code>: Enable. • <code>false</code>: Disable. This is the default value.
native_xrt_trace	[true false]	Enables generation of the Native C/C++ API trace. This also generates the tables for "Host Data Transfer from/to Global memory" in the Profile Summary. • <code>true</code>: Enable. • <code>false</code>: Disable. This is the default value.
opencl_trace	[true false]	Enables generation of OpenCL API host trace. • <code>true</code>: Enable. • <code>false</code>: Disable. This is the default value.
pl_deadlock_detection	[true false]	Enables deadlock detection for PL kernels.
power_profile	[true false]	Enables the polling of power data during the execution of the application. • <code>true</code>: Enable. • <code>false</code>: Disable. This is the default value. Note: This feature is not supported on embedded platforms or AWS.

Table 64: Debug Options (cont'd)

Key	Valid Values	Description
power_profile_interval_ms	<int>(default=20)	Controls the interval of reading the power counters in milliseconds. The default interval is 20 ms. Note: This switch only has an effect if <code>power_profile = true</code> .
profile_api	[true false]	Enables access to HAL API directly from the host application to read counters on device profiling monitors during execution. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
stall_trace	[off all dataflow memory pipe]	Specifies the type of device-side stalls to capture and report in the timeline trace. The default is off. <code>off</code>: Turn off stall trace information. <code>all</code>: Record all stall trace information. <code>dataflow</code>: Intra-kernel streams (for example, writing to full FIFO between dataflow blocks). <code>memory</code>: External memory stalls (for example, AXI4 read from the DDR memory). <code>pipe</code>: Inter-kernel pipe for OpenCL kernels (for example, writing to full pipe between kernels). Note: This switch only has an effect if <code>device_trace</code> is enabled.
trace_buffer_offload_interval_ms	<int>	Controls the reading of device data from the device to the host in milliseconds (ms). The default is 10 ms. Note: This switch only has an effect if <code>device_trace</code> is enabled.
trace_buffer_size	<string>	If the <code>.xclbin</code> was created with memory offload of trace specified, as described in --profile Options , this switch determines the size of the buffer to allocate in memory to capture trace data. The default is 1M. Note: This switch only has an effect if <code>device_trace</code> is enabled.
trace_file_dump_interval_s	<int>	Controls the time between dumping of trace files in seconds (s). The default is 5s. Note: This switch only has an effect if <code>device_trace</code> is enabled.

Table 64: Debug Options (cont'd)

Key	Valid Values	Description
<code>vitis_ai_profile</code>	<code>[true false]</code>	Profile summary and other files come from Vitis AI application layer. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
<code>xocl_debug</code>	<code>[true false]</code>	Generates the <code>xocl.log</code> file when enabled. When any trace options are also enabled, the debug log is added to the <code>xrt.run_summary</code> to view in Vitis Analyzer.
<code>xrt_trace</code>	<code>[true false]</code>	Enables generation of low-level HW shim function trace during HW runs. This will be disabled when used with <code>native_xrt_trace</code> . <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.

AIE_profile_settings Group

The options specified in this group are applied only if `aie_profile=true` under the `[Debug]` group.

Table 65: AI Engine Profile Options

Key	Valid Values	Description
<code>graph_based_aie_metrics</code>	<code><graph name all>:<kernel name all>:<off heat_map stalls execution floating_point write_bandwidths read_bandwidths aie_trace></code>	Specify the metric sets reported by the AI Engine module of AI Engine tiles on a graph-by-graph basis. IMPORTANT! Currently, only <i>all</i> is supported for kernel specification. Controls the configuration of the statistics read from the AIE core performance counters for the entire AI Engine graph application. <code>heat_map</code> : profile active/stall cycles and vector instruction usage <code>stalls</code> : profile the different types of stalls (i.e., memory, stream, lock, and cascade) <code>execution</code> : profile the AI Engine instructions <code>floating_point</code> : profile floating point exceptions <code>write_bandwidths</code> : profile the write bandwidth of streams and cascades <code>read_bandwidths</code> : profile the read bandwidths of streams and cascades <code>aie_trace</code> : profile amount and stalls of event trace from core and memory modules

Table 65: AI Engine Profile Options (cont'd)

Key	Valid Values	Description
graph-based-aie-memory-metrics	<graph name all>:<kernel name all>:<off conflicts dma_locks dma_stalls_s2mm dma_stalls_mm2s write_bandwidths read_bandwidths>	Specify the metric sets reported by the memory module of AI Engine tiles on a graph-by-graph basis. IMPORTANT! Currently, only <i>all</i> is supported for kernel specification. Controls the configuration of statistics read from the AI Engine memory performance counters for the entire AI Engine graph application. conflicts: profile the DMA memory conflicts dma_locks: profile DMA locks and stalls on lock acquire dma_stalls_s2mm: profile stalls on DMA S2MM channels dma_stalls_mm2s: profile stalls on DMA MM2S channels write_bandwidths: profile bandwidths of DMA S2MM channels read_bandwidths: profile bandwidths of DMA MM2S channels
tile-based-aie-metrics	<{<column>,<row>} all>:<off heat_map stalls execution floating_point write_bandwidths read_bandwidths aie_trace> ; {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}:<off heat_map stalls execution floating_point write_bandwidths read_bandwidths aie_trace>	Specify the metric sets reported by the AI Engine module of AI Engine tiles on a tile-by-tile basis. This can be used in conjunction with graph-by-graph selection and will take priority on the specified tiles. Refer to descriptions from <code>graph-based-aie-metrics</code>
tile-based-aie-memory-metrics	<{<column>,<row>} all>:<off conflicts dma_locks dma_stalls_s2mm dma_stalls_mm2s write_bandwidths read_bandwidths> ; {<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}:<off conflicts dma_locks dma_stalls_s2mm dma_stalls_mm2s write_bandwidths read_bandwidths>	Specify the metric sets reported by the memory module of AI Engine tiles on a tile-by-tile basis. This can be used in conjunction with graph-by-graph selection and will take priority on the specified tiles. Refer to descriptions from <code>graph-based-aie-memory-metrics</code>
tile-based-interface-tile-metrics	<column> all>:<off input_bandwidths output_bandwidths packets>[:<channel>] ; <mincolumn>:<maxcolumn>:<off input_bandwidths output_bandwidths packets>[:<channel>]	Specify the metric sets reported by the AI Engine interface tiles on a tile-by-tile basis. Note: Interface tiles are separate from the AI Engine tiles and have different metric sets.
interval_us	<int>	Controls the interval of reading the AI Engine counter values in microseconds (μs). The default interval is 1000 μs. Note: This switch only has an effect if <code>aie_profile = true</code>.

AIE_trace_settings Group

The options specified in this group are applied only if `aie_trace=true` under the `[Debug]` group.

Table 66: AI Engine Trace Options

Key	Valid Values	Description
<code>buffer_size</code>	<code><string></code> (default=8M)	Controls the total size of the buffers allocated for AI Engine event trace. This size is partitioned evenly into the number of different trace streams coming out of the AI Engine. The default is 8M. Note: This switch only has an effect if <code>aie_trace = true</code> .
<code>buffer_offload_interval_us</code>	integer (default=10ms)	Interval, in milliseconds, between reading of PLIO mode AI Engine trace from device to Host memory.
<code>periodic_offload</code>	true/false (default=true)	Enables continuous offload of PLIO mode AI Engine trace. Generated AI Engine trace output files (one per stream) gets appended with new trace data.
<code>file_dump_interval_s</code>	integer (default=5s)	Interval, in seconds, between writing (appending) of raw AI Engine trace data to output files.
<code>graph_based_aie_tile_metrics</code>	string("") <code><graph name all>:<kernel name all>:<off functions functions_partial_stalls functions_all_stalls></code>	Specify the metric sets reported by the AI Engine module of AI Engine tiles on a graph-by-graph basis. IMPORTANT! Currently, only <i>all</i> is supported for kernel specification.
<code>tile_based_aie_tile_metrics</code>	string("") <code><{<column>,<row>} all>:<off functions functions_partial_stalls functions_all_stalls>[:<memory_stalls stream_stalls cascade_stalls lock_stalls>]</code> <code>{<mincolumn>,<minrow>}:<{<maxcolumn>,<maxrow>}>:<off functions functions_partial_stalls functions_all_stalls></code>	Specify the metric sets reported by the AI Engine module of AI Engine tiles on a tile-by-tile basis. IMPORTANT! Currently, only <i>all</i> is supported for kernel specification.
<code>reuse_buffer</code>	true/false (false)	

Emulation Group

The Emulation group of switches apply to the emulation environments and the AMD Vivado™ simulator.

Table 67: Emulation Group Keys and Values

Key	Valid Values	Description
<code>aliveness_message_interval</code>	Any integer	Specifies the interval in seconds that aliveness messages need to be printed. The default is 300.
<code>debug_mode</code>	<code>[off batch gui]</code>	Specifies how the waveform is saved and displayed during emulation. <code>off</code>: Do not launch simulator waveform GUI, and do not save <code>wdb</code> file. This is the default value. <code>batch</code>: Do not launch simulator waveform GUI, but save <code>wdb</code> file <code>gui</code>: Launch simulator waveform GUI, and save <code>wdb</code> file Note: The kernel needs to be compiled with debug enabled (<code>v++ -g</code>) for the waveform to be saved and displayed in the simulator GUI.
<code>kernel-dbg</code>	<code>[true false]</code>	Enables kernel debug functionality during software emulation as described in Command Line Debug Flow for Software Emulation. <code>true</code>: Enable. <code>false</code>: Disable. This is the default value.
<code>print_infos_in_console</code>	<code>[true false]</code>	Controls the printing of emulation info messages to user's console. Emulation info messages are always logged into a file called <code>emulation_debug.log</code> <code>true</code>: Print in user's console. This is the default value. <code>false</code>: Do not print in user console.
<code>print_warnings_in_console</code>	<code>[true false]</code>	Controls the printing emulation warning messages to user's console. Emulation warning messages are always logged into a file called <code>emulation_debug.log</code>. <code>true</code>: Print in user's console. This is the default value. <code>false</code>: Do not print in user console.
<code>print_errors_in_console</code>	<code>[true false]</code>	Controls printing emulation error messages in user's console. Emulation error messages are always logged into the <code>emulation_debug.log</code> file. <code>true</code>: Print in user's console. This is the default value. <code>false</code>: Do not print in user's console.
<code>user_pre_sim_script</code>	Path to Tcl file	For the first run, run simulation in GUI mode. Add signals that you want to add. Copy the commands from the Tcl console and save into a Tcl script. For the next run, pass the Tcl script in batch mode.

Table 67: Emulation Group Keys and Values (cont'd)

Key	Valid Values	Description
<code>user_post_sim_script</code>	Path to Tcl file	Any post operations can be specified in the Tcl and pass to the switch. All the command provided in the Tcl gets executed after simulation is completed.
<code>xtlm_aximm_log</code>	<code>[true false]</code>	Enables the XTLM AXI4 Memory Map transaction logging at runtime and you could see all the transactions in the <code>xsc_report.log</code> file.
<code>xtlm_axis_log</code>	<code>[true false]</code>	Enables the XTLM AXI4-Stream transaction logging at runtime and you could see all the transactions in the <code>xsc_report.log</code> file.
<code>timeout_scale</code>	<code>na/ms/sec/min</code>	Timeout support for <code>clPollStream</code> API in emulation. Provides a scale for the timeout specified in <code>clPollStream</code> API. The timeout specified in the code is specified in ms, and might not work for emulation. Therefore use the <code>timeout_scale</code> to map ms to another scale if needed for emulation. IMPORTANT! <i>Timeout is not enabled in emulation by default. Use this option to enable <code>clPollStream</code> timeout.</i>

Using the Vitis Unified IDE

The Vitis command-line tool flow described in [Section IV: Building and Running the System](#) is also available in the Vitis unified IDE. The process of building and combining the different components of the system project, the Application component, the AI Engine component, the HLS component, and the platform are described in the following sections.

- [Launching Vitis Unified IDE](#)
- [Working with the Vitis Unified IDE](#)
- [Creating an AI Engine Component](#)
- [Creating an HLS Component](#)
- [Creating an Application Component](#)
- [Creating a System Project for Heterogeneous Computing](#)
- [Debugging the System Project and AI Engine Components](#)
- [Vitis Interactive Python Shell](#)



TIP: Before using the Vitis unified IDE, you must first set up the development environment as described in [Setting Up the Vitis Environment](#).

Launching Vitis Unified IDE

Note: The AMD Vitis™ Unified Integrated Design Environment (IDE) supports data center acceleration and embedded system design, AI Engine and High-Level Synthesis (HLS) component creation, platform creation, and embedded software design. You can launch this tool by using the following command:

```
vitis -w <workspace>
```

In the AMD Vitis™ unified IDE, you can create a new embedded application project or platform development project. The `vitis` command launches the Vitis unified IDE with your defined options. It provides options for specifying the workspace and options of the project. The following sections describe the `vitis` command options.

Command Options

The following command options specify how the `vitis` command is configured for the current workspace and project.

```
vitis [--classic | -a | -w | -i | -s | -h | -v]
```

- **-a/--analyze** [**<summary file | folder | waveform file: *.`[wdb|wcfg]`>**]: Open the summary file in the Analysis view. Opening a folder opens the summary files found in the folder. Open the waveform file in a waveform view tab. If no file or folder is specified, opens the Analysis view.
- **-workspace** **<workspace location>**: Specify the workspace directory for Vitis Unified IDE projects.
- **-i/--interactive**: Launches Vitis Python interactive shell.
- **-s/--source** **<python_script>**: Runs the specified Python script.
- **-j/--jupyter**: Launches Vitis Jupyter web UI.
- **-help**: Displays help information for the Vitis command options.
- **-version**: Displays the Vitis tool release version.

Classic Options

You can launch the Vitis classic IDE using the `--classic` option. The options in this section apply to the Vitis classic IDE only.

- **--classic**: Launches the classic Vitis IDE rather than the new unified IDE.



IMPORTANT! *The classic version of the Vitis IDE is deprecated and will be discontinued in a future release.*

- `-debug`: Launches the Vitis IDE to run debug on a command-line project.



TIP: *To view the help for the `vitis -debug` command, use `-debug -help`.*

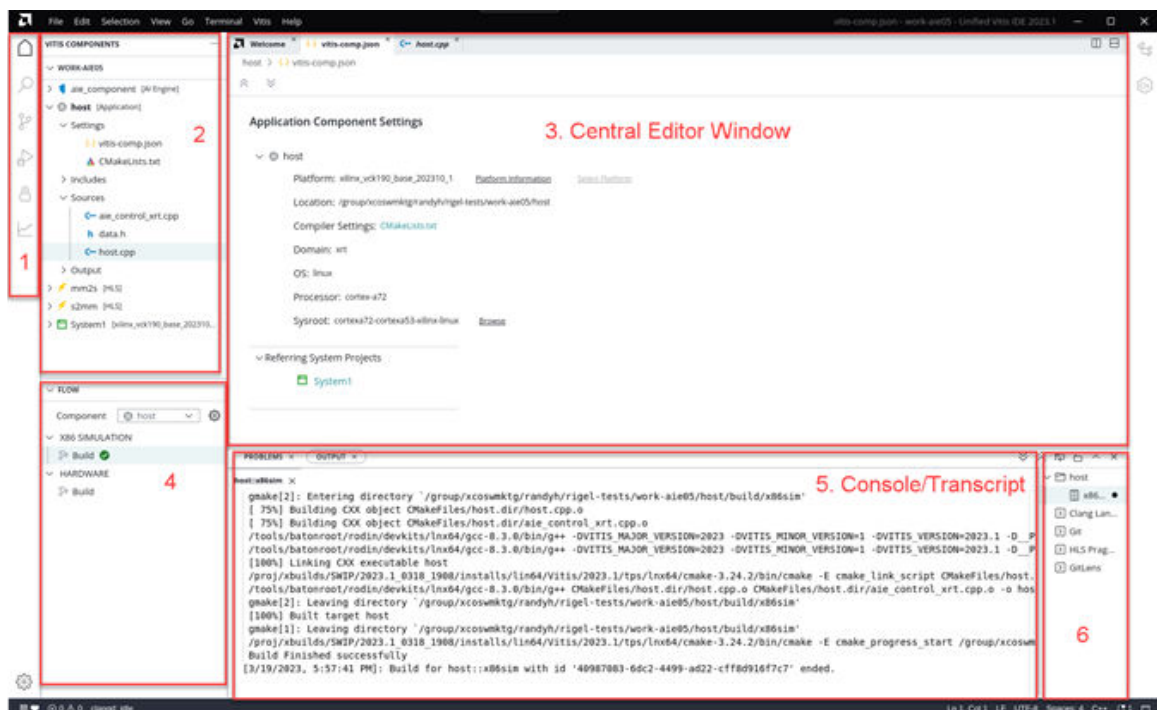
- `{-lp <repository_path>}`: Add `<repository_path>` to the list of Driver/OS/Library search directories for the Vitis classic IDE.
- `-eclipseargs <eclipse arguments>`: Eclipse-specific arguments are passed to Eclipse for the Vitis classic IDE.
- `-vmargs <java vm arguments>`: Additional arguments to be passed to Java VM for the Vitis classic IDE.

Working with the Vitis Unified IDE

Features of the Vitis Unified IDE

The new AMD Vitis™ Unified IDE provides many new features, which are described in brief below:

Figure 64: Vitis Unified IDE



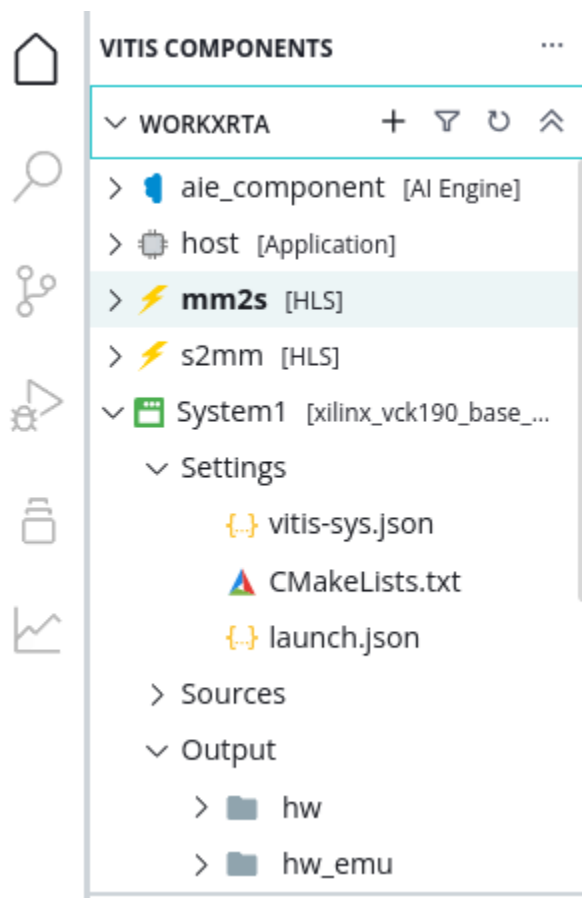
1. The Tool Bar on the left of the screen provides easy access to major features of the tool: the Vitis Component Explorer, the Search function, Source Control, the Debug view, Examples, and the Analysis view.
2. The Vitis Component Explorer can be used to view a virtual hierarchy of the workspace. The view displays a workspace that is structured to help you understand the different elements of the component or project, such a Sources and Outputs folder that don't exist on disk.
3. The Central Editor window is used for editing components, configurations, and source files.

4. The Flow Navigator displays the design flow for the active component. Different components has different work flows, and the work flow of the active component is the one displayed in the Flow Navigator. You can specify the active component by selecting it in the Flow Navigator, or selecting it in the Component Explorer.
5. The Console/Terminal area displays the output transcripts of the tool, and other windows such as the Terminal window and the Pipeline view are also located here. The terminal window displays the folders of the workspace and can be used to run scripts on the content.
6. The Index displays a list of transcripts of the various build steps ran during the session and allows you to reopen a process transcript that was closed earlier.

Using Vitis Component Explorer

You can use the Vitis Component Explorer to manage the components and projects in your workspace.

Figure 65: Vitis Component Explorer



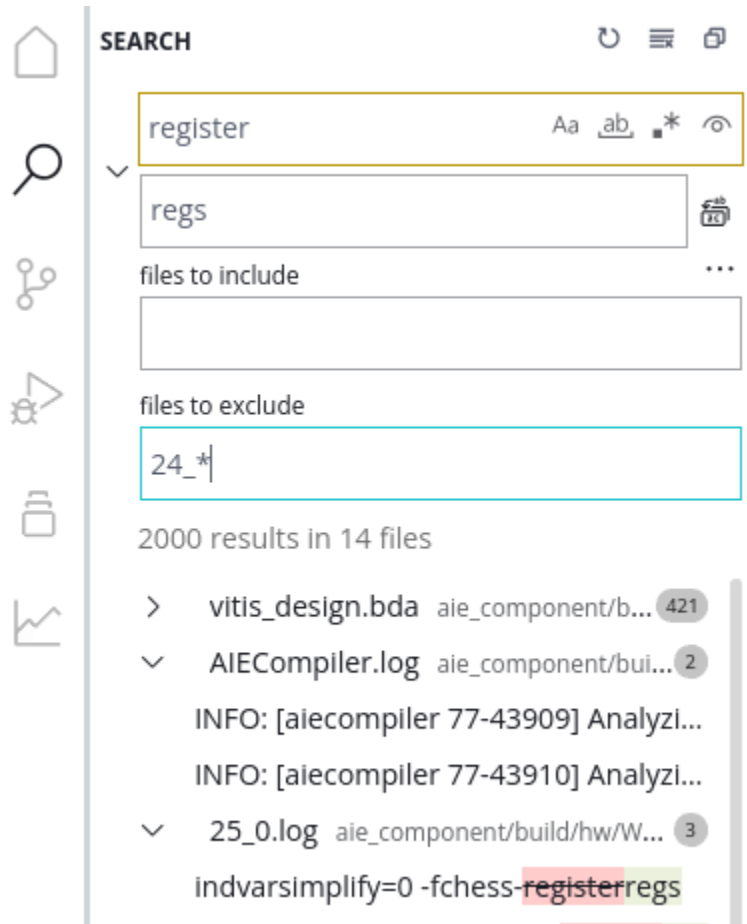
You can use the Component Explorer to perform the following actions:

- **Via the toolbar menu:**
 - Create a New Component.
 - Filter Components to hide or show the components of interest.
 - Refresh the Component Explorer.
 - Collapse All to minimize the displayed content of the components and projects.
- **Via selecting a component or project and using the right-click menu:**
 - Open in Terminal to open a terminal window and changes directory to inside the component.
 - Microcode displays the microcode associated with a specified kernel from the graph. This command is only available for AI Engine components.
 - Show in Flow Navigator: click the component to show it in the Flow Navigator.
 - Delete to remove the component or project from the workspace.
 - Clone Component to create a new component from an existing component. This is useful for design exploration where you preserve the original component as your baseline, and use the cloned component for design exploration. This command does not support System projects.
- **Via right-clicking the Sources folder of a component or project:**
 - New File to create a new file in the component or project.
 - New Folder to create a new folder in the component or project.
 - Open in Terminal to open a terminal window and changes directory to inside the component.
 - Paste to take a copied item and paste it into the component or project. This creates an actual copy of the previously selected and copied item.
 - **Import → Files** to import files into the component or project.
 - **Import → Folders** to import a folder into the component or project.
 - Add Source File or create a New Source File.
- **Via right-clicking an object in the component or project hierarchy:**
 - Copy to copy the current object. This action can be used with Past to copy an object from one component or project to another.
 - Copy Path to copy the absolute path of the object. The path can be pasted into the Terminal view of a configuration file.
 - Copy Relative Path to copy the path of the object relative to the current workspace. The path can be pasted into the Terminal view of a configuration file.
 - Delete to remove the object from the workspace.

Search View

The search view can help users to find and replace text globally within the current workspace. The search result includes text files in the workspace including source files, configuration files, and log files. The search occurs inside of files, and does not search file names.

Figure 66: Search View



As shown in the image above, some of the features of the Search view include the following:

- **Match Case:** Match the case of the provided search string
- **Match Whole Word:** Match only whole words
- **Use Regular Expression:** use regular expression to define the search
- **Include Ignored Files:** Restores ignored files to the search list
- **Replace All:** When replace is enabled, replace all search results

- **Toggle Search Details:** Expand Search view to add Files to Include and Files to Exclude fields
- **Files to include:** Specify a list of files to restrict your search. The listed files can include wildcards, and each entry must be separated by a comma. For example, the following includes (or excludes) the `AIECompiler.log` file and all files with `summary` in the name:

```
AIECompiler.log, *summary*
```

- **Files to exclude:** Specify a list of files to restrict your search. See above for example.
- **Clear Search Results:** Clear the search string, replace string, and search results



TIP: Be careful using the *Replace* function, as it can introduce errors into your designs.

Source Control

Source Control or Version Control technique is widely used in software development flow. This chapter describes how to use the Git integration in Vitis unified IDE .

Source Control

To enable the Source Control view you must initialize your empty workspace as a git repository. After creating an empty workspace, and launching the Vitis Unified IDE to open the workspace, you can add it to your git repository using the following steps:

1. From the Terminal menu, select **New Terminal**. The terminal is opened by default to the folder that is your workspace.
2. In the Terminal window, type in the command `git init` and press **Enter**.

You should see a message such as: *Initialized empty Git repository in /tests/temp/workVADD/.git/*.



IMPORTANT! Using the Source Control view with Git requires you to have a User ID and Password established, and provided to the system.

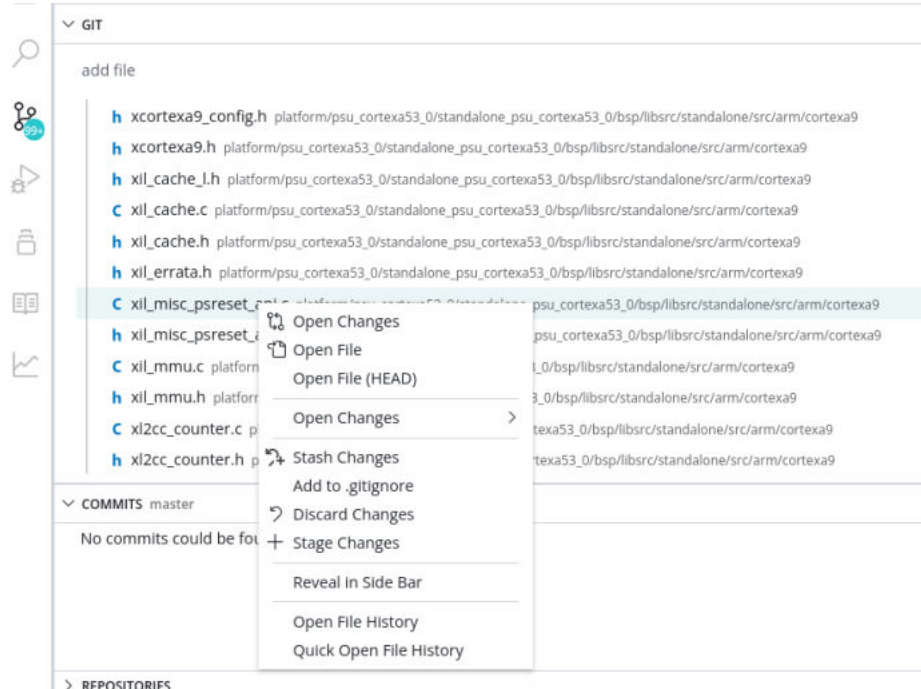
After you create a new component or project in the workspace, Vitis Unified IDE generates a `.gitignore` file. The `.gitignore` file can help you filter out the generated files so that it is easier to pick the files for source control. You can open the `.gitignore` file and edit this file if you have additional requirements.

The Source Control view is a GUI helper for Git. You can use Git commands and the source control view simultaneously for your project. Updates in the command line show up in the source control view and vice versa.

Add New File for Source Control

To add a file for source control, you can do the following:

Right click the file you want to add and select **+ Stage Changes**



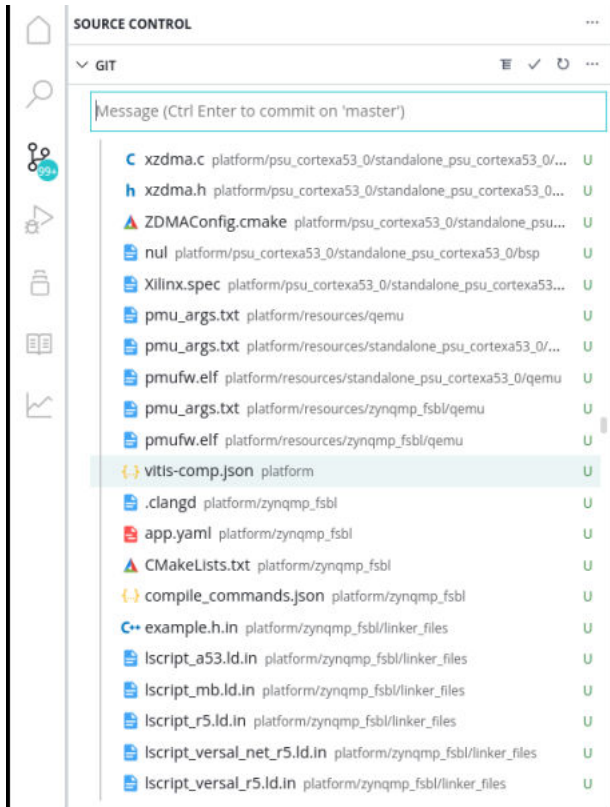
This GUI is equivalent to the following git command.

```
git add <file name>
```

Commit the Changes

To commit the change, you can do either of the following:

Input the commit message and Press **Ctrl + Enter** after you select the git view.



This GUI is equivalent to the following git command:

```
git commit -m <commit message>
```

Push the Project to the Remote Repository

To push to your remote repository, you can do either of the following:

```
git push --set-upstream origin master
```

- origin: this is the remote repo address
- master: this is the branch of your local workspace coed version.

```
git push https://your_repo/vitis_project master
```

Note: The first time you execute `git init`, it will automatically create a branch named **master**. You can use `git branch <branch name>` to create a new branch.

You can find your local project is in the remote repository.

Launch Configurations

Launch configurations save the environment setup for running or debugging the project. The launch configurations are stored in a `launch.json` file associated with a component or System project. The `launch.json` file can contain one or more launch configurations for running or debugging the component or project.

To view and edit launch configurations you can select the `launch.json` file for a component or System project in the Component Explorer view, or select **Open Settings** next to Run or Debug in the Flow Navigator.



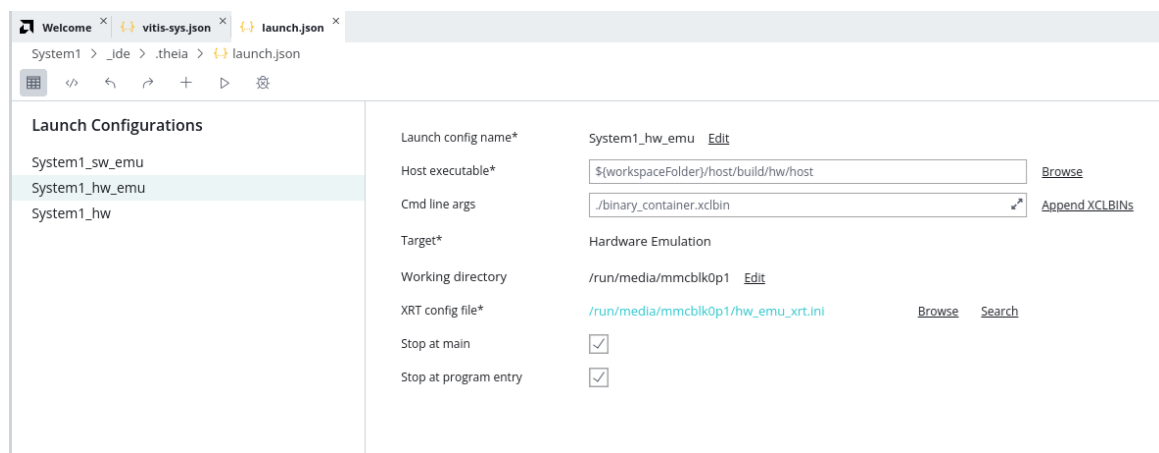
TIP: The Open Settings command is a hidden icon that appears when you place your cursor in the Flow Navigator next to either Run or Debug

Figure 67: Run and Debug in the Flow Navigator



This opens the `launch.json` file and lets you edit the launch configurations contained within. The Launch Configuration editor has a toolbar menu at the top, as shown in the following figure. The figure shows the System Project Launch Configuration editor as an example. The specific contents of the launch configuration will vary depending on the component or project selected.

Figure 68: Launch Configurations



The commands in the toolbar include:

- **Settings Form / Source Editor:** You can toggle between the GUI view and Text Editor view of the `launch.json` file with these commands.

- **Undo/Redo:** undo or redo the last actions.
- **Add Configuration:** You can add a configuration by clicking the + button. The default launch configurations for System projects are: Emulation SW, Emulation HW and Hardware.
- **Run / Debug:** Launch run or debug using the currently selected launch configuration.

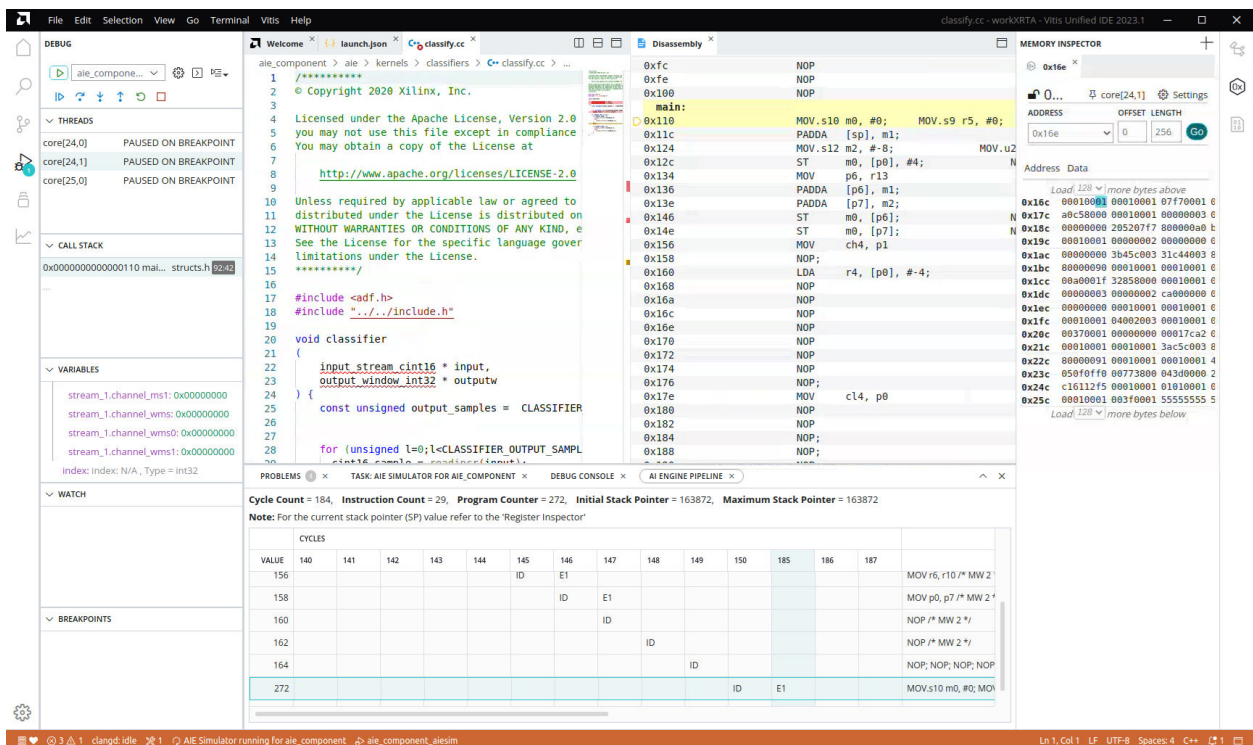
Place your mouse over the configuration name and the Duplicate and Delete commands will appear. Select **Delete** to remove the launch configuration, or **Duplicate** to copy the launch configuration.

In each configuration, you can update the settings to configure the tool prior to running or debugging the component or project. After setting up the launch configuration, you can select Run or Debug commands in the Launch Configuration editor, or by selecting Run or Debug from the Flow Navigator and specifying a launch configuration if more than one is available.

Debug View

The Vitis IDE debug environment has many features found in traditional GUI-based debug environments, such as GDB. You can add break points to the code, step over or step into specific lines of codes, loops, or functions, and examine the state of variables and force them to specific values. The Debug view contains several windows or views as shown in the following figure.

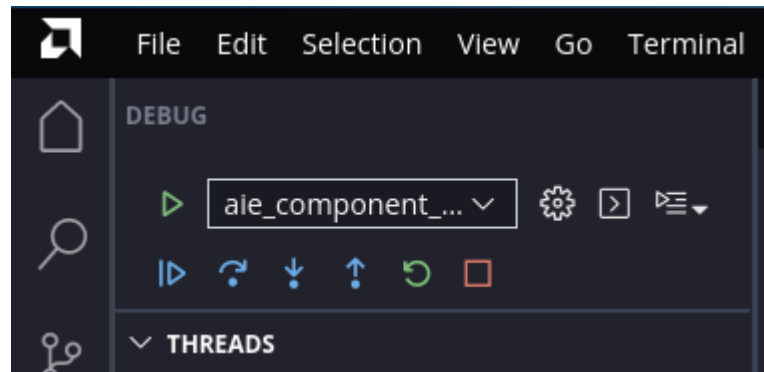
Figure 69: Debug view



- **Control Panel:**

The Debug view Control Panel is displayed in the upper-left corner of the screen as shown in the image below. During debugging, you can use the control buttons such as Continue, Step Over, Step Into, Step Out, Restart, and Stop to control the debugging process.

Figure 70: Debug Control Panel



- **Threads:** Threads shows the related debugging threads. Threads are created and destroyed during the debugging process. You can switch between multiple threads.
- **Call Stack:** Call Stack shows the function call stack being updated as the application is run.
- **Variables:** Variables shows the current value of global and local variables. When switching threads, the variable information is updated.
- **Watch:** Watch shows variables and expressions you have specified to watch. To add watch points select **Add Expression (+)**.
- **Breakpoints:**

Vitis IDE sets break points at the main function of the host component and at the top function of the PL kernels if they can be debugged. To add breakpoints, you can open the source file and click the left side of the line number when a red dot appears. You can remove breakpoints by clicking on a previously added breakpoint.

You can add conditional breakpoints by right-clicking when the red dot appears and select **Add Conditional Breakpoint**. You can also right-click and select **Add Logpoint** to insert a message to be logged when the breakpoint is reached.
- **Source Code Editor:** The Source Code view is opened when Debug is launched from the Flow Navigator or Launch Configuration view.
- **Memory Inspector:** The Memory Inspector displays the content of specific memory addresses.
- **Register Inspector:** The Register Inspector shows the registers of the Cortex-A72 when a breakpoint is triggered in the Application Component source code, and the AI Engine when a breakpoint is triggered in the AI Engine kernel.

- **Disassembly view:** You can open the Disassembly view from the Source Code window right-click menu.
- **Debug Console:** Displays the transcript of the debug process, and any messages received from the tested application, such as from `printf()` statements..



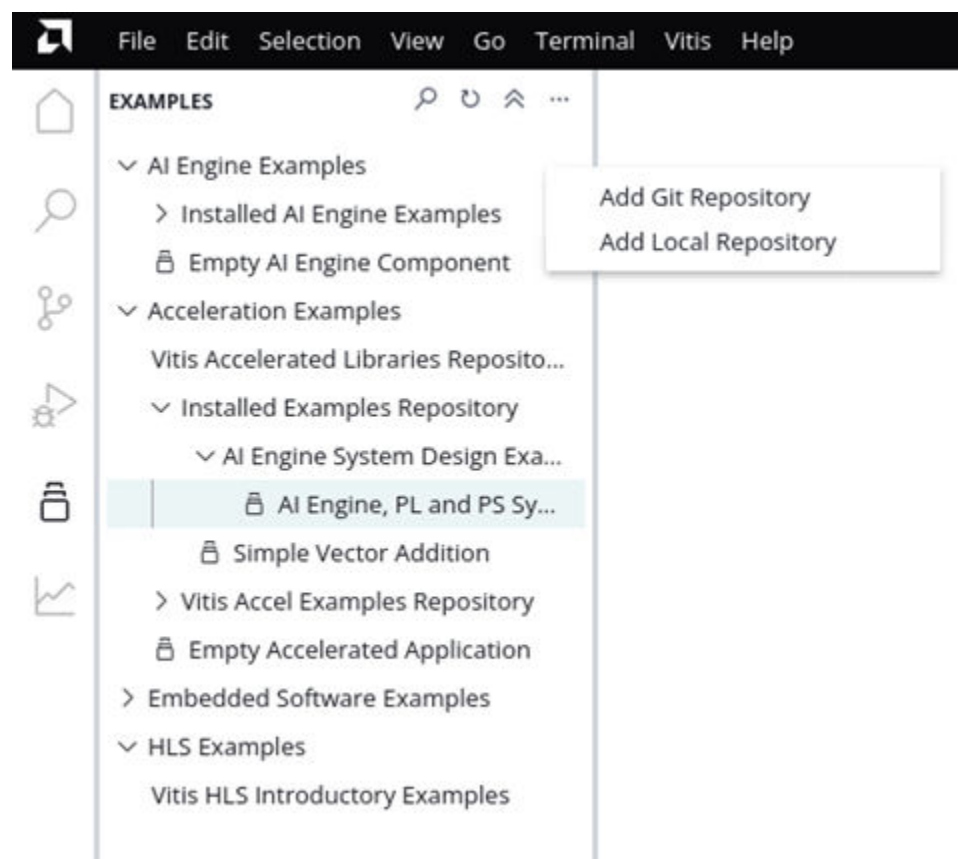
TIP: The Vitis unified IDE also shows the status of the Debug view from the toolbar menu, with a blue circle highlighting the number of running sessions in debug. This can be a helpful reminder when you have changed views and Debug is still running.



Examples View

You can access the Examples view from the left side panel or from **View → Examples**, or by using **Ctrl + Shift + R** shortcut keys. In this view, you can create example System projects to explore the tool, and manage example repositories.

Figure 71: Examples View



To use an example select it from the Examples view and a template will open letting you create a new System project from the example. Select the **Create Application from Template** command and the Create System Project wizard is displayed letting you create a new System project as described in [Creating a System Project](#).

To add a new repository of examples do the following:

1. Click the ellipses (...) option on the right of the view title as shown in the figure above, and select **Add Git Repository** or **Add Local Repository**.
 - If the repository is remote, you can use **Add Git Repository**. You can use Vitis IDE to download the repository.
 - If the repository is local, you can use **Add Local Repository**.
2. You can edit the repository information by hovering over the repository line and click **Edit** button. You can provide or modify the following information for any Git repository or local repository.

Repository Metadata

The following table describes the repository metadata:

Table 68: Repository Metadata

Git Repo	Local Repo	Required
Name	Name	Yes
Id	Id	Yes
Description	Description	No
Git Url	NA	Yes
Branch	NA	Yes
Local Path	Local Path	Yes

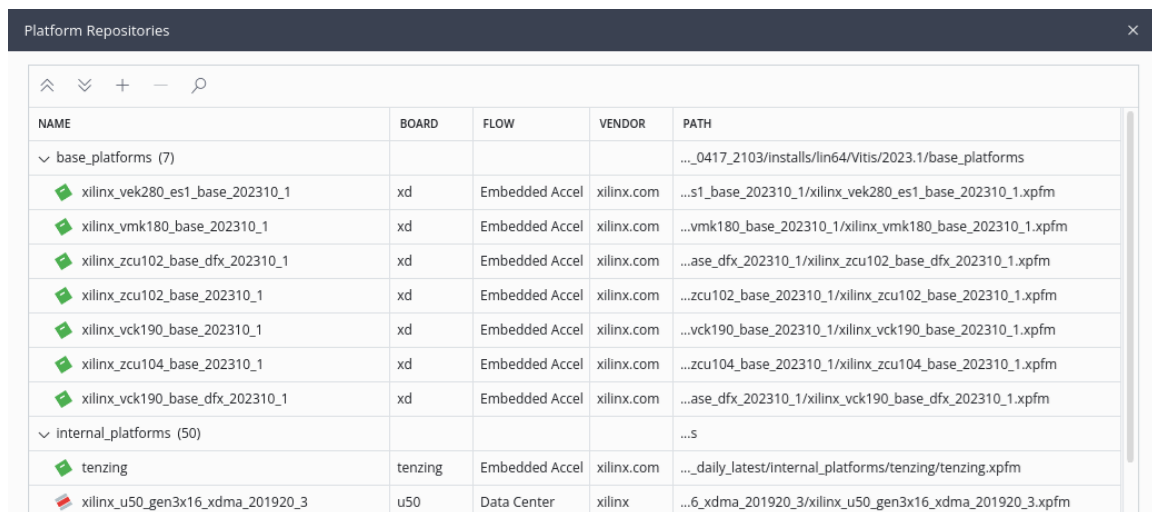
After adding repositories or modifying repository metadata, or if your repository content changes, you need to run the **sync** action. Vitis IDE scans the repository and generates a new map of contents. Right-click on a repository and select **Delete** to remove a repository. It only removes the repository from the Application Examples view, and it does not remove your files on the disk.

Managing Platform Repositories

The Platform Repositories window displays the platforms that are available for use with the new Vitis Unified IDE. The upper part of the table displays the `base_platforms` that are installed with the software installation, and are available to all users. The lower part of the table displays the locations specific as part of the `$PLATFORM_REPO_PATHS` environment variable. This shows the platforms that were installed, in addition to the Vitis tools to work with the tools. To manage platform repositories, do the following:

1. Go to **Vitis → Platform Repositories** in the menu.
2. Click + or - to add or remove a platform search directory from the list.
3. Select a platform directory to view platforms of that directory.
4. Select the **info** link next to a platform to view its detailed information.

Figure 72: Platform Repositories



NAME	BOARD	FLOW	VENDOR	PATH
base_platforms (7)				..._0417_2103/installs/lin64/Vitis/2023.1/base_platforms
xilinx_vek280_es1_base_202310_1	xd	Embedded Accel	xilinx.com	...s1_base_202310_1/xilinx_vek280_es1_base_202310_1.xpfm
xilinx_vmk180_base_202310_1	xd	Embedded Accel	xilinx.com	...vmk180_base_202310_1/xilinx_vmk180_base_202310_1.xpfm
xilinx_zcu102_base_dfx_202310_1	xd	Embedded Accel	xilinx.com	...ase_dfx_202310_1/xilinx_zcu102_base_dfx_202310_1.xpfm
xilinx_zcu102_base_202310_1	xd	Embedded Accel	xilinx.com	...zcu102_base_202310_1/xilinx_zcu102_base_202310_1.xpfm
xilinx_vck190_base_202310_1	xd	Embedded Accel	xilinx.com	...vck190_base_202310_1/xilinx_vck190_base_202310_1.xpfm
xilinx_zcu104_base_202310_1	xd	Embedded Accel	xilinx.com	...zcu104_base_202310_1/xilinx_zcu104_base_202310_1.xpfm
xilinx_vck190_base_dfx_202310_1	xd	Embedded Accel	xilinx.com	...ase_dfx_202310_1/xilinx_vck190_base_dfx_202310_1.xpfm
internal_platforms (50)				...S
tenzing	tenzing	Embedded Accel	xilinx.com	..._daily_latest/internal_platforms/tenzing/tenzing.xpfm
xilinx_u50_gen3x16_xdma_201920_3	u50	Data Center	xilinx	...6_xdma_201920_3/xilinx_u50_gen3x16_xdma_201920_3.xpfm

Code View and Smart Editor Features

The Source Code editor in the Vitis Unified IDE supports the following features:

- Syntax highlight for C, C++, Python, Makefile, CMakeList.txt, XRT INI and v++ CFG file
- Smart editor for XRT INI and v++ CFG files
- Hint for variable names, function names, etc
- Jump to the definition of variables or functions
- Peek the definition of variables or functions so that you need not leave the editing file

- Report the references of variables and function
- Hint for HLS component pragmas

You can open the outline window by selecting the button  on the right, or selecting the **Outline** view from the View menu. The Outline window can list the function names in your source code.

The smart editor for `xrt.ini` files and `v++` config files can switch views between GUI rendering and Text Editor view with the table button  and code button . The GUI rendering will only display the options that it can recognize for the context. For advanced use cases, user needs to edit the configuration file manually. The modifications in one view will update the other view instantly.

The changes will be saved automatically when Auto Save is enabled, or you can manually save changes in a file with keyboard shortcut `Ctrl + S`.

Preferences

The **File → Preferences** menu leads to a variety of user preferences that can be set and maintained to configure the Vitis IDE. The following are items that are associated with the Preferences menu and command.

- **Auto Save:** The Auto Save command on the File menu is enabled by default to allow the tool to save your changes to configuration files, source code, `CMakeLists.txt` files and more. Auto Save triggers and Auto Save Delay values can be defined from the Preferences page.
- **Color Themes:** Specify a Light Theme or a Dark Theme for the Vitis Unified IDE according to your preferences, or available lighting.
- **Open Settings (UI):** Displays the Preferences view, which has a number of settings as indicated by a Table-of-Contents on the left hand side, and a series of options available for you to configure the tool on the right-hand side. The Preferences page presents a wide array of options to configure your design environment, starting with general options like font styles and sizes to configure the environment to your tastes and needs.

Figure 73: Common Preferences

UserWorkspace

Commonly Used

> Vitis

> Embedded Component Paths

> Text Editor

> Workbench

Window

> Features

> Application

> Security

> Extensions

Commonly Used

Files: **Auto Save**

Controls **auto save** of editors that have unsaved changes.

afterDelay

Files: **Auto Save Delay**

Controls the delay in ms after which a dirty editor is saved automatically. Only applies when **Files: Auto Save** is set to **afterDelay**.

1000

Editor: **Font Size**

Controls the font size in pixels.

14

Editor: **Font Family**

Controls the font family.

'Droid Sans Mono', 'monospace', monospace

Editor: **Tab Size**

The number of spaces a tab is equal to. This setting is overridden based on the file contents when **Editor: Detect Indentation** is on.

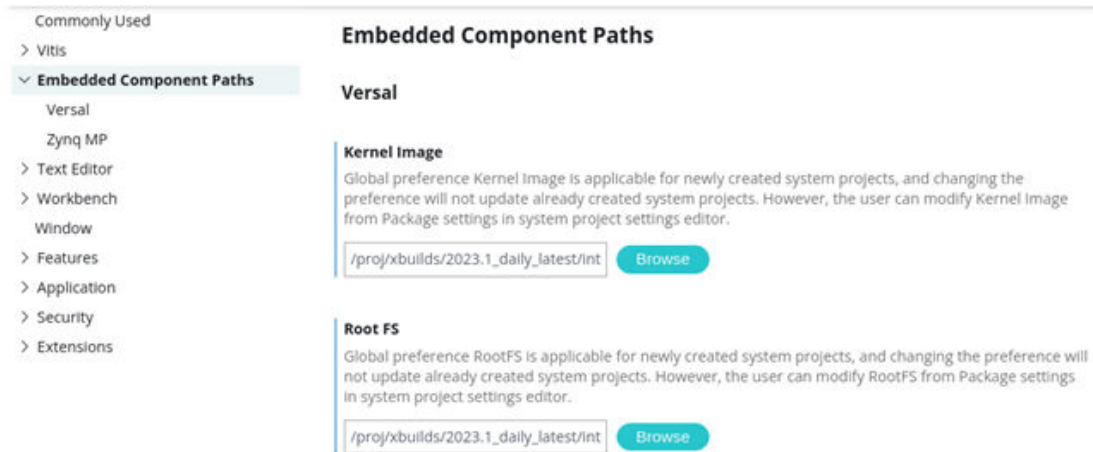
4

- **Open Keyboard Shortcuts:** Define new keyboard shortcuts for the various commands of the tool, as described in [Keyboard Shortcuts](#), [Command Palette](#), and [Quick Find](#).
- **File Icon Theme:** Specify a file icon theme to be used in your environment.

Note: You can press **Ctrl + +** to zoom into the display, or press **Ctrl - -** to zoom out the display.

Embedded Component Paths

Figure 74: Embedded Component Paths



The host component for embedded designs requires user to setup Linux component path like sysroot for cross compilation and linking. The application targeting embedded platforms requires Linux kernel and rootfs for packaging. User can set them at component settings and application settings. If the Embedded Component Paths in the Preferences have been set, they will be used as default path when creating host components and applications. This can be useful when a set of Linux components can be shared by multiple applications. For example, Common Image provided on [Vitis Embedded Platform download page](#) can be used for all pre-built base platforms.

Available through the **File → Preferences → Open Settings (UI)**, you can specify the Embedded Component Paths for the AMD Versal™ devices and AMD Zynq™ UltraScale+™ MPSoCs as needed for your design environment.

- Kernel Image and Root FS are applied to the System project under the Package Settings for embedded platforms.
- Sysroot is applied to Application components for embedded platforms and devices.

All common file paths specified on the Preferences page can be overridden during component or project creation.

Keyboard Shortcuts, Command Palette, and Quick Find

Keyboard Shortcuts

You can review and edit keyboard shortcuts from **File → Preferences → Open Keyboard Shortcuts**, or use shortcut key Alt + Ctrl + Comma. For example:

1. Use Alt + Ctrl + Comma shortcut to open the Keyboard Shortcuts window.

2. Type `build` in the search bar. Matching keyboard shortcuts are displayed.
3. The Build: Hardware key-binding is Alt+Shift+BH.
4. To edit the keyboard shortcut for build hardware, hover over the line of Build: Hardware, click the **Edit Keybinding** icon on the left, input your preferred key-binding in the pop-up window.

Command Palette

The command palette lets you quickly find and execute commands without remembering the keyboard shortcuts or menu location. For example, to run build hardware with the command palette, do the following:

1. Select your component or project in the workspace.
2. Type Ctrl + Shift + P to open the Command Palette menu.
3. Type in the keyword `build` or `run` or `debug` for example, and the Command Palette filters the list of command names until it includes only those commands that match your text entry.
4. Use the mouse, or the up or down arrow keys to scroll through the selection of commands and select the command you want to execute.
5. Hit enter to execute the selected command.

For example, select an HLS component in the Component Explorer and type Ctrl+Shift+P, and then in the Command Palette type C Syn and the tool shows HLS: C Synthesis as one of the options. Select the option and press Enter. The tool runs C synthesis on the selected HLS component.

Quick Find

You can use Ctrl + P to open the Quick Find box. Type in a file name and the tool will begin to find files that match your search string. Select a file and hit enter to open it.

With a file open, you can type @ in the Quick Find box to go to a function within the code, or type `:40` for example, to go to line 40 of the file. You can use this method to quickly find and jump to a function or line number.

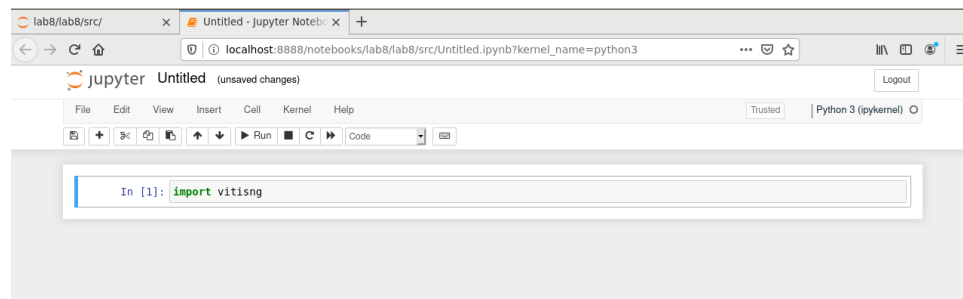
Jupyter Notebook Support

The Jupyter Notebook is a web application for creating and sharing computational documents. It offers a simple, streamlined, document-centric experience. Your code in Jupyter Notebook can produce rich, interactive output: HTML, images, videos, LaTeX, and custom MIME types. The new Vitis IDE integration with Jupyter Notebook allows you to access the Vitis environment remotely, iterate host code rapidly, tune acceleration kernel code and view the interactive output in one environment.

You can use Vitis server commands in this environment to manipulate components and System projects. To launch Jupyter Notebook server, run the following commands:

1. `vitis -j`
2. The Jupyter Notebook server will launch and it will open your default browser to open the web UI.
3. To pass additional options to Jupyter Notebook server, you can append these options. For example, `vitis -j --ip=<ip_address>` specifies the IP address the notebook server will listen on.

Figure 75: Jupyter Notebook



To close the Jupyter Notebook server, press `Ctrl + C` in the console where you launch it. It will stop the server and shut down all kernels. Pressing `Ctrl + C` twice would skip confirmation.

Parallel Compiling

The Vitis Unified IDE provides fast responses to actions. It employs non-blocking build commands to let you continue working and running multiple builds at the same time. If your system is sufficiently powerful, you can launch multiple actions. For example, you can launch build software emulation and hardware emulation at the same time. You can also launch debug software emulation when building hardware emulation is still running.

Note: If your server is not powerful enough, it's possible to see a lag and slow response from the new Vitis IDE if you launch multiple build or emulation jobs at the same time.

The application building job is also parallel if it refers to multiple components. When building the application, Vitis Unified IDE will launch building jobs for the referenced components in parallel.

The Parallel Build feature is disabled by default. You can enable it using the **File → Preferences → Open Settings (UI)** command and selecting the **Vitis → Build → Parallel Build → Disable** setting.

Running Scripts in Batch Mode

The new Vitis IDE provides a powerful command-line interface (CLI). It supports both batch mode and interactive mode. The CLI examples can be found at `<Vitis_Installation_Dir>/cli/examples` directory. The examples cover features for workspace management, application management, end-to-end workflow for data center and embedded use cases.

For example, create a vector addition System project using a script:

1. Review the script at `$XILINX_VITIS/cli/examples/dc_use_case_1.py`.
2. Launch the script using the following command:

```
vitis --source $XILINX_VITIS/cli/examples/dc_use_case_1.py
```



TIP: Some scripts require specific environment variables to be set before running it. For example, edge use case requires kernel, rootfs and sysroot information from environment variables. You are encouraged to review comments in the example scripts before running them.

For more information about Vitis CLI, see [Vitis Interactive Python Shell](#) and [Library Function References](#).

Recreating a Workspace with Scripts

The Vitis Unified IDE keeps a log of the actions taken during the current session of the IDE. These actions let you recreate the workspace, components, and projects if necessary. The actions are recorded to the `builder.py` python script found in `<workspace>/logs/` folder.

Note: The `builder.py` script records the actions taken in the Vitis Unified IDE for the duration of time the IDE is opened. When you close the IDE and reopen it, a new `builder.py` script is recorded. A limited number of older scripts are saved, but can be overwritten.

The `builder.py` script can be useful in the following ways:

- Automate the component and application creation and configuration workflow in script mode so that the flow can be reproduced. This would require saving the `builder.py` created when the initial components and projects are created, and save it for use later.
- To check in for source control only the source files and the workspace creation scripts, rather than the workspace metadata like `vitis-comp.json` and `vitis-sys.json`.

Note: Checking in all workspace metadata is recommended.

Note: If you want to move the workspace to another location, you can move the workspace directory. The workspace is self-contained. Rebuilding the workspace is unnecessary in this case.

To use the `builder.py` script, edit the script as needed to modify paths and such, and then source the script to recreate the workspace:

```
vitis -s ./builder.py
```

Scrollbar control tips

Two methods can be used to control the scrollbar.

1. Leverage the mouse button
 - Move the mouse to the top of the scroll bar, click the left button of your mouse to select the scroll bar, then press the left mouse button to drag the mouse. This time the scroll bar will move up and down or left and right.
2. Leverage the mouse wheel
 - Move the mouse to the top of the scroll bar, click the left button to select the scroll bar, then turn the mouse wheel. This time the scroll bar will start to move up and down. If it is a horizontal scroll bar, you need to hold down the SHIFT key first, and then turn the wheel.

Creating an AI Engine Component

As described in [Introduction to Vitis Tools for Embedded System Designers](#), Adaptive data flow (ADF) graph applications targeting AI Engines can be developed as AI Engine components. These components can be built, simulated, analyzed, and debugged as a standalone component, and then added to a System project as part of an embedded system design, or for application acceleration in a data center. The creation of an AI Engine component is described in the *AI Engine Tools and Flows User Guide* ([UG1076](#)). Refer to that document for more information on building and analyzing the AI Engine graph application.

The use of an AI Engine component in a larger system design can be accomplished from the command line as described in this document under [Section IV: Building and Running the System](#), or in the Vitis unified IDE as described under [Creating a System Project for Heterogeneous Computing](#).

Creating an HLS Component

In an HLS component, the tool synthesizes a C or C++ function into RTL code for implementation in the programmable logic (PL) region of an AMD Versal™ adaptive SoC, AMD Zynq™ MPSoC, or AMD FPGA device. HLS components can be built, simulated, analyzed, and debugged as a standalone component in a bottom-up design flow. The creation of an HLS component is described in the Vitis HLS User Guide ([UG1399](#)). Refer to that document for more information on building and analyzing an HLS component.

The HLS component can be added to a System project as part of an embedded system design, or for application acceleration in a data center. The use of an HLS component in a larger system design is described in this document under [Section IV: Building and Running the System](#), or in the Vitis unified IDE as described under [Creating a System Project for Heterogeneous Computing](#). Refer to that content for more information on using HLS components in a system design.

Creating an Application Component

An Application component can be created for the development of embedded software applications targeted towards embedded processors such as AMD Versal™ adaptive SoC, and AMD Zynq™ MPSoC, as described in *Vitis Embedded Software Development Flow Documentation* (UG1400), or as described in [Introduction to Vitis Tools for Embedded System Designers](#). Application components can also be created to run on x86 processors for use with accelerated systems running on Alveo Data Center acceleration cards as described in [Introduction to Data Center Acceleration for Software Programmers](#). The following steps describe creating an Application component.

1. With the AMD Vitis™ IDE opened, from the main menu select **File → New Component → Application**.



TIP: You can also select the **Create Application Component** command from the Welcome page.

This opens the Name and Location page of the Create Host Component wizard.

2. Specify the Component Name and Component Location and select **Next**. This opens the Platform page.
3. Select a Platform from the list and click **Next**.

The choice of a platform in the Application component is an important one. First, you can choose between a fixed platform to support the *Vitis Embedded Software Development Flow Documentation* (UG1400), or an extensible platform to support the Vitis development flow for application acceleration and heterogeneous system design as described in [Introduction to Data Center Acceleration for Software Programmers](#). Within the Vitis development flow, you can also choose between an embedded processor based design, or a Data Center acceleration card such as an AMD Alveo™ card, or a bare-metal system as described in [Creating a Bare-metal System](#).

There are a number of base platforms installed with the tools, and platforms that you can install and configure as described in [Managing Platform Repositories](#).

4. Depending on the Platform selected, the Summary page is displayed for Data Center platforms, and for Embedded Platforms the Domain page is opened to let you select the processor domain for the Application component and then the Summary page is displayed. Select the domain as needed, review the summary page, and click Finish to create the Application component.

After the Application component is created the `vitis-comp.json` file is opened in the central editor window. You will need to import source files and edit the `UserConfig.make` to specify needed include files or libraries.

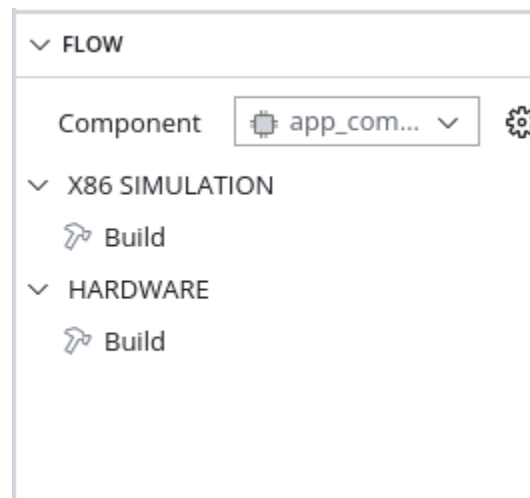
1. Right-click the Sources folder in the expanded view of the Application component in the Vitis Component Explorer.
2. Select **Import** → **Files** or **Import** → **Folders** and import one or more files into the component as needed.
3. In the Compiler Settings of the `vitis-comp.json` select the `UserConfig.make` to open in an editor. Scroll down to Directories, and select **Add Item** under Include Paths.

Edit these settings as needed for your source code.

Building the Application Component

After creating the Application component, and configuring the `CMakeLists.txt` file, you can select the **Component** in the Flow Navigator to make it the active component in the tool. Then select the **Build** command under either the X86 Simulation flow or the Hardware flow as shown in the following image.

Figure 76: Building the Application Component



There are two build commands because the software emulation build uses the PS on X86 approach for the embedded application that requires all the elements of the system to be compiled for x86 targets. This means building the application for use on an X86 processor.



TIP: The Application component assigned to an Alveo Data Center acceleration platform will only have a single build option as it is only built for execution on an X86 processor.

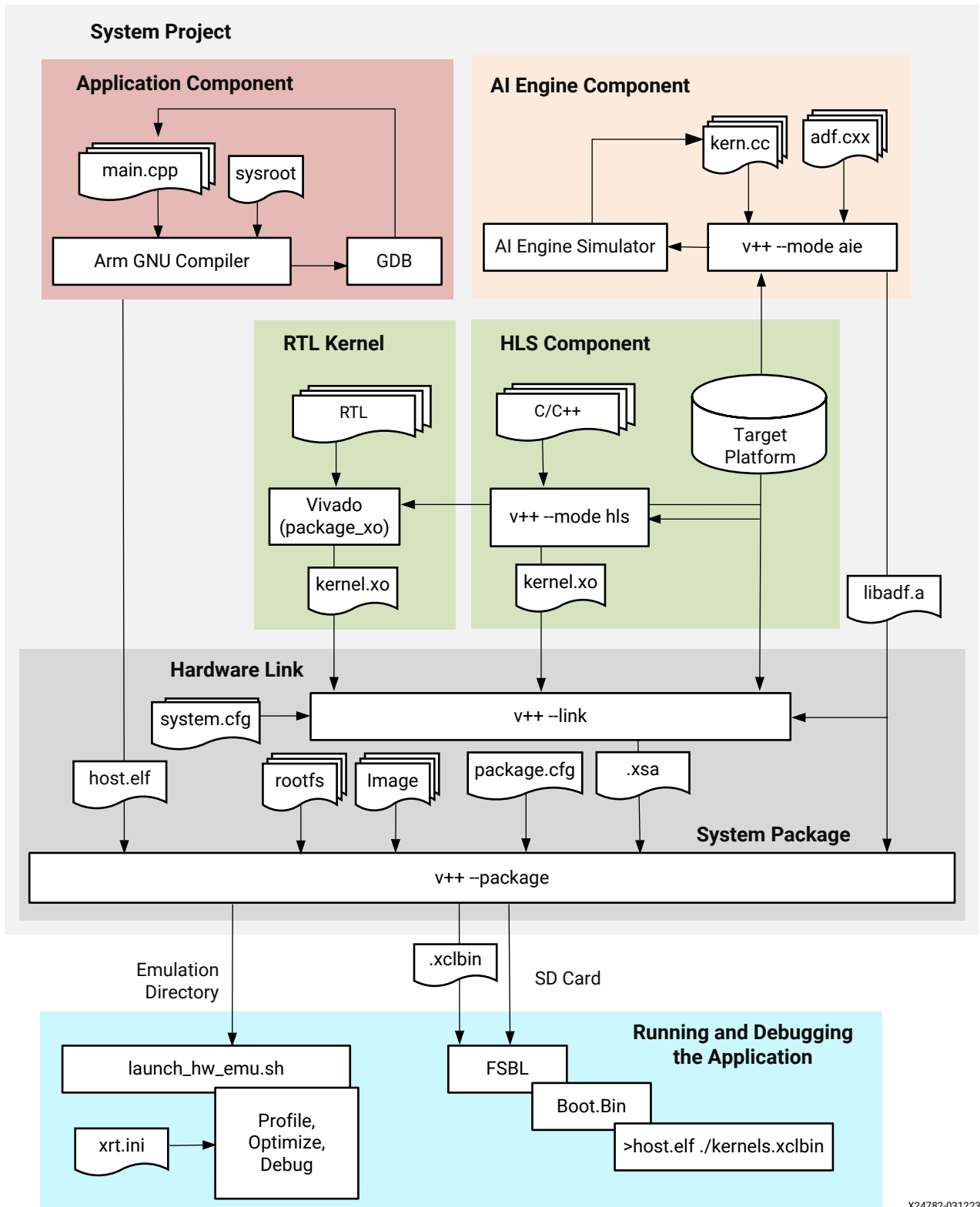
The hardware build of the Application component is compiled using an Arm cross-compiler for use on the embedded processor core. For the hardware emulation build (`-t=hw_emu`) the application is run under the QEMU environment to model the behavior of the Arm processor. For the hardware target (`-t=hw`) the Application component is run on the actual embedded processor core.

The Application component of the Flow Navigator reflects build commands, but does not support Run or Debug commands. This is because the Application component can only be run in the context of a System project. Refer to [Creating a System Project for Heterogeneous Computing](#) for more information.

Creating a System Project for Heterogeneous Computing

In the unified AMD Vitis™ IDE the System project simplifies hardware design by integrating the three domains of an AMD Versal™ device: the AI Engine array, the programmable logic (PL) region, and the processing system (PS). The System integrates the AI Engine component, the HLS component for PL modules, and the Application component for the processor into a single heterogeneous system, as shown in the following figure.

Figure 77: Vitis Tools Flow

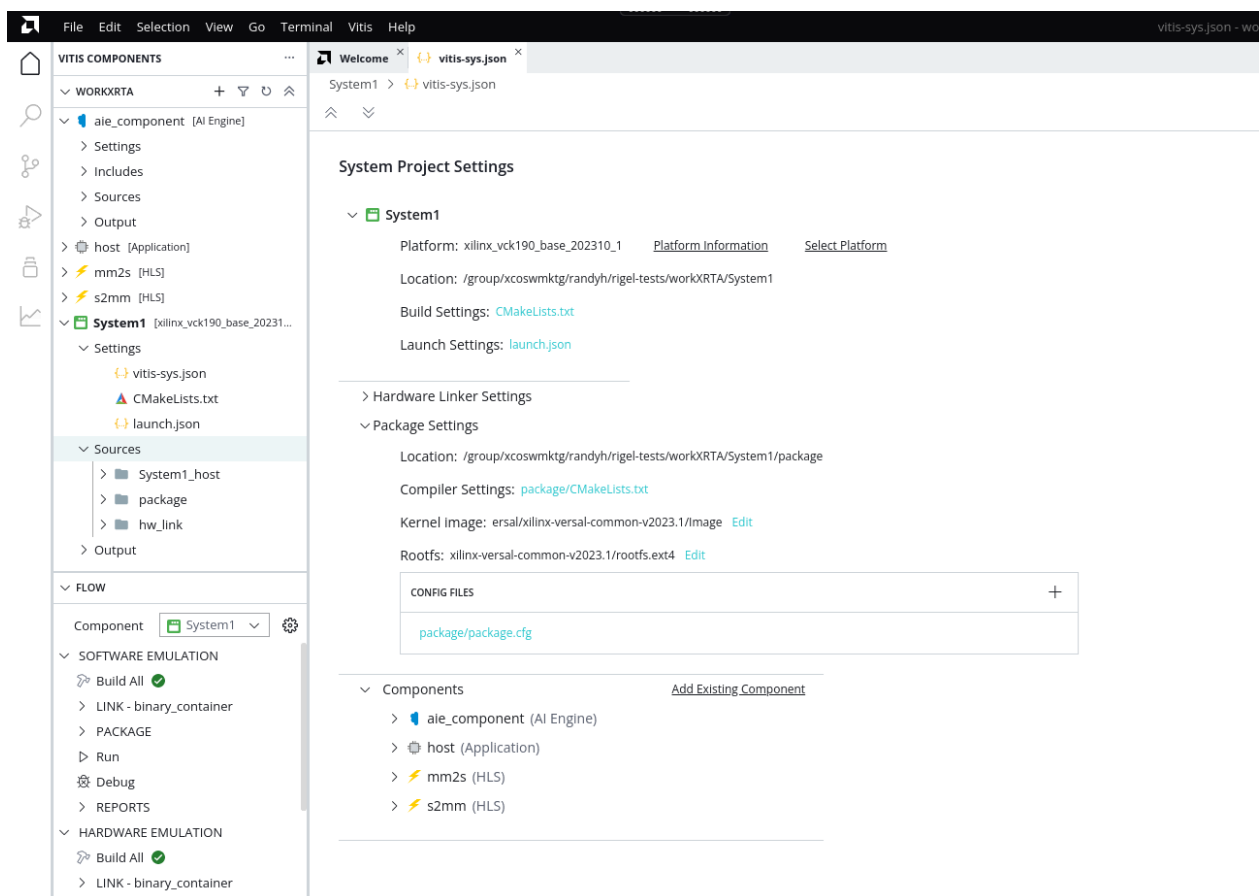


X24782-031223

System Project Structure

The Vitis IDE System project contains components: platforms derived from Platform components, HLS components, AI Engine components, Application components. The System project refers to the components, defines and stores the linking configuration required to build the system, and defines and stores the packaging configuration required to package the system. In the figure below you can see the elements of the System project under the System Project Settings.

Figure 78: Project Structure



- **Application Component:** Software applications that run on x86 CPU for data center acceleration, or on Arm® processors for embedded systems. The host program interacts with the AI Engine kernels and kernels in the PL region.
- **HLS Component:** Programmable logic (PL) kernels that run on FPGA fabric written with C/C++ code and synthesized into RTL code. HLS components are compiled for implementation in the PL region of the target platform using the `v++ --compile --mode hls` command.

- **AI Engine Component:** Asynchronous dataflow graphs and kernels that run on AI Engines of specific Versal devices. AI Engine components are compiled into libadf.a files using the `v++ --compile --mode aie` command.
- **Platform Component:** Defines a custom platform for use in embedded system designs. The acceleration flow also supports the inclusion of predefined platforms installed with the system, or downloaded separately. Custom platforms can be created as described in [Creating a Platform Component](#).
- **HW Linker Settings:** The hardware link process connects the PL kernels and/or AI Engine graph with the platform and builds the hardware system device binary. The `hw_link/binary_container.cfg` file defines the configuration settings for the linking process.
- **Package Settings:** The Package settings define the files required to package the finished system for use. The system package process uses the `v++ --package` command to gather the required files to configure and boot the system, to load and run the application, including the AI Engine graph and PL kernels. This builds the necessary package to run emulation and debug, or run your application on hardware.

Files for project settings are stored in the top directory.

- **vitis-sys.json and vitis-comp.json:** This is the top level settings file for applications and components. It defines the components of the application, their source code and the compiler settings file.
- **CMakeList.txt:** The Vitis IDE uses CMake to manage the project. The top level `CMakeList.txt` defines the global build parameters and adds the component level of the build settings `CmakeList.txt` as sub-modules. You can modify the settings in `CMakeList.txt` by editing content defined between `START OF USER SETTINGS` `START` and `END OF USER SETTINGS`
- **Component/compile_commands.json:** The auto-generated file to enable IntelliSense support using Clang.
- **Component/config.cfg:** The configuration file for PL kernel compilation. It is passed to `v++` compiler.

Build output for each target is stored in its own folder under the `./build` directory. For example,

- `project/build/sw_emu`
- `aie_component/build/x86sim`

The runtime configuration file (`xrt.ini`) is stored in `<project>/<application_component>/runtime/<target>_xrt.ini`. Where `<target>` is software emulation, hardware emulation, or hardware.

Creating a System Project

There are a number of ways to create a System project in the new Vitis IDE. The following steps describe a recommended approach.

1. With the Vitis IDE opened, from the main menu select **File → New Component → System Project**.

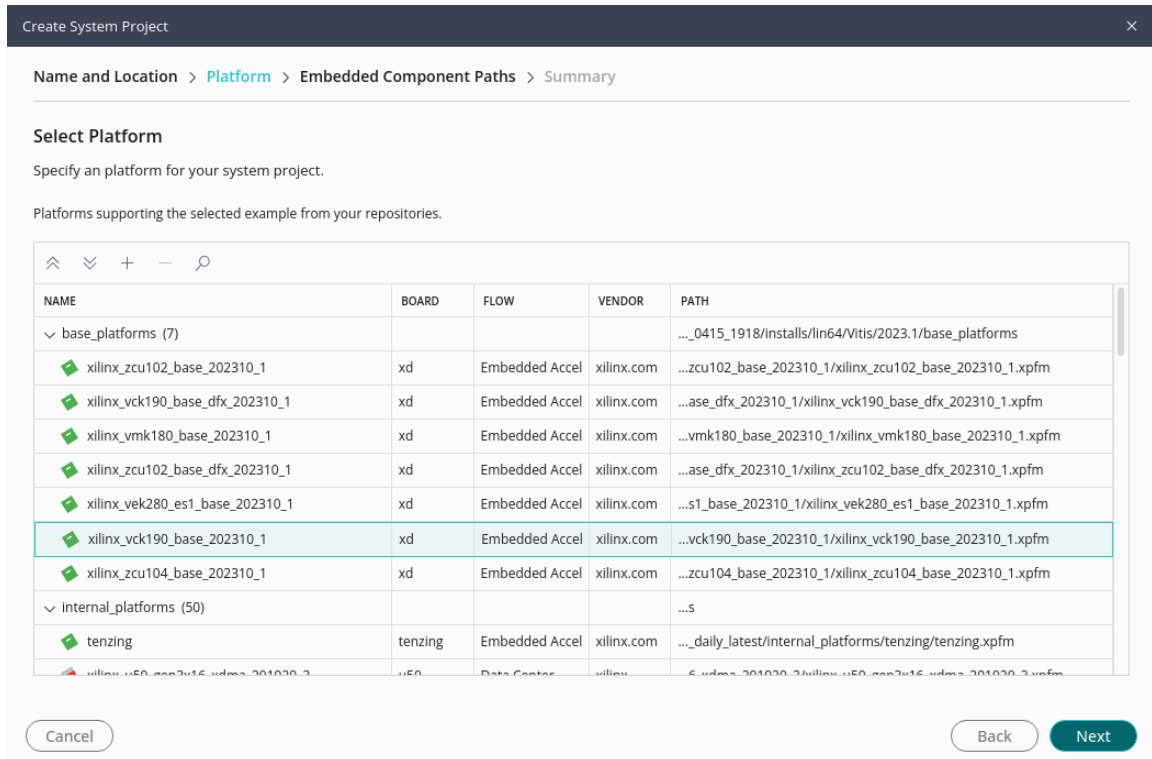


TIP: You can also select the **Create System Project** command from the Welcome page.

This opens the Name and Location page of the Create System Project wizard as shown below.

The screenshot shows the 'Create System Project' wizard with the 'Name and Location' tab selected. The breadcrumb navigation at the top reads 'Name and Location > Platform > Summary'. Below the title, the instruction says 'Choose a name for your system project and specify a directory where system data files will be stored'. There are two input fields: 'System project name' with the text 'system_project' and 'System project location' with a dropdown menu showing '/group/xcoswmktg/andyh/rigel-tests/workXRTA'. A 'Browse' button is next to the location field. Below these fields, it states 'System project will be created at /group/xcoswmktg/andyh/rigel-tests/workXRTA/system_project'. At the bottom, there are 'Cancel' and 'Next' buttons.

2. Enter a Component name and Component location and select **Next**. This opens the Platform page of the wizard as shown below.



- Specify a platform to use for the System project and select **Next**. If you select an embedded platform, this opens the Embedded Components Path page of the wizard as shown below. If you select an AMD Alveo™ Data Center acceleration card, then the Summary view is displayed.

Create System Project

Name and Location > Platform > Embedded Component Paths > Summary

Embedded Component Paths

Specify embedded component paths. If the paths are empty, tool will take the values from the preferences if available.

Kernel Image [Browse](#)

Root FS [Browse](#)

Sysroot [Browse](#)

☐ Update Workspace Preference

Cancel Back Next

The Embedded Component Paths specify the Kernel Image file, the Root FS, and the Sysroot required by the embedded system for compiling embedded applications, and for booting and configuring the embedded systems. These paths can be populated by the Preferences view as described in [Preferences](#), or can be manually entered when the System project is created.

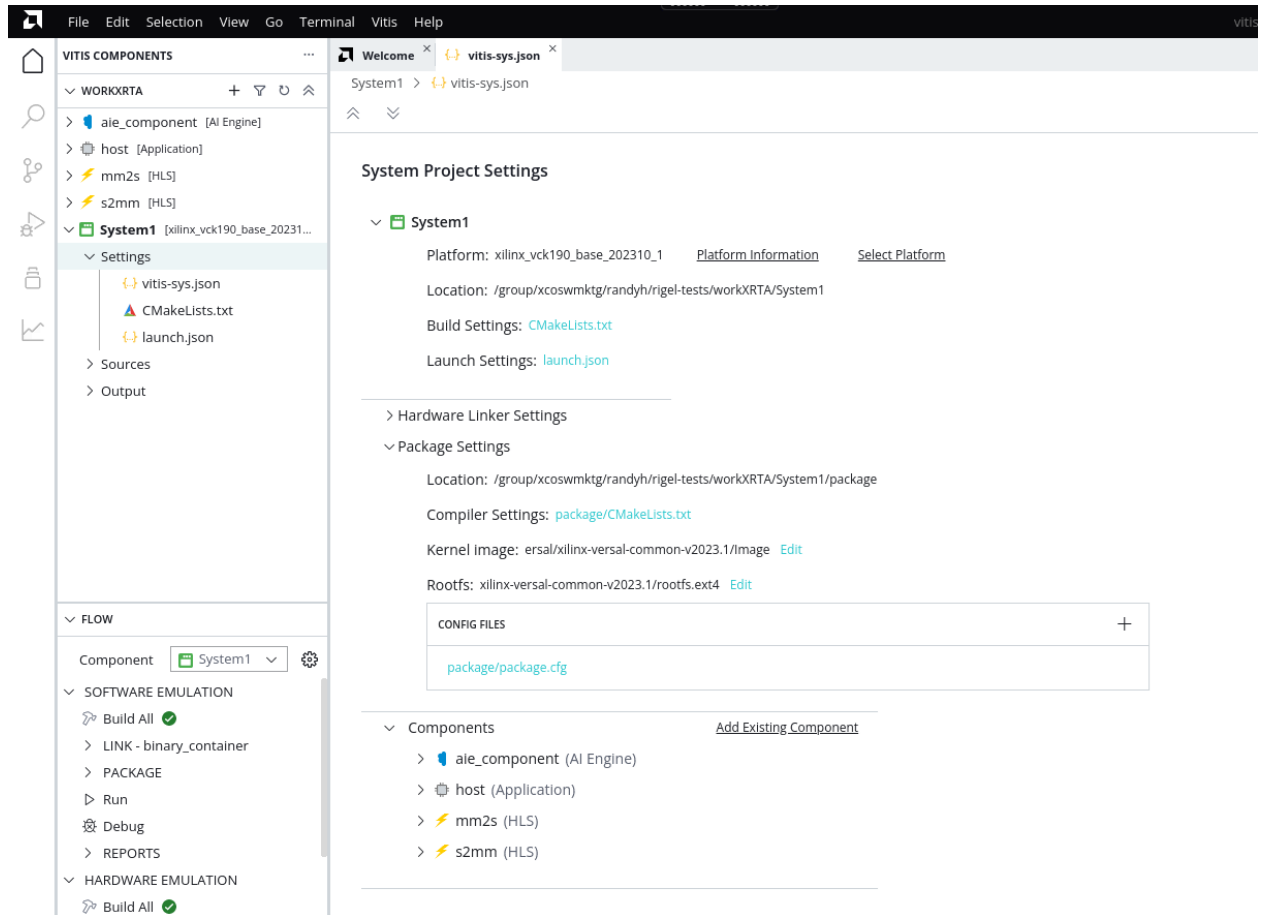


TIP: The Update Workspace Preference check box lets you specify that any manually entered paths should also override or update the Preferences view.

- Click **Next** to display the Summary page.

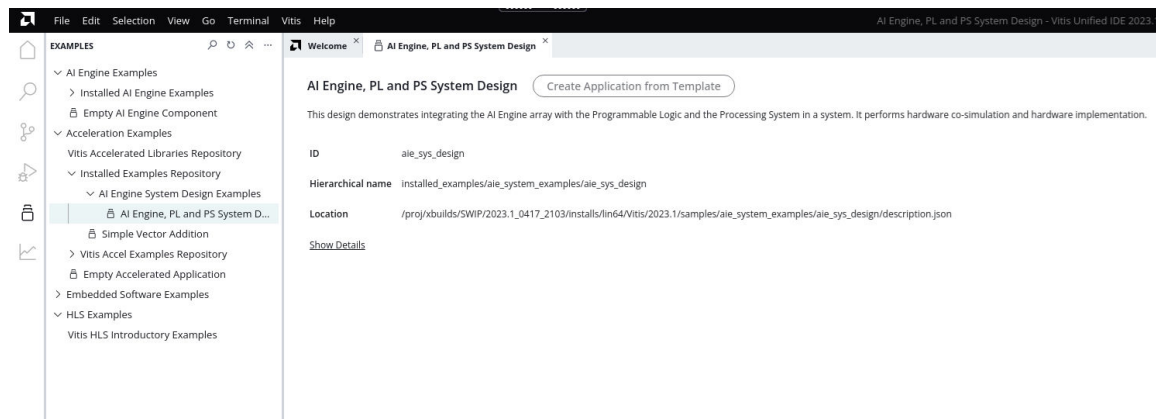
The Summary page displays the choices you have made on the prior pages. Review the summary and select **Finish** to create the System project, or select **Back** to return to earlier pages and change your selections.

When the System project is created the `vitis-sys.json` file for the System is opened in the central editor window, and the System project becomes active in the Flow Navigator. You can see from the red text and the red dot next to the System in the Component Explorer that it is missing key components. Provide the missing components in the next steps.



Creating a System Project from Examples

You can create a System project from the Examples installed with the Vitis IDE.



1. In the Welcome page, click **Examples**. The Examples panel appears as shown above.

2. Select the System project or Component you want to create. There are AI Engine Components, and AI Engine System Projects combining AI Engine components with HLS components, and an Application component to run on the processor. There is a Simple Vector Addition example, and you can download additional example repositories from GitHub, such as the Vitis Accel Examples.
3. When you select an Example it opens the Application Template in the central window. Select **Create Application from Template**.
4. This opens the Create System Project wizard for you to specify the System name and location, and platform. The platforms are restricted by the example you have selected. Click **Finish** from the Summary page to create the example System project.

After the System is created you can configure the project as described in the following sections.

Managing Components in System Projects

After creating the System project you can add components to the design. As previously discussed, a System project can contain multiple components.

You can add components to the System Project Settings in the open `vitis-sys.json` by selecting the **Add Existing Component** command under the Component heading; or you can add them directly to the Binary Container under the Hardware Linker Settings heading.

1. Add the components by selecting the **Add Existing Component** command:
 - Select the type of component to add in the pop-up window: HLS, AI Engine, Host.
 - After you select the type of component to add the tool will present a list of all the components of the selected type in the current workspace. Choose the component to add.
 - If there is only a single binary container the tool will automatically assign it. For DFX platforms that support multiple device binaries you can select the target binary container. If the Application does not have any binary container select the **Container Name** to create a new binary container.
2. Add components directly to the Binary Container:

Note: The Host component is the only type that cannot be added directly to the binary container. You must add it under the Component heading.

- You can select the **Add Pre-Built Binary** command in the Binary Containers configuration box. This command lets you add the `kernel.xo` file or `libadf.a` file from a previously compiled HLS component, RTL kernel, or AI Engine component. The pre-built binary can be added from folders outside the current workspace. Selecting this command opens a file browser letting you navigate to and select the file of interest.



TIP: This method can be used to add a `libadf.a` file that was generated by Model Composer into the System project.

- You can also select to add the component type in the Binary Containers configuration box. You can add Kernels or AI Engine Graphs. This opens an Add Components dialog box that lists all the components of the selected type in the current workspace.



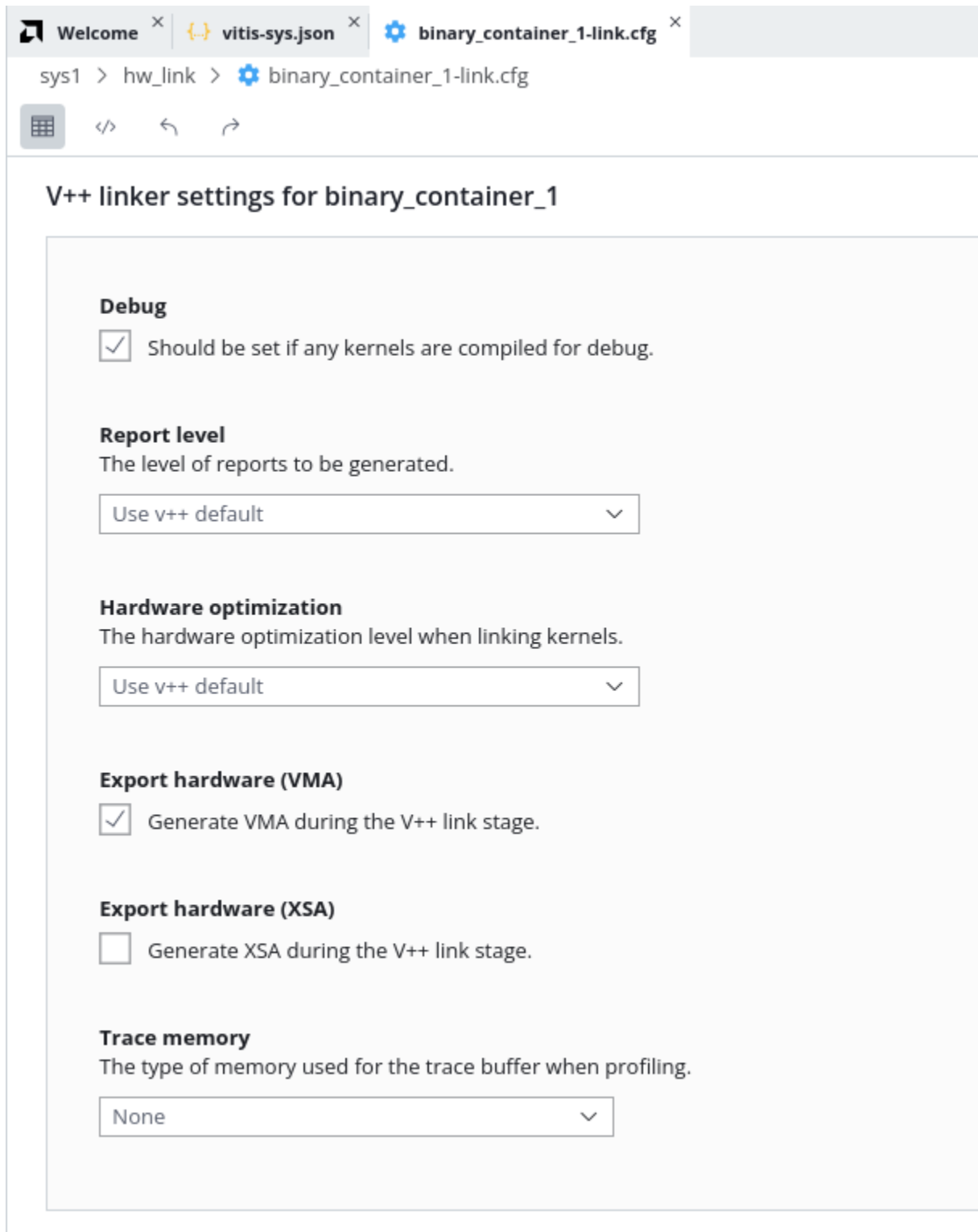
TIP: To remove a component from the System project, you can either select to delete it from the Binary Containers or from the Components. Either will result in the removal of the component from the project.

Defining the HW_Link System Configuration

V++ Linker Settings

The Hardware Link (hw_link) configuration file contains information used by the Vitis compiler (v++) for linking the system. To open the config file from the `vitis-sys.json` file for the System project click the [hw_link/binary_container-link.cfg](#) hyperlink. This will open the Config File Editor in the central editor window as displayed below.

Figure 79: Configuring the Hardware Linker Settings



Note: All of the options found in the `binary_container-link.cfg` are v++ command line options (see [v++ Command](#)).

The V++ linker settings provides the following options:

- **Debug:** Enable debug file generation. When not enabled the HLS component and the system design can be optimized but will not support debug.

Note: This specifies the `-g` option for the `v++ --link` command as described in [v++ General Options](#). To enable debug for the HLS component you must specify `syn.debug.enable` in the HLS component config file (`hls_config.cfg`).

- **Report Level:** Specify the level of detail; included in reports generated by the `v++ --link` command.
- **Hardware Optimization:** This option specifies the optimization level of the AMD Vivado™ implementation results.
- **Export Archive:** Indicates the export of the `.vma` file for use in the Vivado Design Suite as described in [Vitis Export to Vivado Flow](#).

Note: This feature requires the use of a Versal platform or XSA designed with a block design container. If your system project does not meet this requirement the feature will return an error.

- **Export Hardware:** Specifies that an XSA file should be generated from the linked design produced by the `v++` command. This option relates to `param=compiler.addOutputTypes=hw_export` as described at [--advanced Options](#).
- **Trace Memory:** When building the hardware target for the System project, this option lets you specify the type and amount of memory to use for capturing trace data. This option relates to the `--profile.trace_memory` command as described at [--profile Options](#).

Kernel Data

For each PL kernel in the System project there are additional settings available under the Kernel Data heading.

Figure 80: Configuring Kernel Data

Kernel Data								
NAME	COMPUTE UNITS	MEMORY	SLR	PROTOCOL CHECKER	CHIPSCOPE DEBUG	PROFILING DATA TRANSFER	EXECUTE PROFILING	STALL PROFILING
binary_container (2)								
mm2s (1)	1	auto	auto	☐	☐	all	☒	☐
mm2s (3)		auto	Auto	☐	☐	all	☒	☐
mem		Auto		☐	☐	Counters + Trace		
s					☐			
size					☐			
s2mm (1)	1	auto	auto	☐	☐	all	☒	☐
s2mm (3)		auto	Auto	☐	☐	all	☒	☐
mem		Auto		☐	☐	Counters + Trace		
s					☐			
size					☐			

The Kernel Data section refers to config commands that specifically apply to the PL kernels generated from the HLS component, the ports and interfaces, and the debug and profile options available.

The specific options that can be set include:

- **CU Name:** Lets you define the naming sequence when multiple compute units are generated from a PL kernel as described under the [--connectivity Options](#).
- **Compute Units:** Specifies the number of compute units to generate from a PL kernel. Also related to `--connectivity.nk`
- **Memory:** Use this option to specify the connection of kernel ports to system ports within the platform. A primary use case for this option is to connect kernel arguments to specific memory resources as described in [Mapping Kernel Ports to Memory](#).
- **SLR Assignment:** is described in [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#)
- **Protocol Checker:** Described under [--debug Options](#).
- **Chipscope Debug:** Also described under [--debug Options](#).
- **Data Transfer:** Enables monitoring of data ports through special monitor IP that are added into the design for profiling. This option relates to the [--profile.data](#) command
- **Execute Profiling:** This option records the execution times of the kernel and provides minimum port data collection during the system run. This option relates to the `--profile.exec` command.
- **Stall Profiling:** This option relates to the `--profile.stall` command. It adds stall monitoring logic to the device binary (`.xclbin`) which requires the addition of stall ports on the kernel interface. Therefore the `stall` option must be specified during both compilation and linking.



IMPORTANT! The `--profile.stall` command must be specified during compilation of the HLS component using `syn.rtl.kernel_profile=1` in the HLS component config file (`hls_config.cfg`) to be implemented during `v++` linking.

Adding Commands Using the Text Editor

After selecting config items using the Config File editor GUI features, you should take a look at the Text Editor view. You can toggle between the GUI table view and the Text Editor view by using the commands in the toolbar.



Examine the configuration file commands that have been added by the GUI, and be aware of any configuration commands to add to your design that are not presented by the GUI. The Text Editor offers the ability to add configuration commands, whether they are in the GUI or not.

An example of this applies to the use of the AI Engine component in a System project. As described in [Linking the System](#), the AI Engine component must be connected into the platform or to PL kernels using the `--connectivity.sc` command. You can also map the system port to a memory using the `--connectivity.sp` command to specify a connection of a GMIO to a specific memory.



TIP: Keep in mind that the GUI does not provide access to all the command options for linking the system. Think about which options your design will need to use. The Text Editor provides a good way to enter them.

Adding AI Engine Connectivity

1. As explained in [Linking the System](#), for AI Engine graph applications the Vitis compiler needs some instruction on how to make connections between the graph and PL kernels. The number of kernels to be instantiated is already configured in the IDE. However the definition of the connections between the PL kernels and the graph must be specified in the config file as shown below:

```
[connectivity]
stream_connect=mm2s_1.s:ai_engine_0.DataIn1
stream_connect=ai_engine_0.clip_in:polar_clip_1.in_sample
stream_connect=polar_clip_1.out_sample:ai_engine_0.clip_out
stream_connect=ai_engine_0.DataOut1:s2mm_1.s
```

The connection scheme must be defined in the Text Editor, or manually added to the config file for the system project. Connections can be defined as the streaming output of one kernel connecting to the streaming input of a second kernel, or to a streaming input port on an IP implemented in the target platform.

The `system.cfg` file has some differences between the Vitis IDE and the command-line flow. The primary difference is that the IDE provides the connectivity `nk` options of the config file, instantiating a specified number of compute units (CUs) per kernels, as specified in the build settings. In addition, the IDE uses a naming convention for CUs in the form of `<kernel>_#`, where `#` indicates the CU instance. In the case where there is only one CU instance, it has the `_1` extension. This means that for the Vitis IDE, the `system.cfg` should not specify the `nk` option, and the `sc` option should use the CU instance name from the IDE.

Defining the Package Configuration

The Package Settings configuration file contains information used by the Vitis compiler (v++) for packaging the system. To open the config file from the `vitis-sys.json` file for the System project click the **package/package.cfg** hyperlink. This will open the Config File Editor in the central editor window as displayed below.

Figure 81: Configuring Package Settings

System1 > package > package.cfg

Search Settings

All Settings Settings with Values

General

Output directory
Specify absolute or relative path to output directory of package flow.
./package [Browse](#)

Domain name
Specify a domain name.

Boot mode
Boot mode used for running design on emulator or hardware.
sd

Package directories to sd-card
List of directory paths to package into the sd_card directory/image. Valid if boot mode is sd.
 [Browse](#)

Package files to sd-card
ELF or other data file to package into the sd_card directory/image. Valid if boot mode is sd.
 [Browse](#)

Baremetal Elf
<ps.elf,core> For example a72_0.elf,a72-0. Valid for Embedded (Edge) platform.

The packaging process is described in [Building and Packaging the System](#). The specific package commands represented below are documented in [--package Options](#).

The Package config file includes the following sections:

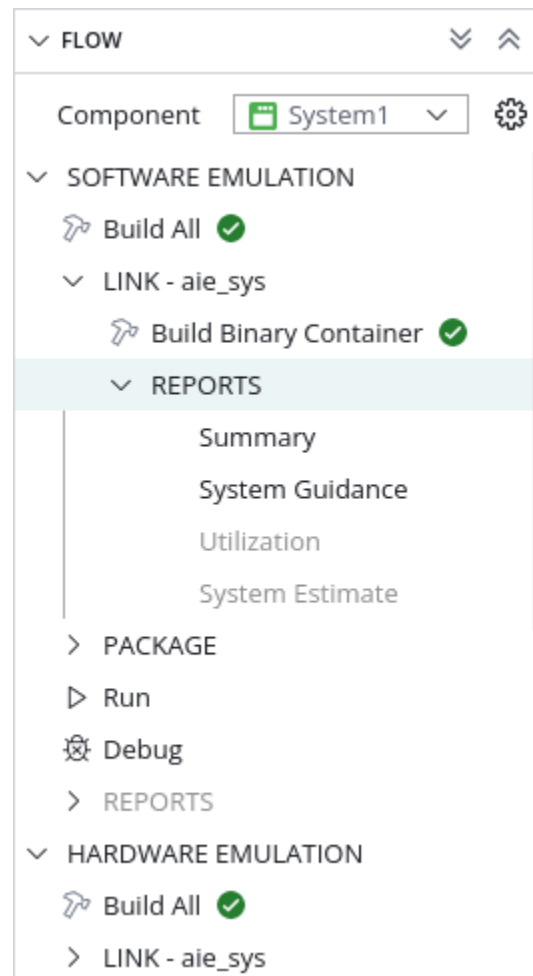
- **General:** Applies to the construction of the SD card or boot files.
 - **Output Directory:** Specifies the absolute or relative path to the output directory of the `--package` command. The default output directory for the Vitis IDE is `./build/<target>/package`.
 - **Domain Name:** Specifies a domain name from the current platform. If this option is not specified, then the tool uses the default domain for the platform.
 - **Boot Mode:** Specifies the boot mode used for booting the device and running the application in emulation or on hardware. For embedded platforms, the default boot mode is SD card.
 - **Package directories to sd-card:** Specifies a directory to package as a sub-folder of the SD card directory/image.
 - **Package files to sd-card:** Specifies an `elf`, `xclbin`, or other files to package into the SD card directory/image.
 - **Baremetal Elf:** This option relates to `--package.ps_elf` and is used when a baremetal ELF file is running on a device processor core.
 - **PS Debug Port:** Specifies the TCP port where emulator listens for incoming connections from the debugger to debug PS cores.
- **Linux Boot:** Specifies options to configure the device boot process for Linux.
 - **Arm Trusted firmware elf:** This option is related to `--package.bl31_elf`, and identifies a trusted firmware ELF that executes on the A72#0 core.
 - **DTB file:** Specifies the absolute or relative path to device tree binary (DTB) used for loading Linux on the APU.
 - **U-Boot ELF:** Specifies a path to U-Boot ELF file which overrides the platform U-Boot.
 - **Image Format:** Specifies `<ext4 | fat32>` output image file format used on the SD card. For embedded platforms with a Linux domain, the default image format is `ext4`. For all others, the image format is `fat32`.
 - **Kernel Image:** Specifies the absolute or relative path to a Linux kernel image file.
 - **Root File System:** Specifies the absolute or relative path to a processed Linux root file system file.
 - **Do Not Create Image:** Bypass SD card image creation when `--package.boot_mode sd` is used.
- **AI Engine:** Defines features related to the AI Engine array.

- **Debug port:** Specifies a TCP port where emulator listens for incoming connections from the debugger to debug Versal AI Engine cores. The default port value is 10100.
- **Do not enable cores:** Specifies that the Versal AI Engine cores are enabled by an embedded processor (PS) application rather than by default at boot time.
- **Enable debug:** When enabled, the tool generates CDO commands to stop the AI Engine cores during PDI load.

Building the System Project

After creating the System project you can select it in the Component Explorer, or choose it from Component drop-down in the Flow Navigator to make it the active component in the tool. Then select **Build All** under the Software Emulation, Hardware Emulation, or Hardware headings as shown in the following image.

Figure 82: Building the System Project



The Build All option combines the Build Binary Container command to run the `v++ --link` command as described in [Linking the System](#), and the Build Package command to run the `v++ --package` command as described in [Packaging the System](#), into a single process step. Alternatively, you can run these as separate process steps. You can expand the Link section and select **Build Binary Container** to build only the `.xclbin` or the `.xsa`. Then also expand the Package section and select **Build Package** command to complete the build process.



TIP: Both the Link and Package commands have their own configuration files that can be managed from the System project `vitis-sys.json` file as previously described.

The System project supports three build targets.

1. Software Emulation is a build for simulation of the C/C++ code of the application and components in the system.
2. Hardware Emulation is a build for simulation of the hardware design in the Vivado logic simulator
3. Hardware is a build that is intended to be run on the physical device.

The builds take progressively longer to complete as you move from 1 to 3. The compilation of a C/C++ model is much quicker than the RTL code implemented into hardware through place and route. So you can use the software emulation to quickly iterate on the logic of your design, then move to hardware emulation to get a cycle accurate simulation of the hardware, finally build and run your hardware for the real performance test.

A transcript of the build process is displayed in the Output console, titled with the System project name and the build target, for example `System1:sw_emu`. When the build is complete you can review the transcript, or the `hw_linker.log` file written to the `<system>/build/<target>/log` folder. You should see *Build Finished successfully* in the transcript, or you could see errors reported instead. The Flow Navigator displays a green circle with a check mark in it, or a red circle with an x depending on the results of the build. If the build completes with errors, review the transcript or the log file to determine the cause and rerun the build as needed.

You can select and expand the folders of the build directories. You can see the output files of the Vitis compiler package process (`v++ --package`) in the output hierarchy. The build process generates the emulation data and boot files needed for the system, and writes them to the `sd_card` folder.

Note: Two folders are created under the Hardware folder, `package` and `package_no_aie_debug`. The `sd_card.img` file within the `package` folder is for hardware debug purposes whereas the `sd_card.img` file in the `package_no_aie_debug` folder is for regular application execution.

After a successful build the Flow Navigator will display a series of reports generated during the compilation process. You can access the reports by expanding Reports under the Link and Package sections. The available reports will vary depending on the build target. You can select any of the available reports to view, or switch to the Analysis view to complete a review of the reports. Refer to [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#) for details.

After the build completes successfully you can choose to **Run** or **Debug** the System project as described in the next sections.

Software Emulation Build

The Vitis Unified IDE uses x86 simulation for software emulation of both Data Center accelerator cards, and Embedded Processor platforms. For embedded systems, simulating on x86 processors is known as the PS on X86 method (see [Compiling an Embedded Application for PS on x86](#)). This requires all the elements of the system to be compiled for x86 targets.

This means the `libadf.a` file that is included in the `v++ --link` command line must be compiled for the `x86sim` target as explained in *Simulating an AI Engine Graph Application* of described in the *AI Engine Tools and Flows User Guide* ([UG1076](#)).

Because C synthesis for HLS components creates a Vitis kernel (`.xo`) file for hardware emulation and hardware design flows, the Vitis IDE must separately compile each HLS component for use in software emulation using the `v++ -c -k` command as described in [Compiling PL Kernels for Software Emulation](#). This extra compilation step for HLS components is specifically for software emulation builds.

Launching Run or Debug of the System Project

Once the System project has been built, you can launch run or debug with the following steps.

1. Open the Launch Configuration editor (`launch.json`) for the System project by either selecting it from the Settings folder of the project in the Component Explorer, or select **Open Settings** next to Run or Debug in the Flow Navigator.

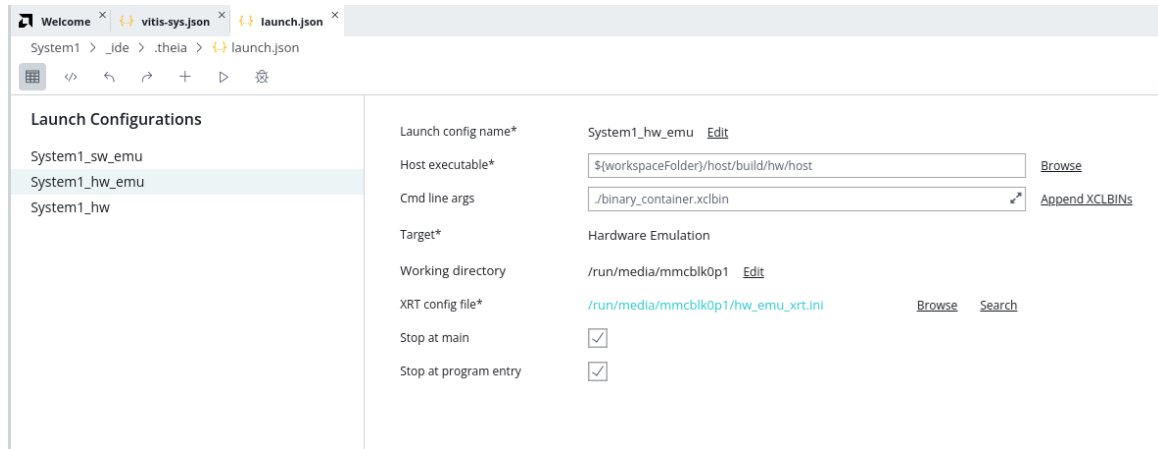


TIP: The Open Settings command is a hidden icon that appears when you place your cursor in the Flow Navigator next to either Run or Debug.



2. Edit an existing launch configuration, or create a new one to modify to establish the specific requirements for the run or debug session you are about to start. You can define separate launch configurations for the three types of targets supported for the System project. The build targets include Software Emulation (`sw_emu`), Hardware Emulation (`hw_emu`), and Hardware (`hw`) as described in *Build Targets in Vitis Unified Software Platform Documentation: Application Acceleration Development* ([UG1393](#)).

Figure 83: Launch Configurations



The preceding figure shows the System project Launch Configuration editor with the options available to configure the System project for Run or Debug.

The preceding figure shows the System project hardware emulation launch configuration which includes the following settings:

- **Name:** The name of the launch configuration.
- **Host Executable:** the name of the host application contained in the Application component.
- **Cmd Line Args:** arguments to be passed to the host application, including the binary container (`.xclbin`) if the code requires it.
- **Target:** The build target that launch configuration should be used with.
- **Working Directory:** This is the working directory that the host application is run from.
- **XRT Config File:** This is the location of the `xrt.ini` file to be used when running the application. Refer to [xrt.ini File](#) for details on the available options.
- **Stop at main:** Enable checkbox to add a breakpoint at the entry point to the main application for debugging.
- **Stop at program entry:** Enable checkbox to add a breakpoint at the entry point to AI Engine program.

After configuring the launch configuration, you can select Run or Debug commands in the Launch Configuration editor, or by selecting Run or Debug from the Flow Navigator. If multiple launch configurations are available for the build target you are running, the Vitis IDE will prompt you to select a launch configuration to use.



IMPORTANT! For embedded platforms running the QEMU environment you must launch the emulator prior to running or debugging the system.

For hardware emulation of embedded devices or embedded platforms you must start the QEMU environment by selecting the **Emulator** command from the Flow Navigator prior to selecting Run or Debug. The QEMU environment takes a few minutes to launch. You should wait to launch Run or Debug until the QEMU is up and running. Refer to [Running Emulation on an Embedded Processor Platform](#) for more information.

When launching hardware emulation, you can specify options for the AI Engine simulator that runs the graph application, as described in *Reusing AI Engine Simulator Options in AI Engine Tools and Flows User Guide* ([UG1076](#)). The options can be specified in the Emulator Arguments field by specifying the following command.

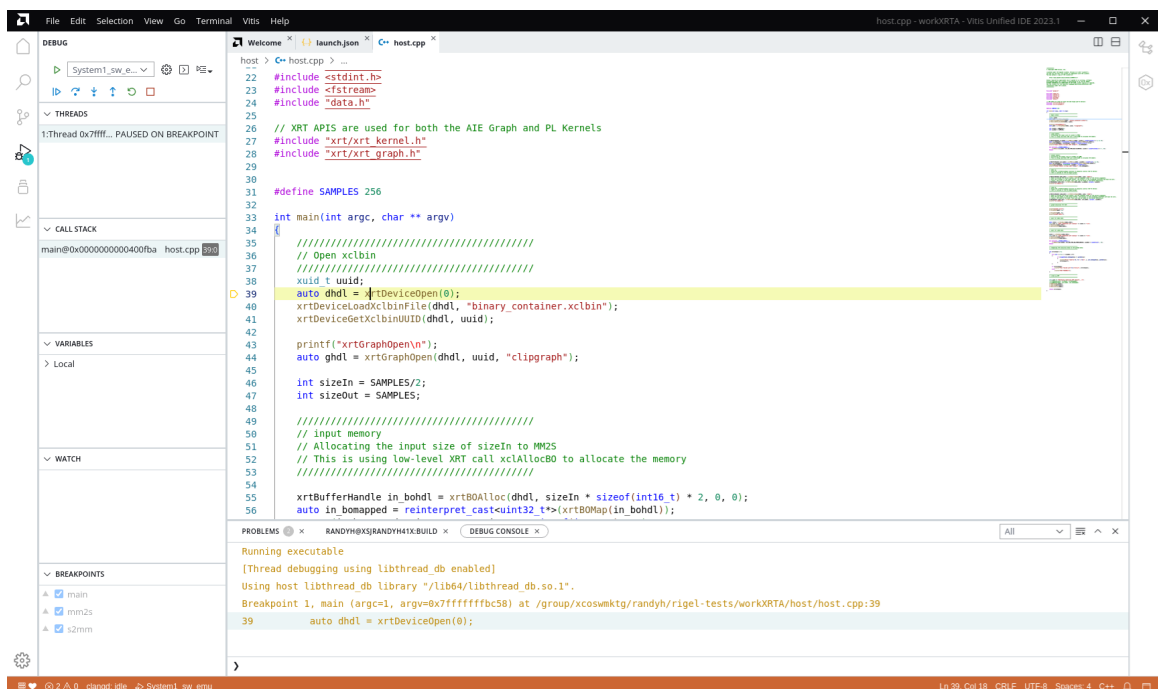
```
-aie-sim-options ${FULL_PATH}/aiesim_options.txt
```

Debugging the System Project

After you have launched the Debug view for an System project, you will see several windows or views displayed as shown in the following figure. The Debug view shows the state of cores that are being debugged. It shows the source file and line number where the debugger stops and what action it is taking (breakpoint/step over/...). Refer to [Debug View](#) for more information on the available features of the tool.

If your System project includes AI Engine components, refer to the Debugging the AI Engine Application chapter in the *AI Engine Tools and Flows User Guide* ([UG1076](#)). If your System project includes PL kernels and Application components only, then refer to for additional information.

Figure 84: Debug View - System Project



Creating a Bare-metal System

Building a bare-metal system requires a deviation from the standard System project flow previously described. The branch in the process occurs after the [Linking the System](#) step, with the bare-metal system requiring a new process branching after the generating the `.xsa` file from the `v++ --link` command. The specific steps required are detailed below.

1. Create a bare-metal fixed platform from the `.xsa` produced by the linking process for the hardware build target. Because the `xilinx_vck190_base_202310_1` base platform does not have a bare-metal domain, you must create a custom platform with one.
 - a. Select the **File → New Component → Platform** command in the Vitis unified IDE. This opens the Create Platform Component wizard as shown.

Create Platform Component

Name and Location > Flow > OS and Processor > Summary

Name and Location

Choose a name for your component and specify a directory where component data files will be stored

Component name: baremetal-platform

Component location: /group/xcoswmktg/andyh/rigel-tests/workAIE [Browse](#)

Component will be created at /group/xcoswmktg/andyh/rigel-tests/workAIE/baremetal-platform

Cancel Next

- b. Specify a Component name and click **Next** to proceed. This displays the Flow page of the wizard where you select **Hardware Design** and specify the `binary_container_1.xsa` from the System project to create the new platform.

- c. After you select the XSA, the Vitis IDE reads the file, determines the Operating system and Processor for the domain defined by the XSA, and populates it in the dialog box.

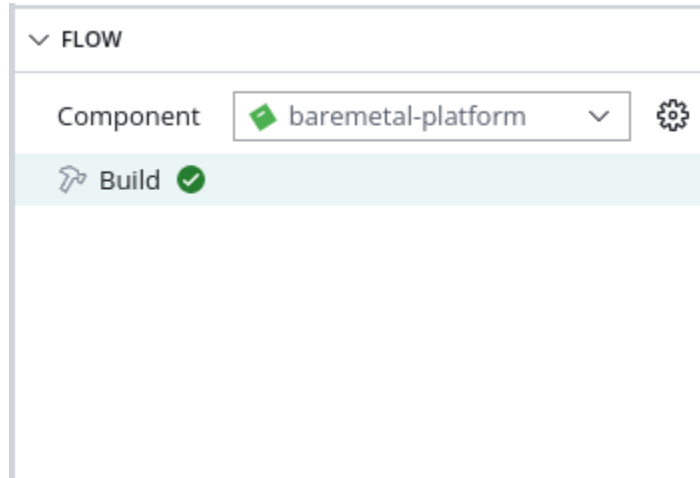
- d. Click **Next** to go to the Summary page and click **Finish** to create the platform project.



TIP: The bare-metal platform is valid for either hardware emulation or hardware builds depending on the fixed-XSA selected for the platform.

- e. Click the **Build** command to build the newly created platform. A copy of the completed platform is written to the export folder of the project, and shows in the Explorer view for the project.

As shown in the following figure, the exported platform has the `platform.xpfm` meta-data file, as well as the `./hw` and `./sw` folders for the different elements of the platform.



The Vitis unified IDE automatically adds the new platform to your platform repository making it available to use in projects. You can also add the file location to your `$PLATFORM_REPO_PATHS` environment variable to make the platform available to the command-line tools as well as the Vitis unified IDE.



IMPORTANT! The bare-metal platform will be only used for building the bare-metal PS application and is not used any other places in the flow.

2. Create a new embedded software Application component.
 - a. Select the **File → New Component → Application** command in the Vitis IDE. This opens the New Application Project wizard.
 - b. Specify a **Component name** and click **Next**.
 - c. Select the **baremetal-platform** you created in the last step, and click **Next** to proceed.
 - d. Review the Domain page and click **Next** to proceed.
 - e. Review the Summary page and click **Finish** to create the Application component.
 - f. Add the source files and build the application as described in [Creating an Application Component](#). The source code for the PS application must be written specifically for the bare-metal project using the provided drivers for accessing the hardware system. For AI Engine designs you must also add the bare-metal AI Engine control file (`aie_control.cpp`), which is created by the `aiecompiler` command, and can be found in the `./Work/ps/c_rts` folder.

- g. Edit the `UserConfig.cmake` file of the Application component located in the Settings folder in the Vitis Components Explorer. This step is only if you have AIE. Edit the Include paths under Directories to include the following:

```
$XILINX_VITIS/aietools/include  
<aie_component>/src
```

- h. After updating the `UserConfig.cmake` file you can build the Application component.
- 3. Package the System project with the new PS application added to the SD card files.
 - a. With the ELF file produced for the bare-metal PS application, you are ready to package the System project and Application component for the bare-metal platform. You must run the package process to generate the final bootable image (PDI), and write the SD card content for booting the device and running the application, as described in [Packaging the System](#).
 - b. Add the following to the Packaging config as described in [Defining the Package Configuration](#):

```
[package]  
ps_elf=../../baremetal_app/Debug/<baremetal_app>.elf,a72-0  
sd_file=<baremetal_app>.elf
```

Note: To enable debugging for both the AI Engine component and the bare-metal PS application, do not add the `--package.ps_elf` option. To debug only the AI Engine component, then add the option as described.

- c. After updating the `package.cfg` file, run the **Build Package** command on the System project.

This adds the ELF file you created in Step 2 above for the bare-metal Application component, assigns it to a processor core (`--package.ps_elf`), and packages the System project files and boot system. In the original Linux-based System project, you added a PS application into the system project to build and debug as part of the system. Here you are building the PS application as part of a separate bare-metal project and adding it as a boot file to the package process in the original System project. This results in a System project build for the platform hardware, and a PS application that runs in the baremetal domain.

Now that you have built the bare-metal system, you can continue to run or debug the application as described in [Bare-Metal Debug from the Vitis Unified IDE](#).



IMPORTANT! You cannot debug the hardware emulation build for bare-metal projects in the Vitis unified IDE.

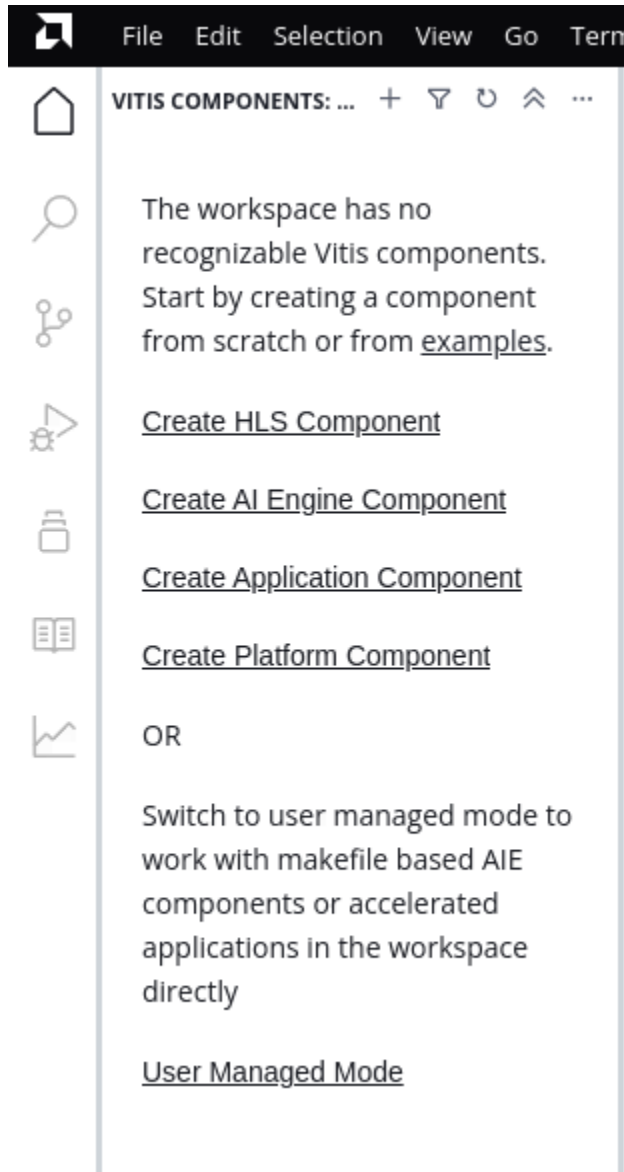
Working with User-Managed Flows

This page describes the steps to use the Vitis unified IDE to build and run user-managed flows, or Makefile flows, for various projects such as AI Engine graph applications, heterogeneous system projects, Data Center acceleration, or embedded software flows.

The user-managed flow lets you configure the tool to build the components of your design using the available commands, Makefile, source files, and config files. It does not require a Makefile, but works well with one. You can also specify command-line tools and compilation commands as needed. Using the Vitis unified IDE with the user-managed flow lets you import existing projects, and report files, and view them in the context of the IDE.

1. Launch the Vitis unified IDE tool and open the project folder as a workspace by selecting the **Open Workspace** command. The project folder can contain a Makefile, or not.

The Vitis Components Explorer will report that the workspace has no recognizable components, and encourage you to create a component or to switch to user managed mode to work with Makefile projects.



2. Select the **User Managed Mode** from the Vitis Components Explorer.

The Vitis Components Explorer view is replaced with the Explorer view to provide a view of the files and folders in the workspace without enhancement.

3. In the workspace folder, right-click and select the **Edit Build Configurations** command from the popup menu.

A `build.json` file is added to the current workspace, and the Build Configurations editor is opened in the IDE.

4. In the Build Configurations editor, create a new build configuration by selecting the **New Build Configuration** command, or the + button in the toolbar menu.

A new build configuration gets created.

- Specify a Build config name
- Specify the Build command to be used. The build command can be taken from a Makefile in the folder. For example:

```
make all TARGET=hw_emu PLATFORM=$PLATFORM_REPO_PATHS/  
xilinx_vck190_base_202320_1
```

Or can be a command line command:

```
v++ -c -t sw_emu --platform xilinx_u250_gen3x16_xdma_4_1_202210_1 --  
config ../src/u250.cfg -k vadd -I../src ../src/vadd.cpp -o sw_emu/  
vadd.xo
```



TIP: You can expand the Build command field by selecting the double-headed arrow on the right. This will provide more space for editing and viewing the command text.

- Specify a Clean command command, also taken from a Makefile:

```
make clean TARGET=hw_emu
```

or command-line:

```
rm -rf sw_emu
```

- Specify a Run directory where the build will occur. The current workspace is the default Run directory.
5. Create additional build configurations as needed for your project.
 6. With the build configurations created, you can build components by right-clicking in the Explorer view and selecting the **Build** command.

If you created multiple build configurations, the IDE will list the available configurations letting you select one. If there is only a single build configuration the tool will run it.



IMPORTANT! Make sure the shell or terminal window from which the Vitis unified IDE is launched is properly configured to run the specified build commands. Any environment variables required by a Makefile for example will also be required by the tool.

7. The component or system will build according to the build configuration commands. The output of the process will be reported to the Output view. If needed, you can open this view from the **View → Output** menu command.

After build the components of your design, you can run or debug them as needed in the IDE by creating a launch configuration for the component. Use the following steps to create and run or debug the components or system project.

1. Right-click in the workspace folder in the Explorer view and select the **Edit Launch Configurations** command from the popup menu.

A `launch.json` file is added to the current workspace, and the Launch Configurations editor is opened in the IDE.

2. Select the **New Launch Configuration** command to define a launch configuration, or select the **+** button in the toolbar menu.

The Create Launch Configuration dialog box is opened as shown in the following figure.



TIP: The tool will offer indicating problems with the specified build directory, such as the tool cannot find a summary file generated by `v++` during the build process. The warning could indicate the component or system is not valid, or did not build properly, and the tool will not proceed.

The Create Launch Configurations dialog box offers three configurations that can be run or debug.

- **AI Engine Graph:**

Select this option for any AI Engine graph application or template. The specified build directory is automatically populated with the available Work directories output by the `v++` command, to enable `x86simulator` or `aiesimulator`.

Click **Submit** to create the launch configuration for AI Engine application. In the launch configuration specify any preferences like Pipeline View, Trace and Profiling. Select the **Play** command to run the component, or select the Debug button for debug.

- **Accelerated Application:**

Select this option for a System project using an extensible platform for embedded system designs or Data Center application acceleration. The specified build directory is automatically mapped with build target, platform and binary container info.

Launch configuration with the few sections gets created for the app, provide the necessary info. and run/debug it.

- **Embedded Application:** There are four options for the Embedded Application, as described below. Refer to *Vitis Embedded Software Development User Guide (UG1400)* for more information.
 - **Attach to Running Target :** To debug the already running target, select this option. Provide the target connection and start running/debugging the application.
 - **Baremetal:** If you have an XSA and want to create a run configuration, map the XSA and click **Submit**. Launch configuration with the few sections gets created for the app, provide the target connection, fsbl and other necessary info. and run/debug it.
 - **Attach to Running Process on Linux Target:** To debug a Linux application on an already running Linux target, select this option and click submit. Launch configuration with the few sections gets created for the app, provide the target connection and host executable info. and run/debug it.
 - **Linux Application with ELF:** If a Linux application has to be run/debugged, select this option and provide the ELF. Launch configuration with the few sections gets created for the app, provide the necessary info. and run/debug it.
- 3. Configure the Launch Configuration as required for the component or system you are trying to run or debug. Refer to [Launch Configurations](#) for additional details.
- 4. After creating the launch configuration, you can run or debug the configuration by right-clicking in the Explorer view and selecting either **Run** or **Debug** commands from the popup menu.
- 5. If the launch configuration is for an embedded system running under QEMU you must start the emulator before launching the configuration. You can do this by selecting **Start Emulator** from the popup menu. This will open the Start Emulator dialog box to review and configure if needed, and click **Start**. Once the emulator is fully up and running, use the launch configuration to run the application or debug it.

Debugging the System Project and AI Engine Components

You can debug AI Engine and HLS components in a standalone capacity, or you can debug the System project, including the top-level PS application, and any additional components. Within this framework, you can also debug applications built from the command line, or system projects built in the Vitis unified IDE. You can debug applications running on the Linux OS, or in bare-metal systems. Finally, you can debug hardware emulation builds that let you simulate the application, or debug the actual application running on hardware. All of these configurations are addressed in the following topics.

The following process is recommended for debugging AI Engine applications and system designs:

1. Use x86 simulation for both single and multi-kernel debug. It supports breakpoints and single stepping using the GNU debugger
2. Use `aiesimulator` to verify timing and to check that it will fit within the program memory and stack/heaps size within the available hardware memory space. The `aiesimulator` and `x86simulator` and their use are described in *Simulating an AI Engine Graph Application* in *AI Engine Tools and Flows User Guide* ([UG1076](#))
3. Software emulation for functional verification of the System project, including Application component, AI Engine component, and c-models for user PL code
4. Hardware emulation for complete integration testing including PL, PS, and AI Engine domains as applicable
5. Hardware-based testing of the final booted system

If code changes are necessary at any step, repeat all steps from the beginning.

Note: It is recommended to use the compiler optimization option `--xlopt = 0` because higher compiler optimizations will reduce debug visibility.

Launching Debug in the Vitis unified IDE

After building the System project, incorporating the kernels running in the AI Engine domain, the PL domain, and an application running in the PS domain, as described in [Creating a System Project for Heterogeneous Computing](#), you can launch debug. These system-level projects can be debugged together from within the Vitis unified IDE using the many features as described in [Debug View](#), or you can debug the individual components like the AI Engine graph application, focusing on a particular design problem.

This section discusses running the Debug Environment from the Vitis unified IDE for hardware emulation and hardware builds.

Using printf for Event Tracing

The simplest form of tracing is to use a formatted `printf()` statement in the code for printing debug messages. Visual inspection of intermediate values, addresses, etc. can help you understand the progress of program execution. No additional include files are necessary for using `printf()` other than standard C/C++ includes (`stdio.h`). You can add `printf()` statements to your code to be processed during simulation, or hardware emulation, and remove them or comment them out for hardware builds.

Adding `printf` statements to your AI Engine kernel code will increase the compiled size of the AI Engine program. Be careful that the compiled size of your kernel code does not exceed the per-AI Engine processor memory limit of 16 KB.



IMPORTANT! You must use the `aiesimulator --profile` command to enable the `printf()` execution during a simulator run. If `--profile` is not specified, the `printf()` function is ignored.

A separate driver and binary is used for this functionality to allow the main simulator to remain as fast as possible. Using the debug simulator driver produces a per-tile profile report under the output directory which gives detailed cycle-level statistics of kernel execution. In addition, using the `--profile` option generates a `run_summary` file that is written to the `./aiesimulator_output` folder that can be viewed as described in [AI Engine Tools and Flows User Guide \(UG1076\)](#).

Software Emulation Debug from the Vitis IDE

To run and debug the application in the Software Emulation build target, you must use the following steps.

1. With the System project active in the Flow Navigator, select the **Build All** command under the SOFTWARE EMULATION heading.

Note: For software emulation the System project is built to run on the x86 processor as described in the [Embedded Processor Emulation Using PS on x86](#), rather than the QEMU environment.

2. After the build completes successfully, select the **Debug** command from the Flow Navigator under the Software Emulation heading.

Note: If you have not setup a launch configuration for the build, you will be prompted to do so as described in [Launch Configurations](#).

3. The Debug view opens in the Vitis unified IDE as shown in [Debug View](#), and connects to the PS application. The application is paused at the entry to `main()` function.

The Source Code editor opens to display a source code file when a breakpoint in that file is triggered.

4. Click **Resume** (▶) from the Control Panel to step forward to the next breakpoint. You can also press Step Over, Step Into, or Step Out as needed.

Looking at the Debug view, you can see the various threads spawned. The view also shows which thread has hit a breakpoint with the status Paused on Breakpoint.

DEBUG

▶

kernels_PeakDetect_system_sw_emu_new

▼

⚙️

▶

☰

▶

||

↺

⬇️

⬆️

↻

⏏️

▼ DEBUG

▼ PeakDetect_system_sw_emu_new (2)

RUNNING

1:Thread 0x7fff7e8e7c0 (LWP 60184)

RUNNING

2:Thread 0x7fff5344700 (LWP 61937)

RUNNING

3:Thread 0x7fff4b43700 (LWP 61942)

RUNNING

▼ kernels_PeakDetect_system_sw_emu_new

PAUSED

1:Thread 1

PAUSED

2:Thread 2

PAUSED

3:Thread 6

PAUSED ON BREAKPOINT

4:Thread 3

PAUSED

5:Thread 4

PAUSED

6:Thread 5

PAUSED

7:Thread 7

PAUSED

8:Thread 8

PAUSED

▼ CALL STACK

mm2s@0x00007ffff536f0f0

mm2s.cpp

22:0

__stub___xlnx_cl_mm2s@0x00007ffff5370c4a

xcl_top.cpp

38:0

xclcpuem::simclEnqueueNDRangeKernel(xclcpuem::cu*)@0x000000000...

xclcpuem::cu_sim_function(void*)@0x000000000045a0a6

start_thread@0x00007ffff69ebe65

clone@0x00007ffff617a88d

The Call Stack view shows the function call stack being updated as emulation runs.

From this point you can do all the debug activities like Step In, Step Over, Step Out, or viewing variables, expressions, and memory locations in the software emulation environment. Refer to [Using the Debug Environment](#) for more information.

Hardware Emulation Debug from the Vitis IDE

Unlike Software Emulation which is compiled to run on an x86 processor even for Arm processor based applications, the Hardware Emulation must be run under the QEMU environment as described in [Section V: Simulating the Application with the Emulation Flow](#). To run and debug the application for the Hardware Emulation build target, you must first launch the QEMU environment using the following steps:

1. Start the QEMU emulation environment by selecting the **Start Emulator** command from the Flow Navigator to open the following dialog box.

The 'Start Emulator' dialog box contains the following fields and controls:

- System Project:** PeakDetect_system
- Target:** Hardware Emulation
- Show Waveform:** ☐
- Enable Trace:** ☒
- Additional Arguments:** Emulator Arguments
- Status:** Not Running
- Enable Profile:** ☒
- Cores:** All

Below these fields is a table with two columns: KEY and VALUE. The table is currently empty, with a '+' button in the top right corner to add new entries.

At the bottom right of the dialog are two buttons: 'Cancel' and 'Start'.

Where:

- **Show Waveform:** Enables the display of the Live simulation Waveform view from within the Vivado Logic Simulator during the Emulation run. This lets you examine waveforms in realtime rather than after. However, it takes more time and resources to run.
- **Enable Trace:** Enables capturing trace data from hardware emulation.
- **Additional Arguments:** Used to pass additional arguments to the `launch_emulator` utility from the command line when starting QEMU. For example, this can be used for specifying options to pass to the AI Engine simulator that runs the graph application, as described in *Reusing AI Engine Simulator Options in AI Engine Tools and Flows User Guide* ([UG1076](#)). The options can be specified as follows:

```
-aie-sim-options ../aiesim_options.txt
```


- **Enable Profile:** Enables profiling of the design as explained in [Profiling the Application](#).
 - **Key/Value Pairs:** Specifies parameters to use when running the application in the form of the parameter name and value.
2. Click **Start** to launch the emulator. Then you must wait for the QEMU environment to boot the emulation system before starting your application.

The TASK: EMULATION output terminal shows a transcript of the QEMU launch and boot process. You will know the process has completed when the `/mnt#` command prompt is displayed in the terminal window. From the command prompt you can type commands in the QEMU environment, and review the transcript of the boot process for details.



IMPORTANT! Do not run the emulation from this window. The Vitis unified IDE will manage the application when launched from the Run or Debug command in the Flow Navigator after the emulator has been started.

3. After the QEMU has started, click **Debug** from the Flow Navigator.

This opens the Debug view in the Vitis unified IDE as shown in the [Debug View](#), and connects to the PS application and AI Engine graph running on their respective cores in the QEMU. The application automatically breaks at the `main()` function for all the ELF files.

4. Click **Resume** (▶) from the Control Panel to start the process.

From this point you can do all the debug activities like step in/step over/viewing variables/point break points in the emulation environment. Refer to [Using the Debug Environment](#) for more information.

Hardware Debug from the Vitis IDE

With the top-level system project built, you can use the following steps to debug the system running in the Hardware platform.

1. Burn `<project>/Hardware/package/sd_card.img` to a physical SD card. This creates a boot-able medium for your target platform.
2. Insert the SD card into the card reader of the [VCK190](#) evaluation kit.
3. Change the boot-mode settings of the card to SD boot mode, and power up the board.
4. After the VCK190 is booted, enter the `mount` command at the command prompt to get a list of mount points. As shown in the following figure, the `mount` command displays mounting information for the system.



TIP: Be sure to capture the proper path for the `cd` command in the next step, and subsequent commands, based on the results of the `mount` command.

```

COM8 - Tera Term VT
File Edit Setup Control Window Help
cgroup on /sys/fs/cgroup/cpuset type cgroup (rw,nosuid,nodev,noexec,relatime,cpu
set)
cgroup on /sys/fs/cgroup/devices type cgroup (rw,nosuid,nodev,noexec,relatime,de
vices)
cgroup on /sys/fs/cgroup/perf_event type cgroup (rw,nosuid,nodev,noexec,relatime
,perf_event)
cgroup on /sys/fs/cgroup/net_cls,net_prio type cgroup (rw,nosuid,nodev,noexec,re
latime,net_cls,net_prio)
cgroup on /sys/fs/cgroup/blkio type cgroup (rw,nosuid,nodev,noexec,relatime,blki
o)
cgroup on /sys/fs/cgroup/memory type cgroup (rw,nosuid,nodev,noexec,relatime,mem
ory)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,relatime,pagesize=2M)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,size=3999452k,nr_inodes=409600)
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /run/media/mmcblk0p1 type vfat (rw,relatime,gid=6,fmask=0007,d
mask=0007,allow_utime=0020,codepage=437,ioccharset=iso8859-1,shortname=mixed,erro
rs=remount-ro)
tmpfs on /run/user/0 type tmpfs (rw,nosuid,nodev,relatime,size=799888k,nr_inodes
=199972,mode=700)
root@versal-rootfs-common-20221:~#

```

- Execute the following commands:

```
cd /run/media/mmcblk0p1
```

- Run `ifconfig` to get the IP address of the target card. The IP address is used to set up a TCF agent connection in Vitis unified IDE. The target needs to connect to the network assigned IP address.
- From within the IDE select or create a target connection to the remote accelerator card as follows:
 - Select the **Vitis → Target Connections** command from the main menu to open the Target Connections dialog box.
 - Select an existing Linux TCF Agent connection, or right-click **Linux TCF Agent** and select the **New Target** command to open the New Target Connection dialog box as shown in the following figure.

Target Connection Details

New Target Connection

Name:

☒ Set as default target

Type:

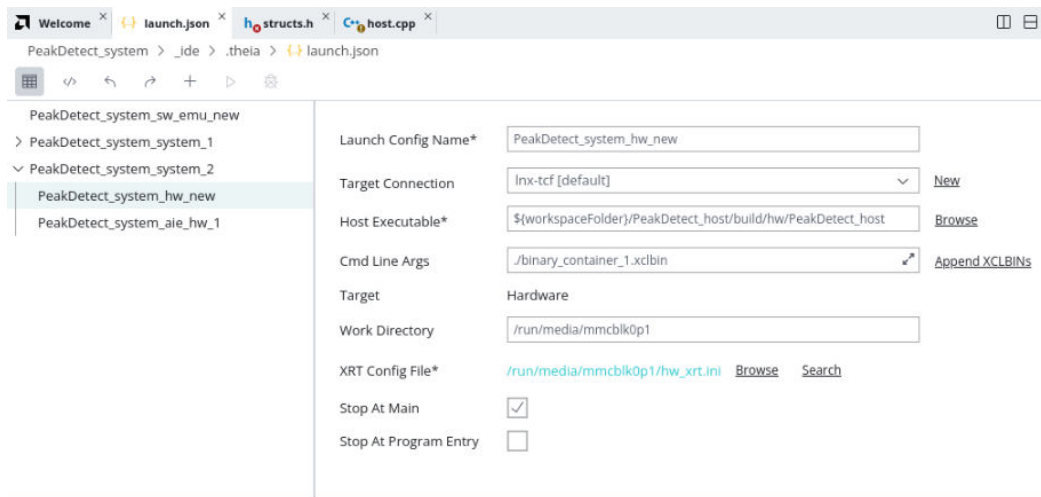
Host:

Port:

> Advanced

Test Connection Cancel OK

- c. Specify the Target Name, enable the **Set as default target** check box, and specify the Host IP address of the accelerator card that you obtained from the `ifconfig` command.
 - d. Click **OK** to close the dialog box and continue.
8. Select the **Open Settings** command in the Flow Navigator next to the Debug command, or open the `launch.json` from the System project in the Vitis Components Explorer. Validate the launch configurations for the Hardware build, or select the **Add Configuration (+)** command to create a new configuration as shown in the following figure:



Be sure to set the following fields on the dialog box as shown in the preceding figure.

- Target Connection: Select the new Linux TCF agent you built with the specified IP address for the accelerator card
 - Host Executable: Specify the PS application to run from the SD card
 - Cmd Line Args: Specify any command line arguments required for the software application, such as the xclbin file to load
 - Work Directory: Specify the remote mount location for the SD card
 - Stop At Main and Stop At Program Entry: To stop the application and AI Engine from running before starting the Debug process
9. Click **Debug** from the Flow Navigator under the **HARDWARE** heading.



TIP: The tool will prompt you to create two launch configurations if you haven't already done so. One for the top-level system project, and a second for the PS application.

This opens the Debug view as shown in [Debug View](#), and connects to the PS application and AI Engine graph running on their respective cores. The application automatically breaks at the `main()` function for all the ELF files.

From this point you can do all the debug activities such as, step in, step over, viewing variables, apply break points in the emulation environment. See [Using the Debug Environment](#) for more information.

Bare-Metal Debug from the Vitis Unified IDE

With the bare-metal system project built as described in [Creating a Bare-metal System](#), you must use the following steps to debug both the AI Engine graph, together with the bare-metal PS application on the Hardware build target.

This process is a bit more complex than debugging a Linux-based System project where the PS application is built as a part of the system project. Here the PS application is built as a separate application component, and must be manually included in the launch configuration for debug. This is detailed in the following steps.

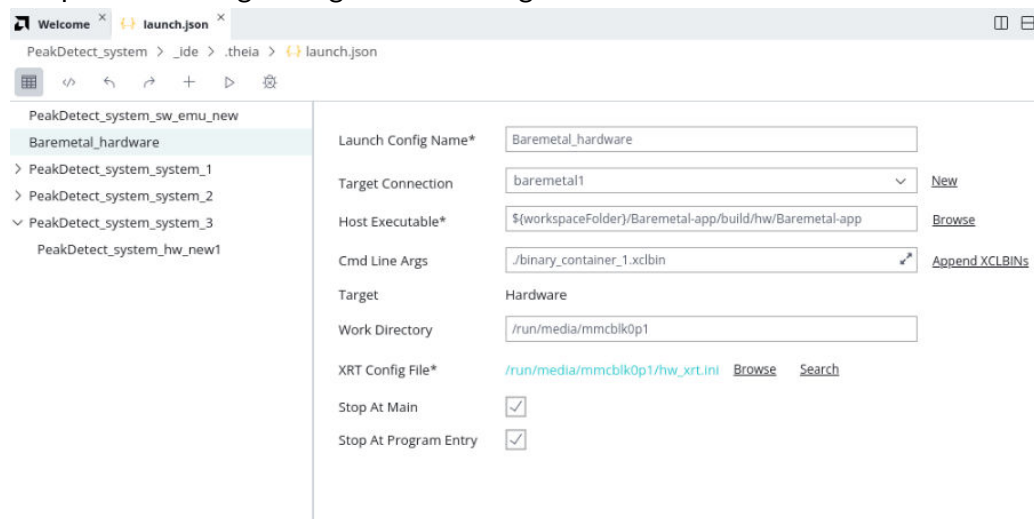
1. Right-click on the top-level system project and select the **Debug As → Debug Configurations** command.

This opens the Debug Configurations dialog box to let you set up the tool.



IMPORTANT! For the Hardware build, you will need to create two Debug configurations: one for the top-level system project, and a second for the bare-metal PS application.

2. In the Debug Configurations dialog box select the **New Launch Configuration** (📄) command to open the Debug Configurations dialog box as shown.



Notice the following fields on the Debug Configurations dialog box:

- **Project:** Reflects the name of the top-level system project which includes the AI Engine graph application, the PL kernels, and the HW-Link projects.
- **Hardware Server:** Specifies a local connection to the board. You can configure this differently for a remotely connected board.
- **Linux TCF Agent:** Is disabled for bare-metal systems.
- **Stop At Main and Stop At Program Entry:** Enable these options to prevent the application from running before you have a chance to start debug.

3. Select **Apply** to save and apply your changes, and select **Close** to close the dialog box.

4. Click **Debug** from the Flow Navigator under the **HARDWARE** heading.



TIP: The tool will prompt you to create two launch configurations if you haven't already done so. One for the top-level system project, and a second for the PS application.

This opens the Debug view as shown in [Debug View](#), and connects to the PS application and AI Engine graph running on their respective cores. The application automatically breaks at the `main()` function for all the ELF files.

From this point you can do all the debug activities: step in, step over, viewing variables, apply break points in the emulation environment. Refer to [Using the Debug Environment](#) for more information.

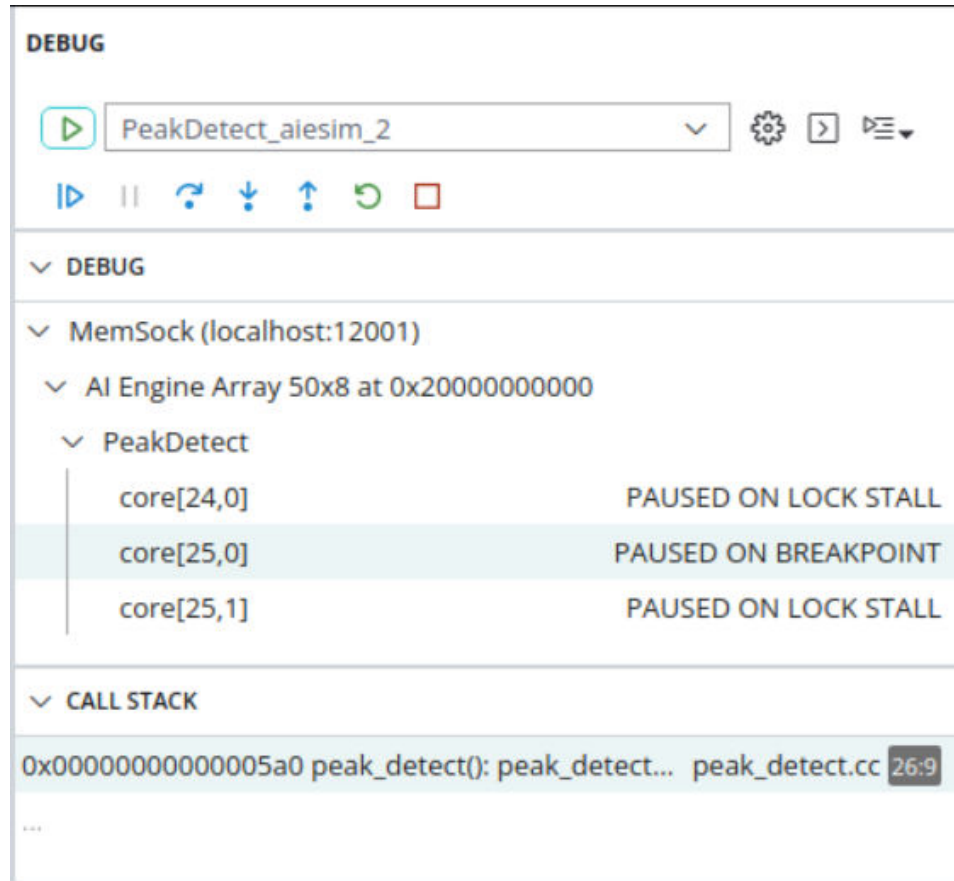
Using the Debug Environment

The Vitis IDE debug environment has many features found in traditional GUI-based debug environments, such as GDB. You can add break points to the code, step over or step into specific lines of codes, loops, or functions, and examine the state of variables and force them to specific values. These are a few of the features in the Vitis IDE debug environment.

After you have launched the Debug perspective, you will see several windows or views displayed, such as the Debug view in the upper left of the display, as shown in the following figure. During the debug process, several windows show the debug state that include call stack, code at breakpoint, step over states, breakpoints view, variables view, registers view, disassemble view, and pipeline view.

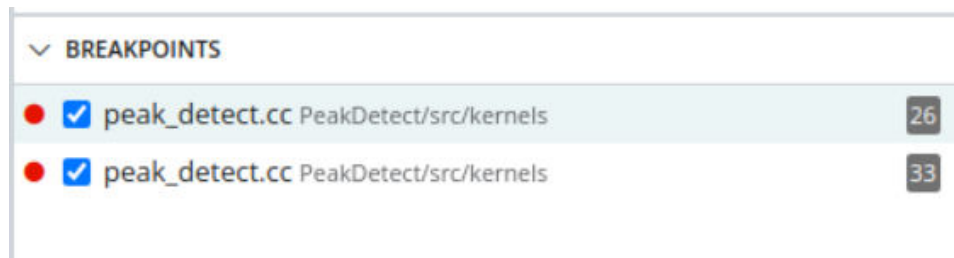
The Debug view shows states of cores that are under debug. It shows where (which file and which line of source code of the file) debugger stops at and with what action it is taking (breakpoint/step over/...) as shown in the following figure.

Figure 85: Debug View



The following figure shows the Breakpoints information with current setup breakpoints. The square with a check mark indicates the breakpoint is enabled. Click on the check mark to clear the check mark and disable breakpoint during debugging. This lets you manage breakpoints without having to remove them or add them back into the code.

Figure 86: Breakpoints View



IMPORTANT! Each AI Engine tile supports four breakpoints when debugging on the AI Engine simulator, or co-simulation. The TCF framework stops at the AI Engine kernel `main()` by default. Breakpoints attached to a `while` statement consume two breakpoint resources. A workaround is to attach the breakpoint inside the `while` loop. This only consumes one breakpoint.

In a source-level debugger, you can trace the source variables allocated to memory or registers. Their location and contents can be visualized. However, when all optimizations are enabled, tracing source variables is not straightforward:

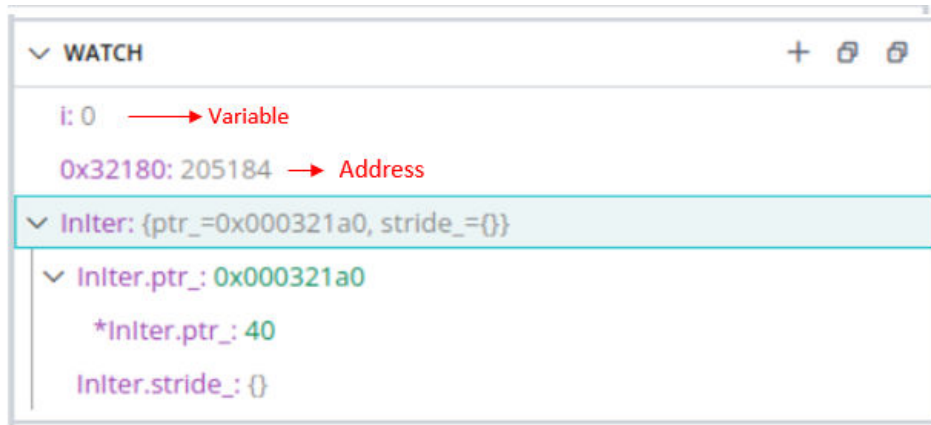
- Local scalar variables typically reside in registers, often different ones at different points in the execution.
 - Variables allocated to memory are not always directly updated. For example, a global variable can be loaded in registers in front of a loop. Then, it can be operated on in registers inside the loop, and only stored back in memory after the loop. This is referred to as optimization redundant store removal.
 - Similarly, stepping through the source code becomes difficult when the code of a single source line is scattered in the eventual object code.
-

Watchpoints

A watchpoint is a type of breakpoint. Watchpoints can be used to stop the execution of a core when the value at an address changes. These are useful in determining the place where the value changes. For example, a variable value can be overwritten unexpectedly, which can break the program flow. Watchpoints help in detecting such cases.

AI Engine architecture supports read/write watchpoints. This means, a watchpoint is triggered on a read or write access, or both (based on your configuration). Each AI Engine tile supports two watchpoints per memory bank. Because every AI Engine core (excluding tiles on boundaries) has access to memory banks in adjacent tiles, a maximum of eight watchpoints can be used per core. However, because the memory banks are shared, the number of available watchpoints per core depends on how many watchpoints are already used from the memory bank. For example, if a core uses all eight watchpoints from its four adjacent memory banks, the other cores which share those four memory banks cannot use watchpoints from the shared memory banks. Debugger keeps track of watchpoints that are allocated from each bank, and throws an error if there are no unused watchpoints in that tile.

1. To add watch point in the Vitis IDE, locate the **WATCH** window in the bottom left side. Hover your mouse on that window and click on the **Add Expression (+)** command as highlighted below.
2. You can enter the memory address or the variable name of interest. You can observe that the values in the watchpoints may appear as "not available", if you select a core other than the core for which the watch point is created. For example, if you created watch points by selecting the core in the debug view, the updated values are seen only for that particular core. For rest all other cores, the watch point values may appear as "not available."



3. You can edit, copy and remove one or all watch points by right-clicking on any watch point.

Triggering of Watchpoints

A watchpoint is triggered on a read access or write access, or both, based on how the watchpoint is configured. A triggered watchpoint causes the core to stop at the instruction that accessed the address. Debugger detects the core has stopped because of the watchpoint and reports that the core has stopped because of a watchpoint.



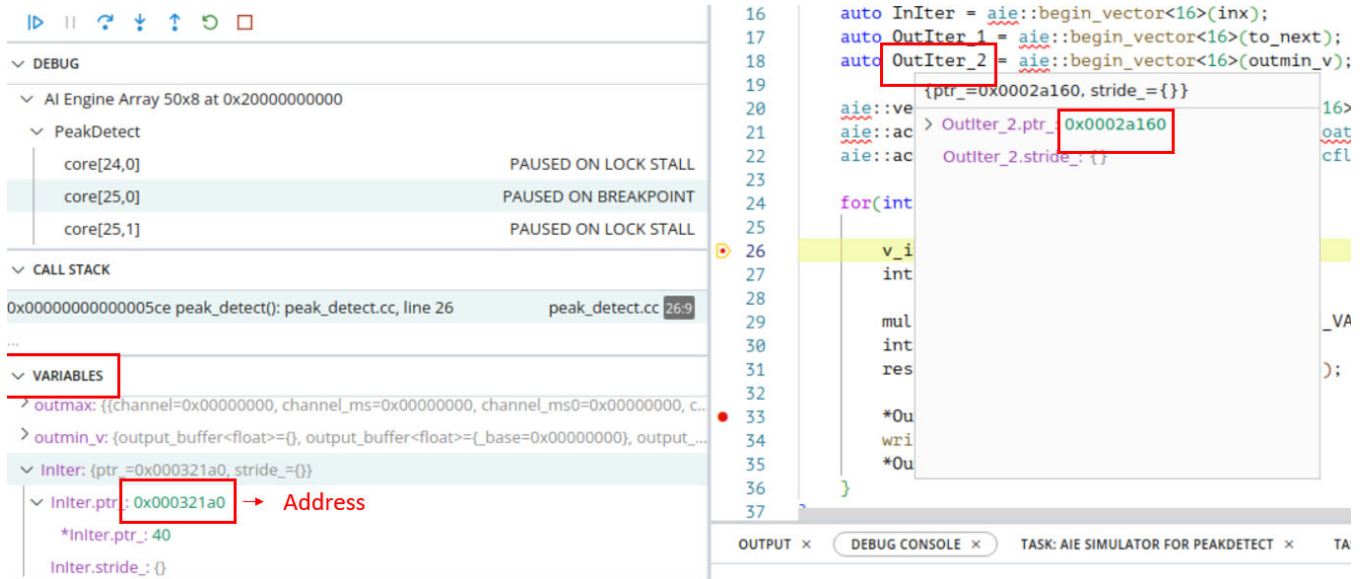
IMPORTANT!

- Watchpoints work on hardware only. AI Engine system C model is not supported.
- Memory banks are shared. It might not be possible to add watchpoints for a core in a specific bank, if another core uses both the watchpoints from that bank.
- Watchpoints must not be set for memory addresses which fall in unused tiles/memory banks. Setting watchpoints to unused tiles can cause AXI errors. Used AI Engine and memory tiles can be found at `Work/aie/active_cores.json`.
- Watchpoints are triggered for memory access in full 16-byte aligned address range.
- Debugger uses two broadcast channels to handle watchpoint events. When enabling watchpoints during debug, ensure that there are no conflicts in using these two broadcast channels.

Variable View and Memory Inspector

The Variables view shows the kernel variables values. Depending on variable type, clicking on a variable shows its type, value, and the address of the variable. For array/structure variables, click on the arrow of the variable expands array/structure content of the array.

Figure 87: Variables View

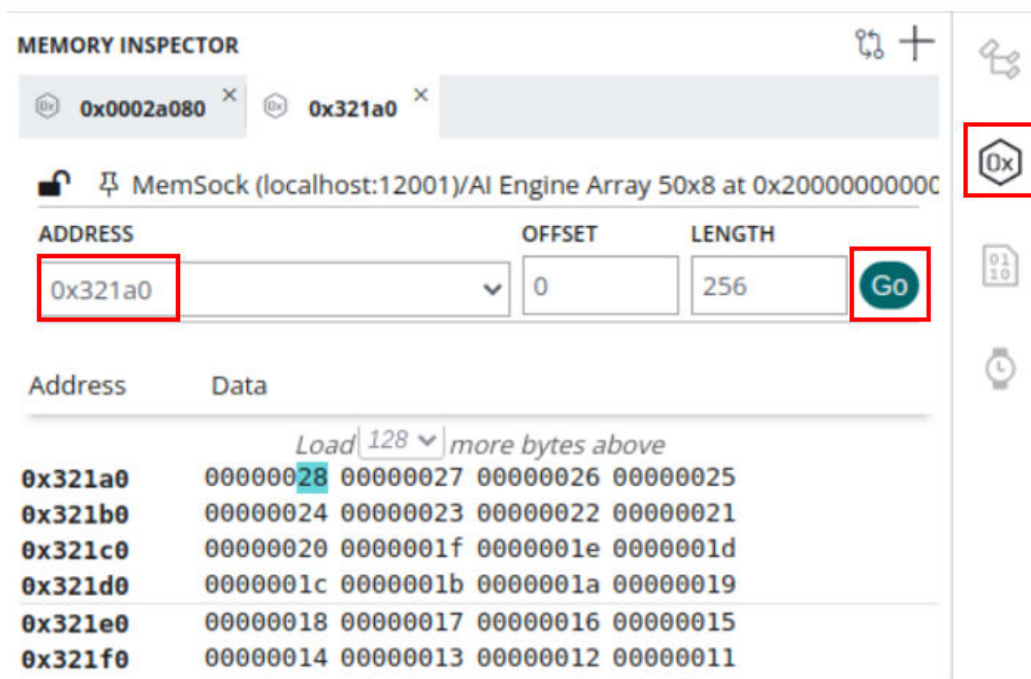


Click on the variable in the VARIABLES view or hover your mouse over the variables in the code view on the right-side as shown above to get the address and add that address in the memory inspector as shown below. As you step-in, step-over the code, the values in the variable and memory view changes.



TIP: You can export the contents of the Memory window by selecting the **Copy to Clipboard** command from the right-click menu to copy and paste the contents into a text editor and save to a file.

Figure 88: Memory Inspector



Register Inspector

In the Registers Inspector, the values are updated during the debug process as you step-in or through the code, and are highlighted as shown in the following figure.

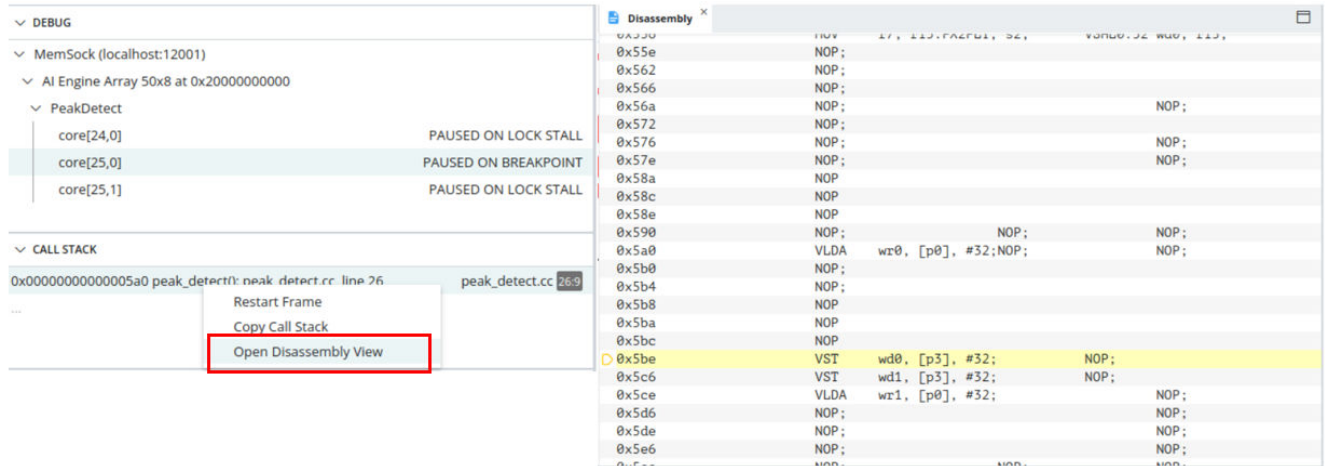
Figure 89: Register Inspector

REGISTER INSPECTOR		
NAME	HEX	DESCRIPTION
 r0	00000008	General purpose register
 r1	00000002	General purpose register
 r2	00000004	General purpose register
 r3	00000001	General purpose register
 r4	42440000	General purpose register
 r5	000000f0	General purpose register
 r6	00000002	General purpose register
 r7	00000000	General purpose register
 r8	00000000	General purpose register
 r9	42600000	General purpose register
 r10	42440000	General purpose register
 r11	00000034	General purpose register
 r12	42580000	General purpose register
 r13	00000037	General purpose register
 r14	42540000	General purpose register
 r15	42500000	General purpose register
 pc	000005ce, At: peak_detec	Program Counter
 fc	00000620	Fetch Counter

Disassembly View

The Disassembly view displays machine code and assembly code. C/C++ source code is also embedded between the lines for source code referencing. In the Explorer view, select the AI Engine core in Debug view and right-click on 'CALL STACK' and select Open Disassembly View. It will bring up the following disassembled code.

Figure 90: Disassembly View



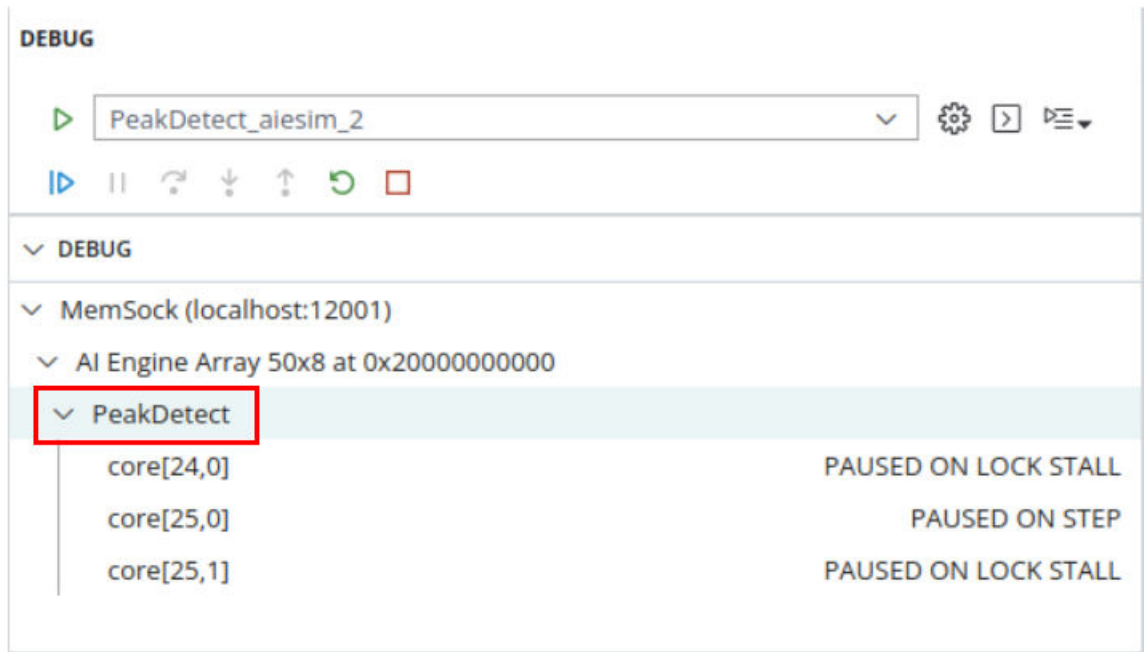
You can step over the disassembly code using the debug controls as shown below. These control commands let you step into/over/return from the source code. Other commands let you resume, stop, terminate, or disconnect the debugger from the Vitis IDE.

Multi-Kernel Debug

The Vitis IDE supports multi-kernel debug and multi-domain (PS and AI Engine) debug. Depending on the application, there can be hundreds of kernels being debugged. Having granular control of each core, in addition to all cores within one domain is important.

Select a single core from the Vitis IDE and click the **Continue** command to resume debug execution for the selected core only. Select one level above all cores within the AI Engine domain, and click the **Continue** command to resume all AI Engine core executions. Resuming execution for all the kernels in the graph is dependent on a variety of conditions such as the availability of data to each of the kernels to avoid stalls, or the setup of individual kernel breakpoints. You must also ensure that the kernels not being debugged are free to run.

Figure 91: Disassembly View

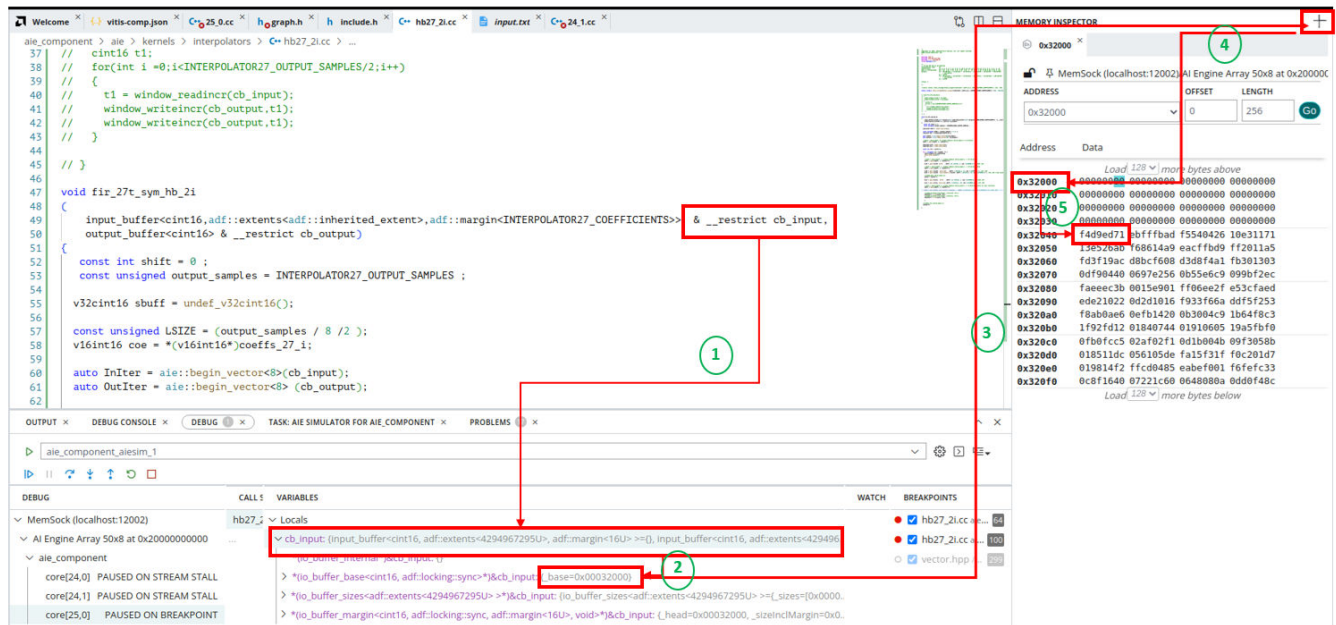
**Note:**

The `main()` function is created automatically by the AI Engine compiler for each core that wraps around the kernel implementation. The Debug view displays the complete source code including the inserted portion.

Viewing Data from Buffer Port Interfaces

During debug, you want to see the value of data passing through the kernel. In the following figure you can trace through objects and observe data stored in the designated memory location from the Vitis IDE.

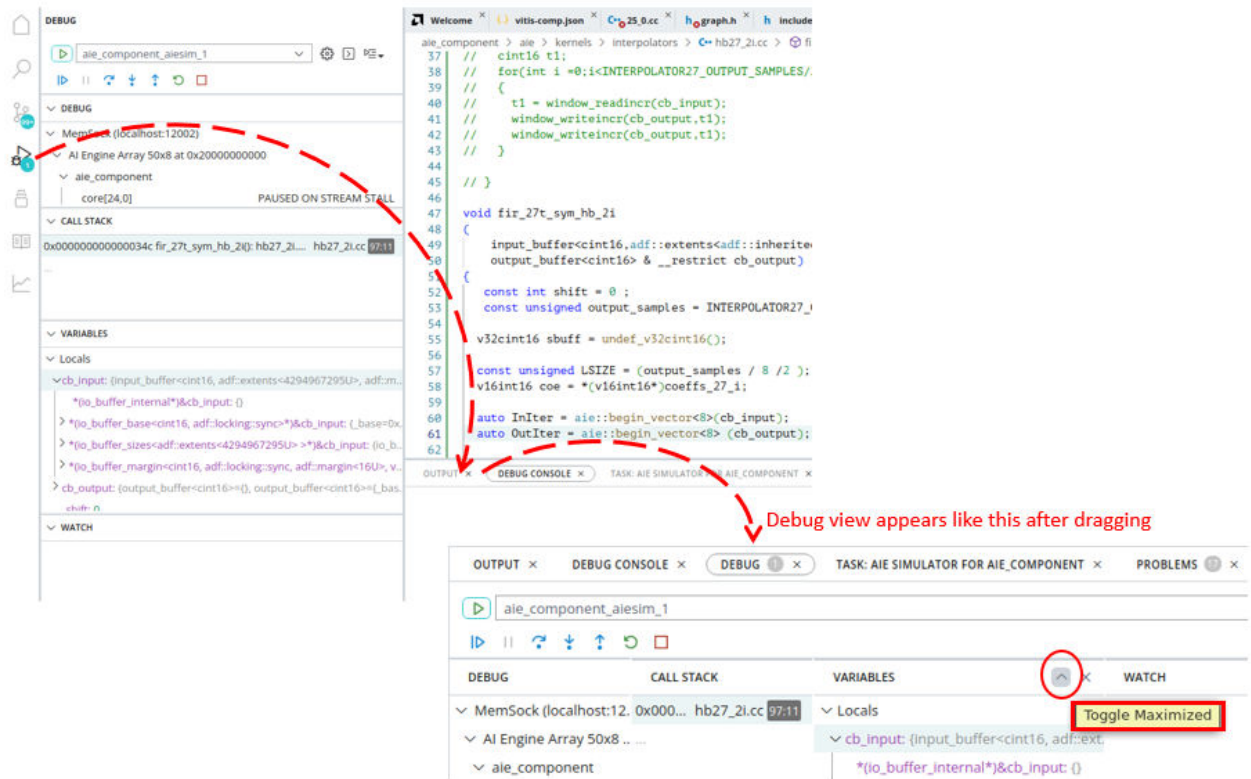
Figure 92: Tracing Data in Vitis unified IDE



1. In preceding code example, Step 1 shows the variable `cb_input` is the reference to `input_buffer` of type `cint16`.
2. In Step 2 move to the `cb_input` variable in the Variables view. This is the pointer representation of the data access buffer port that holds the input data for the kernel. However, the kernel functions merely operate on pointers to the buffer data structures passed to them as arguments. The input buffer port holds the data. Examine the address of the variable `cb_input`. It is at address `0x32000`.
3. In Step 3, enable the Memory Inspector window on the top right and click on the + sign to input the address `0x32000`.
4. In Step 4, the Memory window displays the content at address of `0x32000`. This is the data contained in the data access buffer port defined by the `cb_input0` variable.
5. This example has 16 elements as the margin size and each element is `cint16` type, so the actual data starts from `0x32000 + 0x40 = 0x32040`. You can examine the data contents, export to a file and hover your mouse on the hexadecimal values to display it in a specific data format.

Note: You can click the **Debug** command and drag it from the left side Tool Bar menu to the bottom middle and drop it inside the space where you have the Debug console. This helps in viewing the variables in expanded view by clicking on Arrow button as shown in the following figure. You can drag it back to the left side at any time.

Figure 93: Rearranging Windows



Viewing Data from Stream Interfaces

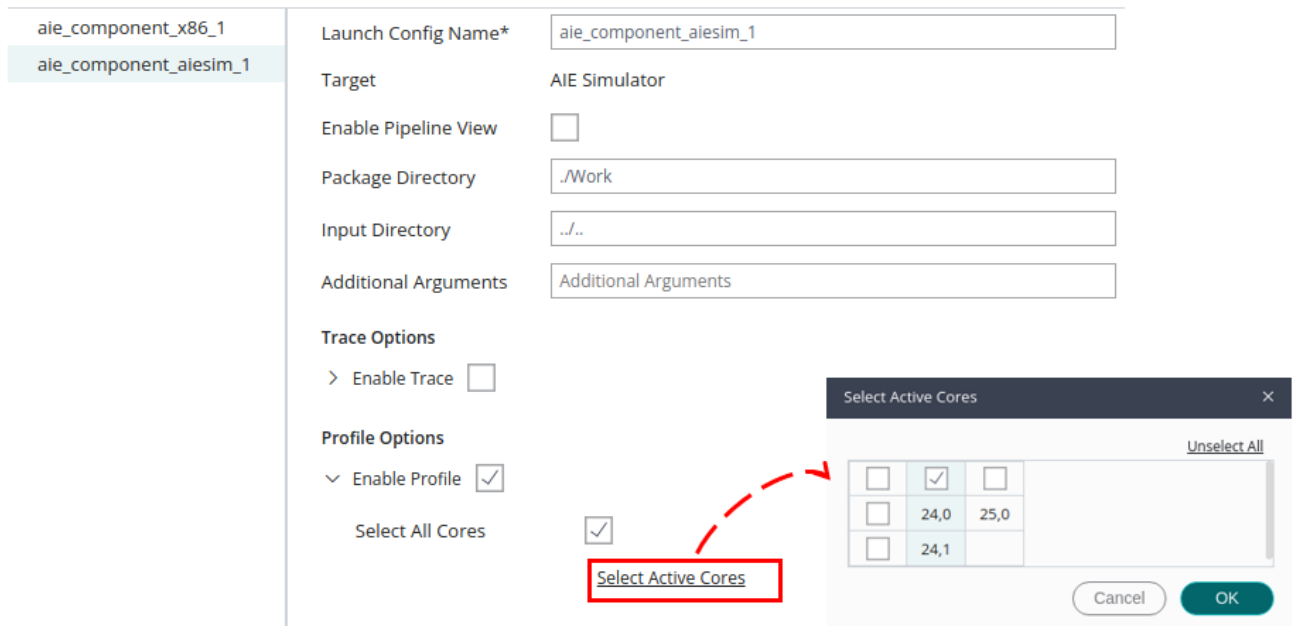
Data flows into and out of an AI Engine kernel through access windows, or a stream interface. During debug you might want to see the value of these data access windows as data is passing through the kernels. In the case of data windows, the Vitis IDE debug environment provides methods to view and access the data as described in [Viewing Data from Buffer Port Interfaces](#). In the case of stream interface connections, it is recommended to add `printf()` statements to your code to let you examine the data passing through the kernel.

★ **IMPORTANT!** Adding `printf()` statements to the code suppresses compiler optimizations, and results in a larger kernel executable program that might not fit into the available memory of the AI Engine processor.

When adding the `printf()` statement in your kernel code you must select **Enable Profile** in the launch configuration for Debug as shown in the following figure. You can also select the cores for which profiling should be enabled. By default, all cores are selected.

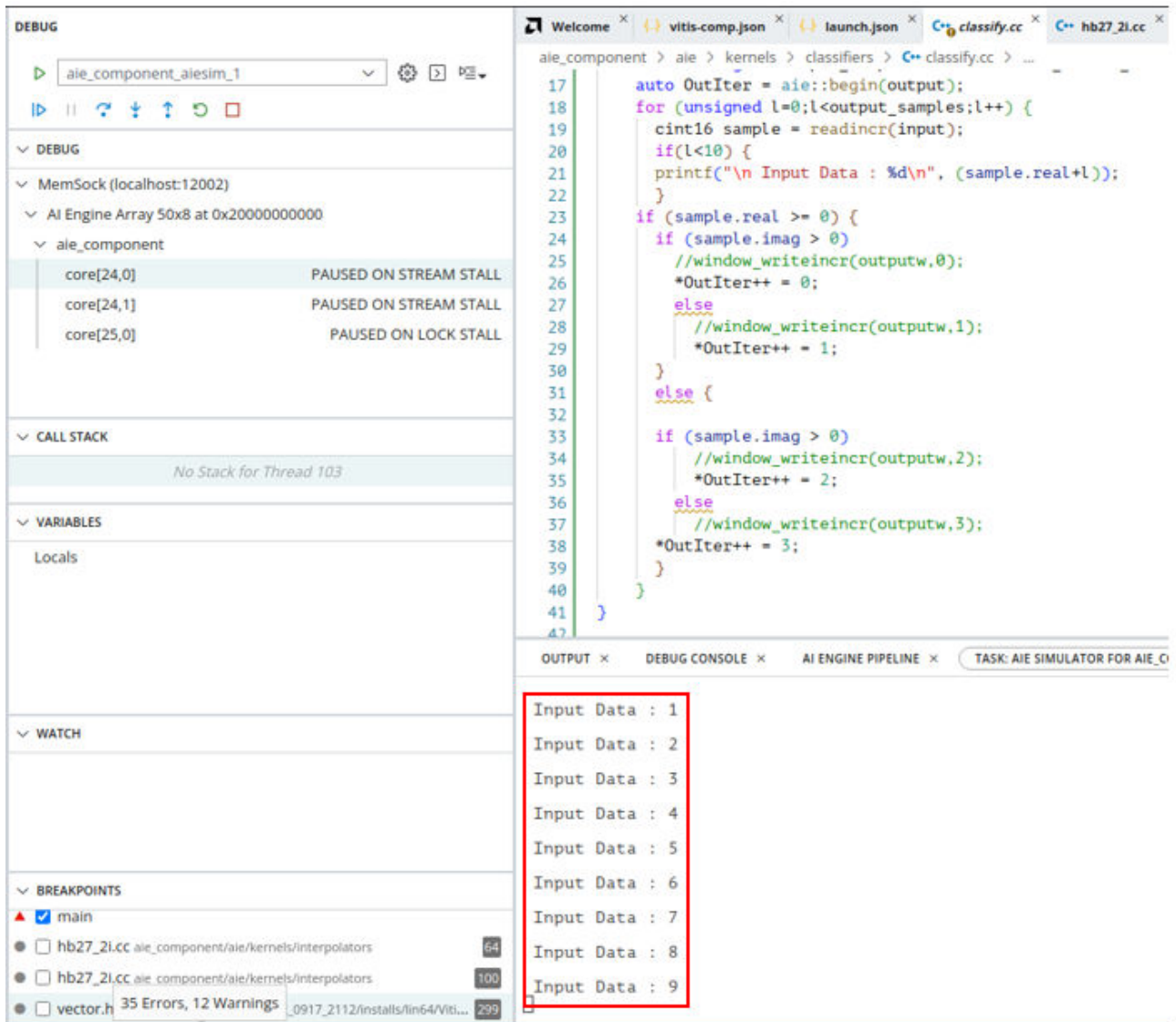
When adding the `printf()` statement in your kernel code you must also add the `--profile` option in the Run Configurations or Debug Configurations dialog box in the Vitis IDE. Add `--profile` to the Arguments tab of the Debug Configuration, along with whatever other options are already specified, as shown in the following figure.

Figure 94: Enable Profiling



Adding the `printf()` statements to your source code, results in the output of streaming data as it is processed by the kernel. The following figure shows an example of such output in the console window. This provides visibility to capture and debug the dataflow through streaming interfaces.

Figure 95: Printing Data from Stream Interfaces



AI Engine Pipeline View

The AI Engine Pipeline view in the Vitis unified IDE allows you to correlate instructions executed in a specific clock cycle with the labels in the Disassembly view. The underlying AI Engine pipeline is exposed in debug mode using the pipeline view. The Vitis unified IDE supports viewing pipeline view for all the kernels in your graph.

To enable the Pipeline view select **Enable Pipeline View** from the Debug launch configuration after the project has been built successfully. The Pipeline view is available with the AI Engine simulator only.

Figure 96: Enable Profiling

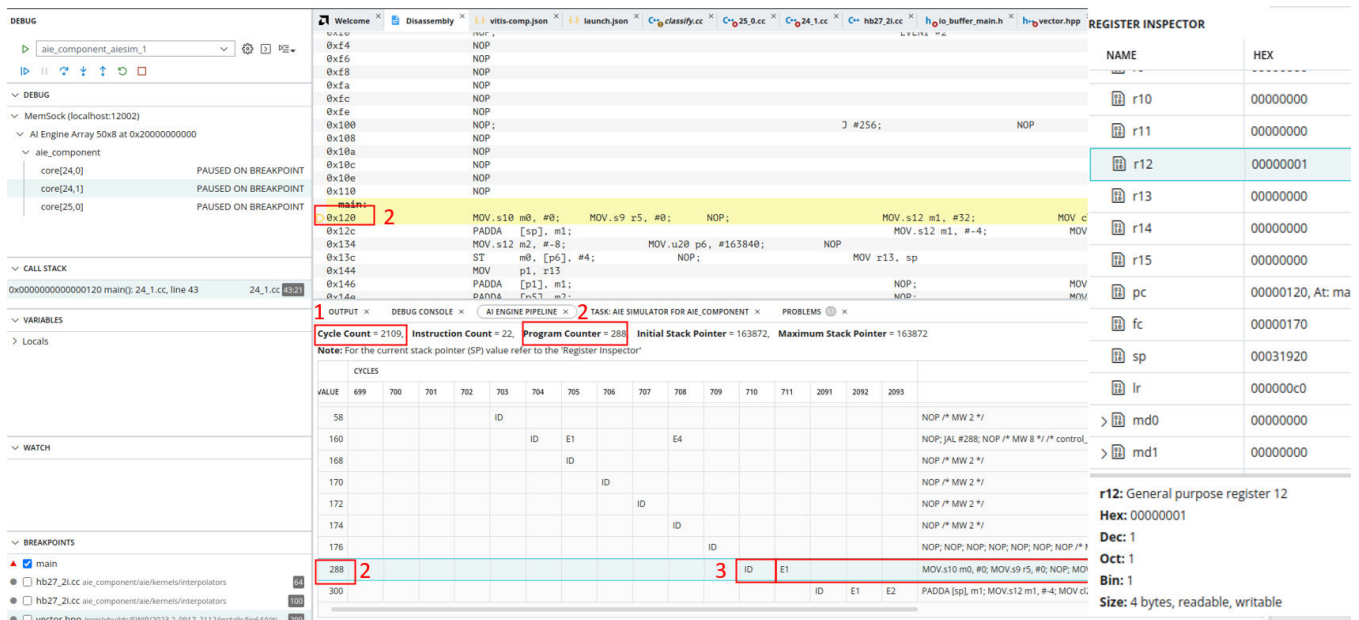
ale_component_x86_1	Launch Config Name*	ale_component_alesim_1
ale_component_alesim_1	Target	AIE Simulator
	Enable Pipeline View	☒
	Package Directory	./Work
	Input Directory	../..
	Additional Arguments	Additional Arguments
	Trace Options	
	> Enable Trace	☐
	Profile Options	
	∨ Enable Profile	☐
	Select All Cores	☒
	Select Active Cores	



TIP: When *Enable Profile* is selected the *Pipeline* view is enabled as well.

Click on **Debug** to start debugging the application. Note the Pipeline view shows up automatically in the Debugger Console window.

Figure 97: Pipeline View



Runtime statistics of the kernel in the pipeline view are highlighted in the previous figure.

1. AI Engine kernel cycle count
2. Program counter
3. ID = Instruction decode
4. E1-E7 are the AI Engine execution stages. Almost all operations in the scalar unit are scheduled in E1 stage of the pipeline besides non-linear operations. The vector unit scheduling spans from the ID stage to the E6 stage. Address Generation Units (AGUs) span over two pipeline stages. The address is ready in the E2 stage of the pipeline. For load units, the data will be available in the AI Engine from the memory module in the E7 stage. For the store unit, the data will be sent out from the AI Engine to the memory module in the E5 or E6 stage of the pipeline, depending on the type of instruction.
5. **Step Over** in the debug view is shown in the following figure. As shown the previous instruction, this is run in E1 stage. The result is updated in Register Inspector.

Figure 98: Stepping Over with Pipeline View

The screenshot displays the Vitis Unified IDE interface during a debug session. The main window shows the AI Engine Pipeline View, which includes a Disassembly window at the top and a Pipeline View table below. The Disassembly window shows the assembly code for the current instruction, with the instruction being stepped over highlighted. The Pipeline View table shows the execution stages (ID, E1, E2, E3, E4, E5, E6, E7) for various instructions. The Register Inspector on the right shows the current state of registers, including the Program Counter (PC) and Stack Pointer (SP).

NAME	HEX	DESCR
r0	00000000	Gener
r1	00000100	Gener
r2	00000000	Gener
r3	00000000	Gener
r4	00000000	Gener
r5	00000000	Gener
r6	00000000	Gener
r7	00000000	Gener
r8	00000000	Gener
r9	00000000	Gener
r10	00000000	Gener
r11	00000000	Gener
r12	00000000	Gener
r13	00000000	Gener
r14	00000000	Gener
r15	00032000	Gener
pc	0000012c	At: main() + 0x...
fc	00000180	Fetch
sp	00031920	Stack

VALUE	876	877	878	879	880	881	882	883	884	885	886	887	283273	283274	283275	283276	283278
56			ID														NOP /# MW 2 *
58			ID														NOP /# MW 2 *
160				ID	E1				E4								NOP; JAL #288;
168				ID													NOP /# MW 2 *
170					ID												NOP /# MW 2 *
172						ID											NOP /# MW 2 *
174							ID										NOP /# MW 2 *
176								ID									NOP; NOP; NOP;
288									ID	E1							MOV.s10 r1, #2
300										ID	E1	E2					PADDA [sp], m0;
312												ID	E1				NOP; MOV.s9 r

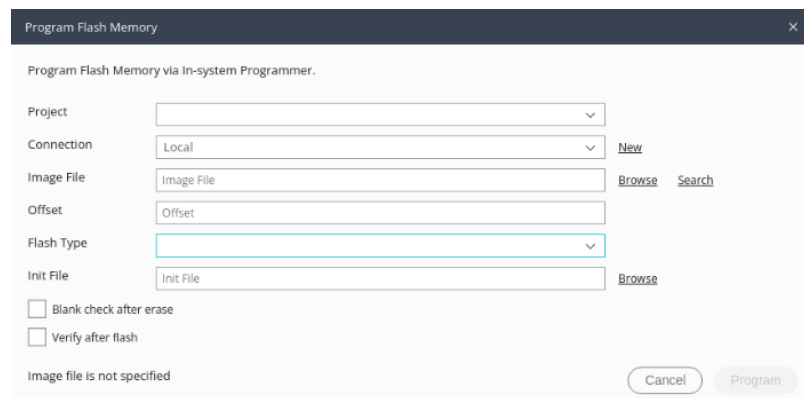
Programming Device and Flash Memory

The Vitis IDE has a feature for programming the board/part using JTAG, allowing you to bypass the need to copy the contents of `sd_card` to an actual SD card to boot on the board.

Programming Flash

After successfully building a System project for hardware, select **Program Flash** from the Vitis menu in the main menu. You should see a dialog box similar to the following:

Figure 99: Program Flash Memory Dialog Box



In this screen specify the **Image File**, which is generally the `BOOT.BIN`, any offset (if necessary for the flash memory), in addition to the **Init File**. The Init File is the partial PDI from the platform you are targeting. To make sure the flash is programmed properly, check the **Verify after flash** check box to confirm it was programmed properly. If the flash memory already has existing data, you can do a more secure erase by checking the **Blank check after erase** check box to make sure it is fully erased before programming.

Make sure that the **Packaging options** in the **System Project Settings** window has -- `package.boot_mode=qspi` set.

Programming the Device

If you do not want to program the flash memory and only want to program the device through JTAG, there is an option under the Vitis menu to select **Program Device**.

Vitis Interactive Python Shell

As described in [Launching the Vitis Unified IDE](#), the Vitis Unified IDE supports an interactive mode that lets you enter commands through the command-line interface (CLI) of a Python command shell. You can launch the tool in interactive mode as follows:

```
vitis -i
```

The Vitis command-line prompt is displayed:

```
Vitis [1]:
```

Note: Type `help()` or `help(vitis)` from the interactive command prompt to explore the available command modules.

As described in [Running Scripts in Batch Mode](#), CLI examples can be found at `<Vitis_Installation_Dir>/cli/examples` directory. There are also examples of command-line use provided in [Platform Component Command Line Flow](#).

The following topics describe some of the features of the Vitis Python command shell.

Auto-Completion of Python Statements

You can provide the starting few letters of a method and double tab to get the list of matching methods.

Shell Basic Commands

Following are the shell commands used in the next generation Vitis Python CLI:

- **cd:** Used to change the current working directory.

```
Vitis [1]: cd /tmp/workspace  
/tmp/workspace
```

- **pwd:** Used to display the current working directory.

```
Vitis [2]: pwd
Out[2]: '/tmp/workspace'
```

- **ls:** Used to list the contents of the current working directory.

```
Vitis [3]: ls
logs/
```

Commands to Edit a File

Following are the commands to edit and execute code in file:

- **edit:** Used to bring up the default editor to edit the file.

```
Vitis [1]: edit <filename>.py
```

- **Launch a specific editor as external command:** Add an exclamation mark prior to the editor executable to launch the editor as external commands.

```
Vitis [1]: !vim <filename>.py
Vitis [1]: !nano <filename>.py
```

- **Multi-line editing in the CLI:** In the new Vitis CLI, you can edit and update the multi-line code using arrow keys.

Commands to Run Python Scripts

Following are the commands to edit and execute code in a file:

- **Run Python code in the next generation Vitis IDE CLI:** Run the named file

```
Vitis [1]: run test.py
```

Option `-t` displaces the timing information at the end of run

```
Vitis [1]: run -t test.py
```

- **Run Python code with the next generation Vitis IDE executable in batch mode:** `vitis` executable can run Python script in batch mode with `-source` option.

```
$ vitis -source $XILINX_VITIS/cli/examples/dc_use_case_1.py
```

Setting and Listing Environment Variables

Following are the commands to set and list environment variables:

- **env:** Used to list all environment variables/values
- **env var:** Used to get value for var
- **env var val:** Used to set value for var
- **env var=val:** Used to set value for var
- **env var=\$val:** Used to set value for var, using python expansion if possible

```
Vitis [1]: env XILINX_VITIS
Out[1]: '/tools/Xilinx/Vitis/2023.2'
```

Command to Display History

Use `history` command to print recent input history.

```
Vitis [1]: ls
vadd/ logs/

Vitis [2]: history
ls
history
```

By default, input history is printed without the line numbers so that it can be pasted directly into an editor. Use `-n` to show line numbers.

```
Vitis [3]: history -n 1
1: ls
```

By default, the input history from the current session is displayed. Ranges of the history can be indicated using the following syntax:

```
Vitis [4]: history -n 1-2
1: ls
2: history
```

Command to Search the History

In the next generation Vitis IDE, you can search the history using the following commands:

- **Ctrl-p (or the up arrow key):** Used to access previous command in the history.
- **Ctrl-n (or the down arrow key):** Used to access the next command in the history.
- **Ctrl-r:** Used to reverse search through the command history.

Command to Display Execution Time

The following command is used to display the next generation Vitis IDE Python methods or expression execution time:

```
time
```

The CPU and wall clock times are printed, and the value of the expression (if any) is returned.

Note: Win 32, system time is always reported as 0, because it cannot be measured.

```
Vitis [1]: time client=vitisng.create_client()  
Vitis Server started on port '59936'.  
CPU times: user 71 ms, sys: 12.8 ms, total: 83.8 ms  
Wall time: 17.2 s
```

Command to Clear the Session

The following command is used to clear the session:

```
clear
```

Library Function References

The new Vitis IDE can create, configure, and build a System project from the command-line using the `vitis` python library. Launch the Vitis tool in interactive mode using the following command:

```
vitis -i
```


Use the following commands while in the interactive mode to get help for the `vitis` library:

```
Vitis [1]: help(vitis)
Vitis [2]: help(vitis.create_client())
```

Working with the Analysis View (Vitis Analyzer)

The Analysis view replaces the AMD Vitis™ Analyzer tool to provide an integrated view of the build and run summaries generated by the Vitis tools. The integrated analysis view provides ready access to view the results of a specific compilation step, or run or debug results, letting you quickly move between development and analysis.

You can open and use Vitis Analyzer in a number of different ways.

1. Use Vitis Analyzer to review specific summary files from a variety of projects by opening the tool and summary directly:
 - Open the tool using the `vitis_analyzer` command (or `vitis -a`) and then select **Open Summary** to view the reports.
 - Open the tool with a specified summary file using `vitis_analyzer <summary_file>` (or `vitis -a <summary>`).



TIP: You can also specify a folder, such as a workspace directory, and the tool will scan the sub-folders for summary reports to open.

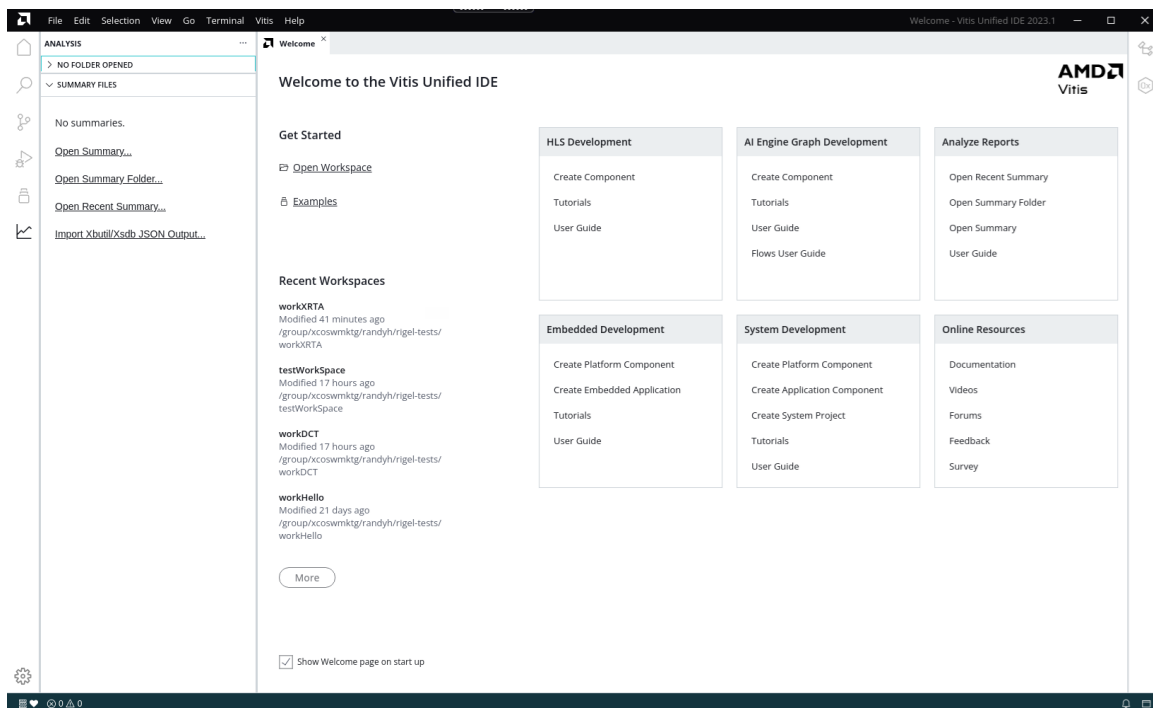
2. Use Vitis Analyzer to review the reports generated for the current workspace in the IDE by selecting the Analysis view to see the hierarchy of reports, or by selecting the reports directly from the Flow Navigator for the active component.
3. Combine these two methods, by using the Analysis view to see the hierarchy of reports generated for the open workspace, and open Summary files independently.



TIP: When you open `vitis_analyzer` for the first time, you are reminded that you are launching the new Vitis analyzer tool. You can also launch the legacy tool by using the `vitis_analyzer --classic` command. You can disable that message for future iterations.

When the Vitis Analyzer is opened from the command line, the tool displays a Welcome screen as shown in the following figure. The Analyze Reports box shows commands specific to the Analysis view of the tool.

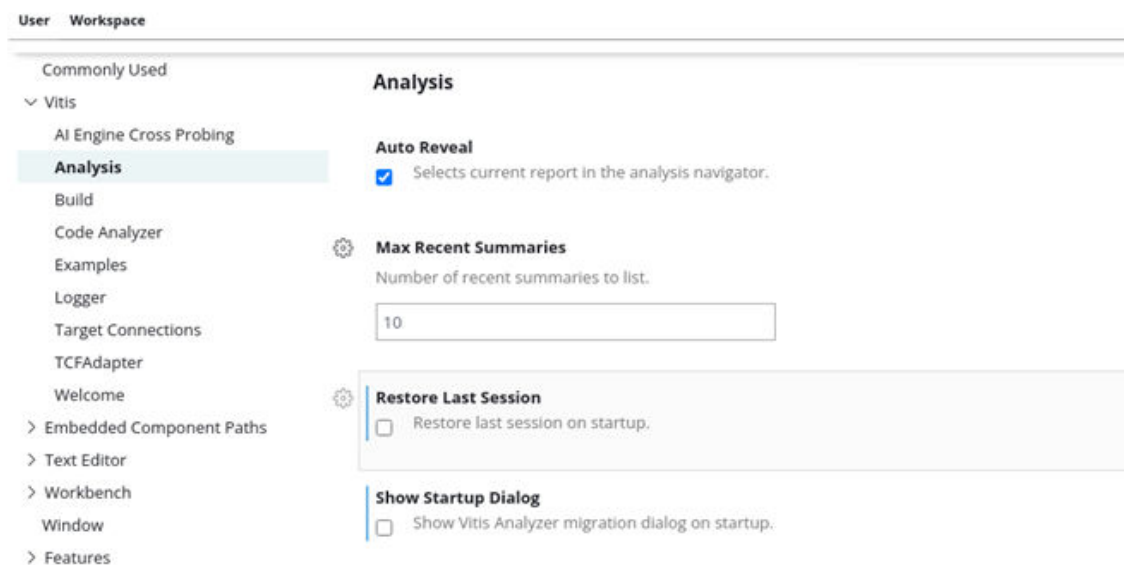
Figure 100: Analysis View



Configuring the Analysis View

Select **File** → **Preferences** → **Open Settings (UI)** to display the preference settings for the Vitis IDE, including the Analysis view as shown below.

Figure 101: Analysis Preferences



- **Auto Reveal:** Highlights the current report in the Analysis navigator. This lets you quickly locate the report you are viewing in the hierarchy of reports. The default is enabled.
- **Max Recent Summaries:** Specifies the number of Summaries to list when presenting Recently Opened Summaries. The default is 10.
- **Restore Last Session:** Opens the Vitis IDE with the summaries opened in the prior session of the tool.
- **Show Startup Dialog:** Displays a dialog box with a message regarding the new features or state of the tool. This dialog box can be dismissed after viewing, and can be re-enabled using this option.

Some additional Preferences that can have an effect on the Analysis view is the Commonly Used Preferences include:

- **Auto Save Delay:** Specifies the delay for the auto-save feature. This lets the tool save any modified files as the work is proceeding.



TIP: *Auto Save is enabled on the File menu.*

- **Font Size:** Specifies the font size for the code editor and log file viewers. The default is 14.
- **Font Family:** Specifies the font to use in reports.

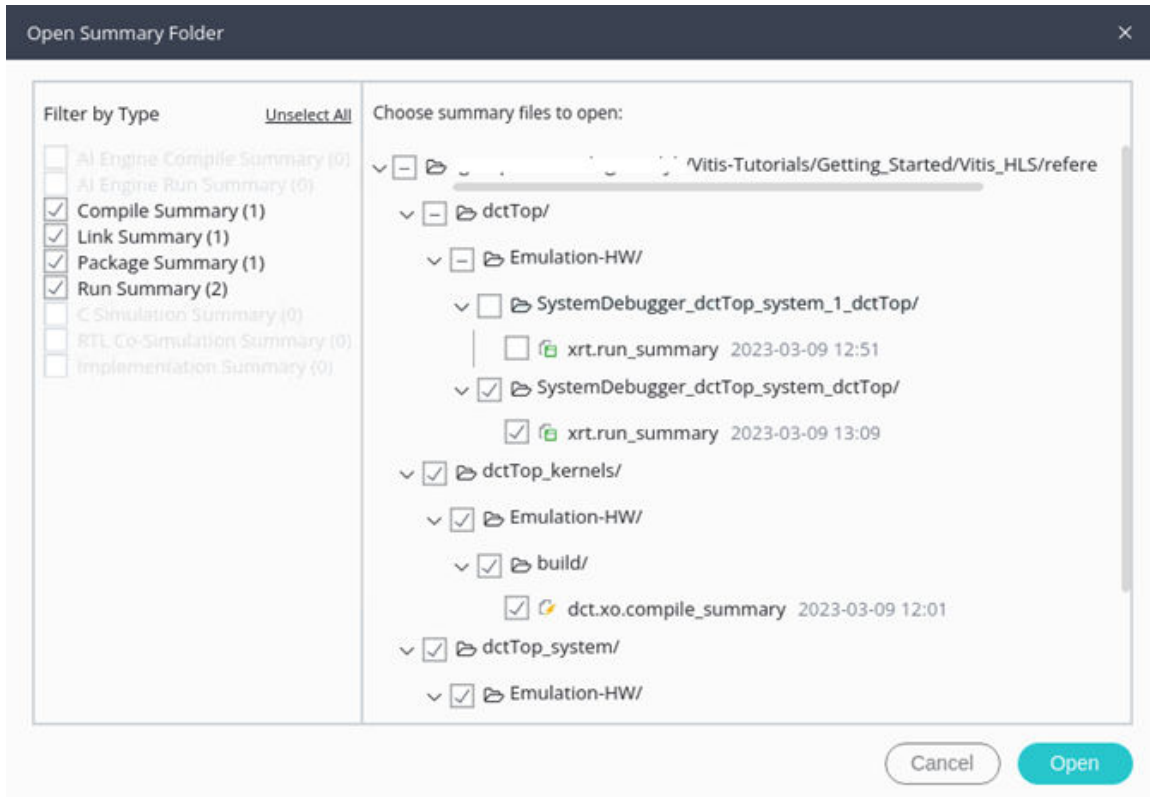
Open Summary Reports

Generally, the Summary reports provide a great overview of the specific steps in building and profiling the application to get a good view of where the application is with regard to performance and optimization.

- For individual components review the Compile Summary.
- For Application and device binary (`xclbin`) refer to the Link Summary, which also loads the Compile Summaries for linked components.
- For embedded processor and AMD Versal™ AI Engine systems, review the Package Summary.
- For performance profiling and event trace data related to the application execution, start with the Run Summary.

The Vitis analyzer offers the ability to open a folder, such as a workspace, using Open Summary Folder from the Welcome view, or from the command line when launching the tool. The tool examines the folder you specify and identifies the various build and run summaries included through the file hierarchy. The Open Summary Folder dialog box opens as shown below.

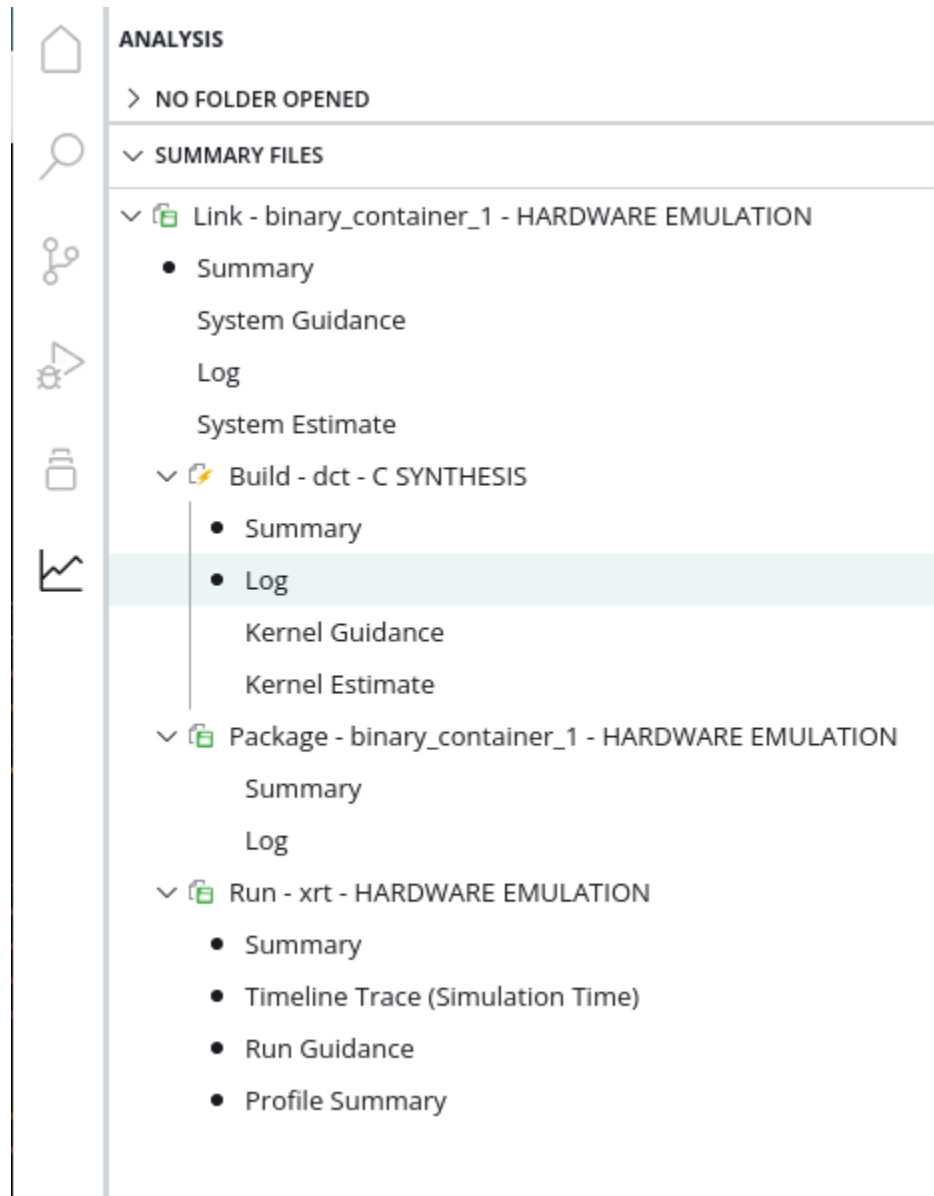
Figure 102: Open Summary Folder



The figure above shows the results of the Open Summary Folder when used on a Vitis IDE workspace. Filter by Type highlights the various types of reports found, and lets you enable or disable the reports of interest. Choose summary files to open lets you select specific files to open. In this way, you are viewing the type and specific reports of interest.

Vitis analyzer displays the various Summary reports in the Analysis view. Related Summaries are grouped into a hierarchy, so that the Link Summary for an Application includes the Compile Summaries and Run Summaries for the various components, in addition to any Run Summary for the Application. Separate Applications, or unrelated Components create separate hierarchies as shown below.

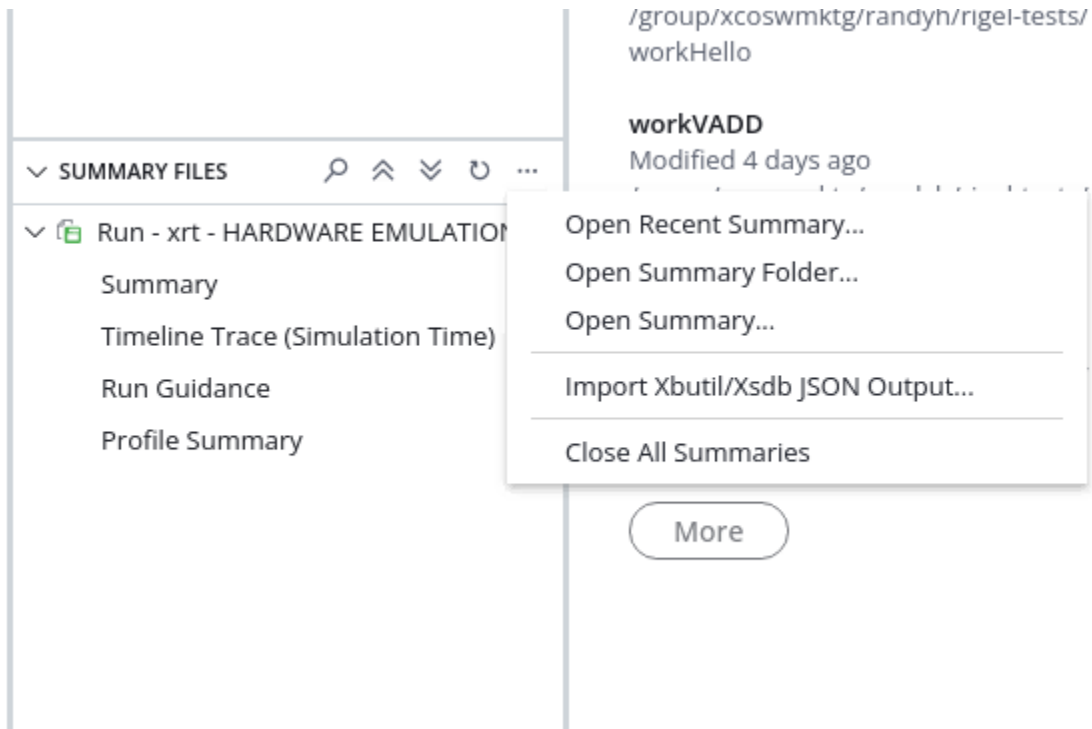
Figure 103: Analysis View Hierarchy



The Summary report provides access to the individual reports that are collected into the summary file, such as the Kernel Estimate, Operation Trace, AI Engine Trace, Timeline Trace, System Estimate, Log, and Timing Summary reports. Refer to [Profiling the Application](#) for more information on the individual reports generated by the build and run processes.

The Summary Files section menu also includes a drop down menu that includes the command Import Xbutil/Xsdb JSON Output as shown in the following figure. This command lets you import a Json file generated by the `xbutil` command or `xsdb` commands to display in the Analysis view.

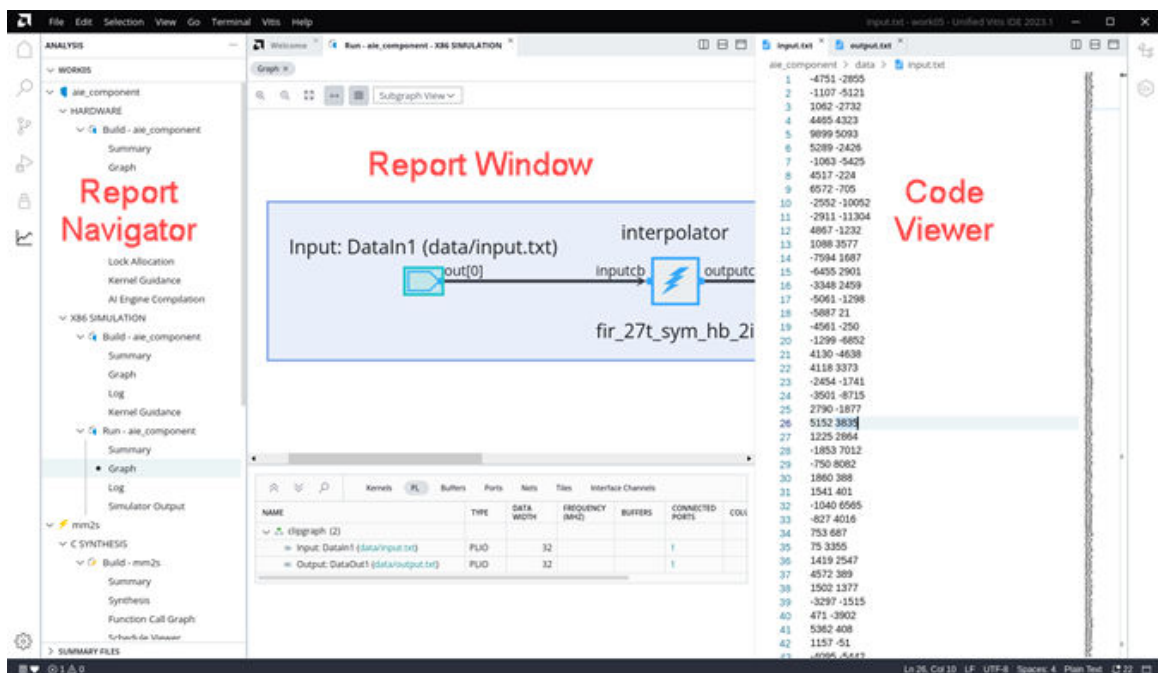
Figure 104: Analysis View Menu



Analysis View Window Manager

As shown below, the Analysis view can be arranged into three views as shown below. The different views can be opened, closed, and rearranged as needed.

Figure 105: Analysis View



- **Report Navigator:** On the left side, this view lists all open summary files and associated reports. The Analysis view displays the hierarchy of reports generated from build and run of components in the open workspace of the Vitis IDE, or displays a selection of Summary files separately opened. You can use this view to quickly find and open a report.

When you click any file in Report Navigator, it opens as a new tab in the Report Window. Opening a file adds a black dot next to the report name in Report Navigator to indicate the report is already open. Selecting an open report will simply bring it to the front in the Report Window.

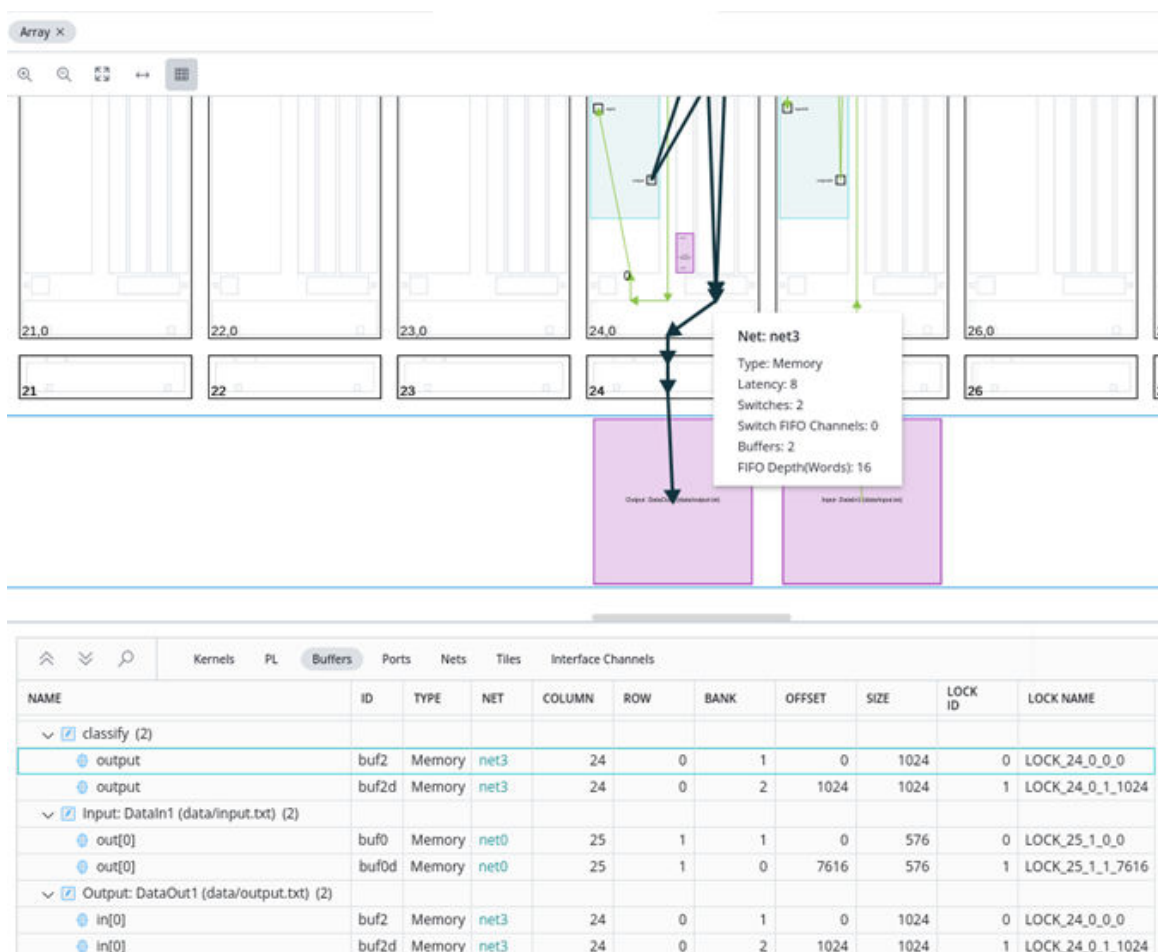
- **Report Window:** The center area displays the contents of the summary files and open reports. You can have multiple reports open in the Reports view, and quickly change from one report to another by selecting the window tab at the top of the view.

All the reports related to the Compile Summary, Link Summary, or Run Summary are grouped together within a single container.

- **Code Viewer:** The optional Code Viewer is opened on the right side of the workspace. This lets you view and edit source code, based on feedback from the various reports. You can open the Source Code window by selecting a link in another report such as the Graph report for instance.

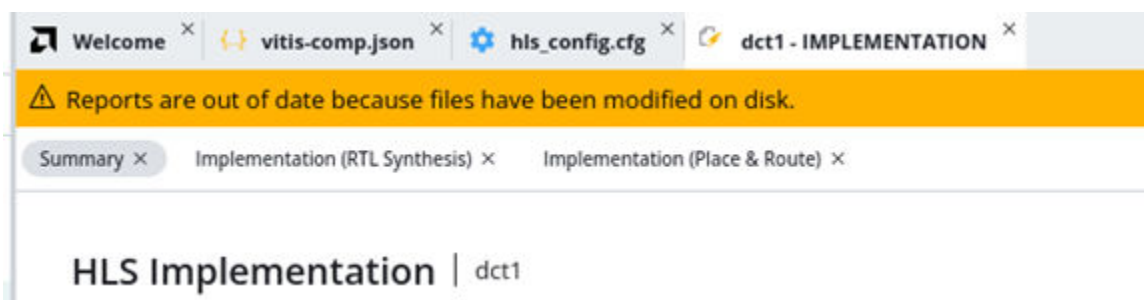
The Analysis view supports cross-probing between reports, such as within the Graph or Array diagrams, and from the Guidance report to other reports. Select the link in a report as shown below, and it will highlight the content in another open report, or open a file in the Code Viewer.

Figure 106: Cross-Probe Between Reports



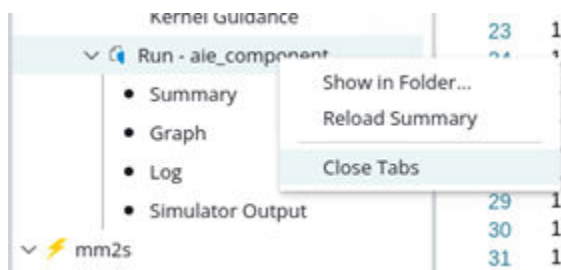
Reports that are currently opened in Vitis analyzer will display an "out-of-date" banner if the summary files or reports have been updated on the disk, due to recompiling or rerunning the application for instance. You can keep working in the open report, or reload the updated files.

Figure 107: Out-of-Date Reports



To close open reports you can right-click the report in the Report Navigator and select **Close Tab**. You can also right-click a Summary file, as shown below and select **Close Tabs** to close all open reports associated with that Summary file.

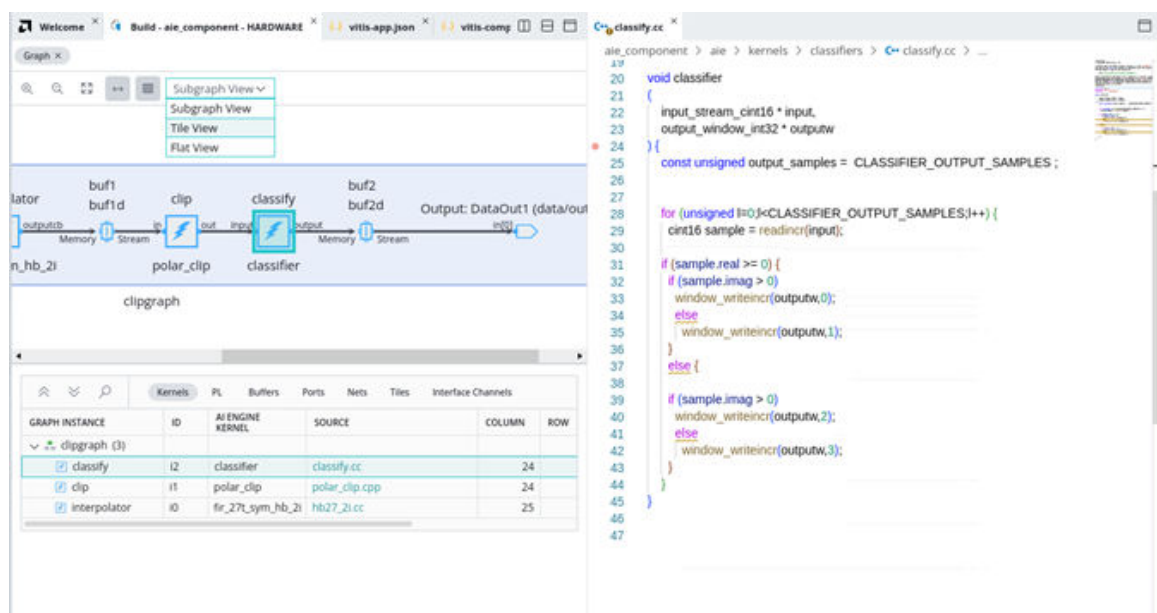
Figure 108: Close Tabs



Viewing AI Engine Graphs and Arrays

The AI Engine Graph and Array diagrams provide a quick overview of the structure of the ADF graph application implemented in the AI Engine tiles.

Figure 109: AI Engine Graph Application



The AI Engine Graph view shows the connectivity of the ADF graph as seen by the AI Engine compiler. The canvas shows nodes representing kernels, kernel arguments, memory buffers, and primary inputs and outputs, with edges drawn to represent the connectivity between these elements. Below the graphical view is a set of tables containing all of the objects drawn in the graphical view: Kernels, Buffers, Ports, Nets, etc. Selecting an element in one of these tables will cross-probe to the graphical view and vice versa.

Figure 110: AI Engine Array Diagram



The Array view shows how the ADF graph is spatially placed on the AI Engine tile array. The array canvas contains all the core and memory components of the array. Kernels are drawn in the core where they are programmed, and connectivity between cores and/or memory are shown with connectivity lines. There is an abstract representation of the PL area as needed to model PL cores and interfaces to the programmable logic region.

To the right of the canvas, a Settings panel that can be used to customize the view of the array, by letting you show or hide elements of the resources and/or tile array.

Importing JSON Output Files

You can output a single snapshot of the AI Engine status to a JSON file at any time after the device has been loaded. This is a static snapshot of the AI Engine running status, along with the events that happened before. Refer to the chapter on Manual AI Engine Status Output in the *AI Engine Tools and Flows User Guide* ([UG1076](#)) for more information. An example command is:

```
xbutil examine -r aie -d 0 -f json -o aie_status_xbutil.json
```

The `aie_status_xbutil.json` file can be imported into the Analysis view using the following specific steps.

1. In the Summary Files section menu of the Analysis view select **Import Xbutil/Xsdb JSON Output**.
2. In the displayed dialog box, set the following options.
 - **Xbutil/Xsdb JSON Output File:** Select the JSON file that was manually generated with the `xbutil` command. For example, select the file `aie_status_xbutil.json`.
 - **AI Engine Compile Summary:** Select the AI Engine compile summary file, for example `./Work/graph.aiecompile_summary`.
 - **Save Run Summary:** The run summary file to be written. The run summary can be used to reload the analysis next time by **Analysis → Open Summary** or **File → Open Recent → Summary**.

The Graph and Array views are shown in the Analysis view.



TIP: If you load the JSON file before loading the compile `workdir`, the information will not be displayed because there is no graph report. An imported JSON file overrides any other JSON file previously set.

Using Vitis Embedded Platforms

This section contains the following chapters:

- [Vitis Embedded Platforms](#)
- [Using Vitis Embedded Platforms](#)
- [Creating Embedded Platforms in Vitis](#)

Vitis Embedded Platforms

Introduction

The AMD Vitis™ software platform is an environment for creating embedded software and accelerated applications on heterogeneous platforms based on FPGAs, AMD Versal™ and AMD Versal™ adaptive SoC devices. The Vitis unified software platform provides a Platform+Kernel structure to help developers focus on the applications. The decoupling of platform and kernel helps make the platform reusable with multiple kinds of kernels and vice versa.

AMD provides pre-built platforms for AMD Alveo™ and embedded evaluation boards. You can also create custom embedded platforms or customize the AMD provided embedded platforms. Your custom platform can be defined to support the Vitis integrated flow, or the Vitis Export to Vivado flow as described in [Linking the System](#).

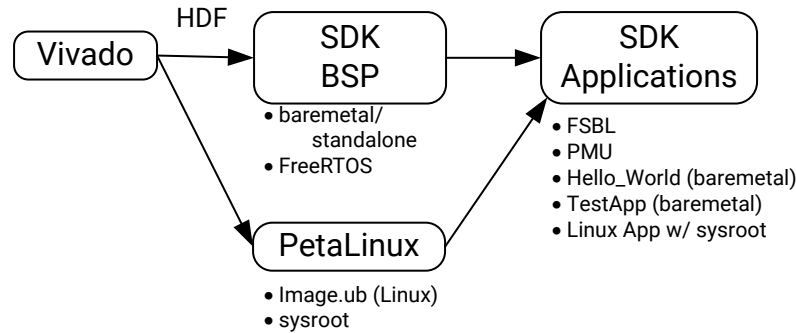
For instructions on installing Embedded Platforms, see [Installing Embedded Platforms](#) in [Vitis Software Platform Installation](#).

Platform Types

The Vitis target platforms can be customized with unique hardware and software components. There are two general types of platforms: fixed platforms and extensible platforms. Fixed platforms support embedded software development and are a direct analog to the hardware definition file that was previously used for software development with the AMD SDK tool. Extensible platforms support the application acceleration development flow, and includes hardware for supporting acceleration kernels, controlling AI Engine for AMD Versal™ adaptive SoC, and software for a target running Linux and the Xilinx Runtime (XRT) library. For more information on the XRT library, see <https://github.com/Xilinx/XRT>.

The following figure shows the traditional SDK flow for embedded software application development. An AMD Hardware Design File (HDF) is exported from the AMD Vivado™ Design Suite. It is used by SDK for board support package (BSP) generation and creating software applications that apply the BSP.

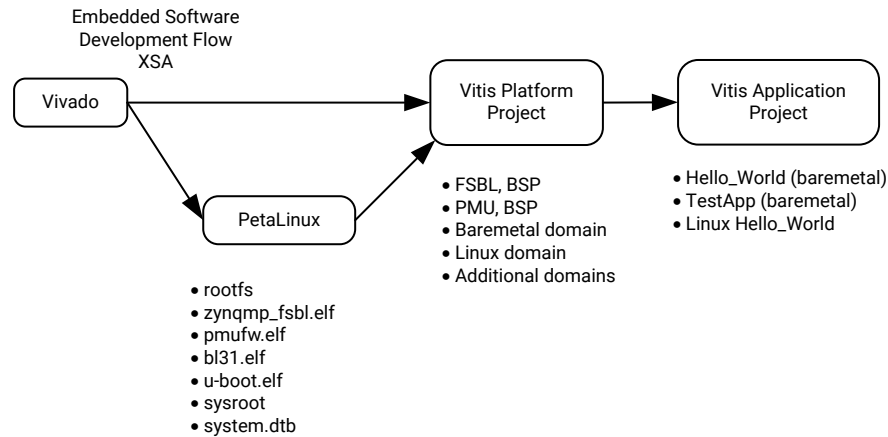
Figure 111: Pre-2019.2 SDK Flow



X233443-102119

The following figure shows the Vitis embedded software development flow that replaces the SDK flow beginning with the 2019.2 release. The hardware specification is now contained in the Xilinx Support Archive (XSA) and is exported from a Vivado design, but is formatted differently from HDF and uses the `.xsa` file name extension.

Figure 112: Vitis Embedded Software Development Flow

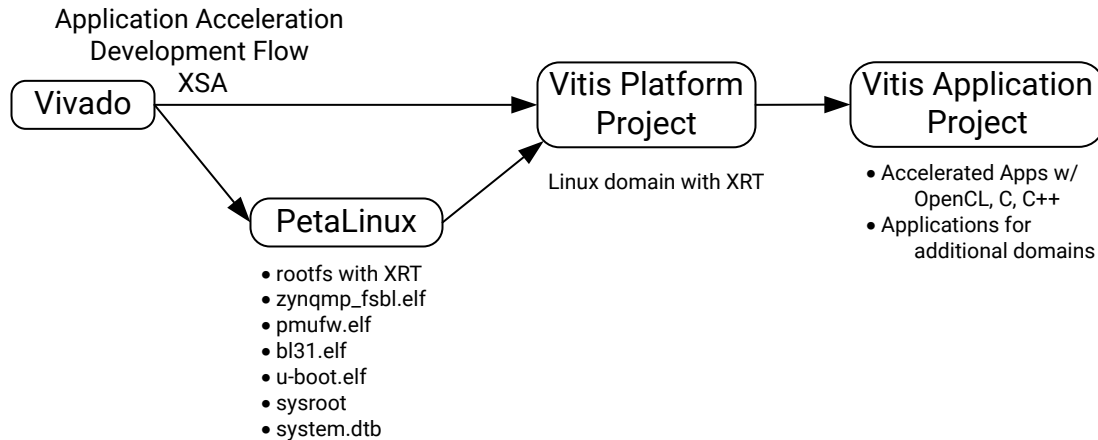


X27497-120522

The Vitis core tools create a platform, BSP, and software boot components such as the FSBL and PMU firmware for the fixed-XSA, and are associated with the Vitis platform. Software applications targeting the fixed-platform can be developed with the Vitis Embedded Software Development flow. Fixed-platforms are not limited to Linux and the XRT library, but can also target processor domains running Baremetal and RTOS operating systems as well. See the *Vitis Unified Software Platform Documentation: Embedded Software Development* ([UG1400](#)) for more information.

In the Vitis Application Acceleration Development flow, the Vivado Design Suite is used to create an extensible-XSA, containing additional IP blocks and metadata to support kernel connectivity. The following figure shows the acceleration software development flow.

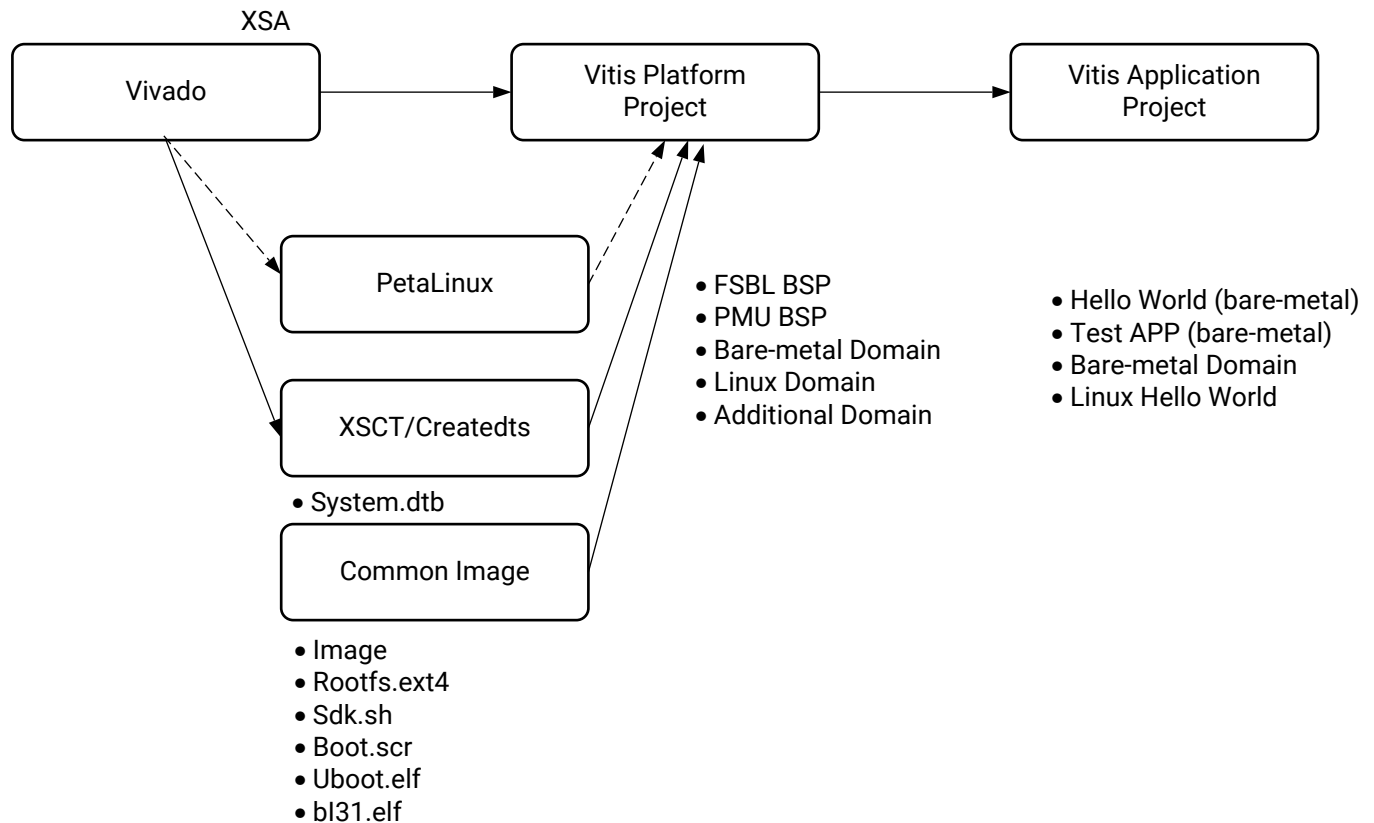
Figure 113: Vitis Acceleration Kernel Flow



X23345-062320

The following figure shows the Vitis embedded software development flow that use common image and the `createdts` command to generate software components and leverage PetaLinux as an alternative beginning with the 2022.1 release.

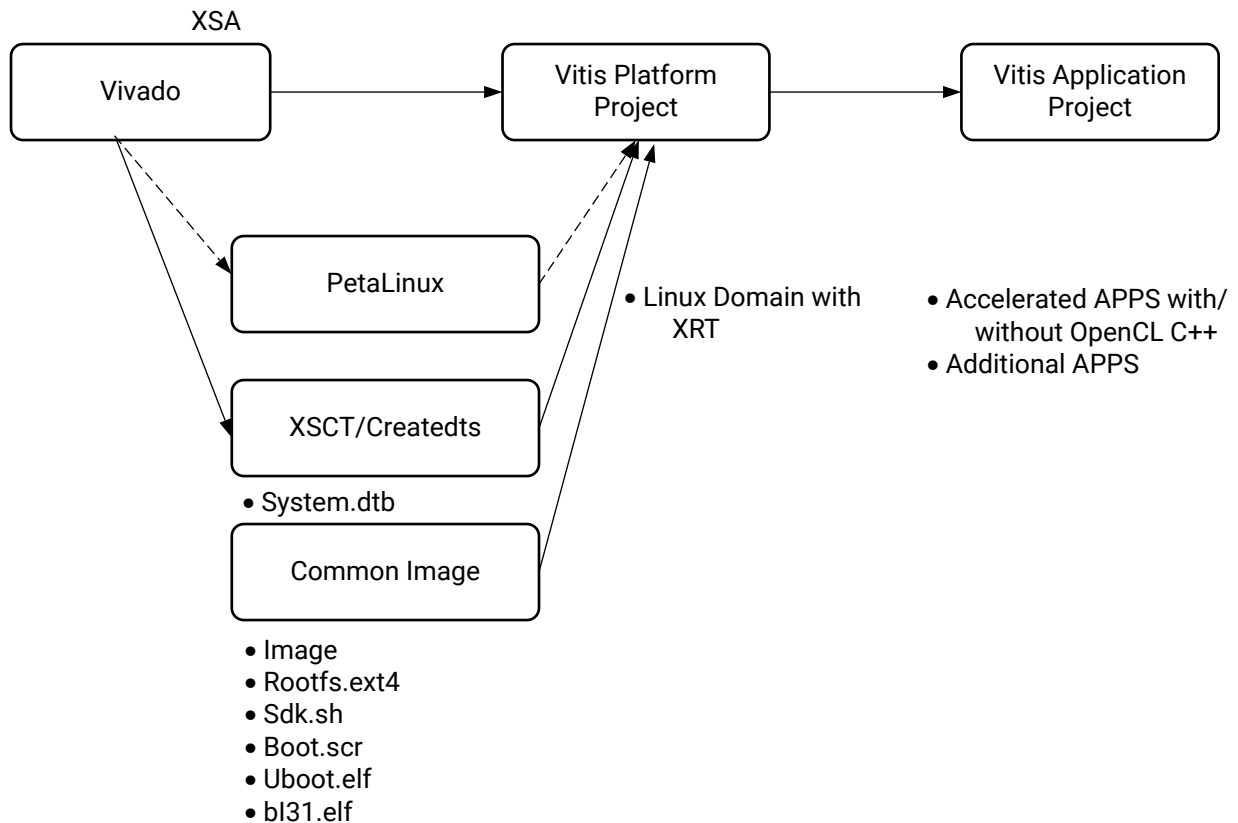
Figure 114: Vitis Embedded Software Development Flow



X26745-060222

In the Vitis Application Acceleration Development flow, the Vivado Design Suite is also used to generate and write an extensible-XSA, containing additional IP blocks and metadata to support kernel connectivity. You start with a common image and `createdts` to generate the DTS for software components and use PetaLinux as an alternative. The following figure shows the acceleration software development flow beginning with the 2022.1 release.

Figure 115: Vitis Acceleration Kernel Flow

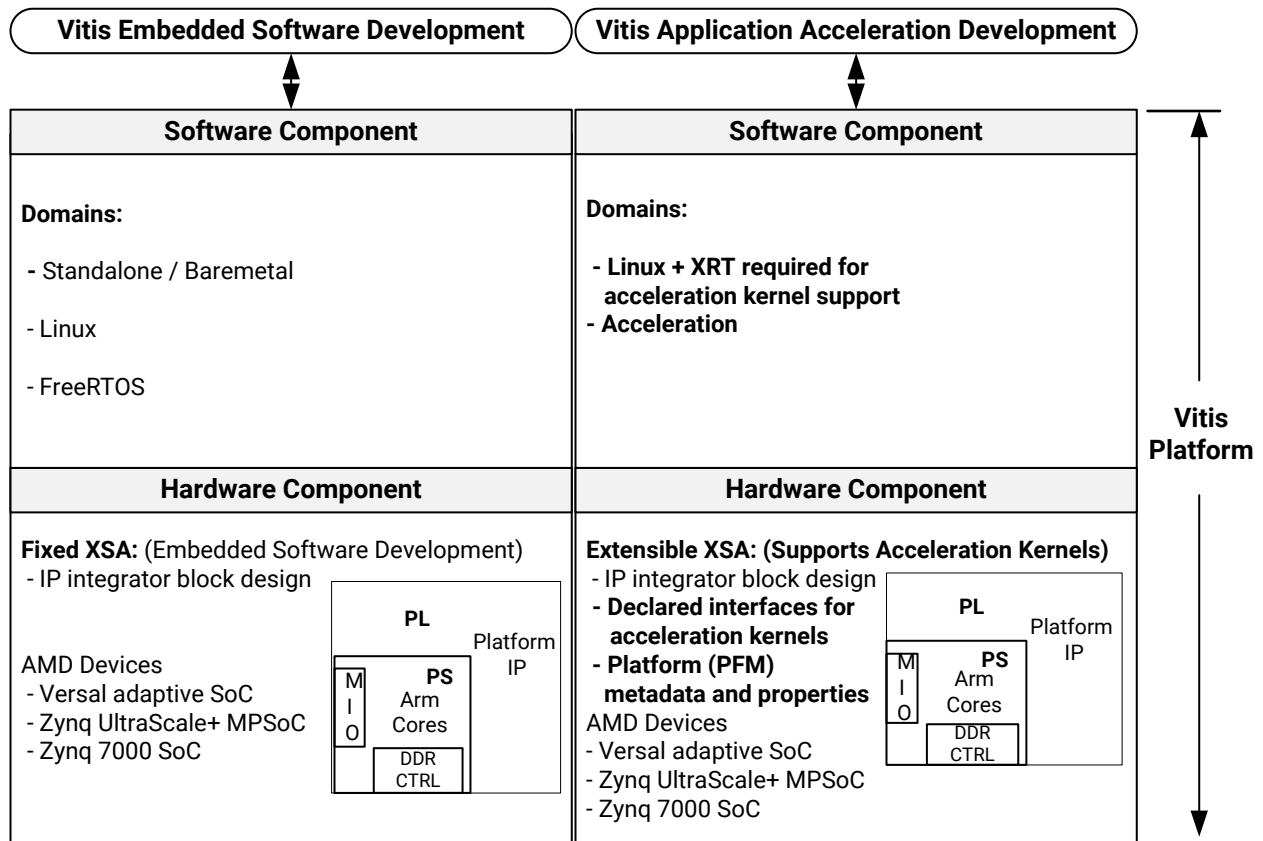


X26764-060622

The Vitis core tool supports application development in multiple languages (OpenCL™, C, C++) but the applications must target a Vitis platform. A target platform consists of hardware and software components as shown in the following figure. The target platform view on the left side of the page is for the Vitis embedded software development flow, whereas the right side of page shows a platform that supports acceleration kernels. The differences include acceleration kernel requirements of a target with Linux + XRT, metadata, and kernel interface declarations.

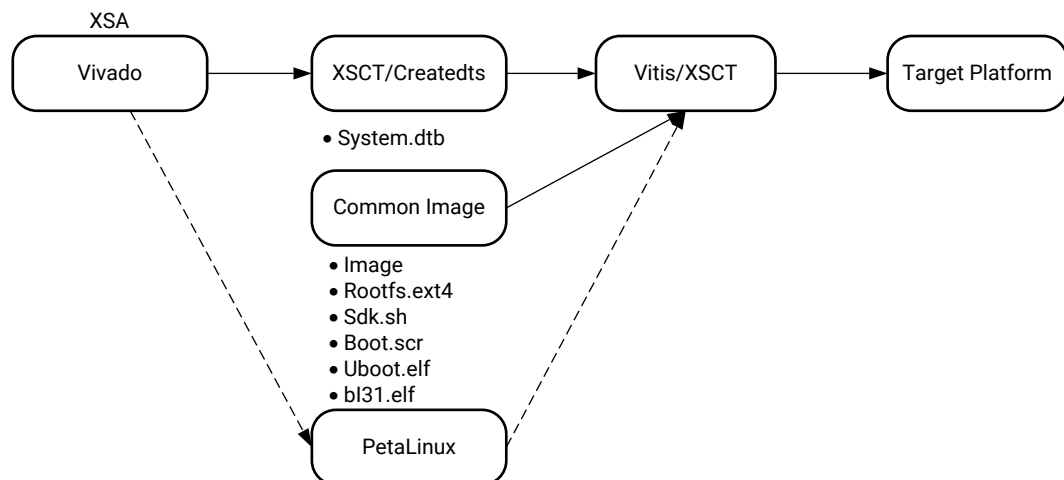
Note: Custom platform generation sources are available in https://github.com/Xilinx/Vitis_Embedded_Platform_Source.

Figure 116: Vitis Target Platforms



X23346-110920

Figure 117: Vitis Platform Project Flow



X26749-060222

Platform Naming Convention

The platform name is used when creating acceleration applications in Vitis or targeting platforms when using the Vitis compiler (v++). Pre-built platform images also use the same file name and directory name.

Pre-built Vitis embedded platforms use the following naming convention.

```
<Vendor>_<Board>_<Feature>_<Vitis Tool Version>_<Release_Version>
```

Where:

- **<Vendor>**: The board vendor. For all AMD-created pre-built platforms, use `xilinx`.
- **<Feature>**: The special function of this platform. For example:
 - `base` indicates that it connects all possible resources for you to use in an acceleration application.
 - `DFX` indicates that it supports AMD Dynamic Function eXchange (DFX).
- **<Vitis Tool Version>**: The specific version of the Vitis development platform that the platform is designed for. This also indicates the version of the AMD Vivado™ Design Suite tools that the pre-built platform is created by.
- **<Release_Version>**: The release version of the platform. The first version is 1.

For example, the following platform names follow the naming convention:

- `xilinx_vck190_base_202310_1`
- `xilinx_vck190_base_dfx_202310_1`
- `xilinx_zcu104_base_202310_1`
- `xilinx_zcu102_base_202310_1`
- `xilinx_zcu102_base_dfx_202310_1`

Note: Platform source code uses a git branch for versions. The directory name is `<Board>_<Feature>` (for example, `zcu102_base`). The platform generated from the source in https://github.com/Xilinx/Vitis_Embedded_Platform_Source has the name `xilinx_zcu102_base_202020_1`.

Embedded Platform Components and Architecture

A platform is the starting point of your Vitis design. Vitis applications are built on top of the platforms.

An embedded platform includes a hardware platform and a software platform.

Hardware Platform

The hardware platform is the static, unchanging portion of your hardware design. It includes the Xilinx Support Archive (XSA) file exported from the Vivado Design Suite.

The hardware platform describes platform hardware setup and the acceleration resources that can be used by acceleration applications, for example, input and output interfaces, clocks, AXI buses, and interrupts. Vitis adds kernels and infrastructure modules to the hardware design as needed to facilitate data movement. Acceleration kernels can share data with platform IPs, but cannot change or modify them. For information about setting up the hardware platform, refer to [Installing Xilinx Runtime and Platforms](#).

Software Platform

The software platform is the environment that runs the software to control acceleration kernels for acceleration applications. Both fixed and extensible platforms require the software platform. It includes the domain setup and boot components setup to reset and configure the hardware platform. The elements of the software platform include the following:

- **Root File System (RFS):** Includes binaries, libraries, and setups for a Linux file system. In the AMD-provided common rootfs, XRT has been installed so that acceleration application can run on this Linux environment.
- **Kernel Image:** A compiled Linux kernel. The common kernel image provided by AMD includes most AMD peripheral drivers.
- **Sysroot:** Used for cross compilation. It provides the libraries to be linked when compiling applications for a target system.

All pre-built AMD platforms have the software platform provided. By default, the platforms have a Linux domain with the Xilinx Runtime (XRT) enable so that acceleration applications can be run on the platform. Because the device tree is unique to each platform, it is provided as a component with the Linux XRT domain inside the platform.

For platform customization, the software platform can be downloaded from the *Common Images for Embedded Vitis Platforms* section of the [Downloads Center](#). The common image packages contain the following components:

- Pre-built Linux kernel
- Pre-built root file system
- boot files (pre-built u-boot.elf, boot.scr, bl31.elf, etc)
- `sdk.sh` script to generate the Sysroot

Linux domain components must be provided when there is a Linux domain in the platform. These components can be generated by PetaLinux, Yocto, or third-party frameworks. Because these components can be shared across all AMD demo boards for the given FPGA family, a common Linux component image generated by PetaLinux is provided for all standard platforms.



TIP: When creating a Linux application in the Vitis IDE, the Linux domain components in the platform setting will be the default settings. You can overwrite these settings with components installed elsewhere.

The source files for embedded platforms are available on GitHub at [Vitis Embedded Platform Source](#). You can use these files as the source of your own custom platform. To regenerate the common Linux components from the platform source files set the environment variable `COMMON_RFS_KRNL_SYSROOT=FALSE` before running `make`.

Using Vitis Embedded Platforms

Packaging Images

The Vitis compiler (`v++`) has three stages:

- `-c` or `--compile` to compile acceleration kernels
- `-l` or `--link` to link acceleration kernels with platform logic.
- `-p` or `--package`, the command `v++ --package` generates both `boot.bin` and the `sd_card` image files

The `--package` command supports both `initramfs` and `Ext4 rootfs` images.

In the Vitis IDE, the package stage is automatically called during the build process. You can add additional package options in the system project detail page by double-clicking the `.sprj` file. Package log files, command configuration files, and output files are stored in the `package` directory under the `Emulation-SW`, `Emulation-HW` or `Hardware` directories.

In command line mode, you can pass in package options as `v++` options or configuration files. For more detailed information about the `v++ --package` option, refer to the [v++ Command](#) or the `v++ -help` command, and the AMD [Vitis Acceleration Examples GitHub repository](#).

Packaging Images with Ext4 rootfs in the Vitis IDE

When `ext4 rootfs` is provided to the Vitis IDE, the generated `sd_card.img` file includes the following:

- The `xclbin` file for PL kernel
- The host application
- The Linux kernel image
- The device tree
- The U-Boot configuration file: `boot.scr`
- The `ext4 rootfs` in the Ext4 partition

To package an image with Ext4 rootfs in the Vitis IDE:

1. Select **File** → **New** → **Application Project** to create a new application project in the Vitis IDE.
2. Select the platform (for example, `xilinx-zcu102-base-202120_1`), and click **Next**.
3. Provide a name for the application project (for example, `vadd`)
4. For the System Project selection, select **Create New**.
5. For the Target processor, select the processor that can run the Linux domain (for example, `psu-cortexa53 SMP`), and click **Next**.
6. In the Domain page, select **xrt** and provide the following application settings:
 - Sysroot path (for example, `xilinx-zynqmp-common-v2021.2/sysroots/cortexa72-cortexa53-xilinx-linux`)
 - Root FS (for example, `xilinx-zynqmp-common-v2021.2/rootfs.ext4`)
 - Kernel Image (for example, `xilinx-zynqmp-common-v2021.2/Image`)
7. Click **Next**.
8. Select an application template (for example, **Vector Addition**) and click **Finish**.
9. Select the system project and click the **Build** button to build the project.
10. Verify that the `sd_card.img` file was created in the `package` directory under the `Emulation-SW`, `Emulation-HW` or `Hardware` directory.



TIP: To change the path for `sysroot`, `rootfs`, or `kernel` after the application project has been created, double-click the `.sprj` file and change the path in the Options dialog box.

Packaging Images with initramfs rootfs in the Vitis IDE

When `initramfs rootfs (rootfs.cpio)` is provided to the Vitis IDE, the generated `sd_card.img` includes the following:

- The `xclbin` file for PL kernel
- The host application
- The Linux kernel image
- The device tree
- The `boot.scr`
- The `init.sh`, `platform_desc.txt`, and `initramfs rootfs` in FAT32 partition

Note: The `sd_card.img` file does not contain the ext4 partition.

1. In the Vitis IDE, select **File** → **New** → **Application Project** to create a new application project.
2. Select the platform (for example, `xilinx-zcu102-base-202010_1`), and click **Next**.

3. Provide the application project name (for example, `vadd`).
4. Select **Create New**.
5. For the target processor, select the processor that can run the Linux domain (for example, `psu_cortexa53 SMP`), and click **Next**.
6. In the Domain page, select the `xrt` domain and provide the application settings as follows:
 - a. Sysroot path (for example, `your_linux_component_dir/sysroots/aarch64-xilinx-linux`)
 - b. Root FS (for example, `your_linux_component_dir/rootfs.cpio.gz.u-boot`)
 - c. Kernel Image (for example, `your_linux_component_dir/Image`)
7. Click **Next**.
8. Select the application template (for example, **Vector Addition**).
9. Select the system project and click the **Build** button to build the project.
10. Verify that the `sd_card.img` file was created in the `package` directory under the `Emulation-SW`, `Emulation-HW` or `Hardware` directory.

Note: The common Linux components package does not provide `initramfs` rootfs. For more information about generating `initramfs` rootfs, refer to *PetaLinux Tools Documentation: Reference Guide* ([UG1144](#)).



TIP: To change the path for `sysroot`, `rootfs`, or `kernel` after the application project has been created, double-click the `.sprj` file and change the path in the Options dialog box.

Writing Images to the SD Card

You can use the Vitis unified software platform accelerated flow to target an embedded platform. This facilitates packaging and creating an SD image with RootFS as an EXT4 partition, because `initramfs` uses Double Data Rate SDRAM (DDR SDRAM) for file system storage. It limits the real usable DDR memory for Linux kernel and applications when the file system size increases. It cannot retain RootFS changes after reboot.

To write EXT4 RootFS to an SD Card:

1. Prepare an SD card binary image file with FAT32 partition for boot and EXT4 partition for RootFS.
2. Write SD card images to the SD card. You can use various tools to do this, such as [Etcher](#) on Windows or `dd` command on Linux.

Note: Refer to AMD [Answer Record 73711](#) for detailed information about these tools.

There are various ways to prepare an SD card image. You can use the `v++` package tool to generate it, or use an open source tool. The `v++` package tool generated `sd_card.img` has two partitions:

- **FAT32 partition:** 1 GB size, initialized with the kernel image provided by common Linux components.
- **EXT4 partition:** 2 GB size, initialized with RootFS provided by common Linux components.

To make the pre-built SD card image boot, you must copy the following boot components to the FAT32 partition:

- `pre-built/BOOT.BIN`
- `boot.scr`, `system.dtb`, `init.sh`, and `platform_desc.txt` in the `xrt/image` directory

The pre-built SD card image can be used for evaluation usage and by Windows users. It does not require Vitis or PetaLinux to be installed.

Note: The `v++ --package` with Ext4 partition is not supported on Windows.

Note: `init.sh` sets up the environment variable `XILINX_XRT` and copies the `platform_desc.txt` file to `/etc/xocl.txt`. You must manually run this after Linux boots up before running any acceleration applications.

Configuring the PL Kernel in DFX Platforms and Non-DFX Platforms

The AMD Dynamic Function eXchange (DFX) feature can change some blocks of PL function while keeping other areas of PL working, allowing you to configure PL kernels on the fly. To use the DFX feature, when the `xclbin` file is generated, configure it with your host application. The new kernels in the `xclbin` take effect immediately without requiring a reboot.

For platforms without DFX features, PL kernel must be packed into `boot.bin`. Copy it to the FAT32 partition on your SD card and reboot the system. Then, configure the `xclbin` file with your host application.

The `xclbin` file contains both bit files for PL kernel and metadata to describe these kernel features and connections. Programming the `xclbin` file on DFX platforms loads the bit file and metadata; programming on non-DFX platforms only loads the metadata.

Running an Acceleration Application on the Board

If you are using the common Linux components that are provided by AMD perform the following to run an acceleration application on the platform:

1. To the SD card, write the `sd_card.img` generated by the Vitis compiler command `v++ --package`.
2. Boot the board.
3. Run the command `cd /mnt/sd-mmcblk0p1/`.
4. Run the command `source init.sh`.
5. Run acceleration application. For example, for vector addition, run `./vadd ./binary_container_1.xclbin`.

Acceleration application uses Xilinx Runtime (XRT) to communicate with acceleration kernels. To set up the environment for XRT, run `init.sh`. This command does the following:

- It sets the environment variable `XILINX_XRT` to `/usr` to allow the application to find the XRT environment.
- It copies `platform_desc.txt` to `/etc/xocl.txt` to inform XRT which platform it is running on.

Note: This was done automatically for embedded platforms in the 2019.2 release of Vitis. Because automatically running `init.sh` may introduce security breaches, common Linux rootfs did not run `init.sh` by default.

Note: If the `sd_card.img` file has already been written to the SD card and you are only updating the application, you can save time in the debugging phase by copying all files from `<Vitis System Project>/Hardware/package/sd_card` to FAT32 partition on SD card to replace existing files. The Ext4 partition does not change in `sd_card.img`.

Software Package Management in PetaLinux rootfs

All PetaLinux rootfs software packages are hosted on <https://petalinux.xilinx.com/sswreleases/rel-v2020/feeds>. You can install these software packages to rootfs when running Linux on the target board as long as the board has Internet access.

To use this feature, you must enable package manager DNF in rootfs. The rootfs in AMD-provided pre-built Linux components provides the DNF package management features by default.

To set up a package feed URL:

1. Visit the package feed repo directory <https://petalinux.xilinx.com/sswreleases/rel-v2020/generic/rpm/repos/>.
2. Download the repo file that matches your SoC device to target board.

```
# Example: ZCU102 uses ZU9EG devices
wget http://petalinux.xilinx.com/sswreleases/rel-v2020/generic/rpm/repos/
zynqmp-generic_eg.repo
# Example: ZCU104 uses ZU7EV devices
wget http://petalinux.xilinx.com/sswreleases/rel-v2020/generic/rpm/repos/
zynqmp-generic_ev.repo
```

3. Copy the downloaded repo file to `/etc/yum.repos.d/`.
4. Clean the cache.

```
dnf clean all
```

To manage packages, use the DNF package manager:

- **Listing available packages:** Use the command `dnf repoquery`.
- **Installing packages from an AMD repository:** Use the command `dnf install <pkg name>`.
- **Installing packages from a local package file:** Use the command `dnf install <pkg file name>`.
- **Installing packages to sysroot:**

When packages are installed on the rootfs of a running board, target has the latest binaries and libraries. When cross compiling on host is needed, these libraries must be added to host side sysroot.

A `sysroot_overlay` script is provided in XRT to extract RPM and update sysroot. This script will extract RPM libraries and include a file update in sysroot.

Besides XRT, this script supports all RPMs for various software packages.

- **Getting the `sysroot_overlay.sh`:** Use the command `wget https://github.com/Xilinx/XRT/blob/master/src/runtime_src/tools/scripts/sysroots_overlay.sh`.

The `sysroot` command description is:

```
./sysroots_overlay.sh --sysroot --rpms-file
```

Where:

- `--sysroot` is the sysroot to be overlaid.
- `--rpms-file` is the RPMs file that contains the RPM file paths to be overlaid.

Examples

The following example is a command to install updated XRT to the common sysroot:

```
./sysroots_overlay.sh -s sysroots/aarch64-xilinx-linux/ -r $PWD/rpm.txt
```

This example shows the contents of an `rpm.txt` file:

```
./xrt-dev-202010.2.6.0-r0.aarch64.rpm  
./xrt-202010.2.6.0-r0.aarch64.rpm
```

Note: This script works only for the local RPMs. You must download RPMs to your host machine to install them to the common sysroot.

Creating Embedded Platforms in Vitis

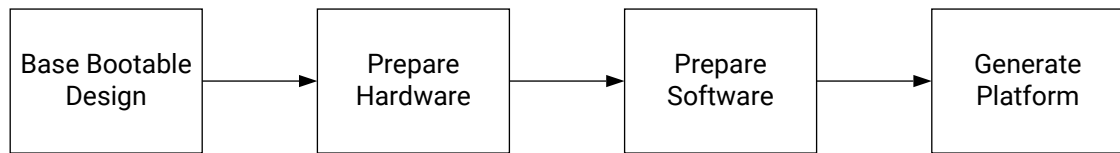
Platform Creation Basics

In the AMD Vitis™ environment acceleration application development flow, the project is divided into two distinct elements: the platform and the processing subsystem. The platform contains essential IP blocks (such as PS for SoCs, NoC and AI Engine for AMD Versal™ adaptive SoCs) and board interface IP blocks (such as high-speed I/Os and memory controllers). The processing subsystem contains the application-specific part of the system and can be composed of both programmable logic and AI Engine blocks. This approach promotes separation of concerns, facilitates concurrent development, and encourages reusability. The application developer is insulated from the low-level details of the platform and can focus on the specifics of the processing subsystem. The platform developer can focus on system bring-up and tuning I/O performance without having to worry about the processing subsystem. This means that the application developer can integrate the subsystem on different platforms, and a platform can be reused with different processing subsystems.

AMD provides pre-built platforms for AMD Alveo™ cards and embedded evaluation boards. You can download these platforms from the [Download Center](#). Efficiently leveraging the decoupling of platforms and subsystems is central to the methodology and the productivity gains offered by the Vitis environment. For embedded designs, AMD recommends a parallel development process where the application team starts working on the subsystem using an AMD pre-built platform while the platform team works independently on bringing-up the custom platform. Rapid progress can be made by working in this manner. Using a pre-built platform means that the subsystem can be developed, integrated, and tested independently using a pre-verified, known-good foundation. After the subsystem is in a sufficiently advanced and stable state, the subsystem can be integrated with appropriate versions of the custom platform. Overall, this approach greatly streamlines the system integration process.

The following figure shows how to create a customized embedded platform.

Figure 118: Platform Creation



X24781-032322

To create a platform, you must have a base bootable design as a starting point. This design can be an AMD base platform design, an existing working design, or a design created from scratch. The following base components must be included in your base bootable design:

- A base hardware design exported from AMD Vivado™ Design Suite
- A base software design that includes Linux kernel, root file system, and device tree

After you have working hardware and board through a Vivado Design Suite design, converting it into a Vitis environment platform requires adding properties to the base components to meet the requirements of the Vitis environment. In general, platform creation consists of the following steps:

1. Add hardware interface parameters and interrupt support in your Vivado Design Suite project and export the XSA.
2. Update the software platform components to enable application acceleration software stacks (enable XRT, update device tree, and so on).
3. Package and generate the platform using XSCT commands or the Vitis IDE.

Note: The platform creation process is usually iterative, and multiple versions of the platform are created throughout the course of the project. In the early phases of the project, you can create platforms with a reduced set of features to facilitate testing of the processing subsystem on the board. In the later phases of the project, you might need to iterate on the platform to respond to specification changes or to improve overall QoR.

Note: Users could also create a platform base on the existing repository platforms. This operation is actually a copy of the repository's platform to the local directory. But a DFX platform is not supported with GUI or XSCT tool.

The Vitis environment uses the properties in the hardware project to recognize the resources in the platforms and link kernels to the platforms. The Vitis environment uses the software stacks to take control of the kernels.

For details on Vitis environment embedded platform creation, see the *Vitis Unified Software Platform Documentation: Embedded Software Development* (UG1400). For step-by-step instructions, see the [Vitis Platform Creation tutorial](#).

Platforms

A platform provides design context to integrate AI Engine design and PL kernels. Platform hardware is represented by an XSA container built using the Vivado Design Suite, and a platform software that includes a board support package (BSP) as well as operating systems, boot files, drivers, libraries, and file systems to link application software and build boot images. The Vitis SDK and PetaLinux tools can be used to build BSPs, in addition to system and application software customized to your implemented hardware that includes AI Engine and PL kernels.

Types of Platforms

There are three type of hardware platforms: DFX, Flat design or BDC (Block Design Container) based. These are encapsulated as XSA files built in Vivado and can support Dynamic Function eXchange (DFX) or represent a flat design. In BDC-based design, AI Engine and AI Engine connected PL subsystem can be placed in the block design and should be added under the top BD as you do in DFX. In DFX platforms, the AI Engine will reside in the reconfigurable partition that the v++ linker will compile into a reconfigurable module. They share the same connectivity abstractions for CPU control, external memory, streaming I/O, and clocking:

- **Extensible Platform:** An extensible platform contains a pre-synthesized block design into which the Vitis v++ compiler configures the AI Engine hardware. The AI Engine hardware configuration is extracted by the v++ compiler from the `libadf.a` file that is the result of a compilation of the AI Engine applicable. The v++ compiler also integrates the PL kernels into the platform's hardware design context. The hardware design context provides connectivity abstractions for CPU control, external memory, streaming I/O, and clock networks. When targeting an extensible hardware platform, the AI Engine `libadf.a` influences the hardware implementation including interfaces between the AI Engine, PL kernels, and platform hardware. For information related to Versal platform clocking, see the *Versal AI Core Series Data Sheet: DC and AC Switching Characteristics* ([DS957](#)).

The Tcl command to export an extensible xsa is: `write_hw_platform -force <filename.xsa>`. If you don't specify any switch with `write_hw_platform`, the Vivado tool generates an extensible XSA.

A base platform is an extensible platform provided by AMD that targets an AMD board such as the VCK190, for example, `xilinx_vck190_base_202310_1.xsa` (flat) or `xilinx_vck190_base_dfx_202310_1.xsa` (DFX). For a flat platform, a generated XSA contains the full PL bitstream. Targeting a DFX platform results in two (or more) generated fixed XSAs, one for the static region that is used to boot the device and remains resident and active at runtime, and one (or more) partial XSAs that represent the partial bitstream and metadata associated with the implemented dynamic region.

A custom platform is an extensible XSA that is built using the Vivado Design Suite. Like a base platform, a custom platform is a Vivado hardware design containing a dynamic region block design in which PFM connectivity attributes have been declared. This platform is generated using the `write_hw_platform` function provided in Vivado. You can build a custom platform starting from a pre-existing Vivado project, from source files provided by AMD for a base platform, or from scratch. For more information, see [Creating Embedded Platforms in Vitis](#).

- **Fixed Platform:** A fixed hardware platform encapsulates an implemented hardware design built by the v++ linker using the Vivado Design Suite. When targeting a fixed hardware platform, the AI Engine compiler must conform to the AI Engine / PL interface constraints defined by the hardware.

The Tcl command to export fixed XSA from Vivado is: `write_hw_platform -fixed <fixed.xsa>`

In this chapter, the base platform `xilinx_vck190_base_202310_1` is used to show command usage. For targeting the DFX platform `xilinx_vck190_base_dfx_202310_1`, see [Packaging for DFX Platforms](#).

Platform Clocking

Platforms have a variety of clocking: processor, PL, and AI Engine clocking. The following table explains the clocking for each.

Table 69: Platform Clocks

Clock	Description
AI Engine	Can be configured in the platform in the AI Engine IP.
Processor	Can be configured in the platform in the CIPS IP.
Programmable Logic (PL)	Can have multiple clocks and can be configured in the platform.
NoC	Device dependent and can be configured in the platform in the CIPS and NoC IP.

Notes:

1. These clocks are derived from the platform and are affected by the device, speed grade, and operating voltage.

For information on Versal device clocks, see *Versal AI Core Series Data Sheet: DC and AC Switching Characteristics* ([DS957](#)).

Platform Creation Requirements

The base design you created in a Vitis platform is static after the platform creation process is complete.

Vitis does modify parameters based on certain IPs (for example, SmartConnect, NoCs) by adding additional master/slave interfaces. In some situations, PS/CIPS interfaces can also be modified and AMD Versal™ adaptive SoC and AI Engine IP is instantiated in the platform.

The following table shows the workflows to validate the base system on your board.

Table 70: Platform Workflows

Workflow	Development	Validation
Basic board bring-up	Processor basic parameter setup.	Standalone Hello world and Memory Test application run properly.
Advanced hardware setup	Enable advanced I/O in Processing System (such as USB, Ethernet, Flash, PCIe®, or RC). Add I/O related IP in PL (such as MIPI, EMAC, or HDMI_TX). Add non-Vitis IP (such as AXI BRAM Controller, or Video Processing Subsystem (VPSS) IP).	If these IP have standalone drivers, test them.
Base software setup	Create PetaLinux project based on hardware platform. Enable kernel drivers. Configure boot mode. Configure rootfs.	Linux boots up successfully. Peripherals work properly in Linux.

Base Component Requirements

Every hardware platform design must contain a Processing System IP block from the IP catalog.

- Versal adaptive SoC, AMD Zynq™ UltraScale+™ MPSoC, and Zynq 7000 SoC devices are supported.
- MicroBlaze™ processors are not supported for controlling acceleration kernels, but can be part of the base hardware.

Creating an Embedded Platform

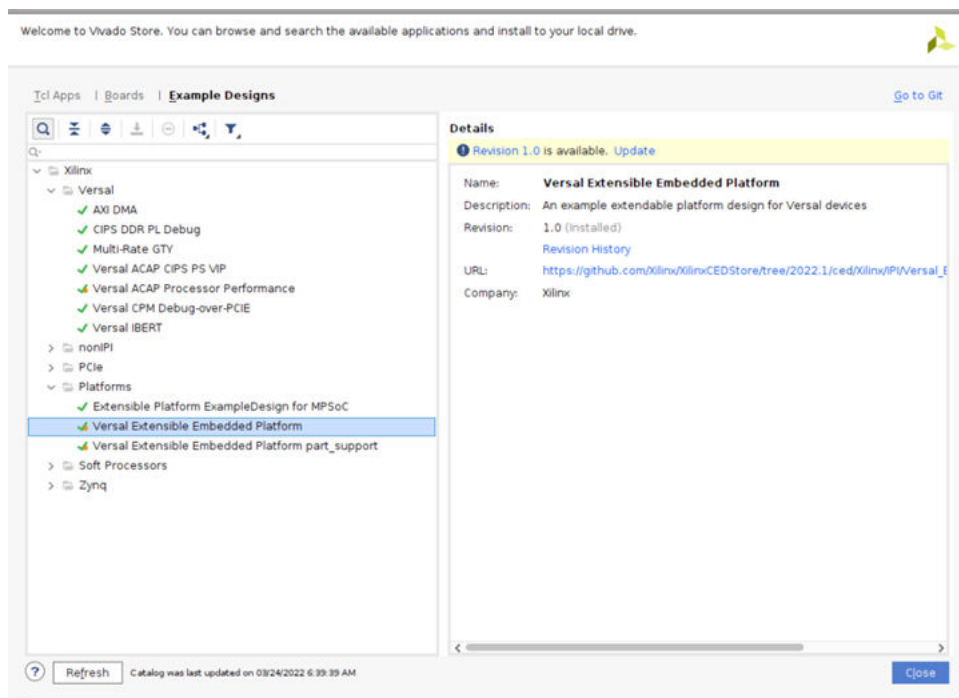
Generate XSA file

1. Leverage CED example design to create hardware platform and export XSA file.
2. Add hardware interface based on fixed platform and export XSA file.

CED Example Usage

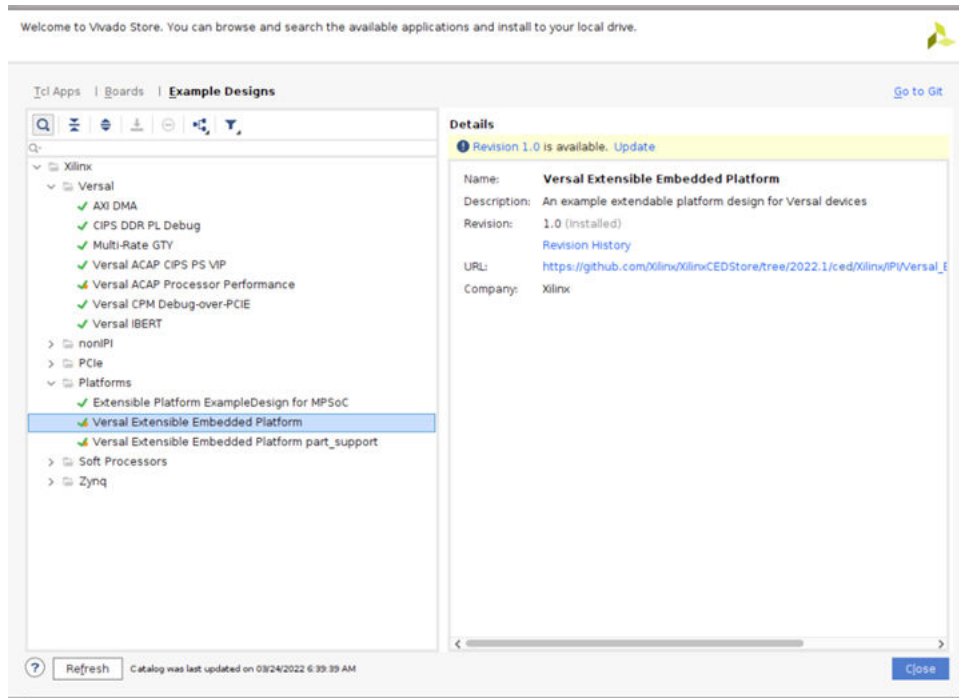
Download Example Platform

1. Launch Vivado.
2. Click **Tools** → **Vivado Store**.
3. Click **OK** to agree to download open source examples from web.
4. Select **Platform** → **Versal Extensible Embedded Platform** and click the download button on the tool bar.
5. Click **Close** after the installation is complete.



Note: In this example, Versal Extensible Embedded Platform is used. Select the appropriate example design you need.

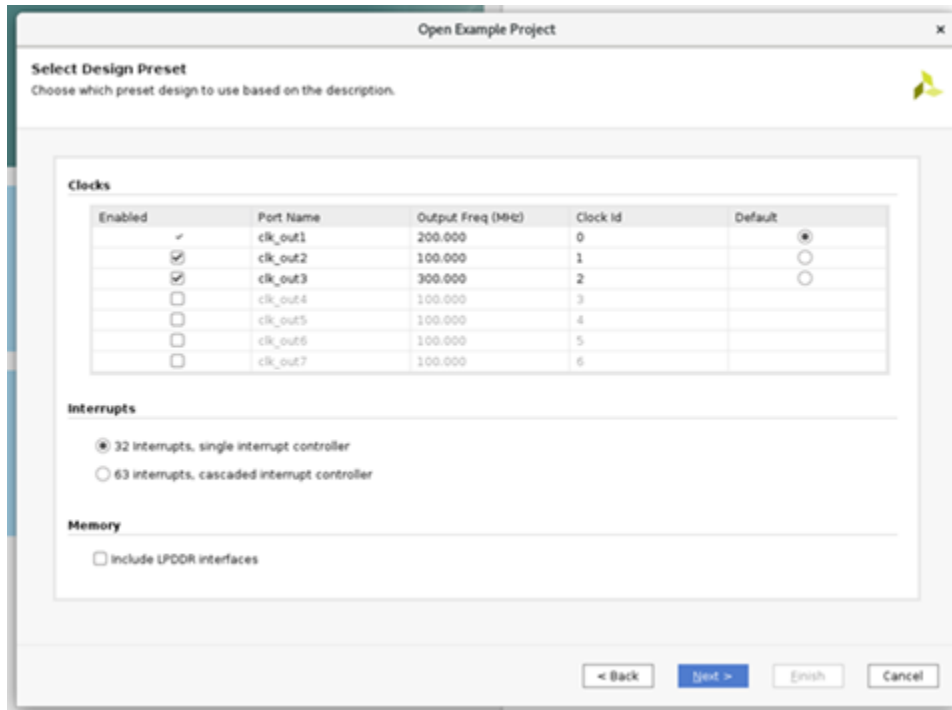
6. Click **Close** after the installation is completed.



Note: In this example Versal Extensible Embedded Platform is used. Select the appropriate example design you need.

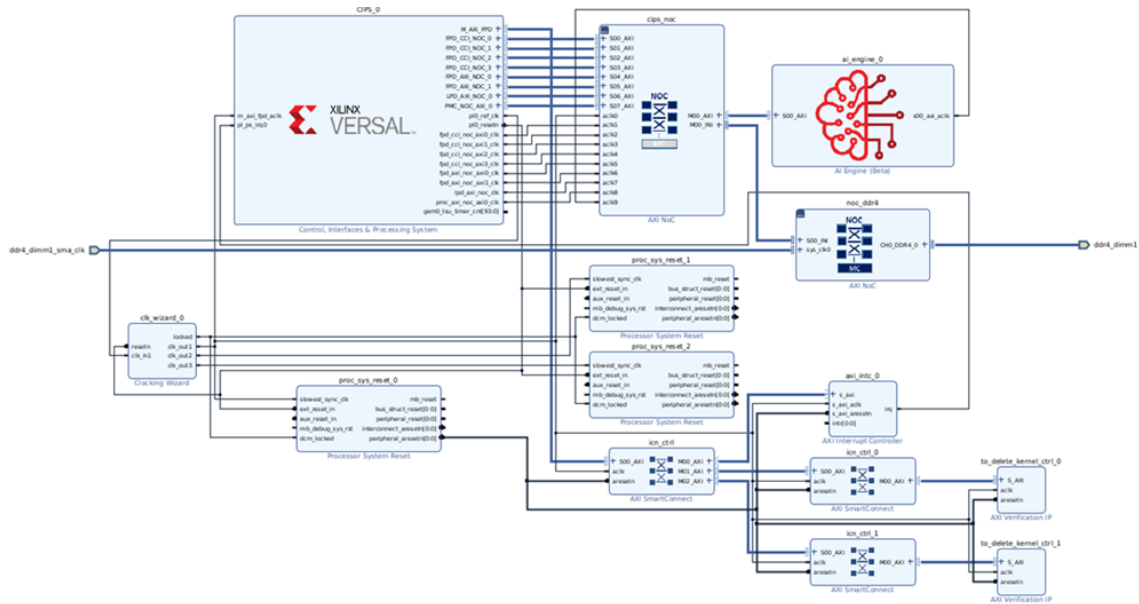
Create CED Example Design

1. Click **File** → **Project** → **Open Example**.
2. Select **Versal Extensible Embedded Platform** in Select Project Template window.
3. Input **project name** and **project location**. Keep **Create project subdirectory** checked. Click **Next**.
4. Select target board in Default Part window. In this example **Versal VCK190 Evaluation Platform** has been selected. Click **Next**.



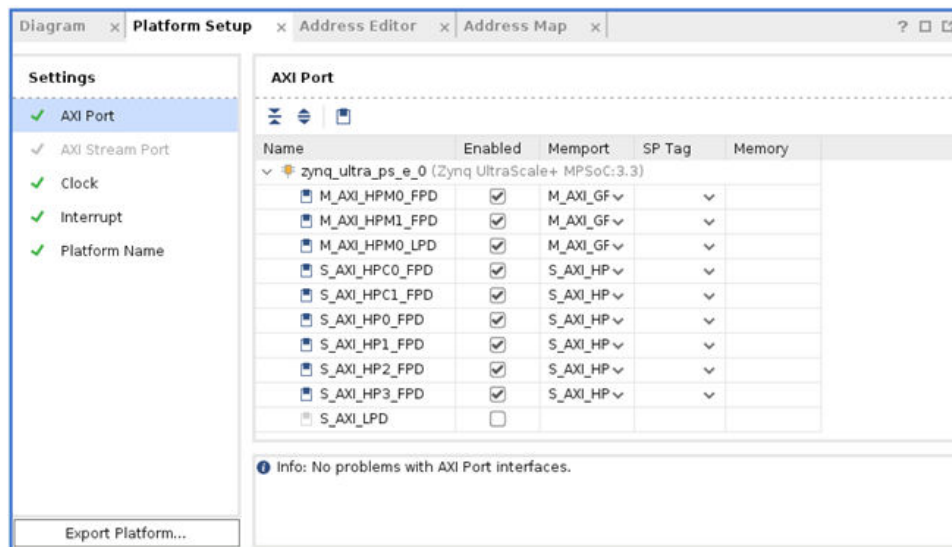
5. Configure Clocks Settings. You can enable more clocks, update output frequency and define default clock in this view. The default settings are being used in this example.
6. Configure Interrupt Settings. You can choose how many interrupt should this platform support. 63 interrupts mode will use two AXI_INTC in cascade mode. The default settings are being used in this example.
7. Configure Memory Settings. By default the example design will only enable DDR4. If you enable LPDDR4, it will enable both DDR4 and LPDDR4. The default settings are being used in this example.
8. Click **Next**.
9. Review the new project summary and click **Finish**.
10. After a while, you will see the design example has been generated.

The generated design instantiated AI Engine, enabled DDR4 controller and connected them to CIPS. It also provides one interrupt controller, three clocks and the associated synchronous reset signals.



Review the Versal Extensible Platform Example Platform Setup

1. Review the AXI port settings.
 - a. In `axi_noc_0`, S01_AXI to S27_AXI are enabled. SP Tag is set to DDR.



- b. In `icn_ctrl_0` and `icn_ctrl_1`, M01_AXI to M15_AXI are enabled. In `icn_ctrl`, M03_AXI and M04_AXI are enabled. Mempport is set to M_AXI_GP. SP Tag is empty. These ports provide the AXI master interfaces to control PL kernels. In the block diagram, `icn_ctrl_0` and `icn_ctrl_1` connects to an AXI Verification IP because the AXI SmartConnect IP requires a load. The AXI Verification IP is used here as a dummy.

Settings

- ✓ AXI Port
- ✓ AXI Stream Port
- ✓ Clock
- ✓ Interrupt
- ✓ Platform Name

AXI Port

Name	Enabled	Memport	SP Tag	Memory
> clips_noc (AXI NoC:1.0)				
> noc_ddr4 (AXI NoC:1.0)				
> icn_ctrl (AXI SmartConnect:1.0)				
icn_ctrl_0 (AXI SmartConnect:1.0)				
S01_AXI	☐			
S02_AXI	☐			
S03_AXI	☐			
S04_AXI	☐			
S05_AXI	☐			
S06_AXI	☐			
S07_AXI	☐			
S08_AXI	☐			
S09_AXI	☐			
S10_AXI	☐			
S11_AXI	☐			
S12_AXI	☐			
S13_AXI	☐			
S14_AXI	☐			
S15_AXI	☐			
M01_AXI	☒	M_AXI_GP	▼	▼
M02_AXI	☒	M_AXI_GP	▼	▼
M03_AXI	☒	M_AXI_GP	▼	▼
M04_AXI	☒	M_AXI_GP	▼	▼

2. Review the Clock settings.

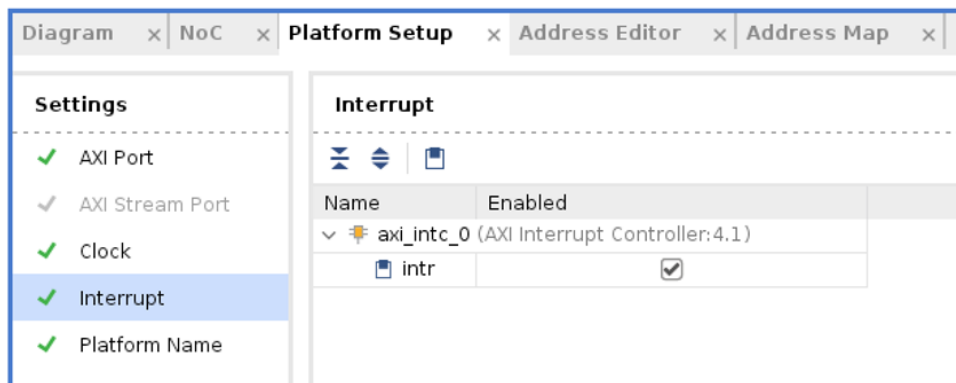
- In Clock tab, clk_out1, clk_out2, clk_out3 from clk_wizard_0 are enabled with id {0,1,2}, frequency {200 MHz, 100 MHz, 300 MHz}. clk_out1 is the default clock. V++ linker will use this clock to connect the kernel if there aren't any clocks specified in the link configuration. The Proc Sys Reset property is set to the synchronous reset signal associated with each clock.

Clock

Name	Enabled	ID	Is Default	Proc Sys Reset	Status	Freque...
CIPS_0 (Control, Interfaces & Processing System:3.0)						
pl0_ref_clk	☐					
clk_wizard_0 (Clocking Wizard:1.0)						
clk_out1	☒	0	☒	/proc_sys_reset_0	fixed	200 MHz
clk_out2	☒	1	☐	/proc_sys_reset_1	fixed	100 MHz
clk_out3	☒	2	☐	/proc_sys_reset_2	fixed	300 MHz

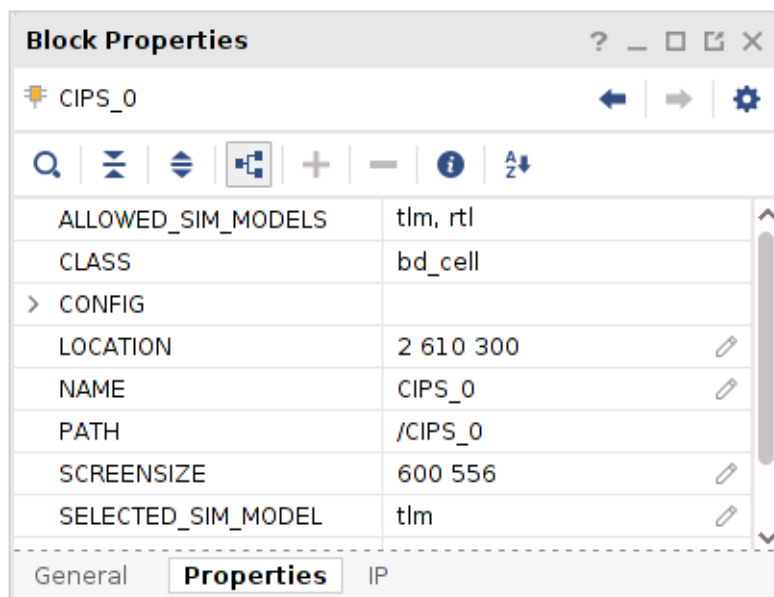
3. Review the Interrupt Tab.

- In Interrupt tab, In0 to In31 port of xlconcat is enabled.



4. Review the Simulation Model.

- a. In Vivado Integrated Design Environment (IDE), select the **CIPS** instance. Check the Block Properties window. In Properties tab, it shows **ALLOWED_SIM_MODELS** is **tlm**, and **rtl**, **SELECTED_SIM_MODEL** is **tlm**. This means this block supports two simulation models. In this example, **tlm** model was selected.



- b. Review the simulation model property for NoC and AI Engine in the block diagram.

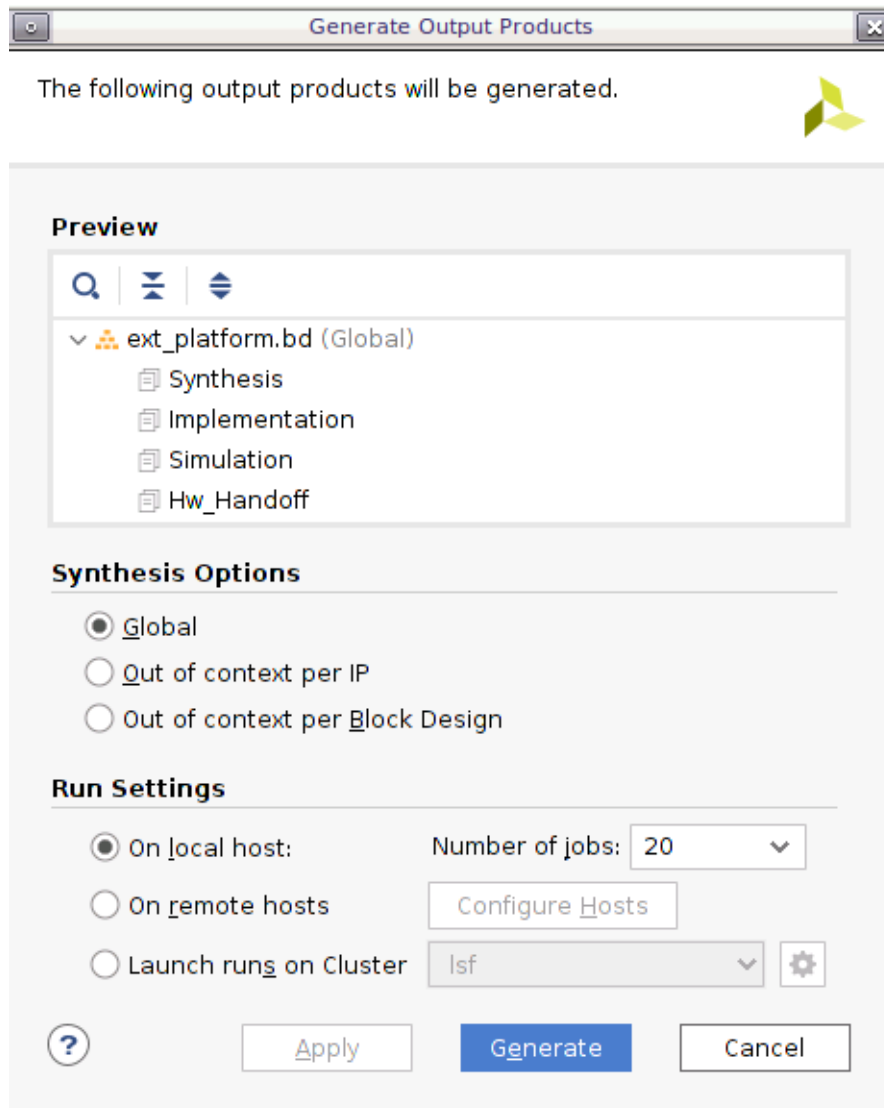
Export Hardware XSA

1. Generate Block Diagram.
 - a. Click **Generate Block Diagram** from Flow Navigator window.

▼ IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design
- Export Platform

- b. Select Synthesis Options to Global to save generation time. Click **Generate** button.



2. Export hardware platform with the following scripts.
- a. Click **File** → **Export** → **Export Platform**. Alternative ways are: Flow Navigator window: **IP Integrator** → **Export Platform**, or the **Export Platform** button on the bottom of Platform Setup tab.

- b. Click **Next** on Export Hardware Platform page.
- c. Select **Hardware**. If there are any IP that don't support simulation, the Hardware XSA and Hardware Emulation XSA need to be generated separately. Click **Next**.
- d. Select **Pre-synthesis**, as a DFX platform is not being created. Click **Next**.
- e. Input the Name and click **Next**.
- f. Update the file name and click **Next**.
- g. Review the summary and click **Finish**.

Export Hardware Emulation XSA

A change in Vitis 2021.2 requires hardware and hardware emulation provide their own XSA files during platform creation step in XSCT. One XSA with both hardware and hardware emulation content is still supported in 2021.2. It will be deprecated in the future.

1. Click **File** → **Export** → **Export Platform**. Alternative ways are: Flow Navigator window: **IP Integrator** → **Export Platform**, or the **Export Platform** button on the bottom of Platform Setup tab.
2. Click **Next** on Export Hardware Platform page.
3. Select **Hardware Emulation**. If there are any IP that don't support simulation, the Hardware XSA and Hardware Emulation XSA need to be generated separately. Click **Next**.
4. Select **Pre-synthesis**, because a DFX platform is not being created. Click **Next**.
5. Input the name and click **Next**.
6. Update the file name and click **Next**.
7. Review the summary and click **Finish**.

Note: Using the same hardware and hardware emulation design for a simple project is okay, but if your project is complicated, platform developers should keep the two designs logically identical. Otherwise your emulation result cannot represent your hardware design.

Adding Hardware Interfaces

The following table shows the possible Vitis inputs and the minimal requirements for an acceleration embedded platform.

Table 71: Available Interfaces for Vitis

Inputs	Types Vitis Can Use	Minimum Requirements for AXI MM Kernels
Control Interfaces	AXI Master Interfaces from PS or from AXI Interconnect IP or SmartConnect IP	One AXI4-Lite Master for kernel control
Memory Interfaces	AXI Slave Interfaces	One memory interface for data exchange

Table 71: Available Interfaces for Vitis (cont'd)

Inputs	Types Vitis Can Use	Minimum Requirements for AXI MM Kernels
Streaming Interfaces	AXI4-Stream Interfaces	Not required
Clock	Multiple clock signals	One clock
Interrupt	Multiple interrupt signals	One Interrupt

General Requirements



IMPORTANT! The source files for all elements of the Vivado project must be local to the project prior to exporting it as an XSA, or an error can be returned when using the platform in the Vitis tool.

- Every IP used in the platform design that is not part of the standard Vivado IP catalog must be local to the Vivado Design Suite project. References to IP repository paths external to the project are not supported when creating extensible XSA.
- Any platform interface, used for linking to kernels by the Vitis compiler, must be an AXI4, AXI4-Lite, AXI4-Stream, interrupt, clock, or reset type of interface.
- Any platform IP that has an AXI interface for linking to kernels by the Vitis compiler must also have associated clock pins to enable `v++` to correctly infer and insert clock domain crossing logic when needed.
- Custom bus type and hardware interfaces on the platform or on kernels are not supported through `v++` linker `--connectivity.sp` and `--connectivity.sc` directives. If a data bus with a custom bus type needs to be connected to kernels by the Vitis compiler, it must be converted to an AXI4, AXI4-Lite, or AXI4-Stream interface.

Project Type

To create a new XSA platform for the Vivado project type, select **RTL Project** and enable **Project is an extensible Vitis platform** check box.

Figure 119: Project Type

Project Type
Specify the type of project to create.

- ☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
 - ☒ Do not specify sources at this time
 - ☒ Project is an extensible Vitis platform
- ☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
 - ☐ Do not specify sources at this time
- ☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.
- ☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.
- ☐ **Example Project**
Create a new Vivado project from a predefined template.



TIP: You will see these settings in the New Project wizard.

When creating a new project, select **Project is an extensible Vitis platform**.

To change an existing Vivado project to an extensible Vitis platform project, select **Project Manager** → **Settings in the Flow Navigator** and enable **Project is an extensible Vitis platform**.

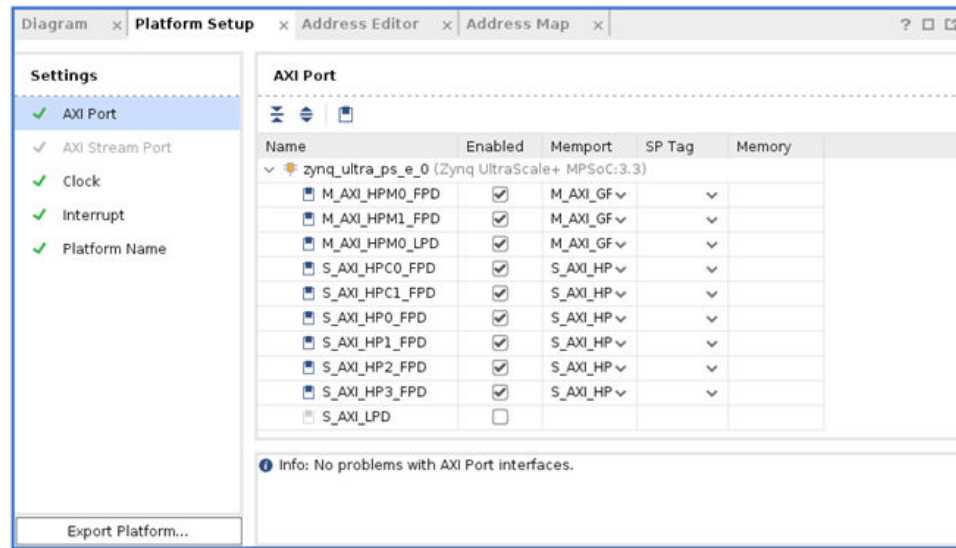
```
set_property platform.extensible true [current_project]
```

Adding Platform Interfaces

If a component in block design has a PFM property, this component can be recognized by v++ linker and can be used by the acceleration kernel.

In Vivado IDE, the Platform interface (PFM) properties can be set in the Platform Setup window if the project is created as an extensible platform project. Click **Window menu** → **Platform Setup** to open the settings.

Figure 120: Platform Setup



TIP: Platform interfaces can be defined manually in the Tcl Console, or by a Tcl script as well.

The four Platform Interface Tcl APIs include:

- AXI memory-mapped interfaces:

```
set_property PFM.AXI_PORT { <port_name> {parameters} <port2>
{parameters} ...} [get_bd_cells <cell_name>]
```

- AXI4-Stream interfaces:

```
set_property PFM.AXIS_PORT { <port_name> {parameters} <port2>
{parameters} ...} [get_bd_cells <cell_name>]
```

- Clocks and resets:

```
set_property PFM.CLOCK { <port_name> {parameters} <port2>
{parameters} ...} [get_bd_cells <cell_name>]
```

- Interrupts:

```
set_property PFM.IRQ {pin_name {id id-number range irq-count}}
[get_bd_cells <cell_name>]
```

The requirements for the PFM Properties are:

- The value of the PFM interface properties must be specified as a Tcl dictionary, a list of name/"value" pairs.



IMPORTANT! The "value" must be quoted, and both the name and value are case-sensitive.

- A `bd_cell` can have multiple PFM interface definitions. However, for each type of PFM interface, all ports are required to be set in a single `set_property` Tcl command.

- For each PFM interface property, the name specified for the port object must match the name of an external port or interface on a `bd_cell`. Each external port or interface object can only have one PFM interface definition.
- Each different type of PFM interface can have different parameters.
- Setting the PFM property with a NULL ("") string will delete previously defined PFM interfaces.

Adding AXI Interfaces

To support AXI interfaces the platform needs to declare at least one AXI control interface with AXI memory-mapped master port (M_AXI) and one memory interface with AXI Slave port (S_AXI). They can be exported from the PS block directly or have an interconnect IP connected. If the platform does not support AXI4 kernels, these interfaces are not required.

For security reasons the DFX decoupler IP is required to control the kernel if it is a DFX platform. The DFX decoupler can turn off the channels during reconfiguration to prevent unexpected requests from the static region cause invalid status on AXI interface and prevent random toggles generated by RP reconfiguration cause unexpected side-effects in static region. XRT will turn on the DFX decoupler before the reconfiguration process and turn it off after the reconfiguration completes.

NoC stub is required in a DFX platform in Vitis Region to export memory interface for the platform. V++ linker can connect the PL kernel memory interfaces to the NoC stub to access memory.

The following is the Tcl command syntax:

```
set_property PFM.AXI_PORT { <port_name> {parameters} <port2>
{parameters} ...} [get_bd_cells <cell_name>]
```

The AXI control interfaces and AXI memory interfaces share the same `PFM.AXI` property. They have different `mempport` types.

- **Memport:** AXI control interface can be defined as `M_AXI_GP`. Memory interfaces use other types: `S_AXI_HP`, `S_AXI_ACP`, `S_AXI_HPC`, or `MIG`.
- **SP Tag ID:** (Optional) A user-defined ID that should start with an alphabetic character. The ID is case-sensitive. The system port tag (`sptag`) is a symbolic identifier that represents a class of platform port connections, such as `S_AXI_HP`, `S_AXI_ACP`,... Multiple block design platform ports can share the same `sptag`. For more information on how `sptags` are used, see [Mapping Kernel Ports to Memory](#).



TIP: The `sptag` property for is not supported `M_AXI_GP` ports.

- **Memory:** (Optional) Specify the associated MIG IP instance and `address_segment`. The memory tag is a unique identifier that combines the Cell Name and Base Name columns in the IP integrator Address Editor. This tag will be associated with connections to the Memory Subsystem HIP, where multiple block design platform ports can share the same memory tag.

Exporting AXI interconnect master and slave ports involves the following requirements:

- All ports on the interconnect used within the platform must precede in index order any declared platform interfaces.
- There can be no gaps in the port indexing.
- The maximum number of master IDs for the S_AXI_ACP port is 8, so on a connected AXI interconnect, available ports to declare must be one of {S00_AXI, S01_AXI, ..., S07_AXI}. Do not declare any ports that are used within the platform itself. Declaring as many as possible will allow `sds++` to avoid cascaded `axi_interconnects`.
- The maximum number of master IDs for an S_AXI_HP or MIG port is 16, so on an connected AXI interconnect, available ports to declare must be one of {S00_AXI, S01_AXI, ..., S15_AXI}. Do not declare any ports that are used within the platform itself. Declaring as many as possible will allow `v++` to avoid cascaded `axi_interconnects` in generated user systems.
- The maximum number of master ports declared on an interconnect connected to an M_AXI_GP port is 64, so on an connected AXI interconnect, available ports to declare must be one of {M00_AXI, M01_AXI, ..., M63_AXI}. Do not declare any ports that are use within the platform itself. Declaring as many as possible will allow `v++` to avoid cascaded `axi_interconnects` in generated user systems.

The following shows an example of defining an AXI master ports on AXI Interconnect IP:

```
set parVal []
for {set i 2} {$i < 64} {incr i} {
  lappend parVal M[format %02d $i]_AXI \
  {mempport "M_AXI_GP"}
}
set_property PFM.AXI_PORT $parVal [get_bd_cells /axi_interconnect_0]
```

The following shows an example of defining AXI memory ports with MIG on SmartConnect IP:

```
set parVal []
for {set i 1} {$i < 16} {incr i} {
  lappend parVal S[format %02d $i]_AXI
  {mempport "MIG" sptag "Bank0"}
}
set_property PFM.AXI_PORT $parVal [get_bd_cells /smartconnect_0]
```

The following is an example of the `PFM.AXI_PORT` setting for control interface and memory interface.

```
set_property PFM.AXI_PORT {
  M_AXI_HPM1_FPD {mempport "M_AXI_GP"}
  S_AXI_HPC0_FPD {mempport "S_AXI_HPC" sptag "HPC0" memory "zynq-ultra-ps-e-0
  HPC0_DDR_LOW"}
  S_AXI_HPC1_FPD {mempport "S_AXI_HPC" sptag "HPC1" memory "zynq-ultra-ps-e-0
```

```
HPC1_DDR_LOW" }
S_AXI_HP0_FPD {mempport "S_AXI_HP" sptag "HP0" memory "zynq-ultra-ps-e-0
HP0_DDR_LOW" }
S_AXI_HP1_FPD {mempport "S_AXI_HP" sptag "HP1" memory "zynq-ultra-ps-e-0
HP1_DDR_LOW" }
S_AXI_HP2_FPD {mempport "S_AXI_HP" sptag "HP2" memory "zynq-ultra-ps-e-0
HP2_DDR_LOW" }
} [get_bd_cells /ps_e]
```



TIP: In the examples above, `zynq-ultra-ps-e-0` is the instance name of the Zynq UltraScale+ MPSoC module, and `HPC0_DDR_LOW` is the address range name.

Adding AXI Stream Interfaces

To support AXI4-Stream stream kernels, the platform needs to declare the corresponding master or slave AXI4-Stream interfaces.

AXI4-Stream kernel interfaces are specified with the PFM.AXIS_PORT sptag interface property and a matching `connectivity.sc` command argument to the `v++` linker.

The following is the Tcl command syntax:

```
set_property PFM.AXIS_PORT { <port_name> {parameters} <port2>
{parameters} ...} [get_bd_cells <cell_name>]
```

Argument Description

- **Port_name:** AXI4-Stream port name.
- **Parameters:** type value: Streaming interface port type. Valid values for type include:
 - M_AXIS: A general-purpose AXI master port
 - S_AXIS: A high-performance AXI slave port

Example

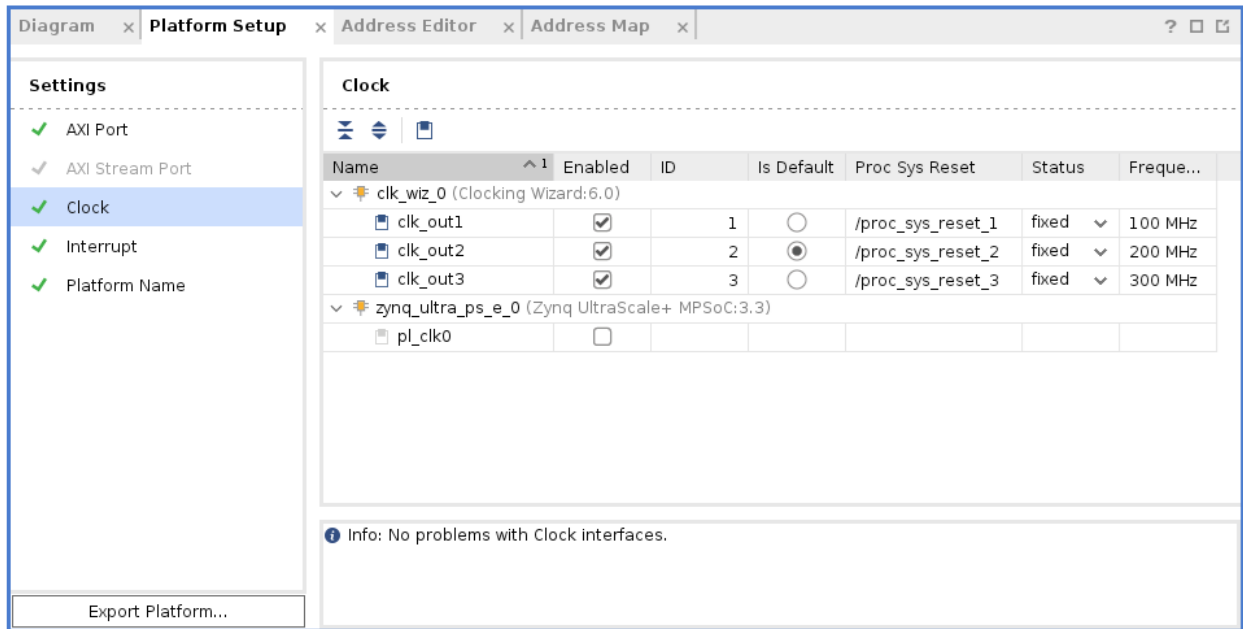
```
set_property PFM.AXIS_PORT {AXIS_P0 {type "S_AXIS"}} [get_bd_cells /
zynq-ultra-ps-e-0]
```

Note: For more information on linking AXI4-Stream interfaces between kernels and platforms, see [Specifying Streaming Connections](#).

Adding Clock and Resets

If it is a DFX platform static region and dynamic region can have their own clock and reset signals. The Clock Wizard in static region is required so that device tree generator (DTG) can generate correct device tree to describe this clock topology.

Figure 121: Platform Setup – Clock



You can export any clock source with the platform, but for each clock you must also export synchronized reset signals using a Processor System Reset IP block in the platform. For details of defining clocks and resets, see the [Vitis-Tutorials/Vitis_Platform_Creation](#). The PFM.CLOCK property can be set on a BD cell, external port, or external interface.

In the preceding figure you can see the details of the platform clocks. There must be at least one enabled clock for the platform and one clock must be specified as the default.

The following is the Tcl command for setting the PFM.CLOCK property:

```
set_property PFM.CLOCK { <port_name> {parameters} \
<port2> {parameters} ... } [get_bd_cells <cell_name>]
```

Adding Interrupts



TIP: The DFX decoupler IP is required for a DFX platform. It is used to connect the interrupt controller in the static region with the concat IP which is used to export the Interrupt signals in the dynamic region. Interrupt Controller outputs to Control, Interfaces, and Processing System (CIPS) IRQ.

The Vitis compiler will automatically connect the kernel `interrupt` signal to an IRQ in the platform during the `v++` linking stage. However, this requires the interrupts on the platform to be defined using the `PFM.IRQ` property as shown below:

```
set_property PFM.IRQ {pin_name {id id_number}} bd_cell
set_property PFM.IRQ {port_name {id id_number range irq_count}}
[get_bd_cell <cell_name>]
```

Argument Description

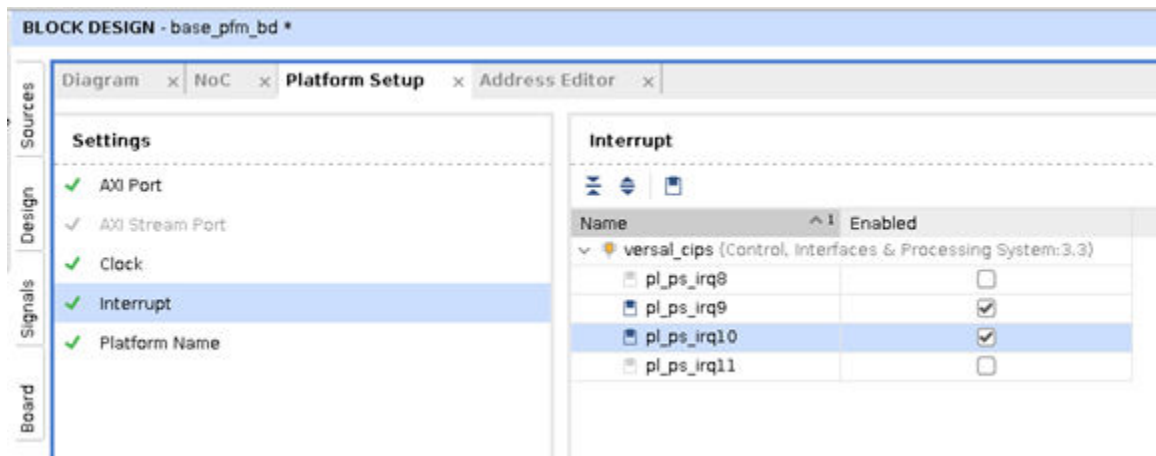
- **Port_name:** IRQ port name of `bd_cell`.
- **id_number:** Integer from 0 to 127 to specify the IRQ number or the starting number if range is specified.
- **irq_count:** Used for labeling interfaces that are otherwise subject to parameter propagation for specifying sizing of a bus (for example, interrupt controller `intr` interface).

The example shows how to enable 32 IRQ inputs to `axi-intc_0` `intr` port.

```
set_property PFM.IRQ {intr {id 0 range 32}} [get_bd_cells /axi-intc_0]
```

The interrupts for AMD Versal™ CIPS IP core can be found on the IP customization dialog box by selecting the PS PMC configuration command and selecting **Interrupts**. Here you can enable the interrupts on the CIPS IP for use on the platform. Then you must enable the interrupts on the platform for connection by `v++` during linking. On the Platform Setup tab of the Block Design, select **Interrupt** and then enable the visible interrupts on the CIPS IP as shown below. Refer to *Creating a Hardware Platform for the Platform-Based Design Flow of the Versal Adaptive SoC Hardware, IP, and Platform Development Methodology Guide* ([UG1387](#)) for more information.

Figure 122: Platform Setup – Interrupt



This results in the following Tcl command for enabling the interrupts:

```
set_property PFM.IRQ {pl_ps_irq9 {is_range "false"} pl_ps_irq10 {is_range "false"}} [get_bd_cells /versal_cips]
```



TIP: Multiple interrupts can also be connected to the CIPS IP through the use of the Concat block.

Platform settings for DFX only

- **Address aperture setting:** Setting the address range for the dynamic region is required so that the CIPS and the SmartConnect can access kernels in the dynamic region after linking stage by V++. Setting the apertures to DDR interfaces is required to export the accessible DDR range for the kernel. Lock the aperture settings by typing the following command in the Tcl console.

```
set_property HDL_ATTRIBUTE.LOCKED TRUE [get_bd_intf_pins /VitisRegion/  
PL_CTRL_S_AXI] set_property HDL_ATTRIBUTE.LOCKED TRUE [get_bd_intf_pins /  
VitisRegion/DDR_0] set_property HDL_ATTRIBUTE.LOCKED TRUE  
[get_bd_intf_pins /VitisRegion/DDR_1] set_property HDL_ATTRIBUTE.LOCKED  
TRUE [get_bd_intf_pins /VitisRegion/DDR_2] set_property  
HDL_ATTRIBUTE.LOCKED TRUE [get_bd_intf_pins /VitisRegion/DDR_3]
```

- **Setup Block Design Container (BDC) for DFX:** Update VitisRegion BDC properties for DFX to freeze the boundary of this container and Enable Dynamic Function eXchange on this container. Configure Dynamic Function eXchange Wizard and add a configuration.
- **Setup the DFX platforms properties:**

```
# Specify that this platform supports  
DFX set_property platform.uses_pr true [current_project]  
# Specify the dynamic region instance path for hardware run  
set_property platform.dr_inst_path {design_1-i/VitisRegion}  
[current_project]  
# Specify the dynamic region instance path for emulation  
set_property platform.emu.dr_bd_inst_path {design_1-wrapper_sim_wrapper/  
design_1-wrapper-i/design_1-i/VitisRegion} [current_project]
```

Exporting the Extensible Platforms

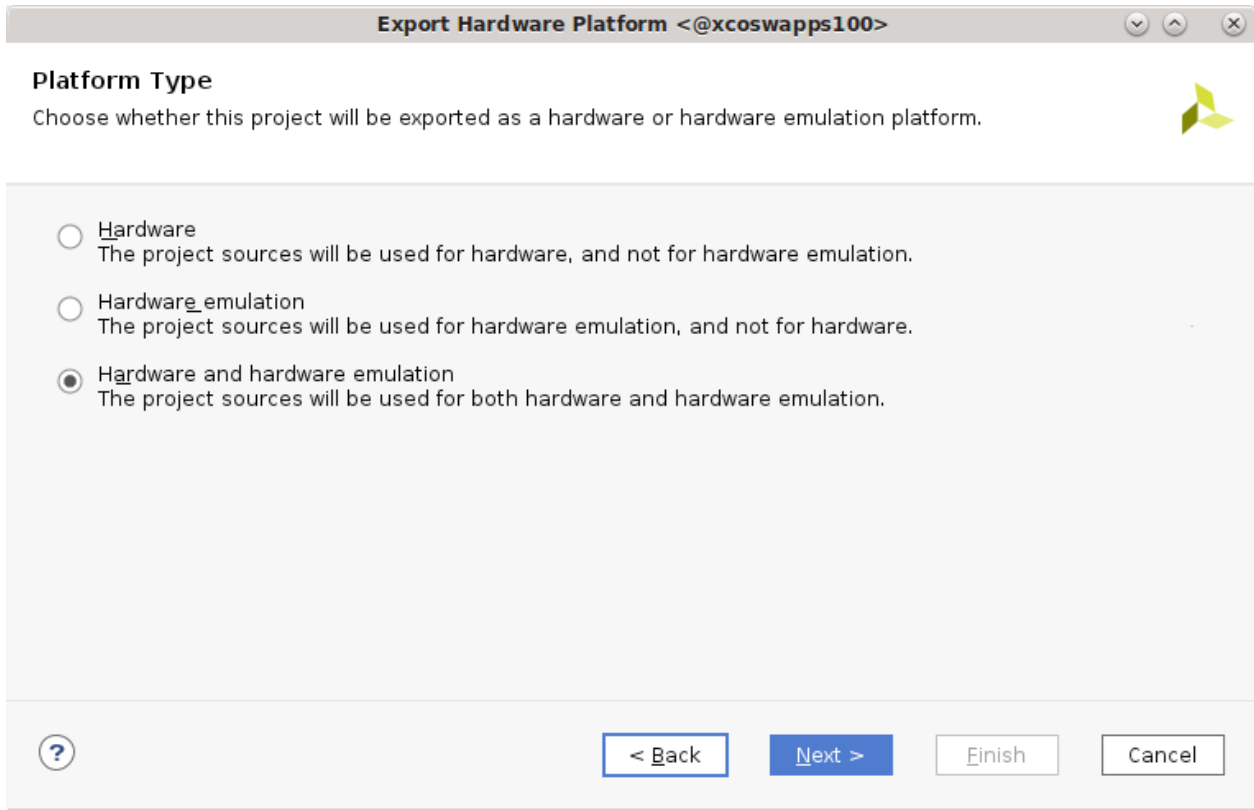
Hardware platforms are encapsulated in XSA file format. There are two kinds of XSA formats: fixed XSA for embedded software development and extensible XSA for Vitis application acceleration projects. To create an embedded platform for the Vitis application acceleration flow, you must use an extensible XSA.

When the Vivado project type is set to extensible Vitis platform, the Export Platform wizard is available from the **File → Export → Export Platform** menu command.



IMPORTANT! The block design must have an HDL wrapper and have output targets generated to export the platform XSA. Use the Create HDL Wrapper from the right-click menu in the Sources window to create the wrapper, and Generate Block Design from the Flow Navigator in the Vivado IDE.

Figure 123: Export Hardware Platform Wizard



Export Hardware Platform <@xcoswapps100>

Platform Type
Choose whether this project will be exported as a hardware or hardware emulation platform.

☐ **Hardware**
The project sources will be used for hardware, and not for hardware emulation.

☐ **Hardware emulation**
The project sources will be used for hardware emulation, and not for hardware.

☒ **Hardware and hardware emulation**
The project sources will be used for both hardware and hardware emulation.



The Export Platform wizard contains five pages to help you export the extensible platform XSA:

- **Platform Type:** Specifies the XSA as supporting hardware emulation, and hardware targets.
- **Platform State:** Specifies the XSA as including platform implementation or only synthesis.
- **Platform Properties:** Defines platform properties and lets you specify Tcl scripts and XDC constraints for the Vitis compiler to use when building the system.
- **Output File:** Specifies the output file name and location.
- **Summary:** Reports the various settings that will be used during export.

In the Export Hardware Platform wizard, select the platform type. There are three types of platforms: platforms intended to run on hardware only, platforms intended for hardware emulation only, or platforms that can run in hardware emulation and on hardware. The differences between these options is that if some modules are not supported by emulation, you should create a separate emulation specific design, export it as "hardware emulation" platform and then use "Combine XSAs" option to combine a hardware XSA and a hardware emulation XSA into one XSA that is capable of performing both jobs.

For basic platforms, use the following steps:

1. Select **Hardware and Hardware Emulation**. Click **Next**.
2. Select **Pre-synthesis for Platform State**. Post-implementation is only needed when creating DFX platforms. Click **Next**.
3. Input **Platform Properties**. Click **Next**.
4. Input the XSA file name and the export target directory. Click **Next**.
5. Check summary and click **Finish**.

You can also perform this in the command line using the following command:

```
set_property pfm_name {vendor:board:name:version} [get_files <bd_file>]
write_hw_platform -hw -force <XSA file>
```

Commands can be used to export the XSA file for DFX platform only.

```
#emulation XSA
set_property platform.platform_state "pre_synth" [current_project]
write_hw_platform -hw_emu -force -file vck190_custom_dfx_hw_emu.xsa
#hardware and RP XSA
set_property platform.platform_state "impl" [current_project]
write_hw_platform -force -fixed -static -file vck190_custom_dfx_static.xsa
write_hw_platform -force -rp design_1_i/VitisRegion vck190_custom_dfx_rp.xsa
```

To create and combine a hardware XSA and a hardware emulation XSA, use the following commands:

```
write_hw_platform -hw <hw_platform>
write_hw_platform -hw_emu <hw_emu_platform>
combine_hw_platform -hw <hw_platform> -hw_emu <hw_emu_platform> -o
<combined_platform>
```

Updating Software Components

Prepare Common Image

AMD provides pre-built image Common Image package for quick platform creation and quick evaluation. The 'common image' packages contain a prebuilt Linux kernel and root file system that can be used with any Zynq™, Zynq™ MP or Versal board for embedded Vitis platform developers. The common image package has following images included:

```
#Versal
├── bl31.elf
├── boot.scr
├── environment-setup-cortexa72-cortexa53-xilinx-linux
├── Image
├── README.txt
├── rootfs.ext4
├── rootfs.manifest
├── rootfs.tar.gz
├── sdk.sh
└── site-config-cortexa72-cortexa53-xilinx-linux
```



```

├── sysroots
├── u-boot.elf
├── version-cortexa72-cortexa53-xilinx-linux

#Zynqmp
├── bl31.elf
├── boot.scr
├── environment-setup-cortexa72-cortexa53-xilinx-linux
├── Image
├── README.txt
├── rootfs.ext4
├── rootfs.manifest
├── rootfs.tar.gz
├── sdk.sh
├── site-config-cortexa72-cortexa53-xilinx-linux
├── sysroots
├── u-boot.elf
├── version-cortexa72-cortexa53-xilinx-linux

```

Download the AMD common image from the [AMD download page](#), and use following command to extract it, and place it in the project folder.

```
tar xvf xilinx-zynqmp-common-v2023.2.tar.gz -C . #. means current directory.
```

Create DTB File

Use the "createdts" command in XSCT tool to generate DTB file. The zocl driver interface requires a device tree node to enable the interrupt connection. Add the -zocl option when using this command. The following code shows the usage of this command with its options.

```
createdts -hw <full path of XSA file> -zocl -platform-name mydevice -git-branch xlnx_rel_v202X.X -board zcu104-revc -compile
```

The `system.dtb` file is located in `<mydevice/psu_cortexaXX_0/device_tree_domain/bsp>` folder.

- `-name`: Platform name
- `-hw`: Hardware XSA file with path
- `-git-branch`: device tree branch
- `-board`: board name of the device. You can check the board name at `/device_tree/data/kernel_dtsi`.
- `-zocl`: enable the zocl driver support
- `-compile`: specify the option to compile the device tree

The following is an example of the zocl device node for your reference.

```

&amba {
    zyxclmm_drm {
        compatible = "xlnx,zocl";
        status = "okay";
        interrupt-parent = <&axi_intc_0>;
    }
}

```

```

interrupts = <0 4>, <1 4>, <2 4>, <3 4>,
             <4 4>, <5 4>, <6 4>, <7 4>,
             <8 4>, <9 4>, <10 4>, <11 4>,
             <12 4>, <13 4>, <14 4>, <15 4>,
             <16 4>, <17 4>, <18 4>, <19 4>,
             <20 4>, <21 4>, <22 4>, <23 4>,
             <24 4>, <25 4>, <26 4>, <27 4>,
             <28 4>, <29 4>, <30 4>, <31 4>;
    };
};

```

For more information, refer to the XRT documentation: <https://xilinx.github.io/XRT/master/html/yocto.html>.

Packaging a Vitis Acceleration Platform

With all requirements prepared for Vitis acceleration platforms, you can package them together and generate the final Vitis acceleration platform. You can do this using either the Vitis Unified IDE or the Python CLI (command line) tool.

- In the Vitis IDE, select **File → New Component → Platform** to create a Vitis platform.
- Using Python CLI, you can use the platform related command to create a platform and the associated domains. For more information about XSCT, see *Vitis Unified Software Platform Documentation: Embedded Software Development* (UG1400).

The platform is an encapsulation of multiple hardware and software components. This capsulation makes it easier to hand off deliveries from hardware-oriented engineers to application developers.

The following files and information are packaged into the platform.

- **Hardware Specification:** This is an extensible XSA file.
- **Software Components:** These are added to the platform as a Linux domain that enables OpenCL runtime. Software components include the following:
 - Boot components
 - BIF file that describes the boot components and their properties for Bootgen to generate the `boot.bin` file.
 - A boot components directory that includes all the files described in the BIF file.
 - Image directory (optional): Contents in this directory will be copied into the FAT32 partition of the final SD card image.
 - Linux domain: The platform requires a Linux domain. The kernel, RootFS, and sysroot info can be added when creating a platform, or when creating an application.
 - Emulation support files (optional)

Root Filesystem

FAT32 and Ext4 partition types are supported by Vitis. The root filesystem is optional in platform creation step because it can be assigned during Vitis application creation step.

An image directory needs to be set during platform creation. All contents in this directory will be packaged into final SD card image. If the target file system is FAT32, the files will be placed to SD card root directory; if the target file system is Ext4, the files will be placed to root directory of the first FAT32 partition.

Boot Components

A BIF file must be provided so that the application build process can package the boot image.

The following is an example of a BIF file:

```
/* linux */
the_ROM_image:
{
  [fsbl_config] a53_x64
  [bootloader] <fsbl.elf>
  [pmufw_image] <pmufw.elf>
  [destination_device=pl] <bitstream>
  [destination_cpu=a53-0, exception_level=e1-3, trustzone] <b131.elf>
  [destination_cpu=a53-0, exception_level=e1-2] <u-boot.elf>
}
```

A boot components directory, including all the files described in the BIF, should also be provided. In this example, the components directory provides `fsbl.elf`, `pmufw.elf`, `b131.elf`, and `u-boot.elf`. These boot components can be generated by PetaLinux or from common images. AMD recommends that you use a common image if there is no system customization.

In the Vitis application building and packaging state, `v++` looks for the files in the boot components directory and replaces the placeholders with real file names and paths. It then calls `Bootgen` to generate `BOOT.BIN`.

Testing Your Platform

Before delivering the platform to the application developers, you should run some basic platform tests to make sure it works properly for acceleration applications.

Generally, you need to make sure the platform can pass these tests:

- **Boot test:** The Vivado project generated implementation result BIT file (from [Adding Hardware Interfaces](#)) and PetaLinux generated images (from [Updating Software Components](#)) should be able to successfully boot to the Linux console.
- **Platforminfo test:** The platform generated in [Packaging a Vitis Acceleration Platform](#) should have a proper platforminfo report for clock and memory information.

- **XRT basic test:** The XRT `xbutil examine` utility should be able to run on the target board and properly report platform information.
- **Vadd test:** Use Vitis to generate a vector addition sample application with the platform. The generated application and `xclbin` should print `test pass` on the board.

Enabling Hardware Emulation for Extensible XSA

The following steps are used for custom platform developers.

1. Create a Vivado project with the necessary BD, RTL, test bench, and other sources.
 - a. Note that in 2020.1, only BD can be used in HW EMU, but starting 2020.2, other sources will also be allowed.
 - b. For Versal adaptive SoCs only, test bench needs to include BD wrapper instead of including BD directly, because Vitis does performs jobs on this level to insert NoC into simulation.
 - c. For Versal adaptive SoCs only, to enable AI Engine in the Vitis platform, the AI Engine block needs to be configured to have only one slave AXI4 Memory-Mapped port enabled and connected to NoC. Vitis based on AI Engine Graph software will make additional auto-connections during `v++` linking stage.
 - d. For DFX platforms, specify correct PFM properties in the dynamic region BD so that the Vitis tools can attach accelerators correctly.
2. Update the design HW Emulation packaging into XSA.
 - a. Before packaging the design into XSA, it is important that your design step through the simulator correctly.
 - b. For Versal adaptive SoCs only, prepare the platform design to enable SystemC models. Update the CIPS and NoC IP setting to change `SELECTED_SIM_MODEL` property to TLM. This ensures that for CIPS IP, the design uses QEMU model on which SW can be run. Similarly, for Zynq 7000 and Zynq UltraScale+ MPSoCs, set the `SELECTED_SIM_MODEL` on the processing system IP instance. Following Tcl command can be used in the design. Also, set the parameter to enable SystemC simulation in Vivado:

```
foreach tlmCell [get_bd_cells * -hierarchical -filter {VLNV =~
":*:axi_noc:*" || VLNV =~ ":*:versal_cips:*"}] {set_property
SELECTED_SIM_MODEL tlm $tlmCell }
set_param bd.generateHybridSystemC true
```

- c. Create a test bench in `sim_1` fileset and instantiate the `<top>` module of your design. For Versal adaptive SoCs, Vivado requires that the user test bench should not instantiate the `<top>` module directly. Instead, it should instantiate `<top>_sim_wrapper` module. A file called `<top>_sim_wrapper.v` is generated when you call the `launch_simulation -scripts_only` command. The interface of this module is the same as your `<top>` module, but it instantiates additional simulations models related to an aggregated NoC module created from various logical NoC modules instantiated in the design. Use the following Vivado Tcl commands to generate the necessary NoC simulation file and use them in your simulation sources.

```
# Ensure that your top of synthesis module is also set as top for
simulation

set_property top <rtl_top> [get_filesets sim_1]

# Generate simulation top for your entire design which would include
# aggregated NOC in the form of xlnoc.bd

launch_simulation -scripts_only
update_compile_order -fileset sim_1

# Set the auto-generated <rtl_top>_sim_wrapper as the sim top

set_property top <rtl_top>_sim_wrapper [get_filesets sim_1]
update_compile_order -fileset sim_1

#Generate the final simulation script which will compile
# the <syn_top>_sim_wrapper and xlnoc.bd modules also
launch_simulation -scripts_only
launch_simulation -step compile
launch_simulation -step elaborate
```

- d. Compile the design, go through the above steps, and start simulation. Because the design is configured to use QEMU, the CIPS IP will not generate any transactions because there is no SW present when doing simulation in Vivado simulator. You will see the following ERROR message in the Vivado simulation, but it indicates that the basic design loads correctly in simulator.

```
#####
#
# Simulation does not work as Versal CIPS Emulation
(SELECTED_SIM_MODEL=tlm) only works with Vitis
tool(launch_emulator.py tool in Vitis)
#
#####

ERROR: [Simtcl 6-50] Simulation engine failed to start: The
Simulation shut down unexpectedly during initialization.
```

Note: To confirm that the design will have correct transactions, you can optionally perform a simulation of the design using the CIPS VIP first before changing it to use TLM (QEMU). First, you must keep the `SELECTED_SIM_MODEL` property to be RTL for NoC and CIPS IP. Also, create a different test bench which drives the CIPS VIP and also meet the requirement of the NoC Verilog model. Refer to the CIPS VIP and NoC IP documentation for additional details on how to set up test bench for Verilog-based simulation.

3. Package the HW Emulation only XSA.



IMPORTANT! The source files for all elements of the Vivado project must be local to the project prior to exporting it as an XSA, or an error can be returned when using the platform in the Vitis tool.

- a. Use **Vivado File → Export → Export Platform** to export a Hardware Emulation platform or use the following Tcl command:

```
set_property platform.platform_state "pre-synth" [current_project]
write_hw_platform -hw_emu -file platform_hw_emu.xsa
```

- b. This XSA can be used with pre-built Linux images or with PetaLinux to create a custom Linux image to create a full platform. Then, the remainder of the Vitis tools can be used to add a kernel to design with the XRT.

Special Considerations for Embedded Platform Creation

Divide Logic Functions to Platform and Kernel

While the designs on FPGA and SoC are getting more complex, it is common for multiple developers or teams to work on a design together. The Vitis software platform provides a clear boundary for application developers and platform developers. Platform developers could include board developers, BSP developers, system software developers, and so on.

In the view of a system architect, some logic functions might be in a gray area: they can be packaged with platforms, or they can work as an acceleration kernel. To help divide the system blocks, here are some general guidelines.

- When a component can be classified as either platform or acceleration kernel, set it as a kernel. The platform should be designed as thin as possible to help reduce the platform repackaging effort when this component needs an update. Also, if you iterate during the development phase, it can save time.
- Platforms should be the abstraction of hardware. When changing a hardware board, the application does not require changes, or very few changes if necessary, to target the new hardware.
- IPs in the platform are controlled by device tree and Linux drivers, acceleration kernels are controlled by XRT. If some IP needs standard Linux driver support, package them into the platform and describe them in the device tree.
- Acceleration kernels usually have standalone features and simple datapath and control path connections. If some components provide infrastructural features shared by multiple kernels or they have complicated connections with other modules, they usually can be packaged in the platform.

The following table shows the recommended platforms and kernels for logic types.

Table 72: Recommended Platforms and Kernels

Logic	Platform	Kernel
Hard Processors (PS of 7000 SoC and Zynq UltraScale+ MPSoC)	Only in Platform	
Soft Processors	Preferred in Platform	OK as an RTL kernel
I/O Interface IP with GPIO and SerDes (External pins, MIPI, Aurora, etc.)	Only in Platform	
I/O Interface Companion IP (DMA for PCIe®, Ethernet MAC for Ethernet PHY, etc.)	Standard and stable IP should be placed in the platform if they are not application specific.	Customized companion IP can work as acceleration kernels.
Traditional memory mapped IP which needs standard Linux driver (VPSS, etc.)	Only in Platform	
HLS AXI memory mapped IP	OK in Platform. Need to write control software or driver manually.	Preferred as Kernel. Controlled by XRT.
Acceleration memory mapped IP follows Vitis kernel register standard and open to XRT		Preferred as Kernel
Vitis Libraries		Only work as Kernel

References

For more information on embedded platforms, see the following links:

- [Vitis Platform Creation tutorial](#)
- [Vitis Embedded Platform Source](#)

Validating an Embedded Platform

As described in [Platform Creation Requirements](#), there are steps you should follow to validate the platform you have created. The following sections describe different actions you can take to examine and exercise the embedded platform for software, graph, and kernel development to ensure the platform meets your needs and expectations.

Check Platform Metadata

During the platform creation process you will define the resources available in the platform. You can perform a quick examination of the platform definition, and the available resources using the `platforminfo` command. This utility can print platform metadata to let you confirm that the defined resources of the packaged platform meet your expectation.

Use the Platform with Simple Test Case

By using the platform with a simple test case, you can ensure that the systems work as expected, without needing to debug the host application, graph, or kernel interactions. A simple test can exercise a specific feature set of the platform, without the complexities of design. AMD provides a number of examples and tutorials for use as test cases, some of which can be found at the following locations:

- https://github.com/Xilinx/Vitis_Accel_Examples
- <https://github.com/Xilinx/Vitis-Tutorials>

A simple test can help to check whether the clock and reset, AXI4 interface settings, and interrupts are correct and functioning as expected.

Run Baremetal Software Emulation

To step back from creating acceleration applications for platform verification, you can run emulation of baremetal host applications to test the hardware inside the platform. This lets you quickly test the embedded platform against the drivers required to access elements of the hardware through the baremetal application. This can help platform developers validate platform peripheral features and custom IP in the platform with simple applications. This validation requires you to convert the extensible platform to a fixed version for baremetal software development. This validation does not need Linux software components.

To create a fixed form of the extensible platform use the following steps.

- **Export a Fixed-XSA from the Extensible Hardware Platform:** As described in [Platform Types](#), there are two general types of platforms: extensible platforms and fixed platforms. Fixed platforms support embedded software development, and do not let you modify the PL kernels or the target platform defined by the device binary. However, the AI Engine flow does let you modify and recompile the graph application on a fixed platform.

A fixed-version of the hardware platform (`.xsa`) can be generated during the linking process of the Vitis compiler (`v++`) when building an Application Project as described in [Building the Device Binary](#).

In the Vitis IDE, the fixed-XSA can be generated by enabling the **Export hardware (XSA)** check box in the Hardware Link Project Settings view. For the command-line flow, the feature must be enabled by adding the following option to a configuration file used during the `v++ --link` command:

```
[advanced]  
param=compiler.addOutputTypes="hw-export"
```


The Vitis compiler exports a fixed hardware specification containing elements of the target platform with any acceleration kernels and the AI Engine graph fixed into the hardware. The fixed-XSA provides a BSP for the system design that you can use for developing embedded software applications.



IMPORTANT! *The fixed-XSA matches the specified build target of the Vitis compiler. So a hardware emulation capable fixed-XSA is generated from the `hw_emu` build target, and a hardware capable fixed-XSA is generated from the `hw` build target.*

- **Generate a Baremetal Platform:**

1. The fixed-XSA can be used to create or generate a baremetal fixed-version of an embedded processor platform, that can be used to validate the platform. Open the Vitis IDE and select **File → New → Platform Project**. This opens the New Platform Project wizard.
2. Specify the Platform project name and click **Next**. This opens the Platform page as shown in the figure below.

3. In this dialog box you can define the platform by specifying the name of the XSA File, selecting the Operating system, Processor domain, and Generate boot components for the platform.

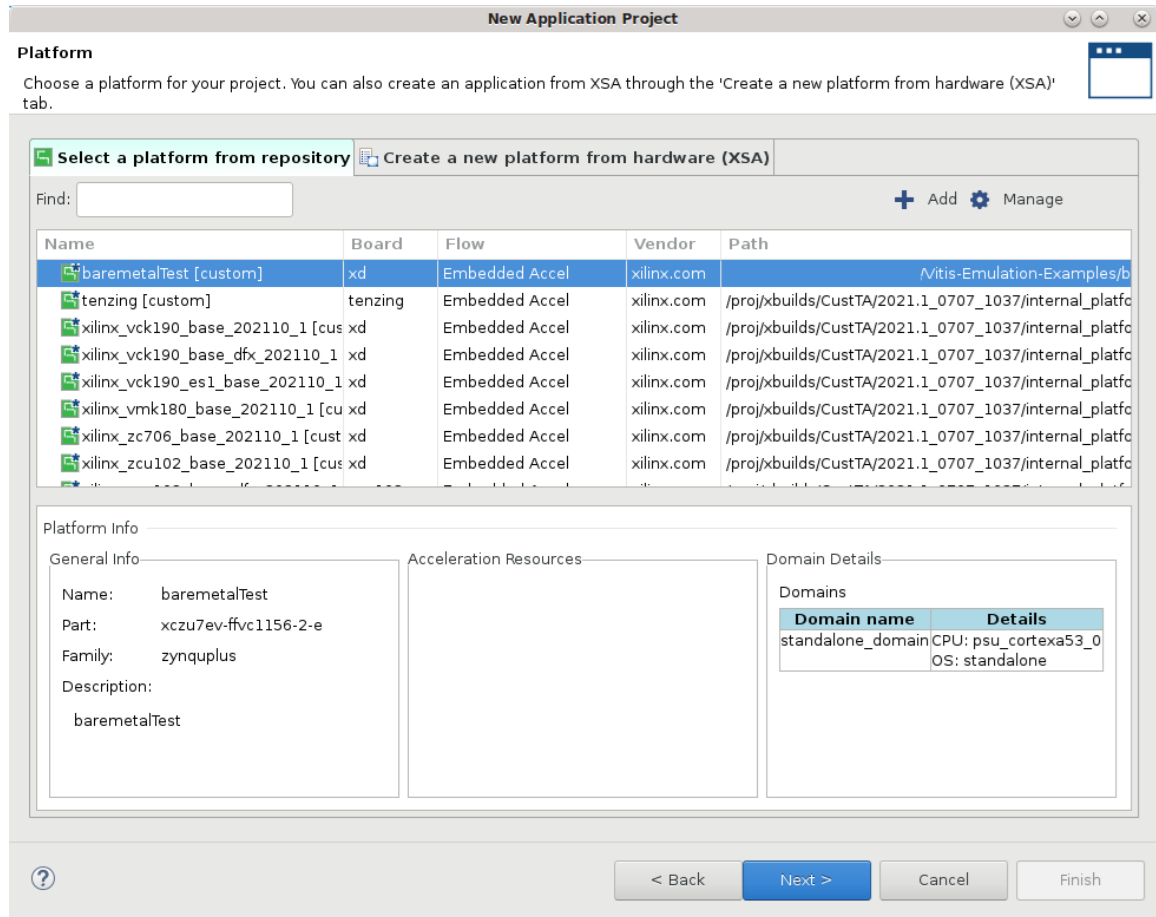
Select **standalone** for Operating system, specify the Processor, and Architecture to define the domain. Enable Generate boot components as shown above, and click **Finish** to generate the baremetal embedded platform.

When the platform is complete, you should see it added to your repository of available platforms. You will use it to create a baremetal application project as described next.

- **Validate the Baremetal Platform with an Application Project:** With the baremetal platform built, you can use a simple application running on the Arm processor to validate the platform. This approach lets you quickly exercise various features of the platform from compiled ELF files.

1. In the Vitis IDE, select **File → New → Application Project**. This opens the New Application Project wizard.

On the Platform page, select the baremetal platform you created in the prior step, and click **Next** to continue.



- The New Application Project wizard presents the Application Project Details page next. Specify the Project name and the System project name, and click **Next** to continue.
- The Domain page displays the domain information for the selected platform. Click **Next** to proceed.
- The Templates page displays a list of Embedded software development templates for the selected platform. Select the **Hello World** template, or another simple application, or select the **Empty Application** to let you specify your own source files, and click **Finish** to create the application project.
- With the baremetal application project in support of the baremetal platform, you are now ready to begin validation. You can use the baremetal platform with the same build target that the fixed-XSA was created from:
 - If the fixed-XSA was generated from a hardware emulation build (`v++ --link -t hw_emu`), it can be used for Emulation-HW builds in the baremetal application project.
 - If the fixed-XSA was generated from a hardware build (`v++ --link -t hw`), it can be used for Hardware builds in the baremetal application project.

In the System Project Settings window, specify the Active build configuration that is compatible with the fixed-XSA of the baremetal platform, as shown below.



TIP: You might also need to specify an `.xclbin` file for the baremetal application project in the Packaging options field as shown in the figure below. This is necessary for the hardware build, and you can copy the `.xclbin` file from the source project of the fixed-XSA into your baremetal Application Project.

System Project Settings Active build configuration: Hardware

General

Project name: `bareMetalApp_system`

Platform: `baremetalTest` ...

Runtime: C/C++

Options

Target: Hardware

Sysroot: ...

Root FS: ...

Kernel Image: ...

Packaging options: `binary_container_1.xclbin` ...

Application Projects

Domain Name	Application Project Name	Build Configuration
standalone_domain	bareMetalApp	Debug

- For the Emulation-HW build configuration, you can run the application in the QEMU environment and see the application interacting with the fixed-platform to validate the system. To run the emulation build, select the **Launch HW Emulator** command from the Assistant view, or the main toolbar menu. QEMU is launched through the TCF agent and in the Emulation Console view you can observe the transcript of QEMU booting and the PS application running.



IMPORTANT! Remember to set the heap/stack size to 2MB in the `lscript.ld` file, or a "Terminate" error will be displayed.



TIP: You can also select the **Debug As** command to launch the interactive debug environment in the Vitis IDE.

Creating a Platform Component

A platform is the starting point of your AMD Vitis™ design. Vitis applications are built on top of the platforms.

An embedded platform includes a hardware platform and a software platform.

Hardware Platform

The hardware platform is the static, unchanging portion of your hardware design. It includes the Xilinx Support Archive (XSA) file exported from the AMD Vivado™ Design Suite.

Hardware platform is divided into two main categories: fixed hardware platform and extensible hardware platform.

The fixed platform contains the hardware specifications like processor configuration properties, peripheral connection information, address map, and device initialization code.

The extensible hardware platform contains the hardware specifications as well as the acceleration resources that can be used by acceleration applications, for example, Input and output interfaces, clocks, AXI buses, and interrupts. Vitis adds kernels and infrastructure modules to the hardware design as needed to facilitate data movement. Acceleration kernels can share data with platform IPs, but cannot change or modify them. For information about setting up the hardware platform, refer to [Installing Xilinx Runtime and Platforms](#).

You can also create the hardware platform using the `--part` option without creating a complete platform design in the Vivado Design Suite. This is a recommended method when targeting devices for which evaluation boards or platforms are not yet available. Using this flow, you can start the development of any AI Engine components, and HLS components of your system design. The steps for the flow are as follows:

1. Compile the AI Engine component with `--part`:

```
v++ -c --mode aie --part ...
```

2. Generate HLS component using `part` specified in the `hls_config.cfg` file:

```
v++ --mode hls --config ...
```

3. Link the system and generate an XSA using `--part`:

```
v++ --link -t hw --part ... -o fixed.xsa
```

4. Package the system and generate the XCLBIN from the fixed XSA:

```
v++ --package --platform fixed.xsa ...
```

Software Platform

The software platform is the environment that runs the software to control acceleration kernels for acceleration applications. It includes the domain setup and boot components setup.

By default, all AMD pre-built platforms have a Linux domain that has enabled Xilinx Runtime (XRT) so that acceleration applications can run on this environment. The pre-built binaries for Linux kernel image and rootfs are located in a separate download file on the PetaLinux download page. See the "Common images for Embedded Vitis platforms" section of the [Xilinx download center](#). Because the device tree is unique to each platform, it is provided as a component with the Linux XRT domain inside the platform.

Linux Domain Components must be provided when there is a Linux domain in the embedded platform. These components can be generated by PetaLinux, Yocto, or third-party frameworks. Because these components can be shared across all AMD demo boards for the given FPGA family, a common Linux component image generated by PetaLinux is provided for AMD Zynq™ 7000 SoC and AMD Zynq™ UltraScale+™ MPSoCs.

The following Linux images can be downloaded from the PetaLinux download page:

- **Root File System (RFS):** Includes binaries, libraries, and setups for a Linux file system. In the AMD-provided common rootfs, XRT has been installed so that acceleration application can run on this Linux environment.
- **Kernel Image:** A compiled Linux kernel. The common kernel image provided by AMD includes most AMD peripheral drivers.
- **Sysroot:** Used for cross compilation. It provides the libraries to be linked when compiling applications for a target system.

Note: Optionally, you can pack Linux domain components into embedded platforms. When creating a Linux application in the Vitis IDE, the Linux domain components in the platform setting will be the default and initial settings if they have been set in the platform. You can overwrite these settings with components installed elsewhere.

AMD pre-built embedded platforms and pre-built common Linux components are provided in separate download files. You can regenerate the common Linux components from the platform source files hosted on the [Vitis Embedded Platform](#) GitHub repository by setting the environment variable `COMMON_RFS_KRNL_SYSROOT=FALSE` before running `make`.

Creating a hardware platform

The hardware platform is divided into two main categories: fixed hardware platform and extensible hardware platform. The following process describes how to create both types of hardware platforms.

Creating a Fixed Hardware Platform

AMD hardware designs are created with the Vivado Design Suite, and can be exported in the proprietary support archive (XSA) file format that can be then used as an element of a Vitis unified software platform, or directly by the Vitis tool for use in a component or project.

The generic steps for creating a fixed-XSA design are as follows:

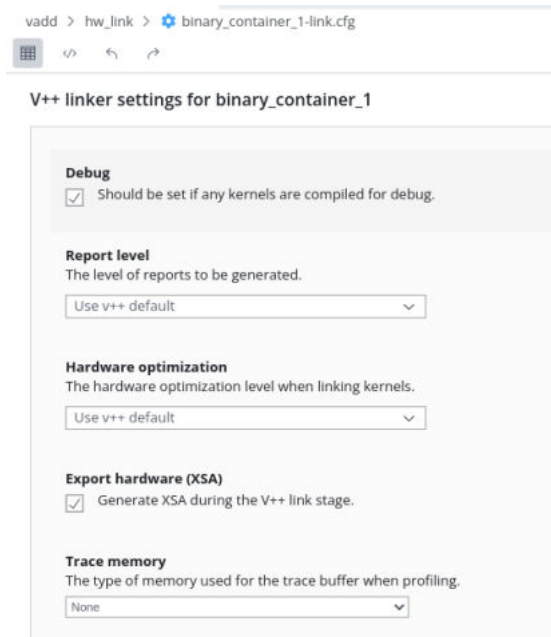
1. Create a Vivado project.
2. Create a block design.
3. Generate the image or bitstream.
4. Export the hardware by selecting **File → Export → Export Hardware**.

For information on creating an embedded design in Vivado Design Suite and generating an XSA file, see the following embedded design tutorials:

- Embedded Design Tutorials: Versal Adaptive Compute Acceleration Platform ([UG1305](#))
- Zynq UltraScale+ MPSoC: Embedded Design Tutorial ([UG1209](#))
- Zynq 7000 SoC: Embedded Design Tutorial ([UG1165](#))

A fixed XSA can be also exported from an extensible platform-based System project during the hardware link process. In the `hw_link/system.cfg` file of a System project select **Export hardware (XSA)** option as shown in the figure below. The hardware link process will generate a fixed XSA file when running the `v++ --link` command. Refer to [Defining the HW_Link System Configuration](#) for more information.

Figure 124: Export Hardware XSA



Creating an Extensible Hardware Platform

Three methods could be used to create an extensible hardware platform:

1. Create a extensible hardware platform from scratch.
2. Add the necessary hardware interfaces based on fixed hardware platform and export extensible XSA file.
3. Leverage CED example design to create hardware platform and export extensible XSA file.

For this part, refer to extensible platform creation in [Creating a hardware platform](#).

Preparing the software component

Prepare the Common Image

You are recommended to use a common image for the software components of your platform. Refer to [Common Images for Embedded Vitis Platforms](#) for more information. If you need to create a PetaLinux image refer to [Vitis Tutorial PetaLinux Customization](#).

Create a Device Tree Binary (DTB) File

Use the `createdts` command in XSCT tool to generate a DTB file. The `zocl` driver interface requires a device tree node to enable the interrupt connection. Add the `-zocl` option when using this command. The following code shows the usage of this command with its options.

```
createdts -hw <full path of XSA file> -zocl -platform-name mydevice \  
-git-branch xlnx_rel_v202X.X -board zcu104-revc -compile
```

- `-platform-name`: platform name.
- `-hw`: hardware XSA file with path.
- `-git-branch`: device tree branch.
- `-board`: board name of the device. You can check the board name at `/device_tree/data/kernel_dtsi`.
- `-zocl`: enable the zocl driver support.
- `-compile`: option to compile the device tree.

The `system.dtb` file is created in the `<mydevice>/psu_cortexaXX_0/device_tree_domain/bsp` folder.

Packaging a platform component

There are a number of ways to create a Platform component in the new Vitis Unified IDE. The following steps describe a recommended approach. For a description of the views and pages mentioned in the steps below refer to [Working with the Vitis Unified IDE](#).

1. With the Vitis IDE open, from File menu options, click **New Component → Platform**.



TIP: You can also select the **Create Platform Component** command from the Welcome page, or from the right-click menu in the Explorer view.

2. This opens the Name and Location page of the Create Platform Component wizard as shown.

The screenshot shows the 'Create Platform Component' wizard with the 'Name and Location' step selected. The breadcrumb navigation is 'Name and Location > XSA > OS and Processor > Summary'. The instruction says 'Choose a name for your component and specify a directory where component data files will be stored'. The 'Component name' field contains 'platform'. The 'Component location' dropdown shows '/group/xcoswmtg/andyh/rigel-tests/work17' with a 'Browse' button next to it. Below the fields, it states 'Component will be created at /group/xcoswmtg/andyh/rigel-tests/work17/platform'. At the bottom right are 'Cancel' and 'Next' buttons.

3. Enter a Component name and Component location and select **Next**.
4. This opens the Select XSA page of the Create Platform Component wizard as shown below.

Figure 125: Create Platform Component - XSA

The screenshot shows the 'Create Platform Component' wizard with the 'Select XSA' step selected. The breadcrumb navigation is 'Name and Location > XSA > OS and Processor > Summary'. The instruction says 'Specify an XSA for your component.'. The 'XSA' dropdown menu is open, showing a list of options: 'vck190', 'vck190' (highlighted), 'vmk180', 'zcu102', and 'zcu106'. A 'Browse' button is to the right of the dropdown. At the bottom left is a 'Back' button, and at the bottom right are 'Cancel' and 'Next' buttons.

5. You can specify the XSA file for your platform component from one of the base platforms available with the Vitis tools located at `$XILINX_VITIS/base_platforms`, or you can select **Browse** to locate an XSA from your `$PLATFORM_REPO_PATHS` or an existing system design. Select **Next** to proceed to Select Operating System and Processor.

Note: If the XSA you selected is a fixed hardware platform, then this Vitis platform is created for embedded design only. If you selected an extensible hardware platform, then this Vitis platform can be used for embedded design, in addition to acceleration applications.

6. After selecting the XSA, the tool reads the XSA and identifies the available processors and operating system domains. From the Select Operating System and Processor page of the Create Platform Component wizard, you can select which OS and processor your platform uses.

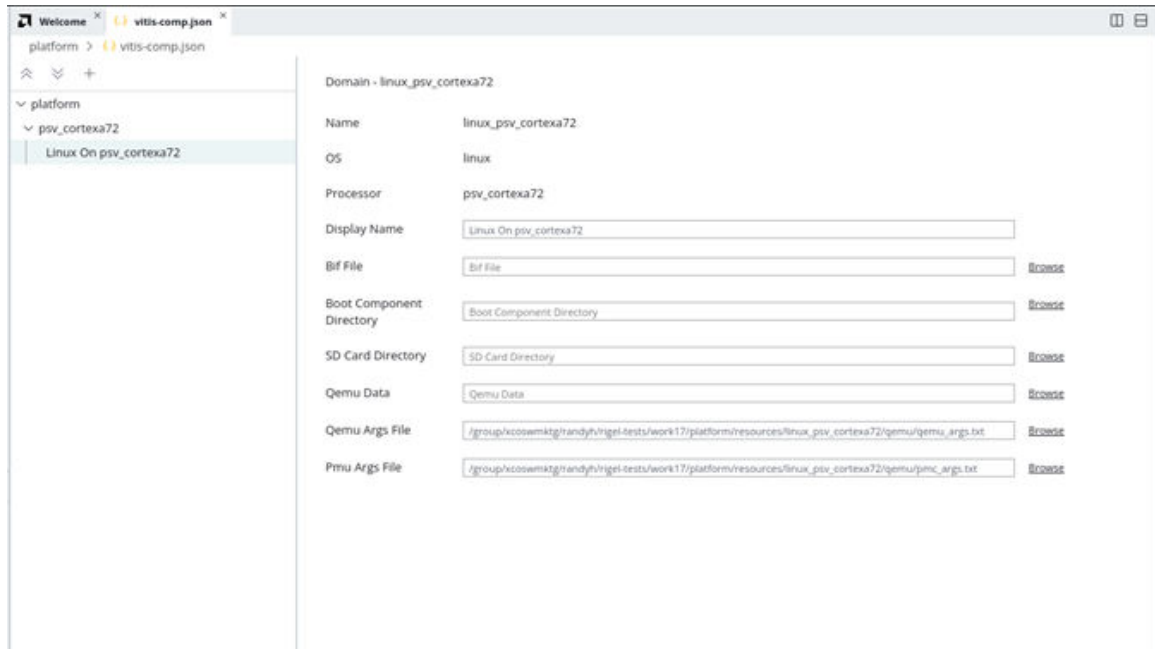
Figure 126: Create Platform Component - OS and Processor

The screenshot shows the 'Create Platform Component' wizard window. The title bar says 'Create Platform Component'. The breadcrumb navigation is 'Name and Location > XSA > OS and Processor > Summary'. The current step is 'Select Operating System and Processor'. Below the title, there is a descriptive text: 'Specify the details for the initial domain to be added to the platform component. More domains can be added, and can edit the domain settings later by opening the platform editor.' There are two dropdown menus: 'Operating system:' with 'standalone' selected, and 'Processor:' with 'psv_cortexa72_0' selected. At the bottom, there are three buttons: 'Back' (disabled), 'Cancel' (disabled), and 'Next' (active).

7. Specify the Operating system, and the Processor for the platform, and click **Next** to proceed to the Summary page.
8. The Summary page reflects the choices you have made on the prior pages. Review the summary and select **Finish** to create the Platform component, or select **Back** to return to earlier pages and change your selections.

When the Platform component is created the `vitis-comp.json` file for the component is opened in the central editor window as shown for the Linux platform below.

Figure 127: Platform Component - vitis-comp.json



Depending on the OS you selected for your platform, and possibly the processor you chose as well, the contents of the platform `vitis-comp.json` can vary.

- For Linux Operating System: As shown above, you need to specify the Bif file, Boot Component Directory, SD Card Directory and as well as the Qemu data. The Qemu Args file are auto-populated by tool.
- For standalone (baremetal) OS: No special operation is required.
- For FreeRTOS: No special operation is required.

After setting, you can build the Platform component by selecting it in the Flow Navigator and selecting the **Build** command. After compilation is finished the tool will populate the platform and its related software and hardware components in the `Output` folder of the Platform component in the Component Explorer.

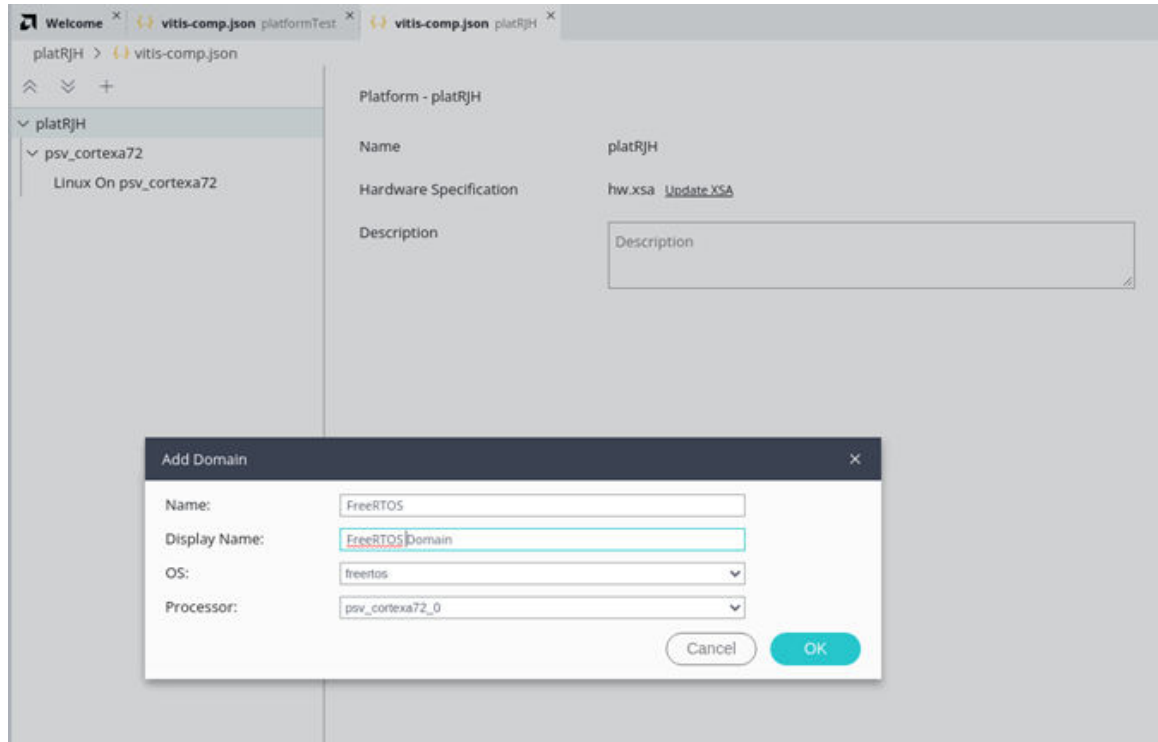
Add a Domain to an Existing Platform

A platform can contain multiple domains, supporting different operating systems and targeting different processors. A Platform component supports three different types of domains: Linux, FreeRTOS, and Standalone (or Baremetal).

1. In the current workspace, expand the Platform component in the Component Explorer to open the Settings folder and select the `vitis-comp.json` file

Note: To create a Platform component refer to [Creating a Platform Component](#).

- Click the '+' button to Add Domain.



- Specify the Name and Display name.
- For the OS select Linux, FreeRTOS, Standalone, or AI Engine Runtime.
- Select from the available Processors. The selection of Processor changes based on the selected OS.
- Click **OK** to add the domain to the Platform.

For a FreeRTOS and Standalone domains a Board Support Package (or BSP) is created for the domain. You can specify the libraries to include in the BSP.

For a Linux domain you can configure additional details of the Linux domain by selecting the new domain in the platform. The Boot Components Directory must contain all the components required by the BIF. These components can be generated by PetaLinux.



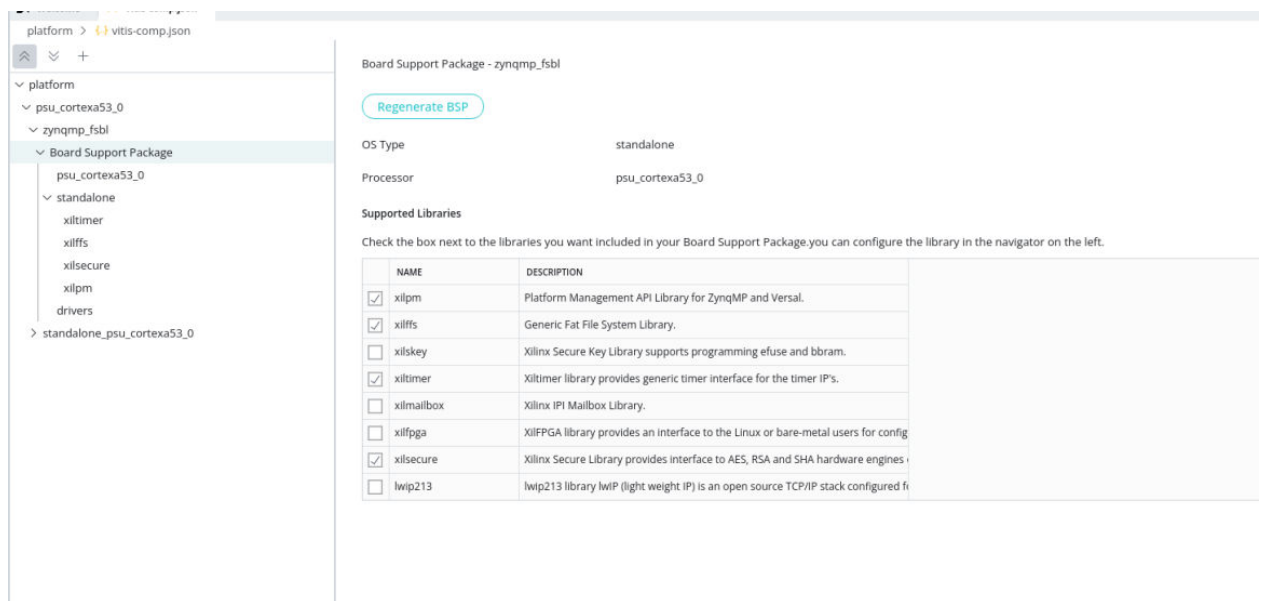
TIP: The components specified in the Linux domain settings will be copied to the platform folder when generating the platform. Adding sysroot to a Linux domain is not supported because Windows does not support copying symbol links in sysroot.

Configuring the BSP (Board support package)

You can modify and configure the BSP code for your standalone and freertos domains..

1. Select the Platform component, expand the **Settings** and click the `vitis-comp.json` file to open it.
2. Select the **Board Support Package** to expand it, and modify, configure it and select the modules according to your requirements.

Figure 128: Platform Component - Board Support Package



3. After setting, you can build the Platform component by selecting it in the Flow Navigator and selecting the **Build** command. After compilation is finished the tool will populate the platform and its related software and hardware components in the `Output` folder of the Platform component in the Component Explorer.

Note: If user does incorrect modifications to the source code and want to revert to the original source code, they can click **Regenerate BSP** button.

Platform Component Command Line Flow

You can also use the command-line flow in the interactive mode to create a Platform component using the steps outlined below.

1. Use the following command to start the Vitis tool in interactive mode:

```
vitis -i
```

2. Import the Vitis tool library:

```
import vitis
```

3. Create a client:

```
client = vitis.create_client()
```

This takes a few seconds, and the tool returns with a `Vitis Server started` message.

4. Specify the workspace to add the Platform component to:

```
client.set_workspace(path="<workspace>")
```

5. Create a platform, specifying the Name, the XSA file, the Operating System, and Processor:

```
platform =  
client.create_platform(name="Platform1",hw="<path_to_xsa>",os="<Operating  
System>",cpu="<processor>")
```

- The XSA file can be specified from one of the supported base platforms found in `$XILINX_VITIS/base_platforms`, or can be specified from `$PLATFORM_REPO_PATHS` or custom designs
- The supported OS are `linux`, `standalone`, `freertos`
- The processors are dependent on the XSA, and can include `psv_cortexa72`, `psv_cortexr5`, `psv_pmc`, `psv_psm` for example.

6. Add domain to a platform:

```
platform = client.get_platform_component(name="<platform name>")  
domain = platform.add_domain(cpu = "processor",os = "Operating  
System",name = "domain name",display_name = "displayed domain name")
```

- The supported OS are `linux`, `standalone`, `freertos`
- The processors are dependent on the XSA, and can include `psv_cortexa72`, `psv_cortexr5`, `psv_pmc`, `psv_psm` for example.

7. For a standalone or freertos domain user could modify the BSP.

```
platform = client.get_platform_component(name="<platform name>")  
domain = platform.get_domain(name="< >")  
status = domain.set_lib(lib_name="<>")  
status = domain.remove_lib(lib_name="< >")
```

- User should get the corresponding platform and domain and then select the module you need or want to delete.

- For a Linux system, add the boot component directory, BIF file, and the SD card directory. First user should get the OS Domain that was created from the concatenation of OS and Processor.

```
platform = client.get_platform_component(name="<platform name>")
domain = platform.get_domain(name="<os_processor>")
status = domain.add_boot_dir(boot_dir="<boot component directory>")
status = domain.add_bif(file_name="<bif file location>")
status = domain.set_sd_dir(path="<sd component directory>")
```

- Build the Platform component. User need to get the corresponding platform and then build it.

```
platform = client.get_platform_component(name="<platform name>")
status = platform.build()
```

After building, the platform is located in the workspace directory. You can also put these commands into a python script (.py) and use the batch mode to source the script as follows.

```
vitis -s <script>.py
```



TIP: You can use *vitis_journal.py*, the log of the commands that were executed in interactive mode, as a source for your script.

Example 1

Example to create a Platform component with a Linux domain:

```
import vitis
client = vitis.create_client()
client.set_workspace(path="new_platform")
platform = client.create_platform(name = "platform1",hw="./src/vck190_custom_hw.xsa",os="linux",cpu="psv_cortexa72")
platform = client.get_platform_component(name="platform1")
domain = platform.get_domain(name="linux_psv_cortexa72")
status = domain.add_boot_dir(boot_dir="./src/boot")
status = domain.set_sd_dir(path="./src/sd_dir")
status = domain.add_bif(file_name="./src/linux.bif")
platform = client.get_platform_component(name="platform1")
domain = platform.add_domain(cpu = "psv_cortexr5_0",os = "freertos",name = "freertos_domain",display_name = "freertos_domain")
domain = platform.get_domain(name="freertos_domain")
status = domain.set_lib(lib_name="xilmailbox")
status = domain.remove_lib(lib_name="xilmailbox")
platform = client.get_platform_component(name="platform1")
status = platform.build()
```


Example 2

Example to create a Platform component with a standalone domain:

```
import vitis
client = vitis.create_client()
client.set_workspace(path="new_platform")
platform = client.create_platform(name="platform2", hw="./src/
vck190_custom_hw.xsa", os="standalone", cpu="psv_cortexa72_0")
platform = client.get_platform_component(name="platform2")
status = platform.build()
```

Additional Information

This section contains the following chapters:

- [Accelerating Data Center Applications with Vitis Software Platform](#)
- [OpenCL Programming](#)
- [Streaming Data Transfers](#)
- [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#)
- [Migrating to a New Target Platform](#)
- [Output Structure of the Vitis Tools](#)
- [Using Vitis System Compilation Mode](#)

Accelerating Data Center Applications with Vitis Software Platform

Introduction

This content focuses on Data Center applications and PCIe®-based acceleration cards, but the concepts developed here are also generally applicable to embedded applications.

Acceleration: An Industrial Analogy

There are distinct differences between CPUs, GPUs, and programmable devices. Understanding these differences is key to efficiently developing applications for each kind of device and achieving optimal acceleration.

Both CPUs and GPUs have pre-defined architectures, with a fixed number of cores, a fixed-instruction set, and a rigid memory architecture. GPUs scale performance through the number of cores and by employing SIMD/SIMT parallelism. In contrast, programmable devices are fully customizable architectures. The developer creates compute units that are optimized for application needs. Performance is achieved by creating deeply pipelined datapaths, rather than multiplying the number of compute units.

Think of a CPU as a group of workshops, with each one employing a very skilled worker. These workers have access to general purpose tools that let them build almost anything. Each worker crafts one item at a time, successively using different tools to turn raw material into finished goods. This sequential transformation process can require many steps, depending on the nature of the task. The workshops are independent, and the workers can all be doing different tasks without distractions or coordination problems.

A GPU also has workshops and workers, but it has considerably more of them, and the workers are much more specialized. They have access to only specific tools and can do fewer things, but they do them very efficiently. GPU workers function best when they do the same few tasks repeatedly, and when all of them are doing the same thing at the same time. After all, with so many different workers, it is more efficient to give them all the same orders.

Programmable devices take this workshop analogy into the industrial age. If CPUs and GPUs are groups of individual workers taking sequential steps to transform inputs into outputs, programmable devices are factories with assembly lines and conveyer belts. Raw materials are progressively transformed into finished goods by groups of workers dispatched along assembly lines. Each worker performs the same task repeatedly and the partially finished product is transferred from worker to worker on the conveyer belt. This results in a much higher production throughput.

Another major difference with programmable devices is that the factories and assembly lines do not already exist, unlike the workshops and workers in CPUs and GPUs. To refine our analogy, a programmable device would be like a collection of empty lots waiting to be developed. This means that the device developer gets to build factories, assembly lines, and workstations, and then customizes them for the required task instead of using general purpose tools. Similar to lot size, device real-estate is not infinite, which limits the number and size of the factories which can be built in the device. Properly architecture and configuring these factories is therefore a critical part of the device programming process.

Traditional software development is about programming functionality on a pre-defined architecture. Programmable device development is about programming an architecture to implement the desired functionality.

Methodology Overview

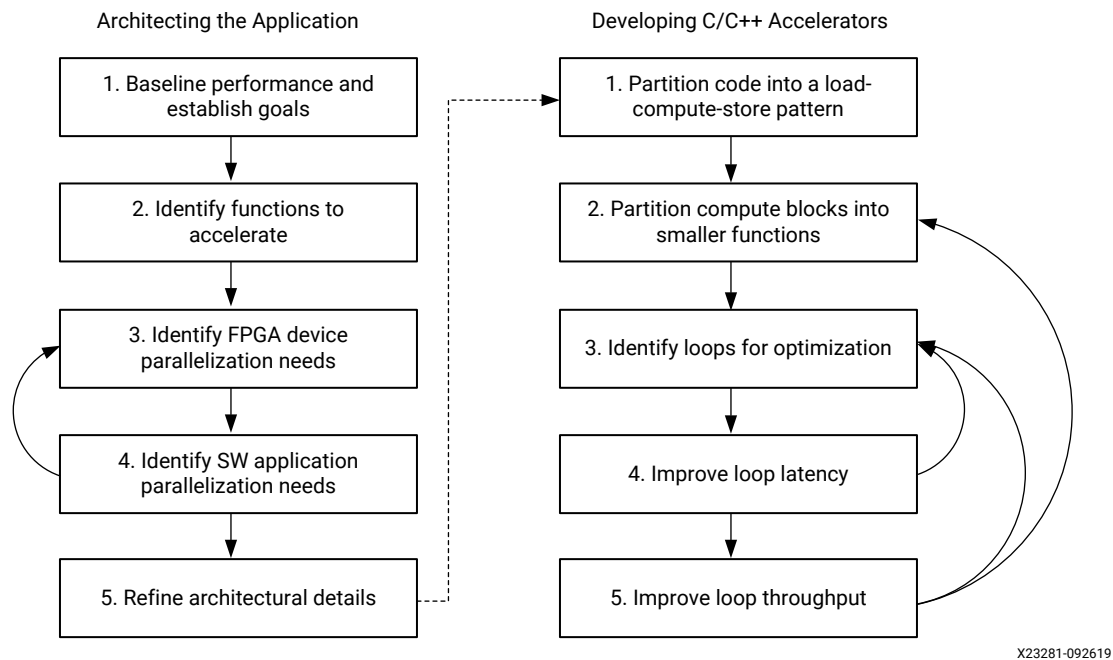
The methodology is comprised of two major phases:

1. Architecting the application
2. Developing the C/C++ kernels

In the first phase, the developer makes key decisions about the application architecture by determining which software functions should be mapped to device kernels, how much parallelism is needed, and how it should be delivered.

In the second phase, the developer implements the kernels. This primarily involves structuring source code and applying the desired compiler pragma to create the desired kernel architecture and meet the performance target.

Figure 129: Methodology Overview



Performance optimization is an iterative process. The initial version of an accelerated application will likely not produce the best possible results. The methodology described in this guide is a process involving constant performance analysis and repeated changes to all aspects of the implementation.

Recommendations

A good understanding of the AMD Vitis™ unified software platform programming and execution model is critical to embarking on a project with this methodology. The following resources provide the necessary knowledge to be productive with the Vitis software platform:

- [Section III: Developing Applications](#)
- [Vitis Application Acceleration Development Flow Tutorials](#) on GitHub.

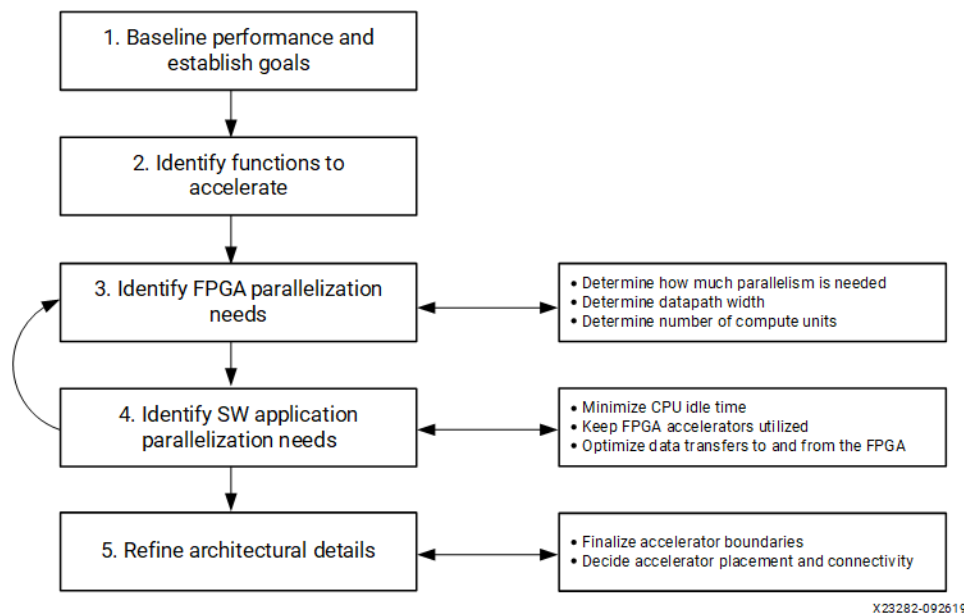
In addition to understanding the key aspects of the Vitis software platform, a good understanding of the following topics will help achieve optimal results with this methodology:

- Application domain
- Software acceleration principles
- Concepts, features and architecture of device
- Features of the targeted device accelerator card and corresponding target platform
- Parallelism in hardware implementations (<http://kastner.ucsd.edu/hlsbook/>)

Methodology for Architecting a Device Accelerated Application

Before beginning the development of an accelerated application, it is important to architect it properly. In this phase, the developer makes key decisions about the architecture of the application and determines factors such as what software functions should be mapped to device kernels, how much parallelism is needed, and how it should be delivered.

Figure 130: Methodology for Architecting the Application



This section walks through the various steps involved in this process. Taking an iterative approach through this process helps refine the analysis and leads to better design decisions.

Step 1: Establish a Baseline Application Performance and Establish Goals

Start by measuring the runtime and throughput performance, to identify bottlenecks of the current application running on your existing platform. These performance numbers should be generated for the entire application (end-to-end), in addition for each major function in the application. The most effective way is to run the application with profiling tools, like `valgrind`, `callgrind`, and `GNU gprof`. The profiling data generated by these tools show the call graph with the number of calls to all functions and their execution time. These numbers provide the baseline for most of the subsequent analysis process. The functions that consume the most execution time are good candidates to be offloaded and accelerated onto FPGAs.

Measure Running Time

Measuring running time is a standard practice in software development. This can be done using common software profiling tools such as gprof, or by instrumenting the code with timers and performance counters.

The following figure shows an example profiling report generated with gprof. Such reports conveniently show the number of times a function is called and the amount of time spent (runtime).

Figure 131: Gprof Output Example

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
70.45	25.54	25.54	256	99.76	99.76	F4(int*, int*, int*)
12.44	30.05	4.51	256	17.61	17.61	F2(int*, int*)
9.91	33.64	3.59	256	14.03	14.03	F1(int*, int*, int*)
7.83	36.48	2.84	256	11.08	11.08	F3(int*, int*)
0.00	36.48	0.00	256	0.00	142.48	F(int*, int*)

Measure Throughput

Throughput is the rate at which data is being processed. To compute the throughput of a given function, divide the volume of data the function processed by the running time of the function.

$$T_{SW} = \max(V_{INPUT}, V_{OUTPUT}) / \text{Running Time}$$

Some functions process a pre-determined volume of data. In this case, simple code inspection can be used to determine this volume. In some other cases, the volume of data is variable. In this case, it is useful to instrument the application code with counters to dynamically measure the volume.

Measuring throughput is as important as measuring running time. While device kernels can improve overall running time, they have an even greater impact on application throughput. As such, it is important to look at throughput as the main optimization target.

Determine the Maximum Achievable Throughput

In most device-accelerated systems, the maximum achievable throughput is limited by the PCIe® bus. PCIe performance is influenced by many different aspects, such as motherboard, drivers, target platform, and transfer sizes. Run DMA tests upfront to measure the effective throughput of PCIe transfers and thereby determine the upper bound of the acceleration potential, such as the xbutl dma test.

Figure 132: Sample Result of `dmatest` on an Alveo U200 Data Center Accelerator Card

```
$ xbutil dmatest
INFO: Found total 1 card(s), 1 are usable
Total DDR size: 65536 MB
Reporting from mem_topology:
Data Validity & DMA Test on bank0
Host -> PCIe -> FPGA write bandwidth = 11381.7 MB/s
Host <- PCIe <- FPGA read bandwidth  =  8358.9 MB/s
Data Validity & DMA Test on bank1
Host -> PCIe -> FPGA write bandwidth = 11235.3 MB/s
Host <- PCIe <- FPGA read bandwidth  =  7485.3 MB/s
INFO: xbutil dmatest succeeded.
```

An acceleration goal that exceeds this upper bound throughput cannot be met as the system is I/O bound. Similarly, when defining kernel performance and I/O requirements, keep this upper bound in mind.

Establish Overall Acceleration Goals

Determining acceleration goals early in the development is necessary because the ratio between the acceleration goal and the baseline performance drives the analysis and decision-making process.

Acceleration goals can be hard or soft. For example, a real-time video application could have the hard requirement to process 60 frames per second. A data science application could have the soft goal to run 10 times faster than an alternative implementation.

Either way, domain expertise is important for setting obtainable and meaningful acceleration goals.

Step 2: Identify Functions to Accelerate

After establishing the performance baseline, the next step is to determine which functions should be accelerated in the device.



TIP: Minimize changes to the existing code at this point so you can quickly generate a working design on the FPGA and get the baselined performance and resource numbers.

When selecting functions to accelerate in hardware, there are two aspects to consider:

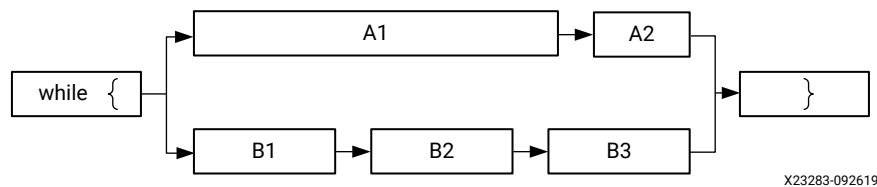
- **Performance bottlenecks:** Which functions are in application hot spots?
- **Acceleration potential:** Do these functions have the potential for acceleration?

Identify Performance Bottlenecks

In a purely sequential application, performance bottlenecks can be easily identified by looking at profiling reports. However, most real-life applications are multi-threaded and it is important to take the effects of parallelism in consideration when looking for performance bottlenecks.

The following figure represents the performance profile of an application with two parallel paths. The width of each rectangle is proportional to the performance of each function.

Figure 133: Application with Two Parallel Paths



The above performance visualization in the context of parallelism shows that accelerating only one of the two paths does not improve the application's overall performance. Because paths A and B re-converge, they are dependent upon each other to finish. Likewise, accelerating A2, even by 100x, does not have a significant impact on the performance of the upper path. Therefore, the performance bottlenecks in this example are functions A1, B1, B2, and B3.

When looking for acceleration candidates, consider the performance of the entire application, instead of only individual functions.

Identify Acceleration Potential

A function that is a bottleneck in the software application does not necessarily have the potential to run faster in a device. A detailed analysis is usually required to accurately determine the real acceleration potential of a given function. However, some simple guidelines can be used to assess if a function has potential for hardware acceleration:

- What is the computational complexity of the function?

Computational complexity is the number of basic computing operations required to execute the function. In programmable devices, acceleration is achieved by creating highly parallel and deeply pipelined data paths. These would be the assembly lines in the earlier analogy. The longer the assembly line and the more stations it has, the more efficient it is compared to a worker taking sequential steps in his workshop.

Good candidates for acceleration are functions where a deep sequence of operations needs to be performed on each input sample to produce an output sample.

- What is the computational intensity of the function?

Computational intensity of a function is the ratio of the total number of operations to the total amount of input and output data. Functions with a high computational intensity are better candidates for acceleration because the overhead of moving data to the accelerator is comparatively lower.

- What is the data access locality profile of the function?

The concepts of data reuse, spatial locality, and temporal locality are useful to assess how much overhead of moving data to the accelerator can be optimized. Spatial locality reflects the average distance between several consecutive memory access operations. Temporal locality reflects the average number of access operations for an address in memory during program execution. The lower these measures the better, because it makes data more easily cacheable in the accelerator, reducing the need to expensive and potentially redundant accesses to global memory.

- How does the throughput of the function compare to the maximum achievable in a device?

Device-accelerated applications are distributed, multi-process systems. The throughput of the overall application does not exceed the throughput of its slowest function. The nature of this bottleneck is application specific and can come from any aspect of the system: I/O, computation or data movement. The developer can determine the maximum acceleration potential by dividing the throughput of the slowest function by the throughput of the selected function.

$$\text{Maximum Acceleration Potential} = T_{\text{Min}} / T_{\text{SW}}$$

On AMD Alveo™ Data Center accelerator cards, the PCIe bus imposes a throughput limit on data transfers. While it may not be the actual bottleneck of the application, it constitutes a possible upper bound and can therefore be used for early estimates. For example, considering a PCIe throughput of 10 Gb/s and a software throughput of 50 MB/s, the maximum acceleration factor for this function is 200x.

These four criteria are not guarantees of acceleration, but they are reliable tools to identify the right functions to accelerate on a device.

Step 3: Identify Device Parallelization Needs

After the functions to be accelerated are identified and the overall acceleration goals are established, the next step is to determine what level of parallelization is needed to meet the goals.

The factory analogy is again helpful to understand what parallelism is possible within kernels.

As described, the assembly line allows the progressive and simultaneous processing of inputs. In hardware, this kind of parallelism is called pipelining. The number of stations on the assembly line corresponds to the number of stages in the hardware pipeline.

Another dimension of parallelism within kernels is the ability to process multiple samples at the same time, similar to putting multiple samples on the conveyor belt at the same time. To accommodate this, the assembly line stations are customized to process multiple samples in parallel. This is effectively defining the width of the datapath within the kernel.

Performance can be further scaled by increasing the number of assembly lines. This can be accomplished by putting multiple assembly lines in a factory, and also by building multiple identical factories with one or more assembly lines in each of them.

The developer needs to determine which combination of parallelization techniques is most effective at meeting the acceleration goals.

Estimate Hardware Throughput without Parallelization

The throughput of the kernel without any parallelization can be approximated as:

$$T_{HW} = \text{Frequency} / \text{Computational Intensity} = \text{Frequency} * \max(V_{INPUT}, V_{OUTPUT}) / V_{OPS}$$

Frequency is the clock frequency of the kernel. This value is determined by the targeted acceleration platform, or target platform. For instance, the maximum kernel clock on an Alveo U200 Data Center accelerator card is 300 MHz.

As previously mentioned, the Computational Intensity of a function is the ratio of the total number of operations to the total amount of input and output data. The formula above clearly shows that functions with a high volume of operations and a low volume of data are better candidates for acceleration.

Determine How Much Parallelism is Needed

After the equation above has been calculated, it is possible to estimate the initial HW/SW performance ratio:

$$\text{Speed-up} = T_{HW} / T_{SW} = F_{max} * \text{Running Time} / V_{ops}$$

Without any parallelization, the initial speed-up will most likely be less than 1.

Next, calculate how much parallelism is needed to meet the performance goal:

$$\text{Parallelism Needed} = T_{Goal} / T_{HW} = T_{Goal} * V_{ops} / (F_{max} * \max(V_{INPUT}, V_{OUTPUT}))$$

This parallelism can be implemented in various ways: by widening the datapath, by using multiple engines, and by using multiple kernel instances. The developer should then determine the best combination given their needs and the characteristics of their application.

Determine How Many Samples the Datapath Should be Processing in Parallel

One possibility is to accelerate the computation by creating a wider datapath and processing more samples in parallel. Some algorithms lend themselves well to this approach, whereas others do not. It is important to understand the nature of the algorithm to determine if this approach will work and if so, how many samples should be processed in parallel to meet the performance goal.

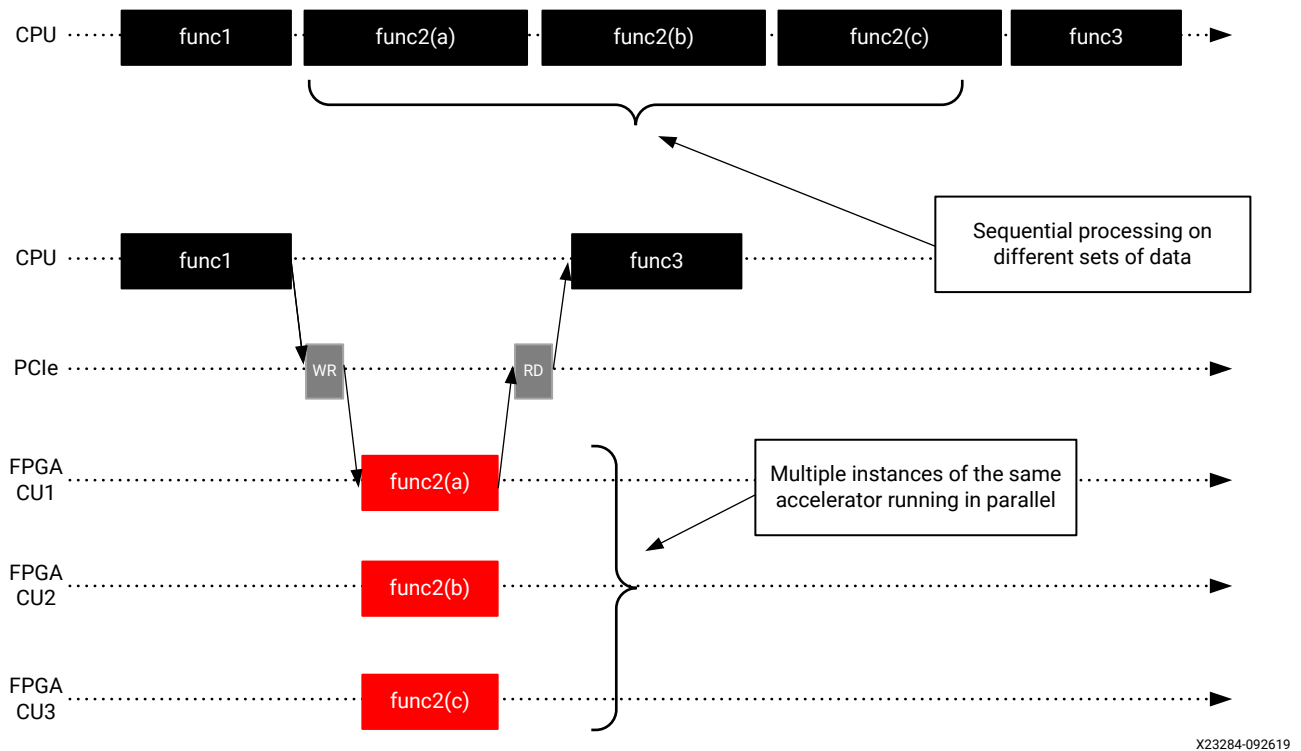
Processing more samples in parallel using a wider datapath improves performance by reducing the latency (running time) of the accelerated function.

Determine How Many Kernels Can and Should be Instantiated in the Device

If the datapath cannot be parallelized (or not sufficiently), then look at adding more kernel instances, as described in [Creating Multiple Instances of a Kernel](#). This is usually referred to as using multiple compute units (CUs).

Adding more kernel instances improves the performance of the application by allowing the execution of more invocations of the targeted function in parallel as shown below. Multiple data sets are processed concurrently by the different instances. Application performance scales linearly with the number of instances, provided that the host application can keep the kernels busy.

Figure 134: Improving Performance with Multiple Compute Units



As illustrated in the [Using Multiple Compute Units](#) tutorial, the Vitis technology makes it easy to scale performance by adding additional instances.

At this point, the developer should have a good understanding of the amount of parallelism necessary in the hardware to meet performance goals and through a combination of datapath width and kernel instances, how that parallelism will be achieved.

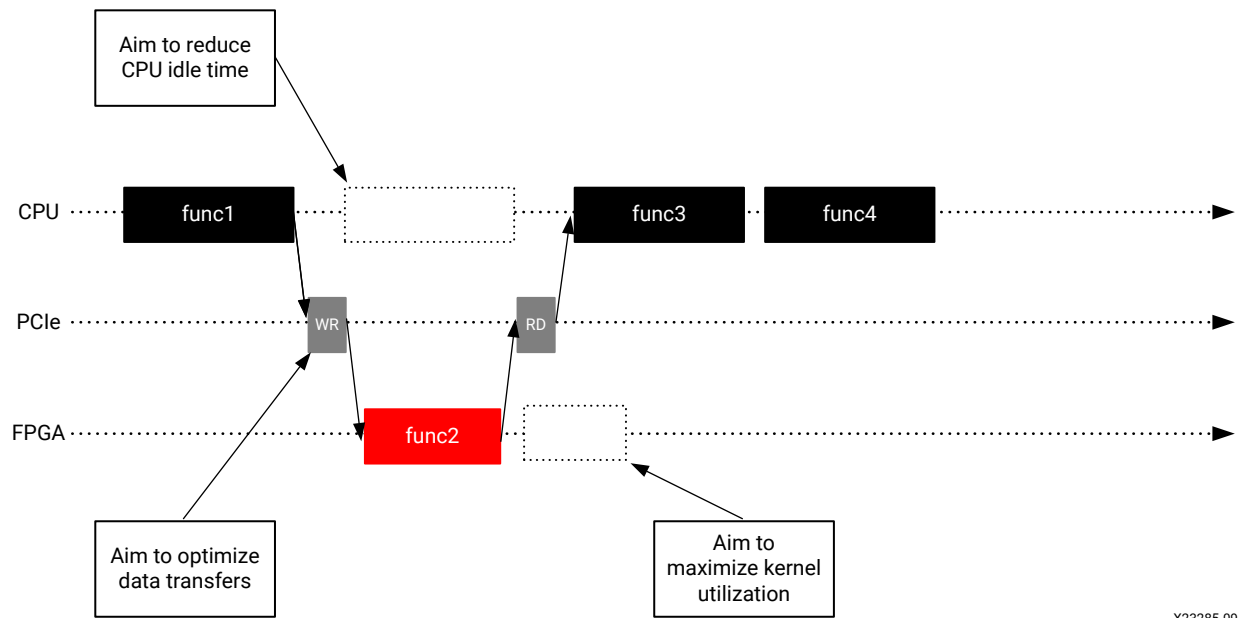
Step 4: Identify Software Application Parallelization Needs

While the hardware device and its kernels are designed to offer potential parallelism, the software application must be engineered to take advantage of this potential parallelism.

Parallelism in the software application is the ability for the host application to:

- Minimize idle time and do other tasks while the device kernels are running.
- Keep the device kernels active performing new computations as early and often as possible.
- Optimize data transfers to and from the device.

Figure 135: Software Optimization Goals



X23285-092619

In the world of factories and assembly lines, the host application would be the headquarters keeping busy and planning the next generation of products while the factories manufacture the current generation.

Similarly, headquarters must orchestrate the transport of goods to and from the factories and send them requests. What is the point of building many factories if the logistics department does not send them raw material or blueprints of what to create?

Minimize CPU Idle Time While the Device Kernels are Running

Device-acceleration is about offloading certain computations from the host processor to the kernels in the device. In a purely sequential model, the application would be waiting idly for the results to be ready and resume processing, as shown in the above figure.

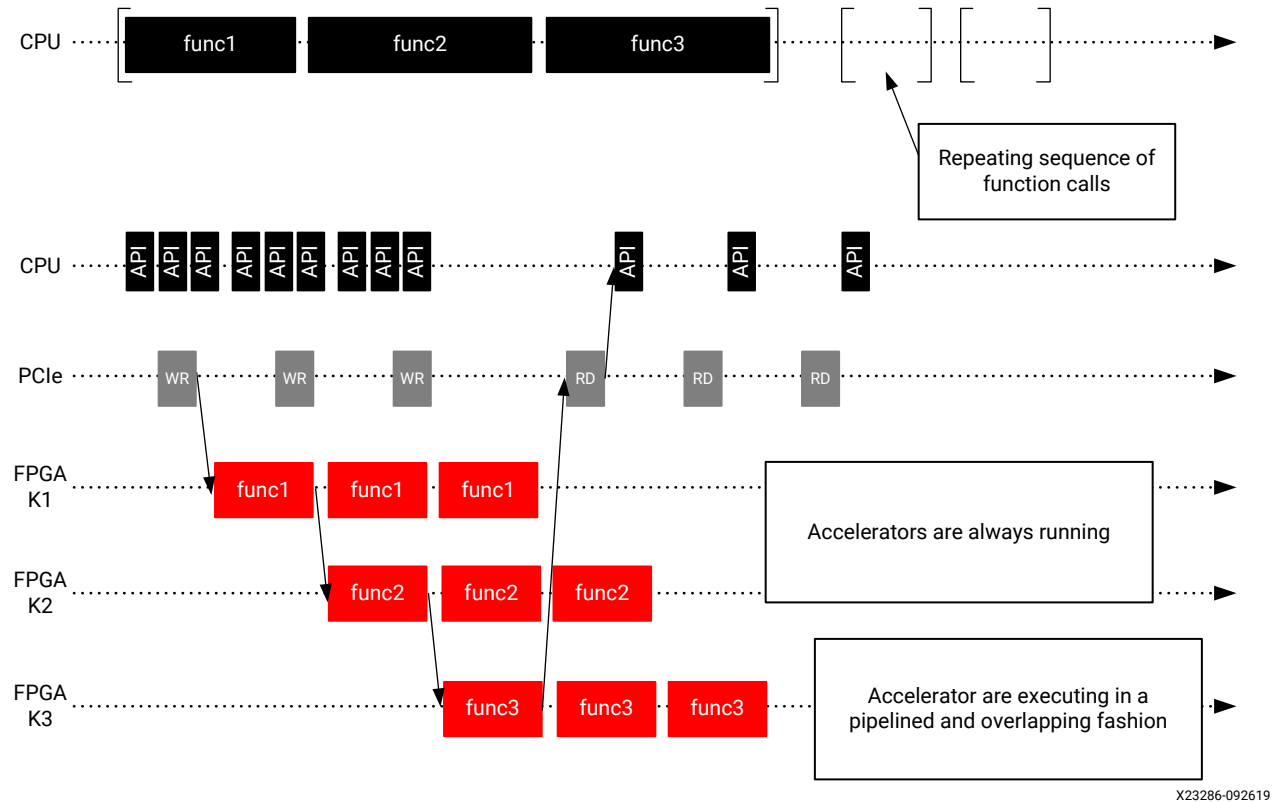
Instead, engineer the software application to avoid such idle cycles. Begin by identifying parts of the application that do not depend on the results of the kernel. Then structure the application so that these functions can be executed on the host in parallel to the kernel running in the device.

Keep the Device Kernels Utilized

Kernels might be present in the device, but they will only run when the application requests them. To maximize performance, engineer the application so that it will keep the kernels busy.

Conceptually, this is achieved by issuing the next requests before the current ones have completed. This results in pipelined and overlapping execution, leading to kernels being optimally utilized, as shown in the following figure.

Figure 136: Pipelined Execution of Accelerators



In this example, the original application repeatedly calls `func1`, `func2` and `func3`. Corresponding kernels (K1, K2, K3) are created for the three functions. A naïve implementation would have the three kernels running sequentially, like the original software application does. However, this means that each kernel is active only a third of the time. A better approach is to structure the software application so that it can issue pipelined requests to the kernels. This allows K1 to start processing a new data set at the same time K2 starts processing the first output of K1. With this approach, the three kernels are constantly running with maximized utilization.

More information on software pipelining can be found in the [Vitis Application Acceleration Development Flow Tutorials](#).

Optimize Data Transfers to and from the Device

In an accelerated application, data must be transferred from the host to the device especially in the case of PCIe-based applications. This introduces latency, which can be very costly to the overall performance of the application.

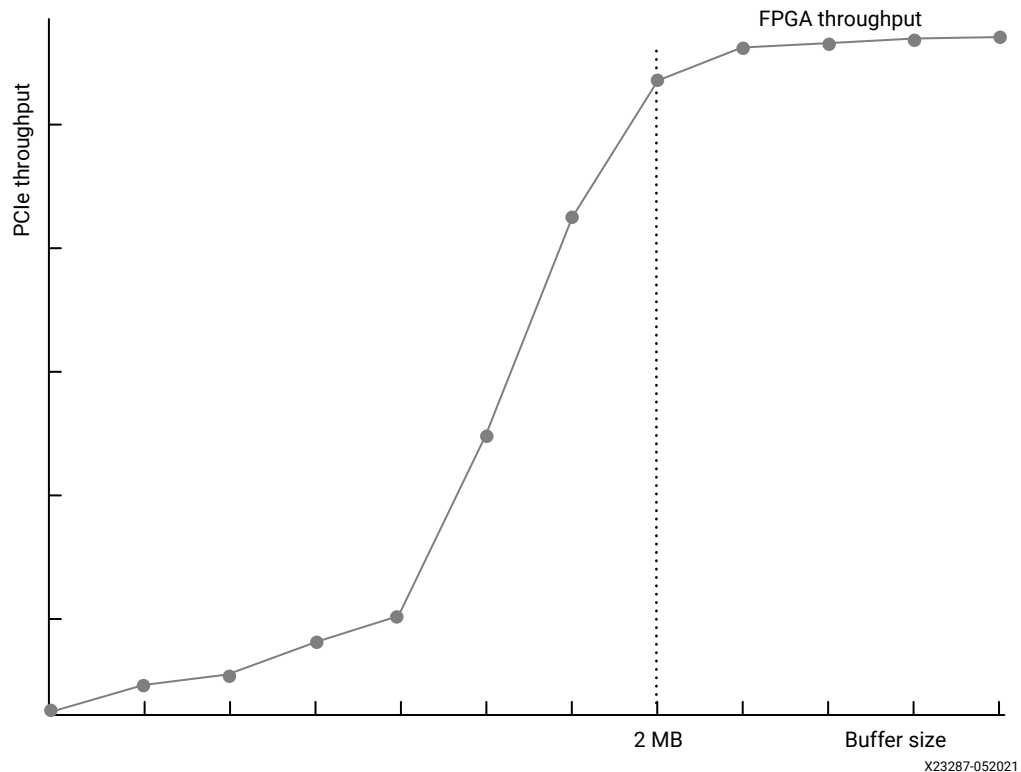
Data needs to be transferred at the right time, otherwise the application performance is negatively impacted if the kernel must wait for data to be available. It is therefore important to transfer data ahead of when the kernel needs it. This is achieved by overlapping data transfers and kernel execution, as described in [Keep the Device Kernels Utilized](#). As shown in the sequence in the previous figure, this technique enables hiding the latency overhead of the data transfers and avoids the kernel having to wait for data to be ready.

Another method of optimizing data transfers is to transfer optimally sized buffers. As shown in the following figure, the effective PCIe throughput varies greatly based on the transferred buffer size. The larger the buffer, the better the throughput, ensuring the accelerators always have data to operate on and are not wasting cycles. It is usually better to make data transfers of 1 MB or more. Running DMA tests upfront can be useful for finding the optimal buffer sizes. Also, when determining optimal buffer sizes, consider the effect of large buffers on resource utilization and transfer latency.

Another method of optimizing data transfers is to transfer optimally sized buffers. The effective data transfer throughput varies greatly based on the size of transferred buffer. The larger the buffer, the better the throughput, ensuring the accelerators always have data to operate on and are not wasting cycles.

As shown in the following figure, on PCIe-based systems it is usually better to make data transfers of 1 MB or more. Running DMA tests in advance using the `xbutil` utility can be useful for finding the optimal buffer sizes.

Figure 137: Performance of PCIe Transfers as a Function of Buffer Size



AMD recommends grouping multiple sets of data in a common buffer to achieve the highest possible throughput.

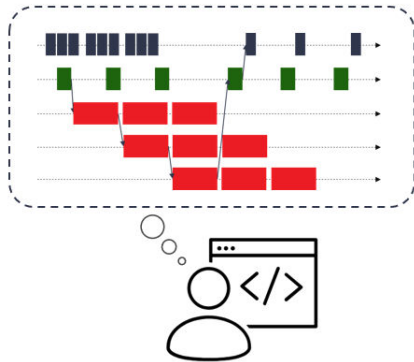
Conceptualize the Desired Application Timeline

The developer should now have a good understanding of what functions need to be accelerated, what parallelism is needed to meet performance goals, and how the application will be delivered.

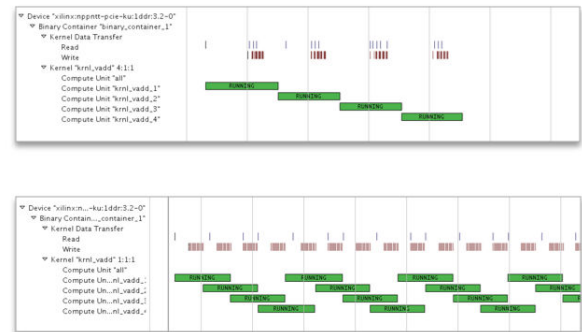
At this point, it is very useful to summarize this information in the form of an expected application timeline. The timeline sequences, such as the ones shown in [Keep the Device Kernels Utilized](#), are very effective ways of representing performance and parallelization in action as the application runs. They represent how the potential parallelism built into the architecture is mobilized by the application.

Figure 138: Timeline Trace

Developer's Application Timeline View



Vitis Application Timeline View



The Vitis software platform generates timeline views from actual application runs. If the developer has a desired timeline in mind, they can compare it to the actual results, identify potential issues, and iterate and converge on the optimal results, as shown in the above figure.

Step 5: Refine Architectural Details

Before proceeding with the development of the application and its kernels, the final step consists of refining and deriving second order architectural details from the top-level decisions made up to this point.

Finalize Kernel Boundaries

As discussed earlier, performance can be improved by creating multiple instances of kernels (compute units). However, adding CUs has a cost in terms of I/O ports, bandwidth, and resources.

In the Vitis software platform flow, kernel ports have a maximum width of 512 bits (64 bytes) and have a fixed cost in terms of device resources. Most importantly, the targeted platform sets a limit on the maximum number of ports which can be used. Be mindful of these constraints and use these ports and their bandwidth optimally.

An alternative to scaling with multiple compute units is to scale by adding multiple engines within a kernel. This approach allows increasing performance in the same way as adding more CUs: multiple data sets are processed concurrently by the different engines within the kernel.

Placing multiple engines in the same kernel takes the fullest advantage of the bandwidth of the kernel's I/O ports. If the datapath engine does not require the full width of the port, it can be more efficient to add additional engines in the kernel than to create multiple CUs with single engines in them.

Putting multiple engines in a kernel also reduces the number of ports and the number of transactions to global memory that require arbitration, improving the effective bandwidth.

On the other hand, this transformation requires coding explicit I/O multiplexing behavior in the kernel. This is a trade-off the developer needs to make.

Decide Kernel Placement and Connectivity

After the kernel boundaries are finalized, the developer knows exactly how many kernels will be instantiated and therefore how many ports will need to be connected to global memory resources.

At this point, it is important to understand the features of the target platform and what global memory resources are available. For instance, the Alveo U200 Data Center accelerator card has 4 x 16 GB banks of DDR4 and 3 x 128 KB banks of PLRAM distributed across three super-logic regions (SLRs). For more information, refer to [Vitis Software Platform Release Notes](#).

If kernels are factories, then global memory banks are the warehouses through which goods transit to and from the factories. The SLRs are like distinct industrial zones where warehouses preexist and factories can be built. While it is possible to transfer goods from a warehouse in one zone to a factory in another zone, this can add delay and complexity.

Using multiple DDRs helps balance the data transfer loads and improves performance. This comes with a cost, however, as each DDR controller consumes device resources. Balance these considerations when deciding how to connect kernel ports to memory banks. As explained in [Mapping Kernel Ports to Memory](#), establishing these connections is done through a simple compiler switch, making it easy to change configurations if necessary.

After refining the architectural details, the developer should have all the information necessary to start implementing the kernels, and ultimately, assembling the entire application.

Methodology for Developing C/C++ Kernels

The Vitis software platform supports kernels modeled in either C/C++ or RTL (Verilog, VHDL, System Verilog). This methodology guide applies to C/C++ kernels. For details on developing RTL kernels, see [Packaging RTL Kernels](#).

The following key kernel requirements for optimal application performance should have already been identified during the architecture definition phase:

- Throughput goal
- Latency goal
- Datapath width
- Number of engines
- Interface bandwidth

These requirements drive the kernel development and optimization process. Achieving the kernel throughput goal is the primary objective, as overall application performance is predicated on each kernel meeting the specified throughput.

The kernel development methodology therefore follows a throughput-driven approach and works from the outside-in. This approach has two phases, as also described in the following figure:

1. Defining and implementing the macro-architecture of the kernel
2. Coding and optimizing the micro-architecture of the kernel

Before starting the kernel development process, it is essential to understand the difference between functionality, algorithm, and architecture; and how they pertain to the kernel development process.

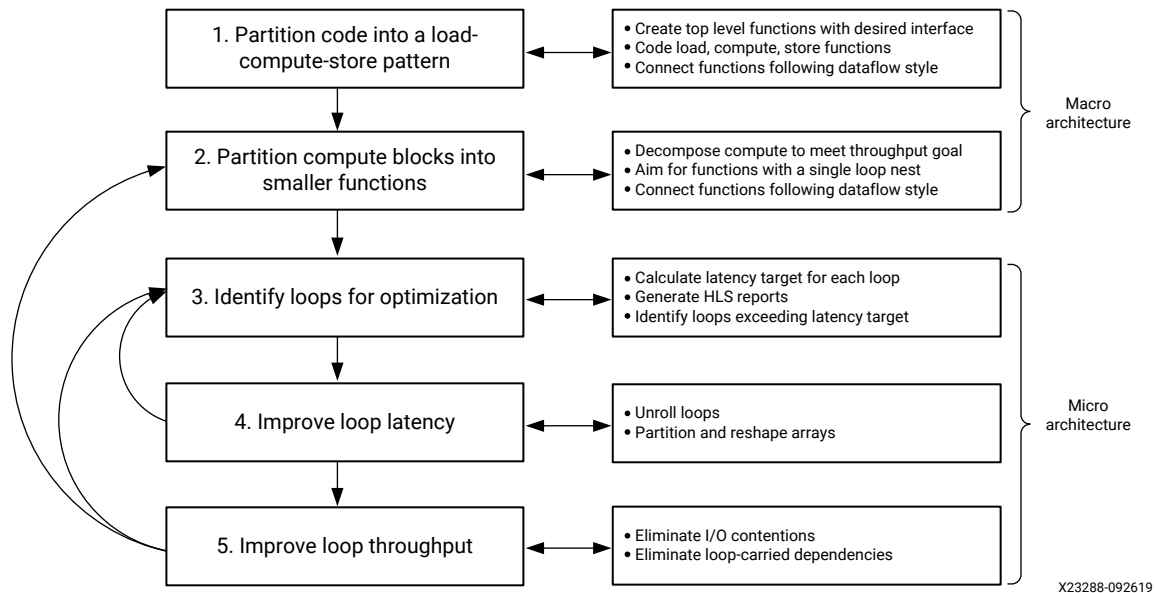
- Functionality is the mathematical relationship between input parameters and output results.
- Algorithm is a series of steps for performing a specific functionality. A given functionality can be performed using a variety of different algorithms. For instance, a sort function can be implemented using a "quick sort" or a "bubble sort" algorithm.
- Architecture, in this context, refers to the characteristics of the underlying hardware implementation of an algorithm. For instance, a particular sorting algorithm can be implemented with more or less comparators executing in parallel, with RAM or register-based storage, and so on.

You must understand that the Vitis compiler generates optimized hardware architectures from algorithms written in C/C++. However, it does not transform a particular algorithm into another one. Even if the software program can be automatically converted (or synthesized) into hardware, achieving acceptable quality of results (QoR), requires additional work such as rewriting the software to help the HLS tool achieve the desired performance goals.

Therefore, because the algorithm directly influences data access locality, in addition to potential for computational parallelism, your choice of algorithm has a major impact on achievable performance, more so than the compiler's abilities or user specified pragmas. To help, you need to understand the best practices for writing good software for execution on the FPGA device. The next few sections discuss how you can first identify some macro-level architectural optimizations to structure your program and then focus on some fine-grained micro-level architectural optimizations to boost your performance goals.

The following methodology assumes that you have identified a suitable algorithm for the functionality that you want to accelerate.

Figure 139: Kernel Development Methodology



About the High-Level Synthesis Compiler

Before starting the kernel development process, the developer should have familiarity with high-level synthesis (HLS) concepts. The HLS compiler turns C/C++ code into RTL designs which then map onto the device fabric.

The HLS compiler is more restrictive than standard software compilers. For example, there are unsupported constructs including: system function calls, dynamic memory allocation and recursive functions. For more information, see the *Vitis High-Level Synthesis User Guide* (UG1399).

More importantly, always keep in mind that the structure of the C/C++ source code has a strong impact on the performance of the generated hardware implementation. This methodology guide will help you structure the code to meet the application throughput goals. For specific information on programming kernels, see [Developing PL Kernels using C++](#).

Verification Considerations

This methodology described in this guide is iterative in nature and involves successive code modifications. AMD recommends verifying the code after each modification. This can be done using standard software verification methods or with the Vitis integrated design environment (IDE) software or hardware emulation flows. In either case, make sure your testing provides sufficient coverage and verification quality.

Step 1: Partition the Code into a Load-Compute-Store Pattern

A kernel is essentially a custom datapath (optimized for the desired functionality) and an associated data storage and motion network. Also referred to as the *memory architecture* or *memory hierarchy* of the kernel, this data storage and motion network is responsible for moving data in and out of the kernel and through the custom datapath as efficiently as possible.

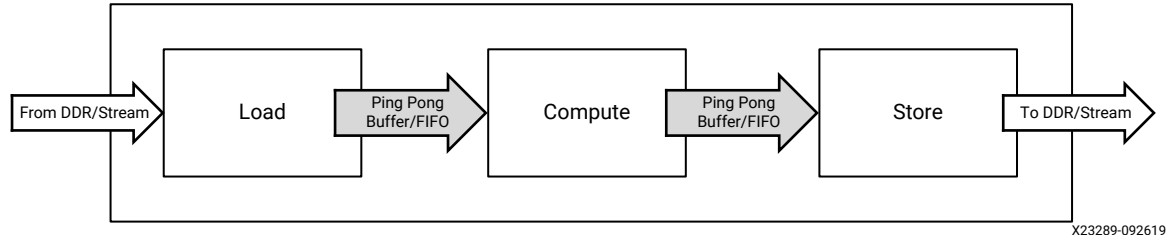
Knowing that kernel accesses to global memory are expensive and that bandwidth is limited, it is very important to carefully plan this aspect of the kernel.

To help with this, the first step of the kernel development methodology requires structuring the kernel code into the load-compute-store pattern.

This means creating a top-level function with:

- Interface parameters matching the desired kernel interface.
- Three sub-functions: load, compute, and store.
- Local arrays or `hls::stream` variables to pass data between these functions.

Figure 140: Load-Compute-Store Pattern



X23289-092619

Structuring the kernel code this way enables task-level pipelining, also known as HLS dataflow. This compiler optimization results in a design where each function can run simultaneously, creating a pipeline of concurrently running tasks. This is the premise of the assembly line in our factory, and this structure is key to achieving and sustaining the desired throughput. For more information about HLS dataflow, see [Task Parallelism](#) in the *Vitis Unified Software Platform Documentation: Application Acceleration Development* (UG1393).

The load function is responsible for moving data external to the kernel (that is, global memory) to the compute function inside the kernel. This function does not perform any data processing but focuses on efficient data transfers, including buffering and caching if necessary.

The compute function, as its name suggests, is where all the processing is done. At this stage of the development flow, the internal structure of the compute function is not important.

The store function mirrors the load function. It is responsible for moving data out of the kernel, taking the results of the compute function and transferring them to global memory outside the kernel.

Creating a load-compute-store structure that meets the performance goals starts by engineering the flow of data within the kernel. Some factors to consider are:

- How does the data flow from outside the kernel into the kernel?
- How fast does the kernel need to process this data?
- How is the processed data written to the output of the kernel?

Understanding and visualizing the data movement as a block diagram will help to partition and structure the different functions within the kernel.

A working example featuring the load-compute-store pattern can be found on the [Vitis Examples](#) GitHub repository.

Create a Top-Level Function with the Desired Interface

The Vitis technology infers kernel interfaces from the parameters of the top-level function. Therefore, start by writing a kernel top-level function with parameters matching the desired interface.

Input parameters should be passed as scalars. Blocks of input and output data should be passed as pointers. Compiler pragmas should be used to finalize the interface definition.

Code the Load and Store Functions

Data transfers between the kernel and global memories have a very big influence on overall system performance. If not properly done, they will throttle the kernel. It is therefore important to optimize the load and store functions to efficiently move data in and out of the kernel and optimally feed the compute function.

The layout of data in global memory matches the layout of data in the software application. This layout must be known when writing the load and store functions. Conversely, if a certain data layout is more favorable for moving data in and out of the kernel, it is possible to adapt buffer layout in the software application. Either way, the kernel developer and application developer need to agree on how data is organized in buffers and global memory.

The following are guidelines for improving the efficiency of data transfers in and out of the kernel.

Match Port Width to Datapath Width

In the Vitis software platform, the port of a kernel can be up to 512 bits wide, which means that a kernel can read or write up to 64 bytes per clock cycle per port.

AMD recommends matching the width of the kernel ports to width of the datapath in the compute function. For instance, if the datapath needs to process 16 bytes in parallel to meet the desired throughput, then ports should be made 128-bit wide to allow reading and writing 16 bytes in parallel.

In some cases, it might be useful to access the full width bits of the interface even if the datapath does not need them. This can help reduce contention when many kernels are trying to access the same global memory bank. However, this will usually lead to additional buffering and internal memory resources in the kernel.

Use Burst Transfers

The first read or write request to global memory is expensive, but subsequent contiguous operations are not. Transferring data in bursts hides the memory access latency and improves bandwidth usage and efficiency of the memory controller.

Atomic accesses to global memory should always be avoided unless absolutely required. The load and store functions should be coded to always infer bursting transaction. This can be done using a `memcpy` operation as shown in the `vadd.cpp` file in the [Vitis Accel Examples](#), or by creating a tight `for` loop accessing all the required values sequentially, as explained in [Section III: Developing Applications](#).

Minimize the Number of Data Transfers from Global Memory

Since accesses to global memory can add significant latency to the application, only make necessary transfers.

The guideline is to only read and write the necessary values, and only do so once. In situations where the same value must be used several times by the compute function, buffer this value locally instead of reading it from global memory again. Coding the proper buffering and caching structure can be key to achieving the throughput goal.

Code the Compute Functions

The compute function is where all the actual processing is done. This first step of the methodology is focused on getting the top-level structure right and optimizing data movement. The priority is to have a function with the right interfaces and make sure the functionality is correct. The following sections focus on the internal structure of the compute function.

Connect the Load, Compute, and Store Functions

Use standard C/C++ variables and arrays to connect the top-level interfaces and the load, compute and store functions. It can also be useful to use the `hls::stream` class, which models a streaming behavior.

Streaming is a type of data transfer in which data samples are sent in sequential order starting from the first sample. Streaming requires no address management and can be implemented with FIFOs. For more information about the `hls::stream` class, see the topic on Using HLS Streams in the Vitis HLS User Guide ([UG1399](#)).

When connecting the functions, use the canonical form required by the HLS compiler. For more information, see the topic on Dataflow Optimization in the Vitis HLS User Guide ([UG1399](#)). This helps the compiler build a high-throughput set of tasks using the dataflow optimization. Key recommendations include:

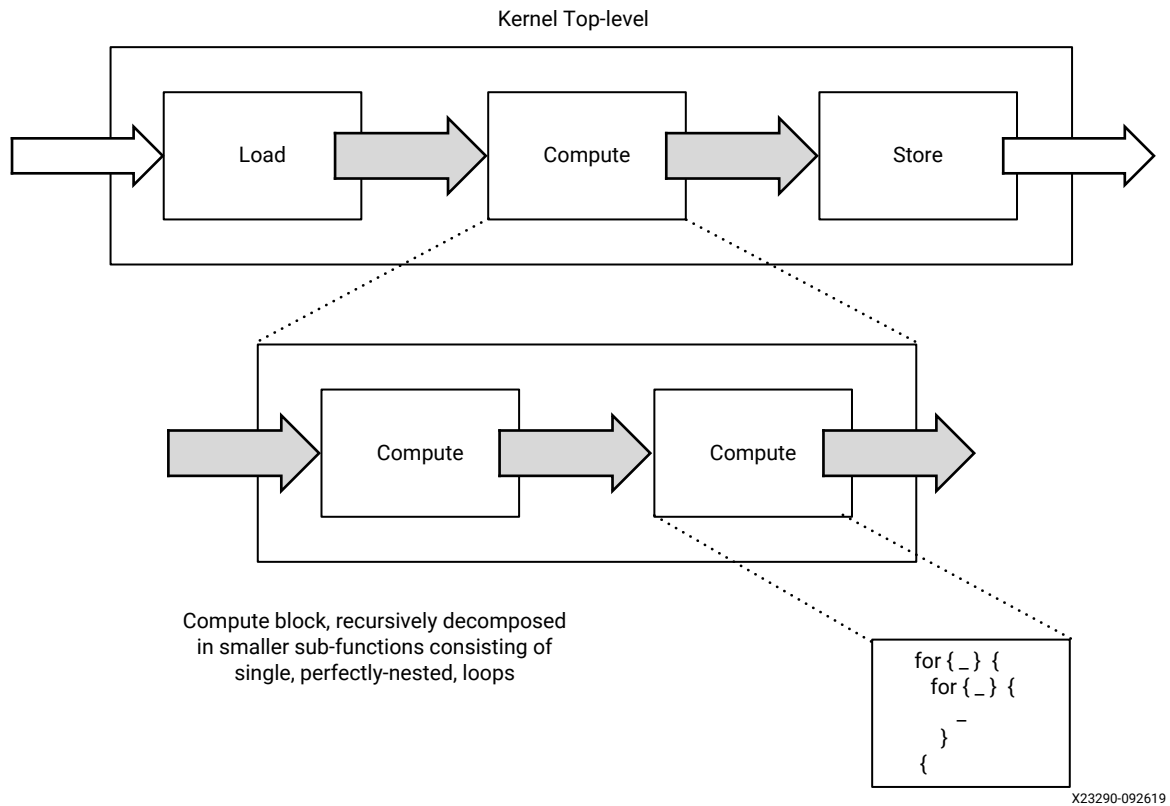
- Data should be transferred in the forward direction only, avoiding feedback whenever possible.
- Each connection should have a single producer and a single consumer.
- Only the load and store functions should access the primary interface of the kernel.

At this point, the developer has created the top-level function of the kernel, coded the interfaces and the load/store functions with the objective of moving data through the kernel at the desired throughput.

Step 2: Partition the Compute Blocks into Smaller Functions

The next step is to refine the main compute function, decomposing it into a sequence of smaller sub-functions, as shown in the following figure.

Figure 141: Compute Block Sub-Functions



Decompose to Identify Throughput Goals

In a dataflow system like the one created with this approach, the slowest task will be the bottleneck.

$$\text{Throughput}(\text{Kernel}) = \min(\text{Throughput}(\text{Task}_1), \text{Throughput}(\text{Task}_2), \dots, \text{Throughput}(\text{Task}_N))$$

Therefore, during the decomposition process, always have the kernel throughput goal in mind and assess whether each sub-function will be able to satisfy this throughput goal.

In the following steps of this methodology, the developer will get actual throughput numbers from running the Vitis HLS compiler. If these results cannot be improved, the developer will have to iterate and further decompose the compute stages.

Aim for Functions with a Single Loop Nest

As a general rule, if a function has sequential loops in it, these loops execute sequentially in the hardware implementation generated by the HLS compiler. This is usually not desirable, as sequential execution hampers throughput.

However, if these sequential loops are pushed into sequential functions, then the HLS compiler can apply the dataflow optimization and generate an implementation that allows the pipelined and overlapping execution of each task. For more information on the dataflow optimization, see [Task Parallelism](#) in the *Vitis Unified Software Platform Documentation: Application Acceleration Development* (UG1393).

During this partitioning and refining process, put sequential loops into individual functions. Ideally, the lowest-level compute block should only contain a single perfectly-nested loop.

Connect Compute Functions Using the Dataflow ‘Canonical Form’

The same rules regarding connectivity within the top-level function apply when decomposing the compute function. Aim for feed-forward connections and having a single producer and consumer for each connecting variable. If a variable must be consumed by more than one function, then it should be explicitly duplicated.

When moving blocks of data from one compute block to another, the developer can choose to use arrays or `hls::stream` objects.

Using arrays requires fewer code changes and is usually the fastest way to make progress during the decomposition process. However, using `hls::stream` objects can lead to designs using less memory resources and having shorter latency. It also helps the developer reason about how data moves through the kernel, which is always an important thing to understand when optimizing for throughput.

Using `hls::stream` objects is usually a good thing to do, but it is up to the developer to determine the most appropriate moment to convert arrays to streams. Some developers will do this very early on while others will do this at the very end, as a final optimization step. This can also be done using the [pragma HLS dataflow](#).

At this stage, maintaining a graphical representation of the architecture of the kernel can be very useful to reason through data dependencies, data movement, control flows, and concurrency.

Step 3: Identify Loops Requiring Optimization

At this point, the developer has created a dataflow architecture with data motion and processing functions intended to sustain the throughput goal of the kernel. The next step is to make sure that each of the processing functions are implemented in a way that deliver the expected throughput.

As explained before, the throughput of a function is measured by dividing the volume of data processed by the latency, or running time, of the function.

$$T = \max(V_{\text{INPUT}}, V_{\text{OUTPUT}}) / \text{Latency}$$

Both the target throughput and the volume of data consumed and produced by the function should be known at this stage of the ‘outside-in’ decomposition process described in this methodology. The developer can therefore easily derive the latency target for each function.

The Vitis HLS compiler generates detailed reports on the throughput and latency of functions and loops. Once the target latencies have been determined, use the HLS reports to identify which functions and loops do not meet their latency target and require attention, as described in [HLS Synthesis Report](#).

The latency of a loop can be calculated as follows:

$$\text{Latency}_{\text{Loop}} = (\text{Steps} + \text{II} \times (\text{TripCount} - 1)) \times \text{ClockPeriod}$$

Where:

- **Steps:** Duration of a single loop iteration, measured in number of clock cycles
- **TripCount:** Number of iterations in the loop.
- **II:** Initiation Interval, the number of clock cycles between the start of two consecutive iterations. When a loop is not pipelined, its II is equal to the number of Steps.

Assuming a given clock period, there are three ways to reduce the latency of a loop, and thereby improve the throughput of a function:

- Reduce the number of Steps in the loop (take less time to perform one iteration).
- Reduce the Trip Count, so that the loop performs fewer iterations.
- Reduce the Initiation Interval, so that loop iterations can start more often.

Assuming a trip count much larger than the number of steps, halving either the II or the trip count can be sufficient to double the throughput of the loop.

Understanding this information is key to optimizing loops with latencies exceeding their target. By default, the Vitis HLS compiler will try to generate loop implementations with the lowest possible II. Start by looking at how to improve latency by reducing the trip count or the number of steps.

Step 4: Improve Loop Latencies

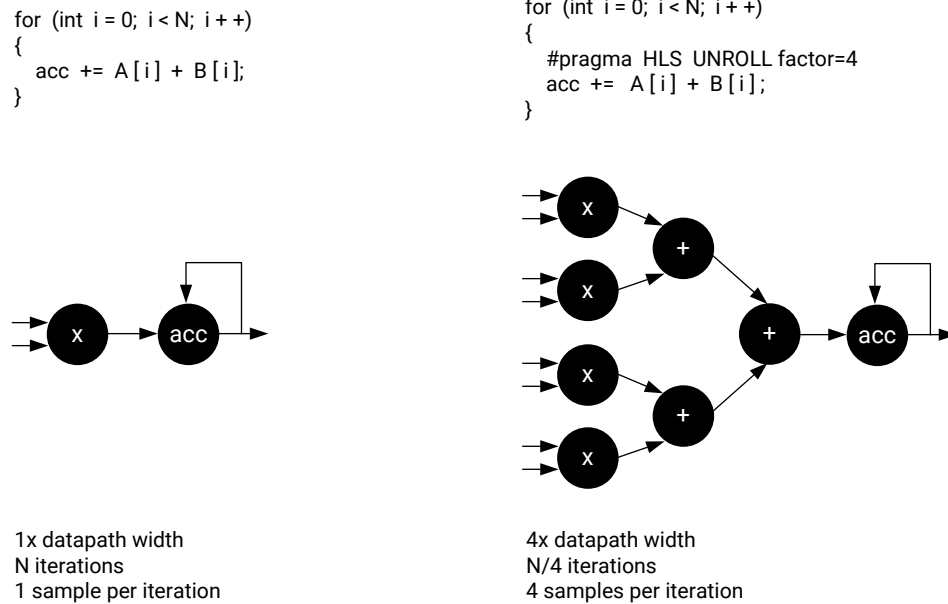
After identifying loops latencies that exceed their target, the first optimization to consider is loop unrolling.

Apply Loop Unrolling

Loop unrolling unwinds the loop, allowing multiple iterations of the loop to be executed together, reducing the loop’s overall trip count.

In the industrial analogy, factories are kernels, assembly lines are dataflow pipelines, and stations are compute functions. Unrolling creates stations which can process multiple objects arriving at the same time on the conveyor belt, which results in higher performance.

Figure 142: Loop Unrolling



X23291-092619

Loop unrolling can widen the resulting datapath by the corresponding factor. This usually increases the bandwidth requirements as more samples are processed in parallel. This has two implications:

- The width of the function I/Os must match the width of the datapath and vice versa.
- No additional benefit is gained by loop unrolling and widening the datapath to the point where I/O requirements exceed the maximum size of a kernel port (512 bits / 64 bytes).

The following guidelines will help optimize the use of loop unrolling:

- Start from the innermost loop within a loop nest.
- Assess which unroll factor would eliminate all loop-carried dependencies.
- For more efficient results, unroll loops with fixed trip counts.
- If there are function calls within the unrolled loop, in-lining these functions can improve results through better resource sharing, although at the expense of longer synthesis times. Note also that the interconnect may become increasingly complex and lead to routing problems later on.
- Do not blindly unroll loops. Always unroll loops with a specific outcome in mind.

Apply Array Partitioning

Unrolling loops changes the I/O requirements and data access patterns of the function. If a loop makes array accesses, as is almost always the case, ensure that the resulting datapath can access all the data it needs in parallel.

If unrolling a loop does not result in the expected performance improvement, this is almost always because of memory access bottlenecks.

By default, the Vitis HLS compiler maps large arrays to memory resources with a word width equal to the size of one array element. In most cases, this default mapping needs to be changed when loop unrolling is applied.

The HLS compiler supports various pragmas to partition and reshape arrays. Consider using these pragmas when loop unrolling to create a memory structure that allows the desired level of parallel accesses.

Unrolling and partitioning arrays can be sufficient to meet the latency and throughput goals for the targeted loop. If so, shift to the next loop of interest. Otherwise, look at additional optimizations to improve throughput.

Step 5: Improve Loop Throughput

If improving loop latency by reducing the trip count was not sufficient, look at ways to reduce the initiation interval (II).

The loop II is the count of clock cycles between the start of two loop iterations. The Vitis HLS compiler will always try to pipeline loops, minimize the II, and start loop iterations as early as possible, ideally starting a new iteration each clock cycle (II=1).

There are two main factors that can limit the II:

- I/O contentions
- Loop-carried dependencies

The HLS Schedule Viewer automatically highlights loop dependencies limiting the II. It is a very useful visualization tool to use when working to improve the II of a loop.

Eliminate I/O Contentions

I/O contentions appear when a given I/O port of internal memory resources must be accessed more than once per loop iteration. A loop cannot be pipelined with an II lower than the number of times an I/O resource is accessed per loop iteration. If port A must be accessed four times in a loop iteration, then the lowest possible II will be 4 in single-port RAM.

The developer needs to assess whether these I/O accesses are necessary or if they can be eliminated. The most common techniques for reducing I/O contentions are:

- Creating internal cache structures

If some of the problematic I/O accesses involve accessing data already accessed in prior loop iterations, then a possibility is to modify the code to make local copies of the values accessed in those earlier iterations. Maintaining a local data cache can help reduce the need for external I/O accesses, thereby improving the potential II of the loop.

This example on the [Vitis Accel Examples](#) GitHub repository illustrates how a shift register can be used locally, cache previously read values, and improve the throughput of a filter.

- Reconfiguring I/Os and memories

As explained earlier in the section about improving latency, the HLS compiler maps arrays to memories, and the default memory configuration can not offer sufficient bandwidth for the required throughput. The array partitioning and reshaping pragmas can also be used in this context to create memory structure with higher bandwidth, thereby improving the potential II of the loop.

Eliminate Loop-Carried Dependencies

The most common case for loop-carried dependencies is when a loop iteration relies on a value computed in a prior iteration. There are differences whether the dependencies are on arrays or on scalar variables. For more information, see [Unrolling Loops](#) in the *Vitis HLS User Guide* (UG1399).

- Eliminating dependencies on arrays

The HLS compiler performs index analysis to determine whether array dependencies exist (read-after-write, write-after-read, write-after-write). The tool may not always be able to statically resolve potential dependencies and will in this case report false dependencies.

Special compiler pragmas can overwrite these dependencies and improve the II of the design. In this situation, be cautious and do not overwrite a valid dependency.

- Eliminating dependencies on scalars

In the case of scalar dependencies, there is usually a feedback path with a computation scheduled over multiple clock cycles. Complex arithmetic operations such as multiplications, divisions, or modulus are often found on these feedback paths. The number of cycles in the feedback path directly limits the potential II and should be reduced to improve II and throughput. To do so, analyze the feedback path to determine if and how it can be shortened. This can potentially be done using HLS scheduling constraints or code modifications such as reducing bit widths.

Advanced Techniques

If an II of 1 is usually the best scenario, it is rarely the only *sufficient* scenario. The goal is to meet the latency and throughput goal. To this extent, various combinations of II and unroll factor are often sufficient.

The optimization methodology and techniques presented in this guide should help meet most goals. The HLS compiler also supports many more optimization options which can be useful under specific circumstances. A complete reference of these optimizations can be found in [HLS Pragmas](#).

Best Practices for Data Center Acceleration with Vitis

Below are some specific things to keep in mind when developing your application code and hardware function in the Vitis core development kit.

- Review the [Accelerating Data Center Applications with Vitis Software Platform](#) section for information about acceleration methodology.
- Look to accelerate functions that have a high ratio of compute time to input and output data volume. Compute time can be greatly reduced using FPGA kernels, but data volume adds transfer latency.
- Accelerate functions that have a self-contained control structure and do not require regular synchronization with the host.
- Transfer large blocks of data from host to global device memory. One large transfer is more efficient than several smaller transfers. Run a bandwidth test to find the optimal transfer size.
- Only copy data back to host when necessary. Data written to global memory by a kernel can be directly read by another kernel. Memory resources include PLRAM (small size but fast access with lowest latency), HBM (moderate size and access speed with some latency), and DDR (large size but slow access with high latency).
- Take advantage of the multiple global memory resources to evenly distribute bandwidth across kernels.
- Maximize bandwidth usage between kernel and global memory by performing 512-bit wide bursts.
- Cache data in local memory within the kernels. Accessing local memories is much faster than accessing global memory.
- In the host application, use events and non-blocking transactions to launch multiple requests in a parallel and overlapping manner.

- In the FPGA, use different kernels to take advantage of task-level parallelism and use multiple CUs to take advantage of data-level parallelism to execute multiple tasks in parallel and further increase performance.
- Within the kernels take advantage of tasks-level with dataflow and instruction-level parallelism with loop unrolling and loop pipelining to maximize throughput.
- Some AMD FPGAs contain multiple partitions called super logic regions (SLRs). Keep the kernel in the same SLR as the global memory bank that it accesses.
- Use software and hardware emulation to validate your code frequently to make sure it is functionally correct.
- Frequently review the Vitis Guidance report as it provides clear and actionable feedback regarding deficiencies in your project.

Migrating to a New Target Platform

This migration content is intended for users who need to migrate their accelerated AMD Vitis™ technology application from one target platform to another. For example, moving an application from an AMD Alveo™ U200 Data Center accelerator card, to an Alveo U280 card.

The following sections are included:

- [Design Migration](#): An overview of the Design Migration Process including the physical aspects of FPGA devices.
- [Migrating Releases](#): Any changes to the host code and design constraints if a new release is used.
- [Modifying Kernel Placement](#): Controlling kernel placements and DDR interface connections.
- [Address Timing](#): Timing issues in the new target platform which might require additional options to achieve performance.

Design Migration

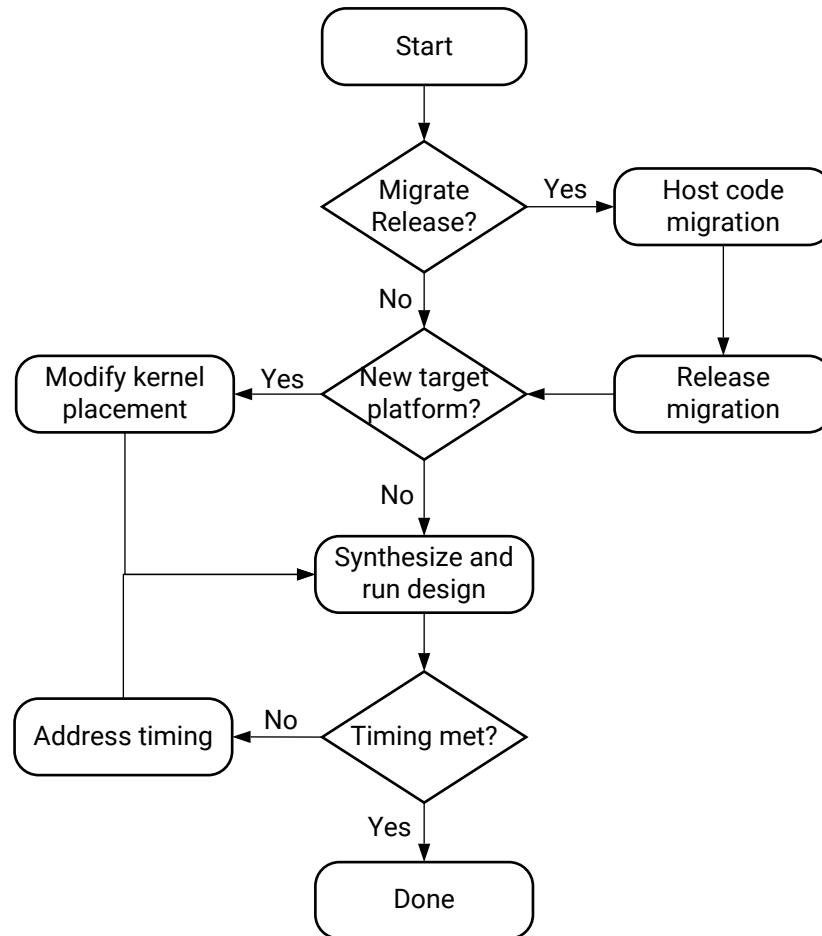
When migrating an application implemented in one target platform to another, it is important to understand the differences between the target platforms and the impact those differences have on the design.

Key considerations:

- Is there a change in the release?
- Does the new target platform contain a different target platform?
- Do the kernels need to be redistributed across the Super Logic Regions (SLRs)?
- Does the design meet the required frequency (timing) performance in the new platform?

The following diagram summarizes the migration flow described in this guide and the topics to consider during the migration process.

Figure 143: Target Platform Migration Flowchart



X21401-092519



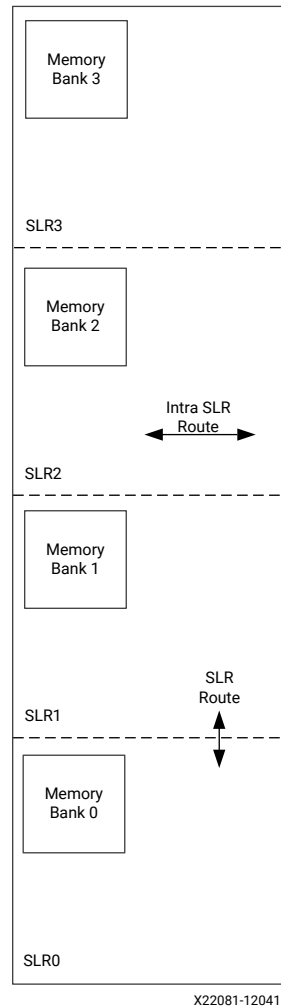
IMPORTANT! Before starting to migrate a design, it is important to understand the architecture of an FPGA and the target platform.

Understanding an FPGA Architecture

Before migrating any design to a new target platform, you should have a fundamental understanding of the FPGA architecture. The following diagram shows the floorplan of an AMD FPGA device. The concepts to understand are:

- SSI Devices
- SLRs
- SLR routing resources
- Memory interfaces

Figure 144: Physical View of AMD FPGA with Four SLR Regions



TIP: The FPGA floorplan shown above is for a SSI device with four SLRs where each SLR contains a DDR Memory interface.

Stacked Silicon Interconnect Devices

A SSI device is one in which multiple silicon dies are connected together through silicon interconnect, and packaged into a single device. An SSI device enables high-bandwidth connectivity between multiple die by providing a much greater number of connections. It also imposes much lower latency and consumes dramatically lower power than either a multiple FPGA or a multi-chip module approach, while enabling the integration of massive quantities of interconnect logic, transceivers, and on-chip resources within a single package. The advantages of SSI devices are detailed in *Xilinx Stacked Silicon Interconnect Technology Delivers Breakthrough FPGA Capacity, Bandwidth, and Power Efficiency* ([WP380](#)).

Super Logic Region

An SLR is a single FPGA die slice contained in an SSI device. Multiple SLR components are assembled to make up an SSI device. Each SLR contains the active circuitry common to most AMD FPGA devices. This circuitry includes large numbers of:

- LUTs
- Registers
- I/O Components
- Gigabit Transceivers
- Block Memory
- DSP Blocks

One or more kernels can be implemented within an SLR. A single kernel can be placed across multiple SLRs if needed.

SLR Routing Resources

The custom hardware implemented on the FPGA is connected via on-chip routing resources. There are two types of routing resources in an SSI device:

- **Intra-SLR Resources:** Intra-SLR routing resource are the fast resources used to connect the hardware logic. The Vitis technology automatically uses the most optimal resources to connect the hardware elements when implementing kernels.
- **Super Long Line (SLL) Resources:** SLLs are routing resources running between SLRs, used to connect logic from one region to the next. These routing resources are slower than intra-SLR routes. However, when a kernel is placed in one SLR, and the DDR it connects to is in another, the Vitis technology automatically implements dedicated hardware to use SLL routing resources without any impact to performance. More information on managing placement are provided in [Modifying Kernel Placement](#).

Memory Interfaces

Each SLR contains one or more memory interfaces. These memory interfaces are used to connect to the DDR memory where the data in the host buffers is copied before kernel execution. Each kernel reads data from the DDR memory and writes the results back to the same DDR memory. The memory interface connects to the pins on the FPGA and includes the memory controller logic.

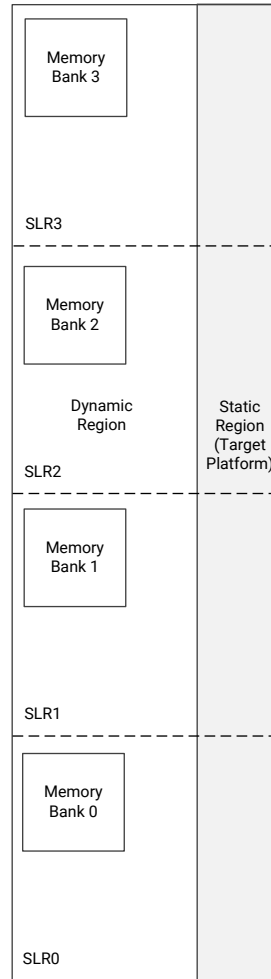
Understanding Target Platforms

In the Vitis technology, a target platform is the hardware design that is implemented onto the FPGA before any custom logic, or accelerators are added. The target platform defines the attributes of the FPGA and is composed of two regions:

- Static region which contains kernel and device management logic.
- Dynamic region where the custom logic of the accelerated kernels is placed.

The figure below shows an FPGA with the target platform applied.

Figure 145: Target Platform on an FPGA with Four SLR Regions



X22082-092519

The target platform, which is a static region that cannot be modified, contains the logic required to operate the FPGA, and transfer data to and from the dynamic region. The static region, shown above in gray, might exist within a single SLR, or as in the above example, might span multiple SLRs. The static region contains:

- DDR memory interface controllers
- PCIe® interface logic
- XDMA logic
- Firewall logic, etc.

The dynamic region is the area shown in white above. This region contains all the reconfigurable components of the target platform and is the region where all the accelerator kernels are placed.

Because the static region consumes some of the hardware resources available on the device, the custom logic to be implemented in the dynamic region can only use the remaining resources. In the example shown above, the target platform defines that all four DDR memory interfaces on the FPGA can be used. This will require resources for the memory controller used in the DDR interface.

Details on how much logic can be implemented in the dynamic region of each target platform is provided in the [Vitis Software Platform Release Notes](#). This topic is also addressed in [Modifying Kernel Placement](#).

Migrating Releases

Before migrating to a new target platform, you should also determine if you will need to target the new platform to a different release of the Vitis technology. If you intend to target a new release, AMD highly recommends to first target the existing platform using the new software release to confirm there are no changes required, and then migrate to a new target platform.

There are two steps to follow when targeting a new release with an existing platform:

- Host Code Migration
- Release Migration



IMPORTANT! Before migrating to a new release, AMD recommends that you review the [Vitis Software Platform Release Notes](#).

Host Code Migration

The `XILINX_XRT` environment variable is used to specify the location of the XRT library environment and must be set before you compile the host code. When the XRT library environment has been installed, the `XILINX_XRT` environment variable can be set by sourcing the `/opt/xilinx/xrt/setup.csh`, or `/opt/xilinx/xrt/setup.sh` file as appropriate. Secondly, ensure that your `LD_LIBRARY_PATH` variable also points to the XRT library installation area.

To compile and run the host code, source the `<INSTALL_DIR>/settings64.csh` or `<INSTALL_DIR>/settings64.sh` file from the Vitis installation.

If you are using the GUI, it will automatically incorporate the new XRT library location and generate the `Makefile` when you build your project.

However, if you are using your own custom `makefile`, you must use the `XILINX_XRT` environment variable to set up the XRT library.

- Include directories are now specified as: `-I${XILINX_XRT}/include` and `-I${XILINX_XRT}/include/CL`
- Library path is now: `-L${XILINX_XRT}/lib`
- OpenCL library will be: `libxilinxopencl.so`, use `-lxilinxopencl` in your `makefile`

Release Migration

After migrating the host code, build the code on the existing target platform using the new release of the Vitis technology. Verify that you can run the project in the Vitis unified software platform using the new release, ensure it completes successfully, and meets the timing requirements.

Issues which can occur when using a new release are:

- Changes to C libraries or library files.
- Changes to kernel path names.
- Changes to the HLS pragmas or pragma options embedded in the kernel code.
- Changes to C/C++/OpenCL compiler support.
- Changes to the performance of kernels: this might require adjustments to the pragmas in the existing kernel code.

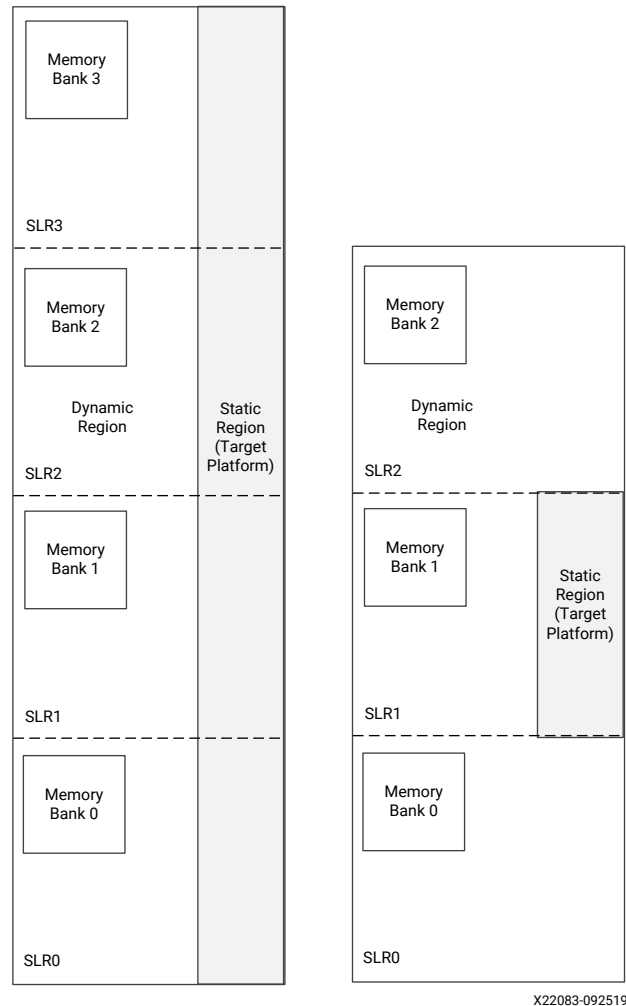
Address these issues using the same techniques you would use during the development of any kernel. At this stage, ensure the throughput performance of the target platform using the new release meets your requirements. If there are changes to the final timing (the maximum clock frequency), you can address these when you have moved to the new target platform. This is covered in [Address Timing](#).

Modifying Kernel Placement

The primary issue when targeting a new platform is ensuring that an existing kernel placement will work in the new target platform. Each target platform has an FPGA defined by a static region. As shown in the figure below, the target platform(s) can be different.

- The target platform on the left has four SLRs, and the static region is spread across all four SLRs.
- The target platform on the right has only three SLRs, and the static region is fully-contained in SLR1.

Figure 146: Comparison of Target Platforms of the Hardware Platform



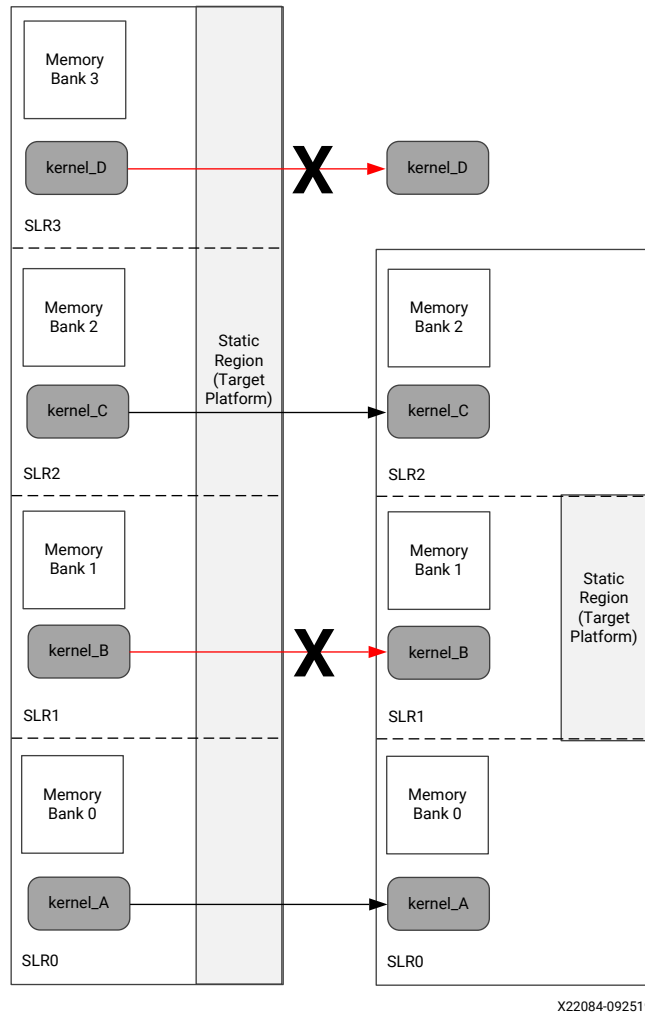
This section explains how to modify the placement of the kernels.

Implications of a New Hardware Platform

The figure below highlights the issue of kernel placement when migrating to a new target platform. In the example below:

- Existing kernel, kernel_B, is too large to fit into SLR2 of the new target platform because most of the SLR is consumed by the static region.
- The existing kernel, kernel_D, must be relocated to a new SLR because the new target platform does not have four SLRs like the existing platform.

Figure 147: Migrating Platforms – Kernel Placement



When migrating to a new platform, you need to take the following actions:

- Understand the resources available in each SLR of the new target platform, as documented in the [Vitis Software Platform Release Notes](#).
- Understand the resources required by each kernel in the design.
- Use the `v++ --config` option to specify which SLR each kernel is placed in, and which DDR bank each kernel connects to. For more details, refer to [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#) and [Mapping Kernel Ports to Memory](#).

These items are addressed in the remainder of this section.

Determining Where to Place the Kernels

To determine where to place kernels, two pieces of information are required:

- Resources available in each SLR of the hardware platform (. xsa).
- Resources required for each kernel.

With these two pieces of information you will then determine which kernel or kernels can be placed in each SLR of the target platform.

Keep in mind when performing these calculation that 10% of the available resources can be used by system infrastructure:

- Infrastructure logic can be used to connect a kernel to a DDR interface if it has to cross an SLR boundary.
- In an FPGA, resources are also used for signal routing. It is never possible to use 100% of all available resources in an FPGA because signal routing also requires resources.

Available SLR Resources

The resources available in each SLR of the various platforms supported by a release can be found in the [Vitis Software Platform Release Notes](#). The table shows an example target platform. In this example:

- SLR description indicates which SLR contains static and/or dynamic regions.
- Resources available in each SLR (LUTs, Registers, RAM, etc.) are listed.

This allows you to determine what resources are available in each SLR.

Table 73: SLR Resources of a Hardware Platform

Area	SLR 0	SLR 1	SLR 2
SLR description	Bottom of device; dedicated to dynamic region.	Middle of device; shared by dynamic and static region resources.	Top of device; dedicated to dynamic region.
Dynamic region Pblock name	pfa_top_i_dynamic_region_pblock_dynamic_SLR0	pfa_top_i_dynamic_region_pblock_dynamic_SLR1	pfa_top_i_dynamic_region_pblock_dynamic_SLR2
Compute unit placement syntax	set_property CONFIG.SLR_ASSIGNMENTS SLR0[get_bd_cells<cu_name>]	set_property CONFIG.SLR_ASSIGNMENTS SLR1[get_bd_cells<cu_name>]	set_property CONFIG.SLR_ASSIGNMENTS SLR2[get_bd_cells<cu_name>]
Global memory resources available in dynamic region			
Memory channels; system port name	bank0 (16 GB DDR4)	bank1 (16 GB DDR4, in static region) bank2 (16 GB DDR4, in dynamic region)	bank3 (16 GB DDR4)
Approximate available fabric resources in dynamic region			
CLB LUT	388K	199K	388K
CLB Register	776K	399K	776K
Block RAM Tile	720	420	720
UltraRAM	320	160	320
DSP	2280	1320	2280

Kernel Resources

The resources for each kernel can be obtained from the System Estimate report.

The System Estimate report is available in the Assistant view after either the Hardware Emulation or Hardware run are complete. An example of this report is shown below.

Figure 148: System Estimate Report

Area Information						
Compute Unit	Kernel Name	Module Name	FF	LUT	DSP	BRAM
smithwaterman_1	smithwaterman	smithwaterman	2925	4304	1	10

- FF refers to the CLB Registers noted in the platform resources for each SLR.
- LUT refers to the CLB LUTs noted in the platform resources for each SLR.
- DSP refers to the DSPs noted in the platform resources for each SLR.
- BRAM refers to the block RAM Tile noted in the platform resources for each SLR.

This information can help you determine the proper SLR assignments for each kernel.

Assigning Kernels to SLRs

Each kernel in a design can be assigned to an SLR region using the `connectivity.slr` option in a configuration file specified for the `v++ --config` command line option. Refer to [Assigning Compute Units to SLRs on Alveo Accelerator Cards](#) for more information.

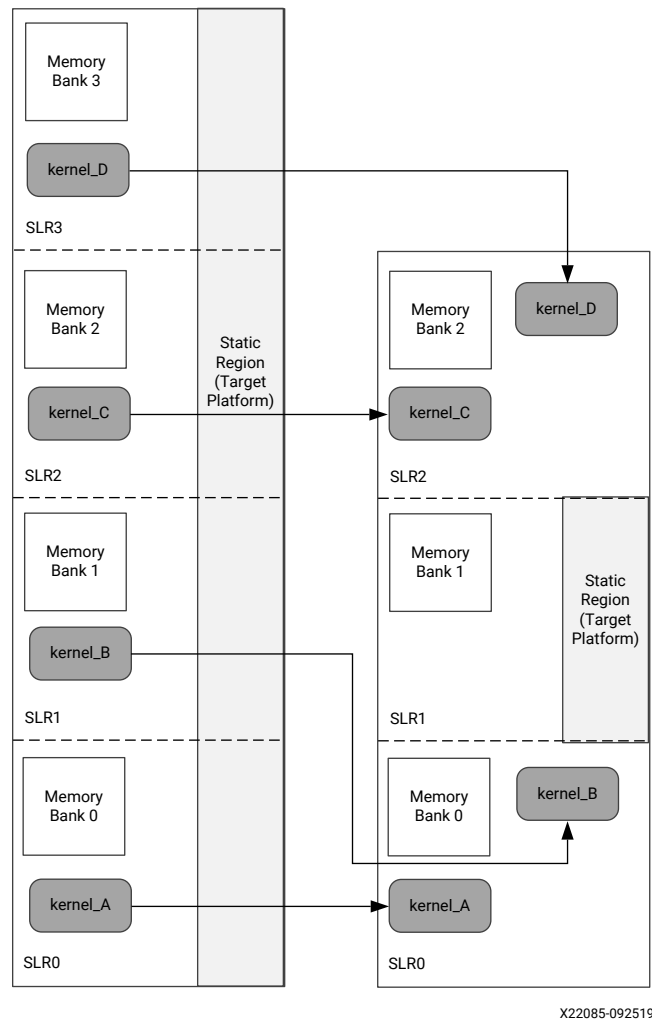
When placing kernels, AMD recommends assigning the specific DDR memory bank that the kernel will connect to using the `connectivity.sp` config option as described in [Mapping Kernel Ports to Memory](#).

For example, the figure below shows an existing target platform that has four SLRs, and a new target platform with three SLRs. The static region is also structured differently between the two platforms. In this migration example:

- Kernel_A is mapped to SLR0.
- Kernel_B, which no longer fits in SLR1, is remapped to SLR0, where there are available resources.
- Kernel_C is mapped to SLR2.
- Kernel_D is remapped to SLR2, where there are available resources.

The kernel mappings are illustrated in the figure below.

Figure 149: Mapping of Kernels Across SLRs



X22085-092519

Specifying Kernel Placement

For the example above, the configuration file to assign the kernels would be similar to the following:

```
[connectivity]
nk=kernel:4:kernel_A.lernel_B.kernel_C.kernel_D

slr=kernel_A:SLR0
slr=kernel_B:SLR0
slr=kernel_C:SLR2
slr=kernel_D:SLR2
```

The `v++` command line to place each of the kernels as shown in the figure above would be:

```
v++ -l --config config.cfg ...
```

Specifying Kernel DDR Interfaces

You should also specify the kernel DDR memory interface when specifying kernel placements. Specifying the DDR interface ensures the automatic pipelining of kernel connections to a DDR interface in a different SLR. This ensures there is no degradation in timing which can reduce the maximum clock frequency.

In this example, using the kernel placements in the above figure:

- Kernel_A is connected to Memory Bank 0.
- Kernel_B is connected to Memory Bank 1.
- Kernel_C is connected to Memory Bank 2.
- Kernel_D is connected to Memory Bank 1.

The configuration file to perform these connections would be as follows, and passed through the `v++ --config` command:

```
[connectivity]
nk=kernel:4:kernel_A.kernel_B.kernel_C.kernel_D

slr=kernel_A:SLR0
slr=kernel_B:SLR0
slr=kernel_C:SLR2
slr=kernel_D:SLR2

sp=kernel_A.arg1:DDR[0]
sp=kernel_B.arg1:DDR[1]
sp=kernel_C.arg1:DDR[2]
sp=kernel_D.arg1:DDR[1]
```



IMPORTANT! When using the `connectivity.sp` option to assign kernel ports to memory banks, you must map all interfaces/ports of the kernel. Refer to [Mapping Kernel Ports to Memory](#) for more information.

Address Timing

Perform a system run and if it completes with no violations, then the migration is successful.

If timing has not been met you might need to specify some custom constraints to help meet timing. Refer to *UltraFast Design Methodology Timing Closure Quick Reference Guide* ([UG1292](#)) for more information on meeting timing.

Custom Constraints

Custom Tcl constraints for floorplanning, placement, and timing of the kernels will need to be reviewed in the context of the new target platform (.xsa). For example, if a kernel needs to be moved to a different SLR in the new target platform, the placement constraints for that kernel will also need to be modified.

In general, timing is expected to be comparable between different target platforms that are based on the 9P AMD Virtex™ UltraScale™ device. Any custom Tcl constraints for timing closure will need to be evaluated and might need to be modified for the new platform.

Custom constraints can be passed to the AMD Vivado™ tools using the [advanced] directives of the v++ configuration file specified by the --config option. Refer to [Managing Vivado Synthesis, Implementation, and Timing Closure](#) more information.

Timing Closure Considerations

Design performance and timing closure can vary when moving across Vitis releases or target platform(s), especially when one of the following conditions is true:

- Floorplan constraints were needed to close timing.
- Device or SLR resource usage was higher than the typical guideline:
 - LUT usage was higher than 70%
 - DSP, RAMB, and UltraRAM usage was higher than 80%
 - FD usage was higher than 50%
- High effort compilation strategies were needed to close timing.

The usage guidelines provide a threshold above which the compilation of the design can take longer, or performance can be lower than initially estimated. For larger designs which usually require using more than one SLR, specify the kernel/DDR association with the v++ --config option, as described in [Mapping Kernel Ports to Memory](#), while verifying that any floorplan constraint ensures the following:

- The usage of each SLR is below the recommended guidelines.
- The usage is balanced across SLRs if one type of hardware resource needs to be higher than the guideline.

For designs with overall high usage, increasing the amount of pipelining in the kernels, at the cost of higher latency, can greatly help timing closure and achieving higher performance.

For quickly reviewing all aspects listed above, use the fail-fast reports generated throughout the Vitis application acceleration development flow using the -R option as described below (refer to [Controlling Report Generation](#) for more information):

- `v++ -R 1`
 - `report_failfast` is run at the end of each kernel synthesis step
 - `report_failfast` is run after `opt_design` on the entire design
 - `opt_design` DCP is saved
- `v++ -R 2`
 - Same reports as with `-R 1`, plus:
 - `report_failfast` is post-placement for each SLR
 - Additional reports and intermediate DCPs are generated

All reports and DCPs can be found in the implementation directory, including kernel synthesis reports:

```
<runDir>/_x/link/vivado/prj/prj.runs/impl_1
```

For more information about timing closure and the fail-fast report, see the *UltraFast Design Methodology Guide for FPGAs and SoCs* ([UG949](#)).

OpenCL Programming

OpenCL Host Application

In the AMD Vitis™ core development kit, host code is written in C or C++ language using the Xilinx Runtime (XRT) API or industry standard OpenCL™ API. The XRT native API is described on the XRT site at https://xilinx.github.io/XRT/master/html/xrt_native_apis.html. The Vitis core development kit supports the OpenCL 1.2 API as described at <https://www.khronos.org/registry/OpenCL/specs/opencvl-1.2.pdf>. XRT extensions to OpenCL are described at https://xilinx.github.io/XRT/master/html/opencvl_extension.html.



TIP: The code examples shown in this text use the OpenCL C language API.

In general, the structure of the host code can be divided into three sections:

1. Setting up the environment.
2. Core command execution including executing one or more kernels.
3. Post processing and release of resources.



TIP: The Vitis core development kit supports the OpenCL Installable Client Driver (ICD) extension (`cl_khr_icd`). This extension allows multiple implementations of OpenCL to co-exist on the same system. For details and installation instructions, refer to [OpenCL Installable Client Driver Loader](#).

Note: For multithreading the host program, exercise caution when calling a `fork()` system call from a Vitis core development kit application. The `fork()` does not duplicate all the runtime threads. Hence, the child process cannot run as a complete application in the Vitis core development kit. It is advisable to use the `posix_spawn()` system call to launch another process from the Vitis software platform application.

OpenCL Installable Client Driver Loader

The AMD Vitis™ environment supports the OpenCL™ Installable Client Driver (ICD) extension (`cl_khr_icd`). This extension allows multiple implementations of OpenCL to co-exist on the same system. The ICD Loader acts as a supervisor for all installed platforms, and provides a standard handler for all API calls.

Applications can choose an OpenCL platform from the list of installed platforms. Based on the platform ID specified by the application, the ICD dispatches the OpenCL host calls to the right runtime.



TIP: This is an optional package to install if your system has or uses multiple versions of the OpenCL library.

AMD does not provide the OpenCL ICD library, so the following should be used to install the library on your system.

Ubuntu

On Ubuntu the ICD library is packaged with the distribution. Install the following packages:

```
sudo apt-get install ocl-icd-libopencl1
sudo apt-get install opencl-headers
sudo apt-get install ocl-icd-opencl-dev
```

RHEL/CentOS

For RHEL/CentOS use EPEL to install the following packages:

```
sudo yum install ocl-icd
sudo yum install ocl-icd-devel
sudo yum install opencl-headers
```

Note: Refer to <https://fedoraproject.org/wiki/EPEL> for information on installing EPEL if needed.

Setting Up the OpenCL Environment

The host code in the Vitis core development kit follows the OpenCL programming paradigm. To setup the runtime environment properly, the host application needs to initialize the standard OpenCL structures: target platform, devices, context, command queue, and program.



TIP: The host code examples and API commands used in this document follow the OpenCL C API. However, XRT also supports the OpenCL C++ wrapper API, and many of the [Vitis Examples](#) are written using the C++ API. For more information on this C++ wrapper API, refer to <https://www.khronos.org/registry/OpenCL/specs/opencl-cplusplus-1.2.pdf>.

Platform

Upon initialization, the host application needs to identify a platform composed of one or more AMD devices. The following code fragment shows a common method of identifying an AMD platform.

```

cl_platform_id platform_id;           // platform id

err = clGetPlatformIDs(16, platforms, &platform_count);

// Find Xilinx Platform
for (unsigned int iplat=0; iplat<platform_count; iplat++) {
    err = clGetPlatformInfo(platforms[iplat],
        CL_PLATFORM_VENDOR,
        1000,
        (void *)cl_platform_vendor,
        NULL);

    if (strcmp(cl_platform_vendor, "Xilinx") == 0) {
        // Xilinx Platform found
        platform_id = platforms[iplat];
    }
}

```

The OpenCL API call `clGetPlatformIDs` is used to discover the set of available OpenCL platforms for a given system. Then, `clGetPlatformInfo` is used to identify AMD device based platforms by matching `cl_platform_vendor` with the string "Xilinx".



RECOMMENDED: Though it is not explicitly shown in the preceding code, or in other host code examples used throughout this chapter, it is always a good coding practice to use error checking after each of the OpenCL API calls. This can help debugging and improve productivity when you are debugging the host and kernel code in the emulation flow, or during hardware execution. The following code fragment is an error checking code example for the `clGetPlatformIDs` command.

```

err = clGetPlatformIDs(16, platforms, &platform_count);
if (err != CL_SUCCESS) {
    printf("Error: Failed to find an OpenCL platform!\n");
    printf("Test failed\n");
    exit(1);
}

```

Devices

After an AMD platform is found, the application needs to identify the corresponding AMD devices.

The following code demonstrates finding all the AMD devices, with an upper limit of 16, by using API `clGetDeviceIDs`.

```

cl_device_id devices[16]; // compute device id
char cl_device_name[1001];

err = clGetDeviceIDs(platform_id, CL_DEVICE_TYPE_ACCELERATOR,
    16, devices, &num_devices);

```

```
printf("INFO: Found %d devices\n", num_devices);

//iterate all devices to select the target device.
for (uint i=0; i<num_devices; i++) {
    err = clGetDeviceInfo(devices[i], CL_DEVICE_NAME, 1024, cl_device_name,
0);
    printf("CL_DEVICE_NAME %s\n", cl_device_name);
}
```



IMPORTANT! The `clGetDeviceIDs` API is called with the `platform_id` and `CL_DEVICE_TYPE_ACCELERATOR` to receive all the available AMD devices.

Sub-Devices

In the Vitis core development kit, sometimes devices contain multiple kernel instances of a single kernel or of different kernels. While the OpenCL API `clCreateSubDevices` allows the host code to divide a device into multiple sub-devices, the Vitis core development kit supports equally divided sub-devices (using `CL_DEVICE_PARTITION_EQUALLY`), each containing one kernel instance.

The following example shows:

1. Sub-devices created by equal partition to execute one kernel instance per sub-device.
2. Iterating over the sub-device list and using a separate context and command queue to execute the kernel on each of them.
3. The API related to kernel execution (and corresponding buffer related) code is not shown for the sake of simplicity, but would be described inside the function `run_cu`.

```
cl_uint num_devices = 0;
cl_device_partition_property props[3] = {CL_DEVICE_PARTITION_EQUALLY,1,0};

// Get the number of sub-devices
clCreateSubDevices(device,props,0,nullptr,&num_devices);

// Container to hold the sub-devices
std::vector<cl_device_id> devices(num_devices);

// Second call of clCreateSubDevices
// We get sub-device handles in devices.data()
clCreateSubDevices(device,props,num_devices,devices.data(),nullptr);

// Iterating over sub-devices
std::for_each(devices.begin(),devices.end(),[kernel](cl_device_id sdev) {

    // Context for sub-device
    auto context = clCreateContext(0,1,&sdev,nullptr,nullptr,&err);

    // Command-queue for sub-device
    auto queue = clCreateCommandQueue(context,sdev,
CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE,&err);

    // Execute the kernel on the sub-device using local context and
    queue run_cu(context,queue,kernel); // Function not shown
});
```



IMPORTANT! As shown in the example, you must create a separate context for each sub-device. Though OpenCL supports a context that can hold multiple devices and sub-devices, XRT requires each device and sub-device to have a separate context.

Context

The `clCreateContext` API is used to create a context that contains an AMD device that will communicate with the host machine.

```
context = clCreateContext(0, 1, &device_id, NULL, NULL, &err);
```

In the code example, the `clCreateContext` API is used to create a context that contains one AMD device. AMD recommends creating only one context per device or sub-device. However, the host program should use multiple contexts if sub-devices are used with one context for each sub-device.

Command Queues

The `clCreateCommandQueue` API creates one or more command queues for each device. The FPGA can contain multiple kernels, which can be either the same or different kernels. When developing the host application, there are two main programming approaches to execute kernels on a device:

1. Single out-of-order command queue: Multiple kernel executions can be requested through the same command queue. XRT dispatches kernels as soon as possible, in any order, allowing concurrent kernel execution on the FPGA.
2. Multiple in-order command queue: Each kernel execution is requested from different in-order command queues. In such cases, XRT dispatches kernels from the different command queues, improving performance by running them concurrently on the device.



RECOMMENDED: For improved performance, AMD recommends using a single out-of-order command queue and manage event dependencies and synchronizations explicitly, instead of using multiple command queues.

The following is an example of standard API calls to create in-order and out-of-order command queues.

```
// Out-of-order Command queue
commands = clCreateCommandQueue(context, device_id,
CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE, &err);

// In-order Command Queue
commands = clCreateCommandQueue(context, device_id, 0, &err);
```

Program

The host and kernel code are compiled separately to create separate executable files: the host program executable and the FPGA binary (.xclbin). When the host application runs, it must load the .xclbin file using the `clCreateProgramWithBinary` API.

The following code example shows how the standard OpenCL API is used to build the program from the .xclbin file.

```

unsigned char *kernelbinary;
char *xclbin = argv[1];

printf("INFO: loading xclbin %s\n", xclbin);

int size=load_file_to_memory(xclbin, (char **) &kernelbinary);
size_t size_var = size;

cl_program program = clCreateProgramWithBinary(context, 1, &device_id,
                                              &size_var, (const unsigned char **) &kernelbinary,
                                              &status, &err);

// Function
int load_file_to_memory(const char *filename, char **result)
{
    uint size = 0;
    FILE *f = fopen(filename, "rb");
    if (f == NULL) {
        *result = NULL;
        return -1; // -1 means file opening fail
    }
    fseek(f, 0, SEEK_END);
    size = ftell(f);
    fseek(f, 0, SEEK_SET);
    *result = (char *)malloc(size+1);
    if (size != fread(*result, sizeof(char), size, f)) {
        free(*result);
        return -2; // -2 means file reading fail
    }
    fclose(f);
    (*result)[size] = 0;
    return size;
}

```

The example performs the following steps:

1. The kernel binary file, .xclbin, is passed in from the command line argument, `argv[1]`.



TIP: Passing the .xclbin through a command line argument is one approach. You can also hardcode the kernel binary file in the host program, define it with an environment variable, read it from a custom initialization file, or another suitable mechanism.

2. The `load_file_to_memory` function is used to load the file contents in the host machine memory space.
3. The `clCreateProgramWithBinary` API is used to complete the program creation process in the specified context and device.

Executing Commands on the FPGA

Once the OpenCL environment is initialized, the host application is ready to issue commands to the device and interact with the kernels. These commands include:

1. Setting up the kernels.
2. Buffer transfer to/from the FPGA.
3. Kernel execution on FPGA.
4. Event synchronization.

Setting Up Kernels

After setting up the runtime environment, such as identifying devices, creating the context, command queue, and program, the host application should identify the kernels that will execute on the device, and set up the kernel arguments.

The OpenCL API `clCreateKernel` should be used to access the kernels contained within the `.xclbin` file (the "program"). The `cl_kernel` object identifies a kernel in the program loaded into the FPGA that can be run by the host application. The following code example identifies two kernels defined in the loaded program.

```
kernel1 = clCreateKernel(program, "<kernel_name_1>", &err);
kernel2 = clCreateKernel(program, "<kernel_name_2>", &err); // etc
```

Setting Kernel Arguments

In the Vitis software platform, two types of arguments can be set for kernel objects:

1. Scalar arguments are used for small data transfer, such as constant or configuration type data. These are write-only arguments from the host application perspective, meaning they are inputs to the kernel.
2. Memory buffer arguments are used for large data transfer. The value is a pointer to a memory object created with the context associated with the program and kernel objects. These can be inputs to, or outputs from the kernel.

Kernel arguments can be set using the `clSetKernelArg` command, as shown in the following example for setting kernel arguments for two scalar and two buffer arguments.

```
// Create memory buffers
cl_mem dev_buf1 = clCreateBuffer(context, CL_MEM_WRITE_ONLY |
CL_MEM_USE_HOST_PTR, size, &host_mem_ptr1, NULL);
cl_mem dev_buf2 = clCreateBuffer(context, CL_MEM_READ_ONLY |
CL_MEM_USE_HOST_PTR, size, &host_mem_ptr2, NULL);

int err = 0;
// Setup scalar arguments
cl_uint scalar_arg_image_width = 3840;
err |= clSetKernelArg(kernel, 0, sizeof(cl_uint), &scalar_arg_image_width);
```

```

cl_uint scalar_arg_image_height = 2160;
err |= clSetKernelArg(kernel, 1, sizeof(cl_uint),
&scalar_arg_image_height);

// Setup buffer arguments
err |= clSetKernelArg(kernel, 2, sizeof(cl_mem), &dev_buf1);
err |= clSetKernelArg(kernel, 3, sizeof(cl_mem), &dev_buf2);

```



IMPORTANT! Although OpenCL allows setting kernel arguments any time before enqueueing the kernel, you should set kernel arguments as early as possible. XRT will error out if you try to migrate a buffer before XRT knows where to put it on the device. Therefore, set the kernel arguments before performing any enqueue operation (for example, `clEnqueueMigrateMemObjects`) on any buffer.

For all kernel buffer arguments you must allocate the buffer on the device global memories. However, sometimes the content of the buffer is not required before the start of the kernel execution. For example, the output buffer content will only be populated during the kernel execution, and hence it is not important prior to kernel execution. In this case, you should specify `clEnqueueMigrateMemObject` with the `CL_MIGRATE_MEM_OBJECT_CONTENT_UNDEFINED` flag so that migration of the buffer will not involve the DMA operation between the host and the device, thus improving performance.

Buffer Allocation on the Device

By default, when kernels are linked to the platform the memory interfaces from all the kernels are connected to a single default global memory bank. As a result, only a single compute unit (CU) can transfer data to and from the global memory bank at one time, limiting the overall performance of the application.

If the device contains only one global memory bank, then this is the only option. However, if the device contains multiple global memory banks, you can customize the global memory bank connections by modifying the memory interface connection for a kernel during linking. The method for performing this is discussed in detail in [Mapping Kernel Ports to Memory](#). Overall performance is improved by using separate memory banks for different kernels or compute units, enabling multiple kernel memory interfaces to concurrently read and write data.



IMPORTANT! XRT must detect the kernel's memory connection to send data from the host program to the correct memory location for the kernel. XRT will automatically find the buffer location from the kernel binary files if `clSetKernelArgs` is used before any enqueue operation on the buffer, such as `clEnqueueMigrateMemObject`.

Buffer Creation and Data Transfer

Interactions between the host program and hardware kernels rely on creating buffers and transferring data to and from the memory in the device. This process makes use of functions like `clCreateBuffer` and `clEnqueueMigrateMemObjects`.



IMPORTANT! A single buffer cannot be bigger than 4 GB, yet to maximize throughput from the host to global memory, AMD also recommends keeping the buffer size at least 2 MB if possible.

There are two methods for allocating memory buffers, and transferring data:

1. [Letting XRT Allocate Buffers](#)
2. [Using Host Pointer Buffers](#)

In the case where XRT allocates the buffer, use `enqueueMapBuffer` to capture the buffer handle. In the second case, allocate the buffer directly with `CL_MEM_USE_HOST_PTR`, so you do not need to capture the handle.



TIP: Do not use `CL_MEM_USE_HOST_PTR` for embedded platforms. Embedded platforms require contiguous memory allocation and should use the `CL_MEM_ALLOC_HOST_PTR` method, as described in [Letting XRT Allocate Buffers](#).

There are a number of coding practices you can adopt to maximize performance and fine-grain control. The OpenCL API supports additional commands for reading and writing buffers. For example, you can use `clEnqueueWriteBuffer` and `clEnqueueReadBuffer` commands in place of `clEnqueueMigrateMemObjects`. However, some of these commands have different effects that must be understood when using them. For example, `clEnqueueReadBufferRect` can read a rectangular region of a buffer object to the host application, but it does not transfer the data from the device global memory to the host. You must first use `clEnqueueReadBuffer` to transfer the data from the device global memory, and then use `clEnqueueReadBufferRect` to read the desired rectangular portion into the host application.

Letting XRT Allocate Buffers

On data center platforms, it is more efficient to allocate memory aligned on 4k page boundaries. On embedded platforms it is more efficient to perform contiguous memory allocation. In either case, you can let the XRT allocate host memory when creating the buffers. This is done by using the `CL_MEM_ALLOC_HOST_PTR` flag when creating the buffers, and then mapping the allocated memory to user-space pointers using `clEnqueueMapBuffer`. With this approach, it is not necessary to create a host space pointer aligned to the 4K boundary.

The `clEnqueueMapBuffer` API maps the specified buffer and returns a pointer created by XRT to this mapped region. Then, fill the host side pointer with your data, followed by `clEnqueueMigrateMemObject` to transfer the data to and from the device. The following code example uses this style:

```
// Two cl_mem buffer, for read and write by kernel
cl_mem dev_mem_read_ptr = clCreateBuffer(context, CL_MEM_ALLOC_HOST_PTR |
CL_MEM_READ_ONLY,
    sizeof(int) * number_of_words, NULL, NULL);

cl_mem dev_mem_write_ptr = clCreateBuffer(context, CL_MEM_ALLOC_HOST_PTR |
CL_MEM_WRITE_ONLY,
    sizeof(int) * number_of_words, NULL, NULL);

cl::Buffer in1_buf(context, CL_MEM_ALLOC_HOST_PTR | CL_MEM_READ_ONLY,
    sizeof(int) * DATA_SIZE, NULL, &err);
```

```
// Setting arguments
clSetKernelArg(kernel, 0, sizeof(cl_mem), &dev_mem_read_ptr);
clSetKernelArg(kernel, 1, sizeof(cl_mem), &dev_mem_write_ptr);

// Get Host side pointer of the cl_mem buffer object
auto host_write_ptr =
clEnqueueMapBuffer(queue, dev_mem_read_ptr, true, CL_MAP_WRITE, 0, bytes, 0, nullptr,
nullptr, &err);
auto host_read_ptr =
clEnqueueMapBuffer(queue, dev_mem_write_ptr, true, CL_MAP_READ, 0, bytes, 0, nullptr,
nullptr, &err);

// Fill up the host_write_ptr to send the data to the FPGA
for(int i=0; i< MAX; i++) {
    host_write_ptr[i] = <.... >
}

// Migrate
cl_mem mems[2] = {host_write_ptr, host_read_ptr};
clEnqueueMigrateMemObjects(queue, 2, mems, 0, 0, nullptr, &migrate_event));

// Schedule the kernel
clEnqueueTask(queue, kernel, 1, &migrate_event, &enqueue_event);

// Migrate data back to host
clEnqueueMigrateMemObjects(queue, 1, &dev_mem_write_ptr,
                           CL_MIGRATE_MEM_OBJECT_HOST, 1, &enqueue_event,
                           &data_read_event);

clWaitForEvents(1, &data_read_event);

// Now use the data from the host_read_ptr
```

To work with an example using `clEnqueueMapBuffer`, refer to [Data Transfer \(C\)](#) in the [Vitis Examples](#) GitHub repository.

Using Host Pointer Buffers



IMPORTANT! Using `CL_MEM_USE_HOST_PTR` is not recommended for embedded platforms. Embedded platforms require contiguous memory allocation and should use the `CL_MEM_ALLOC_HOST_PTR` method, as described in [Letting XRT Allocate Buffers](#).

There are two main parts of a `cl_mem` object: host side pointer and device side pointer. Before the kernel starts its operation, the device side pointer is implicitly allocated on the device side memory (for example, on a specific location inside the device global memory) and the buffer becomes a resident on the device. Using `clEnqueueMigrateMemObjects` this allocation and data transfer occur upfront, much ahead of the kernel execution. This especially helps to enable *software pipelining* if the host is executing the same kernel multiple times, because data transfer for the next transaction can happen when kernel is still operating on the previous data set, and thus hide the data transfer latency of successive kernel executions.

The OpenCL framework provides a number of APIs for transferring data between the host and the device. Typically, data movement APIs, such as `clEnqueueWriteBuffer` and `clEnqueueReadBuffer`, implicitly migrate memory objects to the device after they are enqueued. They do not guarantee when the data is transferred, and this makes it difficult for the host application to synchronize the movement of memory objects with the computation performed on the data.

AMD recommends using `clEnqueueMigrateMemObjects` instead of `clEnqueueWriteBuffer` or `clEnqueueReadBuffer` to improve the performance. Using this API, memory migration can be explicitly performed ahead of the dependent commands. This allows the host application to preemptively change the association of a memory object, through regular command queue scheduling, to prepare for another upcoming command. This also permits an application to overlap the placement of memory objects with other unrelated operations before these memory objects are needed, potentially hiding or reducing data transfer latencies. After the event associated with `clEnqueueMigrateMemObjects` has been marked complete, the host program knows the memory objects are successfully migrated.



TIP: Another advantage of `clEnqueueMigrateMemObjects` is that it can migrate multiple memory objects in a single API call. This reduces the overhead of scheduling and calling functions to transfer data for more than one memory object.

The following code shows the use of `clEnqueueMigrateMemObjects`:

```
int host_mem_ptr[MAX_LENGTH]; // host memory for input vector

// Fill the memory input
for(int i=0; i<MAX_LENGTH; i++) {
    host_mem_ptr[i] = <... >
}

cl_mem dev_mem_ptr = clCreateBuffer(context,
                                   CL_MEM_READ_WRITE | CL_MEM_USE_HOST_PTR,
                                   sizeof(int) * number_of_words, host_mem_ptr, NULL);

clSetKernelArg(kernel, 0, sizeof(cl_mem), &dev_mem_ptr);

err = clEnqueueMigrateMemObjects(commands, 1, dev_mem_ptr, 0, 0,
                                NULL, NULL);
```

Allocating Page-Aligned Host Memory

XRT allocates memory space in 4K boundary for internal memory management. If the host memory pointer is not aligned to a page boundary, XRT performs extra `memcpy` to make it aligned. Hence you should align the host memory pointer with the 4K boundary to save the extra memory copy operation.

The following is an example of how `posix_memalign` is used instead of `malloc` for the host memory space pointer.

```
int *host_mem_ptr; // = (int*) malloc(MAX_LENGTH*sizeof(int));
// Aligning memory in 4K boundary
posix_memalign(&host_mem_ptr, 4096, MAX_LENGTH*sizeof(int));

// Fill the memory input
for(int i=0; i<MAX_LENGTH; i++) {
    host_mem_ptr[i] = <... >
}

cl_mem dev_mem_ptr = clCreateBuffer(context,
                                    CL_MEM_READ_WRITE | CL_MEM_USE_HOST_PTR,
                                    sizeof(int) * number_of_words, host_mem_ptr, NULL);

err = clEnqueueMigrateMemObjects(commands, 1, dev_mem_ptr, 0, 0,
                                NULL, NULL);
```

Sub-Buffers

Though not very common, using sub-buffers can be very useful in specific situations. The following sections discuss the scenarios where using sub-buffers can be beneficial.

Reading a Specific Portion from the Device Buffer

Consider a kernel that produces different amounts of data depending on the input to the kernel. For example, a compression engine where the output size varies depending on the input data pattern and similarity. The host can still read the whole output buffer by using `clEnqueueMigrateMemObjects`, but that is a suboptimal approach as more than the required memory transfer would occur. Ideally the host program should only read the exact amount of data that the kernel has written.

One technique is to have the kernel write the amount of the output data at the start of writing the output data. The host application can use `clEnqueueReadBuffer` two times, first to read the amount of data being returned, and second to read exact amount of data returned by the kernel based on the information from the first read.

```
clEnqueueReadBuffer(command_queue, device_write_ptr, CL_FALSE, 0,
                    sizeof(int) * 1,
                    &kernel_write_size, 0, nullptr, &size_read_event);
clEnqueueReadBuffer(command_queue, device_write_ptr, CL_FALSE,
                    DATA_READ_OFFSET,
                    kernel_write_size, host_ptr, 1, &size_read_event,
                    &data_read_event);
```

With `clEnqueueMigrateMemObject`, which is recommended over `clEnqueueReadBuffer` or `clEnqueueWriteBuffer`, you can adopt a similar approach by using sub-buffers. This is shown in the following code sample.



TIP: The code sample shows only partial commands to demonstrate the concept.

```
//Create a small sub-buffer to read the quantity of data
cl_buffer_region buffer_info_1={0,1*sizeof(int)};
cl_mem size_info = clCreateSubBuffer (device_write_ptr, CL_MEM_WRITE_ONLY,
    CL_BUFFER_CREATE_TYPE_REGION, &buffer_info_1, &err);

// Map the sub-buffer into the host space
auto size_info_host_ptr = clEnqueueMapBuffer(queue, size_info,,, );

// Read only the sub-buffer portion
clEnqueueMigrateMemObjects(queue, 1, &size_info,
    CL_MIGRATE_MEM_OBJECT_HOST,,,);

// Retrive size information from the already mapped size_info_host_ptr
kernel_write_size = .....

// Create sub-buffer to read the required amount of data
cl_buffer_region buffer_info_2={DATA_READ_OFFSET, kernel_write_size};
cl_mem buffer_seg = clCreateSubBuffer (device_write_ptr,
    CL_MEM_WRITE_ONLY,
    CL_BUFFER_CREATE_TYPE_REGION, &buffer_info_2,&err);

// Map the subbuffer into the host space
auto read_mem_host_ptr = clEnqueueMapBuffer(queue, buffer_seg,,,);

// Migrate the subbuffer
clEnqueueMigrateMemObjects(queue, 1, &buffer_seg,
    CL_MIGRATE_MEM_OBJECT_HOST,,,);

// Now use the read data from already mapped read_mem_host_ptr
```

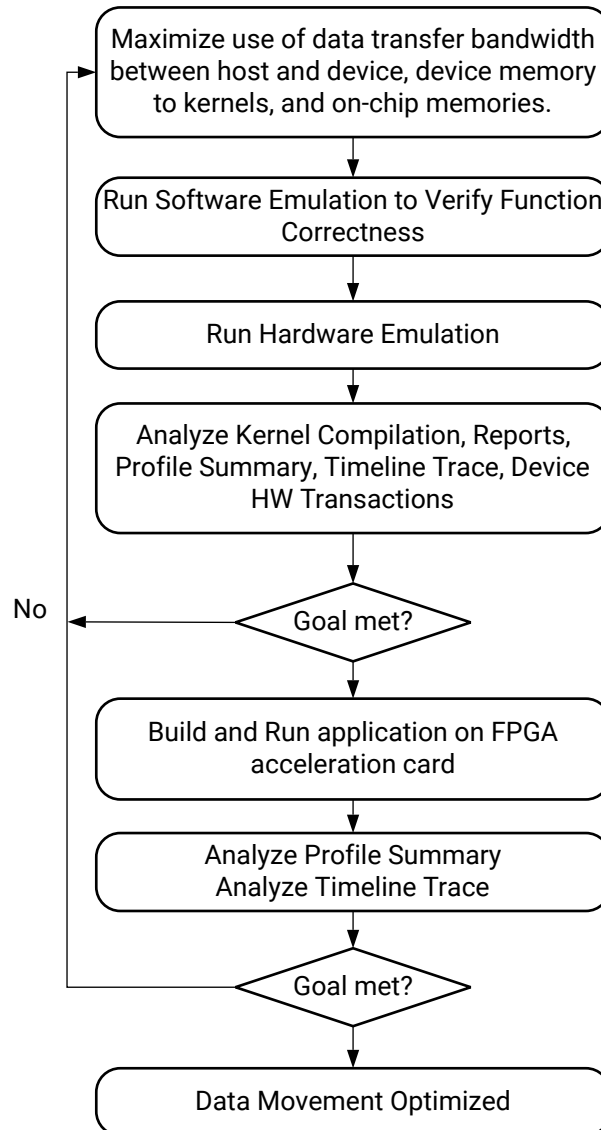
Device Buffer Shared by Multiple Memory Ports or Multiple Kernels

Sometimes memory ports of kernels only require small amounts of data. However, managing small sized buffers, transferring small amounts of data, may have potential performance issues for your application. Alternatively, your host program can create a larger size buffer, divided into smaller sub-buffers. Each sub-buffer is assigned as a kernel argument as discussed in [Setting Kernel Arguments](#), for each of those memory ports requiring small amounts of data.

Once sub-buffers are created they are used in the host code similar to regular buffers. This can improve performance as XRT handles a large buffer in a single transaction, instead of several small buffers and multiple transactions.

Optimizing Data Movement

Figure 150: Optimizing Data Movement Flow



X22239-102320

In the OpenCL execution model, all data is transferred from the host main memory to the global device memory first, and then from the global device memory to the kernel for computation. The computation results are written back from the kernel to the global device memory, and lastly from the global memory to the host main memory. A key factor in determining strategies for kernel data movement optimization is understanding how data can be efficiently moved around between different level of memories maximizing the efficient use of bandwidth on all the memory interfaces.



RECOMMENDED: Optimize the data movement in the application before optimizing computation.

During data movement optimization, it is important to isolate data transfer code from computation code because inefficiency in computation might cause stalls in data movement. You should focus on modifying the data transfer logic in the host and kernel code during this optimization step. The goal is to maximize the system level data throughput by maximizing data transfer bandwidth and device global memory bandwidth usage. It usually takes multiple iterations of running software emulation, hardware emulation, in addition to execution in hardware to achieve optimum performance.

Overlapping Data Transfers with Kernel Computation

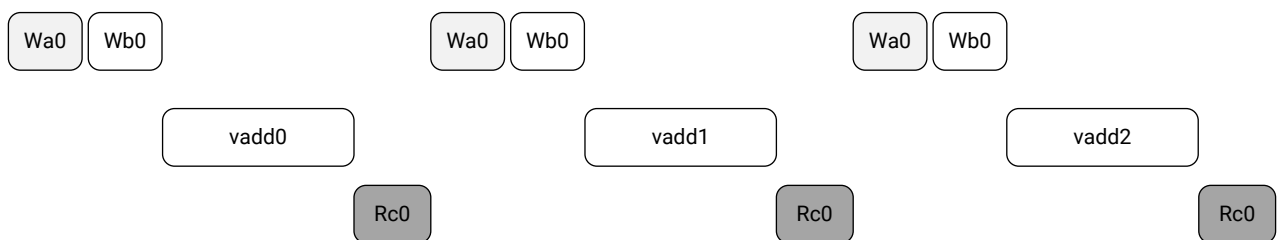
Applications, such as database analytics, have a much larger data set than can be stored in the available global device memory on the acceleration device. They require the complete data to be transferred and processed in blocks. Techniques that overlap the data transfers with the computation are critical to achieve high performance for these applications.

An example can be found in the `vadd` kernel from the [overlap](#) example in the [host](#) category of [Vitis Accelerated Examples](#) on GitHub. This examples demonstrates techniques to overlap Host (CPU) and FPGA computation in the application. In this example, the kernel processes two arrays by adding them together and writing to output. From the host perspective, there are four tasks to perform in this example:

1. Write buffer a (W_a)
2. Write buffer b (W_b)
3. Execute `vadd` kernel
4. Read buffer c (R_c)

Using a simple in-order command queue without data transfer optimization, the overall execution timeline trace should look similar to the one shown below:

Figure 151: Host View of Tasks



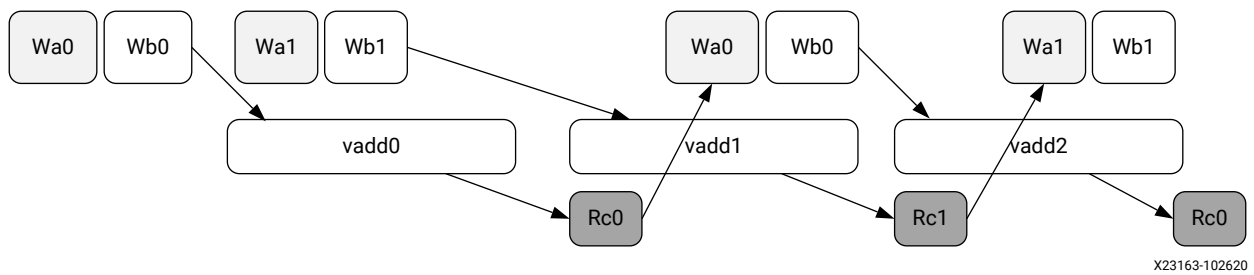
X24770-102720

Using an out-of-order command queue, data transfer and kernel execution can overlap as illustrated in the figure below. In the host code for this example, double buffering is used for all buffers so that the kernel can process one set of buffers while the host can operate on the other set of buffers.

The OpenCL `event` object provides an easy method to set up complex operation dependencies and synchronize host threads and device operations. Events are OpenCL objects that track the status of operations. Event objects are created by kernel execution commands, `read`, `write`, and `copy` commands on memory objects, or user events created using `clCreateUserEvent`.

You can ensure an operation has completed by querying the events returned by these commands. The arrows in the figure below show how event triggering can be set up to achieve optimal performance.

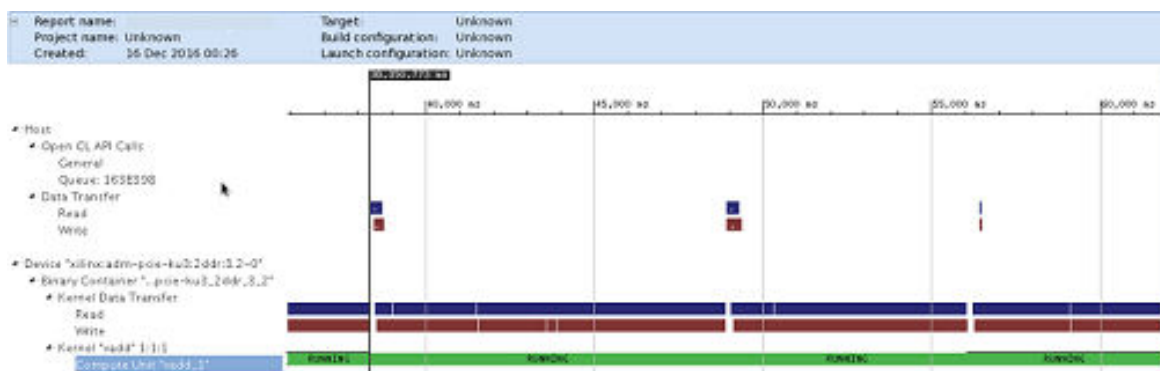
Figure 152: Event Triggering Setup



In the example, the host code (`host.cpp`) enqueues the four tasks in a loop to process the complete data set. It also sets up event synchronization between different tasks to ensure that data dependencies are met for each task. The double buffering is set up by passing different memory objects values to `clEnqueueMigrateMemObjects` API. The event synchronization is achieved by having each API call wait for other event as well as trigger its own event when the API completes.

The Timeline Trace view below clearly shows that the data transfer time is completely hidden, while the compute unit `vadd_1` is running constantly.

Figure 153: Data Transfer Time Hidden in Timeline Trace View



Buffer Memory Segmentation

Allocation and deallocation of memory buffers can lead to memory segmentation in the DDR controllers. This might result in sub-optimal performance of compute units, even if they could theoretically execute in parallel.

This issue occurs most often when multiple pthreads for different compute units are used and the threads allocate and release many device buffers with different sizes every time they enqueue the kernels. In this case, the timeline trace will exhibit gaps between kernel executions and it might seem the processes are sleeping.

Each buffer allocated by runtime should be continuous in hardware. For large memory, it might take some time to wait for that space to be freed, when many buffers are allocated and deallocated. This can be resolved by allocating device buffer and reusing it between different enqueues of a kernel.

Kernel Execution

Often the compute intensive task required by the host application can be defined inside a single kernel, and the kernel is executed only once to work on the entire data range. Because there is an overhead associated with multiple kernel executions, invoking a single monolithic kernel can improve performance. Though the kernel is executed only one time, and works on the entire range of the data, the parallelism is achieved on the FPGA inside the kernel hardware. If properly coded, the kernel is capable of achieving parallelism by various techniques such as instruction-level parallelism (loop pipeline) and function-level parallelism (dataflow). These different kernel coding techniques are discussed in [Developing PL Kernels using C++](#).

When the kernel is compiled to a single hardware instance (or CU) on the FPGA, the simplest method of executing the kernel is using `clEnqueueTask` as shown below.

```
err = clEnqueueTask(commands, kernel, 0, NULL, NULL);
```

XRT schedules the workload, or the data passed through OpenCL buffers from the kernel arguments, and schedules the kernel tasks to run on the accelerator on the AMD FPGA.



IMPORTANT! Though using `clEnqueueNDRangeKernel` is supported (only for OpenCL kernel), AMD recommends using `clEnqueueTask`.

However, sometimes using a single `clEnqueueTask` to run the kernel is not always feasible due to various reasons. For example, the kernel code can become too big and complex to optimize if it attempts to perform all compute intensive tasks in a single execution. Sometimes multiple kernels can be designed performing different tasks on the FPGA in parallel, requiring multiple enqueue commands. Or the host application can be receiving data over time, and not all the data can be processed at one time. Therefore, depending on the situation and application, you may need to

break the data and the task of the kernel into multiple `clEnqueueTask` commands. In this case, an out-of-order command queue, or an in-order command queue can determine how the kernel tasks are processed as explained in [Command Queues](#). In addition, multiple kernel tasks can be implemented as blocking events, or non-blocking events as described in [Event Synchronization](#). These can all affect the performance of the design.

The following topics discuss various methods you can use to run a kernel, run multiple kernels, or run multiple instances of the same kernel on the accelerator.

Reducing Overhead of Kernel Enqueuing

The OpenCL-based execution model supports data parallel and task parallel programming models. An OpenCL host generally needs to call different kernels multiple times. These calls are enqueued in a command queue, either in a certain sequence, or in an out-of-order command queue. Then depending on the availability of compute resources and task data they get scheduled for execution on the device.

Kernel calls can be enqueued for execution on a command queue using `clEnqueueTask`. The dispatching process is executed on the host processor. The dispatcher invokes kernel execution after transferring the kernel arguments to the accelerator running on the device. The dispatcher uses a low-level Xilinx Runtime (XRT) library for transferring kernel arguments and issuing trigger commands for starting the compute. The overhead of dispatching the commands and arguments to the accelerator can be between 30 μ s and 60 μ s, depending on the number of arguments set for the kernel. You can reduce the impact of this overhead by minimizing the number of times the kernel needs to be executed, and minimizing calls to `clEnqueueTask`. Ideally, you should finish all the compute in a single call to `clEnqueueTask`.

You can minimize the calls to `clEnqueueTask` by batching your data and invoking the kernel one time, with a loop wrapped around the original implementation to avoid the overhead of multiple enqueue calls. It can also improve data transfer performance between the host and accelerator, by transferring fewer large data packets rather than many small data packets. For more information on reducing overhead on kernel execution, see [Kernel Execution](#).

The following example shows a simple kernel with given work or data size to process.

```
#define SIZE 256
extern "C" {
    void add(int *a, int *b, int inc){
        int buff_a[SIZE];
        for(int i=0;i<size;i++){
            buff_a[i] = a[i];
        }
        for(int i=0;i<size;i++){
            b[i] = a[i]+inc;
        }
    }
}
```

The following example shows the same simple kernel optimized to process batched data. Depending on the `num_batches` argument the kernel can process multiple inputs of size 256 in a single call and avoid the overhead of multiple `clEnqueueTask` calls. The host application changes to allocate data and buffers in chunks of `SIZE * num_batches`, essentially batching the memory allocation and transfer of data between the host global and device memory.

```
#define SIZE 256
extern "C" {
    void add(int *a , int *b, int inc, int num_batches){
        int buff_a[SIZE];
        for(int j=0;j<num_batches;j++)
        {
            for(int i=0;i<size;i++)
            {
                buff_a[i] = a[i];
            }
            for(int i=0;i<size;i++)
            {
                b[i] = a[i]+inc;
            }
        }
    }
}
```

Task Parallelism Using Different Kernels

Sometimes the compute intensive task required by the host application can be broken into multiple, different kernels designed to perform different tasks on the FPGA in parallel. By using multiple `clEnqueueTask` commands in an out-of-order command queue, for example, you can have multiple kernels performing different tasks, running in parallel. This enables the task parallelism on the FPGA.

Spatial Data Parallelism: Increase Number of Compute Units

Sometimes the compute intensive task required by the host application can process the data across multiple hardware instances of the same kernel, or compute units (CUs) to achieve data parallelism on the FPGA. If a single kernel has been compiled into multiple CUs, the `clEnqueueTask` command can be called multiple times in an out-of-order command queue, to enable data parallelism. Each call of `clEnqueueTask` would schedule a workload of data in different CUs, working in parallel.

Temporal Data Parallelism: Host-to-Kernel Dataflow

Sometimes, the data processed by a compute unit passes from one stage of processing in the kernel, to the next stage of processing. In this case, the first stage of the kernel may be free to begin processing a new set of data. In essence, like a factory assembly line, the kernel can accept new data while the original data moves down the line.

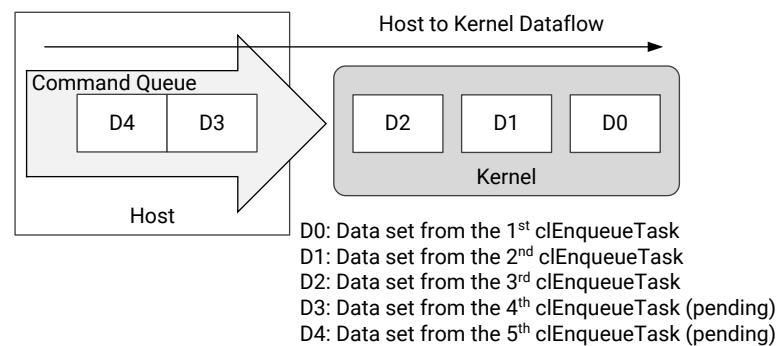
To understand this approach, assume a kernel has only one CU on the FPGA, and the host application enqueues the kernel multiple times with different sets of data. As shown in [Using Host Pointer Buffers](#), the host application can migrate data to the device global memory ahead of the kernel execution, thus hiding the data transfer latency by the kernel execution, enabling *software pipelining*.

However, by default, a kernel can only start processing a new set of data only when it has finished processing the current set of data. Although `clEnqueueMigrateMemObject` hides the data transfer time, multiple kernel executions still remain sequential.

By enabling host-to-kernel dataflow, it is possible to further improve the performance of the accelerator by restarting the kernel with a new set of data while the kernel is still processing the previous set of data. As discussed in [Enabling Host-to-Kernel Dataflow](#), the kernel must implement the `ap_ctrl_chain` interface, and must be written to permit processing data in stages. In this case, XRT restarts the kernel as soon as it is able to accept new data, thus overlapping multiple kernel executions. However, the host program must keep the command queue filled with requests so that the kernel can restart as soon as it is ready to accept new data.

The following is a conceptual diagram for host-to-kernel dataflow.

Figure 154: Host to Kernel Dataflow



X22774-042519

The longer the kernel takes to process a set of data from start to finish, the greater the opportunity to use host-to-kernel dataflow to improve performance. Rather than waiting until the kernel has finished processing one set of data, simply wait until the kernel is ready to begin processing the next set of data. This allows *temporal parallelism*, where different stages of the same kernel processes a different set of data from multiple `clEnqueueTask` commands, in a pipelined manner.

For advanced designs, you can effectively use both the spatial parallelism with multiple CUs to process data, combined with temporal parallelism using host-to-kernel dataflow, overlapping kernel executions on each compute unit.



IMPORTANT! *Embedded processor platforms do not support the host-to-kernel dataflow feature.*

Enabling Host-to-Kernel Dataflow

If a kernel is capable of accepting more data while it is still operating on data from the previous transactions, XRT can send the next batch of data. The kernel then works on multiple data sets in parallel at different stages of the algorithm, thus improving performance. To support host-to-kernel dataflow, the kernel has to implement the `ap_ctrl_chain` protocol using the [pragma HLS interface](#) for the function return:

```
void kernel_name( int *inputs,
...              )// Other input or Output ports
{
#pragma HLS INTERFACE ..... // Other interface pragmas
#pragma HLS INTERFACE ap_ctrl_chain port=return bundle=control
```



IMPORTANT! To take advantage of the host-to-kernel dataflow, the kernel must also be written to process data in stages, such as pipelined at the loop-level as discussed in the *Pipelining Loops* chapter of the Vitis HLS User Guide ([UG1399](#)), or pipelined at the task-level as discussed in the *Dataflow Style Modeling* section in the Vitis HLS User Guide ([UG1399](#)).

Symmetrical and Asymmetrical Compute Units

As discussed in [Creating Multiple Instances of a Kernel](#), multiple compute units (CUs) of a single kernel can be instantiated on the FPGA during the kernel linking process. CUs can be considered symmetrical or asymmetrical with regard to other CUs of the same kernel.

- **Symmetrical:** CUs are considered symmetrical when they have exactly the same `connectivity.sp` options, and therefore have identical connections to global memory. As a result, the Xilinx Runtime can use them interchangeably. A call to `clEnqueueTask` can result in the invocation of any instance in a group of symmetrical CUs.
- **Asymmetrical:** CUs are considered asymmetrical when they do not have exactly the same `connectivity.sp` options, and therefore do not have identical connections to global memory. Using the same setup of input and output buffers, it is not possible for XRT to execute asymmetrical CUs interchangeably.

Kernel Handle and Compute Units

The first time `clSetKernelArg` is called for a given kernel object, XRT identifies the group of symmetrical CUs for subsequent executions of the kernel. When `clEnqueueTask` is called for that kernel, any of the symmetrical CUs in that group can be used to process the task.

If all CUs for a given kernel are symmetrical, a single kernel object is sufficient to access any of the CUs. However, if there are asymmetrical CUs, the host application will need to create a unique kernel object for each group of asymmetrical CUs. In this case, the call to `clEnqueueTask` must specify the kernel object to use for the task, and any matching CU for that kernel can be used by XRT.

Creating Kernel Objects for Specific Compute Units

For creating kernels associated with specific compute units, the `clCreateKernel` command supports specifying the CUs at the time the kernel object is created by the host program. The syntax of this command is shown below:

```
// Create kernel object only for a specific compute unit
cl_kernel kernelA = clCreateKernel(program, "<kernel_name>:
{compute_unit_name}", &err);
// Create a kernel object for two specific compute units
cl_kernel kernelB = clCreateKernel(program, "<kernel_name>:{CU1,CU2}",
&err);
```



IMPORTANT! As discussed in [Creating Multiple Instances of a Kernel](#), the number of CUs is specified by the `connectivity.nk` option in a config file used by the `v++` command during linking. Therefore, whatever is specified in the host program, to create or enqueue kernel objects, must match the options specified by the config file used during linking.

In this case, the Xilinx Runtime identifies the kernel handles (`kernelA`, `kernelB`) for specific CUs, or group of CUs, when the kernel is created. This lets you control which kernel configuration, or specific CU instance is used, when using `clEnqueueTask` from within the host program. This can be useful in the case of asymmetrical CUs, or to perform load and priority management of CUs.

Using Compute Unit Name to Get Handle of All Asymmetrical Compute Units

If a kernel instantiates multiple CUs that are not symmetrical, the `clCreateKernel` command can be specified with CU names to create different CU groups. In this case, the host program can reference a specific CU group by using the `cl_kernel` handle returned by `clCreateKernel`.

In the following example, the kernel `mykernel` has five CUs: K1, K2, K3, K4, and K5. The K1, K2, and K3 CUs are a symmetrical group, having symmetrical connection on the device. Similarly, CUs K4 and K5 form a second symmetrical CU group. The following code segment shows how to address a specific CU group using `cl_kernel` handles.

```
// Kernel handle for Symmetrical compute unit group 1: K1,K2,K3
cl_kernel kernelA = clCreateKernel(program, "mykernel:{K1,K2,K3}", &err);

for(i=0; i<3; i++) {
    // Creating buffers for the kernel_handle1
    ....
    // Setting kernel arguments for kernel_handle1
    ....
    // Enqueue buffers for the kernel_handle1
    ....
    // Possible candidates of the executions K1,K2 or K3
    clEnqueueTask(commands, kernelA, 0, NULL, NULL);
    //
}

// Kernel handle for Symmetrical compute unit group 1: K4, K5
cl_kernel kernelB = clCreateKernel(program, "mykernel:{K4,K5}", &err);

for(int i=0; i<2; i++) {
```

```

// Creating buffers for the kernel_handle2
.....
// Setting kernel arguments for kernel_handle2
.....
// Enqueue buffers for the kernel_handle2
.....
// Possible candidates of the executions K4 or K5
clEnqueueTask(commands, kernelB, 0, NULL, NULL);
}

```

Compute Unit Scheduling

Scheduling kernel operations is key to overall system performance. This becomes even more important when implementing multiple compute units (of the same kernel or of different kernels). This section examines the different command queues responsible for scheduling the kernels.

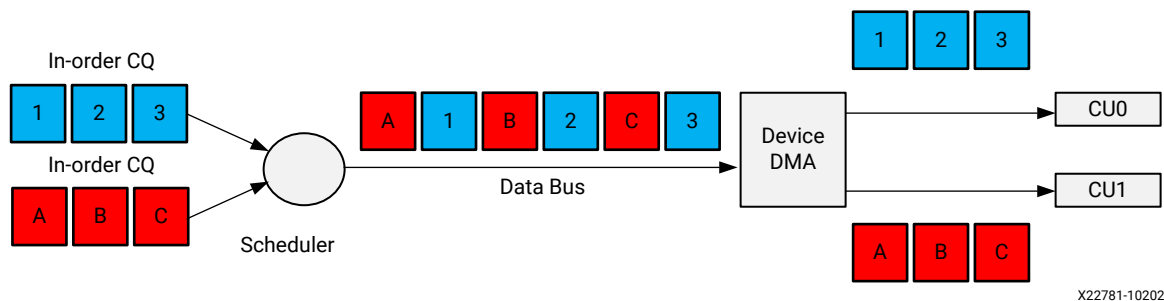
Multiple In-Order Command Queues



RECOMMENDED: For improved performance, AMD recommends using a single out-of-order command queue and manage event dependencies and synchronizations explicitly, instead of using multiple command queues.

The following figure shows an example with two in-order command queues, CQ0 and CQ1. The scheduler dispatches commands from each queue in order, but commands from CQ0 and CQ1 can be pulled out by the scheduler in any order. You must manage synchronization between CQ0 and CQ1 if required.

Figure 155: Example with Two In-Order Command Queues



The following is code extracted from `host.cpp` of the [concurrent_kernel_execution](#) example that sets up multiple in-order command queues and enqueues commands into each queue:

```

OCL_CHECK(
    err,
    cl::CommandQueue ooo_queue(context,
                                device,
                                CL_QUEUE_PROFILING_ENABLE |
                                CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE,
                                &err));
...
printf("[OOO Queue]: Enqueueing scale kernel\n");
OCL_CHECK(
    err,

```

```

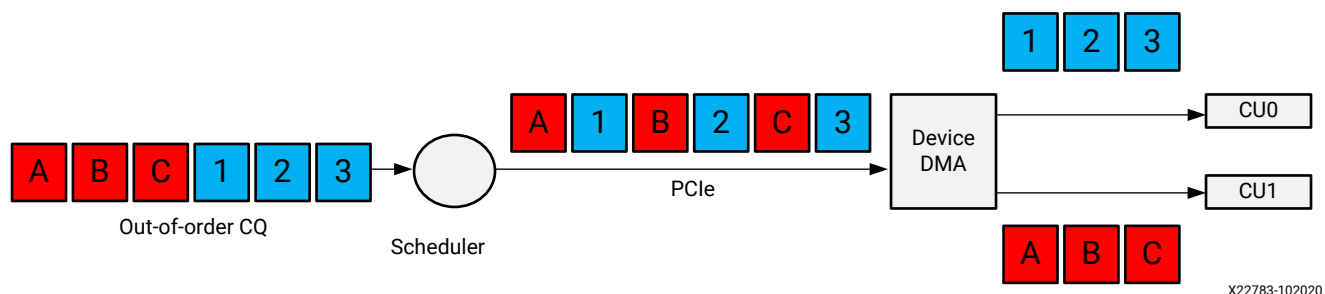
        err = ooo_queue.enqueueTask(
            kernel_mscale, nullptr, &ooo_events[0]));
        set_callback(ooo_events[0], "scale");
    ...
    // This is an out of order queue, events can be executed in any order.
    Since
    // this call depends on the results of the previous call we must pass
    the
    // event object from the previous call to this kernel's event wait list.
    printf("[OOO Queue]: Enqueueing addition kernel (Depends on scale)\n");
    kernel_wait_events.resize(0);
    kernel_wait_events.push_back(ooo_events[0]);
    OCL_CHECK(err,
        err = ooo_queue.enqueueTask(
            kernel_madd,
            &kernel_wait_events, // Event from previous call
            &ooo_events[1]));
    set_callback(ooo_events[1], "addition");
    ...
    // This call does not depend on previous calls so we are passing nullptr
    // into the event wait list. The runtime should schedule this kernel in
    // parallel to the previous calls.
    printf("[OOO Queue]: Enqueueing matrix multiplication kernel\n");
    OCL_CHECK(err,
        err = ooo_queue.enqueueTask(
            kernel_mmult,
            nullptr,
            &ooo_events[2]));
    set_callback(ooo_events[2], "matrix multiplication");

```

Single Out-of-Order Command Queue

The following figure shows an example with a single out-of-order command queue. The scheduler can dispatch commands from the queue in any order. You must manually define event dependencies and synchronizations as required.

Figure 156: Example with Single Out-of-Order Command Queue



X22783-102020

The following is code extracted from `host.cpp` of the [concurrent_kernel_execution](#) example that sets up a single out-of-order command queue and enqueues commands as needed:

```

OCL_CHECK(
    err,
    cl::CommandQueue ooo_queue(context,
                                device,
                                CL_QUEUE_PROFILING_ENABLE |

```



```

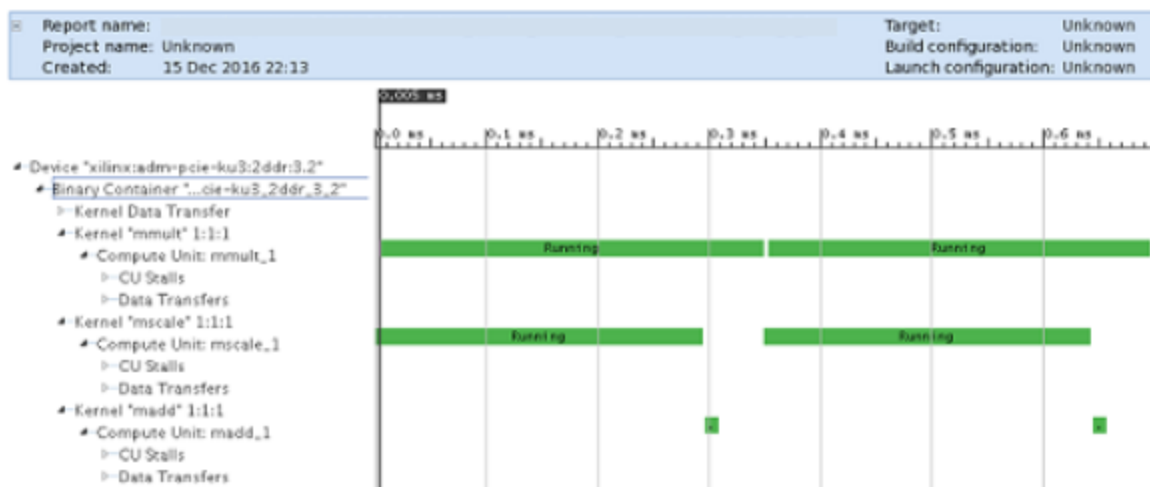
CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE,
    &err));

...
printf("[OOO Queue]: Enqueueing scale kernel\n");
OCL_CHECK(
    err,
    err = ooo_queue.enqueueTask(
        kernel_mscale,nullptr, &ooo_events[0]));
set_callback(ooo_events[0], "scale");
...
// This is an out of order queue, events can be executed in any order.
Since
// this call depends on the results of the previous call we must pass
the
// event object from the previous call to this kernel's event wait list.
printf("[OOO Queue]: Enqueueing addition kernel (Depends on scale)\n");
kernel_wait_events.resize(0);
kernel_wait_events.push_back(ooo_events[0]);
OCL_CHECK(err,
    err = ooo_queue.enqueueTask(
        kernel_madd,
        &kernel_wait_events, // Event from previous call
        &ooo_events[1]));
set_callback(ooo_events[1], "addition");
// This call does not depend on previous calls so we are passing nullptr
// into the event wait list. The runtime should schedule this kernel in
// parallel to the previous calls.
printf("[OOO Queue]: Enqueueing matrix multiplication kernel\n");
OCL_CHECK(err,
    err = ooo_queue.enqueueTask(
        kernel_mmult,
        nullptr,
        &ooo_events[2]));
set_callback(ooo_events[2], "matrix multiplication");

```

The Timeline Trace view shows that the compute unit `mmult_1` is running in parallel with the compute units `mscale_1` and `madd_1`, using both multiple in-order queues and single out-of-order queue methods.

Figure 157: Timeline Trace View Showing `mult_1` Running with `mscale_1` and `madd_1`



Event Synchronization

All OpenCL enqueue-based API calls are asynchronous. These commands will return immediately after the command is enqueued in the command queue. To pause the host program to wait for results, or resolve any dependencies among the commands, an API call such as `clFinish` or `clWaitForEvents` can be used to block execution of the host program.

The following code shows examples for `clFinish` and `clWaitForEvents`.

```
err = clEnqueueTask(command_queue, kernel, 0, NULL, NULL);
// Execution will wait here until all commands in the command queue are
// finished
clFinish(command_queue);

// Create event, read memory from device, wait for read to complete, verify
// results
cl_event readevent;
// host memory for output vector
int host_mem_output_ptr[MAX_LENGTH];
// Enqueue ReadBuffer, with associated event object
clEnqueueReadBuffer(command_queue, dev_mem_ptr, CL_TRUE, 0, sizeof(int) *
number_of_words,
    host_mem_output_ptr, 0, NULL, &readevent );
// Wait for clEnqueueReadBuffer event to finish
clWaitForEvents(1, &readevent);
// After read is complete, verify results
...
```

Note how the commands are used in the example above:

1. The `clFinish` API has been explicitly used to block the host execution until the kernel execution is finished. This is necessary otherwise the host can attempt to read back from the FPGA buffer too early and may read garbage data.
2. The data transfer from FPGA memory to the local host machine is done through `clEnqueueReadBuffer`. Here the last argument of `clEnqueueReadBuffer` returns an event object that identifies this particular read command, and can be used to query the event, or wait for this particular command to complete. The `clWaitForEvents` command specifies a single event (the `readevent`), and waits to ensure the data transfer is finished before verifying the data.

Debugging OpenCL Kernels

For OpenCL kernels, additional runtime checks can be performed during software emulation. These additional checks include:

- Checking whether an OpenCL kernel makes out-of-bounds accesses to the interface buffers (`fsanitize=address`).
- Checking whether the kernel makes accesses to uninitialized local memory (`fsanitize=memory`).

These are Vitis compiler options that are enabled through the `--advanced` compiler option as described in [--advanced Options](#), using the following command syntax:

```
--advanced.param compiler.fsanitize=address,memory
```

When applied, the emulation run produces a debug log with emulation diagnostic messages that are written to `<project_dir>/Emulation-SW/<proj_name>-Default/emulation_debug.log`.

The `fsanitize` directive can also be specified in a config file, as follows:

```
[advanced]
#param=<param_type>:<param_name>.<value>
param=compiler.fsanitize=address,memory
```

The config file is specified on the `v++` command line:

```
v++ -l -t sw_emu --config ./advanced.cfg -o bin_kernel.xclbin
```

Refer to the [Vitis Compiler Configuration File](#) for more information on the `--config` option.

Using `printf()` to Debug OpenCL Kernel

The Xilinx Runtime (XRT) library supports the OpenCL™ `printf()` built-in function within kernels in all build configurations: software emulation, hardware emulation, and during hardware execution.



TIP: The `printf()` function is only supported in all build configurations for OpenCL kernels. For C/C++ kernels, `printf()` is only supported in software emulation.

The following is an example of using `printf()` in the kernel, and the output when the kernel is executed with `global` size of 8:

```
__kernel __attribute__((reqd_work_group_size(1, 1, 1)))
void hello_world(__global int *a)
{
    int idx = get_global_id(0);

    printf("Hello world from work item %d\n", idx);
    a[idx] = idx;
}
```

The output is as follows:

```
Hello world from work item 0
Hello world from work item 1
Hello world from work item 2
Hello world from work item 3
Hello world from work item 4
Hello world from work item 5
Hello world from work item 6
Hello world from work item 7
```



IMPORTANT! *printf()* messages are buffered in the global memory and unloaded when kernel execution is completed. If *printf()* is used in multiple kernels, the order of the messages from each kernel display on the host terminal is not certain. Note, especially when running in hardware emulation and hardware, the hardware buffer size might limit *printf* output capturing.

Assigning DDR Bank in Host Code



IMPORTANT! This is optional and only needed in specific cases as described below.

During the Vitis tool flow, the kernel port to memory bank connectivity can be established using the `--connectivity.sp` switch as described in [Mapping Kernel Ports to Memory](#). The `xclbin` generated by `v++` contains the information about the kernel port to memory connectivity so that XRT can allocate buffers appropriately. When a buffer is created in the host code, XRT automatically assigns the buffer to memory from the kernel `xclbin`, and manages the buffers internally. If a single kernel port is connected to multiple memory banks, XRT always starts from the lower numbered bank.

In most cases, this approach is sufficient. However, in some specific cases you may need to manually assign the buffer location (or special property) in the host code. For this purpose, the AMD OpenCL vendor extension provides a buffer extension called `CL_MEM_XRT_PTR_XILINX` to specifically manage bank assignment in the host code. The following code example shows the required header file and code for assigning input and output buffers to DDR bank 0 and bank 1:

```
#include <CL/cl_ext.h>
...
int main(int argc, char** argv)
{
    ...
    cl_mem_ext_ptr_t inExt, outExt; // Declaring two extensions for both
    buffers
    inExt.flags = 0|XCL_MEM_TOPOLOGY; // Specify Bank0 Memory for input
    memory
    outExt.flags = 1|XCL_MEM_TOPOLOGY; // Specify Bank1 Memory for output
    Memory
    inExt.obj = 0 ; outExt.obj = 0; // Setting Obj and Param to Zero
    inExt.param = 0 ; outExt.param = 0;

    int err;
    //Allocate Buffer in Bank0 of Global Memory for Input Image using
    Xilinx Extension
    cl_mem buffer_inImage = clCreateBuffer(world.context, CL_MEM_READ_ONLY
    | CL_MEM_EXT_PTR_XILINX,
        image_size_bytes, &inExt, &err);
    if (err != CL_SUCCESS){
        std::cout << "Error: Failed to allocate device Memory" << std::endl;
        return EXIT_FAILURE;
    }
    //Allocate Buffer in Bank1 of Global Memory for Input Image using
    Xilinx Extension
    cl_mem buffer_outImage = clCreateBuffer(world.context,
    CL_MEM_WRITE_ONLY | CL_MEM_EXT_PTR_XILINX,
        image_size_bytes, &outExt, NULL);
    if (err != CL_SUCCESS){
```

```

        std::cout << "Error: Failed to allocate device Memory" << std::endl;
        return EXIT_FAILURE;
    }
    ...
}

```

The extension pointer `cl_mem_ext_ptr_t` is a struct as defined below:

```

typedef struct{
    unsigned flags;
    void *obj;
    void *param;
} cl_mem_ext_ptr_t;

```

- Valid values for `flags` are:
 - `XCL_MEM_DDR_BANK0`
 - `XCL_MEM_DDR_BANK1`
 - `XCL_MEM_DDR_BANK2`
 - `XCL_MEM_DDR_BANK3`
 - `<id> | XCL_MEM_TOPOLOGY`

Note: The `<id>` is determined by looking at the Memory Configuration section in the `xxx.xclbin.info` file generated next to the `xxx.xclbin` file. In the `xxx.xclbin.info` file, the global memory (DDR, HBM, PLRAM, etc.) is listed with an index representing the `<id>`.
- `obj` is the pointer to the associated host memory allocated for the CL memory buffer only if `CL_MEM_USE_HOST_PTR` flag is passed to `clCreateBuffer` API, otherwise set it to `NULL`.
- `param` is reserved for future use. Always assign it to 0 or `NULL`.

Here are some specific cases where you might want to use the extension pointer:

- P2P Buffer:** For an explanation and example, refer to <https://xilinx.github.io/XRT/master/html/p2p.html>.
- Host-Memory Buffer:** For an explanation and example, refer to <https://xilinx.github.io/XRT/master/html/hm.html>.
- Allocating the host buffer to a specific bank when the kernel port is connected to multiple banks:** For example, `DDR[0:1]`. This use case is described in detail in the [Using Multiple DDR Banks](#) lab of the *Vitis Optimizing Accelerated FPGA Applications: Bloom Filter Example* tutorial.

Example of Allocating the Host Buffer to a Specific Bank

An example of the third case listed above, where you might need to use `cl_mem_ext_ptr_t`, is when the host and kernel are both accessing the DDR bank simultaneously, and you would like to split the data so that kernel and host access memory banks in a ping-pong fashion. When the host is writing/reading to a specific memory bank, the kernel is writing/reading from another bank so that these host/kernel accesses don't compete and impact performance. For this scenario, you must manage the buffer allocation yourself.

The kernel ports in the `xclbin` are connected to DDR bank1 and bank2, and reading the data from these banks alternatively. The connectivity is established during linking by the Vitis compiler using the `--connectivity.sp` switch:

```
[connectivity]
sp=runOnfpga_1.input_words:DDR[1:2]
```

From the host code, you can send the `input_words` data to DDR banks 1 and 2 alternatively. Two AMD extension pointer (`cl_mem_ext_ptr_t`) objects are created as shown in the example code below. The object flags will determine which DDR bank each buffer will be assigned to for the kernel to access. The kernel argument can be set to `input_words[0]` and `input_words[1]` for consecutive kernel enqueues.

```
#include <CL/cl_ext.h>
...
int main(int argc, char** argv)
{
    cl_mem_ext_ptr_t buffer_words_ext[2];

    buffer_words_ext[0].flags = 1 | XCL_MEM_TOPOLOGY; // DDR[1]
    buffer_words_ext[0].param = 0;
    buffer_words_ext[0].obj = input_doc_words;
    buffer_words_ext[1].flags = 2 | XCL_MEM_TOPOLOGY; // DDR[2]
    buffer_words_ext[1].param = 0;
    buffer_words_ext[1].obj = input_doc_words;
    ...
}
```

Assigning Global Memory for Kernel Code

Creating Multiple AXI Interfaces

OpenCL kernels, C/C++ kernels, and RTL kernels have different methods for assigning function parameters to AXI interfaces.

- For OpenCL kernels, the `--max_memory_ports` option is required to generate one AXI4 interface for each global pointer on the kernel argument. The AXI4 interface name is based on the order of the global pointers on the argument list.

The following code is taken from the example `gmem_2banks_ocl` in the `ocl_kernels` category from the [Vitis Accel Examples](#) on GitHub:

```
__kernel __attribute__((reqd_work_group_size(1, 1, 1)))
void apply_watermark(__global const TYPE * __restrict input,
__global TYPE * __restrict output, int width, int height) {
    ...
}
```

In this example, the first global pointer `input` is assigned an AXI4 name `M_AXI_GMEM0`, and the second global pointer `output` is assigned a name `M_AXI_GMEM1`.

- For C/C++ kernels, multiple AXI4 interfaces are generated by specifying different “bundle” names in the HLS INTERFACE pragma for different global pointers. Refer to [HW Interfaces](#) for more information.

The following is a code snippet from the `gmem_2banks` example that assigns the `input` pointer to the bundle `gmem0` and the `output` pointer to the bundle `gmem1`. The bundle name can be any valid C string, and the AXI4 interface name generated will be `M_AXI_<bundle_name>`. For this example, the input pointer will have AXI4 interface name as `M_AXI_gmem0`, and the output pointer will have `M_AXI_gmem1`. Refer to [pragma HLS interface](#) for more information.

```
#pragma HLS INTERFACE m_axi port=input offset=slave bundle=gmem0
#pragma HLS INTERFACE m_axi port=output offset=slave bundle=gmem1
```

- For RTL kernels, the port names are generated during the import process by the RTL kernel wizard. The default names proposed by the RTL kernel wizard are `m00_axi` and `m01_axi`. If not changed, these names have to be used when assigning a DDR bank through the `connectivity.sp` option in the configuration file. Refer to [Mapping Kernel Ports to Memory](#) for more information.

Post-Processing and FPGA Cleanup

At the end of the host code, all the allocated resources should be released by using proper release functions. If the resources are not properly released, the Vitis core development kit might not be able to generate a correct performance related profile and analysis report.

```
clReleaseCommandQueue(Command_Queue);
clReleaseContext(Context);
clReleaseDevice(Target_Device_ID);
clReleaseKernel(Kernel);
clReleaseProgram(Program);
free(Platform_IDs);
free(Device_IDs);
```

Compiling and Linking the Host Application

Building OpenCL API Host Code for x86

The AMD Vitis™ application acceleration development flow also supports the use of OpenCL API to program your host application. Building OpenCL applications using `g++` uses the following command line:

```
g++ -g -std=c++1y -I$XILINX_XRT/include -L$XILINX_XRT/lib -o host.exe  
host.cpp \  
-lOpenCL -pthread
```

The only difference is the use of the `OpenCL` library for the OpenCL API in place of the `xrt_coreutil` library for the XRT native API.

Note: In [Vitis Accel_Examples](#) you can see the addition of `xc12.cpp` source file, and the `-I../xc12` include statement. These additions to the host program and `g++` command provide access to helper utilities used by the example code, but are generally not required for your own code.

Building OpenCL API Host Code for Arm Processor

The Vitis application acceleration development flow also supports the use of OpenCL API to program your host application. Building OpenCL applications using `g++` uses the following command line:

```
g++ -g -std=c++1y -I$XILINX_XRT/include -L$XILINX_XRT/lib -o host.exe  
host.cpp \  
-lOpenCL -pthread
```

The only difference is the use of the `OpenCL` library for the OpenCL API in place of the `xrt_coreutil` library for the XRT native API.

Note: In [Vitis Accel_Examples](#) you can see the addition of `xc12.cpp` source file, and the `-I../xc12` include statement. These additions to the host program and `g++` command provide access to helper utilities used by the example code, but are generally not required for your own code.

Output Structure of the Vitis Tools

Output Directories of the v++ Command

The directory structure generated by the command-line flow has been organized to let you easily find and access files from the project. By navigating the various `compile`, `link`, `logs`, and `reports` directories, you can easily find generated files. Similarly, each kernel also has a directory structure created.

You can optionally change the directory structure using the following `v++` options:

```
--temp_dir <dir_name>
--log_dir <dir_name>
--report_dir <dir_name>
```

When using `v++` on the command line, by default it creates a directory structure during compile and link. The `.xo` and `xclbin` files are always generated in the current working directory. All the intermediate files are created under the directory specified by the `--temp_dir` option, which defaults to `_x` when `--temp_dir` is not specified. The `link`, `logs`, and `reports` directories default to inside of the `temp_dir`, and contain the respective information on the builds.

The example directory provided below results from the following command lines:

```
## Kernel Compilation command:
v++ -c -t sw_emu --platform xilinx_u200_gen3x16_xdma_2_202110_1 --
config ./src/u200.cfg \
-k vadd -I./src ./src/vadd.cpp -o hw_emu/vadd.xo

## Device Binary Linking Command:
v++ -l -t sw_emu --platform xilinx_u200_gen3x16_xdma_2_202110_1 --
config ./src/u200.cfg \
hw_emu/vadd.xo -o hw_emu/vadd.xclbin
```

The `u200.cfg` file defines the following options:

```
debug=1
save-temps=1

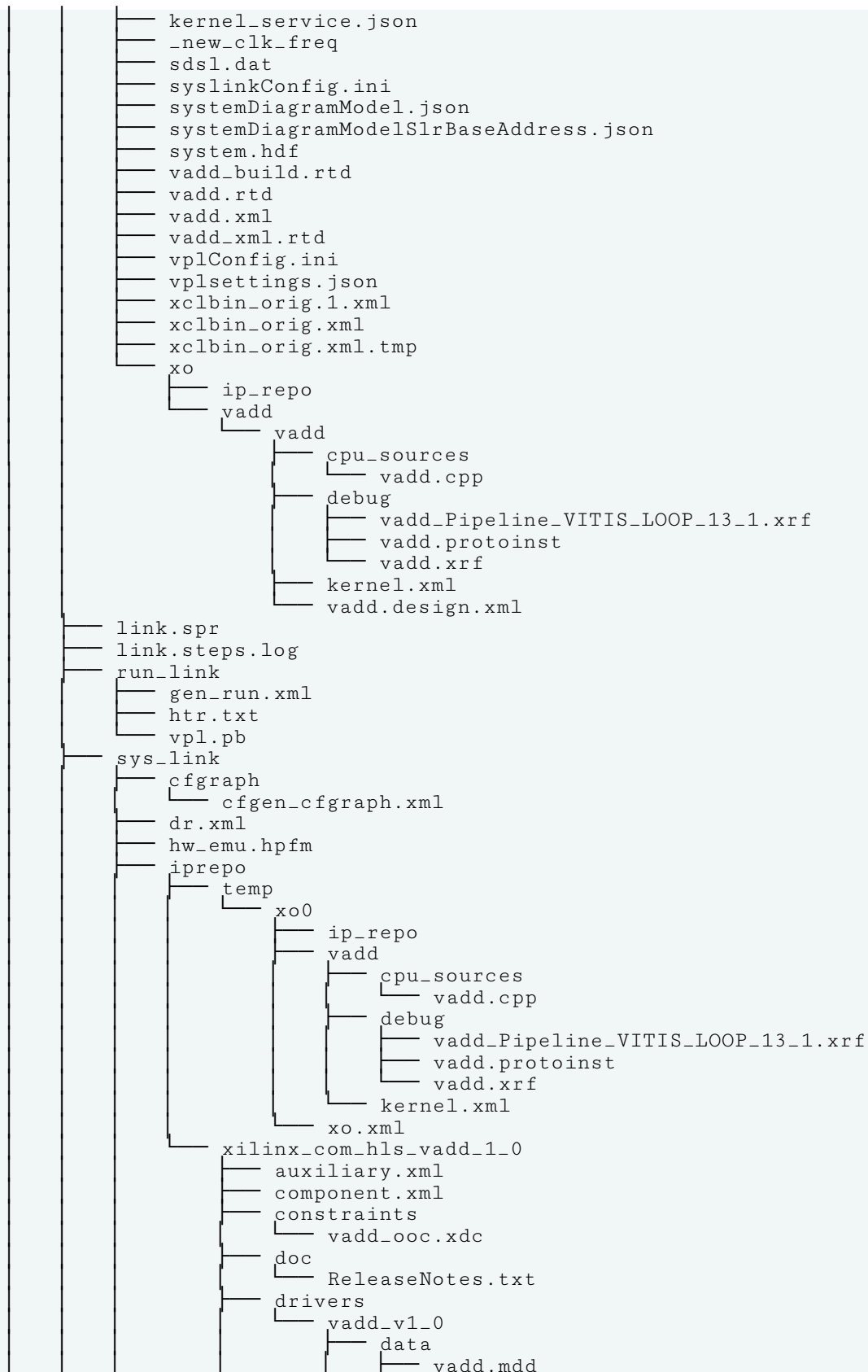
[connectivity]
nk=vadd:1:vadd_1
sp=vadd_1.in1:DDR[1]
```

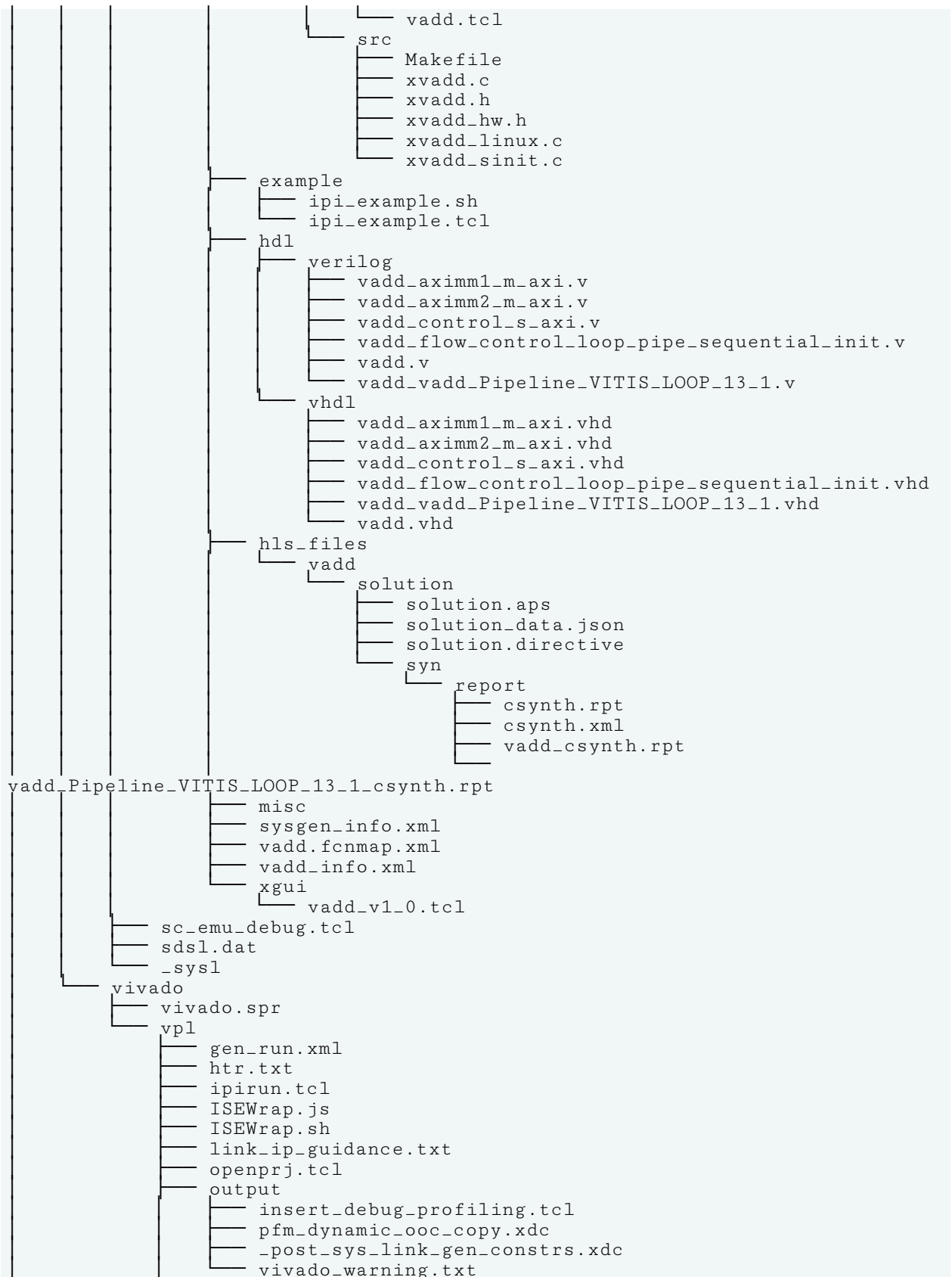
```
sp=vadd_1.in2:DDR[2]
sp=vadd_1.out:DDR[1]

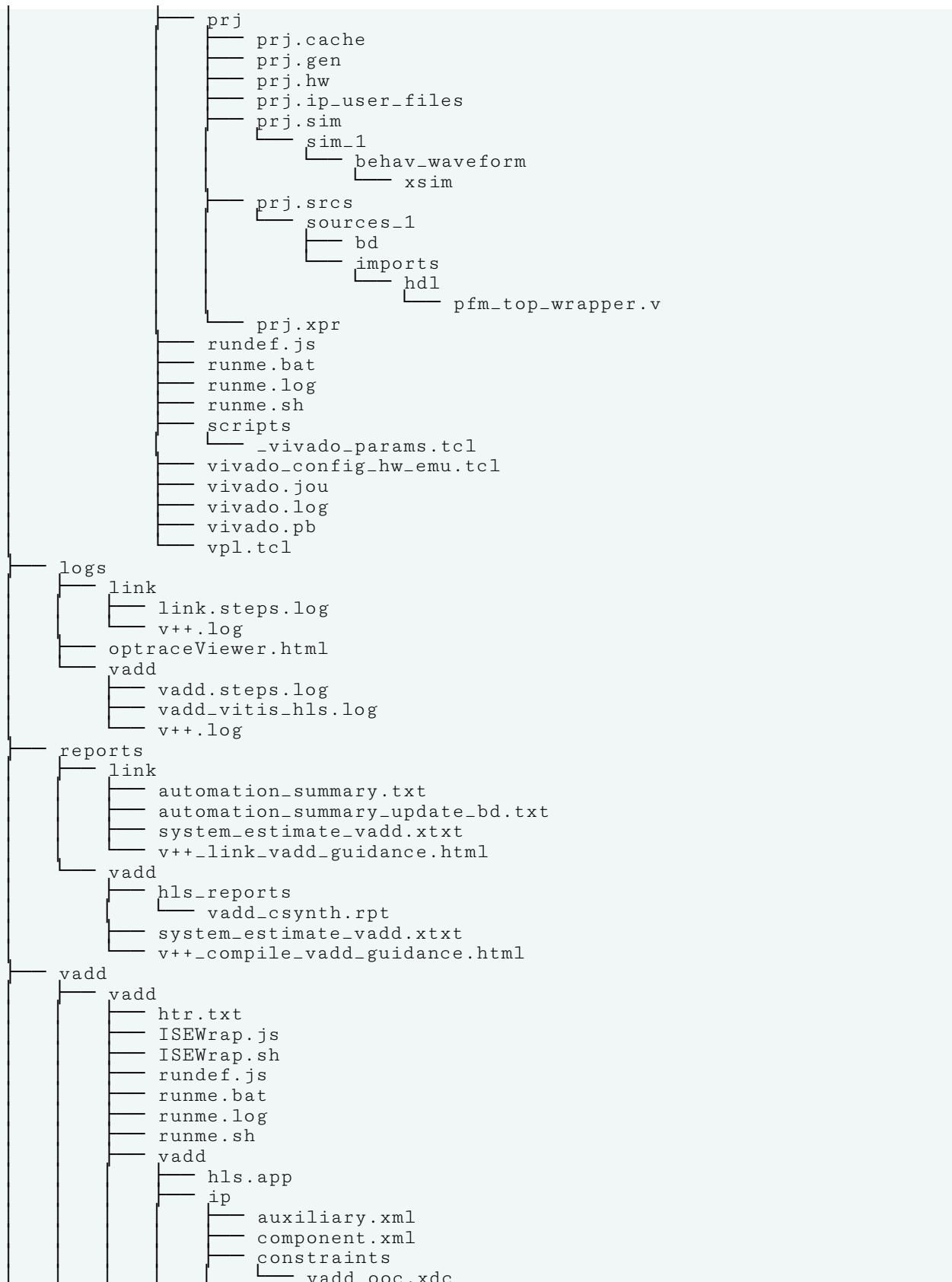
[profile]
data=all:all:all
```

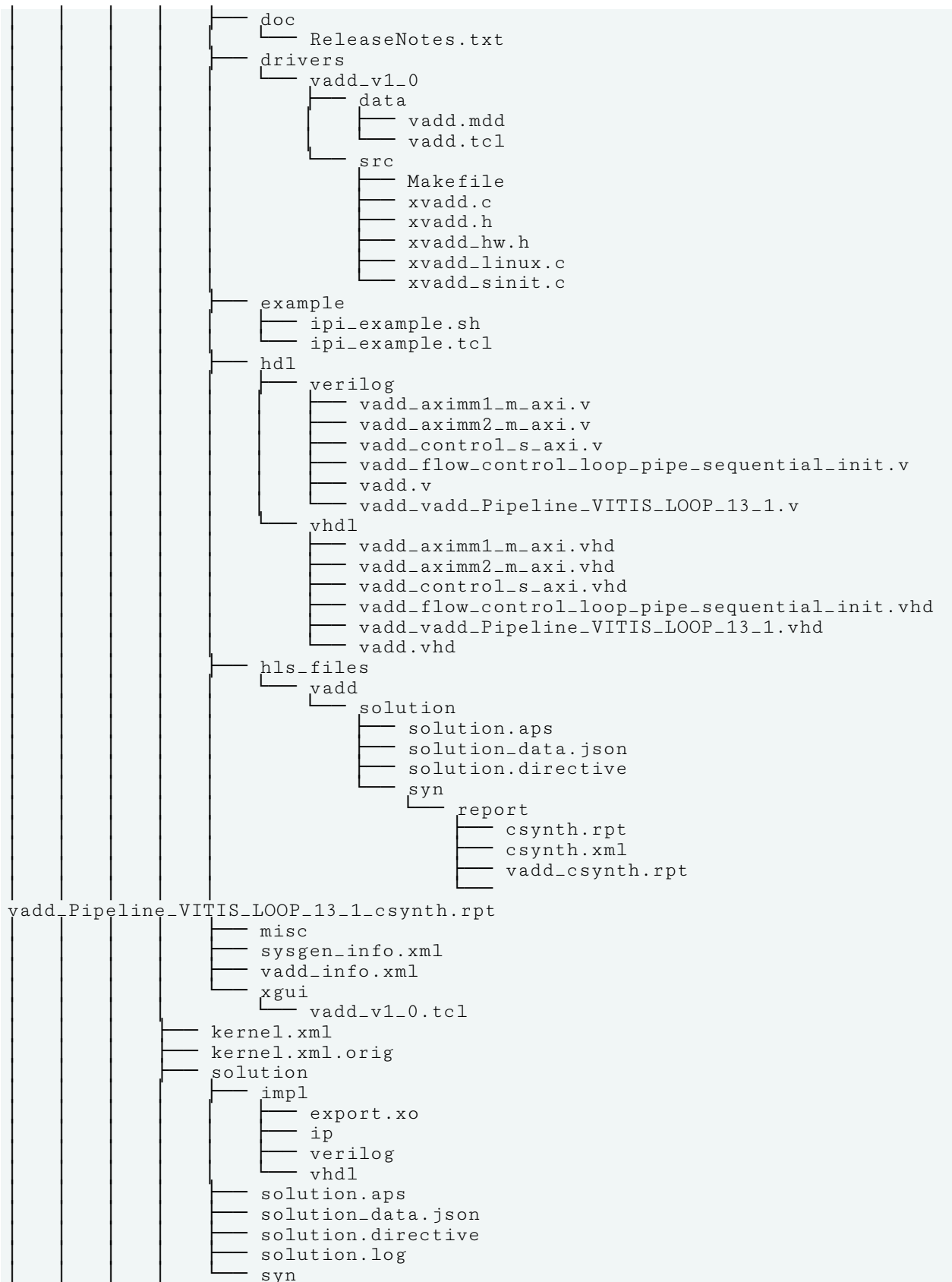
An example of the `temp_dir` output of the `v++ link` command follows:

```
link
├── activetask.json
├── int
│   ├── address_map.xml
│   ├── appendSection.rtd
│   ├── automation_summary.txt
│   ├── automation_summary_update_bd.txt
│   ├── behav_waveform
│   └── xsim
│       ├── compile.log
│       ├── compile.sh
│       ├── dr_behav.protoinst
│       ├── dr_behav.wcfg
│       ├── elaborate.log
│       ├── elaborate.sh
│       ├── glbl.v
│       ├── launch_hw_emu.sh
│       ├── libdpi.so
│       ├── pfm_top_wrapper.tcl
│       ├── pfm_top_wrapper_vhdl.prj
│       ├── pfm_top_wrapper_vlog.prj
│       ├── pfm_top_wrapper_xsc.prj
│       ├── post_sim_tool_scripts.tcl
│       ├── preprocess_profile.tcl
│       ├── pre_sim_tool_scripts.tcl
│       ├── prj.smi
│       ├── profile.tcl
│       ├── protoinst_files
│       ├── run.sh
│       ├── sc_xtlm_bd_d216_interconnect_DDR4_MEM01_0.mem
│       ├── sc_xtlm_bd_d216_interconnect_M00_AXI_MEM00_0.mem
│       ├── sc_xtlm_pfm_dynamic_smartconn_data_0_0.mem
│       ├── sc_xtlm_pfm_top_smartconnect_0_0.mem
│       ├── simulate.log
│       ├── simulate.sh
│       ├── vitis_params.tcl
│       ├── waveform_debug_enable.txt
│       ├── xelab.pb
│       ├── xsc.log
│       ├── xsc.pb
│       ├── xsim.dir
│       ├── xsim.ini
│       ├── xsim.ini.bak
│       ├── xvhdl.log
│       ├── xvhdl.pb
│       ├── xvlog.log
│       └── xvlog.pb
├── behav.xse
├── cf2sw_full.rtd
├── cf2sw.rtd
├── debug_ip_layout.rtd
├── dr.bd.tcl
├── kernel_info.dat
└── _kernel_inst_paths.dat
```









```

├── vadd.design.xml
├── vadd.tcl
├── vitis_hls.log
├── vitis_hls.pb
├── vadd.spr
├── vadd.steps.log
├── v++_compile_vadd_guidance.pb3
└── v++_link_vadd_guidance.pb3

```

Output Directories of `v++ -c --mode aie`

The directory structure generated by the `v++ -c --mode aie` command has been organized to let you easily find and access files from the project.

The example directory provided below results from the following command lines:

```

v++ -c --mode aie --target hw --config ./workAIE/aie_sys_design_aie/
aiecompiler.cfg \
--platform $XILINX_VITIS/base_platforms/xilinx_vck190_base_202320_1/
xilinx_vck190_base_202320_1.xpfm \
--work_dir ./newAIE/hw/Work ./workAIE/aie_sys_design_aie/src/graph.cpp

```

Where `aie_config.cfg` file defines the following options:

```

include=./workAIE/aie_sys_design_aie
include=./workAIE/aie_sys_design_aie/./aie_sys_design_common/src
include=./workAIE/aie_sys_design_aie/src

[aie]
Xchess=main:darts.xargs=-nb
log-level=1
stacksize=1024
heapsize=1024

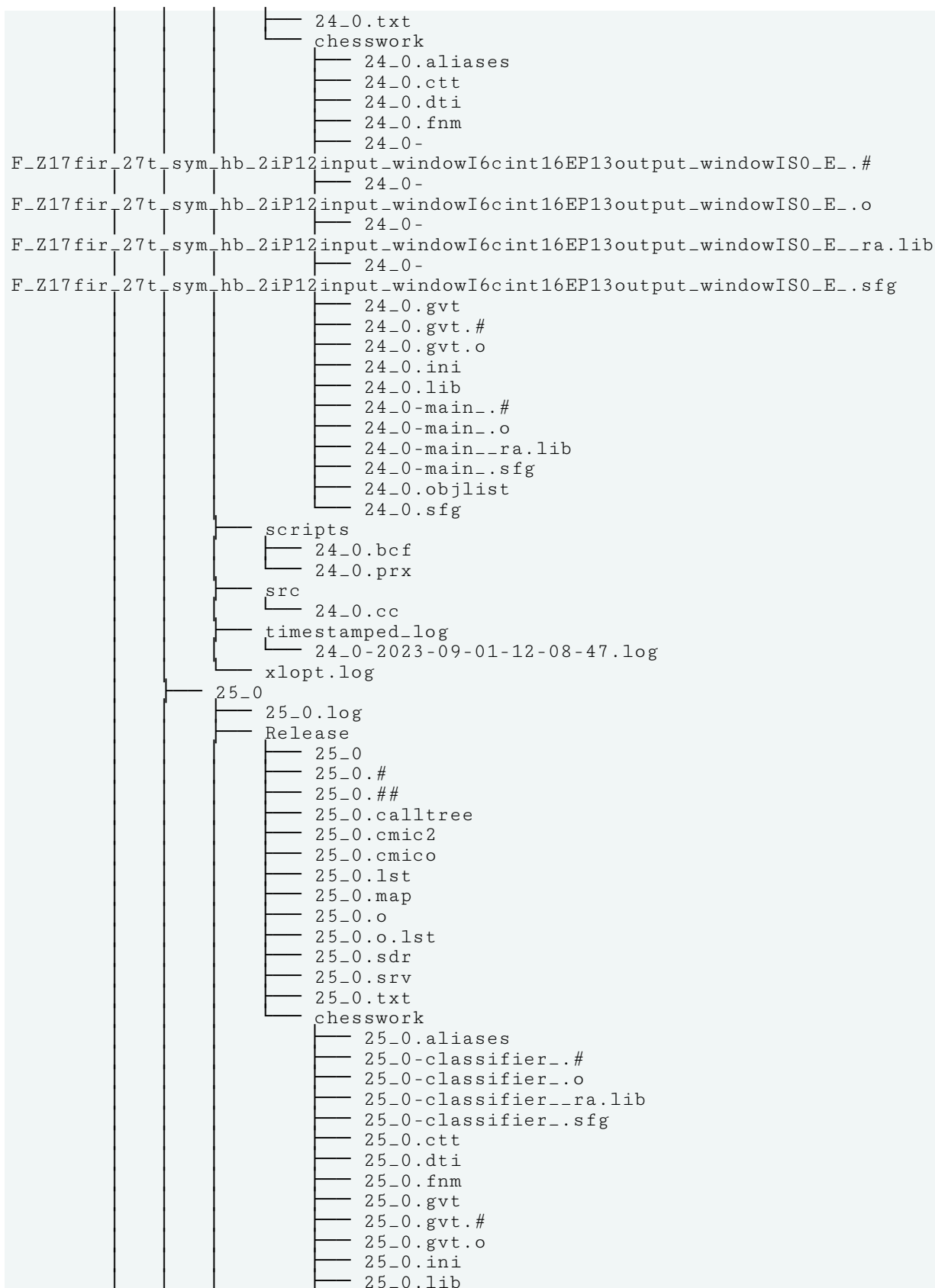
```

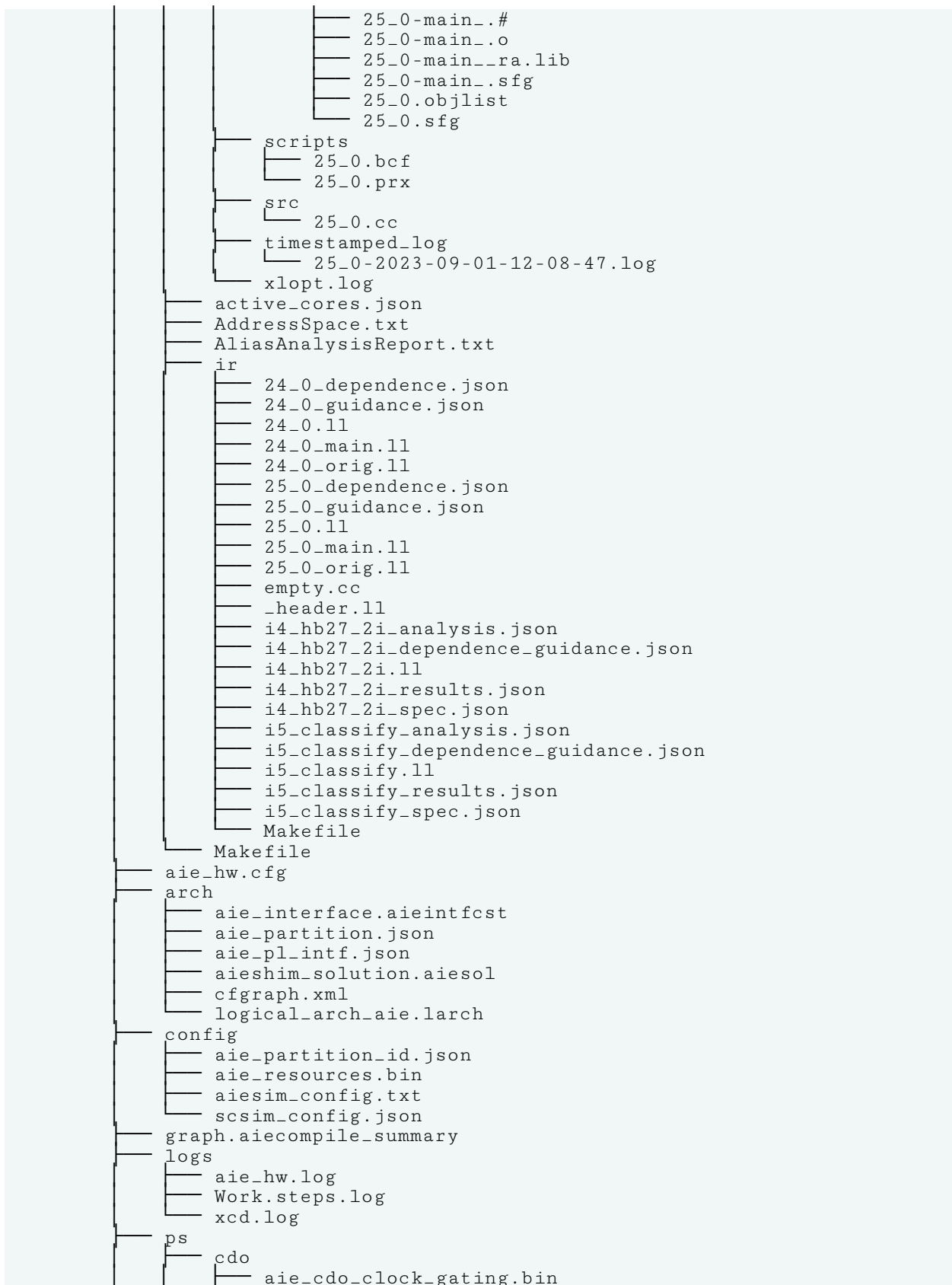
An example of the AI Engine component directory output of the `v++` command follows:

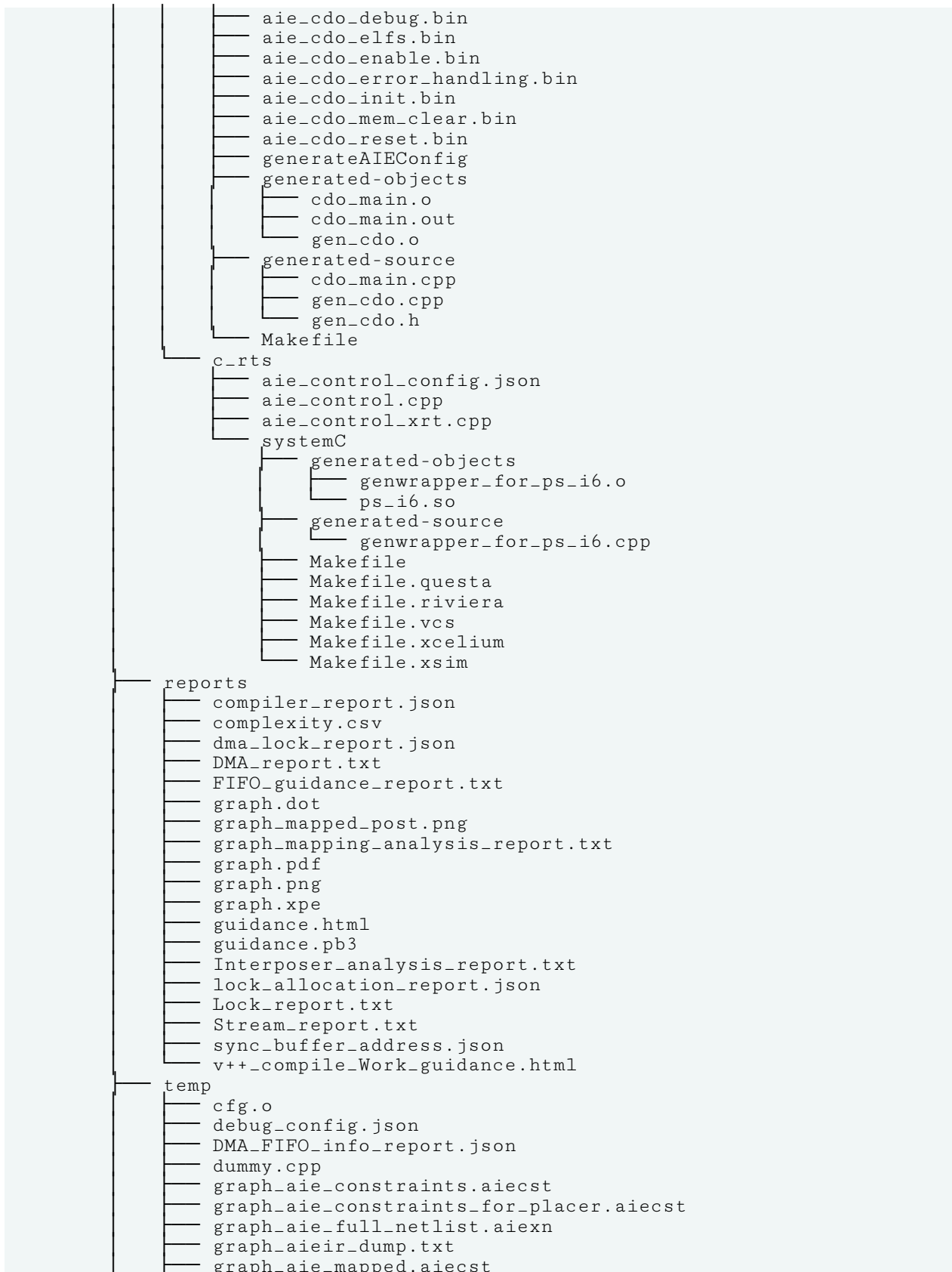
```

newAIE
├── hw
│   ├── Work
│   │   ├── aie
│   │   │   ├── 24_0
│   │   │   │   ├── 24_0.log
│   │   │   │   ├── Release
│   │   │   │   ├── 24_0
│   │   │   │   ├── 24_0.#
│   │   │   │   ├── 24_0.##
│   │   │   │   ├── 24_0.calltree
│   │   │   │   ├── 24_0.cmic2
│   │   │   │   ├── 24_0.cmico
│   │   │   │   ├── 24_0.lst
│   │   │   │   ├── 24_0.map
│   │   │   │   ├── 24_0.o
│   │   │   │   ├── 24_0.o.lst
│   │   │   │   ├── 24_0.sdr
│   │   │   │   └── 24_0.srv

```







```

graph_aie_mapped.aiesol
graph_aie_netlist.aiexn
graph_aie_partitioned.aiesol
graph_aie_premapped.aiesol
graph_aie_prerouted.aiesol
graph_aie_routed.aiecst
graph_aie_routed.aiesol
graph.ii
graph.json
graph_link_design.tcl
graph_mapped.json
graph_mapped_post.dot
graph.out
graph_partition.json
graph_processed.ii
hw.o
log4cfg
router.input
router_soln.dot
router_soln.json
router_soln.pdf
router_soln.png
sw.o
Work.spr

```

Output Directories of `v++ -c --mode hls`

The directory structure generated by the `v++ -c --mode hls` command has been organized to let you easily find and access files from the project.

The example directory provided below results from the following command lines:

```
v++ -c --mode hls --config ./workDCT/dct/hls_config.cfg --work_dir newDCT
```

Where `hls_config.cfg` file defines the following options:

```

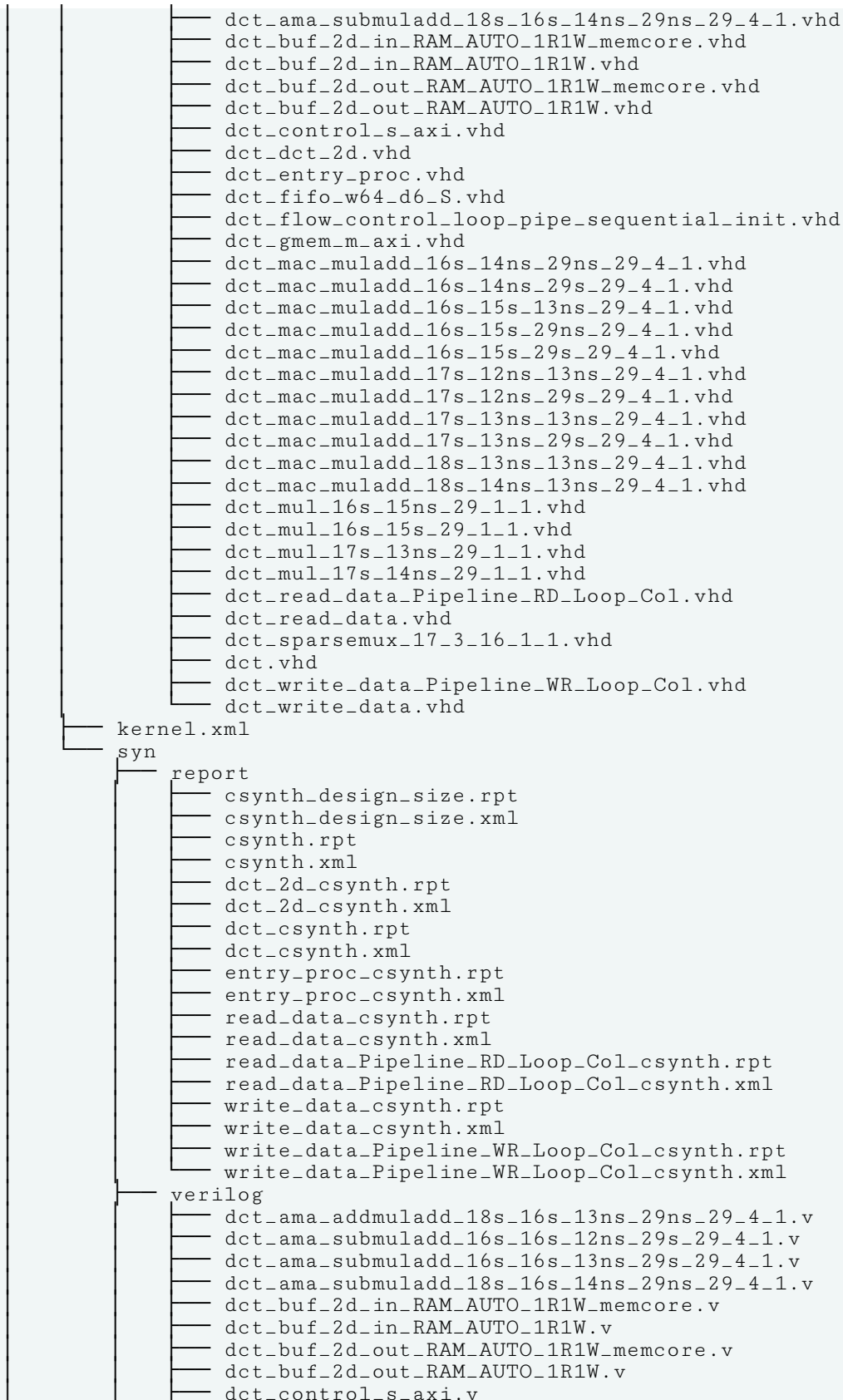
part=xczu7ev-ffvc1156-2-e

[hls]
syn.file=/reference-files/src/dct.cpp
tb.file=/reference-files/src/out.golden.dat
tb.file=/reference-files/src/in.dat
tb.file=/reference-files/src/dct_test.cpp
tb.file=/reference-files/src/dct_coeff_table.txt
syn.top=dct
clock=8ns
clock_uncertainty=12%
csim.code_analyzer=1
csim.clean=true
flow_target=vitis
package.output.format=xo
syn.directive.pipeline=dct_2d II=4
package.output.syn=1
syn.compile.pipeline_loops=6

```

An example of the `temp_dir` output of the `v++ link` command follows:

```
newDCT
├── hls
│   ├── config.cmdline
│   ├── hls_data.json
│   └── impl
│       ├── misc
│       │   ├── drivers
│       │   │   └── dct_v1_0
│       │   │       ├── data
│       │   │       │   ├── dct.mdd
│       │   │       │   ├── dct.tcl
│       │   │       │   └── dct.yaml
│       │   │       └── src
│       │   │           ├── CMakeLists.txt
│       │   │           ├── Makefile
│       │   │           ├── xdct.c
│       │   │           ├── xdct.h
│       │   │           ├── xdct_hw.h
│       │   │           ├── xdct_linux.c
│       │   │           └── xdct_sinit.c
│       │   └── logo.png
│       └── verilog
│           ├── dct_ama_addmuladd_18s_16s_13ns_29ns_29_4_1.v
│           ├── dct_ama_submuladd_16s_16s_12ns_29s_29_4_1.v
│           ├── dct_ama_submuladd_16s_16s_13ns_29s_29_4_1.v
│           ├── dct_ama_submuladd_18s_16s_14ns_29ns_29_4_1.v
│           ├── dct_buf_2d_in_RAM_AUTO_1R1W_memcore.v
│           ├── dct_buf_2d_in_RAM_AUTO_1R1W.v
│           ├── dct_buf_2d_out_RAM_AUTO_1R1W_memcore.v
│           ├── dct_buf_2d_out_RAM_AUTO_1R1W.v
│           ├── dct_control_s_axi.v
│           ├── dct_dct_2d.v
│           ├── dct_entry_proc.v
│           ├── dct_fifo_w64_d6_S.v
│           ├── dct_flow_control_loop_pipe_sequential_init.v
│           ├── dct_gmem_m_axi.v
│           ├── dct_mac_muladd_16s_14ns_29ns_29_4_1.v
│           ├── dct_mac_muladd_16s_14ns_29s_29_4_1.v
│           ├── dct_mac_muladd_16s_15s_13ns_29_4_1.v
│           ├── dct_mac_muladd_16s_15s_29ns_29_4_1.v
│           ├── dct_mac_muladd_16s_15s_29s_29_4_1.v
│           ├── dct_mac_muladd_17s_12ns_13ns_29_4_1.v
│           ├── dct_mac_muladd_17s_12ns_29s_29_4_1.v
│           ├── dct_mac_muladd_17s_13ns_13ns_29_4_1.v
│           ├── dct_mac_muladd_17s_13ns_29s_29_4_1.v
│           ├── dct_mac_muladd_18s_13ns_13ns_29_4_1.v
│           ├── dct_mac_muladd_18s_14ns_13ns_29_4_1.v
│           ├── dct_mul_16s_15ns_29_1_1.v
│           ├── dct_mul_16s_15s_29_1_1.v
│           ├── dct_mul_17s_13ns_29_1_1.v
│           ├── dct_mul_17s_14ns_29_1_1.v
│           ├── dct_read_data_Pipeline_RD_Loop_Col.v
│           ├── dct_read_data.v
│           ├── dct_sparsemux_17_3_16_1_1.v
│           ├── dct.v
│           ├── dct_write_data_Pipeline_WR_Loop_Col.v
│           └── dct_write_data.v
└── vhdl
    ├── dct_ama_addmuladd_18s_16s_13ns_29ns_29_4_1.vhd
    ├── dct_ama_submuladd_16s_16s_12ns_29s_29_4_1.vhd
    └── dct_ama_submuladd_16s_16s_13ns_29s_29_4_1.vhd
```



```

dct_dct_2d.v
dct_entry_proc.v
dct_fifo_w64_d6_S.v
dct_flow_control_loop_pipe_sequential_init.v
dct_gmem_m_axi.v
dct_mac_muladd_16s_14ns_29ns_29_4_1.v
dct_mac_muladd_16s_14ns_29s_29_4_1.v
dct_mac_muladd_16s_15s_13ns_29_4_1.v
dct_mac_muladd_16s_15s_29ns_29_4_1.v
dct_mac_muladd_16s_15s_29s_29_4_1.v
dct_mac_muladd_17s_12ns_13ns_29_4_1.v
dct_mac_muladd_17s_12ns_29s_29_4_1.v
dct_mac_muladd_17s_13ns_13ns_29_4_1.v
dct_mac_muladd_17s_13ns_29s_29_4_1.v
dct_mac_muladd_18s_13ns_13ns_29_4_1.v
dct_mac_muladd_18s_14ns_13ns_29_4_1.v
dct_mul_16s_15ns_29_1_1.v
dct_mul_16s_15s_29_1_1.v
dct_mul_17s_13ns_29_1_1.v
dct_mul_17s_14ns_29_1_1.v
dct_read_data_Pipeline_RD_Loop_Col.v
dct_read_data.v
dct_sparsemux_17_3_16_1_1.v
dct.v
dct_write_data_Pipeline_WR_Loop_Col.v
dct_write_data.v
vhdl
dct_ama_addmuladd_18s_16s_13ns_29ns_29_4_1.vhd
dct_ama_submuladd_16s_16s_12ns_29s_29_4_1.vhd
dct_ama_submuladd_16s_16s_13ns_29s_29_4_1.vhd
dct_ama_submuladd_18s_16s_14ns_29ns_29_4_1.vhd
dct_buf_2d_in_RAM_AUTO_1R1W_memcore.vhd
dct_buf_2d_in_RAM_AUTO_1R1W.vhd
dct_buf_2d_out_RAM_AUTO_1R1W_memcore.vhd
dct_buf_2d_out_RAM_AUTO_1R1W.vhd
dct_control_s_axi.vhd
dct_dct_2d.vhd
dct_entry_proc.vhd
dct_fifo_w64_d6_S.vhd
dct_flow_control_loop_pipe_sequential_init.vhd
dct_gmem_m_axi.vhd
dct_mac_muladd_16s_14ns_29ns_29_4_1.vhd
dct_mac_muladd_16s_14ns_29s_29_4_1.vhd
dct_mac_muladd_16s_15s_13ns_29_4_1.vhd
dct_mac_muladd_16s_15s_29ns_29_4_1.vhd
dct_mac_muladd_16s_15s_29s_29_4_1.vhd
dct_mac_muladd_17s_12ns_13ns_29_4_1.vhd
dct_mac_muladd_17s_12ns_29s_29_4_1.vhd
dct_mac_muladd_17s_13ns_13ns_29_4_1.vhd
dct_mac_muladd_17s_13ns_29s_29_4_1.vhd
dct_mac_muladd_18s_13ns_13ns_29_4_1.vhd
dct_mac_muladd_18s_14ns_13ns_29_4_1.vhd
dct_mul_16s_15ns_29_1_1.vhd
dct_mul_16s_15s_29_1_1.vhd
dct_mul_17s_13ns_29_1_1.vhd
dct_mul_17s_14ns_29_1_1.vhd
dct_read_data_Pipeline_RD_Loop_Col.vhd
dct_read_data.vhd
dct_sparsemux_17_3_16_1_1.vhd
dct.vhd
dct_write_data_Pipeline_WR_Loop_Col.vhd
dct_write_data.vhd
logs

```

```

├── hls_compile.log
├── newDCT.steps.log
├── xcd.log
├── newDCT.hlscompile_summary
├── reports
│   ├── hls_compile.rpt
│   └── v++_compile_newDCT_guidance.html
└── vitis-comp.json

```

Output Directories of the Vitis Unified IDE

Unlike the command-line flow, which is defined largely by the user through command or Makefile, the AMD Vitis™ IDE defines the structure of the projects and output directories in a system design project. In the Vitis IDE, an Application project for acceleration or heterogeneous design can have four different projects associated with it. The projects include:

- **Top-level System project:** This is the project used to build the integrated system design, and is where you will find the consolidated contents of the packaged system design which is the result of the `v++ --package` process
- **Software Application project:** This contains the source and compiled results of the software application which runs on an x86 or Arm processor
- **PL Kernels project:** This project contains the source files for one or more PL kernels used by the system, and the results of the `v++ --compile` command
- **Hardware Linking project:** This project contains the linked system design which is the results of the `v++ --link` command



TIP: Each of the projects below starts with the *Emulation-HW* folder which is where the Vitis IDE places the build content for the hardware emulation build. The files presented here are for that build target. Some of the files at the deeper levels are not displayed, but can be found by navigating into the hierarchy of completed builds.

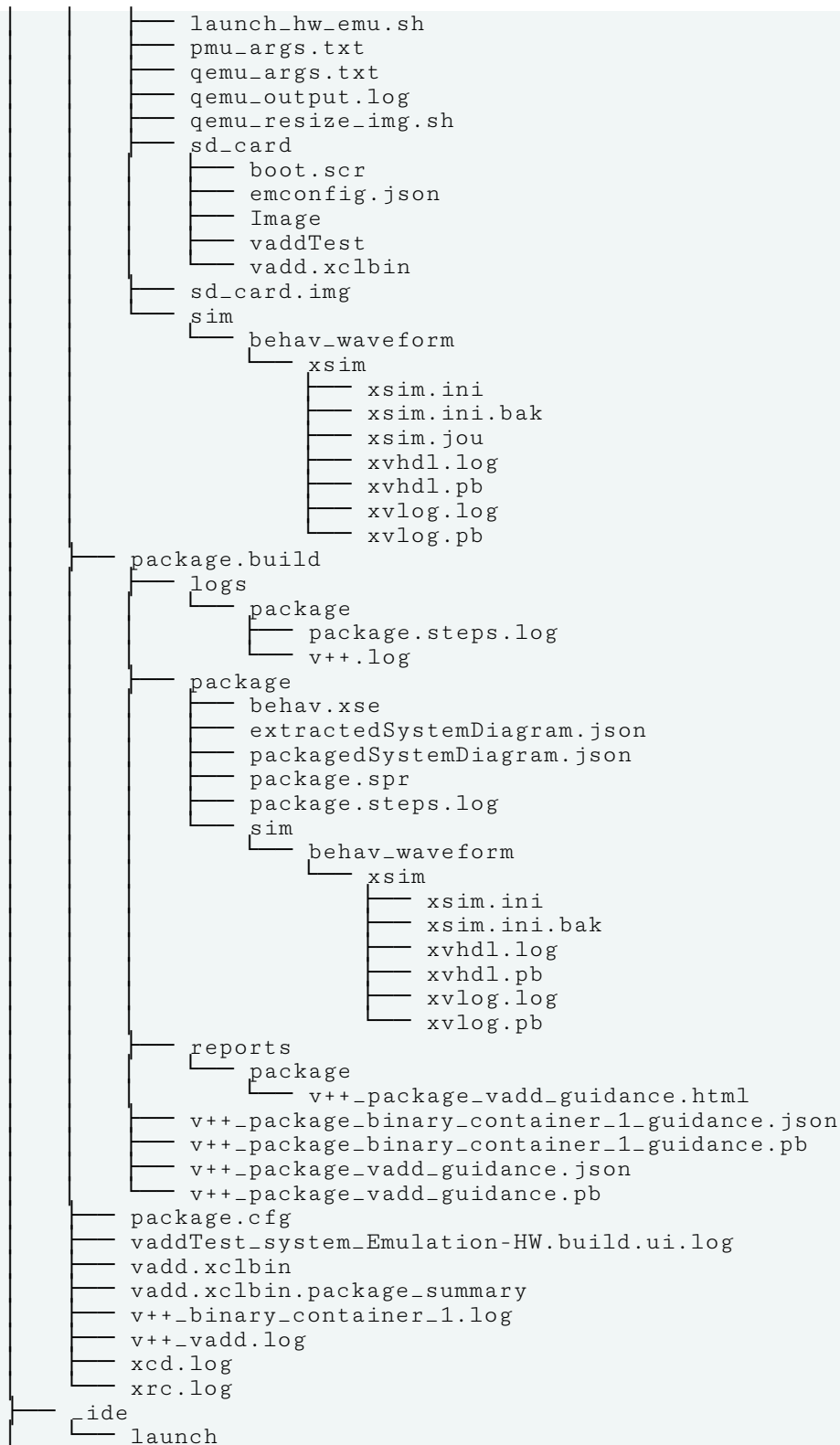
Top-Level System Project

The top-level system project contains all of the other projects as sub-projects. This is where the definition and construction of the system design are provided. This includes the output of the package process, which generates the final fixed platform and device binary, and packages it into an SD card or QSPI image.

```

├── Emulation-HW
│   ├── binary_container_1.xclbin
│   ├── binary_container_1.xclbin.package_summary
│   ├── emulation.pid
│   ├── makefile
│   └── package
│       ├── emu_qemu_scripts
│       │   ├── start_pmc.sh
│       │   └── start_qemu.sh

```




```

├── SystemDebugger_vaddTest_system_1.launch
├── SystemDebugger_vaddTest_system.launch
├── vadd-system.txt
└── vaddTest_system.sprj

```

Software Application Project

The software application project contains the source code for the software application, and the resulting `.o` object files, and `.exe` or `.elf` executable files.

```

├── Emulation-HW
│   ├── emconfig.json
│   ├── guidance.html
│   ├── guidance.json
│   ├── guidance.pb
│   ├── makeemconfig.mk
│   ├── makefile
│   └── src
│       ├── host.o
│       ├── SystemDebugger_vaddTest_system_vaddTest
│       ├── device_trace_0.csv
│       ├── native_trace.csv
│       ├── summary.csv
│       ├── xrt.ini
│       └── xrt.run-summary
│   ├── SystemDebugger_vaddTest_system_vaddTest.launch.log
│   ├── vaddTest
│   ├── vaddTest_Emulation-HW.build.ui.log
│   └── xsa.xml
├── src
│   ├── host.cpp
│   ├── vadd-processor.txt
│   └── vaddTest.prj

```

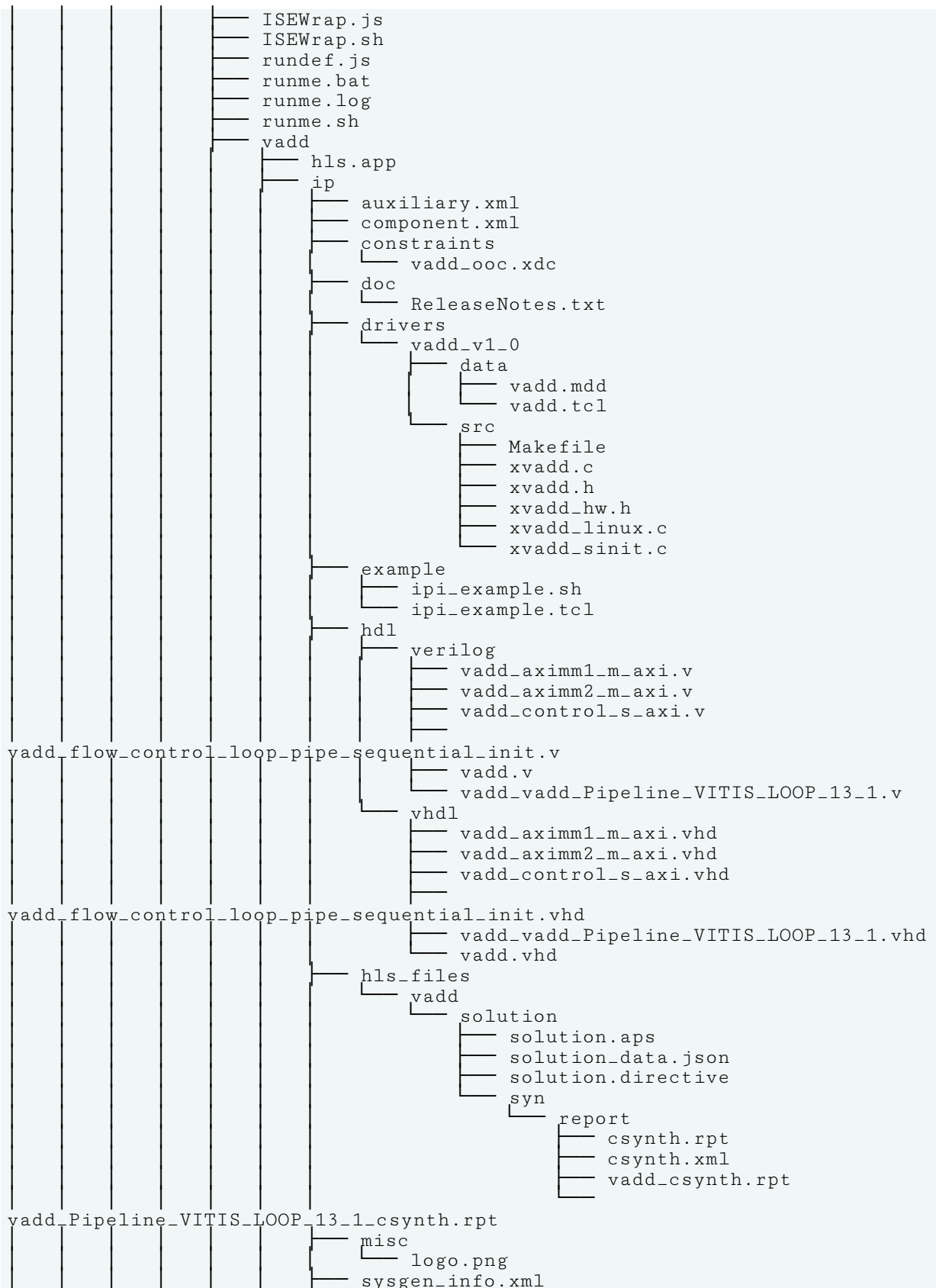
PL Kernels Project

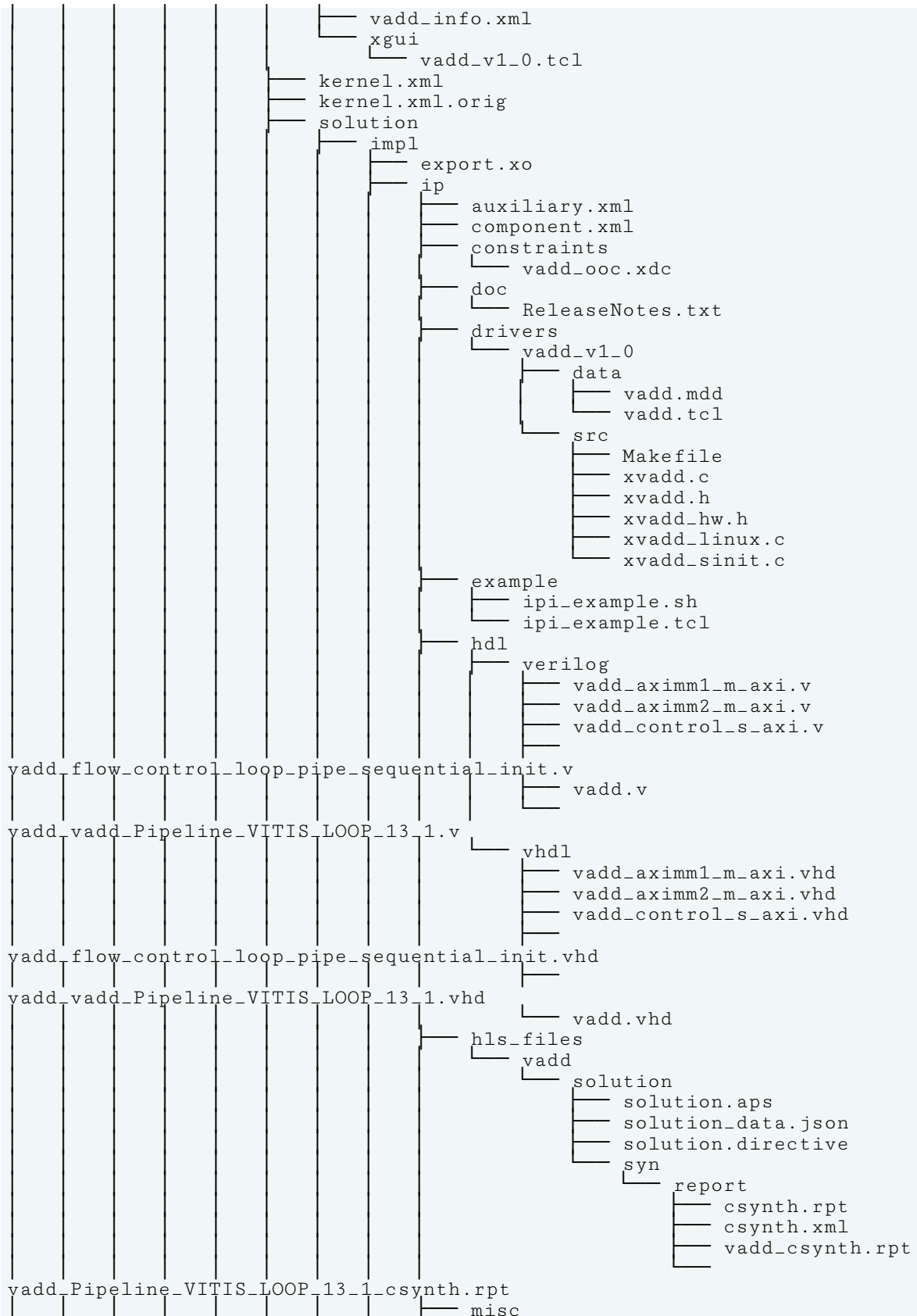
The PL kernels project contains the source code for C++ kernels and the compiled Xilinx object files (`.xo`), as well as RTL kernels (`.xo`), and `libadf.a` files containing AI Engine graph applications. The directory also holds the resulting logs and projects required by Vitis HLS to build the PL kernel objects, such as the `.compile_summary` produced during compilation.

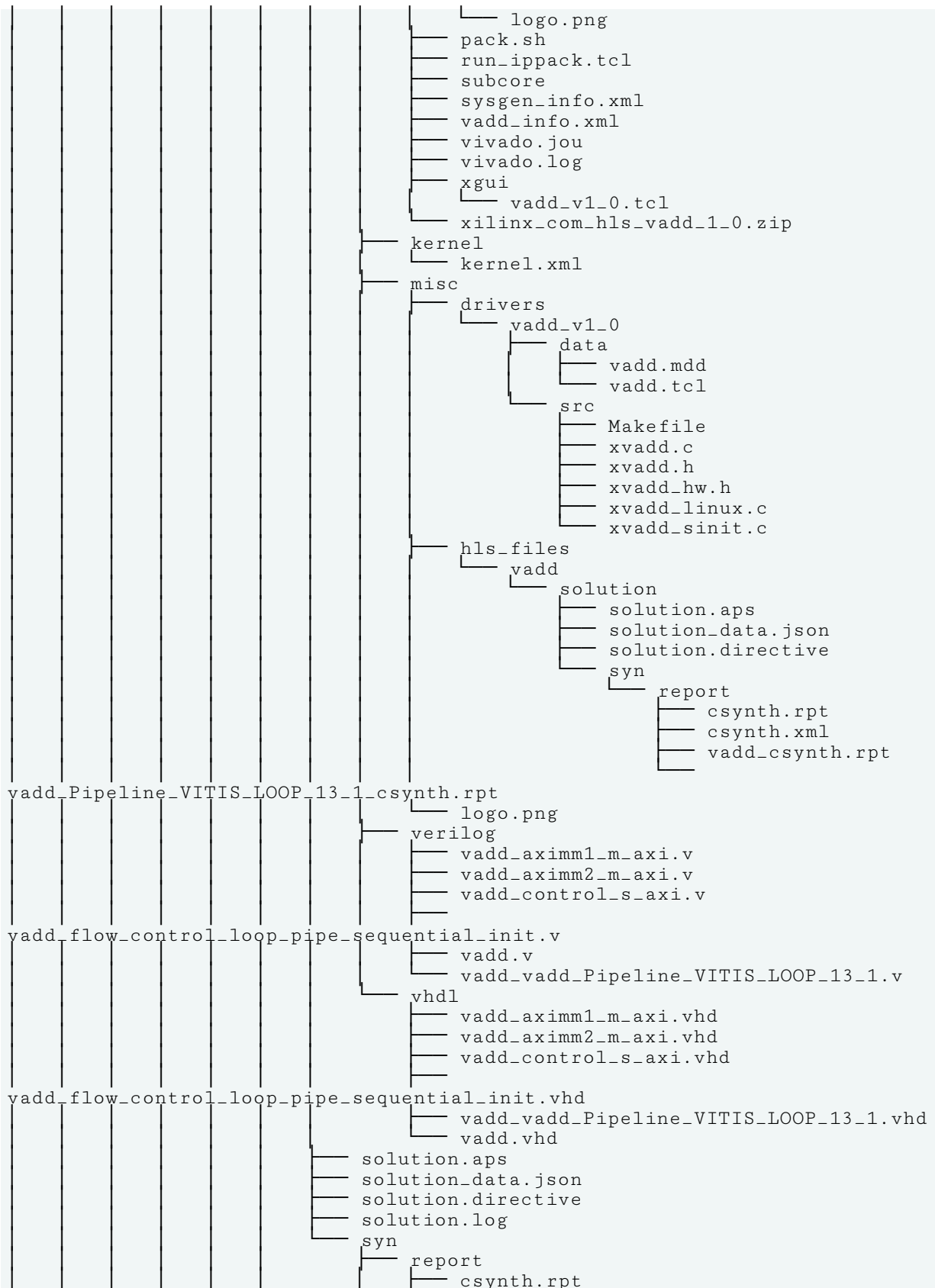
```

├── Emulation-HW
│   ├── build
│   │   ├── logs
│   │   │   ├── vadd
│   │   │   │   ├── vadd.steps.log
│   │   │   │   ├── vadd_vitis_hls.log
│   │   │   │   ├── v++.log
│   │   │   └── v++_vadd.log
│   │   ├── reports
│   │   │   ├── vadd
│   │   │   │   ├── hls_reports
│   │   │   │   └── vadd_csynth.rpt
│   │   └── system_estimate_vadd.xtext
│   └── vadd
│       └── vadd
│           └── htr.txt

```







```

csynth.xml
vadd_csynth.rpt
vadd_csynth.xml
vadd_Pipeline_VITIS_LOOP_13_1_csynth.rpt
vadd_Pipeline_VITIS_LOOP_13_1_csynth.xml
verilog
vadd_aximm1_m_axi.v
vadd_aximm2_m_axi.v
vadd_control_s_axi.v
vadd_flow_control_loop_pipe_sequential_init.v
vadd.v
vadd_vadd_Pipeline_VITIS_LOOP_13_1.v
vhdl
vadd_aximm1_m_axi.vhd
vadd_aximm2_m_axi.vhd
vadd_control_s_axi.vhd
vadd_flow_control_loop_pipe_sequential_init.vhd
vadd_vadd_Pipeline_VITIS_LOOP_13_1.vhd
vadd.vhd
vadd.design.xml
vadd.tcl
vitis_hls.log
vitis_hls.pb
vadd.spr
vadd.steps.log
vadd.mdb
vadd.xo
vadd.xo.compile_summary
guidance.html
guidance.html.bak
guidance.json
guidance.pb
guidance.pb.bak
makefile
vadd-compile.cfg
vaddTest_kernels_Emulation-HW.build.ui.log
xcd.log
src
vadd.cpp
tree-kernel.txt
vaddTest_kernels.prj

```

Hardware Link Project

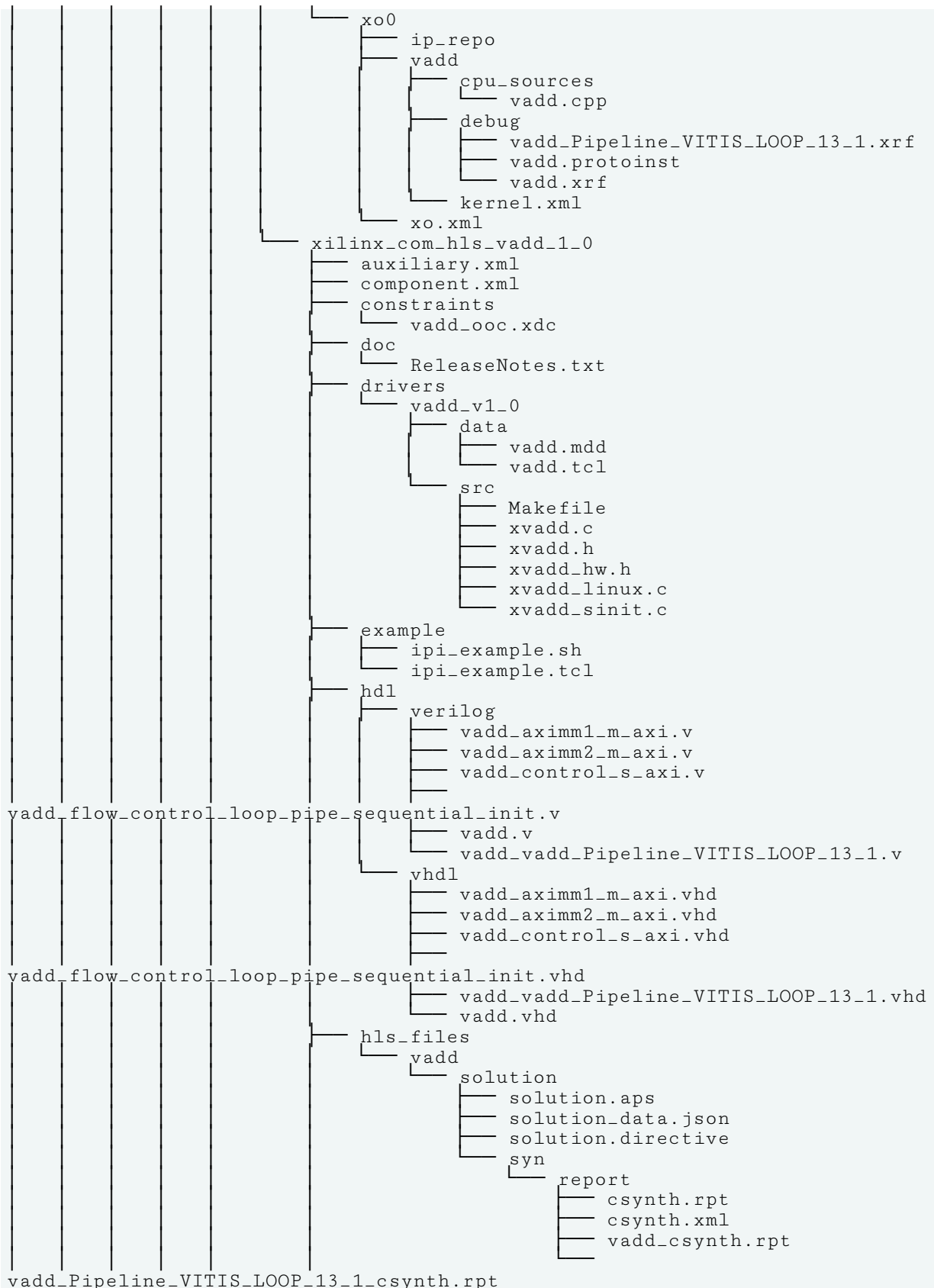
The hardware link project contains the linked fixed hardware platform (.xsa) and the device binary (.xclbin) used to program the AMD device. The directory also holds the resulting logs and projects produced during the linking process, such as the .link_summary that can be viewed in the Vitis analyzer.

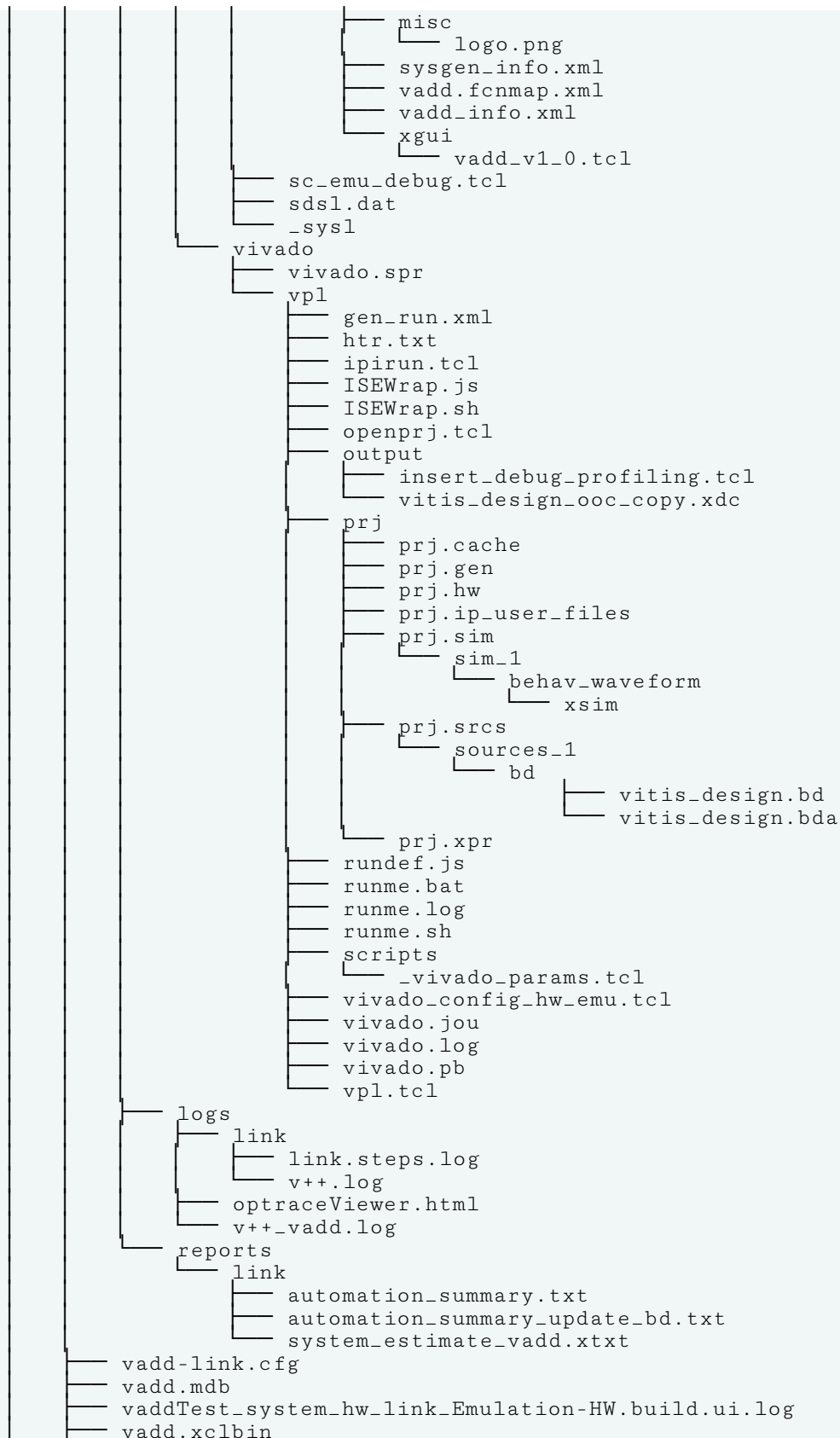
```

Emulation-HW
binary_container_1-link.cfg
guidance.html
guidance.html.bak
guidance.json
guidance.pb
guidance.pb.bak
makefile

```









```
├── vadd.xclbin.info
├── vadd.xclbin.link_summary
├── vadd.xclbin.sh
├── xcd.log
├── vadd-hw-link.txt
└── vaddTest_system_hw_link.prj
```

Python XSDB Commands

This section describes the Python XSDB commands. You can use these commands to implement the functions supported by XSDB. After the list of commands is a section with several usage examples.

Breakpoints

The following are the commands followed by their options (if any) and the Python method for the option (if any).

bpadd

- **-addr <breakpoint-address> :**

```
t = self.session.targets(id=2)
b1 = t.bpadd(addr='main')
```

- **-file <file-name>:**

```
t.bpadd('helloworld.c:90', type='hw', mode=3)
t.bpadd(file='helloworld.c', line=89)
```

- **-line <line-number> :**

```
t.bpadd(file='helloworld.c', line=89)
```

- **-type <breakpoint-type>:**

```
t.bpadd(addr='main', type='hw', mode=2)
```

- **-mode <breakpoint-mode> :**

```
t.bpadd(addr='main', type='hw', mode=2)
```

- **-enable <mode> :**

```
t = self.session.targets(id=2)
b1 = t.bpadd(addr='main', enable=1)
```

- **-ct-input <list> -ct-output <list>:**

```
t.b padd(ct_input=0, ct_output=8, skip_on_step=1)
```

- **-skip-on-step <value>:**

```
t.b padd(ct_input=0, ct_output=8, skip_on_step=1)
```

- **-target-id <id>:**

```
b1 = t.b padd(addr='main', target_id='all')
```

bpdisable

The following are options for the command followed by their Python method.

- **<id-list>:**

```
t.bpdisable(bp_ids=1)  
t.bpdisable(bp_ids=[1, 2])
```

- **-all:**

```
t.bpdisable('--all')
```

bpenable

- **<id-list>:**

```
t.bpenable(bp_ids=1)  
t.bpenable(bp_ids=[1, 2])
```

- **-all:**

```
t.bpenable('--all')
```

bplist

The following is the command's Python method

```
t.bplist()
```

bpremove

- **<id-list>:**

```
t.bpremove(bp_ids=2)  
t.bpremove(bp_ids=[1, 2])
```

- **-all:**

```
t.bpremove('--all')
```

bpstatus

- **<id>:**

```
t.bpstatus(bp_id=3)
```

Connections

The following are the commands followed by their options (if any) and the Python method for the option (if any).

connect

- **-host <host name/ip> :**

```
s.connect(host="xhdbfarmrk9", port=3121)
```

- **-port <port num>:**

```
s.connect(host="xhdbfarmrk9", port=3121)
```

- **-url <url>:**

```
s.connect("--new", url="TCP:xhdbfarmrk9:3121")
```

- **-list:**

```
s.connect("--list")
```

- **-set <channel-id>:**

```
s.connect(set="tcfchan#0")
```

- **-new:**

```
s.connect("--new", url="TCP:xhdbfarmrk9:3121")
```

disconnect

```
s.disconnect()
```

- **<channel-id>:**

```
s.disconnect(chan="tcfchan#1")
```

gdbremote connect

- **-architecture:**

```
session.gdb_connect('localhost:3121')
```

gdbremote disconnect

- **[target-id]:**

```
session.gdb_disconnect(1)
```

targets

```
s.targets()
```

- **[target-id]:**

```
s.targets(1)  
s.targets(id=1)
```

- **-set:**

```
s.targets("--set", filter="name =~ #1")
```

- **-nocase:**

```
s.targets("--no_case", filter="name =~ ARM*")
```

- **-filter:**

```
s.targets(filter="name =~ ARM*")
```

- **-target-properties:**

```
self.session.targets("--target_properties")
```

- **-timeout:**

Device

The following are the commands followed by their options (if any) and the Python method for the option (if any).

device authjtag

```
self.session.device_authjtag("tests/elfs/versal/vck190.pdi")
```

device program

```
self.session.device_program("tests/elfs/versal/vck190.pdi")
```

device status

```
self.session.device_status()
```

- **-jreg-name <jtag-register-name> :**

```
s.device_status('error_status')
```

fpga

- **-file <bitstream-file> :**

```
tgt.fpga(file='tests/elfs/zynq/zc702.bit')
```

- **--partial:**

```
tgt.fpga('--partial', file='tests/elfs/zynq/zc702.bit')
```

- **--no_revision_check:**

```
tgt.fpga('--no_revision_check', file='tests/elfs/zynq/zc702.bit')
```

- **--skip_compatibility_check:**

```
tgt.fpga('--skip_compatibility_check', file='tests/elfs/zynq/zc702.bit')
```

- **--state:**

```
tgt.fpga('--state')
```

- **--config_status:**

```
tgt.fpga('--config_status')
```

- **--ir_status:**

```
tgt.fpga('--ir_status')
```

- **--boot_status:**

```
tgt.fpga('--boot_status')
```

- **--timer_status:**

```
tgt.fpga('--timer_status')
```

- **--cor0_status:**

```
tgt.fpga('--cor0_status')
```

- **--cor1_status:**

```
tgt.fpga('--cor1_status')
```

- **--wbstart_status:**

```
tgt.fpga('--cor1_status')
```

Download

The following are the commands followed by their options (if any) and the Python method for the option (if any).

dow

```
t.dow('tests/elfs/zynq/core0zynq.elf')
```

- **--data:**

```
t.dow('tests/elfs/zynq/data', '-d', addr=0x10000000)
```

- **--clear:**

```
t.dow('tests/elfs/zynq/core0zynq.elf', '-c', relocate_sections=0x30000000)
```

- **--skip_tcm_clear:**

```
t.dow('--skip_tcm_clear', 'tests/elfs/zynq/core0zynq.elf')
```

```
--keepsym
```

- **--force:**

```
t.dow('tests/elfs/zynq/core0zynq.elf', '-f')
```

- **--bypass_cache_synq:**

```
t.dow('tests/elfs/zynq/core0zynq.elf', '-b')
```

- **relocate_sections:**

```
t.dow('tests/elfs/zynq/core0zynq.elf', '-c', relocate_sections=0x30000000)
```

- **-vaddr:**

```
t.dow('tests/elfs/zynq/core0zynq.elf', '-v')
```

verify

```
t.verify('tests/elfs/zynq/core0zynq.elf')
```

- **-data <file> <addr>:**

```
t.verify('tests/elfs/zynq/core0zynq.elf', '-d', addr=0x10000000)
```

- **-force:**

```
t.verify('tests/elfs/zynq/core0zynq.elf', '-f')
```

- **-vaddr:**

```
t.verify('tests/elfs/zynq/core0zynq.elf', '-v')
```

IPI

The following are the commands followed by their options (if any) and the Python method for the option (if any).

plm

- **copy-debug-log:**

```
s.plm_copy_debug_log(0x0)
```

- **set-debug-log:**

```
s.plm_set_debug_log(0x0, 0x4000)
```

- **set-log-level:**

```
s.plm_set_log_level(4)
```

- **log:**

```
s.plm_log(0x0, 0x4000)
f = open('tests/data/plm.log', 'w')
s.plm_log(0x0, 0x4000, handle=f)
```


pmc

```
s.pmc('generic', [0x10241, 0x1c000000], response_size=1)
s.pmc(cmd='generic', data=[0x1030115, 0xffff0000, 0x100], response_size=2)
```

- **-ipi:**

```
s.pmc(cmd='generic', data=[0x1030115, 0xffff0000, 0x100], ipi=5,
response_size=2)
```

JTAG

The following are the commands followed by their options (if any) and the Python method for the option (if any).

jtag claim

The following is the Python method.

```
jt.claim()
```

jtag device_properties

- **<idcode>:**

```
jt.device_properties(0x6ba00477)
```

- **<key> <value> pairs:**

```
props = {'idcode': 0x6ba00477, 'irlen':
4}jt.device_properties(props=props)
DONE
```

jtag disclaim

```
jt.disclaim()
```

jtag frequency

```
jt.frequency()
```

- **-list:**

```
jt.frequency('-l')
```

- **<frequency>:**

```
jt.frequency(freq)
```

jtag lock

```
jt.lock(100)
```

jtag sequence

- **state:**

```
s = self.session
jt3 = s.jtag_targets(3)
jseq = jt3.sequence()
jseq.state("RESET")
```

- **irshift:**

```
jseq.irshift(register='bypass', state="IRUPDATE")
```

- **drshift:**

```
jseq.drshift(capture=True, state="IDLE", tdi=0, bit_len=2)
```

- **delay:**

```
jseq.delay(100)
```

- **get_pin:**

```
jseq.get_pin('TDI')
```

- **set_pin:**

```
jseq.set_pin('TDI', 0)
```

- **atomic:**

```
jseq.atomic()
```

- **run:**

```
jseq.run()
```

- **clear:**

```
jseq.clear()
```

- **delete:**

```
del jseq
```

jtag servers

```
jt.servers()
```

- **-list:**

```
jt.servers('-l')
```

- **-format:**

```
jt.servers('-f')
```

- **-open <server>:**

```
jt.servers(open='xilinx-xvc:localhost:10200')
```

- **-close <server>:**

```
jt.servers(close='xilinx-xvc:localhost:10200')
```

jtag skew

```
jt.skew()
```

- **<clock-skew>:**

```
jt.skew()
```

jtag targets

```
self.session.jtag_targets()
```

- **[target-id]:**

```
self.session.jtag_targets(id=2)
```

- **-set:**

```
self.session.jtag_targets('-s', filter="name == xcvc1902")
```

- **-regexp:**

- **-nocase:**

```
self.session.jtag_targets('-n', filter="name !~ XCVC*")
```

- **-filter <filter-expression>:**

```
self.session.jtag_targets(filter="name == xcvc1902")
```

- **-target-properties:**

```
self.session.jtag_targets('-t')
```

- **-open:**

```
self.session.jtag_targets('-o')
```

- **-close:**

```
self.session.jtag_targets('-c')
```

- **-timeout <sec>:** No method available

jtag unlock

```
jt.unlock()
```

Memory

The following are the commands followed by their options (if any) and the Python method for the option (if any).

init_ps

```
init_data = ["mask_write 0 0x00001FFF 0x00000001",
             "mask_delay 1",
             "mask_poll 4 1 1"]
t.init_ps(init_data)
```

mask_poll

```
t.mask_poll(4,1,1)
```

mask_write

```
t.mask_write(0, 0xFF, 0xFFFF)
```

memmap

- **addr <memory-address>:**

```
t.memmap(addr=0xFC000000, size=0x1000, flags=3)
```

- **alignment <bytes>:**

```
t.memmap(addr=0xFC000000, size=0x1000, flags=3, alignment=4)
```

- **size <memory-size>:**

```
t.memmap(addr=0xFC000000, size=0x1000, flags=3)
```

- **flags <protection-flags>:**

```
t.memmap(addr=0xFC000000, size=0x1000, flags=3)
```

- **--list:**

```
t.memmap('-l')
```

- **--clear:**

```
t.memmap('-c', addr=0xFC000000)
```

- **relocate_sections <addr>:**

```
t.memmap(file='tests/elfs/zynq/core0zynq.elf', ,  
relocate_sections=0x20000000)
```

- **-osa:**

```
t.memmap(addr=0xFC000000, size=0x1000, flags=3, OSA=1)
```

mrd

```
t.mrd(0x00100000, word_size=4, size=10)
```

- **--force:**

```
t.mrd(0x00100002, '-f', word_size=4, size=10)
```

- **size <access-size>:**

```
t.mrd(0x00100000, word_size=4, size=10)
```

- **--value:**

```
x = t.mrd(0x100000, '-v', word_size=4, size=20)
```

- **--bin:**

```
t.mrd(0x00100000, '-b', word_size=8, size=20, file="tests/data/mrd.bin")
```

- **file <file-name>:**

```
t.mrd(0x00100000, '-b', word_size=8, size=20, file="tests/data/mrd.bin")
```

- **-address-space <name>:**

```
t.mrd(0x100, address_space="APR")
t.mrd(0x80090088, address_space="AP1")
```

- **-unaligned-access:**

```
t.mrd(0x00100000, '-u', word_size=4, size=10)
```

mwr

```
t.mwr(0x00100000, word_size=4, size=10, words=[0x01, 0x02, 0x03])
```

- **--force:**

```
t.mwr(0x00100000, '-f', word_size=4, size=10, words=[0x01, 0x02, 0x03])
```

- **--bypass_cache_synq:**

```
t.mwr(0x00100000, '-v', word_size=4, size=10, words=[0x01, 0x02, 0x03])
```

- **size <access-size>:**

```
t.mwr(0x00100000, word_size=4, size=10, words=[0x01, 0x02, 0x03])
```

- **--bin:**

```
t.mwr(0x00100000, '-b', word_size=4, size = 5, file="tests/data/mrd.bin")
```

- **file<file-name>:**

```
t.mwr(0x00100000, '-b', word_size=4, size = 5, file="tests/data/mrd.bin")
```

- **-address-space <name>:**

```
t.mwr(0x80090088, address_space="AP1", words=[0x03186004])
```

- **-unaligned-access:**

```
t.mwr(0x00100000, '-u', word_size=4, size=10, words=[0x01, 0x02, 0x03])
```

osa

```
t.osa('--disable', file='tests/elfs/zynq/core0zynq.elf')
```

- **--disable:**

```
t.osa('--disable', file='tests/elfs/zynq/core0zynq.elf')
```

- **--fast_exec:**

```
t.osa('--fast_exec', file='tests/elfs/zynq/core0zynq.elf')
```

- **--fast_step:**

```
t.osa('--fast_step', file='tests/elfs/zynq/core0zynq.elf')
```

Miscellaneous

The following are the commands followed by their options (if any) and the Python method for the option (if any).

configparams

- **<none>:**

```
s.configparams()
```

- **<parameter>:**

```
s.configparams("silent-mode")
```

- **<parameter> <value>:**

```
s.configparams("silent-mode", 1)
```

- **-all:**

```
s.configparams("--all")
```

loadhw

```
-hw
```

```
-list
```

```
-mem-ranges  
[list {start1 end1} {start2 end2}]
```

loadipxact`<xml-file>``-clear``-list`**mb_drrd**

- **<cmd> <bitlen>:**

`s.mb_drrd(3, 28)`

- **-target-id<id>:** No method available

- **-user <bscan number>:**

`s.mb_drrd(0x07, 288, user=bscan, which=which, target_id=target_id)`

- **-which<instance>:** No method available

mb_drwr

- **<cmd> <data> <bitlen>:**

`s.mb_drwr(1, 0x282, 10)`

- **-target-id <id>:** No method available

- **-user <bscan number>:**

`s.mb_drwr(0x71, value, 8, user=bscan, which=which)`

- **-which <instance>:** No method available

mdm_drrd

- **<cmd> <bitlen>:**

`s.mdm_drrd(0, 32)`

- **-target-id <id>:** No method available

- **-user <bscan number>:** No method available

mdm_drwr

- **<cmd> <data> <bitlen>:**

`s.mdm_drwr(8, 0x40, 8)`

- **-target-id <id>:** No method available
- **-user <bscan number>:** No method available

version

```
s.version('-s')
```

xsdbserver disconnect

```
s.xsdbserver_disconnect()
```

xsdbserver start

```
s.xsdbserver_start()
```

- **-host <addr>:** No method available
- **-port <port>:** No method available

xsdbserver stop

```
s.xsdbserver_stop()
```

xsdbserver version

```
s.xsdbserver_version()
```

Registers

rrd

The following are the commands followed by their options (if any) and the Python method for the option (if any).

- **<none>:**

```
s.targets(2)
s.rrd()
```

- **<register group/name>:**

```
ta = s.targets(2)
# read using session object
s.rrd("usr")
s.rrd("usr r8")# read using target object
ta.rrd("mpcore icdipr ipr95")
```

- **-defs:**

```
s.targets(2)
s.rrd("--defs")
s.rrd("usr", "--defs")
```

- **-no-bits:**

```
s.targets(2)
s.rrd("cpsr", "--no_bits")
```

rwr

- **<register><value>:**

```
s.targets(2)
s.rwr('r0', 0x5555aaaa)
s.rwr('cpsr m', 0x13)
```

Reset

rst

The following are the commands followed by their options (if any) and the Python method for the option (if any).

```
s.rst('--stop', endianness='le', type='cores')
```

- **-processor:** No method available
- **-cores:** No method available
- **-dap:** No method available
- **system:** No method available
- **-srst:** No method available
- **-por:** No method available
- **-ps:** No method available
- **-stop:** No method available
- **-start:**

```
s.rst('--start', type='cores')
```

- **-endianness <value>:**

```
s.rst('--stop', endianness='le', type='cores')
```

- **-code-endianness <value>:**

```
s.rst('--stop', code-endianness='le', type='cores')
```

- **-isa <isa-name>:**

```
s.rst('--stop', isa='ARM', type='cores')
```

- **-clear-registers:** No method available

- **-type <reset type>:**

```
s.rst(type='system')
```

Running

The following are the commands followed by their options (if any) and the Python method for the option (if any).

backtrace, bt

```
# Using session object
s.backtrace()
# Using target object
t.backtrace()
```

- **-maxframes <num>:**

```
# Using session object
s.backtrace(5)
# Using target object
t.backtrace(5)
```

con

```
# Using session
objects.con()
# Using target
objectt.con()
```

- **-addr <address>:** No method available
- **-block:** No method available
- **-timeout <sec>:** No method available

dis

```
t.dis(0x00100000)
```

- **<address>:**

```
t.dis(0x00100000)
```

- **<address> <num>:**

```
t.dis(0x00100000, 5)
```

locals

```
s.locals()
```

- **[variable-name]:**

```
s.locals(name="test")  
# or  
s.locals("test")
```

- **[variable-name [variable-value]]:**

```
s.locals(name="test", value=5)  
# or  
s.locals("test", 7)
```

- **-defs:**

```
s.locals("--defs")  
s.locals("--defs", name="val")
```

- **-dict:**

```
s.locals("--dict")  
s.locals("--dict", name="val")
```

mbprofile

- **-low <addr>:**

```
s.mbprofile(low='low', high='high')
```

- **-high <addr>:** No method available

- **-freq <value>:**

```
s.mbprofile('--count_instr', low='low', high='high')
```

- **-count_instr:** No method available
- **-cumulate:** No method available
- **-start:**

```
s.mbprofile('--start')
```

- **-stop:**

```
s.mbprofile('--stop', out="tests/elfs/mbprofile/gmon_inst_1.out")
```

- **-out <filename>:**

```
s.mbprofile('--stop', out="tests/elfs/mbprofile/gmon_inst_1.out")
```

mbtrace

- **-start:**

```
s.mbtrace('--start')
```

- **-stop:**

```
s.mbtrace('-f', '--stop', out="tests/elfs/mbprofile/gmon_trace.txt")
```

- **-con:**

```
s.mbtrace('-f', '--con', out="tests/elfs/mbprofile/gmon_trace.txt")
```

- **-stp:** No method available
- **-nxt:** No method available
- **-out <filename>:**

```
s.mbtrace('-f', '--con', out="tests/elfs/mbprofile/gmon_trace.txt")
```

- **-level <level>:** No method available
- **-halt:** No method available
- **-save:** No method available
- **-low <addr>:** No method available
- **-high <addr>:** No method available

- **-format <format>:** No method available

nxt

```
# Using session objects.nxt()  
# Using target objectt.nxt()
```

- **[count]:**

```
# Using session objects.nxt(2)  
# Using target objectt.nxt(2)
```

nxti

```
# Using session object  
s.nxti()  
# Using target object  
t.nxti()
```

- **[count]:**

```
# Using session object  
s.nxti(2)  
# Using target object  
t.nxti(2)
```

print

- **[expression]:**

```
s.print()  
# or  
s.print(expr="temp")  
# or  
s.print("temp")  
# or  
s.print("x + y - z")
```

- **[expression] <value>:**

```
s.print(expr="temp", value=5555)  
# or  
s.print("temp", 4444)
```

- **-add <expression>:**

```
s.print("--add", expr="temp")  
# or  
s.print("--add", expr="x + y")
```

- **-defs [expression]:**

```
s.print("--defs")
# or
s.print("--defs", expr="temp")
```

- **-dict [expression]:**

```
s.print("--dict")
# or
s.print("--dict", expr="temp")
```

- **-remove [expression]:**

```
s.print()
# or
s.print("--remove", expr="temp")
```

- **-set <expression>:** No method available

profile

- **-freq <sampling-freq>:**

```
s.profile(freq=10000, addr=0x30000000)
```

- **-scratchaddr <addr>:**

```
s.profile(freq=10000, addr=0x30000000)
```

- **-out <file-name>:**

```
s.profile(out="tests/data/gmon_inst.out")
```

state

```
state = tgt.state()
```

stop

```
# Using session object
s.stop()
# Using target object
t.stop()
```

stp

```
# Using session object
s.stp()
# Using target object
t.stp()
```

- **[count]:**

```
# Using session object
s.stp(2)
# Using target object
t.stp(2)
```

stpi

```
# Using session object
s.stpi()
# Using target object
t.stpi()
```

- **[count]:**

```
# Using session object
s.stpi(2)
# Using target object
t.stpi(2)
```

stpout

```
# Using session object
s.stpout()
# Using target object
t.stpout()
```

- **[count]:**

```
# Using session object
s.stpout(2)
# Using target object
t.stpout(2)
```

STAPL

The following are the commands followed by their options (if any) and the Python method for the option (if any).

stapl config

- **-out <filepath>:**

```
st = self.session.stapl()
st.config(out="tests/data/pystapl.stapl", scan_chain=[{'name':
'xcvc1902'}, {'name': 'xcvm1802'}])
```


- **-handle <filehandle>:**

```
st = self.session.stapl()
handle = open("tests/data/pystapl.stapl", "w+b")
st.config(handle=handle, part=['xcvc1902', 'xcvm1802'])
```

- **-scan-chain <list-of-dicts>:**

```
st = self.session.stapl()
st.config(out="tests/data/pystapl.stapl", scan_chain=[{'name':
'xcvc1902'}, {'name': 'xcvm1802'}])
DONE
```

- **-part <device-name list>:**

```
st = self.session.stapl()
handle = open("tests/data/pystapl.stapl", "w+b")
st.config(handle=handle, part=['xcvc1902', 'xcvm1802'])
```

stapl start

```
st.start()
```

stapl stop

```
st.stop()
```

Streams

The following are the commands followed by their options (if any) and the Python method for the option (if any).

jtagterminal

- **-start:**

```
t.jtagterminal()
```

- **-stop:**

```
t.jtagterminal('--stop')
```

- **-socket:**

```
t.jtagterminal('--socket')
```

readjtaguart

```
t.readjtaguart()
```

- **-start:**

```
t.readjtaguart()
```

- **-stop:**

```
t.readjtaguart('--stop')
```

- **-handle:**

```
t.readjtaguart(file='tests/data/streams.log', mode='w')
```

SVF

The following are the commands followed by their options (if any) and the Python method for the option (if any).

svf con

```
svf = s.svf()  
svf.con()
```

svf config

- **-scan-chain:**

```
svf = s.svf()  
svf.config(scan_chain=[0x14738093, 12, 0x5ba00477, 4], device_index=1,  
cpu_index=0,  
out='/home/rpalla/Desktop/svf1.svf')
```

- **device-index <index>:** No method available
- **cpu-index <processor core>:** No method available
- **out <filename>:** No method available
- **delay <tcks>:** No method available
- **--linkdap:** No method available
- **bscan <user port>:** No method available
- **mk_chunksize <size in bytes>:** No method available

- **exec_mode:** No method available

svf delay

```
svf.delay(ticks=1000)
```

svf dow

- **<file>:**

```
svf.dow(file='/proj/rdi/staff/rpalla/nobkup/wkspc_zynq/hello/Debug/hello.elf')
```

- **-data <file> <addr>:**

```
svf.dow("--data" , file = "data.bin", addr = 0x1000)
```

svf generate

```
svf.generate()
```

svf mwr

```
svf.mwr(0xffff0000,0x14000000)
```

svf rst

```
svf.rst("--processor")
```

svf stop

```
svf.stop()
```

TFile

The following are the commands followed by their options (if any) and the Python method for the option (if any).

tfile close

- **handle = <file_handle>:**

```
tfile = s.tfile()
tfile.ls('/tmp')
handle = tfile.open('/tmp/tfile_test.txt', flags=0x0F)
tfile.write(handle, 'hi, this is a test string')
print(tfile.read(handle, size=5))
fstat = tfile.fstat(handle)
print(fstat)
tfile.close(handle)
```

tfile copy

- **src = <src_file>, dest = <dest_file>:**

```
--owner (-o)
Copy owner information.
--permissions (-p)
Copy permissions.
```

```
tfile.copy('/tmp/rigel.txt', '/tmp/rigel2.txt')
tfile.copy('/tmp/rigel.txt', '/tmp/rigel2.txt', '-o', '-p')
```

tfile fsetstat

- **handle = <file_handle>, file_attr:**

```
handle = tfile.open('/tmp/tfile_test.txt', flags=0x0F)
tfile.write(handle, 'hi, this is a test string')
print(tfile.read(handle, size=5))
fstat = tfile.fstat(handle)
fstat.permissions = 65535
tfile.fsetstat(handle, fstat)
```

tfile fstat

- **handle = <file_handle>:**

```
handle = tfile.open('/tmp/tfile_test.txt', flags=0x0F)
tfile.write(handle, 'hi, this is a test string')
print(tfile.read(handle, size=5))
fstat = tfile.fstat(handle)
```

tfile ls

- **path = <dir_path>:**

```
tfile.ls('/tmp')
```

tfile lstat

- **path = <link>:**

```
lstat = tfile.lstat('/tmp/rigel.txt')
```

tfile mkdir

- **path = <dir_path>:**

```
tfile.mkdir('/tmp/new')
```

tfile open

- **flags = <flags>** read mode = 0x00000001 write mode = 0x00000002 append mode = 0x00000004 create mode= 0x00000008 trunc mode = 0x00000010 excl mode= 0x00000020:

```
tfile.open('/tmp/tfile_test.txt', flags=0x0F)
```

tfile opendir

- **path = <dir_path>:**

```
f = tfile.opendir('/tmp')
```

tfile read

- **handle, size:**

```
handle = tfile.open('/tmp/tfile_test.txt', flags=0x0F)
tfile.write(handle, 'hi, this is a test string')
print(tfile.read(handle, size=5))
```

tfile readdir

- **handle = <dir_handle>:**

```
f = tfile.opendir('/tmp')
print(tfile.readdir(f))
```

tfile readlink

- **path = <file_path>:**

```
print(tfile.readlink('/tmp/rigel1.txt'))
```

tfile realpath

- **path = <file_path>:**

```
print(tfile.realpath('/tmp/../../tmp/rigel.txt'))
```

tfile remove

- **path = <file_path>:**

```
tfile.remove('/tmp/rigel.txt')
```

tfile rename

- **old_path, new_path:**

```
tfile.rename('/tmp/tfile_test.txt', '/tmp/rigel.txt')
```

tfile rmdir

- **path = <dir_path>:**

```
tfile.rmdir('/tmp/new')
```

tfile roots

```
print(tfile.roots())
```

tfile setstat

- **path = <file_path>, attr = <file_attr>:**

```
stat = tfile.stat('/tmp/rigel.txt')
print(stat)
stat.permissions = 65535
tfile.setstat('/tmp/rigel.txt', stat)
```

tfile stat

- **path = <file_path>:**

```
lstat = tfile.stat('/tmp/rigel.txt')
```

file symlink

- **link = <link_path>, target = <target_path>:**

```
tfile.symlink('/tmp/rigel1.txt', '/tmp/rigel.txt')
```

tfile user

```
print(tfile.user())
```

tfile write

- **handle <file_handle>, data:**

```
Optional:
offset=<offset>
The offset (in bytes) relative to the beginning of the
file from where to start writing. Default is 0.
pos=<pos>
Offset in *data* to write. Default is 0.
size=<size>
Number of bytes to write. Default is length of *data*
```

```
handle = tfile.open('/tmp/tfile_test.txt', flags=0x0F)
tfile.write(handle, 'hi, this is a test string')
```

Usage Examples

Debug Operations on Session Object

In this mode, you can create a session object and run debug operations on different debug targets using the same session object.

```
session=start_debug_session()
session.connect(url="TCP:xhdbfarmrkd11:3121")
session.targets(3)
# All subsequent commands are run on target 3, until the target is changed
with targets() function
session.dow("test.elf")
session.bpadd(addr='main')
session.con()
session.targets(4)
# All subsequent commands are run on target 4
session.dow("foo.elf")
session.bpadd(addr='foo')
session.con()
```

Debug Operations on Target Object

You can also use the Target object returned by targets() functions and run debug operations can be performed on these objects. The following is an example:

```
session = start_debug_session()
session.connect(url="TCP:xhdbfarmrkd11:3121")
ta3 = session.targets(3)
ta4 = session.targets(4)
# Run debug commands using target objects
```

```
ta3.dow("test.elf")
ta4.dow("foo.elf")
bp1 = ta3.bpadd(addr='main')
bp2 = ta4.bpadd(addr='foo')
ta3.con()
ta4.con()
...
bp1.status()
bp2.status()
```

For interactive usage, it is recommended to use commands and options instead of functions and arguments, as functions require a lot of extra typing. You have defined an `interactive()` function in `xsdb` module, which supports the commands. The following is an example:

```
Vitis-ng [1]: import xsdb
Vitis-ng [2]: xsdb.interactive()
% conn -host xhdbfarmrkb9
tcfchan#0
% ta
1 APU
   2 ARM Cortex-A9 MPCore #0 (Running)
   3 ARM Cortex-A9 MPCore #1 (Running)
4 xc7z020
% ta 2
<xsdpy._target.Target object at 0x7fb2652d3520>
% stop
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0xffffffff28 (Suspended)
% q
Vitis-ng [3]:
```


Streaming Data Transfers

While transferring data from the host typically requires memory mapped interfaces (`m_axi`) to access global memory, or directly access host memory on some platforms, the AMD Vitis™ core development kit also supports streaming data transfer between kernels. This lets you create kernels that access data from the host system, and then stream it directly to other kernels.

Consider the situation where one kernel is performing some part of the computation, and a second or third kernel completes the operation after receiving the data from the first kernel. With kernel-to-kernel streaming support, data can move directly from one kernel to another without having to transmit back through the global memory. This results in a significant performance improvement. Finally, the data can be passed back to the host application through global memory. An example of this can be found in the [Mixed Kernels Design Tutorial with AXI Stream and Vitis](#) on GitHub.

Streaming Kernel Coding Guidelines

In a PL kernel, the streaming interface directly sending data to or receiving data from another kernel streaming interface is defined by `hls::stream` with the `ap_axiu<D,0,0,0>` data type. The `ap_axiu<D,0,0,0>` data type requires the use of the `ap_axi_sdata.h` header file.

The following example shows the streaming interfaces of the producer and consumer kernels.

```
// Producer kernel - provides output as a data stream
// For simplicity the example only shows the streaming output.

void kernel1 (.... , hls::stream<ap_axiu<32, 0, 0, 0> >& stream_out) {

    for(int i = 0; i < ....; i++) {
        int a = ..... ;           // Internally generated data
        ap_axiu<32, 0, 0, 0> v;    // temporary storage for ap_axiu
        v.data = a;                // Writing the data
        stream_out.write(v);       // Writing to the output stream.
    }
}

// Consumer kernel - reads data stream as input
// For simplicity the example only shows the streaming input.

void kernel2 (hls::stream<ap_axiu<32, 0, 0, 0> >& stream_in, .... ) {

    for(int i = 0; i < ....; i++) {
        ap_axiu<32, 0, 0, 0> v = stream_in.read(); // Reading the input stream
    }
}
```

```
int a = v.data; // Extract the data

// Do further processing
}
```

Because the `hls::stream` data type is defined, the Vitis HLS tool infers `axis` interfaces. The following `INTERFACE` pragmas are shown as an example, but are not required in the code.

```
#pragma HLS INTERFACE axis port=stream_out
#pragma HLS INTERFACE axis port=stream_in
```



TIP: These example kernels show the definition of the streaming input/output ports in the kernel signature, and the handling of the input/output stream in the kernel code. The connection of the streaming interfaces from `kernel1` to `kernel2` must be defined during the linking process as described in [Specifying Streaming Connections](#).

Free-Running Kernels

While the Vitis core development kit provides support for streaming interfaces on PL kernels, it also supports a special kind of data-driven kernel called a free-running kernel. Free-running kernels have no control signal ports, no mechanism for interaction with a software application, and cannot be started or stopped. Free-running kernels have the following characteristics:

- When the device is programmed by the binary container (`xclbin`), free-running kernels start running automatically and do not need to be started from a software application.
- Has no memory input or output port, and therefore interacts with other kernels only through input or output streams
- The kernel operates on the stream data as soon as the data is received and the kernel stalls when data is not available.

The main advantage of a free-running kernel is that it does not follow the C-semantics where all the functions should be executed an equal number of times. This modeling style is more like RTL designs as shown in the example below. The compiler models the kernel to restart automatically after the previous function call finishes. This functionality is similar to a `while(1)` loop in software code, without having to specify the loop in the kernel code.

A free-running kernel only contains `hls::stream` inputs and outputs. The recommended coding guidelines include:

- Use `hls::stream<ap_axiu<D,0,0,0> >` for the kernel interfaces.
- The `hls::stream` data type for the function parameter causes Vitis HLS to infer an AXI4-Stream port (`axis`) for the interface.
- The kernel only supports streaming interfaces (`axis`). It should not use `m_axi` or `s_axilite` interfaces, as shown in the example below.

- The free-running kernel must also specify the following block control protocol using the INTERFACE pragma.

```
#pragma HLS interface ap_ctrl_none port=return
```



TIP: This will create a kernel without control signals or an AXI4-Lite interface. This modeling technique is called a free-running kernel, as it is free of any control handshake and will start automatically and run continuously. For kernels requiring some interaction with software applications, consider using auto-restarting kernels as described in (Vitis High-Level Synthesis User Guide ([UG1399](#))).

The following code example shows a free-running kernel with one input and one output communicating with another kernel.

```
void increment(hls::stream<ap_axiu<32, 0, 0, 0> >& input,
              hls::stream<ap_axiu<32, 0, 0, 0> >& output){
#pragma HLS interface ap_ctrl_none port = return
    ap_axiu<32, 0, 0, 0> v = input.read();
    v.data = v.data + 1;
    output.write(v);
    if (v.last){
        break;
    }
}
```



IMPORTANT! Software emulation is not supported for free-running kernels.

Using Vitis System Compilation Mode

The AMD Vitis™ System Compilation mode (shortened as VSC) lets you create a unified single-source C++ model that captures the system composition, containing both the hardware accelerator code and the host application code (based on C-Threads). For use only in Data Center acceleration projects, VSC creates a system structure that captures design intent specified in the C++ model and generates the necessary host code linking to Xilinx Runtime (XRT) to execute the design. The model incorporates a run time software layer that has many automation features for executing the accelerator hardware and managing data transfers. VSC uses Vitis compilation and linking tools to generate an FPGA bitstream and design-dependent object code, or software drivers, which can be linked with the rest of the application to create an executable.

VSC supports most AMD Alveo™ Data Center accelerator cards with a list available at [Supported Platforms and Startup Examples](#).

This section contains the following chapters:

- [Introduction to Vitis System Compilation Mode](#)
- [Quick Start Example](#)
- [Creating a VSC Makefile](#)
- [Building Hardware](#)
- [Application Software Interface](#)
- [Debugging and Validation](#)
- [Supported Platforms and Startup Examples](#)

Introduction to Vitis System Compilation Mode

A typical hardware acceleration flow consists of identifying compute-intensive portions of applications or algorithms and implementing them as custom-designed hardware. These custom-designed hardware elements are typically called accelerators in domains such as heterogeneous computing; in AMD Vitis™ nomenclature, they are also called kernels. Generally, these functions or kernels take significant execution time when run in pure software on x86 or Arm®-based host machines. With the Vitis acceleration flow, once a function or set of functions is chosen for acceleration, Vitis HLS or an RTL flow based on the AMD Vivado™ Design Suite can be used to design or generate custom hardware. The host application can be written using XRT native APIs, or OpenCL™ C or C++ that links to the Xilinx Runtime (XRT).

The process of designing an accelerated application consists of choosing the compute-intensive functions for acceleration and designing custom hardware to accelerate these functions. Additionally, you must write software that runs on a host machine that orchestrates data movements to and from the accelerator. An important and tedious step in the overall application design consists of optimizing the data movements, especially over PCIe® or a network interface. These data movements consist of transfers between the host and kernel through global memories or network ports, and can be optimized based on the nature of the transfers. The accelerated hardware can also be composed of multiple functions, either as repeated functionality to process data in parallel or as a connected pipeline. Such compositions can consist of the same or different accelerated function instances, or it can be a mix of software and hardware functions.

Thus, the perfect system architecture is developed over an iterative design process. Essentially there are three design aspects being developed together: the kernel code, the supporting application or host code, and the hardware/software system interface and composition. While improving the overall design performance, changing one design aspect will invariably require changing another aspect. Vitis System Compilation mode (shortened as VSC) provides a single source unified C++ model that can make the design exploration process fast and easy. VSC is designed to be a framework to:

- Ease host side development by simplifying host code that controls the accelerated hardware
- Facilitate experimentation with compute unit replication to leverage data parallelism in the application
- Enable task-chaining composition on software and hardware side with multiple accelerated and software functions
- Serve the purpose of abstracting low-level hardware APIs, making it look like software programming
- Easily model concurrent or multi-threaded software code, and making it seamlessly integrate with end-user application code

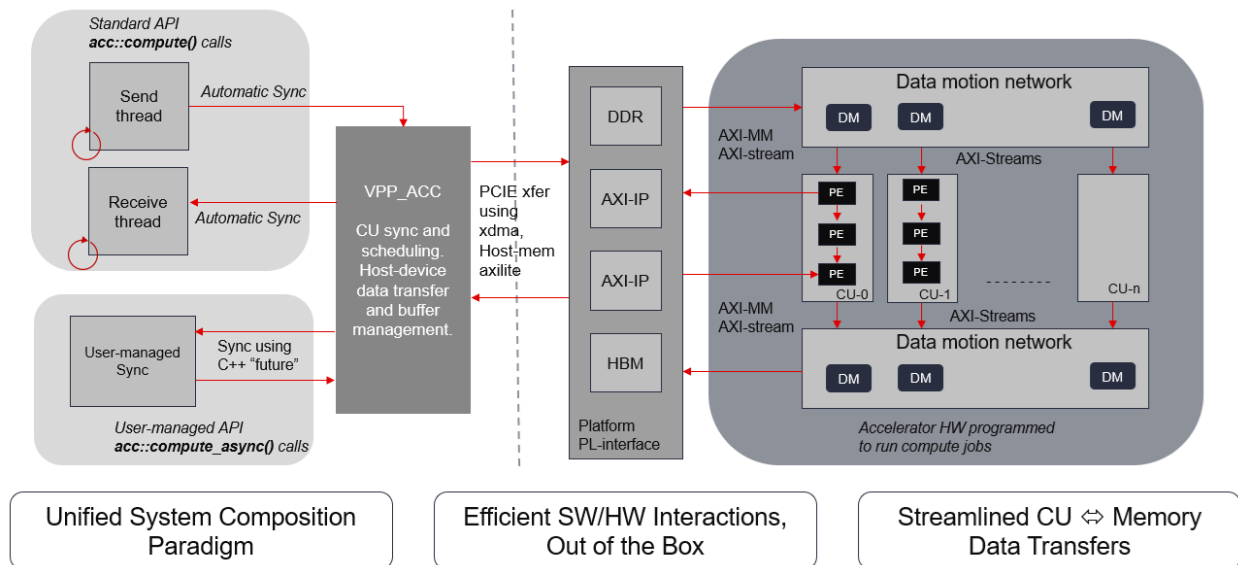
Terminology

- **PL or Programmable Logic:** The part of the FPGA that is made of fundamental programmable building blocks such as flip-flops registers, look-up-tables (LUTs), digital signal processing units (DSPs), RAM-based memory block or clocking circuitry. The PL region can contain one or more accelerators as defined below.
- **Accelerator (ACC):** Designates everything that is custom-generated by the VSC mode and sits inside the programmable logic of the FPGA or adaptive SoC device. The accelerator contains one or more replicated compute units (CUs) in the hardware. In some situations, the term accelerator might also be loosely used to describe the whole Programmable Logic design also including the platform and/or `.xo` kernels generated by the Vitis compile flow. The VSC accelerator is specified in a user-defined C++ class derived from a predefined `VPP_ACC` class. The interface specification contains connections into Vitis platform ports to access the peripheral resources such as global memories or ethernet ports. The accelerator also contains data movers, which are IP designed to efficiently move data from global memory to the compute units and back.
- **Compute Unit (CU):** Designates the composition of one or more processing elements (PEs) as defined below, that connect to global memory and streams to move data to other PEs. The interface is described using the `compute()` method in the accelerator class. This named API acts as the software entry-point function to the accelerator and therefore specifies the hardware-software arguments. The CU makes the system composition self-describing from the source code and not via the use of text config files.
- **Processing Element (PE):** Designates the core building block of a compute functionality that performs specific actions on data. This is a function-call within the scope of the of the `compute()` method in the accelerator class. The processing element's functionality can be written in C++ and each PE is compiled by individually by Vitis HLS.
- **Compose/Composition:** Refers to a method of assembling PEs to form a structural network for a CU. The body of the `compute()` method semantically refers to a composition of PEs in hardware, which is unlike the procedural semantics of a C-function body. VSC will enable validation of such a specification. The PEs collectively compose a CU, and one or more replicated CUs exist in an accelerator.

Hardware and Software Organization

A good system design model makes it very easy to use hardware acceleration for specific functions in an existing application with minimal changes to instantiate compute hardware and run it efficiently. In the Vitis HLS based acceleration flow, the efficiency of the compute hardware will still depend on modeling/coding style and pragmas. In the case of RTL flow, it depends on the chosen architecture. The invocation of accelerated function or CU and interaction with the host should be automated as much as possible, this includes pipelining data through the hardware, using and composing multiple CU's etc.

Figure 158: Hardware and Software Organization Diagram



VSC provides a way to compile your accelerator design and the application software interface from a unified C++ model. The right side of the figure shows the hardware design is a system using the AXI4 framework which is plugged into the dynamic region of a standard Vitis platform. This user-defined composition might consist of replicable compute units (CU), where each CU can be a data-pipelined network of processing elements (PE), and each PE can work on the data:

- in the device memory, typically a DDR, local to the accelerator card having the FPGA
- in a smartSSD connected to the FPGA over PCIe
- arriving to the PE input through one or more AXI4-Stream.

The CUs must connect to platform ports which are typically memory-mapped AXI4 (M_AXI) for data transfers to/from a host CPU through a DDR, or AXI4-Lite for low bandwidth scalar word transfers. The CUs may operate on independent data sets to achieve macro parallelism inherent to the application, achieving compelling acceleration. VSC provides the ability to use a data-mover (DM) for each M_AXI. The DM is an RTL IP that efficiently implements DDR transfers by automating well-defined protocols such as AXI-bursting. The CUs may also transfer data to another user-defined accelerator's CUs through the device memory.

VSC provides an application layer interface as shown on the left-side of the above figure. This is a C++ API interface consisting primarily of two threads for each hardware accelerator, or a cluster of CUs. The send-thread controls forwarding data and launching jobs on the accelerator, while the receive-thread allows gathering results from the accelerator. The send-thread uses a named C-function called `compute()` which acts as the software interface to launch the corresponding call-job on the accelerator. The run time layer will automate the several details in scheduling such

jobs onto CU group and managing efficient data transfers of the `compute()` arguments. These independent threads allow the software to asynchronously interact with the hardware execution, thereby efficiently modeling the application-specific computation and data transfers. The VSC software interface also provides several controls for user-driven synchronization with the hardware.

VSC provides a unified system composition paradigm in C++, provides a runtime layer that allows a hardware composition with streamlined data transfer between CUs and device memory, and efficiently implements hardware-software interactions out of the box.

Because the compilation of hardware is a very time-consuming process, it is important that changes to the hardware code should not trigger recompilation of the hardware. This is avoided by using a specific coding style from the user, and VSC will allow the creation of reusable user-space libraries. Those libraries also act as software stack (of C++ APIs) on top of the hardware accelerator system specified by the user. Such a library may even be used as a dynamic run time shared library to be integrated with a third-party software application.

Design Capture and Modeling

The previous section described the organization, software interfaces, and hardware compiled by VSC from a unified C++ model. The following is a quick summary of some key features of the source C++ model that you can use to easily capture specific design intent:

1. Explicitly model data transfer from host to device and back using two C-threads, to send/receive data to/from each accelerator and its group of CUs
2. Enable performance exploration through guidance parameters, changes such as the use of multiple memory banks, change number of CUs, and concurrent (ping-pong style) data transfers between host and device with minimal or no changes to host or kernel code
3. Replication of CUs to be run in parallel to exploit coarse grain parallelism using only a single numeric guidance parameter
4. Automatic job scheduling on compute cluster (multiple CUs) using round-robin or free-polling
5. Automatic data transfer and accelerator job scheduling on replicated CUs within an accelerator
6. Automatic pipelining of each accelerator job and other optimizations over PCIe®:
 - a. to send/receive data using multiple buffer objects on the host side
 - b. data mover plugin for every CU, based on the data movement pattern guidance
 - i. to copy data between global (DDR/HBM) and on-chip (RAM) memory resources
 - ii. to pre-fetch the next transaction from global memory before the current one finishes on a CU
 - c. Clustered transactions (a sequence of n data sets transferred as one data block) to be processed in one shot, for amortizing PCIe latency

- d. Improve throughput by automatic concurrent (ping-pong style) data transfers using multiple host and device-memory buffers for each CU
 - e. Allow variable payload size using peak memory allocation (allocate for largest data size).
 - f. Dynamic output buffer sizes (allocated at runtime) can be supported, when:
 - i. max buffer size is known at compile-time, and
 - ii. dynamic size is determined by the application code
7. System-level composition using a mix of software (host side) and hardware (compute units):
- a. Hardware-only composition with direct connection (AXI4-Stream) inference. You can create a PE pipeline or a network within each CU, and easily replicate such units.
 - b. Allow free-running PEs with streaming interfaces within a synchronous pipeline (in a CU)
 - c. Mixed hardware and software composition for creating a data processing pipeline. Software tasks can be processing data in-between hardware tasks, with different accelerators compiled into the same xclbin
8. The entire system design is captured in C++ which can be validated in by software compilation of the C++ source and execution
-

Quick Start Example

The convolutional filter example is based on a video filtering use case. An image filter is applied to all the channels of a video frame. The host generates a stream of images for each color channels which are transferred to the device for processing. The RGB video has three parallel channels providing coarse-grain parallelism which allows to process three color channels independently using a separate compute unit per channel. The problem is to design an FPGA based video filter where each video channel can be processed in parallel by a separate CU. The convolutional filter applied to each channel is the same so it is possible that you can build a single compute unit and use three instances of the CU for acceleration. The following sections describe in detail how the host side and CU are modeled and implemented using the Vitis System Compilation (VSC) mode.

Modeling the Convolution Filter

The focus of this section is high-level discussion of modeling the convolutional filter using VSC, and not on the lower level details of the implementation itself. At the top level, the CU is modeled as having a memory interface using three 8-bit pointers (i.e. `char*`) to the color values of the input data and output data, as well as filter coefficients. Some other constant parameters are also passed to the CU like the bias, image height and image width.

The header file `conv_acc_filter.hpp` declares one `compute()` wrapper function used for the host code and software interaction and one or more processing elements (PEs). In this example there is only one PE named `krnl_conv`.

In VSC all accelerators should be written as a class derived from the `VPP_ACC` base class. In the user code, every class that inherits from the `VPP_ACC` class should intend to be an accelerator that compiles to hardware. Specifically the child class should provide a function named `compute()` that is the software entry point to the compiled hardware accelerator, as shown in the example below. Every derived class should have a unique name to represent a unique compute unit function.

```
#pragma once
#include "common.hpp"
#include "vpp_acc.hpp"

class conv_acc : public VPP_ACC<conv_acc, /*NCU*/3>
{
    ACCESS_PATTERN(src, SEQUENTIAL);
    ACCESS_PATTERN(coeffs, SEQUENTIAL);
    ACCESS_PATTERN(dst, SEQUENTIAL);
    // Data copy macros : specifies that size of data to be copied for
    kernel call in
    // case the kernel can process variable size data.
    DATA_COPY(src, src[width*height]);
    DATA_COPY(coeffs, coeffs[FILTER_V_SIZE*FILTER_H_SIZE]);
    DATA_COPY(dst, dst[width*height]);

    // Kernel DDR connections
    SYS_PORT(coeffs, DDR[0]);
    SYS_PORT(src, DDR[0]);
    SYS_PORT(dst, DDR[0]);

public:
    static void compute(
        char                *coeffs,
        float               factor,
        short               bias,
        unsigned short      width,
        unsigned short      height,
        unsigned char        *src,
        unsigned char        *dst
    );
    static void krnl_conv(
        char                *coeffs,
        float               factor,
        short               bias,
        unsigned short      width,
        unsigned short      height,
        unsigned char        *src,
        unsigned char        *dst
    );
};
```

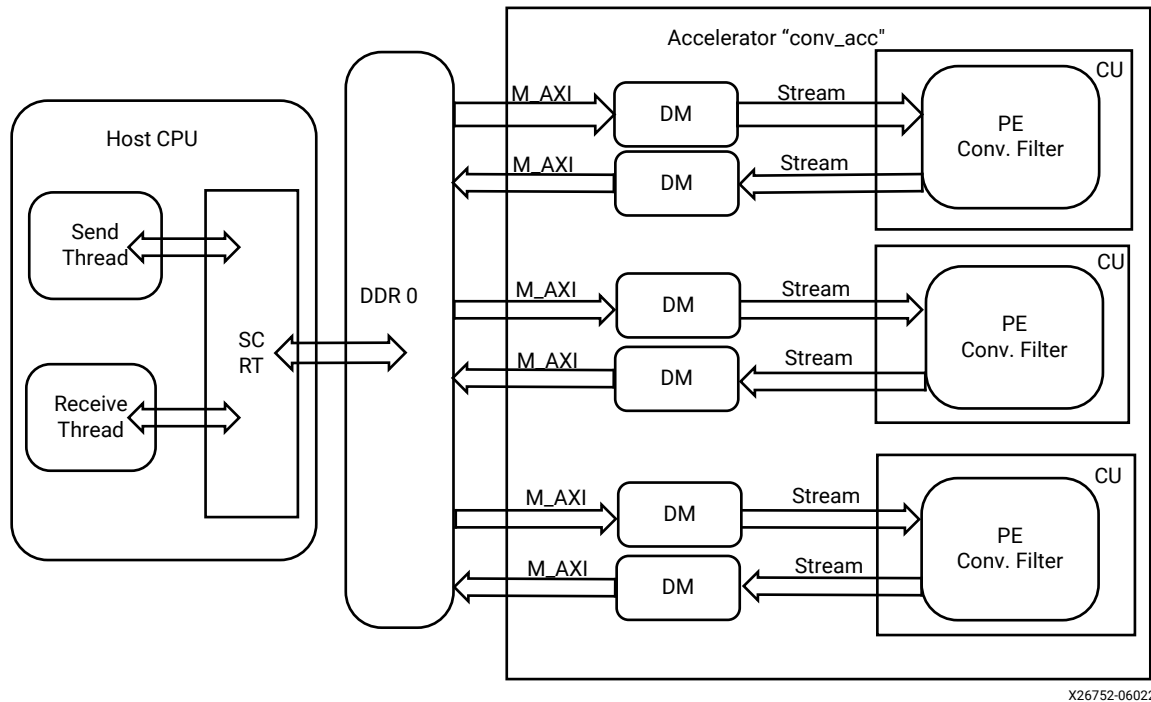
This header file defines a number of different things including the top-level interface of the CU. It declares two static functions `compute()` and `krnl_conv()`. The `compute()` function acts as software entry point on the host side, which is called from the `send_while` thread sending data to the CU (here the `conv_acc` class). The `krnl_conv()` function implements all the functionality of the CU.

The header file also defines some other important items that any kernel model for VSC is required to specify:

Hardware System Architecture

The system architecture and accelerator architecture can be fully specified using an accelerator class derived on the `VPP_ACC` class. In the example code, the convolution filter system architecture, compute composition, number of CUs, and compute connectivity is fully defined by the `conv_acc` class. The following system architecture is generated.

Figure 159: Hardware Architecture



The `conv_acc` class specifies that the `krnl_conv()` function be accelerated by wrapping it inside the `compute()` function. Because a video has three different color channels which can be processed independently using the same functionality, the accelerator class also specifies three CUs to be used. These CUs are replicated and plugged into the memory system using data ACCESS_PATTERN macro. On the host side the figure shows two separate threads for the receive and send threads which interact with the hardware using lower level runtime drivers from VSC.

The CU functionality must be defined via the `compute()` function in a separate `.cpp` file. In this example there is only one processing element (PE), so `compute()` simply wraps it. The following code snippet shows a part of the `.cpp` file with the sub-functions details not provided here. Refer to [Supported Platforms and Startup Examples](#) for more complete examples.

```
// the compute() function wraps the processing function in this example
void conv_acc::compute(
    char          *coeffs,
    float         factor,
    short         bias,
    unsigned short width,
    unsigned short height,
    unsigned char *src,
    unsigned char *dst)
{
    krnl_conv(coeffs, factor, bias, width, height, src, dst);
}

// ... the processing function implements the CU functionality
void conv_acc::krnl_conv(
    char          *coeffs,
    float         factor,
    short         bias,
    unsigned short width,
    unsigned short height,
    unsigned char *src,
    unsigned char *dst) {
    #pragma HLS DATAFLOW

    hls::stream<window,3> window_stream; // Set a stream depth to 3

    // Read incoming pixels and form valid HxV windows
    Window2D(width, height, src, window_stream);

    // Process incoming stream of pixels, and stream pixels out
    Filter2D(width, height, factor, bias, coeffs, window_stream, dst);
}
```

Host Side Data Generation

The convolutional filter consists of three independent CUs which will each process one color channel from the video image stream. The example does not use an actual video stream to keep project simple and stay focused on VSC. The example host code simply generates a random image with three color channels which is passed to a software function which is using the VSC specific code. This results in C-threads for sending and receiving data from host to device plus the `compute()` call.

The function passes all the filter parameters and data pointers to source and destination images. The code snippet below gives the definition of this function. The essential steps include:

- Create buffer pool handles (`srcBufPool...`) to enable sending and receiving data between the host and the device. Attributes indicate if the buffers are inputs or outputs.

- Call `conv_acc::send_while()` using a lambda function. The lambda function allocates buffers on the device, copies host data to the device buffers, then calls the `compute()` function (which ultimately runs the hardware accelerated function). `send_while()` keeps calling the lambda function as long as it return `true`.
- Call `conv_acc::receive_all_in_order()` also using a lambda function to receive the processed buffers.
- Use a `join()` call to wait and synchronize everything.



TIP: Refer to [VPP_ACC Class API](#) for an explanation of the various functions.

```
#include "conv_filter_acc_wrapper.hpp"

int conv_filter_execute_fpga(
    const char      coeffs[FILTER_V_SIZE][FILTER_H_SIZE],
    float           factor,
    short           bias,
    unsigned short   width,
    unsigned short   height,
    unsigned int     numImages,
    YUVImage        srcImage,
    YUVImage        dstImage
)
{
    auto srcBufPool = conv_acc::create_bufpool(vpp::input);
    auto dstBufPool = conv_acc::create_bufpool(vpp::output);
    auto coeffsBufPool = conv_acc::create_bufpool(vpp::input);

    int run = 0;
    int dataSizePerChannel = width * height ;
    // sending input
    conv_acc::send_while([=]()->bool {
        conv_acc::set_handle(run);
        unsigned char * srcBuf = (unsigned char
*)conv_acc::alloc_buf(srcBufPool, 3*dataSizePerChannel);
        unsigned char * dstBuf = (unsigned char
*)conv_acc::alloc_buf(dstBufPool, 3*dataSizePerChannel);
        char * coeffsBuf = (
*)conv_acc::alloc_buf(coeffsBufPool, FILTER_V_SIZE*FILTER_H_SIZE);

        // initialize all input data before parallel computes
        unsigned char * srcChannel[3] = {srcImage.yChannel,
srcImage.uChannel, srcImage.vChannel};
        for (int ch = 0; ch < 3; ch++){
            std::memcpy(srcBuf+ch*dataSizePerChannel, srcChannel[ch],
dataSizePerChannel);
        }
        std::memcpy(coeffsBuf,coeffs,256);
        // execute conv_acc<NCU> parallel computes
        for (int ch = 0; ch < 3; ch++){
            conv_acc::compute(coeffsBuf,
                                factor,
                                bias,
                                width,
                                height,
                                srcBuf + ch*dataSizePerChannel,
                                dstBuf + ch*dataSizePerChannel);
        }
        return (++run < numImages);
    });
}
```

```

    });

    // receive lambda function for receive thread
    conv_acc::receive_all_in_order( [=]() {
        int run = conv_acc::get_handle();
        unsigned char * dstBuf = (unsigned char
*)conv_acc::get_buf(dstBufPool);
        unsigned char * dstChannel[3] = {dstImage.yChannel,
dstImage.uChannel, dstImage.vChannel};
        for (int ch = 0; ch < 3; ch++){
            std::memcpy(dstChannel[ch], dstBuf+ch*dataSizePerChannel,
dataSizePerChannel);
        }
    });
    // wait for both loops to finish
    conv_acc::join();

    return 0;
}

```

Creating a VSC Makefile

Environment Setup

Setup your environment by sourcing both the Vitis setup script and the XRT setup script as shown below:

```

$ source <Vitis_install_path>/Vitis/<version>/settings64.sh
$ source <XRT_install_path>/xrt/setup.sh

```

You should also set up the `PLATFORM_PATH` environment variable to point to the platforms that you have downloaded for your accelerator cards. `PLATFORM_PATH` is used by the Makefile template described below. If you do not set it up, you will need to provide it as a command-line argument to the `make` command.

```

$ export PLATFORM_PATH=<path_to>/platforms

```



TIP: The `PLATFORM_PATH` environment variable is the same as the `PLATFORM_REPO_PATHS`, and points to directories containing platform files (`.xpfm`).

Using the Template Makefile

A template Makefile is provided in the Vitis installation folder that can be included by your Makefile to greatly ease the development of custom Makefiles. The following describes variables required to use the template Makefile by your own Makefile:

```
# Accelerator source files (ie. Vitis HLS code, or vpp_acc code)
# must be specified here; they will be processed by v++ --compile
ACC_SRCS := src/conv_filter_acc.cpp

# Host source files are defined here:
HOST_SRCS := src/host.cpp src/function_cpu.cpp

# Include tool provided Makefile template:
include ${XILINX_VITIS}/system_compiler/examples/vpp_sc.mk
```

The build targets of the template Makefile include: `all`, `run`, `build`, `clean`, and `ultraclean`. A valid `make` command to build the design for hardware emulation (`hw_emu`) could be:

```
$ make DEVICE=<platform> TARGET=hw_emu build
```

A `make` command to run the target as well could be:

```
$ make DEVICE=<platform> TARGET=hw_emu run
```



IMPORTANT! VSC uses `g++ -dumpversion` to get the version number of `g++`, and stops the build if the version is older than 7.1.0. However, on some OSes this command returns only the major version, for example version 7.5.0 is seen as version 7, and returns the following error:

```
vpp_sc.mk: Using /usr/bin/g++
vpp_sc.mk: g++ (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0 <Installation path
of Vitis 2023.2>/system_compiler/examples/vpp_sc.mk:29:
*** gcc version lower then 7.1.0 is not supported. Stop.
```

To resolve this error use the `make` command with `GCC_VERSION=75000`. In this case the version is returned as 75000 in the template Makefile, and the build will continue.

Writing a Custom Makefile

Your custom Makefile should provide similar functions as the template Makefile. The basic steps to compile in VSC are similar to other Vitis tool flows, as shown below:

```
v++ --compile my_acc.cpp -o my_acc.o
v++ --link my_acc.o -o hw.xclbin # also outputs hw.o
g++ -c main.cpp -o main.o
g++ -o host.exe main.o my_acc.o hw.o -lvpp_acc -lxrt_core -lpthread
```

Some things to notice in these commands:

1. The `v++` compile command does not use the `-k` option to specify a kernel name. This is because the source code defines a class derived from the `VPP_ACC` class that enables the Vitis tools to compile the VSC system.
2. The `v++ --compile` compile command produces `.o` files instead of `.xo` files. These files are used by Vitis compiler and `g++` to build the hardware and host systems.
3. The `v++ --link` command will use the `.o` files as inputs and generate both an `.xclbin` for the hardware accelerator, and a `hw.o` which contains compiled objects for the VSC hardware-software interfaces required by the host application and used by `g++`.
4. The commands will also need to have specific options for the respective tools and modes, for example `--platform` and `--target`.

Alternatively, you can create a shared (dynamically loadable) library to let the accelerator be easily integrated into a third-party application. For example, `libmy_acc.so` as given below:

```
v++ -c my_acc.cpp -o my_acc.o
v++ -l my_acc.o kernel.xo -o hw.xclbin # also produces hw.o
g++ -c -fPIC app.cpp -o app.o
## Command to create the shared library:
g++ -shared -fPIC -o libmy_acc.so app.o my_acc.o \
    -Wl,--whole-archive ${XILINX_VITIS}/system_compiler/lib/x86/
libvpp_acc.a \
    -Wl,--no-whole-archive
## The link command using the shared library
g++ -o host.exe main.o -lmy_acc -lxrt_hw -lpthread
```

OS and Tool-Chain Compatibility

The VSC tools should work with any Linux distribution which is compatible with RHEL/CentOS 8 toolchain (including `gcc-8.3` and `binutils-2.30`).

RHEL/CentOS 7 is also supported, with the following requirements for building and running:

1. Use of `devtoolset-8`. For example:
 - It can be installed with `sudo yum install devtoolset-8`
 - It can be used in a shell as `scl enable devtoolset-8 bash` for Vitis compilation
2. The host code needs to be compiled with this flag: `-D_GLIBCXX_USE_CXX11_ABI=0` explicitly when compiling with `g++`.



TIP: When using the VSC template Makefile this is automatically handled, as described above.

3. Final linking needs to use `${XILINX_VITIS}/system_compiler/lib/centos7/libvpp_acc.a`.



IMPORTANT! Because of the need to disable the use of `CXX11 ABI` while building on RHEL/CentOS 7, the executable cannot be run on a RHEL/CentOS 8 platform, and vice versa.

Building Hardware

This chapter will describe the key aspects of building the accelerator hardware using VSC. The first section describes the user-defined C++ accelerator class specification, and the following sections describe creating the hardware interface with user-guidance macros, the data transfer types, and the different styles of system composition for the accelerator.

Accelerator HW Interface

User-Defined Accelerator Class

Vitis enables VSC for hardware and host-code compilation when the source code contains a class derived from the `VPP_ACC` class. The class derived from `VPP_ACC` will be compiled as an accelerator.

A VSC accelerator interface is defined in the single-source C++ model as a class definition that is required in a C++ header file. Multiple accelerator class definitions can be found in one compilation unit (as a single `v++ --compile` command), all of which will be implemented in the same PL hardware. You might also provide multiple accelerators across multiple compilation units. However each compilation unit must define the complete kernel (or HLS) code for that unit.

Every user-defined accelerator class:

- Must be derived from the Vitis pre-defined `VPP_ACC` class
- Must have a static method named `compute()`
- The arguments and functionality of `compute()` can be user-defined
- The class template has two arguments:
 - The first argument must be the same as the user-defined class name
 - The second argument is the number of compute units to be replicated in the hardware

An example accelerator class definition (`xmmult`) is provided below:

```
#include "vpp_acc.hpp"
class xmmult : VPP_ACC<xmmult, /*NCU=*/4>
{
public:
    // Platform port connections
    SYS_PORT(A, DDR[0]);
    SYS_PORT(B, DDR[1]);
    SYS_PORT(C, DDR[2]);
    SYS_PORT_PFM(u50, A, (HBM[0]:HBM[4]:HBM[8]:HBM[12]));
    SYS_PORT_PFM(u50, B, (HBM[1]:HBM[5]:HBM[9]:HBM[13]));
    SYS_PORT_PFM(u50, C, (HBM[2]:HBM[6]:HBM[10]:HBM[14]));
    // Data interfaces
```

```

ACCESS_PATTERN(A, SEQUENTIAL);
ACCESS_PATTERN(B, RANDOM);
DATA_COPY(A, A[SZ]); // move to local memory
DATA_COPY(B, B[SZ]);
ZERO_COPY(C); // direct AXI-mm

// define the SW entry point of the accelerator
static void compute(data_t* A, data_t* B, data_t* C);
// define the HW top-level of the accelerator (HLS top)
static void mmult(data_t* A, data_t* B, data_t* C);
};

```

This class interface model captures all hardware system-related considerations in a single unified source and allows easy integration with the application layer through the `compute()` function as described in [The `compute\(\)` API](#). The `compute()` function is the host application's entry point to the hardware accelerator.

The class definition also contains guidance macros that refer to `compute()` arguments. By providing these, you will guide VSC to make specific hardware choices during implementation. Refer to [Guidance Macros](#) for more information.

The example shown above describes the accelerator class named `xmmult`, which will be compiled by VSC to have four CUs in hardware (`/*NCU= */4`). The accelerator has a PE (or kernel) code defined in the `mmult()` function, which is called within `compute()` that takes three arguments A, B, and C. For each of these arguments, two types of guidance macros are specified: memory port connections, and data access.

1. Platform port connections using `SYS_PORT()` and `SYS_PORT_PFM()` macros.
 - a. These are typically global memory I/O connections or other AXI4 interface connections available in the hardware platform used during Vitis compilation. In this example, the first three `SYS_PORT()` macros connect the three A, B, and C arguments of `compute()` to different DDR banks (0, 1 and 2).



TIP: The `SYS_PORT()` guidance macro connectivity for each argument applies to all the CU instances (`NCU=4`) in the `xmmult` accelerator, because it only specifies one memory bank.

- b. The three `SYS_PORT_PFM()` macros apply global memory connections in two ways:
 - i. It only applies when the target platform name contains `u50`, as in the U50 Alveo card.
 - ii. For each of the four CUs, each argument connects to a different HBM banks. This is because the syntax used for the `SYS_PORT_PFM()` for each argument specifies connectivity four times:

```
SYS_PORT_PFM(u50, A, (HBM[0]:HBM[4]:HBM[8]:HBM[12]));
```

2. Data access is specified using `ACCESS_PATTERN()`, `DATA_COPY()`, and `ZERO_COPY()` macros:

- a. The `ACCESS_PATTERN()` macros directs VSC to infer a sequential or random access for data transfer. In the example, argument A is defined as sequentially accessed and therefore can be implemented with a stream connection. Argument B is accessed randomly and therefore requires a local (on-chip) memory buffer to support randomized data access from the `PE mmult()`.
- b. For arguments A and B, the `DATA_COPY()` macros direct VSC to infer a local memory (RAM) next to the accelerator.
- c. The `ZERO_COPY()` macro directs VSC to create a memory-mapped AXI (M_AXI) connection for argument C.

Guidance Macros

The guidance macros that VSC supports allow the function arguments (both PE and compute) in the accelerator class to use to various types of hardware interfaces. The following defines the different types of guidance macros:

- **SYS_CLOCK_ID(<PE-name>, <clock-ID>);:**
 - <PE-name> specifies a processing element function instantiated in the body of the `compute()` function.
 - <clock-ID> is an integer value referring to any clock supported by the platform and is available for connection in the user-logic partition. When this macro is specified, the kernel implementation will use the corresponding clock net for the kernel. The available clock IDs for a platform can be found using the `platforminfo` command.



TIP: When two PEs are connected through AXI-streams, clock-ID may be different if those PEs are marked free-running. In this case, VSC will automatically insert clock-domain crossing (CDC) connection for data transfer between the PEs. When a free-running PE is connected to a PE that it not, they both need to have the same clock-ID.

- **SYS_PORT(<port>, <global_memory>);:**

Specify the platform interface to use for a given argument of the `compute()` function. The global memory is typically a memory bank that will be used to data transfer to/from the FPGA.

- <port> refers to the name of a specific `compute()` argument.
- <global_memory> can be specified in one of the following forms.
 - <bank-ID>: A single bank ID that applies to all CU instances. For example DDR, DDR[1], or HBM[5]. The bank names for a platform can be found by using the `platforminfo` command.



IMPORTANT! Observe the following limitations:

- Host memory (HOST[0]) is only supported for the X3 hybrid platforms (xilinx_x3522p*)
- Specifying memory bank ranges (for example HBM[0:3]) is not supported

- Specifying PLRAM is not supported

- (<CU1-bank-ID>:<CU2-bank-ID>:...:<NCU-bank-ID>): Within parenthesis, a list of bank-IDs for each CU separated by colons. The bank-IDs are specified for each CU in numerical order, but does not include the CU name:
(HBM[0]:HBM[4]:HBM[8]:HBM[12]). The number of entries must match the specified number of CUs in the class (/ *NCU= */ 4).

- **SYS_PORT_PFM(<substr>, <port>, <global_memory>);:**

This can be used to configure accelerator port connections for specific platforms, but defined through a single class header. For example in the code given below, port-A will be connected to HBM[0] for a u50 platform, and to DDR[0] for all other platforms.

```
SYS_PORT(A, DDR[0]);
SYS_PORT_PFM(u50, A, HBM[0]);
```

- The <substr> refers to a sub-string of the platform name. For example, using u50 would employ the SYS_PORT_PFM connections only when the platform name contains the specified string.
- The <port> and <global_memory> arguments work as described above for SYS_PORT macros.



IMPORTANT! When multiple *SYS_PORT* and *SYS_PORT_PFM* macros are provided for the same <port>, VSC will apply the last suitable *SYS_PORT* or *SYS_PORT_PFM* guidance macro that is read.

- **ACCESS_PATTERN(<port>, <pattern>);:**

Enables VSC to infer a data mover between the hardware accelerator interface and the global memory in the device.

- <port> refers to the name of a specific `compute()` argument.
- <pattern> defines one of two different memory access patterns:
 - **SEQUENTIAL:** data transfers occur through AXI4-Stream connections to the acceleration interface. The CU (or kernel) code must strictly follow a sequential access pattern on the corresponding argument, otherwise it will lead to incorrect hardware behavior. For example, pointer indices should be sequentially incremented as with the coding style `pointer[i++]` or `*pointer++`.
 - **RANDOM:** data transfers into an on-chip memory acting as a cache to the accelerator. Therefore the CU code does not need to follow a sequential access pattern.



IMPORTANT! On-chip memory resources are limited (for example, typically 32 Kbits per BRAM which could be accessed as 1024 words of 32 bits). If a large payload size per compute job uses too many on-chip RAMs, it can lead to timing closure issues in the Vivado tools. It might be better to use a *ZERO_COPY* guidance macro connecting the accelerator directly to global memory as described below.

- **DATA_COPY(<port>, <port>[<Num>]);:**

Infers a data-mover IP between the global memory and the accelerator interface. At the runtime of each `compute()` call this data-mover IP will copy the data for the specific `compute()` argument from (or to) a source memory specified by `SYS_PORT` or `SYS_PORT_PFM` guidance macro, or from (or to) local on-chip memory.

- `<port>` refers to the name of a specific `compute()` argument.
- `<port>[<Num>]` specifies the number of array elements that the (array or pointer) argument refers to. `Num` can be an expression of C-constants and/or scalar arguments of `compute()`. This allows the accelerator to have a dynamic payload size determined at run time, and enables automatic bursting on AXI4 connections, data width conversion, and padding for the user-defined argument data-types.



IMPORTANT! When using `DATA_COPY` along with `RANDOM` access pattern, the corresponding argument in the prototype of the `compute` API must be declared as an array with fixed size. For example: `compute(int A[10], ...)`.

- **ZERO_COPY(<port>);:**

Directs VSC to not infer a data-mover IP. Instead, let the accelerator use a AXI4 interface directly connected to the specified global memory for the specified argument of the `compute()` function.

- `<port>` refers to the name of a specific `compute()` argument.

- **ASSIGN_SLR(<PE>, <SLR-IDS>);:**

VSC will request Vivado to place the related logic of the named PE into the specified SLR(s). However, this is only a request and the final determination made during placement.

- `<PE>`: Specifies the name of the processing element.



TIP: If the `compute()` function is specified, the SLR assignment is applied to all PEs inside the `compute()` function.

- `<SLR-IDS>`: Specifies the SLRs to use to place the PE. This can be specified in one of the following forms.
 - `<SLR-ID>`: Applies the specified SLR-ID to all CU instances of this PE.
 - `(<CU1-SLR-ID> : ... : <NCU-SLR-ID>)`: Within parenthesis, a list of SLR-IDs separated by colons. These SLR-IDs are assigned to the CU instances. The number of entries must match the specified number of CUs in the class (`/ *NCU= */ 4`).

- **FREE_RUNNING(<PE>);:**

Enables the named PE function to be marked as free-running or an always executing kernel in hardware. Refer to [Accelerator System Composition](#) for more information.

The `compute()` API

The `compute()` method is a special user-defined method in the user-defined accelerator class definition which is used to represent the compute unit. The arguments of `compute()` provide the software interface of the CU to the host-side application, and the body of `compute()` specifies the hardware composition of the CU. The accelerator class must have one or more additional methods defined, each of which may be individually called only within the body of `compute()`. Each such function called is a processing element (PE) in the hardware. The body of `compute()` can be used to create a structural composition of PEs.

The `compute()` body has the following features:

- Is a structural network of processing elements:
 - Using AXI4 standard protocols in the hardware.
 - Using a start/stop synchronization per `compute()` job
 - Can represent a synchronized hardware pipeline
- Only calls to PE functions and local variable declarations, and no other C-language constructs, are allowed in the `compute()` body
- Each PE is a processing element running in parallel in hardware:
 - The code of this function is implemented as an FSM and datapath using Vitis HLS
 - PE functions can include Vitis HLS pragmas (`#pragma HLS`)
- PEs can be connected to global memory or other platform AXI4 ports
- PEs can be connected to each other through AXI4-Stream interfaces
- A PE can be free-running:
 - Unaware of transaction start/stop with no `ap_ctrl` signals
 - Always executing (data-driven) without a reset/start state
 - Only AXI4-Stream interfaces are allowed

An example `compute()` function definition is given below:

```
typedef vpp::stream<float, STREAM_DEPTH> InternalStream;
void pipelined_cu::compute(float* A, float* B, float* E, int M) {
    static InternalStream STR_X("str_X");
    static InternalStream STR_Y("str_Y");
    mmult(A, B, STR_X, M);
    incr_10(STR_X, STR_Y, M);
    incr_20(STR_Y, E, M);
}
```

- This example is a hardware pipeline of three PEs connected by two internal AXI4-Stream interfaces: `STR_X` and `STR_Y`.

- The `mmult` PE reads matrices A and B from global memory using their associated pointers and writes to the stream `STR_X`.
- The `incr_10` PE reads from stream `STR_X` and writes to stream `STR_Y`.
- The `incr_20` PE read from stream `STR_Y` and writes to global memory E.
- The scalar argument M is the dimension of the matrices and is provided to all the PE in the CU.

This model can be used to define various types of accelerator system compositions.

The `compute()` interface is intended to succinctly capture a hardware system while providing a simple software application interface. The following section describes the native C++ data types allowed at the `compute()` interface. The entire accelerator class definition with `compute()` and other PEs, along with an application code can be functionally validated as described in [Debugging and Validation](#).

Interface Data Types

All the basic C++ data types are supported for `compute()` arguments: `bool`, `char`, `int`, `short`, `long`, `float`, `double`. These C++ type modifiers are supported as well: `signed`, `unsigned`, `short` and `long`.

In addition, arbitrary precision data types are also supported, `ap_int<N>` and `ap_uint<N>`, as described in [Arbitrary Precision \(AP\) Data Types](#) in the Vitis HLS tool. The `ap_int` and `ap_uint` are pre-defined data types in Vitis HLS, where N is a integer with a max limit of 1024. This allows finer bit-width control, especially to handle data packing or to match a global memory data bus width.

The `compute()` interface arguments can be classified as Scalar types or Buffer types of arguments as described below.

Scalar Type

A scalar argument is a basic data-type that represents a single word passed to the `compute()` call. VSC will infer a scalar argument type when any of the basic data types described earlier are used as a pass-by-value argument of `compute()`. For example `size` and `value` are scalars here: `compute(int size, float value);`



TIP: The transfer of a scalar word is done using the AXI4-Lite hardware interface.

A user-defined C-struct can be used as a pass-by-value scalar argument, with the following rules:

- Struct cannot have a field with C++ bit specifiers, for example: `int field_x:4;`
- Struct cannot have a pointer field, for example: `int* ptr_field;`

- The total byte-size of the struct must be a strict power of two. If not, the user is required to declare a dummy field to fill in the remaining bytes (e.g. `char pad[remainder]`).

Scalar arguments are always passed by value, and are typically inputs to the accelerator. A standard C++ reference argument is not allowed. For example:

```
compute(int size, float& result_value) // reference argument is not allowed
```

However, a C++ pointer type can be used in place of a reference when a PE function argument is still a reference. For example:

```
my_acc::compute(int inp, data_t *out_p) { // reference not allowed
    my_PE(inp, *out_p); // deref the ptr for my_PE out_ref
    my_PE_two(inp ... ); // passing inp to multiple PEs is allowed
}
my_acc::my_PE(int in, data_t &out_ref) { // reference is allowed
    ...
    // application code
    my_acc::send_while ... { ...
        out_p = my_acc::alloc_buf(bp, 1); // required
        my_acc::compute(inp, out_p);
    }
```

VSC will infer a scalar for `out_p`, when this output argument when all these conditions apply:

- It must be pointing to only one element.



IMPORTANT! The application code must allocate memory of size 1 using `alloc_buf()` as described in [VPP_ACC Class API](#).

- The kernel code writes exactly one value to it, for example: `*out_p = result;`
- No VSC guidance macros are specified on this argument.

A scalar argument of `compute()` can be passed to multiple PEs. In hardware a single AXI4-Lite port will drive individual scalar registers of every PE function that takes this scalar argument.

Buffer Type

A buffer is a pointer or array argument to the `compute()` function, which can hold one or more elements. The contents of the buffer is transferred to/from the device based on the corresponding platform interface (SYS_PORT connection). The base (element) type of the pointer or array argument can be any basic data types described in the earlier section. In addition, a user-defined C-struct data type can be also be used as a base element type.

Note: A buffer type argument requires global memory space both on the device and in the CPU host. Therefore, when using a pointer type the application code is required to call memory allocation (`alloc_buf` described in [VPP_ACC Class API](#)).

When using a C-struct as a base type for an pointer or an array, the following rules apply:

- Struct cannot have field with C++ bit specifiers, for example: `int field_x:4;`
- Struct cannot have a pointer field, for example: `int* ptr_field;`



IMPORTANT! The C++ data modifiers (like `const`, `register`, `volatile`) are not-allowed at the `compute()` interface.

The following example shows coding styles, both allowed and not-allowed, for buffer type arguments:

```
// accelerator interface
my_acc::compute(int A, int *B, int C[10]) {
...
// application code
int S, *BB, *CC;
my_acc::send_while ... { ...
    BB = my_acc::alloc_buf( ... ); // required
    CC = my_acc::alloc_buf( ... ); // required
    compute (S, AA, BB);
}
```

In the example above, the `compute()` arguments A and B are treated the same way in the application code so that it is required to allocate memory for these buffer arguments.



TIP: A buffer argument of `compute()` can only be passed to a single PE function within.

In the example, A is a scalar input argument. The caller only passes an integer argument to the `compute()` call, the value is transferred to the device using AXI4-Lite. Whereas, B and C are buffer arguments. They can be either input, output, bi-directional or remote. The caller has to allocate memory for this buffer using `alloc_buf()`.

Platform Specific Hardware Config

As mentioned in [Guidance Macros](#), `SYS_PORT` connections can be customized per platform using the `SYS_PORT_PFM` macro. But if a new platform needs to be added, and thus the header file gets modified, then `make` will want to rebuild the hardware for already existing and compiled platforms which is not desirable. A good way to prevent this is to create separate configuration header files for each platform, and use the `-I CFLAG` to include the correct one for a given platform. For example:

```
ifneq (, $(findstring u50, $(DEVICE)))
    EXTRA_CFLAGS := -I include/u50
else
    EXTRA_CFLAGS := -I include/u200
endif
```

The accelerator header file includes a common file (example `config.hpp`), which is customized and provided in separate directories to make it easier to have platform-specific configuration, like number of compute units and port mapping as shown in this example:

Table 74: Additional Makefile Variables

In the ACC class specification	include/u50/config.hpp	include/u200/config.hpp
<pre>#include "config.hpp" // Accelerator class class pipelined_cu : public VPP_ACC<pipelined_cu, NCU> { ... SYS_PORT(A, PORT_MAP_A); SYS_PORT(B, PORT_MAP_B); SYS_PORT(E, PORT_MAP_E); }</pre>	<pre>#define NCU 2 // global memory connections to Accelerator ports #define PORT_MAP_A (HBM[0]:HBM[2]) #define PORT_MAP_B (HBM[0]:HBM[2]) #define PORT_MAP_E (HBM[0]:HBM[2])</pre>	<pre>#define NCU 3 // global memory connections to Accelerator ports #define PORT_MAP_A (DDR[0]:DDR[1]:DDR[2]) #define PORT_MAP_B (DDR[0]:DDR[1]:DDR[2]) #define PORT_MAP_E (DDR[0]:DDR[1]:DDR[2])</pre>

Accelerator System Composition

As described in [the `compute\(\)` API](#) section, the body of the `compute()` function represents a structural composition of PEs, which is unlike the procedural C semantics but can be validated in the Vitis tools by running software emulation. The `compute()` scope can only hold two types of C++ statements:

1. Function calls to represent hardware PEs. The structural semantics of the `compute()` body allows various hardware composition styles that are described in the following sections.
2. Static `vpp::stream` object declarations to represent data streams in-between PEs. This is described in more detail in the following section.

Stream connections using `vpp::stream`

Each argument of a PE function has to be connected to either an argument of the top-level `compute()` function, or to a statically declared stream. A `vpp::stream` inherits from the Vitis HLS `hls::stream` which is described in the HLS Stream Library section of the Vitis HLS User Guide ([UG1399](#)).

The `vpp::stream` object provides a few additional features:

- The `vpp::stream` constructor takes an additional argument, `post_check`, the default value is `true`. Most stream variables will be expected to be empty at the end of each `compute()` call, but others might be designed to carry over data across multiple `compute()` calls. In software emulation, by default `vpp::stream` will be checked to be empty at the end of each `compute()` call, and will assert if it is not. The optional `post_check` argument lets you turn off this assertion.
- The `DEPTH` parameter allows the user to specify a FIFO depth to be implemented in hardware, in addition to being used during software and hardware emulation.



TIP: The default value of `DEPTH` is set to 1024 and that is typically an over-utilization of resources in hardware compilation. You are recommended to specify the depth for streams used in the scope of `compute()` to conserve resources. You can run software emulation with a small depth of 2 to test functionality and help determine the required depth.

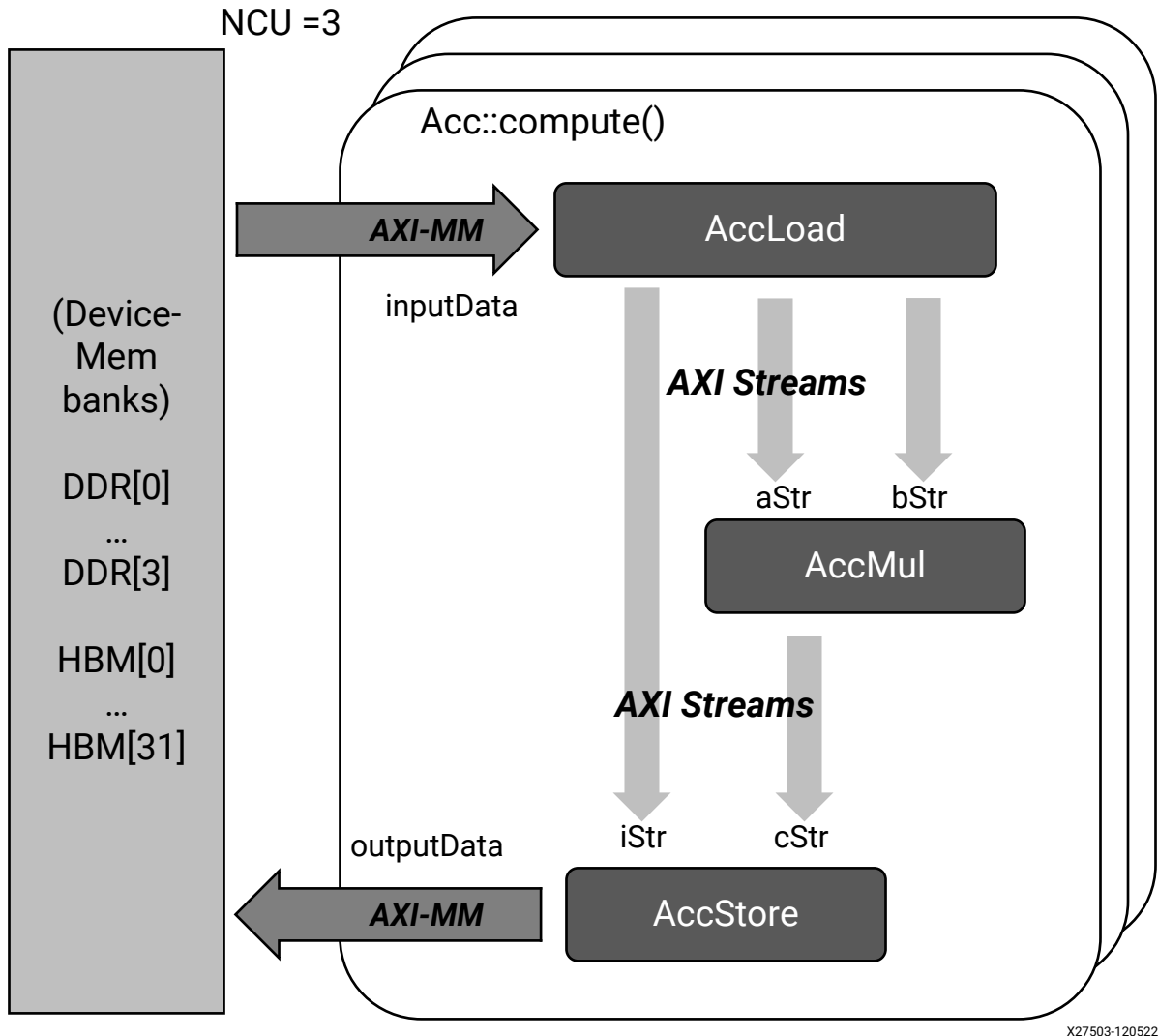
- A `vpp::stream` variable can be passed down to a PE-function argument which can be of type `hls::stream`.
- The `vpp::stream` variable must be declared `static` to ensure proper functional validation in Vitis software emulation. In hardware, a stream/FIFO is not emptied implicitly and its content is persistent across different runs. It behaves in the same way as a `static` variable.

The following sections describe popular styles of composing a pipelined system of PEs using basic C++ coding within the `compute()` body scope. The PEs in the pipeline can use direct AXI4-Stream connections or AXI4 (M_AXI) global memory access to move data across PEs.

Single-path Synchronous Pipeline

The following figure shows an example of a single-path pipeline which has three PEs, namely `AccLoad`, `AccMul`, and `AccStore`. The PEs `AccLoad` and `AccStore` access data stored in the global memory through M_AXI channels. The accelerator class header ties the `inputData` and `outputData` ports to DDR[0]. In this case, the `ZERO_COPY` code was used for these PEs to directly access the global memory.

Figure 160: Single-path Pipeline



X27503-120522

The following is the code example for the figure.

```
typedef vpp::stream<DT> STREAM;
class Acc : public VPP_ACC<Acc, NCU>
{
    ZERO_COPY(inputData);
    ZERO_COPY(outputData);
    SYS_PORT(inputData, DDR[0]);
    SYS_PORT(outputData, DDR[0]);
public:
    static void compute(DT* inputData, DT* outputData);

    static void AccLoad(DT* inputData, STREAM& aStr,
                        STREAM& bStr, STREAM& iStr);
    static void AccMul(STREAM& aStr, STREAM& bStr,
                       STREAM& cStr);
    static void AccStore(STREAM& iStr, STREAM& cStr,
                         DT* outputData);
};
```

```

}
void Acc::compute(DT* inputData, DT* outputData)
{
    static STREAM aStr, bStr, cStr, iStr;

    AccLoad (inputData, aStr, bStr, iStr);
    AccMul  (aStr, bStr, cStr);
    AccStore(iStr, cStr, outputData);
}
void Acc::AccMul(STREAM& aStr, STREAM& bStr, STREAM& cStr)
{
    for (int i = 0 ; i < N_WORDS ; i ++) {
        int res = aStr.read() * bStr.read();
        cStr.write(res);
    }
}

```

The `compute()` function body represents a hardware pipeline. There are three function calls corresponding to the PEs, and there are four local stream variables declared:

1. `AccLoad` takes `inputData` and writes to three streams
2. `AccMul` processes a fixed number of words in input streams `aStr` and `bStr` and writes the results to `cStr`
3. The `AccStore` function will further process the incoming data in `iStr` and `cStr` to write results on `outputData` connected to DDR[0] port.

The PEs in this system will execute in a synchronous fashion such that data flows through in a pipelined fashion. Every call of `compute()` will load `inputData` and trigger all PEs for a new transaction. Every call to `compute()` requires every PE to complete execution (start and stop) exactly once. This example is a pipeline with 3-stages, or PEs chained in a single-path. Thus, with a simple C++ coding style the user can create a hardware pipeline.

The `VPP_ACC` class allows replication of such pipeline using the `NCU` parameter. If `NCU` is more than 1, then the hardware contains as many replicated pipelines. The calls to `compute()` are automatically loaded in the next available pipeline slot. Thus, the application layer remains simple and easy to maintain, and automates running data through multiple pipelines in the hardware.

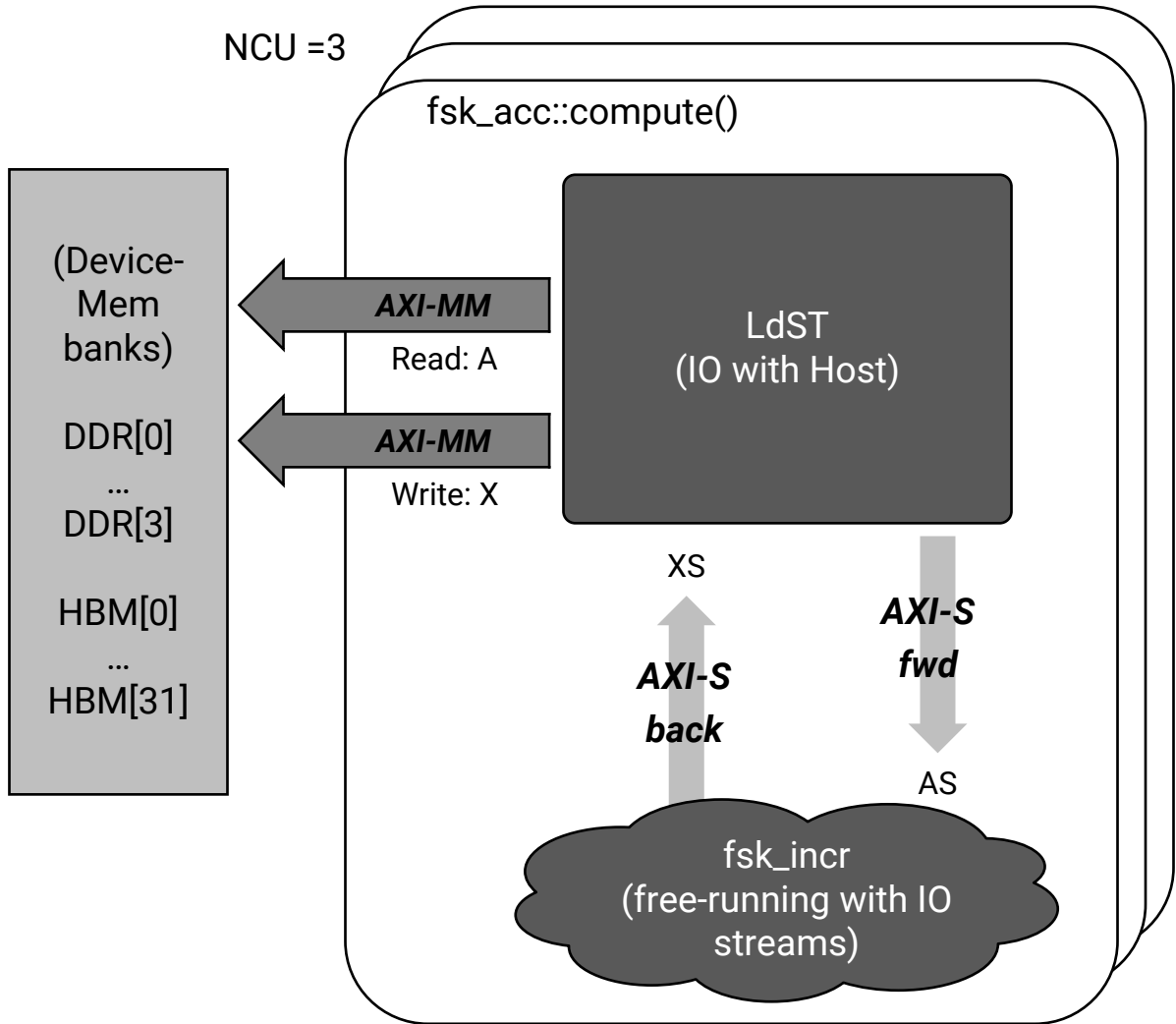
Free-Running PE

The PEs in a pipeline operate synchronously on transactions passing through. For every `compute()` call a PE will start and stop exactly once. However, when a PE is marked as `FREE_RUNNING` as described in [Guidance Macros](#), it has the following hardware semantics:

- The PE does not start, stop, or reset per transaction or `compute()` call. It is an HLS kernel with the `ap_none` control interface as described in [Block-Level Control Protocols](#)
- The interface must have only AXI4-Stream arguments, or scalar inputs
- Operation is data-driven, with the PE acting only on the input stream words, and unaware of the payload size of the transaction

- The PE begins execution immediately after the hardware bitstream is programmed into the device

Figure 161: Free-Running



X27504-120522

The figure above shows a diagram of the free-running PE. This accelerator contains two PEs, a `LdStr` PE that has global memory access, and the `fsk_incr` which is a free-running PE. In the `compute()` scope these PEs are connected by two AXI4-Stream interfaces: `AS` that moves words from `LdStr` to `fsk_incr`, and `XS` which is the feedback path.

The code for this example is provided below.

```
class fsk_acc : public VPP_ACC<fsk_acc, NCU>
{
    ZERO_COPY(A);
    ZERO_COPY(X);
    SYS_PORT(A, DDR[0]);
    SYS_PORT(X, DDR[0]);
    FREE_RUNNING (fsk_incr);
}
```

```

public:
    static void compute(DT* A, DT* X, int sz);
    static void loadstore(DT* A, DT* X, hls::stream<DT>& AS,
                          hls::stream<DT>& XS);
    static void fsk_incr(hls::stream<DT>& AS, hls::stream<DT>& XS);
};

Void fsk_acc::compute(DT* A, DT* X, int sz)
{
    static vpp::stream<DT> AS, XS;
    ldst(A, X, sz, AS, XS);
    fsk_incr(AS, XS);
}

void fsk_acc::ldst(DT* A, DT* X, int sz, hls::stream<DT>& AS,
                  hls::stream<DT>& XS)
{
    for (int i = 0; i < sz; i++) {
        AS.write(A[i]);
    }
    for (int i = 0; i < sz; i++) {
        XS.read(X[i]);
    }
}

void fsk_acc::fsk_incr(hls::stream<DT>& AS, hls::stream<DT>& XS)
{
    DT val;
    AS.read(val);
    XS.write(val + 1);
}

```

The `LdSt` PE operates on `sz` words reading and writing to the global memory ports `A` and `X` respectively. Whereas, the free-running PE `fsk_incr` is agnostic to `sz`, and reacts only to the words on the incoming `AS` stream.

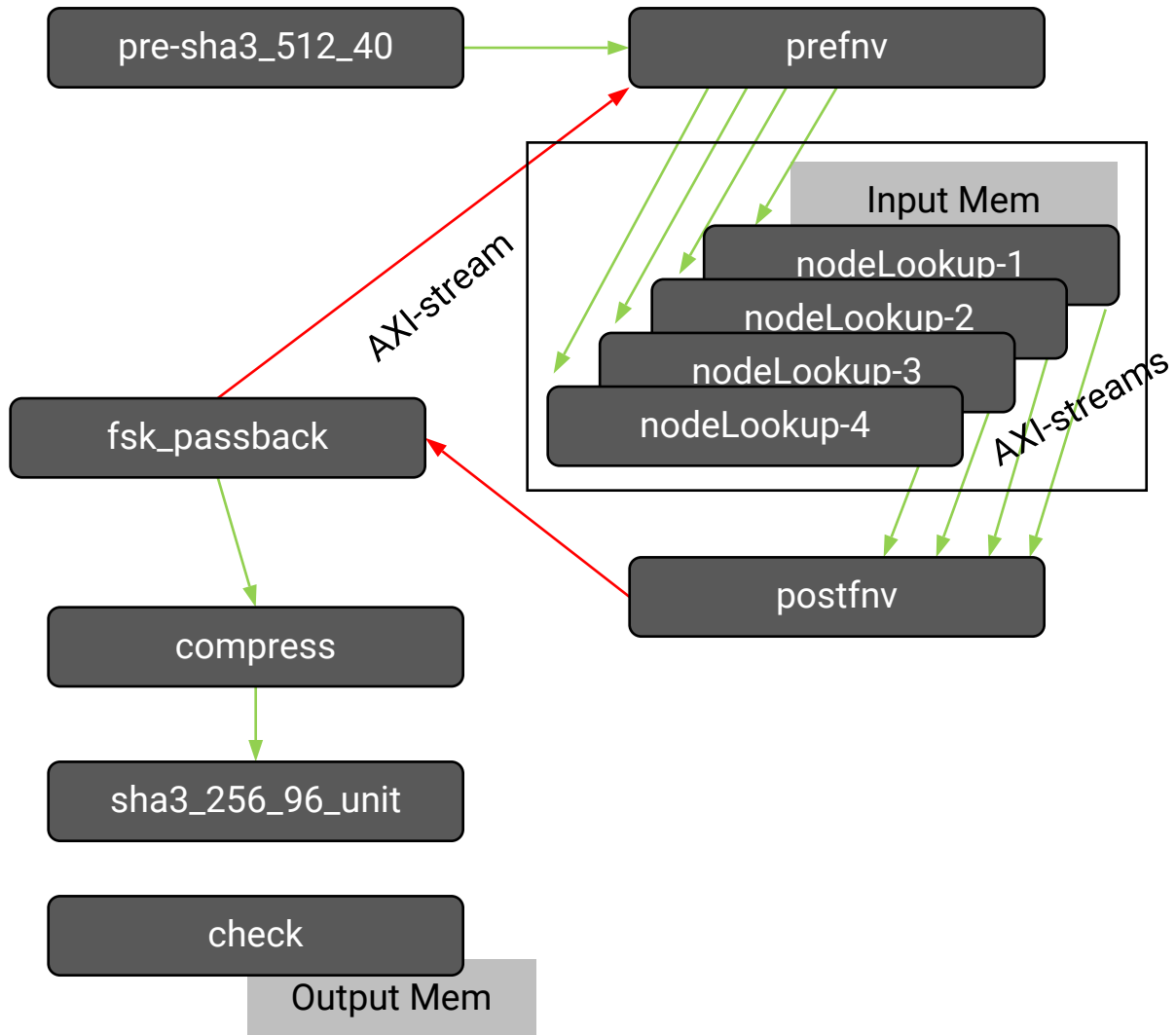
The free-running semantics described earlier greatly simplifies the implementation of a free-running PE, often simplifying the FPGA usage and routing resources required. It enables the design of a streaming pipeline design where the intermediate PEs can be free-running, thereby operating only on the input AXI4-Stream.

With any pipeline composition, when hardware replication is enabled (NCU is more than 1), the hardware contains as many replicated pipelines, and `compute()` jobs run on available pipeline slot. Thus, the application layer remains simple and automates running data through multiple pipelines in the hardware.

Streaming Network of PEs

Using stream variables between functions in the `compute()` scope, you can design an arbitrary network of PEs streaming data across the PEs. The body of the `compute()` method semantically describes a structural composition of PEs. It is unlike the procedural semantics in the C-language, and VSC allows software-emulation based validation of the `compute()` body semantics. An example using such a network is a design developed for Ethereum hashing - a popular algorithm used in cryptocurrency mining.

Figure 162: Streaming Network



The picture shows the system architecture of this design. It is a pipelined network of PEs connected by AXI4-Stream. There are four PEs, `nodeLookup-1` to `4` that read from global memory, and each of these also read from an input stream produced by the PE `prefnv`. The resulting AXI4-Stream from these four PEs are consumed by the PE `postfnv`.

There is an AXI4-Stream feedback loop from `postfnv` in `fsk_passback` and back to `prefnv`. This loop is expected to converge after iterating several times over data flowing through the AXI4-Stream. The entire system of PEs will deterministically start and stop execution for each `compute()` call.

Such streaming architectures are typically efficient in utilizing FPGA resources, and particularly lower in routing resources compared to using AXI4 M_AXI interfaces. Therefore, such architectures have the potential to achieve higher clock frequency and better accelerator performance.

This VSC model is written entirely in C++ and it captures the network with function calls in the `compute()` scope. The C++ model can be functionally validated in VSC using software emulation, without needing to compile any hardware. This enables early validation of the original design intent in the Vitis tools.



TIP: A less efficient way to compose a system is by creating multiple accelerators (different derivative classes from `VPP_ACC`), composing a pipeline in the application layer as described in [Multi-Accelerator Pipeline Composition](#).

Data Transfer Interface Considerations

In a system design, it is important to correctly specify the mode of data transfer between the accelerator and the host. The following sections provide more details on the design aspects.

Global Memory I/O

VSC allows two modes of transfers between the global memory and the accelerator interface as described in [Guidance Macros](#):

1. `DATA_COPY` - drop-in a data mover, an efficiently designed RTL IP, that will automate certain features like bursting and data width manipulation on M_AXI interface to global memory. On the accelerator side, this interface supports both sequential access of data as in the Vitis HLS `ap_fifo` interface, and random access of data as in the Vitis HLS `ap_memory` interface.
2. `ZERO_COPY` - connects the M_AXI interface from the global memory platform port to the accelerator

The following considerations are recommended for definition of the `compute()` function interface:

1. If the data size (all compute arguments) is too big to fit into the device DDR, split up the large `alloc_buf` into smaller chunks of computation over multiple send iterations.
2. It is generally recommended to use `DATA_COPY` and `SEQUENTIAL` access pattern, especially when the accelerator code does sequential input data access.
3. If the accelerator code does random access of the data, and the code cannot be modified to access the data sequentially, then use the random access pattern if the data fits into device RAM (BRAM or URAM). Otherwise use `ZERO_COPY`.

Sequential Access Pattern

For a PE port `data` that is accessed sequentially by the kernel code, use `ACCESS_PATTERN(data, SEQUENTIAL);`

VSC requires the size of the data at runtime. This must be done through the `DATA_COPY` macro. For example, `DATA_COPY(data, data[numData]);`

Both `data` and `numData` will have to be passed as arguments to the kernel, even if the kernel for example would not use `numData`, it still has to be provided as an argument. In general, `numData` can be any expression as long as it can be evaluated at runtime in terms of the kernel function arguments. The following example is allowed in the accelerator class declaration, though perhaps not very practical:

```
DATA_COPY(data, data[m * log(n) + 5]);
...
static void PE_func(int n, int m, float* data);
```

For both input and output data, the exact size of the `DATA_COPY` is important and has to exactly match the amount of data the kernel is reading. If the size does not match the design there can be functional issues like:

- A hang at runtime, if the kernel reads more data than provided by the application code, or
- the kernel will read garbage data, if the previous kernel call did not read all the data from a previous compute call

To prevent and debug this, you can use `ZERO_COPY` to make sure that the kernel code works properly.

Random Access Pattern

If the kernel has to access a `compute()` function argument, `data`, in a random fashion, use `ACCESS_PATTERN(data, RANDOM)`:



TIP: The random access pattern will require a local FIFO buffer which has size limitations imposed by BRAMs, which are typically in 32 Kb blocks. The on-chip memory will require as many BRAMs as the size of the user-defined argument.

Therefore, you must ensure that the data would fit in the on-chip FPGA memory. The kernel code must declare the data as a static array, for example:

```
ACCESS_PATTERN(data, RANDOM);
...
static void PE_func(int n, int m, float data[64]);
```

If the data size is accessed randomly and too big for on chip FPGA memory, you should use `ZERO_COPY(data)`.



IMPORTANT! Do not use an `ACCESS_PATTERN` macro together with `ZERO_COPY`.

The unit amount of data transferred between host and the global memory is not necessarily the same as the `DATA_COPY` size. It is actually determined by the size argument of `VPP_ACC::alloc_buf()` call. This size can be bigger than the data size needed for each kernel compute, for example when sending data for multiple `compute()` calls in one-shot. Thus, clustering PCIe data transfers say for `N` calls to `compute()` is easily done as follows:

```
send_while ... { ...
    clustered_buffer = acc::alloc_buf( N * size );
    for (i = 0; i < N; i++) { ...
        acc::compute(&clustered_buf[ i * size ] ...
    }
}
```

1. Allocate the appropriate data buffer size for the `N` compute calls
2. Call `compute()` `N` times where each call indexes into the clustered buffer

Compute Payload Data Type

The `compute()` data type also determines the data layout on the global memory and therefore will affect accelerator performance. To allow the kernel to access the data as fast as possible it is important to choose the appropriate data type for the `compute()` arguments. For example, if the kernel is processing integers and is required to process one integer every clock cycle (the HLS `II = 1`), then the interface can use an array of integers, such as:

```
static void compute ( int* A );
```

Consider another example with the following two coding styles that add four integers in every compute call.

<pre>// --- acc interface DATA_COPY(data, data[numData*4]); // --- application code int data[numData*4]; ... static void acc::compute(int* data, ...); // --- 4-cycle kernel code void PE_func(int* data, int numData, int *out) { for (int i=0; i < numData; i++) { int o = i * 4; out = data[o+0]+data[o+1]+data[o+2]+data[o +3]; } }</pre>	<pre>// --- acc interface DATA_COPY(data, data[numData]); // --- application code struct data_t { int i[4]; }; data_t *data; ... static void acc::compute(data_t* data, ...); // --- 1-cycle kernel code with packed data type void PE_func(data_t* di, int numData, int *out) { for (int i=0; i < numData; i++) { data_t data = di[i]; out = data.i[0]+data.i[1]+data.i[2]+data.i[3]; } }</pre>
--	---

The kernel adds up every four integers from the input array. The straightforward implementation on the left would need 4 clock cycles for each result, assuming one cycle per memory access. However, it would be better to pack all 4 integers into a single global memory access, as shown on the right. Therefore, in this case it is recommended to:

- Use a C-struct to pack all the data and pass to the kernel

- Ensure the correct data copy size is provided, using `DATA_COPY(data, data[numData]);`

Some other key points to note:

- Using `int* data[4]` will not pack the integers and will result in the same hardware as `int* data;`
- Typically packing more than 64 bytes (for a 512-bit M_AXI bus width) will not improve performance, but should also not degrade performance

Application Software Interface

Host Code Organization

VSC provides a simple template for application code development and managing software calls to the accelerator hardware. Regardless of the hardware composition this template provides a unified style of creating a C++ software API.

```
auto arg1BP = myACC::create_bufpool( .... ); (1)
....
myACC::send_while([= .... ]() // (2)
{
    int* arg1 = myAcc::alloc_buf<int>(arg1BP, .... ); // (4)
    ....
    myACC::compute( arg1, .... ); // (5)
    ....
    return ( while_cond ); // (3)
}
myACC::receive_all_in_order([= .... ]() // (6)
{
    ....
}
myACC::join(); // (7)
```

The structure of this template shown in this pseudo-code above has the following parts. Refer to [VPP_ACC Class API](#) for details of these elements.

- `create_bufpool()` - Creates a buffer pool for each pointer argument passed to the accelerator and provides the specification of the argument data (for example: input, output, and remote).
- `send_while()` - A thread to control the overall scheduling of jobs on the accelerator, providing data for each job by using a lambda function.

- `return ( while_cond );` - The body of the lambda function executes in a loop and must return a Boolean value. The `return` statement in the `send_while` body allows the user to continue running the loop as long as the value returned is true, and to stop the loop and exit the `send_while` thread when the value returned is false. A `return` statement could be `(+sent_value<MAX_SEND)` if `sent_value` was set to 0 before declaring and using the lambda function.
- `alloc_buf()` - Allocation of a memory buffer object from the buffer pool for the current loop iteration.
- `compute()` - The software call to schedule one job execution of the `compute()` function on the accelerator hardware.
- `receive_all_in_order()` - A thread that waits on the results from the scheduled jobs. This is another user-defined lambda function and executes in a loop as long as the `send_while()` thread runs.
- `join()` - Waits on the completion of the send and receive threads.

The following sections provide additional details for creating the application code.

VPP_ACC Class API

You can define a hardware accelerator derived from the `VPP_ACC` base class, and build the hardware and software interface with VSC. This section describes the software API provided by the `VPP_ACC` class.

Controlling the Accelerator

The `VPP_ACC` class API provides methods for scheduling jobs on the accelerator hardware, processing results, and other software controls at run time.

- **`send_while()`:**

This API executes the `SendBody` repeatedly until it returns a boolean false; the pseudo-code of the `send_while` is similar to `do{ bool ret=f() } while(ret==true);`

```
void send_while(SendBody f, VPP_CC& cc = *s_default_cc);
```

Table 75: `send_while()` arguments

Argument	Description
SendBody f	SendBody is a user-defined C++ lambda function. The lambda function captures variables from the enclosing scope. Notably all used buffer pool variables need to be captured. They should be captured by value. Using <code>[=]</code> will automatically capture those by value as well any other variable you might use inside the lambda function. Any variable which gets modified inside the lambda function needs to be passed by reference. For example <code>[=, &m]</code>. Passing variables by reference unnecessarily can result in degraded host code performance, but on the other hand, passing a large class object by value might lead to an unnecessary deep copy. The latter however is unlikely the needed for the send (or receive) functions. TIP: Any variable which needs to be passed by reference can be captured explicitly with <code>[=, &var]</code>.
VPP_CC& cc	Optional argument used for grouping CUs. For example it can be used to specify which multi-card cluster of CUs to use as described in CU Cluster and Multi-Card Support .

- **compute():**

As described in [The compute\(\) API](#), the `compute()` method is a special user-defined method in the derived `VPP_ACC` accelerator class definition which is used to represent the CU and contain the processing elements (PEs).

```
void compute(Args ...);
```

- a call to the hardware accelerator that schedules one job
- one or multiple `compute()` calls can be made inside the `SendBody` function
- each `compute()` call is non-blocking and will return immediately, but will block when the task pipeline is full.
- in the background, a `compute()` call will make sure that all its inputs get transferred to the device and then executed on any available CU
- once all `compute()` calls of an iteration have finished, the output buffers are transferred back to the host and a `receive_all` iteration will be started for that iteration
- The following conditions must be followed by the application code, and are asserted during software emulation
 - once a `compute()` call has been made, input buffers and file buffers cannot be modified anymore, and no more calls to `alloc_buf` or `file_buf` can be made in that iteration.
 - output buffers cannot be read or written until data is received in the corresponding `receive` iteration
- **receive_all_xxx():**

Executes a C++ lambda function repeatedly, either in order or ASAP, whenever a compute request completes to receive data results from the hardware accelerator. Exits when `send_while()` has exited and all iterations are received.

```
void receive_all_in_order(RecvBody f, VPP_CC& cc = *s_default_cc);
```

```
void receive_all_asap(RecvBody f, VPP_CC& cc = *s_default_cc);
```

Table 76: `receive_all_xxx()` arguments

Argument	Description
RecvBody f	RecvBody is a user-defined C++ lambda function. Refer to the explanation of lambda functions in <code>send_while()</code>
VPP_CC& cc	Optional argument used for grouping CUs. For example it can be used to specify which multi-card cluster of CUs to use as described in CU Cluster and Multi-Card Support .

- **`receive_one_xxx()`:**

As described in [Multi-Accelerator Pipeline Composition](#), this is used to receive one iteration of this accelerator inside the `send_while()` loop of another accelerator.

```
void receive_one_in_order(RecvBody f, VPP_CC& cc = *s_default_cc);
```

```
void receive_one_asap(RecvBody f, VPP_CC& cc = *s_default_cc);
```

Table 77: `receive_one_xxx()` arguments

Argument	Description
RecvBody f	RecvBody is a user-defined C++ lambda function. Refer to the explanation of lambda functions in <code>send_while()</code>
VPP_CC& cc	Optional argument used for grouping CUs. For example it can be used to specify which multi-card cluster of CUs to use as described in CU Cluster and Multi-Card Support .

- **`join()`:**

Wait for send and receive loops to finish.

```
void join(VPP_CC& cc = *s_default_cc);
```

Table 78: `join()` arguments

Argument	Description
VPP_CC& cc	Optional argument used for grouping CUs. For example it can be used to specify which multi-card cluster of CUs to use as described in CU Cluster and Multi-Card Support .

- **`set_ncu()`:**

Set the number of CUs the driver should use. Use this method before starting the send/receive loop to establish the number of CUs the `compute()` function should use.

```
void VPP_ACC::set_ncu(int ncu);
```

Table 79: `set_ncu()` arguments

Argument	Description
int ncu	The number of CUs specified (ncu) should be ($1 \leq \text{ncu} \leq \text{NCU}$) where NCU is the template parameter of <code>VPP_ACC< , NCU></code> , as described in User-Defined Accelerator Class .

- **get_ncu():**

Returns the number of CU currently used by the driver, as previously set by `VPP_ACC::set_ncu`, or if not modified, the NCU template parameter of `VPP_ACC< . . . . , NCU>`.

```
int VPP_ACC::get_ncu();
```

- **get_NCU:**

Returns the number of CUs implemented in HW (i.e. the value of the NCU template parameter provided when building hardware, and specified in the base class `VPP_ACC< . . . . , NCU>`).

```
int VPP_ACC::get_NCU();
```

Setup I/O for Computation

The API methods described here are used to setup input and output buffers to the hardware accelerator.

- **create_bufpool():**

Creates and returns an opaque class object to be used in other methods that require a buffer handle, such as `alloc_buf()`. Use before starting the send/receive loops.

```
VPP_BP VPP_ACC::create_bufpool(vpp::Mode m, vpp::FileXFer = vpp::none);
```


Table 80: `create_bufpool()` arguments

Argument	Description
<code>vpp::Mode m</code>	Can specify any of the following values to denote the data transfer type for each <code>compute()</code> argument: <code>vpp::input</code>: data transfers into the accelerator <code>vpp::output</code>: data transfers out of the accelerator <code>vpp::bidirectional</code>: data transfer into and out of the accelerator <code>vpp::remote</code>: data is resident only on the device memory connected to the accelerator, and not send or received by the host code
<code>vpp::FileXFer = vpp::none</code>	Can specify any of the following values to indicate the location of a file for data transfer: <code>vpp::p2p</code>: file is transferred over the P2P bridge to the accelerator. This works only on platforms that support the P2P feature, for example the U2 card with a connected smartSSD. <code>vpp::h2c</code>: file is transfer from a host CPU (connected file server) to the card over PCIe. This is standard for most Alveo cards connected to a host CPU over PCIe. <code>vpp::none</code>: uses regular buffer objects, not supporting a file transfer. This is the default value.

- **`alloc_buf()`:**

Returns a pointer to the buffer object. Use inside the send thread's lambda function.

```
void* VPP_ACC::alloc_buf(VPP_BP bp, int byte_sz);
```

```
T* VPP_ACC::alloc_buf<T>(VPP_BP bp, int numT);
```



IMPORTANT! The buffer gets allocated from the given buffer pool. The lifetime of the buffer is until the end of the matching receive iteration. At that point the buffer will automatically be returned to the buffer pool.

Table 81: `alloc_buf()` arguments

Argument	Description
<code>VPP_BP bp</code>	A buffer pool object returned by <code>create_bufpool()</code>
<code>int byte_sz</code>	Specifies the number of bytes for the requested buffer
<code>int numT</code>	Specifies the number of elements for the requested <code><T></code> array buffer

- **`file_buf()`:**

This method will map a given file, or part of the file to a buffer from the specified buffer pool object. Use inside the send thread's lamda function. The `file_buf()` method can be called multiple times to map multiple files (or file segments) to different locations in a single buffer.

The method returns a pointer to the buffer object (which is a host handle). The host cannot be used to read or write it.

```
void* VPP_ACC::file_buf(VPP_BP bp, int fd, int byte_sz, off_t
fd_byte_offset=0, off_t buf_byte_offset=0);
```

```
T* VPP_ACC::file_buf<T>(VPP_BP bp, int fd, int numT, off_t fd_T_index=0,
off_t buf_T_index);
```

Table 82: file_buf() arguments

Argument	Description
VPP_BP bp	A buffer pool object returned by <code>create_bufpool()</code> .
int fd	The file descriptor to read from or write to (or 0 when using <code>custom_sync_outputs()</code> as described below). In P2P mode the file is opened with <code>O_DIRECT</code> flag.
int byte_sz	Specifies the number of bytes for the requested buffer. In P2P mode, this must align to the file system block size (4 kB).
fd_offset	Offset in the file to read from/write to.
buf_offset	Offset in the buffer to write to/read from.
int numT	Specifies the number of elements for the requested <T> array buffer. In P2P mode, this must align to the file system block size (4 kB).
fd_T_index	The array index in the file to start reading from/writing to.
buf_t_index	The buffer index to start writing to/reading from.

Additional notes:

- The statement `T* buf = file_buf<T>(bp, fd, num, fd_idx, buf_idx);` is the same as `T* buf = (T*)file_buf(bp, fd, num*sizeof(T), fd_idx*sizeof(T), buf_idx*sizeof(T));`
- The actual size of the buffer will be adjusted as required. As a result the buffer returned by the last call needs to be used in the `compute()` call(s). Once used in a compute call, no more mappings can be added in that iteration.
- See `file_filter_sc` under **Startup Example** in [Supported Platforms and Startup Examples](#)
- `get_buf()`:

- Use inside the receive loop associated with a matching send loop. This returns the buffer object which was allocated in the matching send iteration.

```
void* VPP_ACC::get_buf(VPP_BP bp);
```

```
T* VPP_ACC::get_buf<T>(VPP_BP bp);
```

Table 83: get_buf() arguments

Argument	Description
VPP_BP bp	A buffer pool object returned by <code>create_bufpool()</code>

- **transfer_buf():**

This method is to be used in multi-accelerator composition, as described in [Multi-Accelerator Pipeline Composition](#), to transfer ownership of a buffer from one accelerator using a `receive_one_xxx()` method inside the `send_while` of another accelerator, to that other accelerator. This extends the lifetime of the buffer till the end of the receive iteration matching the current send iteration.



TIP: This is especially useful for `vpp::remote buffers`, because then the buffer will remain on the device and no copying or syncing will be needed.

```
void* VPP_ACC::transfer_buf(VPP_BP bp);
```

```
T* VPP_ACC::transfer_buf<T>(VPP_BP bp);
```

Table 84: transfer_buf() arguments

Argument	Description
VPP_BP bp	A buffer pool object returned by <code>create_bufpool()</code>

- **custom_sync_outputs():**

This method can be called in the body of a `send_while` loop, before the call to the `compute()` function. This lets you provide a custom sync function to sync output buffers back to the host application. It is useful when only some (and not all) output buffers data need to be transferred back from the hardware accelerator.



IMPORTANT! This will disable any automatic syncing of all output buffers.

```
void custom_sync_outputs(std::function<void()> sync_outputs_fn)
```

Table 85: `custom_sync_outputs()` arguments

Argument	Description
<code>sync_outputs_fn</code>	Specifies the custom sync function that will be called automatically for each iteration of the <code>send_while</code> loop when the compute tasks of the iteration have finished. When the <code>sync_outputs_fn</code> returns AND all requested <code>sync_output()</code> calls are complete, a receive will be triggered for the <code>send_while</code> loop iteration.

- **`sync_output()`:**

This method is to be called inside the `sync_output_fn` passed to the `custom_sync_outputs()` method. It will perform the requested sync in the background, returning a future which the caller can check for completion of the transfer.

```
std::future<void> sync_output(void* buf, size_t byte_sz, off_t
byte_offset = 0);
```

```
std::future<void> sync_output<T>(T* buf, size_t numT, off_t Tindex = 0);
```

Table 86: `sync_output()` arguments

Argument	Description
<code>buf</code>	Buffer pointer obtained as a capture from the <code>sendBody()</code> scope.
<code>byte_sz</code>	Specifies the number of bytes for the requested buffer.
<code>byte_offset</code>	Offset in the buffer to write to/read from.
<code>numT</code>	Specifies the number of elements for the requested <code><T></code> array buffer. In P2P mode, this must align to the file system block size (4 kB).
<code>Tindex</code>	The buffer index to start writing to/reading from.

- **`sync_output_to_file()`:**

This method is to be called inside the `sync_output_fn` passed to the `custom_sync_outputs()` method. It will perform the requested sync in the background, returning a future which the caller can check for completion of the transfer.

```
std::future<void> sync_output_to_file(void* buf, int fd, size_t byte_sz,
off_t fd_byte_offset = 0, off_t buf_byte_offset = 0);
```

```
std::future<void> sync_output_to_file<T>(T* buf, int fd, size_t numT,
off_t fd_T_index = 0, off_t buf_T_index = 0);
```

Table 87: `sync_output_to_file()` arguments

Argument	Description
<code>buf</code>	Buffer pointer obtained as a capture from the <code>sendBody()</code> scope.

Table 87: `sync_output_to_file()` arguments (cont'd)

Argument	Description
<code>fd</code>	The file descriptor to write to.
<code>byte_sz</code>	Specifies the number of bytes for the requested buffer.
<code>fd_offset</code>	Offset in the file to read from/write to.
<code>buf_offset</code>	Offset in the buffer to write to/read from.
<code>numT</code>	Specifies the number of elements for the requested <code><T></code> array buffer. In P2P mode, this must align to the file system block size (4 kB).
<code>fd_T_index</code>	The array index in the file to start reading from/writing to.
<code>buf_t_index</code>	The buffer index to start writing to/reading from.

- **`set_handle()`:**

Use inside the `sendBody` to identify any objects you want associated with the current iteration of the `send_while` loop.

```
void VPP_ACC::set_handle(intptr_t hndl);
```

```
void VPP_ACC::set_handle<T>(T hndl);
```

Table 88: `set_handle()` arguments

Argument	Description
<code>hndl</code>	Anything you might want to associate with the current iteration of the <code>send_while</code> loop. TIP: With the templated form any class which has a simple assignment/copy operator can be used as a handle.

- **`get_handle()`:**

Used inside the `RecvBody`, this method returns the handle of an object that was set (`set_handle()`) in the matching send iteration of the `send_while` loop.

```
intptr_t VPP_ACC::get_handle();
```

```
T VPP_ACC::get_handle<T>();
```

Special Data Transfer Models

This section describes certain specialized data transfers to and from the accelerator, such as parts of device (result) buffers, and different styles of file transfers allowing a single accelerator computation to simultaneously process multiple files.

Customized Transfer of I/O Sub-Buffers

Sometimes you do not want to sync the whole output buffer back to the host, and the sizes of what you want to sync back are also not known up front in the host code. A good example is compression. Especially if the compression algorithm is compressing multiple chunks into the same output buffer and you want to sync back only the exact compressed chunks from the output buffer. As shown in the following example, this can be done by registering a function which will control exactly what will be synced back once the data is available.

```
xfilter::custom_sync_outputs([=]()
{
    auto fut = xfilter::sync_output<int>(outSz, chunks, 0);
    fut.get();
    for (int chunk = 0; chunk < chunks; ++chunk) {
        xfilter::sync_output<int>(out, outSz[chunk], chunk * chunkSz);
    }
});
```

The `custom_sync_outputs()` method registers a callback function which will determine which buffer/sub-buffers will be synced back to the host. This method would have to be called inside the `send_while` body, before the first `compute()` call.



IMPORTANT! Calling `custom_sync_outputs()` disables any automatic transfer-back of output buffers.

Inside the callback function the user-defined code has full control of what is synced back and if there needs to be synchronization. The example above syncs back an `outSz` buffer. This buffer contains the sizes of the compressed chunks. The `sync_output` API returns a `std::future`. Calling `fut.get()` on that future will make the code wait for that sync to finish. Then the code transfers all chunks, with their exact sizes, back to the host. These calls will also return a future, but there is no need to synchronize for those syncs anymore. The System Compilation runtime layer will make sure that all those syncs have finished before starting the corresponding `receive_all` iteration.

File Transfer Modes

System Compilation mode also supports easy reading and writing to files as shown in the `file_filter_sc` in [Supported Platforms and Startup Examples](#). This can be enabled in the `create_bufpool()` method using either of the following modes as discussed in [VPP_ACC Class API](#):

- `vpp::p2p` mode : the file is transferred over the peer-to-peer PCIe bridge to the accelerator. This works only on platforms that support the P2P feature, for example the U2 card with a connected smartSSD.
- `vpp::h2c` mode : file is transfer from a host CPU (connected file server) to the card over PCIe. This is standard for most Alveo cards connected to a host CPU over PCIe.

This simple file-transfer switch (`vpp::p2p` and `vpp::h2c`) makes accelerator design portable across platforms. It is a simple matter to test a design for any platform using software emulation on a typical host CPU (connected to a file server) hosting the data files. However, eventually, the design must be compiled for a specific platform supporting P2P, such as the U2 card connected to a smartSSD, thereby allowing direct porting without changing the design sources.



TIP: Running hardware emulation or running on hardware will not work on an Alveo platform that does not support P2P.

The following example code demonstrates this:

```
auto inBP = my_acc::create_bufpool(vpp::input , P2P ? vpp::p2p :
vpp::h2c);
my_acc::send_while([=]() {
    if (P2P) o_flags |= O_DIRECT;
    int fd = open(fnm, o_flags, s_flags);
    DT* in = (DT*)xfilter::file_buf(inBP, fd, fsz);
    my_acc::compute(in, ...);
    ...
});
```

Based on the value of the `P2P` flag, this code will either do peer-to-peer (P2P) transfer of a host mapped NVMe device file, or it will do host file to device transfer (H2C). In the P2P case, the file will be loaded into the device memory directly from the NVMe device, without any data transfer through the host. In case of H2C, the System Compilation runtime layer will automatically transfer the file from the host to the device buffer (or vice versa for outputs).

To enable P2P the file has to be opened with `O_DIRECT` flag specified as shown in the example above. When in P2P mode, the host pointer, as returned by the call to `VPP_ACC::file_buf`, is a handle for the compute call argument, and cannot be read from or written to.

Multi-File Buffers

As described in [VPP_ACC Class API](#), you can make multiple calls to the `file_buf` method before calling the `compute()` method, to map multiple files, or multiple file segments into a single device buffer. The code example below shows small portions of multiple files being processed simultaneously by the accelerator in one `compute()` call.

```
void* VPP_ACC::file_buf(VPP_BP bp, int fd, size_t sz, off_t fos = 0, off_t
bos = 0)

xfilter::send_while([=, &total_out_size]()
{
    static int iter = 0;
    int* in;
    // collect all "chunks" input files into one "in" buffer
    for (int chunk = 0; chunk < chunks; ++chunk) {
        std::stringstream nm;
        nm << DATA << iter << '-' << chunk << ".orig";
        int ifd = open(nm.str().c_str(), rd_o_flags);
        assert(ifd > 2);
        in = xfilter::file_buf<int>(inBP, ifd, chunkSz, 0, chunk * chunkSz);
    }
});
```

```

    }
    // prepare output buffer to be able to hold all chunks
    int* out = xfilter::file_buf<int>(outBP, 0, chunks * chunkSz, 0);
    // output buffer to provide the actual filtered size of each chunk
    int* outSz = xfilter::alloc_buf<int>(outSzBP, chunks);
    ....
    xfilter::compute(chunks, chunkSz, in, out, outSz);
    ....
});

```

The code creates an input (`in`) buffer that holds these file segments, and an output (`out`) buffer that holds the processed output data. In every `send_while` iteration the `file_buf()` provides the `chunkSz` and read offset (`chunk*chunkSz`) for each file associated with the `in` buffer. In the subsequent call to `compute()` all those files segments will be written to the input or read from the output device buffers.

Note: The assignment `in = xfilter::file_buf<...>` is repeated in the for-loop so that the last returned pointer is assigned to `in`. This is important to follow while writing the application code.

Custom Transfer of Output files

You can also use custom sync for file buffers, in which an output buffer can be synced to a file descriptor in `custom_sync_outputs()`. As explained in [VPP_ACC Class API](#), you can do this by calling the `sync_output_to_file()` method as shown in the following example.

```

VPP_ACC::sync_output_to_file(void* buf, int fd, size_t byte_sz,
                             off_t fd_byte_offset = 0,
                             off_t buf_byte_offset = 0);

```

In this case files added by a call to `VPP_ACC::file_buf(bufPool, fd, sz)` will not get synced automatically. So a call to the `file_buf` API is not needed to actually add a file, but it is required only to return a host pointer. Adding a dummy file like this is the best approach:

```
VPP_ACC::file_buf(bufPool, 0, 0);
```

Here's an example code snippet:

```

my_acc::send_while([=]() {
    DT* out = (DT*)my_acc::file_buf(outBP, 0, 0);
    ...
    my_acc::custom_sync_outputs([=]() {
        ...
        auto fut = my_acc::sync_output_to_file(out, fd, sz, fd_offset,
        buf_offset);
        ...
    });
    ...
}

```

Accelerator Execution Models

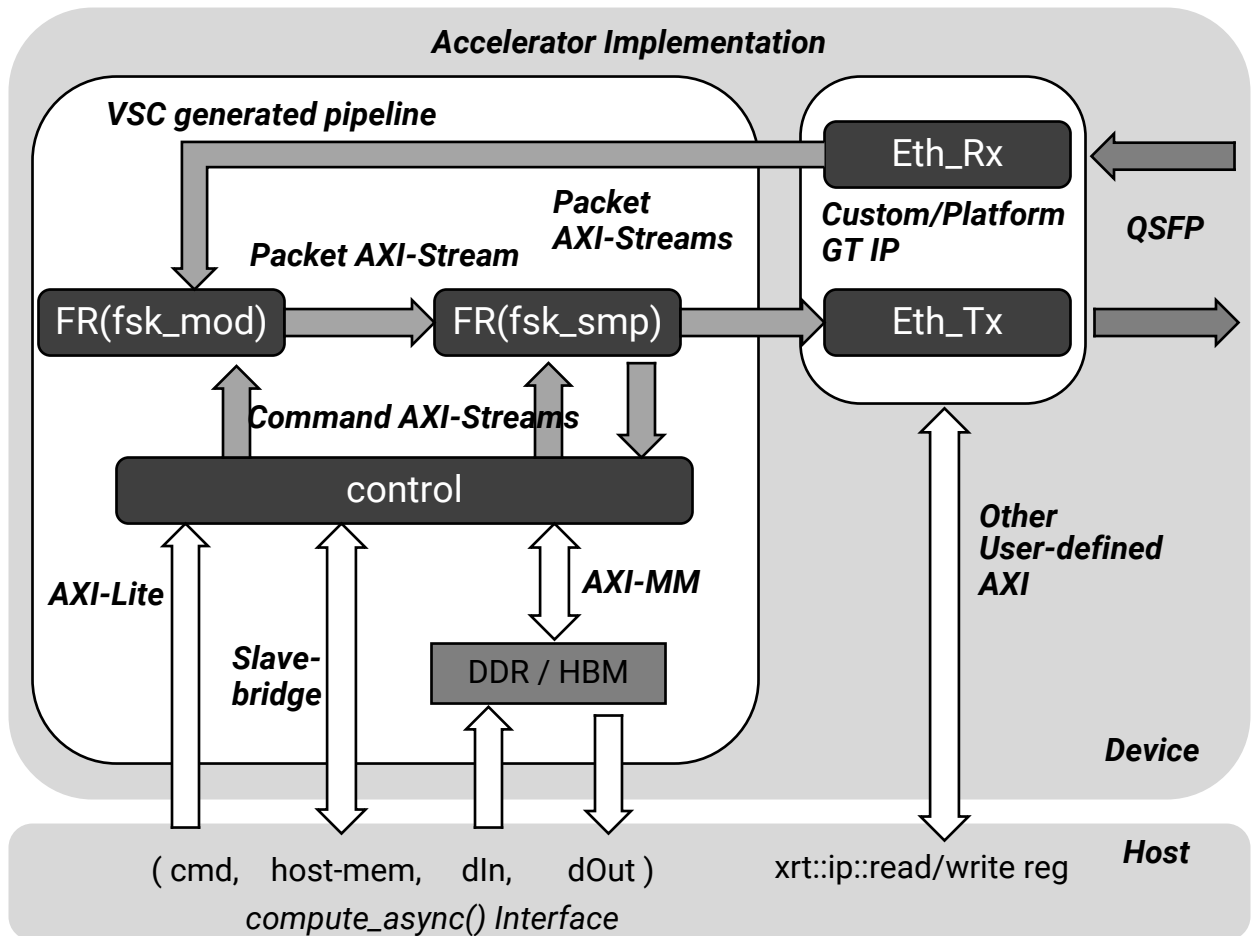
The following sections describe various styles of accelerator execution models. Each one is uniquely defines a system architecture that may used for an accelerator application domain.

Asynchronous Host Control of Accelerator

The VSC mode allows compilation of accelerators with CUs that contain user-defined hardware pipelines, as described in [Building Hardware](#). Such a pipeline is composed of PEs that connect to each other through AXI4-Stream and can also connect to platform ports that are AXI4 connections, such as global memory or IO interfaces such as an ethernet QSFP port. The platform will provide IP that translate such interfaces into AXI4-Stream ports which can be connected to PEs in the user-defined pipeline.

Using VSC such hardware pipelines can be easily configured to dynamically change processing behavior at runtime from an application running in the host CPU. The following describes how such an accelerator can be created. An example system composition is show in the picture given below.

Figure 163: Accelerator Implementation



X27506-120522

The `Eth_Rx` and `Eth_Tx` modules are typically platform IP that translate AXI4-Stream words into ethernet packets. These can also be custom IP with user-defined AXI4 interfaces.

The rest of the accelerator pipeline, shown in the white box, is created with VSC using AXI4-Stream connections. The PEs in the pipeline are user-defined functionality, such as packet processing like an internet protocol packet filter. In this example there is a pipeline created with two tasks, which are the PEs called `mod` and `smp`. Additionally, the system is composed of another control PE that has AXI4-Stream connections to these pipeline PEs. Example accelerator code is provided below, with the `.hpp` file on the left and the `.cpp` on the right.

<pre>// -- file: ETH.hpp -- #include "vpp_acc.hpp" class ETH : public VPP_ACC<ETH,1> { public: ZERO_COPY(dIn); ZERO_COPY(dOut); SYS_PORT(dIn, MEM_BANK0); SYS_PORT(dOut, MEM_BANK1); FREE_RUNNING(fsk_mod); FREE_RUNNING(fsk_smp); FREE_RUNNING(eth_rx); FREE_RUNNING(eth_tx); static void compute(int cmd, Pkt* dIn, Pkt* dOut); static void control(...) static void fsk_mod(...) static void fsk_smp(...) static void eth_tx(...) static void eth_rx(...) };</pre>	<pre>// -- file: ETH.cpp -- void ETH::compute(int cmd, Pkt* dIn, Pkt* dOut) { static IntStream dropS("drop"); static IntStream addS("add"); static IntStream reqS("req"); static PktStream smpS("smp"); static BitStream getS("get"); static IntStream sntS("snt"); static PktStream Ax("Ax", /*post-check=*/ false); static PktStream Bx("Bx", /*post-check=*/ false); static PktStream Cx("Cx", /*post-check=*/ false); control(cmd, dIn, dOut, dropS, addS, reqS, smpS, getS, sntS); eth_rx (Ax); fsk_mod(Ax, Bx, dropS, addS); fsk_smp(Bx, Cx, reqS, smpS); eth_tx (Cx, getS, sntS); }</pre>
---	---

In this example, five PEs are defined including the `eth_tx` and `eth_rx` which mock the platform IP behavior in receiving and transmitting words in the AXI4-Stream. The `compute()` scope implements the accelerator pipeline using AXI4-Stream connections between these PEs. The `control` PE can send command words on these streams and the task PEs (`fsk_mod` and `fsk_smp`) monitor these command AXI4-Stream and react by changing behavior. The `fsk_smp` PE reacts by sampling a requested number of packets back to the `control` PE. The `fsk_mod` PE reacts by adding a value to the packet data or by dropping packets that are being passed from `Eth_Rx` into `Eth_Tx`.

The pipeline PEs, `fsk_mod` and `fsk_smp`, are `FREE_RUNNING` as described in [Guidance Macros](#) because they are never-ending PEs driven to operation by the words in their input streams.

The `control` PE talks to the host CPU through two `SYS_PORT` connections for interface argument data pointers for input (`dIn`) and output (`dOut`), in addition to the scalar command argument (`cmd`). The `control` PE is not free-running and reacts to `compute()` calls from the host CPU. This system composition is entirely user-defined including the nature of the commands and corresponding PE functionality.

The host code snapshot is shown here and the entire example is available on GitHub.

```
// -- file: host.cpp --
#include "vpp_acc_core.hpp" // required
#include "ETH.hpp"
int config_sample(int sz)
{
    printf("main: sample %d\n", sz);

    Pkt* sample = (Pkt*)ETH::alloc_buf(sz * sizeof(Pkt), vpp::output);
    Pkt* config = (Pkt*)ETH::alloc_buf(sizeof(Pkt), vpp::input);
    config[0].dt = sz;

    auto fut = ETH::compute_async(cmd_sample, config, sample);
    fut.get();
    print_sample(sample, sz);
    int pkt_nr = sample[0].nr;
    ETH::free_buf(config);
    ETH::free_buf(sample);
    return pkt_nr;
}
```

The job commands issued by the VSC host code, specifically using the `compute_async()` API, enables the control PE to translate the command and in-turn pass configuration words to the pipeline PEs through the command streams. This snapshot shows a user-defined API that issues a packet sampling command. A `sample` buffer of a required `sz` is allocated at run time, and the `compute_async()` call will trigger the control PE to capture `sz` number of packets and return the words back to the host. The `fut` returned by `compute()` is blocking in the host code until the results are available. However, the `compute_async()` as name denotes is an asynchronous call that triggers the accelerator. Once the sample words are returned and processed by the host and the corresponding buffers can be freed.

Because host control in this case is not a continuous pipeline of compute jobs, and only an occasional, non-timing critical job, the `send_while/receive_all` thread will not manage this. Instead, the synchronization is application managed using the `compute_async()` API defined in `vpp_acc_core.hpp`.



IMPORTANT! This `vpp_acc_core.hpp` header file needs to be included only in the host code file, and before any `vpp_acc.hpp` is included.

With VSC such hardware pipelines can be composed and be controlled asynchronously from a host CPU. One of the applications for such an accelerator is a packet processing accelerator on a NIC card. For example the X3 Hybrid platforms provides ethernet transmission and reception IPs which convert ethernet packets arriving at the QSFP ports into AXI4-Stream, through a MAC interface. Furthermore, a NIC interface allows the accelerator to provide data to a connected host CPU over PCIe, or a Host-Memory access may be used for direct host CPU memory access over PCIe. Using VSC, the accelerator packet processing pipeline can be composed on the PL and can be controlled by a CPU asynchronously over PCIe.

CU Cluster and Multi-Card Support

If a host machine has only one accelerator card installed, VSC will try to use that card. There will be a fatal error if the card does not match the platform that the design was compiled for. However, on a host machine which has multiple cards installed, VSC will by default pick the first card that exactly matches the platform the design was compiled for. The host code can override the default in two ways:

- Setting environment variable `XILINX_SC_CARD` to the desired `<cardIndex>`.
- From the host code, call the following API before making any other calls.

```
VPP_ACC call: my_acc::add_card(<cardIndex>)
```



TIP: With multiple cards installed, if there is any mismatch between the names of installed platforms and the platform the design was compiled for VSC will not identify a default card. In such a case you must specify the card index as shown above. For example, platform `xilinx_u2_gen3x4_xdma_gc_base_2` will not match a design compiled for `xilinx_u2_gen3x4_xdma_gc_2_202110_1`, even though the platforms are compatible.

As shown in the `sysc_multi_card` example in [Supported Platforms and Startup Examples](#), if the host has identical accelerator cards installed you can use multiple cards to run your VSC accelerator. This is supported in a mode where all CUs of any given card are running as a separate compute cluster as explained below.

The separate compute cluster mode is useful for performance improvement in scenarios like the U2 card with a local smartSSD. This example code show below creates CU-clusters and assigns a card to each of them. Then, the user code can perform data selection based on the index-*i* to ensure that the subsequent `compute()` job will automatically use the card-*i* because the selected data-*i* resides on the same SSD.

```

VPP_CC* cuCluster = new VPP_CC[ncards];
for (int i = 0; i < ncards; ++i) {
    my_acc::add_card(cuCluster[i], i);
}
for (int i = 0; i < ncards; ++i) {
    my_acc::send_while(
        [=]() -> bool {
            ... // data-i selection
        }, cuCluster[i]);
    my_acc::receive_all_in_order(
        [=]() {
            ...
        }, cuCluster[i]);
}
for (int i = 0; i < ncards; ++i) {
    my_acc::join(cuCluster[i]);
}

```

Multi-Accelerator Pipeline Composition

In VSC you can also create multiple accelerators with different functionality in a single `.xclbin` and runtime environment. With such composition you can create a pipeline, where two or more VSC accelerators operating of different data sets in a pipelined fashion, as shown in the `sysc_compose` example in [Supported Platforms and Startup Examples](#). There are two possible use models described in the following sections.

ACC1-CPU-ACC2 Pipeline

This model defines a pipeline of tasks where the first accelerator computes an intermediate result which is then processed by a host application, and then further processed by a second accelerator. The output buffer of the first accelerator needs to be modified (modify while copying) and then passed to the second accelerator. Example code is provided below.

```

auto inBP      = my_acc1::create_bufpool(vpp::input);
auto tmpoBP    = my_acc1::create_bufpool(vpp::output);
auto tmpiBP    = my_acc2::create_bufpool(vpp::input);
auto outBP     = my_acc2::create_bufpool(vpp::output);
my_acc1::send_while(
    [=]()->bool {
        int* in = my_acc1::alloc_buf<int>(inBP, inSz);
        int* tmp = my_acc1::alloc_buf<int>(tmpoBP, tmpSz);
        my_acc1::compute(in, tmp, ...);
        ...;
        return ...;
    });
my_acc2::send_while(
    [=]()->bool {
        int* tmp2 = my_acc2::alloc_buf<int>(tmpiBP, tmpSz);
        bool cond = my_acc1::receive_one_in_order( // or receive_one_asap
            [=]() {
                int* tmp1 = my_acc1::get_buf<int>(tmpoBP);
                ...; // tmp1 -> copy and modify -> tmp2
            });
        if (!cond) return false;
        int* out = my_acc2::alloc_buf<int>(outBP, outSz);
        my_acc2::compute(tmp2, out, ...);
        ...;
        return true;
    });
my_acc2::receive_all_in_order(
    [=]() {
        int* out = xfilter1::get_buf<int>(outBP);
        ...;
    });
my_acc1::join();
my_acc2::join();

```

This code uses the dedicated `receive_one_in_order` API within the scope of the `send_while` loop of the second accelerator, `my_acc2`. The `receive_one_in_order` or `received_one asap` APIs requires a user-defined lambda function body, as discussed in [VPP_ACC Class API](#).

In this case, the `my_acc1::receive_one_in_order` (or `receive_one_asap`) will wait for the next job in-order (or asap) to finish, and then execute the lambda function body. Because it is called from the `send_while` of the second accelerator, `my_acc2` computes in lock-step with the results generated from the first accelerator `my_acc1`. This API returns a boolean value which will be true when the `send_while` loop has exited and there are no more jobs to be received.

ACC1-ACC2 Pipeline

This model defines a pipeline of tasks where an output buffer of the first accelerator can be used directly (without any host CPU synchronization) by the second accelerator. As described in [VPP_ACC Class API](#), the `transfer_buf()` API takes a buffer pool object and returns the buffer corresponding to the correct iteration. Using this API allows seamless transfer of the result buffer from the first to the second accelerator without needing to copy data. VSC runtime automatically manages the re-use of such transferred buffers across accelerators and job iterators.

The following is a code example:

```
auto inBP      = my_acc1::create_bufpool(vpp::input);
auto tmpBP     = my_acc1::create_bufpool(vpp::remote);
auto outBP     = my_acc2::create_bufpool(vpp::output);
my_acc1::send_while(
    [=]()->bool {
        int* in = my_acc1::alloc_buf<int>(inBP, inSz);
        int* tmp = my_acc1::alloc_buf<int>(tmpBP, tmpSz);
        my_acc1::compute(in, tmp, ...);
        ...;
        return ...;
    });
my_acc2::send_while(
    [=]()->bool {
        int* tmp;
        bool cond = my_acc1::receive_one_in_order( // or receive_one_asap
            [=, &tmp]() {
                tmp = my_acc1::transfer_buf<int>(tmpBP);
            });
        if (!cond) return false;
        int* out = my_acc2::alloc_buf<int>(outBP, outSz);
        my_acc2::compute(tmp, out, ...);
        ...;
        return true;
    });
my_acc2::receive_all_in_order(
    [=]() {
        int* out = xfilter1::get_buf<int>(outBP);
        ...;
    });
my_acc1::join();
my_acc2::join();
```

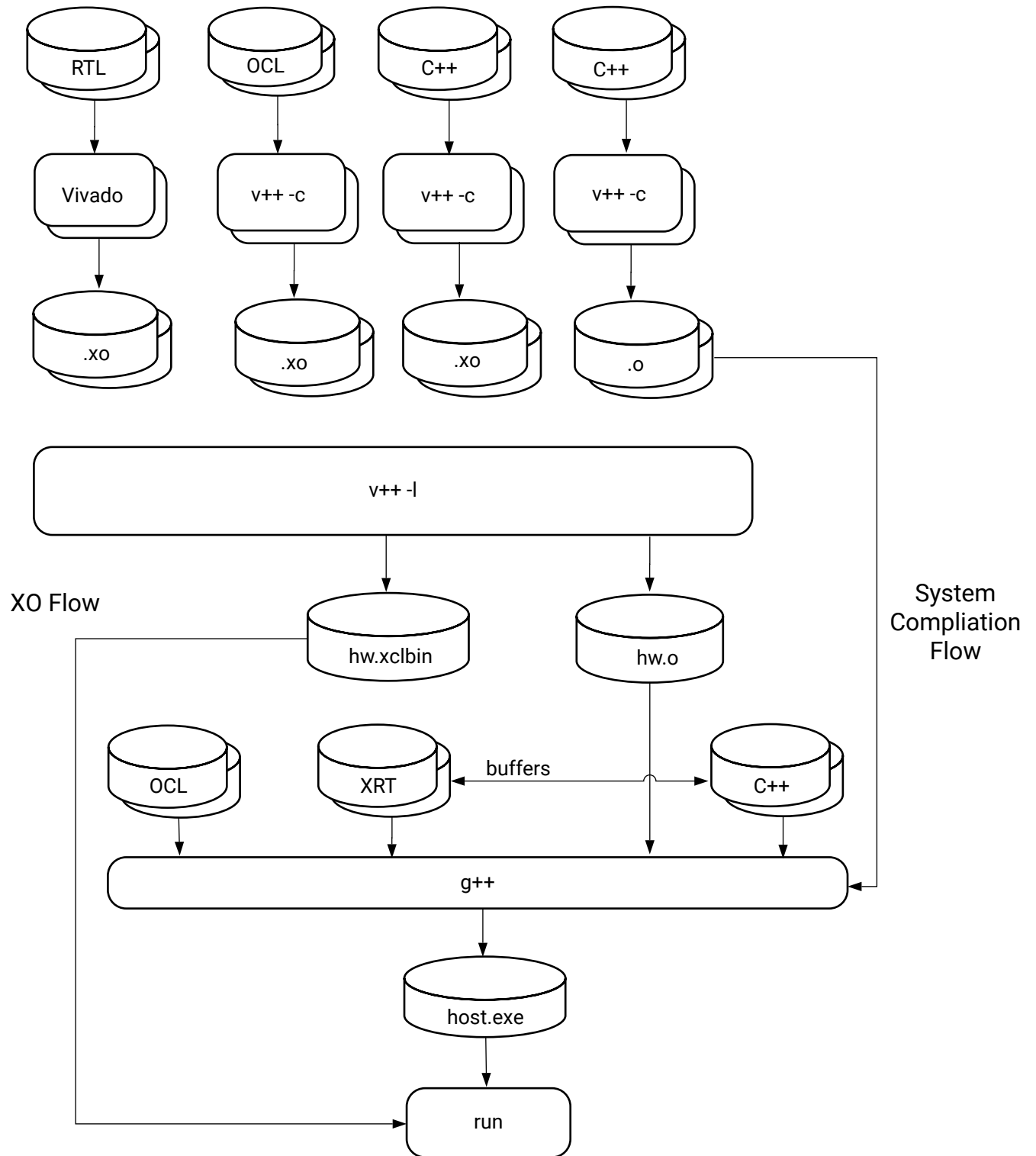
Interoperability with XO and RTL Kernels



IMPORTANT! *There is no software emulation support for the XO design flow described here.*

Vitis System Compilation mode can also be used with standard Vitis kernels (.xo) and the application acceleration design flow. Therefore, you can use VSC with Vitis HLS C++ kernels, as described in *Vitis High-Level Synthesis User Guide* ([UG1399](#)), and RTL kernels, as described in [RTL Kernel Development Flow](#).

Figure 164: XO Flow



X26762-060322

Given a VSC accelerator code, `sc_acc.cpp`, and a Vitis kernel, `kernel.xo`, the following would be the command lines required to build the project:

```
v++ -c sc_acc.cpp -o sc_acc.o
v++ -l sc_acc.o kernel.xo -o hw.xclbin # also produces hw.o

g++ -c host.cpp -o host.o
g++ -l -o host.exe host.o sc_acc.o hw.o -lvpp_acc -lxrt_hw -lpthread
```

In the scenario as it has been described, the `kernel.xo` is already compiled. So, the VSC accelerator code (`sc_acc.cpp`) needs to be compiled as shown in the first `v++` command above. The output of this is the object file `sc_acc.o`. The `v++` command recognizes the VSC accelerator code, and compiles it properly into the object file.

The `v++ --link` command is then run to link the `.xo` and `.o` files into the `.xclbin`. This produces a `hw.o` file in addition to the `hw.xclbin` file. The `hw.o` file is required for compilation of the host application by the `g++` command as is shown in the next steps.

The `g++` command compiles the host code, and then links the various object files (`host.o`, `sc_acc.o`, and `hw.o`) to produce the `host.exe` executable. Notice the addition of the `vpp_acc` and the `xrt_hw` libraries to the `g++` command line as they are required for the linking process.

In a standard Vitis flow, using native-XRT or OpenCL code, the user host code has to explicitly load the `.xclbin`. In a mixed VSC-XO flow the user host code can still load the `.xclbin` explicitly, but is not required. Instead, the device-ID can be obtained by calling API `vpp::sc::get_xrt_device()`.

Buffers objects can be easily passed between the native XRT and VSC APIs. The two possible buffer transfer scenarios are described in the following sections.

Passing Buffer from XO Kernel to VPP_ACC

The following API can be used to pass an `xrt::bo` over to a VSC accelerator:

```
std::shared_ptr<...> vpp::sc::set_xrt_bo(xrt::device, xrt::bo, void* buf,
int memBank);
```

This `buf` can then be used in the VSC accelerator `compute()` calls as long as the returned shared pointer keeps references.



TIP: Such buffers will not be auto-synced from/to the device, unlike buffers obtained by `vpp_acc::alloc_buf`.

Following is an example use case:

```
xrt::memory_group memA = 0;
auto Abo = xrt::bo(device, bytes, memA);
auto Abuf = Abo.map();
auto Aref = vpp::sc::set_xrt_bo(device, Abo, Abuf, memA);
auto Bbp = my_acc::create_bufpool(vpp::input);
my_cc::send_while([=]()->bool {
    auto Bbuf = my_acc::alloc_buf(Bbp, bytes);
    my_acc::compute(Abuf, Bbuf, ...);
    ...
})
```

Passing a Buffer from VPP_ACC to XO Kernel

To go from the VSC accelerator buffer to an `xrt::bo` use the following code:

```
xrt::bo vpp::sc::get_xrt_bo(void* buf);
```



TIP: Be aware though that the lifetime of such a buffer will be fully controlled by the VSC runtime layer. The lifetime of a VSC buffer starts from the time it gets allocated in an iteration of the `send_while` loop, using `vpp_acc::alloc_buf`, until the end of the corresponding `receive_all` iteration.

For example:

```
Acc::send_while([=]()->bool {
    auto* Abuf = Acc::alloc_buf(Abp, size);
    xrt::bo Abo = vpp::sc::get_xrt_bo(Abuf);
    auto run = xo_kernel(Abo, ...);
    run.wait();
    ...
})
```

Debugging and Validation

This section describes various types of debugging and validation features available for use with the VSC mode, which includes software emulation, hardware emulation, and profiling the actual execution on the physical accelerator card. All of these methods use the standard Vitis interfaces as described in [Section V: Simulating the Application with the Emulation Flow](#) and [Section VI: Profiling and Debugging the Application](#).

Software Emulation

VSC uses a single source C++ model which can be functionally validated fast. The software emulation target (`-t sw_emu`) must be specified during `v++` compile step. For this target, VSC will compile the accelerator and application source files using C-compilation and will not run a hardware compilation. The Vitis HLS tool will not be run to create RTL, and the object files are linked using `g++`.

The following is an example code of an accelerator with two PEs, `ldSt` and `fsk`. The `ldSt` PE writes `sz` words in the AXI4-Stream `L`, and the `fsk` PE simply copies those words back in the feedback stream `S`. The `fsk` function body reads one word from `L` and writes one word to `S`. The `fsk` PE is marked free-running and therefore is agnostic to `sz`.

```
void compute(...) {
    hls::stream<T> L, S;
    ldSt(..., L, S); // S is feedback
    fsk(L, S);      // fsk is free-running
}
void ldSt(..., hls::stream<T>& L,
           hls::stream<T>& S) {
    for (int i=0; i<sz; i++) {
        L << input[i];
    }
    // Error-1: non-empty stream (i < sz-1)
    // Error-2: deadlock stream (i < sz+1)
    for (int i=0; (i < sz); i++) {
        S >> output[i];
    }
}
void fsk(hls::stream<T>& L,
         hls::stream<T>& S) {
    T word;
    L >> word;
    S << word;
}
```

Because the C++ source is entirely built with C-compilation the process is very fast. Additionally, VSC checks the model against certain hardware behavior semantics of the VSC C++ model:

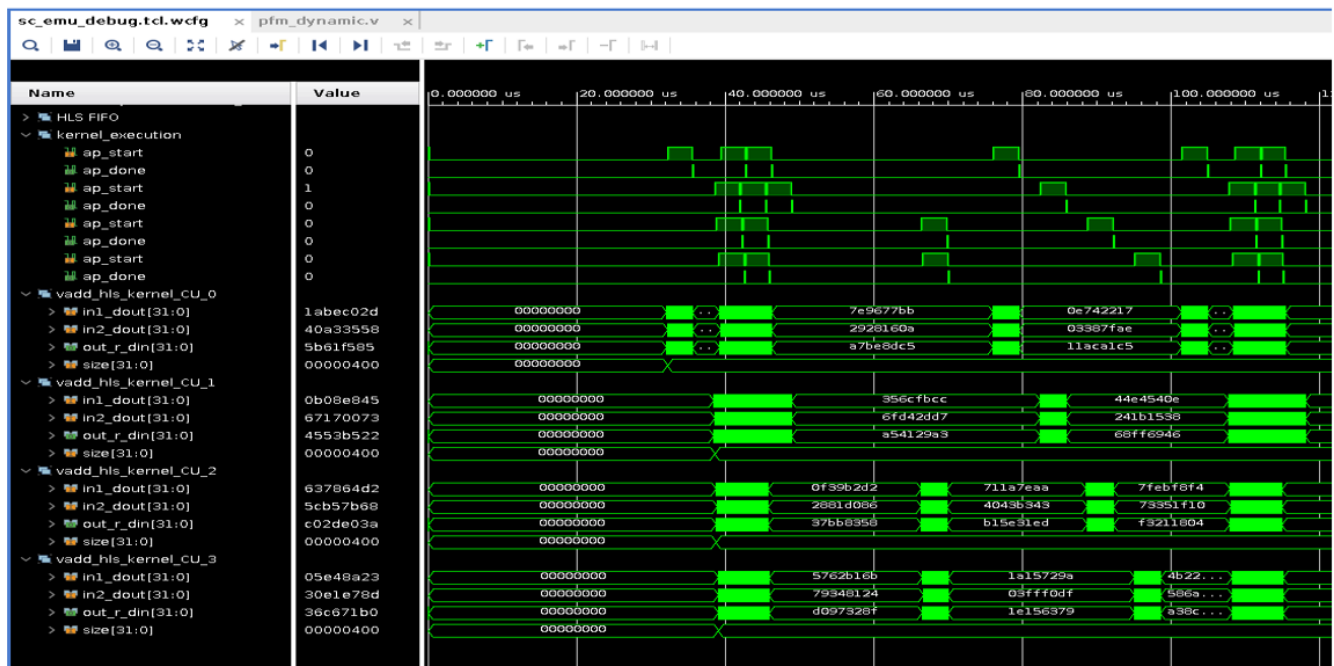
- Automatic runtime assertion for host-device data movement, in the application layer
 - Input data buffer might not be written after it is synced to the device
 - Output data buffer might not be read before the result is synced from the device
- The CUs and PE are executed in parallel, to model the hardware semantics. Therefore,
 - feedback connections, that are non-procedural C++ code, can be functionally validated
 - free-running PEs can be functionally validated along with regular PEs
- Certain erroneous hardware behavior can be detected
- Error-1: If the second loop in `ldSt` was using "`i < sz-1`" then it will lead to non-empty AXI4-Stream '`S`', which will be asserted
- Error-2: If the second loop in `ldSt` was using "`i < sz+1`" then it will lead to a deadlock as `ldSt` is expecting one more word than what is written to stream '`S`'. This scenario will be captured as a hang in this early C-based validation.

Hardware Emulation

VSC mode when compiled for the hardware emulation target (`-t hw_emu`) will generate RTL from the accelerator sources and run RTL simulation along with the application layer code. To view simulation waveforms the user can enable the following switch in the `xrt.ini` file:

```
[Emulation]
debug_mode=gui
```

Figure 165: Simulation Waveform View



The picture above shows the waveforms viewer in the Vivado XSim interface. By default, VSC will create grouped waveform objects corresponding to the `compute()` interface. The kernel code must be compiled with `-g` flag, otherwise the grouping will error out in Vivado XSim. In this design, there are four instances (NCU=4) of the accelerator enumerated as grouped objects `vadd*CU_0` through `vadd*_CU_3`. Each of these groups further contains the signals that correspond to `compute()` arguments, `in1`, `in2`, `out`, and `size`. The group `kernel_execution` is also auto-created and contains the `ap_start` and `ap_done` signals for each CU instance.



TIP: In hardware emulation, the timing of the start signals is not timing accurate with respect to realistic hardware behavior. This is because interactions of the host with the emulation model (data transfer latency over PCIe or with DDR/HBM memories), and application will not reflect real-time hardware execution behavior. However, the timing from the start to a stop of a `compute()` call is cycle-accurate.

Profiling Execution on the Card

VSC Profiling allows getting real-time statistics of the application execution along with the accelerator card. There are three parts to the profiling statistics:

1. profile summary: summary tables with runtime statistics from an application execution
2. application traces: A graph of timeline traces as observed from the application layer
3. hardware traces: A graph of timelines traces of the hardware interfaces within the accelerator

Profiling can be enabled with these settings in the `xrt.ini` file:

```
[Debug]
sc_profile=true
xrt_trace=true
```

When the application is run on the card, at the end of the execution, the file `xrt.run_summary` will be created for viewing in the Vitis Analyzer tool as described in [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

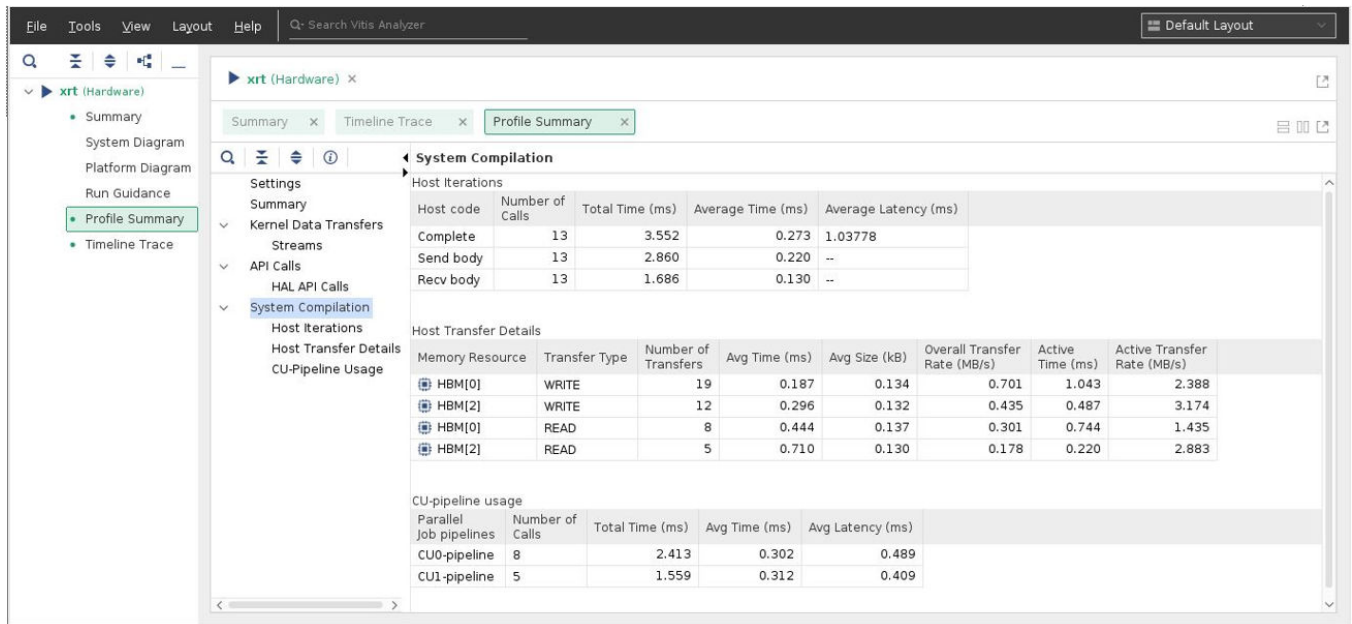
Profile Summary

The `xrt.run_summary` file can be loaded into Vitis Analyzer and an example of the results is shown in the picture. The application run summary is presented under the System Compilation section of the Profile Summary tab. It contains the following three tables,

1. Host Iterations: This table shows a summary of the send and the receive threads iterations, and the number of compute() calls, issued by the application layer and useful runtime statistics. In this example, 13 compute jobs were run with a total runtime of 3.552 milliseconds.
2. Host Transfer Details: This table captures the number of data transfers between the application code (running on a host) and the device. In this example, data is transferred using the HBM banks 0 and 2. Several average runtime statistics on the data transfers are shown.
3. CU-pipeline usage: This table captures the activity in the compute unit pipelines. Each row represents a CU in the hardware and average run time statics are shown, In this example, the 13 compute jobs were distributed with 8 on CU0 and 5 on CU1.

Note: These columns have tooltips in Vitis Analyzer. If the mouse hovers over any column header in the table, a pop-up will show the explanation of the column. For example, hovering over "Average Time (ms)" will show "= Total time (in milliseconds) / Number of calls."

Figure 166: Profile Summary

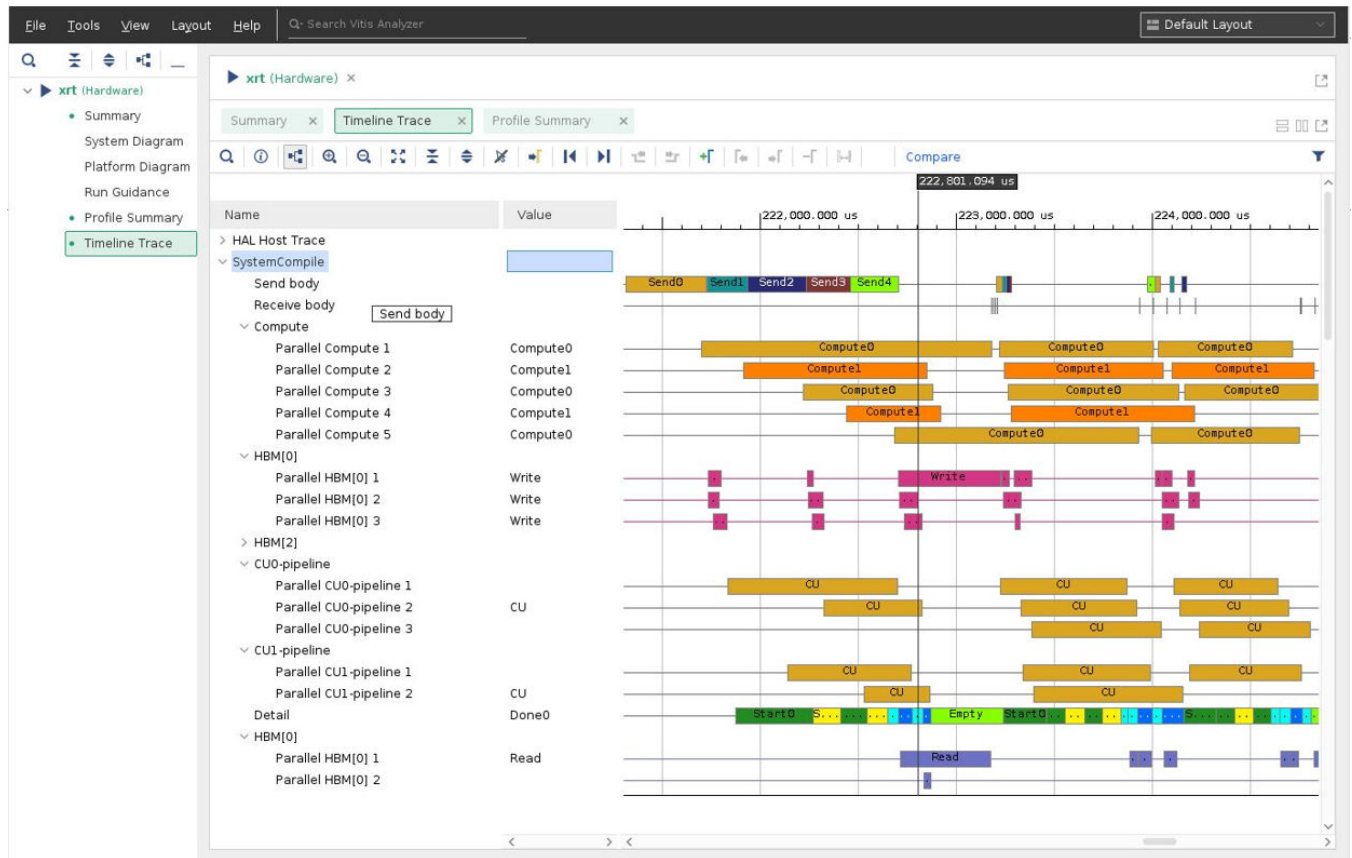


Timeline Trace

Profiling also shows a timeline trace for various phases of run in the application layer, as shown in the picture. This is presented under the "SystemCompile" tab in Vitis Analyzer. These timeline trace rows are explained below.

- **Send body:** each box captures the start and end of a send iteration, including the execution of any user-written code in the send lambda function.
- **Receive body:** each box captures the start and end of a receive iteration, including any user-written code. In this example, the receive iteration is very fast (tiny in the picture) compared to that of send.
- **Compute:** each box captures the start of a compute call until the application receives a result (a `get()` on the C++ future).
- **Input transfers:** these are input data transfers for each compute job. In this example, they are writes to the HBM memory bank.
- **CU-pipeline:** the application layer maintains multiple software job pipelines, typically more than the number of CUs in the hardware, for efficient execution. Each row is a separate software pipeline, and it captures every job execution from the start of submission in this pipeline until the completion of the job by the hardware.
- **Detail:** this row captures some key events in the execution model, primarily the issuance of the start signal and receiving the done for each compute call.
- **Output transfers:** these are output data transfers for each compute job. In this example, they are data reads from the HBM memory bank.

Figure 167: Timeline Trace



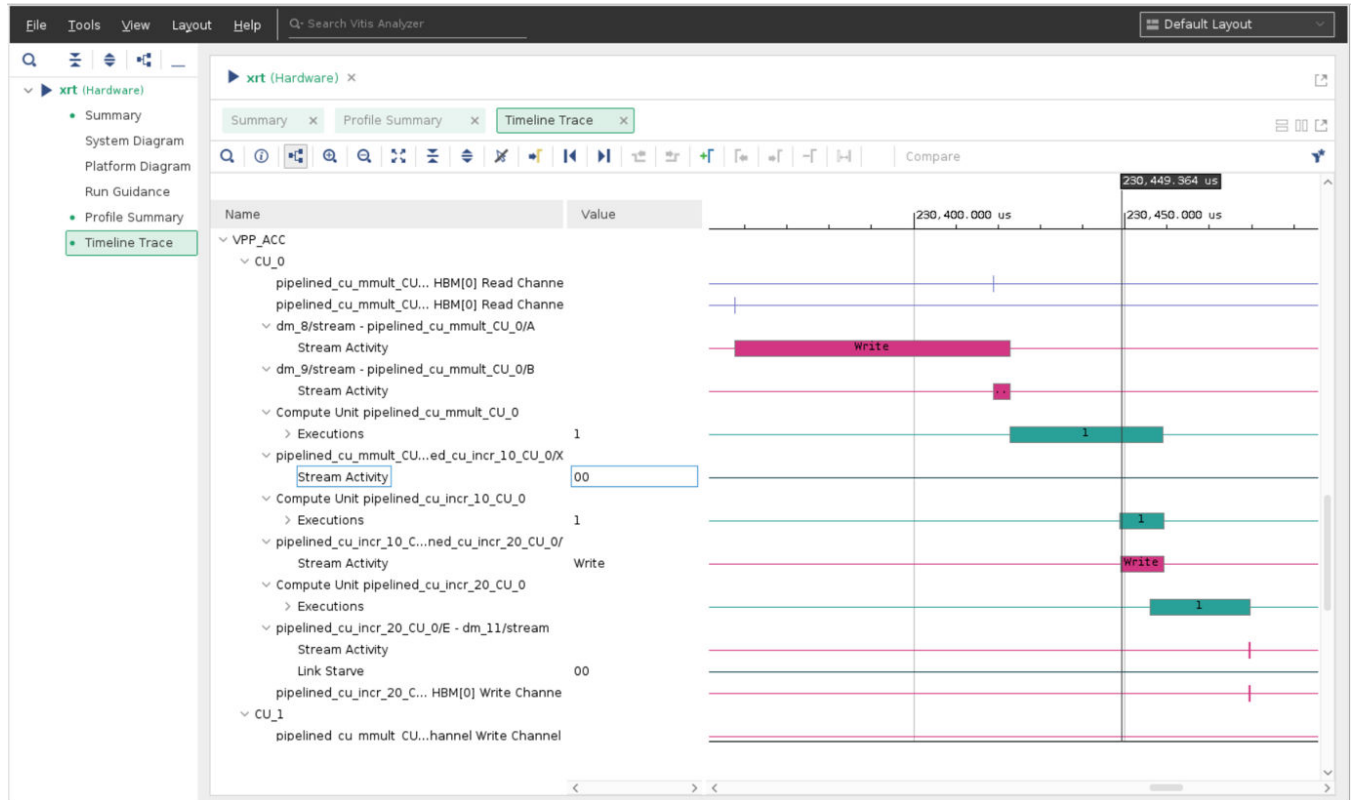
Hardware Event Tracing

You can also profile hardware events within the accelerator system composition. Particularly, hardware events on the AXI4 ports of the platform connections and PE interfaces can be captured and presented as a timeline trace.

The picture below is for an example of an accelerator with two CUs, where each CU is a pipeline of three PEs in a chain: `mmult` → `incr_10` → `incr_20`. As seen in the picture, the rows are topologically organized to show the sequence of events following the order of the pipeline.

The picture shows the execution of a single compute call. The first two rows show two input arguments being read from a HBM bank. The `DATA_COPY` and `SEQUENTIAL` access macros allow a data mover (`dm_8` and `dm_9`) to stream this data to the PE `mmult` which then executes. The `mmult` then writes to a stream connected to the subsequent PE `incr_10`, which in turn writes to another stream to the PE `incr_20`. This PE finally triggers the output data mover streams which eventually write results back to the HBM bank.

Figure 168: HW Event Trace



Profiling in VSC mode can be enabled with these settings,

1. The accelerator class requires these macros,
 - a. `PROFILE_KERNEL` ("PE function names"): To enable tracing the start and stop of every PE execution
 - b. `PROFILE_PORT` ("PE argument names"): To enable profiling of AXI ports for any of the PE arguments
 - c. The keyword `all` can be used for profiling all PEs or all ports.
Note: Using `all` on accelerator with many AXI ports (every PE argument and platform port connection) can cause Vivado routing issues, preventing the design from closing timing. Then, it is recommended to create hardware traces on specific ports as required.
 - d. Modifying these macros will trigger a full Vitis compile and linking including Vivado place and route.
2. Hardware traces can be enabled with this setting in the `xrt.ini` file:

```
[Debug]
device_trace=[fine|coarse]
```


PROFILE_KERNEL Examples

1. To enable profiling for all kernels:

```
PROFILE_KERNEL( "all" );
```

2. To enable profiling for all instances of *hls_kernel* function:

```
PROFILE_KERNEL( "hls_kernel" );
```

3. To enable profiling for compute unit 0 and 2 of *hls_df_kernel* function:

```
PROFILE_KERNEL( "hls_df_kernel[0] hls_df_kernel[2]" );
```

PROFILE_PORT Examples

1. To enable profiling for all ports of all kernels:

```
PROFILE_PORT( "all" );
```

2. To enable profiling for all ports of *hls_kernel* function on compute unit 0 and 2:

```
PROFILE_PORT( "hls_kernel[0]/all hls_kernel[2]/all" );
```

3. To enable profiling for all ports of all instances of *hls_df_kernel*:

```
PROFILE_PORT( "hls_df_kernel/all" );
```

4. To enable profiling on port A in *hls_kernel* for compute unit 0 and port C in *hls_kernel* for compute unit 2:

```
PROFILE_PORT( "hls_kernel[0]/A hls_kernel[2]/C" );
```

5. To enable profiling on port A for all instances of *hls_kernel* and port C for all instances of *hls_kernel*:

```
PROFILE_PORT( "hls_kernel/A hls_kernel/C" );
```

Hardware Event Tracing Issues and Solutions

Hardware events are captured in the device FIFO or global memory buffers and sent back (offloaded) to the CPU to generate the timeline traces. Hardware trace events can be dropped in different scenarios as explained in the table.

Table 89: Hardware Trace Event Scenarios

Issue	Solution
Once the FIFO or DDR/HBM buffer gets full, all subsequent hardware trace events will be dropped.	Look for a runtime warning and rerun with a larger buffer size, using this setting in <code>xrt.ini</code> <code>trace_buffer_size=<size></code> , where size must fit within the global memory limits.

Table 89: Hardware Trace Event Scenarios (cont'd)

Issue	Solution
Using the same global memory for compute data transfers can cause congestion leading to dropped hardware trace events.	Use a resource for offloading that is different from what the <code>compute()</code> IO uses. The macro <code>PROFILE_OFFLOAD</code> can be used to specify the profiling offload method in the accelerator class. <pre>PROFILE_OFFLOAD("FIFO" "DDR[0-3]" "HBM[0-31]");</pre> TIP: This cannot be used when targeting compilation for <i>hw_emu</i>, or it will cause a profiling logic insertion error. It will also be ignored for the <i>sw_emu</i> target.
The application testing created too many hardware events.	Lower the number of hardware events generated with any of these setting in the <code>xrt.ini</code> file, <code>device_trace=coarse</code> : generates the fewest hardware events <code>device_trace=fine</code> : generates more hardware events and greater detail <code>stall_trace=true</code> : generates the most hardware events and can quickly fill up any size buffer. Do not use it.

Supported Platforms and Startup Examples

Platforms

While VSC mode should work on most of the Vitis supported platforms. Here is the list of platforms in production that VSC mode is tested on,

- xilinx_u200_gen3x16_xdma_2_202110_1
- xilinx_u55c_gen3x16_xdma_3_202210_1
- xilinx_u50_gen3x4_xdma_2_202010_1
- xilinx_u50_gen3x16_xdma_5_202210_1
- xilinx_u25_gen3x8_xdma_1_202010_1
- xilinx_u250_gen3x16_xdma_3_1_202020_1
- xilinx_u2_gen3x4_xdma_flat_gc_2_202110_1
- xilinx_u2_gen3x4_xdma_gc_2_202110_1

The following are not supported in this release

- All NoDMA Platforms such as xilinx_u50_gen3x16_nodma_1_202110_1
- Embedded platforms such as those using Zynq MPSoC

With the following specific platforms, hardware emulation is not supported and running emulation will lead to an error.

- xilinx_u25_gen3x8_xdma_1_202010_1
- xilinx_u250_gen3x16_xdma_2_1_202010_1

However, validation with software emulation and profiling on the physical card should work as expected. The newer versions of these platforms will support emulation with VSC.

Start-Up Examples

The following examples cover some of the popular design features supported by VSC. They are published on GitHub and the table provides the links to each of those.

Table 90: Popular Design Features Supported by VSC

Example	Name	Feature Coverage
Convolution Filter	quick_start_sc	This example is described in detail the quick-start section. It covers some basic VSC features such as multiple CUs.
Hardware Emulation	debug_profile_sc	This example covers validation features: 1. hardware emulation. 2. Profiling the accelerator execution on the card.
CU-to-GMIO transfer types	gmio_transfers_sc	Global memory IO transfers.
File transfers to CU	file_filter_sc	This examples covers multiple features of specialized IO transfers: 1. P2P and H2C file transfer using ping-pong buffers. Refer to Special Data Transfer Models for more details. 2. Create and use multi-file buffer objects that can work with multi-CU computation. Refer to Special Data Transfer Models for more details. 3. User-defined custom synchronization of input and output buffers to ACC.
Free-running PEs with AXI4-Streams	streaming_sc	ACC containing free-running processing elements with feedback AXI4-Stream connections. Showcase stream depth. Software emulation.
Multiple ACCs with CPU	mult_acc_compose_sc	Allow multiple accelerators (VPP_ACC) in one xclbin, and compose them into a pipeline, with or without CPU processing in-between the PEs.
Multi-card accelerators	mult_card_sc	Application code controlling a multi-card multiple accelerator design.

Vitis C-Library Examples

The following Vitis C-library APIs are some design examples using the VSC mode.

Table 91: Design Examples Using the VSC Mode

API	Platform	API Level
regexEngine	U50	L3
U.2 Gunzip + CSV	U.2	L3
GeoSpatial-KNN	U50	L2
CRC32C	U.2/U50/U200	L3

Additional Resources and Legal Notices

Finding Additional Documentation

Documentation Portal

The AMD Adaptive Computing Documentation Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Documentation Portal, go to <https://docs.xilinx.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

Revision History

Getting Started with Vitis Revision History

The following table shows the revision history for [Section I: Getting Started with Vitis](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
No changes	
07/17/2023 Version 2023.1	
No changes	
05/10/2023 Version 2023.1	
No changes	

Introduction to Vitis Flows Revision History

The following table shows the revision history for [Section II: Introduction to Vitis Flows](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
Section II: Introduction to Vitis Flows	Revised flowchart for Fixed Platforms versus Extensible Platforms .
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
Building and Packaging the System	Updated block diagram.
07/17/2023 Version 2023.1	
No changes	
05/10/2023 Version 2023.1	
Building and Packaging the System	Added content for Vitis Export to Vivado Flow in System Linking subsection.
Vitis Export to Vivado Flow	This section is new.

Developing Applications Revision History

The following table shows the revision history for [Section III: Developing Applications](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
Interrupt	Added a note.
05/10/2023 Version 2023.1	
No changes	

Building and Running the Application Revision History

The following table shows the revision history for [Section IV: Building and Running the System](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
Flow Details and Implementation	Clarified steps for Vitis Export to Vivado flow.
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
Vitis Export Flow Guidelines and Limitations	Added a limitation of -t = hw only.
Flow Details and Implementation	Removed hardware emulation.
05/10/2023 Version 2023.1	
Kernel Interface Requirements	Added s_AXI4-Lite address width to RTL Kernel
Linking the System	Revised figure Figure 23: Linking the System Design to incorporate Export to Vivado Flow

Simulating the Application with the Emulation Flow Revision History

The following table shows the revision history for [Section V: Simulating the Application with the Emulation Flow](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
Working with I/O Traffic Generators	Clarified APIs and API parameters
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
Debug Techniques in Hardware Emulation	Replaced VCD Analyzer with Vitis Analyzer.
07/17/2023 Version 2023.1	
No changes	

Section	Revision Summary
05/10/2023 Version 2023.1	
Running Emulation on an Embedded Processor Platform	Updated procedures to include scp from QEMU shell to copy files from QEMU to local directory (see Step 5).

Profiling, Optimizing, and Debugging the Application Revision History

The following table shows the revision history for [Section VI: Profiling and Debugging the Application](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
No changes	
05/10/2023 Version 2023.1	
No changes	

Vitis Commands and Utilities Revision History

The following table shows the revision history for [Section VII: Vitis Commands and Utilities](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
--clock Options	Clarified clock options.
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
launch_emulator Utility, Versal PS and PMC Arguments for QEMU, Zynq UltraScale+ MPSoC PS and PMU Arguments for QEMU, and Zynq 7000 PS Arguments for QEMU	Added a tip.
xrt.ini File	Updated platforms for poier_profile.
05/10/2023 Version 2023.1	
No changes	

Working with the Analysis View (Vitis Analyzer) Revision History

The following table shows the revision history for [Section IX: Working with the Analysis View \(Vitis Analyzer\)](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	

Section	Revision Summary
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
No changes	
05/10/2023 Version 2023.1	
Importing JSON Output Files	Added section.

Using the Vitis IDE Revision History

The following table shows the revision history for [Section VIII: Using the Vitis Unified IDE](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
RTL Kernel Wizard	Removed from section, this is deprecated.
07/17/2023 Version 2023.1	
Generate RTL Kernel	Documented third-party netlist support.
05/10/2023 Version 2023.1	
No changes	

Using Vitis Embedded Platforms Revision History

The following table shows the revision history for [Section X: Using Vitis Embedded Platforms](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
Adding Hardware Interfaces	Added an image.
05/10/2023 Version 2023.1	
No changes	

Additional Information

The following table shows the revision history for [Section XI: Additional Information](#).

Section	Revision Summary
12/13/2023 Version 2023.2	
No changes	
11/09/2023 Version 2023.2	
Entire section	Updated procedures to work with the Vitis Unified IDE.
07/17/2023 Version 2023.1	
No changes	
05/10/2023 Version 2023.1	
No changes	

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING

OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2019-2023 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Alveo, Kria, UltraScale+, Versal, Virtex, Vitis, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. OpenCL and the OpenCL logo are trademarks of Apple Inc. used by permission by Khronos. PCI, PCIe, and PCI Express are trademarks of PCI-SIG and used under license. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.